Contents

1. Algebra . . . 26
2. Cheatsheet . . . 27
   2.1. Arithmetic Operations . . . 27
   2.2. Exponents and Radicals . . . 27
   2.3. Factoring Special Polynomials . . . 27
   2.4. Binomial Theorem . . . 27
   2.5. Quadratic Formula . . . 27
   2.6. Inequalities and Absolute Value . . . 27
3. Logarithms . . . 28
   3.1. Log . . . 28
   3.2. Ln . . . 28
4. Linear Algebra . . . 29
5. Cheatsheet . . . 30
   5.1. Matrix . . . 37
   5.2. Matrix Factorization . . . 39
   5.3. Singular Value Decomposition (SVD) . . . 39
6. Vectors . . . 42
7. Real Coordinate Spaces . . . 43
8. Vector Operations . . . 43
   8.1. Vector Addition . . . 43
   8.2. Vector Subtraction . . . 44
   8.3. Scalar Multiplication . . . 45
9. Norm . . . 46
   9.1. Euclidean norm (ℓ_2) . . . 46
   9.2. From dot product . . . 47
   9.3. Properties (norm axioms) . . . 47
   9.4. Geometric interpretation . . . 47
   9.5. Other common norms (ℓ_p family) . . . 47
   9.6. Normalizing a vector . . . 48
   9.7. Connections . . . 48
10. Unit Vector . . . 48
11. Dot Product . . . 49
   11.1. Geometric formula . . . 49
   11.2. Connection to norm . . . 49
   11.3. Properties . . . 50
   11.4. Matrix form . . . 50
   11.5. See also . . . 50
12. Inner Product . . . 50
   12.1. Axioms . . . 50
   12.2. Standard examples . . . 51
   12.3. Induced norm . . . 51
   12.4. Cauchy–Schwarz (general form) . . . 51
   12.5. Orthogonality . . . 51
   12.6. Angle between vectors . . . 51

12.7. Why inner products matter . . . 51
12.8. See also . . . 52
13. Cauchy–Schwarz Inequality . . . 52
14. Triangle Inequality . . . 54
14.1. Vector Triangle Inequality . . . 54
15. Orthogonality . . . 55
15.1. Orthogonal sets and orthonormal sets . . . 55
15.2. Key facts . . . 56
15.3. Orthogonal complement . . . 56
15.4. Four fundamental subspaces (and their complements) . . . 56
15.5. Orthogonal projection . . . 56
15.6. Why orthogonality matters . . . 57
15.7. See also . . . 57
16. Angles Between Vectors . . . 57
17. Cartesian Coordinates . . . 59
18. Polar Coordinates . . . 60
19. Parametric Line . . . 60
20. Planes . . . 64
20.1. Point–normal form . . . 64
20.2. Reading the normal off the equation . . . 65
20.3. See also . . . 65
21. Point–Plane Distance . . . 66
21.1. Generalization . . . 66
21.2. Connections . . . 66
22. Plane–Plane Distance . . . 66
22.1. Distance between parallel planes . . . 67
22.2. Strategy: pick a point on P_1, measure to P_2 . . . 67
22.3. Non-parallel planes . . . 67
22.4. Connections . . . 67
23. Hyperplane . . . 68
23.1. Dimension of a hyperplane . . . 68
23.2. Through the origin . . . 68
23.3. Distance from a point to a hyperplane . . . 68
23.4. Half-spaces . . . 68
23.5. Where hyperplanes show up . . . 68
24. Cross Product . . . 69
24.1. Proof: $\vec{c}$ is orthogonal to both $\vec{a}$ and $\vec{b}$ . . . 69
24.2. See also . . . 70
25. Cross Product Magnitude . . . 70
25.1. Geometric meaning . . . 70
25.2. Connection to the determinant . . . 70
25.3. Connections . . . 70
26. Dot vs Cross Product . . . 70
26.1. Side-by-side . . . 70
26.2. Recovering the angle . . . 71
26.3. Pythagorean identity . . . 71

26.4. Connections . . . 71
27. Vector Space . . . 71
28. Linear Combination . . . 72
29. Span . . . 72
30. Linear Independence . . . 75
31. Subspace . . . 78
31.1. Dimension of a Subspace . . . 78
32. Basis . . . 78
33. Dimension . . . 78
33.1. Properties . . . 79
33.2. Computing dimensions of common subspaces . . . 79
33.3. See also . . . 79
34. Coordinate Vector . . . 79
34.1. The standard basis is implicit . . . 80
34.2. Why coordinates matter . . . 80
34.3. Connection to change of basis . . . 81
34.4. See also . . . 81
35. Change of Basis . . . 81
35.1. Change of basis matrix . . . 81
35.2. Standard basis as anchor . . . 81
35.3. Change of basis for a linear transformation . . . 82
35.4. Why it matters . . . 82
35.5. See also . . . 82
36. Gram–Schmidt . . . 82
36.1. The algorithm . . . 83
36.2. Why it works . . . 83
36.3. Numerical instability . . . 84
36.4. Connection to QR . . . 84
36.5. See also . . . 84
37. Matrices . . . 84
37.1. Matrix-Vector Products . . . 84
38. Matrix–Vector Product . . . 86
38.1. Two equivalent views . . . 87
38.2. Linear transformation view . . . 87
38.3. Linearity . . . 87
39. Matrix Multiplication . . . 87
39.1. Properties . . . 88
39.2. Geometric meaning . . . 88
39.3. Special cases . . . 88
40. Transpose . . . 88
40.1. Properties . . . 89
40.2. Special cases . . . 89
40.3. Connection to dot product . . . 89
41. Trace . . . 89
41.1. Properties . . . 90
41.2. Trace as inner product . . . 90
41.3. Where it shows up . . . 90

42. Identity Matrix . . . 90
42.1. Why it's "the identity" . . . 90
42.2. Key properties . . . 91
42.3. Where it shows up . . . 91
43. Zero Matrix . . . 91
43.1. Why it's "the zero" . . . 91
43.2. Key properties . . . 91
43.3. Where the zero vector matters . . . 92
44. Diagonal Matrix . . . 92
44.1. Why diagonal matrices are useful . . . 92
44.2. Special cases . . . 92
44.3. Where they show up . . . 93
45. Triangular Matrix . . . 93
45.1. Upper triangular (U) . . . 93
45.2. Lower triangular (L) . . . 93
45.3. Properties (apply to both upper and lower) . . . 93
45.4. Why they matter . . . 93
45.5. Where they show up . . . 94
46. Symmetric Matrix . . . 94
46.1. Skew-symmetric matrices . . . 94
46.2. Why symmetric matrices are special . . . 94
46.3. Decomposing any matrix . . . 95
46.4. Where they show up . . . 95
47. Orthogonal Matrix . . . 95
47.1. Geometric meaning . . . 95
47.2. Properties . . . 95
47.3. Where they show up . . . 96
48. Linear Equations . . . 96
48.1. Solving Systems of Linear Equations . . . 96
49. Augmented Matrix . . . 99
49.1. Why use it . . . 100
49.2. Reading solutions . . . 100
49.3. Connections . . . 100
50. Row Echelon Form . . . 100
50.1. Elementary row operations . . . 101
50.2. REF vs RREF . . . 102
50.3. See also . . . 102
51. RREF . . . 102
51.1. RREF vs REF . . . 103
51.2. Uniqueness . . . 103
51.3. Reading the solution . . . 103
52. Gaussian Elimination . . . 104
53. Gauss–Jordan Elimination . . . 105
53.1. Two phases . . . 105
53.2. Why bother going past REF? . . . 106
53.3. Inverse via Gauss–Jordan . . . 106

53.4. Cost 107
53.5. See also 107
54. Substitution 107
55. Addition / Subtraction 108
55.1. Addition or Subtraction Method 108
56. Linear System Solutions 108
56.1. 1. Unique solution 108
56.2. 2. No solution (inconsistent) 109
56.3. 3. Infinitely many solutions (free variables) 109
56.4. Rouché–Capelli summary 109
56.5. See also 109
57. Linear System Special Cases 110
57.1. 1. Dependent equations 110
57.2. 2. Underdetermined systems 110
57.3. 3. Inconsistent systems 110
57.4. See also 111
58. Homogeneous System 111
58.1. When are there non-trivial solutions? 111
58.2. Connection to general systems 112
58.3. See also 112
59. Linear Transformation 112
59.1. The two axioms 112
59.2. Consequences 113
59.3. Equivalent definition: matrix–vector product 113
59.4. Non-examples 113
59.5. See also 113
60. Matrix Representation 113
60.1. How to find the matrix 114
60.2. Why this works (sketch) 114
60.3. Identity transformation 114
60.4. See also 114
61. Domain 115
61.1. Domain vs codomain vs image 115
61.2. For matrices 115
61.3. See also 115
62. Codomain 115
62.1. Codomain vs image 116
62.2. For matrices 116
62.3. See also 116
63. Surjective, Injective, Bijective 116
63.1. Injective (one-to-one) 116
63.2. Surjective (onto) 117
63.3. Bijective (one-to-one **and** onto) 117
63.4. Summary table for linear $T : \mathbb{R}^n \to \mathbb{R}^m$ given by matrix A 117
63.5. Connection to inverse 117
63.6. See also 117
64. Image 118

64.1. Matrix representation 118
64.2. Image = Column space 118
64.3. Properties 119
64.4. Three names for the same thing 119
65. Image of Subset 119
65.1. Connections 120
66. Preimage 120
66.1. Properties 121
66.2. Connections 121
67. Kernel 121
67.1. Why the kernel is a subspace 121
67.2. Matrix form: kernel ↔ null space 121
67.3. Injectivity 122
67.4. Connections 122
68. Column Space 122
69. Null Space 123
69.1. Computing the null space 124
69.2. Nullity 124
69.3. Kernel vs null space 124
69.4. Left null space 125
69.5. See also 125
70. Rank 125
70.1. Matrix Representation of Systems of Equations 125
71. Rank–Nullity Theorem 126
71.1. Why it's true (sketch) 126
71.2. Why it matters 127
71.3. Equivalent statements 127
71.4. See also 127
72. Projection 127
73. Composition of Linear Transformations 132
73.1. Composition ↔ matrix multiplication 133
73.2. Properties of composition 133
73.3. See also 133
74. Transformation Sum 133
74.1. Matrix correspondence 133
74.2. Connections 134
75. Transformation Scalar Multiplication 134
75.1. Matrix correspondence 134
75.2. Combined with sum: vector space of transformations 134
75.3. Connections 135
76. Rotation Matrix 135
76.1. 2D rotation 135
76.2. Properties 135
76.3. 3D rotations (about coordinate axes) 136
76.4. General 3D rotation 136
76.5. Eigenvalues 136
76.6. Connection to special orthogonal group 136

76.7. See also 137
77. Reflection Matrix 137
77.1. Reflection across a line through the origin (2D) 137
77.2. Reflection across a hyperplane (general) 137
77.3. Properties 138
77.4. Rotations vs reflections 138
77.5. Householder transformations 138
77.6. See also 138
78. Scaling Matrix 139
78.1. Uniform vs non-uniform 139
78.2. Properties 139
78.3. Negative scaling 139
78.4. Scaling in a general direction 140
78.5. Application to images / graphics 140
78.6. See also 140
79. Shear Matrix 140
79.1. 2D shears 140
79.2. Properties 141
79.3. Why $\det = 1$ 141
79.4. Composition of shears 141
79.5. Application: graphics and italics 141
79.6. Shears in higher dimensions 142
79.7. See also 142
80. Minor 142
80.1. (i, j)-minor (cofactor expansion) 142
80.2. Variants 142
80.3. Where minors show up 143
81. Principal Minor 143
81.1. Levels (order) 143
81.2. Where they show up 143
81.3. See also 144
82. Leading Principal Minor 144
82.1. Sylvester's criterion (positive-definiteness) 144
82.2. See also 144
83. Determinant 144
83.1. From first principles: the 2×2 case 145
83.2. 3×3: signed volume of the parallelepiped 146
83.3. General $n \times n$: cofactor expansion 146
83.4. Key properties 146
83.5. What $\det(A)$ tells you about A 147
83.6. Geometric interpretation 147
83.7. Computing $\det$ in practice 147
83.8. Connection to other concepts 147
84. Cramer's Rule 147
84.1. Why it works (sketch) 148
84.2. Practical remarks 148

84.3. When it fails 149
84.4. See also 149
85. Adjugate 149
86. Unimodularity 155
86.1. Unimodular Matrix 155
86.2. Totally Unimodular Matrix 155
87. Matrix Inverse 156
87.1. Calculating inverse 156
88. LU Decomposition 157
89. QR Decomposition 159
89.1. Construction via Gram–Schmidt 159
89.2. Why it matters 160
89.3. Computation methods 161
89.4. See also 161
90. Cholesky Decomposition 161
90.1. Existence and uniqueness 161
90.2. Algorithm (forward construction) 161
90.3. Why use Cholesky 162
90.4. Relation to LU and SVD 162
90.5. See also 162
91. SVD 162
91.1. Singular values 163
91.2. Geometric interpretation 163
91.3. Compact / reduced SVD 163
91.4. Best low-rank approximation 163
91.5. Connection to symmetric matrices 164
91.6. Connection to pseudoinverse 164
91.7. See also 164
92. Pseudoinverse 164
92.1. Definition via SVD 164
92.2. Four defining properties 165
92.3. Special cases 165
92.4. Least-squares solution 165
92.5. Why it generalizes the inverse 166
92.6. See also 166
93. Eigenvectors & Eigenvalues 166
93.1. Eigen 166
94. Characteristic Polynomial 169
94.1. Why the roots are the eigenvalues 169
94.2. Structure of the polynomial 169
94.3. Multiplicity 169
94.4. Cayley–Hamilton 170
94.5. See also 170
95. Diagonalization 170
95.1. When is A diagonalizable? 170
95.2. How to diagonalize 170
95.3. Why it matters: power and exponential 171

95.4. Spectral decomposition (real symmetric case) 171
95.5. Failure: non-diagonalizable matrices 171
95.6. See also 171
96. Spectral Theorem 172
96.1. What it guarantees 172
96.2. Spectral expansion 172
96.3. Generalizations 173
96.4. Why it matters 173
96.5. See also 173
97. Jordan Canonical Form 173
97.1. Jordan blocks 174
97.2. Diagonalizable as a special case 174
97.3. Why it matters 175
97.4. Computation note 175
97.5. See also 175
98. Quadratic Form 175
98.1. Definiteness 176
98.2. Tests for positive definiteness 176
98.3. Geometric pictures (2D) 176
98.4. Where they show up 177
98.5. Diagonalization (principal axis theorem) 177
98.6. See also 177
99. Real Analysis 178
100. Sets 179
101. Calculus 183
102. Calculus I 184
102.1. Cheatsheet 184
102.2. Limits 191
102.3. Derivatives 195
102.4. Higher-Order Derivatives 200
102.5. Power Rule 200
102.6. Product Rule 205
102.7. Quotient Rule 206
102.8. Chain Rule 208
102.9. Implicit Differentiation 209
102.10. Derivatives of Inverse Functions 210
102.11. Inverse Functions 213
102.12. L'Hôpital's Rule 216
102.13. Mean Value Theorem 217
102.14. Extreme Value Theorem 218
102.15. Critical Points 219
102.16. Derivative Test 220
102.17. Arc Length 226
102.18. Series 226
102.19. Taylor Series 227
103. Calculus II 229
103.1. Cheatsheet 229

103.2. Chain Rule . . . 231
103.3. Double Integrals . . . 232
103.4. Gradient . . . 232
103.5. Hessian Matrix . . . 233
103.6. Jacobian . . . 234
103.7. Semi-Definiteness . . . 235
104. Calculus III . . . 236
104.1. Vector-Valued Functions . . . 236
105. Integration . . . 236
105.1. Rules . . . 236
105.2. Leibniz's Rule . . . 236
105.3. Leibniz's Rule . . . 237
106. Differential Equations . . . 237
106.1. Slope Field (Direction Field) . . . 241
107. Geometry . . . 244
108. Cheatsheet . . . 245
108.1. Geometric Formulas . . . 245
108.2. Trigonometric Functions . . . 245
109. Probability Theory . . . 246
110. Cheatsheet . . . 247
111. Sample Space . . . 249
112. Sets . . . 249
112.1. Infinite Collection of Sets . . . 251
113. Axioms . . . 251
113.1. Probability Axioms . . . 251
113.2. Discrete Uniform Law . . . 255
113.3. Probability Calculations Steps . . . 255
114. De Morgan's Laws . . . 257
114.1. Complement of a union . . . 257
114.2. Complement of an intersection . . . 257
114.3. Scaling . . . 257
115. Sequences . . . 258
116. Series . . . 259
117. Countability . . . 260
117.1. Countable and Uncoutable Sets . . . 260
118. Bonferroni Inequality . . . 260
119. Conditional Probability . . . 262
120. Multiplication Rule . . . 263
121. Total Probability Theorem . . . 264
122. Bayes' Rule . . . 264
122.1. Beyes' Rule . . . 264
123. Independence . . . 267
123.1. Independence of Complements . . . 267
123.2. Conditional Independence . . . 268
123.3. Pairwise Independence . . . 268
123.4. Reliability . . . 269
124. Counting Principle . . . 269

125. Expected Value . . . 271
125.1. Discrete . . . 271
125.2. Continuous . . . 271
125.3. Partial (truncated) expectation . . . 271
125.4. Conditional expectation . . . 271
125.5. LOTUS . . . 272
126. Expected Value . . . 272
126.1. Discrete . . . 272
126.2. Continuous . . . 272
127. Linearity of Expectations . . . 272
128. Joint, Marginal & Conditional Probability . . . 273
128.1. Joint Probability . . . 273
128.2. Marginal Probability . . . 273
128.3. Conditional Probability . . . 273
128.4. Convolution . . . 273
129. Mixture Distributions . . . 275
130. Law of Total Expectation . . . 276
131. Law of Total Variance . . . 276
132. I.I.D. . . . 276
133. Moment Generating Function . . . 277
134. Markov's Inequality . . . 278
135. Chebyshev's Inequality . . . 278
136. Statistics . . . 279
137. Cheatsheet . . . 280
138. Descriptive Statistics . . . 281
138.1. Mean . . . 281
138.2. Median . . . 281
138.3. Mode . . . 282
138.4. Range . . . 282
138.5. Variance . . . 282
138.6. Standard Deviation . . . 283
138.7. Interquartile Range . . . 284
139. Probability Distributions . . . 284
139.1. Bernoulli . . . 284
139.2. Gaussian . . . 285
139.3. Differentiating Normal Distribution . . . 286
139.4. Student's t . . . 289
139.5. Binomial . . . 290
139.6. Poisson . . . 290
139.7. Exponential . . . 291
139.8. Differentiating the Normal Distribution . . . 291
140. Functions . . . 294
140.1. PDF . . . 294
140.2. PMF . . . 295
140.3. CDF . . . 295
140.4. PPF . . . 296
140.5. SF . . . 296

140.6. Standard Normal Loss Function 297
140.7. Standard Normal Loss Function 299
140.8. Likelihood 301
141. Error Metrics 302
141.1. MD 302
141.2. MAE 303
141.3. MSE 303
141.4. RMSE 303
141.5. MPE 303
141.6. MAPE 304
141.7. R^2 304
141.8. MSLE 305
141.9. Cross-Entropy Loss 305
142. Hypothesis Testing 305
142.1. Null Hypothesis 305
142.2. Alternative Hypothesis 306
142.3. Type I Error 306
142.4. Type II Error 306
142.5. One-Sample Z-Test for Mean 306
142.6. One-Sample Two-Tail Z-Test for Mean 308
142.7. Two-Sample Z-Test for Proportion 310
142.8. One-Sample T-Test for Mean 311
142.9. One-Sample Two-Tail T-Test for Mean 315
142.10. Independent t-Test 316
142.11. Paired t-Test 317
142.12. Chi-Square Goodness of Fit 317
142.13. Chi-Square Independence 320
142.14. One-Way ANOVA 323
142.15. Two-Way ANOVA 327
142.16. A/B Testing 334
143. Regression Analysis 335
143.1. Simple Linear Regression 335
143.2. Multiple Linear Regression 337
143.3. Logistic Regression 339
143.4. Ridge 340
143.5. Lasso 340
143.6. p-value 340
143.7. F-statistic 341
143.8. Confidence Intervals 341
144. Correlation 345
144.1. Pearson 345
144.2. Spearman Rank 346
144.3. Covariance & Correlation 346
145. Non-Parametric Statistics 348
145.1. Mann–Whitney U 348
145.2. Wilcoxon Signed-Rank 348
145.3. Kolmogorov–Smirnov 348

145.4. Kruskal–Wallis . . . 348
146. Maximum Likelihood Estimation . . . 348
147. Generalized Linear Models . . . 349
148. Cross Validation . . . 352
149. Markov Chains . . . 352
149.1. Checkout Counter Model . . . 352
149.2. n-Step Transition Probabilities . . . 353
149.3. Key Recursion . . . 353
149.4. Random Initial State . . . 353
150. Monte Carlo Simulation . . . 354
151. Design of Experiments . . . 354
151.1. Factorial Designs . . . 354
151.2. Response Surface Methodology . . . 354
151.3. Analysis of Interaction Effects . . . 354
152. Principal Component Analysis . . . 354
152.1. PCA as application . . . 354
153. AIC & BIC . . . 356
153.1. AIC . . . 356
153.2. BIC . . . 356
154. Bootstrap & Resampling . . . 358
155. Types of Statistics . . . 358
156. Time Series . . . 359
157. Moving Averages . . . 360
157.1. Overview . . . 360
157.2. Naive . . . 361
157.3. Cumulative . . . 363
157.4. Simple Moving Average . . . 364
157.5. Weighted Moving Average . . . 366
158. Decomposition . . . 368
158.1. Classical . . . 368
158.2. STL . . . 370
159. Frequency . . . 373
159.1. Fourier . . . 373
159.2. Periodogram . . . 375
160. Exponential Smoothing (ETS) . . . 377
160.1. Summary . . . 377
160.2. A-N-N . . . 382
160.3. A-N-A . . . 384
160.4. A-N-M . . . 386
160.5. A-A-N . . . 387
160.6. A-A-A . . . 389
160.7. A-A-M . . . 391
160.8. A-Ad-N . . . 393
160.9. A-Ad-A . . . 394
160.10. A-Ad-M . . . 396
160.11. A-M-N . . . 398
160.12. A-M-A . . . 400

160.13. A-M-M 401
160.14. A-Md-N 403
160.15. A-Md-A 405
160.16. A-Md-M 407
160.17. M-N-N 409
160.18. M-N-A 410
160.19. M-N-M 412
160.20. M-A-N 413
160.21. M-A-A 415
160.22. M-A-M 417
160.23. M-Ad-N 419
160.24. M-Ad-A 421
160.25. M-Ad-M 423
160.26. M-M-N 424
160.27. M-M-A 426
160.28. M-M-M 428
160.29. M-Md-N 430
160.30. M-Md-A 431
160.31. M-Md-M 433
161. ARIMA Family 435
161.1. Summary 435
161.2. AR 437
161.3. MA 439
161.4. ARMA 441
161.5. ARIMA 442
161.6. SARIMA 445
161.7. SARIMAX 446
161.8. VAR 448
161.9. VMA 451
161.10. VARMA 453
161.11. VARIMA 456
161.12. VARIMAX 458
162. System Dynamics 462
163. System Dynamics 463
163.1. Core concepts 463
163.2. When to use 463
163.3. When not to use 463
163.4. Vs other approaches 463
163.5. Classic SD applications 463
163.6. Software 464
163.7. See also 464
164. Stocks Flows 464
164.1. Mathematical foundation 464
164.2. Units 464
164.3. Three classes of stock-flow systems 465
164.4. Bathtub metaphor 465
164.5. Stocks have memory; flows don’t 465

164.6. Connection to differential equations . . . 465
164.7. See also . . . 466
165. Causal Loop Diagrams . . . 466
165.1. Anatomy . . . 466
165.2. Loop polarity . . . 466
165.3. Example: population growth . . . 466
165.4. Common patterns / archetypes . . . 467
165.5. From CLD to quantitative model . . . 467
165.6. Common pitfalls . . . 467
165.7. See also . . . 467
166. Feedback Loops . . . 467
166.1. Two types . . . 467
166.2. Behavior patterns . . . 468
166.3. First-order behavior . . . 468
166.4. Combined loops: limits to growth . . . 468
166.5. Second-order: oscillation . . . 468
166.6. Why feedback loops matter . . . 469
166.7. Identifying loops in a system . . . 469
166.8. See also . . . 469
167. Delays . . . 469
167.1. Three kinds . . . 469
167.2. Pipeline (pure) delay . . . 469
167.3. First-order exponential delay . . . 470
167.4. Higher-order Erlang delays . . . 470
167.5. Why delays drive oscillation . . . 470
167.6. Common SD examples . . . 470
167.7. See also . . . 470
168. Second-Order . . . 471
168.1. Generic equation . . . 471
168.2. Behavior by ζ . . . 471
168.3. Damped frequency and period . . . 471
168.4. Where 2nd-order systems arise . . . 471
168.5. Beer game equations as 2nd-order . . . 471
168.6. Stability via Routh-Hurwitz . . . 472
168.7. See also . . . 472
169. Smoothing . . . 472
169.1. First-order smoothing (exponential smoothing) . . . 472
169.2. Higher-order smoothing . . . 472
169.3. TREND function . . . 472
169.4. FORECAST function . . . 473
169.5. Why smoothing models decision-making . . . 473
169.6. Phase lag . . . 473
169.7. See also . . . 473
170. Stock Management . . . 473
170.1. The classical formula . . . 473
170.2. The supply-line neglect parameter . . . 474
170.3. Anchoring and adjustment . . . 474

170.4. Why this generates bullwhip . . . 474
170.5. Real-world example . . . 474
170.6. See also . . . 475
171. Bullwhip SD . . . 475
171.1. Chen-Drezner-Ryan-Simchi-Levi formula . . . 475
171.2. Key insights . . . 475
171.3. Sources of bullwhip (Lee-Padmanabhan-Whang 1997) . . . 475
171.4. SD beer-game replication . . . 476
171.5. Mitigations . . . 476
171.6. See also . . . 476
172. Beer Game . . . 476
172.1. Setup . . . 476
172.2. The shock . . . 476
172.3. The result (consistent across hundreds of plays) . . . 477
172.4. Sterman's decision-rule estimate . . . 477
172.5. What works . . . 477
172.6. Mathematical structure . . . 477
172.7. Variations . . . 477
172.8. Why it matters . . . 477
172.9. See also . . . 478
173. Bass Diffusion . . . 478
173.1. Model . . . 478
173.2. Structure . . . 478
173.3. Behavior . . . 478
173.4. Empirical fit . . . 479
173.5. Generalizations . . . 479
173.6. Why it matters . . . 479
173.7. See also . . . 479
174. Logistic Growth . . . 479
174.1. Model . . . 479
174.2. Closed-form solution . . . 480
174.3. Phases . . . 480
174.4. Structurally . . . 480
174.5. Where it shows up . . . 480
174.6. Limitations . . . 480
174.7. Compared to Bass diffusion . . . 480
174.8. Parameter estimation . . . 481
174.9. See also . . . 481
175. SIR SEIR . . . 481
175.1. SIR model . . . 481
175.2. Basic reproduction number R_0 . . . 481
175.3. Herd immunity threshold . . . 482
175.4. SEIR: add Exposed . . . 482
175.5. Behavior . . . 482
175.6. Variants . . . 482
175.7. Where used . . . 483
175.8. Limitations . . . 483

175.9. See also 483
176. Lotka-Volterra 483
176.1. Behavior: closed orbits in the phase plane 483
176.2. Oscillation dynamics 483
176.3. Limitations and extensions 484
176.4. Beyond two species 484
176.5. Outside biology 484
176.6. See also 484
177. Phase Plane 485
177.1. For a 2-state system 485
177.2. Key features 485
177.3. Linearizing at an equilibrium 485
177.4. Classification by eigenvalues 485
177.5. Lotka-Volterra as example 486
177.6. Limit cycles and chaos 486
177.7. Routh-Hurwitz stability for higher dimensions 486
177.8. See also 486
178. Numerical Integration 486
178.1. Euler method (default for SD) 487
178.2. Time step (Δt) selection rule 487
178.3. Runge-Kutta 2 (RK2 / midpoint) 487
178.4. Runge-Kutta 4 (RK4) 487
178.5. Which to use? 487
178.6. Stiff systems 488
178.7. Initialization in equilibrium 488
178.8. See also 488
179. Validation 488
179.1. Structure-oriented tests (does the model capture what it should?) 488
179.2. Behavior-oriented tests (does the model produce realistic output?) 489
179.3. Quantitative fit: Theil's U decomposition 489
179.4. Behavior modes matter more than point fits 489
179.5. Calibration methods 489
179.6. Common pitfalls 489
179.7. Sensitivity analysis types 490
179.8. See also 490
180. Operations Research 491
181. Optimization 492
181.1. Cheatsheet 492
181.2. MILP Idioms 497
181.3. Standard Form 499
181.4. Linear Programming 500
181.5. Simplex Method 513
181.6. Simplex Cheatsheet 565
181.7. Input / Output 568
181.8. Integer Programming 572
181.9. Branch and Bound 590
181.10. Mixed Integer Programming 596

181.11. Non-Linear Programming 597
181.12. Convexity 603
181.13. Convex Analysis 604
181.14. Gradient Descent 618
181.15. Newton's Method 629
181.16. Heuristic Algorithms 635
181.17. Duality 635
181.18. Multi-Commodity Network Flow 645
181.19. Sensitivity Analysis 656
181.20. Network Flow 661
181.21. Ford–Fulkerson 665
181.22. Edmonds–Karp 669
181.23. Transportation Problem 669
181.24. Assignment Problem 670
181.25. Hungarian Algorithm 671
181.26. Transshipment Problem 673
181.27. Min-Cost Flow Algorithms 674
181.28. Network Simplex 675
181.29. Min Spanning Tree 676
181.30. Floyd-Warshall 677
181.31. TSP 679
181.32. TSP 2opt 680
181.33. Christofides 682
181.34. VRP 683
181.35. VRP Time Windows 684
181.36. Clarke-Wright 686
181.37. Daganzo Continuous 687
181.38. Facility Location 688
181.39. UFLP 690
181.40. CFLP 691
181.41. p-Median 693
181.42. p-Center 694
181.43. Set Covering 696
181.44. Max Covering 697
181.45. Center of Gravity 699
181.46. Stochastic Programming 701
181.47. Two-stage Recourse 702
181.48. Scenario Trees 703
181.49. Chance Constraints 704
181.50. Sample Average Approximation 706
181.51. EVPI vs VSS 708
181.52. Lagrange Relaxation 709
181.53. Lagrangian 713
181.54. KKT Conditions 714
181.55. Dynamic Programming 739
181.56. Bellman Equation 740
181.57. Backward Induction 742

181.58. Forward Induction 743
181.59. Stochastic DP 745
181.60. Value Iteration 746
181.61. Policy Iteration 748
181.62. Regression 750
181.63. Support Vector Machine 754
182. Queuing Theory 754
182.1. M/M/1 754
182.2. M/M/c 758
182.3. Little's Law 761
182.4. Kingman VUT 762
182.5. Sakasegawa 763
182.6. Linking Equation 764
182.7. Erlang B 766
182.8. Erlang C 767
182.9. MMc Abandonment 769
182.10. Square-Root Staffing 770
183. Decision Analysis 772
183.1. Decision Analysis 772
183.2. Decision Trees 773
183.3. EMV 774
183.4. EVPI 775
183.5. EVSI 777
183.6. Decision Criteria 778
183.7. Utility Theory 780
184. Control Charts 782
184.1. Cheatsheet 782
184.2. C-Chart 783
184.3. P-Chart 784
184.4. U-Chart 785
184.5. NP-Chart 785
184.6. X-Chart 786
185. Process Capability 787
185.1. Cheatsheet 787
185.2. Cp 788
185.3. Cpk 791
185.4. Cpm 793
185.5. Pp 793
185.6. Ppk 795
186. Causes of Variation 796
186.1. Common 796
186.2. Special 796
187. Acceptance Sampling 797
188. Critical Path Method (CPM) 797
189. Supply Chain 798
190. Planning 799
190.1. Aggregate Planning 799

190.2. MPS 801
190.3. MRP 802
190.4. DRP 804
190.5. RCCP 806
190.6. Silver-Meal 807
190.7. Least Unit Cost 808
190.8. Part-Period Balancing 809
191. Manufacturing 811
191.1. Factory Physics 811
191.2. Best Worst PWC 812
191.3. Critical WIP 814
191.4. CONWIP 815
191.5. Kanban Sizing 816
191.6. Takt Time 817
191.7. Line Balancing 819
191.8. Theory of Constraints 821
191.9. Learning Curves 822
191.10. Heijunka 824
192. Bullwhip Effect 826
192.1. Why it matters 826
192.2. The four classical causes (Lee, Padmanabhan, Whang 1997) 826
192.3. Mitigation strategies 827
192.4. The beer game 828
193. Classification 829
193.1. ABC 829
193.2. XYZ 831
193.3. ABC × XYZ 833
193.4. FSN 835
193.5. VED 837
194. Metrics 840
194.1. Inventory Turnover 840
194.2. Days of Supply 842
194.3. Order Cycle Time 845
194.4. Perfect Order Rate 845
194.5. Stockout Rate 845
194.6. GMROI 848
195. Service Levels 850
195.1. Cycle Service Level 850
195.2. Fill Rate 852
195.3. Ready Rate 854
196. Stocks 856
196.1. Cycle Stock 856
196.2. Lead Time 858
196.3. Lead Time 859
196.4. Safety Stock 860
196.5. Anticipation Stock 862
196.6. Decoupling Stock 864

196.7. Pipeline Stock 867
197. Newsvendor 869
197.1. Overview 869
197.2. Discrete 872
197.3. Expected Profit 874
197.4. Shortage Penalty 876
197.5. Multiplicative 878
197.6. Multi-Product 880
197.7. Capacity-Constrained 882
198. Multi-Echelon 885
198.1. Multi-Echelon 885
198.2. Clark-Scarf 886
198.3. Multi-Echelon Inventory 889
198.4. Clark-Scarf view (stochastic service) 891
198.5. Graves-Willems 892
198.6. Stochastic-service 894
198.7. METRIC 895
198.8. VARI-METRIC 897
199. Risk Pooling 898
199.1. Risk Pooling 898
199.2. Square-Root Law 900
199.3. Correlated Pooling 902
199.4. Location Pooling 904
199.5. Product Pooling 905
199.6. Lead-time Pooling 907
200. EOQ 908
200.1. EOQ 908
200.2. EPQ 916
200.3. Backorders 919
200.4. Lost Sales 921
200.5. Quantity Discounts 923
200.6. Capacity-Constrained 926
200.7. Joint Replenishment 929
200.8. Multi-Echelon 931
200.9. Perishables 934
200.10. Wagner–Whitin 936
201. Policies 938
201.1. Overview 938
201.2. Base Stock 940
201.3. (s, S) 942
201.4. (Q, r) 944
201.5. (nQ, r) 947
201.6. (R, S) 949
201.7. (R, s, S) 950
201.8. (R, nQ, s) 953
202. Graph Theory 957
203. Overview 958

204. Machine Learning 959
205. Bias–Variance Tradeoff 960
205.1. Bias Variance Tradoff 960
206. Classification 964
206.1. Support Vector Machine 964
206.2. XGBoost 964
207. Graphs 964
207.1. Graph Neural Network 964
208. Economics 970
209. Supply & Demand 971
210. Utility 971
211. Production Functions 971
212. Externality 971
213. Network Effect 971
214. Algorithms 972
215. Dijkstra 973
216. Organisations 975
217. Actor Network 976
217.1. Actor-Network Theory (ANT) 976
218. Bounded Rationality 976
219. Bureaucracy 976
219.1. Bureaucratic Theory 976
220. Cybernetic 976
221. Embeddedness 976
222. Functional Theory of Stratification 976
223. Institutional Theory 976
224. Network Theories 976
224.1. World-Systems Theory 976
224.2. Global Commodity Chains (GCC) 976
224.3. Global Value Chains (GVC) 976
224.4. Global Production Networks (GPN) 977
225. Resource Dependency 977
225.1. Resource Dependency Theory (RDT) 977
226. Scientific Management 977
227. Transaction Costs 977
228. Organizational Ecology 979
229. Certifications 980
230. CSCP 981
230.1. Supply Chains, Demand Management, and Forecasting 981
231. Supply Chain Analytics 990
231.1. Inventory Turns 990
232. ACCA F3 — Financial Accounting 994
232.1. Cheatsheet 994
2. The context and purpose of financial reporting 994
3. Accounting principles, concepts and qualitative characteristics 996
4. The use of double-entry bookkeeping and accounting systems 997
5. Recording transactions and events 998

232.6. Overview 998
232.7. Stakeholders 1002
232.8. Balance Sheet 1003
232.9. Income Statement 1003
232.10. Cash Flow Statement 1005
233. Appendix 1006
234. Scheduling Code 1007
235. White Paper 1009
235.1. Administrative Hierarchy and Demographics 1011
235.2. Schema 1014
236. References 1018
Bibliography 1019

1. Algebra

2. Cheatsheet

2.1. Arithmetic Operations

$a(b+c) = ab + ac$

$\frac{a+c}{b} = \frac{a}{b} + \frac{c}{b}$

$\frac{a}{b} + \frac{c}{d} = \frac{ad+bc}{db}$

$\frac{\frac{a}{b}}{\frac{c}{d}} = \frac{a}{b} \times \frac{d}{c} = \frac{ad}{bc}$

2.2. Exponents and Radicals

$x^m x^n = x^{m+n}$

$(x^m)^n = x^{mn}$

$(xy)^n = x^n + y^n$

$x^{\frac{1}{n}} = \sqrt[n]{x}$

$\sqrt[n]{xy} = \sqrt[n]{x}\sqrt[n]{y}$

$\frac{x^m}{x^n} = x^{m-n}$

$x^{-n} = \frac{1}{x^n}$

$\left(\frac{x}{y}\right)^n = \frac{x^n}{y^n}$

$x^{\frac{m}{n}} = \sqrt[n]{x^m} = (\sqrt[n]{x})^m$

$\sqrt[n]{\frac{x}{y}} = \frac{\sqrt[n]{x}}{\sqrt[n]{y}}$

2.3. Factoring Special Polynomials

$x^2 - y^2 = (x+y)(x-y)$

$x^3 + y^3 = (x+y)(x^2 - xy + y^2)$

$x^3 - y^3 = (x-y)(x^2 + xy + y^2)$

2.4. Binomial Theorem

$(x+y)^n = \sum_{k=0}^{n} \begin{bmatrix} n \\ k \end{bmatrix} x^{n-k} y^k$

$(x+y)^2 = x^2 + 2xy + y^2$

$(x-y)^2 = x^2 - 2xy + y^2$

$(x+y)^3 = x^3 + 3x^2y + 3xy^2 + y^3$

$(x-y)^3 = x^3 - 3x^2y + 3xy^2 - y^3$

2.5. Quadratic Formula

If $ax^2 + bx + c = 0$, then $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$

2.6. Inequalities and Absolute Value

If $a < b$ and $b < c$, then $a < c$

If $a < b$, then $a + c < b + c$

If $a < b$ and $c > 0$, then $ca < cb$

If $a < b$ and $c < 0$, then $ca > cb$

If $a > 0$, then

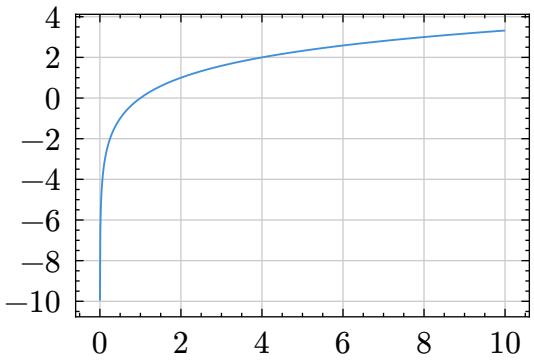
$|x| = a \qquad \text{means} \qquad x = a \quad \text{or} \quad x = -a$

$|x| < a$ means $-a < x < a$

$|x| > a$ means $x > a$ or $x < -a$

3. Logarithms

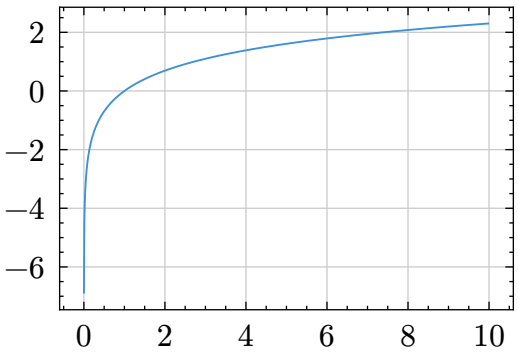
3.1. Log



A logarithm turns multiplication into addition.

$$\log(a \cdot b) = \log(a) + \log(b)$$

3.2. Ln



4. Linear Algebra

5. Cheatsheet

Vector	$\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$	
Scalar Vector Multiplication	$c \in \mathbb{R} \qquad \vec{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$ $c\vec{x} = \begin{bmatrix} cx_1 \\ cx_2 \\ \vdots \\ cx_n \end{bmatrix}$	
Dot Product	$\vec{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \qquad \vec{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$ $\vec{x} \cdot \vec{y} = \sum_{i=1}^{n} x_i y_i$ $= x_1 y_1 + x_2 y_2 + ... + x_n y_n$ $= \vec{x}^T \vec{y}$	
Cross Product ($\mathbb{R}^3$)	$\vec{a} = \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} \qquad \vec{b} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$ $\vec{c} = \vec{a} \times \vec{b}$ $\vec{c} = \begin{bmatrix} a_2 b_3 - a_3 b_2 \\ a_3 b_1 - a_1 b_3 \\ a_1 b_2 - a_2 b_1 \end{bmatrix}$	Returns a vector orthogonal to the two vectors

Vector Space	Closure under addition $\vec{u}, \vec{v} \in V \quad \Rightarrow \quad \vec{u} + \vec{v} \in V$ Closure under scalar multiplication $\vec{v} \in V \wedge c \in \mathbb{R} \quad \Rightarrow \quad c\vec{v} \in V$ Commutativity of addition $\vec{u} + \vec{v} = \vec{v} + \vec{u}$ Associativity of addition $(\vec{u} + \vec{v}) + \vec{w} = \vec{u} + (\vec{v} + \vec{w})$ Additive identity $\exists \mathbf{0} \in V \mid \vec{v} + \mathbf{0} = \vec{v}$ Additive inverse $\forall \vec{v} \in V \exists -\vec{v} \in V \mid \vec{v} + (-\vec{v}) = 0$ Scalar multiplication (compatibility) $a(b\vec{v}) = (ab)\vec{v}$ Distributivity over vector addition $a(\vec{u} + \vec{v}) = a\vec{u} + a\vec{v}$ Distributivity over scalar addition $(a + b)\vec{v} = a\vec{v} + b\vec{v}$ Multiplicative identity $1\vec{v} = \vec{v}$	
Subspace	Non-emptiness $\mathbf{0} \in V$ Closure under addition $\text{If } \vec{u}, \vec{v} \in V, \text{then } \vec{u} + \vec{v} \in V$ Closure under scalar multiplication $\text{If } \vec{v} \in V, c \in \mathbb{R}, \text{then } c\vec{v} \in V$	A subspace is a subset of a vector space that is itself a vector space, satisfying the same axioms as the original. If V is a vector space in $\mathbb{R}^n$, then the subspace U is always contained in $\mathbb{R}^n$, meaning $U \subseteq \mathbb{R}^n$
Vector Addition	$\vec{u} = (u_1, u_2, ..., u_n)$ $\vec{v} = (v_1, v_2, ..., v_n)$ $\vec{u} + \vec{v} = (u_1 + v_1, u_2 + v_2, ..., u_n + v_n)$	

Dot Product	$\vec{u} \cdot \vec{v} = \sum_{i=1}^{n} u_i v_i$	
Orthogonality	$\vec{u} \cdot \vec{v} = \mathbf{0}$	Angle between the two vectors is $90°$
Angle between vectors	$\Theta = \arccos\left(\frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\|_2 \cdot \|\vec{v}\|_2}\right)$	
L_1 Norm (Manhattan)	$\|\vec{u}\|_1 = \sum_{i=1}^{n} \|u_i\|$	
L_2 Norm (Euclidean)	$\|\vec{u}\|_2 = \sqrt{\sum_{i=1}^{n} u_i^2}$	
L_1 Distance (Manhattan)	$d(\vec{u}, \vec{v}) = \sum_{i=1}^{n} \|u_i - v_i\|$	
L_2 Distance (Euclidean)	$d(\vec{u}, \vec{v}) = \sqrt{\sum_{i=1}^{n} (u_i - v_i)^2}$	
Projection	$\text{proj}_w(\vec{v}) = \frac{\vec{v} \cdot \vec{w}}{\vec{w} \cdot \vec{w}} \vec{w}$	
Linear Independence		A set of vectors is linearly independent if no vector in the set can be written as a linear combination of the others A set of vectors $\{\vec{v}_1, \vec{v}_2, ..., \vec{v}_n\}$ is linearly independent if the only solution to the the equation $c_1\vec{v}_1 + c_2\vec{v}_2 + ... + c_n\vec{v}_n = \mathbf{0}$ is $c_1 = c_2 = ... = c_n = 0$
Transformation	$T : \mathbb{R}^n \to \mathbb{R}^m$ $T(\vec{v}) = A\vec{v}$ • Additivity $T(\vec{u} + \vec{v}) = T(\vec{u}) + T(\vec{v})$ • Homogeneity $T(c\vec{u}) = cT(\vec{u})$	• Surjective (onto) Every element in B is the image of at least one element in A. The transformation covers the entire codomain. $\text{Range}(T) = B$ • Injective (one-to-one) Different inputs in A map to different outputs in B.The transformation is information-

		preserving — doesn't collapse distinct vectors together $T(\vec{x}_1) = T(\vec{x}_2) \Rightarrow x_1 = x_2$ Or equivalently: $\ker(T) = \{\mathbf{0}\}$
Domain	$T : V \to W$ $\text{Domain}(T) = V$	Set of all **input vectors** V that the **transformation** acts on
Codomain	$T : V \to W$ $\text{Codomain}(T) = W$	Set of all possible **output vectors** W to which elements of the domain V are mapped under the **transformation**
Transpose	$\det(A) = \det(A^T)$ $(AB)^T = B^T A^T$ $(A^T)^{-1} = (A^{-1})^T$	
Image	$T : V \to W$ $\text{Im}(T) = \{\vec{w} \in W \mid \vec{w} = T(\vec{v}) \text{ for some } \vec{v} \in V\}$	The image of a **transformation** $T : V \to W$ is the set of all possible outputs $T(v)$ for $v \in V$: • It is a subspace of the codomain W • If T is represented by a matrix A, the image of T is the column space of A
Preimage	$T : V \to W$ $T^{-1}(\vec{w}) = \{\vec{v} \in V \mid T(\vec{v}) = \vec{w}\}$	The preimage of a transformation refers to the set of all elements in the domain that map to a particular element or subset in the codomain
Span	$\text{Span}(\{v_1, v_2, ..., v_k\}) = \left\{\sum_{i=1}^{n} c_i \vec{v}_i \mid c_i \in \mathbb{R}\right\}$	The span of a set of vectors is the collection of all possible linear combinations of those vectors
Composition	$T_1 : \mathbb{R}^n \to \mathbb{R}^m \qquad T_2 : \mathbb{R}^m \to \mathbb{R}^p$ $T_1(\vec{v}) = A\vec{v} \qquad T_2(\vec{v}) = B\vec{v}$ $T \circ S(\vec{v}) = T_2(T_1(\vec{v}))$	

	$T_2 \circ T_1(\vec{v}) = B(A\vec{v}) = (BA)\vec{v}$ • Additivity $(T_3 \circ T_2) \circ T_1 = T_3 \circ (T_2 \circ T_1)$ • Homogeneity $T_2 \circ T_1(c\vec{u}) = c(T_2 \circ T_1)(\vec{u})$ • Identity Transformation $I \circ T = T \qquad T \circ I = T$	
Column Space (Range)	$\text{Col}(A) = \{Ax \mid \vec{x} \in \mathbb{R}^n\}$ Or equivalently $A = [\vec{c}_1 \quad \vec{c}_2 \quad ... \quad \vec{c}_3]$ $\text{Col}(A) = \text{span}(\vec{c}_1, \vec{c}_2, ..., \vec{c}_3)$	The column space (or range) of a **matrix** A is the set of all linear combinations of its columns
Determinant	$\det(A)$	The determinant of a square matrix A measure of the "scale factor" by which the matrix A transforms a space • $\det(A) \neq 0$ ‣ A does not collapse the space ‣ A has full rank ‣ A's columns are linearly independent ‣ A is invertable • $\det(A) = 0$ ‣ A collapses the space into lower dimension ‣ A does not have full rank ‣ A's columns are linearly dependent ‣ A is non-invertable (singular)
Invertibility	$\det(A) \neq 0 \implies \text{Invertible}$	

<table>
<tr>
<td></td>
<td>$\det(A) = 0 \implies$
Non-Invertible
$AA^{-1} = A^{-1}A = I_n$
$(AB)^{-1} = B^{-1}A^{-1}$
$(A^T)^{-1} = (A^{-1})^T$</td>
<td></td>
</tr>
<tr>
<td>Basis</td>
<td>Linear Independence
$c_1\vec{v_1} + c_2\vec{v_2} + ... + c_k\vec{v_k} = \mathbf{0}$
$\Rightarrow c_1 = c_2 = ... = c_k = 0$
Spanning
$\forall \vec{v} \in V, \exists c_1, ..., c_k \in \mathbb{R} \quad s.t.$
$\vec{v} = c_1\vec{v_1} + ... + c_k\vec{v_k}$</td>
<td>• A basis of a vector space V is a set of linearly independent vectors that span the space
• Every vector in V can be uniquely written as a linear combination of the basis vectors
E.g.:</td>
</tr>
<tr>
<td>Dimension</td>
<td>$\dim(V)$</td>
<td>Number of linearly independent vectors (basis) in a vector space V
$V \subseteq \mathbb{R}^n$
<table><tr><td>$\dim(V) = 0$</td><td>$V = \{\mathbf{0}\}$</td></tr><tr><td>$\dim(V) = 1$</td><td>V is a line through the origin in $\mathbb{R}^n$</td></tr><tr><td>$\dim(V) = 2$</td><td>V is a plane through the origin in $\mathbb{R}^n$</td></tr><tr><td>$\dim(V) = k$</td><td>V is a k-dimensional flat subspace of $\mathbb{R}^n$</td></tr><tr><td>$\dim(V) = n$</td><td>$V = \mathbb{R}^n$</td></tr></table>• $\mathbb{R}^2$ has dimension 2:
‣ A basis: $\left\{ \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \end{bmatrix} \right\}$
• $\mathbb{R}^3$ has dimension 3:</td>
</tr>
</table>

		▸ A basis: $\left\{ \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \right\}$
Rank	$\text{Rank}(A) = \dim(\text{Col}(A)) = \dim(\text{Row}(A))$	• The rank of a **matrix** A is the **dimension** of its column space (or row space) • Number of linearly independent columns (or rows)
Eigen	$Ax = \lambda x, \quad x \neq \mathbf{0}$	Set of all nonzero vectors $\vec{x}$ such that when the transformation represented by matrix A is applied to $\vec{x}$, the result is a scaled version of $\vec{x}$ itself These vectors lie along directions that are preserved by the transformation: • $\lvert\lambda\rvert > 1$: stretched • $0 < \lvert\lambda\rvert < 1$: shrunk • $\lambda < 0$: flipped • $\lambda = 1$: stay the same
Null Space (kernel)	$\text{Null}(A) = \{\vec{x} \in \mathbb{R}^n \mid A\vec{x} = \mathbf{0}\}$	The null space of a matrix A is the set of all input vectors that get mapped to the zero vector when you multiply them by A
Identity Matrix	$I_n = \begin{bmatrix} 1 & 0 & 0 & \dots & 0 \\ 0 & 1 & 0 & \dots & 0 \\ 0 & 0 & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & 1 \end{bmatrix}$	
Matrix Inverse	$A \cdot A^{-1} = I$	
RREF	1. Row Swapping (Interchange) $R_1 \leftrightarrow R_2$	

	2. Row Scaling (Multiplication) $$R_1 \to \frac{1}{3}R_1$$ 3. Row Addition (Replacement) $$R_1 \to R_1 - 2R_2$$	

5.1. Matrix

$$\overset{n}{\underset{m}{\begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}}}$$

5.1.1. Matrix Vector Product

$$\overset{n}{\underset{m}{\begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}}} \quad \overset{}{\underset{n}{\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}}}$$

$$\begin{bmatrix} a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n \\ a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n \\ \vdots \\ a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n \end{bmatrix}$$

5.1.2. Matrix Multiplication

p

n

$$A = \begin{matrix} m \end{matrix} \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix} \qquad B = \begin{matrix} n \end{matrix} \begin{bmatrix} b_{11} & b_{12} & \dots & b_{1p} \\ b_{21} & b_{22} & \dots & b_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ b_{n1} & b_{n2} & \dots & b_{np} \end{bmatrix}$$

$$A = \begin{matrix} m \end{matrix} \begin{bmatrix} \overset{n}{\begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \end{bmatrix}} \\ \begin{bmatrix} a_{21} & a_{22} & \dots & a_{2n} \end{bmatrix} \\ \vdots \\ \begin{bmatrix} a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix} \end{bmatrix} \qquad B = \begin{matrix} n \end{matrix} \begin{bmatrix} \overset{p}{\begin{bmatrix} b_{11} \\ b_{21} \\ \vdots \\ b_{n1} \end{bmatrix}} & \overset{}{\begin{bmatrix} b_{12} \\ b_{22} \\ \vdots \\ b_{n2} \end{bmatrix}} & \dots & \overset{}{\begin{bmatrix} b_{1p} \\ b_{2p} \\ \vdots \\ b_{np} \end{bmatrix}} \end{bmatrix}$$

A: Row Representation

$$A = \begin{bmatrix} r_1 \\ r_2 \\ \vdots \\ r_m \end{bmatrix}$$

$$r_i = \begin{bmatrix} a_{i1} & a_{i2} & \dots & a_{in} \end{bmatrix}, \quad \text{for } i = 1, 2, ..., m$$

B: Column Representation

$$B = \begin{bmatrix} c_1 & c_2 & \dots & c_p \end{bmatrix}$$

$$c_j = \begin{bmatrix} b_1 j \\ b_2 j \\ \vdots \\ b_{nj} \end{bmatrix}, \quad \text{for } j = 1, 2, ..., p$$

$$C = \begin{matrix} m \end{matrix} \begin{bmatrix} \overset{p}{r_1 \cdot c_1} & r_1 \cdot c_2 & \dots & r_1 \cdot c_p \\ r_2 \cdot c_1 & r_2 \cdot c_2 & \dots & r_2 \cdot c_p \\ \vdots & \vdots & \ddots & \vdots \\ r_m \cdot c_1 & r_m \cdot c_2 & \dots & r_m \cdot c_p \end{bmatrix}$$

5.1.3. Transpose

$$A^T$$

m

n

$$A = \begin{matrix} m \end{matrix} \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} \qquad A^T = \begin{matrix} n \end{matrix} \begin{bmatrix} a_{11} & a_{21} & \cdots & a_{m1} \\ a_{12} & a_{22} & \cdots & a_{m2} \\ \vdots & \vdots & \ddots & \vdots \\ a_{1n} & a_{2n} & \cdots & a_{mn} \end{bmatrix}$$

$$A = \begin{matrix} m \end{matrix} \begin{bmatrix} \overset{n}{\overbrace{\begin{matrix} a_{11} & a_{12} & \cdots & a_{1j} & \cdots & a_{1n} \end{matrix}}} \\ \begin{matrix} a_{21} & a_{22} & \cdots & a_{2j} & \cdots & a_{2n} \end{matrix} \\ \begin{matrix} \vdots & \vdots & & \vdots & & \vdots \end{matrix} \\ \begin{matrix} a_{i1} & a_{i2} & \cdots & a_{ij} & \cdots & a_{in} \end{matrix} \\ \begin{matrix} \vdots & \vdots & & \vdots & & \vdots \end{matrix} \\ \begin{matrix} a_{m1} & a_{m2} & \cdots & a_{ij} & \cdots & a_{mn} \end{matrix} \end{bmatrix} \qquad A^T = \begin{matrix} n \end{matrix} \begin{bmatrix} \overset{m}{\overbrace{\begin{matrix} a_{11} & a_{21} & \cdots & a_{i1} & \cdots & a_{m1} \end{matrix}}} \\ \begin{matrix} a_{12} & a_{22} & \cdots & a_{i2} & \cdots & a_{m2} \end{matrix} \\ \begin{matrix} \vdots & \vdots & & \vdots & & \vdots \end{matrix} \\ \begin{matrix} a_{1j} & a_{2j} & \cdots & a_{ij} & \cdots & a_{mj} \end{matrix} \\ \begin{matrix} \vdots & \vdots & & \vdots & & \vdots \end{matrix} \\ \begin{matrix} a_{1n} & a_{2n} & \cdots & a_{in} & \cdots & a_{mn} \end{matrix} \end{bmatrix}$$

5.2. Matrix Factorization

5.2.1. LU Decomposition

$$A = LU$$

where:

- L: lower triangular matrix (entries above the diagonal are zero)
- U: upper triangular matrix (entries below the diagonal are zero)

1. $m \times n$ Matrix (with $m \geq n$)

$$L = \begin{matrix} m \end{matrix} \begin{bmatrix} \overset{n}{\overbrace{\begin{matrix} l_{11} & l_{12} & \cdots & l_{1n} \end{matrix}}} \\ \begin{matrix} l_{21} & l_{22} & \cdots & l_{2n} \end{matrix} \\ \begin{matrix} \vdots & \vdots & \ddots & \vdots \end{matrix} \\ \begin{matrix} l_{m1} & l_{m2} & \cdots & l_{mn} \end{matrix} \end{bmatrix} \qquad U = \begin{matrix} n \end{matrix} \begin{bmatrix} \overset{n}{\overbrace{\begin{matrix} u_{11} & u_{12} & \cdots & u_{1n} \end{matrix}}} \\ \begin{matrix} u_{21} & u_{22} & \cdots & u_{2n} \end{matrix} \\ \begin{matrix} \vdots & \vdots & \ddots & \vdots \end{matrix} \\ \begin{matrix} u_{n1} & u_{n2} & \cdots & u_{nn} \end{matrix} \end{bmatrix}$$

2. $m \times n$ Matrix (with $m < n$)

see QR Decomposition

5.3. Singular Value Decomposition (SVD)

$$A = U\Sigma V^T$$

$$U = \begin{matrix} \\ m \end{matrix} \overset{m}{\begin{bmatrix} u_{11} & u_{12} & \cdots & u_{1m} \\ u_{21} & u_{22} & \cdots & u_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ u_{m1} & u_{m2} & \cdots & u_{mm} \end{bmatrix}} \quad \Sigma = \begin{matrix} \\ m \end{matrix} \overset{n}{\begin{bmatrix} \varepsilon_{11} & \varepsilon_{12} & \cdots & \varepsilon_{1m} \\ \varepsilon_{21} & \varepsilon_{22} & \cdots & \varepsilon_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ \varepsilon_{n1} & \varepsilon_{n2} & \cdots & \varepsilon_{nm} \end{bmatrix}} \quad V^T = \begin{matrix} \\ n \end{matrix} \overset{n}{\begin{bmatrix} v_{11} & v_{12} & \cdots & v_{1n} \\ v_{21} & v_{22} & \cdots & v_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ v_{n1} & v_{n2} & \cdots & v_{nn} \end{bmatrix}}$$

	$\left[\begin{array}{ccc\|c} 1 & 2 & -1 & 2 \\ 2 & 3 & 1 & 5 \\ 3 & 4 & -2 & 4 \end{array}\right]$
$R_2 \to R_2 - 2R_1$ $R_3 \to R_3 - 3R_1$	$\left[\begin{array}{ccc\|c} 1 & 2 & -1 & 2 \\ 0 & -1 & 3 & 1 \\ 0 & -2 & 1 & -2 \end{array}\right]$
$R_2 \to -R_2$	$\left[\begin{array}{ccc\|c} 1 & 2 & -1 & 2 \\ 0 & 1 & -3 & -1 \\ 0 & -2 & 1 & -2 \end{array}\right]$
$R_1 \to R_1 - 2R_2$ $R_3 \to R_3 + 2R_2$	$\left[\begin{array}{ccc\|c} 1 & 0 & 5 & 4 \\ 0 & 1 & -3 & -1 \\ 0 & 0 & -5 & -4 \end{array}\right]$
$R_3 \to \frac{1}{-5} R_3$	$\left[\begin{array}{ccc\|c} 1 & 0 & 5 & 4 \\ 0 & 1 & -3 & -1 \\ 0 & 0 & 1 & \frac{4}{5} \end{array}\right]$
$R_1 \to R_1 - 5R_3$ $R_2 \to R_2 + 3R_3$	$\left[\begin{array}{ccc\|c} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & \frac{7}{5} \\ 0 & 0 & 1 & \frac{4}{5} \end{array}\right]$

Example

$$A = \begin{bmatrix} 2 & -1 & -3 \\ -4 & 2 & 6 \end{bmatrix}$$

1. Null Space

The **null-space** of A, denoted $N(A)$, consists of all vectors $\vec{x} \in \mathbb{R}^3$ such that $Ax = 0$. The set of all such vectors is the **pre-image** of the zero vector under the transformation defined by A. In other words, $N(A) = \{x \in \mathbb{R}^3 \mid A\vec{x} = 0\}$, which represents the set of vectors that A maps to zero.

$$N(A) = \{\vec{x} \in \mathbb{R}^3 \mid A\vec{x} = \mathbf{0}\}$$

To find the null space $N(A)$ of the matrix A, we can use the **row-reduced echelon form (RREF)**. By augmenting the matrix A with a zero column and performing row operations, we reduce it to the form:

$$\begin{bmatrix} 2 & -1 & -3 \\ -4 & 2 & 6 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

	$\left[\begin{array}{ccc\|c} 2 & -1 & -3 & 0 \\ -4 & 2 & 6 & 0 \end{array}\right]$
$R_1 \to \frac{R_1}{2}$ $R_2 \to \frac{R_2}{4}$	$\left[\begin{array}{ccc\|c} 1 & -\frac{1}{2} & -\frac{3}{2} & 0 \\ -1 & \frac{1}{2} & \frac{3}{2} & 0 \end{array}\right]$
$R_2 \to R_2 - R_1$	$\left[\begin{array}{ccc\|c} 1 & -\frac{1}{2} & -\frac{3}{2} & 0 \\ 0 & 0 & 0 & 0 \end{array}\right]$

$$\begin{bmatrix} 1 & -\frac{1}{2} & -\frac{3}{2} \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$x_1 - \frac{1}{2}x_2 - \frac{3}{2}x_2 = 0$$

$$x_1 = \frac{1}{2}x_2 + \frac{3}{2}x_2$$

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = x_2 \begin{bmatrix} \frac{1}{2} \\ 1 \\ 0 \end{bmatrix} + x_3 \begin{bmatrix} \frac{3}{2} \\ 0 \\ 1 \end{bmatrix}$$

$$N(a) = \text{span}\left(\left\{\begin{bmatrix} \frac{1}{2} \\ 1 \\ 0 \end{bmatrix} \begin{bmatrix} \frac{3}{2} \\ 0 \\ 1 \end{bmatrix}\right\}\right)$$

The dimension of the **null-space** is the number of vectors in this basis, which is 2. This is important because the dimension of the null space gives us insight into how many degrees of freedom exist in the system of equations $Ax = 0$

2. Column Space

$$C(A) = \text{span}\left(\left\{\begin{bmatrix} 2 \\ -4 \end{bmatrix} \begin{bmatrix} -1 \\ 2 \end{bmatrix} \begin{bmatrix} -3 \\ 6 \end{bmatrix}\right\}\right)$$

$$= \text{span}\left(\left\{\begin{bmatrix} 2 \\ -4 \end{bmatrix}\right\}\right)$$

3. Basis

$$\begin{bmatrix} 2 \\ -4 \end{bmatrix}$$

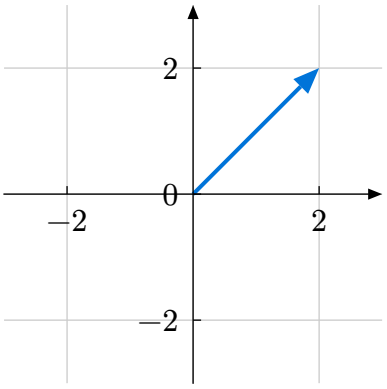
4. Rank

Number of vector in the basis of our column space

$$\text{Rank}(A) = 1$$

6. Vectors

$$\vec{v} = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}$$



7. Real Coordinate Spaces

N-dimensional Real Coordinate Space

$$\mathbb{R}^n$$

$$\vec{x} \in \mathbb{R}^n$$

All possible real-valued n-tuples

$$\vec{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

8. Vector Operations

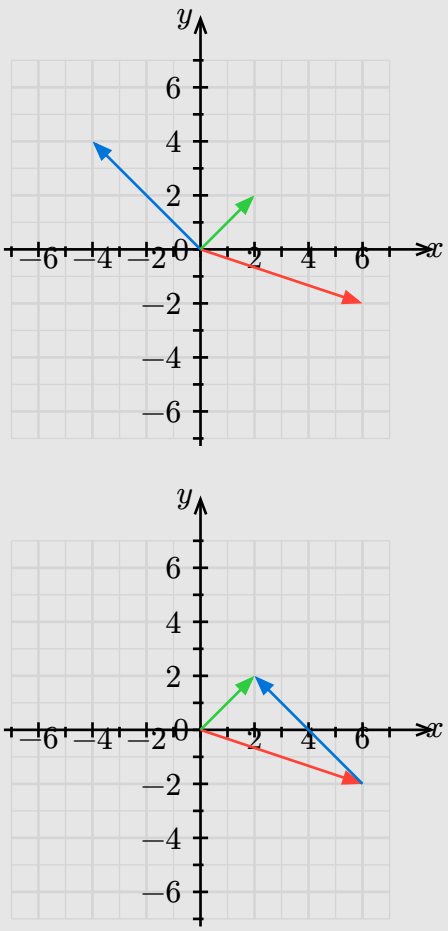
8.1. Vector Addition

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} x_1 + y_1 \\ x_2 + y_2 \\ x_3 + y_3 \end{bmatrix}$$

Example

$$\vec{a} = \begin{bmatrix} 6 \\ -2 \end{bmatrix} \qquad \vec{b} = \begin{bmatrix} -4 \\ 4 \end{bmatrix}$$

$$\vec{a} + \vec{b} = \begin{bmatrix} 6 + -4 \\ -2 + 4 \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \end{bmatrix}$$



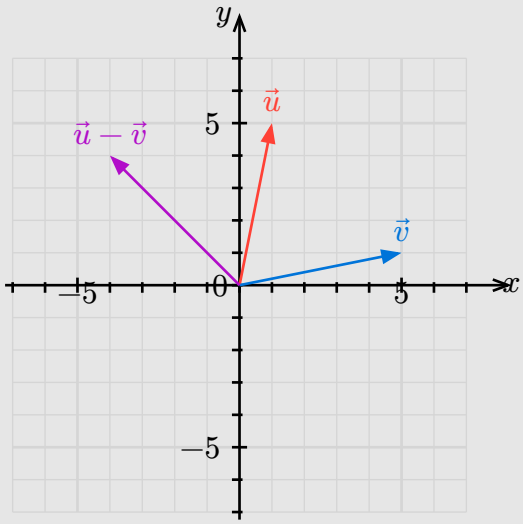
8.2. Vector Subtraction

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} - \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} x_1 - y_1 \\ x_2 - y_2 \\ x_3 - y_3 \end{bmatrix}$$

Example

$$\vec{u} = \begin{bmatrix} 1 \\ 5 \end{bmatrix} \qquad \vec{v} = \begin{bmatrix} 5 \\ 1 \end{bmatrix}$$

$$\vec{u} - \vec{v} = \begin{bmatrix} 1 - 5 \\ 5 - 1 \end{bmatrix} = \begin{bmatrix} -4 \\ 4 \end{bmatrix}$$



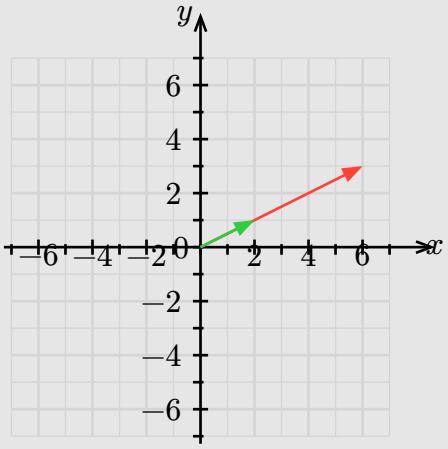
8.3. Scalar Multiplication

$$c \times \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} c \times x_1 \\ c \times x_2 \\ c \times x_3 \end{bmatrix}$$

Example

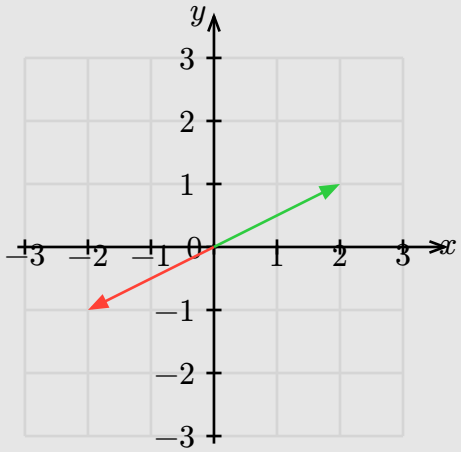
$$\vec{a} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

$$3\vec{a} = 3 \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \cdot 2 \\ 3 \cdot 1 \end{bmatrix} = \begin{bmatrix} 6 \\ 3 \end{bmatrix}$$



$$\vec{a} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

$$-1\vec{a} = -1 \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \cdot 2 \\ -1 \cdot 1 \end{bmatrix} = \begin{bmatrix} -2 \\ -1 \end{bmatrix}$$



9. Norm

A scalar that measures the **length** (or **magnitude**) of a vector.

Notation: $\|\vec{v}\|$.

9.1. Euclidean norm (ℓ_2)

The default norm — Pythagoras applied componentwise.

For $\vec{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix} \in \mathbb{R}^n$:

$$\|\vec{v}\| = \sqrt{v_1^2 + v_2^2 + ... + v_n^2}$$

Example

$$\vec{v} = \begin{bmatrix} 3 \\ 4 \end{bmatrix} \qquad \|\vec{v}\| = \sqrt{3^2 + 4^2} = \sqrt{25} = 5$$

Geometrically, this is the distance from the origin to the point $(3, 4)$.

9.2. From dot product

The Euclidean norm is the dot product of a vector with itself, square-rooted:

$$\|\vec{v}\| = \sqrt{\vec{v} \cdot \vec{v}}$$

(See Dot Product.)

This connection makes norms central to projections, angles between vectors, and least-squares regression.

9.3. Properties (norm axioms)

For any vectors $\vec{u}, \vec{v} \in \mathbb{R}^n$ and scalar $c \in \mathbb{R}$:

- **Non-negativity**: $\|\vec{v}\| \geq 0$
- **Definiteness**: $\|\vec{v}\| = 0 \quad \Leftrightarrow \quad \vec{v} = \mathbf{0}$
- **Absolute homogeneity**: $\|c\,\vec{v}\| = |c|\,\|\vec{v}\|$
- **Triangle inequality**: $\|\vec{u} + \vec{v}\| \leq \|\vec{u}\| + \|\vec{v}\|$ (see Triangle Inequality)

Any function $\mathbb{R}^n \to \mathbb{R}$ satisfying these four axioms is a **norm**.

9.4. Geometric interpretation

- $\|\vec{v}\|$ = distance from the origin to the point $\vec{v}$
- $\|\vec{u} - \vec{v}\|$ = distance between the two points $\vec{u}$ and $\vec{v}$
- The set $\{\vec{x} : \|\vec{x}\| = 1\}$ is the **unit sphere** (a circle in $\mathbb{R}^2$, sphere in $\mathbb{R}^3$)

9.5. Other common norms (ℓ_p family)

For $p \geq 1$:

$$\|\vec{v}\|_p = \left(\sum_{i=1}^{n} |v_i|^p \right)^{\frac{1}{p}}$$

Special cases:

Norm	Formula	Geometry of unit ball
ℓ_1 (Manhattan / taxicab)	$\sum_i \lvert v_i \rvert$	diamond
ℓ_2 (Euclidean — default)	$\sqrt{\sum_i v_i^2}$	circle / sphere
ℓ_∞ (Chebyshev / max)	$\max_i \lvert v_i \rvert$	square / cube

Example

$\vec{v} = \begin{bmatrix} 3 \\ -4 \end{bmatrix}$:

$$\|\vec{v}\|_1 = |3| + |-4| = 7$$
$$\|\vec{v}\|_2 = \sqrt{9+16} = 5$$
$$\|\vec{v}\|_\infty = \max(3,4) = 4$$

9.6. Normalizing a vector

Dividing a vector by its norm gives a unit vector in the same direction:

$$\hat{v} = \frac{\vec{v}}{\|}\vec{v}\|, \quad \|\hat{v}\| = 1$$

9.7. Connections

- **Dot Product** — $\|\vec{v}\|^2 = \vec{v} \cdot \vec{v}$
- **Cauchy–Schwarz Inequality** — $|\vec{u} \cdot \vec{v}| \leq \|\vec{u}\| \, \|\vec{v}\|$
- **Triangle Inequality** — $\|\vec{u} + \vec{v}\| \leq \|\vec{u}\| + \|\vec{v}\|$
- **Unit Vector** — vectors with $\|\vec{v}\| = 1$
- **Angles Between Vectors** — $\cos(\theta) = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \, \|\vec{v}\|}$

10. Unit Vector

A vector that has a norm (length) of exactly 1.

For a vector $\vec{v}$ in n-dimensional space, the unit vector $\hat{v}$ in the same direction is:

$$\hat{v} = \frac{\vec{v}}{\| \vec{v} \|}$$

Key properties:

- $\|\hat{v}\| = 1$
- $\hat{v}$ points in the same direction as $\vec{v}$

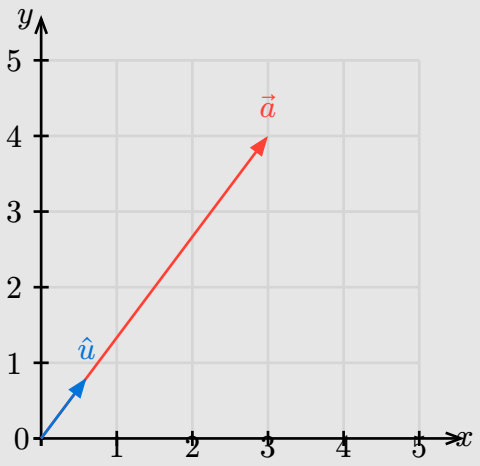
Example

Find the unit vector with the same direction as $\vec{a} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$.

$$\| \vec{a} \| = \sqrt{3^2 + 4^2} = \sqrt{25} = 5$$

$$\hat{u} = \frac{\vec{a}}{\|}\vec{a}\| = \begin{bmatrix} \frac{3}{5} \\ \frac{4}{5} \end{bmatrix}$$

$$\| \hat{u} \| = \sqrt{\left(\frac{3}{5}\right)^2 + \left(\frac{4}{5}\right)^2} = \sqrt{\frac{9}{25} + \frac{16}{25}} = \sqrt{1} = 1 \ \checkmark$$



11. Dot Product

The **dot product** (or **scalar product**) of two vectors of the same dimension returns a scalar — the sum of componentwise products.

$$\vec{a} \cdot \vec{b} = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix} \cdot \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix} = a_1 b_1 + a_2 b_2 + \ldots + a_n b_n = \sum_{i=1}^{n} a_i b_i$$

Example

$\vec{a} = \begin{bmatrix} 2 \\ -1 \\ 3 \end{bmatrix}, \vec{b} = \begin{bmatrix} 4 \\ 5 \\ -2 \end{bmatrix}$:

$$\vec{a} \cdot \vec{b} = 2 \cdot 4 + (-1) \cdot 5 + 3 \cdot (-2) = 8 - 5 - 6 = -3$$

11.1. Geometric formula

$$\vec{a} \cdot \vec{b} = \|\vec{a}\| \, \|\vec{b}\| \cos\theta$$

where θ is the angle between $\vec{a}$ and $\vec{b}$ — and $\| \cdot \|$ is the norm.

This links the algebraic dot product to geometric intuition: the dot product measures how much the vectors point in the same direction.

11.2. Connection to norm

The square of the norm is the dot product of a vector with itself:

$$\|\vec{a}\|^2 = \vec{a} \cdot \vec{a}$$

See Norm for the full story.

11.3. Properties

Commutative:

$$\vec{v} \cdot \vec{w} = \vec{w} \cdot \vec{v}$$

Distributive over vector addition:

$$(\vec{v} + \vec{w}) \cdot \vec{x} = \vec{v} \cdot \vec{x} + \vec{w} \cdot \vec{x}$$

Compatible with scalar multiplication:

$$(c\vec{v}) \cdot \vec{w} = c(\vec{v} \cdot \vec{w})$$

Bilinear: linear in each argument separately (the three properties above combined).

11.4. Matrix form

For column vectors, the dot product is just matrix multiplication of $\vec{a}^T$ with $\vec{b}$:

$$\vec{a} \cdot \vec{b} = \vec{a}^T \vec{b}$$

(See Transpose.)

11.5. See also

- **Norm** — $\|\vec{v}\| = \sqrt{\vec{v} \cdot \vec{v}}$
- **Angles Between Vectors** — $\cos\theta = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \, \|\vec{b}\|}$
- **Cauchy–Schwarz Inequality**
- **Triangle Inequality**
- **Orthogonality** — $\vec{a} \perp \vec{b} \Leftrightarrow \vec{a} \cdot \vec{b} = 0$
- **Cross Product** + **Dot vs Cross**
- **Inner Product** — generalization

12. Inner Product

An **inner product** is a function $\langle \cdot, \cdot \rangle : V \times V \to \mathbb{R}$ that generalizes the dot product to arbitrary vector spaces.

12.1. Axioms

For all $\vec{u}, \vec{v}, \vec{w} \in V$ and scalar $c \in \mathbb{R}$:

1. **Symmetry**: $\langle \vec{u}, \vec{v} \rangle = \langle \vec{v}, \vec{u} \rangle$
2. **Linearity in the first argument**:

$$\langle c\vec{u} + \vec{w}, \vec{v} \rangle = c\langle \vec{u}, \vec{v} \rangle + \langle \vec{w}, \vec{v} \rangle$$

(by symmetry, also linear in the second argument)

3. **Positive-definiteness**: $\langle \vec{v}, \vec{v} \rangle \geq 0$, with equality only when $\vec{v} = \mathbf{0}$

A vector space with an inner product is an **inner product space**.

12.2. Standard examples

Dot product on $\mathbb{R}^n$: $\langle \vec{u}, \vec{v} \rangle = \sum_{i=1}^{n} u_i v_i = \vec{u}^T \vec{v}$

Weighted inner product: $\langle \vec{u}, \vec{v} \rangle_W = \vec{u}^T W \vec{v}$ where W is symmetric positive-definite

Function space inner product (continuous functions on $[a, b]$):

$$\langle f, g \rangle = \int_a^b f(x) g(x)\, dx$$

Frobenius inner product on matrices:

$$\langle A, B \rangle_F = \operatorname{tr}(A^T B) = \sum_{i,j} A_{ij} B_{ij}$$

12.3. Induced norm

Every inner product induces a norm:

$$\|\vec{v}\| = \sqrt{\langle \vec{v}, \vec{v} \rangle}$$

This norm satisfies the parallelogram law $\|\vec{u} + \vec{v}\|^2 + \|\vec{u} - \vec{v}\|^2 = 2\,\|\vec{u}\|^2 + 2\,\|\vec{v}\|^2$ — a necessary and sufficient condition for a norm to come from an inner product.

12.4. Cauchy–Schwarz (general form)

The Cauchy–Schwarz inequality holds in every inner product space:

$$|\langle \vec{u}, \vec{v} \rangle| \leq \|\vec{u}\|\,\|\vec{v}\|$$

12.5. Orthogonality

Two vectors are **orthogonal** if their inner product is zero:

$$\vec{u} \perp \vec{v} \;\Leftrightarrow\; \langle \vec{u}, \vec{v} \rangle = 0$$

This generalizes perpendicularity in $\mathbb{R}^n$. See Orthogonality.

12.6. Angle between vectors

$$\cos\theta = \frac{\langle \vec{u}, \vec{v} \rangle}{\|\vec{u}\|\,\|\vec{v}\|}$$

In abstract spaces this **defines** the angle (between non-zero vectors).

12.7. Why inner products matter

- **Generalize geometry** (length, angle, projection) to any vector space
- **Function spaces**: e.g., Hilbert spaces in functional analysis use $\langle f, g \rangle = \int fg$
- **Quantum mechanics**: state space is a complex inner product space (Hilbert space)

- **Statistics**: covariance is an inner product on centered random variables
- **Machine learning**: kernel methods replace explicit inner products with implicit ones via **kernels**

12.8. See also

- **Dot Product** — special case in $\mathbb{R}^n$
- **Norm** — derived from the inner product
- **Orthogonality**
- **Gram–Schmidt Process** — orthogonalization via inner products
- **Cauchy–Schwarz Inequality**

13. Cauchy–Schwarz Inequality

$$|\vec{u} \cdot \vec{v}| \leq \|\vec{u}\| \, \|\vec{v}\|$$
$$|\vec{u} \cdot \vec{v}| = \|\vec{u}\| \, \|\vec{v}\| \quad \text{when} \quad \vec{u} = c\vec{v}$$

Where:

- $\vec{u} \cdot \vec{v}$: dot product of vectors $\vec{u}$ and $\vec{v}$
- $\|\vec{u}\|$ and $\|\vec{v}\|$: magnitudes (lengths) of vectors $\vec{u}$ and $\vec{v}$

Step 1: Understand the dot product

The dot product of two vectors $\vec{u} = [u_1, u_2, ..., u_n]$ and $\vec{v} = [v_1, v_2, ..., v_n]$ is calculated as:

$$\vec{u} \cdot \vec{v} = u_1 v_1 + u_2 v_2 + ... + u_n v_n$$

The magnitude (or norm) of a vector $\vec{u}$ is:

$$\|\vec{u}\| = \sqrt{u_1^2 + u_2^2 + ... + u_n^2}$$

Step 2: Define a new function

We introduce a parameter $t \in \mathbb{R}$ and define a new vector:

$$w(t) = u - tv$$

Now, consider the dot product of this new vector with itself, which is always non-negative because it represents the square of the magnitude of $w(t)$:

$$w(t) \cdot w(t) \geq 0$$

This inequality makes sense because the dot product of any vector with itself is the square of its magnitude, and **a square is always non-negative**.

Step 3: Expand the dot product

Expand $w(t) \cdot w(t)$:

$$w(t) \cdot w(t) = (\vec{u} - t\vec{v}) \cdot (\vec{u} - t\vec{v})$$

Now, we apply the distributive property of the dot product, which behaves similarly to the distributive property of multiplication. We expand each term:

$$(\vec{u} - t\vec{v}) \cdot (\vec{u} - t\vec{v}) = \vec{u} \cdot \vec{u} - t(\vec{u} \cdot \vec{v}) - t(\vec{v} \cdot \vec{u}) + t^2(\vec{v} \cdot \vec{v})$$

Since the dot product is commutative ($\vec{u} \cdot \vec{v} = \vec{v} \cdot \vec{u}$), we can rewrite this as:

$$\vec{u} \cdot \vec{u} - 2t(\vec{u} \cdot \vec{v}) + t^2(\vec{v} \cdot \vec{v})$$

This simplifies to:

$$\|\vec{u}\|^2 - 2t(\vec{u} \cdot \vec{v}) + t^2 \ \|\vec{v}\|^2$$

We've now expressed the result of expanding the dot product as a quadratic expression in t, where

- $\|\vec{u}\|^2$ is a constant term,
- $-2t(\vec{u} \cdot \vec{v})$ is the linear term in t
- $t^2 \ \|\vec{v}\|^2$ is the quadratic term

Step 4: Treat as a quadratic equation

Now that we have the quadratic expression:

$$\|\vec{v}\|^2 \ t^2 - 2(\vec{u} \cdot \vec{v})t + \|\vec{u}\|^2 \geq 0$$

We recognize this as a standard quadratic inequality of the form $at^2 + bt + c \geq 0$, where:

- $a = \|\vec{v}\|^2$
- $b = -2(\vec{u} \cdot \vec{v})$
- $c = \|\vec{u}\|^2$

For any quadratic expression $at^2 + bt + c$ to always be non-negative, its discriminant must be less than or equal to zero. The discriminant of a quadratic equation $at^2 + bt + c = 0$ is given by:

$$\Delta = b^2 - 4ac$$

Substituting in the values of a, b, and c from our expression:

$$\Delta = (-2(\vec{u} \cdot \vec{v}))^2 - 4 \cdot \|\vec{v}\|^2 \cdot \|\vec{u}\|^2$$

Simplifying:

$$\Delta = 4(\vec{u} \cdot \vec{v})^2 - 4 \ \|\vec{v}\|^2 \ \|\vec{u}\|^2$$

Step 5: Apply the discriminant condition

For the quadratic inequality to hold, the discriminant must be less than or equal to zero:

$$\Delta = 4(\vec{u} \cdot \vec{v})^2 - 4 \ \|\vec{v}\|^2 \ \|\vec{u}\|^2 \leq 0$$

Divide by 4:

$$\Delta = (\vec{u} \cdot \vec{v})^2 \leq \|\vec{v}\|^2 \ \|\vec{u}\|^2$$

Take the square root of both sides:

$$|\vec{u} \cdot \vec{v}| \leq \|\vec{u}\| \ \|\vec{v}\|$$

Example

$$\vec{u} = \begin{bmatrix} 1 \\ \\ 2 \end{bmatrix} \qquad \vec{v} = \begin{bmatrix} 3 \\ \\ 4 \end{bmatrix}$$

Step 1: Compute the dot product $\vec{u} \cdot \vec{v}$

$$\vec{u} \cdot \vec{v} = (1)(3) + (2)(4) = 3 + 8 = 11$$

Step 2: Compute the norms of $\vec{u}$ and $\vec{v}$

- The norm $\|\vec{u}\|$ is:

$$\|\vec{u}\| = \sqrt{1^2 + 2^2} = \sqrt{1 + 4} = \sqrt{5}$$

- The norm $\|\vec{v}\|$ is:

$$\|\vec{v}\| = \sqrt{3^2 + 4^2} = \sqrt{9 + 16} = \sqrt{25} = 5$$

Step 3: Verify the Cauchy-Schwarz inequality

The inequality states:

$$|\vec{u} \cdot \vec{v}| \leq \|\vec{u}\| \, \|\vec{v}\|$$

Substitute the values:

$$|11| \leq 5\sqrt{5} = 11.18$$

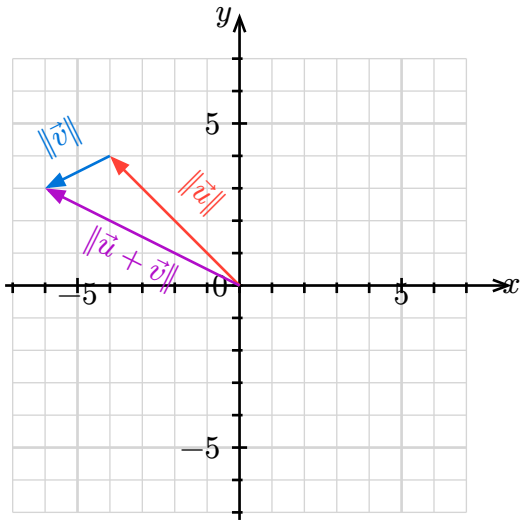
Since $11 \leq 11.18$, the inequality holds.

The Cauchy-Schwarz inequality is satisfied for the vectors $\vec{u} = \begin{bmatrix} 1 \\ \\ 2 \end{bmatrix}$ and $\vec{v} = \begin{bmatrix} 3 \\ \\ 4 \end{bmatrix}$

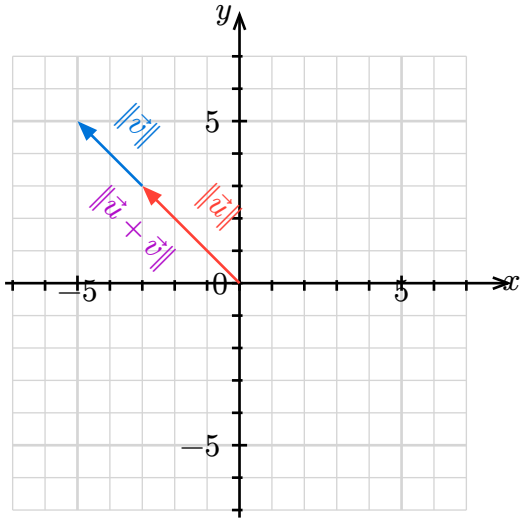
14. Triangle Inequality

14.1. Vector Triangle Inequality

$$\|\vec{u} + \vec{v}\| \leq \|\vec{u}\| + \|\vec{v}\|$$



$$\|\vec{u}+\vec{v}\| = \|\vec{u}\| + \|\vec{v}\|$$



$$\vec{u} = c\vec{v} \qquad c > 0$$

15. Orthogonality

Two vectors $\vec{u}, \vec{v}$ are **orthogonal** (written $\vec{u} \perp \vec{v}$) if their dot product is zero:

$$\vec{u} \perp \vec{v} \Leftrightarrow \vec{u} \cdot \vec{v} = 0$$

In $\mathbb{R}^2$ and $\mathbb{R}^3$, this is the same as **perpendicular**. In higher dimensions or abstract inner product spaces, orthogonality generalizes the notion.

15.1. Orthogonal sets and orthonormal sets

A set $\{\vec{v}_1, ..., \vec{v}_k\}$ is:

- **Orthogonal** if every pair is orthogonal: $\vec{v}_i \cdot \vec{v}_j = 0$ for $i \neq j$
- **Orthonormal** if also each is a unit vector: $\|\vec{v}_i\| = 1$

A useful concise formula uses the Kronecker delta:

$$\vec{v}_i \cdot \vec{v}_j = \delta_{ij} = \begin{cases} 1 \text{ if } i = j \\ 0 \text{ if } i \neq j \end{cases}$$

15.2. Key facts

- **Linear independence**: any orthogonal set of non-zero vectors is automatically linearly independent
- **Pythagoras**: if $\vec{u} \perp \vec{v}$, then $\|\vec{u} + \vec{v}\|^2 = \|\vec{u}\|^2 + \|\vec{v}\|^2$
- **Easy coordinates**: in an orthonormal basis $\{\vec{q}_1, ..., \vec{q}_n\}$, the coordinates of $\vec{x}$ are just dot products:

$$\vec{x} = (\vec{x} \cdot \vec{q}_1)\vec{q}_1 + ... + (\vec{x} \cdot \vec{q}_n)\vec{q}_n$$

15.3. Orthogonal complement

For a subspace $W \subseteq \mathbb{R}^n$, its **orthogonal complement** is:

$$W^\perp = \{\vec{x} \in \mathbb{R}^n : \vec{x} \cdot \vec{w} = 0 \text{ for all } \vec{w} \in W\}$$

— the set of vectors orthogonal to **every** vector in W.

Properties:

- $W^\perp$ is itself a subspace
- $W \cap W^\perp = \{\mathbf{0}\}$
- $\dim(W) + \dim(W^\perp) = n$
- $(W^\perp)^\perp = W$
- $\mathbb{R}^n = W \oplus W^\perp$ (every vector decomposes uniquely as $\vec{w} + \vec{w}^\perp$)

15.4. Four fundamental subspaces (and their complements)

For an $m \times n$ matrix A, four subspaces and two pairings:

$$\underbrace{\text{Col}(A)}_{\subset \mathbb{R}^m} \perp \underbrace{\text{Null}(A^T)}_{\subset \mathbb{R}^m}$$

$$\underbrace{\text{Row}(A)}_{\subset \mathbb{R}^n} \perp \underbrace{\text{Null}(A)}_{\subset \mathbb{R}^n}$$

The column space is orthogonal to the **left** null space; the row space (= column space of A^T) is orthogonal to the null space.

15.5. Orthogonal projection

For a subspace W with orthonormal basis $\{\vec{q}_1, ..., \vec{q}_k\}$, the projection of $\vec{x}$ onto W is:

$$\text{proj}_W(\vec{x}) = \sum_{i=1}^{k} (\vec{x} \cdot \vec{q}_i)\vec{q}_i$$

In matrix form with $Q = [\vec{q}_1 \quad \cdots \quad \vec{q}_k]$: $\text{proj}_W(\vec{x}) = QQ^T\vec{x}$. The matrix $P = QQ^T$ is the **projection matrix** — see Projection.

15.6. Why orthogonality matters

- **Best approximation**: projecting onto a subspace gives the closest point — basis of least-squares fitting
- **Decoupling**: in an orthonormal basis, coordinates are independent — no cross-terms, simpler computations
- **Orthogonal matrices** preserve distances and angles — rigid transformations
- **Spectral theorem**: real symmetric matrices have orthogonal eigenvectors
- **Gram–Schmidt** builds orthonormal bases from any basis

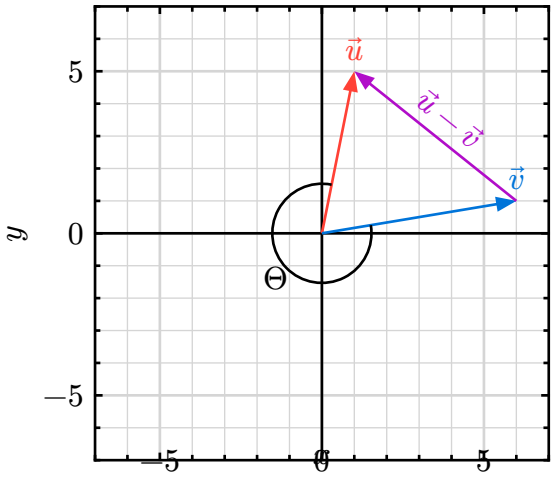
15.7. See also

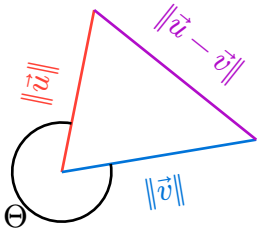
- **Dot Product**
- **Projection**
- **Gram–Schmidt**
- **Orthogonal Matrix**
- **Inner Product** — generalization

16. Angles Between Vectors

The scalar $\|\vec{u}\|$ is the length of the vector $\vec{u}$

Say $\vec{u}, \vec{v} \in \mathbb{R}^n$





Law of Cosines

$$c^2 = a^2 + b^2 - 2ab \cdot \cos(C)$$

Where:

- a, b and c: lengths of the sides of a triangle
- C: angle opposite the side c

$$\|\vec{u}-\vec{v}\|^2 = \|\vec{u}\|^2 + \|\vec{v}\|^2 - 2\,\|\vec{u}\|\,\|\vec{v}\| \cdot \cos(\Theta)$$

$$(\vec{u}-\vec{v}) \cdot (\vec{u}-\vec{v}) =$$

$$\vec{u}\cdot\vec{u} - \vec{u}\cdot\vec{v} - \vec{v}\cdot\vec{u} + \vec{v}\cdot\vec{v} =$$

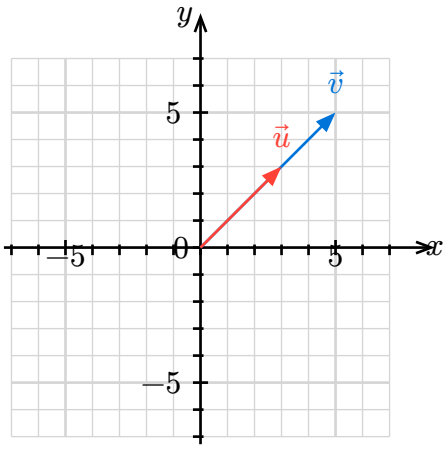
$$\cancel{\|\vec{u}\|^2} \cancel{-2}(\vec{u}\cdot\vec{v}) + \cancel{\|\vec{v}\|} = \cancel{\|\vec{u}\|^2} + \cancel{\|\vec{v}\|^2} \cancel{-2}\,\|\vec{u}\|\,\|\vec{v}\| \cdot \cos(\Theta)$$

$$\boxed{\vec{u}\cdot\vec{v} = \|\vec{u}\|\,\|\vec{v}\| \cdot \cos(\Theta)}$$

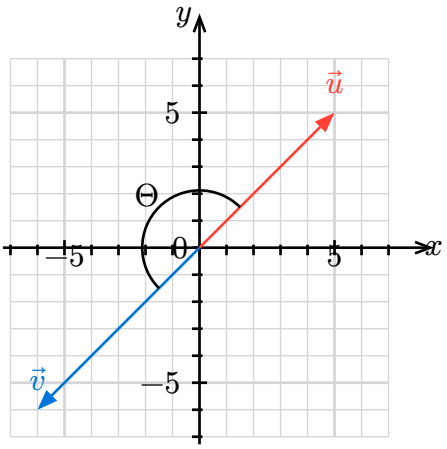
$$\boxed{\frac{\vec{u}\cdot\vec{v}}{\|\vec{u}\|\,\|\vec{v}\|} = \cos(\Theta)}$$

$$\boxed{\Theta = \arccos\left(\frac{\vec{a}\cdot\vec{b}}{\|\vec{a}\|\,\|\vec{b}\|}\right)}$$

So, if $\vec{u}$ is a scalar multiple of $\vec{v}$ ($\vec{u} = c\vec{v}$) where $c > 0$, then $\Theta = 0°$



And, if $\vec{u}$ is a scalar multiple of $\vec{v}$ ($\vec{u} = c\vec{v}$) where $c < 0$, then $\Theta = 180°$



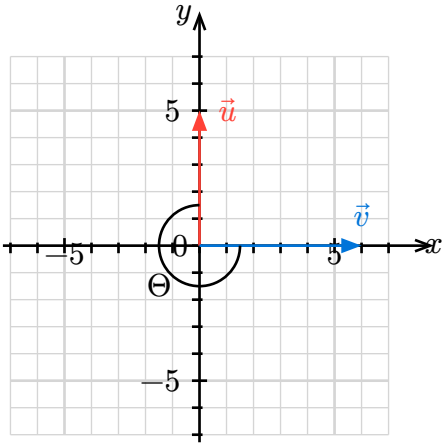
$\vec{u}$ and $\vec{v}$ are perpendicular if the angle Θ between them is $90°$

$$\vec{u} \cdot \vec{v} = \|\vec{u}\| \, \|\vec{v}\| \cdot \cos(90)$$
$$\vec{u} \cdot \vec{v} = 0$$

If $\vec{u}$ and $\vec{v}$ are perpendicular, then $\vec{u} \cdot \vec{v} = 0$

If $\vec{u}$ and $\vec{v}$ are non-zer0 and $\vec{u} \cdot \vec{v} = 0$, then they are perpendicular

If $\vec{u} \cdot \vec{v} = 0$ then $\vec{u}$ and $\vec{v}$ are **orthogonal**.

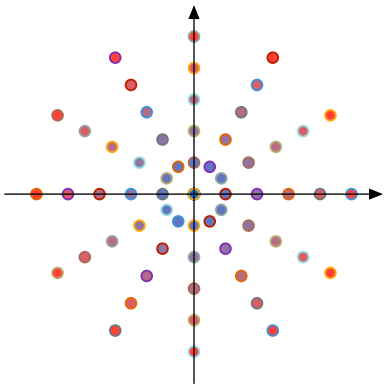


17. Cartesian Coordinates

Describe a point's position using perpendicular axes.

The coordinates are (x, y), where:

- x: horizontal distance from the origin
- y: vertical distance from the origin

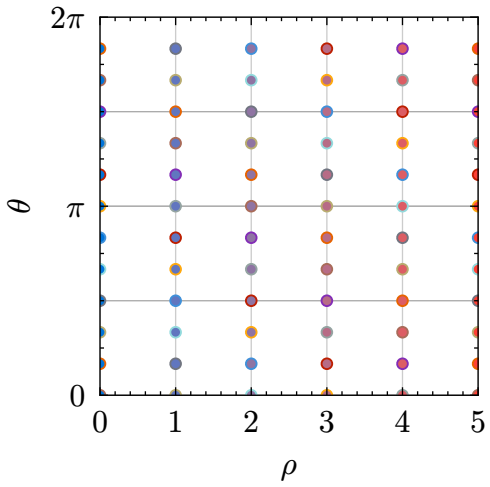


18. Polar Coordinates

Describe a point's position using a distance from the origin and an angle from a reference direction.

The coordinates are (r, θ), where:

- r: distance from the origin
- θ: angle from the positive x-axis (in radians or degrees)



19. Parametric Line

Set L of all points (i.e., line) equal to the set of all vectors $\vec{x}$ plus some scalar t times the vector $\vec{v}$ such that t can be any real number ($\mathbb{R}$)

$$L = \{\vec{x} + t\vec{v} \mid t \in \mathbb{R}\}$$

Example

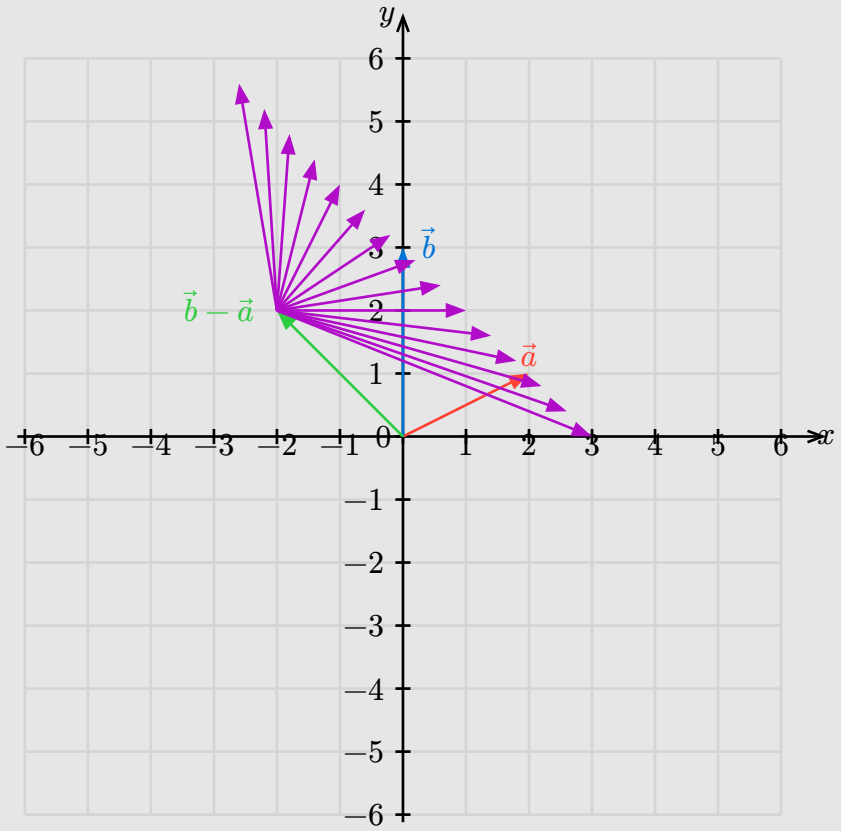
$$\vec{a} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

$$\vec{b} = \begin{bmatrix} 0 \\ 3 \end{bmatrix}$$

$$t = 1$$

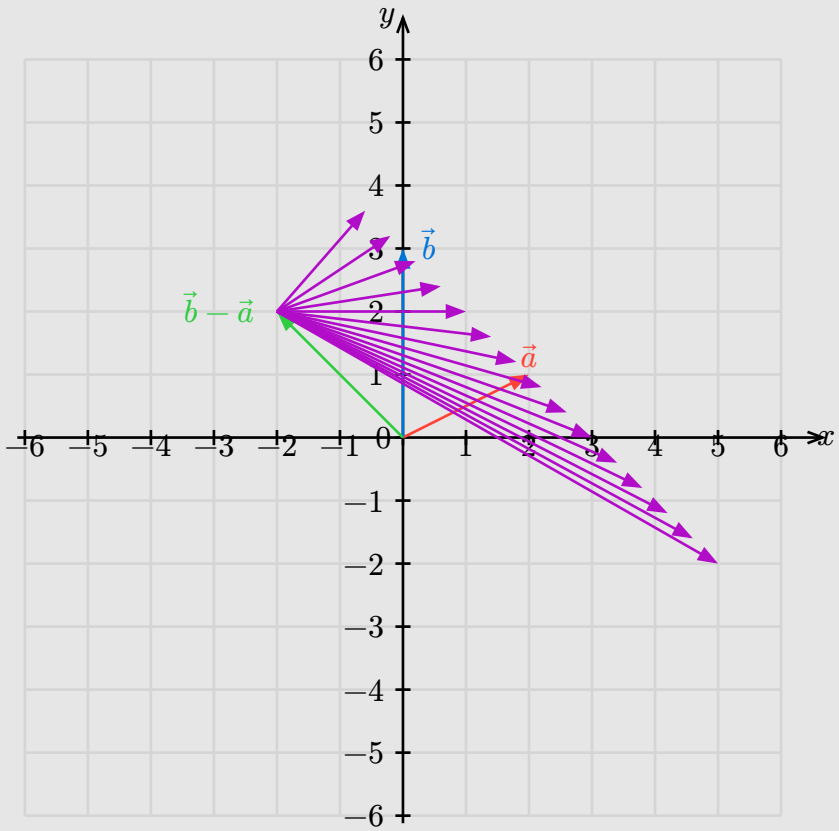
The line L can be defined as:

$$\begin{aligned} \vec{b} + t\left(\vec{b} - \vec{a}\right) &= \begin{bmatrix} 0 \\ 3 \end{bmatrix} + 1\left(\begin{bmatrix} 0 \\ 3 \end{bmatrix} - \begin{bmatrix} 2 \\ 1 \end{bmatrix}\right) \\ &= \begin{bmatrix} 0 \\ 3 \end{bmatrix} + 1\begin{bmatrix} -2 \\ 2 \end{bmatrix} \\ &= \begin{bmatrix} -2 \\ 5 \end{bmatrix} \end{aligned}$$



The line L can also be defined as:

$$\begin{aligned}\vec{a}+t\left(\vec{b}-\vec{a}\right) &= \begin{bmatrix}2\\1\end{bmatrix}+1\left(\begin{bmatrix}0\\3\end{bmatrix}-\begin{bmatrix}2\\1\end{bmatrix}\right)\\ &= \begin{bmatrix}2\\1\end{bmatrix}+1\begin{bmatrix}-2\\2\end{bmatrix}\\ &= \begin{bmatrix}0\\3\end{bmatrix}\end{aligned}$$



Generalization

$$P_1 = \begin{bmatrix}x_1\\x_2\\\vdots\\x_n\end{bmatrix} \qquad P_2 = \begin{bmatrix}x_1\\x_2\\\vdots\\x_n\end{bmatrix}$$

$$L = P_1 + t(P_1 - P_2) \mid t \in \mathbb{R}$$

Example

$$\vec{P}_1 = \begin{bmatrix} -1 \\ 2 \end{bmatrix} \qquad \vec{P}_2 = \begin{bmatrix} 0 \\ 3 \end{bmatrix}$$

$$\vec{P}_1 - \vec{P}_2 = \begin{bmatrix} -1 \\ -1 \end{bmatrix}$$

- L starts at P_1 and moves toward P_2 as t decreases

$$L = P_1 + t(P_1 - P_2) = \begin{bmatrix} -1 \\ 2 \end{bmatrix} + t \begin{bmatrix} -1 \\ -1 \end{bmatrix}$$

$$x = -1 + t(-1)$$
$$y = 2 + t(-1)$$

- L starts at P_2 and moves toward P_1 as t increases

$$L = P_2 + t(P_1 - P_2) = \begin{bmatrix} 0 \\ 3 \end{bmatrix} + t \begin{bmatrix} -1 \\ -1 \end{bmatrix}$$

$$x = 0 + t(-1)$$
$$y = 3 + t(-1)$$

- L starts at P_1 and moves toward P_2 as t increases

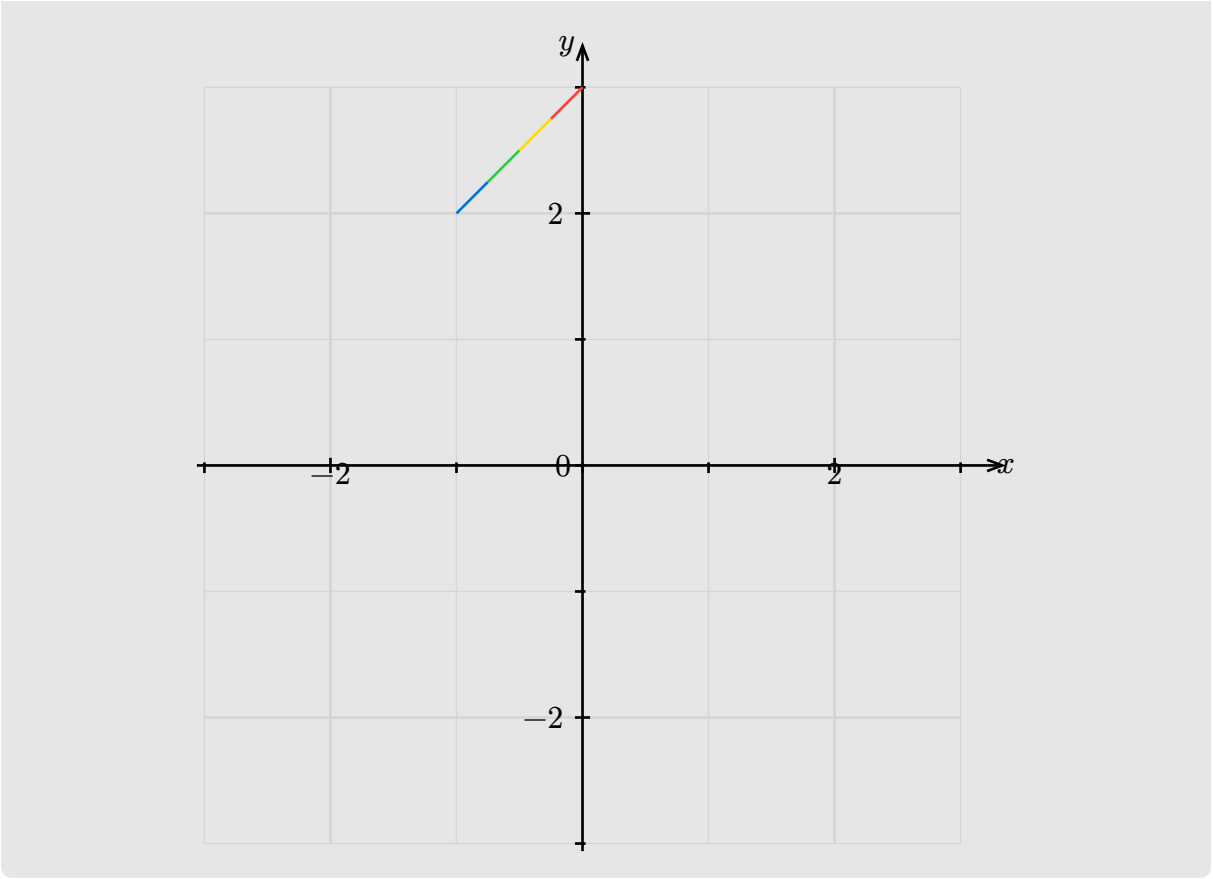
$$L = P_1 + t(P_2 - P_1) = \begin{bmatrix} -1 \\ 2 \end{bmatrix} + t \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$x = -1 + t(1)$$
$$y = 2 + t(1)$$

- L starts at P_2 and moves toward P_1 as t decreases

$$L = P_2 + t(P_2 - P_1) = \begin{bmatrix} 0 \\ 3 \end{bmatrix} + t \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$x = 0 + t(1)$$
$$y = 3 + t(1)$$



20. Planes

A plane in $\mathbb{R}^3$ is the set of points (x, y, z) satisfying a single linear equation:

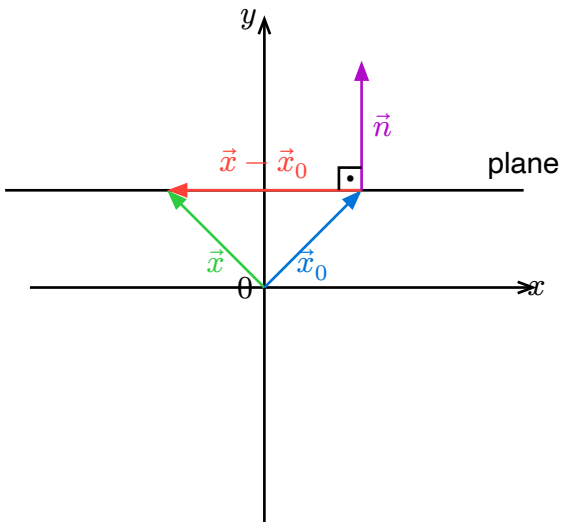
$$Ax + By + Cz = D$$

The vector $\vec{n} = (A, B, C)$ is the **normal vector** — perpendicular to every vector lying in the plane.

20.1. Point–normal form

If $\vec{x}_0$ is a known point on the plane and $\vec{n}$ is the normal, then any other point $\vec{x}$ lies on the plane iff $\vec{x} - \vec{x}_0$ is perpendicular to $\vec{n}$:

$$\vec{n} \cdot (\vec{x} - \vec{x}_0) = 0$$



Expanding componentwise:

$$n_1(x - x_0) + n_2(y - y_0) + n_3(z - z_0) = 0$$

Example

Find the plane equation given $\vec{n} = (1, 3, -2)$ and a point on the plane $\vec{x}_0 = (1, 2, 3)$.

$$1(x - 1) + 3(y - 2) - 2(z - 3) = 0$$

Simplify:

$$x - 1 + 3y - 6 - 2z + 6 = 0 \;\Rightarrow\; x + 3y - 2z = 1$$

20.2. Reading the normal off the equation

In the form $Ax + By + Cz = D$, the coefficients are exactly the components of the normal vector:

$$\vec{n} = (A, B, C)$$

Example

$$-3x + \sqrt{2}y + 7z = \pi \qquad \Rightarrow \qquad \vec{n} = \left(-3, \sqrt{2}, 7\right)$$

20.3. See also

- **Point–Plane Distance** — formula and example
- **Distance Between Planes** — parallel and non-parallel cases
- **Hyperplane** — n-dimensional generalization
- **Cross Product** — building a normal from two in-plane vectors

21. Point–Plane Distance

For a plane in $\mathbb{R}^3$ defined by $Ax + By + Cz = D$ and a point (x_0, y_0, z_0), the perpendicular distance from the point to the plane is:

$$d = \frac{|Ax_0 + By_0 + Cz_0 - D|}{\sqrt{A^2 + B^2 + C^2}}$$

The denominator is the norm of the plane's normal vector $\vec{n} = (A, B, C)$.

The signed distance (drop the absolute value) tells you which side of the plane the point is on:

$$d_{\text{signed}} = \frac{Ax_0 + By_0 + Cz_0 - D}{\sqrt{A^2 + B^2 + C^2}}$$

Example

Plane: $x - 2y + 3z = 5$. Point: $(2, 3, 1)$.

$$\begin{aligned} d &= \frac{1 \cdot 2 - 2 \cdot 3 + 3 \cdot 1 - 5}{\sqrt{1^2 + (-2)^2 + 3^2}} \\ &= \frac{2 - 6 + 3 - 5}{\sqrt{1 + 4 + 9}} \\ &= \frac{-6}{\sqrt{14}} \approx -1.60 \end{aligned}$$

The negative sign means the point lies on the opposite side of the plane from the normal direction.

21.1. Generalization

The same formula generalizes to a hyperplane in $\mathbb{R}^n$ defined by $\vec{a} \cdot \vec{x} = b$:

$$d = \frac{|\vec{a} \cdot \vec{p} - b|}{\| \vec{a} \|}$$

21.2. Connections

- **Planes** — definition and normal vector
- **Projection** — same idea: project the displacement vector onto the unit normal
- **Hyperplane** — n-dimensional generalization

22. Plane–Plane Distance

Two planes in $\mathbb{R}^3$ are either:

- **Identical** — distance 0
- **Parallel** — separated by a constant perpendicular distance
- **Intersecting** — they cross along a line, distance 0 at the intersection

22.1. Distance between parallel planes

Two planes are parallel iff their normal vectors are parallel:

$$P_1 : A_1x + B_1y + C_1z = D_1$$
$$P_2 : A_2x + B_2y + C_2z = D_2$$

are parallel iff $(A_1, B_1, C_1) = k(A_2, B_2, C_2)$ for some $k \neq 0$.

Rewriting both with the **same** normal vector $\vec{n} = (A, B, C)$:

$$P_1 : \vec{n} \cdot \vec{x} = D_1$$
$$P_2 : \vec{n} \cdot \vec{x} = D_2$$

The perpendicular distance between them is:

$$d = \frac{|D_1 - D_2|}{\sqrt{A^2 + B^2 + C^2}} = \frac{|D_1 - D_2|}{\|}\vec{n}\|$$

Example

$$P_1 : x + 2y + 2z = 5$$
$$P_2 : x + 2y + 2z = 14$$

Same normal $(1, 2, 2)$, $\|\vec{n}\| = \sqrt{1 + 4 + 4} = 3$.

$$d = \frac{|5 - 14|}{3} = \frac{9}{3} = 3$$

22.2. Strategy: pick a point on P_1, measure to P_2

Equivalent computation: any point $\vec{x}_0$ on P_1 has point–plane distance to P_2 equal to the inter-plane distance.

22.3. Non-parallel planes

If the normal vectors are not parallel, the planes intersect along a **line** in $\mathbb{R}^3$ — distance 0. Any pair of non-parallel planes intersects (in 3-D).

The intersection line's direction is $\vec{n}_1 \times \vec{n}_2$ (a vector orthogonal to both normals — see Cross Product).

22.4. Connections

- **Planes**
- **Point–Plane Distance**
- **Cross Product** — direction of intersection line

23. Hyperplane

A **hyperplane** in $\mathbb{R}^n$ is an $(n-1)$-dimensional affine subspace — the natural generalization of a line in $\mathbb{R}^2$ or a plane in $\mathbb{R}^3$.

Defined by a single linear equation:

$$\vec{a} \cdot \vec{x} = b \qquad \text{or equivalently} \qquad a_1 x_1 + a_2 x_2 + ... + a_n x_n = b$$

where $\vec{a} \neq \mathbf{0}$ is the **normal vector** and $b \in \mathbb{R}$ is the **offset**.

23.1. Dimension of a hyperplane

In $\mathbb{R}^n$, the equation $\vec{a} \cdot \vec{x} = b$ imposes one linear constraint. The solution set has dimension $n-1$:

- $\mathbb{R}^2$: hyperplane = **line**
- $\mathbb{R}^3$: hyperplane = **plane**
- $\mathbb{R}^n$: hyperplane = $(n\{-\}1)$-dimensional flat

23.2. Through the origin

If $b = 0$, the hyperplane passes through the origin and is a linear subspace (closed under addition and scalar multiplication). Otherwise it's an **affine** subspace — a translated subspace.

23.3. Distance from a point to a hyperplane

Given hyperplane $\vec{a} \cdot \vec{x} = b$ and point $\vec{p} \in \mathbb{R}^n$:

$$\text{dist}(\vec{p}, H) = \frac{|\vec{a} \cdot \vec{p} - b|}{\| \vec{a} \|}$$

(See Norm for $\|\vec{a}\|$.)

23.4. Half-spaces

A hyperplane splits $\mathbb{R}^n$ into two **half-spaces**:

$$H^+ = \{\vec{x} : \vec{a} \cdot \vec{x} \geq b\}$$
$$H^- = \{\vec{x} : \vec{a} \cdot \vec{x} \leq b\}$$

These are the building blocks of **polyhedra** (intersections of finitely many half-spaces).

23.5. Where hyperplanes show up

- **Linear programming**: feasible regions are intersections of half-spaces (polyhedra)
- **Support vector machines**: the decision boundary is a hyperplane $\vec{w} \cdot \vec{x} + b = 0$
- **Planes** in $\mathbb{R}^3$ — the $n = 3$ case
- **Linear constraints** in optimization: each $\vec{a}_i^T \vec{x} \leq b_i$ defines a half-space

24. Cross Product

Only defined in $\mathbb{R}^3$. Returns a vector orthogonal to the two inputs.

$$\vec{a} = \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} \qquad \vec{b} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

$$\vec{c} = \vec{a} \times \vec{b} = \begin{bmatrix} a_2 b_3 - a_3 b_2 \\ a_3 b_1 - a_1 b_3 \\ a_1 b_2 - a_2 b_1 \end{bmatrix}$$

Example

$$\vec{a} = \begin{bmatrix} 1 \\ -7 \\ 1 \end{bmatrix} \quad \vec{b} = \begin{bmatrix} 5 \\ 2 \\ 4 \end{bmatrix}$$

$$\vec{c} = \vec{a} \times \vec{b} = \begin{bmatrix} -7 \cdot 4 - 1 \cdot 2 \\ 1 \cdot 5 - 1 \cdot 4 \\ 1 \cdot 2 - (-7) \cdot 5 \end{bmatrix} = \begin{bmatrix} -30 \\ 1 \\ 37 \end{bmatrix}$$

24.1. Proof: $\vec{c}$ is orthogonal to both $\vec{a}$ and $\vec{b}$

Two vectors are orthogonal when their dot product is zero.

Orthogonal to $\vec{a}$:

$$\vec{c} \cdot \vec{a} = \begin{bmatrix} a_2 b_3 - a_3 b_2 \\ a_3 b_1 - a_1 b_3 \\ a_1 b_2 - a_2 b_1 \end{bmatrix} \cdot \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix}$$

$$= a_1 a_2 b_3 - a_1 a_3 b_2 + a_2 a_3 b_1 - a_2 a_1 b_3 + a_3 a_1 b_2 - a_3 a_2 b_1$$

$$= \cancel{a_1 a_2 b_3} \cancel{- a_1 a_3 b_2} + \cancel{a_2 a_3 b_1} \cancel{- a_2 a_1 b_3} + \cancel{a_3 a_1 b_2} \cancel{- a_3 a_2 b_1} = 0$$

Orthogonal to $\vec{b}$: same argument with the roles of $\vec{a}, \vec{b}$ swapped.

$$\vec{c} \cdot \vec{b} = b_1 a_2 b_3 - b_1 a_3 b_2 + b_2 a_3 b_1 - b_2 a_1 b_3 + b_3 a_1 b_2 - b_3 a_2 b_1$$

$$= \cancel{b_1 a_2 b_3} \cancel{- b_1 a_3 b_2} + \cancel{b_2 a_3 b_1} \cancel{- b_2 a_1 b_3} + \cancel{b_3 a_1 b_2} \cancel{- b_3 a_2 b_1} = 0$$

24.2. See also

- **Cross Product Magnitude** — $\|\vec{a} \times \vec{b}\| = \|\vec{a}\| \, \|\vec{b}\| \sin\theta$
- **Dot vs Cross** — side-by-side comparison

25. Cross Product Magnitude

The norm of the cross product is related to the angle between the two vectors:

$$\|\vec{a} \times \vec{b}\| = \|\vec{a}\| \, \|\vec{b}\| \sin(\theta)$$

where $\theta \in [0, \pi]$ is the angle between $\vec{a}$ and $\vec{b}$.

This is the **signed area of the parallelogram** spanned by the two vectors:

$$\text{Area} = \|\vec{a}\| \, \|\vec{b}\| \sin(\theta)$$

25.1. Geometric meaning

- $\sin(\theta) = 0$ when $\vec{a}, \vec{b}$ are **parallel** (or anti-parallel) — parallelogram degenerates to a line segment, area 0
- $\sin(\theta) = 1$ when $\vec{a}, \vec{b}$ are **perpendicular** — maximum area $\|\vec{a}\| \, \|\vec{b}\|$

25.2. Connection to the determinant

The magnitude is the absolute value of the 3×3 determinant with $\vec{a}, \vec{b}$ as two columns and any orthonormal third column — equivalently the area of the projection of the parallelogram into the plane perpendicular to that third axis.

For a triple of vectors, the **triple product** $\vec{a} \cdot \left(\vec{b} \times \vec{c}\right)$ gives the signed volume of the parallelepiped — the 3×3 determinant with those vectors as columns.

25.3. Connections

- **Cross Product** — the operation itself
- **Dot vs Cross** — $\cos$ side-by-side
- **Determinant** — signed parallelogram / parallelepiped volume
- **Angles Between Vectors** — recovering θ from the formula

26. Dot vs Cross Product

The dot product and cross product are the two fundamental products of vectors. They give complementary geometric information.

26.1. Side-by-side

	Dot product $\vec{a} \cdot \vec{b}$	**Cross product** $\vec{a} \times \vec{b}$
Result type	scalar	vector (in $\mathbb{R}^3$ only)

	Dot product $\vec{a} \cdot \vec{b}$	Cross product $\vec{a} \times \vec{b}$
Formula	$\vec{a} \cdot \vec{b} = \sum a_i b_i$	$\begin{bmatrix} a_2 b_3 - a_3 b_2 \\ a_3 b_1 - a_1 b_3 \\ a_1 b_2 - a_2 b_1 \end{bmatrix}$
Geometric formula	$\lvert\vec{a}\rvert \ \lvert\vec{b}\rvert \cos\theta$	$\lvert\vec{a}\rvert \ \lvert\vec{b}\rvert \sin\theta \ \hat{n}$
Geometric meaning	length of projection of $\vec{a}$ onto $\vec{b}$, scaled by $\lvert\vec{b}\rvert$	vector perpendicular to both, magnitude = parallelogram area
Zero when	vectors are perpendicular	vectors are parallel
Maximum when	vectors are parallel	vectors are perpendicular
Commutative?	yes: $a \cdot b = b \cdot a$	no: $a \times b = -(b \times a)$

26.2. Recovering the angle

Combining the two formulas:

$$\cos\theta = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \ \|\vec{b}\|} \qquad \sin\theta = \frac{\|\vec{a} \times \vec{b}\|}{\|\vec{a}\| \ \|\vec{b}\|}$$

$$\theta = \arctan\left(\|\vec{a} \times \vec{b}\frac{\|}{\vec{a} \cdot \vec{b}}\right)$$

(This `atan2`-style form is numerically more stable than $\arccos$ alone, especially near $\theta = 0$ or π.)

26.3. Pythagorean identity

$$\|\vec{a} \times \vec{b}\|^2 + \left(\vec{a} \cdot \vec{b}\right)^2 = \|\vec{a}\|^2 \, \|\vec{b}\|^2$$

(Direct from $\sin^2 + \cos^2 = 1$.)

26.4. Connections

- **Dot Product**
- **Cross Product** + **Cross Product Magnitude**
- **Angles Between Vectors**

27. Vector Space

Example

Let's say your factory can produce up to 300 units of product 1, 500 units of product 2, and 400 units of product 3. The set of all possible production combinations forms a vector space:

$$x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

$$0 \leq x_1 \leq 300$$
$$0 \leq x_2 \leq 500$$
$$0 \leq x_3 \leq 400$$

28. Linear Combination

Set of vector

$$v_1, v_2, ..., v_n \in \mathbb{R}^n$$

Where

- $v_1, v_2, ..., v_n$: set of vectors
- $\mathbb{R}^n$: set of all ordered tuples of n real numbers

Linear combination of those vector

$$c_1 v_1 + c_2 v_2 + ... + c_n v_n$$

$$c_1, c_2, ..., c_n \in \mathbb{R}$$

Where:

- $c_1, c_2, ..., c_n$: constants or weights

Example

$$\vec{a} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \qquad \vec{b} = \begin{bmatrix} 0 \\ 3 \end{bmatrix}$$

$$0\vec{a} + 0\vec{b}$$

$$3\vec{a} + 2\vec{b}$$

29. Span

Represents the subspace of the vector space that is “covered” by these vectors through their linear combinations

If you have a set of vectors $v_1, v_2, ..., v_n$, the span of these vectors is the set of all vectors that can be written as:

$$\text{Span}(v_1, v_2, ..., v_n) = \{c_1 v_1 + c_2 v_2 + ... + c_n v_n \mid c_1, c_2, ..., c_n \in \mathbb{R}\}$$

Any vector in $\mathbb{R}^2$ can be represented by a linear combination with some combination of these vectors

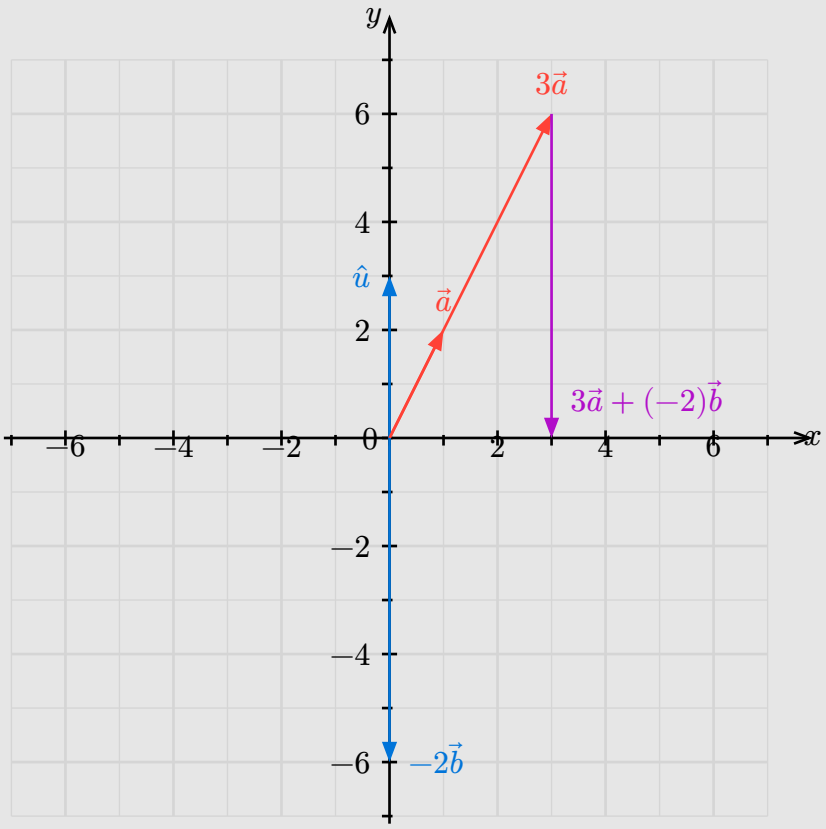
Example

1. Spanning $\mathbb{R}^2$

$$\text{Span}\left(\vec{a}, \vec{b}\right) = \mathbb{R}^2$$

$$\vec{a} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \qquad \vec{b} = \begin{bmatrix} 0 \\ 3 \end{bmatrix}$$

$$3\vec{a} + (-2)\vec{b} = \begin{bmatrix} 3-0 \\ 6-6 \end{bmatrix} = \begin{bmatrix} 3 \\ 0 \end{bmatrix}$$



Any point $\vec{x}$ can be represented as a linear combination of $\vec{a}$ and $\vec{b}$

1. Define the vectors

$$\vec{a} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \qquad \vec{b} = \begin{bmatrix} 0 \\ 3 \end{bmatrix} \qquad \vec{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

2. Express $\vec{x}$ as a linear combinations

$$c_1\vec{a} + c_2\vec{b} = \vec{x}$$

Which expands to

$$c_1 \begin{bmatrix} 1 \\ 2 \end{bmatrix} + c_2 \begin{bmatrix} 0 \\ 3 \end{bmatrix} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

3. Set up the system of equations

$$1c_1 + 0c_2 = x_1 \qquad (1)$$
$$2c_1 + 3c_2 = x_2 \qquad (2)$$

4. Express c_1: From equation (1), we can directly express c_1

$$c_1 = x_1$$

5. Substitute c_1 into equation (2)

$$2x_1 + 3c_2 = x_2$$

Rearranging gives:

$$3c_2 = x_2 - 2x_1$$

6. Solve for c_2 : Dividing both sides by 3 yields

$$c_2 = \frac{x_2 - 2x_1}{3}$$

7. Example with specific values: Let's say we want to find c_1 and c_2 when $\vec{x} = \begin{bmatrix} 2 \\ 2 \end{bmatrix}$

Substitute $x_1 = 2$ and $x_2 = 2$

$$c_1 = x_1 = 2$$
$$c_2 = \frac{2 - 2 \cdot 2}{3} = -\frac{2}{3}$$

8. Final linear combination: Now, substituting c_1 and c_2 back into the linear combination

$$2\vec{a} - \frac{2}{3}\vec{b} = \begin{bmatrix} 2 \\ 2 \end{bmatrix}$$

Verifying

$$2 \begin{bmatrix} 1 \\ 2 \end{bmatrix} + \frac{1}{3} \begin{bmatrix} 0 \\ 3 \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \end{bmatrix}$$

9. This shows that

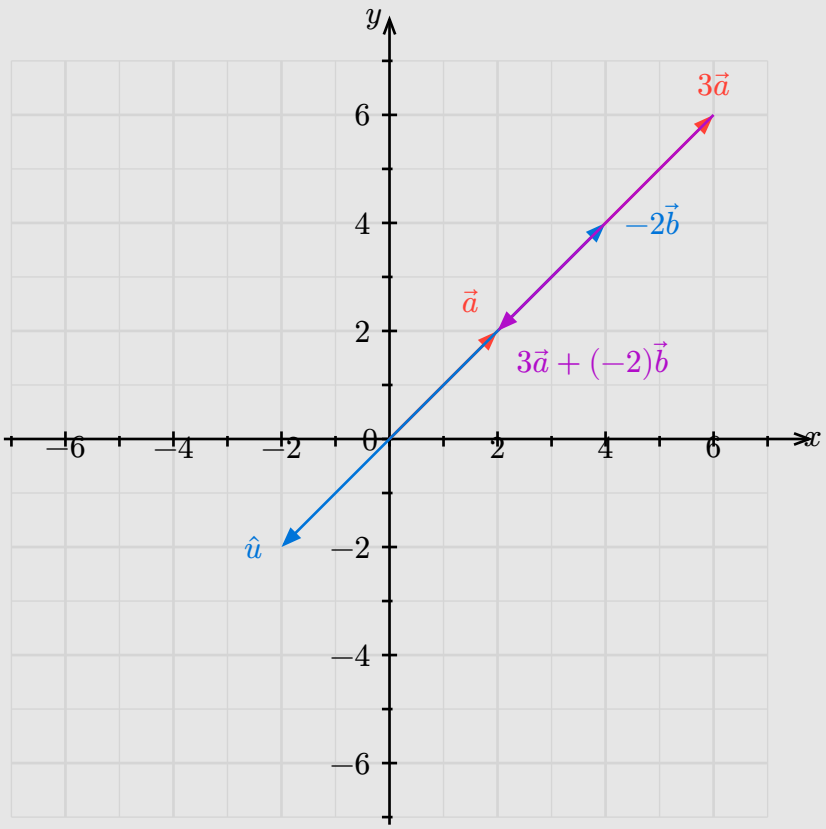
$$2\vec{a} - \frac{2}{3}\vec{b} = \vec{x}$$

2. Spanning Line in $\mathbb{R}^2$

Any linear combination of $\vec{a}$ and $\vec{b}$ will produce vectors that lie along the same line. This is the line through the origin in the direction of $\vec{a}$ (or $\vec{b}$), with all points on the line being scalar multiples of $\vec{a}$

$$\vec{a} = \begin{bmatrix} 2 \\ 2 \end{bmatrix} \qquad \vec{b} = \begin{bmatrix} -2 \\ -2 \end{bmatrix}$$

$$3\vec{a} + (-2)\vec{b} = \begin{bmatrix} 6-4 \\ 6-4 \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \end{bmatrix}$$



30. Linear Independence

1. Definition of Linear Independence

The set of vectors

$$S = \{v_1, v_2, ..., v_n\}$$

is said to be **linearly independent** if the only solution to the equation

$$c_1 v_1 + c_2 v_2 + ... + c_n v_n = \mathbf{0}$$

is $c_1 = c_2 = ... = c_n = 0$. In other words, no vector in the set can be written as a linear combination of the others.

If at least one constant c_i is non-zero, the set is linearly dependent.

Example

Example 1: Testing for Linear Independence

Problem: Is the following set of vectors **linearly dependent**?

$$S = \{\vec{v}_1 \vec{v}_2\}$$

Where:

$$\vec{v}_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix} \quad \text{and} \quad \vec{v}_2 = \begin{bmatrix} 3 \\ 2 \end{bmatrix}$$

For a set of vectors to be **linearly independent**, the only solution to the equation:

$$c_1\vec{v}_1 + c_2\vec{v}_2 = \mathbf{0}$$

must be $c_1 = 0$ and $c_2 = 0$

In this case:

$$c_1 \begin{bmatrix} 2 \\ 1 \end{bmatrix} + c_2 \begin{bmatrix} 3 \\ 2 \end{bmatrix} = \mathbf{0}$$

If not only the zero solution exists (i.e., if c_1 or c_2 can be non-zero), the set is **linearly dependent**.

Step 1. Set up the system of equations:

1. $2c_1 + 3c_2 = 0$
2. $1c_1 + 2c_2 = 0$

2. Eliminate one variable

$$2 \times (1c_1 + 2c_2 = 0) \Rightarrow 2c_1 + 4c_2 = 0$$

Now the system is

$$2c_1 + 3c_2 = 0$$
$$2c_1 + 4c_2 = 0$$

3. Subtract the equations

$$(2c_1 + 3c_2) - (2c_1 + 4c_2) = 0$$

Simplifies to:

$$(2c_1 - 2c_1) + (4c_2 - 3c_2) = 0$$

So:

$$c_2 = 0$$

4. Substitute back to find c_1

Now that we know $c_2 = 0$, substitute this value into one of the original equations. Let's use the second equation:

$$1c_1 + 2c_2 = 0$$

Substitute $c_2 = 0$:

$$1c_1 + 2(0) = 0$$
$$c_1 = 0$$

Conclusion:

Since $c_1 = 0$ $c_2 = 0$, the set of vectors S is **linearly independent**. These vecots span $\mathbb{R}^2$.

Example 2: Testing for Linear Dependence

Problem: Is the following set of vectors **linearly dependent**?

$$S = \{\vec{v}_1 \vec{v}_2\}$$

Where:

$$\vec{v}_1 = \begin{bmatrix} 2 \\ 3 \end{bmatrix} \quad \text{and} \quad \vec{v}_2 = \begin{bmatrix} 4 \\ 6 \end{bmatrix}$$

The span of this set is the collection of all vectors that can be formed by linear combinations of $\vec{v}_1$ and $\vec{v}_2$:

$$c_1 v_1 + c_2 v_2$$

Since $v_2 = 2v_1$, the linear combination becomes:

$$c_1 v_1 + c_2(2v_1) = (c_1 + 2c_2)v_1$$
$$c_1 \begin{bmatrix} 2 \\ 3 \end{bmatrix} + c_2 \begin{bmatrix} 4 \\ 6 \end{bmatrix}$$
$$c_1 \begin{bmatrix} 2 \\ 3 \end{bmatrix} + c_2 2 \begin{bmatrix} 2 \\ 3 \end{bmatrix}$$
$$(c_1 + 2c_2) \begin{bmatrix} 2 \\ 3 \end{bmatrix}$$
$$c_3 \begin{bmatrix} 2 \\ 3 \end{bmatrix}$$

Thus, any linear combination of these vectors is just a scalar multiple of v_1. The span is a single line in $\mathbb{R}^2$, and the vectors are **linearly dependent**.

For any two **colinear** vectors in $\mathbb{R}^2$, their span reduces to a single line.

One vector in the set can be represented by some combination of other vectors in the set

2. General Rule

In $\mathbb{R}^n$, if you have more than n vectors, at least one vector must be linearly dependent on the others, meaning the set cannot be linearly independent.

31. Subspace

31.1. Dimension of a Subspace

Number of elements in a basis for the subspace

32. Basis

Non-redundant set of vectors that span $\mathbb{R}^n$

A basis of a vector space is a set of vectors that satisfies two conditions:

1. **Linear Independence**: no vector in the set can be written as a linear combination of the others — equivalently, the only solution to

$$c_1\vec{v}_1 + c_2\vec{v}_2 + ... + c_n\vec{v}_n = \mathbf{0}$$

   is $c_1 = c_2 = ... = c_n = 0$.

2. **Spanning**: every vector $\vec{v} \in V$ can be expressed as a linear combination of the basis vectors:

$$\vec{v} = c_1\vec{v}_1 + c_2\vec{v}_2 + ... + c_n\vec{v}_n$$

Example

Consider the vector space $\mathbb{R}^2$ (the 2-dimensional Euclidean space). A common basis for $\mathbb{R}^2$ is $\{e_1, e_2\}$, where:

$$e_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \qquad e_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

This set is a basis because:

- Linear Independence: The only solution to $c_1e_1 + c_2e_2 = \mathbf{0}$ is $c_1 = c_2 = 0$.
- Spanning: Any vector $\vec{v} = \begin{bmatrix} x \\ y \end{bmatrix} \in \mathbb{R}^2$ can be written as $\vec{v} = xe_1 + xe_2$

This means $\{e_1, e_2\}$ is a basis for $\mathbb{R}^2$, and the dimension of $\mathbb{R}^2$ is 2.

33. Dimension

The **dimension** of a vector space V is the number of vectors in any basis of V.

$$\dim(V) = (\text{number of vectors in a basis})$$

This number doesn't depend on which basis you pick — every basis of the same vector space has the same size.

Example

- $\dim(\mathbb{R}^n) = n$ (the standard basis $e_1, ..., e_n$ has n vectors)
- $\dim$ of the xy-plane in $\mathbb{R}^3$ is 2
- $\dim(\{\mathbf{0}\}) = 0$ (the trivial space has the empty basis)
- $\dim$ of the space of $m \times n$ matrices is mn

33.1. Properties

For subspaces $U, W \subseteq V$:

- $\dim(U) \leq \dim(V)$
- $\dim(U) = \dim(V)$ implies $U = V$ (no proper subspace of full dimension)
- $\dim(U + W) = \dim(U) + \dim(W) - \dim(U \cap W)$ (**Grassmann formula**)
- For a linear transformation $T : V \to W$:

  $$\dim(V) = \dim(\ker(T)) + \dim(\text{im}(T))$$

  This is the Rank–Nullity Theorem

33.2. Computing dimensions of common subspaces

For an $m \times n$ matrix A:

- $\dim(\text{Col(A)}) = \text{rank}(A)$
- $\dim(\text{Null(A)}) = n - \text{rank}(A)$ (the **nullity**)
- $\dim(\text{Row(A)}) = \text{rank}(A)$ (column rank = row rank)

33.3. See also

- **Basis** — what dimension counts
- **Rank** — dimension of column / row / image
- **Rank–Nullity Theorem**

34. Coordinate Vector

Given a basis $B = \{\vec{v}_1, \vec{v}_2, ..., \vec{v}_n\}$ of a vector space V, every $\vec{x} \in V$ has a **unique** representation as a linear combination of the basis vectors:

$$\vec{x} = c_1\vec{v}_1 + c_2\vec{v}_2 + ... + c_n\vec{v}_n$$

The **coordinate vector** of $\vec{x}$ with respect to B collects those coefficients:

$$[\vec{x}]_B = \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_n \end{bmatrix}$$

The bracket-subscript notation $[\vec{x}]_B$ emphasizes that **the coordinates depend on the basis**.

34.1. The standard basis is implicit

In $\mathbb{R}^n$, when we write $\vec{x} = \begin{bmatrix} 3 \\ -1 \\ 5 \end{bmatrix}$, we tacitly mean coordinates with respect to the **standard basis** $E = \{\vec{e}_1, ..., \vec{e}_n\}$:

$$[\vec{x}]_E = \begin{bmatrix} 3 \\ -1 \\ 5 \end{bmatrix}$$

When you change to a different basis, the same vector gets different coordinates.

Example

Let $B = \left\{ \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ -1 \end{bmatrix} \right\}$ — a basis of $\mathbb{R}^2$.

Express $\vec{x} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$ in basis B. Solve:

$$c_1 \begin{bmatrix} 1 \\ 1 \end{bmatrix} + c_2 \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$$

$$c_1 + c_2 = 3$$
$$c_1 - c_2 = 1$$

$\rightarrow c_1 = 2, c_2 = 1$. So $[\vec{x}]_B = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$, while $[\vec{x}]_E = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$.

34.2. Why coordinates matter

Coordinates make abstract vectors **computable**: once we fix a basis, every vector is a column of numbers and every linear transformation is a matrix.

Different bases reveal different aspects:

- **Standard basis**: most natural for $\mathbb{R}^n$
- **Eigenbasis**: makes the matrix of a linear transformation diagonal (when one exists — see Diagonalization)

- **Orthonormal basis**: distances and angles match the standard formulas
- **Adapted basis**: chosen to simplify a specific subspace structure

34.3. Connection to change of basis

When you switch between two bases B and B', the coordinate vectors are related by an invertible matrix — see Change of Basis.

34.4. See also

- **Basis**
- **Change of Basis**
- **Vector Space**
- **Matrix Representation** — coordinates of $T(\vec{x})$ in terms of coordinates of $\vec{x}$

35. Change of Basis

The same vector or transformation gets different coordinate representations depending on which basis you use. **Change of basis** converts between two such representations.

35.1. Change of basis matrix

Given two bases $B = \left\{\vec{b}_1, ..., \vec{b}_n\right\}$ and $B' = \left\{\vec{b}'_1, ..., \vec{b}'_n\right\}$ of the same space, the **change-of-basis matrix from B' to B** is

$$P_{B' \to B} = \begin{bmatrix} \left[\vec{b}'_1\right]_B & \left[\vec{b}'_2\right]_B & \dots & \left[\vec{b}'_n\right]_B \end{bmatrix}$$

— each column is the B'-basis vector expressed in B-coordinates.

It satisfies:

$$[\vec{x}]_B = P_{B' \to B} \, [\vec{x}]_{B'}$$

The reverse direction uses the inverse:

$$[\vec{x}]_{B'} = P^{-1}_{B' \to B} \, [\vec{x}]_B$$

35.2. Standard basis as anchor

For a basis $B = \left\{\vec{b}_1, ..., \vec{b}_n\right\}$ of $\mathbb{R}^n$, write the basis vectors as columns of a matrix:

$$P = \begin{bmatrix} \vec{b}_1 & \vec{b}_2 & \dots & \vec{b}_n \end{bmatrix}$$

Then $[\vec{x}]_E = P \, [\vec{x}]_B$ (standard $\leftrightarrow$ B conversion via plain matrix–vector product).

Example

$B = \left\{ \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ -1 \end{bmatrix} \right\}$ in $\mathbb{R}^2$, $\vec{x} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$ (standard coords).

$$P = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \qquad P^{-1} = \frac{1}{2}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

$$[\vec{x}]_B = P^{-1}\vec{x} = \frac{1}{2}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}\begin{bmatrix} 3 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

$\rightarrow \vec{x} = 2\vec{b}_1 + 1\vec{b}_2$.

35.3. Change of basis for a linear transformation

If a linear transformation has matrix A in basis B and matrix A' in basis B', with change-of-basis matrix P (mapping B' coords to B coords):

$$A' = P^{-1}AP$$

This is **similarity**. Two matrices related by $A' = P^{-1}AP$ for some invertible P are **similar** — they represent the same linear transformation in different bases.

Similar matrices share:

- **Determinant**: $\det(A') = \det(A)$
- **Trace**: $\mathrm{tr}(A') = \mathrm{tr}(A)$
- **Eigenvalues** (with the same multiplicities)
- **Rank**

35.4. Why it matters

- **Diagonalization** is exactly the search for a basis P in which $A' = P^{-1}AP$ is diagonal — making computation trivial.
- Many problems become easier in an **eigenbasis**, **orthonormal basis**, or specially-chosen adapted basis.

35.5. See also

- **Coordinate Vector**
- **Basis**
- **Diagonalization**
- **Eigenvectors & Eigenvalues**

36. Gram–Schmidt

Given a linearly independent set $\{\vec{v}_1, \vec{v}_2, ..., \vec{v}_k\}$, the **Gram–Schmidt process** produces an orthonormal set $\{\vec{q}_1, \vec{q}_2, ..., \vec{q}_k\}$ that spans the same subspace.

It “straightens out” the basis: removes the parallel components between successive vectors so they become mutually orthogonal, then normalizes each.

36.1. The algorithm

Process the input vectors one at a time. For each new vector, subtract off its projection onto the orthonormal vectors already built, then normalize.

For $j = 1, 2, ..., k$:

$$\vec{u}_j = \vec{v}_j - \sum_{i=1}^{j-1} (\vec{v}_j \cdot \vec{q}_i)\vec{q}_i \qquad \text{(subtract projections off the already-orthonormal vectors)}$$

$$\vec{q}_j = \frac{\vec{u}_j}{\|\vec{u}_j\|} \qquad \text{(normalize)}$$

The first step is just normalization: $\vec{q}_1 = \frac{\vec{v}_1}{\|\vec{v}_1\|}$.

Example

$\vec{v}_1 = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}, \vec{v}_2 = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}$ in $\mathbb{R}^3$.

Step 1: $\vec{q}_1 = \frac{\vec{v}_1}{\|\vec{v}_1\|} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}$.

Step 2: project $\vec{v}_2$ onto $\vec{q}_1$ and subtract:

$$\vec{v}_2 \cdot \vec{q}_1 = \frac{1}{\sqrt{2}}$$

$$\vec{u}_2 = \vec{v}_2 - \frac{1}{\sqrt{2}}\vec{q}_1 = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix} - \frac{1}{2}\begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} -\frac{1}{2} \\ \frac{1}{2} \\ 1 \end{bmatrix}$$

Normalize: $\|\vec{u}_2\| = \sqrt{\frac{1}{4} + \frac{1}{4} + 1} = \sqrt{\frac{3}{2}}$.

$$\vec{q}_2 = \sqrt{\frac{2}{3}} \begin{bmatrix} -\frac{1}{2} \\ \frac{1}{2} \\ 1 \end{bmatrix} = \frac{1}{\sqrt{6}} \begin{bmatrix} -1 \\ 1 \\ 2 \end{bmatrix}$$

Verify: $\vec{q}_1 \cdot \vec{q}_2 = 0$ ✓ and $\|\vec{q}_1\| = \|\vec{q}_2\| = 1$ ✓.

36.2. Why it works

After subtracting the projections, $\vec{u}_j$ is orthogonal to every previously-built $\vec{q}_i$. Inductively, after step j, the set $\{\vec{q}_1, ..., \vec{q}_j\}$ is orthonormal and spans the same subspace as $\{\vec{v}_1, ..., \vec{v}_j\}$.

36.3. Numerical instability

Classical Gram–Schmidt (the version above) is **unstable in floating point** — small rounding errors compound, and the computed $\vec{q}_j$'s drift away from being truly orthogonal.

Better alternatives:

- **Modified Gram–Schmidt** — subtract projections one at a time as you go, instead of all at once; much more stable
- **Householder reflections** — used to compute QR in practice
- **Givens rotations** — for sparse / structured input

For pencil-and-paper or symbolic use, classical Gram–Schmidt is fine. For numerical libraries, Householder is standard.

36.4. Connection to QR

The Gram–Schmidt process is **exactly** the construction behind the QR decomposition: the orthonormal vectors form Q, and the projection coefficients form R.

36.5. See also

- **Orthogonality**
- **Projection**
- **QR Decomposition**
- **Orthogonal Matrix**
- **Basis**

37. Matrices

$m \times n$ matrix $\boldsymbol{A}$

- m: rows
- n: columns

$$\boldsymbol{A} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}$$

37.1. Matrix-Vector Products

$$\boldsymbol{A} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}$$

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

$$\boldsymbol{A}\ \mathbf{x} = \begin{bmatrix} a_{11}x_1 + a_{12}x_2 + ... + a_{1n}x_n \\ a_{21}x_1 + a_{22}x_2 + ... + a_{2n}x_n \\ \vdots \\ a_{m1}x_1 + a_{m2}x_2 + ... + a_{mn}x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

For the dot product to be defined, the number of columns in the matrix $\boldsymbol{A}$ (which is n) must match the number of elements in the vector $\vec{x}$ (also n).

The result of multiplying matrix $\boldsymbol{A}$ and vector $\vec{x}$ will be a column vector with dimensions $m \times 1$, where m is the number of rows in the matrix $\boldsymbol{A}$

$$(m \times n) \cdot (n \times 1) = m \times 1$$

1. As Row vectors

$$\mathbf{a} = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix}$$

$$\mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

$$\mathbf{a}^T = [a_1, a_2, ..., a_n]$$
$$\mathbf{b}^T = [b_1, b_2, ..., b_n]$$

$$\boldsymbol{A} = \begin{bmatrix} [a_1, a_2, ..., a_n] \\ [b_1, b_2, ..., b_n] \end{bmatrix}$$

$$\boldsymbol{A} = \begin{bmatrix} \mathbf{a} \\ \mathbf{b} \end{bmatrix}$$

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

$$\begin{bmatrix} \mathbf{a}^T \\ \mathbf{b}^T \end{bmatrix} \cdot \mathbf{x} = \begin{bmatrix} \mathbf{a} \cdot \mathbf{x} \\ \mathbf{b} \cdot \mathbf{x} \end{bmatrix}$$

2. As Column Vectors

$$\vec{a} = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix}$$

$$\vec{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

$$\boldsymbol{A} = \begin{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix} & \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix} \end{bmatrix}$$

$$\boldsymbol{A} = \begin{bmatrix} \vec{a} & \vec{b} \end{bmatrix}$$

$$\vec{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

$$\boldsymbol{A}\vec{x} = x_1\vec{a} + x_2\vec{b}$$

38. Matrix–Vector Product

A special case of matrix multiplication: a matrix times a single column vector.

For an $m \times n$ matrix A and a vector $\vec{x} \in \mathbb{R}^n$, the product $A\vec{x} \in \mathbb{R}^m$ has entries:

$$(A\vec{x})_i = \sum_{j=1}^{n} a_{ij}\, x_j$$

Example

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \qquad \vec{x} = \begin{bmatrix} 7 \\ 8 \\ 9 \end{bmatrix}$$

$$A\vec{x} = \begin{bmatrix} 1 \cdot 7 + 2 \cdot 8 + 3 \cdot 9 \\ 4 \cdot 7 + 5 \cdot 8 + 6 \cdot 9 \end{bmatrix} = \begin{bmatrix} 50 \\ 122 \end{bmatrix}$$

38.1. Two equivalent views

1. Row view (dot products): each entry is a dot product of a row of A with $\vec{x}$.

2. Column view (linear combination): $A\vec{x}$ is a linear combination of A's columns, weighted by the entries of $\vec{x}$:

$$A\vec{x} = x_1\vec{a}_1 + x_2\vec{a}_2 + ... + x_n\vec{a}_n$$

where $\vec{a}_j$ is the j-th column of A.

This view explains why $A\vec{x} = \vec{b}$ has a solution iff $\vec{b}$ lies in the column space of A.

38.2. Linear transformation view

Every matrix defines a linear transformation $T_A : \mathbb{R}^n \to \mathbb{R}^m$:

$$T_A(\vec{x}) = A\vec{x}$$

Conversely, every linear transformation between finite-dimensional spaces is a matrix–vector product (after fixing bases).

38.3. Linearity

$$A(\vec{x} + \vec{y}) = A\vec{x} + A\vec{y}$$

$$A(c\vec{x}) = c(A\vec{x})$$

A applied to the zero vector returns the zero vector.

39. Matrix Multiplication

For matrices A ($m \times n$) and B ($n \times p$), the product $C = AB$ is the $m \times p$ matrix with entries:

$$c_{ij} = \sum_{k=1}^{n} a_{ik}\, b_{kj}$$

Each entry c_{ij} is the dot product of **row i of A** with **column j of B**.

Compatibility: inner dimensions must match — A's columns must equal B's rows.

Example

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \qquad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

$$AB = \begin{bmatrix} 1 \cdot 5 + 2 \cdot 7 & 1 \cdot 6 + 2 \cdot 8 \\ 3 \cdot 5 + 4 \cdot 7 & 3 \cdot 6 + 4 \cdot 8 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

39.1. Properties

- **Associative**: $(AB)C = A(BC)$
- **Distributive**: $A(B + C) = AB + AC$ and $(A + B)C = AC + BC$
- **Not commutative**: in general $AB \neq BA$
- **Identity**: $IA = AI = A$
- **Transpose**: $(AB)^T = B^T A^T$
- **Inverse** (when defined): $(AB)^{-1} = B^{-1} A^{-1}$
- **Determinant**: $\det(AB) = \det(A) \det(B)$
- **Zero product**: $AB = 0$ does **not** imply $A = 0$ or $B = 0$

39.2. Geometric meaning

If A and B represent linear transformations, then AB represents the **composition** — apply B first, then A (see Composition of Linear Transformations).

39.3. Special cases

- AB where B is a single column → matrix–vector product
- AB where both are vectors → dot product (1×1 result)
- **Outer product**: column vector × row vector = rank-1 matrix

40. Transpose

The **transpose** A^T of a matrix A flips it across its main diagonal — rows become columns:

$$(A^T)_{ij} = A_{ji}$$

If A is $m \times n$, then A^T is $n \times m$.

Example

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \qquad A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

40.1. Properties

- **Involution**: $(A^T)^T = A$
- **Sum**: $(A + B)^T = A^T + B^T$
- **Scalar**: $(c\,A)^T = c\,A^T$
- **Product** (order reversed): $(AB)^T = B^T A^T$
- **Inverse**: $(A^T)^{-1} = (A^{-1})^T$ (when A is invertible)
- **Determinant**: $\det(A^T) = \det(A)$
- **Rank**: $\operatorname{rank}(A^T) = \operatorname{rank}(A)$

40.2. Special cases

- **Symmetric**: $A^T = A$
- **Skew-symmetric**: $A^T = -A$
- **Orthogonal**: $A^T = A^{-1}$, i.e. $A^T A = I$
- **Vector transpose**: a column vector becomes a row vector and vice-versa

40.3. Connection to dot product

For column vectors $\vec{u}, \vec{v} \in \mathbb{R}^n$:

$$\vec{u} \cdot \vec{v} = \vec{u}^T \vec{v}$$

(see Dot Product)

41. Trace

The **trace** of a square matrix A is the sum of its main diagonal entries:

$$\operatorname{tr}(A) = \sum_{i=1}^{n} a_{ii}$$

Example

$$A = \begin{bmatrix} 2 & 7 & 1 \\ 0 & -3 & 5 \\ 4 & 6 & 8 \end{bmatrix} \qquad \operatorname{tr}(A) = 2 + (-3) + 8 = 7$$

41.1. Properties

- **Linearity**: $\operatorname{tr}(A+B) = \operatorname{tr}(A) + \operatorname{tr}(B)$ and $\operatorname{tr}(c\,A) = c\operatorname{tr}(A)$
- **Transpose**: $\operatorname{tr}(A^T) = \operatorname{tr}(A)$
- **Cyclic property**: $\operatorname{tr}(AB) = \operatorname{tr}(BA)$
    - More generally: $\operatorname{tr}(ABC) = \operatorname{tr}(BCA) = \operatorname{tr}(CAB)$
- **Similarity invariance**: $\operatorname{tr}\left(P^{-1}AP\right) = \operatorname{tr}(A)$
    - The trace doesn't change under change of basis
- **Sum of eigenvalues**: $\operatorname{tr}(A) = \sum_{i=1}^{n} \lambda_i$ (counted with multiplicity, including complex)
- **Identity**: $\operatorname{tr}(I_n) = n$

41.2. Trace as inner product

For matrices, the **Frobenius inner product** is defined via the trace:

$$\langle A, B\rangle_F = \operatorname{tr}(A^T B) = \sum_{i,j} A_{ij} B_{ij}$$

It induces the **Frobenius norm** $\|A\|_F = \sqrt{\operatorname{tr}(A^T A)}$.

41.3. Where it shows up

- **Sum of eigenvalues** — quick check via $\operatorname{tr}(A)$
- **Variance / covariance**: $\operatorname{tr}(\mathrm{Cov})$ = total variance across all dimensions
- **Loss functions** in machine learning — $\|y - X\beta\|^2 = \operatorname{tr}\left((y-X\beta)(y-X\beta)^T\right)$
- **Quantum mechanics** — expectation values: $\left\langle \hat{O} \right\rangle = \operatorname{tr}\left(\rho \hat{O}\right)$
- **Characteristic polynomial**: trace appears as a coefficient

42. Identity Matrix

The **identity matrix** I_n is the $n \times n$ diagonal matrix with 1 on every diagonal entry and 0 elsewhere.

$$I_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \qquad I_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \qquad I_n = \begin{bmatrix} 1 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{bmatrix}$$

In index notation: $(I_n)_{ij} = \delta_{ij}$ (the **Kronecker delta**).

42.1. Why it's "the identity"

It's the **multiplicative identity** for matrix multiplication:

$$I_n A = A I_n = A$$

(provided shapes are compatible).

Equivalently, the identity linear transformation $I : \mathbb{R}^n \to \mathbb{R}^n$, $\vec{x} \mapsto \vec{x}$, has matrix I_n.

42.2. Key properties

- **Determinant**: $\det(I_n) = 1$
- **Trace**: $\operatorname{tr}(I_n) = n$
- **Rank**: $\operatorname{rank}(I_n) = n$ (full)
- **Inverse**: $I_n^{-1} = I_n$
- **Transpose**: $I_n^T = I_n$
- **Eigenvalues**: 1 with multiplicity n
- **Powers**: $I_n^k = I_n$ for all k

42.3. Where it shows up

- Defining the matrix inverse: $AA^{-1} = A^{-1}A = I$
- Eigenvalue equation: $\det(A - \lambda I) = 0$ (eigenvalues)
- Block-matrix algebra
- Characterization of orthogonal matrices: $Q^TQ = I$

43. Zero Matrix

The **zero matrix** $0_{m\times n}$ is the $m \times n$ matrix with every entry equal to 0.

$$0_{2\times 3} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

The **zero vector** is the $n \times 1$ zero matrix:

$$\mathbf{0} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

43.1. Why it's "the zero"

It's the **additive identity** for matrix addition:

$$A + 0 = 0 + A = A$$

And the **absorbing element** for matrix multiplication:

$$A \cdot 0 = 0 \cdot A = 0$$

(provided shapes are compatible — note the result 0 may have different shape than the input 0.)

43.2. Key properties

- **Determinant**: $\det(0_{n\times n}) = 0$ — singular
- **Rank**: $\operatorname{rank}(0) = 0$

- **Trace**: $\text{tr}(0_{n\times n}) = 0$
- **Null space**: $\text{null}(0) = \mathbb{R}^n$ — every vector maps to zero

43.3. Where the zero vector matters

- The zero vector is in **every** subspace (subspace axiom)
- **Linear independence** test: $\sum c_i v_i = \mathbf{0}$ has only the trivial solution
- **Kernel** of a linear map: vectors that map to $\mathbf{0}$
- **Homogeneous systems**: $Ax = \mathbf{0}$ — always has at least the trivial solution $x = \mathbf{0}$

44. Diagonal Matrix

A square matrix whose only non-zero entries lie on the **main diagonal**:

$$D = \begin{bmatrix} d_1 & 0 & \dots & 0 \\ 0 & d_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & d_n \end{bmatrix}$$

Often written as $\text{diag}(d_1, d_2, \dots, d_n)$.

Example

$$D = \begin{bmatrix} 3 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 5 \end{bmatrix}$$

44.1. Why diagonal matrices are useful

Operations become trivial — they act componentwise:

- **Multiplication by vector**: $D\,\vec{x} = (d_1 x_1, d_2 x_2, \dots, d_n x_n)$
- **Powers**: $D^k = \text{diag}(d_1^k, \dots, d_n^k)$
- **Inverse** (when each $d_i \neq 0$): $D^{-1} = \text{diag}\left(\frac{1}{d_1}, \dots, \frac{1}{d_n}\right)$
- **Determinant**: $\det(D) = d_1 d_2 \dots d_n$
- **Trace**: $\text{tr}(D) = d_1 + d_2 + \dots + d_n$
- **Eigenvalues**: $d_1, d_2, \dots, d_n$ themselves
- **Eigenvectors**: standard basis vectors $e_1, \dots, e_n$

44.2. Special cases

- **Identity matrix**: all $d_i = 1$
- **Zero matrix** (square): all $d_i = 0$
- **Scalar matrix**: all $d_i = c$ for the same scalar c (i.e. $c\,I$)

44.3. Where they show up

- **Diagonalization**: turning a general matrix into a diagonal one via change of basis
- **SVD**: $A = U\Sigma V^T$ where Σ is diagonal
- **Eigen-decomposition**: $A = PDP^{-1}$ where D is diagonal of eigenvalues

45. Triangular Matrix

A square matrix in which all entries on one side of the main diagonal are zero.

45.1. Upper triangular (U)

All entries **below** the main diagonal are zero: $U_{ij} = 0$ for $i > j$.

$$U = \begin{bmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{bmatrix}$$

45.2. Lower triangular (L)

All entries **above** the main diagonal are zero: $L_{ij} = 0$ for $i < j$.

$$L = \begin{bmatrix} l_{11} & 0 & 0 \\ l_{21} & l_{22} & 0 \\ l_{31} & l_{32} & l_{33} \end{bmatrix}$$

45.3. Properties (apply to both upper and lower)

- **Determinant**: product of diagonal entries

$$\det(L) = \prod_{i=1}^{n} l_{ii}$$

- **Trace**: sum of diagonal entries
- **Eigenvalues**: the diagonal entries themselves
- **Sum / product / inverse** of triangular matrices is triangular (same orientation)
- **Transpose** flips orientation: L^T is upper triangular

45.4. Why they matter

Triangular systems are easy to solve by **back substitution** (upper) or **forward substitution** (lower):

$$\begin{bmatrix} 2 & 3 & 1 \\ 0 & 1 & -1 \\ 0 & 0 & 4 \end{bmatrix} \vec{x} = \begin{bmatrix} 7 \\ 2 \\ 8 \end{bmatrix} \quad \Rightarrow \quad x_3 = 2, x_2 = 4, x_1 = -2$$

Solve from the bottom row up — each row has one new unknown.

45.5. Where they show up

- **Row Echelon Form**: upper triangular (after Gaussian elimination)
- **LU Decomposition**: $A = LU$ — lower × upper triangular
- **Cholesky Decomposition**: $A = LL^T$ for symmetric positive-definite A
- **QR Decomposition**: $A = QR$ where R is upper triangular

46. Symmetric Matrix

A square matrix that equals its own transpose:

$$A^T = A \qquad \text{i.e. } A_{ij} = A_{ji} \text{ for all } i, j$$

Symmetric matrices are mirror-images across the main diagonal.

Example

$$A = \begin{bmatrix} 2 & -1 & 4 \\ -1 & 5 & 0 \\ 4 & 0 & 3 \end{bmatrix} \qquad A^T = A \ \checkmark$$

46.1. Skew-symmetric matrices

The mirror flips sign: $A^T = -A$, equivalently $A_{ij} = -A_{ji}$.

Diagonal entries must be zero (since $A_{ii} = -A_{ii}$).

$$S = \begin{bmatrix} 0 & 3 & -2 \\ -3 & 0 & 5 \\ 2 & -5 & 0 \end{bmatrix}$$

46.2. Why symmetric matrices are special

- **Real eigenvalues**: every eigenvalue of a real symmetric matrix is real
- **Orthogonal eigenvectors**: eigenvectors corresponding to **distinct** eigenvalues are orthogonal
- **Spectral theorem**: every real symmetric matrix is **orthogonally diagonalizable**:

  $$A = QDQ^T$$

  where Q is orthogonal and D is diagonal of eigenvalues
- Defines a **quadratic form** $\vec{x}^T A \vec{x}$

46.3. Decomposing any matrix

Any square matrix M uniquely splits into a symmetric and skew-symmetric part:

$$M = \underbrace{\frac{1}{2}(M + M^T)}_{\text{symmetric}} + \underbrace{\frac{1}{2}(M - M^T)}_{\text{skew-symmetric}}$$

46.4. Where they show up

- **Covariance matrices** — always symmetric (and positive semi-definite)
- **Quadratic forms** — only the symmetric part matters
- **Hessian matrices** — symmetric for C^2 functions (Schwarz's theorem)
- **Gram matrices** $X^T X$ — always symmetric and positive semi-definite
- **Adjacency matrices** of undirected graphs — symmetric

47. Orthogonal Matrix

A square matrix Q whose transpose is its inverse:

$$Q^T Q = Q Q^T = I$$

Equivalently, the columns (and rows) of Q form an **orthonormal set** — each is a unit vector and they are mutually orthogonal:

$$q_i \cdot q_j = \begin{cases} 1 \text{ if } i = j \\ 0 \text{ if } i \neq j \end{cases}$$

47.1. Geometric meaning

An orthogonal matrix represents a **rigid** linear transformation of $\mathbb{R}^n$: it preserves lengths and angles.

For any vectors $\vec{u}, \vec{v} \in \mathbb{R}^n$:

- **Lengths**: $\|Q\vec{v}\| = \|\vec{v}\|$ (norm preserved)
- **Angles**: $(Q\vec{u}) \cdot (Q\vec{v}) = \vec{u} \cdot \vec{v}$ (dot product preserved)

47.2. Properties

- **Determinant**: $\det(Q) = \pm 1$
  - $+1 \rightarrow$ orientation preserving (a **rotation**)
  - $-1 \rightarrow$ orientation reversing (includes **reflections**)
- **Eigenvalues**: all have absolute value 1 (they sit on the unit circle in $\mathbb{C}$)
- **Inverse**: $Q^{-1} = Q^T$ — extremely cheap to invert
- **Closed under product**: $Q_1 Q_2$ orthogonal (Q_1, Q_2 orthogonal)
- **Orthogonal group** $O(n) = \{Q \in \mathbb{R}^{n \times n} : Q^T Q = I\}$
- **Special orthogonal group** $SO(n) = \{Q \in O(n) : \det(Q) = 1\}$ — pure rotations

Example

2×2 rotation matrix by angle θ — see Rotation Matrix:

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

$R(\theta)^T R(\theta) = I$, $\det = \cos^2\theta + \sin^2\theta = 1$.

47.3. Where they show up

- **QR decomposition**: $A = QR$ — orthogonal × upper triangular
- **SVD**: $A = U\Sigma V^T$ — both U and V are orthogonal
- **Spectral theorem**: real symmetric matrices diagonalize via orthogonal Q
- **Gram–Schmidt**: produces orthonormal columns from any basis
- **Coordinate frame changes** in robotics, computer graphics, physics

48. Linear Equations

48.1. Solving Systems of Linear Equations

Linear Equation

$$y = a_1x_1 + a_1x_2 + ... + a_nx_n$$

1. Consistency

Whether a system of linear equations has at least one solution

Example

Consistent System

$$x + y = 3$$
$$x - y = 1$$

This system has a unique solution

$$(x, y) = (2, 1)$$

Inconsistent System

$$x + y = 3$$
$$x + y = 5$$

This system is inconsistent (equations contradict each other, no solution can satisfy both)

2. Independence

Whether the equations in the system provide unique and non-redundant information about the variables

Example

Independent Equations

$$x + y = 3$$
$$x - y = 1$$

Neither equation can be derived from the other (they provide unique information and intersect at a single point)

Dependent Equations

$$x + y = 3$$
$$2x + 2y = 6$$

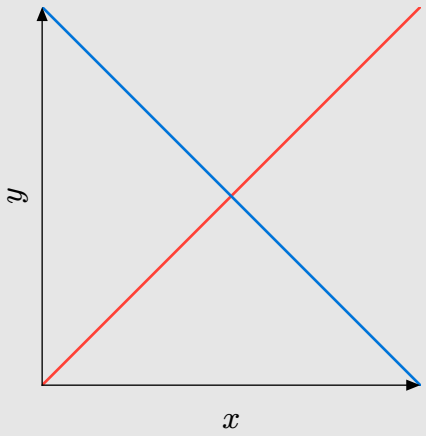
Second equation is just a multiple of the first equation (they describe the same line)

3. Recognizing Systems with No Solution or Infinite Solutions

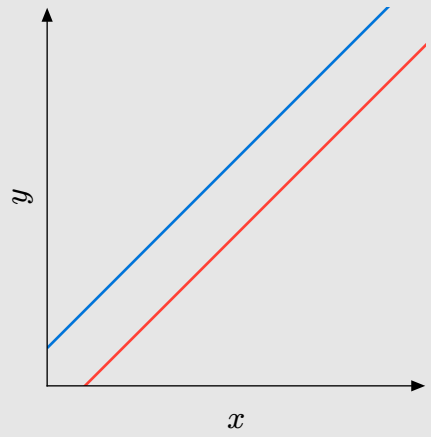
Example

$$3x + 2y = 6 \qquad \text{(Equation 1)}$$
$$6x + 4y = 12 \qquad \text{(Equation 2)}$$

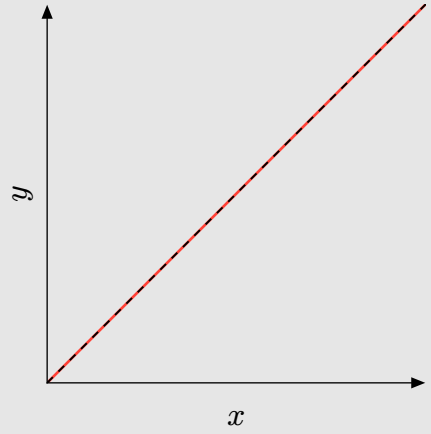
Unique Solution (Consistent and Independent):



No Solution (Inconsistent):



Infinitely Many Solutions (Consistent and Dependent):



2. Matrix Representation

System of Equations

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + ... + a_{1n}x_n &= b_1 \\ a_{21}x_1 + a_{22}x_2 + ... + a_{2n}x_n &= b_2 \\ &\vdots \\ a_{m1}x_1 + a_{m2}x_2 + ... + a_{mn}x_n &= b_m \end{aligned}$$

Matrix Representation

Coefficient vector (A)

$$A = \begin{bmatrix} a_{11} & a_{12} & ... & a_{1n} \\ a_{21} & a_{22} & ... & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{2m1} & a_{m2} & ... & a_{mn} \end{bmatrix}$$

Variable vector (x)

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

Constant vector (b)

$$b = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix}$$

Matrix equation

$$Ax = b$$

linear_eq.py

```python
from scipy.linalg import solve

X = np.array([
  [1, 1, 1],
  [2, -1, 3],
  [3, 4, -1]
])

Y = np.array([6, 14, 1])

intersection_point = solve(X, Y)
```

49. Augmented Matrix

When solving the linear system $A\vec{x} = \vec{b}$, the **augmented matrix** $\left[A \mid \vec{b}\right]$ stacks the coefficient matrix and the right-hand side into one $m \times (n+1)$ matrix:

$$\left[A \mid \vec{b}\right] = \left[\begin{array}{cccc|c} a_{11} & a_{12} & \dots & a_{1n} & b_1 \\ a_{21} & a_{22} & \dots & a_{2n} & b_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} & b_m \end{array}\right]$$

The vertical bar separates the coefficients from the constants — purely visual; row operations treat it as one matrix.

Example

System:

$$2x_1 + 3x_2 = 7$$
$$x_1 - x_2 = 1$$

Augmented form:

$$\left[A \mid \vec{b}\right] = \left[\begin{array}{cc|c} 2 & 3 & 7 \\ 1 & -1 & 1 \end{array}\right]$$

49.1. Why use it

Row-reducing $\left[A \mid \vec{b}\right]$ to REF (via Gaussian elimination) or RREF (via Gauss–Jordan) reads off the solution directly — no need to substitute back manually if you go all the way to RREF.

49.2. Reading solutions

After row reduction:

- **Unique solution**: $\operatorname{rank}(A) = \operatorname{rank}\left(\left[A \mid \vec{b}\right]\right) = n$
- **Infinite solutions**: $\operatorname{rank}(A) = \operatorname{rank}\left(\left[A \mid \vec{b}\right]\right) < n$ (free variables)
- **No solution**: $\operatorname{rank}(A) < \operatorname{rank}\left(\left[A \mid \vec{b}\right]\right)$ — a row $[0, 0, ..., 0 \mid c]$ with $c \neq 0$ appears

This is the **Rouché–Capelli theorem** expressed in terms of augmented-matrix ranks.

49.3. Connections

- Gaussian Elimination — the algorithm that operates on $[A \mid b]$
- RREF — the canonical row-reduced form
- Homogeneous System — when $\vec{b} = \mathbf{0}$
- Linear System Solutions

50. Row Echelon Form

A matrix is in **Row Echelon Form (REF)** if it satisfies all of:

1. **Pivot** — each non-zero row has a leading non-zero entry called a **pivot**
2. **Zeros below pivots** — each pivot has zeros below it in its column
3. **Rightward staircase** — each pivot is strictly to the right of the pivot in the row above
4. **Zero rows at bottom** — any all-zero rows appear at the bottom

$$\left[\begin{array}{cccc|c} 1 & a_{12} & a_{13} & a_{14} & b_1 \\ 0 & 1 & a_{23} & a_{24} & b_2 \\ 0 & 0 & 1 & a_{34} & b_3 \\ 0 & 0 & 0 & 0 & 0 \end{array}\right]$$

50.1. Elementary row operations

The three operations that don't change the solution set of a system, used to drive a matrix toward REF:

1. **Row swap**: exchange two rows
2. **Row scale**: multiply a row by a non-zero scalar
3. **Row replacement**: add a multiple of one row to another

These are applied via Gaussian elimination.

Example

System:

$$\begin{aligned} 2x + y + z &= 8 \\ -3x - y + 2z &= -11 \\ -2x + y + 2z &= -3 \end{aligned}$$

Augmented matrix:

$$\left[\begin{array}{ccc|c} 2 & 1 & 1 & 8 \\ -3 & -1 & 2 & -11 \\ -2 & 1 & 2 & -3 \end{array}\right]$$

Step 1: $R_1 \to \frac{1}{2}R_1$:

$$\left[\begin{array}{ccc|c} 1 & 0.5 & 0.5 & 4 \\ -3 & -1 & 2 & -11 \\ -2 & 1 & 2 & -3 \end{array}\right]$$

Step 2: $R_2 \to R_2 + 3R_1$, $R_3 \to R_3 + 2R_1$:

$$\left[\begin{array}{ccc|c} 1 & 0.5 & 0.5 & 4 \\ 0 & 0.5 & 3.5 & 1 \\ 0 & 2 & 3 & 5 \end{array}\right]$$

Step 3: $R_2 \to 2R_2$:

$$\left[\begin{array}{ccc|c} 1 & 0.5 & 0.5 & 4 \\ 0 & 1 & 7 & 2 \\ 0 & 2 & 3 & 5 \end{array}\right]$$

Step 4: $R_3 \to R_3 - 2R_2$:

$$\left[\begin{array}{ccc|c} 1 & 0.5 & 0.5 & 4 \\ 0 & 1 & 7 & 2 \\ 0 & 0 & -11 & 1 \end{array}\right]$$

Step 5: $R_3 \to -\frac{1}{11}R_3$ (now in REF):

$$\left[\begin{array}{ccc|c} 1 & 0.5 & 0.5 & 4 \\ 0 & 1 & 7 & 2 \\ 0 & 0 & 1 & -\frac{1}{11} \end{array}\right]$$

Step 6: back-substitute.

$$z = -\frac{1}{11}, \quad y = 2 - 7z = \frac{29}{11}, \quad x = 4 - 0.5y - 0.5z = \frac{31}{11}$$

50.2. REF vs RREF

REF is not unique. The stronger RREF (Reduced Row Echelon Form) — pivots equal 1 and zeros above as well as below — **is** unique.

50.3. See also

- **RREF** — canonical reduced form
- **Linear System Solutions** — unique / none / infinite
- **Linear System Special Cases** — when zero rows appear
- **Gaussian Elimination** — the algorithm

51. RREF

A matrix is in **Reduced Row Echelon Form** (RREF) if it satisfies all of the following:

1. **Echelon form** — see REF:
    - Any all-zero rows are at the bottom
    - Leading entries (**pivots**) move strictly right as you go down
2. **Each pivot is 1** (the leading entry of every non-zero row)
3. **Each pivot is the only non-zero entry in its column** (zeros above and below)

Example

RREF:

$$\begin{bmatrix} 1 & 0 & 0 & 4 \\ 0 & 1 & 0 & -2 \\ 0 & 0 & 1 & 7 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Not RREF (the pivot in column 2 is 3, not 1, and column 1 has a non-zero above the pivot):

$$\begin{bmatrix} 1 & 5 & 2 \\ 0 & 3 & 4 \\ 0 & 0 & 0 \end{bmatrix}$$

51.1. RREF vs REF

| Form | Pivots | Above pivots | |—|—|—| | REF | non-zero | unrestricted | | RREF | exactly 1 | all 0 |

REF comes from Gaussian elimination RREF is what Gauss–Jordan elimination produces.

51.2. Uniqueness

Given a matrix A, its REF is **not** unique (depends on row operations chosen), but its **RREF is unique**. So RREF is a canonical form — useful for:

- Reading off the rank: number of non-zero rows
- Reading off the solution to $Ax = b$ directly from $[A \mid b]$ in RREF
- Identifying linearly independent columns: the **pivot columns**
- Computing the null space basis from free variables

51.3. Reading the solution

After reducing augmented matrix $\left[A \mid \vec{b}\right]$ to RREF:

Example

$$\left[\begin{array}{ccccc|c} 1 & 0 & 3 & 0 & 5 & 4 \\ 0 & 1 & -2 & 0 & 1 & 7 \\ 0 & 0 & 0 & 1 & 6 & 2 \end{array}\right]$$

Pivot columns: 1, 2, 4 → pivot variables x_1, x_2, x_4. **Free columns**: 3, 5 → free variables x_3, x_5.

Solution (parametric):

$$
\begin{aligned}
x_1 &= 4 - 3x_3 - 5x_5 \\
x_2 &= 7 + 2x_3 - x_5 \\
x_4 &= 2 - 6x_5 \\
x_3, x_5 &\ \text{free}
\end{aligned}
$$

52. Gaussian Elimination

Convert a matrix into its row echelon form (REF) or reduced row echelon form (RREF)

$$
\begin{aligned}
a_{11}x_1 + a_{12}x_2 + ... + a_{1n}x_n &= b_1 \\
a_{21}x_1 + a_{22}x_2 + ... + a_{2n}x_n &= b_2 \\
&\vdots \\
a_{m1}x_1 + a_{m2}x_2 + ... + a_{mn}x_n &= b_m
\end{aligned}
$$

1. Create an augmented matrix

$$
A = \left[\begin{array}{cccc|c}
a_{11} & a_{12} & ... & a_{1n} & b_1 \\
a_{21} & a_{22} & ... & a_{2n} & b_2 \\
\vdots & \vdots & \ddots & \vdots & b_3 \\
a_{2m1} & a_{m2} & ... & a_{mn} & b_m
\end{array}\right]
$$

2. Forward Elimination

Eliminate the element in the i-th of the k-th column ($k > i$)

$$
R_k \leftarrow R_k - \frac{a_{ki}}{a_{ii}} R_i
$$

Where

- a_{ii}: Pivot element
- R_k: k-th row
- R_i: i-th row

3. Back Substitution
4. Reduced Row Echelon Form (RREF)

Example

$$
\begin{aligned}
2x_1 + 3x_2 &= 5 \\
4x_1 + 5x_2 &= 5
\end{aligned}
$$

1. Create an augmented matrix

$$
\left[\begin{array}{cc|c}
2 & 3 & 5 \\
4 & 5 & 6
\end{array}\right]
$$

2. Forward Elimination

$R_k \leftarrow R_k - \frac{a_{ki}}{a_{ii}} R_i$

$R_k \leftarrow R_k - \frac{a_{21}}{a_{11}} R_i$

$R_2 \leftarrow R_2 - \frac{4}{2} R_1$

$R_2 \leftarrow R_2 - 2 \times R_1$

$$\left[\begin{array}{cc|c} 2 & 3 & 5 \\ 4-2\times 2 & 5-2\times 3 & 6-2\times 5 \end{array}\right]$$

Simplifies to:

$$\left[\begin{array}{cc|c} 2 & 3 & 5 \\ 0 & -1 & -4 \end{array}\right]$$

System is now:

$$\begin{aligned} 2x_1 + 3x_2 &= 5 \\ -1x_2 &= -4 \end{aligned}$$

3. Back Substitution

$$\begin{aligned} -1x_2 &= -4 \\ x_2 &= 4 \end{aligned}$$

Substitute:

$$\begin{aligned} 2x_1 + 3(4) &= 5 \\ 2x_1 + 12 &= 5 \\ x_1 &= -3.5 \end{aligned}$$

Solution:

$$\begin{aligned} x_1 &= -3.5 \\ x_2 &= 4 \end{aligned}$$

53. Gauss–Jordan Elimination

An extension of Gaussian elimination that takes the matrix all the way to **Reduced Row Echelon Form** (RREF) — not just to upper-triangular REF.

53.1. Two phases

1. **Forward elimination** (same as Gaussian elimination): pivot to get zeros **below** each pivot, achieve REF (upper-triangular form).
2. **Back-substitution turned into elimination**: pivot upward to also clear **above** each pivot, and scale each pivot row so the pivot equals 1.

Result: each pivot is 1, with all-zero columns above and below it — the canonical RREF.

53.2. Why bother going past REF?

After full Gauss–Jordan, the solution to $A\vec{x} = \vec{b}$ can be **read directly** off the right-hand side of the augmented matrix $\left[A \mid \vec{b}\right]$ — no back substitution needed.

Also useful for:

- Computing the inverse: row-reduce $[A \mid I]$ to $[I \mid A^{-1}]$
- Finding a basis for the null space
- Reading off rank (number of pivots) and linearly independent columns (the pivot columns)

Example

Apply Gauss–Jordan to:

$$\left[\begin{array}{cc|c} 1 & 2 & 5 \\ 3 & 7 & 16 \end{array}\right]$$

Forward: $R_2 \to R_2 - 3R_1$ gives

$$\left[\begin{array}{cc|c} 1 & 2 & 5 \\ 0 & 1 & 1 \end{array}\right]$$

Backward: $R_1 \to R_1 - 2R_2$ gives

$$\left[\begin{array}{cc|c} 1 & 0 & 3 \\ 0 & 1 & 1 \end{array}\right]$$

Read the solution off: $x_1 = 3,\ x_2 = 1$.

53.3. Inverse via Gauss–Jordan

To compute A^{-1} for an $n \times n$ invertible matrix, form $[A \mid I_n]$ and row-reduce until the left half is the identity:

$$[A \mid I_n] \longrightarrow [I_n \mid A^{-1}]$$

Example

$A = \begin{bmatrix} 2 & 1 \\ 5 & 3 \end{bmatrix}$. Augment with I_2:

$$\left[\begin{array}{cc|cc} 2 & 1 & 1 & 0 \\ 5 & 3 & 0 & 1 \end{array}\right]$$

Row-reduce to $[I \mid A^{-1}]$:

$$\left[\begin{array}{cc|cc} 1 & 0 & 3 & -1 \\ 0 & 1 & -5 & 2 \end{array}\right]$$

$$A^{-1} = \begin{bmatrix} 3 & -1 \\ -5 & 2 \end{bmatrix}$$

53.4. Cost

Gauss–Jordan is $O(n^3)$ — same asymptotic complexity as Gaussian elimination + back substitution, but with a slightly larger constant. For solving a single system, plain Gaussian elimination with back-substitution is marginally faster; for the inverse or RREF directly, Gauss–Jordan is the natural choice.

53.5. See also

- **Gaussian Elimination** — the forward phase only
- **REF** / **RREF**
- **Matrix Inverse**

54. Substitution

Example

$$\begin{aligned} x + y &= 10 \qquad (\text{Equation } 1) \\ 2x - y &= 5 \qquad (\text{Equation } 2) \end{aligned}$$

1. Solve Equation 1 for y

$$y = 10 - x$$

2. Substitute into Equation 2

$$2x - (10 - x) = 5$$

3. Solve for x:

$$\begin{aligned} 2x - 10 + x &= 5 \\ 3x - 10 &= 5 \\ 3x &= 15 \\ x &= 5 \end{aligned}$$

4. Find y using value of x

$$y = 10 - x$$
$$y = 10 - 5$$
$$y = 5$$

55. Addition / Subtraction

55.1. Addition or Subtraction Method

Example

$$3x + 2y = 12 \qquad \text{(Equation 1)}$$
$$2x - 2y = 4 \qquad \text{(Equation 2)}$$

1. Add the equations

$$(3x + 2y) + (2x - 2y) = 12 + 4$$
$$5x = 16$$
$$x = \frac{16}{5}$$
$$x = 3.2$$

2. Substitute

$$3(3.2) + 2y = 12$$
$$9.6 + 2y = 12$$
$$2y = 12 - 9.6y = \frac{2.4}{2}$$
$$y = 1.2$$

56. Linear System Solutions

A linear system $A\vec{x} = \vec{b}$ has **exactly** one of three outcomes:

1. **Unique solution** — exactly one $\vec{x}$ satisfies the system
2. **No solution** — the system is **inconsistent**
3. **Infinitely many solutions** — the system is **underdetermined**

The REF (or RREF) of the augmented matrix $\left[A \mid \vec{b}\right]$ tells you which case you're in.

56.1. 1. Unique solution

Every column of A has a pivot, no free variables, and no inconsistency:

$$\left[\begin{array}{cccc|c} 1 & a_{12} & a_{13} & a_{14} & b_1 \\ 0 & 1 & a_{23} & a_{24} & b_2 \\ 0 & 0 & 1 & a_{34} & b_3 \\ 0 & 0 & 0 & 1 & b_4 \end{array}\right]$$

Equivalently: $\operatorname{rank}(A) = \operatorname{rank}\Big(\big[A \mid \vec{b}\big]\Big) = n$ (the number of unknowns).

56.2. 2. No solution (inconsistent)

A row of the form $[0, 0, \ldots, 0 \mid c]$ with $c \neq 0$ — translates to "$0 = c$" which is impossible.

$$\left[\begin{array}{cc|c} 1 & 1 & 3 \\ 0 & 0 & 1 \end{array}\right]$$

The second row says $0x + 0y = 1$. No $\vec{x}$ satisfies this.

Equivalently: $\operatorname{rank}(A) < \operatorname{rank}\Big(\big[A \mid \vec{b}\big]\Big)$.

56.3. 3. Infinitely many solutions (free variables)

Some columns of A have no pivot — those correspond to **free variables** you can set arbitrarily; the **pivot variables** are determined by them.

$$\left[\begin{array}{cccc|c} 1 & a_{12} & 0 & a_{14} & b_1 \\ 0 & 0 & 1 & a_{24} & b_2 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{array}\right]$$

Columns 2 and 4 have no pivot → x_2, x_4 are free.

Equivalently: $\operatorname{rank}(A) = \operatorname{rank}\Big(\big[A \mid \vec{b}\big]\Big) < n$. The solution set is an **affine subspace** of dimension $n - \operatorname{rank}(A)$ — see Rank–Nullity.

56.4. Rouché–Capelli summary

Case	Rank condition	Solution set
Unique	$\operatorname{rank}(A) = \operatorname{rank}([A \mid b]) = n$	single point
None	$\operatorname{rank}(A) < \operatorname{rank}([A \mid b])$	empty
Infinite	$\operatorname{rank}(A) = \operatorname{rank}([A \mid b]) < n$	affine subspace of dim $n - \operatorname{rank}(A)$

56.5. See also

- **Row Echelon Form** — the algorithm
- **Linear System Special Cases** — when zero rows appear
- **Rank** and **Rank–Nullity**
- **Homogeneous System** — special case $\vec{b} = \mathbf{0}$

57. Linear System Special Cases

When you reduce an augmented matrix $\left[A \mid \vec{b}\right]$ to REF, rows of all zeros (or zero coefficients with non-zero RHS) signal one of three structural situations.

57.1. 1. Dependent equations

One equation is a linear combination of others — they carry the same information.

$$2x + 4y = 8$$
$$x + 2y = 4$$

The second equation is half the first — same line. After row reduction:

$$\left[\begin{array}{cc|c} 1 & 2 & 4 \\ 0 & 0 & 0 \end{array}\right]$$

The zero row means one equation was redundant. The system has **infinitely many solutions** (a whole line in $\mathbb{R}^2$).

57.2. 2. Underdetermined systems

More variables than independent equations — fewer constraints than unknowns.

$$x + y + z = 2$$
$$2x + 3y + z = 5$$

Two equations in three unknowns. After row reduction:

$$\left[\begin{array}{ccc|c} 1 & 1 & 1 & 2 \\ 0 & 1 & -1 & 1 \end{array}\right]$$

Both equations are independent, but z is a free variable. **Infinitely many solutions** (a line in $\mathbb{R}^3$).

57.3. 3. Inconsistent systems

A zero coefficient row with a **non-zero** RHS — translates to $0 = c$ for $c \neq 0$, which is impossible.

$$x + y = 3$$
$$2x + 2y = 7$$

(Same left-hand side scaled by 2, but right-hand side doesn't scale to match.)

$$\left[\begin{array}{cc|c} 1 & 1 & 3 \\ 0 & 0 & 1 \end{array}\right]$$

The second row reads $0 = 1$. **No solution**.

57.4. See also

- **Row Echelon Form**
- **Linear System Solutions** — Rouché–Capelli summary
- **Rank** — diagnoses dependence

58. Homogeneous System

A linear system is **homogeneous** if its right-hand side is the zero vector:

$$A\vec{x} = \mathbf{0}$$

Every homogeneous system is **consistent** — it always has at least the **trivial solution** $\vec{x} = \mathbf{0}$.

58.1. When are there non-trivial solutions?

The non-trivial solutions form the null space of A (equivalently, the kernel of the linear map $T(x) = Ax$):

$$\mathrm{Sol}(Ax = \mathbf{0}) = \mathrm{null}(A) = \ker(T)$$

By the Rank–Nullity Theorem, for an $m \times n$ matrix A:

$$\dim(\mathrm{null}(A)) = n - \mathrm{rank}(A)$$

So:

$\mathrm{rank}(A)$	**Solution set**
$= n$	only the trivial solution $\vec{x} = \mathbf{0}$
$< n$	a subspace of dimension $n - \mathrm{rank}(A) > 0$ — **infinitely many** solutions

Notably: an $n \times n$ homogeneous system has a non-trivial solution **iff** A is singular (i.e., $\det(A) = 0$).

Example

Solve $A\vec{x} = \mathbf{0}$ for $A = \begin{bmatrix} 1 & 2 & -1 \\ 2 & 4 & -2 \end{bmatrix}$.

Row-reduce: $R_2 \to R_2 - 2R_1 = (0, 0, 0)$. So $\mathrm{rank} = 1$, nullity $= 3 - 1 = 2$.

From row 1: $x_1 + 2x_2 - x_3 = 0$, so $x_1 = -2x_2 + x_3$. With x_2, x_3 free:

$$\vec{x} = x_2 \begin{bmatrix} -2 \\ 1 \\ 0 \end{bmatrix} + x_3 \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$$

Two-parameter family of solutions — a plane through the origin in $\mathbb{R}^3$.

58.2. Connection to general systems

If $A\vec{x} = \vec{b}$ has solution $\vec{x}_p$ (a particular solution), the **full** solution set is:

$$\vec{x}_p + \text{null}(A) = \{\vec{x}_p + \vec{x}_h : \vec{x}_h \in \text{null}(A)\}$$

— the particular solution plus any solution to the **associated homogeneous system**. This is why homogeneous systems are the "structural skeleton" of solution sets.

58.3. See also

- **Null Space** / **Kernel**
- **Linear System Solutions**
- **Rank–Nullity Theorem**
- **Zero Vector**

59. Linear Transformation

A function $T : \mathbb{R}^n \to \mathbb{R}^m$ between vector spaces is a **linear transformation** if it preserves vector addition and scalar multiplication.

59.1. The two axioms

For any vectors $\vec{a}, \vec{b} \in \mathbb{R}^n$ and scalar $c \in \mathbb{R}$:

1. Additivity

$$T\left(\vec{a} + \vec{b}\right) = T(\vec{a}) + T\left(\vec{b}\right)$$

2. Homogeneity (scalar multiplication)

$$T(c\,\vec{a}) = c\,T(\vec{a})$$

The two combine into a single condition: $T\left(c\,\vec{a} + d\,\vec{b}\right) = c\,T(\vec{a}) + d\,T\left(\vec{b}\right)$ for all scalars and vectors.

Example

$T : \mathbb{R}^2 \to \mathbb{R}^2$ defined by $T\left(\begin{bmatrix} x \\ y \end{bmatrix}\right) = \begin{bmatrix} 2x \\ 3y \end{bmatrix}$ is linear.

Check additivity: $\vec{a} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \vec{b} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$:

$$T(\vec{a}) + T\left(\vec{b}\right) = \begin{bmatrix} 2 \\ 6 \end{bmatrix} + \begin{bmatrix} 6 \\ 3 \end{bmatrix} = \begin{bmatrix} 8 \\ 9 \end{bmatrix}$$

$$T\left(\vec{a} + \vec{b}\right) = T\left(\begin{bmatrix} 4 \\ 3 \end{bmatrix}\right) = \begin{bmatrix} 8 \\ 9 \end{bmatrix} \quad \checkmark$$

Check homogeneity: $c = 3$, $\vec{a} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$:

$$c\,T(\vec{a}) = 3 \begin{bmatrix} 2 \\ 6 \end{bmatrix} = \begin{bmatrix} 6 \\ 18 \end{bmatrix}$$

$$T(c\,\vec{a}) = T\left(\begin{bmatrix} 3 \\ 6 \end{bmatrix} \right) = \begin{bmatrix} 6 \\ 18 \end{bmatrix} \quad \checkmark$$

59.2. Consequences

Linearity forces T to be very rigid. From the two axioms:

- $T(\mathbf{0}) = \mathbf{0}$ (apply homogeneity with $c = 0$)
- $T(-\vec{a}) = -T(\vec{a})$
- T maps lines through the origin to lines through the origin (or to the origin itself)
- T maps the origin to the origin
- T maps parallelograms to parallelograms (or degenerate shapes)

59.3. Equivalent definition: matrix–vector product

Every linear transformation can be written as $T(\vec{x}) = A\vec{x}$ for some matrix A — see Matrix Representation. This means **linear transformation $\mathbb{R}^n \to \mathbb{R}^m \iff m \times n$ matrix**.

59.4. Non-examples

- $T(x) = x + 1$: not linear (translates the origin off itself)
- $T\left(\begin{bmatrix} x \\ y \end{bmatrix} \right) = \begin{bmatrix} x^2 \\ y \end{bmatrix}$: not linear (squaring isn't homogeneous)
- $T\left(\begin{bmatrix} x \\ y \end{bmatrix} \right) = \begin{bmatrix} xy \\ 0 \end{bmatrix}$: not linear (product of components)

59.5. See also

- **Matrix–Vector Product** — every matrix is a linear transformation
- **Matrix Representation** — every linear transformation is a matrix
- **Image** / **Kernel** — output range / null space
- **Composition** — composing linear transformations
- **Rotation**, **Reflection**, **Scaling**, **Shear**

60. Matrix Representation

Every linear transformation $T : \mathbb{R}^n \to \mathbb{R}^m$ can be uniquely represented by an $m \times n$ matrix A such that:

$$T(\vec{x}) = A\vec{x}$$

Conversely, every $m \times n$ matrix gives a linear transformation. So **linear transformations $\mathbb{R}^n \to \mathbb{R}^m$ are the same as $m \times n$ matrices**.

60.1. How to find the matrix

Apply T to each standard basis vector $\vec{e}_1, ..., \vec{e}_n$. The results become the columns of A:

$$A = \begin{bmatrix} T(\vec{e}_1) & T(\vec{e}_2) & ... & T(\vec{e}_n) \end{bmatrix}$$

60.2. Why this works (sketch)

Any input $\vec{x} \in \mathbb{R}^n$ is a linear combination of standard basis vectors:

$$\vec{x} = x_1\vec{e}_1 + x_2\vec{e}_2 + ... + x_n\vec{e}_n$$

Applying T and using linearity:

$$\begin{aligned} T(\vec{x}) &= T(x_1\vec{e}_1 + ... + x_n\vec{e}_n) \\ &= x_1T(\vec{e}_1) + ... + x_nT(\vec{e}_n) \\ &= \begin{bmatrix} T(\vec{e}_1) & ... & T(\vec{e}_n) \end{bmatrix}\vec{x} \\ &= A\vec{x} \end{aligned}$$

The columns of A are **exactly** the images of the standard basis under T.

Example

$T : \mathbb{R}^2 \to \mathbb{R}^2$ rotates by 90° counterclockwise.

$$T(\vec{e}_1) = T\left(\begin{bmatrix} 1 \\ 0 \end{bmatrix}\right) = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad T(\vec{e}_2) = T\left(\begin{bmatrix} 0 \\ 1 \end{bmatrix}\right) = \begin{bmatrix} -1 \\ 0 \end{bmatrix}.$$

$$A = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$

Verify: $A\begin{bmatrix} 2 \\ 3 \end{bmatrix} = \begin{bmatrix} -3 \\ 2 \end{bmatrix}$ — rotating $(2, 3)$ by 90°.

60.3. Identity transformation

The identity $I : \mathbb{R}^n \to \mathbb{R}^n$, $I(\vec{x}) = \vec{x}$, has matrix I_n (the identity matrix), since $I(\vec{e}_i) = \vec{e}_i$ for each standard basis vector.

60.4. See also

- **Linear Transformation** — definition
- **Matrix–Vector Product** — the operation that makes a matrix into a transformation
- **Change of Basis** — what happens to the matrix when you switch bases
- **Rotation Matrix**, **Reflection Matrix**, **Scaling Matrix**, **Shear Matrix** — common examples

61. Domain

For a function $f : X \to Y$, the **domain** is the set X of inputs.

$$X = \text{domain}(f) = \text{set of all valid inputs}$$

For a linear transformation $T : \mathbb{R}^n \to \mathbb{R}^m$, the domain is $\mathbb{R}^n$.

61.1. Domain vs codomain vs image

- **Domain** — where the inputs live (X)
- **Codomain** — where outputs are **allowed** to live (Y)
- **Image / Range** — where outputs **actually** land (a subset of the codomain)

Example

$f(x) = x^2$ from $\mathbb{R}$ to $\mathbb{R}$:

- Domain: $\mathbb{R}$
- Codomain: $\mathbb{R}$
- Image: $[0, \infty)$ — only non-negative outputs

The image is a strict subset of the codomain (negatives like -3 are in the codomain but not the image).

61.2. For matrices

If $T(\vec{x}) = A\vec{x}$ with A an $m \times n$ matrix, then:

- Domain = $\mathbb{R}^n$ (number of **columns** of A)
- Codomain = $\mathbb{R}^m$ (number of **rows** of A)
- Image = column space of $A \subseteq \mathbb{R}^m$

61.3. See also

- **Codomain**
- **Image** — what lands in the codomain
- **Linear Transformation**

62. Codomain

For a function $f : X \to Y$, the **codomain** is the set Y where outputs are declared to live — though outputs don't necessarily fill it.

$$Y = \text{codomain}(f)$$

The actual subset of Y that gets hit by f is the image (or **range**) — always a subset of the codomain, sometimes strict.

Example

$f : \mathbb{R} \to \mathbb{R}$, $f(x) = x^2$.

- Codomain: $\mathbb{R}$
- Image: $[0, \infty)$

Negative reals like -3 are in the codomain (they're "allowed" outputs by the type signature) but not in the image (no real x produces them).

62.1. Codomain vs image

The choice of codomain is partly conventional. Two functions can be the "same" with different codomains:

$$f_1 : \mathbb{R} \to \mathbb{R}, \;\; f_1(x) = x^2$$
$$f_2 : \mathbb{R} \to [0, \infty), \;\; f_2(x) = x^2$$

f_2 is **surjective** (image = codomain); f_1 is not.

62.2. For matrices

If $T(\vec{x}) = A\vec{x}$ with A an $m \times n$ matrix:

- **Codomain**: $\mathbb{R}^m$ (set by the number of rows)
- **Image**: column space of A, generally a subspace of $\mathbb{R}^m$
- **Surjective** iff the columns of A span $\mathbb{R}^m$ — i.e., $\text{rank}(A) = m$

62.3. See also

- **Domain**
- **Image** / **Column Space**
- **Surjective / Injective / Bijective**

63. Surjective, Injective, Bijective

Three properties a function $T : V \to W$ might (or might not) have, with clean characterizations for linear transformations.

63.1. Injective (one-to-one)

Different inputs produce different outputs — no two distinct inputs collide:

$$\vec{x}_1 \neq \vec{x}_2 \;\Rightarrow\; T(\vec{x}_1) \neq T(\vec{x}_2)$$

For a linear T, equivalently:

$$\ker(T) = \{\mathbf{0}\}$$

(See Kernel — only the zero vector maps to zero.)

63.2. Surjective (onto)

Every element of the codomain is hit by some input:

$$\forall \vec{y} \in W,\ \exists \vec{x} \in V \text{ with } T(\vec{x}) = \vec{y}$$

For a linear $T : \mathbb{R}^n \to \mathbb{R}^m$, equivalently:

$$\text{im}(T) = \mathbb{R}^m, \quad \text{i.e. } \text{rank}(A) = m$$

(See Image / Rank.)

63.3. Bijective (one-to-one and onto)

Both injective and surjective. Each output has **exactly one** preimage. Bijective linear transformations are **invertible**.

For a linear $T : \mathbb{R}^n \to \mathbb{R}^m$, bijective requires $m = n$ (square matrix) **and** A is invertible.

63.4. Summary table for linear $T : \mathbb{R}^n \to \mathbb{R}^m$ given by matrix A

Property	Equivalent (rank)	Equivalent (kernel/image)	Possible only if
Injective	$\text{rank}(A) = n$	$\ker(T) = \{\mathbf{0}\}$	$m \geq n$ (full column rank)
Surjective	$\text{rank}(A) = m$	$\text{im}(T) = \mathbb{R}^m$	$n \geq m$ (full row rank)
Bijective	$\text{rank}(A) = m = n$	both above	$m = n$ and A invertible

Example

$$A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix} - T : \mathbb{R}^2 \to \mathbb{R}^3.$$

- $\text{rank} = 2 = n \to$ injective
- $\text{rank} = 2 < 3 = m \to$ not surjective
- Not bijective ($m \neq n$)

63.5. Connection to inverse

A linear T has an inverse iff it's bijective iff A is square and invertible.

When invertible, T^{-1} corresponds to A^{-1} (see Matrix Inverse).

63.6. See also

- **Kernel** / **Image**
- **Rank** / **Rank–Nullity Theorem**
- **Matrix Inverse**

64. Image

The **image** (or **range**) of a linear transformation $T : \mathbb{R}^n \to \mathbb{R}^m$ is the set of all possible outputs:

$$\text{im}(T) = T(\mathbb{R}^n) = \{T(\vec{x}) \mid \vec{x} \in \mathbb{R}^n\}$$

This is a generalization of the image of a subset applied to the entire domain.

64.1. Matrix representation

If T is represented by an $m \times n$ matrix A with columns $\vec{a}_1, ..., \vec{a}_n$, then for any input $\vec{x} = (x_1, ..., x_n)$:

$$T(\vec{x}) = A\vec{x} = x_1\vec{a}_1 + x_2\vec{a}_2 + ... + x_n\vec{a}_n$$

— a linear combination of A's columns.

64.2. Image = Column space

The image of T equals the column space of A:

$$\text{im}(T) = \text{Col}(A) = \text{span}(\vec{a}_1, \vec{a}_2, ..., \vec{a}_n)$$

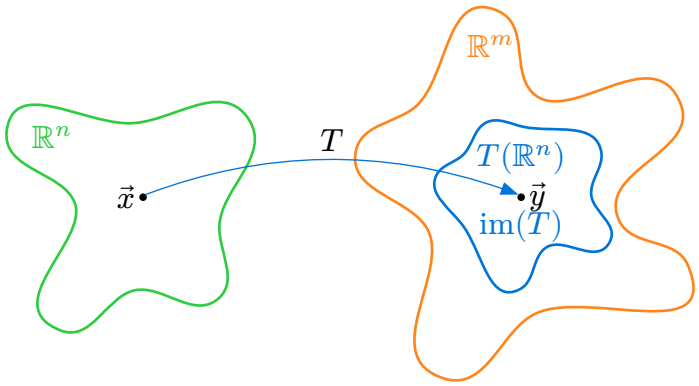
Example

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}$$

$$T\left(\begin{bmatrix} 1 \\ 0 \end{bmatrix}\right) = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, T\left(\begin{bmatrix} 0 \\ 1 \end{bmatrix}\right) = \begin{bmatrix} 1 \\ 3 \end{bmatrix}.$$

$$\text{im}(T) = \text{span}\left(\begin{bmatrix} 2 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 3 \end{bmatrix}\right) = \mathbb{R}^2$$

(The two columns are linearly independent, so they span $\mathbb{R}^2$.)



64.3. Properties

- $\mathrm{im}(T)$ is a subspace of the codomain $\mathbb{R}^m$
- $\dim(\mathrm{im}(T)) = \mathrm{rank}(A) =$ rank of the matrix
- **Rank–Nullity**: $\dim(\mathrm{im}(T)) + \dim(\ker(T)) = n$
- T is **surjective** iff $\mathrm{im}(T) = \mathbb{R}^m$ iff A has full row rank

64.4. Three names for the same thing

- **Image** of T — emphasis on the linear-map perspective
- **Range** of T — same, common in analysis
- **Column space** of A — same, matrix-algebra perspective

65. Image of Subset

For a linear transformation $T : \mathbb{R}^n \to \mathbb{R}^m$ and a subset $V \subseteq \mathbb{R}^n$, the **image of V under T** is:

$$T(V) = \{T(\vec{x}) \mid \vec{x} \in V\}$$

— the set of all outputs you get by applying T to every input in V.

Example

$$\vec{x}_0 = \begin{bmatrix} -2 \\ -2 \end{bmatrix} \qquad \vec{x}_1 = \begin{bmatrix} -2 \\ 2 \end{bmatrix} \qquad \vec{x}_2 = \begin{bmatrix} 2 \\ -2 \end{bmatrix}$$

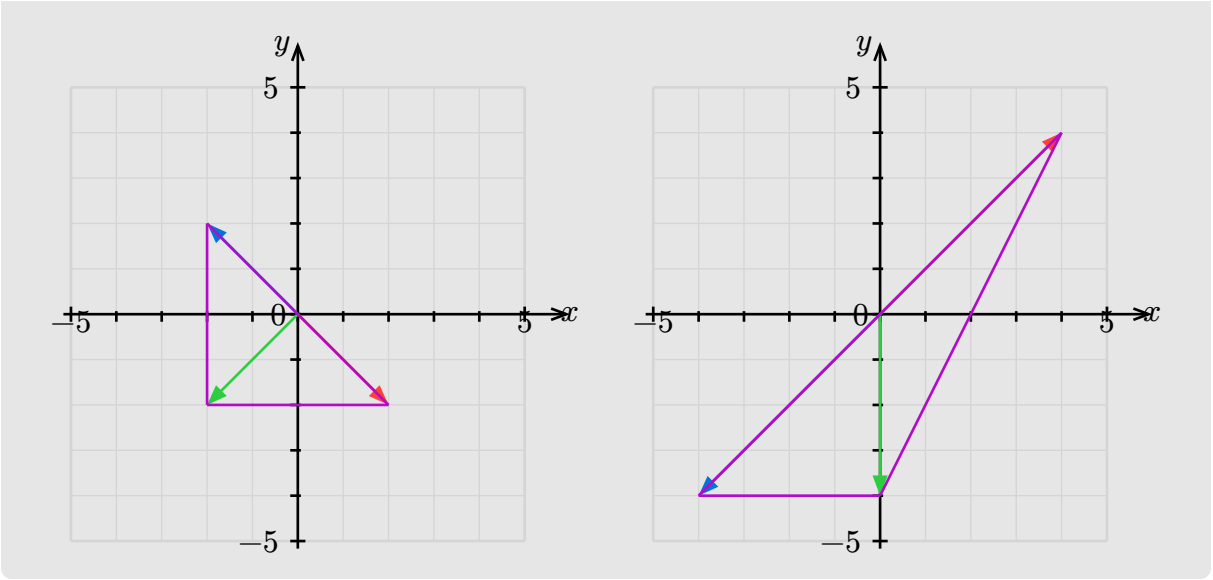
$$L_0 = \{\vec{x}_0 + t(\vec{x}_1 - \vec{x}_0) \mid 0 \le t \le 1\}$$
$$L_1 = \{\vec{x}_1 + t(\vec{x}_2 - \vec{x}_1) \mid 0 \le t \le 1\}$$
$$L_2 = \{\vec{x}_2 + t(\vec{x}_0 - \vec{x}_2) \mid 0 \le t \le 1\}$$

Triangle: $S = \{L_0, L_1, L_2\}$.

Apply transformation $T(\vec{x}) = A\vec{x}$ with $A = \begin{bmatrix} 1 & -1 \\ 2 & 0 \end{bmatrix}$:

$$T(\vec{x}_0) = \begin{bmatrix} 0 \\ -4 \end{bmatrix} \qquad T(\vec{x}_1) = \begin{bmatrix} -4 \\ -4 \end{bmatrix} \qquad T(\vec{x}_2) = \begin{bmatrix} 4 \\ 4 \end{bmatrix}$$

By linearity, line segments map to line segments: $T(L_0)$ is the image of L_0, and $T(S)$ is the image of the whole triangle.



65.1. Connections

- **Image of a transformation** — the case $V = \mathbb{R}^n$ (the whole domain)
- **Preimage** — the inverse direction (which inputs produce a given output set)

66. Preimage

For a transformation $T : \mathbb{R}^n \to \mathbb{R}^m$ and a subset $S \subseteq \mathbb{R}^m$, the **preimage of S under T** is the set of inputs that map into S:

$$T^{-1}(S) = \{\vec{x} \in \mathbb{R}^n \mid T(\vec{x}) \in S\}$$

For a single point $\vec{y} \in \mathbb{R}^m$, the preimage $T^{-1}(\vec{y})$ may be empty, a single point, or an entire affine subspace — depending on T.

Example

Let $T : \mathbb{R}^2 \to \mathbb{R}^2$ be defined by $A = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$, so $T(\vec{x}) = \begin{bmatrix} 2x_1 \\ 3x_2 \end{bmatrix}$.

Preimage of $\begin{bmatrix} 4 \\ 6 \end{bmatrix}$:

Solve $A\vec{x} = \begin{bmatrix} 4 \\ 6 \end{bmatrix}$:

$$2x_1 = 4 \;\Rightarrow\; x_1 = 2$$
$$3x_2 = 6 \;\Rightarrow\; x_2 = 2$$

$$T^{-1}\left(\begin{bmatrix} 4 \\ 6 \end{bmatrix}\right) = \left\{\begin{bmatrix} 2 \\ 2 \end{bmatrix}\right\}$$

Preimage of $\begin{bmatrix} 2 \\ 3 \end{bmatrix}$: similarly $T^{-1}\left(\begin{bmatrix} 2 \\ 3 \end{bmatrix}\right) = \left\{\begin{bmatrix} 1 \\ 1 \end{bmatrix}\right\}$.

Preimage of the set $S = \left\{\begin{bmatrix} 4 \\ 6 \end{bmatrix}, \begin{bmatrix} 2 \\ 3 \end{bmatrix}\right\}$:

$$T^{-1}(S) = \left\{\begin{bmatrix} 2 \\ 2 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \end{bmatrix}\right\}$$

66.1. Properties

- $T^{-1}(\varnothing) = \varnothing$
- $T^{-1}(\mathbb{R}^m) = \mathbb{R}^n$
- $T^{-1}(S_1 \cup S_2) = T^{-1}(S_1) \cup T^{-1}(S_2)$
- $T^{-1}(S_1 \cap S_2) = T^{-1}(S_1) \cap T^{-1}(S_2)$
- For a linear T: $T^{-1}(\mathbf{0})$ is exactly the kernel of T
- For linear T and any $\vec{y} \in \operatorname{im}(T)$: $T^{-1}(\vec{y}) = \vec{x}_p + \ker(T)$ for any particular solution $\vec{x}_p$ — an **affine subspace** parallel to the kernel

66.2. Connections

- **Image** — the forward direction
- **Image of a subset** — applying T to a set rather than T^{-1}
- **Kernel** — preimage of the zero vector under a linear map

67. Kernel

For a linear transformation $T : \mathbb{R}^n \to \mathbb{R}^m$, the **kernel** (or **null space** of T) is the set of vectors that map to the zero vector:

$$\ker(T) = \{\vec{x} \in \mathbb{R}^n \mid T(\vec{x}) = \mathbf{0}\}$$

The kernel is the preimage of $\mathbf{0}$.

67.1. Why the kernel is a subspace

For any $\vec{x}, \vec{y} \in \ker(T)$ and scalar c:

- $T(\vec{x} + \vec{y}) = T(\vec{x}) + T(\vec{y}) = \mathbf{0} + \mathbf{0} = \mathbf{0}$ → closed under addition
- $T(c\,\vec{x}) = c\,T(\vec{x}) = c\,\mathbf{0} = \mathbf{0}$ → closed under scalar multiplication
- $\mathbf{0} \in \ker(T)$ since $T(\mathbf{0}) = \mathbf{0}$

Hence $\ker(T)$ is a subspace of $\mathbb{R}^n$.

67.2. Matrix form: kernel ↔ null space

If T is represented by a matrix A (so $T(\vec{x}) = A\vec{x}$), the kernel of T equals the null space of A:

$$\ker(T) = \text{null}(A) = \{\vec{x} \mid A\vec{x} = \mathbf{0}\}$$

Example

Let $T : \mathbb{R}^2 \to \mathbb{R}^2$ with $A = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$. Solve $A\vec{x} = \mathbf{0}$:

$$2x_1 = 0 \Rightarrow x_1 = 0$$
$$3x_2 = 0 \Rightarrow x_2 = 0$$

Only the trivial solution: $\ker(T) = \{\mathbf{0}\}$. The transformation is **injective**.

67.3. Injectivity

A linear transformation T is **injective** (one-to-one) if and only if $\ker(T) = \{\mathbf{0}\}$.

Proof sketch: if $T(\vec{x}_1) = T(\vec{x}_2)$, then $T(\vec{x}_1 - \vec{x}_2) = \mathbf{0}$, so $\vec{x}_1 - \vec{x}_2 \in \ker(T)$. The kernel being trivial forces $\vec{x}_1 = \vec{x}_2$.

67.4. Connections

- **Null Space** — kernel of the matrix representation
- **Rank–Nullity**: $\dim(\ker(T)) + \dim(\text{im}(T)) = n$
- **Image** — the "output side" counterpart
- **Preimage** — generalization to non-zero target sets
- **Zero Vector** — what kernel elements map to

68. Column Space

The **columns space** (or range) of matrix A is span of its columns vectors

If the matrix A has columns $\vec{a}_1, \vec{a}_2, ..., \vec{a}_n$, then the column space of A is defined as:

$$\text{Col}(A) = \{\vec{y} \in \mathbb{R}^m \mid \vec{y} = A\vec{x} \text{ for some } \vec{x} \in \mathbb{R}^n\}$$

or equivalently,

$$\text{Col}(A) = \text{span}(\{\vec{a}_1, \vec{a}_2, ..., \vec{a}_n\})$$

Example

Consider the simple example of a 2×2 matrix:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 6 \end{bmatrix}$$

The matrix has two columns:

$$\vec{a}_1 = \begin{bmatrix} 1 \\ 3 \end{bmatrix} \quad \text{and} \quad \vec{a}_2 = \begin{bmatrix} 2 \\ 6 \end{bmatrix}$$

The column space, denoted $\text{Col}(A)$, is the span of these two vectors:

$$\text{Col}(A) = \text{span}\left(\left\{ \begin{bmatrix} 1 \\ 3 \end{bmatrix}, \begin{bmatrix} 2 \\ 6 \end{bmatrix} \right\} \right)$$

Finding the Column Space

We observe that the two columns $\vec{a}_1$ and $\vec{a}_2$ are **linearly dependent**:

$$\vec{a}_2 = k\vec{a}_1$$

This means that $\vec{a}_2$ is a scalar multiple of $\vec{a}_1$, the the two columns are **linearly dependent**. As a result, the column space is spanned by just one vector, $\vec{a}_1$, because any linear combination of $\vec{a}_1$ and $\vec{a}_2$ can be reduced to a multiple of $\vec{a}_1$.

Therefore, the column space of A is:

$$\text{Col}(A) = \text{span}\left(\left\{ \begin{bmatrix} 1 \\ 3 \end{bmatrix} \right\} \right)$$

which represents all vectors of the form:

$$c \begin{bmatrix} 1 \\ 3 \end{bmatrix} = \begin{bmatrix} c \\ 3c \end{bmatrix} \quad \text{for any scalar } c$$

In other words, the column space is a line in $\mathbb{R}^2$ through the origin in the direction of

Rank of A

The rank of A, which is the **dimension of its column space**, is 1 because there is only one linearly independent column

This means the column space is the span of the columns of A, or all vectors that can be formed by taking linear combinations of the columns of A.

69. Null Space

The **null space** (or kernel) of a matrix $\boldsymbol{A}$ is the set of all vectors $\vec{x}$ that satisfy:

$$\boldsymbol{A}\vec{x} = \mathbf{0}$$

For an $m \times n$ matrix A:

- $\vec{x} \in \mathbb{R}^n$
- $\mathbf{0} \in \mathbb{R}^m$

The null space is a subspace of $\mathbb{R}^n$. By the Rank–Nullity Theorem:

$$\dim(\text{Null}(A)) = n - \text{rank}(A)$$

69.1. Computing the null space

Row-reduce A to RREF. The non-pivot columns correspond to **free variables**; the pivot columns are expressed in terms of those free variables. A basis for the null space is built from one vector per free variable.

Example

$$\boldsymbol{A} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 \\ 4 & 3 & 2 & 1 \end{bmatrix}$$

Find $\vec{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}$ with $\boldsymbol{A}\vec{x} = \boldsymbol{0}$:

$$\begin{cases} 1x_1 + 1x_2 + 1x_3 + 1x_4 = 0 \\ 1x_1 + 2x_2 + 3x_3 + 4x_4 = 0 \\ 4x_1 + 3x_2 + 2x_3 + 1x_4 = 0 \end{cases}$$

Augmented matrix:

$$\left[\begin{array}{cccc|c} 1 & 1 & 1 & 1 & 0 \\ 1 & 2 & 3 & 4 & 0 \\ 4 & 3 & 2 & 1 & 0 \end{array}\right]$$

Row-reduce to find the null space. (See Gaussian Elimination.)

69.2. Nullity

The **nullity** of A is the dimension of its null space:

$$\text{nullity}(A) = \dim(\text{Null}(A))$$

After row-reducing A to RREF, nullity equals the number of **non-pivot columns** (= number of free variables).

69.3. Kernel vs null space

For a linear transformation $T(\vec{x}) = A\vec{x}$:

$$\ker(T) = \text{Null}(A)$$

Same set, different names: **kernel** is the linear-map perspective, **null space** is the matrix perspective. See Kernel.

69.4. Left null space

The null space of A^T is called the **left null space** of A:

$$\text{Null}(A^T) = \{\vec{y} : \vec{y}^T A = \mathbf{0}\}$$

It's orthogonal to the column space of A. Together with $\text{Null}(A)$ and $\text{Row}(A)$, these are the **four fundamental subspaces**.

69.5. See also

- **Kernel** — same concept, transformation perspective
- **Rank–Nullity Theorem**
- **Rank**
- **Homogeneous System**
- **Column Space** — the "output side" counterpart

70. Rank

Dimension of the column **space**

$$\text{rank}(A) = \dim(C(A))$$

70.1. Matrix Representation of Systems of Equations

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + ... + a_{1m}x_m &= b_1 \\ a_{21}x_1 + a_{22}x_2 + ... + a_{2m}x_m &= b_2 \\ &\vdots \\ a_{n1}x_1 + a_{n2}x_2 + ... + a_{nm}x_m &= b_n \end{aligned}$$

Coefficient Matrix (A):

$$A = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1m} \\ a_{21} & a_{22} & \dots & a_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nm} \end{bmatrix}$$

Variable Vector (x):

$$\mathrm{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix}$$

Constant Vector (b):

$$\mathrm{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

$$\mathrm{Ax} = b$$

Example

The system of equations:

$$\begin{aligned} 2x_1 + 3x_2 + 5x_3 &= 100 \\ 4x_1 + 2x_2 + 1x_3 &= 80 \\ 1x_1 + 5x_2 + 2x_3 &= 60 \end{aligned}$$

Can be represented as a matrix equation:

$$\begin{bmatrix} 2 & 3 & 5 \\ 4 & 2 & 1 \\ 1 & 5 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 100 \\ 80 \\ 60 \end{bmatrix}$$

71. Rank–Nullity Theorem

For a linear transformation $T : V \to W$ between finite-dimensional vector spaces:

$$\underbrace{\dim(V)}_{n} = \underbrace{\dim(\ker(T))}_{\text{nullity}} + \underbrace{\dim(\mathrm{im}(T))}_{\text{rank}}$$

In matrix form, for an $m \times n$ matrix A:

$$n = \dim(\mathrm{null(A)}) + \mathrm{rank(A)}$$

(See Dimension, Null Space, and Rank.)

The dimension of the input space splits **exactly** into the part that gets sent to zero (the kernel) and the part that survives as the image.

71.1. Why it's true (sketch)

Pick a basis $\{k_1, ..., k_p\}$ for $\ker(T)$, then extend it to a basis $\{k_1, ..., k_p, v_1, ..., v_q\}$ for V (so $n = p + q$).

Then $\{T(v_1), ..., T(v_q)\}$ is a basis for $\mathrm{im}(T)$ — they span (because the k's map to 0) and are linearly independent (a non-trivial dependency would land back in $\ker(T)$, contradicting linear independence of the v's with the k's).

So $\dim(\mathrm{im}(T)) = q$, and $\dim(\ker(T)) + \dim(\mathrm{im}(T)) = p + q = n$. ∎

71.2. Why it matters

It's the bookkeeping identity for linear maps. Lots of immediate consequences:

- **Injectivity from dimensions**: if $T : \mathbb{R}^n \to \mathbb{R}^m$ has $n > m$, then $\dim(\ker(T)) \geq n - m > 0$ — **not injective**.
- **Surjectivity from dimensions**: if $T : \mathbb{R}^n \to \mathbb{R}^m$ with $n < m$, then $\dim(\text{im}(T)) \leq n < m$ — **not surjective**.
- **Square invertibility**: a square matrix A is invertible iff $\ker(A) = \{\mathbf{0}\}$ iff $\text{im}(A) = \mathbb{R}^n$ — by rank–nullity these are equivalent for square A.
- **Solution space dimension**: the solution set of $A\vec{x} = \vec{b}$ (when consistent) has dimension $n - \text{rank}(A)$ — see Linear System Solutions.

Example

$A = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \end{bmatrix}$. The two rows are dependent $\to \text{rank}(A) = 1$.

$n = 3 \to$ nullity $= 3 - 1 = 2$.

$A\vec{x} = \mathbf{0}$ has a 2-dimensional solution space (a plane through the origin in $\mathbb{R}^3$).

71.3. Equivalent statements

For an $m \times n$ matrix A:

- $\text{rank}(A) + \text{nullity}(A) = n$ (number of columns)
- $\text{rank}(A) = \text{rank}(A^T)$ — **row rank = column rank**
- $\text{nullity}(A^T) = m - \text{rank}(A)$ — gives the dimension of the **left** null space

71.4. See also

- **Rank**
- **Null Space** / **Kernel**
- **Image** / **Column Space**
- **Surjective / Injective / Bijective**

72. Projection

The projection of a vector $\vec{x}$, onto a line L, denoted as $\text{Proj}_L(\vec{x})$, is a vector that lies on the line L, such that the difference between $\vec{x}$ and its projection, $\text{Proj}_L(\vec{x}) - \vec{x}$, is orthogonal to L

$\text{Proj}_L(\vec{x})$ can be seen as the "shadow"cast by $\vec{x}$ onto L when light shines perpendicularly to L.

$$\text{Proj}_L(\vec{x}) = c\vec{v} = \left(\frac{\vec{x} \cdot \vec{v}}{\vec{v} \cdot \vec{v}}\right)\vec{v}$$

Equivalently

$$\text{Proj}_L(\vec{x}) = c\vec{v} = \left(\frac{\vec{x}\cdot\vec{v}}{\|\vec{v}\|^2}\right)\vec{v}$$

Where:

- $\vec{v}$ is a direction vector for the line L
- $c = \frac{\vec{x}\cdot\vec{v}}{\vec{v}\cdot\vec{v}}$ is a scalar

Example

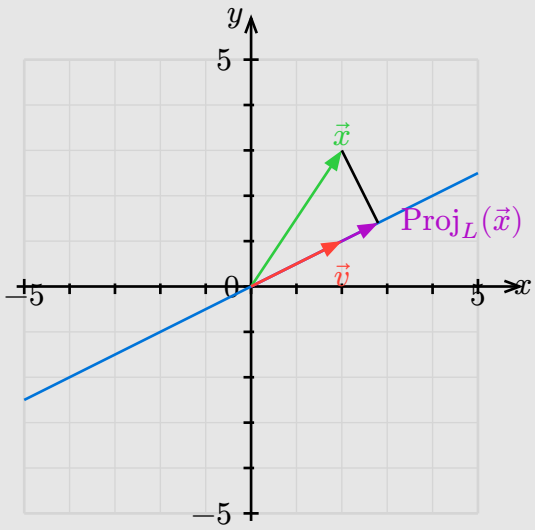
$$\vec{x} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}, \qquad \vec{v} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

1. Define the line L as all vectors of the form $c\vec{v}$, where c is a scalar:

$$\begin{aligned} L &= \{c\vec{v} \mid c \in \mathbb{R}\} \\ &= \left\{ c\begin{bmatrix} 2 \\ 1 \end{bmatrix} \mid c \in \mathbb{R} \right\} \end{aligned}$$

2. Compute the projection using

$$\begin{aligned} \text{Proj}_L(\vec{x}) &= \frac{\vec{x}\cdot\vec{v}}{\vec{v}\cdot\vec{v}}\vec{v} \\ &= \frac{\begin{bmatrix} 2 \\ 3 \end{bmatrix} \cdot \begin{bmatrix} 2 \\ 1 \end{bmatrix}}{\begin{bmatrix} 2 \\ 1 \end{bmatrix} \cdot \begin{bmatrix} 2 \\ 1 \end{bmatrix}} \begin{bmatrix} 2 \\ 1 \end{bmatrix} \\ &= \frac{7}{5}\begin{bmatrix} 2 \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} 2.8 \\ 1.4 \end{bmatrix} \end{aligned}$$



Projection as a Transformation

As a matrix vector product

$$L = \{c\vec{v} \mid c \in \mathbb{R}\}$$

$$\text{Proj}_L : \mathbb{R}^n \to \mathbb{R}^n$$

$$\vec{v} \cdot \vec{v} = \| \vec{v} \|^2$$

If $\vec{v}$ is a unit vector:

$$\|\vec{v}\| = 1$$

Then

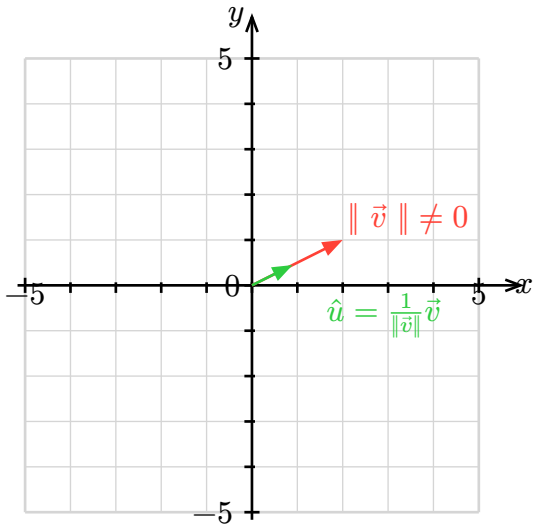
$$\begin{aligned} \text{Proj}_L(\vec{x}) &= \left(\frac{\vec{x} \cdot \vec{v}}{\vec{v} \cdot \vec{v}}\right)\vec{v} \\ &= \left(\frac{\vec{x} \cdot \vec{v}}{\| \vec{v} \|^2}\right)\vec{v} \end{aligned}$$

If we redefine our line L as all the scalar multiples of out unit vector $\hat{u}$:

$$L = \{c\hat{u} \mid c \in \mathbb{R}\}$$

Simplifies to:

$$\boxed{(\vec{x} \cdot \hat{u})\hat{u}}$$



Projection as a Linear Transformation

Let $\hat{u} \in \mathbb{R}^n$ be a unit vector:

$$\hat{u} = \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_n \end{bmatrix}, \quad \text{where } \|\hat{u}\| = 1$$

The projection matrix A is:

$$A = \hat{u}\hat{u}^T = \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_n \end{bmatrix} [u_1 \quad u_2 \quad \dots \quad u_n]$$

Expands to:

$$A = \begin{bmatrix} u_1u_1 & u_1u_2 & \dots & u_1u_n \\ u_2u_1 & u_2u_2 & \dots & u_2u_n \\ \vdots & \vdots & \ddots & \vdots \\ u_nu_1 & u_nu_2 & \dots & u_nu_n \end{bmatrix}$$

For any $\vec{x} \in \mathbb{R}^n$, the projection of $\vec{x}$ onto the line spanned by $\hat{u}$ is:

$$\text{Proj}_L(\vec{x}) = A\vec{x} = (\hat{u} \cdot \vec{x})\hat{u}$$

Example

Consider a vector $\vec{v}$ in $\mathbb{R}^2$:

$$\vec{v} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

1. Construct the Unit Vector

$$\| \vec{v} = \sqrt{2^2 + 1^2} = \sqrt{5}$$

$$\hat{u} = \frac{\vec{v}}{\| \vec{v} \|} = \frac{1}{\sqrt{5}} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{2}{\sqrt{5}} \\ \frac{1}{\sqrt{5}} \end{bmatrix}$$

This unit vector $\hat{u}$ defines the line L, which consists of all the scalar multiples og $\vec{v}$:

$$L = \{c\vec{v} \mid c \in \mathbb{R}\}$$

2. Derive the Projection Matrix

The projection of any vector $\vec{x}$ onto the line L is given by:

$$\text{Proj}_L(\vec{x}) = A\vec{x}$$

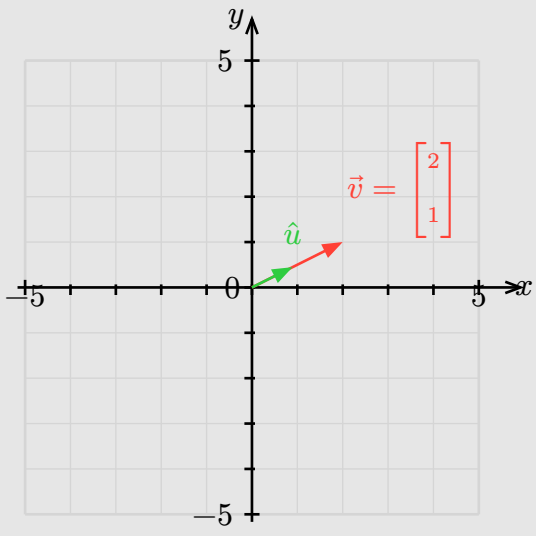
Where A is the projection matrix. To construct A we use the formula:

$$
\begin{aligned}
A &= \hat{u}\hat{u}^T \\
&= \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} \begin{bmatrix} u_1 & u_2 \end{bmatrix} \\
&= \begin{bmatrix} u_1 \cdot u_1 & u_1 \cdot u_2 \\ u_2 \cdot u_1 & u_2 \cdot u_2 \end{bmatrix} \\
&= \begin{bmatrix} \frac{2}{\sqrt{5}} \\ \frac{1}{\sqrt{5}} \end{bmatrix} \begin{bmatrix} \frac{2}{\sqrt{5}} & \frac{1}{\sqrt{5}} \end{bmatrix} \\
&= \begin{bmatrix} \left(\frac{2}{\sqrt{5}}\right)^2 & \frac{1}{\sqrt{5}}\frac{2}{\sqrt{5}} \\ \frac{2}{\sqrt{5}}\frac{1}{\sqrt{5}} & \left(\frac{1}{\sqrt{5}}\right)^2 \end{bmatrix} \\
&= \begin{bmatrix} \frac{4}{5} & \frac{2}{5} \\ \frac{2}{5} & \frac{1}{5} \end{bmatrix}
\end{aligned}
$$

3. Applying the Projection

To project any vector $\vec{x}$ onto L, we multiply $\vec{x}$ by the matrix A:

$$
\begin{aligned}
\text{Proj}_L(\vec{x}) &= A\vec{x} \\
&= \begin{bmatrix} \frac{4}{5} & \frac{2}{5} \\ \frac{2}{5} & \frac{1}{5} \end{bmatrix} \vec{x}
\end{aligned}
$$



1. Additivity of Projections (Linearity with respect to addition)

$$\begin{aligned}\text{Proj}_L\left(\vec{a}+\vec{b}\right) &= \left(\left(\vec{a}+\vec{b}\right)\cdot\hat{u}\right)\\ &= \left(\vec{a}\cdot\hat{u}+\vec{b}\cdot\hat{u}\right)\hat{u}\\ &= (\vec{a}\cdot\hat{u})\hat{u}+\left(\vec{b}\cdot\hat{u}\right)\hat{u}\\ &= \text{Proj}_L(\vec{a})+\text{Proj}_L\left(\vec{b}\right)\end{aligned}$$

2. Homogeneity of Projections (Linearity with respect to scalar multiplication)

$$\begin{aligned}\text{Proj}_L(c\vec{a}) &= (c\vec{a}\cdot\hat{u})\hat{u}\\ &= c(\vec{a}\cdot\hat{u})\hat{u}\\ &= c\text{Proj}_L(\vec{a})\end{aligned}$$

General Properties of A

1. Idempotence

$$A^2 = A$$

2. Symmetry

$$A^T = A$$

3. Rank

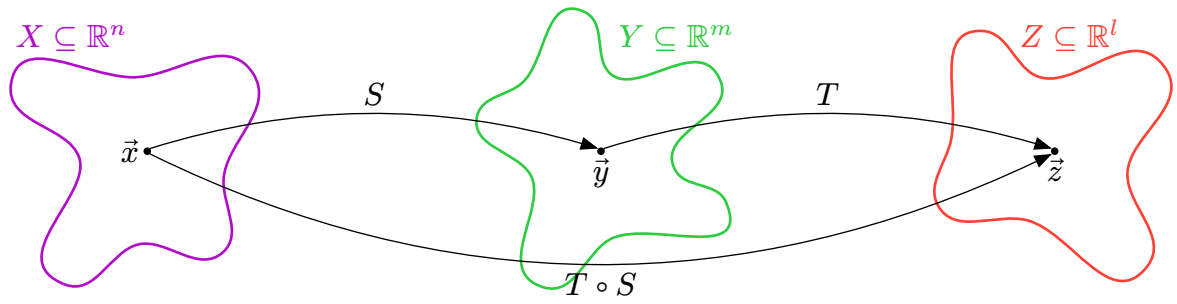
$$\text{rank}(A) = 1$$

Because $\hat{u}\hat{u}^T$ projects onto a one-dimensional subspace spanned by $\hat{u}$

73. Composition of Linear Transformations

For two linear transformations $S : X \to Y$ and $T : Y \to Z$, the **composition** $T \circ S : X \to Z$ applies S first, then T:

$$(T \circ S)(\vec{x}) = T(S(\vec{x}))$$



The composition of two linear transformations is itself linear:

Additivity:

$$(T \circ S)(\vec{x}+\vec{y}) = T(S(\vec{x})+S(\vec{y})) = T(S(\vec{x}))+T(S(\vec{y})) = (T \circ S)(\vec{x})+(T \circ S)(\vec{y})$$

Homogeneity: same argument with scalar pulled through.

73.1. Composition ↔ matrix multiplication

If S has matrix A ($m \times n$) and T has matrix B ($l \times m$), then:

$$(T \circ S)(\vec{x}) = T(A\vec{x}) = B(A\vec{x}) = (BA)\vec{x}$$

So composition corresponds to **matrix multiplication** — with the matrices in **reverse** order (the transformation applied first sits on the right):

$$T \circ S \;\leftrightarrow\; BA$$

This single observation is the entire reason matrix multiplication is defined the way it is.

73.2. Properties of composition

- **Associative**: $(R \circ T) \circ S = R \circ (T \circ S)$ — follows from matrix multiplication being associative
- **Not commutative**: in general $T \circ S \neq S \circ T$ — different transformations don't commute (and matrices generally don't either)
- **Identity**: composing with I (the identity transformation) leaves any transformation unchanged
- **Inverse**: $(T \circ S)^{-1} = S^{-1} \circ T^{-1}$ (when defined) — peel off in reverse order

73.3. See also

- **Linear Transformation**
- **Matrix Multiplication**
- **Matrix Representation** — how transformations become matrices
- **Matrix Inverse**

74. Transformation Sum

For linear transformations $T, S : \mathbb{R}^n \to \mathbb{R}^m$, define the sum $T + S : \mathbb{R}^n \to \mathbb{R}^m$ pointwise:

$$(T + S)(\vec{x}) = T(\vec{x}) + S(\vec{x})$$

The sum is itself linear.

74.1. Matrix correspondence

If T has matrix A and S has matrix B (so $T(\vec{x}) = A\vec{x}$ and $S(\vec{x}) = B\vec{x}$), then $T + S$ has matrix $A + B$:

Let $A = \begin{bmatrix} \vec{a}_1 & \vec{a}_2 & \dots & \vec{a}_n \end{bmatrix}$ and $B = \begin{bmatrix} \vec{b}_1 & \vec{b}_2 & \dots & \vec{b}_n \end{bmatrix}$ (column-wise).

$$
\begin{aligned}
(T+S)(\vec{x}) &= T(\vec{x}) + S(\vec{x}) \\
&= A\vec{x} + B\vec{x} \\
&= \sum_{i=1}^{n} x_i \vec{a}_i + \sum_{i=1}^{n} x_i \vec{b}_i \\
&= \sum_{i=1}^{n} x_i \left(\vec{a}_i + \vec{b}_i\right) \\
&= (A+B)\vec{x}
\end{aligned}
$$

So in matrix terms: **transformation addition = matrix addition**, performed entrywise.

74.2. Connections

- **Scalar multiplication of transformations** — companion operation
- **Linear Transformation** — the underlying object
- **Matrix Multiplication** — composition is the multiplicative analog (composition ↔ product, sum ↔ sum)

75. Transformation Scalar Multiplication

For a linear transformation $T : \mathbb{R}^n \to \mathbb{R}^m$ and a scalar $c \in \mathbb{R}$, define the scaled transformation $cT : \mathbb{R}^n \to \mathbb{R}^m$ pointwise:

$$(cT)(\vec{x}) = c\,T(\vec{x})$$

The scaled transformation is itself linear.

75.1. Matrix correspondence

If T has matrix $A = \begin{bmatrix} \vec{a}_1 & \vec{a}_2 & \dots & \vec{a}_n \end{bmatrix}$, then cT has matrix cA:

$$
\begin{aligned}
(cT)(\vec{x}) &= c(T(\vec{x})) \\
&= c\left(\sum_{i=1}^{n} x_i \vec{a}_i\right) \\
&= \sum_{i=1}^{n} x_i (c\vec{a}_i) \\
&= (cA)\vec{x}
\end{aligned}
$$

So **scaling a transformation = scaling its matrix entrywise**.

75.2. Combined with sum: vector space of transformations

Together with sum, scalar multiplication makes the set of linear transformations $\mathbb{R}^n \to \mathbb{R}^m$ into a vector space of dimension mn — isomorphic to the space of $m \times n$ matrices.

75.3. Connections

- **Transformation Sum** — companion operation
- **Vector Operations** — same structure (scaling distributes over sum) at the vector level
- **Linear Transformation** — the underlying object

76. Rotation Matrix

A rotation matrix rotates vectors about the origin by an angle θ without changing their lengths.

76.1. 2D rotation

Rotation by angle θ **counterclockwise** about the origin:

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

Applied to $\vec{x} = \begin{bmatrix} x \\ y \end{bmatrix}$:

$$R(\theta)\vec{x} = \begin{bmatrix} x\cos\theta - y\sin\theta \\ x\sin\theta + y\cos\theta \end{bmatrix}$$

Example

Rotate $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$ by $\theta = 90°$ ($\frac{\pi}{2}$):

$$R\left(\frac{\pi}{2}\right) = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$

$$R\left(\frac{\pi}{2}\right)\begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

The unit x-vector becomes the unit y-vector. ✓

76.2. Properties

- **Orthogonal**: $R(\theta)^T R(\theta) = I$, so $R^{-1} = R^T$ — see Orthogonal Matrix
- **Determinant**: $\det(R(\theta)) = \cos^2\theta + \sin^2\theta = 1$ — orientation-preserving
- **Composition**: $R(\alpha)R(\beta) = R(\alpha + \beta)$ — rotations add
- **Inverse**: $R(\theta)^{-1} = R(-\theta)$
- **Preserves lengths and angles**

76.3. 3D rotations (about coordinate axes)

About the x-axis:

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta \\ 0 & \sin\theta & \cos\theta \end{bmatrix}$$

About the y-axis:

$$R_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{bmatrix}$$

About the z-axis:

$$R_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

76.4. General 3D rotation

Any 3D rotation is a composition of rotations about coordinate axes. Common parameterizations:

- **Euler angles**: $R_z(\alpha)R_y(\beta)R_x(\gamma)$ — three angles
- **Axis–angle (Rodrigues formula)**: rotate by angle θ about unit axis $\hat{k}$:

$$R = I + \sin\theta\, K + (1 - \cos\theta)K^2$$

where K is the skew-symmetric "cross-product matrix" of $\hat{k}$

- **Quaternions**: $R = q \cdot \vec{x} \cdot q^*$ — avoids gimbal lock, used in 3D graphics / robotics

76.5. Eigenvalues

For a 2D rotation by θ:

$$\lambda = e^{\pm i\theta} = \cos\theta \pm i\sin\theta$$

Real eigenvalues only when $\theta = 0$ ($\lambda = 1$) or $\theta = \pi$ ($\lambda = -1$). Otherwise the eigenvalues are complex — rotations have no real eigenvectors (no fixed direction in the plane).

76.6. Connection to special orthogonal group

The set of all $n \times n$ rotation matrices forms the **special orthogonal group** $SO(n)$:

$$SO(n) = \{Q \in \mathbb{R}^{n\times n} : Q^TQ = I, \det(Q) = 1\}$$

Excluding the -1 determinant case removes reflections — see Reflection Matrix.

76.7. See also

- **Orthogonal Matrix**
- **Reflection Matrix**
- **Scaling Matrix**
- **Shear Matrix**
- **Linear Transformation**

77. Reflection Matrix

A reflection matrix maps each point to its **mirror image** across a fixed line (in 2D), plane (in 3D), or hyperplane (in $\mathbb{R}^n$).

77.1. Reflection across a line through the origin (2D)

Line making angle θ with the x-axis:

$$R_\theta = \begin{bmatrix} \cos 2\theta & \sin 2\theta \\ \sin 2\theta & -\cos 2\theta \end{bmatrix}$$

Special cases:

Axis	Matrix
x-axis ($\theta = 0$)	$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$
y-axis ($\theta = \frac{\pi}{2}$)	$\begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$
Line $y = x$ ($\theta = \frac{\pi}{4}$)	$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$
Line $y = -x$ ($\theta = -\frac{\pi}{4}$)	$\begin{bmatrix} 0 & -1 \\ -1 & 0 \end{bmatrix}$

77.2. Reflection across a hyperplane (general)

For a hyperplane through the origin with unit normal $\hat{n}$, the reflection is the **Householder matrix**:

$$R = I - 2\hat{n}\hat{n}^T$$

This subtracts twice the projection onto $\hat{n}$ from each input.

Example

Reflect across the plane $z = 0$ in $\mathbb{R}^3$. Normal: $\hat{n} = (0, 0, 1)$.

$$R = I - 2\begin{bmatrix}0\\0\\1\end{bmatrix}(0,0,1) = \begin{bmatrix}1&0&0\\0&1&0\\0&0&1\end{bmatrix} - 2\begin{bmatrix}0&0&0\\0&0&0\\0&0&1\end{bmatrix} = \begin{bmatrix}1&0&0\\0&1&0\\0&0&-1\end{bmatrix}$$

z flips sign, x and y preserved. ✓

77.3. Properties

- **Orthogonal**: $R^T R = I$, so $R^{-1} = R^T = R$ (reflections are self-inverse — reflecting twice is the identity)
- **Determinant**: $\det(R) = -1$ — orientation-reversing
- **Eigenvalues**: -1 (for the direction perpendicular to the mirror) and $+1$ with multiplicity $n - 1$ (for directions parallel to the mirror)
- **Preserves lengths and angles** (it's orthogonal)
- **Involution**: $R^2 = I$

77.4. Rotations vs reflections

	Rotation	Reflection
$\det$	$+1$	-1
Orientation	preserved	reversed
Self-inverse?	no (unless 0 or π)	yes
Symmetric?	no	yes

Composition: two reflections produce a rotation (by 2θ, where θ is the angle between the mirrors).

77.5. Householder transformations

Reflections of the form $R = I - 2\vec{v}\frac{\vec{v}^T}{\vec{v}^T\vec{v}}$ (Householder transformations) are the building blocks of stable numerical algorithms for:

- **QR decomposition** — alternative to Gram–Schmidt, much more stable
- **Tridiagonalization** of symmetric matrices (preprocessing step for eigenvalue computation)

77.6. See also

- **Rotation Matrix**
- **Orthogonal Matrix**
- **Scaling Matrix**
- **Shear Matrix**
- **Projection** — related but $\det = 0$ (collapses dimension)

78. Scaling Matrix

A **scaling matrix** stretches (or compresses) each coordinate independently. It's a diagonal matrix:

$$S = \begin{bmatrix} s_1 & 0 & \dots & 0 \\ 0 & s_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & s_n \end{bmatrix}$$

The i-th coordinate gets multiplied by s_i:

$$S\vec{x} = \begin{bmatrix} s_1 x_1 \\ s_2 x_2 \\ \vdots \\ s_n x_n \end{bmatrix}$$

78.1. Uniform vs non-uniform

- **Uniform scaling**: all $s_i = c$, i.e. $S = cI$ — stretches every direction by the same factor c. Shapes are similar (same angles, scaled lengths).
- **Non-uniform scaling**: s_i's differ — distorts shapes. A circle becomes an ellipse.

Example

$S = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$.

$S \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$. The unit circle $x^2 + y^2 = 1$ maps to the ellipse $\left(\frac{x}{2}\right)^2 + \left(\frac{y}{3}\right)^2 = 1$.

78.2. Properties

- **Determinant**: $\det(S) = s_1 s_2 \dots s_n$ — total volume change factor
- **Inverse**: $S^{-1} = \operatorname{diag}\left(\frac{1}{s_1}, \frac{1}{s_2}, \dots, \frac{1}{s_n}\right)$ (when all $s_i \neq 0$)
- **Singular when any $s_i = 0$**: the corresponding direction is collapsed
- **Eigenvalues**: the s_i's themselves
- **Eigenvectors**: the standard basis $e_1, e_2, \dots, e_n$
- **Already in diagonal form** — diagonalization is trivial

78.3. Negative scaling

Scaling by $s_i < 0$ flips the i-th coordinate:

- One negative entry: orientation reverses ($\det < 0$) — combination of a scaling and a reflection
- Two negative entries: orientation preserved ($\det > 0$) — same as a 180° rotation by a scaling

78.4. Scaling in a general direction

To scale by factor c along a **non-axis** direction $\hat{v}$, conjugate by a rotation that aligns $\hat{v}$ with a coordinate axis:

$$S_{\hat{v},c} = R^T \begin{bmatrix} c & 0 \\ 0 & 1 \end{bmatrix} R$$

Or, more directly: $S_{\hat{v},c} = I + (c-1)\hat{v}\hat{v}^T$ — adds $c-1$ extra of the projection onto $\hat{v}$.

78.5. Application to images / graphics

- $S = \mathrm{diag}(s, s)$ — zoom in/out uniformly
- $S = \mathrm{diag}(2, 1)$ — stretch horizontally
- $S = \mathrm{diag}(1, -1)$ — flip vertically (= reflection)
- Singular value decomposition (SVD) writes every linear map as rotate–**scale**–rotate, so non-uniform scaling is the "shape-distorting" core of any matrix

78.6. See also

- **Diagonal Matrix**
- **Rotation Matrix**
- **Reflection Matrix**
- **Shear Matrix**
- **SVD** — every matrix is rotate–scale–rotate

79. Shear Matrix

A **shear matrix** shifts each point in one direction by an amount proportional to its coordinate in another. The result: parallelograms slide into other parallelograms, preserving area.

79.1. 2D shears

Horizontal shear (each point shifts horizontally by a multiple of its y-coordinate):

$$S_x(k) = \begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix}$$

$$S_x(k) \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x + ky \\ y \end{bmatrix}$$

Vertical shear:

$$S_y(k) = \begin{bmatrix} 1 & 0 \\ k & 1 \end{bmatrix}$$

$$S_y(k) \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x \\ y + kx \end{bmatrix}$$

Example

Horizontal shear with $k = 1$ applied to the unit square:

$$\begin{bmatrix} 0 \\ 0 \end{bmatrix} \to \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad \begin{bmatrix} 1 \\ 0 \end{bmatrix} \to \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad \begin{bmatrix} 1 \\ 1 \end{bmatrix} \to \begin{bmatrix} 2 \\ 1 \end{bmatrix} \quad \begin{bmatrix} 0 \\ 1 \end{bmatrix} \to \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Square becomes a parallelogram. Area $= 1$ — preserved.

79.2. Properties

- **Determinant**: $\det(S) = 1$ — **area / volume preserving**
- **Inverse**: shear by $-k$ in the same direction:

$$S_x(k)^{-1} = S_x(-k) = \begin{bmatrix} 1 & -k \\ 0 & 1 \end{bmatrix}$$

- **Eigenvalues**: 1 (repeated) — both with the same eigenvector $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$ for the horizontal shear; not diagonalizable when $k \neq 0$
- **Fixed line**: the axis being sheared along (e.g., the x-axis stays put for horizontal shear)
- **Triangular**: shear matrices are upper / lower triangular

79.3. Why $\det = 1$

Geometrically: a parallelogram slides — its base is unchanged, and its height (perpendicular distance) is unchanged. Area $=$ base $\times$ height stays the same. The same logic generalizes to n dimensions.

79.4. Composition of shears

The product of two shears (in the same direction) is another shear:

$$S_x(k_1)S_x(k_2) = S_x(k_1 + k_2)$$

Two shears in **different** directions can produce non-shear results (composition need not be triangular).

79.5. Application: graphics and italics

- **Italic typography**: italic letters are upright letters with a horizontal shear applied

- **Graphics transforms**: any 2D affine transformation can be decomposed into translate, rotate, scale, **shear**
- **Solving linear systems**: row operations are shears applied to the matrix — that's why row reduction preserves the row-space dimensions but reshapes the picture

79.6. Shears in higher dimensions

A general shear in $\mathbb{R}^n$ adds a multiple of one coordinate to another. The matrix is the identity with a single off-diagonal non-zero entry:

$$I + k\, e_i e_j^T, \quad i \neq j$$

79.7. See also

- **Rotation Matrix**
- **Reflection Matrix**
- **Scaling Matrix**
- **Triangular Matrix**
- **Linear Transformation**

80. Minor

A **minor** of a matrix A is the determinant of a smaller square matrix obtained by deleting some rows and some columns of A. The deleted rows and columns **do not need to share indices** — that distinguishes a minor from a principal minor.

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix}$$

Pick rows $\{1, 3\}$ and columns $\{2, 4\}$ — different sets, this is **not** a principal minor:

$$\begin{vmatrix} a_{12} & a_{14} \\ a_{32} & a_{34} \end{vmatrix}$$

80.1. (i, j)-minor (cofactor expansion)

The most-used minor is the (i, j)-minor M_{ij}: delete row i and column j from A and take the determinant of what's left. Used in cofactor expansion:

$$\det(A) = \sum_{j=1}^{n} (-1)^{i+j} a_{ij}\, M_{ij}$$

80.2. Variants

- **Principal minor** — same row and column indices deleted

- **Leading principal minor** — keep only the top-left $k \times k$ block
- **Cofactor** $C_{ij} = (-1)^{i+j} M_{ij}$ — signed minor used in adjugate and inverse formulas

80.3. Where minors show up

- **Determinant** via cofactor expansion
- **Adjugate** matrix and the inverse formula $A^{-1} = \frac{\text{adj}(A)}{\det(A)}$
- **Rank** — equals the size of the largest non-zero minor
- **Cramer's Rule** — see Cramer's Rule

81. Principal Minor

A **principal minor** of an $n \times n$ matrix A is the determinant of a square submatrix whose diagonal is a subset of A's diagonal.

To form a principal minor: pick a subset of indices $I \subseteq \{1, 2, ..., n\}$, then delete every row and column **not** in I. The resulting submatrix is square; its determinant is the principal minor.

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix}$$

Pick rows/columns $I = \{1, 3\}$:

$$\begin{vmatrix} a_{11} & a_{13} \\ a_{31} & a_{33} \end{vmatrix}$$

81.1. Levels (order)

A principal minor formed from $|I| = k$ indices is called a **level-k** (or **order-k**) principal minor.

For an $n \times n$ matrix:

- There are n levels (from $k = 1$ to $k = n$)
- The number of level-k principal minors is $\binom{n}{k}$
- Total number of principal minors: $\sum_{k=0}^{n} \binom{n}{k} = 2^n$ (including the trivial empty one)

81.2. Where they show up

- **Sylvester's criterion** (positive-definiteness test): a symmetric matrix is positive definite iff every leading principal minor is positive
- **Positive semi-definiteness**: requires **every** principal minor (not just leading) to be non-negative
- **Convex analysis** — checking definiteness of the Hessian via principal minors

81.3. See also

- **Leading Principal Minor** — minors from the top-left corner
- **Minor** — general minors (rows and columns can differ)
- **Determinant**

82. Leading Principal Minor

A **leading principal minor** of order k is the determinant of the **top-left** $k \times k$ submatrix of A.

That is, take rows and columns indexed $1, 2, ..., k$ — no other choices.

For a 3×3 matrix, the three leading principal minors are:

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \quad \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \quad \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

Order 1: a_{11}. Order 2: $\det \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}$. Order 3: $\det(A)$ itself.

82.1. Sylvester's criterion (positive-definiteness)

A symmetric matrix A is **positive definite** iff **all** leading principal minors are strictly positive:

$$\Delta_1 > 0, \ \Delta_2 > 0, \ ..., \ \Delta_n > 0$$

where Δ_k is the order-k leading principal minor.

For **negative definite**: signs alternate — $\Delta_1 < 0, \Delta_2 > 0, \Delta_3 < 0, ...$ (i.e. $(-1)^k \Delta_k > 0$).

Example

$$A = \begin{bmatrix} 2 & -1 \\ -1 & 3 \end{bmatrix}$$

$\Delta_1 = 2 > 0$, $\Delta_2 = 2 \cdot 3 - (-1)(-1) = 5 > 0$. So A is positive definite.

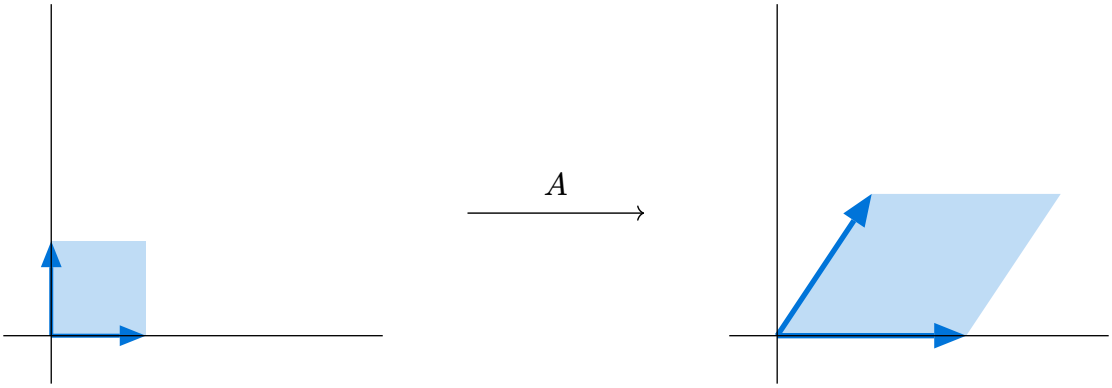
82.2. See also

- **Principal Minor** — general principal minors (any subset of indices)
- **Minor** — general (cofactor) minors
- **Determinant**

83. Determinant

The determinant of a **square** matrix is the **signed scale factor** by which it scales area (or volume).

$\det(A)$



$\text{Area} = 1$ $\text{Area} = 3$

83.1. From first principles: the 2×2 case

Take two vectors $\vec{a}, \vec{b} \in \mathbb{R}^2$:

$$\vec{a} = \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} \qquad \vec{b} = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix}$$

They span a parallelogram. Its **signed area** is:

$$\text{Area} = a_1 b_2 - a_2 b_1$$

That signed area is exactly the determinant of the matrix whose columns are $\vec{a}, \vec{b}$:

$$\det \begin{bmatrix} a_1 & b_1 \\ a_2 & b_2 \end{bmatrix} = a_1 b_2 - a_2 b_1$$

Sign:

- $+$ if $\vec{b}$ is counterclockwise from $\vec{a}$ (preserved orientation)
- $-$ if clockwise (orientation flipped)
- 0 if $\vec{a}$ and $\vec{b}$ are collinear (parallelogram degenerates)

Example

$$A = \begin{bmatrix} 3 & 1 \\ 0 & 2 \end{bmatrix} \qquad \det(A) = 3 \cdot 2 - 1 \cdot 0 = 6$$

The unit square $[0, 1]^2$ maps to a parallelogram of area 6 — six times bigger.

$$B = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix} \qquad \det(B) = 1 \cdot 4 - 2 \cdot 2 = 0$$

The columns are parallel (one is twice the other) — image collapses to a line, area 0.

83.2. 3×3: signed volume of the parallelepiped

Three column vectors $\vec{a}, \vec{b}, \vec{c} \in \mathbb{R}^3$ span a parallelepiped. Its signed volume is:

$$\det \begin{bmatrix} a_1 & b_1 & c_1 \\ a_2 & b_2 & c_2 \\ a_3 & b_3 & c_3 \end{bmatrix} = a_1(b_2c_3 - b_3c_2) - a_2(b_1c_3 - b_3c_1) + a_3(b_1c_2 - b_2c_1)$$

This is the **cofactor expansion** along the first column — three 2×2 determinants, alternating sign.

83.3. General $n \times n$: cofactor expansion

For an $n \times n$ matrix A, expand along **any** row i (or any column j):

$$\det(A) = \sum_{j=1}^{n} (-1)^{i+j} a_{ij}\, M_{ij}$$

where M_{ij} is the **minor** — the determinant of the $(n-1) \times (n-1)$ submatrix obtained by deleting row i and column j of A (see Minor).

The factor $(-1)^{i+j}$ produces the **checkerboard sign pattern**:

$$\begin{bmatrix} + & - & + & \dots \\ - & + & - & \dots \\ + & - & + & \dots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix}$$

The recursion bottoms out at the 1×1 case: $\det([a]) = a$.

83.4. Key properties

- **Identity**: $\det(I_n) = 1$
- **Multiplicativity**: $\det(AB) = \det(A)\det(B)$
- **Transpose**: $\det(A^T) = \det(A)$
- **Inverse**: $\det(A^{-1}) = \frac{1}{\det(A)}$ (when defined)
- **Scalar multiple**: $\det(c\,A) = c^n \det(A)$ for an $n \times n$ matrix
- **Triangular matrix**: $\det =$ product of diagonal entries
- **Row swap**: flips the sign
- **Row scaled by c**: multiplies $\det$ by c
- **Row $+$ multiple of another row**: leaves $\det$ unchanged (basis of Gaussian-elimination computation)

83.5. What $\det(A)$ tells you about A

$\det(A)$	Implication
$\neq 0$	A is invertible (see Matrix Inverse); columns are linearly independent (see Linear Independence); columns span $\mathbb{R}^n$; A has full rank (see Rank); $Ax = b$ has a unique solution for every b.
$= 0$	A is **singular** — non-invertible; columns are linearly dependent; image is a strict subspace; $Ax = 0$ has non-trivial solutions; the linear map collapses dimension.

83.6. Geometric interpretation

For a linear map $T : \mathbb{R}^n \to \mathbb{R}^n$ given by matrix A:

- $|\det(A)|$ = factor by which T scales n-dimensional volume
- $\text{sign}(\det(A))$ = whether T preserves $(+)$ or reverses $(-)$ orientation

Example

Reflection across the y-axis:

$$R = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} \qquad \det(R) = -1$$

Areas preserved (factor $|-1| = 1$); orientation reversed (sign).

83.7. Computing $\det$ in practice

- **2×2 / 3×3**: direct formula above.
- **Larger matrices**: cofactor expansion is $O(n!)$ — impractical past $n \approx 10$.
- **Standard method**: row-reduce to triangular form, multiply diagonal entries, track row-operation effects on $\det$ (see Gaussian Elimination). This is $O(n^3)$.
- **LU decomposition** (see LU Decomposition): $\det(A) = \det(L)\det(U) = \prod$ of U's diagonal entries (since L has unit diagonal).

83.8. Connection to other concepts

- **Minor** — building block for cofactor expansion
- **Adjugate** — matrix of cofactors, used in $A^{-1} = \frac{\text{adj}(A)}{\det(A)}$
- **Eigenvectors & Eigenvalues** — eigenvalues are roots of $\det(A - \lambda I) = 0$
- **Unimodularity** — matrices with $\det = \pm 1$

84. Cramer's Rule

For a square system $A\vec{x} = \vec{b}$ where A is $n \times n$ and invertible (i.e., $\det(A) \neq 0$), each component of the solution is a ratio of determinants:

$$x_i = \frac{\det(A_i)}{\det(A)}$$

where A_i is the matrix A with its i-th column replaced by $\vec{b}$.

Example

Solve

$$2x + y = 5$$
$$x + 3y = 10$$

Then $A = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}$, $\vec{b} = \begin{bmatrix} 5 \\ 10 \end{bmatrix}$, and $\det(A) = 6 - 1 = 5$.

Replace column 1:

$$A_1 = \begin{bmatrix} 5 & 1 \\ 10 & 3 \end{bmatrix} \qquad \det(A_1) = 15 - 10 = 5$$

Replace column 2:

$$A_2 = \begin{bmatrix} 2 & 5 \\ 1 & 10 \end{bmatrix} \qquad \det(A_2) = 20 - 5 = 15$$

$$x = \frac{5}{5} = 1, \qquad y = \frac{15}{5} = 3$$

84.1. Why it works (sketch)

Let E_i be the identity with the i-th column replaced by $\vec{x}$. Then $AE_i = A_i$ (column-by-column), so

$$\det(A_i) = \det(AE_i) = \det(A)\det(E_i) = \det(A)\, x_i$$

(since E_i is upper triangular with x_i on its i-th diagonal entry).

84.2. Practical remarks

Cramer's rule is **theoretically beautiful** but **computationally awful** for $n > 3$:

- Computes $n + 1$ determinants of $n \times n$ matrices
- Cost: $O(n!)$ via cofactor expansion, or $O(n^4)$ even with row reduction
- Standard methods (Gaussian elimination, LU decomposition) cost $O(n^3)$

Use Cramer's rule for:

- Symbolic solutions (small systems with parameters)
- Theoretical proofs
- Closed-form expressions in 2×2 or 3×3 cases

Don't use it for serious numerical work.

84.3. When it fails

If $\det(A) = 0$, the formula divides by zero — meaning A is singular and the system either has no solution or infinitely many (depending on $\vec{b}$). See Linear System Solutions.

84.4. See also

- **Determinant**
- **Matrix Inverse** — Cramer's rule is essentially $A^{-1}\vec{b}$ with the inverse expanded via cofactors
- **Adjugate** — alternative inverse formula
- **Gaussian Elimination** — the practical alternative

85. Adjugate

$$\operatorname{adj}(A)$$

1. Compute **Minor** M_{ij}

For each entry A_{ij} of A, take the determinant of the submatrix that remains when row i and column j are removed

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \quad \det\left(\begin{bmatrix} a_{22} & a_{23} \\ a_{32} & a_{33} \end{bmatrix}\right)$$

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \quad \det\left(\begin{bmatrix} a_{12} & a_{13} \\ a_{32} & a_{33} \end{bmatrix}\right)$$

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \quad \det\left(\begin{bmatrix} a_{12} & a_{13} \\ a_{22} & a_{23} \end{bmatrix}\right)$$

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \quad \det\left(\begin{bmatrix} a_{21} & a_{23} \\ a_{31} & a_{33} \end{bmatrix}\right)$$

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \quad \det\left(\begin{bmatrix} a_{11} & a_{13} \\ a_{31} & a_{33} \end{bmatrix}\right)$$

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \quad \det\left(\begin{bmatrix} a_{11} & a_{13} \\ a_{21} & a_{23} \end{bmatrix}\right)$$

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \quad \det\left(\begin{bmatrix} a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix}\right)$$

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \quad \det\left(\begin{bmatrix} a_{11} & a_{12} \\ a_{31} & a_{32} \end{bmatrix}\right)$$

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \quad \det\left(\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}\right)$$

2. Compute **Cofactor** C_{ij}

Multiply the minor by a sign depending on position

$$C_{ij} = (-1)^{i+j} M_{ij}$$

3. **Adjugate** (Adjoint)

Take the transpose:

$$\text{adj}(A) = C^T$$

Example

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 4 & 5 \\ 1 & 0 & 6 \end{bmatrix}$$

Step 1. Compute minors and cofactors

- **Entry (1, 1)**: remove row 1 and column 1

Minor:

$$\begin{bmatrix} 1 & 2 & 3 \\ 0 & 4 & 5 \\ 1 & 0 & 6 \end{bmatrix}$$

$$\det\left(\begin{bmatrix} 4 & 5 \\ 0 & 6 \end{bmatrix}\right) = 24$$

Cofactor:

$$\begin{aligned} C_{11} &= (-1)^{1+1} \cdot 24 \\ &= 1 \cdot 24 \\ &= 24 \end{aligned}$$

Cofactor matrix

$$C = \begin{bmatrix} 24 & & \\ & & \\ & & \end{bmatrix}$$

- **Entry (1, 2)**: remove row 1 and column 2

Minor:

$$\begin{bmatrix} \boxed{1} & \boxed{2} & \boxed{3} \\ 0 & \boxed{4} & 5 \\ 1 & \boxed{0} & 6 \end{bmatrix}$$

$$\det\left(\begin{bmatrix} 0 & 5 \\ 1 & 6 \end{bmatrix}\right) = -5$$

Cofactor:

$$\begin{aligned} C_{12} &= (-1)^{1+2} \cdot -5 \\ &= -1 \cdot -5 \\ &= 5 \end{aligned}$$

Cofactor matrix

$$C = \begin{bmatrix} 24 & 5 & \\ & & \\ & & \end{bmatrix}$$

- **Entry (1, 3)**: remove row 1 and column 3

Minor:

$$\begin{bmatrix} \boxed{1} & \boxed{2} & \boxed{3} \\ 0 & 4 & \boxed{5} \\ 1 & 0 & \boxed{6} \end{bmatrix}$$

$$\det\left(\begin{bmatrix} 0 & 4 \\ 1 & 0 \end{bmatrix}\right) = -4$$

Cofactor:

$$\begin{aligned} C_{13} &= (-1)^{1+3} \cdot -4 \\ &= 1 \cdot -4 \\ &= -4 \end{aligned}$$

Cofactor matrix

$$C = \begin{bmatrix} 24 & 5 & -4 \\ & & \\ & & \end{bmatrix}$$

- **Entry (2, 1)**: remove row 2 and column 1

Minor:

$$\begin{bmatrix} \color{#ffcccc}{1} & \color{#ffcccc}{2} & \color{#ffcccc}{3} \\ \color{#ffcccc}{0} & \color{#ffcccc}{4} & \color{#ffcccc}{5} \\ \color{#ffcccc}{1} & \color{#ffcccc}{0} & \color{#ffcccc}{6} \end{bmatrix}$$

$$\det\left(\begin{bmatrix} 2 & 3 \\ 0 & 6 \end{bmatrix}\right) = 12$$

Cofactor:

$$\begin{aligned} C_{21} &= (-1)^{2+1} \cdot 12 \\ &= -1 \cdot 12 \\ &= -12 \end{aligned}$$

Cofactor matrix

$$C = \begin{bmatrix} 24 & 5 & -4 \\ -12 & & \\ & & \end{bmatrix}$$

- **Entry (2, 2)**: remove row 2 and column 2

Minor:

$$\begin{bmatrix} 1 & \color{#ffcccc}{2} & 3 \\ \color{#ffcccc}{0} & \color{#ffcccc}{4} & \color{#ffcccc}{5} \\ 1 & \color{#ffcccc}{0} & 6 \end{bmatrix}$$

$$\det\left(\begin{bmatrix} 1 & 3 \\ 1 & 6 \end{bmatrix}\right) = 3$$

Cofactor:

$$\begin{aligned} C_{22} &= (-1)^{2+2} \cdot 3 \\ &= 1 \cdot 3 \\ &= 3 \end{aligned}$$

Cofactor matrix

$$C = \begin{bmatrix} 24 & 5 & -4 \\ -12 & 3 & \\ & & \end{bmatrix}$$

- **Entry (2, 3)**: remove row 2 and column 3

Minor:

$$\begin{bmatrix} 1 & 2 & 3 \\ 0 & 4 & 5 \\ 1 & 0 & 6 \end{bmatrix}$$

$$\det\left(\begin{bmatrix} 1 & 2 \\ 1 & 0 \end{bmatrix}\right) = -2$$

Cofactor:

$$\begin{aligned} C_{22} &= (-1)^{2+3} \cdot -2 \\ &= -1 \cdot -2 \\ &= 2 \end{aligned}$$

Cofactor matrix

$$C = \begin{bmatrix} 24 & 5 & -4 \\ -12 & 3 & 2 \\ & & \end{bmatrix}$$

- **Entry (3, 1)**: remove row 3 and column 1

Minor:

$$\begin{bmatrix} 1 & 2 & 3 \\ 0 & 4 & 5 \\ 1 & 0 & 6 \end{bmatrix}$$

$$\det\left(\begin{bmatrix} 2 & 3 \\ 4 & 5 \end{bmatrix}\right) = -2$$

Cofactor:

$$\begin{aligned} C_{31} &= (-1)^{3+1} \cdot -2 \\ &= 1 \cdot -2 \\ &= -2 \end{aligned}$$

Cofactor matrix

$$C = \begin{bmatrix} 24 & 5 & -4 \\ -12 & 3 & 2 \\ -2 & & \end{bmatrix}$$

- **Entry (3, 2)**: remove row 3 and column 2

Minor:

$$\begin{bmatrix} 1 & 2 & 3 \\ 0 & 4 & 5 \\ 1 & 0 & 6 \end{bmatrix}$$

$$\det\left(\begin{bmatrix} 1 & 3 \\ 0 & 5 \end{bmatrix}\right) = 5$$

Cofactor:

$$\begin{aligned} C_{32} &= (-1)^{3+2} \cdot -5 \\ &= -1 \cdot 5 \\ &= -5 \end{aligned}$$

Cofactor matrix

$$C = \begin{bmatrix} 24 & 5 & -4 \\ -12 & 3 & 2 \\ -2 & -5 & \end{bmatrix}$$

- **Entry (3, 3)**: remove row 3 and column 3

Minor:

$$\begin{bmatrix} 1 & 2 & 3 \\ 0 & 4 & 5 \\ 1 & 0 & 6 \end{bmatrix}$$

$$\det\left(\begin{bmatrix} 1 & 2 \\ 0 & 4 \end{bmatrix}\right) = 4$$

Cofactor:

$$\begin{aligned} C_{33} &= (-1)^{3+3} \cdot 4 \\ &= 1 \cdot 4 \\ &= 4 \end{aligned}$$

Cofactor matrix

$$C = \begin{bmatrix} 24 & 5 & -4 \\ -12 & 3 & 2 \\ -2 & -5 & 4 \end{bmatrix}$$

Step 2. Cofactor matrix

$$C = \begin{bmatrix} 24 & 5 & -4 \\ -12 & 3 & 2 \\ -2 & -5 & 4 \end{bmatrix}$$

Step 3. Transpose to get adjugate

$$\operatorname{adj}(A) = C^T = \begin{bmatrix} 24 & -12 & -2 \\ 5 & 3 & -5 \\ -4 & 2 & 4 \end{bmatrix}$$

Relation to **inverse**:

$$A^{-1} = \frac{1}{\det(A)} \operatorname{adj}(A) \qquad \text{if} \det(A) \neq 0$$

Relation to **identity**:

$$A \cdot \operatorname{adj}(A) = \operatorname{adj}(A) \cdot A = \det(A) I_n$$

86. Unimodularity

86.1. Unimodular Matrix

Integer square matrix whose determinant is +1 or −1:

$$\det(A) = \pm 1$$

- Always invertible, and its inverse is also an integer matrix
- Applying such a matrix (transformation) to an integer vector gives another integer vector

86.2. Totally Unimodular Matrix

A totally unimodular (TU) matrix is a (not necessarily square) matrix in which every square submatrix (determinant of any square submatrix of any size) has determinant in:

$$\{-1, 0, +1\}$$

$$A = \begin{bmatrix} 1 & 0 & -1 & 0 \\ 0 & -1 & 0 & 1 \\ -1 & 0 & 1 & 0 \end{bmatrix}$$

$$\det\left(\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}\right) = 0 \qquad \det\left(\begin{bmatrix} 0 & -1 & 0 \\ -1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}\right) = 0$$

87. Matrix Inverse

$$A \cdot A^{-1} = A^{-1} \cdot A = I$$

Where I is the identity matrix (like the number 1 in regular multiplication)

- For scalars: $2 \times \frac{1}{2} = 1$
- For matrices: $A \times A - 1 = I$

A matrix A is invertible (nonsingluar) if:

- It is square ($n \times n$)
- $\det(A) \neq 0$

87.1. Calculating inverse

Row Reduction (Gauss-Jordan Elimination)

Augment A with the identity matrix I, and perform row operations until the left side becomes I. The right side will then be A^{-1}

$$[A \mid I] \quad \longrightarrow \quad [I \mid A^{-1}]$$

Example

$$A = \begin{bmatrix} 2 & 1 \\ 5 & 3 \end{bmatrix}$$

Step 1. Set up augmented matrix

$$[A \mid I] = \left[\begin{array}{cc|cc} 2 & 1 & 1 & 0 \\ 5 & 3 & 0 & 1 \end{array}\right]$$

Step 2. Make the pivot of the first column equal to 1

$$\left(R_1 \to \frac{1}{2} R_1\right)$$

$$\left[\begin{array}{cc|cc} 1 & 0.5 & 0.5 & 0 \\ 5 & 3 & 0 & 1 \end{array}\right]$$

Step 3: Eliminate the first column in row 2

$$R_2 \to R_2 - 5R_1$$

$$\left[\begin{array}{cc|cc} 1 & 0.5 & 0.5 & 0 \\ 0 & 0.5 & -2.5 & 1 \end{array}\right]$$

Step 4: Make the pivot in row 2 a 1

$$R_2 \to 2R_2$$

$$\left[\begin{array}{cc|cc} 1 & 0.5 & 0.5 & 0 \\ 0 & 1 & -5 & 2 \end{array}\right]$$

Step 5: Eliminate the 0.5 above the pivot

$$R_1 \to R_1 - 0.5R_2$$

$$\left[\begin{array}{cc|cc} 1 & 0 & 3 & -1 \\ 0 & 1 & -5 & 2 \end{array}\right]$$

So the inverse is:

$$A^{-1} = \begin{bmatrix} 3 & -1 \\ -5 & 2 \end{bmatrix}$$

88. LU Decomposition

Given a matrix A, LU decomposition aims to express A as:

$$A = LU$$

Where:

- L: Lower triangular matrix (all elements above the diagonal are zero)
- U: Upper triangular matrix (all elements below the diagonal are zero)

1. Solve $Ly = b$ Using Forward Substitution

$$Ly = b$$

Where:

- L: Lower triangular matrix (all elements above the diagonal are zero)
- y: Intermediate vector we are solving for
- b: Right-hand side vector

$$\begin{bmatrix} L_{11} & 0 & 0 \\ L_{21} & L_{22} & 0 \\ L_{31} & L_{32} & L_{33} \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

- First row: $L_{11}y_1 = b_1$, so $y_1 = \frac{b_1}{L_{11}}$
- Second row: $L_{21}y_1 + L_{22}y_2 = b_2$, substitute y_1 into this equation and solve for y_2:

$$y_2 = \frac{b_2 - L_{21}y_1}{L_{22}}$$

- Third row: $L_{31}y_1 + L_{32}y_2 + L_{33}y_2 = b_3$, substitute y_1 and y_2 into this equation, solve for y_3:

$$y_3 = \frac{b_2 - L_{31}y_1 - L_{32}y_2}{L_{33}}$$

2. Solve $Ux = y$ Using Backward Substitution

$$Ux = y$$

Where:

- U: Upper triangular matrix (all elements below the diagonal are zero)
- y: Vector of unknowns (solution)
- b: Vector computed from the forward substitution step

$$\begin{bmatrix} U_{11} & U_{12} & U_{13} \\ 0 & U_{22} & U_{23} \\ 0 & 0 & U_{33} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix}$$

- Third row: $U_{33}x_3 = y_3$, so $x_3 = \frac{y_3}{U_{33}}$
- Second row: $U_{22}x_2 + U_{23}x_3 = y_2$, substitute x_3 from the previous step and solve for x_2:

$$x_2 = \frac{y_2 - U_{23}x_3}{U_{22}}$$

- First row: $U_{11}x_1 + U_{12}x_2 + U_{13}x_3 = y_1$, substitute x_2 and x_3 from the previous step and solve for x_1:

$$x_1 = \frac{y_1 - U_{12}x_2 - U_{13}x_3}{U_{11}}$$

Example

$$A = \begin{bmatrix} 2 & 3 & 1 \\ 4 & 7 & 3 \\ 6 & 18 & 5 \end{bmatrix} \qquad b = \begin{bmatrix} 5 \\ 12 \\ 31 \end{bmatrix}$$

1. Factor A into L and U:

$$L = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 3 & 6 & 1 \end{bmatrix} \qquad U = \begin{bmatrix} 2 & 3 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & -2 \end{bmatrix}$$

2. Solve $Ax = b$
   - $y_1 = \frac{b_1}{L_{11}} = \frac{5}{1} = 5$
   - $y_2 = \frac{b_2 - L_{21}y_1}{L_{22}} = \frac{12 - 2\times5}{1} = 2$
   - $y_2 = \frac{b_3 - L_{31}y_1 - L_{32}y_2}{L_{33}} = \frac{31 - 4\times5 - 3\times2}{1} = 5$

So,

$$y = \begin{bmatrix} 5 \\ 2 \\ 5 \end{bmatrix}$$

3. Solve $Ux = y$
   - $x_3 = \frac{y_3}{U_{33}} = \frac{5}{-}2 = -2.5$
   - $x_2 = \frac{y_2 - U_{23}x_3}{U_{22}} = \frac{2 - 1 \times -2.5}{1} = 4.5$
   - $x_1 = \frac{y_1 - U_{12}x_2 - U_{13}x_3}{U_{11}} = \frac{5 - 3 \times 4.5 - 1 \times -2.5}{2} = -4$

So,

$$x = \begin{bmatrix} -4 \\ 4.5 \\ -2.5 \end{bmatrix}$$

89. QR Decomposition

Every $m \times n$ matrix A with $m \geq n$ can be factored as:

$$A = QR$$

where:

- Q is $m \times n$ with **orthonormal columns** (so $Q^TQ = I_n$)
- R is $n \times n$ **upper triangular**

If A is full column rank, the decomposition is unique up to signs on the diagonal of R.

For square A (full rank), Q is a full orthogonal matrix and R is square upper triangular.

89.1. Construction via Gram–Schmidt

The columns of Q are the result of applying Gram–Schmidt orthogonalization to the columns of A:

- $a_1, a_2, ..., a_n$ are the columns of A
- $q_1, q_2, ..., q_n$ are orthonormalized versions
- R records the coefficients of each a_j in terms of $q_1, ..., q_j$

Explicitly, $R_{ij} = q_i^T a_j$ for $i \leq j$ and 0 for $i > j$.

Example

$$A = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{bmatrix}$$

Apply Gram–Schmidt to columns:

- $q_1 = \frac{a_1}{\|} a_1 \| = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}$
- Project a_2 off q_1: $a_2 - (q_1^T a_2) q_1 = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix} - \frac{1}{\sqrt{2}} \cdot \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} -\frac{1}{2} \\ \frac{1}{2} \\ 1 \end{bmatrix}$
- $q_2 =$ that, normalized: $\frac{1}{\sqrt{6}} \begin{bmatrix} -1 \\ 1 \\ 2 \end{bmatrix}$

$$Q = \begin{bmatrix} \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{6}} \\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{6}} \\ 0 & \frac{2}{\sqrt{6}} \end{bmatrix}$$

$$R = Q^T A = \begin{bmatrix} \sqrt{2} & \frac{1}{\sqrt{2}} \\ 0 & \sqrt{\frac{3}{2}} \end{bmatrix}$$

89.2. Why it matters

QR is the workhorse for **least-squares problems**:

Given $A\vec{x} = \vec{b}$ overdetermined (more equations than unknowns), the least-squares solution

$$\hat{\vec{x}} = \arg\min_{\vec{x}} \|A\vec{x} - \vec{b}\|^2$$

solves the **normal equations** $A^T A\vec{x} = A^T\vec{b}$. Using $A = QR$:

$$R\vec{x} = Q^T\vec{b}$$

Then back-substitute on the upper-triangular R — much more numerically stable than forming $A^T A$ directly.

Other uses:

- **Eigenvalue** computation via the QR algorithm — iteratively factor $A_k = Q_k R_k$ and update $A_{k+1} = R_k Q_k$; converges to a triangular matrix whose diagonal gives the eigenvalues
- **Orthonormalizing a basis** numerically (Gram–Schmidt is unstable in floating point; Householder QR is much more reliable)

- **Solving square systems** as an alternative to LU

89.3. Computation methods

- **Classical Gram–Schmidt** — conceptually clean but numerically unstable
- **Modified Gram–Schmidt** — small variation, much more stable
- **Householder reflections** — uses orthogonal reflections, very stable, $O(mn^2)$
- **Givens rotations** — useful for sparse / structured matrices

89.4. See also

- **Gram–Schmidt Process** — the orthogonalization underlying QR
- **Orthogonal Matrix**
- **Triangular Matrix**
- **LU** / **Cholesky** / **SVD**

90. Cholesky Decomposition

Every **symmetric positive-definite** matrix A has a unique factorization:

$$A = LL^T$$

where L is **lower triangular** with positive diagonal entries.

The Cholesky decomposition is the **symmetric square root** of a positive-definite matrix — like writing a positive number a as $\sqrt{a} \cdot \sqrt{a}$.

90.1. Existence and uniqueness

Cholesky exists iff A is:
1. **Symmetric**: $A^T = A$
2. **Positive definite**: $\vec{x}^T A\vec{x} > 0$ for all $\vec{x} \neq \mathbf{0}$

If either fails, Cholesky fails. For positive **semi-definite** A a related decomposition exists but is non-unique.

For symmetric **indefinite** A, use the LDL^T variant (D diagonal with arbitrary signs).

90.2. Algorithm (forward construction)

Compute L column by column, $j = 1, ..., n$:

$$L_{jj} = \sqrt{A_{jj} - \sum_{k=1}^{j-1} L_{jk}^2}$$

$$L_{ij} = \frac{1}{L_{jj}} \left(A_{ij} - \sum_{k=1}^{j-1} L_{ik} L_{jk} \right), \text{ for } i > j$$

Cost: about $\frac{n^3}{3}$ — **half** of LU decomposition (we exploit symmetry).

Example

$$A = \begin{bmatrix} 4 & 12 & -16 \\ 12 & 37 & -43 \\ -16 & -43 & 98 \end{bmatrix}$$

$$L = \begin{bmatrix} 2 & 0 & 0 \\ 6 & 1 & 0 \\ -8 & 5 & 3 \end{bmatrix}$$

Verify: $LL^T = A$. ✓

90.3. Why use Cholesky

1. **Solving $A\vec{x} = \vec{b}$**: split into $L\vec{y} = \vec{b}$ (forward substitution) then $L^T\vec{x} = \vec{y}$ (backward substitution). Twice as fast as LU for symmetric positive-definite A.
2. **Testing positive-definiteness**: an attempt to compute Cholesky fails (negative argument to `sqrt`, or division by zero) iff A is not positive definite. Cheap test.
3. **Sampling from multivariate Gaussian**: if $\Sigma = LL^T$ is the covariance matrix and $\vec{z} \sim \mathcal{N}(\mathbf{0}, I)$ (standard normal), then $\vec{x} = L\vec{z}$ is $\mathcal{N}(\mathbf{0}, \Sigma)$.
4. **Linear least squares** (alternative to QR): the normal equations $A^TA\vec{x} = A^T\vec{b}$ have a symmetric positive-definite system matrix A^TA; Cholesky solves it.
5. **Determinant**: $\det(A) = \prod L_{ii}^2$ — much cheaper than expanding directly.

90.4. Relation to LU and SVD

- LU: $A = LU$ — general matrices, L lower-triangular, U upper-triangular
- Cholesky: $A = LL^T$ — symmetric positive-definite case, $U = L^T$
- SVD: $A = U\Sigma V^T$ — completely general, gives singular values, more expensive

For symmetric positive-definite A, Cholesky is the fastest and most stable choice.

90.5. See also

- **Symmetric Matrix**
- **Quadratic Form** — positive-definite condition
- **Triangular Matrix**
- **LU** / **QR** / **SVD**

91. SVD

Every real matrix A — square or rectangular, full-rank or rank-deficient — has a **Singular Value Decomposition**:

$$A = U\Sigma V^T$$

where, for an $m \times n$ matrix A:

- U is $m \times m$ orthogonal — columns are the **left singular vectors**
- Σ is $m \times n$ diagonal with non-negative entries $\sigma_1 \geq \sigma_2 \geq \ldots \geq 0$ — the **singular values**
- V is $n \times n$ orthogonal — columns are the **right singular vectors**

91.1. Singular values

The singular values are the **square roots of the eigenvalues** of $A^T A$ (which is symmetric positive semi-definite):

$$\sigma_i = \sqrt{\lambda_i(A^T A)}$$

Number of non-zero singular values = rank of A.

91.2. Geometric interpretation

Every linear transformation $A : \mathbb{R}^n \to \mathbb{R}^m$ decomposes into:

1. **Rotate / reflect** in the input space (multiply by V^T)
2. **Stretch** along the new axes by factors $\sigma_1, \sigma_2, \ldots$ (multiply by Σ)
3. **Rotate / reflect** in the output space (multiply by U)

So every matrix is a **rotation–stretch–rotation**. The singular values measure how much A stretches each direction.

Example

$$A = \begin{bmatrix} 3 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}$$

Already in SVD form: $U = I_3$, $\Sigma = A$, $V = I_2$. Singular values: $\sigma_1 = 3, \sigma_2 = 1$.

A maps the unit circle in $\mathbb{R}^2$ to an ellipse in $\mathbb{R}^3$ with semi-axes 3 and 1.

91.3. Compact / reduced SVD

For rank-r matrix A, drop the zero singular values:

$$A = U_r \Sigma_r V_r^T$$

where U_r is $m \times r$, Σ_r is $r \times r$ diagonal, V_r is $n \times r$. Same product, less storage.

91.4. Best low-rank approximation

The truncated SVD (keeping the top k singular values) is the **best rank-k approximation** of A in both Frobenius and spectral norms:

$$A_k = \sum_{i=1}^{k} \sigma_i u_i v_i^T$$

This is the engine behind:

- **Image / signal compression**
- **Recommender systems** / matrix factorization
- **Noise reduction** — drop tiny singular values

91.5. Connection to symmetric matrices

If A is symmetric positive semi-definite, the SVD coincides with the eigendecomposition (Spectral Theorem): $U = V$ and Σ = eigenvalues.

For general A, $A^T A$ and AA^T are both symmetric:

- **Right singular vectors** of A = eigenvectors of $A^T A$ (with eigenvalues σ_i^2)
- **Left singular vectors** of A = eigenvectors of AA^T (with same eigenvalues σ_i^2)

91.6. Connection to pseudoinverse

When A is not square or not invertible, the **Moore–Penrose** pseudoinverse uses the SVD:

$$A^+ = V\Sigma^+ U^T$$

where Σ^+ inverts the non-zero singular values and transposes the shape.

91.7. See also

- **Spectral Theorem** — symmetric special case
- **Pseudoinverse**
- **QR** / **Cholesky** / **LU** — other matrix factorizations
- **Orthogonal Matrix**

92. Pseudoinverse

The **Moore–Penrose pseudoinverse** A^+ generalizes the matrix inverse to **every** matrix — square, rectangular, full-rank, or rank-deficient.

For invertible square A, $A^+ = A^{-1}$. For other matrices, A^+ provides the “best possible inverse” in a precise least-squares sense.

92.1. Definition via SVD

If $A = U\Sigma V^T$ is the SVD of A, then:

$$A^+ = V\Sigma^+ U^T$$

where Σ^+ is the **transpose** of Σ with each non-zero singular value σ_i replaced by $\frac{1}{\sigma_i}$ (zero singular values stay zero).

92.2. Four defining properties

A^+ is uniquely characterized by the **Moore–Penrose conditions**:

1. $AA^+A = A$
2. $A^+AA^+ = A^+$
3. $(AA^+)^T = AA^+$ — symmetric
4. $(A^+A)^T = A^+A$ — symmetric

These force a unique A^+ for every matrix.

92.3. Special cases

Full column rank ($m \geq n$, $\text{rank}(A) = n$):

$$A^+ = (A^TA)^{-1}A^T$$

Full row rank ($m \leq n$, $\text{rank}(A) = m$):

$$A^+ = A^T(AA^T)^{-1}$$

Square invertible: $A^+ = A^{-1}$.

92.4. Least-squares solution

For the overdetermined system $A\vec{x} = \vec{b}$ (more equations than unknowns), the **minimum-norm least-squares solution** is:

$$\hat{\vec{x}} = A^+\vec{b}$$

Properties:

- If a solution to $A\vec{x} = \vec{b}$ exists, $\hat{\vec{x}}$ is one (the one with smallest norm)
- If no exact solution exists, $\hat{\vec{x}}$ minimizes $\|A\vec{x} - \vec{b}\|$

This is the foundation of linear regression and least-squares fitting.

Example

$$A = \begin{bmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix} \qquad \vec{b} = \begin{bmatrix} 3 \\ 4 \\ 5 \end{bmatrix}$$

No exact solution (rows 2 and 3 are inconsistent with non-zero b entries).

SVD: $\sigma_1 = 1$, $\sigma_2 = 0$. So $\Sigma^+ = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$ (transpose of Σ with σ_2 left zero).

$$A^+ = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \qquad \hat{\vec{x}} = A^+\vec{b} = \begin{bmatrix} 3 \\ 0 \end{bmatrix}$$

This minimizes $\|A\vec{x} - \vec{b}\|$ and has minimum norm among minimizers.

92.5. Why it generalizes the inverse

- Replaces “exact solution” with “best least-squares approximation”
- Handles rank-deficient systems (where standard inverse is undefined)
- Handles rectangular systems (where standard inverse is undefined)
- Reduces to ordinary inverse exactly when ordinary inverse exists

92.6. See also

- **SVD** — the construction
- **Matrix Inverse** — special case
- **QR** — used to compute the pseudoinverse in the full-rank case
- **Linear Regression** — pseudoinverse gives OLS coefficients $\hat{\beta} = (X^T X)^{-1} X^T y = X^+ y$

93. Eigenvectors & Eigenvalues

93.1. Eigen

$$Ax = \lambda x$$

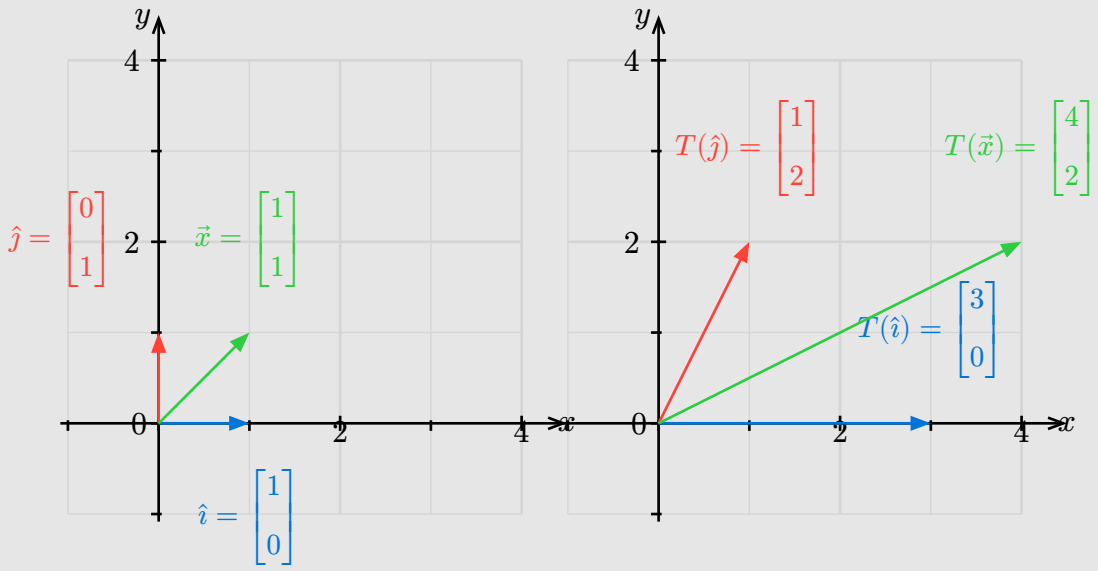
When a transformation A is applied to a vector $\mathbf{x}$, it is equivalent to scaling the vector by a factor of λ. The vector $\mathbf{x}$ is called an **eigenvector** of the transformation A, and the scalar λ is called the corresponding **eigenvalue**.

Example

Transformation

Cectors $\hat{\imath}, \hat{\jmath}, \vec{x}$ under transformation A

$$A = \begin{bmatrix} 3 & 1 \\ 0 & 2 \end{bmatrix}$$



$$T(\hat{\imath}) = A\hat{\imath} = \begin{bmatrix} 3 & 1 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 3 \\ 0 \end{bmatrix}$$

$$T(\hat{j}) = A\hat{j} = \begin{bmatrix} 3 & 1 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

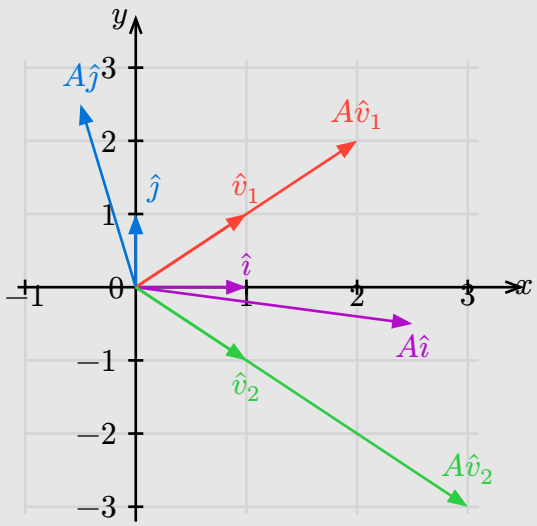
$$T(\vec{x}) = A\vec{x} = \begin{bmatrix} 3 & 1 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 2 \end{bmatrix}$$

Example

Transformation

Vectors $\hat{\imath}, \hat{\jmath}, \hat{v}_1, \hat{v}_2$ under transformation A

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$



An $n \times n$ matrix A has exactly n eigenvalues because the eigenvalues are the **roots** of its characteristic polynomial, which is degree n

Characteristic polynomial:

$$p(\lambda) = \det(A - \lambda I)$$

Example

2×2 Matrix

$$A = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$$

$$\det(A - \lambda I) = (2 - \lambda)(3 - \lambda) = 0$$

Solve for λ:

$$\lambda_1 = 2, \lambda_2 = 3$$

3×3 Matrix

$$A = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 4 \end{bmatrix}$$

$$\det(A - \lambda I) = (1 - \lambda)(2 - \lambda)(4 - \lambda) = 0$$

Solve for λ:

$$\lambda_1 = 1, \lambda_2 = 2, \lambda_3 = 4$$

94. Characteristic Polynomial

For an $n \times n$ matrix A, the **characteristic polynomial** is:

$$p_{A(\lambda)} = \det(A - \lambda I)$$

where I is the $n \times n$ identity matrix.

$p_{A(\lambda)}$ is a polynomial in λ of degree n. Its **roots** are the eigenvalues of A.

94.1. Why the roots are the eigenvalues

By definition, λ is an eigenvalue of A iff there exists a non-zero $\vec{v}$ with $A\vec{v} = \lambda\vec{v}$, i.e. $(A - \lambda I)\vec{v} = \mathbf{0}$. A non-trivial solution exists iff $A - \lambda I$ is singular iff $\det(A - \lambda I) = 0$. So the eigenvalues are exactly the roots of p_A.

Example

$$A = \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix}$$

$$A - \lambda I = \begin{bmatrix} 2 - \lambda & 1 \\ 0 & 3 - \lambda \end{bmatrix}$$

$$p_A(\lambda) = (2 - \lambda)(3 - \lambda) - 0 = \lambda^2 - 5\lambda + 6$$

Roots: $\lambda^2 - 5\lambda + 6 = 0 \;\Rightarrow\; \lambda \in \{2, 3\}$.

The eigenvalues of A are 2 and 3 (the diagonal entries — a feature of triangular matrices).

94.2. Structure of the polynomial

For A an $n \times n$ matrix, $p_{A(\lambda)} = (-1)^n\lambda^n + \ldots$ with:

- **Leading coefficient**: $(-1)^n$ (the sign depends on n; some conventions use $\lambda^n - \operatorname{tr}(A)\lambda^{n-1} + \ldots$)
- **Coefficient of λ^{n-1}**: $(-1)^{n-1}\ \operatorname{tr}(A)$ — gives the trace
- **Constant term**: $\det(A)$ (set $\lambda = 0$)

For 2×2 matrices:

$$p_{A(\lambda)} = \lambda^2 - \operatorname{tr}(A)\lambda + \det(A)$$

94.3. Multiplicity

- **Algebraic multiplicity** of λ_i: its multiplicity as a root of p_A.
- **Geometric multiplicity** of λ_i: $\dim(\ker(A - \lambda_i I))$ — the number of linearly independent eigenvectors for λ_i.

Always: geometric multiplicity $\leq$ algebraic multiplicity. Equality everywhere is required for the matrix to be diagonalizable.

94.4. Cayley–Hamilton

Every square matrix satisfies its own characteristic polynomial:

$$p_{A(A)} = 0$$

(substituting the matrix A for λ — and interpreting the constant term as a multiple of I). This lets you express A^k for any $k \geq n$ in terms of lower powers $I, A, A^2, ..., A^{n-1}$.

94.5. See also

- **Eigenvectors & Eigenvalues**
- **Determinant**
- **Diagonalization**
- **Trace**

95. Diagonalization

A square matrix A is **diagonalizable** if it can be written as

$$A = PDP^{-1}$$

where D is a diagonal matrix and P is an invertible matrix.

The diagonal entries of D are the eigenvalues of A, and the columns of P are the corresponding eigenvectors.

95.1. When is A diagonalizable?

An $n \times n$ matrix is diagonalizable iff it has n linearly independent eigenvectors.

Equivalent conditions:

- A has n distinct eigenvalues $\rightarrow$ automatically diagonalizable
- Every eigenvalue's **geometric multiplicity** equals its **algebraic multiplicity** (see Characteristic Polynomial)
- $\mathbb{R}^n$ has a basis consisting of eigenvectors of A (an **eigenbasis**)

Some matrices are **not** diagonalizable — e.g., $\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$ has only one eigenvalue (1) of algebraic multiplicity 2 but geometric multiplicity 1. For non-diagonalizable matrices, see Jordan Canonical Form.

95.2. How to diagonalize

1. Find the eigenvalues $\lambda_1, ..., \lambda_n$ from $\det(A - \lambda I) = 0$.
2. For each λ_i, find a basis for $\ker(A - \lambda_i I)$ — the eigenvectors.
3. Assemble eigenvectors into columns of P, eigenvalues into the diagonal of D (in matching order).

4. Verify by computing PDP^{-1}.

Example

$A = \begin{bmatrix} 4 & -2 \\ 1 & 1 \end{bmatrix}$.

Characteristic polynomial: $\det(A - \lambda I) = (4-\lambda)(1-\lambda) - (-2)(1) = \lambda^2 - 5\lambda + 6 = (\lambda - 2)(\lambda - 3)$.

Eigenvalues: $\lambda_1 = 2, \lambda_2 = 3$.

Eigenvectors: solve $(A - 2I)\vec{v} = \mathbf{0} \to \vec{v}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$. And $(A - 3I)\vec{v} = \mathbf{0} \to \vec{v}_2 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$.

$$P = \begin{bmatrix} 1 & 2 \\ 1 & 1 \end{bmatrix} \qquad D = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$$

95.3. Why it matters: power and exponential

In diagonal form, matrix arithmetic is trivial:

$$A^k = (PDP^{-1})^k = PD^kP^{-1}$$

with D^k just powering each diagonal entry. This makes:

- A^k for large k tractable (Markov chains, dynamics)
- **Matrix exponential** $e^A = Pe^DP^{-1}$ (linear ODE solutions)
- Solving linear recurrences in closed form

95.4. Spectral decomposition (real symmetric case)

If A is real and symmetric, then it's **always** diagonalizable and can be done with an orthogonal P:

$$A = QDQ^T, \text{ where } Q^TQ = I$$

This is the Spectral Theorem — the gold standard of diagonalization.

95.5. Failure: non-diagonalizable matrices

$$A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$$

has only the eigenvalue 1 (with algebraic multiplicity 2) and only one independent eigenvector $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$. Not diagonalizable — the best you can do is the Jordan form.

95.6. See also

- **Eigenvectors & Eigenvalues**

- **Characteristic Polynomial**
- **Spectral Theorem** — symmetric case
- **Jordan Canonical Form** — non-diagonalizable case
- **Change of Basis** — what P does geometrically

96. Spectral Theorem

The **Spectral Theorem** says that every real symmetric matrix is **orthogonally diagonalizable**:

$$A = QDQ^T$$

where:

- D is a diagonal matrix whose entries are the (always **real**) eigenvalues of A
- Q is an orthogonal matrix whose columns are an **orthonormal** basis of eigenvectors of A

Equivalently, $Q^TQ = I$, and the columns of Q are orthonormal eigenvectors.

96.1. What it guarantees

For a real symmetric matrix A:

1. **All eigenvalues are real** (no complex eigenvalues, even if A's entries don't suggest it)
2. **Eigenvectors for distinct eigenvalues are orthogonal** (automatic)
3. **Eigenvectors for repeated eigenvalues can be chosen orthonormal** (within that eigenspace, via Gram–Schmidt)
4. The full set of n orthonormal eigenvectors forms a basis of $\mathbb{R}^n$

The trio (1)+(2)+(3) is what makes the theorem work. No real symmetric matrix is missing eigenvectors.

96.2. Spectral expansion

Write $Q = [q_1 \quad q_2 \quad \cdots \quad q_n]$ and $D = \text{diag}(\lambda_1, ..., \lambda_n)$. Then:

$$A = \sum_{i=1}^{n} \lambda_i \, q_i q_i^T$$

The matrix decomposes into a sum of rank-1 **spectral projectors** $q_i q_i^T$, each scaled by an eigenvalue. This is the **spectral decomposition**.

Example

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

Eigenvalues: $\lambda_1 = 3, \lambda_2 = 1$.

Orthonormal eigenvectors: $q_1 = \frac{1}{\sqrt{2}}\begin{bmatrix}1\\1\end{bmatrix}$, $q_2 = \frac{1}{\sqrt{2}}\begin{bmatrix}1\\-1\end{bmatrix}$.

$$Q = \frac{1}{\sqrt{2}}\begin{bmatrix}1 & 1\\1 & -1\end{bmatrix} \qquad D = \begin{bmatrix}3 & 0\\0 & 1\end{bmatrix} \qquad A = QDQ^T$$

Spectral expansion: $A = 3q_1q_1^T + 1q_2q_2^T = \frac{3}{2}\begin{bmatrix}1 & 1\\1 & 1\end{bmatrix} + \frac{1}{2}\begin{bmatrix}1 & -1\\-1 & 1\end{bmatrix}$.

96.3. Generalizations

- **Complex Hermitian matrices** ($A^* = A$, where A^* is the conjugate transpose): same theorem, with Q replaced by a **unitary** matrix U ($U^*U = I$).
- **Normal matrices** ($A^*A = AA^*$): same theorem in $\mathbb{C}^n$ — unitarily diagonalizable.

96.4. Why it matters

- **Quadratic forms**: $\vec{x}^T A\vec{x}$ simplifies to $\sum \lambda_i y_i^2$ in the eigenbasis $y = Q^T x$. Definiteness is read off the signs of eigenvalues.
- **PCA / dimensionality reduction**: the covariance matrix is symmetric → orthogonal eigenbasis = principal components.
- **Optimization**: the Hessian is symmetric → its spectrum classifies critical points.
- **Quantum mechanics**: observables are Hermitian operators; their eigenvalues are the possible measured values.
- **SVD** extends the idea to non-square / non-symmetric matrices.

96.5. See also

- **Symmetric Matrix**
- **Orthogonal Matrix**
- **Diagonalization**
- **Eigenvectors & Eigenvalues**
- **SVD**

97. Jordan Canonical Form

When a matrix is **not** diagonalizable (geometric multiplicity $<$ algebraic multiplicity for some eigenvalue), the Jordan canonical form is the next-best thing — **almost** diagonal, with 1's just above the diagonal in certain blocks.

Every square matrix A over $\mathbb{C}$ (or over $\mathbb{R}$ if all eigenvalues are real) is **similar** to its Jordan form J:

$$A = PJP^{-1}$$

where J is **block diagonal**:

$$J = \begin{bmatrix} J_{\lambda_1,k_1} & 0 & \dots & 0 \\ 0 & J_{\lambda_2,k_2} & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & J_{\lambda_r,k_r} \end{bmatrix}$$

97.1. Jordan blocks

A **Jordan block** $J_{\lambda,k}$ is a $k \times k$ upper triangular matrix with λ on the diagonal and 1 on the superdiagonal:

$$J_{\lambda,1} = [\lambda]$$

$$J_{\lambda,2} = \begin{bmatrix} \lambda & 1 \\ 0 & \lambda \end{bmatrix}$$

$$J_{\lambda,3} = \begin{bmatrix} \lambda & 1 & 0 \\ 0 & \lambda & 1 \\ 0 & 0 & \lambda \end{bmatrix}$$

If A has k linearly independent eigenvectors for eigenvalue λ (geometric multiplicity k) and total multiplicity m (algebraic multiplicity), the corresponding Jordan structure has k blocks for λ, whose sizes sum to m.

97.2. Diagonalizable as a special case

If A is diagonalizable, every Jordan block is 1×1 — i.e., $J = D$ is just diagonal. The Jordan form generalizes diagonalization to handle “missing eigenvectors.”

Example

$$A = \begin{bmatrix} 5 & 1 \\ 0 & 5 \end{bmatrix}$$

Eigenvalue $\lambda = 5$ with algebraic multiplicity 2, geometric multiplicity 1 (only eigenvector $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$). Not diagonalizable.

Jordan form: $J = \begin{bmatrix} 5 & 1 \\ 0 & 5 \end{bmatrix} = A$ (already in Jordan form).

Example

$$A = \begin{bmatrix} 2 & 1 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{bmatrix}$$

Eigenvalues: 2 (algebraic mult 2, geometric mult 1) and 3 (mult 1).

Jordan form has one 2×2 block for $\lambda = 2$ and one 1×1 block for $\lambda = 3$:

$$J = \begin{bmatrix} 2 & 1 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{bmatrix} = A$$

97.3. Why it matters

- **Matrix exponential** $e^A = Pe^J P^{-1}$, and e^J on Jordan blocks has a known closed form (involving polynomial factors t^k alongside $e^{\lambda t}$ in the time evolution of linear ODEs)
- **Solving linear systems of ODEs** $\vec{x}'(t) = A\vec{x}(t)$ with repeated eigenvalues produces polynomial-times-exponential terms — directly traceable to non-trivial Jordan blocks
- **Generalized eigenvectors** — the "missing" basis vectors live in $\ker\big((A - \lambda I)^k\big)$ for $k > 1$

97.4. Computation note

Computing the Jordan form numerically is **unstable** — small perturbations can turn a non-trivial Jordan block into a tight cluster of distinct eigenvalues. For numerical work, prefer the SVD or the Schur form (always available, real or complex, and stable).

97.5. See also

- **Diagonalization**
- **Eigenvectors & Eigenvalues**
- **Characteristic Polynomial** — algebraic multiplicity
- **Spectral Theorem** — when Jordan blocks are all 1×1 (symmetric / Hermitian case)

98. Quadratic Form

A **quadratic form** is a polynomial expression in n variables where every term has total degree exactly 2:

$$Q(\vec{x}) = \sum_{i,j} a_{ij} x_i x_j$$

It can always be written compactly using a symmetric matrix A:

$$Q(\vec{x}) = \vec{x}^T A \vec{x}$$

The matrix A is assumed symmetric without loss of generality — if B isn't symmetric, replacing it with $\frac{B+B^T}{2}$ gives the same quadratic form.

Example

$$Q(x, y) = 3x^2 + 5xy + 2y^2$$

As a matrix product (split the $5xy$ symmetrically as $\frac{5}{2}xy + \frac{5}{2}yx$):

$$A = \begin{bmatrix} 3 & \frac{5}{2} \\ \frac{5}{2} & 2 \end{bmatrix}, \quad Q(\vec{x}) = \vec{x}^T A \vec{x}$$

98.1. Definiteness

Quadratic forms are classified by the sign of $Q(\vec{x})$ over all non-zero $\vec{x}$:

Type	Sign of Q	All eigenvalues of A
Positive definite	> 0 always	> 0
Positive semi-definite	≥ 0 always	≥ 0
Negative definite	< 0 always	< 0
Negative semi-definite	≤ 0 always	≤ 0
Indefinite	takes both signs	mixed

The eigenvalue characterization comes from the Spectral Theorem: diagonalizing $A = QDQ^T$ and changing variables $\vec{y} = Q^T \vec{x}$ gives

$$Q(\vec{x}) = \sum_{i=1}^{n} \lambda_i y_i^2$$

— a sum of squares weighted by eigenvalues. The signs of the eigenvalues determine the signs of Q.

98.2. Tests for positive definiteness

For a symmetric A, equivalent conditions:

1. All eigenvalues > 0
2. All leading principal minors > 0 (**Sylvester's criterion**)
3. $A = LL^T$ admits a Cholesky decomposition
4. $\vec{x}^T A \vec{x} > 0$ for all $\vec{x} \neq \mathbf{0}$

98.3. Geometric pictures (2D)

For $Q(x, y) = \vec{x}^T A \vec{x}$, the level sets $Q(\vec{x}) = c$ are conic sections aligned with the eigenvectors of A:

- **Positive definite**: ellipse — closed, bounded, axes are eigenvectors with lengths $\frac{1}{\sqrt{\lambda_i}}$
- **Negative definite**: ellipse (with negative c)
- **Indefinite**: hyperbola — open curves with asymptotes
- **Positive semi-definite (rank 1)**: parallel lines

98.4. Where they show up

- **Semi-definiteness / Hessian**: the second-order Taylor approximation of a function near a critical point is a quadratic form. Definiteness classifies the critical point as min / max / saddle.
- **Norms**: $\|\vec{x}\|_2^2 = \vec{x}^T\vec{x} = \vec{x}^T I\vec{x}$ — a quadratic form with $A = I$
- **Mahalanobis distance**: $(\vec{x} - \vec{\mu})^T\Sigma^{-1}(\vec{x} - \vec{\mu})$ — a quadratic form using the inverse covariance
- **Multivariate Gaussian density**: $\exp\left(-\frac{1}{2}\,\vec{x}^T\Sigma^{-1}\vec{x}\right)$
- **Quadratic programming**: optimize $\frac{1}{2}\vec{x}^TQ\vec{x} + c^T\vec{x}$ subject to linear constraints
- **Kinetic energy / potential energy** in mechanics
- **Variance** of a linear combination: $\mathrm{Var}(a^TX) = a^T\ \mathrm{Cov}(X)a$

98.5. Diagonalization (principal axis theorem)

Every quadratic form can be brought to **diagonal form** by an orthogonal change of variables — equivalently, by rotating the coordinate axes to align with the eigenvectors of A:

$$\vec{y} = Q^T\vec{x} \qquad \Rightarrow \qquad Q(\vec{x}) = \lambda_1 y_1^2 + \lambda_2 y_2^2 + ... + \lambda_n y_n^2$$

This is the “principal axes” of the conic / quadric.

98.6. See also

- **Symmetric Matrix**
- **Spectral Theorem**
- **Leading Principal Minor** — Sylvester’s criterion
- **Cholesky Decomposition**
- **Semi-Definiteness** in Calculus II

99. Real Analysis

100. Sets

Definition: Set

A set is a collection of objects called elements or numbers

Definition: Empty Set $\varnothing$

Set with no elements

Notation

- $a \in S$: a is an element of S
- $a \notin S$: a is not an element of S
- $\forall$: for all
- $\exists$: there exists
- $\Rightarrow$: implies
- $\Leftrightarrow$: if and only if

Definition:

1. if $A \subset B$ if $a \in A \Rightarrow a \in B$
2. $A = B$ if

$$A \subset B \wedge B \subset A \tag{1}$$

3. $A \subsetneq B$ if $A \subset B \wedge A \neq B$

Set Building Notation

All x satisfying property $P(x)$

$\{x \in A, P(x)\}$ or $\{x, P(X)\}$

Examples

1. $\mathbb{N} = \{1, 2, 3, ...\}$
2. $\mathbb{Z} = \{0, 1, -1, 2, -2, ...\}$
3. $\mathbb{Q} = \left\{\frac{m}{n} : m, n \in \mathbb{Z} \quad n \neq 0\right\}$
4. Odd numbers: $\{2m - 1 : m \in \mathbb{N}\}$

$$\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R} (\subset \mathbb{C})$$

Definition

1. Union: $A \cup B = \{x : x \in A \vee x \in B\}$
2. Intersection: $A \cap B. = \{z : x \in A \wedge x \in B\}$
3. Difference: $A \setminus B = \{x : x \in A \wedge x \notin B\}$
4. Complement: $A^C = \{x : x \notin A\}$
5. Disjount: $A \cap B = \varnothing$

De Morgan

If A, B, C are sets, then:

1. $(B \cup C)^C = B^C \cap C^C$
2. $(B \cap C)^C = B^C \cup C^C$
3. $A \setminus (B \cup C) = (A \setminus B) \cap (A \setminus C)$
4. $A \setminus (B \cap C) = (A \setminus B) \cup (A \setminus C)$

Proof

1.

Let B, C be sets.

We want to prove that:

$$(B \cup C)^C = B^C \cap C^C$$

Since to prove the equality of two sets we need to show that Equation 1 holds:

We need to show that:

$$(B \cup C)^C \subset B^C \cap C^C \tag{2}$$

and

$$B^C \cap C^C \subset (B \cup C)^C \tag{3}$$

WTS Equation 2:

Let

$$x \in (B \cup C)^C$$

Then

$$\begin{gathered} x \notin B \cup C \\ \Rightarrow \\ x \notin B \wedge x \notin C \\ \Rightarrow \\ x \in B^C \wedge x \in C^C \\ \Rightarrow \\ x \in B^C \cap C^C \end{gathered}$$

Thus, $(B \cup C)^C \subset B^C \cap C^C$

WTS Equation 3:

Let

$$x \in B^C \cap C^C$$

Then

$$\begin{gathered} x \in B^C \wedge C^C \\ \Rightarrow \\ x \notin B \wedge x \notin C \Rightarrow \\ x \notin B \cup C \Rightarrow \\ x \in (B \cup C)^C \end{gathered}$$

Thus, $B^C \cap C^C \subset (B \cup C)^C$

Thus, $B^C \cap C^C = (B \cup C)^C$

Theorem: Induction

$\mathbb{N} = \{1, 2, 3, ...\}$ has an ordering

Axiom: Well ordering property

If $S \subset \mathbb{N}$ and $S \neq \varnothing$ then S has a least element: i.e., $\exists x \in S s.t. x \leq y \forall y \in S$

Let $P(n)$ be a statement depending on $n \in \mathbb{N}$. Assume:
1. (Base Case) $P(1)$ is True
2. (Induction Step) If $P(m)$ is True, then $P(m+1)$ is True

Then $P(n)$ is true for all $n \in \mathbb{N}$

Proof

Let $S = \{n \in \mathbb{N} : P(n) \text{ is not True}\}$

WTS: $S = \varnothing$

We will prove this by contradiction

(Assume the negation of the statement $S \neq \varnothing$ and derive a false statement. Rules of logic say that $S \neq \varnothing$ is false.)

Suppose $S \neq \varnothing$, by the well ordering property (WOP) of $\mathbb{N}$, S has a least element $x \in S$. Since $P(1)$ is True (assumption), $1 \notin S \rightarrow\Rightarrow x \neq 1$, this particular $x > 1$. Since x is the least element of S and $x - 1 < x$, $x - 1 \notin S$. Thus, by the definition of S, $P(x-1)$ is True. By the second property of induction (Induction Step) $\Rightarrow P(X)$ is True $\Rightarrow x \notin S$.

From $S \neq \varnothing$, we conclude $\exists x \in \mathbb{N}$, $x \in S$ and $x \notin S$. Contradiction.

Thus, $S = \varnothing$

Using Induction

We want to prove $\forall n \in \mathbb{N}, P(n)$ is True, we need to do 2 things:
1. Prove base case $P(1)$
2. Prove, if $P(m)$ is True, $P(m+1)$ is True

Theorem

$P(n)$

$\forall c \neq 1, \forall n \in \mathbb{N}$,

$$1 + c + c^2 + ... + c^n = \frac{1 - c^{n+1}}{1 - c} \tag{4}$$

Proof

Prove Equation 4 by induction: 1) (Basic Case)

$$1 + c^1 = \frac{1 - c^{1+1}}{1 - c} = 1 + c$$

2) (Induction Step)

$$1 + c + ... + c^m = \frac{1 - c^{m+1}}{1 - c}$$

We want to prove Equation 4 holds for $n = m + 1$

We have:

$$
\begin{aligned}
1 + c + ... + c^m + c^{m+1} &= \frac{1 - c^{m+1}}{1 - c} + c^{m+1} \\
&= \frac{1 - \cancel{c^{m+1}} + \cancel{c^{m+1}} - c^{m+2}}{1 - c} \\
&= \frac{1 - c^{(m+1)+1}}{1 - c}
\end{aligned}
$$

Thus, Equation 4 holds for $n = m + 1$. So by induction, Equation 4 holds for all $n \in \mathbb{N}$

Theorem

If $c \geq -1$, then $\forall n \in \mathbb{N}$,

$$(1 + c)^n \geq 1 + nc \tag{5}$$

Proof (By Induction)

1) (Base Case)

$$(1 + c)^1 \geq 1 + 1c$$

So, Equation 5

101. Calculus

102. Calculus I

102.1. Cheatsheet

102.1.1. Limits

Epsilon-Delta

For every distance ε around L, there's a δ-range around a that keeps $f(x)$ within ε of L.

$$\lim_{x \to x_0} f(x) = L \iff \forall \varepsilon > 0, \exists \delta > 0 \text{ s.t. } 0 < |x - x_0| < \delta \Rightarrow |f(x) - L| < \varepsilon$$

The limit of $f(x)$ as x approaches x_0 equals L if and only if, for every $\varepsilon > 0$, there exists a $\delta > 0$ such that, whenever $0 < |x - x_0| < \delta$, implies that $|f(x) - L| < \varepsilon$

Limit Type	Name	Quantifiers
$x \to x_0, f(x) \to L$	Epsilon-Delta	$\forall \varepsilon, \exists \delta$
$x \to x_0, f(x) \to \infty$	M-Delta	$\forall M, \exists \delta$
$x \to \infty, f(x) \to L$	epsilon-N	$\forall \varepsilon, \exists N$
$x \to \infty, f(x) \to \infty$	M-N	$\forall M, \exists N$

Finite $\to$ Finite (**Epsilon-Delta**)

$$\lim_{x \to x_0} f(x) = L \iff \forall \varepsilon > 0, \exists \delta > 0 \text{ s.t. } 0 < |x - x_0| < \delta \Rightarrow |f(x) - L| < \varepsilon$$

Finite $\to$ Infinity (**M-Delta**)

$+\infty$

$$\lim_{x \to x_0} f(x) = +\infty \iff \forall M > 0, \exists \delta > 0 \text{ s.t. } 0 < |x - x_0| < \delta \Rightarrow f(x) > M$$

$-\infty$

$$\lim_{x \to x_0} f(x) = -\infty \iff \forall M > 0, \exists \delta > 0 \text{ s.t. } 0 < |x - x_0| < \delta \Rightarrow f(x) < -M$$

Infinity $\to$ Finite (**Epsilon-N**)

$+\infty$

$$\lim_{x \to +\infty} f(x) = L \iff \forall \varepsilon > 0, \exists N > 0 \text{ s.t. } x > N \Rightarrow |f(x) - L| < \varepsilon$$

$-\infty$

$$\lim_{x \to +\infty} f(x) = L \iff \forall \varepsilon > 0, \exists N > 0 \text{ s.t. } x < -N \Rightarrow |f(x) - L| < \varepsilon$$

Infinity $\to$ Infinity (**M-N**)

$$\lim_{x\to\pm\infty} f(x) = \pm\infty \iff \forall M > 0, \exists N > 0 \text{ s.t. } x < N \Rightarrow f(x) > M$$

102.1.2. Derivatives

$$\frac{df}{dx}(x) = f'(x) = \lim_{h\to 0} \frac{f(x+h) - f(x)}{h}$$

Example

Let $f(x) = x^2$

a. Find $f'(x)$

$$f'(x) = \boxed{2x}$$

b. Prove a

$$\begin{aligned}
f'(x) &= \lim_{h\to 0} \frac{f(x+h) - f(x)}{(\cancel{x}+h) - \cancel{x}} \\
&= \lim_{h\to 0} \frac{f(x+h) - f(x)}{h} \\
&= \lim_{h\to 0} \frac{(x+h)^2 - x^2}{h} \\
&= \lim_{h\to 0} \frac{\cancel{x^2} + 2xh + h^2 - \cancel{x^2}}{h} \\
&= \lim_{h\to 0} \frac{\cancel{h}(2x+h)}{\cancel{h}} \\
&= \lim_{h\to 0} (2x+h) \\
&= \lim_{h\to 0} (2x+0) \\
&= \boxed{2x}
\end{aligned}$$

c. Prove b

$$\lim_{x\to x_0} f(x) = L \iff \forall \varepsilon > 0, \exists \delta > 0 \text{ s.t. } 0 < |x - x_0| < \delta \Rightarrow |f(x) - L| < \varepsilon$$

p.f.:

Let $\varepsilon > 0$

Choose $\delta = \varepsilon$

Suppose $0 < |h - 0| < \delta$

Check

$$\left| \frac{(x+h)^2 - x^2}{h} - 2x \right|$$

$$
\begin{aligned}
&= \left| \frac{\cancel{x^2} + 2xh + h^2 - \cancel{x^2}}{h} - 2x \right| \\
&= \left| \frac{\cancel{h}(2x + h)}{\cancel{h}} - 2x \right| \\
&= |\cancel{2x} + h - \cancel{2x}| \\
&= \boxed{|h| < \delta = \varepsilon}
\end{aligned}
$$

102.1.3. Integrals

102.1.4. Differential

If you have a function:

$$y = f(x)$$

Then the differential $\mathrm{d}u$ is defined as:

$$\mathrm{d}y = f'(x) \cdot \mathrm{d}x$$

This means:

- $f'(x) = \frac{\mathrm{d}y}{\mathrm{d}x}$ is the derivative
- $\mathrm{d}x$ is a small change in x
- $\mathrm{d}y$ is the corresponding small change in y

So:

$$\mathrm{d}y = \frac{\mathrm{d}y}{\mathrm{d}x} \cdot \mathrm{d}x$$

Example

Let's say“

$$y = x^2$$

Then:

$$\frac{\mathrm{d}y}{\mathrm{d}x} = 2x$$

$$\frac{d}{\mathrm{d}x}\left(\int f(x)\,\mathrm{d}x\right) = f(x)$$

$$\int \left(\frac{d}{\mathrm{d}x} f(x)\right)\mathrm{d}x = f(x)$$

Operation	Notation	Input	Output	Meaning
Derivative	$\frac{df}{dx}$	Function $f(x)$	Function $f'(x)$	Slope/Rate of change at each x
Indefinite Integral	$\int f(x)dx$	Function $f(x)$	Family of functions $F(x)+C$	Function whose slope is $f(x)$
Definite Integral	$\int_a^b f(x)dx$	Function $f(x)$ Bounds $[a,b]$	Number	Total signed area between a and b

Rule	$\frac{d}{\mathrm{d}x}$ Rule	$\frac{d}{\mathrm{d}x}$ Example	$\int$ Rule	$\int$ Example
Constant	$\frac{d}{\mathrm{d}x}[c] = 0$	$\frac{d}{\mathrm{d}x}[7] = 0$	$\int c\,\mathrm{d}x = cx + C$	$\int 7\,\mathrm{d}x = 7x + C$
Power	$\frac{d}{\mathrm{d}x}[x^n] = nx^{n-1}$	$\frac{d}{\mathrm{d}x}[x^4] = 4x^3$	$\int x^n\,\mathrm{d}x = \frac{x^{n+1}}{n+1} + C$ $(n \neq -1)$	$\int x^3\,\mathrm{d}x = \frac{x^4}{4} + C$
Constant Multiple	$\frac{d}{\mathrm{d}x}[cf] = cf'$	$\frac{d}{\mathrm{d}x}[3x^2] = 3 \cdot 2x = 6x$	$\int cf\,\mathrm{d}x = c\int f\,\mathrm{d}x$	$\int 3x^2\,\mathrm{d}x = 3 \cdot \frac{x^3}{3} = x^3 + C$
Sum	$\frac{d}{dx}[f+g] = f' + g'$	$\frac{d}{\mathrm{d}x}(x^2+x) = \frac{d}{\mathrm{d}x}(x^2) + \frac{d}{\mathrm{d}x}(x) = 2x + 1$	$\int (f+g)\,\mathrm{d}x = \int f\,\mathrm{d}x + \int g\,\mathrm{d}x$	$\int (x^2+x)\,\mathrm{d}x = \int x^2\,\mathrm{d}x + \int x\,\mathrm{d}x = \frac{x^3}{3} + \frac{3^2}{2} + C$

Rule	$\frac{d}{dx}$ Rule	$\frac{d}{dx}$ Example	$\int$ Rule	$\int$ Example
Difference	$\frac{d}{dx}[f-g]=f'-g'$	$\frac{d}{dx}(x^2-x)$ $=\frac{d}{dx}(x^2)-\frac{d}{dx}(x)$ $=2x-1$	$\int(f-g)\,dx$ $=\int f\,dx-\int g\,dx$	$\int(x^2-x)\,dx$ $=\int x^2\,dx-\int x\,dx$ $=\frac{x^3}{3}-\frac{x^2}{2}+C$
Product	$\frac{d}{dx}[fg]=f'g+fg'$	$\frac{d}{dx}[x\sin x]$ $=1\cdot\sin x+x\cdot\cos x$	Integration by Parts $\int u\,dv=uv-\int v\,du$	
Quotient	$\frac{d}{dx}\left[\frac{f}{g}\right]=\frac{f'g-fg'}{g^2}$	$\frac{d}{dx}\left[\frac{x^2}{x+1}\right]$ $=\frac{2x(x+1)-x^2(1)}{(x+1)^2}$ $=\frac{x^2+2x}{(x+1)^2}$	Algebraic Manipulation / Substitution	
Chain	$\frac{d}{dx}[f(g(x))]$ $=f'(g(x))\cdot g'(x)$	$\frac{d}{dx}[\sin(x^2)]$ $=\cos(x^2)\cdot 2x$	Integration by Substitution $\int f(g(x))g'(x)\,dx$ $=\int f(u)\,du$	
Exponential	$\frac{d}{dx}[e^x]=e^x$	$\frac{d}{dx}[e^x]=e^x$	$\int e^x\,dx=e^x+C$	$\int e^x\,dx=e^x+C$
	$\frac{d}{dx}[a^x]=a^x\ln(a)$	$\frac{d}{dx}[2^x]=2^x\ln 2$	$\int a^x dx=\frac{a^x}{\ln a}+C$	$\int 2^x\,dx=\frac{2^x}{\ln 2}+C$
Logarithmic	$\frac{d}{dx}[\ln(x)]=\frac{1}{x}$	$\frac{d}{dx}[\ln(x)]=\frac{1}{x}$	$\int\ln(x)\,dx$ $=x\ln x-x+C$	$\int\ln(x)\,dx$ $=x\ln x-x+C$
	$\frac{d}{dx}[\log_a(x)]=\frac{1}{x\ln(a)}$	$\frac{d}{dx}\log_2(x)=\frac{1}{x\ln(2)}$	$\int\log_a(x)\,dx=$ $\frac{x\ln x}{\ln a}-\frac{x}{\ln a}+C$	$\int\log_{10}(x)\,dx$ $=\frac{x\ln x}{\ln 10}-\frac{x}{\ln 10}+C$

Rule	$\frac{d}{\mathrm{d}x}$ Rule	$\frac{d}{\mathrm{d}x}$ Example	$\int$ Rule	$\int$ Example
Sin	$\frac{d}{dx}[\sin(x)] = \cos(x)$	$\frac{d}{dx}[\sin(x)] = \cos(x)$	$\int \sin(x)\,\mathrm{d}x = -\cos x + C$	$\int \sin(x)\,\mathrm{d}x = -\cos x + C$
Cos	$\frac{d}{dx}[\cos(x)] = -\sin(x)$	$\frac{d}{dx}[\cos(x)] = -\sin(x)$	$\int \cos(x)\,\mathrm{d}x = \sin x + C$	$\int \cos(x)\,\mathrm{d}x = \sin x + C$
Tan	$\frac{d}{dx}[\tan(x)] = \sec^2(x)$	$\frac{d}{dx}[\tan(x)] = \sec^2(x)$	$\int \tan(x)\,\mathrm{d}x = -\ln\lvert\cos x\rvert + C$	$\int \tan(x)\,\mathrm{d}x = -\ln\lvert\cos x\rvert + C$

102.1.5. Product Rule $\longrightarrow$ Integration by Parts

$$\int u\,\mathrm{d}v = uv - \int v\,\mathrm{d}u$$

Example

Given a function:

$$f(x) = x \cdot e^x$$

Integrate (by parts):

$$\int x \cdot e^x\,\mathrm{d}x$$

Step 1: Choose u and $\mathrm{d}v$

We choose:

- $u = x$ (easy to **differentiate**)
- $\mathrm{d}v = e^x$ (easy to **integrate**)

Step 2: Compute $\mathrm{d}u$ and v

- $u = x \longrightarrow \mathrm{d}u = \mathrm{d}x$
- $\mathrm{d}v = e^x\,\mathrm{d}x \longrightarrow v = \int e^x\,\mathrm{d}x = e^x$

Step 3: Plug into formula

$$\begin{aligned}\int x \cdot e^x\,\mathrm{d}x &= u \cdot v - \int v \cdot \mathrm{d}u \\ &= x \cdot e^x\,\mathrm{d}x - \int e^x \cdot \mathrm{d}x\end{aligned}$$

Step 4: Compute the remaining integral:

$$\int e^x\,\mathrm{d}x = e^x$$

Step 5: Finish the expression:

$$\begin{aligned}\int x \cdot e^x &= x \cdot e^x - e^x + C \\ &= e^x(x-1) + C\end{aligned}$$

102.1.6. Chain Rule $\longrightarrow$ u-Substitution

$$\int f(g(x))g'(x)\,\mathrm{d}x = \int f(u)\,\mathrm{d}u$$

102.1.7. Quotient Rule $\longrightarrow$ Algebraic Manipulation / Substitution

102.2. Limits

102.2.1. Properties of Limits

102.2.1.1. Continuous

102.2.1.1.1. Addition, Subtraction, Multiplication, Division

$$\lim_{x\to c}(f(x) * g(x)) = \lim_{x\to c} f(x) * \lim_{x\to c} g(x)$$
$$* \in \{+, -, \times, \div\}$$

102.2.1.1.2. Constant

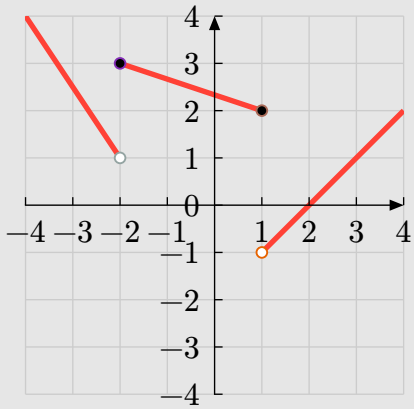
$$\lim_{x\to c} kf(x) = k\lim_{x\to c} f(x)$$

102.2.1.2. Non-continuous

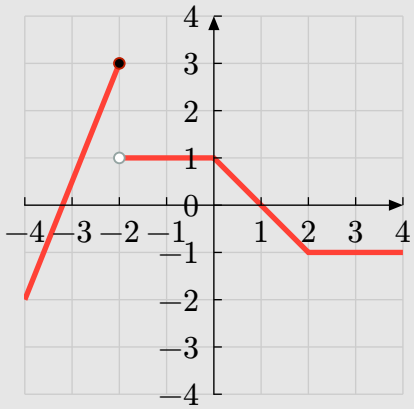
Even though the limit for either function may not exist, their $*$ can exist as long as

$$\lim_{x\to c^-}(f(x) * g(x)) \quad = \quad \lim_{x\to c^+}(f(x) * g(x))$$
$$* \in \{+, -, \times, \div\}$$

Example



$f(x)$



$g(x)$

Problem 1: Limit Exists

$$\lim_{x\to -2}(f(x) + g(x))$$

1. Left-hand limit ($x \to -2^-$)

$$\lim_{x\to -2^-} f(x) = 1 \qquad \lim_{x\to -2^-} g(x) = 3$$

Adding these

$$\begin{aligned}\lim_{x\to -2^-}(f(x)+g(x)) &= \lim_{x\to -2^-} f(x) + \lim_{x\to -2^-} g(x)\\ &= 1+3\\ &= 4\end{aligned}$$

2. Right-hand limit ($x \to -2^+$)

$$\lim_{x\to -2^+} f(x) = 3 \qquad \lim_{x\to -2^+} g(x) = 1$$

Adding these

$$\begin{aligned}\lim_{x\to -2^+}(f(x)+g(x)) &= \lim_{x\to -2^+} f(x) + \lim_{x\to -2^+} g(x)\\ &= 3+1\\ &= 4\end{aligned}$$

3. Since both the left-hand and right-hand limits agree

$$\lim_{x\to -2}(f(x)+g(x)) = 4$$

Problem 2: Limit Does Not Exist

$$\lim_{x\to 1}(f(x)+g(x))$$

1. Left-hand limit ($x \to 1^-$)

$$\lim_{x\to 1^-} f(x) = 2 \qquad \lim_{x\to 1^-} g(x) = 0$$

Adding these

$$\begin{aligned}\lim_{x\to 1^-}(f(x)+g(x)) &= \lim_{x\to 1^-} f(x) + \lim_{x\to 1^-} g(x)\\ &= 2+0\\ &= 2\end{aligned}$$

2. Right-hand limit ($x \to 1^+$)

$$\lim_{x\to 1^+} f(x) = -1 \qquad \lim_{x\to 1^+} g(x) = 0$$

Adding these

$$\begin{aligned}\lim_{x\to 1^+}(f(x)+g(x)) &= \lim_{x\to 1^+} f(x) + \lim_{x\to 1^+} g(x)\\ &= -1+0\\ &= -1\end{aligned}$$

3. Since both the left-hand and right-hand limits do not agree, the limit $\lim_{x\to c}(f(x)+g(x))$ does not exist

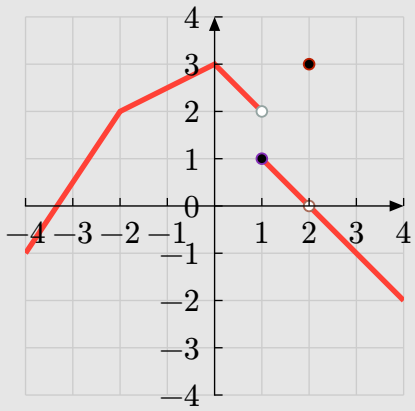
102.2.1.3. Composite Functions

$$\lim_{x\to c} f(g(x)) = f\left(\lim_{x\to c} g(x)\right)$$

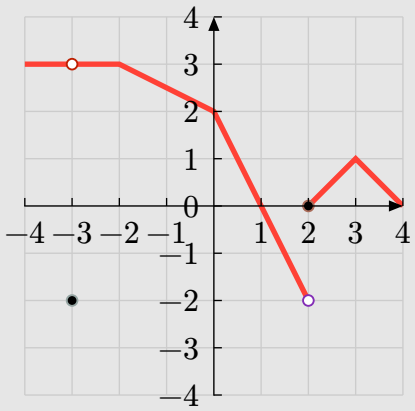
For this to hold true, two important conditions must be satisfied:

- **Inner limit exists**: The limit $\lim_{x \to c}$ must exist and equal some value L. That is, as x gets arbitrarily close to c, $g(x)$ approaches a well-defined number L
- **Continuity of the outer function**: The function f must be continuous at the point L. Continuity ensures that f behaves predictably near L, without any jumps, gaps, or undefined points

Example



$f(x)$



$g(x)$

Problem 1: Inner Limit & Continuity Exist

$$\lim_{x \to -3} f(g(x))$$

1. Inner limit $\lim_{x \to -3} g(x)$

Observing $g(x)$, as $x \to -3$, $g(x) \to 3$. The inner limit $L = 3$ exists

$$\lim_{x \to -3} g(x) = 3$$

2. Continuity of $f(x)$ at $x = 3$

Observing $f(x)$, $f(3) = -1$. Since $f(x)$ is continuous at $x = 3$, the composite limit holds

$$\begin{aligned} f\left(\lim_{x \to -3} g(x)\right) &= f(3) \\ &= -1 \end{aligned}$$

Problem 2: Inner Limit Does Not Exist

$$\lim_{x \to 2} f(g(x))$$

1. Inner limit $\lim_{x \to 2} g(x)$

Observing $g(x)$, as $x \to 2$, $g(x) \to 2$. The inner limit does not exist

Problem 3: Continuity Does Not Exist

$$\lim_{x \to 0.5} f(g(x))$$

1. Inner limit $\lim_{x \to 0.5} g(x)$

Observing $g(x)$, as $x \to 0.5$, $g(x) \to 1$. The inner limit $L = 1$ exists

$$\lim_{x \to 0.5} g(x) = 1$$

2. Continuity of $f(x)$ at $x = 1$

Observing $f(x)$, $f(1)$ is not continuous. Since $f(x)$ is not continuous at $x = 1$, the composite limit does not hold

102.2.1.4. Limits by Direct Substitution

Example

Limit exists

$$\begin{aligned}\lim_{x \to -1} (6x^2 + 5x - 1) &= 6(-1)^2 + 5(-1) - 1 \\ &= 6 - 5 - 1 \\ &= 0\end{aligned}$$

Limit does not exist (Undefined)

$$\begin{aligned}\lim_{x \to 1} \frac{x}{\ln(x)} &= \frac{1}{\ln(1)} \\ &= \frac{1}{0}\end{aligned}$$

102.2.1.4.1. Limits of Piecewise Functions

102.2.1.4.2. Absolute Value

102.2.1.5. Limits by Factoring

102.2.1.6. Limits by Rationalizing

102.2.1.7. Continuity & Differentiability at a Point

Example

Piecewise function:

$$f(x) = \begin{cases} x^2 & \text{if } x < 3 \\ 6x - 9 & \text{if } x \geq 3 \end{cases}$$

1. Check for **Continuity**
    - Value of $f(3)$

$$f(3) = 6(3) - 9 = \boxed{9}$$

    - Left-Hand Limit (LHL)

$$\lim_{x \to 3^-} f(x) = 3^2 = \boxed{9}$$

- Right-Hand Limit (RHL)

$$\lim_{x \to 3^+} f(x) = 6(3) - 9 = \boxed{9}$$

Since $f(3) = \lim_{x \to 3^-} f(x) = \lim_{x \to 3^+} f(x)$, $f(x)$ is **continuous** at $x = 3$

2. Check for **Differentiability**

- Left-Hand Derivative (LHD)

$$\begin{aligned}
\lim_{x \to 3^-} \frac{f(x) - f(3)}{x - 3} &= \frac{x^2 - 3^2}{x - 3} \\
&= \frac{x^2 - 9}{x - 3} \\
&= \frac{(x + 3)(x - 3)}{x - 3} \\
&= x + 3 \\
&= \boxed{6}
\end{aligned}$$

- Right-Hand Derivative

$$\begin{aligned}
\lim_{x \to 3^+} \frac{f(x) - f(3)}{x - 3} &= \frac{(6x - 9) - 3^2}{x - 3} \\
&= \frac{6x - 9 - 9}{x - 3} \\
&= \frac{6x - 18}{x - 3} \\
&= \frac{6(x - 3)}{x - 3} \\
&= \boxed{6}
\end{aligned}$$

Since the left-hand and right-hand derivatives are equal, $f(x)$ is **differentiable** at $x = 3$

Conclusion: $f(x)$ is both continuous & differentiable at $x = 3$

Example

102.3. Derivatives

Measures the instantaneous rate of change of a function

Derivative of of a function $f(x)$

$$f'(x) = \frac{d}{dx} f(x)$$

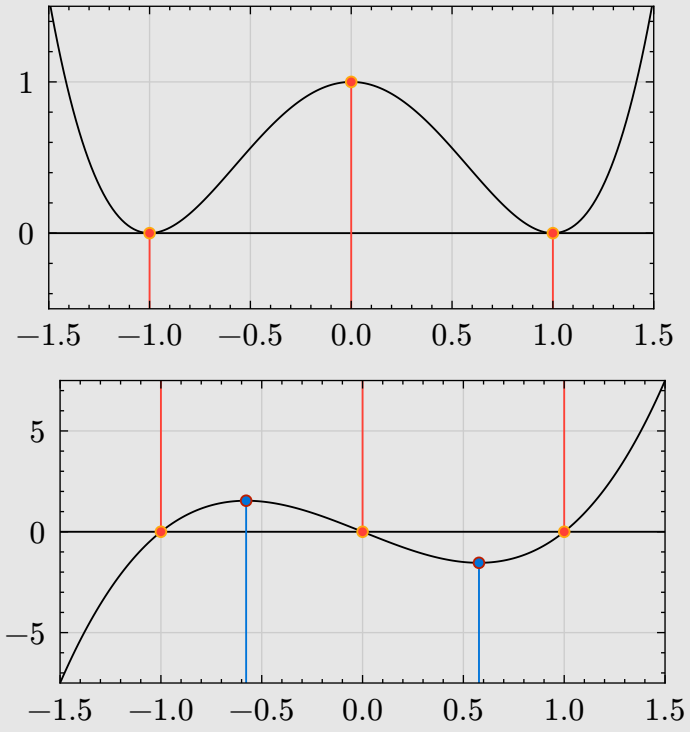
General Pattern:

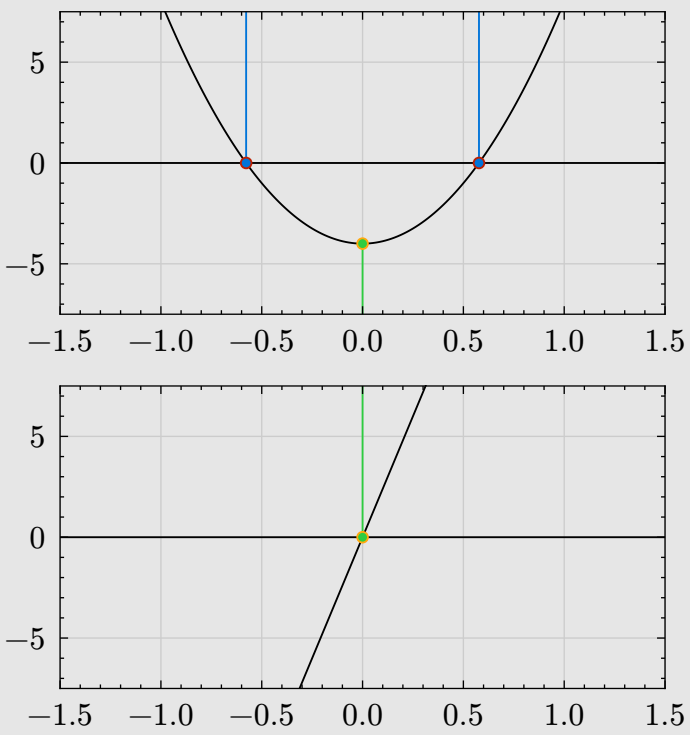
- **Linear term**: If a function is linear (e.g., $ax + b$), the first derivative will be a constant a.
- **Constant term**: The derivative of a constant is zero.
- **Higher-degree polynomials**: For polynomials of higher degrees, each derivative decreases the degree of the polynomial until you get a constant, after which all further derivatives are zero.

Example

Consider the quartic function: $f(x) = x^4 - 2x^2 + 1$

- First derivative: $f'(x) = 4x^3 - 4x$ (**cubic**)
- Second derivative: $f''(x) = 12x^2 - 4$ (**quadratic**)
- Third derivative: $f'''(x) = 24x$ (**linear**)
- Fourth derivative: $f''''(x) = 24$ (**constant**)
- Fourth derivative: $f''''(x) = 0$ (**zero**)





102.3.1. Power Rule

$$f(x) = x^n$$
$$f'(x) = nx^{n-1}$$

Example

$$f(x) = x^3$$
$$f'(x) = 3x^2$$

102.3.2. Product Rule

$$f(x) = u(x) \cdot v(x)$$
$$f'(x) = u'(x)v(x) + u(x)v'(x)$$

Example

$$f(x) = x^2 \cdot x^3$$

$$u(x) = x^2$$

$$v(x) = x^3$$

1. Compute the derivatives of $u(x)$ and $v(x)$

$$u'(x) = 2x$$

$$v'(x) = 3x^2$$

2. Apply rule

$$\begin{aligned} f'(x) &= (2x)(x^3) + (x^2)(3x^2) \\ &= 2x^2 + 3x^4 \\ &= 5x^4 \end{aligned}$$

102.3.3. Quotient Rule

$$f(x) = \frac{u(x)}{v(x)}$$

$$f'(x) = \frac{u'(x)v(x) - u(x)v'(x)}{v(x)^2}$$

Example

$$f(x) = \frac{x^3}{x^2}$$

$$u(x) = x^3$$

$$v(x) = x^2$$

1. Compute the derivatives of $u(x)$ and $v(x)$

$$u'(x) = 3x^2$$

$$v'(x) = 2x$$

2. Apply rule

$$\begin{aligned} f'(x) &= \frac{(3x^2)(x^2) - (x^3)(2x)}{(x^2)^2} \\ &= \frac{3x^4 - 2x^4}{x^4} \\ &= \frac{x^4}{x^4} \\ &= 1 \end{aligned}$$

102.3.4. Chain Rule

$$f(g(x))$$

$$f'(x) = f'(g(x)) \cdot g'(x)$$

Example

$$f(x) = (2x+1)^3$$

- Outer function:

$$f(u) = u^3$$

- Inner function

$$g(x) = 2x + 1$$

1. Compute the derivatives of $f'(u)$ and $g'(x)$

$$f'(u) = 3u^2$$

$$g'(x) = 2$$

2. Apply rule

$$\begin{aligned} f'(x) &= 3(2x+1)^2 \cdot 2 \\ &= 6(2x+1)^2 \end{aligned}$$

102.3.5. Others

1. Trigonometric Functions

$$\frac{d}{dx}[\sin(x)] = \cos(x)$$

$$\frac{d}{dx}[\cos(x)] = -\sin(x)$$

$$\frac{d}{dx}[\tan(x)] = \sec^2(x)$$

$$\frac{d}{dx}[\cot(x)] = \csc^2(x)$$

$$\frac{d}{dx}[\sec(x)] = \sec(x)\tan(x)$$

$$\frac{d}{dx}[\csc(x)] = -\csc(x)\cot(x)$$

2. Inverse Trigonometric Functions

$$\frac{d}{dx}[\arcsin(x)] = \frac{1}{\sqrt{1-x^2}}$$

$$\frac{d}{dx}[\arccos(x)] = -\frac{1}{\sqrt{1-x^2}}$$

$$\frac{d}{dx}[\arctan(x)] = \frac{1}{1+x^2}$$

$$\frac{d}{dx}[\operatorname{arccot}(x)] = -\frac{1}{1+x^2}$$

$$\frac{d}{dx}[\operatorname{arcsec}(x)] = \frac{1}{|x|\sqrt{x^2-1}}$$

$$\frac{d}{dx}[\operatorname{arccsc}(x)] = -\frac{1}{|x|\sqrt{x^2-1}}$$

3. Exponential and Logarithmic Functions

$$\frac{d}{dx}[e^x] = e^x$$

$$\frac{d}{dx}[a^x] = a^x \ln(a)$$

$$\frac{d}{dx}[\ln(x)] = \frac{1}{x}$$

$$\frac{d}{dx}\left[\log_{a(x)}\right] = \frac{1}{x\ln(a)}$$

Definition 0: Stationary Point

A stationary point of a function $f(x)$ is a solution to

$$f'(x) = 0$$

That is, where the slope (derivative) is zero.

- *In 1D, this means the tangent line is horizontal*
- *In nD, it means the gradient vector $\nabla f(x) = 0$*

102.4. Higher-Order Derivatives

$$\frac{d^2y}{dx^2}$$

102.5. Power Rule

$$f(x) = x^n, \qquad n \neq 0$$
$$f'(x) = nx^{n-1}$$

Example

$$f(x) = x^3$$
$$f'(x) = 3x^2$$

Example

$$\begin{aligned}\frac{d}{dx}\left(\sqrt[3]{x^2}\right) &= \frac{d}{dx}\left((x^2)^{\frac{1}{3}}\right)\\ &= \frac{d}{dx}\left(x^{2\times\frac{1}{3}}\right)\\ &= \frac{d}{dx}\left(x^{\frac{2}{3}}\right)\\ &= \frac{d}{dx}\left(\frac{2}{3}x^{-\frac{1}{3}}\right)\end{aligned}$$

102.5.1. Constant Rule

$$\frac{d}{dx}[k] = 0$$

Example

$$\frac{d}{dx}[-3] = 0$$

102.5.2. Constant Multiple Rule

$$\begin{aligned}\frac{d}{dx}[kf(x)] &= k\frac{d}{dx}[f(x)]\\ &= kf'(x)\end{aligned}$$

Example

$$\begin{aligned}\frac{d}{dx}[2x^5] &= 2\frac{d}{dx}[x^5]\\ &= 2\cdot 5x^4\\ &= 10x^4\end{aligned}$$

102.5.3. Sum Rule

$$\begin{aligned}\frac{d}{dx}[f(x)+g(x)] &= \frac{d}{dx}[f(x)] + \frac{d}{dx}[g(x)]\\ &= f'(x)+g'(x)\end{aligned}$$

Example

$$\begin{aligned}\frac{d}{dx}[x^3+x^{-4}] &= \frac{d}{dx}[x^3] + \frac{d}{dx}[x^{-4}]\\ &= 3x^2 + (-4x^{-5})\\ &= 3x^2 - 4x^{-5}\end{aligned}$$

102.5.4. Difference Rule

$$\begin{aligned}\frac{d}{dx}[f(x) - g(x)] &= \frac{d}{dx}[f(x)] - \frac{d}{dx}[g(x)] \\ &= f'(x) - g'(x)\end{aligned}$$

Example

$$\begin{aligned}\frac{d}{dx}[x^4 - x^3] &= \frac{d}{dx}[x^4] - \frac{d}{dx}[x^3] \\ &= 4x^3 - 3x^2\end{aligned}$$

102.5.5. Square Root

Example

$$\begin{aligned}\frac{d}{dx}\sqrt[4]{x} &= \frac{d}{dx}x^{\frac{1}{4}} \\ &= \frac{1}{4}x^{\frac{1}{4}-1} \\ &= \frac{1}{4} \cdot x^{-\frac{3}{4}} \\ &= \frac{1}{4} \cdot \frac{1}{x^{3/4}} \\ &= \frac{1}{4x^{3/4}}\end{aligned}$$

102.5.6. Derivative of a Polynomial

Example

$$\begin{aligned}f(x) &= 2x^3 - 7x^2 + 3x - 100 \\ f'(x) &= 2 \cdot 3x^2 - 7 \cdot 2x + 3 + 0\end{aligned}$$

Example

$$h(x) = 3f(x) + 2g(x)$$

Evaluate $\frac{d}{dx}h(x)$ at $x = 9$

$$\begin{aligned}\frac{d}{dx}(h(x)) &= \frac{d}{dx}(3f(x)+2g(x)) \\ &= \frac{d}{dx}3f(x)+\frac{d}{dx}2g(x) \\ &= 3\frac{d}{dx}f(x)+2\frac{d}{dx}g(x)\end{aligned}$$

Evaluate $h'(9)$

$$h'(9) = 3f'(9)+2g'(9)$$

Example

$$\begin{aligned}g(x) &= \frac{2}{x^3}-\frac{1}{x^2} \\ \frac{d}{dx}(g(x)) &= \frac{d}{dx}(2x^{-3}-1x^{-2}) \\ g'(x) &= 2\cdot(-3)x^{-4}-(-2)x^{-3} \\ &= -6x^{-4}+2x^{-3} \\ g'(2) &= -6\cdot 2^{-4}+2\cdot 2^{-3} \\ &= -\frac{6}{2^4}+\frac{2}{2^3} \\ &= -\frac{3}{8}+\frac{2}{8} \\ &= -\frac{1}{8}\end{aligned}$$

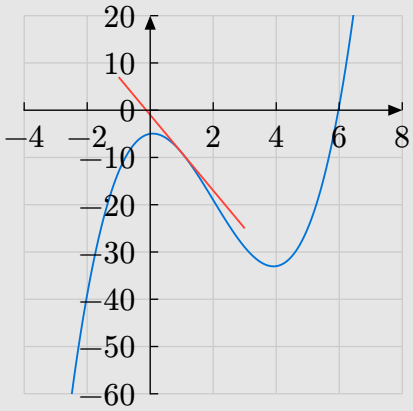
Example

–9 –8

$$\begin{aligned}f(x) &= x^3-6x^2+x-5 \\ y &= mx+b \\ f'(1) &= -8\ y = -8\ x+b \\ -9 &= -8\cdot 1+b \\ -9 &= -8+b \\ -9+8 &= -8+8+b \\ b &= -1\end{aligned}$$

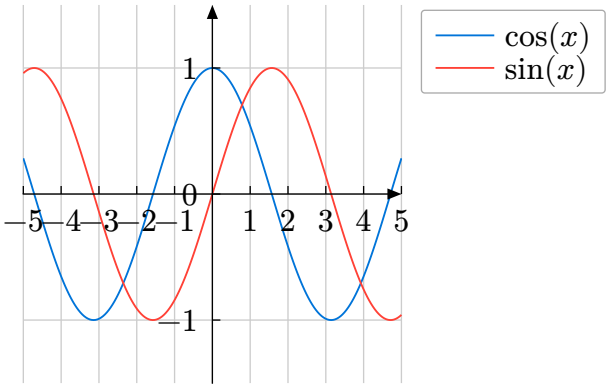
Tangent line to $f(x)$ at $x=1$

$$y = -8x-1$$



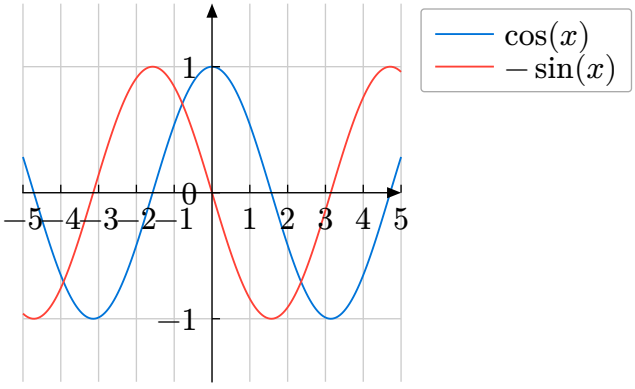
102.5.7. Sin

$$\frac{d}{dx}\sin(x) = \cos(x)$$



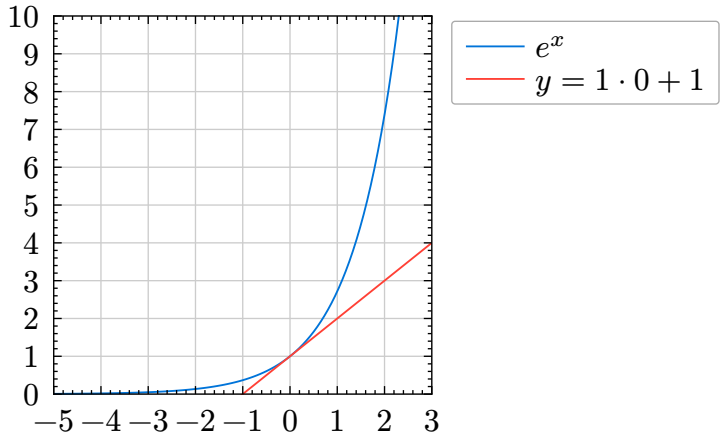
102.5.8. Cos

$$\frac{d}{dx}\cos(x) = -\sin(x)$$



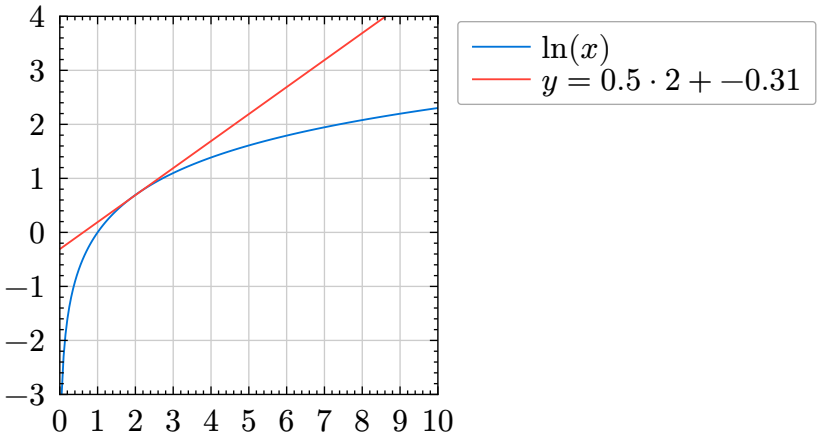
102.5.9. e^x

$$\frac{d}{dx}e^x = e^x$$



102.5.10. $\ln(x)$

$$\ln(x) = \frac{1}{x}$$



102.6. Product Rule

$$\frac{d}{dx}[f(x)g(x)] = f'(x)g(x) + f(x)g'(x)$$

Example

$$\frac{d}{dx}[x^2 \sin(x)]$$

$$f(x) = x^2 \qquad g(x) = \sin(x)$$
$$f'(x) = 2x \qquad g'(x) = \cos(x)$$

$$\frac{d}{dx}[x^2 \sin(x)] = 2x \sin(x) + x^2 \cos(x)$$

102.7. Quotient Rule

$$\frac{d}{dx}\left[\frac{f(x)}{g(x)}\right] = \frac{f'(x)g(x) - f(x)g'(x)}{[g(x)]^2}$$

Example

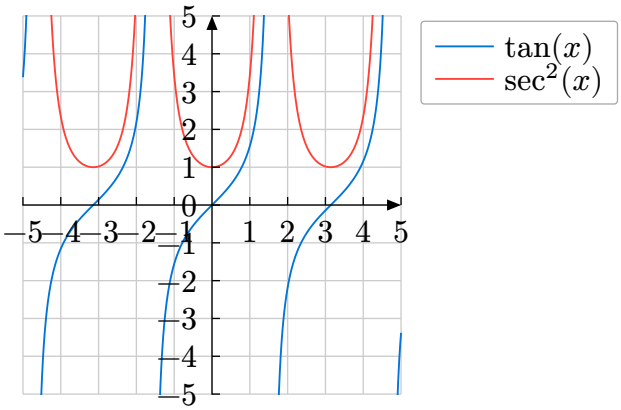
$$\frac{d}{dx}\left[\frac{x^2}{\cos(x)}\right]$$

$$f(x) = x^2 \qquad g(x) = \cos(x)$$
$$f'(x) = 2x \qquad g'(x) = -\sin(x)$$

$$\frac{d}{dx}\left[\frac{x^2}{\cos(x)}\right] = \frac{2x\cos(x) - x^2(-\sin(x))}{[\cos(x)]^2}$$

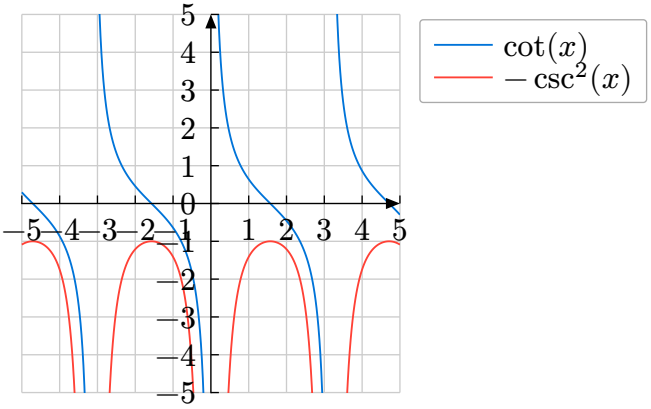
102.7.1. $\tan(x)$

$$\begin{aligned}
\frac{d}{dx}[\tan(x)] &= \frac{d}{dx}\left[\frac{\sin(x)}{\cos(x)}\right] \\
&= \frac{\cos(x)\cdot\cos(x) - \sin(x)\cdot -\sin(x)}{\cos^2(x)} \\
&= \frac{\cos^2(x) + \sin^2(x)}{\cos^2(x)} \\
&= \boxed{\frac{1}{\cos^2(x)}} \\
&= \boxed{\sec^2(x)}
\end{aligned}$$



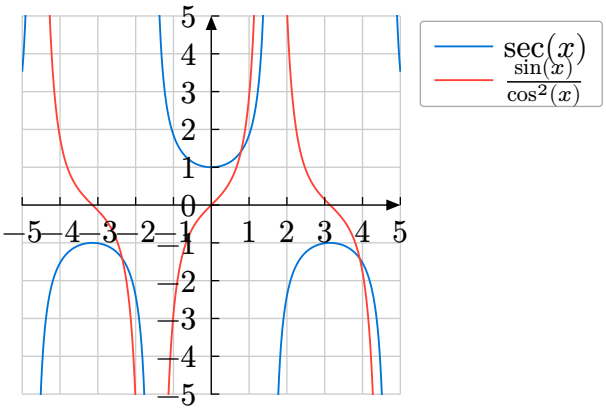
102.7.2. $\cot(x)$

$$\begin{aligned}\frac{d}{dx}[\cot(x)] &= \frac{d}{dx}\left[\frac{\cos(x)}{\sin(x)}\right] \\ &= \frac{\cos(x)\cdot\cos(x) - \sin(x)\cdot -\sin(x)}{\cos^2(x)} \\ &= \frac{-\sin^2(x) - \cos^2(x)}{} \\ &= \boxed{-\frac{1}{\sin^2}(x)} \\ &= \boxed{-\csc^2(x)}\end{aligned}$$



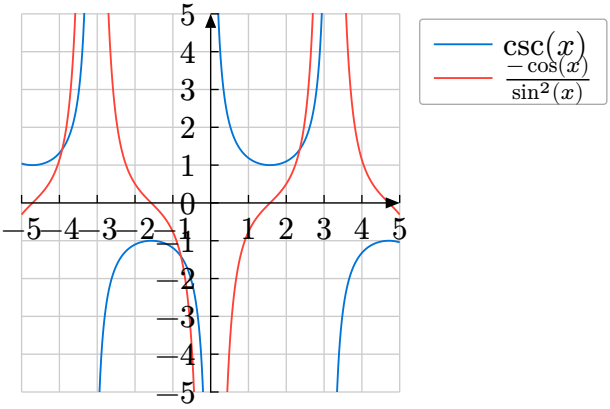
102.7.3. $\sec(x)$

$$\begin{aligned}\frac{d}{dx}[\sec(x)] &= \frac{d}{dx}\left[\frac{1}{\cos(x)}\right] \\ &= \frac{0\cdot\cos(x) - 1\cdot -\sin(x)}{\cos^2(x)} \\ &= \frac{0 + 1\cdot\sin(x)}{\cos^2(x)} \\ &= \boxed{\frac{\sin(x)}{\cos^2(x)}} \\ &= \frac{\sin(x)}{\cos(x)}\cdot\frac{1}{\cos(x)} \\ &= \boxed{\tan(x)\cdot\sec(x)}\end{aligned}$$



102.7.4. $\csc(x)$

$$\begin{aligned}\frac{d}{dx}[\csc(x)] &= \frac{d}{dx}\left[\frac{1}{\sin(x)}\right] \\ &= \frac{0 \cdot \sin(x) - 1 \cdot \cos(x)}{\sin^2(x)} \\ &= \frac{0 - 1 \cdot \cos(x)}{\sin^2(x)} \\ &= \boxed{\frac{-\cos(x)}{\sin^2(x)}} \\ &= -\frac{\cos(x)}{\sin(x)} \cdot \frac{1}{\sin(x)} \\ &= \boxed{\cot(x) \cdot \csc(x)}\end{aligned}$$



102.8. Chain Rule

Compute the derivative of a composite function

$$\begin{aligned}h'(x) &= \frac{d}{dx}[f(g(x))] \\ &= f'(g(x)) \cdot g'(x)\end{aligned}$$

More generally:

$$y = f_1(f_2(f_3(...f_n(x)...)))$$

$$\frac{dy}{dx} = f_1'(f_2(f_3(...f_n(x)...))) \cdot f_2'(f_3(...f_n(x)...)) \cdot f_3'(...f_n(x)...) \cdot ... \cdot f_n'(x)$$

Example

$$\frac{d}{dx}[f(g(x))] = f'(g(x)) \cdot g'(x)$$

$$\frac{d}{dx}\left[e^{\sin(x)}\right]$$

$$\frac{d}{dx}[e^x] = \frac{1}{x} \qquad \frac{d}{dx}[\sin(x)] = \cos(x)$$

$$f'(g(x)) = \frac{1}{\sin(x)} \qquad g'(x) = \cos(x)$$

$$\frac{d}{dx}\left[\underbrace{\ln\left(\overbrace{\sin(x)}^{g(x)}\right)}_{f(g(x))}\right] = \frac{1}{\sin(x)} \cdot \cos(x)$$

$$f(x) = \cos^3(x) = (\cos(x))^3$$

$$f(x) = v(u(x))$$

$$f'(x) = v'(u(x)) \cdot u'(x)$$

$$\begin{aligned} f'(x) &= \frac{dv}{du} \cdot \frac{du}{dx} \\ &= \frac{d(\cos(x))^3}{d\cos(x)} \cdot \frac{d\cos(x)}{x} \\ &= 3(\cos(x))^2 \cdot -\sin(x) \\ &= -3(\cos(x))^2 \sin(x) \end{aligned}$$

102.9. Implicit Differentiation

When a function is not explicitly solved for one variable in terms of another. E.g.:

$$x^2 + y^2 = 1$$

Instead of solving for y explicitly in terms of x, implicit differentiation allows you to differentiate both sides of an equation directly, treating y as an implicit function of x.

Steps for Implicit Differentiation

1. Differentiate both sides of the equation with respect to x, treating y as a function of x
2. Apply the chain rule whenever differentiating y, since $y = y(x)$
2. Solve for $\frac{dx}{dy}$

Example

$$x^2 + y^2 = 1$$

1. Differentiate both sides

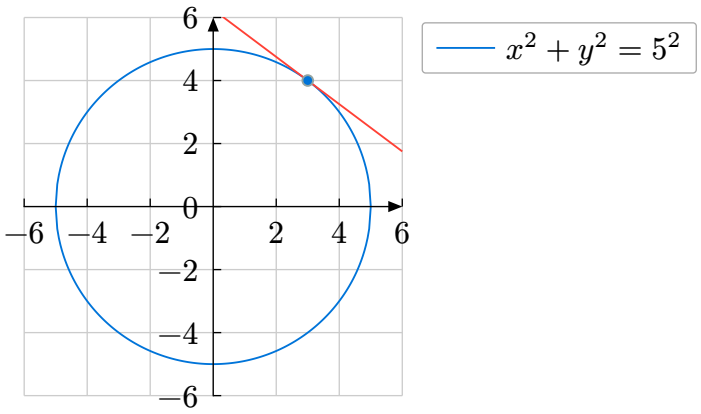
$$\frac{d}{dx}[x^2 + y^2] = \frac{d}{dx}[1]$$
$$\frac{d}{dx}[x^2] + \frac{d}{dx}[y^2] = \frac{d}{dx}[1]$$
$$\frac{d}{dx}[x^2] + \frac{d}{dx}[y^2] = 0$$

2. Apply the chain rule to y^2

$$2x + 2y\frac{dy}{dx} = 0$$

3. Slove for $\frac{dy}{dx}$

$$2y \cdot \frac{dy}{dx} = -2x$$
$$\frac{dy}{dx} = \frac{-2x}{2y}$$
$$\frac{dy}{dx} = -\frac{x}{y}$$



$$x^2 + y^2 = 5^2$$
$$2xdx + 2ydy = 0$$
$$\frac{dy}{dx} = -\frac{x}{y}$$

102.10. Derivatives of Inverse Functions

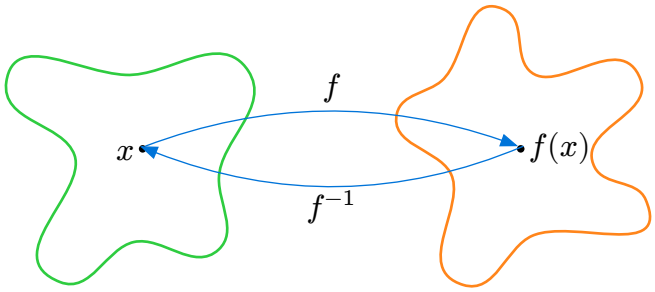
Finding the derivative of the inverse function $f^{-1}(x)$ at a given point directly from the function $f(x)$. Instead of explicitly computing the inverse function $f^{-1}(x)$, we use the inverse function derivative formula:

$$\frac{d}{dx}[f^{-1}(x)] = \frac{1}{f'(f^{-1}(x))}$$

This approach allows us to determine the derivative of the inverse function without needing to express $f^{-1}(x)$ explicitly. Instead, we find the value of x that satisfies $f(x) = a$ (where a a is the given point), evaluate $f'(x)$, and apply the formula.

1. Definition of Inverse Function

$$f(f^{-1}(x)) = x$$
$$f^{-1}(f(x)) = x$$



2. Differentiate Both Sides

Differenatiate both sides with respect to x

$$\frac{d}{dx}[f(f^{-1}(x))] = \frac{d}{dx}[x]$$

The right-hand side simplifies to:

$$\frac{d}{dx}[f(f^{-1}(x))] = 1$$

Using chain-rule:

$$\frac{d}{dx}[f(g(x))] = f'(g(x)) \cdot g'(x)$$

The left side expands as:

$$\frac{d}{dx}[f(f^{-1}(x))] = f'(f^{-1}(x)) \cdot \frac{d}{dx}[f^{-1}(x)]$$

This we get:

$$f'(f^{-1}(x)) \cdot \frac{d}{dx}[f^{-1}(x)] = 1$$

3. Solve for $\frac{d}{dx}[f^{-1}(x)]$

Rearrange to isolate $\frac{d}{dx}[f^{-1}(x)]$:

$$\frac{d}{dx}[f^{-1}(x)] = \frac{1}{f'(f^{-1}(x))}$$

Example

Given the function:

$$f(x) = x^3$$

We want to find $\frac{d}{dx} f^{-1}(x)$ at $x = 0.5$ using the inverse function derivative formula:

$$\frac{d}{dx} f^{-1}(x) = \frac{1}{f'(f^{-1}(x))}$$

---

1. Compute $f'(x)$

Differenatiate $f(x)$:

$$f'(x) = 3x^2$$

---

2. Solve for x such that $f(x) = 0.5$

We need to find x such that:

$$x^3 = 0.5$$

Solving for x:

$$x = \sqrt[3]{0.5}$$

---

3. Compute $f'\left(\sqrt[3]{0.5}\right)$

Evaluate the derivative at $x = \sqrt[3]{0.5}$:

$$f'\left(\sqrt[3]{0.5}\right) = 3\left(\sqrt[3]{0.5}\right)^2$$

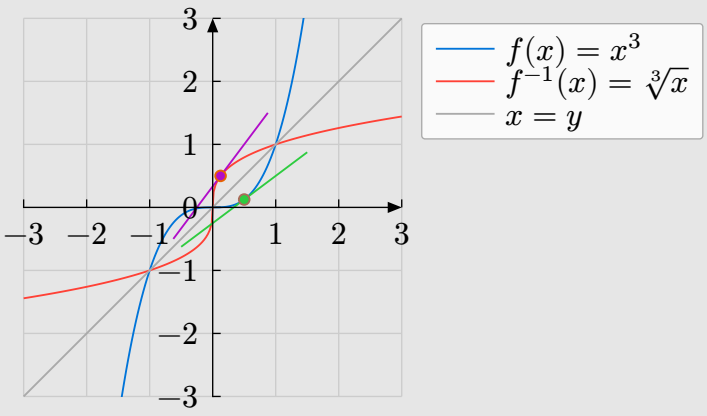
---

4. Use the formula

$$\frac{d}{dx} f^{-1}(0.5) = \frac{1}{3\left(\sqrt[3]{0.5}\right)^2}$$

---

5. Interpretation



The expression:

$$\frac{d}{dx} f^{-1}(0.5) = \frac{1}{3\left(\sqrt[3]{0.5}\right)^2}$$

represents the derivative of the inverse function $f^{-1}(x)$ evaluated at $x = 0.5$. This means it gives the slope of the tangent line to the inverse function at $x = 0.5$.

inverse.py

```python
from sympy import symbols, solve

# y = x**3 + x

x, y = symbols('x y')

f = x**3 + x - y

inverse = solve(f, x)

print(inverse)
```

102.10.1. Derivative Inverse Sin

$$\frac{d}{dx}[\sin^{-1}(x)] = \frac{1}{\sqrt{1-x^2}}$$

$$\frac{d}{dx}[\arcsin(x)] = \frac{1}{\sqrt{1-x^2}}$$

102.10.2. Derivative Inverse Cos

$$\frac{d}{dx}[\cos^{-1}(x)] = -\frac{1}{\sqrt{1-x^2}}$$

$$\frac{d}{dx}[\arccos(x)] = -\frac{1}{\sqrt{1-x^2}}$$

102.10.3. Derivative Inverse Tan

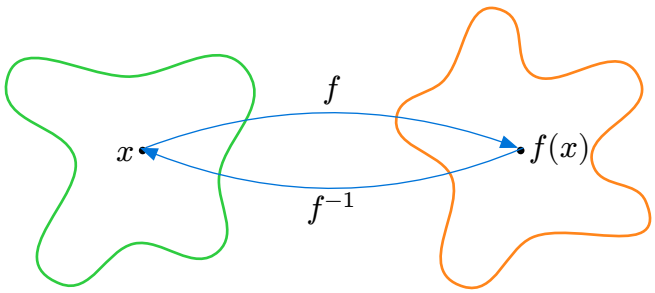
$$\frac{d}{dx}[\tan^{-1}(x)] = -\frac{1}{1+x^2}$$

$$\frac{d}{dx}[\arctan(x)] = -\frac{1}{1+x^2}$$

102.11. Inverse Functions

1. Definition

$$f(f^{-1}(x)) = x$$
$$f^{-1}(f(x)) = x$$



A function $f : A \to B$ has an inverse function f^{-1} if and only if f is **bijective** (i.e., both one-to-one and onto):

- **Injective (One-to-One)**:

  $f(x_1) = f(x_2)$ implies $x_1 = x_2$

  No two inputs map to the same output

- **Surjective (Onto)**:

  Every element in B is mapped from some element in A

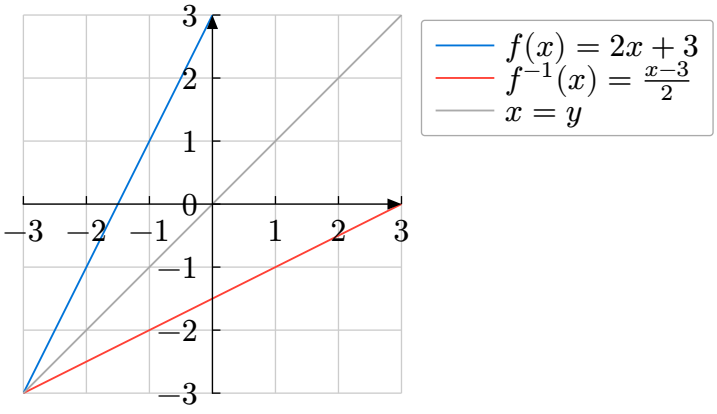
2. Finding Inverse Function

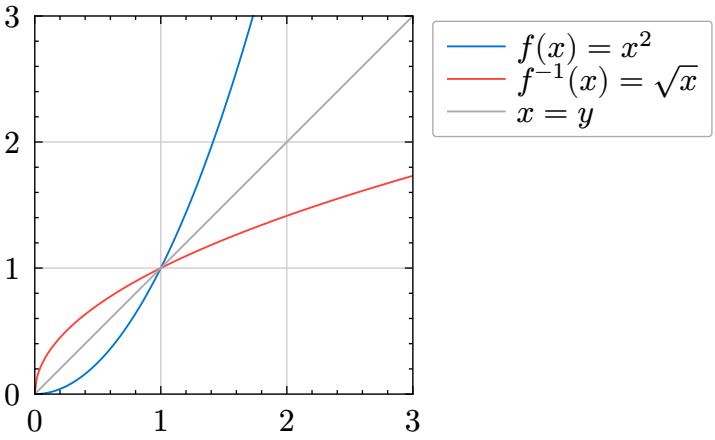
To determine f^{-1}:

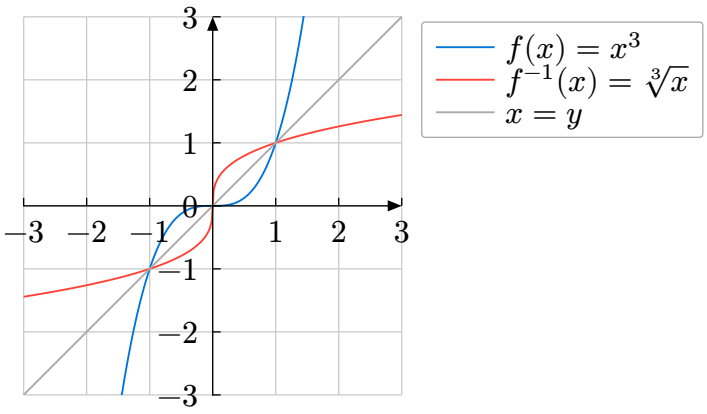
- Express y in terms of x: $y = f(x)$
- Solve for x in terms of y
- Swap x and y, remaining y as $f^{-1}(x)$

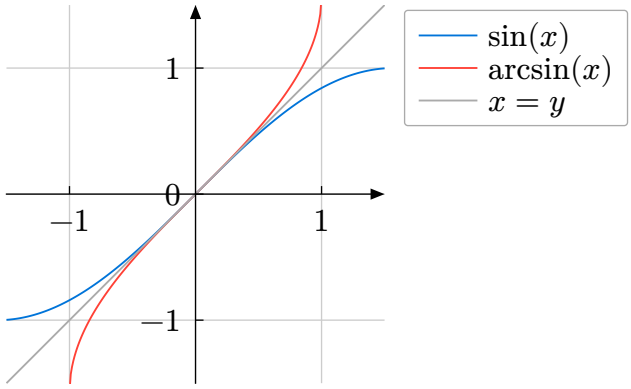
3. Graphical Representation

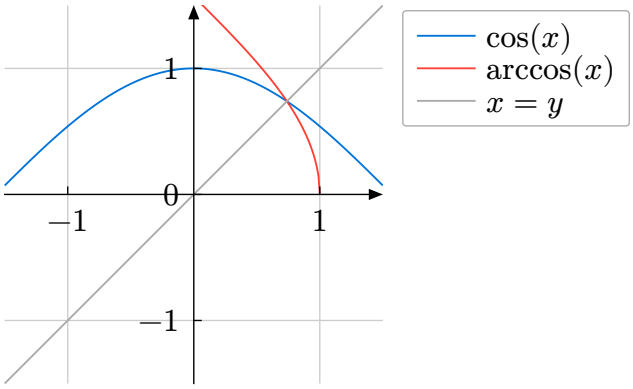
The graph of f^{-1} is a reflection of the graph of f access the line $x = y$

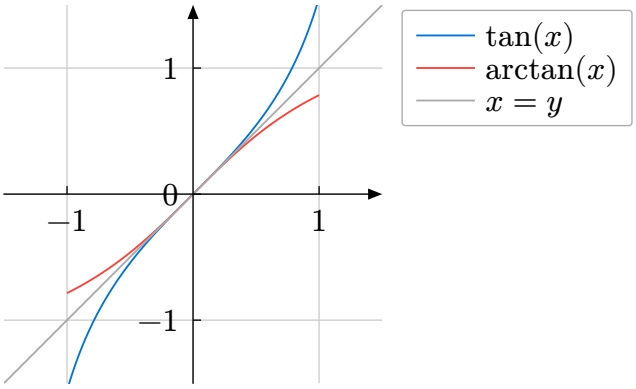



$f(x) = x^2$
$f^{-1}(x) = \sqrt{x}$
$x = y$

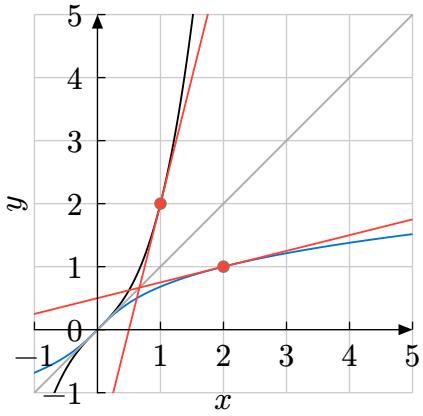

$f(x) = x^3$
$f^{-1}(x) = \sqrt[3]{x}$
$x = y$


$\sin(x)$
$\arcsin(x)$
$x = y$


$\cos(x)$
$\arccos(x)$
$x = y$



2. Derivative of Inverse Functions



102.12. L'Hôpital's Rule

Evaluating limits that result in a indeterminite form like $\frac{0}{0}$ or $\frac{\infty}{\infty}$

If $\lim_{x\to a} f(x) = 0$ and $\lim_{x\to a} g(x) = 0$ (or both go to $\pm\infty$), and $f(x)$ and $g(x)$ are differentiable near a, then:

$$\lim_{x\to a} \frac{f(x)}{g(x)} = \lim_{x\to a} \frac{f'(x)}{g'(x)}$$

Example

Consider:

$$\lim_{x\to 0} \frac{1-\cos(x)}{x^2}$$

Step 1: Direct Substitution

Substituting $x = 0$:

$$\frac{1-\cos(0)}{0^2} = \frac{1-1}{0} = \frac{0}{0}$$

Since this is an indeterminate form, we apply L'Hôpital's Rule.

Step 2: First Application of L'Hôpital's Rule

Differentiate the numerator and denominator:

- Numerator: $f(x) = 1 - \cos(x) \Rightarrow f'(x) = \sin(x)$
- Denominator: $g(x) = x^2 \Rightarrow g'(x) = 2x$

Thus, applying L'Hôpital's Rule:

$$\lim_{x \to 0} \frac{1 - \cos(x)}{x^2} = \lim_{x \to 0} \frac{\sin(x)}{2x}$$

Step 3: Second Check for Indeterminate Form

Substituting $x = 0$:

$$\frac{\sin(0)}{2(0)} = \frac{0}{0}$$

Since this is still an indeterminate form, we apply L'Hôpital's Rule again.

Step 4: Second Application of L'Hôpital's Rule

Differentiate again:

- Numerator: $f'(x) = \sin(x) \Rightarrow f''(x) = \cos(x)$
- Denominator: $g'(x) = 2x \Rightarrow g''(x) = 2$

Applying L'Hôpital's Rule again:

$$\lim_{x \to 0} \frac{\sin(x)}{2x} = \lim_{x \to 0} \frac{\cos(x)}{2}$$

Step 5: Evaluate the Limit

Now, substituting $x = 0$:

$$\frac{\cos(0)}{2} = \frac{1}{2}$$

Final Answer:

$$\boxed{\lim_{x \to 0} \frac{1 - \cos(x)}{x^2} = \frac{1}{2}}$$

102.13. Mean Value Theorem

Let $f : [a, b] \to \mathbb{R}$ be a function that satisfies the following conditions:

1. f is **continuous** on the closed interval $[a, b]$
2. f is **differentiable** on the open interval (a, b)

Then there exists at least one point $c \in (a, b)$ such that

$$f'(c) = \frac{f(b) - f(a)}{b - a}$$

This means that the instantaneous rate of change (derivative) at some point c is equal to the average rate of change over the entire interval

Example

Consider $f(x) = x^2$ on $[1, 3]$

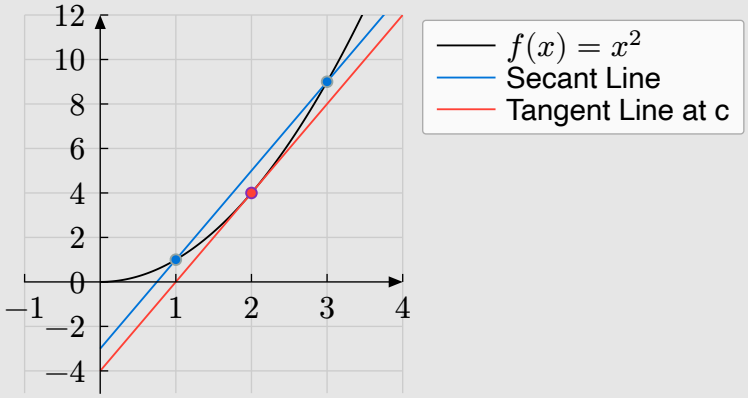
- The average rate of change is:

$$\frac{f(3) - f(1)}{3 - 1} = \frac{9 - 1}{2} = 4$$

- The derivative for $f(x)$ is $f'(x) = 2x$
- Setting $f'(c) = 4$, we solve:

$$2c = 4 \Rightarrow c = 2$$

Thus, at $c = 2$, the instantaneous rate of change matches the average rate of change



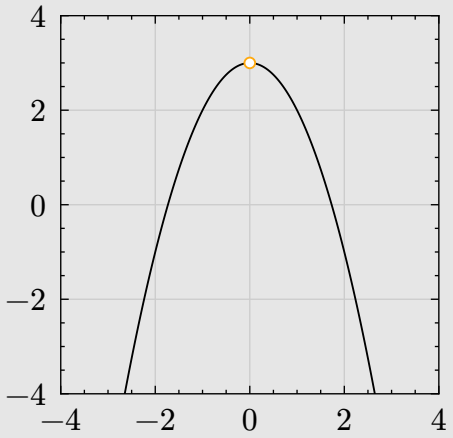
102.14. Extreme Value Theorem

The Extreme Value Theorem states that if a function $f(x)$ is continuous on a closed interval $[a, b]$, then $f(x)$ must attain both a maximum and a minimum value within that interval. This means there exist points $c, d \in [a, b]$ such that:

$$f(c) \geq f(x) \quad \text{and} \quad f(d) \leq f(x) \quad \text{for all} \quad x \in [a, b]$$

1. **Continuity**: The function must be continuous on $[a, b]$. Discontinuities (jumps, asymptotes, holes) can prevent the function from attaining an extreme value

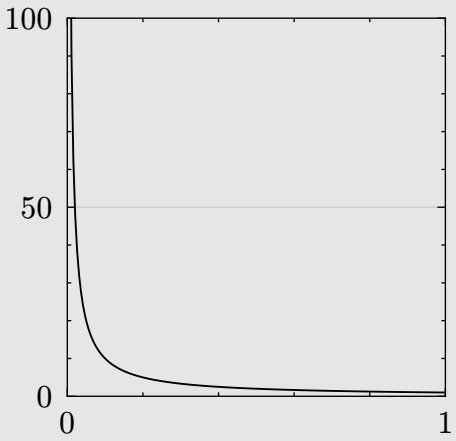
Example



2. **Closed Interval**: If the function is defined on an open interval (a, b), an extremum may not exist

Example

$f(x) = \frac{1}{x}$on $(0, 1]$ has no maximum because it keeps increasing as $x \to 0$.



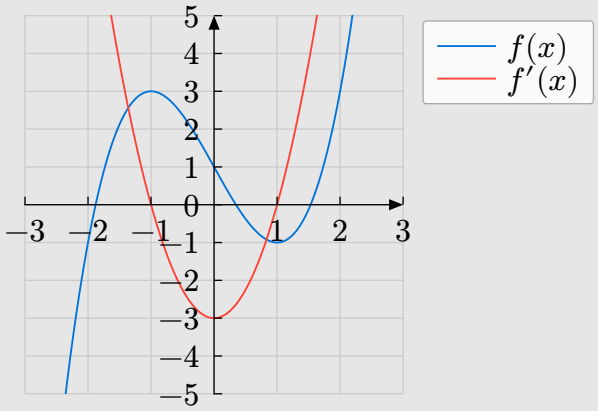
102.15. Critical Points

A critical point of a function $f(x)$ is a point in the domain where either:

- $f'(x) = 0$
- $f'(x)$ is undefined

Example

$f(x) = \frac{1}{x}$ on $(0, 1]$ has no maximum because it keeps increasing as $x \to 0$.



102.15.1. Global vs. Local Extrema

- $f(c)$ is a **relative maximum** if $f(c) \geq f(x)$ for all $x \in (c-h, c+h)$ for $h > 0$
- $f(d)$ is a **relative minimum** if $f(d) \leq f(x)$ for all $x \in (d-h, d+h)$ for $h > 0$

102.15.2. First and Second Derivative Tests

First Derivative Test and the Second Derivative Test are used to classify critical points, and both aim to determine whether the point is a local maximum, local minimum, or neither

102.16. Derivative Test

Find critical points by solving $f'(x) = 0$

1^{st} Derivative	$+ \longrightarrow -$	Local Max	Concave Down
	$- \longrightarrow +$	Local Min	Concave Up
2^{nd} Derivative	$f''(c) < 0$	Local Max	Concave Down
	$f''(c) > 0$	Local Min	Concave Up

102.16.1. First Derivative Test

If $f'(x)$ changes sign around a critical point c, we can determine if $f(c)$ is a local maximum or minimum:

- If $f'(x)$ changes from **positive to negative** at c, then $f(c)$ is a **local maximum**
- If $f'(x)$ changes from **negative to positive** at c, then $f(c)$ is a **local minimum**
- If $f'(x)$does **not** change sign, $f(c)$ is **not** a local extremum

Example

If $f'(x)$ changes from **negative to positive** at c, then $f(c)$ is a **local minimum**

Suppose:

$$f(x) = x^2, \qquad f'(x) = 2x$$

Step 1. Find critical point

Solve $f'(x) = 0$:

$$2x = 0 \implies x = 0$$

So, $x = 0$ is the only critical point

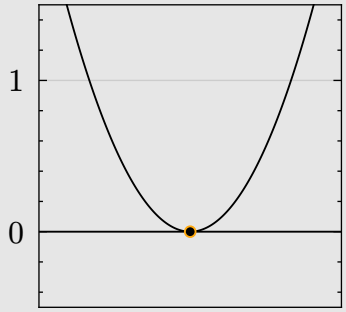
Step 2. Test sign of $f'(x)$ around $x = 0$

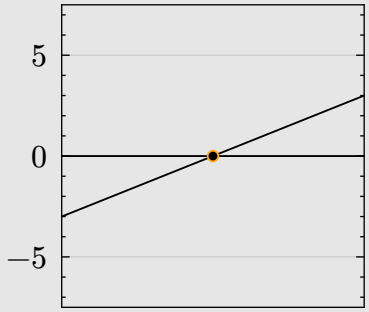
- **Left**: $x = -1$

$$f'(-1) = 2(-1) = -2 < 0 \implies f \text{ is } \mathbf{decreasing}$$

- **Right**: $x = 1$:

$$f'(1) = 2(1) = 2 > 0 \implies f \text{ is } \mathbf{increasing}$$





Step 3. Interpretation

Since $f'(x)$ changes from **negative** to **positive** at $x = 0$,

$$f(0) = 0 \implies \mathbf{local\ minimum}$$

Example

If $f'(x)$ changes from **positive to negative** at c, then $f(c)$ is a **local maximum**

Suppose:

$$f(x) = -x^2, \qquad f'(x) = -2x$$

Step 1. Find critical point

Solve $f'(x) = 0$

$$-2x = 0 \implies x = 0$$

So, $x = 0$ is the only critical point

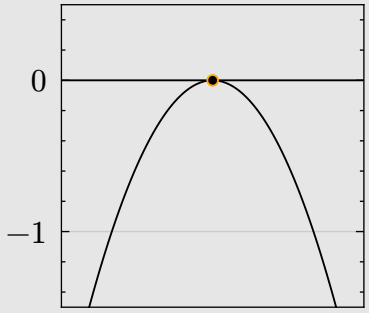
Step 2. Test sign of $f'(x)$ around $x = 0$

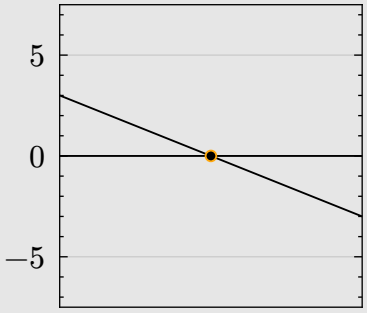
- **Left**: $x = -1$

$$f'(-1) = -2(-1) = 2 > 0 \quad \Longrightarrow \quad f \text{ is } \mathbf{increasing}$$

- **Right**: $x = 1$

$$f'(1) = -2(1) = -2 < 0 \quad \Longrightarrow \quad f \text{ is } \mathbf{decreasing}$$





Step 3. Interpretation

Since $f'(x)$ changes from **positive** to **negative** at $x = 0$,

$$f(0) = 0 \quad \Longrightarrow \mathbf{local\ maximum}$$

Example

If $f'(x)$does **not** change sign, $f(c)$ is **not** a local extremum

Suppose

$$f(x) = x^3, \qquad f'(x) = 3x^2$$

Step 1. Find critical point

Solve $f'(x) = 0$

$$3x^2 = 0 \quad \Longrightarrow \quad x = 0$$

So, $x = 0$ is the only critical point

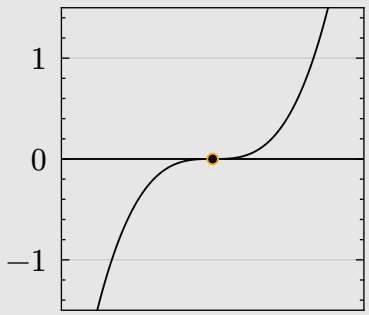
Step 2. Test sign of $f'(x)$ around $x = 0$

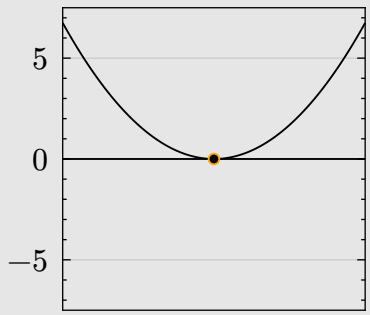
- **Left**: $x = -1$

$$f'(-1) = 3(-1)^2 = 3 > 0 \quad \Longrightarrow \quad f \text{ is } \mathbf{increasing}$$

- **Right**: $x = 1$

$$f'(1) = 3(1)^2 = 3 > 0 \quad \Longrightarrow \quad f \text{ is } \mathbf{increasing}$$





Step 3. Interpretation

Since $f'(x)$ **does not change sign** at $x = 0$,

$$f(0) = 0 \quad \Longrightarrow \quad \textbf{not a local extremum}$$

but rather a point of inflection

102.16.2. Second Derivative Test

If $f''(x)$ is continuous near a critical point c, and $f'(c) = 0$, then:

- If $f''(c) > 0$, then $f(c)$ is a **local minimum** (concave up)
- If $f''(c) < 0$, then $f(c)$ is a **local maximum** (concave down)
- If $f''(c) = 0$, the test is **inconclusive** — use the first derivative test or other methods

Example

If $f''(c) > 0$, then $f(c)$ is a **local minimum** (concave up)

Suppose

$$f(x) = x^2, \qquad f'(x) = 2x, \qquad f''(x) = 2$$

Step 1. Find critical points

Solve $f'(x) = 0$

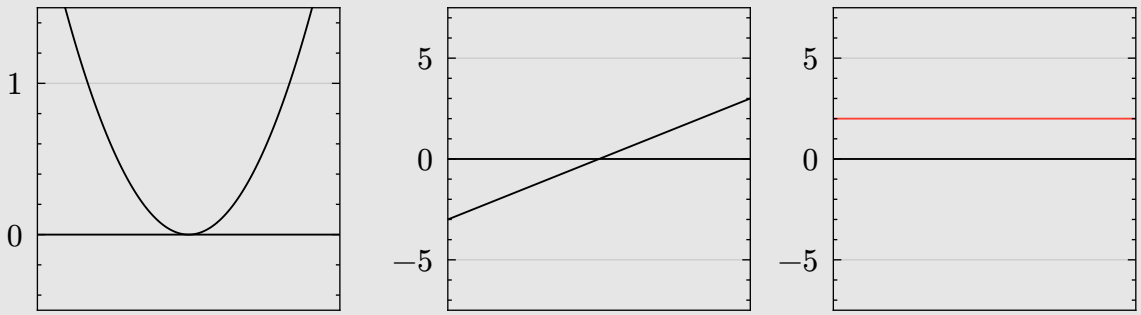
$$2x = 0 \quad \Longrightarrow \quad x = 0$$

So, $x = 0$ is the only critical point

Step 2. Evaluate the second derivative at $x = 0$

$$f''(0) = 2 > 0$$

Since the second derivative is positive, the function is **concave up** near $x = 0$



Step 3. Interpretation

$$f(0) = 0 \implies \textbf{local minimum}$$

Example

If $f''(c) < 0$, then $f(c)$ is a **local maximum** (concave down)

Suppose

$$f(x) = -x^2, \qquad f'(x) = -2x, \qquad f''(x) = -2$$

Step 1. Find critical points

Solve $f'(x) = 0$

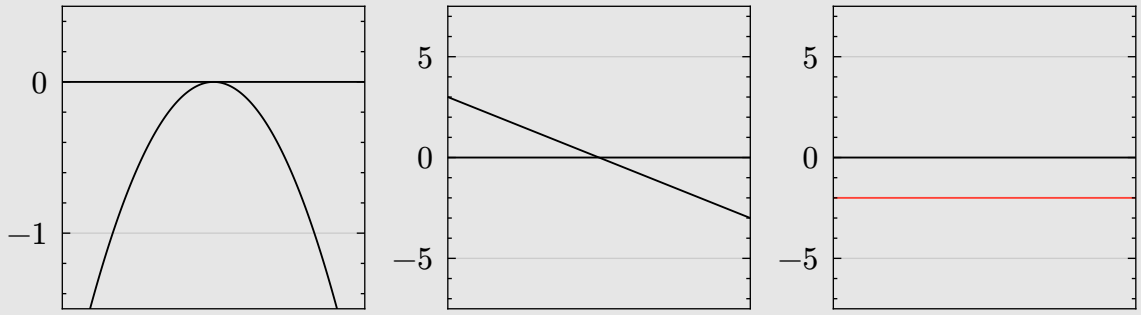
$$-2x = 0 \implies x = 0$$

So, $x = 0$ is the only critical point

Step 2. Evaluate the second derivative at $x = 0$

$$f''(0) = -2 < 0$$

Since the second derivative is negative, the function is **concave down** near $x = 0$



Step 3. Interpretation

$$f(0) = 0 \implies \textbf{local maximum}$$

Example

If $f''(c) = 0$, the test is **inconclusive** — use the first derivative test or other methods

Suppose

$$f(x) = x^3, \qquad f'(x) = 3x^2, \qquad f''(x) = 6x$$

Step 1. Find critical points

Solve $f'(x) = 0$

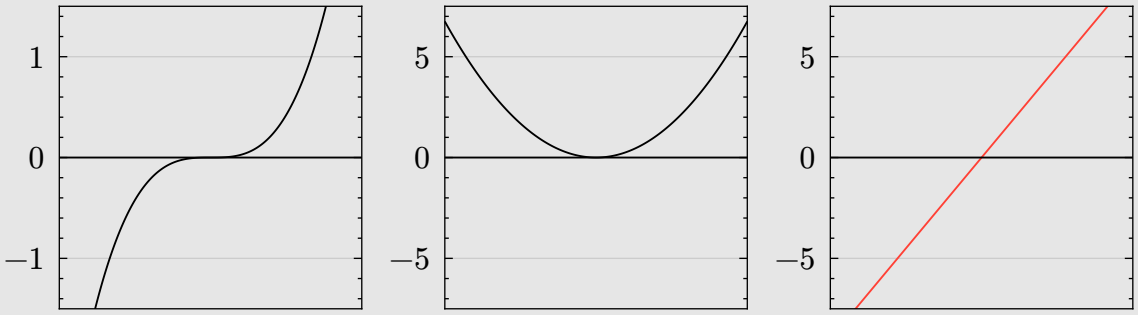
$$3x^2 = 0 \implies x = 0$$

So, $x = 0$ is the only critical point

Step 2. Evaluate the second derivative at $x = 0$

$$f''(0) = 6(0) = 0$$

Since the second derivative is 0, the test in **inconclusive**



Step 3. Use another method

Test sign of $f'(x)$ around $x = 0$

- **Left**: $x = -1$

$$f'(-1) = 3(-1)^2 = 3 > 0 \implies f \text{ is } \mathbf{increasing}$$

- **Right**: $x = 1$

$$f'(1) = 3(1)^2 = 3 > 0 \implies f \text{ is } \mathbf{increasing}$$

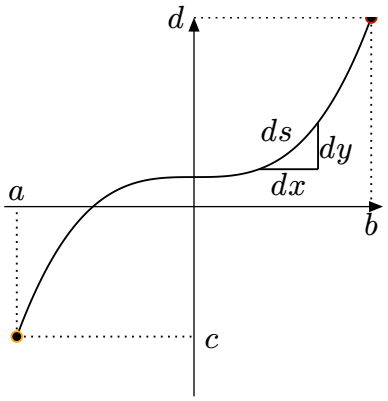
Step 4. Interpretation

Since $f'(x)$ **does not change sign** at $x = 0$,

$$f(0) = 0 \implies \mathbf{not\ a\ local\ extremum}$$

but rather a point of inflection

102.17. Arc Length



$$ds = \sqrt{(dx)^2 + (dy)^2}$$

$$s = \int ds = \int \sqrt{(dx)^2 + (dy)^2}$$

$$\int_a^b \sqrt{1 + \left(\frac{dy}{dx}\right)^2} \quad dx$$

$$\int_c^d \sqrt{\left(\frac{dy}{dx}\right)^2 + 1} \quad dy$$

102.18. Series

Given the series $(a_n) = a_1 + a_2 + ...$, then

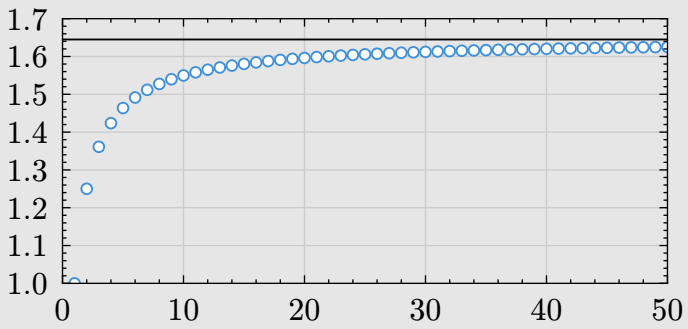
$$\sum_{n=1}^{\infty} a_n = a_1 + a_2 + ...$$

We form a sequence associated with the series, the sequence (s_n) of its **partial sums**:

$$\begin{aligned} s_1 &= a_1 \\ s_2 &= a_1 + a_2 \\ s_3 &= a_1 + a_2 + a_3 \\ &\vdots \\ s_n &= a_1 + a_2 + ... + a_n \\ &\vdots \end{aligned}$$

Example

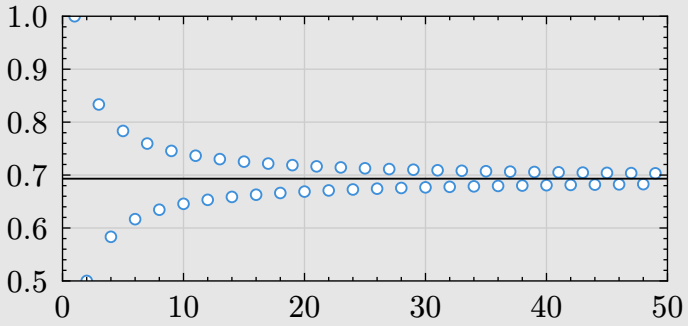
$$\sum_{n=1}^{\infty} \frac{1}{n^2}$$



102.18.1. Alternating

Example

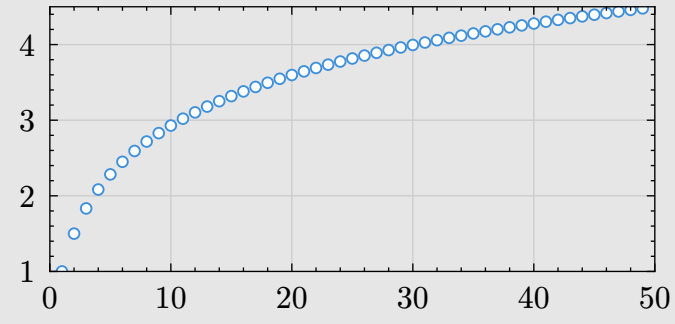
$$\sum_{n=1}^{\infty} \frac{(-1)^{n+1}}{n}$$



102.18.2. Non-Convergent

Example

$$\sum_{n=1}^{\infty} \frac{1}{n}$$



102.19. Taylor Series

Taylor Series (Expansion)

Taylor polynomial

Approximate a complicated function near a point a by a polynomial made from the function's derivatives at that point

$$f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!}(x-a)^n$$

$$f(x) = \underbrace{\underbrace{\underbrace{f(x_0)}_{*} + f'(x_0)(x-x_0)}_{**} + \frac{1}{2!}f''(a)(x-a) + ... + \frac{1}{n!}f^{(n)}(x_0)(x-a)}_{***},$$

Where:

- $*$: Ensures value of the polynomial matches with the value of f when $x = a$
- $**$: Ensures slope of the polynomial matches with the slope of f when $x = a$
- $***$: Ensures rate of change of the polynomial matches with the rate of change of f when $x = a$

Where:

- $f^{(n)}(a)$: n-th derivative of f evaluated at a

Example

Setup

$$f(x) = x^2$$

Expand it around the point $a = 1$

Step 1: Compute Derivatives

- $f(x) = x^2$
- $f'(x) = 2x$
- $f''(x) = 2$

Step 2: Evaluate at $x = 1$

- $f(1) = 1^2 = 1$
- $f'(1) = 2 \cdot 1 = 2$
- $f''(1) = 2$

Step 3: Write the Taylor expansion

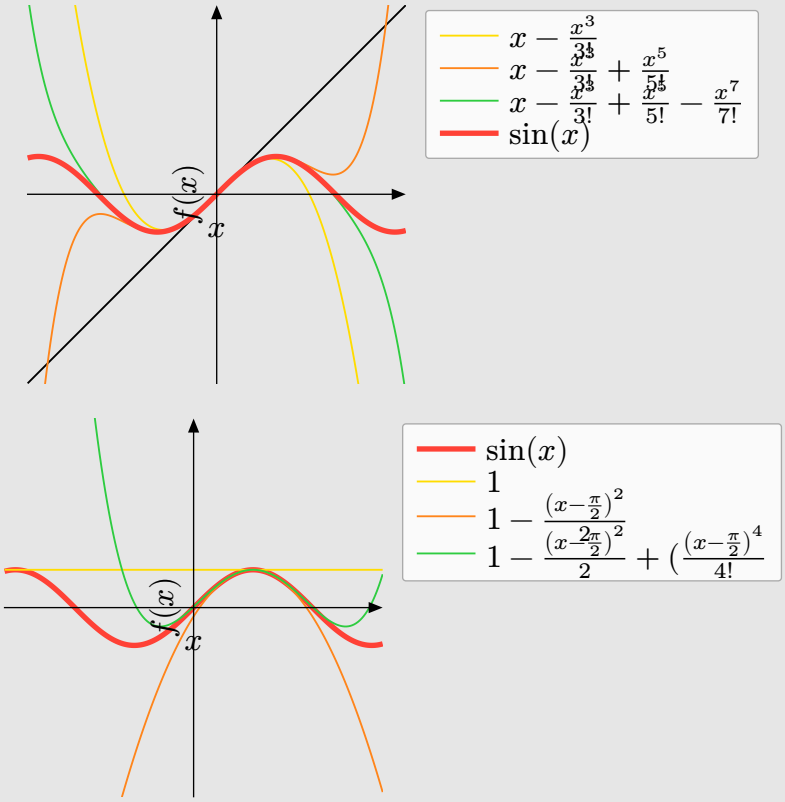
$$\begin{aligned} f(x) &= f(1) + f'(1)(x-1) + \frac{f''(1)}{2!}(x-1)^2 \\ &= 1 + 2(x-1) + \frac{2}{2}(x-1)^2 \\ &= 1 + 2(x-1) + (x-1)^2 \end{aligned}$$

Step 3: Expand

$$1 + 2(x-1) + (x-1)^2 = 1 + 2x - 2 + x^2 + 1 = x^2$$

It gives back the original function exactly

Example



Resources

- Tyler Series (3B1B)

103. Calculus II

103.1. Cheatsheet

103.1.1. Derivative Objects

Gradient	$f : \mathbb{R}^n \to \mathbb{R}$	$n \times 1$	Vector of first derivatives; special case of Jacobian (scalar output)

Jacobian	$F : \mathbb{R}^n \to \mathbb{R}^m$	$m \times n$	Matrix of first derivatives; general case for vector-valued functions
Hessian	$f : \mathbb{R}^n \to \mathbb{R}$	$n \times n$	Matrix of second derivatives; Jacobian of the gradient

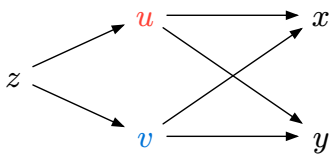
	Input/Output	Derivatives	Notation
Gradient (Derivative) (1×1)	$f : \mathbb{R} \to \mathbb{R}$ One Input **One Output** Scalar Function	**First** Order	$\nabla f(x) = f'(x) = \left[\frac{\partial f}{\partial x}\right]$
Jacobian $(m \times 1)$	$f : \mathbb{R} \to \mathbb{R}^m$ One Input **Multiple Output** Vector Function	**First** Order	$J_f(x) = \begin{bmatrix} \frac{\partial f_1}{\partial x} \\ \frac{\partial f_2}{\partial x} \\ \vdots \\ \frac{\partial f_m}{\partial x} \end{bmatrix}$
Gradient $(n \times 1)$	$f : \mathbb{R}^n \to \mathbb{R}$ Multiple Input **One Output** Scalar Function	**First** Order	$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}$
Jacobian $(m \times n)$	$f : \mathbb{R}^n \to \mathbb{R}^m$ Multiple Input **Multiple Output** Vector Function	**First** Order	$J_f(\mathbf{x}) = \begin{bmatrix} \nabla g_1(x)^T \\ \nabla g_2(x)^T \\ \vdots \\ \nabla g_m(x)^T \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$
Hessian $(n \times n)$	$f : \mathbb{R}^n \to \mathbb{R}$ Multiple Input One Output Scalar Function	**Second** Order	$\nabla^2 f(\mathbf{x}) = H_f(\mathbf{x}) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$
Laplacian	Trace of the Hessian (sum of diagonal entries)		$\operatorname{tr}(H_f(\mathbf{x})) = \sum_{i=1}^{n} \frac{\partial^2 f}{\partial x_i^2}(x)$

			$H_f(\mathbf{x}) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$

Gradient	1^{st} derivative	$n \times 1$
Hessian	2^{nd} derivative	$n \times n$
3^{rd} Order Tensor	3^{rd} derivative	$n \times n \times n$
$\vdots$		
k-th Order Tensor	k^{th} derivative	$n \times n \times ... \times n \quad (k \text{ times})$

103.2. Chain Rule

$$z = f(u, v)$$
$$u = f(x, y) \qquad v = f(x, y)$$



The derivative of z with respect to x is the sum of all possible paths from z to x:

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial u}\frac{\partial u}{\partial x} + \frac{\partial z}{\partial v}\frac{\partial v}{\partial x}$$

Example

Let:

- $u = x + y$
- $v = xy$
- $z = u^2 + \sin(v)$

We want to compute:

$$\frac{\partial z}{\partial x}$$

Step 1: Apply the multivariable chain rule

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial u}\frac{\partial u}{\partial x} + \frac{\partial z}{\partial v}\frac{\partial v}{\partial x}$$

Step 2: Compute partial derivatives

- $\frac{\partial u}{\partial x} = \frac{\partial}{\partial x}(x+y) = 1 + 0 =$ $\boxed{1}$
- $\frac{\partial v}{\partial x} = \frac{\partial}{\partial x}(xy) = 1y =$ $\boxed{\text{y}}$
- $\frac{\partial z}{\partial u} = \frac{\partial}{\partial u}(u^2 + \sin(v)) = 2u + 0 =$ $\boxed{\text{2u}}$
- $\frac{\partial z}{\partial v} = \frac{\partial}{\partial v}(u^2 + \sin(v)) = 0 + \cos(v) =$ $\boxed{\cos(v)}$

Step 3: Plug in:

$$\begin{aligned}\frac{\partial z}{\partial x} &= 2u \cdot 1 + \cos(v) \cdot y \\ &= 2(x+y) + y\cos(xy)\end{aligned}$$

103.3. Double Integrals

103.3.1. 1. Double Indefinite Integral

$$\int\int f(x,y)\,\mathrm{d}x\,\mathrm{d}y$$

First Integration:

Integrate with respect to x (treat y as constant):

$$\int f(x,y)\,\mathrm{d}x = F(x,y) + C_1(y)$$

- $F(x,y)$: antiderivative of f w.r.t. x
- $C_1(y)$: Constant of integration (can depend on y)

Second Integration:

Integrate with respect to y (treat x as constant):

$$\int [F(x,y) + C_1(y)]\,\mathrm{d}y = G(x,y) + C_2(x)$$

- $G(x,y)$: antiderivative of $F(x,y)$ with respect to y

Final Output:

A family of functions $G(x,y) + C_2$, representing all functions whose mixed partial derivative $\frac{\partial^2}{\partial y \partial x}$ is $f(x,y)$

103.4. Gradient

The gradient $\nabla f(x,y)$ is a vector of expressions (equations)

If

$$f : \mathbb{R}^n \to \mathbb{R}$$

The gradient is always **a vector in the same space as the input variables**

$$\nabla f(x) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix} \in \mathbb{R}^n$$

Example

$$f(x, y) = x^2 + xy + y^2$$

$$\nabla f(x, y) = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix} = \begin{bmatrix} 2x + y \\ x + 2y \end{bmatrix}$$

Evaluating at a point

If you plug in numbers for x and y, each component becomes a scalar:

$$(x, y) = (1, 2) \quad \Rightarrow \quad \nabla f(1, 2) = \begin{bmatrix} 2 \cdot 1 + 2 \\ 1 + 2 \cdot 2 \end{bmatrix} = \begin{bmatrix} 4 \\ 5 \end{bmatrix}$$

103.5. Hessian Matrix

Square matrix of second-order partial derivatives

Describes the local curvature of a multivariable function

$$f : \mathbb{R}^n \to \mathbb{R}$$

Input: $\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$

Output: Scalar

$$H_f(\mathbf{x}) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n^2 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$

Diagonal entries: $\frac{\partial^2 f}{\partial x_i^2}$

Second derivative of f with respect to a single variable twice

How the slope of f changes as you move along the x_1 direction

Off-diagonal entries: $\frac{\partial^2 f}{\partial x_i \partial x_j}$ where $i \neq j$

How the slope in one direction changes when you move in a different direction

$\frac{\partial^2 f}{\partial x_1 \partial x_2}$: How the rate of change of f along x_1 changes as you move in the x_2 direction

Symmetric (Clairaut-Schwarz theorem):

$$\frac{\partial^2 f}{\partial x_i \partial x_j} = \frac{\partial^2 f}{\partial x_j \partial x_i}$$

$$H_f(x) = H_f(x)^T$$

Example

$$f(x, y) = x^3 y + y^2$$

- Symbolic Hessian

$$\nabla^2 f(x, y) = \begin{bmatrix} \frac{\partial^2 f}{\partial x^2} & \frac{\partial^2 f}{\partial x \partial y} \\ \frac{\partial^2 f}{\partial y \partial x} & \frac{\partial^2 f}{\partial y^2} \end{bmatrix} = \begin{bmatrix} 6xy & 3x^2 \\ 3x^2 & 2 \end{bmatrix}$$

- Evaluate at $(x, y) = (1, 2)$

$$\nabla^2 f(1, 2) = \begin{bmatrix} 6 \cdot 1 \cdot 2 & 3 \cdot 1^2 \\ 3 \cdot 1^2 & 2 \end{bmatrix} = \begin{bmatrix} 12 & 3 \\ 3 & 2 \end{bmatrix}$$

Each entry is now a scalar, and the Hessian is numeric

103.6. Jacobian

For a vector valued function

$$g(x) = \begin{bmatrix} g_1(x) \\ g_2(x) \\ \vdots \\ g_m(x) \end{bmatrix} \quad g : \mathbb{R}^n \to \mathbb{R}^m$$

The Jacobian:

$$J_g(x) = \begin{bmatrix} \nabla g_1(x)^T \\ \nabla g_2(x)^T \\ \vdots \\ \nabla g_m(x)^T \end{bmatrix}$$

103.7. Semi-Definiteness

103.7.1. Positive/Negative Definite/Indefinite/Semidefinite Matrices

Let $\mathbf{x} \in \mathbb{R}^n$ be any nonzero vector. Look at the quadratic form:

$$\mathbf{x}^T H \mathbf{x}$$

$q(x) = \mathbf{x}^T H \mathbf{x}$ $H \mathbf{x} = \lambda \mathbf{x}$			
	Definition	**Eigen Values**	**Geometry**
Positive Definite	$q(\mathbf{x}) > 0 \quad \forall \mathbf{x} \neq \mathbf{0}$	$\lambda_i > 0$ $H \succ 0$	**Min** (bowl-shaped)
Negative Definite	$q(\mathbf{x}) < 0 \quad \forall \mathbf{x} \neq \mathbf{0}$	$\lambda_i < 0$ $H \prec 0$	**Max** (inverted bowl)
Positive Semidefinite	$q(\mathbf{x}) \geq 0 \quad \forall \mathbf{x}$	$\lambda_i \geq 0$ $H \succeq 0$	**Min** but **flat** in some directions
Negative Semidefinite	$q(\mathbf{x}) \leq 0 \quad \forall \mathbf{x}$	$\lambda_i \leq 0$ $H \preceq 0$	**Max** but **flat** in some directions
Indefinite	some $q(\mathbf{x_1}) > 0$ some $q(\mathbf{x_2}) < 0$	some $\lambda_i > 0$ some $\lambda_j < 0$	**Saddle** Up in some directions Down in others

104. Calculus III

104.1. Vector-Valued Functions

105. Integration

105.1. Rules

105.1.1. Integral

Measures the accumulation of quantities

Integral (antiderivative) of a function $f(x)$

$$F(x) = \int f(x)dx$$

Area under a curve between two points

105.1.1.1. Power Rule

105.1.1.2. Sum & Difference Rule

105.1.1.3. Constant Multiple Rule

105.1.1.4. Integration by Substitution

105.1.1.5. Integration by Parts

105.1.1.6. Trigonometric Integrals

105.1.1.7. Exponential and Logarithmic Integrals

105.1.1.8. Definite Integrals of Even and Odd Functions

105.1.1.9. Improper Integrals

105.2. Leibniz's Rule

Differentiation under the integral sign. When the integrand **and** the limits depend on a parameter Q, the derivative splits into three pieces: one from the integrand changing, two from the moving limits.

$$\frac{\mathrm{d}}{\mathrm{d}Q}\int_{a(Q)}^{b(Q)} h(x,Q)\,\mathrm{d}x = \underbrace{\int_{a(Q)}^{b(Q)} \frac{\partial h}{\partial Q}\,\mathrm{d}x}_{\text{integrand varies}} + \underbrace{h(b(Q),Q)\cdot b'(Q)}_{\text{upper limit moves}} - \underbrace{h(a(Q),Q)\cdot a'(Q)}_{\text{lower limit moves}}$$

Reading the three terms:

- **Integrand term** — freeze the limits, differentiate inside.
- **Upper-limit term** — rate the top edge sweeps out new area: h at $x = b(Q)$ times $b'(Q)$.

- **Lower-limit term** — area lost at the bottom edge, subtracted.

105.2.1. Special cases

- **Fixed limits** (a, b constant) — only the integrand term survives:

$$\frac{\mathrm{d}}{\mathrm{d}Q}\int_a^b h(x,Q)\,\mathrm{d}x = \int_a^b \frac{\partial h}{\partial Q}\,\mathrm{d}x$$

- **Constant integrand, moving upper limit** (the fundamental theorem):

$$\frac{\mathrm{d}}{\mathrm{d}Q}\int_a^Q h(x)\,\mathrm{d}x = h(Q)$$

Example

The newsvendor overage term $\int_0^Q (Q-x)f(x)\,\mathrm{d}x$ has $h(x,Q) = (Q-x)f(x)$, upper limit $b(Q) = Q$, lower limit fixed:

$$\frac{\mathrm{d}}{\mathrm{d}Q}\int_0^Q (Q-x)f(x)\,\mathrm{d}x = \int_0^Q f(x)\,\mathrm{d}x + \underbrace{(Q-Q)f(Q)}_{=0} = F(Q)$$

The boundary term vanishes because the integrand is zero at $x = Q$.

105.3. Leibniz's Rule

$$\frac{\mathrm{d}}{\mathrm{d}Q}\int_{a(Q)}^{b(Q)} h(x,Q)\,\mathrm{d}x = \underbrace{\int_{a(Q)}^{b(Q)} \frac{\partial h}{\partial Q}\,\mathrm{d}x}_{\text{swap}} + \underbrace{h(b,Q)\cdot b'(Q) - h(a,Q)\cdot a'(Q)}_{\text{boundary term}}$$

106. Differential Equations

A differential equation specifies a relationship between an unknown function and its derivatives

Solving for (an) unknown function(s) that satisfy this relationship

Initial or boundary conditions to select a unique solution

106.0.1. Linearity

The **dependent** variable and its derivatives each appear only to the first power, are not multiplied together, and are not inside any other function

Example

Linear

$$\frac{\mathrm{d}y}{\mathrm{d}t} + y = 0$$

Non-Linear

$$\frac{\mathrm{d}y}{\mathrm{d}t} + y^2 = 0$$
$$\frac{\mathrm{d}y}{\mathrm{d}t} + \sin(y) = 0$$
$$\frac{\mathrm{d}y}{\mathrm{d}t} + e^y = 0$$

106.0.2. Homogeneity

- **Homogeneous**: No terms with just the **independent** variable of **constant**
- **Nonhomogeneous**: Has term(s) with just the **independent** variable or **constants**

Example

Homogeneous

$$\frac{\mathrm{d}y}{\mathrm{d}t} + y = 0$$

Non-Homogeneous

$$\frac{\mathrm{d}y}{\mathrm{d}t} + y = 1$$
$$\frac{\mathrm{d}y}{\mathrm{d}t} + y = \sin(t)$$
$$\frac{\mathrm{d}y}{\mathrm{d}t} + y = e^t$$

106.0.3. Order & Degree

- Order: highest derivative
- Degree: exponent on highest derivative

Example

Order

$$\frac{\mathrm{d}y}{\mathrm{d}x} + y = x \qquad \text{1st order}$$
$$\frac{\mathrm{d}^2y}{\mathrm{d}x^2} + y = x \qquad \text{2nd order}$$

Degree

$$\left(\frac{\mathrm{d}^2y}{\mathrm{d}x^2}\right)^3 + y = x \qquad \text{3rd degree}$$

106.0.4. Autonomous

The **independent** variable does not appear explicitly in the equation (rate of change depends only on the dependent variable)

Example

- Autonomous

$$\frac{\mathrm{d}y}{\mathrm{d}t} = y$$

- Non-Autonomous

$$\frac{\mathrm{d}y}{\mathrm{d}t} = y + t$$

106.0.5. Ordinary Differential Equation (ODE)

- **One** independent variable
- Involves **ordinary derivatives** with respect to that variable

106.0.6. Partial Differential Equation (PDE)

- **Multiple** independent variables
- Involves **partial derivatives** with respect to those variables

106.0.7. Summary

- Inputs
  - Equation itself
  - Initial/boundary conditions (optional)
- Outputs
  - Function(s) that satisfy it
- Defined for **continuous variables**

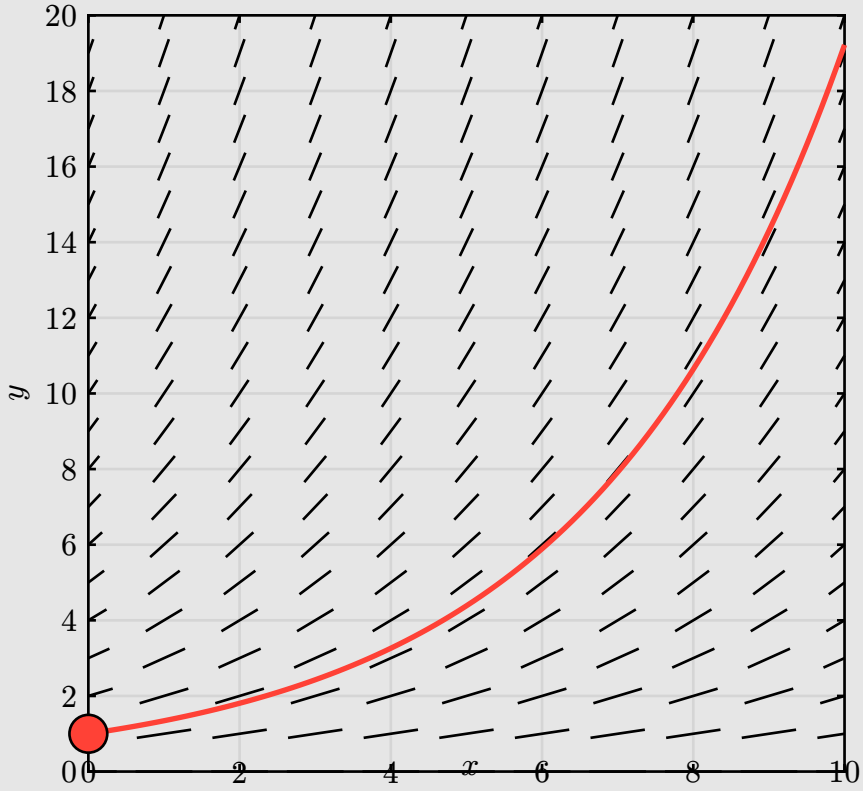
Example

Non-coupled ODE

Exponential Population Growth

$$\frac{\mathrm{d}}{\mathrm{d}t}P(t) = r \cdot P(t)$$

- $P(t)$: population at time t
- $\frac{\mathrm{d}}{\mathrm{d}t}P(t)$: rate of change of the population at time t
- r: growth rate constant
- $r \cdot P(t)$: growth contribution proportional to the current population



Example

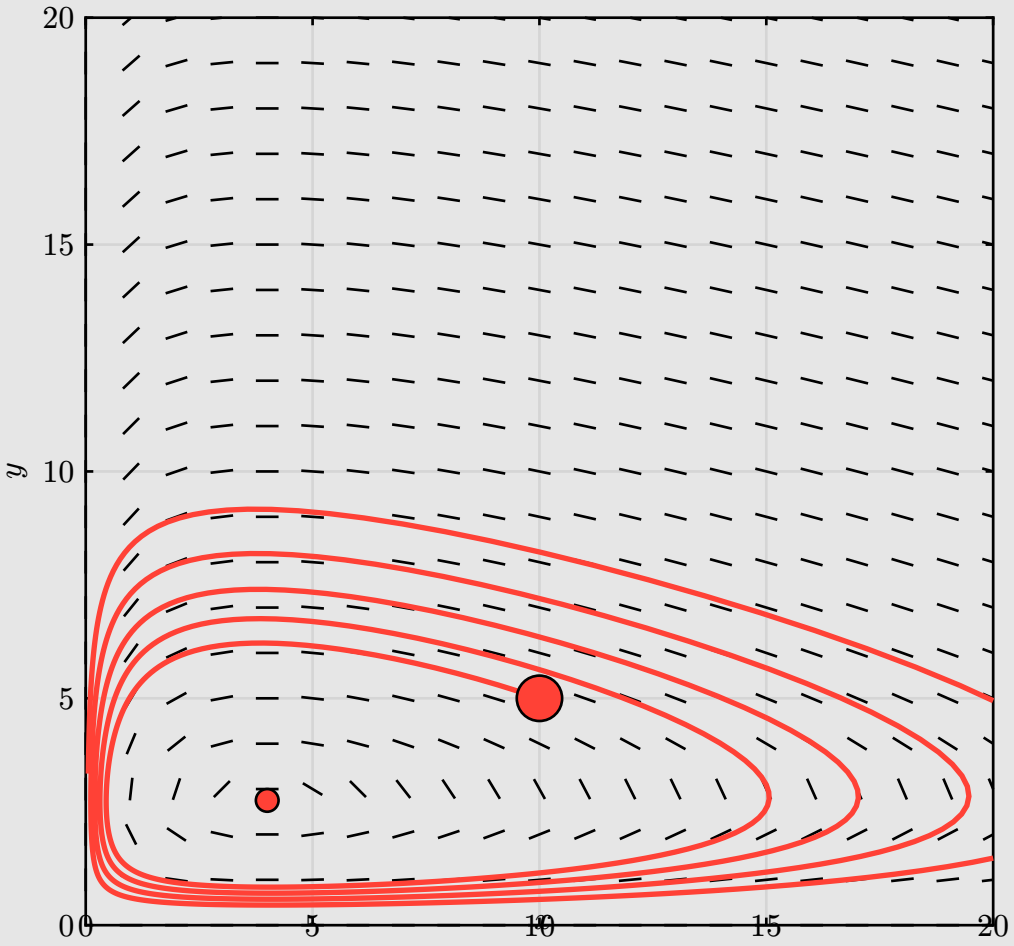
Coupled ODE

Lotka-Volterra ODEs (predator-prey)

$$\frac{\mathrm{d}}{\mathrm{d}t}x(t) = \alpha x(t) - \beta x(t)y(t)$$
$$\frac{\mathrm{d}}{\mathrm{d}t}y(t) = \delta x(t)y(t) - \gamma y(t)$$

- $x(t)$ = prey population
- $y(t)$ = predator population
- $\frac{\mathrm{d}}{\mathrm{d}t}x(t)$: The rate of change of the prey population at time t
- $\frac{\mathrm{d}}{\mathrm{d}t}y(t)$: The rate of change of the predator population at time t
- α: Prey growth rate — how fast the prey multiply in the absence of predators

- β: Predation rate — how effectively predators consume prey
- δ: Predator growth rate — how much predator population increases when consuming prey
- γ: Predator death rate — how fast predators die without enough food
- $\alpha x(t)$: prey naturally reproduce at a rate proportional to how many exist now
- $-\beta x(t)y(t)$: prey lost to predation, proportional to encounters with predators
- $\delta x(t)y(t)$: predators grow in number based on how much prey they consume
- $-\gamma y(t)$: predators die naturally when there isn't enough food

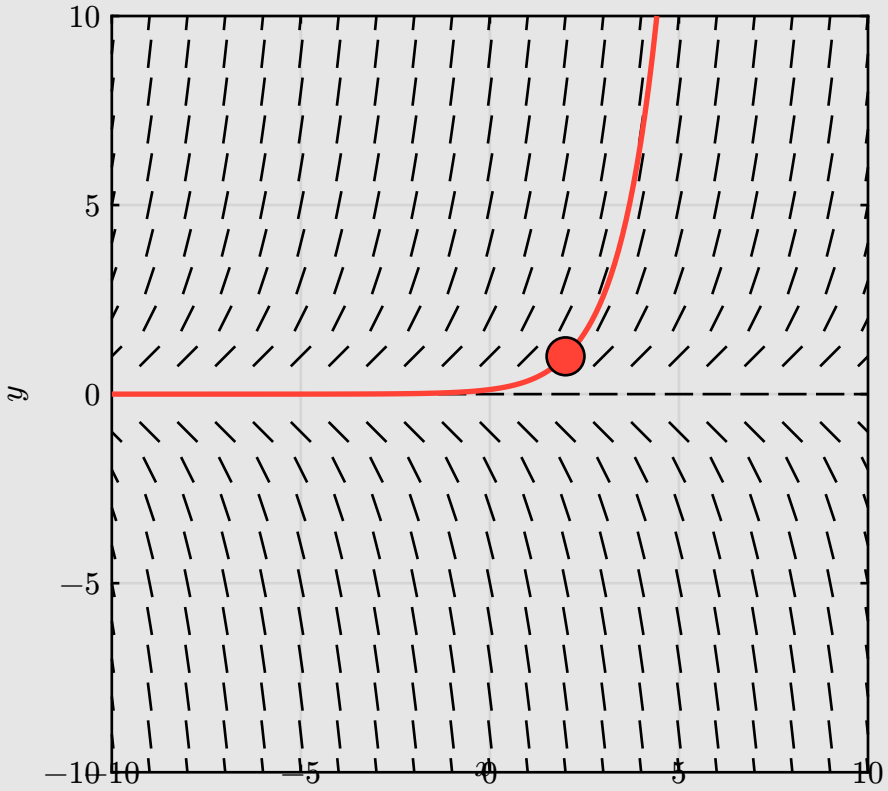


106.1. Slope Field (Direction Field)

Example

IVP (Initial Value Problem)

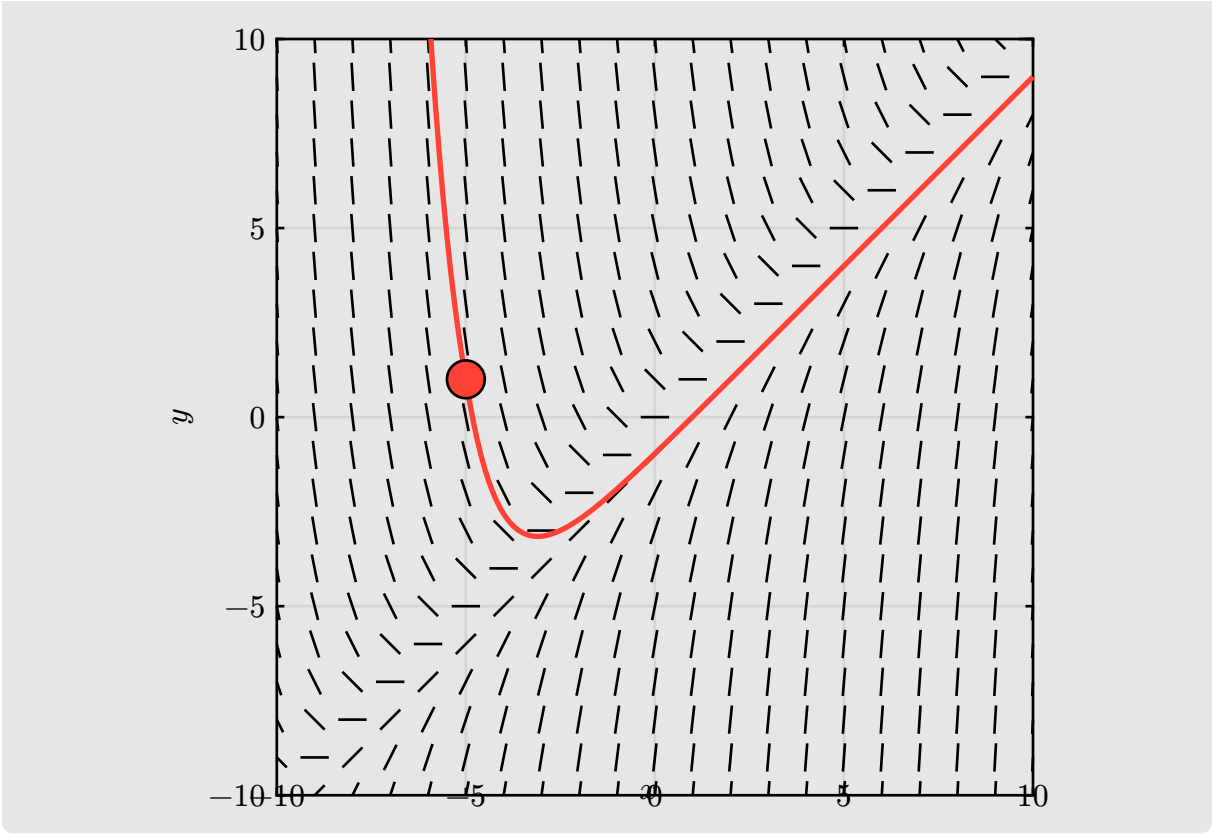
$$\begin{cases} \frac{\mathrm{d}y}{\mathrm{d}x} = y \\ y(2) = 1 \end{cases}$$



Example

IVP (Initial Value Problem)

$$\begin{cases} \frac{\mathrm{d}y}{\mathrm{d}x} = x - y \\ y(-5) = 1 \end{cases}$$


10
5
0
−5
−10
−10
−5
0
5
10
y
x

107. Geometry

108. Cheatsheet

108.1. Geometric Formulas

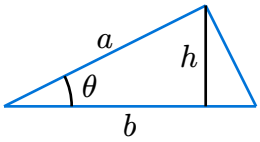
Formulas for:

- Area A
- Circumference C
- Volume V

Triangle

$A = \frac{1}{2}bh$

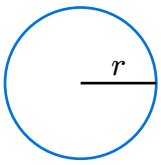
$A = \frac{1}{2}ab\sin(\theta)$



Circle

$A = \pi r^2$

$C = 2\pi r$



Sector of Circle

$A = \frac{1}{2}r^2\theta$

$s = r\theta$ (θ in radians)

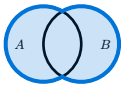
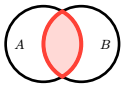
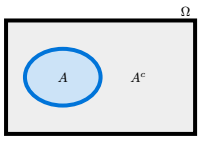
108.2. Trigonometric Functions

Function	Definition	Reciprocal
$\sin(x)$	$\frac{\text{opposite}}{\text{hypotenuse}}$	$\frac{1}{\sin(x)} = \csc(x)$
$\cos(x)$	$\frac{\text{adjacent}}{\text{hypotenuse}}$	$\frac{1}{\cos(x)} = \sec(x)$
$\tan(x)$	$\frac{\sin(x)}{\cos(x)} = \frac{\text{opposite}}{\text{adjacent}}$	$\frac{1}{\tan(x)} = \cot(x)$
$\csc(x)$	$\frac{\sin(x)}{\cos(x)} = \frac{\text{hypotenuse}}{\text{opposite}}$	$\frac{1}{\csc(x)} = \sin(x)$
$\sec(x)$	$\frac{\text{hypotenuse}}{\text{adjacent}}$	$\frac{1}{\sec(x)} = \cos(x)$
$\cot(x)$	$\frac{\cos(x)}{\sin(x)} = \frac{\text{adjacent}}{\text{opposite}}$	$\frac{1}{\cot(x)} = \tan(x)$

geometric_formulas

109. Probability Theory

110. Cheatsheet

Non-negativity	$P(A) \geq 0$	
Normalization	$P(\Omega) = 1$	
Addition Rule (Disjoint)	$\bigcap_{i=1}^{n} A_i = \varnothing$ $\Rightarrow$ $P\left(\bigcup_{i=1}^{n} A_i\right) = \sum_{i=1}^{n} P(A_i)$	
Union	$\bigcup_{i=1}^{n} A_i$	
Intersection	$\bigcap_{i=1}^{n} A_i$	
Empty set	$P(\varnothing) = 0$	
Disjoint (Mutually Exclusive)	$A \cap B = \varnothing$	
Probability Bound	$0 \leq P(A) \leq 1$	
Compliment	A^c	
Compliment Rule	$P(A) + P(A^c) = 1$ $P(A^c) = 1 - P(A)$	
Commutative Law (Union)	$S \cup T = T \cup S$	
Associative Law (Union)	$S \cup (T \cup U) = (S \cup T) \cup U$ $= S \cup T \cup U$	
Distributive Law (Intersection over Union)	$S \cap (T \cup U) = (S \cap T) \cup (S \cap U)$	
Distributive Law (Union over Intersection)	$S \cup (T \cap U) = (S \cup T) \cap (S \cup U)$	
Involution (Complement of a Complement)	$(S^c)^c = S$	
Complement Law (Disjointedness)	$(S \cap S^c) = \varnothing$	
Identity Law (Union with Universal Set)	$S \cup \Omega = \Omega$	
Identity Law (Intersection with Universal Set)	$S \cap \Omega = S$	

Independence	Two processes are independent if knowing the outcome of one provides **no useful information** about the outcome of the other	
Union Bound	$P(A \cup B) \leq P(A) + P(B)$	
Discrete Uniform Law	$P(A) = k\frac{1}{n}$	
Finite Additivity **Disjoint Events**	$P\left(\bigcup_{i=1}^{n} A_i\right) = \sum_{i=1}^{n} P(A_i)$	
Countable Additivity **Disjoint Events**	$P\left(\bigcup_{i}^{\infty} A_i\right) = \sum_{i=1}^{\infty} P(A_i)$	
De Morgan Laws	$\left(\bigcap_{n} S_n\right)^c = \bigcup_{n} S_n^c$	
	$\left(\bigcup_{n} S_n\right)^c = \bigcap_{n} S_n^c$	
Bonferroni Inequality	$P(\cap_{i=1}^{n} A_i) \geq \sum_{i=1}^{n} P(A_i) - (n-1)$	$P(S \cap T) \geq P(S) + P(T) - 1$
Monotonicity	$A \subseteq B \Longrightarrow P(A) \leq P(B)$	
Continuity of Probability	$A_1 \subseteq A_2 \subseteq \ldots$ $\Rightarrow$ $P\left(\bigcup_{n=1}^{\infty} A_n\right) = \lim_{n\to\infty} P(A_n)$	
	$A_1 \supseteq A_2 \supseteq \ldots$ $\Rightarrow$ $P\left(\bigcap_{n=1}^{\infty} A_n\right) = \lim_{n\to\infty} P(A_n)$	
Conditional probability	$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$	
Multiplication Rule (Independent)	$P\left(\bigcap_{i=1}^{n} A_i\right) = \prod_{i=1}^{n} P(A_i)$	
Law of Total Probability	$P(A) = \sum_{i} P(A \mid B_i)P(B_i)$	
Beyes' Theorem	$P(B_j \mid A) = \frac{P(A \mid B_j)P(B_j)}{\sum_{i=1}^{n} P(A \mid B_i)P(B_i)}$	
Independence	$P\left(\bigcap_{i=1}^{n} A_i\right) = \prod_{i=1}^{n} P(A_i)$	$P(A \cap B) = P(A) \cdot P(B)$
Complement Independence	For any $B_i \in \{A_i, A_i^c\}$, $P\left(\bigcap_{i=1}^{n} A_i\right) = \prod_{i=1}^{n} P(A_i)$	If $P(A \cap B) = P(A)P(B)$ then: $P(A \cap B^c) = P(A)P(B^c)$ $P(A^c \cap B) = P(A^c)P(B)$ $P(A^c \cap B^c) = P(A^c)P(B^c)$
Conditional Independence	$P\left(\bigcap_{i=1}^{n} A_i \mid C\right) = \prod_{i=1}^{n} P(A_i \mid C)$	$P(A \cap B \mid C) = P(A \mid C)P(B \mid C)$
Expected Value (Discrete)	$E(X) = \sum_{i=1}^{k} x_i P(X = x_i)$	

Expected Value (Continuous)	$E(X) = \int_{-\infty}^{\infty} x f_X(x) dx$	
Variance (Discrete)	$\begin{aligned}\operatorname{Var}(X) &= E[(X - E(X))^2] \\ &= \sum_{i=1}^{k} (x_i - E(X))^2 P(X = x_i)\end{aligned}$	
Variance (Continuous)	$\begin{aligned}\operatorname{Var}(X) &= E[(X - E(X))^2] \\ &= \int_{-\infty}^{\infty} (x - E(X))^2 f_X(x) dx\end{aligned}$	
Linear Combination	$\sum_{i=1}^{n} a_i X_i$	$a_1 X_1 + a_2 X_2 + \ldots + a_n X_n$
Expected Value of Linear Combination	$E\left(\sum_{i=1}^{n} a_i X_i\right) = \sum_{i=1}^{n} a_i E(X_i)$	$\begin{aligned}&E(a_1 X_1 + x_2 X_2 + \ldots a_n X_n) \\ &= a_1 E(X_1) + a_2 E(X_2) + \ldots + a_n E(X_n)\end{aligned}$
Variance (Independent) of Linear Combination	$\operatorname{Var}\left(\sum_{i=1}^{n} a_i X_i\right) = \sum_{i=1}^{n} a_i^2 \operatorname{Var}(X_i)$	
Variance (Dependent) of Linear Combination	$\operatorname{Var}\left(\sum_{i=1}^{n} a_i X_i\right) = \sum_{i=1}^{n} a_i^2 \operatorname{Var}(X_i) + 2 \sum_{1 \le i < j \le n} a_i a_j \operatorname{Cov}(X_i, X_j)$	

111. Sample Space

A probability model begins with two tasks:

1. Specify the set of all possible **outcomes**
2. Specify beliefs about how **likely** each outcome is

The Sample Space

- The sample space, denoted by Ω, is the set of all possible outcomes of an experiment
- The elements of Ω, called outcomes, must satisfy:
  - **Mutually exclusivity**: No two distinct outcomes can occur simultaneously
  - **Collective exhaustiveness**: Every possible result of the experiment must be included in Ω

112. Sets

A **set** is a collection of distinct elements

- $\{a, b, c, d\}$: a finite set
- $\mathbb{R}$: an infinite set

Membership

If S is a set and x is an element of S:

$$x \in S$$

If S is a set and x is not an element of S:

$$x \notin S$$

Example

Let S be the set of real numbers whose cosine is greater than $\frac{1}{2}$:

$$S = \left\{ x \in \mathbb{R} : \cos(x) > \frac{1}{2} \right\}$$

Universal Sets and Complements

Ω: universal set (sample space)

Compliment of a set

$$x \in S^c \quad \text{iff } x \in \Omega \text{ and } x \notin S$$

Key identities:

$$(S^c)^c = S$$

$\varnothing$: the empty set

$$\Omega^c = \varnothing$$

Subsets

If set S is a subset of the set T:

$$S \subset T$$

This means:

$$x \in S \Rightarrow x \in T$$

Union

$$S \cup T$$

Membership Rule

$$x \in S \cup T \quad \Leftrightarrow \quad x \in S \vee x \in T$$

Intersection

$$S \cap T$$

Membership Rule

$$x \in S \cap T \quad \Leftrightarrow \quad x \in S \wedge x \in T$$

112.1. Infinite Collection of Sets

Let

$$S_n \quad n = 1, 2, ...$$

Countable Union

$$x \in \bigcup_n S_n \quad \text{iff} \quad x \in S_n \textbf{ for some } n$$

Countable Intersection

$$x \in \bigcap_n S_n \quad \text{iff} \quad x \in S_n \textbf{ for all } n$$

Set properties

$$S \cup T = T \cup S$$

$$S \cup (T \cup U) = (S \cup T) \cup U = S \cup T \cup U$$

$$S \cap (T \cup U) = (S \cap T) \cup (S \cap U)$$

$$S \cup (T \cap U) = (S \cup T) \cap (S \cup U)$$

$$(S^c)^c = S$$

$$(S \cap S^c) = \varnothing$$

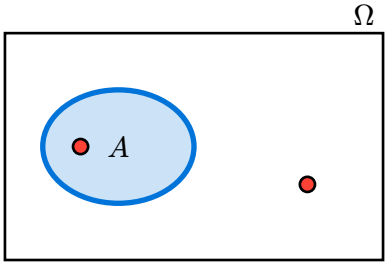
$$S \cup \Omega = \Omega$$

$$S \cap \Omega = S$$

113. Axioms

113.1. Probability Axioms

- Event: subset of the sample space
- Probability $P(A)$ is assigned to each event A
- After the experiment is performed, either the observed outcome lies in A (event A occurs) or it does not



113.1.1. Non-Negativity Axiom

Probability of any event is always non-negative

$$P(A) \geq 0$$

113.1.2. Normalization Axiom

The probability of the sample space is always 1

$$P(\Omega) = 1$$

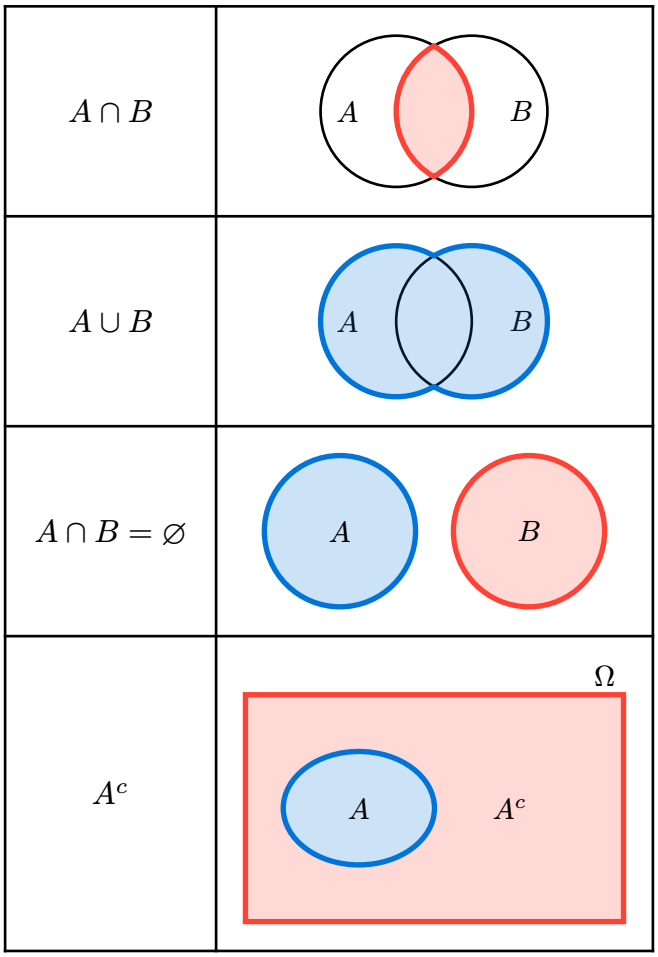
113.1.3. Finite Additivity Axiom

If two events are disjoint (mutually exclusive),

$$A \cap B = \varnothing$$

then

$$P(A \cup B) = P(A) + P(B)$$



Consequences of the Axioms:

113.1.4. $0 \leq P(A) \leq 1$

Since

$$A \cup A^c = \Omega, \qquad A \cap A^c = \varnothing$$

we have

$$1 = P(\Omega) = P(A) + P(A^c)$$

Thus,

$$P(A) = 1 - P(A^c) \overset{(a)}{\leq} 1$$

113.1.5. $P(\varnothing) = 0$

$$\begin{aligned} 1 &= P(\Omega) \\ &= P(\Omega) + P(\Omega^c) \\ &= 1 + P(\varnothing) \quad \Rightarrow \quad P(\varnothing) = 0 \end{aligned}$$

113.1.6. $P(A) + P(A^c) = 1$

Follows immediately from additivity applied to A and A^c.

113.1.7. Probability of Finite Unions of Disjoint Events

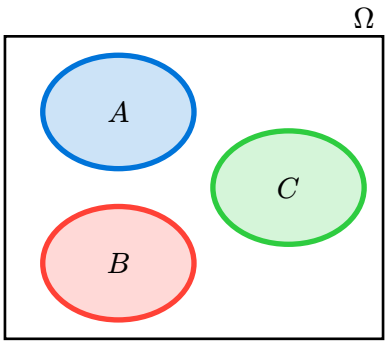
For three mutually disjoint sets:

$$P(A \cup B \cup C) = P(A) + P(B) + P(C)$$

More generally, if:

$$A_i \cap A_j = \varnothing \quad \text{for } i \neq j$$

then:

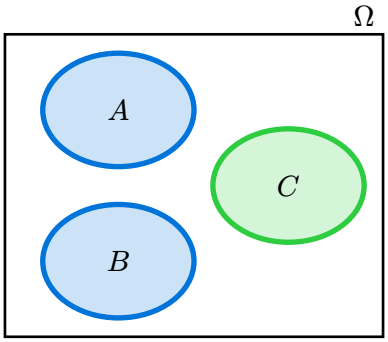


From (Finite) Additivity, we have:

$$P(A \cup B) = P(A) + P(B)$$

So,

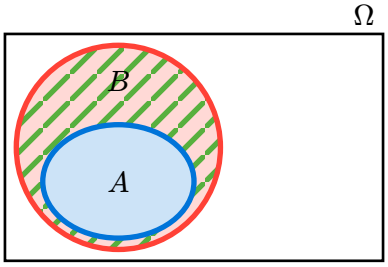
$$\begin{aligned} P(A \cup B \cup C) &= P((A \cup B) \cup C) \\ &= P(A \cup B) + P(C) \\ &= P(A) + P(B) + P(C) \end{aligned}$$



More generally, by induction:

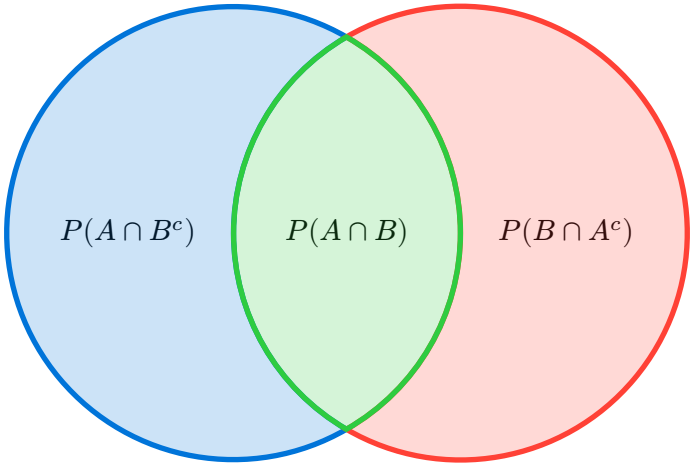
$$P\left(\bigcup_{i=1}^{n} A_i\right) = \sum_{i=1}^{n} P(A_i) \qquad A_i \cap A_j = \varnothing, \quad \text{for } i \neq j$$

113.1.8. If $A \subset B$ then $P(A) \leq P(B)$



$$B = A \cup (B \cap A^c)$$
$$P(B) = P(A) + P(B \cap A^c) \geq P(A)$$

113.1.9. $P(A \cup B) = P(A) + P(B) = P(A \cap B)$



$$P(A \cup B) = P(A \cap B^c) + P(A \cap B) + P(B \cap A^c)$$

$$P(A) + P(B) - P(A \cap B) = (P(A \cap B^c) + P(A \cap B)) + (\cancel{P(A \cap B)} + P(B \cap A^c)) \cancel{- P(A \cap B)}$$
$$= P(A \cap B^c) + P(A \cap B) + P(B \cap A^c))$$

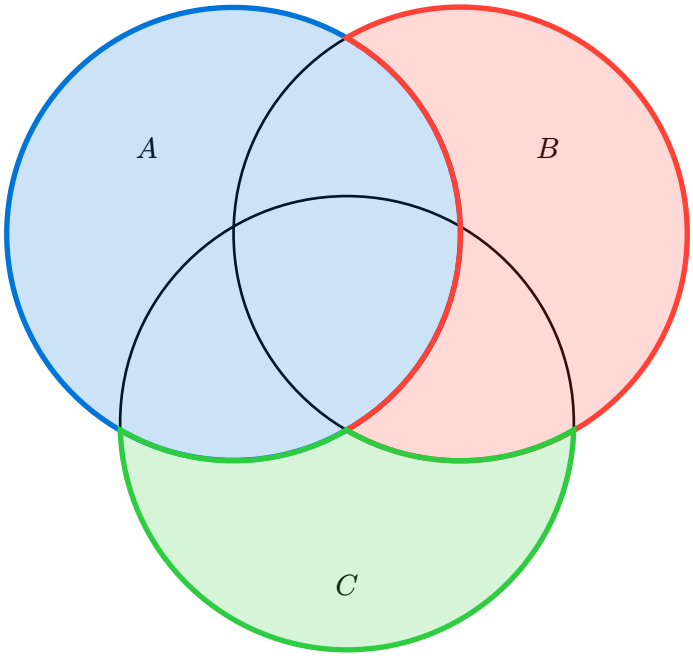
113.1.10. Union Bound

$$P(A \cup B) \leq P(A) + P(B)$$

Since $P(A \cap B) \geq 0$

113.1.11. Union of more than two overlapping (non disjoint) sets

$$P(A \cup B \cup C) = P(A) + P(A^c \cap B) + P(A^c \cap B^c \cap C)$$



$$A \cup B \cup C = A \cup B \cap A^c \cup C \cap A^c \cap B^c$$
$$P(A \cup B \cup C) = P(A) + P(B \cap A^c) + P(C \cap A^c \cap B^c)$$

113.2. Discrete Uniform Law

- Assume Ω consists of n equally likely elements
- Assume A consists of k elements

$$P(A) = k\frac{1}{n}$$

113.3. Probability Calculations Steps

- Specify a sample space
- Specify a probability law
- Identify an event of interest
- Calculate

Discrete but infinite sample space

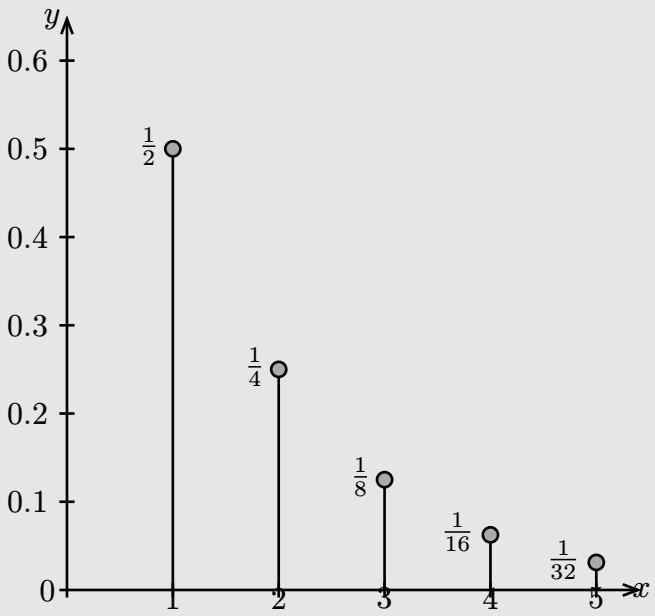
Example

Number of coin tosses until we observe a heads toss

Sample Space

$$\Omega = \{1, 2, ..., \infty\}$$

We are given $P(n) = \frac{1}{2^n}, \quad n = 1, 2, ...$



$$\sum_{n=1}^{\infty} \frac{1}{2^n} = \frac{1}{2} \sum_{n=0}^{\infty} \frac{1}{2^n} = \frac{1}{2} \cdot \frac{1}{1 - \left(\frac{1}{2}\right)} = 1$$

$$\begin{aligned} P(\text{outcome is even}) &= P(\{2, 4, ...\}) \\ &= P(\{2\} \cup \{4\} \cup ...) \\ &= P(2) + P(4) + ... \\ &= \frac{1}{2^2} + \frac{1}{2^4} + ... \\ &\frac{1}{4}\left(1 + \frac{1}{4} + \frac{1}{4^2} + ...\right) \\ &= \frac{1}{4} \cdot \frac{1}{1 - \frac{1}{4}} \\ &= \frac{1}{3} \end{aligned}$$

113.3.1. Countable Additivity Axiom

If $a_1, A_2, A_3...$ is an infinite **sequence** of **disjoint** events, then $P(A_1 \cup A_2 \cup A_3...) = P(A_1) + P(A_2) + P(A_3) + ...$

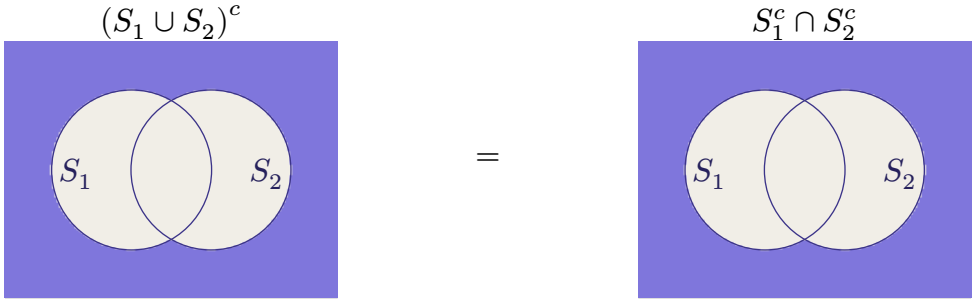
$$P\left(\bigcup_{i=1}^{\infty} A_i\right) = \sum_{i=1}^{\infty} P(A_i) \qquad A_i \cap A_j = \varnothing, \quad \text{for } i \neq j$$

- Additivity holds only for “countable” sequences of events
  - The unit square (or real line) if **not countable**

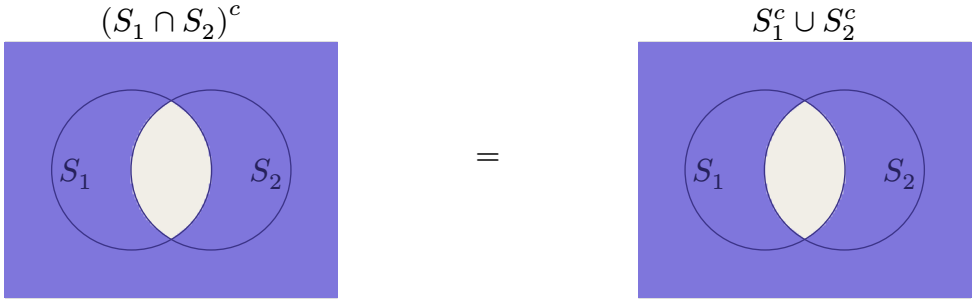
114. De Morgan’s Laws

Allows going back and forth between unions and intersections

114.1. Complement of a union



114.2. Complement of an intersection



114.3. Scaling

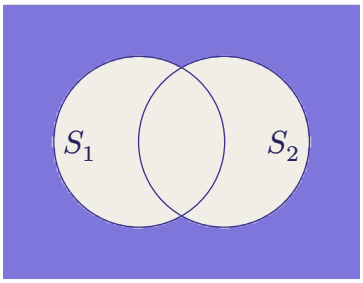
The general statement quantifies over an arbitrary index set:

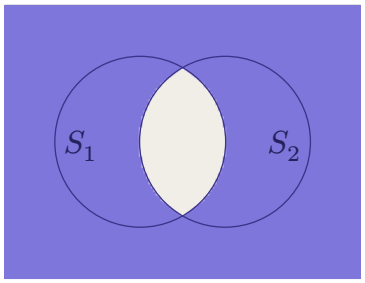
$$\left(\bigcup_{n \in I} S_n\right)^c = \bigcap_{n \in I} S_n^c \qquad \left(\bigcap_{n \in I} S_n\right)^c = \bigcup_{n \in I} S_n^c$$

Two circles cannot show why I may be infinite, but two, three and four can show the **direction of travel**. Only the family changes below; the predicates are untouched.

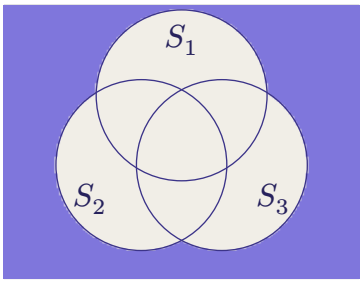
$$\left(\bigcup_n S_n\right)^c = \bigcap_n S_n^c \qquad \left(\bigcap_n S_n\right)^c = \bigcup_n S_n^c$$

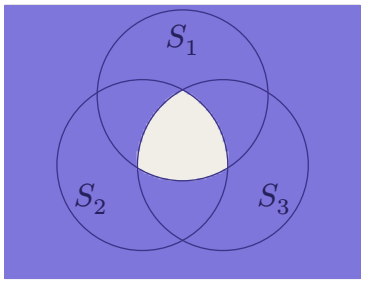
$n = 2$



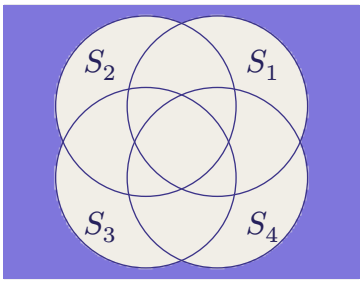


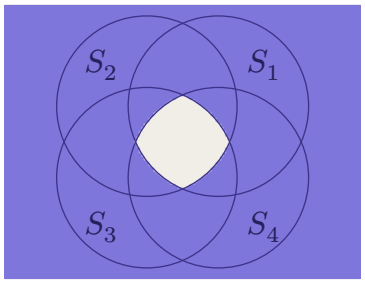
$n = 3$





$n = 4$





115. Sequences

$$a_1, a_2, a_3, ...$$

$$S = \{a_i\} \quad i \in \mathbb{N} = \{1, 2, 3, ...\}$$

Formal definition:

$$f : \mathbb{N} \to S$$

$$f(i) = a_i$$

Convergence

$$\lim_{i \to \infty} a_i = a$$

$$\forall \varepsilon > 0 \exists N \in \mathbb{N} \quad \forall n \geq N : |a_n - a| < \varepsilon$$

For every $\varepsilon > 0$, there exists $N \in \mathbb{N}$ such that for all $n \geq N$, $|a_n - a| < \varepsilon$

Combination of Sequences

$$\left.\begin{array}{l} a_i \to a \\ b_i \to b \end{array}\right\} \quad \Rightarrow \quad \begin{array}{c} a_i + b_i \to a + b \\ a_i b_i \to ab \end{array}$$

$$g : \text{continuous} \quad \Rightarrow \quad g(a_i) \to g(a)$$

When does a sequence converge?

1. Increasing sequence

$$a_i \leq a_{i+1} \quad \forall i$$

Then eiher:

- Sequence vonverges to ∞
- Sequence converges to some real number a

2.

If

$|a_i - a| \leq b_i \quad \forall i$

and

$b_i \to 0$

then

$a_i \to a$

116. Series

$$\sum_{i=1}^{\infty} a_i = \lim_{n\to\infty} \sum_{i=1}^{n} a_i$$

Provided the limit exists:

- If $a_i \geq 0$: limit exists
- If a_i do not have the same sign:
    - Limit need not exists
    - Limit may exists but be different if we sum in a different order
        - Limit exists and **independent of order** of summation if

$$\sum_{i=1}^{\infty} |a_i| \leq \infty$$

Geometric Series

$$S = \sum_{i=0}^{\infty} \alpha^i = 1 + \alpha + \alpha^2 + ... = \frac{1}{1-\alpha} \quad |\alpha| < 1$$

Derivation:

$$(1-\alpha)(1+\alpha+...\alpha^n) = 1 - \alpha^{n+1}$$
$$n \to \infty$$
$$(1-\alpha)S = 1$$
$$S = \frac{1}{1-\alpha}$$

Summation in series with multiple indices

$$\sum_{i\geq 1, j\geq 1} a_{ij}$$

If the sum of the absolute value of the terms of the series are finite,

$$\sum |a_{ij}| < \infty$$

the order in which the elements are summed will not matter

E.g.:

$$\sum_{i=1}^{\infty}\left(\sum_{j=1}^{\infty} a_{ij}\right) = \sum_{j=1}^{\infty}\left(\sum_{i=1}^{\infty} a_{ij}\right)$$

117. Countability

117.1. Countable and Uncoutable Sets

1. Countable (Discrete)

One-to-one correspondence with positive integers

$$\{a_1, a_2, a_3 ...\} = \Omega$$

E.g.,

- Positive integers $\{1, 2, 3, ...\}$
- Integers $\{0, -1, 1, -2, 2, -3, 3, ...\}$
- Pairs, triples, quadruples, … of positive integers
- rational numbers q, with $0 < q < 1$

$\left\{\frac{1}{2}, \frac{1}{3}, \frac{2}{3}, \frac{1}{4}, \frac{2}{4}, \frac{3}{4}, \frac{1}{5}, \frac{2}{5}, ...\right\}$

2. Uncountable (Continuous)

- Intervals ($[0, 1]$)
- Reals
- Plane

118. Bonferroni Inequality

1. Two Sets

$$P(S \cap T) \geq P(S) + P(T) - 1$$

Proof: Bonferroni Inequality (2 Sets)

1. Start with **union bound**:

$$P(A^c \cup B^c) \leq P(A^c) + P(B^c)$$

2. And **De Morgan's Law**:

$$A^c \cup B^c = (A \cap B)^c$$

3. Rewrite all terms in terms of probabilities of A and B:

$$P(A^c) = 1 - P(A) \qquad P(B^c) = 1 - P(B)$$

Insert into the union bound:

$$P((A \cap B)^c) \leq (1 - P(A)) + (1 - P(B))$$

4. Use complement again:

$$1 - P(A \cap B) \leq 2 - P(A) - P(B)$$

5. Rearrange:

$$P(A \cap B) \geq P(A) + P(B) - 1$$

2. n Sets

$$P\left(\bigcap_{i=1}^{n} A_i\right) \geq \sum_{i=1}^{n} A_i - (n-1)$$

$$P(A_1, \cap \ldots \cap A_n) \geq P(A_1) + \ldots + P(A_n) - (n-1)$$

Proof: Bonferroni Inequality (n Sets)

1. Start with the **union bound**

$$P\left(\bigcup_{i=1}^{n} A_i^c\right) \leq \sum_{i=1}^{n} P(A_i^c)$$

2. **De Morgan's Law**

$$\bigcup_{i=1}^{n} A_i^c = \left(\bigcap_{i=1}^{n} A_i\right)^c$$

So we can rewrite the LHS as:

$$P\left(\left(\bigcap_{i=1}^{n} A_i\right)^c\right)$$

3. Rewrite each complement probability

For each i:

$$P(A_i^c) = 1 - P(A_i)$$

Thus:

$$P\left(\left(\bigcap_{i=1}^{n} A_i\right)^c\right) \leq \sum_{i=1}^{n} (1 - P(A_i))$$

4. Use the complement relation $P(E^c) = 1 - P(E)$

$$1 - P\left(\bigcap_{i=1}^{n} A_i\right) \leq n - \sum_{i=1}^{n} P(A_i)$$

5. Rearrange to obtain the lower bound

Move terms around:

$$P\left(\bigcap_{i=1}^{n} A_n\right) \geq \sum_{\{i=1\}}^{n} P(A_i) - (n-1)$$

119. Conditional Probability

Probability of A given that B has occurred

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$$

Note:

$$P(A \mid B) \geq 0$$

$$P(A \mid A) = 1$$

$$P(\Omega \mid A) = 1$$

Additivity:

$$P(A \cup C \mid B) = P(A \mid B) + P(C \mid B)$$
$$\text{if } A \cap C = \varnothing$$

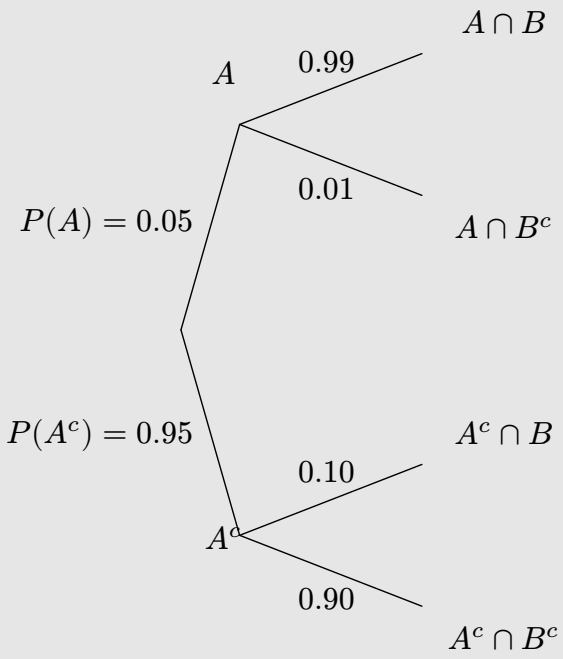
$$\begin{aligned}\frac{P((A \cup C) \cap B)}{P(B)} &= \frac{P((A \cap B) \cup (C \cap B))}{P(B)} \\ &= \frac{P(A \cap B) + P(C \cap B)}{P(B)} \\ &= P(A \mid B) + P(C \mid B)\end{aligned}$$

Example

Event A: Airplane is flying above

Event B: Something registers on radar screen

$$P(A) = 0.05$$
$$P(A^c) = 0.95$$



$$P(A \cap B) = P(A) \cdot P(B \mid A) = 0.05 \cdot 0.99$$

$$P(B) = 0.05 \cdot 0.99 + 0.95 \cdot 0.10$$

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)} = \frac{0.05 \cdot 0.99}{0.05 \cdot 0.99 + 0.95 \cdot 0.10}$$

120. Multiplication Rule

For n events:

$$P\left(\bigcap_{i=1}^{n} A_i\right) = \prod_{i=1}^{n} P\left(A_i \mid \bigcap_{j=1}^{i-1} A_j\right)$$

2 events:

$$P(A_1 \cap A_2) = P(A_1)P(A_2 \mid A_1)$$

3 events:

$$P(A_1 \cap A_2 \cap A_3) = P(A_1)P(A_2 \mid A_1)P(A_3 \mid A_1 \cap A_2)$$

4 events:

$$P(A_1 \cap A_2 \cap A_3 \cap A_4) = P(A_1)P(A_2 \mid A_1)P(A_3 \mid A_1 \cap A_2)P(A_4 \mid A_1 \cap A_2 \cap A_3)$$

121. Total Probability Theorem

Let $B_1, B_2, ..., B_n$ be a partition of the sample space Ω, meaning:

1. $B_i \cap B_j = \varnothing \forall i \neq j$ (disjoint)
2. $\bigcap_{i=1}^{n} B_i = \Omega$ (cover the whole space)

Then for any event A:

$$\begin{aligned} P(A) &= \sum_{i=1}^{n} P(A \cap B_i) \\ &= \sum_{i=1}^{n} P(A \mid B_i) P(B_i) \end{aligned}$$

The total probability theorem arises by breaking an event into pieces using a partition, and each piece is expressed using the multiplication rule, which itself comes from the definition of conditional probability.

1. Conditional Probability:

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$$

2. Multiplication Rule

$$P(A \cap B) = P(A \mid B) P(B)$$

3. Total Probability Theorem

$$P(A) = \sum_{i=1}^{n} P(A \mid B_i) P(B_i)$$

122. Bayes' Rule

122.1. Beyes' Rule

$$P(A \mid B) = \frac{P(B \mid A) P(A)}{P(B)}$$

Start with the multiplication rule in both directions:

$$\begin{aligned} P(A \cap B) &= P(A) P(B \mid A) \\ P(A \cap B) &= P(B) P(A \mid B) \end{aligned}$$

Set them equal:

$$P(A) P(B \mid A) = P(B) P(A \mid B)$$

Solve for $P(A \mid B)$:

$$P(A \mid B) = \frac{P(B \mid A) P(A)}{P(B)}$$

So Bayes' Rule is the multiplication rule rearranged.

Start: Definition of conditional probability

$$P(A_i \mid B) = \frac{P(A_i \cap B)}{P(B)}$$

1. **Numerator**: $P(A_i \cap B)$ (Multiplication Rule)

Using the multiplication rule:

$$P(A_i \cap B) = P(A_i)P(B \mid A_i)$$

This expresses the joint probability as *prior* $\times$ *likelihood*

So the numerator comes directly from multiplying:

- prior $P(A_i)$
- likelihood $P(B \mid A_i)$

2. **Denominator**: $P(B)$ (Total Probability Theorem)

If $A_1, ..., A_n$ form a partition, then the total probability theorem gives:

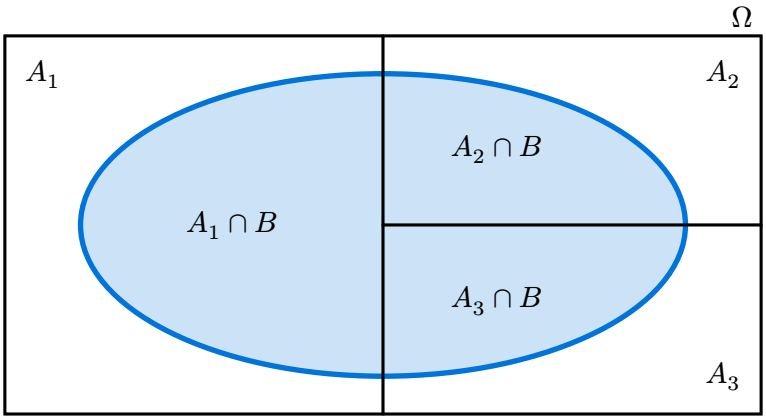
$$P(B) = \sum_{j=1}^{n} P(B \mid A_j)P(A_j)$$

This decomposes the probability of the "evidence" B into all possible scenarios A_j

3. Put numerator & denominator together

Substitute both results into the definition:

$$P(A_i \mid B) = \frac{P(A_i)P(B \mid A_i)}{\sum_{j=1}^{n} P(B \mid A_j)P(A_j)}$$



Interpretation

- Numerator: how strongly scenario A_i explains the evidence B (*likelihood* $\times$ *prior*)
- Denominator: total probability of seeing the evidence at all (*sum of all scenario contributions*)

So Bayes' Rule is literally:

$$\text{Posterior} = \frac{\text{Likelihood} \times \text{Prior}}{\text{Total Evidence}}$$

Example

Identifying Source of a Defective Part

A company sources a critical component from three suppliers:

- Supplier A_1: 50% of parts
- Supplier A_2: 30% of parts
- Supplier A_3: 20% of parts

Each supplier has a different defect rate:

- $P(B \mid A_1) = 0.01$
- $P(B \mid A_2) = 0.03$
- $P(B \mid A_3) = 0.05$

Here:

- A_i: part comes from supplier i
- B: part is defective

1. Total Probability Theorem: What is the overall defect rate?

We compute

$$P(B) = \sum_{i=1}^{3} P(B \mid A_i) P(A_i)$$

Plug in numbers:

$$\begin{aligned} P(B) &= 0.01(0.50) + 0.03(0.30) + 0.05(0.20) \\ &= 0.005 + 0.009 + 0.010 = 0.024 \end{aligned}$$

Overall defect rate = 2.4%

2. Multiplication Rule: Joint probability from each supplier

- Supplier 1:

$$\begin{aligned} P(A_1 \cap B) &= P(A_1) P(B \mid A_1) \\ &= 0.50 \times 0.01 \\ &= 0.005 \end{aligned}$$

- Supplier 2:

$$\begin{aligned} P(A_2 \cap B) &= P(A_2) P(B \mid A_2) \\ &= 0.30 \times 0.03 \\ &= 0.009 \end{aligned}$$

- Supplier 3:

$$\begin{aligned} P(A_3 \cap B) &= P(A_3)P(B \mid A_3) \\ &= 0.20 \times 0.08 \\ &= 0.016 \end{aligned}$$

This term appears in the numerator of Bayes' Rule.

3. Bayes' Rule: If a part is defective, what is the probability it came from supplier 3?

$$P(A_3 \mid B) = \frac{P(A_3)P(B \mid A_3)}{P(B)}$$

Substitute:

$$\begin{aligned} P(A_3 \mid B) &= \frac{0.20 \cdot 0.05}{0.024} \\ &= \frac{0.010}{0.024} \\ &\approx 0.4167 \end{aligned}$$

About a 41.7% chance the defective part came from supplier 3.

Even though supplier 3 supplies only 20% of parts, it becomes the most likely source of a defect because its defect rate is highest.

123. Independence

Two events are said to be independent if the occurrence of one event does not change our beliefs about the other

Events A and B are independent from each other if:

$$P(A \cap B) = P(A) \cdot P(B)$$

Implies:

$$\begin{aligned} P(B \mid A) &= P(B) \\ P(A \mid B) &= P(A) \end{aligned}$$

Applies even if $P(A) = 0$

123.1. Independence of Complements

If A and B are independent, then the following pairs are also independent:

- A and B^c
- A^c and B
- A^c and B^c

Formally:

If

$$P(A \cap B) = P(A)P(B)$$

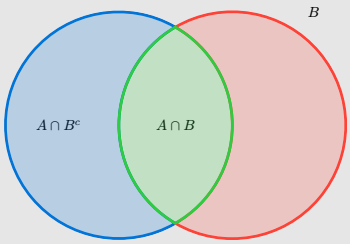
then:

$$P(A \cap B^c) = P(A)P(B^c)$$
$$P(A^c \cap B) = P(A^c)P(B)$$
$$P(A^c \cap B^c) = P(A^c)P(B^c)$$

Proof

Start with:

$$A = (A \cap B) \cup (A \cap B^c)$$



Since $(A \cap B)$ and $(A \cap B^c)$ are disjoint, by the Additivity Axiom:

$$\begin{aligned} P(A) &= P(A \cap B) + P(A \cap B^c) \\ &= P(A)P(B) + P(A \cap B^c) \end{aligned}$$

Rearanging:

$$\begin{aligned} P(A \cap B^c) &= P(A) - P(A)P(B) \\ &= P(A)(1 - P(B)) \\ &= P(A)P(B^c) \end{aligned}$$

Thus we are back at out definition of independence:

$$P(A \cap B^c) = P(A)P(B^c)$$

123.2. Conditional Independence

$$P(A \cap B \mid C) = P(A \mid C)P(B \mid C)$$

123.3. Pairwise Independence

- **Mutual (collective) independence** $\Rightarrow$ pairwise independence

  If a collection of events is mutually independent, then every pair of events in the collection is independent.

- **Pairwise independence** ⇒ mutual (collective) independence

  A collection of events can be pairwise independent but not mutually independent.

123.4. Reliability

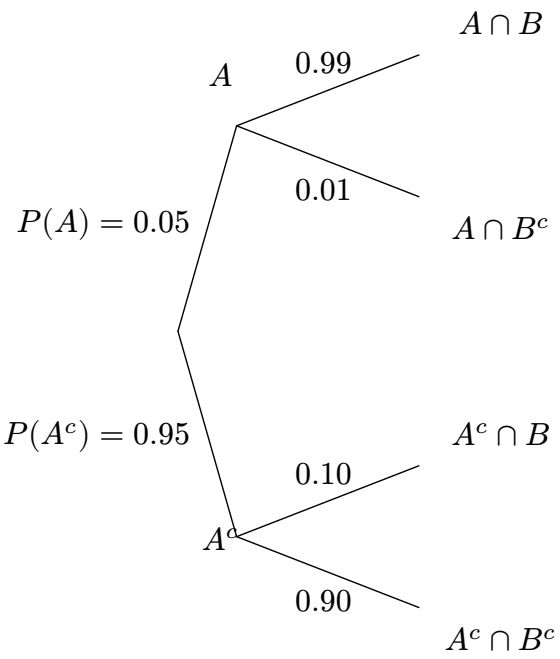
124. Counting Principle

Count the total number of possible outcomes in a multi-step process

- The first stage has n_1 possible outcomes,
- The second stage has n_2 possible outcomes,
- …
- The k-th stage has n_k possible outcomes

then the total number of outcomes is:

$$n_1 \cdot n_2 \cdot ... \cdot n_k = \prod_{i=1}^{k} n_i$$



Permutations

Number of k-element subsets of a given n-element set

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

Because $0! = 1$:

$$\binom{n}{n} = 1$$

$$\binom{n}{0} = 1$$

$$\sum_{k=0}^{n} \binom{n}{k} = \binom{n}{0} + \binom{n}{1} + ... + \binom{n}{n} = 2^n$$

Ordering n elements	$n!$
Ordering k out of n	$P(n,k) = \frac{n!}{(n-k)!}$
Choosing and ordering k out of m	$\binom{m}{k} k!$
Number of subsets of n	2^n
Sequences of length n from m choices	m^n

Example

Find the probability that 6 rolls of a fair 6-sided die all give different numbers

$$P(\text{all different}) = \frac{\text{number of favorable outcomes}}{\text{total outcomes}}$$

- Total outcomes for 6 rolls: 6^6
- Favorable outcomes (all numbers different): $6! = 6 \cdot 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1$

Therefore:

$$\frac{6!}{6^6}$$

Combination

Definition: Number of k-element subsets of a given n-element set

$$\binom{n}{k}$$

Two ways to construct an **ordered** sequence of k **distinct** items:

- Choose the k items one at a time

$$n(n-1)(n-2)...(n-k+1) = \frac{n!}{(n-k)!}$$

- Choose k items, then order them

$$\binom{n}{k} k!$$

125. Expected Value

The **expected value** $\mathbb{E}[X]$ is the probability-weighted average of a random variable — its long-run mean over many draws.

125.1. Discrete

Weight each value x_i by its probability p_i and sum:

$$\mathbb{E}[X] = \sum_{i=1}^{n} x_i \cdot p_i$$

Example

Fair die: $\mathbb{E}[X] = \sum_{k=1}^{6} k \cdot \frac{1}{6} = \frac{21}{6} = 3.5$

125.2. Continuous

Swap the sum for an integral, and the mass p_i for $f(x)\,\mathrm{d}x$:

$$\mathbb{E}[X] = \int_{-\infty}^{\infty} x \cdot f(x)\,\mathrm{d}x$$

125.3. Partial (truncated) expectation

Average x over only a window $[a, b]$ — same integrand, restricted limits:

$$\int_{a}^{b} x \cdot f(x)\,\mathrm{d}x = \mathbb{E}\left[X \cdot 1_{\{a \le X \le b\}}\right]$$

where $1_{\{(\cdot)\}}$ is the **indicator** (1 when the condition holds, 0 otherwise). This is **not** a mean — it is not renormalized, so it shrinks toward 0 as the window captures less probability.

125.4. Conditional expectation

Renormalize by the probability of the window — now it **is** a true average, given that X landed in $[a, b]$:

$$\mathbb{E}[X \mid a \le X \le b] = \frac{\int_{a}^{b} x \cdot f(x)\,\mathrm{d}x}{P(a \le X \le b)}$$

125.5. LOTUS

Law of the Unconscious Statistician. To average a function $g(X)$, integrate g against the density of X — no need to first find the distribution of $g(X)$:

$$\mathbb{E}[g(X)] = \int_{-\infty}^{\infty} g(x) \cdot f(x)\,\mathrm{d}x$$

---

126. Expected Value

126.1. Discrete

$$\mathbb{E}[X] = \sum_{i=1}^{n} x_i \cdot p_i$$

126.2. Continuous

$$\mathbb{E}[X] = \int_{-\infty}^{\infty} x \cdot f(x)\,\mathrm{d}x$$

126.2.1. Partial

Truncated or restricted expectation. Specifically:

$$\int_{-\infty}^{\infty} x \cdot f(x)\,\mathrm{d}x = \mathbb{E}\left[X \cdot 1_{\{n \le X \le m\}}\right]$$

Where $1_{\{\cdot\}}$ is the indicator function — it's 1 when the condition is true, 0 otherwise.

126.2.2. Conditional

$$\mathbb{E}[X \mid n \le X \le m] = \frac{\int_{-\infty}^{\infty} x \cdot f(x)\,\mathrm{d}x}{P(n \le X \le m)}$$

126.2.3. LOTUS

Law of the Unconscious Statistician

$$\int_{-\infty}^{\infty} g(x) \cdot f(x)\,\mathrm{d}x$$

127. Linearity of Expectations

$$E[X+Y] = E[X] + E[Y]$$

$$E[aX] = aE[X]$$

128. Joint, Marginal & Conditional Probability

Setup

Imagine we have 100 emails. We track two things about each one: is it spam or not, and does it contain the word “free” or not. Here’s the count of every combination:

	F	F^c	margin
S	0.20	0.10	0.30
S^c	0.05	0.65	0.70
margin	0.25	0.75	1.00

128.1. Joint Probability

$$P(S \cap F) = 0.20$$

128.2. Marginal Probability

$$P(A) = \sum_{i=1}^{n} P(A \cap B_i)$$

$$\begin{aligned} P(S) &= P(S \cap F) + P(S \cap F^c) \\ &= 0.20 + 0.10 \\ &= 0.30 \end{aligned}$$

128.3. Conditional Probability

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$$

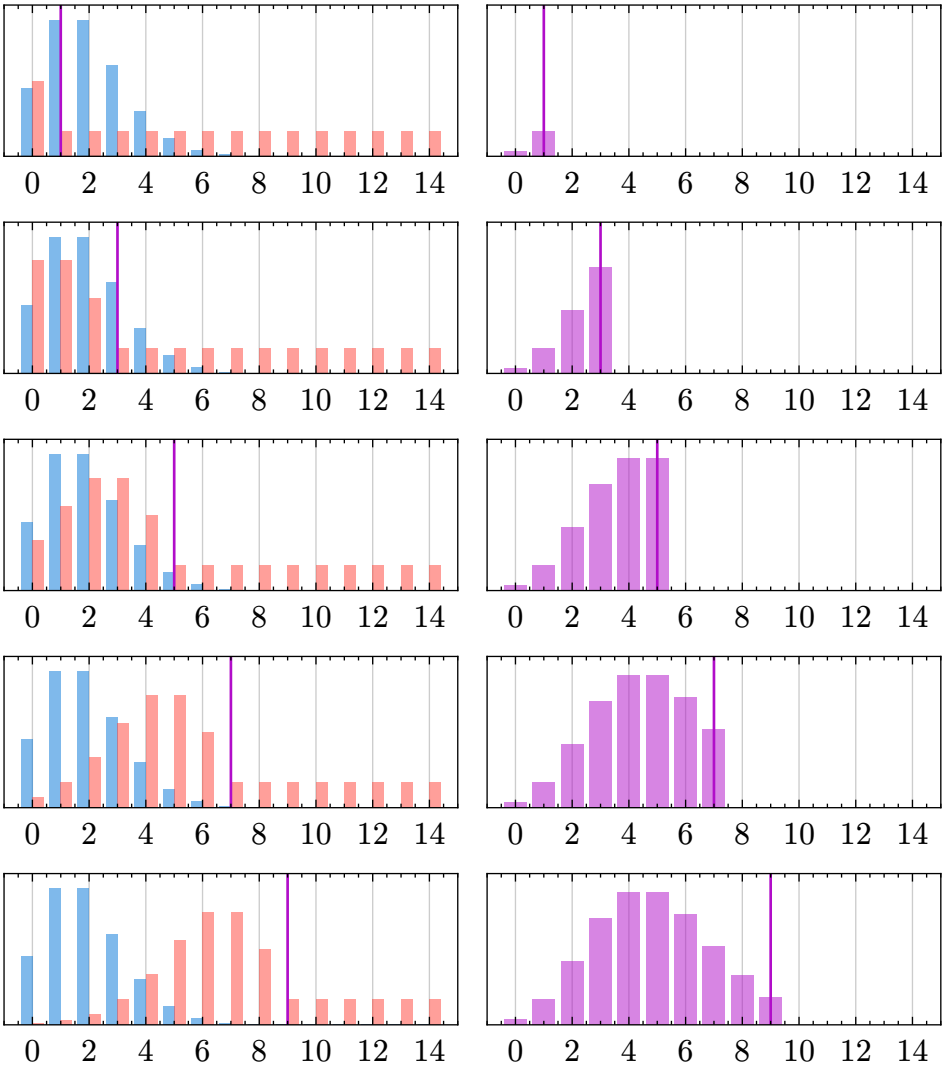
$$P(S \mid F) = \frac{P(S \cap F)}{P(F)} = \frac{0.20}{0.25} = 0.80$$

128.4. Convolution

You have two independent random variables, X and Y. You add them: $Z = X + Y$. What’s the distribution of Z?

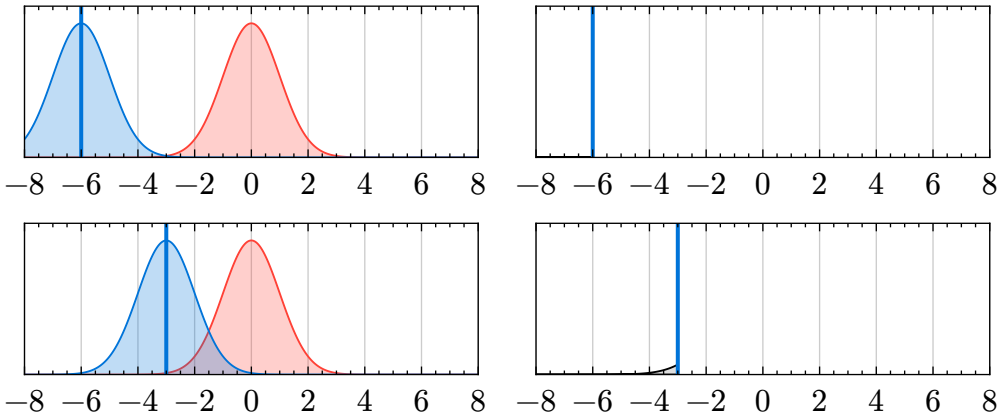
128.4.1. Discrete

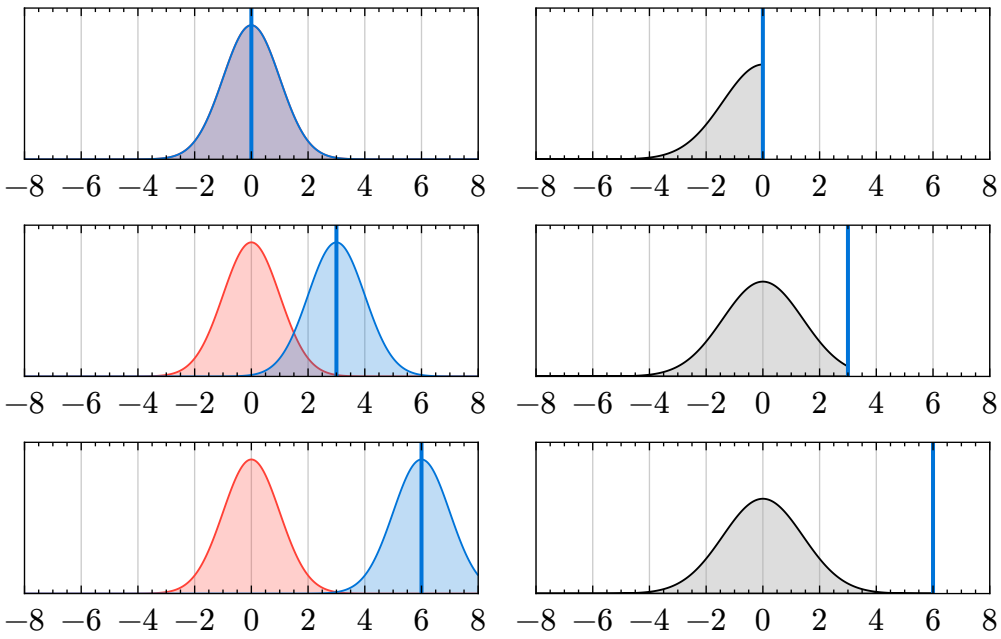
$$P(Z = z) = \sum_{x} P(X = x) P(Y = z - x)$$



128.4.2. Continuous

$$f_Z(z) = \int_{-\infty}^{+\infty} f_X(x) f_Y(z - x)$$





129. Mixture Distributions

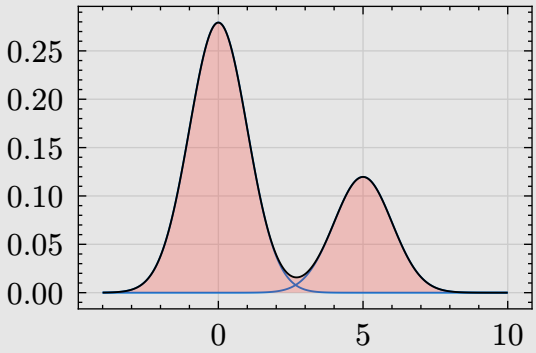
A mixture is what you get when you randomly pick one of several distributions and then draw from it. You're not combining the random variables — you're choosing between them.

$$f(x) = \sum_i w_i f_i(x) \qquad w_i \geq 0, \sum_i w_i = 1$$

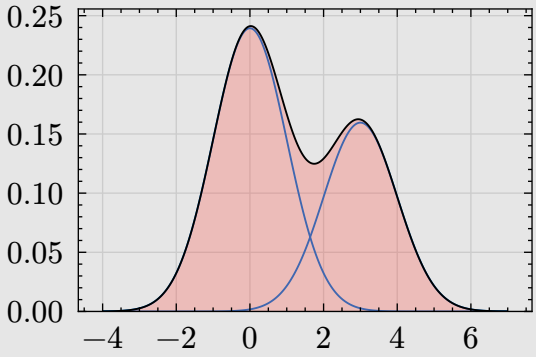
- Flip a biased coin: with probability w_1, pick component 1, with probability $w_2 = 1 - w_1$ pick component 2.
- Draw your value from whichever component the coin selected.

Example

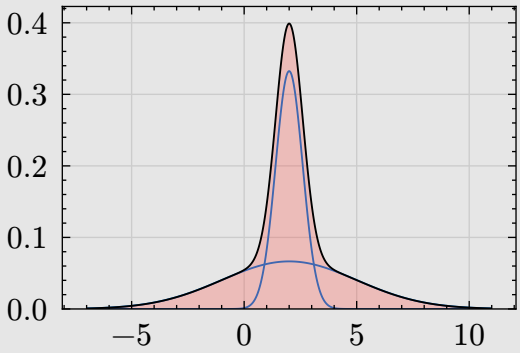
$$f(x) = 0.7 \cdot \mathcal{N}(x; 0, 1) + 0.3 \cdot \mathcal{N}(x; 5, 1)$$



$$f(x) = 0.6 \cdot \mathcal{N}(0, 1) + 0.4 \cdot \mathcal{N}(3, 1)$$



$$f(x) = 0.5 \cdot \mathcal{N}(2, 0.6) + 0.5 \cdot \mathcal{N}(2, 3)$$



130. Law of Total Expectation

- Tower property
- Law of iterated expectations
- Smoothing theorem
- Adam's law

$$\mathbb{E}[X] = \mathbb{E}[\mathbb{E}[X \mid Y]]$$

131. Law of Total Variance

- Variance decomposition formula
- Conditional variance formula
- Eve's law

$$\operatorname{Var}(X) = \underbrace{\mathbb{E}[\operatorname{Var}(X \mid Y)]}_{\text{within-group}} + \underbrace{\operatorname{Var}(\mathbb{E}[X \mid Y])}_{\text{between-group}}$$

132. I.I.D.

- **Independent** — the joint distribution factorizes; cross-terms (covariances) vanish.
- **Identically distributed** — every variable shares the same mean, variance, and distribution shape, so you can replace n different quantities with one common one.

133. Moment Generating Function

For a random variable X, its MGF is:

$$\begin{aligned} G(t) &= \sum_{i=1}^{\infty} \frac{G^{(k)}(0)}{k!} \\ &= \sum_{i=1}^{\infty} \frac{E[X^k]}{k!} t^k \\ &= E\left[\sum_{i=1}^{\infty} \frac{X^k t^k}{k!}\right] \\ &= E\left[\sum_{i=1}^{\infty} \frac{(tX)^k}{k!}\right] \\ &= E[e^{tX}] \end{aligned}$$

Therefore:

$$M_X(t) = E[e^{tX}]$$

In general $M_X^k(0) = E[X^k]$. So you turn the crank – take k derivatives, plug in $t = 0$ and the k-th moment falls out. The whole sequence of moments (mean via the first, variance via the second, skewness via the third, …) is encoded in this one function, recoverable by differentiation.

Moment	Name	f	Definition
1st	Mean	$G^1(0) = E[X^k]$	where the distribution sits
2nd	Variance	$G^2(0) =$	How spread out it is
3rd	Skewness	$G^3(0) =$	How lopsided it is (longer left or right tail)
4th	Kurtosis	$G^4(0) =$	How heavy the tails are, how peaked
5th …	No names	$G^{(k)}(0) =$	No intuitive picture

Example

134. Markov’s Inequality

135. Chebyshev’s Inequality

136. Statistics

137. Cheatsheet

Test Type	Purpose	Assumptions	Non-Parametric Equivalent	Non-Parametric Test Purpose	Non-Parametric Test Assumptions
One-Sample t-Test	Compare sample mean to a known value	Normality of the sample distribution	Wilcoxon Signed-Rank Test	Compare median of a sample to a known value	Distribution-free; does not assume normality
Independent t-Test	Compare means between two independent groups	Normality, equal variances in groups	Mann-Whitney U Test	Compare distributions between two independent groups	Distribution-free; does not assume normality or equal variances
Paired t-Test	Compare means between two related groups	Normality of the difference scores	Wilcoxon Signed-Rank Test	Compare the median of paired differences	Distribution-free; does not assume normality
Two-Sample t-Test	Compare means between two independent groups	Normality, equal variances in groups	Mann-Whitney U Test	Compare distributions between two independent groups	Distribution-free; does not assume normality or equal variances
One-Way ANOVA	Compare means across three or more groups	Normality, equal variances among groups	Kruskal-Wallis Test	Compare distributions across three or more independent groups	Distribution-free; does not assume normality or equal variances
Repeated Measures ANOVA	Compare means across	Normality, sphericity (equal	Friedman Test	Compare distributions across	Distribution-free; does not

Test Type	Purpose	Assumptions	Non-Parametric Equivalent	Non-Parametric Test Purpose	Non-Parametric Test Assumptions
	multiple related groups	variances of differences)		multiple related groups	assume normality
ANCOVA					

138. Descriptive Statistics

138.1. Mean

Sum of all the values divided by the number of values

$$\mu = \frac{\sum_{i=1}^{n} x_i}{n}$$

Example

$$[1, 2, 3]$$

$$\bar{x} = \frac{1 + 2 + 3}{3} = 150$$

138.2. Median

Middle value in a set of values when they are arranged in ascending or descending order

Example

1. **Odd** Number of Values

$$[1, 2, 3]$$

   - **Step 1**: Arrange the Data in Ascending Order

$$[1, 2, 3]$$

   - **Step 2**: Identify the Median

$$\text{Median} = 2$$

2. **Even** Number of Values

$$[1, 2, 3, 4]$$

- **Step 1**: Arrange the Data in Ascending Order

$$[1, 2, 3, 4]$$

- **Step 2**: Identify the Median

$$\text{Median} = \frac{2+3}{2} = \frac{5}{2} = 2.5$$

138.3. Mode

Value that appears most frequently

Example

$$[1, 1, 2, 3]$$

- **Step 1**: Identify the Most Frequent Number

$$\begin{aligned} 1 &: 2 \\ 2 &: 1 \\ 3 &: 1 \end{aligned}$$

- **Step 2**: Determine the Mode

$$\text{Mode} = 1$$

138.4. Range

$$\text{Range} = \max - \min$$

Example

$$[1, 2, 3, 4, 5]$$

- **Step 1**: Identify the Maximum and Minimum Values

$$\begin{aligned} \max &= 5 \\ \min &= 1 \end{aligned}$$

- **Step 2**: Calculate the Range

$$\text{Range} = 5 - 1 = 4$$

138.5. Variance

Quantifies the spread or dispersion of a set of data points in a dataset

- Population

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^{n} (x_i - \mu)^2$$

- Sample

$$s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2$$

Example

$$[10,\ 12,\ 13,\ 17,\ 20,\ 24]$$

Step 1: Find mean

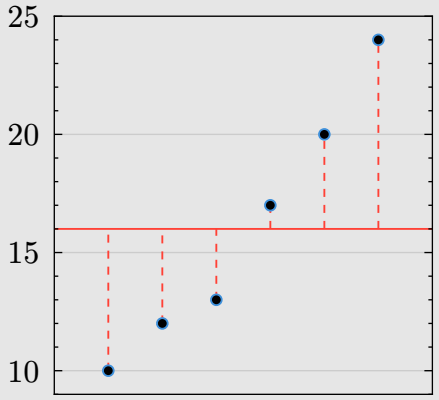
$$\bar{x} = \frac{10 + 12 + 13 + 17 + 20 + 24}{6} = \frac{96}{6} = 16$$

Step 2: Subtract the Mean and Square the result

$$(10 - 16)^2 = -6^2 = 36$$
$$(12 - 16)^2 = -4^2 = 16$$
$$(13 - 16)^2 = -3^2 = 9$$
$$(17 - 16)^2 = 1^2 = 1$$
$$(20 - 16)^2 = 4^2 = 16$$
$$(24 - 16)^2 = 8^2 = 64$$

Step 3: Calculate variance

$$s^2 = \frac{36 + 16 + 9 + 1 + 16 + 64}{6 - 1} = \frac{142}{5} = 28.4$$



138.6. Standard Deviation

Quantifies the amount of variation or dispersion in a set of data points

- Population

$$\sigma = \sqrt{\sigma^2}$$

$$\sigma = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_i - \mu)^2}$$

- Sample

$$s = \sqrt{s^2}$$

$$s = \sqrt{\frac{1}{n-1}\sum_{i=1}^{N}(x_i - \bar{x})^2}$$

138.7. Interquartile Range

$$\mathrm{IQR} = \mathrm{Q3} - \mathrm{Q1}$$

Example

$$[1, 2, 3, 4, 5, 6, 7]$$

Step 1: Arrange the Data in Ascending Order

$$[1, 2, 3, 4, 5, 6, 7]$$

Step 2: Find the Quartiles

1. Calculate the Median (Q2)

$$\text{Median } (\mathrm{Q2}) = 4$$

2. Find First Quartile (Q1)

Q1 is the median of the first half of the dataset

$$\mathrm{Q1} = 2$$

3. Find Third Quartile (Q3)

Q3 is the median of the second half of the dataset

$$\mathrm{Q3} = 5$$

Step 3: Calculate the Interquartile Range (IQR)

$$\mathrm{IQR} = 5 - 2 = 3$$

139. Probability Distributions

139.1. Bernoulli

139.1.1. Bernoulli Trial (Binomial trial)

A Bernoulli trial is a random experiment with exactly two possible outcomes: “success” and “failure,” where the probability of success (p) remains constant across trials.

$P(\text{success}) = p$

$P(\text{failure}) = q$

$p = 1 - q$

$q = 1 - p$

$p + q = 1$

Example

- Flipping a fair coin where “heads” is considered a **success** ($p = 0.5$) and “tails” a **failure** ($1 - p = 0.5$).
- Rolling a die and checking if the outcome is a 6 (**success**) or not (**failure**). If the die is fair, the probability of success is $p = \frac{1}{6}$ and the probability of failure is $1 - p = \frac{5}{6}$.
- Checking whether a shipment arrives on time (**success**) or is delayed (**failure**). If historical data shows that a shipment has a 90% chance of arriving on time, the probability of success (p) is 0.9, and the probability of failure ($1 - p$) is 0.1.

139.1.1.1. Odds

Odds for (o_f)

$$o_f = \frac{p}{q} = \frac{p}{1-p} = \frac{1-q}{q}$$

Odds against (o_a)

$$o_a = \frac{q}{p} = \frac{1-p}{p} = \frac{q}{1-q}$$

$$o_f = \frac{1}{o_a}$$

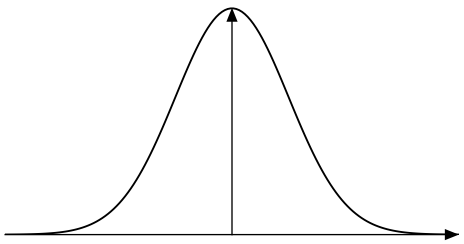
$$o_a = \frac{1}{o_f}$$

$$o_f \cdot o_a = 1$$

139.2. Gaussian

139.2.1. Gaussian (Normal) distribution

$$f(x \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$



139.2.1.1. Derivative

We want $f'(x) = \frac{\mathrm{d}}{\mathrm{d}x} f(x)$, treating μ, σ as constants.

The constant factor pulls out, leaving the exponential:

$$f'(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \cdot \frac{\mathrm{d}}{\mathrm{d}x} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

Chain rule. Let $u(x) = -\frac{(x-\mu)^2}{2\sigma^2}$, so the exponential is $\exp(u(x))$ and

$$\frac{\mathrm{d}}{\mathrm{d}x} \exp(u(x)) = \exp(u(x)) \cdot u'(x)$$

Differentiate u. Pull out $-\frac{1}{2\sigma^2}$, then chain-rule $(x-\mu)^2$:

$$u'(x) = -\frac{1}{2\sigma^2} \cdot 2(x-\mu) = -\frac{x-\mu}{\sigma^2}$$

Assemble. The last two factors reconstruct $f(x)$ itself:

$$f'(x) = \underbrace{\frac{1}{\sqrt{2\pi\sigma^2}} \cdot \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)}_{f(x)} \cdot \left(-\frac{x-\mu}{\sigma^2}\right)$$

$$\boxed{f'(x) = -\frac{x-\mu}{\sigma^2} \cdot f(x)}$$

Setting $\mu = 0$, $\sigma = 1$ collapses f to the standard normal ϕ:

Distribution	Derivative
General normal $f(x)$	$f'(x) = -\frac{x-\mu}{\sigma^2} \cdot f(x)$
Standard normal $\phi(z)$	$\phi'(z) = -z \cdot \phi(z)$

139.3. Differentiating Normal Distribution

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

We want $f'(x) = \frac{\mathrm{d}}{\mathrm{d}x} f(x)$

- μ, σ: constants
- x: variable

Step 1. Structure

Two factors:

$$f(x) = \underbrace{\frac{1}{\sqrt{2\pi\sigma^2}}}_{\text{constant}} \cdot \underbrace{\exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)}_{\text{function of } x}$$

First factor is a constant (does not depend on x):

$$f'(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \cdot \frac{\mathrm{d}}{\mathrm{d}x} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

Step 2: Set up the chain rule

$$u(x) = -\frac{(x-\mu)^2}{2\sigma^2}$$

Then

$$\exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) = \exp(u(x))$$

The chain rule:

$$\frac{\mathrm{d}}{\mathrm{d}x} \exp(u(x)) = \exp(u(x)) \cdot u'(x)$$

We need 2 things:

- $\exp(u(x)) = \exp(u(x))$
- $u'(x)$

Step 3. Differentiate $u(x)$

$$u(x) = -\frac{(x-\mu)^2}{2\sigma^2}$$

3.a. Pull out constant

$$u(x) = -\frac{1}{2\sigma^2} \cdot (x-\mu)^2$$

The factor $-\frac{1}{2}\sigma^2$ depends only on σ, not x. Constants pull out of derivatives:

$$u'(x) = -\frac{1}{2\sigma^2} \cdot \frac{\mathrm{d}}{\mathrm{d}x}(x-\mu)^2$$

Now we need $\frac{\mathrm{d}}{\mathrm{d}x}(x-\mu)^2$

3.b. Apply chain rule again to $(x-\mu)^2$

This is itself a composition:

- Outer function: v^2 where $v = x - \mu$
- Inner function: $v(x) = x - \mu$

Chain rule:

$$\frac{\mathrm{d}}{\mathrm{d}x}(x-\mu)^2 = \frac{\mathrm{d}}{\mathrm{d}v}(v^2) \cdot \frac{\mathrm{d}v}{\mathrm{d}x}$$

Outer derivative:

$$\frac{\mathrm{d}}{\mathrm{d}v}(v^2) = 2v = 2(x-\mu)$$

Inner derivative: $v(x) = x - \mu$

Since μ is a constant, $\frac{\mathrm{d}u}{\mathrm{d}x} = 0$. So:

$$\frac{\mathrm{d}v}{\mathrm{d}x} = \frac{\mathrm{d}}{\mathrm{d}x}(x-\mu) = \frac{\mathrm{d}}{\mathrm{d}x}(x) - \frac{\mathrm{d}}{\mathrm{d}x}(\mu) = 1 - 0 = 1$$

Combine:

$$\frac{\mathrm{d}}{\mathrm{d}x}(x-\mu)^2 = 2(x-\mu)\cdot 1 = 2(x-\mu)$$

3.c. Put Step 3 together

Substitute back into expression for $u'(x)$

$$u'(x) = -\frac{1}{2\sigma^2}\cdot(x-\mu)$$

Simplify be canceling the 2's:

$$u'(x) = -\left(2\frac{x-\mu}{2\sigma^2}\right) = -\frac{x-\mu}{\sigma^2}$$

So:

$$u'(x) = -\frac{x-\mu}{\sigma^2}$$

Step 4. Apply chain tule from Step 2

We had:

$$\frac{\mathrm{d}}{\mathrm{d}x}\exp(u(x)) = \exp(u(x))\cdot u'(x)$$

Plug in:

- $\exp(u(x)) = \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$
- $u'(x) = -\frac{x-\mu}{\sigma^2}$

So:

$$\frac{\mathrm{d}}{\mathrm{d}x}\exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) = \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)\cdot\left(-\frac{x-\mu}{\sigma^2}\right)$$

Move the factor to the front for clarity:

$$-\frac{x-\mu}{\sigma^2}\cdot\exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

Step 5. Putting everything together

Recall from Step 1:

$$f'(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \cdot \frac{\mathrm{d}}{\mathrm{d}x} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

Substitute what we found:

$$f'(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \cdot \left[-\frac{x-\mu}{\sigma^2} \cdot \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)\right]$$

Pull the $-\frac{x-\mu}{\sigma^2}$ out front:

$$f'(x) = -\frac{x-\mu}{\sigma^2} \cdot \frac{1}{\sqrt{2\pi\sigma^2}} \cdot \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

Step 6. Recognize $f(x)$ on the right side

Look at the last two factors

$$\frac{1}{\sqrt{2\pi\sigma^2}} \cdot \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) = f(x)$$

That's exactly the original PDF. So:

$$f'(x) = -\frac{x-\mu}{\sigma^2} \cdot f(x)$$

Step 7. Standard Normal

Does this collapse to $\phi'(z)$ when $\mu = 0$ and $\sigma = 1$?

Plug in $\mu = 0$, $\sigma = 1$:

$$f'(x) = -\frac{x-0}{1^2} \cdot f(x) = -x \cdot f(x)$$

With $\mu = 0$ and $\sigma = 1$, $f(x)$ becomes $\phi(x)$, so:

$$\phi(x) = -x \cdot \phi(x)$$

Distribution	Derivative
General Normal $f(x)$	$f'(x) = -\frac{x-\mu}{\sigma^2} \cdot f(x)$
Standard Normal $\phi(x)$	$\phi'(z) = -z \cdot \phi(z)$

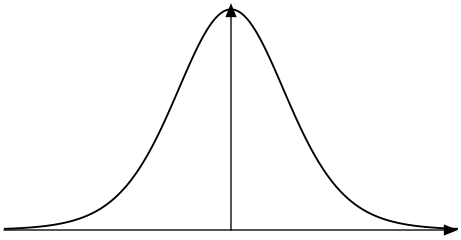
139.4. Student's t

139.4.1. t-Distribution

$$f(t \mid \nu) = \frac{\Gamma\left(\frac{\nu+1}{2}\right)}{\sqrt{\nu\pi}\,\Gamma\left(\frac{\nu}{2}\right)} \left(1 + \frac{t^2}{\nu}\right)^{-\frac{\nu+1}{2}}$$

Where:

- t: t-statistic
- ν (or df): degrees of freedom
- Γ: Gamma function (generalizes the factorial function)



Continuous probability distribution for estimating the mean of a normally distributed population in situations where:

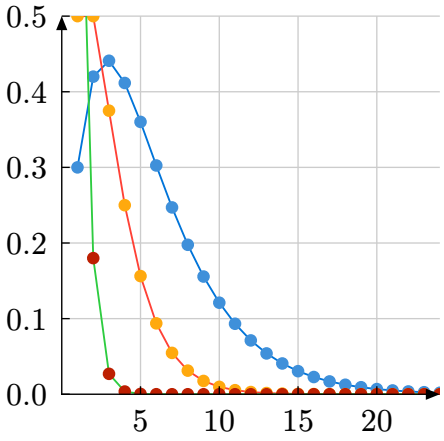
- Sample size is small
- Population standard deviation is unknown

Similar in shape to the normal distribution but has heavier tails, which means it gives more probability to values further from the mean

139.5. Binomial

Discrete probability distribution that describes the number of successes in a fixed number of independent Bernoulli trials, each with the same probability of success

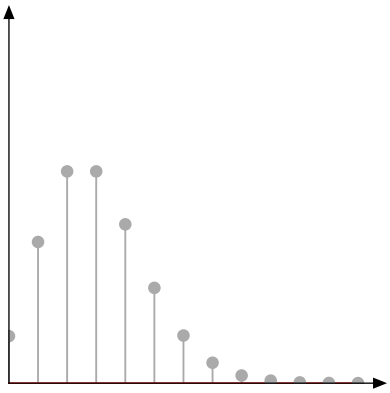
$$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}$$



139.6. Poisson

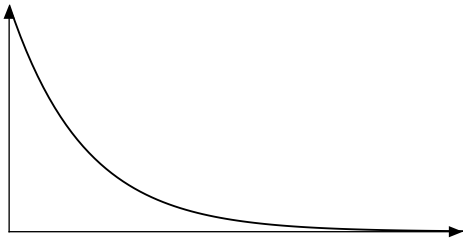
Used to model the number of events that occur within a fixed interval of time or space, given a constant mean rate and assuming that these events occur independently of each other

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}$$



139.7. Exponential

$$f(x \mid \lambda) = \lambda e^{-\lambda x}$$



139.8. Differentiating the Normal Distribution

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

We want $f'(x) = \frac{\mathrm{d}}{\mathrm{d}x} f(x)$

- μ, σ: constants
- x: variable

Step 1. Structure

Two factors:

$$f(x) = \underbrace{\frac{1}{\sqrt{2\pi\sigma^2}}}_{\text{constant}} \cdot \underbrace{\exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)}_{\text{function of } x}$$

First factor is a constant (does not depend on x):

$$f'(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \cdot \frac{\mathrm{d}}{\mathrm{d}x} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

Step 2: Set up the chain rule

$$u(x) = -\frac{(x-\mu)^2}{2\sigma^2}$$

Then

$$\exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) = \exp(u(x))$$

The chain rule:

$$\frac{\mathrm{d}}{\mathrm{d}x}\exp(u(x)) = \exp(u(x)) \cdot u'(x)$$

We need 2 things:

- $\exp(u(x)) = \exp(u(x))$
- $u'(x)$

Step 3. Differentiate $u(x)$

$$u(x) = -\frac{(x-\mu)^2}{2\sigma^2}$$

3.a. Pull out constant

$$u(x) = -\frac{1}{2\sigma^2} \cdot (x-\mu)^2$$

The factor $-\frac{1}{2}\sigma^2$ depends only on σ, not x. Constants pull out of derivatives:

$$u'(x) = -\frac{1}{2\sigma^2} \cdot \frac{\mathrm{d}}{\mathrm{d}x}(x-\mu)^2$$

Now we need $\frac{\mathrm{d}}{\mathrm{d}x}(x-\mu)^2$

3.b. Apply chain rule again to $(x-\mu)^2$

This is itself a composition:

- Outer function: v^2 where $v = x - \mu$
- Inner function: $v(x) = x - \mu$

Chain rule:

$$\frac{\mathrm{d}}{\mathrm{d}x}(x-\mu)^2 = \frac{\mathrm{d}}{\mathrm{d}v}(v^2) \cdot \frac{\mathrm{d}v}{\mathrm{d}x}$$

Outer derivative:

$$\frac{\mathrm{d}}{\mathrm{d}v}(v^2) = 2v = 2(x-\mu)$$

Inner derivative: $v(x) = x - \mu$

Since μ is a constant, $\frac{\mathrm{d}u}{\mathrm{d}x} = 0$. So:

$$\frac{\mathrm{d}v}{\mathrm{d}x} = \frac{\mathrm{d}}{\mathrm{d}x}(x-\mu) = \frac{\mathrm{d}}{\mathrm{d}x}(x) - \frac{\mathrm{d}}{\mathrm{d}x}(\mu) = 1 - 0 = 1$$

Combine:

$$\frac{\mathrm{d}}{\mathrm{d}x}(x-\mu)^2 = 2(x-\mu) \cdot 1 = 2(x-\mu)$$

3.c. Put Step 3 together

Substitute back into expression for $u'(x)$

$$u'(x) = -\frac{1}{2\sigma^2} \cdot (x - \mu)$$

Simplify be canceling the 2's:

$$u'(x) = -\left(2\frac{x-\mu}{2\sigma^2}\right) = -\frac{x-\mu}{\sigma^2}$$

So:

$$u'(x) = -\frac{x-\mu}{\sigma^2}$$

Step 4. Apply chain tule from Step 2

We had:

$$\frac{\mathrm{d}}{\mathrm{d}x}\exp(u(x)) = \exp(u(x)) \cdot u'(x)$$

Plug in:

- $\exp(u(x)) = \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$
- $u'(x) = -\frac{x-\mu}{\sigma^2}$

So:

$$\frac{\mathrm{d}}{\mathrm{d}x}\exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) = \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) \cdot \left(-\frac{x-\mu}{\sigma^2}\right)$$

Move the factor to the front for clarity:

$$-\frac{x-\mu}{\sigma^2} \cdot \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

Step 5. Putting everything together

Recall from Step 1:

$$f'(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \cdot \frac{\mathrm{d}}{\mathrm{d}x}\exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

Substitute what we found:

$$f'(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \cdot \left[-\frac{x-\mu}{\sigma^2} \cdot \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)\right]$$

Pull the $-\frac{x-\mu}{\sigma^2}$ out front:

$$f'(x) = -\frac{x-\mu}{\sigma^2} \cdot \frac{1}{\sqrt{2\pi\sigma^2}} \cdot \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

Step 6. Recognize $f(x)$ on the right side

Look at the last two factors

$$\frac{1}{\sqrt{2\pi\sigma^2}} \cdot \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) = f(x)$$

That's exactly the original PDF. So:

$$f'(x) = -\frac{x-\mu}{\sigma^2} \cdot f(x)$$

Step 7. Standard Normal

Does this collapse to $\phi'(z)$ when $\mu = 0$ and $\sigma = 1$?

Plug in $\mu = 0$, $\sigma = 1$:

$$f'(x) = -\frac{x-0}{1^2} \cdot f(x) = -x \cdot f(x)$$

With $\mu = 0$ and $\sigma = 1$, $f(x)$ becomes $\phi(x)$, so:

$$\phi(x) = -x \cdot \phi(x)$$

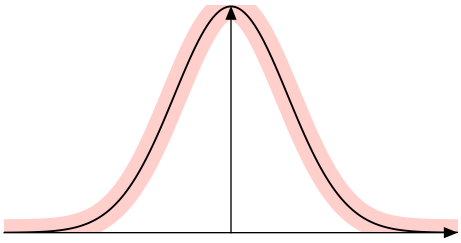
Distribution	Derivative
General Normal $f(x)$	$f'(x) = -\frac{x-\mu}{\sigma^2} \cdot f(x)$
Standard Normal $\phi(x)$	$\phi'(z) = -z \cdot \phi(z)$

140. Functions

140.1. PDF

140.1.1. PDF (Probability Density Function)

Function that describes the likelihood of a continuous random variable taking on a particular value



Properties:

- The area under the curve of a PDF over the entire range of possible values equals 1
- The PDF itself is non-negative everywhere
- The probability that the variable falls within a certain range is given by the integral of the PDF over that range

140.1.1.1. Relationship to the CDF

The PDF is the derivative of the CDF; integrating the PDF back recovers the CDF:

$$f(x) = \frac{\mathrm{d}}{\mathrm{d}x} F(x) \quad \Longleftrightarrow \quad F(x) = \int_{-\infty}^{x} f(t)\,\mathrm{d}t$$

$$f(x) = \frac{\mathrm{d}}{\mathrm{d}x} F(x)$$

140.2. PMF

140.2.1. PMF (Probability Mass Function)

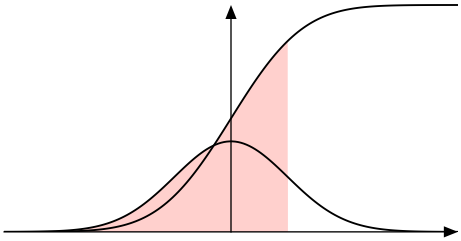
Frequency distribution that provides the probability that a categorical variable takes on each of its possible values

140.3. CDF

140.3.1. CDF (Cumulative Distribution Function)

Gives the probability that X will take a value less than or equal to x

$$F(x) = P(X \leq x)$$



1. Categorical

$$F(x) = \int_{-\infty}^{x} f(t)dt$$

2. Continuous

$$F(x) = \sum_{t \leq x} P(X = t)$$

cdf.py

```python
from scipy.stats import norm

x = 1
mu = 0
sigma = 1

norm.cdf(x, loc=mu, scale=sigma)
```

$$F(x) = \int_{-\infty}^{x} f(t)\,\mathrm{d}t$$

140.4. PPF

140.4.1. PPF (Percent-Point Function)

Gives the value x such that the probability of a random variable being less than or equal to x is equal to a given probability p

ppf.py

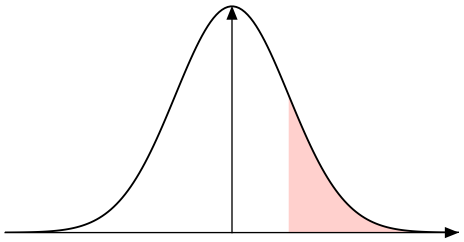
```
from scipy.stats import norm

p = 0.95
mu = 0
sigma = 1

x_value = norm.ppf(p, loc=mu, scale=sigma)
```

140.5. SF

140.5.1. SF (Survival Function)

Probability that a certain event has not occurred by a certain time

$$S(t) = P(T > t)$$



Relationship to PDF:

$$S(t) = 1 - F(t)$$

sf.py

```
from scipy.stats import norm

z = 3.4
mu = 0
sigma = 1

```

```
norm.sf(z, loc=mu, scale=sigma)
```

140.6. Standard Normal Loss Function

The **standard normal loss function**, denoted $L(z)$ or $G(z)$, is the **expected shortfall above a threshold z** for a standard normal random variable $Z \sim \mathcal{N}(0,1)$:

$$L(z) = E[(Z-z)^+] = \int_z^\infty (t-z)\varphi(t)\,dt$$

where φ is the standard normal PDF and $(x)^+ = \max(x, 0)$.

140.6.1. Closed form

Integration by parts gives:

$$L(z) = \varphi(z) - z[1-\Phi(z)]$$

where Φ is the standard normal CDF. Both $\varphi(z)$ and $\Phi(z)$ are tabulated; combining them gives $L(z)$.

Example

$$L(0) = \varphi(0) - 0 = \frac{1}{\sqrt{2\pi}} \approx 0.3989$$
$$L(1) = \varphi(1) - 1\cdot[1-\Phi(1)] = 0.2420 - 0.1587 \approx 0.0833$$
$$L(2) = \varphi(2) - 2\cdot[1-\Phi(2)] = 0.0540 - 2\cdot 0.0228 \approx 0.0085$$

$L(z)$ decreases rapidly with z — large thresholds have negligible expected shortfall.

140.6.2. Derivation: tail minus threshold

Where does the closed form come from? Split the excess $(t-z)$ accumulated over the tail into **total tail value** minus **threshold counted across the tail**.

Quantity A — tail value. The full value t over the tail, weighted by likelihood. Using $t\varphi(t) = -\varphi'(t)$:

$$A = \int_z^\infty t\,\varphi(t)\,dt = \varphi(z)$$

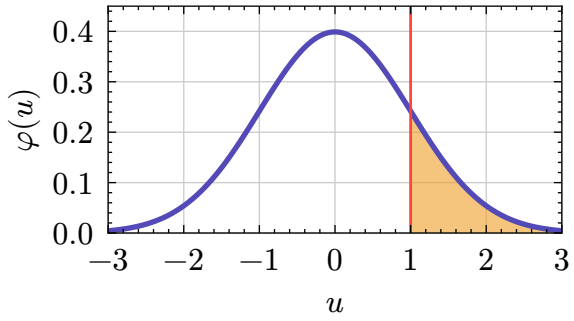
Quantity B — threshold across the tail. Count z once per tail outcome:

$$B = \int_z^\infty z\,\varphi(t)\,dt = z[1-\Phi(z)]$$

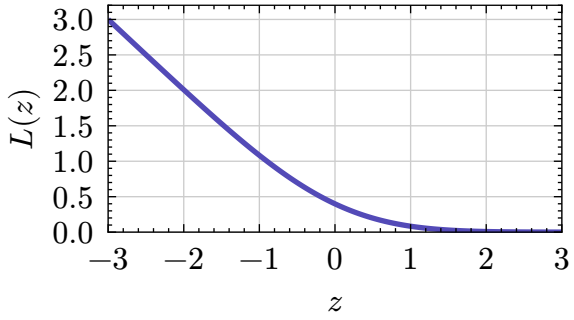
Subtract. Each tail outcome contributes t to A and z to B, so $A - B$ strips the baseline and leaves exactly the excess $(t-z)$:

$$L(z) = A - B = \underbrace{\int_z^\infty t\varphi(t)\,dt}_{\text{total tail value}} - \underbrace{\int_z^\infty z\varphi(t)\,dt}_{\text{threshold across tail}} = \varphi(z) - z[1 - \Phi(z)]$$

The shaded tail below is the region both integrals run over; $L(z)$ is the per-tail gap between the two.



$L(z)$ itself decreases monotonically and convexly toward 0:



140.6.3. Table of values

z	$L(z)$	z	$L(z)$	z	$L(z)$
0.0	0.3989	1.0	0.0833	2.0	0.0085
0.1	0.3509	1.1	0.0686	2.1	0.0065
0.2	0.3069	1.2	0.0561	2.2	0.0049
0.3	0.2668	1.3	0.0455	2.3	0.0037
0.4	0.2304	1.4	0.0367	2.4	0.0027
0.5	0.1978	1.5	0.0293	2.5	0.0020
0.6	0.1687	1.6	0.0232	2.6	0.0015
0.7	0.1429	1.7	0.0183	2.7	0.0011
0.8	0.1202	1.8	0.0143	2.8	0.0008
0.9	0.1004	1.9	0.0111	2.9	0.0005

For $z < 0$: $L(z) = L(-z) + (-z) = L(|z|) - z$ (use symmetry of φ).

140.6.4. Use case: safety-stock derivations

The reason this function is everywhere in supply-chain math: the expected shortfall (in stock units) per replenishment cycle, when demand during lead time is $\mathcal{N}(\mu_L, \sigma_L^2)$ and the reorder point is $r = \mu_L + z\sigma_L$:

$$\mathrm{E}[\text{shortfall per cycle}] = \sigma_L \, L(z)$$

So $L(z)$ converts "how many sigma above the mean did I stock" into "expected units short per replenishment." This drives:

- **Fill rate**: $\beta = 1 - \frac{\sigma_L \, L(z)}{Q}$ — see Fill Rate
- **Safety stock**: solve $L(z) = \frac{(1-\beta)Q}{\sigma_L}$ for z, then $\mathrm{SS} = z\sigma_L$
- **Newsvendor expected lost sales**: $\sigma \, L(z^*)$ at the optimal critical-ratio quantile
- **Newsvendor expected leftover**: $\sigma \, [L(z) + z]$

140.6.5. Properties

- **Non-increasing**: $L(z)$ decreases monotonically as z increases (more inventory → less expected shortfall)
- **Asymptotic**: $L(z) \to 0$ as $z \to \infty$; $L(z) \sim -z$ as $z \to -\infty$
- **Derivative**: $L'(z) = -[1 - \Phi(z)] = -P(Z > z)$ (note: $L'(z) \leq 0$, confirming monotonicity)
- **Convexity**: $L(z)$ is convex — second derivative $L''(z) = \varphi(z) \geq 0$

140.6.6. Connection to standard normal PDF / CDF

$$L(z) = \varphi(z) - z\,\overline{\Phi}(z), \qquad \overline{\Phi}(z) = 1 - \Phi(z)$$

— see PDF, CDF, SF.

140.6.7. See also

- **Gaussian Distribution**
- **Fill Rate**
- **Safety Stock**
- **Newsvendor**

140.7. Standard Normal Loss Function

$$L(k) = \mathbb{E}[(Z - k)^+]$$

$L(k)$ is the expected amount by which Z exceeds the threshold k:

- $Z < k$: excess is 0
- $Z > k$: excess is $Z - k$. Average this across all the ways Z could come in above k, weighted by probability

$$\boxed{L(k) = \int_k^\infty (z - k)\phi(z)\,\mathrm{d}z}$$

$$\boxed{L(k) = \phi(k) - k \cdot (1 - \Phi(k))}$$

We want $\mathbb{E}[\max(X - k, 0)]$ - the average excess of X above the threshold k, where the excess is **zero** when $X \leq k$.

Imaging 2 related quantities:

Quantity A: Tail Contribution

Sum up the value of X at each tail outcome, weighted by probability:

$$A = \int_k^\infty u \cdot \phi(u)\,\mathrm{d}u = \phi(k)$$

This is "if X is in the tail, what value does it take, on average - weighted by likelihood?" Note that this counts the **full** value of u, not the excess $u - k$.

Quantity B: Threshold Contribution

$$B = \int_k^\infty k \cdot \phi(u)\,\mathrm{d}u = k \cdot [1 - \Phi(k)] = k \cdot \mathrm{sf}(k)$$

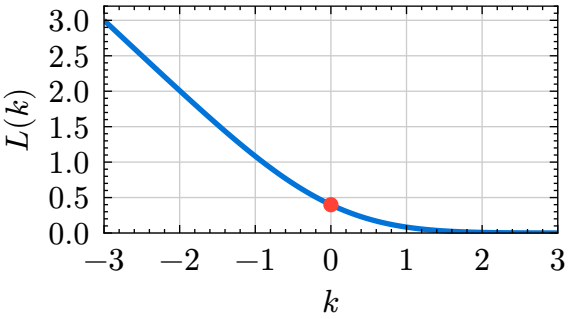
This is "for every tail outcome, count k once, weighted by probability of being there"

Subtraction

Each tail outcome at value u contributes u to A and k to B. So the difference A - B contributes (u - k) per tail outcome - **exactly the excess we want**

$$L(k) = A - B = \underbrace{\int_k^\infty u\phi(u)\,\mathrm{d}u}_{\text{total tail values}} - \underbrace{\int_k^\infty k\phi(u)\,\mathrm{d}u}_{\text{treshold counted across tail}}$$

The subtraction strips off the "baseline up to k" porton of each tail outcome, leaving only the part **above** k. Without that subtraction, we'd be over counting by k for every tail occurrence.

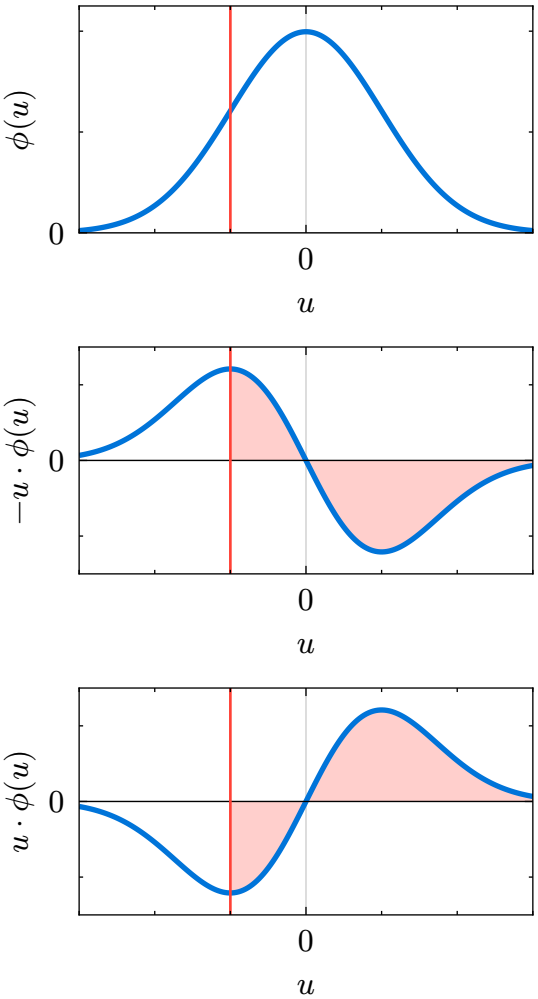


$$L(k) = \phi(k) - k \cdot (1 - \Phi(k))$$

```python
from scipy.stats import norm
import numpy as np

mu, sigma = 50, 12
c_u, c_o = 5, 3

# Optimal Q from critical ratio
cr = c_u / (c_u + c_o)
k_star = norm.ppf(cr)
Q_star = mu + sigma * k_star

# Standard normal loss function
```

```python
def L(k):
    return norm.pdf(k) - k * (1 - norm.cdf(k))

expected_lost_sales = sigma * L(k_star)
expected_sales = mu - expected_lost_sales
expected_leftover = Q_star - mu + sigma * L(k_star)
```

$\phi(u)$	PDF
$\phi'(u) = -u\phi(u)$	Derivative of PDF
$u\phi(u) = -\phi'(u)$	Negative derivative of PDF



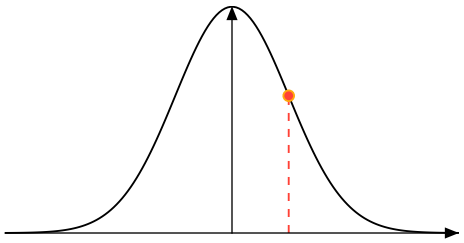
140.8. Likelihood

$$\mathcal{L}\left(\theta \mid x\right) = f(x \mid \theta)$$

Where:

- θ: model parameters

- x: observed data



140.8.1. PDF vs. Likelihood

Same formula, read two ways — what is held **fixed** and what **varies** swap:

$$\phi(x;\mu,\sigma) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

Quantity	Fixed	Varies	Integrates / sums to 1
PDF $\phi(x;\mu,\sigma)$	μ, σ	x	Yes
Likelihood $\mathcal{L}(\mu,\sigma \mid x)$	x	μ, σ	No

- **PDF**: parameters fixed, sweep x — a curve over outcomes that integrates to 1.
- **Likelihood**: data fixed, sweep the parameters — **not** a density in μ, σ, so it need not integrate to 1.

For n independent observations the likelihood is the product of per-point densities:

$$\mathcal{L}(\mu,\sigma \mid x_1,...,x_n) = \prod_{i=1}^{n} \phi(x_i;\mu,\sigma)$$

141. Error Metrics

141.1. MD

141.1.1. MD (Mean Deviation / Bias)

Average of the signed forecast errors $e_t = x_t - \hat{x}_t$. Captures **bias** — whether the forecast systematically over- or under-shoots.

$$\mathrm{MD} = \frac{1}{n}\sum_{t=1}^{n} e_t = \frac{1}{n}\sum_{t=1}^{n}(x_t - \hat{x}_t)$$

- $\mathrm{MD} \approx 0$: forecast is unbiased on average
- $\mathrm{MD} > 0$: forecast under-predicts (actuals exceed forecast)
- $\mathrm{MD} < 0$: forecast over-predicts

Caveat. Positive and negative errors cancel out, so MD says nothing about **accuracy** — a forecast that alternates wildly between $+100$ and -100 has MD $= 0$. Pair MD with MAE / RMSE to separate bias from accuracy.

141.2. MAE

141.2.1. MAE (Mean Absolute Error)

Average of squared differences between predicted ($\hat{y}_i$) and actual values (y_i)

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

141.3. MSE

141.3.1. MSE (Mean Squared Error)

Average of squared differences between predicted ($\hat{y}_i$) and actual (y_i) values

$$\mathrm{MSE} = \frac{1}{2}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

141.4. RMSE

141.4.1. RMSE (Root Mean Squared Error)

square root of the average squared differences between predicted ($\hat{y}_i$) and actual (y_i) values

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

141.5. MPE

141.5.1. MPE (Mean Percent Error)

Signed-percentage version of MD. Captures **relative bias** — whether the forecast over- or under-shoots in percentage terms.

$$\mathrm{MPE} = \frac{1}{n}\sum_{t=1}^{n}\frac{e_t}{x_t} = \frac{1}{n}\sum_{t=1}^{n}\frac{x_t - \hat{x}_t}{x_t}$$

- $\mathrm{MPE} \approx 0$: forecast is unbiased on a percentage scale
- $\mathrm{MPE} > 0$: under-prediction (actuals exceed forecast)
- $\mathrm{MPE} < 0$: over-prediction

Caveats.

- Positive and negative errors cancel — same warning as MD. Pair with MAPE to separate bias from accuracy.
- Undefined / unstable when any $x_t = 0$ or x_t is small relative to the error.

141.6. MAPE

141.6.1. MAPE (Mean Absolute Percentage Error)

Average percentage difference between predicted ($\hat{y}_i$) and actual (y_i) values

$$\text{MAPE} = \frac{100}{n} \sum_{i=1}^{n} \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

141.7. R^2

Proportion of variance explained by the model

$$R^2 = 1 - \frac{\sum_{i=1}^{n} (y_i - \hat{y}_i)^2}{\sum_{i=1}^{n} (y_i - \bar{y}_i)^2}$$

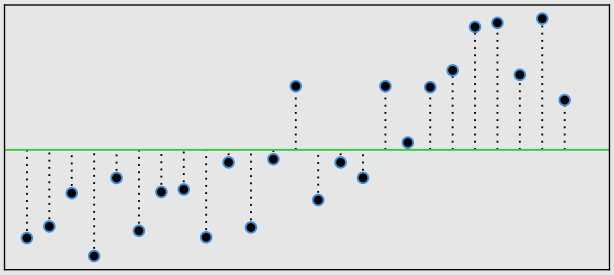
- 1: model explains all the variance in the dependent variable
- 0: model explains none of the variance in the dependent variable

Example

Step 1: Fit the Regression Model

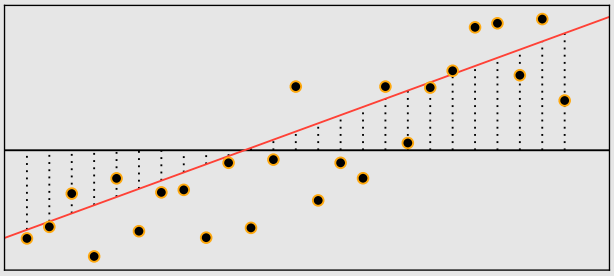
Step 2: Compute **Total Sum of Squares** (SST)

$$\text{SST} = \sum_{i=1}^{n} (y_i - \bar{y})^2$$



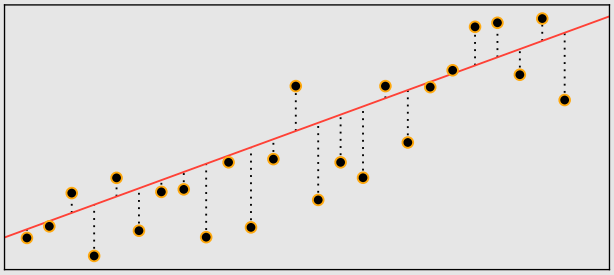
Step 3: Compute **Regression Sum of Squares** (SSR)

$$\text{SSR} = \sum_{i=1}^{n} = (\hat{y}_i - \bar{y})^2$$



Step 4: Compute **Residual Sum of Squares** (SSE)

$$\text{SSE} = \sum_{i=1}^{n} = (y_i - \hat{y}_i)^2$$



Step 5: Calculate R^2

$$R^2 = \frac{\text{SSR}}{\text{SST}} = 1 - \frac{\text{SSE}}{\text{SST}}$$

141.7.1. Adj R-squared

Adjusts the R^2 value based on the number of predictors (penalty for adding non-informative variables)

$$\text{Adj } R^2 = 1 - (1 - R^2)\frac{n-1}{n-p-1}$$

141.8. MSLE

141.8.1. MSLE (Mean Squared Logarithmic Error)

When predictions and actual values span several orders of magnitude

$$\text{MSLE} = \frac{1}{n}\sum_{i=1}^{n}(\log(1+y_i) - \log(1+\hat{y}_i))^2$$

141.9. Cross-Entropy Loss

141.9.1. Cross-Entropy Loss (Log Loss)

Binary and multi-class classification

$$\text{Log Loss} = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)]$$

142. Hypothesis Testing

142.1. Null Hypothesis

142.1.1. Null (H_0)

No effect or difference (observed effect is due to sampling variability)

$$H_0 : \mu = \mu_0$$

142.2. Alternative Hypothesis

142.2.1. Alternative (H_1 or H_a)

Presence of an effect or a difference

- Two-Tailed

$$H_1 : \mu \neq \mu_0$$

- Right-Tailed

$$H_1 : \mu > \mu_0$$

- Left-Tailed

$$H_1 : \mu < \mu_0$$

142.3. Type I Error

142.3.1. Type I (α)

$$\alpha = P(\text{Reject } H_0 \mid H_0 \text{ is true})$$

142.4. Type II Error

142.4.1. Type II (β)

$$\beta = P(\text{Fail to Reject } H_0 \mid H_1 \text{ is true})$$

142.5. One-Sample Z-Test for Mean

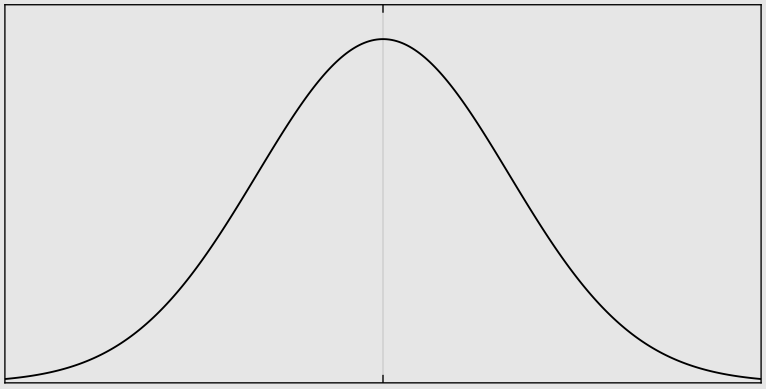
Warning: Pipulation standard deviation, σ, is known.

$$H_0 : \mu$$
$$H_a : \mu$$

Example

Your favorite candy company claims their chocolate bars have a mean weight of 200 grams, and their population standard deviation is 5 grams. You've noticed that they've seemed lighter than usual. You take a simple random sample of 35 candy bars and find they have a mean weight of 198 grams. At α = 0.05 test the company's claim.

$$H_0 : \mu \geq 200$$
$$H_a : \mu < 200$$

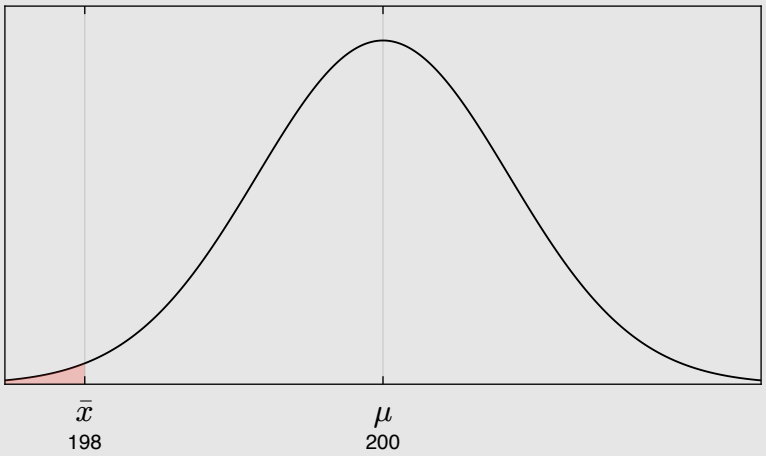


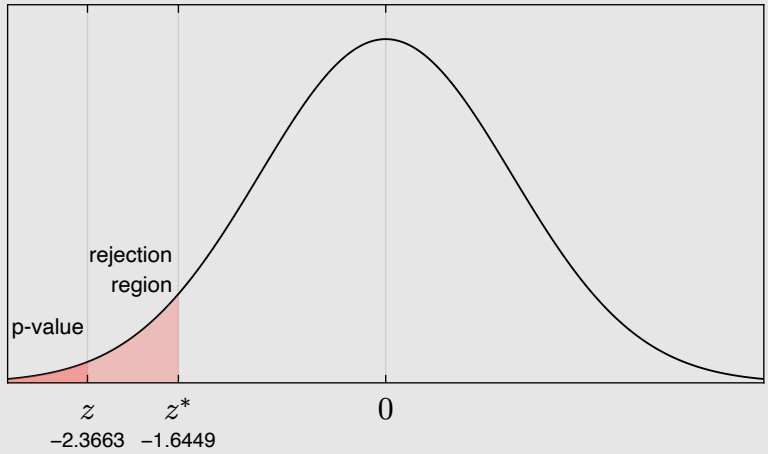
$$z = \frac{\bar{x} - \mu}{\sigma/\sqrt{n}}$$

Where:

- $\bar{x} = 198$
- $\mu = 200$
- $\sigma = 5$
- $n = 35$
- $\sigma/\sqrt{n} = 5/\sqrt{35} = 0.8452$

$$z = \frac{198 - 200}{5/\sqrt{35}} = -2.3663$$





$$P(z \leq -2.3663) = 0.009$$

$$z\text{-value} < \text{critical-value}$$
$$-2.3663 < -1.6449$$

$$p\text{-value} < \alpha\text{-value}$$
$$0.009 < 0.05$$

Reject H_0. At $\alpha = 0.05$ there is sufficient evidence that the mean weight of the chocolate bars is less than 200 grams.

Code

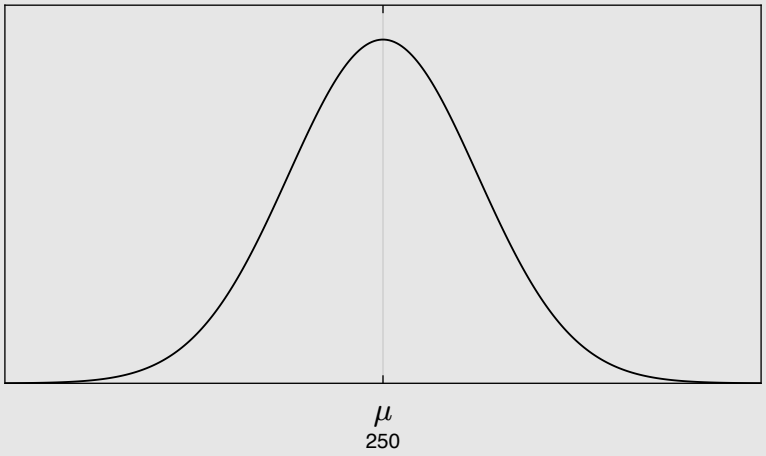
```python
import numpy as np
from scipy.stats import norm

mu = 200.0
sigma = 5.0
xbar = 198
n = 35
alpha = 0.05

cv = norm.ppf(alpha)

z = (xbar - mu) / (sigma / np.sqrt(n))
p = norm.cdf(z)

if z <= cv:
    print("Reject null hypothesis")
else:
    print("Fail to reject null hypothesis")
```

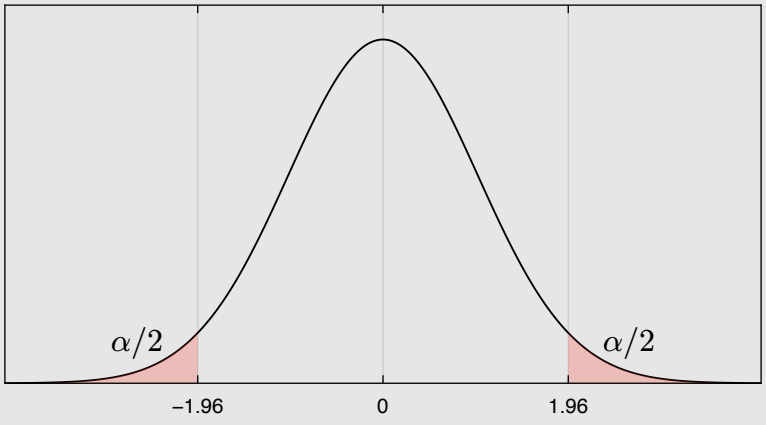
142.6. One-Sample Two-Tail Z-Test for Mean

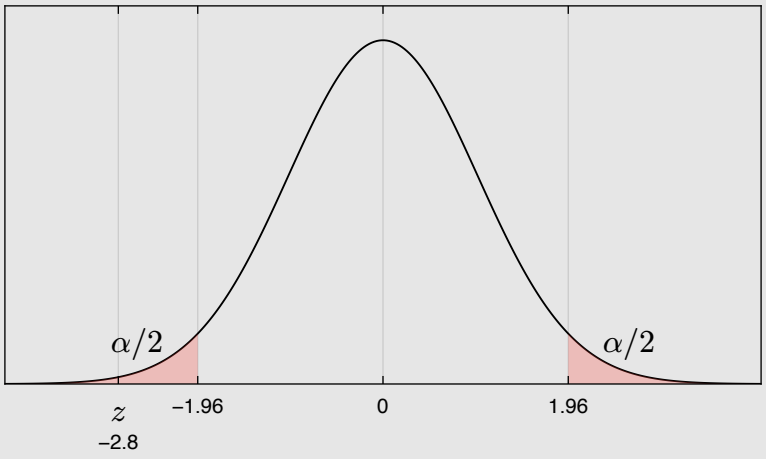
Example

A café's automatic dispenser is calibrated to pour 250 mL per cup. The manufacturer specifies that pour volume has a known standard deviation of 9 mL. After a maintenance visit, the owner wants to check whether the machine has drifted off target in either direction. A simple random sample of 36 pours has a mean volume of 245.8 mL. At α = 0.05, test whether the mean pour volume differs from the calibrated setting.

$$H_0 : \mu = 250$$
$$H_a : \mu \neq 250$$





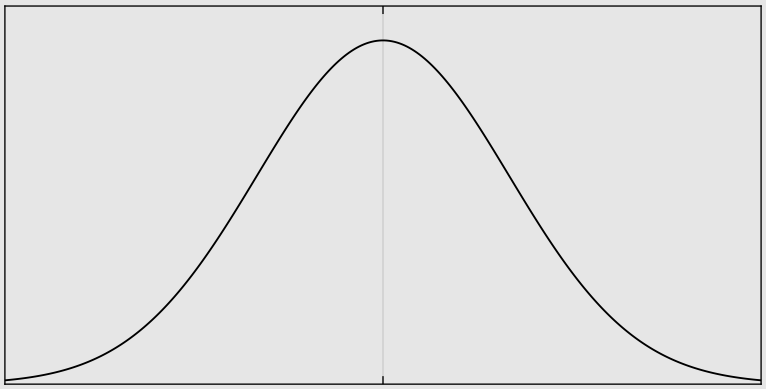


142.7. Two-Sample Z-Test for Proportion

Example

Your friend is convinced studying doesn't help. You believe it does, so you take 2 simple random samples of your Stat course: one for students who studied > 4 hrs and one for those who studied < 4 hrs. Of the 120 students who studied > 4 hrs, 103 passed. Of the 80 who studied less, 57 passed. At a = 0.05 test your claim.

$$H_0 : p_{>4} - p_{<4} \leq 0$$
$$H_a : p_{>4} - p_{<4} > 0$$



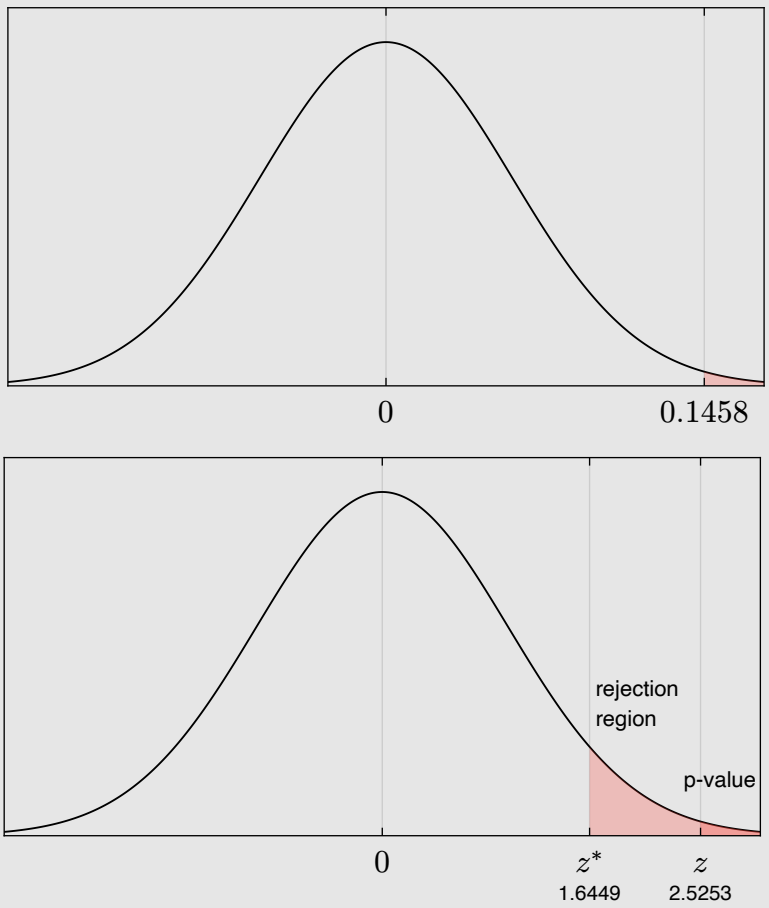
$$z = \frac{\hat{p}_{>4} - \hat{p}_{<4}}{\sqrt{\hat{p}_c(1-\hat{p}_c)\left(\frac{1}{n_{>4}} + \frac{1}{n_{<4}}\right)}}$$

Where:

- $\hat{p}_{>4} = \frac{103}{120} = 0.8583$
- $\hat{p}_{<4} = \frac{57}{80} = 0.7125$
- $n_{>4} = 120$
- $n_{<4} = 80$
- $\hat{p}_c = \frac{x_{>4}+x_{<4}}{n_{>4}+n_{<4}} = \frac{103+57}{120+80} = 0.8$

$$z = \frac{0.8583 - 0.7125}{\sqrt{0.8(1-0.8)\left(\frac{1}{120} + \frac{1}{80}\right)}} = 2.5253$$

$$\hat{p}_{>4} - \hat{p}_{<4} = 0.8583 - 0.7125 = 0.1458$$



$$P(z \geq 2.5253) = 0.0058$$

$$z\text{-value} > \text{critical-value}$$
$$2.5253 > 1.6449$$

$$p\text{-value} < \alpha\text{-value}$$
$$0.0058 < 0.05$$

Reject H_0. At $\alpha = 0.05$ there is sufficient evidence that students who studied more than 4 hours passed at a higher rate.

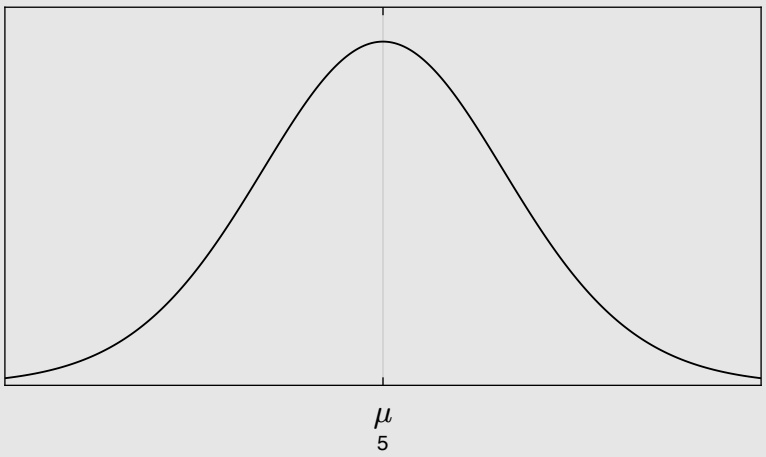
142.8. One-Sample T-Test for Mean

Population standard deviation, σ, is unknown.

Example

An airline claims their mean departure time is 5 minutes late, but you believe their flights are more delayed than that. You take a simple random sample of 20 flights. Your sample has a mean delay of 12 mins with a sample SD of 27 mins. Assume population flight times are normally distributed. At $\alpha = 0.05$ test the airline's claims.

$$H_0 : \mu \leq 5$$
$$H_a : \mu > 5$$

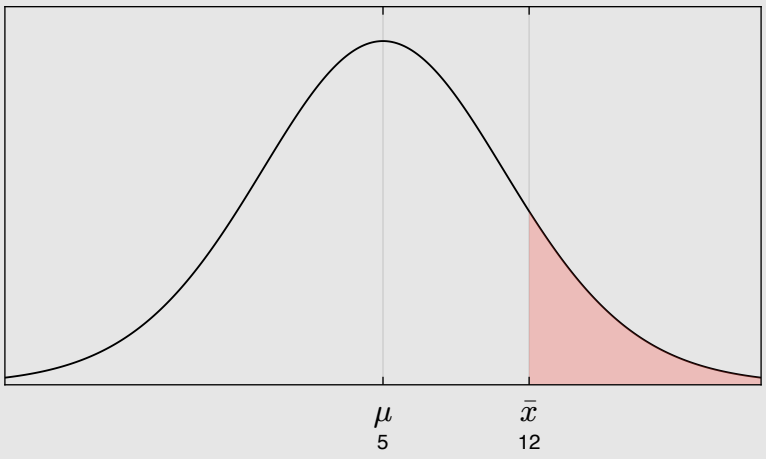


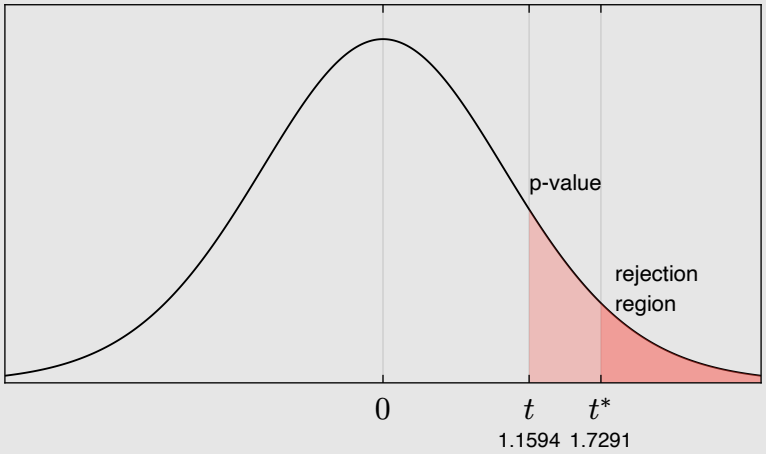
$$t = \frac{\bar{x} - \mu}{s/\sqrt{n}}$$

Where:

- $\bar{x} = 12$
- $\mu = 5$
- $s = 27$
- $n = 20$
- $\mathrm{df} = n - 1 = 19$
- $s/\sqrt{n} = 27/\sqrt{20} = 6.0374$

$$t = \frac{12 - 5}{27/\sqrt{20}} = 1.1594$$





$$P(t \geq 1.1594) = 0.1303$$

$$t\text{-value} < \text{critical-value}$$
$$1.1594 < 1.7291$$

$$p\text{-value} > \alpha\text{-value}$$
$$0.1303 > 0.05$$

Fail to reject H_0. At $\alpha = 0.05$ there is not sufficient evidence that the mean departure delay is more than 5 minutes.

Code

```python
import numpy as np
from scipy.stats import t

mu = 5
s = 27
xbar = 12
n = 20
df = n - 1
alpha = 0.05

cv = t.ppf(1 - alpha, df = df)

t_statistic = (xbar - mu) / (s / np.sqrt(n))
p = t.cdf(z)

if cv > t_statistic:
    print("Reject null hypothesis")
else:
    print("Fail to reject null hypothesis")
```

142.8.1. One-sample

Tests if the mean of a single sample differs from a known or hypothesized population mean.

$$t = \frac{\bar{x} - \mu_0}{\frac{s}{\sqrt{n}}}$$

- $\bar{x}$: sample mean
- μ_0: hypothesized population mean
- s: sample standard deviation
- n: sample size

Example

$$[78, 82, 89]$$

Step 1: State Hypotheses

$$H_0 : \mu = 85 \text{ cm}$$

$$H_1 : \mu \neq 85 \text{ cm}$$

Step 2: Summarize Data

- Sample values:

$$x_1 = 78, x_2 = 82, x_3 = 89$$

- Sample size:

$$n = 3$$

Step 3: Calculate Sample Mean ($\bar{x}$)

$$\bar{x} = \frac{x_1 + x_2 + x_3}{n} = \frac{78 + 82 + 89}{3} = 83$$

Step 4: Calculate Sample Standard Deviation:

$$s = \sqrt{\frac{\sum_{i=1}^{n} (x_i - \bar{x})^2}{n - 1}}$$

- Find the deviations from the mean and square them

$$(x_1 - \bar{x})^2 = (78 - 83)^2 = (-5)^2 = 25$$

$$(x_2 - \bar{x})^2 = (82 - 83)^2 = (-1)^2 = 1$$

$$(x_3 - \bar{x})^2 = (89 - 83)^2 = (6)^2 = 36$$

- Sum of squared deviations

$$\text{SSD} = 25 + 1 + 36$$

- Calculate variance of sample

$$s^2 = \frac{\text{SSD}}{n - 1} = \frac{62}{3 - 1} = \frac{62}{2} = 31$$

- Calculate Sample Standard Deviation

$$s = \sqrt{s^2} = \sqrt{32} = 5.57$$

Step 5: Calculate the Test Statistic

$$t = \frac{\bar{x} - \mu_0}{\frac{s}{\sqrt{n}}} = \frac{83 - 85}{\frac{5.57}{\sqrt{3}}} = -\frac{3}{3.22} = -0.62$$

Step 6: Determine the Degrees of Freedom

$$df = n - 1 = 15 - 1 = 14$$

Step 7: Find Critical t-value

- For a two-tailed test at a significance level (α) of 0.05 and 2 degrees of freedom (df)

$$4.303$$

Step 8: Compare the t-Value to the Critical t-Value

- If the absolute value of the test statistic is greater than the critical t-value, reject the null hypothesis.
- If the absolute value of the test statistic is less than the critical t-value, fail to reject the null hypothesis.

Step 8: Find the p-Value

$$t = 4.303 \text{ and } df = 3$$

$$\text{p-value} = 0.58$$

t_test_one_sample.py

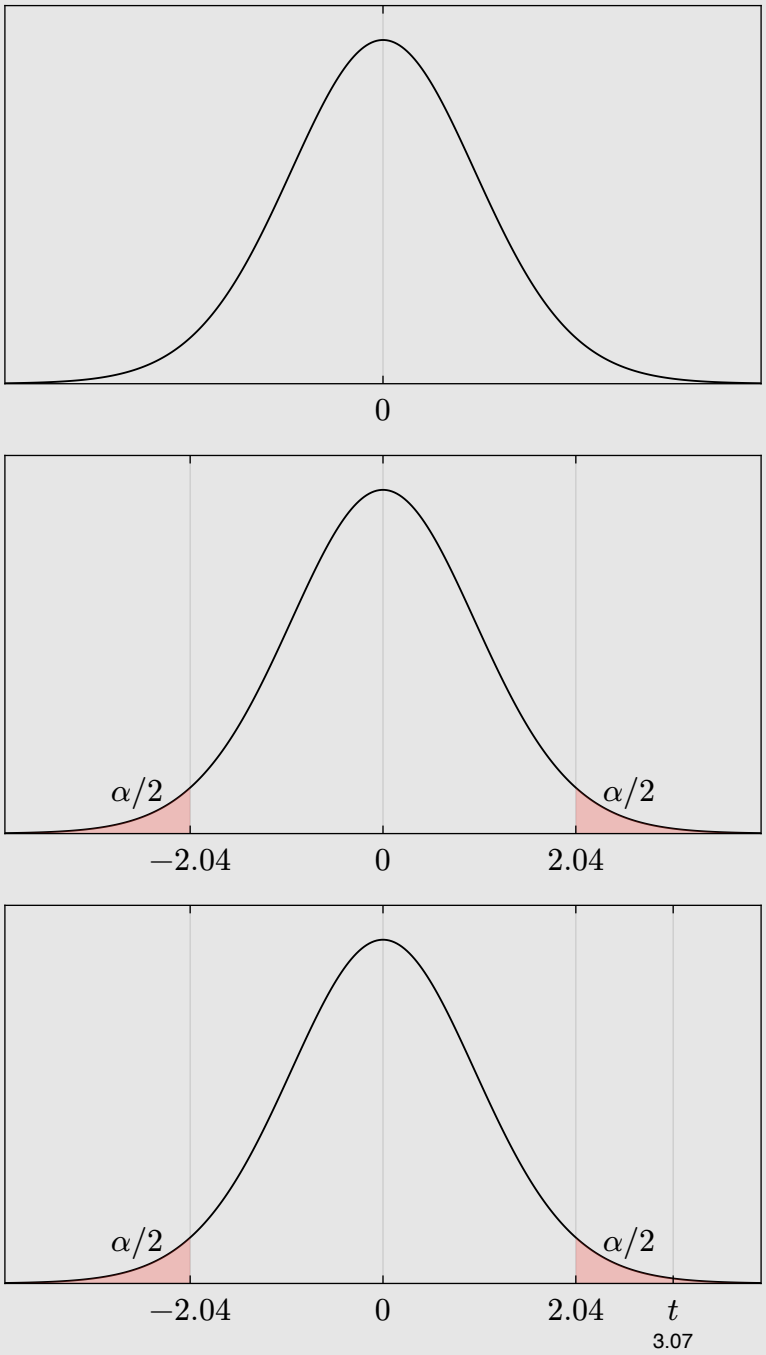
```python
from scipy import stats
rvs = stats.uniform.rvs(size=50)
stats.ttest_1samp(rvs, popmean=0.5)

```

142.9. One-Sample Two-Tail T-Test for Mean

Example

A snack company's energy bars are labeled as containing 20 grams of protein. A consumer group suspects the actual mean protein content differs from the label — it could be over or under. They take a simple random sample of 31 bars and measure a mean of 21.40 grams with a sample standard deviation of 2.54 grams. Assume protein content is normally distributed. At α = 0.05, test whether the mean protein content differs from the labeled amount.

$$H_0 : \mu = 20$$
$$H_a : \mu \neq 20$$



Since $|t| = 3.07 > 2.04$, reject H_0 ($p \approx 0.0045$). There is sufficient evidence at $\alpha = 0.05$ that the mean protein content differs from 20 grams.

142.10. Independent t-Test

142.10.1. Independent

Compares the means of two independent samples.

$$t = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{\left(\frac{s_1^2}{n_1}\right) + \left(\frac{s_2^2}{n_2}\right)}}$$

- $\bar{x}_1$ and $\bar{x}_2$: sample means
- s_1^2 and s_2^2: sample variances
- s_1 and n_2: sample sizes

Example

t_test_independent

```
from scipy import stats
rvs1 = stats.norm.rvs(loc=5, scale=10, size=500)
rvs2 = stats.norm.rvs(loc=5, scale=10, size=500)
stats.ttest_ind(rvs1, rvs2)
```

142.11. Paired t-Test

142.11.1. Paired

$$t = \frac{\bar{D}}{\frac{s_D}{\sqrt{n}}}$$

- $\bar{D}$: mean of the differences between paired observations
- s_D: standard deviation of the differences
- n: number of pairs

142.12. Chi-Square Goodness of Fit

Compares an **observed** categorical distribution to a **theoretical** categorical distribution.

$$\mathrm{X}^2 = \sum_{i=1}^{k} \frac{(o_i - e_i)^2}{e_i}$$

- o_i: **observed** frequency in category i
- e_i: **expected** frequency in category i

$$\mathrm{df} = k - 1$$

- k: number of categories

Example

Your professor gave a hard test. He said 45% got A's, 20% B's, 15% C's, 10% D's and 10% F's. You don't believe him. You take a simple random sample of 60 students and find 20 got A's, 15 B's, 8 O's, 12 D's and 5 F's. At $\alpha = 0.05$, test.

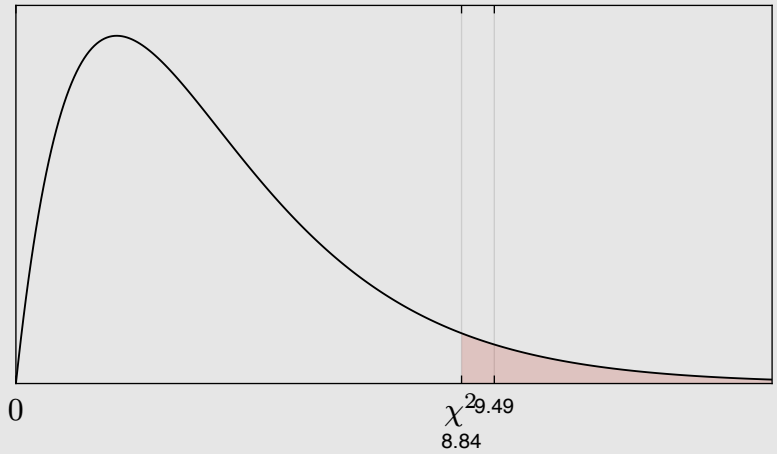
Category	A	B	C	D	F	Total
Expected % (O_i)	45%	20%	15%	10%	10%	100%
Observed # (E_i)	20	15	8	12	5	60
Expected #	$60 \cdot 0.45 = 27$	$60 \cdot 0.20 = 12$	$60 \cdot 0.15 = 9$	$60 \cdot 0.10 = 6$	$60 \cdot 0.10 = 6$	60
$\frac{(O-E)^2}{E}$	1.815	0.750	0.111	6.000	0.167	

1. Hypothesis

$$H_0 : p_A = 0.45, p_B = 0.20, p_C = 0.15, p_D = 0.10, p_F = 0.10$$
$$\text{(the professor's claimed distribution is correct)}$$
$$H_a : \text{at least one } \ p_i \text{ differs from its claimed value}$$

2. Test Statistic & P-Value

$$\begin{aligned}
\chi^2 &= \sum_{i=1}^{n} \frac{(o_i - e_i)^2}{e_i} \\
&= z_A^2 + z_B^2 + z_C^2 + z_D^2 + z_F^2 \\
&= \frac{(20-27)^2}{27} + \frac{(15-12)^2}{12} + \frac{(8-9)^2}{9} + \frac{(12-6)^2}{6} + \frac{(5-6)^2}{6} \\
&= 1.81 + 0.75 + 0.11 + 6.00 + 0.17 \\
&= 8.84
\end{aligned}$$



$$p\text{-value} = 0.0652$$
$$\alpha = 0.05$$
$$\text{critical value} = \chi^2_{0.05,4} = 9.49$$
$$\chi^2 = 8.84$$

3. Decision

Since $\chi^2 = 8.84 < 9.49$ (equivalently $p = 0.0652 > \alpha = 0.05$), we **fail to reject** H_0.

4. Conclusion

At the 5% significance level there is not enough evidence to conclude that the professor's claimed grade distribution differs from the true one.

Example

Complaints by Quarter (testing for a uniform distribution)

	Q1	Q2	Q3	Q4	$\sum$
o_i	43	52	54	40	189

Under H_0 all four quarters are equally likely, so $e_i = N/k = 189/4$ for every i.

	Q1	Q2	Q3	Q4	$\sum$
o_i	43	52	54	40	189
e_i	$\frac{189}{4}$	$\frac{189}{4}$	$\frac{189}{4}$	$\frac{189}{4}$	189

	Q1	Q2	Q3	Q4	$\sum$
o_i	43	52	54	40	189
e_i	47.25	47.25	47.25	47.25	189

	$\frac{(43-47.25)^2}{47.25}$	$\frac{(52-47.25)^2}{47.25}$	$\frac{(54-47.25)^2}{47.25}$	$\frac{(40-47.25)^2}{47.25}$	$\sum$
	0.38	0.48	0.96	1.11	**2.94**

$$\mathrm{X}^2 = 0.38 + 0.48 + 0.96 + 1.11 = 2.94$$

$\mathrm{df} = k - 1 = 3$, and $\mathrm{X}^2_{0.05,3} = 7.815$. Since $2.94 < 7.815$ we fail to reject H_0 ($p \approx 0.40$): the complaints are consistent with a uniform distribution across quarters.

chi2_got.py

```
from scipy import stats
f_obs = np.array([43, 52, 54, 40])
f_exp = np.array([47, 47, 47, 47])
stats.chisquare(f_obs=f_obs, f_exp=f_exp)
```

142.13. Chi-Square Independence

Compares two **observed** categorical distributions.

$$\mathrm{X}^2 = \sum_{i=1}^{k} \frac{(o_{ij} - e_{ij})^2}{e_{ij}}$$

- o_{ij}: observed frequency in cell (i, j)
- e_{ij}: expected frequency in category (i, j) calculated as:

$$E_{ij} = \frac{R_i \times C_j}{N}$$

- R_i: Row total for row i
- C_j: Column total for column j
- N: Total number of observations

$$\mathrm{df} = (\mathrm{rows} - 1)(\mathrm{cols} - 1)$$

Example

You own a café, and you think you can make more money by getting people to buy larger drinks in the morning. You want to see if the time of day and coffee size are related, so you collect a simple random sample of last month's orders. The observed data is below. At $\alpha = 0.05$ test if they are related.

Observed	Small	Medium	Large	Total
Morning	7	14	29	50
Afternoon	7	7	15	29
Evening	10	7	4	21
Total	24	28	48	100

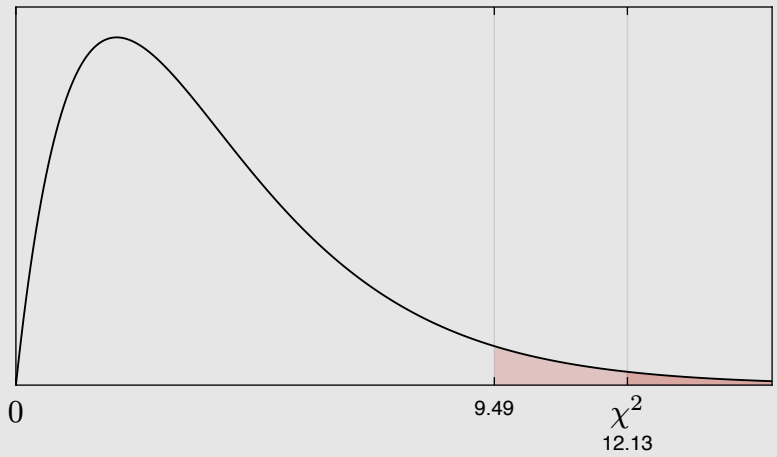
$$\text{expected} = \frac{\text{row total} \times \text{col total}}{\text{table total}}$$

Expected	Small	Medium	Large	Total
Morning	$\frac{50 \cdot 24}{100} = 12$	$\frac{50 \cdot 28}{100} = 14$	$\frac{50 \cdot 48}{100} = 24$	50
Afternoon	$\frac{29 \cdot 24}{100} = 6.96$	$\frac{29 \cdot 28}{100} = 8.12$	$\frac{29 \cdot 48}{100} = 13.92$	29
Evening	$\frac{21 \cdot 24}{100} = 5.04$	$\frac{21 \cdot 28}{100} = 5.88$	$\frac{21 \cdot 48}{100} = 10.08$	21
Total	24	28	48	100

H_0 : the two variables are independent

H_a : the 2 variables are related

$$\begin{aligned}\chi^2 = &\frac{(7-12)^2}{12} + \frac{(14-14)^2}{14} + \frac{(29-24)^2}{24} + \\ &\frac{(7-6.96)^2}{6.96} + \frac{(7-8.12)^2}{8.12} + \frac{(15-13.92)^2}{13.92} + \\ &\frac{(10-5.04)^2}{5.04} + \frac{(7-5.88)^2}{5.88} + \frac{(4-10.08)^2}{10.08} \\ = &\ 12.13\end{aligned}$$



p-value < alpha

0.0164 < 0.05

Critical value < Chi^2 statistic

9.49 < 12.13

Example

Genre Preference by Age Group

	Action	Comedy	Drama	
Under 30	60	50	30	140
Above 30	20	40	60	120
	80	90	90	260

	Action	Comedy	Drama	
Under 30	$\frac{140 \cdot 80}{260}$	$\frac{140 \cdot 90}{260}$	$\frac{140 \cdot 90}{260}$	140
Above 30	$\frac{120 \cdot 80}{260}$	$\frac{120 \cdot 90}{260}$	$\frac{120 \cdot 90}{260}$	120
	80	90	90	260

	Action	Comedy	Drama	
Under 30	43.08	48.46	48.46	140
Above 30	36.92	41.54	41.54	120
	80	90	90	260

	Action	Comedy	Drama	
Under 30	$\frac{(60-43.08)^2}{43.08}$	$\frac{(50-48.46)^2}{48.46}$	$\frac{(30-48.46)^2}{48.46}$	140
Above 30	$\frac{(20-36.92)^2}{36.92}$	$\frac{(40-41.54)^2}{41.54}$	$\frac{(60-41.54)^2}{41.54}$	120
	80	90	90	260

	Action	Comedy	Drama	$\sum$
Under 30	6.65	0.05	7.03	13.73
Above 30	7.76	0.06	8.21	16.02
$\sum$	14.40	0.11	15.24	$\mathbf{29.75}$

$$\text{Chi}^2 = 6.65 + 0.05 + 7.03 + 7.76 + 0.06 + 8.21 = 29.75$$

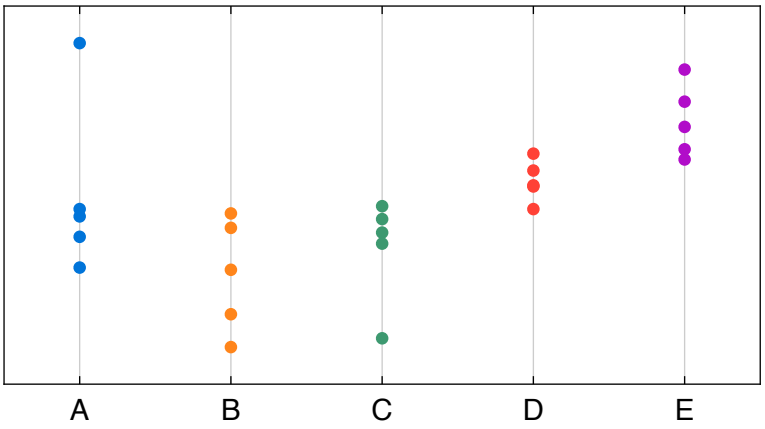
Since $\text{X}^2 = 29.75 > \text{X}^2_{0.05,2} = 5.991$, reject H_0: genre preference depends on age group, driven almost entirely by action and drama (14.40 and 15.24) rather than comedy (0.11).

142.14. One-Way ANOVA

Compares the means of three or more groups based on one independent variable

- H_0: $\mu_1 = \mu_2 = ... = \mu_k$
- H_1: At least one μ_i differs from the others

142.14.1. The data



142.14.2. Variation around the grand mean

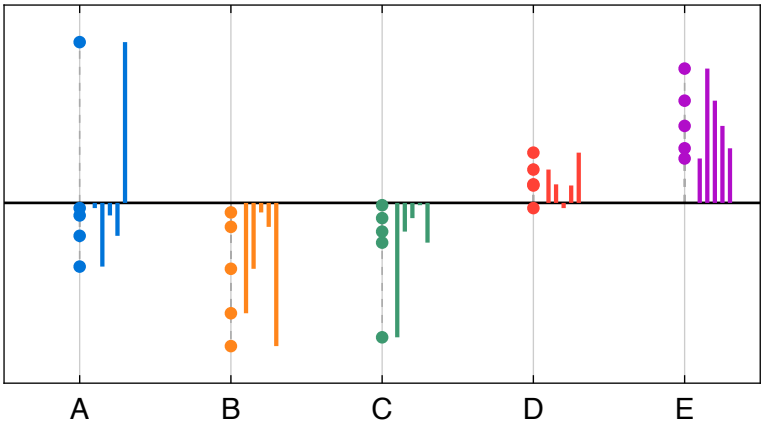
Total Variation (SS_{total}): every observation measured against the grand mean.

$$\text{SS}_{\text{total}} = \sum_{i=1}^{k} \sum_{j=1}^{n_i} \left(X_{ij} - \bar{X}_{\text{overall}}\right)^2$$

- X_{ij}: Observation j in group i
- $\bar{X}_{\text{overall}}$: Overall mean of all groups

It splits into two parts, which the next two sections build up separately:

$$\text{SS}_{\text{total}} = \text{SS}_{\text{between}} + \text{SS}_{\text{within}}$$



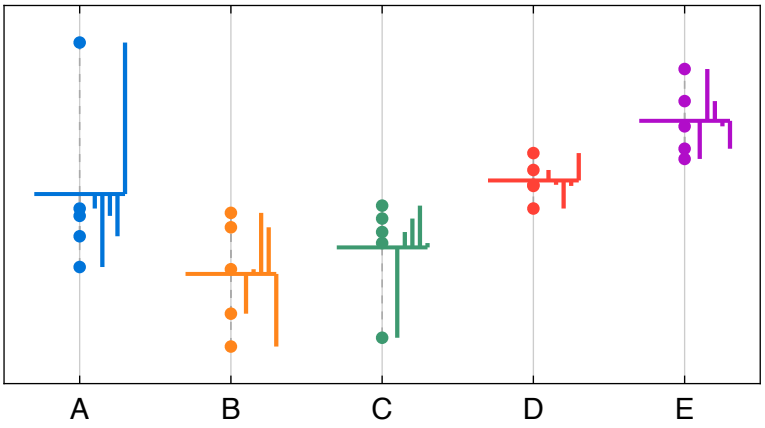
$$\text{SS}_{\text{total}} = \square^2 + \square^2 \approx 388$$

142.14.3. Variation around the group means

Within-Group Variation ($\text{SS}_{\text{within}}$): every observation measured against its own group mean.

$$\text{SS}_{\text{within}} = \sum_{i=1}^{k} \sum_{j=1}^{n_i} \left(X_{ij} - \bar{X}_i\right)^2$$

- $\bar{X}_i$: Mean of group i



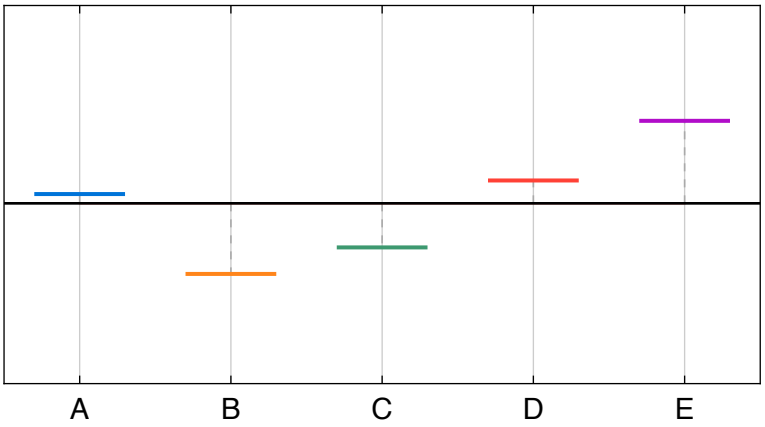
$$\begin{aligned}\mathrm{SS}_{\text{within}} = {}& {}^2 + {}^2 + {}^2 + {}^2 + {}^2 \\ & + {}^2 + {}^2 + {}^2 + {}^2 + {}^2 \\ & + {}^2 + {}^2 + {}^2 + {}^2 + {}^2 \\ & + {}^2 + {}^2 + {}^2 + {}^2 + {}^2 \\ & + {}^2 + {}^2 + {}^2 + {}^2 + {}^2 \\ & \approx 179.2\end{aligned}$$

142.14.4. Variation between the group means

Between-Group Variation ($\mathrm{SS}_{\text{between}}$): each group mean measured against the grand mean, weighted by group size.

$$\mathrm{SS}_{\text{between}} = \sum_{i=1}^{k} n_i \left(\bar{X}_i - \bar{X}_{\text{overall}}\right)^2$$

- n_i: Number of observations in group i



Equivalently, it is whatever the group means account for out of the total:

$$\mathrm{SS}_{\text{between}} = \mathrm{SS}_{\text{total}} - \mathrm{SS}_{\text{within}} \approx 388 - 179.2 \approx 208.8$$

142.14.5. Degrees of freedom

- $k = 5$: number of groups, one mean each
- $N = 25$: total number of observations
- $k - 1 = 4$: between-group degrees of freedom (one grand mean is fixed)
- $N - k = 20$: within-group degrees of freedom (one mean fixed per group)

142.14.6. Mean squares

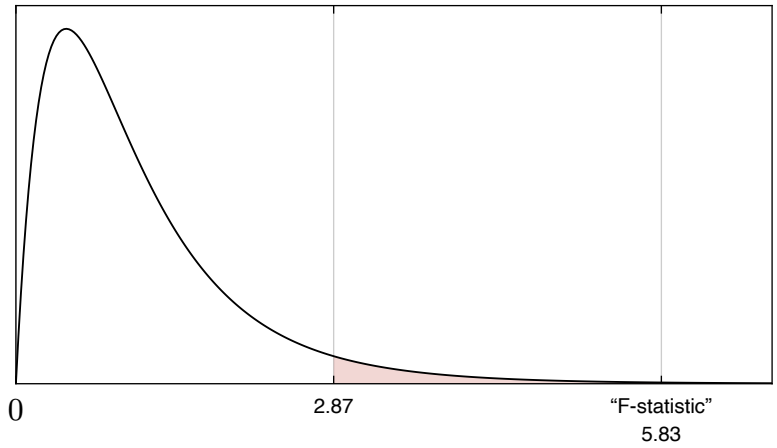
Dividing each sum of squares by its degrees of freedom turns it into a variance estimate.

$$\mathrm{MS}_{\text{between}} = \frac{\mathrm{SS}_{\text{between}}}{k - 1} \approx \frac{208.8}{4} \approx 52.2$$

$$\mathrm{MS}_{\text{within}} = \frac{\mathrm{SS}_{\text{within}}}{N - k} \approx \frac{179.2}{20} \approx 8.96$$

142.14.7. F statistic

$$F = \frac{\text{MS}_{\text{between}}}{\text{MS}_{\text{within}}} = \frac{\frac{\text{SS}_{\text{between}}}{k-1}}{\frac{\text{SS}_{\text{within}}}{N-k}} \approx \frac{\frac{208.8}{4}}{\frac{179.2}{20}} \approx 5.83$$



142.14.8. Decision rule

- Compare the F-statistic to the critical value from the F-distribution table with $k - 1 = 4$ and $N - k = 20$ degrees of freedom (based on chosen significance level α).
- Alternatively, compare the p-value to the significance level α.
- Reject H_0 if the F-statistic is greater than the critical value or if the p-value is less than α, indicating that at least one group mean is significantly different.

Code

```python
from scipy import stats

rng = np.random.default_rng(0)
groups = [rng.normal(loc, 3, 5) for loc in (10, 9, 8, 12, 13)]

f, p = stats.f_oneway(*groups)
```

Code

```python
import pandas as pd, statsmodels.api as sm
from statsmodels.formula.api import ols

rng = np.random.default_rng(0)
groups = [rng.normal(loc, 3, 5) for loc in (10, 9, 8, 12, 13)]

df = pd.DataFrame({
    "y": np.concatenate(groups),
    "g": np.repeat(list("ABCDE"), 5),
})

```

```
model = ols("y ~ C(g)", data=df).fit()
sm.stats.anova_lm(model, typ=2)
```

142.15. Two-Way ANOVA

Compares the means of groups formed by two independent variables, and asks whether the two effects simply add up or whether they modify each other.

$$X_{ijk} = \mu + \alpha_i + \beta_j + (\alpha\beta)_{ij} + \varepsilon_{ijk}$$

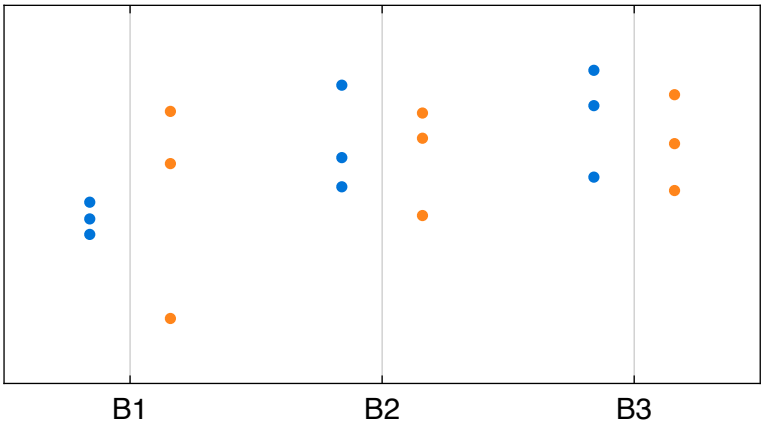
- α_i: effect of level i of factor A
- β_j: effect of level j of factor B
- $(\alpha\beta)_{ij}$: interaction, the part of cell (i, j) that the two main effects fail to explain
- ε_{ijk}: error, assumed $N(0, \sigma^2)$

Three null hypotheses, tested separately:

- H_0^A: $\alpha_1 = ... = \alpha_a = 0$ (no factor A main effect)
- H_0^B: $\beta_1 = ... = \beta_b = 0$ (no factor B main effect)
- H_0^{AB}: all $(\alpha\beta)_{ij} = 0$ (the effects are additive)

142.15.1. The data

Six cells: two levels of factor A (colour) crossed with three levels of factor B (position), three replicates each.



142.15.2. Variation around the grand mean

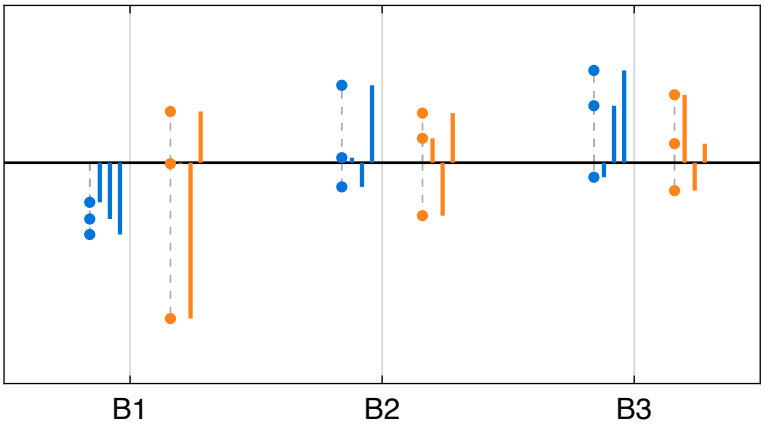
Total Variation ($\mathrm{SS}_{\mathrm{total}}$): every observation measured against the grand mean, ignoring both factors.

$$\mathrm{SS}_{\mathrm{total}} = \sum_{i=1}^{a}\sum_{j=1}^{b}\sum_{k=1}^{n}\left(X_{ijk} - \bar{X}_{\mathrm{overall}}\right)^2$$

- X_{ijk}: replicate k in cell (i, j)
- $\bar{X}_{\mathrm{overall}}$: overall mean of all observations

It splits into four parts, which the next sections build up separately:

$$\mathrm{SS}_{\text{total}} = \mathrm{SS}_A + \mathrm{SS}_B + \mathrm{SS}_{AB} + \mathrm{SS}_{\text{error}}$$



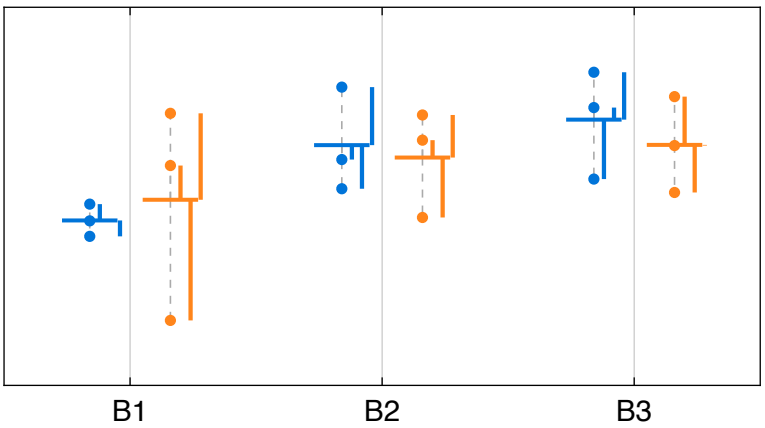
$$\begin{aligned}\mathrm{SS}_{\text{total}} = {} & \blacksquare^2 + \blacksquare^2 + \blacksquare^2 \\ & + \blacksquare^2 + \blacksquare^2 + \blacksquare^2 \\ & + \blacksquare^2 + \blacksquare^2 + \blacksquare^2 \\ & + \blacksquare^2 + \blacksquare^2 + \blacksquare^2 \\ & + \blacksquare^2 + \blacksquare^2 + \blacksquare^2 \\ & + \blacksquare^2 + \blacksquare^2 + \blacksquare^2 \\ \approx {} & 63.5\end{aligned}$$

142.15.3. Variation around the cell means

Error Variation ($\mathrm{SS}_{\text{error}}$): every observation measured against its own cell mean. Nothing about A or B can explain this, so it becomes the yardstick for all three tests.

$$\mathrm{SS}_{\text{error}} = \sum_{i=1}^{a} \sum_{j=1}^{b} \sum_{k=1}^{n} \left(X_{ijk} - \bar{X}_{ij}\right)^2$$

- $\bar{X}_{ij}$: mean of cell (i, j)



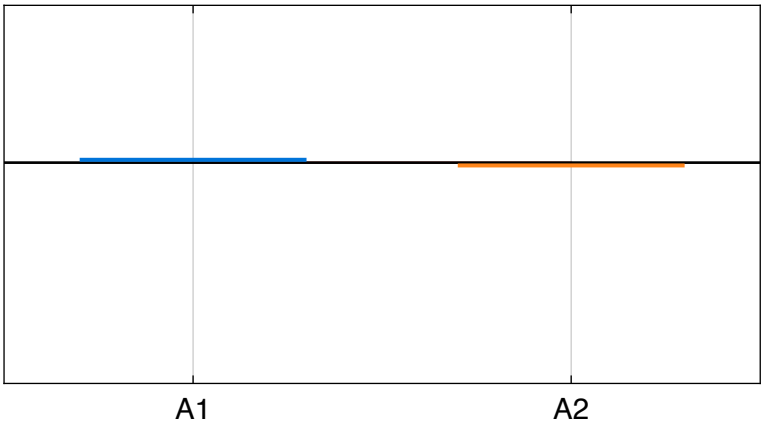
$$\begin{aligned}\mathrm{SS}_{\text{error}} = {}& {}^2 + {}^2 + {}^2 \\ & + {}^2 + {}^2 + {}^2 \\ & + {}^2 + {}^2 + {}^2 \\ & + {}^2 + {}^2 + {}^2 \\ & + {}^2 + {}^2 + {}^2 \\ & + {}^2 + {}^2 + {}^2 \\ \approx {}& 43\end{aligned}$$

142.15.4. Variation between the levels of factor A

Main Effect of A (SS_A): collapse factor B entirely, average each row of the design down to one number, and measure those against the grand mean.

$$\mathrm{SS}_A = bn \sum_{i=1}^{a} \left(\bar{X}_{i\cdot} - \bar{X}_{\text{overall}}\right)^2$$

- $\bar{X}_{i\cdot}$: mean of all observations at level i of A, pooled over B
- $bn = 9$: observations behind each row mean



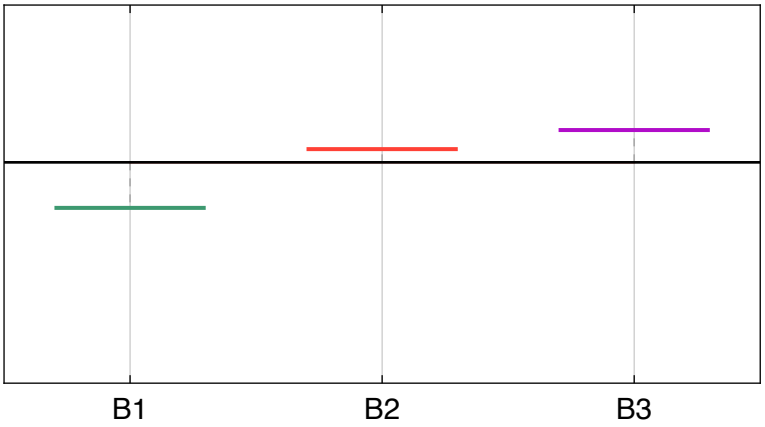
$$\mathrm{SS}_A = 9\left({}^2 + {}^2\right) \approx 0.1$$

142.15.5. Variation between the levels of factor B

Main Effect of B (SS_B): the mirror image, collapsing factor A instead.

$$\mathrm{SS}_B = an \sum_{j=1}^{b} \left(\bar{X}_{\cdot j} - \bar{X}_{\text{overall}}\right)^2$$

- $\bar{X}_{\cdot j}$: mean of all observations at level j of B, pooled over A
- $an = 6$: observations behind each column mean



$$\mathrm{SS}_B = 6(.^2 + .^2 + .^2) \approx 18.7$$

142.15.6. Variation left over: the interaction

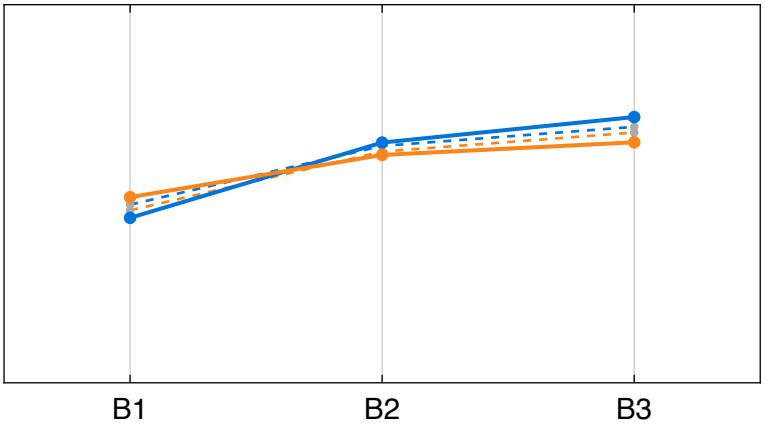
If the two factors acted independently, each cell mean would be predicted by its row and column effects alone:

$$\hat{X}_{ij} = \bar{X}_{i\cdot} + \bar{X}_{\cdot j} - \bar{X}_{\text{overall}}$$

Interaction Variation (SS_{AB}): how far the actual cell means sit from that additive prediction.

$$\mathrm{SS}_{AB} = n \sum_{i=1}^{a} \sum_{j=1}^{b} \left(\bar{X}_{ij} - \bar{X}_{i\cdot} - \bar{X}_{\cdot j} + \bar{X}_{\text{overall}}\right)^2$$

Dashed lines are the additive prediction, solid lines the actual cell means. Under H_0^{AB} the solid lines would be parallel.



$$\begin{aligned}\mathrm{SS}_{AB} = 3(&.^2 + .^2 + .^2 \\ &+ .^2 + .^2 + .^2) \\ &\approx 1.6\end{aligned}$$

Equivalently, it is whatever is left once the main effects and the error are taken out:

$$\mathrm{SS}_{AB} = \mathrm{SS}_{\text{total}} - \mathrm{SS}_A - \mathrm{SS}_B - \mathrm{SS}_{\text{error}} \approx 1.6$$

142.15.7. Degrees of freedom

- $a = 2$: levels of factor A

- $b = 3$: levels of factor B
- $n = 3$: replicates per cell, $N = 18$ observations in total
- $a - 1 = 1$: one grand mean is fixed, so the row effects lose one
- $b - 1 = 2$: likewise for the column effects
- $(a - 1)(b - 1) = 2$: the interaction is a grid of deviations whose rows and columns must each sum to zero
- $ab(n - 1) = 12$: one mean fixed per cell

They add up the same way the sums of squares do:

$$N - 1 = 17 = 1 + 2 + 2 + 12$$

Replication is what makes $\mathrm{df}_{\text{error}} > 0$. With $n = 1$ there is nothing left over inside a cell, $ab(n - 1) = 0$, and the interaction has to be sacrificed as the error term instead of being tested.

142.15.8. Mean squares

Dividing each sum of squares by its degrees of freedom turns it into a variance estimate.

$$\mathrm{MS}_A = \frac{\mathrm{SS}_A}{a - 1} \approx \frac{0.1}{1} \approx 0.14$$

$$\mathrm{MS}_B = \frac{\mathrm{SS}_B}{b - 1} \approx \frac{18.7}{2} \approx 9.36$$

$$\mathrm{MS}_{AB} = \frac{\mathrm{SS}_{AB}}{(a - 1)(b - 1)} \approx \frac{1.6}{2} \approx 0.8$$

$$\mathrm{MS}_{\text{error}} = \frac{\mathrm{SS}_{\text{error}}}{ab(n - 1)} \approx \frac{43}{12} \approx 3.59$$

142.15.9. F statistics

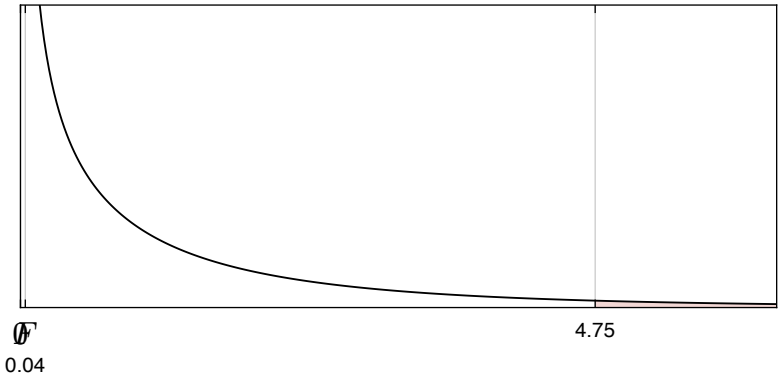
Each effect is compared against the same error variance. In a fixed-effects model all three denominators are $\mathrm{MS}_{\text{error}}$.

$$F_A = \frac{\mathrm{MS}_A}{\mathrm{MS}_{\text{error}}} \approx \frac{0.14}{3.59} \approx 0.04 \quad (1, 12)$$

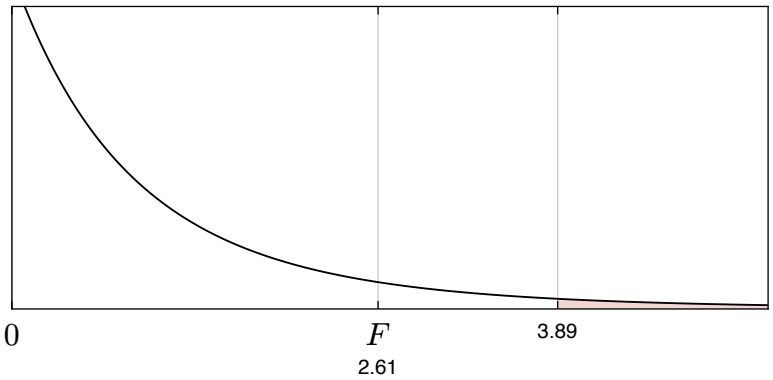
$$F_B = \frac{\mathrm{MS}_B}{\mathrm{MS}_{\text{error}}} \approx \frac{9.36}{3.59} \approx 2.61 \quad (2, 12)$$

$$F_{AB} = \frac{\mathrm{MS}_{AB}}{\mathrm{MS}_{\text{error}}} \approx \frac{0.8}{3.59} \approx 0.22 \quad (2, 12)$$

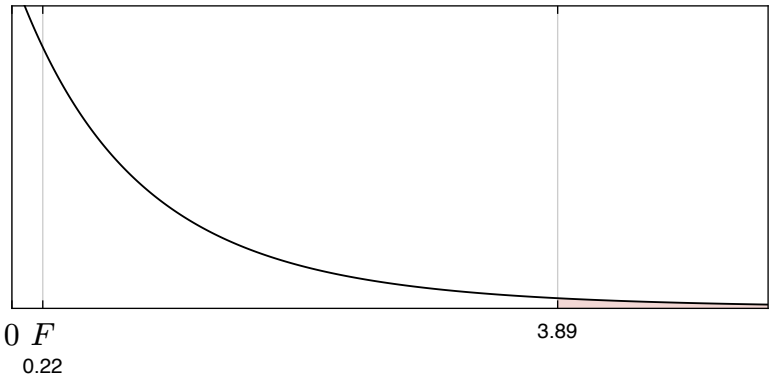
Factor A:



Factor B:



Interaction $A \times B$:



142.15.10. ANOVA table

Source	SS	df	MS	F
A	0.1	1	0.14	0.04
B	18.7	2	9.36	2.61
$A \times B$	1.6	2	0.8	0.22
Error	43	12	3.59	
Total	63.5	17		

142.15.11. Decision rule

- Compare each F-statistic to the critical value from the F-distribution with its own numerator degrees of freedom and $ab(n-1) = 12$ in the denominator (based on chosen significance level 0.05).
- Alternatively, compare each p-value to the significance level 0.05.
- Reject an H_0 if its F-statistic exceeds the critical value or its p-value is below 0.05.
- Read the interaction first. If H_0^{AB} is rejected, the main effects describe averages over conditions where the effect genuinely differs, so they are usually reported as secondary to the cell means themselves.

Two assumptions the decomposition above quietly relies on:

- Balance. Equal n in every cell is what makes the four sums of squares orthogonal and additive. With unequal cells they no longer sum to $\mathrm{SS}_{\text{total}}$, and the order of terms starts to matter (Type I / II / III sums of squares).
- Fixed effects. If a factor is random, its main effect is tested against MS_{AB} rather than $\mathrm{MS}_{\text{error}}$.

Code

```python
import numpy as np
from scipy import stats

rng = np.random.default_rng(0)

n = 3
cell_means = np.array([[10, 12, 14],
                       [14, 13, 12]])

y = rng.normal(np.repeat(cell_means.ravel(), n), 2)
A = np.repeat(["A1", "A2"], 3 * n)
B = np.tile(np.repeat(["B1", "B2", "B3"], n), 2)
```

Code

```python
import pandas as pd, statsmodels.api as sm
from statsmodels.formula.api import ols

df = pd.DataFrame({"y": y, "A": A, "B": B})

# Full model: both main effects plus the interaction
model = ols("y ~ C(A) * C(B)", data=df).fit()
sm.stats.anova_lm(model, typ=2)
```

Code

```python
# Additive model: interaction assumed away, its SS folded into the residual
additive = ols("y ~ C(A) + C(B)", data=df).fit()
sm.stats.anova_lm(additive, typ=2)

# The two models differ by exactly SS_AB on df_AB degrees of freedom
sm.stats.anova_lm(additive, model)
```

142.16. A/B Testing

Step 1: Problem Statement

- User Funnel (Journey)
- Success Metrics:
  - Measurable: Can the behavior be measured using collected data?
  - Attributable: Can the behavior (effect) be assigned to the treatment (cause)?
  - Sensitive: Does the metric have low variability that can distinguish the treatment from the control?
  - Timely: How long will it take?

Step 2: Hypothesis Testing

- Hypothesis Statement (H_0, H_a)
- Set Significance Level (α usually 0.05)
- Set Statistical Power (usually 0.8)
- Set Minimum Detectable Effect (MD usually 1% lift)

Step 3: Experiment Design

- Set Randomization Unit
- Target Population
- Determine Sample Size

$$n = \frac{16\sigma^2}{\delta^2}$$

Where:

- σ: sample standard deviation
- δ: difference between control & measurement
- Determine Duration of Experiment

Step 4: Run Experiment

Step 5: Validity Check

- Instrumentation Effect
- External Factors
- Selection Bias
- Sample Ratio Mismatch
- Novelty Effect
-

Step 6: Interpret Results

Step 7: Make Decision

143. Regression Analysis

143.1. Simple Linear Regression

$$y = \beta_0 + \beta_1 x_1 + \varepsilon$$

- y: dependent variable
- x: independent variables
- β_0: intercept (value of y when $x = 0$)
- β_1: slope (change in y for a one-unit change in x)
- ε: error term (difference between the actual data points and the predicted values)

Example

Data

x (Hours Studied)	y (Test Score)
1	2
2	3
3	5
4	4
5	6

Step 1: Calculate Means

$$\bar{x} = 3$$

$$\bar{y} = 4$$

Step 2: Calculating Slope β_1

$$\beta_1 = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

- The numerator

$$\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y}) = 9$$

- The denominator

$$\sum_{i=1}^{n} (x_i - \bar{x})^2 = 10$$

- The slope β_1 is

$$\beta_1 = \frac{9}{10} = 0.9$$

Step 3: Calculate Intercept β_0

$$\beta_1 = \bar{y} - \beta_1 \bar{x} = 1.3$$

Step 4: Calculate p-Value

- Calculate Standard Error of the Slope (SE_{β_1})

$$SE_{\beta_1} = \sqrt{\frac{\sum_{n=1}^{n} (y_i - \hat{y}_i)^2}{(n-2) \cdot \sum_{i=1}^{n} (x_i - \bar{x})^2}}$$

- Calculate the Residual Sum of Squares (RSS)

$$RSS = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

- Calculate the t-statistic for the Slope

$$t = \frac{\beta_1}{SE_{\beta_1}}$$

- Determine Degrees of Freedom

$$\text{df} = n - 2$$

- Look up the p-value corresponding to t with 3 degrees of freedom

Step 5: Calculate R^2 (R^2_{adj})

$$R^2 = \frac{\text{SS}_{\text{reg}}}{\text{SS}_{\text{total}}}$$

SS_{reg} (Regression Sum of Squares): sum of the squared differences between the predicted $\hat{y}$ values and the mean of the observed y values.

$$\text{SS}_{\text{reg}} = \sum_{i=1}^{n} (\hat{y}_i - \bar{y})^2$$

SS_{total} (Total Sum of Squares): sum of the squared differences between the observed y values and the mean of the observed y values.

$$\text{SS}_{\text{total}} = \sum_{i=1}^{n} (y_i - \bar{y})^2$$

Adjusting for the number of independent variables

$$R^2_{\text{adj}} = 1 - \left(\frac{(1 - R^2)(n - 1)}{n - k - 1} \right)$$

- n: number of observations
- k: number of independent variables

143.2. Multiple Linear Regression

Causal vs. extrapolative. Time-series methods (SMA, ETS, ARIMA) build a forecast from the demand series alone — extracting level, trend, and seasonality from $x_1, ..., x_t$. **Causal** models flip the framing: demand is **driven** by exogenous regressors (price, weather, promotions, demographics, day-of-week, …) and we fit a relationship between them.

$$Y_i = \beta_0 + \beta_1 x_{1,i} + ... + \beta_k x_{k,i} + \varepsilon_i, \quad \varepsilon_i \sim \text{iid } \mathcal{N}(0, \sigma^2)$$

- **Model parameters** (unknown — to be estimated)
  - $\beta_0 \in \mathbb{R}$: intercept
  - $\beta_j \in \mathbb{R}$, $j = 1, ..., k$: slope on regressor x_j
  - $\sigma^2 > 0$: noise variance
- **Data**
  - $(x_{1,i}, ..., x_{k,i}, y_i)$ for $i = 1, ..., n$: one row per observation
- **Output**
  - $\hat{b}_0, \hat{b}_1, ..., \hat{b}_k$: fitted coefficients
  - $\hat{y}_i = \hat{b}_0 + \sum_{j=1}^{k} \hat{b}_j x_{j,i}$: predicted value
  - $e_i = y_i - \hat{y}_i$: residual

143.2.1. Why “least squares”?

Three candidate loss functions for the residuals $e_i = y_i - \hat{y}_i$:

Loss	What it captures	Verdict
$\sum_i e_i$	Bias only	Positive and negative errors cancel — useless for accuracy
$\sum_i \lvert e_i \rvert$	Bias + accuracy	No closed-form minimiser — intractable analytically (OK numerically)
$\sum_i e_i^2$	Bias + accuracy + smooth	**Closed form via calculus** ✓

OLS picks $\hat{b}$ to minimise the **Residual Sum of Squares**:

$$\text{RSS}(b_0, ..., b_k) = \sum_{i=1}^{n} \left(y_i - b_0 - b_1 x_{1,i} - ... - b_k x_{k,i} \right)^2$$

For the **univariate** case ($k = 1$), setting first-order conditions to zero gives a clean closed form:

$$\hat{b}_1 = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n}(x_i - \bar{x})^2}, \quad \hat{b}_0 = \bar{y} - \hat{b}_1 \bar{x}$$

For the **multivariate** case, the same exercise in matrix form yields the **normal equations**:

$$\hat{b} = (X^T X)^{-1} X^T y$$

where X is the $n \times (k+1)$ design matrix (a column of ones for the intercept, plus one column per regressor).

143.2.2. Goodness of fit (R^2)

Total variation in y around its mean splits cleanly into the part the model explains and the part it leaves behind:

$$\underbrace{\sum_{i=1}^{n}(y_i - \bar{y})^2}_{\text{TSS (total)}} = \underbrace{\sum_{i=1}^{n}(\hat{y}_i - \bar{y})^2}_{\text{RSS (regression)}} + \underbrace{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}_{\text{ESS (residual)}}$$

The **coefficient of determination** is the fraction of variation the model explains:

$$R^2 = \frac{\text{RSS}}{\text{TSS}} = 1 - \frac{\text{ESS}}{\text{TSS}} \in [0, 1]$$

- $R^2 = 1$: model fits perfectly (all residuals zero)
- $R^2 = 0$: model no better than predicting $\bar{y}$ for everyone

Cautions on R^2:

- R^2 never decreases when you add a regressor → use **adjusted** $R^2 = 1 - (1 - R^2)\frac{n-1}{n-k-1}$ when comparing models of different sizes.
- High R^2 does not imply causality — only that variation co-moves.
- Each coefficient still needs its own t-test / confidence interval; the F-test on R^2 only says "**some** coefficient is non-zero".

Example

Trend + summer dummy on monthly demand (20 months, Jan year 1 — Aug year 2).

Model — level, linear trend, and a summer indicator:

$$F_i = \beta_0 + \beta_1 \cdot \text{Period}_i + \beta_2 \cdot \text{Summer}_i, \quad i = 1, \ldots, 20$$

where $\text{Period}_i \in \{1, \ldots, 20\}$ is a linear time index and $\text{Summer}_i \in \{0, 1\}$ flags May–Aug.

Data

Mo.	y	t	S	Mo.	y	t	S
Jan	3 025	1	0	Nov	3 499	11	0
Feb	3 047	2	0	Dec	3 598	12	0
Mar	3 079	3	0	Jan	3 596	13	0
Apr	3 136	4	0	Feb	3 721	14	0

Mo.	y	t	S	Mo.	y	t	S
May	3 454	5	1	Mar	3 745	15	0
Jun	3 661	6	1	Apr	3 650	16	0
Jul	3 554	7	1	May	4 157	17	1
Aug	3 692	8	1	Jun	4 221	18	1
Sep	3 407	9	0	Jul	4 238	19	1
Oct	3 410	10	0	Aug	4 008	20	1

Fit (e.g. `numpy.linalg.lstsq`, `statsmodels.OLS`, or Excel's regression tool):

$$\hat{F}_i = 2969.14 + 48.03 \cdot \text{Period}_i + 303.51 \cdot \text{Summer}_i$$

Diagnostics

- $R^2 = 0.958$, adj. $R^2 = 0.953$, residual standard error ≈ 79.21
- All three coefficients have $p < 0.001$ (t-stats: intercept 79.8, period 15.0, summer 8.05)

Interpretation

- **Intercept** (2969): baseline demand at $\text{Period} = 0$ in a non-summer month
- **Period** (48): underlying trend adds ≈ 48 units per month
- **Summer** (304): summer months run ≈ 304 units above the trend, holding period fixed

Forecast next month (Sep, $i = 21$, $S = 0$):

$$\hat{F}_{21} = 2969.14 + 48.03(21) + 303.51(0) = 3977.77$$

143.2.3. When to reach for OLS

Strengths:

- Coefficients are **interpretable** — each one quantifies a specific driver
- Confidence intervals, t-tests, and F-tests come for free
- Easy to fold in exogenous regressors (weather, promotions, demographics, …) that pure time-series methods cannot consume
- Plays nicely with categorical predictors via dummy variables

Limitations:

- Treats every observation equally — no down-weighting of stale data the way SES does
- Linear **in coefficients** (transformations of x help, but the structure is rigid)
- Assumes residuals are iid normal; serial correlation in time-series residuals deflates standard errors (fix: Newey–West SEs, ARMA errors, or fall back to ETS)
- Forecasting future y requires future x — fine when "Period" is the regressor, awkward when it's "weather"

143.3. Logistic Regression

Binary classification (1 or 0, true or false, yes or no) based on one or more predictor variables

Sigmoid function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Where:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + ... + \beta_n x_n$$

Example

143.4. Ridge

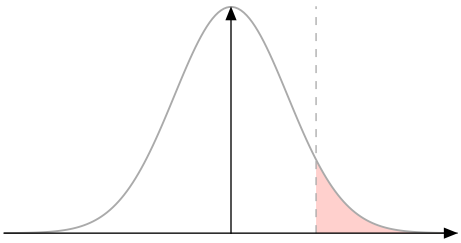
143.5. Lasso

143.6. p-value

Probability of obtaining results at least as extreme as the observed results

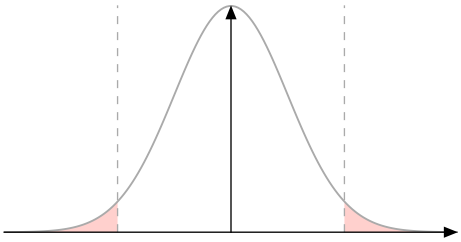
1. One-Tailed

$$p = P(Z > z_{\text{observed}})$$



2. Two-Tailed

$$p = 2 \cdot P(Z > |z_{\text{observed}}|)$$



p_value.py

```
z = 2.1
df = 3
scipy.stats.norm.sf(abs(z), df=df)
```

143.7. F-statistic

$$F = \frac{\text{MSR}}{\text{MSE}}$$

Where:

- Mean Square Regression (MSR):

$$\text{MSR} = \frac{\text{SSR}}{\text{df}_{\text{regression}}}$$

- $\text{df}_{\text{regression}} = p$

Where:

- p: Number of independent
- Mean Square Error (MSE):

$$\text{MSE} = \frac{\text{SSE}}{\text{df}_{\text{error}}}$$

- $\text{df}_{\text{error}} = n - p - 1$

Where:

- p: Number of independent
- n: Number of observations
- 1: Constant (for intercept)

143.8. Confidence Intervals

Range within which we can be confident that the true value (population parameter) lies, based on the sample data

$$E\left[\bar{X}\right] = \mu \qquad \sigma_{\bar{X}} = \frac{\sigma}{\sqrt{n}}$$

Known population standard deviation (σ):

$$\text{CI} = \bar{x} \pm z_{\alpha/2}\frac{\sigma}{\sqrt{n}}$$

- $\bar{x}$: sample mean
- $z_{\alpha/2}$: upper $\alpha/2$ quantile of $N(0,1)$
- σ: population standard deviation (known)
- n: sample size
- $1-\alpha$: confidence level

Unknown population standard deviation (σ):

$$\text{CI} = \bar{x} \pm t_{\alpha/2,n-1}\frac{s}{\sqrt{n}}$$

- $\bar{x}$: sample mean
- $t_{\alpha/2,n-1}$: upper $\alpha/2$ quantile of t_{n-1}
- s: sample standard deviation, $\text{ddof} = 1$
- $n-1$: degrees of freedom
- $1-\alpha$: confidence level

Both assume an i.i.d. sample from a normal population; the first is also valid for large n by the CLT, and the second is approximately so. Note $t_{\alpha/2,n-1} \to z_{\alpha/2}$ as $n \to \infty$.

143.8.1. Where the confidence interval comes from

What we know. If $X_1, \ldots, X_n$ are i.i.d. draws from a population with mean μ and standard deviation σ, then the sample mean $\bar{X}$ has

$$E[\bar{X}] = \mu \qquad \mathrm{SE} := \sigma_{\bar{X}} = \frac{\sigma}{\sqrt{n}}$$

and its shape is normal — exactly if the population is normal, approximately by the CLT otherwise. We write SE (the *standard error*) for $\sigma/\sqrt{n}$ from here on.

The question. How far can $\bar{X}$ stray from μ? Pick a distance b and ask for the b that makes the sample mean land within b of the truth 90% of the time:

$$P\big(\mu - b \le \bar{X} \le \mu + b\big) = 0.90$$

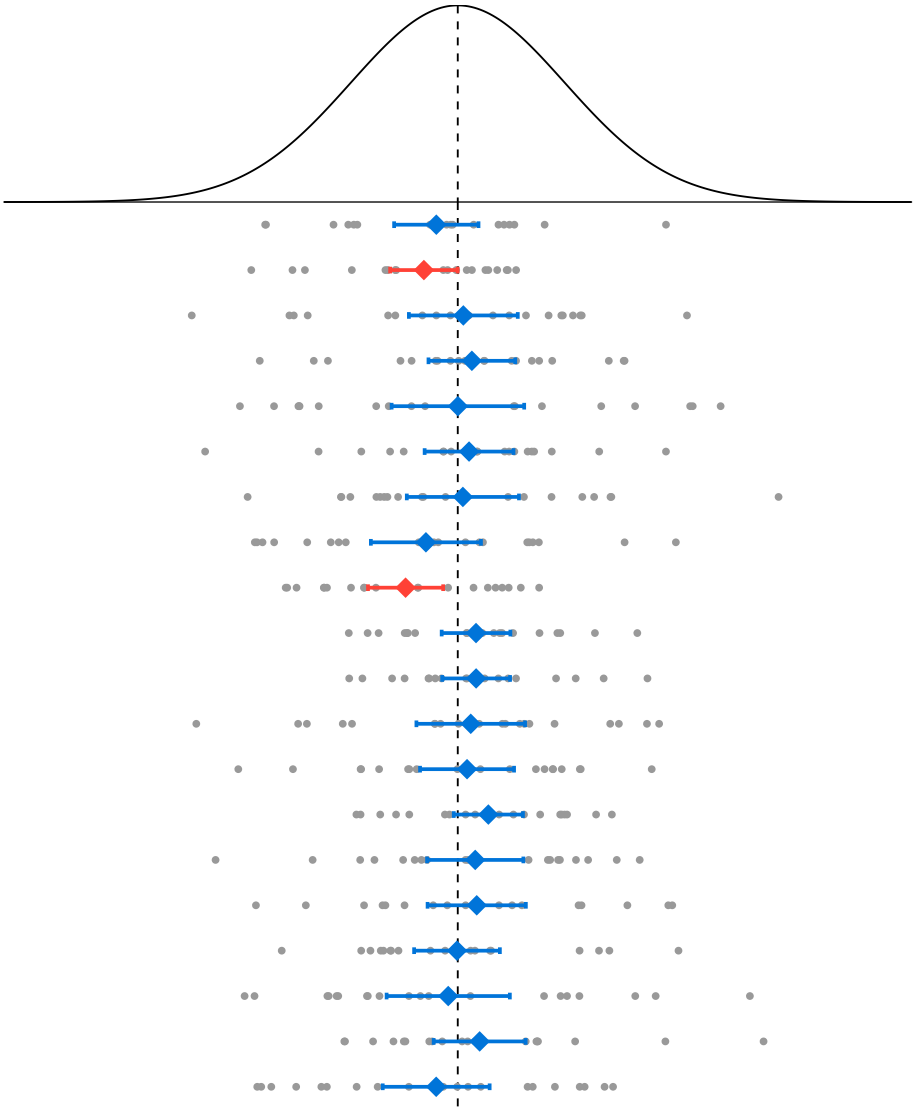
Note what is random here: μ is a fixed (unknown) number and $\bar{X}$ is the quantity that varies from sample to sample.

143.8.1.1. Step 1 — Standardise

We cannot look up probabilities for $\bar{X}$ directly, but we can for the standard normal Z. So rewrite the event until the middle term *is* a Z:

$P\big(\mu - b \le \bar{X} \le \mu + b\big)$	$= P\big(-b \le \bar{X} - \mu \le b\big)$	subtract the mean from all three parts
	$= P\left(-\frac{b}{\sigma/\sqrt{n}} \le \frac{\bar{X}-\mu}{\sigma/\sqrt{n}} \le \frac{b}{\sigma/\sqrt{n}}\right)$	divide all three parts by SE (positive, so the inequalities keep their direction)
	$= P(-z \le Z \le z)$	the middle term is now (value - its mean) / (its sd), i.e. standard normal, so $z = b/(\sigma/\sqrt{n})$
	$= P\left(-\frac{z\sigma}{\sqrt{n}} \le \bar{X} - \mu \le \frac{z\sigma}{\sqrt{n}}\right)$	substitute $b = z\sigma/\sqrt{n}$ into line 2
	$= P\left(-\frac{z\sigma}{\sqrt{n}} - \bar{X} \le -\mu \le \frac{z\sigma}{\sqrt{n}} - \bar{X}\right)$	subtract $\bar{X}$ from all three parts
	$= P\left(\frac{z\sigma}{\sqrt{n}} + \bar{X} \ge \mu \ge -\frac{z\sigma}{\sqrt{n}} + \bar{X}\right)$	multiply by -1, so the inequalities flip
	$= P\left(\bar{X} - \frac{z\sigma}{\sqrt{n}} \le \mu \le \bar{X} + \frac{z\sigma}{\sqrt{n}}\right)$	rewrite reading right-to-left

$$\left[\bar{x} - z\frac{s}{\sqrt{n}}, \quad \bar{x} + z\frac{s}{\sqrt{n}}\right]$$



Example

Code

```
from scipy.stats import t, norm
import numpy as np

xbar = 10
s = 5
n = 1000
conf = 0.90
```

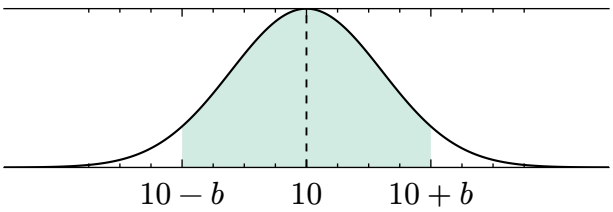
```python

df = n - 1
alpha = 1 - conf
p = 1 - alpha / 2
crit = t.ppf(p, df=df)
se = s / np.sqrt(n)
moe = crit * se

print(f"Degrees of Freedom:     {df}")
print(f"Alpha:                  {alpha:.2f}")
print(f"Cumulative Probability: {p:.3f}")
print(f"Critical Value (t):     {crit:.4f}")
print(f"Critical Value (z):     {norm.ppf(p):.4f}")
print(f"Standard Error:         {se:.4f}")
print(f"Margin of Error:        {moe:.4f}")

lower, upper = xbar - moe, xbar + moe
print(f"Confidence Interval:    [{lower:.2f}, {upper:.2f}]")
print(f"Check:                  {t.interval(conf, df, loc=xbar,
scale=se)}")
```

Code

```python
import numpy as np
from scipy import stats

x = np.array([12.1, 11.8, 13.4, 12.9, 12.2, 13.0, 11.5, 12.7])

stats.t.interval(0.95, len(x) - 1, loc=x.mean(), scale=stats.sem(x))
# (np.float64(12.005...), np.float64(13.019...))
```

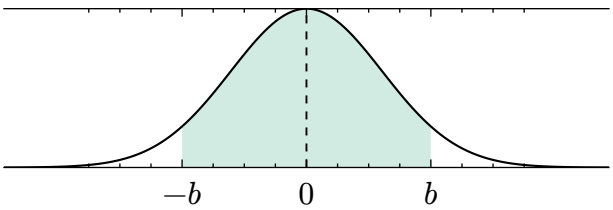
1. Sampling distribution

$$\bar{X} \sim N\left(10, \frac{5^2}{n}\right)$$



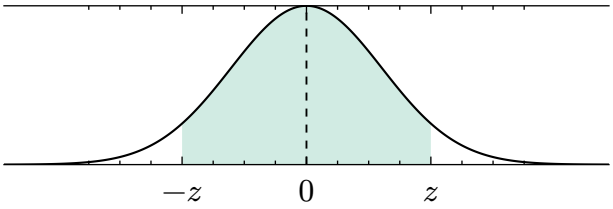
2. Subtract 10

$$P\left(-b \leq \bar{X} - 10 \leq b\right)$$



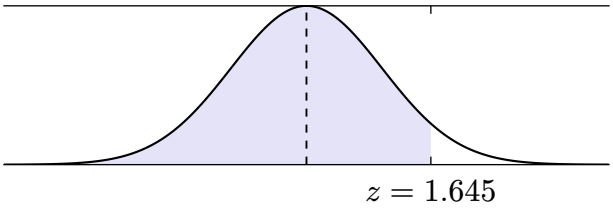
3. Divide by $5/\sqrt{n}$

$$Z = \frac{\bar{X} - 10}{5/\sqrt{n}}$$



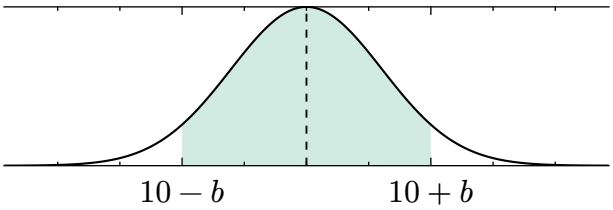
4. Look up the critical value

$$P(Z \leq z) = 1 - \frac{\alpha}{2} = 0.95$$



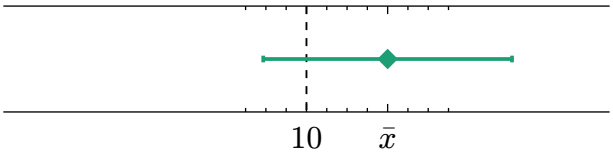
5. Back-substitute

$$b = 1.645\frac{5}{\sqrt{n}}$$



6. Rearrange for 10

$$\bar{x} \pm 1.645\frac{s}{\sqrt{n}}$$



144. Correlation

144.1. Pearson

Linear relationships

$$r = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2 \times (y_i - \bar{y})^2}}$$

Interpretation:

- 1: perfect positive linear relationship
- –1: perfect negative linear relationship
- 0: no linear relationship

Example

$$X : [1, 2, 3, 4, 5]$$
$$Y : [2, 4, 6, 8, 10]$$

Step 1: Calculate the Means of X and Y

$$\bar{X} = 3$$
$$\bar{Y} = 6$$

Step 2: Calculate the Differences from the Mean

144.2. Spearman Rank

144.2.1. Spearman's Rank

Non-linear relationships

$$\rho = 1 - \frac{6 \sum_{i=1}^{n} d_i^2}{n(n^2 - 1)}$$

Where

- ρ: Spearman rank correlation coefficient
- d_i: difference between the ranks of corresponding values
- n: number of observations

Rank: position of a value within a data set when the values are ordered in ascending order

Observation i	X	Rank X	Y	Rank Y	d_i = Rank X - Rank Y	d_i^2
1	3	3	8	5	–2	4
2	1	1	6	3	–2	4
3	4	4	7	4	0	0
4	2	2	4	1	1	1
5	5	5	5	2	3	9

Interpretation

- ρ = 1: Perfect positive correlation
- ρ = –1: Perfect negative correlation
- ρ = 0: No correlation

144.3. Covariance & Correlation

144.3.1. Covariance

$$\operatorname{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])] = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y]$$
$$\frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{n}$$

Example

$$X = (1, 2, 3)$$
$$Y = (2, 4, 6)$$

$$\mathbb{E}[X] = \frac{1+2+3}{3} = 2$$
$$\mathbb{E}[Y] = \frac{2+4+6}{3} = 4$$

x_i	y_i	$x_i - \mathbb{E}[X]$	$y_i - \mathbb{E}[Y]$	product
1	2	–1	–2	2
2	4	0	0	0
3	6	1	2	2

$$\mathrm{Cov}(X, Y) = \frac{2+0+2}{3} = \frac{4}{3} \approx 1.33$$

Cross-check with the formula:

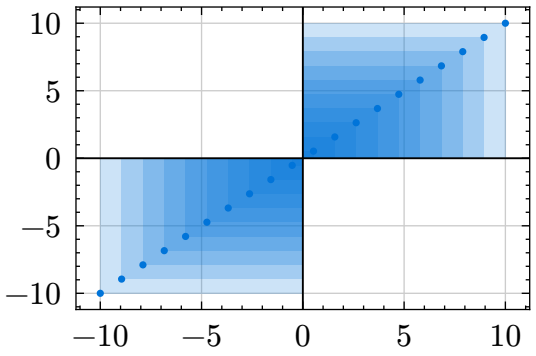
$$\mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y]$$

$$\mathbb{E}[XY] = \frac{(1)(2) + (2)(4) + (3)(6)}{3} = \frac{2+8+18}{3} = \frac{28}{3}$$
$$\mathbb{E}[X]\mathbb{E}[Y] = (2)(4) = 8$$
$$\mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y] = \frac{28}{3} - 8 = \frac{4}{3}$$

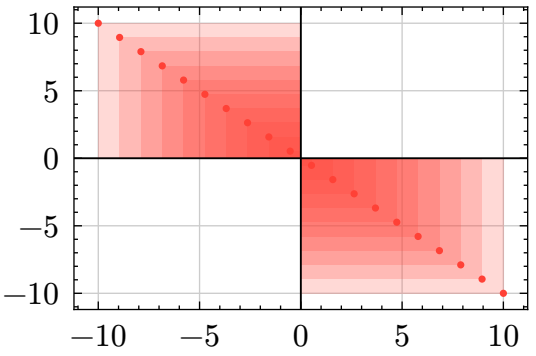
Covariance is unbounded; correlation is the normalized version:

$$\rho_{XY} = \frac{\mathrm{Cov}(X, Y)}{\sigma_X \sigma_Y}$$

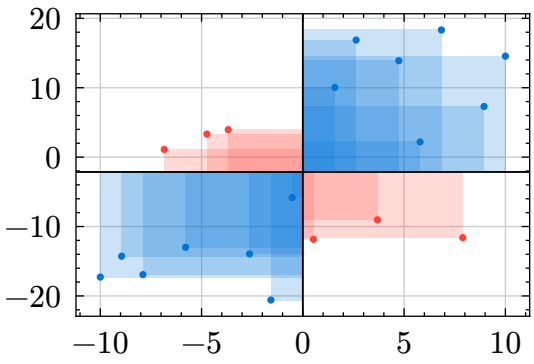
144.3.2. Covariance and Correlation



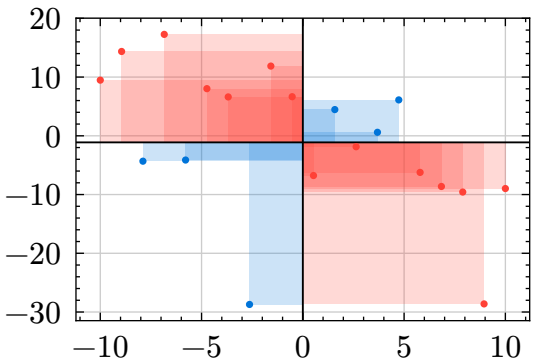
$\mathrm{Cov} = 36.84$
$\mathrm{Corr} = 1$



$\mathrm{Cov} = -36.84$
$\mathrm{Corr} = -1$



$\text{Cov} = 44.38$
$\text{Corr} = 0.59$



$\text{Cov} = -38.96$
$\text{Corr} = -0.53$

145. Non-Parametric Statistics

145.1. Mann–Whitney U

Determine whether there is a significant difference between the distributions of two independent samples (used as an alternative to the independent samples t-test when the assumptions of normality are not met)

145.2. Wilcoxon Signed-Rank

Compare two paired samples or to assess whether the median of a single sample is different from a specified value (used as a non-parametric alternative to the paired t-test when the data does not meet the assumptions of normality)

145.3. Kolmogorov–Smirnov

Determine if a sample is drawn from a population with a specific distribution.

It compares the empirical distribution function (EDF) of the sample with the cumulative distribution function (CDF) of the reference distribution.

145.4. Kruskal–Wallis

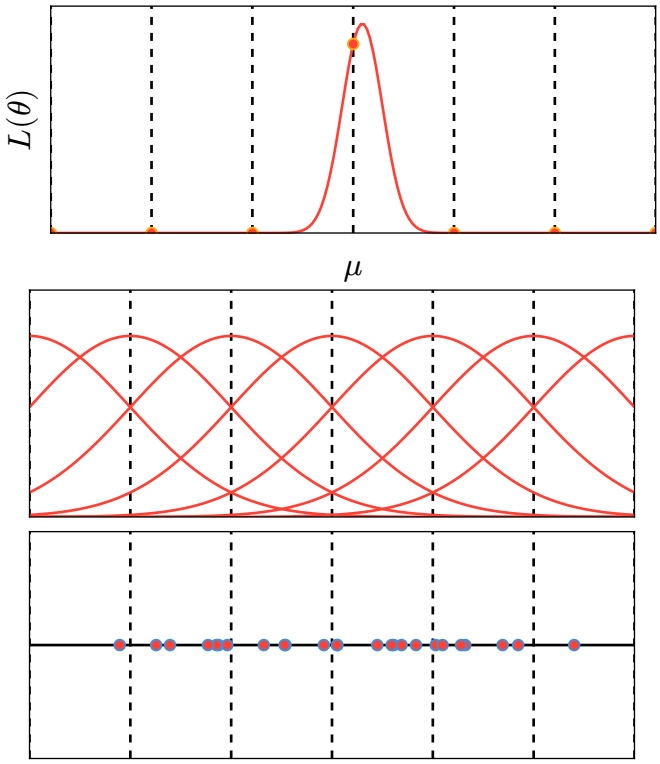
Determine if there are significant differences between the medians of three or more independent groups (extension of the Mann-Whitney U test, which is used for comparing two groups)

146. Maximum Likelihood Estimation

$$L(\theta) = \prod_{i=1}^{n} f(x_i \mid \theta)$$

Where:

- $f(x \mid \theta)$: probability density (or mass) function for each observation



Log Likelihood

$$\ell(\theta) = \log L(\theta) = \sum_{i=1}^{n} \log f(x_i \mid \theta)$$

147. Generalized Linear Models

Three components:

1. Random Component

Specifies the distribution of the response variable Y.

Y can come from the exponential family of distributions (which includes Normal, Binomial, Poisson, Gamma, etc.)

2. Systematic Component

Specifies the linear predictor, a linear combination of explanatory variables:

$$\eta = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + ... + \beta_p x_p$$

3. Link Function

Connects the expected value of Y, denoted $\mu = E[Y]$, to the linear predictor η:

$$g(\mu) - \eta$$

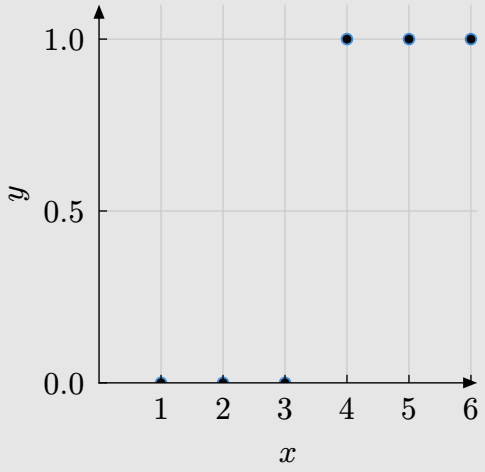
Where $g(\cdot)$ is called the link function

Model	Response Type	Distribution	Link Function
Linear Regression	Continuous	Normal	$g(\mu) = \mu$ (identity)
Logistic Regression	Binary (0, 1)	Binomial	$g(\mu) = \log\left(\frac{\mu}{1-\mu}\right)$ (logit)
Poisson Regression	Count	Poisson	$g(\mu) = \log(\mu)$
Gamma Regression	Positive continuous	Gamma	$g(\mu) = \log(\mu)$

Example

Logistic Regression

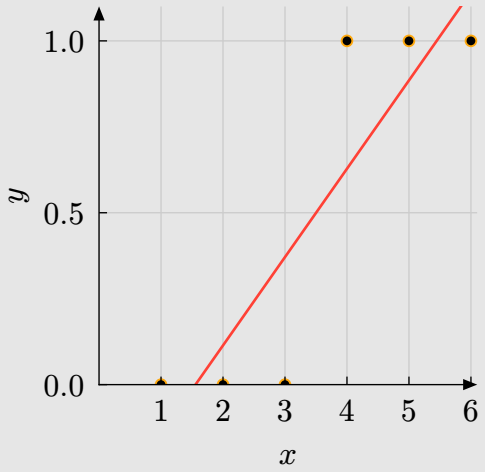
x	y
1	0
2	0
3	0
4	1
5	1
6	1



Fit linear model

$$\eta = \beta_0 + \beta_1 x$$

$$\beta_0 = -0.4$$
$$\beta_1 = 0.257$$



Logit link function:

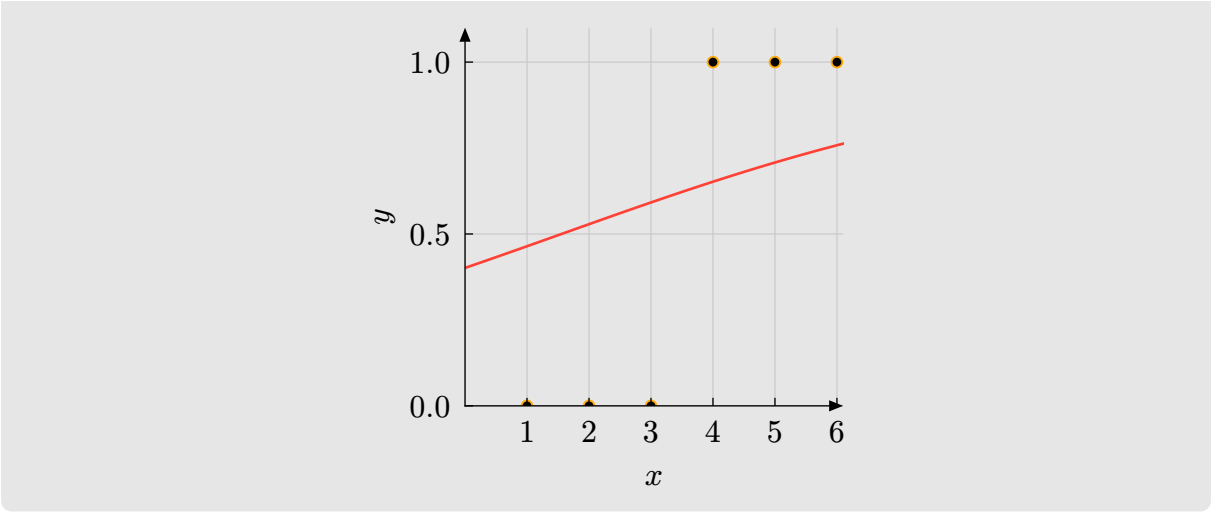
$$g(p) = \log \frac{p}{1-p} = \eta$$

Model

$$\log \frac{p}{1-p} = \beta_0 + \beta_1 x$$

Equivalently

$$p = \frac{1}{1 + e^{-(\beta_0 + b_1 x)}}$$

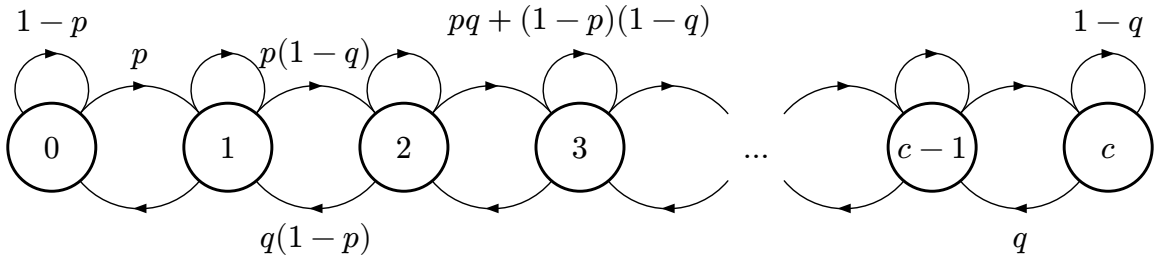


148. Cross Validation

149. Markov Chains

149.1. Checkout Counter Model

- Discrete time ($n = 0, 1, ...$)
- Custommer arrivals: Benoulli(p)
    - Geometric arrival times
- Custommer service times: Geometric(q)
- State X_n: Number of cusommers at time n



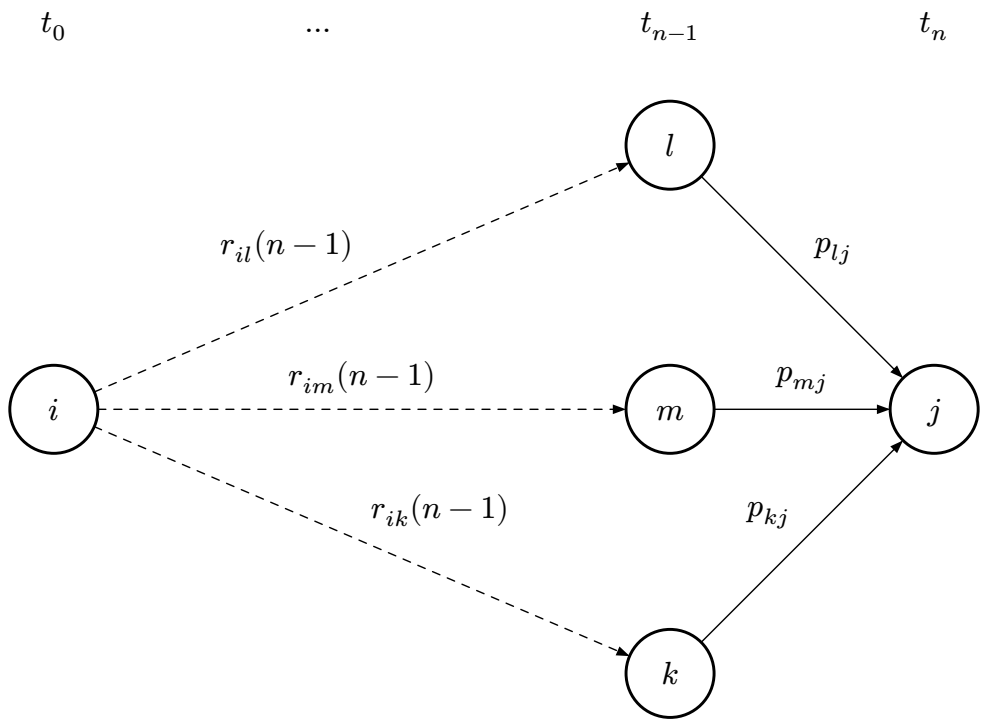
Where:

- p: Probability of having an arrival
- q: Probability of having an departure
- pq: Probability of having an arrival and a departure simultaneously
- $1-p$: Probability of not having a custommer arrival
- $1-q$: Probability of not having a custommer departure
- $p(1-q)$: Probability of custommer arrival without a custommer departure
- $q(1-p)$: Probability of custommer departure without a custommer arrival

149.2. n-Step Transition Probabilities

State occupance probabilities given initial state i:

$$r_{ij} = P(X_n = j \mid X_0 = i)$$



149.3. Key Recursion

The total probability of ending at with state j is the sum of probabilities of the different paths to get to state j

$$r_{ij} = \sum_{k=1}^{m} r_{ik}(n-1)p_{kj}$$

149.4. Random Initial State

$$P(X_n = j) = \sum_{i=1}^{m} P(X_0 = i)r_{ij}(n)$$

150. Monte Carlo Simulation

151. Design of Experiments

151.1. Factorial Designs

151.2. Response Surface Methodology

151.3. Analysis of Interaction Effects

152. Principal Component Analysis

152.1. PCA as application

Covariance Matrix

Principle components are always perpendicular

Let X_j denote the j-th feature, and $X_1, X_2, ..., X_d$ are your d features:

$$C = \begin{bmatrix} \text{var}(X_1) & \text{cov}(X_1, X_2) & ... & \text{cov}(X_1, X_d) \\ \text{cov}(X_2, X_1) & \text{var}(X_2) & ... & \text{cov}(X_2, X_d) \\ \vdots & \vdots & \ddots & \vdots \\ \text{cov}(X_d, X_1) & \text{cov}(X_d, X_2) & ... & \text{var}(X_d) \end{bmatrix}$$

Example

$$X = \{(0,2), (2,0), (4,3), (6,4), (8,6)\}$$

Step 1 Center

$$\overline{x} = \frac{0+2+4+6+8}{5} = 5 \qquad \overline{y} = \frac{2+0+3+4+6}{5} = 3$$

Subtract (5, 4) from each point

$$\bar{X} = \{(-4,-1), (-2,-3), (0,0), (2,1), (4,3)\}$$

Step 2 Covariance matrix

$$\text{var}(x) = \frac{16+4+0+4+16}{4}$$

$$\text{var}(y) = \frac{1+9+0+1+9}{4}$$

$$\text{cov}(x,y) = \frac{(-4)(-1)+(-2)(-3)+0+(2)(1)+(4)(3)}{4} = 6$$

$$C = \begin{bmatrix} 10 & 6 \\ 6 & 5 \end{bmatrix}$$

- top-left = 10 → how much &x& spreads on its own
- bottom-right = 5 → how much y spreads on its own (x spreads twice as much)
- off-diagonal = 6 → how x and y move together (positive, so they rise together)

Step 3 — Eigenvalues and eigenvectors

Each principal component is an eigenvector v of C: a direction that C only stretches, never rotates.

$$Cv = \lambda v \quad \Rightarrow \quad (C - \lambda I)v = 0$$

Eigenvalues — force a non-trivial solution with $\det(C - \lambda I) = 0$:

$$\begin{aligned} \det(C - \lambda I) &= (10-\lambda)(5-\lambda) - 36 \\ &= \lambda^2 - 15\lambda + 14 \\ &= (\lambda - 14)(\lambda - 1) = 0 \end{aligned}$$

$$\Rightarrow \quad \lambda_1 = 14 \quad \text{or} \quad \lambda_2 = 1$$

Eigenvectors — substitute each λ back into $(C - \lambda I)v = 0$.

For $\lambda_1 = 14$, with $C - 14I = \begin{bmatrix} -4 & 6 \\ 6 & -9 \end{bmatrix}$:

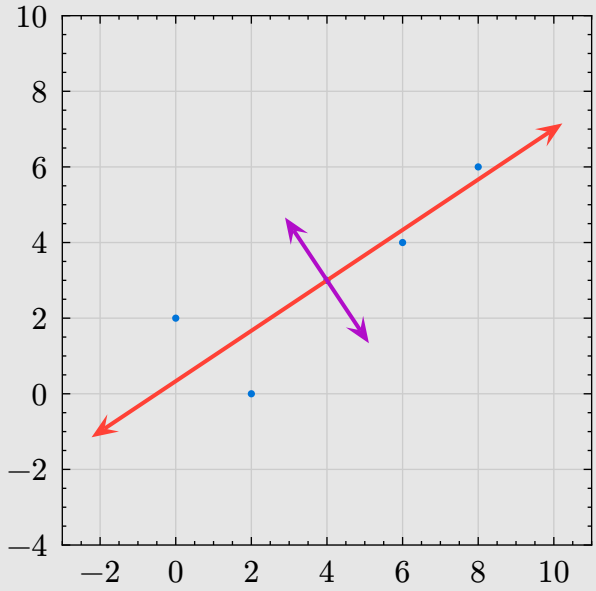
$$-4v_1 + 6v_2 = 0 \quad \Rightarrow \quad 2v_1 = 3v_2 \quad \Rightarrow \quad v \propto \begin{bmatrix} 3 \\ 2 \end{bmatrix}$$

For $\lambda_2 = 1$, with $C - 1I = \begin{bmatrix} 9 & 6 \\ 6 & 4 \end{bmatrix}$:

$$9v_1 + 6v_2 = 0 \quad \Rightarrow \quad 3v_1 = -2v_2 \quad \Rightarrow \quad v \propto \begin{bmatrix} 2 \\ -3 \end{bmatrix}$$

Normalise to unit length:

$$\text{PC}_1 = \frac{1}{\sqrt{13}} \begin{bmatrix} 3 \\ 2 \end{bmatrix} \quad \text{PC}_2 = \frac{1}{\sqrt{13}} \begin{bmatrix} 2 \\ -3 \end{bmatrix}$$



153. AIC & BIC

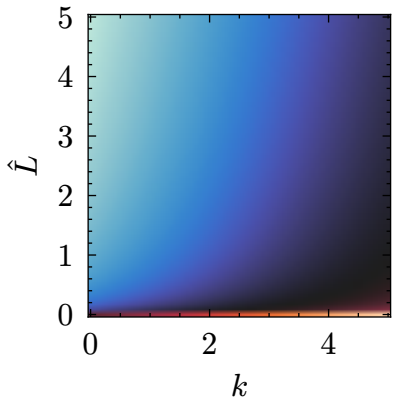
Picking between competing models

Maximized likelihood

153.1. AIC

$$\text{AIC} = \underbrace{2k}_{\text{penalty term}} - \underbrace{2 \cdot \ln\left(\hat{L}\right)}_{\text{fit term}}$$

- k: number of parameters in the model
- n: number of data points

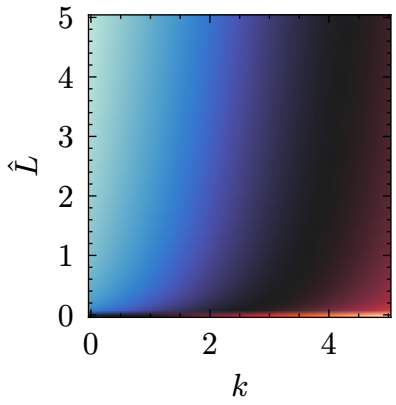


153.2. BIC

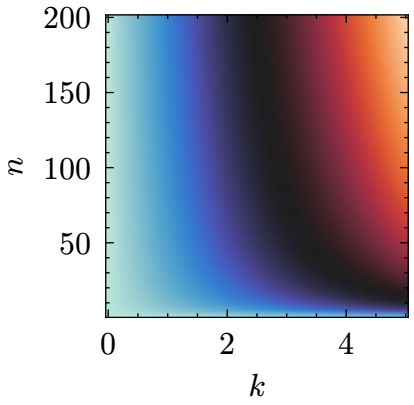
$$\text{BIC} = \underbrace{k \cdot \ln(n)}_{\text{penalty term}} - \underbrace{2 \cdot \ln\left(\hat{L}\right)}_{\text{fit term}}$$

- k: number of parameters in the model

- n: number of data points



Fix n, plot over k and $\hat{L}$



Fix $\hat{L}$, plot over k and n

Explanation:

- The fit term, $-2 \cdot \ln(L)$, is identical in both (Better fit $\rightarrow$ bigger $\hat{L}$ $\rightarrow$ bigger ln($\hat{L}$) $\rightarrow$ this term goes down $\rightarrow$ criterion goes down
- The penalty term is the only difference:
  - AIC charges 2 per parameter
  - BIC charges ln(n) per parameter

Example

You have n = 100 data points and two candidate models for them:

- Model A (simpler): k = 3 parameters, maximized log-likelihood ln($\hat{L}$) = –120
- Model B (fancier): k = 6 parameters, maximized log-likelihood ln($\hat{L}$) = –116

Model B fits better — its log-likelihood is higher (-116 > –120), which it should be, since it has more parameters to play with. The whole question is whether those 3 extra parameters are worth it.

AIC $2k - 2 \cdot \ln\left(\hat{L}\right)$

- Model A: 2·3 - 2·(-120) = 6 + 240 = 246
- Model B: 2·6 - 2·(-116) = 12 + 232 = 244

AIC picks Model B (244 < 246)

BIC $k \cdot \ln(n) - 2 \cdot \ln\left(\hat{L}\right)$

- Model A: 3·4.6 - 2·(-120) = 13.8 + 240 = 253.8
- Model B: 6·4.6 - 2·(-116) = 27.6 + 232 = 259.6

BIC picks Model A (253.8 < 259.6)

Same data, same two models, opposite verdicts. Here's the why, and it's exactly that per-parameter cost from before: Going from A to B, the fit term drops by 8 (from 240 to 232) — that's the reward for the better fit. The 3 extra parameters cost you:

BIC charges more per parameter (because ln(100) > 2), so it demands a bigger fit improvement before it'll accept extra complexity. That's why it leans toward simpler models.

154. Bootstrap & Resampling

155. Types of Statistics

- Descriptive: summarize data you actually have
- Inferential: generalize from a sample to a broader population
- Predictive: cares about forecasting accuracy on new data regarless of whether the model is interpretable (inferential cares about understanding the data-generating process)
- Exploratory hypothesis generating rather than testing

156. Time Series

157. Moving Averages

157.1. Overview

157.1.1. Moving Averages

A family of **forecasting methods** that produce a forecast as a **weighted average of past observations**. Each method is a different choice of weights along a single axis: **how much should past observations count?**

157.1.1.1. The weighting-scheme axis

Two extremes and two interpolators. All five methods can be written in the form

$$\hat{x}_{t,t+1} = \sum_{i=0}^{N-1} w_i x_{t-i}$$

— they only differ in how they choose the weights w_i (and the window N).

Method	Window	Weights	Formula	See
Naïve	1	(1, 0, 0, …)	$\hat{x}_{t,t+1} = x_t$	[naive.typ] (naive.typ)
SMA	M	$(1/M, ..., 1/M, 0, ...)$	$\hat{x}_{t,t+1} = \frac{1}{M}\sum_{i=0}^{M-1} x_{t-i}$	[sma.typ](sma.typ)
WMA	M	$(w_0, w_1, ..., w_{M-1}, 0, ...)$	$\hat{x}_{t,t+1} = \frac{\sum w_i x_{t-i}}{\sum w_i}$	[wma.typ] (wma.typ)
SES	all	$\alpha(1-\alpha)^i$ (geometric decay)	$\hat{x}_{t,t+1} = \alpha\sum_{i=0}^{t}(1-\alpha)^i x_{t-i}$	ETS(A,N,N)
Cumulative	t	$(1/t, 1/t, ..., 1/t)$	$\hat{x}_{t,t+1} = \frac{1}{t}\sum_{i=1}^{t} x_i$	[cumulative.typ] (cumulative.typ)

157.1.1.2. Reading the axis

Two endpoints and two interpolators:

- **Naïve** is the case $M = 1$: only the most recent observation matters. Maximum responsiveness, no smoothing.
- **Cumulative** is the case $M = t$: every observation since the start counts equally. Maximum smoothing, no responsiveness.

Between them, two interpolators:

- **SMA** and **WMA**: finite window of size M, with explicit weights (equal for SMA, custom for WMA). Choosing M trades responsiveness vs smoothing.
- **SES**: infinite window with geometrically-decaying weights. Choosing α trades responsiveness ($\alpha \to 1$, looks like Naïve) vs smoothing ($\alpha \to 0$, looks like Cumulative).

The full ETS family (all 30 cells, see `ets/`) generalizes SES by adding trend and seasonal components on top of the level.

157.1.1.3. When to use each

Method	Use when
Naïve	Random-walk demand (no structure to exploit). Useful baseline — every other method should beat it.
SMA	Smooth, slowly-changing demand without trend. Easy to explain to non-technical stakeholders.
WMA	Same as SMA, but you want to give more weight to recent observations without going to full geometric decay.
SES	Slowly-changing level + want infinite-history weighting. Most operational forecasting starts here.
Cumulative	Estimating a long-run mean (for stable processes) or as a slow-baseline in regime-detection.

157.1.1.4. Comparing them on the same data

For a **stationary** series, SMA, WMA, SES, and Cumulative all converge to the same long-run mean — they only differ in how fast they react to changes. For a **trending** series, all of them lag behind (they're averages — they smooth out the trend); use ETS or ARIMA if you have trend.

The next four files cover Naïve, SMA, WMA, and Cumulative; SES is in [ets/a-n-n.typ](../ets/a-n-n.typ) as `ETS(A, N, N)`.

157.2. Naive

157.2.1. Naïve Forecast

The simplest possible forecast: **next value equals last value**.

$$\hat{x}_{t,t+1} = x_t$$

For a h-step-ahead forecast: $\hat{x}_{t,t+h} = x_t$ — flat extrapolation of the last observation.

157.2.1.1. When it's the right choice

The naïve forecast is **optimal** for a **random walk** — a series where each step is the previous value plus mean-zero noise:

$$x_t = x_{t-1} + \varepsilon_t, \quad \varepsilon_t \sim \text{iid}(0, \sigma^2)$$

Examples of approximate random walks:

- Daily stock prices (efficient-market hypothesis)
- Daily exchange rates
- High-frequency series with no seasonality or trend

For these, **no model can beat naïve** because there's no exploitable structure. Naïve is the textbook **baseline**: any forecaster should outperform it on series with structure, and **not** outperforming it is a flag that the data is essentially a random walk.

157.2.1.2. Variants

Variant	Formula
Naïve	$\hat{x}_{t,t+h} = x_t$
Seasonal naïve	$\hat{x}_{t,t+h} = x_{t+h-m}$ — repeat the value from one period ago, where m is the seasonal period
Drift	$\hat{x}_{t,t+h} = x_t + h \cdot (x_t - x_1)/(t-1)$ — naïve + average historical drift

Seasonal naïve is the surprisingly-good baseline for highly seasonal series (retail, weather, energy demand). Drift is the equivalent baseline for series with a clear trend.

157.2.1.3. Naïve as a benchmark

Always evaluate forecasts **relative** to naïve. Define the **MASE** (mean absolute scaled error):

$$\text{MASE} = \frac{\text{MAE of model}}{\text{MAE of naïve}}$$

$\text{MASE} < 1$: the model beats naïve. $\text{MASE} \geq 1$: it doesn't — try a different model or accept that the series is a random walk.

The same logic applies to seasonal series with seasonal naïve as the denominator.

Example: Naïve forecast and the random-walk test

Given (last 5 days of a series):

t	1	2	3	4	5
x_t	100	102	101	104	103

Step 1 — naïve 1-step forecast at $t = 5$

$$\hat{x}_{5,6} = x_5 = 103$$

Step 2 — naïve h-step forecasts

$$\hat{x}_{5,6} = \hat{x}_{5,7} = \hat{x}_{5,8} = \ldots = 103$$

Flat at 103 forever.

Step 3 — drift variant

Average historical step: $(x_5 - x_1)/(5-1) = (103 - 100)/4 = 0.75$.

$$\hat{x}_{5,6} = 103 + 1 \cdot 0.75 = 103.75$$

$$\hat{x}_{5,10} = 103 + 5 \cdot 0.75 = 106.75$$

Step 4 — interpret

Since the series wandered up and down between 100 and 104 with no clear trend, naïve's flat 103 is probably about as good as you can do. If a forecaster predicts something fancy and gets MASE > 1, the series is essentially noise — switch to naïve, save the effort.

157.3. Cumulative

Average of **every** observation since the start of the series. The expanding-window endpoint of the moving-averages axis.

$$s_t = \frac{1}{t}\sum_{i=1}^{t} x_i$$

Forecast: $\hat{x}_{t,t+1} = s_t$ — predict the long-run average.

157.3.1. Behavior

The cumulative average can also be written **recursively**:

$$s_t = \frac{(t-1)s_{t-1} + x_t}{t} = s_{t-1} + \frac{x_t - s_{t-1}}{t}$$

The second form makes the dynamics clear:

- New observation x_t is compared to the running mean s_{t-1}.
- The error $(x_t - s_{t-1})$ is added back, scaled by $1/t$.

So the **effective smoothing parameter** of the cumulative average is $\alpha_t = 1/t$ — **decreasing over time**. As more data arrives, the cumulative average becomes increasingly insensitive to new observations.

157.3.2. Comparison to other moving averages

- **Naïve** ($M = 1$, $\alpha = 1$): only most-recent matters; ignores history.
- **SMA** (fixed M): finite window; weight per observation is $1/M$.
- **SES** (fixed α): infinite window; weight on most recent is α, decaying geometrically.
- **Cumulative** ($M = t$, $\alpha_t = 1/t$): infinite window with **equal** weights.

The relationship between M, α, and effective memory:

- $\alpha = 1/M$ for SMA and an "average age" perspective on SES.
- For the cumulative average, $\alpha_t \to 0$ as $t \to \infty$ — effectively **infinite** memory.

157.3.3. When to use

- **Estimating a long-run mean** of a stationary process: cumulative converges to the true mean by the law of large numbers.
- **Slow baseline** in regime detection or anomaly detection: a rapid moving average can be compared against the cumulative average to detect drift or shifts.
- **Online statistics**: classical "running mean" computed in a single pass.

Don't use it as a **forecast** for a non-stationary series. Cumulative gives equal weight to data from years ago and yesterday — useless if the level has shifted.

157.3.4. Connection to MLE

The cumulative average is the **maximum likelihood estimator** of the population mean for an i.i.d. process. So if you genuinely believe the data are i.i.d. with constant mean, cumulative average is **optimal** — best you can do.

If you don't believe i.i.d. (which is true of almost all real time series), use SES, ETS, or ARIMA instead.

Example

Given:

t	1	2	3	4	5	6
x_t	10	20	30	20	12	24

Iterate:

t	x_t	$s_t = \frac{1}{t}\sum_{i=1}^{t} x_i$	$\alpha_t = 1/t$
1	10	10/1 = 10.00	1.00
2	20	(10+20)/2 = 15.00	0.50
3	30	(10+20+30)/3 = 20.00	0.33
4	20	(10+20+30+20)/4 = 20.00	0.25
5	12	(10+20+30+20+12)/5 = 18.40	0.20
6	24	(10+20+30+20+12+24)/6 = 19.33	0.17

Notice:

- s_t stabilizes around 19–20 even as new data wiggles around.
- The effective α_t shrinks: by $t = 6$, the cumulative average reacts to a new observation with weight only 0.17. By $t = 100$, weight 0.01 — essentially frozen.
- This means the cumulative average is a **late** indicator. If the underlying mean shifts at $t = 100$, it'll take many observations before s_t catches up.

157.4. Simple Moving Average

157.4.1. SMA (Simple Moving Average)

For a time series y_t, the SMA of window size k at time t is:

$$s_t = \frac{1}{k}\sum_{i=0}^{k-1} y_{t-i}$$

The average of the last k values. Equal weights $1/k$ on each. The forecast is $\hat{x}_{t,t+1} = s_t$.

157.4.1.1. Choosing the window size k

The window size trades two effects:

- **Small k** (1–3): forecast tracks recent observations closely. Responsive to changes; noisy.
- **Large k** (10+): forecast smooths out short-term noise. Stable; lags behind real changes.

Rules of thumb:

- For **non-trending, smooth** series: k around 5–10 typically works.
- For **high-frequency noise**: bigger k to filter the noise.
- For **responsive forecasting** (regime detection): smaller k.

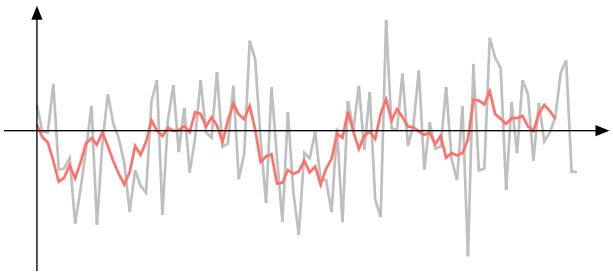
157.4.1.2. Lag

SMA **lags** the true level. If the underlying series is rising linearly, the SMA tracks roughly $k/2$ time steps behind — because the average of the last k values has a center of mass at $t - (k-1)/2$. Larger $k \longrightarrow$ larger lag.

This is why SMA is poor for **trending** series — it's structurally late. Use ETS or ARIMA when there's trend.

157.4.1.3. Visualization

The SMA filters the high-frequency noise and produces a smoother curve, but with a horizontal lag relative to the data.



- **Model parameters**
  - $k \in \mathbb{Z}_{\geq 1}$: window size (number of past observations to average)
- **Observation**
  - y_i: observed value at time i, for $i \in \{t-k+1, ..., t\}$
- **Output**
  - $s_t = \hat{x}_{t,t+1}$: one-step-ahead forecast for time $t+1$, given information up to t

Example

$k = 3$.

Data

t	1	2	3	4	5	6	7	8
x_t	10	20	30	20	12	24	36	24

Init — no parameters to seed; the first forecast is at $t = 4$ once we have $k = 3$ observations.

First forecast — at $t = 4$, using x_1, x_2, x_3:

$$\hat{x}_{3,4} = \frac{10 + 20 + 30}{3} = 20, \quad e_4 = 20 - 20 = 0$$

Iterate:

t	x_t	$\hat{x}_{t-1,t}$	e_t
4	20	(10 + 20 + 30) / 3 = 20.00	0.00
5	12	(20 + 30 + 20) / 3 = 23.33	−11.33
6	24	(30 + 20 + 12) / 3 = 20.67	3.33
7	36	(20 + 12 + 24) / 3 = 18.67	17.33
8	24	(12 + 24 + 36) / 3 = 24.00	0.00
9	—	(24 + 36 + 24) / 3 = 28.00	—

sma.py

```
pl.col('X').rolling_mean(window_size)
```

157.5. Weighted Moving Average

157.5.1. WMA (Weighted Moving Average)

Same finite-window structure as SMA, but each observation in the window gets a **custom weight** — usually with more weight on more recent observations.

$$\mathrm{WMA}_t = \frac{\sum_{i=0}^{M-1} w_i \cdot x_{t-i}}{\sum_{i=0}^{M-1} w_i}$$

where w_i is the weight on the observation i steps before t (so w_0 weights the most recent value, w_{M-1} the oldest in the window).

The denominator $\sum w_i$ normalizes the weights so the WMA is on the same scale as the data; if the weights already sum to 1, the denominator drops out.

157.5.1.1. Common weight schemes

- **Linearly decreasing** (most common): $w_i = M - i$. So $w_0 = M$, $w_1 = M - 1$, …, $w_{M-1} = 1$. Most weight on the most recent.

- **Triangular**: linearly increasing then decreasing — emphasizes the **middle** of the window.
- **Custom**: any non-negative weights you choose.

157.5.1.2. Comparison to SMA

- **SMA**: equal weight $1/M$ on every observation in the window.
- **WMA**: more weight on recent observations (with linearly-decreasing weights).

WMA is more responsive to recent changes than SMA at the same window size M, because the new observation gets a relatively large weight (e.g., $M/(M(M+1)/2) = 2/(M+1)$ for linearly decreasing — compared to $1/M$ for SMA).

157.5.1.3. Comparison to SES

WMA with linearly-decreasing weights is conceptually similar to SES, with two differences:

- WMA truncates at M; SES uses the entire history (with geometrically-decaying weights).
- WMA's most-recent weight depends on M (e.g., $2/(M+1)$); SES uses the explicit smoothing parameter α.

For $M \to \infty$ with linearly-decreasing weights, WMA approaches a **geometric-weighted** form similar to SES — but with polynomially-decaying tails rather than exponentially-decaying.

Example

Data

- Day 1: \$10
- Day 2: \$12
- Day 3: \$14
- Day 4: \$13
- Day 5: \$15

Weights (linearly decreasing, $M = 3$):

- $w_0 = 3$ (most recent)
- $w_1 = 2$
- $w_2 = 1$ (oldest in window)

Denominator: $3 + 2 + 1 = 6$.

Forecasts:

$$\text{WMA}_3 = \frac{3 \cdot 14 + 2 \cdot 12 + 1 \cdot 10}{6} = \frac{42 + 24 + 10}{6} = 12.67$$

$$\text{WMA}_4 = \frac{3 \cdot 13 + 2 \cdot 14 + 1 \cdot 12}{6} = \frac{39 + 28 + 12}{6} = 13.17$$

$$\text{WMA}_5 = \frac{3 \cdot 15 + 2 \cdot 13 + 1 \cdot 14}{6} = \frac{45 + 26 + 14}{6} = 14.17$$

Compare to SMA with $M = 3$ on the same data:

$$\text{SMA}_3 = \frac{10 + 12 + 14}{3} = 12.00$$

$$\text{SMA}_4 = \frac{12 + 14 + 13}{3} = 13.00$$

$$\text{SMA}_5 = \frac{14 + 13 + 15}{3} = 14.00$$

WMA values are slightly higher than SMA because the most-recent (and largest) observations get more weight. On a **trending up** series, WMA tracks the trend better; on a **random** series, both are similar.

wma.py

```
weights = [3, 2, 1]
pl.col('X').rolling_map(
    lambda s: sum(w * x for w, x in zip(weights, s)) / sum(weights),
    window_size=len(weights),
)
```

158. Decomposition

158.1. Classical

Decompose a time series y_t into four components that **add up** (or **multiply**) back to the original:

- **Level** L_t: baseline value around which the series fluctuates
- **Trend** T_t: long-term progression or direction
- **Seasonal** S_t: regular, repeating patterns over a fixed period m
- **Noise** N_t: irregular fluctuations not explained by the other components (residuals)

Two functional forms — additive and multiplicative — depending on whether the seasonal swing is **constant** in absolute units or **proportional** to the level.

158.1.1. Additive form

$$y_t = L_t + T_t + S_t + N_t$$

Use when the **amplitude** of seasonal swings is roughly constant over time, regardless of the level. Example: temperature (summer is 10°C above average whether the long-run average is 15°C or 25°C).

158.1.2. Multiplicative form

$$y_t = L_t \cdot T_t \cdot S_t \cdot N_t$$

Use when the seasonal amplitude **scales with** the level. Example: retail sales (December peak is 30% above the trend, whether the trend is at $1M or $10M).

A multiplicative decomposition can always be converted to an additive one via $\log y_t = \log L_t + \log T_t + \log S_t + \log N_t$ — useful trick if your tools support only additive.

158.1.3. How to choose

Plot the series. If the seasonal swing **grows** with the level → multiplicative. If it stays roughly constant → additive. When unsure, try both and check which has more uniform residuals.

158.1.4. Procedure (additive case, integer period m)

The classic step-by-step:

1. **Estimate trend** via centered moving average of size m. For even m (typical with monthly data, $m = 12$), use a 2-step MA: first an m-MA, then a 2-MA. Result: $\hat{T}_t$.
2. **Detrend**: $y_t - \hat{T}_t$.
3. **Estimate seasonal indices**: average the detrended values within each seasonal slot (e.g., all Januaries, all Februaries, etc.). Adjust so the seasonal indices sum to zero (additive) or average to 1 (multiplicative). Result: $\hat{S}_t$ (depends only on $t \bmod m$).
4. **Residual / noise**: $\hat{N}_t = y_t - \hat{T}_t - \hat{S}_t$.

158.1.5. Limitations of classical decomposition

- **Constant seasonal pattern**: classical decomposition assumes the seasonal effect is the **same** in every cycle. Real series often have **evolving** seasonality (Christmas effect grows over decades). Use STL ([stl.typ](stl.typ)) for those.
- **Endpoint problem**: the centered MA has gaps at the start and end (size $m/2$ each). Modern methods (STL, X-13) handle this better.
- **Sensitivity to outliers**: a single anomalous month can distort the seasonal index for that slot. Robust methods (STL with robust loess) address this.

For a quick first pass, classical decomposition is fine. For production forecasting, prefer STL.

Example

Given (24 months of monthly sales data, additive case):

Month	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
	Year 1	110	105	120	130	125	115	108	112	122	135	150
165		Year 2	120	115	130	140	135	125	118	122	132	145
160	175											

Step 1 — trend via 12-MA (centered, requires 2-MA on top)

At each month t (for $t \in \{7, \ldots, 18\}$ where the centered window fits), compute the 12-month centered moving average. For January year 2 ($t = 13$):

$$\hat{T}_{13} = (\text{avg of months } 8..18 \text{ first, then average pairs}) \approx 130$$

(rough estimate; actual computation involves a 12-MA followed by a 2-MA at each midpoint)

Step 2 — detrend

$y_t - \hat{T}_t$ leaves the seasonal + noise components:

Sample: $y_{13} - \hat{T}_{13} \approx 120 - 130 = -10$ (Jan year 2 is 10 below trend).

Step 3 — average detrended values per month

Pool **all** Januaries: $(110 - 116) + (120 - 130) \approx -8$, average ≈ -8. Pool **all** Decembers: $(165 - 121) + (175 - 142) \approx +37$. … and so on for each month.

Adjust so seasonal indices sum to 0:

	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
$\hat{S}$	-10	-15	0	$+10$	$+5$	-5	-12	-8	$+2$	$+15$	$+30$	$+45$

Sum: $\approx 57 \longrightarrow$ adjust by -4.75 per month so they sum to 0 (skipped here for brevity).

Step 4 — residuals

$\hat{N}_t = y_t - \hat{T}_t - \hat{S}_t$. If the model fits well, residuals are small and structureless.

Step 5 — interpret

- **Trend**: +10 units / year (rough slope between year-1 and year-2 averages).
- **Seasonal**: clear December peak, January-February through.
- **Noise**: small, well-explained.

Use: forecast next month by extending the trend and adding the seasonal index. For Jan year 3: $\hat{y}_{25} = \hat{T}_{25} + \hat{S}_{\text{Jan}} \approx 145 + (-10) = 135$.

158.2. STL

158.2.1. STL Decomposition (Seasonal-Trend using Loess)

Modern alternative to classical decomposition. **Robust**, handles **changing** seasonal patterns, and produces **complete** trend/seasonal estimates with no endpoint gaps.

Developed by Cleveland, Cleveland, McRae, and Terpenning (1990). Standard tool in `statsmodels`, `R::stats`, `forecast::stl`.

158.2.1.1. What it does

Decompose a series additively:

$$y_t = T_t + S_t + R_t$$

where T is trend, S is seasonal, R is the remainder (noise). Multiplicative decomposition is handled by first taking $\log y_t$, decomposing additively, then exponentiating.

Same components as classical decomposition, but the **estimation method** is different:

- **Loess** (Locally Estimated Scatterplot Smoothing): a non-parametric regression that fits local low-order polynomials to nearby points. Produces **smooth** trend and seasonal curves.
- **Iterative**: STL alternates between estimating trend and seasonal components, refining each pass. After few iterations the components stabilize.
- **Robustness option**: a robust-loess variant downweights outliers, making STL insensitive to anomalies.

158.2.1.2. Key parameters

Parameter	Typical value	Effect
`np` (period)	seasonal period m (e.g., 12 for monthly, 7 for daily-with-weekly-seasonality)	Defines the seasonal cycle
`ns` (seasonal window)	7-15 (must be odd ≥ 7)	Smaller → seasonal pattern can change rapidly. Larger → seasonal pattern is locked-in over time. Set based on how stable seasonality is.
`nt` (trend window)	auto: $1.5m/(1-1.5/ns)$	Larger → smoother trend. Smaller → trend tracks short-term moves.
`nl` (low-pass window)	next odd integer $\geq m$	Internal; usually leave default.
`robust`	True / False	If True, downweights outliers via a second pass. Use when there are known anomalies.

158.2.1.3. Why STL beats classical decomposition

Issue	Classical	STL
Seasonal pattern	Constant across cycles	Can evolve gradually (controlled by `ns`)
Endpoints	Gaps of $m/2$ at start and end	Smooth all the way through
Outliers	Sensitive — one anomaly distorts seasonal index	Robust mode downweights anomalies
Multiplicative decomp	Separate procedure	Take logs, decompose additively, exponentiate

158.2.1.4. Workflow

1. Plot the series, look at seasonality and trend.
2. Choose `np` (= seasonal period).
3. Try default STL — usually good. Inspect residuals: should be structureless.

4. If seasonal pattern is **clearly** shifting, lower `ns`. If residuals show seasonal structure (under-fit), lower `ns`.
5. If outliers, enable robust mode.
6. Use the components for: forecasting (forecast each component, recombine), anomaly detection (flag large residuals), seasonal-adjustment (subtract seasonal for trend-only view).

158.2.1.5. When STL doesn't work

- **Multiple seasonalities**: STL assumes one seasonal period. For daily data with both weekly and yearly seasonality, use MSTL (Multiple STL) or Fourier-based methods.
- **Non-additive structure**: if seasonality interacts with trend in a way that's not pure multiplicative, STL on $\log y$ won't capture it. Use a structural state-space model.
- **Short series**: STL needs at least **two full seasonal cycles**; ideally more.

Example

Given: 5 years of monthly retail sales with growing seasonal swing.

Setup:

- $m = 12$ (monthly data with yearly seasonality)
- $ns = 13$ (seasonal pattern can evolve slowly)
- Robust mode on (Black-Friday-style anomalies expected)

Output: three series, each of length 60.

- **Trend** T_t: smooth upward curve, captures the multi-year growth.
- **Seasonal** S_t: 12-month repeating pattern, but the December peak grows from year 1 (+10%) to year 5 (+18%) — that's the evolving seasonality STL captures.
- **Remainder** R_t: small, mean-zero, ideally autocorrelation-free.

Use the output:

- **Forecast**: extend trend (e.g., linear extrapolation or random-walk) + use the most-recent seasonal pattern.
- **Seasonally-adjusted series**: $y_t - S_t$. Useful for tracking underlying performance ignoring seasonal effects (Black Friday, summer, etc.).
- **Anomaly detection**: large residuals $|R_t|$ flag anomalous months.

Code (Python):

```python
from statsmodels.tsa.seasonal import STL
result = STL(y, period=12, seasonal=13, robust=True).fit()
trend = result.trend
seasonal = result.seasonal
remainder = result.resid
```

Plot all four (`result.plot()`) for visual diagnosis.

159. Frequency

159.1. Fourier

A **frequency-domain** representation of a time series. Decompose the series into a sum of sines and cosines at different frequencies. Useful for finding **dominant cycles** that aren't obvious in the time-domain view.

159.1.1. Discrete Fourier Transform (DFT)

For a series $y_0, y_1, ..., y_{N-1}$ of length N:

$$Y_k = \sum_{t=0}^{N-1} y_t \cdot e^{-2\pi i k t/N}, \quad k = 0, 1, ..., N-1$$

The output Y_k is a complex number. Its magnitude $|Y_k|$ measures how much the **frequency** k/N (cycles per time unit) is present in the signal. The phase $\arg(Y_k)$ tells you where the cycle peaks.

The inverse DFT recovers the original series:

$$y_t = \frac{1}{N} \sum_{k=0}^{N-1} Y_k \cdot e^{2\pi i k t/N}$$

159.1.2. Frequencies and periods

For a series sampled at one observation per unit time, the DFT covers frequencies k/N for $k = 0, 1, ..., N-1$:

- $k = 0$: DC component (mean of the series)
- $k = 1$: one cycle over the entire window — period N
- $k = 2$: two cycles over the window — period $N/2$
- …
- $k = N/2$: Nyquist frequency — period 2 (highest detectable cycle)
- $k > N/2$: redundant (mirror of lower frequencies; this is **aliasing**)

So you only look at the first $N/2$ frequencies for a real-valued series. The **period** corresponding to bin k is N/k.

159.1.3. FFT — the fast version

The Fast Fourier Transform (Cooley-Tukey 1965) computes the DFT in $O(N \log N)$ rather than the naive $O(N^2)$. It's the standard implementation in every numerical library (NumPy `fft.fft`, MATLAB `fft`, etc.).

Use FFT in practice; conceptually it computes exactly the DFT above.

159.1.4. What you can do with the Fourier transform

159.1.4.1. 1. Find dominant cycles

Plot $|Y_k|^2$ (the **power spectrum**) vs frequency. Peaks indicate dominant cycles. Example uses:

- Found a peak at $k = 12$ in 144 months of data → period 12 → annual cycle.
- Found a peak at $k = 1$ in 365 days → period 365 → annual cycle (in daily data).
- Multiple peaks → multiple seasonalities (e.g., daily data has both weekly **and** yearly peaks).

This is the core of periodogram.typ.

159.1.4.2. 2. Filter (frequency-domain)

Remove specific frequencies:

- **Low-pass filter**: keep low frequencies, drop high → smooths the series.
- **High-pass filter**: keep high, drop low → removes trend, isolates noise/cycles.
- **Band-pass filter**: keep a specific range → isolate one cycle.

Procedure: FFT → zero out unwanted frequency bins → inverse FFT.

159.1.4.3. 3. Forecast with Fourier features

Add `sin(2 pi k t \/ m)` and `cos(2 pi k t \/ m)` as regressors in a linear forecasting model, where m is the period. A handful of harmonics ($k = 1, 2, 3$) captures most of a smooth seasonal pattern.

This is how Prophet (Facebook) and many modern forecasting libraries handle seasonality without a fixed seasonal index.

159.1.4.4. 4. Spectral density estimation

Smoothed version of the periodogram → see periodogram.typ.

159.1.5. Limitations

- **Stationarity assumption**: standard FFT assumes the series is stationary. For series with changing frequency content (e.g., a chirp), use **short-time Fourier transform** (STFT) or wavelets.
- **Boundary effects**: a finite series treated as periodic introduces artifacts at the edges. Apply a **windowing function** (Hann, Hamming) before FFT to reduce these.
- **Resolution**: frequency resolution is $1/N$ — you can only distinguish cycles at multiples of this. Low frequencies need long series.

159.1.6. Connection to seasonality

If your series has a known seasonal period m, you'd expect a peak in the spectrum at $k = N/m$. Conversely, **unknown** periodicities are **discovered** via the spectrum — perhaps weekly + monthly + yearly all show up in the same series. Fourier analysis is the natural tool for this.

Example: Fourier analysis of monthly retail sales

Given: $N = 60$ months of sales data (5 years).

Step 1 — apply FFT

```
import numpy as np
```

```
Y = np.fft.fft(y)
power = np.abs(Y[:N//2])**2  # only first half (positive frequencies)
freq = np.arange(N//2) / N    # cycles per month
period = 1 / np.where(freq > 0, freq, np.inf)  # months per cycle
```

Step 2 — find peaks

Suppose the power spectrum shows large peaks at:

- $k = 5 \to$ frequency $\frac{5}{60} = \frac{1}{12} \to$ period 12 (monthly cycle = annual)
- $k = 10 \to$ frequency $\frac{10}{60} = \frac{1}{6} \to$ period 6 (semi-annual)
- $k = 1 \to$ frequency $\frac{1}{60} \to$ period 60 (long-term trend)

Step 3 — interpret

- The annual peak at $k = 5$ is the dominant seasonality.
- The semi-annual peak might be a back-to-school spike on top of the holiday spike.
- $k = 1$ at the lowest frequency captures the multi-year trend.

Step 4 — use

Pick a few harmonics ($k = 5, 10$, possibly 15) and use them as Fourier features in a regression: forecast = trend + linear combination of $\sin(2\pi kt/12)$ and $\cos(2\pi kt/12)$. The Fourier representation lets you smoothly interpolate seasonal effects between cycles.

159.2. Periodogram

Estimate the **power spectral density** (PSD) of a time series — i.e., how much variance is at each frequency. The simplest spectrum estimator; built directly from the Fourier transform.

159.2.1. Definition

For a series $y_0, y_1, ..., y_{N-1}$ with DFT Y_k (see fourier.typ):

$$I(k/N) = \frac{1}{N} |Y_k|^2, \quad k = 0, 1, ..., N-1$$

The periodogram is the function I evaluated at the discrete frequencies k/N. For a real-valued series, $I(k/N) = I(1 - k/N)$ (symmetric around 0.5), so you only plot the first $N/2$ bins.

159.2.2. What it tells you

- **Peaks at frequency k/N**: the series has a strong cyclic component with period N/k.
- **Flat spectrum**: white noise (no preferred frequency).
- **$1/f$-shaped spectrum**: low-frequency power dominates → trending or persistent series.

The **area** under the periodogram equals the variance of the series (Parseval's theorem). So peaks tell you what **fraction of variance** is explained by each frequency.

159.2.3. Why raw periodograms are noisy

The periodogram is **not consistent**: even with infinite data, its variance at each frequency does **not** decrease. So the raw periodogram is a **jagged** estimate — peaks are real, but their estimated heights are noisy.

Two standard fixes:

159.2.3.1. 1. Smoothing the periodogram

Average the periodogram over a small window of nearby frequencies (a **spectral window**). Trades resolution for variance reduction.

$$\hat{f}(k/N) = \sum_{j=-m}^{m} W_j \cdot I((k+j)/N)$$

with weights W_j summing to 1 (e.g., Bartlett, Hann, Tukey windows).

159.2.3.2. 2. Welch's method

Split the series into overlapping segments. Compute the periodogram of each segment. Average. Reduces variance at the cost of resolution; the standard implementation in `scipy.signal.welch`.

159.2.4. Significance test

How big does a peak need to be before we believe it?

For Gaussian white noise input, $2I(k/N)/\sigma^2 \sim \chi^2_2$ at each frequency. So a peak with $I > c\sigma^2$ has a known p-value.

Rule of thumb: peaks more than 2-3× the surrounding noise floor are likely real; smaller peaks often vanish under significance testing.

159.2.5. Workflow

1. Detrend the series (subtract a linear trend, or take first differences). Trend dominates the low-frequency spectrum and hides cycles.
2. Apply a **windowing function** (Hann, Hamming) before FFT to reduce edge artifacts.
3. Compute periodogram via FFT.
4. Smooth (or use Welch's method) to get a stable estimate.
5. Plot $\log(I)$ vs frequency — log scale reveals weak peaks.
6. Identify peaks; period = $1/$frequency.

159.2.6. Comparison to autocorrelation analysis

Periodogram and **autocorrelation function** (ACF) contain **equivalent information** (Wiener-Khinchin theorem):

$$\text{Autocorrelation}(\tau) = \text{inverse Fourier transform of } \ I(f)$$

A peak in the periodogram at frequency f corresponds to oscillating autocorrelation at period $1/f$. Use whichever is easier to read for your data:

- **Periodogram**: better for finding **unknown** periodicities.
- **ACF**: better for confirming and characterizing a **known** periodicity.

Most time-series analyses use both.

Example: Periodogram of daily energy demand

Given: 2 years (730 days) of daily electricity demand.

Step 1 — preprocess

- Detrend: subtract a linear regression line (removes long-term load growth).
- Apply a Hann window: $w_t = 0.5(1 - \cos(2\pi t/N))$.

Step 2 — FFT and periodogram

```
from scipy.signal import periodogram
freq, psd = periodogram(y_detrended, fs=1.0, window='hann')
period = 1 / freq[1:]  # skip DC
```

Step 3 — read peaks

Suppose `psd` peaks at:

- $f = 1/7$ (period = 7 days) → strong weekly cycle (weekday vs weekend pattern)
- $f = 1/365$ (period = 365 days) → annual cycle (heating/cooling load)
- $f = 1/3.5$ (period = 3.5 days) → secondary peak; possibly mid-week vs end-of-week subdivision

Step 4 — interpret

Two clear seasonalities (weekly and annual) — confirms what you'd see in classical / STL decomposition, but reveals them **quantitatively** without imposing a seasonal period in advance.

Step 5 — use

- For **forecasting**: include Fourier features at periods 7 and 365.
- For **decomposition**: STL with two seasonal components, or MSTL.
- For **anomaly detection**: a day with unusual frequency content (e.g., a power outage) shows up in the residuals after removing the dominant frequencies.

The periodogram is the **diagnostic step** that tells you which seasonal periods to model.

160. Exponential Smoothing (ETS)

160.1. Summary

160.1.1. ETS taxonomy (state space form)

■ level l_t ■ trend b_t ■ seasonality s_t

Each model has an *observation equation* (top, separated by a line) and *state equations* for each component plus a point-forecast formula. ε_t is the one-step-ahead innovation, assumed mean zero. m is the seasonal period. $m_h^+ = ((h-1) \bmod m) + 1$ picks the appropriate

seasonal index for an h-step-ahead forecast. Damped models use $\varphi \in (0, 1)$. Smoothing parameters $\alpha, \beta, \gamma \in (0, 1)$.

Every one of the 30 cells gives you the same two-step pattern:

1. Compute the innovation from the observation equation, solving for ε_t:
    - Additive errors: $\varepsilon_t = x_t - \mu_t$
    - Multiplicative errors: $\varepsilon_t = \frac{x_t - \mu_t}{\mu_t}$

where μ_t is whatever the observation equation says x_t equals when $\varepsilon_t = 0$ (i.e., the conditional mean).

2. Update the states by plugging ε_t into the state equations.

That's it. The same loop works for all 30 models — only the specific formulas for μ_t and the state updates change.

- **Error (E)**:
    - Additive (A)
    - Multiplicative (M)
- **Trend (T)**:
    - None (N)
    - Additive (A)
    - Additive damped (Ad)
    - Multiplicative (M)
    - Multiplicative damped (Md)
- **Seasonal (S)**:
    - None (N)
    - Additive (A)
    - Multiplicative (M)

160.1.1.1. Trend = N (no trend)

ETS(A,N,N)

Simple exponential smoothing

$$x_t = l_{t-1} + \varepsilon_t$$

$$l_t = l_{t-1} + \alpha\varepsilon_t$$
$$\hat{x}_{t+h \mid t} = l_t$$

ETS(M,N,N)

SES, multiplicative errors

$$x_t = l_{t-1}(1 + \varepsilon_t)$$

$$l_t = l_{t-1}(1 + \alpha\varepsilon_t)$$
$$\hat{x}_{t+h \mid t} = l_t$$

ETS(A,N,A)

SES + additive seasonality

$$x_t = l_{t-1} + s_{t-m} + \varepsilon_t$$

$$l_t = l_{t-1} + \alpha\varepsilon_t$$
$$s_t = s_{t-m} + \gamma\varepsilon_t$$
$$\hat{x}_{t+h \mid t} = l_t + s_{t+h-m_h^+}$$

ETS(M,N,A)

SES + add. seasonality, mult. errors

$$x_t = (l_{t-1} + s_{t-m})(1 + \varepsilon_t)$$

$$l_t = l_{t-1} + \alpha(l_{t-1} + s_{t-m})\varepsilon_t$$
$$s_t = s_{t-m} + \gamma(l_{t-1} + s_{t-m})\varepsilon_t$$
$$\hat{x}_{t+h \mid t} = l_t + s_{t+h-m_h^+}$$

ETS(A,N,M)

SES + multiplicative seasonality

$$x_t = l_{t-1}s_{t-m} + \varepsilon_t$$

$$\begin{aligned} l_t &= l_{t-1} + \alpha\varepsilon_t/s_{t-m} \\ s_t &= s_{t-m} + \gamma\varepsilon_t/l_{t-1} \\ \hat{x}_{t+h\,|\,t} &= l_t s_{t+h-m_h^+} \end{aligned}$$

ETS(M,N,M)

SES + mult. seasonality and errors

$$x_t = l_{t-1}s_{t-m}(1+\varepsilon_t)$$

$$\begin{aligned} l_t &= l_{t-1}(1+\alpha\varepsilon_t) \\ s_t &= s_{t-m}(1+\gamma\varepsilon_t) \\ \hat{x}_{t+h\,|\,t} &= l_t s_{t+h-m_h^+} \end{aligned}$$

160.1.1.2. Trend = A (additive trend)

ETS(A,A,N)

Holt's linear trend

$$x_t = l_{t-1} + b_{t-1} + \varepsilon_t$$

$$\begin{aligned} l_t &= l_{t-1} + b_{t-1} + \alpha\varepsilon_t \\ b_t &= b_{t-1} + \beta\varepsilon_t \\ \hat{x}_{t+h\,|\,t} &= l_t + hb_t \end{aligned}$$

ETS(M,A,N)

Holt's linear, multiplicative errors

$$x_t = (l_{t-1} + b_{t-1})(1+\varepsilon_t)$$

$$\begin{aligned} l_t &= (l_{t-1} + b_{t-1})(1+\alpha\varepsilon_t) \\ b_t &= b_{t-1} + \beta(l_{t-1} + b_{t-1})\varepsilon_t \\ \hat{x}_{t+h\,|\,t} &= l_t + hb_t \end{aligned}$$

ETS(A,A,A)

Additive Holt-Winters

$$x_t = l_{t-1} + b_{t-1} + s_{t-m} + \varepsilon_t$$

$$\begin{aligned} l_t &= l_{t-1} + b_{t-1} + \alpha\varepsilon_t \\ b_t &= b_{t-1} + \beta\varepsilon_t \\ s_t &= s_{t-m} + \gamma\varepsilon_t \\ \hat{x}_{t+h\,|\,t} &= l_t + hb_t + s_{t+h-m_h^+} \end{aligned}$$

ETS(M,A,A)

Additive HW, multiplicative errors

$$x_t = (l_{t-1} + b_{t-1} + s_{t-m})(1+\varepsilon_t)$$

$$\begin{aligned} l_t &= l_{t-1} + b_{t-1} + \alpha\mu_t\varepsilon_t \\ b_t &= b_{t-1} + \beta\mu_t\varepsilon_t \\ s_t &= s_{t-m} + \gamma\mu_t\varepsilon_t \\ \hat{x}_{t+h\,|\,t} &= l_t + hb_t + s_{t+h-m_h^+} \\ \text{where}\quad \mu_t &= l_{t-1} + b_{t-1} + s_{t-m} \end{aligned}$$

ETS(A,A,M)

HW with mult. seasonality

$$x_t = (l_{t-1} + b_{t-1})s_{t-m} + \varepsilon_t$$

$$\begin{aligned} l_t &= l_{t-1} + b_{t-1} + \alpha\varepsilon_t/s_{t-m} \\ b_t &= b_{t-1} + \beta\varepsilon_t/s_{t-m} \\ s_t &= s_{t-m} + \gamma\varepsilon_t/(l_{t-1} + b_{t-1}) \\ \hat{x}_{t+h\,|\,t} &= (l_t + hb_t)s_{t+h-m_h^+} \end{aligned}$$

ETS(M,A,M)

Multiplicative Holt-Winters

$$x_t = (l_{t-1} + b_{t-1})s_{t-m}(1+\varepsilon_t)$$

$$\begin{aligned} l_t &= (l_{t-1} + b_{t-1})(1+\alpha\varepsilon_t) \\ b_t &= b_{t-1} + \beta(l_{t-1} + b_{t-1})\varepsilon_t \\ s_t &= s_{t-m}(1+\gamma\varepsilon_t) \\ \hat{x}_{t+h\,|\,t} &= (l_t + hb_t)s_{t+h-m_h^+} \end{aligned}$$

160.1.1.3. Trend = Ad (damped additive trend)

ETS(A,Ad,N)

Damped linear trend

$$x_t = l_{t-1} + \varphi b_{t-1} + \varepsilon_t$$

$$\begin{aligned} l_t &= l_{t-1} + \varphi b_{t-1} + \alpha\varepsilon_t \\ b_t &= \varphi b_{t-1} + \beta\varepsilon_t \\ \hat{x}_{t+h\,|\,t} &= l_t + (\varphi + \varphi^2 + \ldots + \varphi^h)b_t \end{aligned}$$

ETS(M,Ad,N)

Damped linear, mult. errors

$$x_t = (l_{t-1} + \varphi b_{t-1})(1+\varepsilon_t)$$

$$\begin{aligned} l_t &= (l_{t-1} + \varphi b_{t-1})(1+\alpha\varepsilon_t) \\ b_t &= \varphi b_{t-1} + \beta(l_{t-1} + \varphi b_{t-1})\varepsilon_t \\ \hat{x}_{t+h\,|\,t} &= l_t + (\varphi + \varphi^2 + \ldots + \varphi^h)b_t \end{aligned}$$

ETS(A,Ad,A)

Damped additive Holt-Winters

$$x_t = l_{t-1} + \varphi b_{t-1} + s_{t-m} + \varepsilon_t$$

$$\begin{aligned} l_t &= l_{t-1} + \varphi b_{t-1} + \alpha\varepsilon_t \\ b_t &= \varphi b_{t-1} + \beta\varepsilon_t \\ s_t &= s_{t-m} + \gamma\varepsilon_t \\ \hat{x}_{t+h\,|\,t} &= l_t + (\varphi + \varphi^2 + \ldots + \varphi^h)b_t + s_{t+h-m_h^+} \end{aligned}$$

ETS(M,Ad,A)

Damped additive HW, mult. errors

$$x_t = (l_{t-1} + \varphi b_{t-1} + s_{t-m})(1 + \varepsilon_t)$$

$$\begin{aligned} l_t &= l_{t-1} + \varphi b_{t-1} + \alpha\mu_t\varepsilon_t \\ b_t &= \varphi b_{t-1} + \beta\mu_t\varepsilon_t \\ s_t &= s_{t-m} + \gamma\mu_t\varepsilon_t \\ \hat{x}_{t+h\,|\,t} &= l_t + (\varphi + \varphi^2 + \ldots + \varphi^h)b_t + s_{t+h-m_h^+} \\ \text{where}\quad \mu_t &= l_{t-1} + \varphi b_{t-1} + s_{t-m} \end{aligned}$$

ETS(A,Ad,M)

Damped HW, mult. seasonality

$$x_t = (l_{t-1} + \varphi b_{t-1})s_{t-m} + \varepsilon_t$$

$$\begin{aligned} l_t &= l_{t-1} + \varphi b_{t-1} + \alpha\varepsilon_t/s_{t-m} \\ b_t &= \varphi b_{t-1} + \beta\varepsilon_t/s_{t-m} \\ s_t &= s_{t-m} + \gamma\varepsilon_t/(l_{t-1} + \varphi b_{t-1}) \\ \hat{x}_{t+h\,|\,t} &= \left(l_t + (\varphi + \varphi^2 + \ldots + \varphi^h)b_t\right)s_{t+h-m_h^+} \end{aligned}$$

ETS(M,Ad,M)

Damped multiplicative Holt-Winters

$$x_t = (l_{t-1} + \varphi b_{t-1})s_{t-m}(1 + \varepsilon_t)$$

$$\begin{aligned} l_t &= (l_{t-1} + \varphi b_{t-1})(1 + \alpha\varepsilon_t) \\ b_t &= \varphi b_{t-1} + \beta(l_{t-1} + \varphi b_{t-1})\varepsilon_t \\ s_t &= s_{t-m}(1 + \gamma\varepsilon_t) \\ \hat{x}_{t+h\,|\,t} &= \left(l_t + (\varphi + \varphi^2 + \ldots + \varphi^h)b_t\right)s_{t+h-m_h^+} \end{aligned}$$

160.1.1.4. Trend = M (multiplicative trend)

ETS(A,M,N)

Multiplicative trend

$$x_t = l_{t-1}b_{t-1} + \varepsilon_t$$

$$\begin{aligned} l_t &= l_{t-1}b_{t-1} + \alpha\varepsilon_t \\ b_t &= b_{t-1} + \beta\varepsilon_t/l_{t-1} \\ \hat{x}_{t+h\,|\,t} &= l_t b_t^h \end{aligned}$$

ETS(M,M,N)

Mult. trend, mult. errors

$$x_t = l_{t-1}b_{t-1}(1 + \varepsilon_t)$$

$$\begin{aligned} l_t &= l_{t-1}b_{t-1}(1 + \alpha\varepsilon_t) \\ b_t &= b_{t-1}(1 + \beta\varepsilon_t) \\ \hat{x}_{t+h\,|\,t} &= l_t b_t^h \end{aligned}$$

ETS(A,M,A)

Mult. trend, additive seasonality

$$x_t = l_{t-1}b_{t-1} + s_{t-m} + \varepsilon_t$$

$$\begin{aligned} l_t &= l_{t-1}b_{t-1} + \alpha\varepsilon_t \\ b_t &= b_{t-1} + \beta\varepsilon_t/l_{t-1} \\ s_t &= s_{t-m} + \gamma\varepsilon_t \\ \hat{x}_{t+h\,|\,t} &= l_t b_t^h + s_{t+h-m_h^+} \end{aligned}$$

ETS(M,M,A)

Mult. trend + add. season., mult. errors

$$x_t = (l_{t-1}b_{t-1} + s_{t-m})(1 + \varepsilon_t)$$

$$\begin{aligned} l_t &= l_{t-1}b_{t-1} + \alpha\mu_t\varepsilon_t \\ b_t &= b_{t-1} + \beta\mu_t\varepsilon_t/l_{t-1} \\ s_t &= s_{t-m} + \gamma\mu_t\varepsilon_t \\ \hat{x}_{t+h\,|\,t} &= l_t b_t^h + s_{t+h-m_h^+} \\ \text{where}\quad \mu_t &= l_{t-1}b_{t-1} + s_{t-m} \end{aligned}$$

ETS(A,M,M)

Mult. trend, mult. seasonality

$$x_t = l_{t-1}b_{t-1}s_{t-m} + \varepsilon_t$$

$$\begin{aligned} l_t &= l_{t-1}b_{t-1} + \alpha\varepsilon_t/s_{t-m} \\ b_t &= b_{t-1} + \beta\varepsilon_t/(l_{t-1}s_{t-m}) \\ s_t &= s_{t-m} + \gamma\varepsilon_t/(l_{t-1}b_{t-1}) \\ \hat{x}_{t+h\,|\,t} &= l_t b_t^h s_{t+h-m_h^+} \end{aligned}$$

ETS(M,M,M)

Fully multiplicative

$$x_t = l_{t-1}b_{t-1}s_{t-m}(1 + \varepsilon_t)$$

$$\begin{aligned} l_t &= l_{t-1}b_{t-1}(1 + \alpha\varepsilon_t) \\ b_t &= b_{t-1}(1 + \beta\varepsilon_t) \\ s_t &= s_{t-m}(1 + \gamma\varepsilon_t) \\ \hat{x}_{t+h\,|\,t} &= l_t b_t^h s_{t+h-m_h^+} \end{aligned}$$

160.1.1.5. Trend = Md (damped multiplicative trend)

ETS(A,Md,N)

Damped multiplicative trend

$$x_t = l_{t-1}b_{t-1}^{\varphi} + \varepsilon_t$$

$$\begin{aligned} l_t &= l_{t-1}b_{t-1}^{\varphi} + \alpha\varepsilon_t \\ b_t &= b_{t-1}^{\varphi} + \beta\varepsilon_t / l_{t-1} \\ \hat{x}_{t+h\,|\,t} &= l_t b_t^{\varphi+\varphi^2+\ldots+\varphi^h} \end{aligned}$$

ETS(M,Md,N)

Damped mult. trend, mult. errors

$$x_t = l_{t-1}b_{t-1}^{\varphi}(1+\varepsilon_t)$$

$$\begin{aligned} l_t &= l_{t-1}b_{t-1}^{\varphi}(1+\alpha\varepsilon_t) \\ b_t &= b_{t-1}^{\varphi}(1+\beta\varepsilon_t) \\ \hat{x}_{t+h\,|\,t} &= l_t b_t^{\varphi+\varphi^2+\ldots+\varphi^h} \end{aligned}$$

ETS(A,Md,A)

Damped mult. trend, add. seasonality

$$x_t = l_{t-1}b_{t-1}^{\varphi} + s_{t-m} + \varepsilon_t$$

$$\begin{aligned} l_t &= l_{t-1}b_{t-1}^{\varphi} + \alpha\varepsilon_t \\ b_t &= b_{t-1}^{\varphi} + \beta\varepsilon_t / l_{t-1} \\ s_t &= s_{t-m} + \gamma\varepsilon_t \\ \hat{x}_{t+h\,|\,t} &= l_t b_t^{\varphi+\varphi^2+\ldots+\varphi^h} + s_{t+h-m_h^+} \end{aligned}$$

ETS(M,Md,A)

Damped mult. trend + add. season., mult. errors

$$x_t = (l_{t-1}b_{t-1}^{\varphi} + s_{t-m})(1+\varepsilon_t)$$

$$\begin{aligned} l_t &= l_{t-1}b_{t-1}^{\varphi} + \alpha\mu_t\varepsilon_t \\ b_t &= b_{t-1}^{\varphi} + \beta\mu_t\varepsilon_t / l_{t-1} \\ s_t &= s_{t-m} + \gamma\mu_t\varepsilon_t \\ \hat{x}_{t+h\,|\,t} &= l_t b_t^{\varphi+\varphi^2+\ldots+\varphi^h} + s_{t+h-m_h^+} \\ \text{where}\quad \mu_t &= l_{t-1}b_{t-1}^{\varphi} + s_{t-m} \end{aligned}$$

ETS(A,Md,M)

Damped mult. trend, mult. seasonality

$$x_t = l_{t-1}b_{t-1}^{\varphi}s_{t-m} + \varepsilon_t$$

$$\begin{aligned} l_t &= l_{t-1}b_{t-1}^{\varphi} + \alpha\varepsilon_t / s_{t-m} \\ b_t &= b_{t-1}^{\varphi} + \beta\varepsilon_t / (l_{t-1}s_{t-m}) \\ s_t &= s_{t-m} + \gamma\varepsilon_t / (l_{t-1}b_{t-1}^{\varphi}) \\ \hat{x}_{t+h\,|\,t} &= l_t b_t^{\varphi+\varphi^2+\ldots+\varphi^h} s_{t+h-m_h^+} \end{aligned}$$

ETS(M,Md,M)

Fully multiplicative with damped trend

$$x_t = l_{t-1}b_{t-1}^{\varphi}s_{t-m}(1+\varepsilon_t)$$

$$\begin{aligned} l_t &= l_{t-1}b_{t-1}^{\varphi}(1+\alpha\varepsilon_t) \\ b_t &= b_{t-1}^{\varphi}(1+\beta\varepsilon_t) \\ s_t &= s_{t-m}(1+\gamma\varepsilon_t) \\ \hat{x}_{t+h\,|\,t} &= l_t b_t^{\varphi+\varphi^2+\ldots+\varphi^h} s_{t+h-m_h^+} \end{aligned}$$

Model	Name
ETS(A,N,N)	Simple exponential smoothing
ETS(A,N,A)	SES with additive seasonality
ETS(A,N,M)	SES with multiplicative seasonality
ETS(A,A,N)	Holt's linear trend
ETS(A,A,A)	Additive Holt-Winters
ETS(A,A,M)	Holt-Winters with multiplicative seasonality
ETS(A,Ad,N)	Damped linear trend
ETS(A,Ad,A)	Damped additive Holt-Winters
ETS(A,Ad,M)	Damped Holt-Winters with multiplicative seasonality
ETS(A,M,N)	Multiplicative trend
ETS(A,M,A)	Multiplicative trend with additive seasonality
ETS(A,M,M)	Multiplicative trend and seasonality
ETS(A,Md,N)	Damped multiplicative trend
ETS(A,Md,A)	Damped multiplicative trend with additive seasonality

ETS(A,Md,M)	Damped multiplicative trend and seasonality
ETS(M,N,N)	SES with multiplicative errors
ETS(M,N,A)	SES with additive seasonality, multiplicative errors
ETS(M,N,M)	SES with multiplicative seasonality and errors
ETS(M,A,N)	Holt's linear trend with multiplicative errors
ETS(M,A,A)	Additive Holt-Winters with multiplicative errors
ETS(M,A,M)	Multiplicative Holt-Winters
ETS(M,Ad,N)	Damped linear trend with multiplicative errors
ETS(M,Ad,A)	Damped additive Holt-Winters with multiplicative errors
ETS(M,Ad,M)	Damped multiplicative Holt-Winters
ETS(M,M,N)	Multiplicative trend with multiplicative errors
ETS(M,M,A)	Multiplicative trend, additive seasonality, multiplicative errors
ETS(M,M,M)	Fully multiplicative
ETS(M,Md,N)	Damped multiplicative trend with multiplicative errors
ETS(M,Md,A)	Damped multiplicative trend, additive seasonality, mult. errors
ETS(M,Md,M)	Fully multiplicative with damped trend

160.1.1.6. ETS (Exponential Smoothing)

ETS stands for **Error, Trend, Seasonality** — a family of forecasting models that decomposes a time series into up to three components and updates each one recursively from new observations.

160.1.1.6.1. The weighting-scheme axis

Before exponential smoothing, ask: **how much should past observations count?** Two extremes and two interpolators:

Naïve	Moving average	SES	Cumulative
Only the last point matters	*Last M points, equal weight*	*All history, geometric decay*	*All history, equal weight*
$\hat{x}_{t,t+1} = x_t$	$\hat{x}_{t,t+1} = \frac{1}{M} \sum_{i=t-M+1}^{t} x_i$	$\hat{x}_{t,t+1} = \alpha \sum_{i=0}^{t} (1-\alpha)^i x_{t-i}$	$\hat{x}_{t,t+1} = \frac{1}{t} \sum_{i=1}^{t} x_i$
Weights: $(1, 0, 0, ..., 0)$	Weights: $(\frac{1}{M}, ..., \frac{1}{M}, 0, ..., 0)$	Weights: $\alpha, \alpha(1-\alpha), \alpha(1-\alpha)^2, ...$	Weights: $(\frac{1}{t}, \frac{1}{t}, ..., \frac{1}{t})$

Naïve ($M = 1$) and Cumulative ($M = t$) are the two extremes of a single "how much history counts" axis. MA sits between them with a finite equal-weight window; SES sits between them with infinite geometrically-decaying weights. **The rest of this section builds on SES.**

160.2. A-N-N

Simple exponential smoothing

$$\mathrm{ETS}(A, N, N)$$

$$x_t = l_{t-1} + \varepsilon_t$$

$$l_t = l_{t-1} + \alpha\varepsilon_t$$

$$\hat{x}_{t+h \mid t} = l_t$$

Example: $\mathrm{ETS}(A, N, N)$

Given

- Smoothing parameter: $\alpha = 0.5$
- Initial level: $l_0 = 12$
- Data:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
x_t	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

Step 1 — formula

Observation:

$$x_t = l_{t-1} + \varepsilon_t$$

Innovation (rearrange):

$$\varepsilon_t = x_t - l_{t-1}$$

State update:

$$l_t = l_{t-1} + \alpha\varepsilon_t$$

One-step-ahead forecast:

$$\hat{x}_{t \mid t-1} = l_{t-1}$$

Step 2 — apply at $t = 1$

Use $l_0 = 12$ and $x_1 = 12$:

$$\hat{x}_{1 \mid 0} = l_0 = 12$$

$$\varepsilon_1 = x_1 - l_0 = 12 - 12 = 0$$

$$l_1 = l_0 + \alpha\varepsilon_1 = 12 + 0.5 \cdot 0 = 12$$

Step 3 — iterate

Values rounded to 4 decimal places; arithmetic performed at full precision.

t	x_t	$\hat{x}_{t \mid t-1} = l_{t-1}$	$\varepsilon_t = x_t - l_{t-1}$	$l_t = l_{t-1} + 0.5\varepsilon_t$
1	12	12	0	12
2	10	12	−2	11
3	8	11	−3	9.5
4	11	9.5	1.5	10.25

5	14	10.25	3.75	12.125
6	12	12.125	-0.125	12.0625
7	9	12.0625	-3.0625	10.5313
8	13	10.5313	2.4688	11.7656
9	16	11.7656	4.2344	13.8828
10	14	13.8828	0.1172	13.9414
11	11	13.9414	-2.9414	12.4707
12	15	12.4707	2.5293	13.7354
13	18	13.7354	4.2646	15.8677
14	16	15.8677	0.1323	15.9338
15	13	15.9338	-2.9338	14.4669
16	17	14.4669	2.5331	15.7335

Forecast for any future $t > 16$: $\hat{x}_{t \mid 16} = l_{16} = 15.7335$ (flat — SES has no trend).

160.3. A-N-A

SES + additive seasonality

$$\mathrm{ETS}(A, N, A)$$

$$x_t = l_{t-1} + s_{t-m} + \varepsilon_t$$

$$l_t = l_{t-1} + \alpha\varepsilon_t$$

$$s_t = s_{t-m} + \gamma\varepsilon_t$$

$$\hat{x}_{t+h \mid t} = l_t + s_{t+h-m_h^+}$$

Example: $\mathrm{ETS}(A, N, A)$

Given

- Smoothing parameters: $\alpha = 0.5$, $\gamma = 0.2$
- Initial states: $l_0 = 12$, $(s_{-3}, s_{-2}, s_{-1}, s_0) = (2, 0, -3, 1)$, seasonal period $m = 4$
- Data:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
x_t	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

Step 1 — formula

Observation:

$$x_t = l_{t-1} + s_{t-m} + \varepsilon_t$$

Conditional mean (one-step-ahead forecast $\hat{x}_{t \mid t-1} = \mu_t$):

$$\mu_t = l_{t-1} + s_{t-m}$$

Innovation:

$$\varepsilon_t = x_t - \mu_t$$

State updates:

$$l_t = l_{t-1} + \alpha\varepsilon_t$$

$$s_t = s_{t-m} + \gamma\varepsilon_t$$

Forecast h steps ahead from time t (using current-period states):

$$\hat{x}_{t+h \mid t} = l_t + s_{t+h-m_h^+}$$

where $m_h^+ = ((h-1) \bmod m) + 1$ picks the right seasonal slot for the period h steps ahead (cycles through $1, 2, \ldots, m$).

Step 2 — apply at $t = 1$

$$\mu_1 = 12 + 2 = 14$$

$$\varepsilon_1 = x_1 - \mu_1 = 12 - 14 = -2$$

$$l_1 = 12 + 0.5 \cdot (-2) = 11$$

$$s_1 = 2 + 0.2 \cdot (-2) = 1.6$$

Step 3 — iterate

Each column header is the equation that produced its values. Values rounded to 4 decimal places; arithmetic performed at full precision.

t	x_t	$\mu_t = l_{t-1} + s_{t-m}$	ε_t	$l_t = l_{t-1} + \alpha\varepsilon_t$	$s_t = s_{t-m} + \gamma\varepsilon_t$
1	12	14	−2	11	1.6
2	10	11	−1	10.5	−0.2
3	8	7.5	0.5	10.75	−2.9
4	11	11.75	−0.75	10.375	0.85
5	14	11.975	2.025	11.3875	2.005
6	12	11.1875	0.8125	11.7937	−0.0375
7	9	8.8937	0.1063	11.8469	−2.8787
8	13	12.6969	0.3031	11.9984	0.9106
9	16	14.0034	1.9966	12.9967	2.4043
10	14	12.9592	1.0408	13.5171	0.1707
11	11	10.6384	0.3616	13.6979	−2.8064
12	15	14.6086	0.3914	13.8937	0.9889
13	18	16.298	1.702	14.7447	2.7447
14	16	14.9153	1.0847	15.287	0.3876

15	13	12.4806	0.5194	15.5467	−2.7025
16	17	16.5356	0.4644	15.7789	1.0818

160.4. A-N-M

SES + multiplicative seasonality

$$\mathrm{ETS}(A, N, M)$$

$$x_t = l_{t-1} s_{t-m} + \varepsilon_t$$

$$l_t = l_{t-1} + \alpha \varepsilon_t / s_{t-m}$$

$$s_t = s_{t-m} + \gamma \varepsilon_t / l_{t-1}$$

$$\hat{x}_{t+h \mid t} = l_t s_{t+h-m_h^+}$$

Example: $\mathrm{ETS}(A, N, M)$

Given

- Smoothing parameters: $\alpha = 0.5$, $\gamma = 0.2$
- Initial states: $l_0 = 12$, $(s_{-3}, s_{-2}, s_{-1}, s_0) = (1.2, 1, 0.8, 1)$, seasonal period $m = 4$
- Data:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
x_t	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

Step 1 — formula

Observation:

$$x_t = l_{t-1} s_{t-m} + \varepsilon_t$$

Conditional mean (one-step-ahead forecast $\hat{x}_{t \mid t-1} = \mu_t$):

$$\mu_t = l_{t-1} s_{t-m}$$

Innovation:

$$\varepsilon_t = x_t - \mu_t$$

State updates:

$$l_t = l_{t-1} + \alpha \varepsilon_t / s_{t-m}$$

$$s_t = s_{t-m} + \gamma \varepsilon_t / l_{t-1}$$

Forecast h steps ahead from time t (using current-period states):

$$\hat{x}_{t+h \mid t} = l_t s_{t+h-m_h^+}$$

where $m_h^+ = ((h-1) \bmod m) + 1$ picks the right seasonal slot for the period h steps ahead (cycles through $1, 2, \ldots, m$).

Step 2 — apply at $t = 1$

$$\mu_1 = 12 \cdot 1.2 = 14.4$$

$$\varepsilon_1 = x_1 - \mu_1 = 12 - 14.4 = -2.4$$

$$l_1 = 12 + 0.5 \cdot (-2.4)/1.2 = 11$$

$$s_1 = 1.2 + 0.2 \cdot (-2.4)/12 = 1.16$$

Step 3 — iterate

Each column header is the equation that produced its values. Values rounded to 4 decimal places; arithmetic performed at full precision.

t	x_t	$\mu_t = l_{t-1}s_{t-m}$	ε_t	$l_t = l_{t-1} + \alpha\varepsilon_t/s_{t-m}$	$s_t = s_{t-m} + \gamma\varepsilon_t/l_{t-1}$
1	12	14.4	−2.4	11	1.16
2	10	11	−1	10.5	0.9818
3	8	8.4	−0.4	10.25	0.7924
4	11	10.25	0.75	10.625	1.0146
5	14	12.325	1.675	11.347	1.1915
6	12	11.1407	0.8593	11.7846	0.997
7	9	9.3379	−0.3379	11.5714	0.7866
8	13	11.7407	1.2593	12.1919	1.0364
9	16	14.5271	1.4729	12.81	1.2157
10	14	12.7711	1.2289	13.4263	1.0162
11	11	10.5618	0.4382	13.7049	0.7932
12	15	14.2037	0.7963	14.089	1.048
13	18	17.1279	0.8721	14.4477	1.2281
14	16	14.681	1.319	15.0967	1.0344
15	13	11.9743	1.0257	15.7433	0.8068
16	17	16.4993	0.5007	15.9822	1.0544

160.5. A-A-N

Holt's linear trend

$$\mathrm{ETS}(A, A, N)$$

$$x_t = l_{t-1} + b_{t-1} + \varepsilon_t$$

$$l_t = l_{t-1} + b_{t-1} + \alpha\varepsilon_t$$

$$b_t = b_{t-1} + \beta\varepsilon_t$$

$$\hat{x}_{t+h \mid t} = l_t + hb_t$$

Example: $\mathrm{ETS}(A, A, N)$

Given

- Smoothing parameters: $\alpha = 0.5$, $\beta = 0.4$
- Initial states: $l_0 = 12$, $b_0 = 0.5$
- Data:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
x_t	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

Step 1 — formula

Observation:

$$x_t = l_{t-1} + b_{t-1} + \varepsilon_t$$

Conditional mean (one-step-ahead forecast $\hat{x}_{t \mid t-1} = \mu_t$):

$$\mu_t = l_{t-1} + b_{t-1}$$

Innovation:

$$\varepsilon_t = x_t - \mu_t$$

State updates:

$$l_t = l_{t-1} + b_{t-1} + \alpha\varepsilon_t$$

$$b_t = b_{t-1} + \beta\varepsilon_t$$

Forecast h steps ahead from time t (using current-period states):

$$\hat{x}_{t+h \mid t} = l_t + hb_t$$

where $h \in \{1, 2, 3, \ldots\}$ is the forecast horizon (how many steps ahead).

Step 2 — apply at $t = 1$

$$\mu_1 = 12 + 0.5 = 12.5$$

$$\varepsilon_1 = x_1 - \mu_1 = 12 - 12.5 = -0.5$$

$$l_1 = 12 + 0.5 + 0.5 \cdot (-0.5) = 12.25$$

$$b_1 = 0.5 + 0.4 \cdot (-0.5) = 0.3$$

Step 3 — iterate

Each column header is the equation that produced its values. Values rounded to 4 decimal places; arithmetic performed at full precision.

t	x_t	$\mu_t = l_{t-1} + b_{t-1}$	ε_t	$l_t = l_{t-1} + b_{t-1} + \alpha\varepsilon_t$	$b_t = b_{t-1} + \beta\varepsilon_t$
1	12	12.5	-0.5	12.25	0.3
2	10	12.55	-2.55	11.275	-0.72
3	8	10.555	-2.555	9.2775	-1.742

4	11	7.5355	3.4645	9.2677	−0.3562
5	14	8.9116	5.0884	11.4558	1.6792
6	12	13.135	−1.135	12.5675	1.2252
7	9	13.7927	−4.7927	11.3963	−0.6919
8	13	10.7045	2.2955	11.8522	0.2263
9	16	12.0786	3.9214	14.0393	1.7949
10	14	15.8342	−1.8342	14.9171	1.0612
11	11	15.9783	−4.9783	13.4892	−0.9301
12	15	12.5591	2.4409	13.7795	0.0463
13	18	13.8258	4.1742	15.9129	1.716
14	16	17.6289	−1.6289	16.8144	1.0644
15	13	17.8788	−4.8788	15.4394	−0.8871
16	17	14.5523	2.4477	15.7761	0.092

160.6. A-A-A

Additive Holt-Winters

$$\mathrm{ETS}(A, A, A)$$

$$x_t = l_{t-1} + b_{t-1} + s_{t-m} + \varepsilon_t$$

$$l_t = l_{t-1} + b_{t-1} + \alpha\varepsilon_t$$

$$b_t = b_{t-1} + \beta\varepsilon_t$$

$$s_t = s_{t-m} + \gamma\varepsilon_t$$

$$\hat{x}_{t+h \mid t} = l_t + hb_t + s_{t+h-m_h^+}$$

Example: $\mathrm{ETS}(A, A, A)$

Given

- Smoothing parameters: $\alpha = 0.5$, $\beta = 0.4$, $\gamma = 0.2$
- Initial states: $l_0 = 12$, $b_0 = 0.5$, $(s_{-3}, s_{-2}, s_{-1}, s_0) = (2, 0, -3, 1)$, seasonal period $m = 4$
- Data:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
x_t	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

Step 1 — formula

Observation:

$$x_t = l_{t-1} + b_{t-1} + s_{t-m} + \varepsilon_t$$

Conditional mean (one-step-ahead forecast $\hat{x}_{t \mid t-1} = \mu_t$):

$$\mu_t = l_{t-1} + b_{t-1} + s_{t-m}$$

Innovation:

$$\varepsilon_t = x_t - \mu_t$$

State updates:

$$l_t = l_{t-1} + b_{t-1} + \alpha\varepsilon_t$$

$$b_t = b_{t-1} + \beta\varepsilon_t$$

$$s_t = s_{t-m} + \gamma\varepsilon_t$$

Forecast h steps ahead from time t (using current-period states):

$$\hat{x}_{t+h \mid t} = l_t + hb_t + s_{t+h-m_h^+}$$

where $h \in \{1, 2, 3, ...\}$ is the forecast horizon (how many steps ahead); $m_h^+ = ((h-1) \bmod m) + 1$ picks the right seasonal slot for the period h steps ahead (cycles through $1, 2, ..., m$).

Step 2 — apply at $t = 1$

$$\mu_1 = 12 + 0.5 + 2 = 14.5$$

$$\varepsilon_1 = x_1 - \mu_1 = 12 - 14.5 = -2.5$$

$$l_1 = 12 + 0.5 + 0.5 \cdot (-2.5) = 11.25$$

$$b_1 = 0.5 + 0.4 \cdot (-2.5) = -0.5$$

$$s_1 = 2 + 0.2 \cdot (-2.5) = 1.5$$

Step 3 — iterate

Each column header is the equation that produced its values. Values rounded to 4 decimal places; arithmetic performed at full precision.

t	x_t	$\mu_t = l_{t-1} + b_{t-1} + s_{t-m}$	ε_t	$l_t = l_{t-1} + b_{t-1} + \alpha\varepsilon_t$	$b_t = b_{t-1} + \beta\varepsilon_t$	$s_t = s_{t-m} + \gamma\varepsilon_t$
1	12	14.5	−2.5	11.25	−0.5	1.5
2	10	10.75	−0.75	10.375	−0.8	−0.15
3	8	6.575	1.425	10.2875	−0.23	−2.715
4	11	11.0575	−0.0575	10.0287	−0.253	0.9885
5	14	11.2757	2.7243	11.1379	0.8367	2.0449
6	12	11.8246	0.1754	12.0623	0.9069	−0.1149
7	9	10.2542	−1.2542	12.3421	0.4052	−2.9658
8	13	13.7358	−0.7358	12.3794	0.1109	0.8413
9	16	14.5351	1.4649	13.2227	0.6968	2.3378
10	14	13.8046	0.1954	14.0172	0.775	−0.0758

11	11	11.8264	−0.8264	14.379	0.4444	−3.1311
12	15	15.6648	−0.6648	14.4911	0.1785	0.7084
13	18	17.0074	0.9926	15.1659	0.5756	2.5363
14	16	15.6656	0.3344	15.9086	0.7093	−0.009
15	13	13.4868	−0.4868	16.3745	0.5146	−3.2285
16	17	17.5975	−0.5975	16.5904	0.2756	0.5889

160.7. A-A-M

HW with mult. seasonality

$$\mathrm{ETS}(A, A, M)$$

$$x_t = (l_{t-1} + b_{t-1})s_{t-m} + \varepsilon_t$$

$$l_t = l_{t-1} + b_{t-1} + \alpha\varepsilon_t / s_{t-m}$$

$$b_t = b_{t-1} + \beta\varepsilon_t / s_{t-m}$$

$$s_t = s_{t-m} + \gamma\varepsilon_t / (l_{t-1} + b_{t-1})$$

$$\hat{x}_{t+h \mid t} = (l_t + hb_t)s_{t+h-m_h^+}$$

Example: $\mathrm{ETS}(A, A, M)$

Given

- Smoothing parameters: $\alpha = 0.5$, $\beta = 0.4$, $\gamma = 0.2$
- Initial states: $l_0 = 12$, $b_0 = 0.5$, $(s_{-3}, s_{-2}, s_{-1}, s_0) = (1.2, 1, 0.8, 1)$, seasonal period $m = 4$
- Data:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
x_t	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

Step 1 — formula

Observation:

$$x_t = (l_{t-1} + b_{t-1})s_{t-m} + \varepsilon_t$$

Conditional mean (one-step-ahead forecast $\hat{x}_{t \mid t-1} = \mu_t$):

$$\mu_t = (l_{t-1} + b_{t-1})s_{t-m}$$

Innovation:

$$\varepsilon_t = x_t - \mu_t$$

State updates:

$$l_t = l_{t-1} + b_{t-1} + \alpha\varepsilon_t / s_{t-m}$$

$$b_t = b_{t-1} + \beta\varepsilon_t / s_{t-m}$$

$$s_t = s_{t-m} + \gamma\varepsilon_t / (l_{t-1} + b_{t-1})$$

Forecast h steps ahead from time t (using current-period states):

$$\hat{x}_{t+h \mid t} = (l_t + hb_t)s_{t+h-m_h^+}$$

where $h \in \{1, 2, 3, ...\}$ is the forecast horizon (how many steps ahead); $m_h^+ = ((h-1) \bmod m) + 1$ picks the right seasonal slot for the period h steps ahead (cycles through $1, 2, ..., m$).

Step 2 — apply at $t = 1$

$$\mu_1 = (12 + 0.5) \cdot 1.2 = 15$$

$$\varepsilon_1 = x_1 - \mu_1 = 12 - 15 = -3$$

$$l_1 = 12 + 0.5 + 0.5 \cdot (-3)/1.2 = 11.25$$

$$b_1 = 0.5 + 0.4 \cdot (-3)/1.2 = -0.5$$

$$s_1 = 1.2 + 0.2 \cdot (-3)/(12 + 0.5) = 1.152$$

Step 3 — iterate

Each column header is the equation that produced its values. Values rounded to 4 decimal places; arithmetic performed at full precision.

t	x_t	$\mu_t = (l_{t-1} + b_{t-1})s_{t-m}$	ε_t	$l_t = l_{t-1} + b_{t-1} + \alpha\varepsilon_t / s_{t-m}$	$b_t = b_{t-1} + \beta\varepsilon_t / s_{t-m}$	$s_t = s_{t-m} + \gamma\varepsilon_t / (l_{t-1} + b_{t-1})$
1	12	15	−3	11.25	−0.5	1.152
2	10	10.75	−0.75	10.375	−0.8	0.986
3	8	7.66	0.34	9.7875	−0.63	0.8071
4	11	9.1575	1.8425	10.0787	0.107	1.0402
5	14	11.734	2.266	11.1693	0.8938	1.1965
6	12	11.8948	0.1052	12.1164	0.9365	0.9878
7	9	10.5351	−1.5351	12.102	0.1757	0.7836
8	13	12.7718	0.2282	12.3874	0.2635	1.044
9	16	15.1367	0.8633	13.0117	0.5521	1.2101
10	14	13.3982	0.6018	13.8684	0.7958	0.9967
11	11	11.4906	−0.4906	14.3512	0.5454	0.7769
12	15	15.5513	−0.5513	14.6325	0.3341	1.0366
13	18	18.1117	−0.1117	14.9204	0.2972	1.2086
14	16	15.1669	0.8331	15.6356	0.6316	1.0076
15	13	12.6378	0.3622	16.5003	0.8181	0.7813

16	17	17.9514	−0.9514	16.8594	0.4509	1.0256

160.8. A-Ad-N

Damped linear trend

$$\mathrm{ETS}(A, \mathrm{Ad}, N)$$

$$x_t = l_{t-1} + \varphi b_{t-1} + \varepsilon_t$$

$$l_t = l_{t-1} + \varphi b_{t-1} + \alpha \varepsilon_t$$

$$b_t = \varphi b_{t-1} + \beta \varepsilon_t$$

$$\hat{x}_{t+h \mid t} = l_t + \left(\varphi + \varphi^2 + \ldots + \varphi^h\right) b_t$$

Example: $\mathrm{ETS}(A, \mathrm{Ad}, N)$

Given

- Smoothing parameters: $\alpha = 0.5$, $\beta = 0.4$, $\varphi = 0.8$
- Initial states: $l_0 = 12$, $b_0 = 0.5$
- Data:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
x_t	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

Step 1 — formula

Observation:

$$x_t = l_{t-1} + \varphi b_{t-1} + \varepsilon_t$$

Conditional mean (one-step-ahead forecast $\hat{x}_{t \mid t-1} = \mu_t$):

$$\mu_t = l_{t-1} + \varphi b_{t-1}$$

Innovation:

$$\varepsilon_t = x_t - \mu_t$$

State updates:

$$l_t = l_{t-1} + \varphi b_{t-1} + \alpha \varepsilon_t$$

$$b_t = \varphi b_{t-1} + \beta \varepsilon_t$$

Forecast h steps ahead from time t (using current-period states):

$$\hat{x}_{t+h \mid t} = l_t + \left(\varphi + \varphi^2 + \ldots + \varphi^h\right) b_t$$

where $h \in \{1, 2, 3, \ldots\}$ is the forecast horizon (how many steps ahead).

Step 2 — apply at $t = 1$

$$\mu_1 = 12 + 0.8 \cdot 0.5 = 12.4$$

$$\varepsilon_1 = x_1 - \mu_1 = 12 - 12.4 = -0.4$$

$$l_1 = 12 + 0.8 \cdot 0.5 + 0.5 \cdot (-0.4) = 12.2$$

$$b_1 = 0.8 \cdot 0.5 + 0.4 \cdot (-0.4) = 0.24$$

Step 3 — iterate

Each column header is the equation that produced its values. Values rounded to 4 decimal places; arithmetic performed at full precision.

t	x_t	$\mu_t = l_{t-1} + \varphi b_{t-1}$	ε_t	$l_t = l_{t-1} + \varphi b_{t-1} + \alpha\varepsilon_t$	$b_t = \varphi b_{t-1} + \beta\varepsilon_t$
1	12	12.4	−0.4	12.2	0.24
2	10	12.392	−2.392	11.196	−0.7648
3	8	10.5842	−2.5842	9.2921	−1.6455
4	11	7.9757	3.0243	9.4878	−0.1067
5	14	9.4025	4.5975	11.7012	1.7537
6	12	13.1042	−1.1042	12.5521	0.9613
7	9	13.3211	−4.3211	11.1605	−0.9594
8	13	10.393	2.607	11.6965	0.2753
9	16	11.9167	4.0833	13.9584	1.8535
10	14	15.4412	−1.4412	14.7206	0.9063
11	11	15.4457	−4.4457	13.2228	−1.0532
12	15	12.3803	2.6197	13.6901	0.2053
13	18	13.8544	4.1456	15.9272	1.8225
14	16	17.3852	−1.3852	16.6926	0.9039
15	13	17.4157	−4.4157	15.2079	−1.0432
16	17	14.3733	2.6267	15.6867	0.2161

160.9. A-Ad-A

Damped additive Holt-Winters

$$\mathrm{ETS}(A, \mathrm{Ad}, A)$$

$$x_t = l_{t-1} + \varphi b_{t-1} + s_{t-m} + \varepsilon_t$$

$$l_t = l_{t-1} + \varphi b_{t-1} + \alpha\varepsilon_t$$

$$b_t = \varphi b_{t-1} + \beta\varepsilon_t$$

$$s_t = s_{t-m} + \gamma\varepsilon_t$$

$$\hat{x}_{t+h \mid t} = l_t + \left(\varphi + \varphi^2 + \ldots + \varphi^h\right) b_t + s_{t+h-m_h^+}$$

Example: $\mathrm{ETS}(A, \mathrm{Ad}, A)$

Given

- Smoothing parameters: $\alpha = 0.5$, $\beta = 0.4$, $\varphi = 0.8$, $\gamma = 0.2$
- Initial states: $l_0 = 12$, $b_0 = 0.5$, $(s_{-3}, s_{-2}, s_{-1}, s_0) = (2, 0, -3, 1)$, seasonal period $m = 4$
- Data:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
x_t	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

Step 1 — formula

Observation:

$$x_t = l_{t-1} + \varphi b_{t-1} + s_{t-m} + \varepsilon_t$$

Conditional mean (one-step-ahead forecast $\hat{x}_{t \mid t-1} = \mu_t$):

$$\mu_t = l_{t-1} + \varphi b_{t-1} + s_{t-m}$$

Innovation:

$$\varepsilon_t = x_t - \mu_t$$

State updates:

$$l_t = l_{t-1} + \varphi b_{t-1} + \alpha \varepsilon_t$$

$$b_t = \varphi b_{t-1} + \beta \varepsilon_t$$

$$s_t = s_{t-m} + \gamma \varepsilon_t$$

Forecast h steps ahead from time t (using current-period states):

$$\hat{x}_{t+h \mid t} = l_t + (\varphi + \varphi^2 + \ldots + \varphi^h) b_t + s_{t+h-m_h^+}$$

where $h \in \{1, 2, 3, \ldots\}$ is the forecast horizon (how many steps ahead); $m_h^+ = ((h-1) \bmod m) + 1$ picks the right seasonal slot for the period h steps ahead (cycles through $1, 2, \ldots, m$).

Step 2 — apply at $t = 1$

$$\mu_1 = 12 + 0.8 \cdot 0.5 + 2 = 14.4$$

$$\varepsilon_1 = x_1 - \mu_1 = 12 - 14.4 = -2.4$$

$$l_1 = 12 + 0.8 \cdot 0.5 + 0.5 \cdot (-2.4) = 11.2$$

$$b_1 = 0.8 \cdot 0.5 + 0.4 \cdot (-2.4) = -0.56$$

$$s_1 = 2 + 0.2 \cdot (-2.4) = 1.52$$

Step 3 — iterate

Each column header is the equation that produced its values. Values rounded to 4 decimal places; arithmetic performed at full precision.

t	x_t	$\mu_t = l_{t-1} + \varphi b_{t-1} + s_{t-m}$	ε_t	$l_t = l_{t-1} + \varphi b_{t-1} + \alpha\varepsilon_t$	$b_t = \varphi b_{t-1} + \beta\varepsilon_t$	$s_t = s_{t-m} + \gamma\varepsilon_t$
1	12	14.4	−2.4	11.2	−0.56	1.52
2	10	10.752	−0.752	10.376	−0.7488	−0.1504
3	8	6.777	1.223	10.3885	−0.1098	−2.7554
4	11	11.3006	−0.3006	10.1503	−0.2081	0.9399
5	14	11.5038	2.4962	11.2319	0.832	2.0192
6	12	11.7471	0.2529	12.0239	0.7667	−0.0998
7	9	9.882	−0.882	12.1964	0.2606	−2.9318
8	13	13.3447	−0.3447	12.2325	0.0706	0.8709
9	16	14.3082	1.6918	13.1349	0.7332	2.3576
10	14	13.6216	0.3784	13.9106	0.7379	−0.0241
11	11	11.5692	−0.5692	14.2164	0.3627	−3.0456
12	15	15.3774	−0.3774	14.3178	0.1392	0.7954
13	18	16.7867	1.2133	15.0358	0.5966	2.6003
14	16	15.4889	0.5111	15.7686	0.6817	0.0781
15	13	13.2684	−0.2684	16.1798	0.438	−3.0993
16	17	17.3257	−0.3257	16.3674	0.2202	0.7303

160.10. A-Ad-M

Damped Holt-Winters, multiplicative seasonality

$$\mathrm{ETS}(A, \mathrm{Ad}, M)$$

$$x_t = (l_{t-1} + \varphi b_{t-1}) s_{t-m} + \varepsilon_t$$

$$l_t = l_{t-1} + \varphi b_{t-1} + \alpha\varepsilon_t / s_{t-m}$$

$$b_t = \varphi b_{t-1} + \beta\varepsilon_t / s_{t-m}$$

$$s_t = s_{t-m} + \gamma\varepsilon_t / (l_{t-1} + \varphi b_{t-1})$$

$$\hat{x}_{t+h \mid t} = \left(l_t + \left(\varphi + \varphi^2 + \ldots + \varphi^h\right) b_t\right) s_{t+h-m_h^+}$$

Example: $\mathrm{ETS}(A, \mathrm{Ad}, M)$

Given

- Smoothing parameters: $\alpha = 0.5$, $\beta = 0.4$, $\varphi = 0.8$, $\gamma = 0.2$
- Initial states: $l_0 = 12$, $b_0 = 0.5$, $(s_{-3}, s_{-2}, s_{-1}, s_0) = (1.2, 1, 0.8, 1)$, seasonal period $m = 4$
- Data:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
x_t	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

Step 1 — formula

Observation:

$$x_t = (l_{t-1} + \varphi b_{t-1}) s_{t-m} + \varepsilon_t$$

Conditional mean (one-step-ahead forecast $\hat{x}_{t \mid t-1} = \mu_t$):

$$\mu_t = (l_{t-1} + \varphi b_{t-1}) s_{t-m}$$

Innovation:

$$\varepsilon_t = x_t - \mu_t$$

State updates:

$$l_t = l_{t-1} + \varphi b_{t-1} + \alpha \varepsilon_t / s_{t-m}$$

$$b_t = \varphi b_{t-1} + \beta \varepsilon_t / s_{t-m}$$

$$s_t = s_{t-m} + \gamma \varepsilon_t / (l_{t-1} + \varphi b_{t-1})$$

Forecast h steps ahead from time t (using current-period states):

$$\hat{x}_{t+h \mid t} = \left(l_t + \left(\varphi + \varphi^2 + \ldots + \varphi^h\right) b_t\right) s_{t+h-m_h^+}$$

where $h \in \{1, 2, 3, \ldots\}$ is the forecast horizon (how many steps ahead); $m_h^+ = ((h-1) \bmod m) + 1$ picks the right seasonal slot for the period h steps ahead (cycles through $1, 2, \ldots, m$).

Step 2 — apply at $t = 1$

$$\mu_1 = (12 + 0.8 \cdot 0.5) \cdot 1.2 = 14.88$$

$$\varepsilon_1 = x_1 - \mu_1 = 12 - 14.88 = -2.88$$

$$l_1 = 12 + 0.8 \cdot 0.5 + 0.5 \cdot (-2.88)/1.2 = 11.2$$

$$b_1 = 0.8 \cdot 0.5 + 0.4 \cdot (-2.88)/1.2 = -0.56$$

$$s_1 = 1.2 + 0.2 \cdot (-2.88)/(12 + 0.8 \cdot 0.5) = 1.1535$$

Step 3 — iterate

Each column header is the equation that produced its values. Values rounded to 4 decimal places; arithmetic performed at full precision.

t	x_t	$\mu_t = (l_{t-1} + \varphi b_{t-1}) s_{t-m}$	ε_t	$l_t = l_{t-1} + \varphi b_{t-1} + \alpha \varepsilon_t / s_{t-m}$	$b_t = \varphi b_{t-1} + \beta \varepsilon_t / s_{t-m}$	$s_t = s_{t-m} + \gamma \varepsilon_t / (l_{t-1} + \varphi b_{t-1})$
1	12	14.88	−2.88	11.2	−0.56	1.1535
2	10	10.752	−0.752	10.376	−0.7488	0.986

3	8	7.8216	0.1784	9.8885	−0.5098	0.8037
4	11	9.4806	1.5194	10.2403	0.1999	1.0321
5	14	11.9972	2.0028	11.2683	0.8544	1.1921
6	12	11.7847	0.2153	12.0611	0.7709	0.9896
7	9	10.1885	−1.1885	11.9383	0.0252	0.7849
8	13	12.3418	0.6582	12.2774	0.2752	1.0431
9	16	14.8979	1.1021	12.9598	0.59	1.2097
10	14	13.2924	0.7076	13.7894	0.758	1.0002
11	11	11.2993	−0.2993	14.2052	0.4539	0.7807
12	15	15.1956	−0.1956	14.4745	0.2881	1.0404
13	18	17.7887	0.2113	14.7924	0.3004	1.2126
14	16	15.0349	0.9651	15.5151	0.6263	1.013
15	13	12.5045	0.4955	16.3335	0.7549	0.7869
16	17	17.6212	−0.6212	16.6388	0.3651	1.033

160.11. A-M-N

Multiplicative trend

$$\mathrm{ETS}(A, M, N)$$

$$x_t = l_{t-1} b_{t-1} + \varepsilon_t$$

$$l_t = l_{t-1} b_{t-1} + \alpha \varepsilon_t$$

$$b_t = b_{t-1} + \beta \varepsilon_t / l_{t-1}$$

$$\hat{x}_{t+h \mid t} = l_t b_t^h$$

Example: $\mathrm{ETS}(A, M, N)$

Given

- Smoothing parameters: $\alpha = 0.5$, $\beta = 0.4$
- Initial states: $l_0 = 12$, $b_0 = 1$
- Data:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
x_t	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

Step 1 — formula

Observation:

$$x_t = l_{t-1} b_{t-1} + \varepsilon_t$$

Conditional mean (one-step-ahead forecast $\hat{x}_{t \mid t-1} = \mu_t$):

$$\mu_t = l_{t-1} b_{t-1}$$

Innovation:

$$\varepsilon_t = x_t - \mu_t$$

State updates:

$$l_t = l_{t-1} b_{t-1} + \alpha \varepsilon_t$$

$$b_t = b_{t-1} + \beta \varepsilon_t / l_{t-1}$$

Forecast h steps ahead from time t (using current-period states):

$$\hat{x}_{t+h \mid t} = l_t b_t^h$$

where $h \in \{1, 2, 3, \ldots\}$ is the forecast horizon (how many steps ahead).

Step 2 — apply at $t = 1$

$$\mu_1 = 12 \cdot 1 = 12$$

$$\varepsilon_1 = x_1 - \mu_1 = 12 - 12 = 0$$

$$l_1 = 12 \cdot 1 + 0.5 \cdot 0 = 12$$

$$b_1 = 1 + 0.4 \cdot 0/12 = 1$$

Step 3 — iterate

Each column header is the equation that produced its values. Values rounded to 4 decimal places; arithmetic performed at full precision.

t	x_t	$\mu_t = l_{t-1} b_{t-1}$	ε_t	$l_t = l_{t-1} b_{t-1} + \alpha \varepsilon_t$	$b_t = b_{t-1} + \beta \varepsilon_t / l_{t-1}$
1	12	12	0	12	1
2	10	12	−2	11	0.9333
3	8	10.2667	−2.2667	9.1333	0.8509
4	11	7.7716	3.2284	9.3858	0.9923
5	14	9.3135	4.6865	11.6568	1.192
6	12	13.8951	−1.8951	12.9476	1.127
7	9	14.5918	−5.5918	11.7959	0.9542
8	13	11.2561	1.7439	12.1281	1.0134
9	16	12.2903	3.7097	14.1451	1.1357
10	14	16.065	−2.065	15.0325	1.0773
11	11	16.195	−5.195	13.5975	0.9391
12	15	12.7694	2.2306	13.8847	1.0047
13	18	13.9502	4.0498	15.9751	1.1214
14	16	17.9143	−1.9143	16.9571	1.0735
15	13	18.2027	−5.2027	15.6014	0.9507
16	17	14.8327	2.1673	15.9163	1.0063

160.12. A-M-A

Multiplicative trend, additive seasonality

$$\mathrm{ETS}(A, M, A)$$

$$x_t = l_{t-1} b_{t-1} + s_{t-m} + \varepsilon_t$$

$$l_t = l_{t-1} b_{t-1} + \alpha \varepsilon_t$$

$$b_t = b_{t-1} + \beta \varepsilon_t / l_{t-1}$$

$$s_t = s_{t-m} + \gamma \varepsilon_t$$

$$\hat{x}_{t+h \mid t} = l_t b_t^h + s_{t+h-m_h^+}$$

Example: $\mathrm{ETS}(A, M, A)$

Given

- Smoothing parameters: $\alpha = 0.5$, $\beta = 0.4$, $\gamma = 0.2$
- Initial states: $l_0 = 12$, $b_0 = 1$, $(s_{-3}, s_{-2}, s_{-1}, s_0) = (2, 0, -3, 1)$, seasonal period $m = 4$
- Data:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
x_t	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

Step 1 — formula

Observation:

$$x_t = l_{t-1} b_{t-1} + s_{t-m} + \varepsilon_t$$

Conditional mean (one-step-ahead forecast $\hat{x}_{t \mid t-1} = \mu_t$):

$$\mu_t = l_{t-1} b_{t-1} + s_{t-m}$$

Innovation:

$$\varepsilon_t = x_t - \mu_t$$

State updates:

$$l_t = l_{t-1} b_{t-1} + \alpha \varepsilon_t$$

$$b_t = b_{t-1} + \beta \varepsilon_t / l_{t-1}$$

$$s_t = s_{t-m} + \gamma \varepsilon_t$$

Forecast h steps ahead from time t (using current-period states):

$$\hat{x}_{t+h \mid t} = l_t b_t^h + s_{t+h-m_h^+}$$

where $h \in \{1, 2, 3, ...\}$ is the forecast horizon (how many steps ahead); $m_h^+ = ((h-1) \bmod m) + 1$ picks the right seasonal slot for the period h steps ahead (cycles through $1, 2, ..., m$).

Step 2 — apply at $t = 1$

$$\mu_1 = 12 \cdot 1 + 2 = 14$$

$$\varepsilon_1 = x_1 - \mu_1 = 12 - 14 = -2$$

$$l_1 = 12 \cdot 1 + 0.5 \cdot (-2) = 11$$

$$b_1 = 1 + 0.4 \cdot (-2)/12 = 0.9333$$

$$s_1 = 2 + 0.2 \cdot (-2) = 1.6$$

Step 3 — iterate

Each column header is the equation that produced its values. Values rounded to 4 decimal places; arithmetic performed at full precision.

t	x_t	$\mu_t = l_{t-1}b_{t-1} + s_{t-m}$	ε_t	$l_t = l_{t-1}b_{t-1} + \alpha\varepsilon_t$	$b_t = b_{t-1} + \beta\varepsilon_t/l_{t-1}$	$s_t = s_{t-m} + \gamma\varepsilon_t$
1	12	14	−2	11	0.9333	1.6
2	10	10.2667	−0.2667	10.1333	0.9236	−0.0533
3	8	6.3595	1.6405	10.1798	0.9884	−2.6719
4	11	11.0616	−0.0616	10.0308	0.986	0.9877
5	14	11.4901	2.5099	11.145	1.0861	2.102
6	12	12.0509	−0.0509	12.0788	1.0842	−0.0635
7	9	10.4243	−1.4243	12.3841	1.0371	−2.9568
8	13	13.8308	−0.8308	12.4277	1.0102	0.8215
9	16	14.6569	1.3431	13.2265	1.0535	2.3706
10	14	13.8701	0.1299	13.9985	1.0574	−0.0375
11	11	11.8452	−0.8452	14.3794	1.0332	−3.1258
12	15	15.6789	−0.6789	14.5179	1.0144	0.6857
13	18	17.097	0.903	15.1779	1.0392	2.5512
14	16	15.7359	0.2641	15.9055	1.0462	0.0153
15	13	13.5145	−0.5145	16.383	1.0333	−3.2287
16	17	17.6137	−0.6137	16.6211	1.0183	0.563

160.13. A-M-M

Mult. trend, mult. seasonality

$$\mathrm{ETS}(A, M, M)$$

$$x_t = l_{t-1}b_{t-1}s_{t-m} + \varepsilon_t$$

$$l_t = l_{t-1} b_{t-1} + \alpha \varepsilon_t / s_{t-m}$$

$$b_t = b_{t-1} + \beta \varepsilon_t / (l_{t-1} s_{t-m})$$

$$s_t = s_{t-m} + \gamma \varepsilon_t / (l_{t-1} b_{t-1})$$

$$\hat{x}_{t+h \mid t} = l_t b_t^h s_{t+h-m_h^+}$$

Example: $\mathrm{ETS}(A, M, M)$

Given

- Smoothing parameters: $\alpha = 0.5$, $\beta = 0.4$, $\gamma = 0.2$
- Initial states: $l_0 = 12$, $b_0 = 1$, $(s_{-3}, s_{-2}, s_{-1}, s_0) = (1.2, 1, 0.8, 1)$, seasonal period $m = 4$
- Data:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
x_t	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

Step 1 — formula

Observation:

$$x_t = l_{t-1} b_{t-1} s_{t-m} + \varepsilon_t$$

Conditional mean (one-step-ahead forecast $\hat{x}_{t \mid t-1} = \mu_t$):

$$\mu_t = l_{t-1} b_{t-1} s_{t-m}$$

Innovation:

$$\varepsilon_t = x_t - \mu_t$$

State updates:

$$l_t = l_{t-1} b_{t-1} + \alpha \varepsilon_t / s_{t-m}$$

$$b_t = b_{t-1} + \beta \varepsilon_t / (l_{t-1} s_{t-m})$$

$$s_t = s_{t-m} + \gamma \varepsilon_t / (l_{t-1} b_{t-1})$$

Forecast h steps ahead from time t (using current-period states):

$$\hat{x}_{t+h \mid t} = l_t b_t^h s_{t+h-m_h^+}$$

where $h \in \{1, 2, 3, \ldots\}$ is the forecast horizon (how many steps ahead); $m_h^+ = ((h-1) \bmod m) + 1$ picks the right seasonal slot for the period h steps ahead (cycles through $1, 2, \ldots, m$).

Step 2 — apply at $t = 1$

$$\mu_1 = 12 \cdot 1 \cdot 1.2 = 14.4$$

$$\varepsilon_1 = x_1 - \mu_1 = 12 - 14.4 = -2.4$$

$$l_1 = 12 \cdot 1 + 0.5 \cdot (-2.4)/1.2 = 11$$

$$b_1 = 1 + 0.4 \cdot (-2.4)/(12 \cdot 1.2) = 0.9333$$

$$s_1 = 1.2 + 0.2 \cdot (-2.4)/(12 \cdot 1) = 1.16$$

Step 3 — iterate

Each column header is the equation that produced its values. Values rounded to 4 decimal places; arithmetic performed at full precision.

t	x_t	$\mu_t = l_{t-1}b_{t-1}s_{t-m}$	ε_t	$l_t = l_{t-1}b_{t-1} + \alpha\varepsilon_t/s_{t-m}$	$b_t = b_{t-1} + \beta\varepsilon_t/(l_{t-1}s_{t-m})$	$s_t = s_{t-m} + \gamma\varepsilon_t/(l_{t-1}b_{t-1})$
1	12	14.4	−2.4	11	0.9333	1.16
2	10	10.2667	−0.2667	10.1333	0.9236	0.9948
3	8	7.4876	0.5124	9.6798	0.9489	0.8109
4	11	9.1853	1.8147	10.0927	1.0239	1.0395
5	14	11.9874	2.0126	11.2015	1.0927	1.199
6	12	12.1759	−0.1759	12.1511	1.0864	0.9919
7	9	10.7049	−1.7049	12.1493	1.0172	0.7851
8	13	12.8459	0.1541	12.4317	1.022	1.042
9	16	15.2334	0.7666	13.0253	1.0426	1.211
10	14	13.4706	0.5294	13.8471	1.059	0.9997
11	11	11.5129	−0.5129	14.3373	1.0401	0.7781
12	15	15.5389	−0.5389	14.6539	1.0257	1.0348
13	18	18.202	−0.202	14.9469	1.0211	1.2083
14	16	15.2587	0.7413	15.6336	1.041	1.0094
15	13	12.6634	0.3366	16.4906	1.052	0.7823
16	17	17.9523	−0.9523	16.8888	1.0297	1.0238

160.14. A-Md-N

Damped multiplicative trend

$$\mathrm{ETS}(A, \mathrm{Md}, N)$$

$$x_t = l_{t-1}b_{t-1}^{\varphi} + \varepsilon_t$$

$$l_t = l_{t-1}b_{t-1}^{\varphi} + \alpha\varepsilon_t$$

$$b_t = b_{t-1}^{\varphi} + \beta\varepsilon_t/l_{t-1}$$

$$\hat{x}_{t+h\,|\,t} = l_t b_t^{\varphi+\varphi^2+\ldots+\varphi^h}$$

Example: $\mathrm{ETS}(A, \mathrm{Md}, N)$

Given

- Smoothing parameters: $\alpha = 0.5$, $\beta = 0.4$, $\varphi = 0.8$
- Initial states: $l_0 = 12$, $b_0 = 1$
- Data:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
x_t	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

Step 1 — formula

Observation:

$$x_t = l_{t-1} b_{t-1}^{\varphi} + \varepsilon_t$$

Conditional mean (one-step-ahead forecast $\hat{x}_{t \mid t-1} = \mu_t$):

$$\mu_t = l_{t-1} b_{t-1}^{\varphi}$$

Innovation:

$$\varepsilon_t = x_t - \mu_t$$

State updates:

$$l_t = l_{t-1} b_{t-1}^{\varphi} + \alpha \varepsilon_t$$

$$b_t = b_{t-1}^{\varphi} + \beta \varepsilon_t / l_{t-1}$$

Forecast h steps ahead from time t (using current-period states):

$$\hat{x}_{t+h \mid t} = l_t b_t^{\varphi + \varphi^2 + \ldots + \varphi^h}$$

where $h \in \{1, 2, 3, \ldots\}$ is the forecast horizon (how many steps ahead).

Step 2 — apply at $t = 1$

$$\mu_1 = 12 \cdot 1^{0.8} = 12$$

$$\varepsilon_1 = x_1 - \mu_1 = 12 - 12 = 0$$

$$l_1 = 12 \cdot 1^{0.8} + 0.5 \cdot 0 = 12$$

$$b_1 = 1^{0.8} + 0.4 \cdot 0 / 12 = 1$$

Step 3 — iterate

Each column header is the equation that produced its values. Values rounded to 4 decimal places; arithmetic performed at full precision.

t	x_t	$\mu_t = l_{t-1} b_{t-1}^{\varphi}$	ε_t	$l_t = l_{t-1} b_{t-1}^{\varphi} + \alpha \varepsilon_t$	$b_t = b_{t-1}^{\varphi} + \beta \varepsilon_t / l_{t-1}$
1	12	12	0	12	1
2	10	12	−2	11	0.9333
3	8	10.4093	−2.4093	9.2047	0.8587
4	11	8.1485	2.8515	9.5742	1.0092

5	14	9.6444	4.3556	11.8222	1.1893
6	12	13.581	−1.581	12.7905	1.0953
7	9	13.7565	−4.7565	11.3782	0.9268
8	13	10.7066	2.2934	11.8533	1.0216
9	16	12.0577	3.9423	14.0288	1.1503
10	14	15.6915	−1.6915	14.8457	1.0703
11	11	15.6748	−4.6748	13.3374	0.9299
12	15	12.5839	2.4161	13.792	1.016
13	18	13.9678	4.0322	15.9839	1.1297
14	16	17.6219	−1.6219	16.8109	1.0619
15	13	17.6382	−4.6382	15.3191	0.9388
16	17	14.565	2.435	15.7825	1.0144

160.15. A-Md-A

Damped mult. trend, add. seasonality

$$\mathrm{ETS}(A, \mathrm{Md}, A)$$

$$x_t = l_{t-1} b_{t-1}^{\varphi} + s_{t-m} + \varepsilon_t$$

$$l_t = l_{t-1} b_{t-1}^{\varphi} + \alpha \varepsilon_t$$

$$b_t = b_{t-1}^{\varphi} + \beta \varepsilon_t / l_{t-1}$$

$$s_t = s_{t-m} + \gamma \varepsilon_t$$

$$\hat{x}_{t+h \mid t} = l_t b_t^{\varphi+\varphi^2+\ldots+\varphi^h} + s_{t+h-m_h^+}$$

Example: $\mathrm{ETS}(A, \mathrm{Md}, A)$

Given

- Smoothing parameters: $\alpha = 0.5$, $\beta = 0.4$, $\varphi = 0.8$, $\gamma = 0.2$
- Initial states: $l_0 = 12$, $b_0 = 1$, $(s_{-3}, s_{-2}, s_{-1}, s_0) = (2, 0, -3, 1)$, seasonal period $m = 4$
- Data:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
x_t	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

Step 1 — formula

Observation:

$$x_t = l_{t-1} b_{t-1}^{\varphi} + s_{t-m} + \varepsilon_t$$

Conditional mean (one-step-ahead forecast $\hat{x}_{t \mid t-1} = \mu_t$):

$$\mu_t = l_{t-1} b_{t-1}^{\varphi} + s_{t-m}$$

Innovation:

$$\varepsilon_t = x_t - \mu_t$$

State updates:

$$l_t = l_{t-1} b_{t-1}^{\varphi} + \alpha \varepsilon_t$$

$$b_t = b_{t-1}^{\varphi} + \beta \varepsilon_t / l_{t-1}$$

$$s_t = s_{t-m} + \gamma \varepsilon_t$$

Forecast h steps ahead from time t (using current-period states):

$$\hat{x}_{t+h \mid t} = l_t b_t^{\varphi+\varphi^2+\ldots+\varphi^h} + s_{t+h-m_h^+}$$

where $h \in \{1, 2, 3, \ldots\}$ is the forecast horizon (how many steps ahead); $m_h^+ = ((h-1) \bmod m) + 1$ picks the right seasonal slot for the period h steps ahead (cycles through $1, 2, \ldots, m$).

Step 2 — apply at $t = 1$

$$\mu_1 = 12 \cdot 1^{0.8} + 2 = 14$$

$$\varepsilon_1 = x_1 - \mu_1 = 12 - 14 = -2$$

$$l_1 = 12 \cdot 1^{0.8} + 0.5 \cdot (-2) = 11$$

$$b_1 = 1^{0.8} + 0.4 \cdot (-2)/12 = 0.9333$$

$$s_1 = 2 + 0.2 \cdot (-2) = 1.6$$

Step 3 — iterate

Each column header is the equation that produced its values. Values rounded to 4 decimal places; arithmetic performed at full precision.

t	x_t	$\mu_t = l_{t-1} b_{t-1}^{\varphi} + s_{t-m}$	ε_t	$l_t = l_{t-1} b_{t-1}^{\varphi} + \alpha \varepsilon_t$	$b_t = b_{t-1}^{\varphi} + \beta \varepsilon_t / l_{t-1}$	$s_t = s_{t-m} + \gamma \varepsilon_t$
1	12	14	−2	11	0.9333	1.6
2	10	10.4093	−0.4093	10.2047	0.9314	−0.0819
3	8	6.6408	1.3592	10.3204	0.998	−2.7282
4	11	11.3041	−0.3041	10.152	0.9866	0.9392
5	14	11.6433	2.3567	11.2217	1.0821	2.0713
6	12	11.8714	0.1286	12.0176	1.0698	−0.0561
7	9	9.9557	−0.9557	12.206	1.0236	−2.9193
8	13	13.3754	−0.3754	12.2485	1.0066	0.8641
9	16	14.3841	1.6159	13.1207	1.058	2.3945

10	14	13.6701	0.3299	13.8912	1.0562	0.0098
11	11	11.5931	−0.5931	14.2158	1.0276	−3.0379
12	15	15.3934	−0.3934	14.3326	1.011	0.7854
13	18	16.8529	1.1471	15.0319	1.0408	2.6239
14	16	15.5303	0.4697	15.7553	1.045	0.1038
15	13	13.2821	−0.2821	16.179	1.0287	−3.0943
16	17	17.3345	−0.3345	16.3818	1.0146	0.7185

160.16. A-Md-M

Damped mult. trend, mult. seasonality

$$\mathrm{ETS}(A, \mathrm{Md}, M)$$

$$x_t = l_{t-1} b_{t-1}^{\varphi} s_{t-m} + \varepsilon_t$$

$$l_t = l_{t-1} b_{t-1}^{\varphi} + \alpha \varepsilon_t / s_{t-m}$$

$$b_t = b_{t-1}^{\varphi} + \beta \varepsilon_t / (l_{t-1} s_{t-m})$$

$$s_t = s_{t-m} + \gamma \varepsilon_t / (l_{t-1} b_{t-1}^{\varphi})$$

$$\hat{x}_{t+h \mid t} = l_t b_t^{\varphi+\varphi^2+\ldots+\varphi^h} s_{t+h-m_h^+}$$

Example: $\mathrm{ETS}(A, \mathrm{Md}, M)$

Given

- Smoothing parameters: $\alpha = 0.5$, $\beta = 0.4$, $\varphi = 0.8$, $\gamma = 0.2$
- Initial states: $l_0 = 12$, $b_0 = 1$, $(s_{-3}, s_{-2}, s_{-1}, s_0) = (1.2, 1, 0.8, 1)$, seasonal period $m = 4$
- Data:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
x_t	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

Step 1 — formula

Observation:

$$x_t = l_{t-1} b_{t-1}^{\varphi} s_{t-m} + \varepsilon_t$$

Conditional mean (one-step-ahead forecast $\hat{x}_{t \mid t-1} = \mu_t$):

$$\mu_t = l_{t-1} b_{t-1}^{\varphi} s_{t-m}$$

Innovation:

$$\varepsilon_t = x_t - \mu_t$$

State updates:

$$l_t = l_{t-1} b_{t-1}^{\varphi} + \alpha \varepsilon_t / s_{t-m}$$

$$b_t = b_{t-1}^{\varphi} + \beta \varepsilon_t / (l_{t-1} s_{t-m})$$

$$s_t = s_{t-m} + \gamma \varepsilon_t / (l_{t-1} b_{t-1}^{\varphi})$$

Forecast h steps ahead from time t (using current-period states):

$$\hat{x}_{t+h \mid t} = l_t b_t^{\varphi+\varphi^2+\ldots+\varphi^h} s_{t+h-m_h^+}$$

where $h \in \{1, 2, 3, \ldots\}$ is the forecast horizon (how many steps ahead); $m_h^+ = ((h-1) \bmod m) + 1$ picks the right seasonal slot for the period h steps ahead (cycles through $1, 2, \ldots, m$).

Step 2 — apply at $t = 1$

$$\mu_1 = 12 \cdot 1^{0.8} \cdot 1.2 = 14.4$$

$$\varepsilon_1 = x_1 - \mu_1 = 12 - 14.4 = -2.4$$

$$l_1 = 12 \cdot 1^{0.8} + 0.5 \cdot (-2.4)/1.2 = 11$$

$$b_1 = 1^{0.8} + 0.4 \cdot (-2.4)/(12 \cdot 1.2) = 0.9333$$

$$s_1 = 1.2 + 0.2 \cdot (-2.4)/(12 \cdot 1^{0.8}) = 1.16$$

Step 3 — iterate

Each column header is the equation that produced its values. Values rounded to 4 decimal places; arithmetic performed at full precision.

t	x_t	$\mu_t = l_{t-1} b_{t-1}^{\varphi} s_{t-m}$	ε_t	$l_t = l_{t-1} b_{t-1}^{\varphi} + \alpha \varepsilon_t / s_{t-m}$	$b_t = b_{t-1}^{\varphi} + \beta \varepsilon_t / (l_{t-1} s_{t-m})$	$s_t = s_{t-m} + \gamma \varepsilon_t / (l_{t-1} b_{t-1}^{\varphi})$
1	12	14.4	−2.4	11	0.9333	1.16
2	10	10.4093	−0.4093	10.2047	0.9314	0.9921
3	8	7.7127	0.2873	9.8204	0.9588	0.806
4	11	9.4956	1.5044	10.2478	1.0282	1.0317
5	14	12.1549	1.8451	11.2737	1.0846	1.1952
6	12	11.9357	0.0643	12.0627	1.0694	0.9932
7	9	10.2583	−1.2583	11.9474	1.0034	0.7862
8	13	12.3594	0.6406	12.2902	1.0235	1.0424
9	16	14.965	1.035	12.9537	1.0469	1.2118
10	14	13.3466	0.6534	13.7668	1.0577	1.0029
11	11	11.32	−0.32	14.1951	1.0341	0.7817
12	15	15.1986	−0.1986	14.4854	1.0218	1.0397
13	18	17.858	0.142	14.796	1.0206	1.2137
14	16	15.0837	0.9163	15.4965	1.0412	1.0151

15	13	12.5116	0.4884	16.3171	1.0489	0.7878
16	17	17.6251	−0.6251	16.6521	1.0242	1.0323

160.17. M-N-N

SES, multiplicative errors

$$\mathrm{ETS}(M, N, N)$$

$$x_t = l_{t-1}(1 + \varepsilon_t)$$

$$l_t = l_{t-1}(1 + \alpha\varepsilon_t)$$

$$\hat{x}_{t+h \mid t} = l_t$$

Example: $\mathrm{ETS}(M, N, N)$

Given

- Smoothing parameters: $\alpha = 0.5$
- Initial states: $l_0 = 12$
- Data:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
x_t	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

Step 1 — formula

Observation:

$$x_t = (l_{t-1})(1 + \varepsilon_t)$$

Conditional mean (one-step-ahead forecast $\hat{x}_{t \mid t-1} = \mu_t$):

$$\mu_t = l_{t-1}$$

Innovation:

$$\varepsilon_t = (x_t - \mu_t)/\mu_t$$

State updates:

$$l_t = l_{t-1}(1 + \alpha\varepsilon_t)$$

Forecast h steps ahead from time t (using current-period states):

$$\hat{x}_{t+h \mid t} = l_t$$

Step 2 — apply at $t = 1$

$$\mu_1 = 12 = 12$$

$$\varepsilon_1 = (x_1 - \mu_1)/\mu_1 = (12 - 12)/12 = 0$$

$$l_1 = 12(1 + 0.5 \cdot 0) = 12$$

Step 3 — iterate

Each column header is the equation that produced its values. Values rounded to 4 decimal places; arithmetic performed at full precision.

t	x_t	$\mu_t = l_{t-1}$	ε_t	$l_t = l_{t-1}(1 + \alpha\varepsilon_t)$
1	12	12	0	12
2	10	12	−0.1667	11
3	8	11	−0.2727	9.5
4	11	9.5	0.1579	10.25
5	14	10.25	0.3659	12.125
6	12	12.125	−0.0103	12.0625
7	9	12.0625	−0.2539	10.5312
8	13	10.5312	0.2344	11.7656
9	16	11.7656	0.3599	13.8828
10	14	13.8828	0.0084	13.9414
11	11	13.9414	−0.211	12.4707
12	15	12.4707	0.2028	13.7354
13	18	13.7354	0.3105	15.8677
14	16	15.8677	0.0083	15.9338
15	13	15.9338	−0.1841	14.4669
16	17	14.4669	0.1751	15.7335

160.18. M-N-A

SES + add. seasonality, mult. errors

$$\mathrm{ETS}(M, N, A)$$

$$x_t = (l_{t-1} + s_{t-m})(1 + \varepsilon_t)$$

$$l_t = l_{t-1} + \alpha(l_{t-1} + s_{t-m})\varepsilon_t$$

$$s_t = s_{t-m} + \gamma(l_{t-1} + s_{t-m})\varepsilon_t$$

$$\hat{x}_{t+h \mid t} = l_t + s_{t+h-m_h^+}$$

Example: $\mathrm{ETS}(M, N, A)$

Given

- Smoothing parameters: $\alpha = 0.5$, $\gamma = 0.2$
- Initial states: $l_0 = 12$, $(s_{-3}, s_{-2}, s_{-1}, s_0) = (2, 0, -3, 1)$, seasonal period $m = 4$
- Data:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
x_t	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

Step 1 — formula

Observation:

$$x_t = (l_{t-1} + s_{t-m})(1 + \varepsilon_t)$$

Conditional mean (one-step-ahead forecast $\hat{x}_{t \mid t-1} = \mu_t$):

$$\mu_t = l_{t-1} + s_{t-m}$$

Innovation:

$$\varepsilon_t = (x_t - \mu_t)/\mu_t$$

State updates:

$$l_t = l_{t-1} + \alpha \mu_t \varepsilon_t$$

$$s_t = s_{t-m} + \gamma \mu_t \varepsilon_t$$

Forecast h steps ahead from time t (using current-period states):

$$\hat{x}_{t+h \mid t} = l_t + s_{t+h-m_h^+}$$

where $m_h^+ = ((h-1) \bmod m) + 1$ picks the right seasonal slot for the period h steps ahead (cycles through $1, 2, \ldots, m$).

Step 2 — apply at $t = 1$

$$\mu_1 = 12 + 2 = 14$$

$$\varepsilon_1 = (x_1 - \mu_1)/\mu_1 = (12 - 14)/14 = -0.1429$$

$$l_1 = 12 + 0.5 \cdot 14 \cdot (-0.1429) = 11$$

$$s_1 = 2 + 0.2 \cdot 14 \cdot (-0.1429) = 1.6$$

Step 3 — iterate

Each column header is the equation that produced its values. Values rounded to 4 decimal places; arithmetic performed at full precision.

t	x_t	$\mu_t = l_{t-1} + s_{t-m}$	ε_t	$l_t = l_{t-1} + \alpha \mu_t \varepsilon_t$	$s_t = s_{t-m} + \gamma \mu_t \varepsilon_t$
1	12	14	−0.1429	11	1.6
2	10	11	−0.0909	10.5	−0.2
3	8	7.5	0.0667	10.75	−2.9
4	11	11.75	−0.0638	10.375	0.85
5	14	11.975	0.1691	11.3875	2.005
6	12	11.1875	0.0726	11.7937	−0.0375
7	9	8.8937	0.0119	11.8469	−2.8787

8	13	12.6969	0.0239	11.9984	0.9106
9	16	14.0034	0.1426	12.9967	2.4043
10	14	12.9592	0.0803	13.5171	0.1707
11	11	10.6384	0.034	13.6979	−2.8064
12	15	14.6086	0.0268	13.8937	0.9889
13	18	16.298	0.1044	14.7447	2.7447
14	16	14.9153	0.0727	15.287	0.3876
15	13	12.4806	0.0416	15.5467	−2.7025
16	17	16.5356	0.0281	15.7789	1.0818

160.19. M-N-M

SES + mult. seasonality and errors

$$\mathrm{ETS}(M, N, M)$$

$$x_t = l_{t-1} s_{t-m} (1 + \varepsilon_t)$$

$$l_t = l_{t-1} (1 + \alpha \varepsilon_t)$$

$$s_t = s_{t-m} (1 + \gamma \varepsilon_t)$$

$$\hat{x}_{t+h \mid t} = l_t s_{t+h-m_h^+}$$

Example: $\mathrm{ETS}(M, N, M)$

Given

- Smoothing parameters: $\alpha = 0.5$, $\gamma = 0.2$
- Initial states: $l_0 = 12$, $(s_{-3}, s_{-2}, s_{-1}, s_0) = (1.2, 1, 0.8, 1)$, seasonal period $m = 4$
- Data:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
x_t	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

Step 1 — formula

Observation:

$$x_t = (l_{t-1} s_{t-m})(1 + \varepsilon_t)$$

Conditional mean (one-step-ahead forecast $\hat{x}_{t \mid t-1} = \mu_t$):

$$\mu_t = l_{t-1} s_{t-m}$$

Innovation:

$$\varepsilon_t = (x_t - \mu_t)/\mu_t$$

State updates:

$$l_t = l_{t-1}(1 + \alpha\varepsilon_t)$$

$$s_t = s_{t-m}(1 + \gamma\varepsilon_t)$$

Forecast h steps ahead from time t (using current-period states):

$$\hat{x}_{t+h \mid t} = l_t s_{t+h-m_h^+}$$

where $m_h^+ = ((h-1) \bmod m) + 1$ picks the right seasonal slot for the period h steps ahead (cycles through $1, 2, \ldots, m$).

Step 2 — apply at $t = 1$

$$\mu_1 = 12 \cdot 1.2 = 14.4$$

$$\varepsilon_1 = (x_1 - \mu_1)/\mu_1 = (12 - 14.4)/14.4 = -0.1667$$

$$l_1 = 12(1 + 0.5 \cdot (-0.1667)) = 11$$

$$s_1 = 1.2(1 + 0.2 \cdot (-0.1667)) = 1.16$$

Step 3 — iterate

Each column header is the equation that produced its values. Values rounded to 4 decimal places; arithmetic performed at full precision.

t	x_t	$\mu_t = l_{t-1}s_{t-m}$	ε_t	$l_t = l_{t-1}(1+\alpha\varepsilon_t)$	$s_t = s_{t-m}(1+\gamma\varepsilon_t)$
1	12	14.4	−0.1667	11	1.16
2	10	11	−0.0909	10.5	0.9818
3	8	8.4	−0.0476	10.25	0.7924
4	11	10.25	0.0732	10.625	1.0146
5	14	12.325	0.1359	11.347	1.1915
6	12	11.1407	0.0771	11.7846	0.997
7	9	9.3379	−0.0362	11.5714	0.7866
8	13	11.7407	0.1073	12.1919	1.0364
9	16	14.5271	0.1014	12.81	1.2157
10	14	12.7711	0.0962	13.4263	1.0162
11	11	10.5618	0.0415	13.7049	0.7932
12	15	14.2037	0.0561	14.089	1.048
13	18	17.1279	0.0509	14.4477	1.2281
14	16	14.681	0.0898	15.0967	1.0344
15	13	11.9743	0.0857	15.7433	0.8068
16	17	16.4993	0.0303	15.9822	1.0544

160.20. M-A-N

Holt's linear, multiplicative errors

$$\text{ETS}(M, A, N)$$

$$x_t = (l_{t-1} + b_{t-1})(1 + \varepsilon_t)$$

$$l_t = (l_{t-1} + b_{t-1})(1 + \alpha\varepsilon_t)$$

$$b_t = b_{t-1} + \beta(l_{t-1} + b_{t-1})\varepsilon_t$$

$$\hat{x}_{t+h \mid t} = l_t + hb_t$$

Example: $\text{ETS}(M, A, N)$

Given

- Smoothing parameters: $\alpha = 0.5$, $\beta = 0.4$
- Initial states: $l_0 = 12$, $b_0 = 0.5$
- Data:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
x_t	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

Step 1 — formula

Observation:

$$x_t = (l_{t-1} + b_{t-1})(1 + \varepsilon_t)$$

Conditional mean (one-step-ahead forecast $\hat{x}_{t \mid t-1} = \mu_t$):

$$\mu_t = l_{t-1} + b_{t-1}$$

Innovation:

$$\varepsilon_t = (x_t - \mu_t)/\mu_t$$

State updates:

$$l_t = (l_{t-1} + b_{t-1})(1 + \alpha\varepsilon_t)$$

$$b_t = b_{t-1} + \beta(l_{t-1} + b_{t-1})\varepsilon_t$$

Forecast h steps ahead from time t (using current-period states):

$$\hat{x}_{t+h \mid t} = l_t + hb_t$$

where $h \in \{1, 2, 3, ...\}$ is the forecast horizon (how many steps ahead).

Step 2 — apply at $t = 1$

$$\mu_1 = 12 + 0.5 = 12.5$$

$$\varepsilon_1 = (x_1 - \mu_1)/\mu_1 = (12 - 12.5)/12.5 = -0.04$$

$$l_1 = (12 + 0.5)(1 + 0.5 \cdot (-0.04)) = 12.25$$

$$b_1 = 0.5 + 0.4(12 + 0.5) \cdot (-0.04) = 0.3$$

Step 3 — iterate

Each column header is the equation that produced its values. Values rounded to 4 decimal places; arithmetic performed at full precision.

t	x_t	$\mu_t = l_{t-1} + b_{t-1}$	ε_t	$l_t = (l_{t-1} + b_{t-1})(1 + \alpha\varepsilon_t)$	$b_t = b_{t-1} + \beta(l_{t-1} + b_{t-1})\varepsilon_t$
1	12	12.5	−0.04	12.25	0.3
2	10	12.55	−0.2032	11.275	−0.72
3	8	10.555	−0.2421	9.2775	−1.742
4	11	7.5355	0.4598	9.2677	−0.3562
5	14	8.9116	0.571	11.4558	1.6792
6	12	13.135	−0.0864	12.5675	1.2252
7	9	13.7927	−0.3475	11.3963	−0.6919
8	13	10.7045	0.2144	11.8522	0.2263
9	16	12.0786	0.3247	14.0393	1.7949
10	14	15.8342	−0.1158	14.9171	1.0612
11	11	15.9783	−0.3116	13.4892	−0.9301
12	15	12.5591	0.1944	13.7795	0.0463
13	18	13.8258	0.3019	15.9129	1.716
14	16	17.6289	−0.0924	16.8144	1.0644
15	13	17.8788	−0.2729	15.4394	−0.8871
16	17	14.5523	0.1682	15.7761	0.092

160.21. M-A-A

Additive HW, multiplicative errors

$$\mathrm{ETS}(M, A, A)$$

$$x_t = (l_{t-1} + b_{t-1} + s_{t-m})(1 + \varepsilon_t)$$

$$l_t = l_{t-1} + b_{t-1} + \alpha\mu_t\varepsilon_t$$

$$b_t = b_{t-1} + \beta\mu_t\varepsilon_t$$

$$s_t = s_{t-m} + \gamma\mu_t\varepsilon_t$$

$$\hat{x}_{t+h\,|\,t} = l_t + hb_t + s_{t+h-m_h^+}$$

where

$$\mu_t = l_{t-1} + b_{t-1} + s_{t-m}$$

Example: $\mathrm{ETS}(M, A, A)$

Given

- Smoothing parameters: $\alpha = 0.5$, $\beta = 0.4$, $\gamma = 0.2$
- Initial states: $l_0 = 12$, $b_0 = 0.5$, $(s_{-3}, s_{-2}, s_{-1}, s_0) = (2, 0, -3, 1)$, seasonal period $m = 4$
- Data:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
x_t	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

Step 1 — formula

Observation:

$$x_t = (l_{t-1} + b_{t-1} + s_{t-m})(1 + \varepsilon_t)$$

Conditional mean (one-step-ahead forecast $\hat{x}_{t \mid t-1} = \mu_t$):

$$\mu_t = l_{t-1} + b_{t-1} + s_{t-m}$$

Innovation:

$$\varepsilon_t = (x_t - \mu_t)/\mu_t$$

State updates:

$$l_t = l_{t-1} + b_{t-1} + \alpha \mu_t \varepsilon_t$$

$$b_t = b_{t-1} + \beta \mu_t \varepsilon_t$$

$$s_t = s_{t-m} + \gamma \mu_t \varepsilon_t$$

Forecast h steps ahead from time t (using current-period states):

$$\hat{x}_{t+h \mid t} = l_t + h b_t + s_{t+h-m_h^+}$$

where $h \in \{1, 2, 3, ...\}$ is the forecast horizon (how many steps ahead); $m_h^+ = ((h - 1) \bmod m) + 1$ picks the right seasonal slot for the period h steps ahead (cycles through $1, 2, ..., m$).

Step 2 — apply at $t = 1$

$$\mu_1 = 12 + 0.5 + 2 = 14.5$$

$$\varepsilon_1 = (x_1 - \mu_1)/\mu_1 = (12 - 14.5)/14.5 = -0.1724$$

$$l_1 = 12 + 0.5 + 0.5 \cdot 14.5 \cdot (-0.1724) = 11.25$$

$$b_1 = 0.5 + 0.4 \cdot 14.5 \cdot (-0.1724) = -0.5$$

$$s_1 = 2 + 0.2 \cdot 14.5 \cdot (-0.1724) = 1.5$$

Step 3 — iterate

Each column header is the equation that produced its values. Values rounded to 4 decimal places; arithmetic performed at full precision.

t	x_t	$\mu_t = l_{t-1} + b_{t-1} + s_{t-m}$	ε_t	$l_t = l_{t-1} + b_{t-1} + \alpha\mu_t\varepsilon_t$	$b_t = b_{t-1} + \beta\mu_t\varepsilon_t$	$s_t = s_{t-m} + \gamma\mu_t\varepsilon_t$
1	12	14.5	−0.1724	11.25	−0.5	1.5
2	10	10.75	−0.0698	10.375	−0.8	−0.15
3	8	6.575	0.2167	10.2875	−0.23	−2.715
4	11	11.0575	−0.0052	10.0287	−0.253	0.9885
5	14	11.2757	0.2416	11.1379	0.8367	2.0449
6	12	11.8246	0.0148	12.0623	0.9069	−0.1149
7	9	10.2542	−0.1223	12.3421	0.4052	−2.9658
8	13	13.7358	−0.0536	12.3794	0.1109	0.8413
9	16	14.5351	0.1008	13.2227	0.6968	2.3378
10	14	13.8046	0.0142	14.0172	0.775	−0.0758
11	11	11.8264	−0.0699	14.379	0.4444	−3.1311
12	15	15.6648	−0.0424	14.4911	0.1785	0.7084
13	18	17.0074	0.0584	15.1659	0.5756	2.5363
14	16	15.6656	0.0213	15.9086	0.7093	−0.009
15	13	13.4868	−0.0361	16.3745	0.5146	−3.2285
16	17	17.5975	−0.034	16.5904	0.2756	0.5889

160.22. M-A-M

Multiplicative Holt-Winters

$$\text{ETS}(M, A, M)$$

$$x_t = (l_{t-1} + b_{t-1})s_{t-m}(1 + \varepsilon_t)$$

$$l_t = (l_{t-1} + b_{t-1})(1 + \alpha\varepsilon_t)$$

$$b_t = b_{t-1} + \beta(l_{t-1} + b_{t-1})\varepsilon_t$$

$$s_t = s_{t-m}(1 + \gamma\varepsilon_t)$$

$$\hat{x}_{t+h \mid t} = (l_t + hb_t)s_{t+h-m_h^+}$$

Example: $\text{ETS}(M, A, M)$

Given

- Smoothing parameters: $\alpha = 0.5$, $\beta = 0.4$, $\gamma = 0.2$
- Initial states: $l_0 = 12$, $b_0 = 0.5$, $(s_{-3}, s_{-2}, s_{-1}, s_0) = (1.2, 1, 0.8, 1)$, seasonal period $m = 4$
- Data:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

x_t	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

Step 1 — formula

Observation:

$$x_t = ((l_{t-1} + b_{t-1})s_{t-m})(1 + \varepsilon_t)$$

Conditional mean (one-step-ahead forecast $\hat{x}_{t \mid t-1} = \mu_t$):

$$\mu_t = (l_{t-1} + b_{t-1})s_{t-m}$$

Innovation:

$$\varepsilon_t = (x_t - \mu_t)/\mu_t$$

State updates:

$$l_t = (l_{t-1} + b_{t-1})(1 + \alpha\varepsilon_t)$$

$$b_t = b_{t-1} + \beta(l_{t-1} + b_{t-1})\varepsilon_t$$

$$s_t = s_{t-m}(1 + \gamma\varepsilon_t)$$

Forecast h steps ahead from time t (using current-period states):

$$\hat{x}_{t+h \mid t} = (l_t + hb_t)s_{t+h-m_h^+}$$

where $h \in \{1, 2, 3, ...\}$ is the forecast horizon (how many steps ahead); $m_h^+ = ((h-1) \bmod m) + 1$ picks the right seasonal slot for the period h steps ahead (cycles through $1, 2, ..., m$).

Step 2 — apply at $t = 1$

$$\mu_1 = (12 + 0.5) \cdot 1.2 = 15$$

$$\varepsilon_1 = (x_1 - \mu_1)/\mu_1 = (12 - 15)/15 = -0.2$$

$$l_1 = (12 + 0.5)(1 + 0.5 \cdot (-0.2)) = 11.25$$

$$b_1 = 0.5 + 0.4(12 + 0.5) \cdot (-0.2) = -0.5$$

$$s_1 = 1.2(1 + 0.2 \cdot (-0.2)) = 1.152$$

Step 3 — iterate

Each column header is the equation that produced its values. Values rounded to 4 decimal places; arithmetic performed at full precision.

t	x_t	$\mu_t = (l_{t-1} + b_{t-1})s_{t-m}$	ε_t	$l_t = (l_{t-1} + b_{t-1})(1 + \alpha\varepsilon_t)$	$b_t = b_{t-1} + \beta(l_{t-1} + b_{t-1})\varepsilon_t$	$s_t = s_{t-m}(1 + \gamma\varepsilon_t)$
1	12	15	−0.2	11.25	−0.5	1.152
2	10	10.75	−0.0698	10.375	−0.8	0.986

3	8	7.66	0.0444	9.7875	−0.63	0.8071
4	11	9.1575	0.2012	10.0787	0.107	1.0402
5	14	11.734	0.1931	11.1693	0.8938	1.1965
6	12	11.8948	0.0088	12.1164	0.9365	0.9878
7	9	10.5351	−0.1457	12.102	0.1757	0.7836
8	13	12.7718	0.0179	12.3874	0.2635	1.044
9	16	15.1367	0.057	13.0117	0.5521	1.2101
10	14	13.3982	0.0449	13.8684	0.7958	0.9967
11	11	11.4906	−0.0427	14.3512	0.5454	0.7769
12	15	15.5513	−0.0355	14.6325	0.3341	1.0366
13	18	18.1117	−0.0062	14.9204	0.2972	1.2086
14	16	15.1669	0.0549	15.6356	0.6316	1.0076
15	13	12.6378	0.0287	16.5003	0.8181	0.7813
16	17	17.9514	−0.053	16.8594	0.4509	1.0256

160.23. M-Ad-N

Damped linear, mult. errors

$$\mathrm{ETS}(M, \mathrm{Ad}, N)$$

$$x_t = (l_{t-1} + \varphi b_{t-1})(1 + \varepsilon_t)$$

$$l_t = (l_{t-1} + \varphi b_{t-1})(1 + \alpha\varepsilon_t)$$

$$b_t = \varphi b_{t-1} + \beta(l_{t-1} + \varphi b_{t-1})\varepsilon_t$$

$$\hat{x}_{t+h \mid t} = l_t + \left(\varphi + \varphi^2 + \ldots + \varphi^h\right) b_t$$

Example: $\mathrm{ETS}(M, \mathrm{Ad}, N)$

Given

- Smoothing parameters: $\alpha = 0.5$, $\beta = 0.4$, $\varphi = 0.8$
- Initial states: $l_0 = 12$, $b_0 = 0.5$
- Data:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
x_t	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

Step 1 — formula

Observation:

$$x_t = (l_{t-1} + \varphi b_{t-1})(1 + \varepsilon_t)$$

Conditional mean (one-step-ahead forecast $\hat{x}_{t \mid t-1} = \mu_t$):

$$\mu_t = l_{t-1} + \varphi b_{t-1}$$

Innovation:

$$\varepsilon_t = (x_t - \mu_t)/\mu_t$$

State updates:

$$l_t = (l_{t-1} + \varphi b_{t-1})(1 + \alpha\varepsilon_t)$$

$$b_t = \varphi b_{t-1} + \beta(l_{t-1} + \varphi b_{t-1})\varepsilon_t$$

Forecast h steps ahead from time t (using current-period states):

$$\hat{x}_{t+h \mid t} = l_t + \left(\varphi + \varphi^2 + \ldots + \varphi^h\right) b_t$$

where $h \in \{1, 2, 3, \ldots\}$ is the forecast horizon (how many steps ahead).

Step 2 — apply at $t = 1$

$$\mu_1 = 12 + 0.8 \cdot 0.5 = 12.4$$

$$\varepsilon_1 = (x_1 - \mu_1)/\mu_1 = (12 - 12.4)/12.4 = -0.0323$$

$$l_1 = (12 + 0.8 \cdot 0.5)(1 + 0.5 \cdot (-0.0323)) = 12.2$$

$$b_1 = 0.8 \cdot 0.5 + 0.4(12 + 0.8 \cdot 0.5) \cdot (-0.0323) = 0.24$$

Step 3 — iterate

Each column header is the equation that produced its values. Values rounded to 4 decimal places; arithmetic performed at full precision.

t	x_t	$\mu_t = l_{t-1} + \varphi b_{t-1}$	ε_t	$l_t = (l_{t-1} + \varphi b_{t-1})(1 + \alpha\varepsilon_t)$	$b_t = \varphi b_{t-1} + \beta(l_{t-1} + \varphi b_{t-1})\varepsilon_t$
1	12	12.4	−0.0323	12.2	0.24
2	10	12.392	−0.193	11.196	−0.7648
3	8	10.5842	−0.2442	9.2921	−1.6455
4	11	7.9757	0.3792	9.4878	−0.1067
5	14	9.4025	0.489	11.7012	1.7537
6	12	13.1042	−0.0843	12.5521	0.9613
7	9	13.3211	−0.3244	11.1605	−0.9594
8	13	10.393	0.2508	11.6965	0.2753
9	16	11.9167	0.3427	13.9584	1.8535
10	14	15.4412	−0.0933	14.7206	0.9063
11	11	15.4457	−0.2878	13.2228	−1.0532
12	15	12.3803	0.2116	13.6901	0.2053
13	18	13.8544	0.2992	15.9272	1.8225
14	16	17.3852	−0.0797	16.6926	0.9039
15	13	17.4157	−0.2535	15.2079	−1.0432

16	17	14.3733	0.1827	15.6867	0.2161

160.24. M-Ad-A

Damped additive HW, mult. errors

$$\text{ETS}(M, \text{Ad}, A)$$

$$x_t = (l_{t-1} + \varphi b_{t-1} + s_{t-m})(1 + \varepsilon_t)$$

$$l_t = l_{t-1} + \varphi b_{t-1} + \alpha \mu_t \varepsilon_t$$

$$b_t = \varphi b_{t-1} + \beta \mu_t \varepsilon_t$$

$$s_t = s_{t-m} + \gamma \mu_t \varepsilon_t$$

$$\hat{x}_{t+h \mid t} = l_t + (\varphi + \varphi^2 + \ldots + \varphi^h) b_t + s_{t+h-m_h^+}$$

where

$$\mu_t = l_{t-1} + \varphi b_{t-1} + s_{t-m}$$

Example: $\text{ETS}(M, \text{Ad}, A)$

Given

- Smoothing parameters: $\alpha = 0.5$, $\beta = 0.4$, $\varphi = 0.8$, $\gamma = 0.2$
- Initial states: $l_0 = 12$, $b_0 = 0.5$, $(s_{-3}, s_{-2}, s_{-1}, s_0) = (2, 0, -3, 1)$, seasonal period $m = 4$
- Data:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
x_t	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

Step 1 — formula

Observation:

$$x_t = (l_{t-1} + \varphi b_{t-1} + s_{t-m})(1 + \varepsilon_t)$$

Conditional mean (one-step-ahead forecast $\hat{x}_{t \mid t-1} = \mu_t$):

$$\mu_t = l_{t-1} + \varphi b_{t-1} + s_{t-m}$$

Innovation:

$$\varepsilon_t = (x_t - \mu_t)/\mu_t$$

State updates:

$$l_t = l_{t-1} + \varphi b_{t-1} + \alpha \mu_t \varepsilon_t$$

$$b_t = \varphi b_{t-1} + \beta \mu_t \varepsilon_t$$

$$s_t = s_{t-m} + \gamma \mu_t \varepsilon_t$$

Forecast h steps ahead from time t (using current-period states):

$$\hat{x}_{t+h \mid t} = l_t + (\varphi + \varphi^2 + \ldots + \varphi^h) b_t + s_{t+h-m_h^+}$$

where $h \in \{1, 2, 3, \ldots\}$ is the forecast horizon (how many steps ahead); $m_h^+ = ((h-1) \bmod m) + 1$ picks the right seasonal slot for the period h steps ahead (cycles through $1, 2, \ldots, m$).

Step 2 — apply at $t = 1$

$$\mu_1 = 12 + 0.8 \cdot 0.5 + 2 = 14.4$$

$$\varepsilon_1 = (x_1 - \mu_1)/\mu_1 = (12 - 14.4)/14.4 = -0.1667$$

$$l_1 = 12 + 0.8 \cdot 0.5 + 0.5 \cdot 14.4 \cdot (-0.1667) = 11.2$$

$$b_1 = 0.8 \cdot 0.5 + 0.4 \cdot 14.4 \cdot (-0.1667) = -0.56$$

$$s_1 = 2 + 0.2 \cdot 14.4 \cdot (-0.1667) = 1.52$$

Step 3 — iterate

Each column header is the equation that produced its values. Values rounded to 4 decimal places; arithmetic performed at full precision.

t	x_t	$\mu_t = l_{t-1} + \varphi b_{t-1} + s_{t-m}$	ε_t	$l_t = l_{t-1} + \varphi b_{t-1} + \alpha \mu_t \varepsilon_t$	$b_t = \varphi b_{t-1} + \beta \mu_t \varepsilon_t$	$s_t = s_{t-m} + \gamma \mu_t \varepsilon_t$
1	12	14.4	−0.1667	11.2	−0.56	1.52
2	10	10.752	−0.0699	10.376	−0.7488	−0.1504
3	8	6.777	0.1805	10.3885	−0.1098	−2.7554
4	11	11.3006	−0.0266	10.1503	−0.2081	0.9399
5	14	11.5038	0.217	11.2319	0.832	2.0192
6	12	11.7471	0.0215	12.0239	0.7667	−0.0998
7	9	9.882	−0.0892	12.1964	0.2606	−2.9318
8	13	13.3447	−0.0258	12.2325	0.0706	0.8709
9	16	14.3082	0.1182	13.1349	0.7332	2.3576
10	14	13.6216	0.0278	13.9106	0.7379	−0.0241
11	11	11.5692	−0.0492	14.2164	0.3627	−3.0456
12	15	15.3774	−0.0245	14.3178	0.1392	0.7954
13	18	16.7867	0.0723	15.0358	0.5966	2.6003
14	16	15.4889	0.033	15.7686	0.6817	0.0781
15	13	13.2684	−0.0202	16.1798	0.438	−3.0993
16	17	17.3257	−0.0188	16.3674	0.2202	0.7303

160.25. M-Ad-M

Damped multiplicative Holt-Winters

$$\text{ETS}(M, \text{Ad}, M)$$

$$x_t = (l_{t-1} + \varphi b_{t-1}) s_{t-m} (1 + \varepsilon_t)$$

$$l_t = (l_{t-1} + \varphi b_{t-1})(1 + \alpha \varepsilon_t)$$

$$b_t = \varphi b_{t-1} + \beta (l_{t-1} + \varphi b_{t-1}) \varepsilon_t$$

$$s_t = s_{t-m}(1 + \gamma \varepsilon_t)$$

$$\hat{x}_{t+h \mid t} = \left(l_t + \left(\varphi + \varphi^2 + \ldots + \varphi^h\right) b_t\right) s_{t+h-m_h^+}$$

Example: $\text{ETS}(M, \text{Ad}, M)$

Given

- Smoothing parameters: $\alpha = 0.5$, $\beta = 0.4$, $\varphi = 0.8$, $\gamma = 0.2$
- Initial states: $l_0 = 12$, $b_0 = 0.5$, $(s_{-3}, s_{-2}, s_{-1}, s_0) = (1.2, 1, 0.8, 1)$, seasonal period $m = 4$
- Data:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
x_t	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

Step 1 — formula

Observation:

$$x_t = ((l_{t-1} + \varphi b_{t-1}) s_{t-m})(1 + \varepsilon_t)$$

Conditional mean (one-step-ahead forecast $\hat{x}_{t \mid t-1} = \mu_t$):

$$\mu_t = (l_{t-1} + \varphi b_{t-1}) s_{t-m}$$

Innovation:

$$\varepsilon_t = (x_t - \mu_t)/\mu_t$$

State updates:

$$l_t = (l_{t-1} + \varphi b_{t-1})(1 + \alpha \varepsilon_t)$$

$$b_t = \varphi b_{t-1} + \beta (l_{t-1} + \varphi b_{t-1}) \varepsilon_t$$

$$s_t = s_{t-m}(1 + \gamma \varepsilon_t)$$

Forecast h steps ahead from time t (using current-period states):

$$\hat{x}_{t+h \mid t} = \left(l_t + \left(\varphi + \varphi^2 + \ldots + \varphi^h\right) b_t\right) s_{t+h-m_h^+}$$

where $h \in \{1, 2, 3, ...\}$ is the forecast horizon (how many steps ahead); $m_h^+ = ((h-1) \bmod m) + 1$ picks the right seasonal slot for the period h steps ahead (cycles through $1, 2, ..., m$).

Step 2 — apply at $t = 1$

$$\mu_1 = (12 + 0.8 \cdot 0.5) \cdot 1.2 = 14.88$$

$$\varepsilon_1 = (x_1 - \mu_1)/\mu_1 = (12 - 14.88)/14.88 = -0.1935$$

$$l_1 = (12 + 0.8 \cdot 0.5)(1 + 0.5 \cdot (-0.1935)) = 11.2$$

$$b_1 = 0.8 \cdot 0.5 + 0.4(12 + 0.8 \cdot 0.5) \cdot (-0.1935) = -0.56$$

$$s_1 = 1.2(1 + 0.2 \cdot (-0.1935)) = 1.1535$$

Step 3 — iterate

Each column header is the equation that produced its values. Values rounded to 4 decimal places; arithmetic performed at full precision.

t	x_t	$\mu_t = (l_{t-1} + \varphi b_{t-1}) s_{t-m}$	ε_t	$l_t = (l_{t-1} + \varphi b_{t-1})(1 + \alpha\varepsilon_t)$	$b_t = \varphi b_{t-1} + \beta(l_{t-1} + \varphi b_{t-1})\varepsilon_t$	$s_t = s_{t-m}(1 + \gamma\varepsilon_t)$
1	12	14.88	−0.1935	11.2	−0.56	1.1535
2	10	10.752	−0.0699	10.376	−0.7488	0.986
3	8	7.8216	0.0228	9.8885	−0.5098	0.8037
4	11	9.4806	0.1603	10.2403	0.1999	1.0321
5	14	11.9972	0.1669	11.2683	0.8544	1.1921
6	12	11.7847	0.0183	12.0611	0.7709	0.9896
7	9	10.1885	−0.1166	11.9383	0.0252	0.7849
8	13	12.3418	0.0533	12.2774	0.2752	1.0431
9	16	14.8979	0.074	12.9598	0.59	1.2097
10	14	13.2924	0.0532	13.7894	0.758	1.0002
11	11	11.2993	−0.0265	14.2052	0.4539	0.7807
12	15	15.1956	−0.0129	14.4745	0.2881	1.0404
13	18	17.7887	0.0119	14.7924	0.3004	1.2126
14	16	15.0349	0.0642	15.5151	0.6263	1.013
15	13	12.5045	0.0396	16.3335	0.7549	0.7869
16	17	17.6212	−0.0353	16.6388	0.3651	1.033

160.26. M-M-N

Mult. trend, mult. errors

$$\mathrm{ETS}(M, M, N)$$

$$x_t = l_{t-1} b_{t-1} (1 + \varepsilon_t)$$

$$l_t = l_{t-1} b_{t-1} (1 + \alpha \varepsilon_t)$$

$$b_t = b_{t-1} (1 + \beta \varepsilon_t)$$

$$\hat{x}_{t+h \mid t} = l_t b_t^h$$

Example: $\mathrm{ETS}(M, M, N)$

Given

- Smoothing parameters: $\alpha = 0.5$, $\beta = 0.4$
- Initial states: $l_0 = 12$, $b_0 = 1$
- Data:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
x_t	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

Step 1 — formula

Observation:

$$x_t = (l_{t-1} b_{t-1})(1 + \varepsilon_t)$$

Conditional mean (one-step-ahead forecast $\hat{x}_{t \mid t-1} = \mu_t$):

$$\mu_t = l_{t-1} b_{t-1}$$

Innovation:

$$\varepsilon_t = (x_t - \mu_t)/\mu_t$$

State updates:

$$l_t = l_{t-1} b_{t-1} (1 + \alpha \varepsilon_t)$$

$$b_t = b_{t-1} (1 + \beta \varepsilon_t)$$

Forecast h steps ahead from time t (using current-period states):

$$\hat{x}_{t+h \mid t} = l_t b_t^h$$

where $h \in \{1, 2, 3, ...\}$ is the forecast horizon (how many steps ahead).

Step 2 — apply at $t = 1$

$$\mu_1 = 12 \cdot 1 = 12$$

$$\varepsilon_1 = (x_1 - \mu_1)/\mu_1 = (12 - 12)/12 = 0$$

$$l_1 = 12 \cdot 1(1 + 0.5 \cdot 0) = 12$$

$$b_1 = 1(1 + 0.4 \cdot 0) = 1$$

Step 3 — iterate

Each column header is the equation that produced its values. Values rounded to 4 decimal places; arithmetic performed at full precision.

t	x_t	$\mu_t = l_{t-1} b_{t-1}$	ε_t	$l_t = l_{t-1} b_{t-1}(1+\alpha\varepsilon_t)$	$b_t = b_{t-1}(1+\beta\varepsilon_t)$
1	12	12	0	12	1
2	10	12	−0.1667	11	0.9333
3	8	10.2667	−0.2208	9.1333	0.8509
4	11	7.7716	0.4154	9.3858	0.9923
5	14	9.3135	0.5032	11.6568	1.192
6	12	13.8951	−0.1364	12.9476	1.127
7	9	14.5918	−0.3832	11.7959	0.9542
8	13	11.2561	0.1549	12.1281	1.0134
9	16	12.2903	0.3018	14.1451	1.1357
10	14	16.065	−0.1285	15.0325	1.0773
11	11	16.195	−0.3208	13.5975	0.9391
12	15	12.7694	0.1747	13.8847	1.0047
13	18	13.9502	0.2903	15.9751	1.1214
14	16	17.9143	−0.1069	16.9571	1.0735
15	13	18.2027	−0.2858	15.6014	0.9507
16	17	14.8327	0.1461	15.9163	1.0063

160.27. M-M-A

Mult. trend + add. season., mult. errors

$$\mathrm{ETS}(M, M, A)$$

$$x_t = (l_{t-1} b_{t-1} + s_{t-m})(1 + \varepsilon_t)$$

$$l_t = l_{t-1} b_{t-1} + \alpha \mu_t \varepsilon_t$$

$$b_t = b_{t-1} + \beta \mu_t \varepsilon_t / l_{t-1}$$

$$s_t = s_{t-m} + \gamma \mu_t \varepsilon_t$$

$$\hat{x}_{t+h \mid t} = l_t b_t^h + s_{t+h-m_h^+}$$

where

$$\mu_t = l_{t-1} b_{t-1} + s_{t-m}$$

Example: $\mathrm{ETS}(M, M, A)$

Given

- Smoothing parameters: $\alpha = 0.5$, $\beta = 0.4$, $\gamma = 0.2$
- Initial states: $l_0 = 12$, $b_0 = 1$, $(s_{-3}, s_{-2}, s_{-1}, s_0) = (2, 0, -3, 1)$, seasonal period $m = 4$
- Data:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
x_t	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

Step 1 — formula

Observation:

$$x_t = (l_{t-1} b_{t-1} + s_{t-m})(1 + \varepsilon_t)$$

Conditional mean (one-step-ahead forecast $\hat{x}_{t \mid t-1} = \mu_t$):

$$\mu_t = l_{t-1} b_{t-1} + s_{t-m}$$

Innovation:

$$\varepsilon_t = (x_t - \mu_t)/\mu_t$$

State updates:

$$l_t = l_{t-1} b_{t-1} + \alpha \mu_t \varepsilon_t$$

$$b_t = b_{t-1} + \beta \mu_t \varepsilon_t / l_{t-1}$$

$$s_t = s_{t-m} + \gamma \mu_t \varepsilon_t$$

Forecast h steps ahead from time t (using current-period states):

$$\hat{x}_{t+h \mid t} = l_t b_t^h + s_{t+h-m_h^+}$$

where $h \in \{1, 2, 3, \ldots\}$ is the forecast horizon (how many steps ahead); $m_h^+ = ((h-1) \bmod m) + 1$ picks the right seasonal slot for the period h steps ahead (cycles through $1, 2, \ldots, m$).

Step 2 — apply at $t = 1$

$$\mu_1 = 12 \cdot 1 + 2 = 14$$

$$\varepsilon_1 = (x_1 - \mu_1)/\mu_1 = (12 - 14)/14 = -0.1429$$

$$l_1 = 12 \cdot 1 + 0.5 \cdot 14 \cdot (-0.1429) = 11$$

$$b_1 = 1 + 0.4 \cdot 14 \cdot (-0.1429)/12 = 0.9333$$

$$s_1 = 2 + 0.2 \cdot 14 \cdot (-0.1429) = 1.6$$

Step 3 — iterate

Each column header is the equation that produced its values. Values rounded to 4 decimal places; arithmetic performed at full precision.

t	x_t	$\mu_t = l_{t-1} b_{t-1} + s_{t-m}$	ε_t	$l_t = l_{t-1} b_{t-1} + \alpha \mu_t \varepsilon_t$	$b_t = b_{t-1} + \beta \mu_t \varepsilon_t / l_{t-1}$	$s_t = s_{t-m} + \gamma \mu_t \varepsilon_t$
1	12	14	−0.1429	11	0.9333	1.6
2	10	10.2667	−0.026	10.1333	0.9236	−0.0533

3	8	6.3595	0.258	10.1798	0.9884	−2.6719
4	11	11.0616	−0.0056	10.0308	0.986	0.9877
5	14	11.4901	0.2184	11.145	1.0861	2.102
6	12	12.0509	−0.0042	12.0788	1.0842	−0.0635
7	9	10.4243	−0.1366	12.3841	1.0371	−2.9568
8	13	13.8308	−0.0601	12.4277	1.0102	0.8215
9	16	14.6569	0.0916	13.2265	1.0535	2.3706
10	14	13.8701	0.0094	13.9985	1.0574	−0.0375
11	11	11.8452	−0.0714	14.3794	1.0332	−3.1258
12	15	15.6789	−0.0433	14.5179	1.0144	0.6857
13	18	17.097	0.0528	15.1779	1.0392	2.5512
14	16	15.7359	0.0168	15.9055	1.0462	0.0153
15	13	13.5145	−0.0381	16.383	1.0333	−3.2287
16	17	17.6137	−0.0348	16.6211	1.0183	0.563

160.28. M-M-M

Fully multiplicative

$$\mathrm{ETS}(M, M, M)$$

$$x_t = l_{t-1} b_{t-1} s_{t-m}(1 + \varepsilon_t)$$

$$l_t = l_{t-1} b_{t-1}(1 + \alpha\varepsilon_t)$$

$$b_t = b_{t-1}(1 + \beta\varepsilon_t)$$

$$s_t = s_{t-m}(1 + \gamma\varepsilon_t)$$

$$\hat{x}_{t+h \mid t} = l_t b_t^h s_{t+h-m_h^+}$$

Example: $\mathrm{ETS}(M, M, M)$

Given

- Smoothing parameters: $\alpha = 0.5$, $\beta = 0.4$, $\gamma = 0.2$
- Initial states: $l_0 = 12$, $b_0 = 1$, $(s_{-3}, s_{-2}, s_{-1}, s_0) = (1.2, 1, 0.8, 1)$, seasonal period $m = 4$
- Data:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
x_t	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

Step 1 — formula

Observation:

$$x_t = (l_{t-1} b_{t-1} s_{t-m})(1 + \varepsilon_t)$$

Conditional mean (one-step-ahead forecast $\hat{x}_{t \mid t-1} = \mu_t$):

$$\mu_t = l_{t-1} b_{t-1} s_{t-m}$$

Innovation:

$$\varepsilon_t = (x_t - \mu_t)/\mu_t$$

State updates:

$$l_t = l_{t-1} b_{t-1} (1 + \alpha \varepsilon_t)$$

$$b_t = b_{t-1} (1 + \beta \varepsilon_t)$$

$$s_t = s_{t-m} (1 + \gamma \varepsilon_t)$$

Forecast h steps ahead from time t (using current-period states):

$$\hat{x}_{t+h \mid t} = l_t b_t^h s_{t+h-m_h^+}$$

where $h \in \{1, 2, 3, ...\}$ is the forecast horizon (how many steps ahead); $m_h^+ = ((h-1) \bmod m) + 1$ picks the right seasonal slot for the period h steps ahead (cycles through $1, 2, ..., m$).

Step 2 — apply at $t = 1$

$$\mu_1 = 12 \cdot 1 \cdot 1.2 = 14.4$$

$$\varepsilon_1 = (x_1 - \mu_1)/\mu_1 = (12 - 14.4)/14.4 = -0.1667$$

$$l_1 = 12 \cdot 1 (1 + 0.5 \cdot (-0.1667)) = 11$$

$$b_1 = 1 (1 + 0.4 \cdot (-0.1667)) = 0.9333$$

$$s_1 = 1.2 (1 + 0.2 \cdot (-0.1667)) = 1.16$$

Step 3 — iterate

Each column header is the equation that produced its values. Values rounded to 4 decimal places; arithmetic performed at full precision.

t	x_t	$\mu_t = l_{t-1} b_{t-1} s_{t-m}$	ε_t	$l_t = l_{t-1} b_{t-1} (1 + \alpha \varepsilon_t)$	$b_t = b_{t-1} (1 + \beta \varepsilon_t)$	$s_t = s_{t-m} (1 + \gamma \varepsilon_t)$
1	12	14.4	−0.1667	11	0.9333	1.16
2	10	10.2667	−0.026	10.1333	0.9236	0.9948
3	8	7.4876	0.0684	9.6798	0.9489	0.8109
4	11	9.1853	0.1976	10.0927	1.0239	1.0395
5	14	11.9874	0.1679	11.2015	1.0927	1.199
6	12	12.1759	−0.0144	12.1511	1.0864	0.9919
7	9	10.7049	−0.1593	12.1493	1.0172	0.7851

8	13	12.8459	0.012	12.4317	1.022	1.042
9	16	15.2334	0.0503	13.0253	1.0426	1.211
10	14	13.4706	0.0393	13.8471	1.059	0.9997
11	11	11.5129	−0.0446	14.3373	1.0401	0.7781
12	15	15.5389	−0.0347	14.6539	1.0257	1.0348
13	18	18.202	−0.0111	14.9469	1.0211	1.2083
14	16	15.2587	0.0486	15.6336	1.041	1.0094
15	13	12.6634	0.0266	16.4906	1.052	0.7823
16	17	17.9523	−0.053	16.8888	1.0297	1.0238

160.29. M-Md-N

Damped mult. trend, mult. errors

$$\mathrm{ETS}(M, \mathrm{Md}, N)$$

$$x_t = l_{t-1} b_{t-1}^{\varphi} (1 + \varepsilon_t)$$

$$l_t = l_{t-1} b_{t-1}^{\varphi} (1 + \alpha \varepsilon_t)$$

$$b_t = b_{t-1}^{\varphi} (1 + \beta \varepsilon_t)$$

$$\hat{x}_{t+h \mid t} = l_t b_t^{\varphi + \varphi^2 + \ldots + \varphi^h}$$

Example: $\mathrm{ETS}(M, \mathrm{Md}, N)$

Given

- Smoothing parameters: $\alpha = 0.5$, $\beta = 0.4$, $\varphi = 0.8$
- Initial states: $l_0 = 12$, $b_0 = 1$
- Data:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
x_t	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

Step 1 — formula

Observation:

$$x_t = (l_{t-1} b_{t-1}^{\varphi})(1 + \varepsilon_t)$$

Conditional mean (one-step-ahead forecast $\hat{x}_{t \mid t-1} = \mu_t$):

$$\mu_t = l_{t-1} b_{t-1}^{\varphi}$$

Innovation:

$$\varepsilon_t = (x_t - \mu_t)/\mu_t$$

State updates:

$$l_t = l_{t-1} b_{t-1}^{\varphi}(1 + \alpha\varepsilon_t)$$

$$b_t = b_{t-1}^{\varphi}(1 + \beta\varepsilon_t)$$

Forecast h steps ahead from time t (using current-period states):

$$\hat{x}_{t+h \mid t} = l_t b_t^{\varphi+\varphi^2+\ldots+\varphi^h}$$

where $h \in \{1, 2, 3, \ldots\}$ is the forecast horizon (how many steps ahead).

Step 2 — apply at $t = 1$

$$\mu_1 = 12 \cdot 1^{0.8} = 12$$

$$\varepsilon_1 = (x_1 - \mu_1)/\mu_1 = (12 - 12)/12 = 0$$

$$l_1 = 12 \cdot 1^{0.8}(1 + 0.5 \cdot 0) = 12$$

$$b_1 = 1^{0.8}(1 + 0.4 \cdot 0) = 1$$

Step 3 — iterate

Each column header is the equation that produced its values. Values rounded to 4 decimal places; arithmetic performed at full precision.

t	x_t	$\mu_t = l_{t-1} b_{t-1}^{\varphi}$	ε_t	$l_t = l_{t-1} b_{t-1}^{\varphi}(1 + \alpha\varepsilon_t)$	$b_t = b_{t-1}^{\varphi}(1 + \beta\varepsilon_t)$
1	12	12	0	12	1
2	10	12	−0.1667	11	0.9333
3	8	10.4093	−0.2315	9.2047	0.8587
4	11	8.1485	0.3499	9.5742	1.0092
5	14	9.6444	0.4516	11.8222	1.1893
6	12	13.581	−0.1164	12.7905	1.0953
7	9	13.7565	−0.3458	11.3782	0.9268
8	13	10.7066	0.2142	11.8533	1.0216
9	16	12.0577	0.327	14.0288	1.1503
10	14	15.6915	−0.1078	14.8457	1.0703
11	11	15.6748	−0.2982	13.3374	0.9299
12	15	12.5839	0.192	13.792	1.016
13	18	13.9678	0.2887	15.9839	1.1297
14	16	17.6219	−0.092	16.8109	1.0619
15	13	17.6382	−0.263	15.3191	0.9388
16	17	14.565	0.1672	15.7825	1.0144

160.30. M-Md-A

Damped mult. trend + add. season., mult. errors

$$\mathrm{ETS}(M, \mathrm{Md}, A)$$

$$x_t = (l_{t-1} b_{t-1}^{\varphi} + s_{t-m})(1 + \varepsilon_t)$$

$$l_t = l_{t-1} b_{t-1}^{\varphi} + \alpha \mu_t \varepsilon_t$$

$$b_t = b_{t-1}^{\varphi} + \beta \mu_t \varepsilon_t / l_{t-1}$$

$$s_t = s_{t-m} + \gamma \mu_t \varepsilon_t$$

$$\hat{x}_{t+h \mid t} = l_t b_t^{\varphi + \varphi^2 + \ldots + \varphi^h} + s_{t+h-m_h^+}$$

where

$$\mu_t = l_{t-1} b_{t-1}^{\varphi} + s_{t-m}$$

Example: $\mathrm{ETS}(M, \mathrm{Md}, A)$

Given

- Smoothing parameters: $\alpha = 0.5$, $\beta = 0.4$, $\varphi = 0.8$, $\gamma = 0.2$
- Initial states: $l_0 = 12$, $b_0 = 1$, $(s_{-3}, s_{-2}, s_{-1}, s_0) = (2, 0, -3, 1)$, seasonal period $m = 4$
- Data:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
x_t	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

Step 1 — formula

Observation:

$$x_t = (l_{t-1} b_{t-1}^{\varphi} + s_{t-m})(1 + \varepsilon_t)$$

Conditional mean (one-step-ahead forecast $\hat{x}_{t \mid t-1} = \mu_t$):

$$\mu_t = l_{t-1} b_{t-1}^{\varphi} + s_{t-m}$$

Innovation:

$$\varepsilon_t = (x_t - \mu_t)/\mu_t$$

State updates:

$$l_t = l_{t-1} b_{t-1}^{\varphi} + \alpha \mu_t \varepsilon_t$$

$$b_t = b_{t-1}^{\varphi} + \beta \mu_t \varepsilon_t / l_{t-1}$$

$$s_t = s_{t-m} + \gamma \mu_t \varepsilon_t$$

Forecast h steps ahead from time t (using current-period states):

$$\hat{x}_{t+h \mid t} = l_t b_t^{\varphi + \varphi^2 + \ldots + \varphi^h} + s_{t+h-m_h^+}$$

where $h \in \{1, 2, 3, ...\}$ is the forecast horizon (how many steps ahead); $m_h^+ = ((h-1) \bmod m) + 1$ picks the right seasonal slot for the period h steps ahead (cycles through $1, 2, ..., m$).

Step 2 — apply at $t = 1$

$$\mu_1 = 12 \cdot 1^{0.8} + 2 = 14$$

$$\varepsilon_1 = (x_1 - \mu_1)/\mu_1 = (12 - 14)/14 = -0.1429$$

$$l_1 = 12 \cdot 1^{0.8} + 0.5 \cdot 14 \cdot (-0.1429) = 11$$

$$b_1 = 1^{0.8} + 0.4 \cdot 14 \cdot (-0.1429)/12 = 0.9333$$

$$s_1 = 2 + 0.2 \cdot 14 \cdot (-0.1429) = 1.6$$

Step 3 — iterate

Each column header is the equation that produced its values. Values rounded to 4 decimal places; arithmetic performed at full precision.

t	x_t	$\mu_t = l_{t-1} b_{t-1}^{\varphi} + s_{t-m}$	ε_t	$l_t = l_{t-1} b_{t-1}^{\varphi} + \alpha \mu_t \varepsilon_t$	$b_t = b_{t-1}^{\varphi} + \beta \mu_t \varepsilon_t / l_{t-1}$	$s_t = s_{t-m} + \gamma \mu_t \varepsilon_t$
1	12	14	−0.1429	11	0.9333	1.6
2	10	10.4093	−0.0393	10.2047	0.9314	−0.0819
3	8	6.6408	0.2047	10.3204	0.998	−2.7282
4	11	11.3041	−0.0269	10.152	0.9866	0.9392
5	14	11.6433	0.2024	11.2217	1.0821	2.0713
6	12	11.8714	0.0108	12.0176	1.0698	−0.0561
7	9	9.9557	−0.096	12.206	1.0236	−2.9193
8	13	13.3754	−0.0281	12.2485	1.0066	0.8641
9	16	14.3841	0.1123	13.1207	1.058	2.3945
10	14	13.6701	0.0241	13.8912	1.0562	0.0098
11	11	11.5931	−0.0512	14.2158	1.0276	−3.0379
12	15	15.3934	−0.0256	14.3326	1.011	0.7854
13	18	16.8529	0.0681	15.0319	1.0408	2.6239
14	16	15.5303	0.0302	15.7553	1.045	0.1038
15	13	13.2821	−0.0212	16.179	1.0287	−3.0943
16	17	17.3345	−0.0193	16.3818	1.0146	0.7185

160.31. M-Md-M

Fully multiplicative with damped trend

$$\mathrm{ETS}(M, \mathrm{Md}, M)$$

$$x_t = l_{t-1} b_{t-1}^{\varphi} s_{t-m} (1 + \varepsilon_t)$$

$$l_t = l_{t-1} b_{t-1}^{\varphi} (1 + \alpha \varepsilon_t)$$

$$b_t = b_{t-1}^{\varphi} (1 + \beta \varepsilon_t)$$

$$s_t = s_{t-m} (1 + \gamma \varepsilon_t)$$

$$\hat{x}_{t+h \mid t} = l_t b_t^{\varphi + \varphi^2 + \ldots + \varphi^h} s_{t+h-m_h^+}$$

Example: $\mathrm{ETS}(M, \mathrm{Md}, M)$

Given

- Smoothing parameters: $\alpha = 0.5$, $\beta = 0.4$, $\varphi = 0.8$, $\gamma = 0.2$
- Initial states: $l_0 = 12$, $b_0 = 1$, $(s_{-3}, s_{-2}, s_{-1}, s_0) = (1.2, 1, 0.8, 1)$, seasonal period $m = 4$
- Data:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
x_t	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

Step 1 — formula

Observation:

$$x_t = (l_{t-1} b_{t-1}^{\varphi} s_{t-m})(1 + \varepsilon_t)$$

Conditional mean (one-step-ahead forecast $\hat{x}_{t \mid t-1} = \mu_t$):

$$\mu_t = l_{t-1} b_{t-1}^{\varphi} s_{t-m}$$

Innovation:

$$\varepsilon_t = (x_t - \mu_t)/\mu_t$$

State updates:

$$l_t = l_{t-1} b_{t-1}^{\varphi} (1 + \alpha \varepsilon_t)$$

$$b_t = b_{t-1}^{\varphi} (1 + \beta \varepsilon_t)$$

$$s_t = s_{t-m} (1 + \gamma \varepsilon_t)$$

Forecast h steps ahead from time t (using current-period states):

$$\hat{x}_{t+h \mid t} = l_t b_t^{\varphi + \varphi^2 + \ldots + \varphi^h} s_{t+h-m_h^+}$$

where $h \in \{1, 2, 3, \ldots\}$ is the forecast horizon (how many steps ahead); $m_h^+ = ((h-1) \bmod m) + 1$ picks the right seasonal slot for the period h steps ahead (cycles through $1, 2, \ldots, m$).

Step 2 — apply at $t = 1$

$$\mu_1 = 12 \cdot 1^{0.8} \cdot 1.2 = 14.4$$

$$\varepsilon_1 = (x_1 - \mu_1)/\mu_1 = (12 - 14.4)/14.4 = -0.1667$$

$$l_1 = 12 \cdot 1^{0.8}(1 + 0.5 \cdot (-0.1667)) = 11$$

$$b_1 = 1^{0.8}(1 + 0.4 \cdot (-0.1667)) = 0.9333$$

$$s_1 = 1.2(1 + 0.2 \cdot (-0.1667)) = 1.16$$

Step 3 — iterate

Each column header is the equation that produced its values. Values rounded to 4 decimal places; arithmetic performed at full precision.

t	x_t	$\mu_t = l_{t-1}b_{t-1}^{\varphi}s_{t-m}$	ε_t	$l_t = l_{t-1}b_{t-1}^{\varphi}(1+\alpha\varepsilon_t)$	$b_t = b_{t-1}^{\varphi}(1+\beta\varepsilon_t)$	$s_t = s_{t-m}(1+\gamma\varepsilon_t)$
1	12	14.4	−0.1667	11	0.9333	1.16
2	10	10.4093	−0.0393	10.2047	0.9314	0.9921
3	8	7.7127	0.0373	9.8204	0.9588	0.806
4	11	9.4956	0.1584	10.2478	1.0282	1.0317
5	14	12.1549	0.1518	11.2737	1.0846	1.1952
6	12	11.9357	0.0054	12.0627	1.0694	0.9932
7	9	10.2583	−0.1227	11.9474	1.0034	0.7862
8	13	12.3594	0.0518	12.2902	1.0235	1.0424
9	16	14.965	0.0692	12.9537	1.0469	1.2118
10	14	13.3466	0.049	13.7668	1.0577	1.0029
11	11	11.32	−0.0283	14.1951	1.0341	0.7817
12	15	15.1986	−0.0131	14.4854	1.0218	1.0397
13	18	17.858	0.008	14.796	1.0206	1.2137
14	16	15.0837	0.0607	15.4965	1.0412	1.0151
15	13	12.5116	0.039	16.3171	1.0489	0.7878
16	17	17.6251	−0.0355	16.6521	1.0242	1.0323

161. ARIMA Family

161.1. Summary

161.1.1. ARIMA family (Box-Jenkins models)

■ AR $\varphi(B)$ ■ MA $\theta(B)$ ■ differencing $(1-B)^d$

Each model is written using the *backshift operator* B, where $Bx_t = x_{t-1}$ and $B^k x_t = x_{t-k}$. The compact form

$$\varphi(B)(1-B)^d x_t = c + \theta(B)\varepsilon_t$$

expresses ARIMA as a product of three operators acting on the series: the AR polynomial, the differencing operator, and the MA polynomial. ε_t is white noise with mean zero and variance σ^2.

For multivariate models, $\boldsymbol{x}_t \in \mathbb{R}^K$ is a vector of K time series, and operator coefficients become $K \times K$ matrices.

161.1.1.1. Univariate models

$\mathrm{AR}(p)$

Autoregressive

$$\varphi(B)x_t = c + \varepsilon_t$$

$$\varphi(B) = 1 - \varphi_1 B - \varphi_2 B^2 - \ldots - \varphi_p B^p$$

i.e., $x_t = c + \varphi_1 x_{t-1} + \ldots + \varphi_p x_{t-p} + \varepsilon_t$
Stationary if all roots of $\varphi(B) = 0$ lie outside the unit circle.

Parameters: $\varphi_1, \ldots, \varphi_p, c, \sigma^2$
Orders: p

$\mathrm{MA}(q)$

Moving average

$$x_t = c + \theta(B)\varepsilon_t$$

$$\theta(B) = 1 + \theta_1 B + \theta_2 B^2 + \ldots + \theta_q B^q$$

i.e.,

$$x_t = c + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \ldots + \theta_q \varepsilon_{t-q}$$

Always stationary. Invertible if roots of $\theta(B) = 0$ lie outside the unit circle.

Parameters: $\theta_1, \ldots, \theta_q, c, \sigma^2$
Orders: q

$\mathrm{ARMA}(p, q)$

Autoregressive moving average

$$\varphi(B)x_t = c + \theta(B)\varepsilon_t$$

$$\varphi(B) = 1 - \varphi_1 B - \ldots - \varphi_p B^p$$

$$\theta(B) = 1 + \theta_1 B + \ldots + \theta_q B^q$$

Combines AR and MA structure. Requires stationary input series.

Parameters: $\varphi_1, \ldots, \varphi_p, \theta_1, \ldots, \theta_q, c, \sigma^2$
Orders: p, q

$\mathrm{ARIMA}(p, d, q)$

Autoregressive integrated moving average

$$\varphi(B)(1 - B)^d x_t = c + \theta(B)\varepsilon_t$$

$$\varphi(B) = 1 - \varphi_1 B - \ldots - \varphi_p B^p$$
$$\theta(B) = 1 + \theta_1 B + \ldots + \theta_q B^q$$

Differencing operator $(1 - B)y_t = y_t - y_{t-1}$ removes trend.
Apply d times to make non-stationary series stationary.

Parameters: $\varphi_1, \ldots, \varphi_p, \theta_1, \ldots, \theta_q, c, \sigma^2$
Orders: p, d, q

$\mathrm{SARIMA}(p, d, q)(P, D, Q)_m$

Seasonal ARIMA

$$\varphi(B)\Phi(B^m)(1 - B)^d(1 - B^m)^D x_t = c + \theta(B)\Theta(B^m)\varepsilon_t$$

$$\Phi(B^m) = 1 - \Phi_1 B^m - \ldots - \Phi_P B^{Pm}$$
$$\Theta(B^m) = 1 + \Theta_1 B^m + \ldots + \Theta_Q B^{Qm}$$

Seasonal differencing $(1 - B^m)y_t = y_t - y_{t-m}$ removes annual/periodic patterns. Period m (e.g., 12 for monthly).

Parameters: $\varphi_1, \ldots, \varphi_p, \Phi_1, \ldots, \Phi_P, \theta_1, \ldots, \theta_q, \Theta_1, \ldots, \Theta_Q, c, \sigma^2$
Orders: p, P, d, D, q, Q, m

$\mathrm{SARIMAX}(p, d, q)(P, D, Q)_m$

Seasonal ARIMA with exogenous regressors

$$\varphi(B)\Phi(B^m)(1 - B)^d(1 - B^m)^D y_t$$
$$= c + \sum_{i=1}^{k} \beta_i x_{i,t} + \theta(B)\Theta(B^m)\varepsilon_t$$

y_t = endogenous (target) series $x_{i,t}$ = exogenous regressors, $i = 1, \ldots, k$ β_i = regression coefficients on exogenous variables The X adds linear regression on external predictors to SARIMA.

Parameters: $\varphi_1, \ldots, \varphi_p, \Phi_1, \ldots, \Phi_P, \theta_1, \ldots, \theta_q, \Theta_1, \ldots, \Theta_Q, \beta_1, \ldots, \beta_k, c, \sigma^2$
Orders: p, P, d, D, q, Q, m, k (regressors)

161.1.1.2. Multivariate (vector) models

For all models below, $\boldsymbol{x}_t = (x_{1,t}, ..., x_{K,t})^\top \in \mathbb{R}^K$ is a vector of K jointly modeled time series. Operator coefficients $\boldsymbol{\Phi}_i$ and $\boldsymbol{\Theta}_j$ are $K \times K$ matrices, and ε_t is multivariate white noise with covariance matrix $\boldsymbol{\Sigma} \in \mathbb{R}^{K \times K}$.

$\mathrm{VAR}(p)$

Vector autoregression

$$\boldsymbol{\Phi}(B)\boldsymbol{x}_t = \boldsymbol{c} + \varepsilon_t$$

$$\boldsymbol{\Phi}(B) = \boldsymbol{I} - \boldsymbol{\Phi}_1 B - ... - \boldsymbol{\Phi}_p B^p$$

i.e.,

$$\boldsymbol{x}_t = \boldsymbol{c} + \boldsymbol{\Phi}_1 \boldsymbol{x}_{t-1} + ... + \boldsymbol{\Phi}_p \boldsymbol{x}_{t-p} + \varepsilon_t$$

Each component depends on p lags of all K series.

Parameters: $\boldsymbol{\Phi}_1, ..., \boldsymbol{\Phi}_p$ (K^2 each), $\boldsymbol{c}$ (K), $\boldsymbol{\Sigma}$ ($K(K+1)/2$)
Orders: p, K (series)

$\mathrm{VMA}(q)$

Vector moving average

$$\boldsymbol{x}_t = \boldsymbol{c} + \boldsymbol{\Theta}(B)\varepsilon_t$$

$$\boldsymbol{\Theta}(B) = \boldsymbol{I} + \boldsymbol{\Theta}_1 B + ... + \boldsymbol{\Theta}_q B^q$$

i.e., $\boldsymbol{x}_t = \boldsymbol{c} + \varepsilon_t + \boldsymbol{\Theta}_1 \varepsilon_{t-1} + ... + \boldsymbol{\Theta}_q \varepsilon_{t-q}$

Each component is a linear combination of past shocks across all series.

Parameters: $\boldsymbol{\Theta}_1, ..., \boldsymbol{\Theta}_q$ (K^2 each), $\boldsymbol{c}$ (K), $\boldsymbol{\Sigma}$ ($K(K+1)/2$)
Orders: q, K (series)

$\mathrm{VARMA}(p, q)$

Vector autoregressive moving average

$$\boldsymbol{\Phi}(B)\boldsymbol{x}_t = \boldsymbol{c} + \boldsymbol{\Theta}(B)\varepsilon_t$$

$$\boldsymbol{\Phi}(B) = \boldsymbol{I} - \boldsymbol{\Phi}_1 B - ... - \boldsymbol{\Phi}_p B^p$$
$$\boldsymbol{\Theta}(B) = \boldsymbol{I} + \boldsymbol{\Theta}_1 B + ... + \boldsymbol{\Theta}_q B^q$$

Vector analog of ARMA. Identifiability requires care (e.g., echelon forms).

Parameters: $\boldsymbol{\Phi}_1, ..., \boldsymbol{\Phi}_p, \boldsymbol{\Theta}_1, ..., \boldsymbol{\Theta}_q, \boldsymbol{c}, \boldsymbol{\Sigma}$
Orders: p, q, K

$\mathrm{VARIMA}(p, d, q)$

Vector ARIMA

$$\boldsymbol{\Phi}(B)(1-B)^d \boldsymbol{x}_t = \boldsymbol{c} + \boldsymbol{\Theta}(B)\varepsilon_t$$

Differencing applied component-wise to each of the K series.
In practice, cointegrated systems use VECM (Vector Error Correction) rather than VARIMA — VECM models the long-run equilibrium directly.

Parameters: $\boldsymbol{\Phi}_1, ..., \boldsymbol{\Phi}_p, \boldsymbol{\Theta}_1, ..., \boldsymbol{\Theta}_q, \boldsymbol{c}, \boldsymbol{\Sigma}$
Orders: p, d, q, K

$\mathrm{VARIMAX}(p, d, q)$

Vector ARIMA with exogenous regressors

$$\boldsymbol{\Phi}(B)(1-B)^d \boldsymbol{y}_t = \boldsymbol{c} + \boldsymbol{B}\boldsymbol{x}_t + \boldsymbol{\Theta}(B)\varepsilon_t$$

$\boldsymbol{y}_t \in \mathbb{R}^K$ = endogenous vector series
$\boldsymbol{x}_t \in \mathbb{R}^k$ = exogenous regressor vector
$\boldsymbol{B} \in \mathbb{R}^{K \times k}$ = matrix of regression coefficients
Each of the K series can depend on the same set of k exogenous variables.

Parameters: $\boldsymbol{\Phi}_1, ..., \boldsymbol{\Phi}_p, \boldsymbol{\Theta}_1, ..., \boldsymbol{\Theta}_q, \boldsymbol{B}$ ($K \times k$), $\boldsymbol{c}, \boldsymbol{\Sigma}$
Orders: p, d, q, K, k (regressors)

161.2. AR

Autoregressive

$\mathrm{AR}(p)$

$$\varphi(B)x_t = c + \varepsilon_t$$

$$\varphi(B) = 1 - \varphi_1 B - \varphi_2 B^2 - \ldots - \varphi_p B^p$$

i.e.,

$$x_t = c + \varphi_1 x_{t-1} + \ldots + \varphi_p x_{t-p} + \varepsilon_t$$

Stationary if all roots of $\varphi(B) = 0$ lie outside the unit circle

Parameters: $\varphi_1, \ldots, \varphi_p$, c, σ^2
Orders: p

Example: AR(1)

Given

- Order: $p = 1$
- Parameters: $\varphi_1 = 0.5$, $c = 0$
- Data:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
x_t	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

Step 1 — formula

Substitute $p = 1$ into the AR(p) recursion:

$$x_t = c + \varphi_1 x_{t-1} + \varepsilon_t$$

Forecast (set $\varepsilon_t = 0$):

$$\hat{x}_t = c + \varphi_1 x_{t-1}$$

Innovation:

$$\varepsilon_t = x_t - \hat{x}_t$$

Step 2 — apply at $t = 2$

Plug in $c = 0$, $\varphi_1 = 0.5$, and $x_1 = 12$:

$$\hat{x}_2 = 0 + 0.5 \cdot 12 = 6$$

$$\varepsilon_2 = x_2 - \hat{x}_2 = 10 - 6 = 4$$

Step 3 — iterate

t	x_t	$\hat{x}_t = 0.5 \cdot x_{t-1}$	$\varepsilon_t = x_t - \hat{x}_t$
2	10	$0.5 \cdot 12 = 6$	$10 - 6 = 4$
3	8	$0.5 \cdot 10 = 5$	$8 - 5 = 3$
4	11	$0.5 \cdot 8 = 4$	$11 - 4 = 7$
5	14	$0.5 \cdot 11 = 5.5$	$14 - 5.5 = 8.5$

6	12	$0.5 \cdot 14 = 7$	$12 - 7 = 5$
7	9	$0.5 \cdot 12 = 6$	$9 - 6 = 3$
8	13	$0.5 \cdot 9 = 4.5$	$13 - 4.5 = 8.5$
9	16	$0.5 \cdot 13 = 6.5$	$16 - 6.5 = 9.5$
10	14	$0.5 \cdot 16 = 8$	$14 - 8 = 6$
11	11	$0.5 \cdot 14 = 7$	$11 - 7 = 4$
12	15	$0.5 \cdot 11 = 5.5$	$15 - 5.5 = 9.5$
13	18	$0.5 \cdot 15 = 7.5$	$18 - 7.5 = 10.5$
14	16	$0.5 \cdot 18 = 9$	$16 - 9 = 7$
15	13	$0.5 \cdot 16 = 8$	$13 - 8 = 5$
16	17	$0.5 \cdot 13 = 6.5$	$17 - 6.5 = 10.5$
17	—	$0.5 \cdot 17 = 8.5$	—

Residuals are large because AR(1) cannot capture the trend or seasonality in x — that is the role of ARIMA / SARIMA / ETS, applied to the **same** data in their own examples.

161.3. MA

Moving average

$$\mathrm{MA}(q)$$

$$x_t = c + \theta(B)\varepsilon_t$$

$$\theta(B) = 1 + \theta_1 B + \theta_2 B^2 + ... + \theta_q B^q$$

i.e.,

$$x_t = c + \varepsilon_t + \theta_1 \varepsilon_{t-1} + ... + \theta_q \varepsilon_{t-q}$$

Always stationary. Invertible if roots of $\theta(B) = 0$ lie outside the unit circle.

Parameters: $\theta_1, ..., \theta_q$, c, σ^2
Orders: q

Example: $\mathrm{MA}(1)$

Given

- Order: $q = 1$
- Parameters: $\theta_1 = 0.5$, $c = 0$
- Initial innovation: $\varepsilon_0 = 0$
- Data:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

x_t	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

Step 1 — formula

Substitute $q = 1$ into the MA(q) recursion:

$$x_t = c + \varepsilon_t + \theta_1 \varepsilon_{t-1}$$

Forecast (set $\varepsilon_t = 0$):

$$\hat{x}_t = c + \theta_1 \varepsilon_{t-1}$$

Innovation:

$$\varepsilon_t = x_t - \hat{x}_t$$

Step 2 — apply at $t = 1$

Plug in $c = 0$, $\theta_1 = 0.5$, and $\varepsilon_0 = 0$:

$$\hat{x}_1 = 0 + 0.5 \cdot 0 = 0$$

$$\varepsilon_1 = x_1 - \hat{x}_1 = 12 - 0 = 12$$

Step 3 — iterate

Each row uses the **previous** innovation ε_{t-1}. Values rounded to 4 decimal places.

t	x_t	$\hat{x}_t = 0.5 \cdot \varepsilon_{t-1}$	$\varepsilon_t = x_t - \hat{x}_t$
1	12	$0.5 \cdot 0 = 0$	$12 - 0 = 12$
2	10	$0.5 \cdot 12 = 6$	$10 - 6 = 4$
3	8	$0.5 \cdot 4 = 2$	$8 - 2 = 6$
4	11	$0.5 \cdot 6 = 3$	$11 - 3 = 8$
5	14	$0.5 \cdot 8 = 4$	$14 - 4 = 10$
6	12	$0.5 \cdot 10 = 5$	$12 - 5 = 7$
7	9	$0.5 \cdot 7 = 3.5$	$9 - 3.5 = 5.5$
8	13	$0.5 \cdot 5.5 = 2.75$	$13 - 2.75 = 10.25$
9	16	$0.5 \cdot 10.25 = 5.125$	$16 - 5.125 = 10.875$
10	14	$0.5 \cdot 10.875 = 5.4375$	$14 - 5.4375 = 8.5625$
11	11	$0.5 \cdot 8.5625 = 4.2813$	$11 - 4.2813 = 6.7188$
12	15	$0.5 \cdot 6.7188 = 3.3594$	$15 - 3.3594 = 11.6406$
13	18	$0.5 \cdot 11.6406 = 5.8203$	$18 - 5.8203 = 12.1797$
14	16	$0.5 \cdot 12.1797 = 6.0898$	$16 - 6.0898 = 9.9102$
15	13	$0.5 \cdot 9.9102 = 4.9551$	$13 - 4.9551 = 8.0449$
16	17	$0.5 \cdot 8.0449 = 4.0225$	$17 - 4.0225 = 12.9775$

17	—	$0.5 \cdot 12.9775 = 6.4888$	—

Like AR(1), MA(1) cannot capture the trend or seasonality in x.

161.4. ARMA

Autoregressive moving average

$$\text{ARMA}(p, q)$$

$$\varphi(B)x_t = c + \theta(B)\varepsilon_t$$

$$\varphi(B) = 1 - \varphi_1 B - ... - \varphi_p B^p$$

$$\theta(B) = 1 + \theta_1 B + ... + \theta_q B^q$$

Combines AR and MA structure. Requires stationary input series.

Parameters: $\varphi_1, ..., \varphi_p$, $\theta_1, ..., \theta_q$, c, σ^2
Orders: p, q

Example: $\text{ARMA}(1, 1)$

Given

- Orders: $p = 1$, $q = 1$
- Parameters: $\varphi_1 = 0.5$, $\theta_1 = 0.5$, $c = 0$
- Initial conditions: $x_0 = 0$, $\varepsilon_0 = 0$
- Data:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
x_t	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

Step 1 — formula

Substitute $p = 1$, $q = 1$ into the ARMA(p, q) recursion:

$$x_t = c + \varphi_1 x_{t-1} + \varepsilon_t + \theta_1 \varepsilon_{t-1}$$

Forecast (set $\varepsilon_t = 0$):

$$\hat{x}_t = c + \varphi_1 x_{t-1} + \theta_1 \varepsilon_{t-1}$$

Innovation:

$$\varepsilon_t = x_t - \hat{x}_t$$

Step 2 — apply at $t = 1$

Plug in $c = 0$, $\varphi_1 = 0.5$, $\theta_1 = 0.5$, $x_0 = 0$, $\varepsilon_0 = 0$:

$$\hat{x}_1 = 0 + 0.5 \cdot 0 + 0.5 \cdot 0 = 0$$

$$\varepsilon_1 = x_1 - \hat{x}_1 = 12 - 0 = 12$$

Step 3 — iterate

Each row uses the **previous** observation x_{t-1} and the **previous** innovation ε_{t-1}. Values rounded to 4 decimal places.

t	x_t	$\hat{x}_t = 0.5x_{t-1} + 0.5\varepsilon_{t-1}$	$\varepsilon_t = x_t - \hat{x}_t$
1	12	$0.5 \cdot 0 + 0.5 \cdot 0 = 0$	$12 - 0 = 12$
2	10	$0.5 \cdot 12 + 0.5 \cdot 12 = 6 + 6 = 12$	$10 - 12 = -2$
3	8	$0.5 \cdot 10 + 0.5 \cdot (-2) = 5 - 1 = 4$	$8 - 4 = 4$
4	11	$0.5 \cdot 8 + 0.5 \cdot 4 = 4 + 2 = 6$	$11 - 6 = 5$
5	14	$0.5 \cdot 11 + 0.5 \cdot 5 = 5.5 + 2.5 = 8$	$14 - 8 = 6$
6	12	$0.5 \cdot 14 + 0.5 \cdot 6 = 7 + 3 = 10$	$12 - 10 = 2$
7	9	$0.5 \cdot 12 + 0.5 \cdot 2 = 6 + 1 = 7$	$9 - 7 = 2$
8	13	$0.5 \cdot 9 + 0.5 \cdot 2 = 4.5 + 1 = 5.5$	$13 - 5.5 = 7.5$
9	16	$0.5 \cdot 13 + 0.5 \cdot 7.5 = 6.5 + 3.75 = 10.25$	$16 - 10.25 = 5.75$
10	14	$0.5 \cdot 16 + 0.5 \cdot 5.75 = 8 + 2.875 = 10.875$	$14 - 10.875 = 3.125$
11	11	$0.5 \cdot 14 + 0.5 \cdot 3.125 = 7 + 1.5625 = 8.5625$	$11 - 8.5625 = 2.4375$
12	15	$0.5 \cdot 11 + 0.5 \cdot 2.4375 = 5.5 + 1.2188 = 6.7188$	$15 - 6.7188 = 8.2813$
13	18	$0.5 \cdot 15 + 0.5 \cdot 8.2813 = 7.5 + 4.1406 = 11.6406$	$18 - 11.6406 = 6.3594$
14	16	$0.5 \cdot 18 + 0.5 \cdot 6.3594 = 9 + 3.1797 = 12.1797$	$16 - 12.1797 = 3.8203$
15	13	$0.5 \cdot 16 + 0.5 \cdot 3.8203 = 8 + 1.9102 = 9.9102$	$13 - 9.9102 = 3.0898$
16	17	$0.5 \cdot 13 + 0.5 \cdot 3.0898 = 6.5 + 1.5449 = 8.0449$	$17 - 8.0449 = 8.9551$
17	—	$0.5 \cdot 17 + 0.5 \cdot 8.9551 = 8.5 + 4.4775 = 12.9775$	—

161.5. ARIMA

Autoregressive integrated moving average

$$\mathrm{ARIMA}(p, d, q)$$

$$\varphi(B)(1 - B)^d x_t = c + \theta(B)\varepsilon_t$$

$$\varphi(B) = 1 - \varphi_1 B - \ldots - \varphi_p B^p$$

$$\theta(B) = 1 + \theta_1 B + \ldots + \theta_q B^q$$

Differencing operator $(1 - B)y_t = y_t - y_{t-1}$ removes trend.
Apply d times to make non-stationary series stationary.

Parameters: $\varphi_1, \ldots, \varphi_p$, $\theta_1, \ldots, \theta_q$, c, σ^2
Orders: p, d, q

Example: $\mathrm{ARIMA}(1,1,1)$

Given

- Orders: $p = 1$, $d = 1$, $q = 1$
- Parameters: $\varphi_1 = 0.5$, $\theta_1 = 0.5$, $c = 0$
- Initial conditions: $d_1 = 0$, $\varepsilon_1 = 0$
- Data:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
x_t	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

Step 1 — formula

Substitute $p = 1$, $d = 1$, $q = 1$ into the ARIMA recursion:

$$(1 - \varphi_1 B)(1 - B)x_t = c + (1 + \theta_1 B)\varepsilon_t$$

Difference — apply $(1 - B)$ to convert x_t to a stationary series:

$$d_t := (1 - B)x_t = x_t - x_{t-1}$$

In the differenced space, the model is $\mathrm{ARMA}(1,1)$:

$$d_t = c + \varphi_1 d_{t-1} + \varepsilon_t + \theta_1 \varepsilon_{t-1}$$

Forecast the difference (set $\varepsilon_t = 0$):

$$\hat{d}_t = c + \varphi_1 d_{t-1} + \theta_1 \varepsilon_{t-1}$$

Undifference — convert the difference forecast back to a forecast for x_t:

$$\hat{x}_t = x_{t-1} + \hat{d}_t$$

Innovation:

$$\varepsilon_t = d_t - \hat{d}_t = x_t - \hat{x}_t$$

Step 2 — apply at $t = 2$

First difference: $d_2 = x_2 - x_1 = 10 - 12 = -2$.

Plug in $\varphi_1 = 0.5$, $\theta_1 = 0.5$, $d_1 = 0$, $\varepsilon_1 = 0$:

$$\hat{d}_2 = 0 + 0.5 \cdot 0 + 0.5 \cdot 0 = 0$$

$$\hat{x}_2 = x_1 + \hat{d}_2 = 12 + 0 = 12$$

$$\varepsilon_2 = x_2 - \hat{x}_2 = 10 - 12 = -2$$

Step 3 — iterate

Two-stage pipeline at each t: difference $\to$ ARMA-forecast the difference $\to$ undifference. Values rounded to 4 decimal places.

t	x_t	$d_t = x_t - x_{t-1}$	$\hat{d}_t = 0.5 d_{t-1} + 0.5\varepsilon_{t-1}$	$\hat{x}_t = x_{t-1} + \hat{d}_t$	$\varepsilon_t = x_t - \hat{x}_t$
2	10	-2	$0.5(0) + 0.5(0) = 0$	$12 + 0 = 12$	-2
3	8	-2	$0.5(-2) + 0.5(-2) = -2$	$10 + (-2) = 8$	0
4	11	3	$0.5(-2) + 0.5(0) = -1$	$8 + (-1) = 7$	4
5	14	3	$0.5(3) + 0.5(4) = 3.5$	$11 + 3.5 = 14.5$	-0.5
6	12	-2	$0.5(3) + 0.5(-0.5) = 1.25$	$14 + 1.25 = 15.25$	-3.25
7	9	-3	$0.5(-2) + 0.5(-3.25) = -2.625$	$12 + (-2.625) = 9.375$	-0.375
8	13	4	$0.5(-3) + 0.5(-0.375) = -1.6875$	$9 + (-1.6875) = 7.3125$	5.6875
9	16	3	$0.5(4) + 0.5(5.6875) = 4.8438$	$13 + 4.8438 = 17.8438$	-1.8438
10	14	-2	$0.5(3) + 0.5(-1.8438) = 0.5781$	$16 + 0.5781 = 16.5781$	-2.5781
11	11	-3	$0.5(-2) + 0.5(-2.5781) = -2.2891$	$14 + (-2.2891) = 11.7109$	-0.7109
12	15	4	$0.5(-3) + 0.5(-0.7109) = -1.8555$	$11 + (-1.8555) = 9.1445$	5.8555
13	18	3	$0.5(4) + 0.5(5.8555) = 4.9277$	$15 + 4.9277 = 19.9277$	-1.9277
14	16	-2	$0.5(3) + 0.5(-1.9277) = 0.5361$	$18 + 0.5361 = 18.5361$	-2.5361
15	13	-3	$0.5(-2) + 0.5(-2.5361) = -2.2681$	$16 + (-2.2681) = 13.7319$	-0.7319

16	17	4	$0.5(-3) + 0.5(-0.7319) = -1.8660$	$13 + (-1.8660) = 11.1340$	5.8660
17	—	—	$0.5(4) + 0.5(5.8660) = 4.9330$	$17 + 4.9330 = 21.9330$	—

161.6. SARIMA

Seasonal ARIMA

$\text{SARIMA}(p,d,q)(P,D,Q)_m$

$$\varphi(B)\Phi(B^m)(1-B)^d(1-B^m)^D x_t = c + \theta(B)\Theta(B^m)\varepsilon_t$$

$$\Phi(B^m) = 1 - \Phi_1 B^m - \ldots - \Phi_P B^{Pm}$$

$$\Theta(B^m) = 1 + \Theta_1 B^m + \ldots + \Theta_Q B^{Qm}$$

Seasonal differencing $(1-B^m)y_t = y_t - y_{t-m}$ removes annual/periodic patterns. Period m (e.g., 12 for monthly).

Parameters: $\varphi_1, \ldots, \varphi_p$, $\Phi_1, \ldots, \Phi_P$, $\theta_1, \ldots, \theta_q$, $\Theta_1, \ldots, \Theta_Q$, c, σ^2
Orders: p, P, d, D, q, Q, m

Example: $\text{SARIMA}(1,0,0)(1,0,0)_4$

Given

- Orders: $p = 1$, $P = 1$, $d = 0$, $D = 0$, $q = 0$, $Q = 0$, $m = 4$
- Parameters: $\varphi_1 = 0.5$, $\Phi_1 = 0.5$, $c = 0$
- Data:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
x_t	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

Step 1 — formula

Substitute orders into the SARIMA recursion. With $d = D = 0$ and $q = Q = 0$:

$$(1 - \varphi_1 B)(1 - \Phi_1 B^m)x_t = c + \varepsilon_t$$

Expand the operator product:

$$(1 - \varphi_1 B - \Phi_1 B^m + \varphi_1\Phi_1 B^{m+1})x_t = c + \varepsilon_t$$

$$x_t - \varphi_1 x_{t-1} - \Phi_1 x_{t-m} + \varphi_1\Phi_1 x_{t-m-1} = c + \varepsilon_t$$

Forecast (set $\varepsilon_t = 0$):

$$\hat{x}_t = c + \varphi_1 x_{t-1} + \Phi_1 x_{t-m} - \varphi_1\Phi_1 x_{t-m-1}$$

Innovation:

$$\varepsilon_t = x_t - \hat{x}_t$$

Pre-compute the cross term: $\varphi_1 \Phi_1 = 0.5 \cdot 0.5 = 0.25$.

Step 2 — apply at $t = 6$ (first usable step: needs $x_{t-m-1} = x_1$)

Plug in $\varphi_1 = 0.5$, $\Phi_1 = 0.5$, $x_5 = 14$, $x_2 = 10$, $x_1 = 12$:

$$\hat{x}_6 = 0.5 \cdot 14 + 0.5 \cdot 10 - 0.25 \cdot 12 = 7 + 5 - 3 = 9$$

$$\varepsilon_6 = x_6 - \hat{x}_6 = 12 - 9 = 3$$

Step 3 — iterate

Each row uses lag-1 (x_{t-1}), lag-m (x_{t-4}), and lag-$(m+1)$ (x_{t-5}).

t	x_t	$x_{t-1}, x_{t-4}, x_{t-5}$	$\hat{x}_t = 0.5x_{t-1} + 0.5x_{t-4} - 0.25x_{t-5}$	$\hat{x}_t$	ε_t
6	12	$14, 10, 12$	$0.5(14) + 0.5(10) - 0.25(12) = 7 + 5 - 3$	9	3
7	9	$12, 8, 10$	$0.5(12) + 0.5(8) - 0.25(10) = 6 + 4 - 2.5$	7.5	1.5
8	13	$9, 11, 8$	$0.5(9) + 0.5(11) - 0.25(8) = 4.5 + 5.5 - 2$	8	5
9	16	$13, 14, 11$	$0.5(13) + 0.5(14) - 0.25(11) = 6.5 + 7 - 2.75$	10.75	5.25
10	14	$16, 12, 14$	$0.5(16) + 0.5(12) - 0.25(14) = 8 + 6 - 3.5$	10.5	3.5
11	11	$14, 9, 12$	$0.5(14) + 0.5(9) - 0.25(12) = 7 + 4.5 - 3$	8.5	2.5
12	15	$11, 13, 9$	$0.5(11) + 0.5(13) - 0.25(9) = 5.5 + 6.5 - 2.25$	9.75	5.25
13	18	$15, 16, 13$	$0.5(15) + 0.5(16) - 0.25(13) = 7.5 + 8 - 3.25$	12.25	5.75
14	16	$18, 14, 16$	$0.5(18) + 0.5(14) - 0.25(16) = 9 + 7 - 4$	12	4
15	13	$16, 11, 14$	$0.5(16) + 0.5(11) - 0.25(14) = 8 + 5.5 - 3.5$	10	3
16	17	$13, 15, 11$	$0.5(13) + 0.5(15) - 0.25(11) = 6.5 + 7.5 - 2.75$	11.25	5.75
17	—	$17, 18, 15$	$0.5(17) + 0.5(18) - 0.25(15) = 8.5 + 9 - 3.75$	13.75	—

Forecasts now sit much closer to the data than AR(1) — the seasonal lag x_{t-4} tracks the within-cycle pattern, and lag-1 picks up the trend.

161.7. SARIMAX

Seasonal ARIMA with exogenous regressors

$\mathrm{SARIMAX}(p, d, q)(P, D, Q)_m$

$$\varphi(B)\Phi(B^m)(1-B)^d(1-B^m)^D y_t = c + \sum_{i=1}^{k} \beta_i x_{i,t} + \theta(B)\Theta(B^m)\varepsilon_t$$

- y_t = endogenous (target) series
- $x_{i,t}$ = exogenous regressors, $i = 1, ..., k$
- β_i = regression coefficients on exogenous variables
- The X adds linear regression on external predictors to SARIMA.

Parameters: $\varphi_1, ..., \varphi_p$, $\Phi_1, ..., \Phi_P$, $\theta_1, ..., \theta_q$, $\Theta_1, ..., \Theta_Q$, $\beta_1, ..., \beta_k$, c, σ^2
Orders: p, P, d, D, q, Q, m, k (regressors)

Example: $\text{SARIMAX}(1,0,0)(1,0,0)_4 + 1$ exog

Given

- Orders: $p = 1$, $P = 1$, $d = 0$, $D = 0$, $q = 0$, $Q = 0$, $m = 4$, $k = 1$
- Parameters: $\varphi_1 = 0.5$, $\Phi_1 = 0.5$, $\beta_1 = 0.5$, $c = 0$
- Endogenous data $y_t = x_t$:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
y_t	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

- Exogenous regressor z_t:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
z_t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

Step 1 — formula

Substitute orders into the SARIMAX recursion. With $d = D = 0$ and $q = Q = 0$:

$$(1 - \varphi_1 B)(1 - \Phi_1 B^m) y_t = c + \beta_1 z_t + \varepsilon_t$$

Expand the operator product (same as SARIMA, plus the exogenous term):

$$y_t - \varphi_1 y_{t-1} - \Phi_1 y_{t-m} + \varphi_1 \Phi_1 y_{t-m-1} = c + \beta_1 z_t + \varepsilon_t$$

Forecast (set $\varepsilon_t = 0$):

$$\hat{y}_t = c + \beta_1 z_t + \varphi_1 y_{t-1} + \Phi_1 y_{t-m} - \varphi_1 \Phi_1 y_{t-m-1}$$

Innovation:

$$\varepsilon_t = y_t - \hat{y}_t$$

Pre-compute the cross term: $\varphi_1 \Phi_1 = 0.5 \cdot 0.5 = 0.25$.

Step 2 — apply at $t = 6$ (first usable step: needs $y_{t-m-1} = y_1$)

Plug in $\beta_1 = 0.5$, $\varphi_1 = 0.5$, $\Phi_1 = 0.5$, $z_6 = 6$, $y_5 = 14$, $y_2 = 10$, $y_1 = 12$:

$$\hat{y}_6 = 0 + 0.5 \cdot 6 + 0.5(14) + 0.5(10) - 0.25(12)$$

$$\hat{y}_6 = 3 + 7 + 5 - 3 = 12$$

$$\varepsilon_6 = y_6 - \hat{y}_6 = 12 - 12 = 0$$

Step 3 — iterate

Each row adds the exogenous contribution $\beta_1 z_t$ on top of the SARIMA forecast.

t	y_t	$\beta_1 z_t$	SARIMA part: $0.5y_{t-1} + 0.5y_{t-4} - 0.25y_{t-5}$	$\hat{y}_t$	ε_t
6	12	$0.5(6) = 3$	$0.5(14) + 0.5(10) - 0.25(12) = 9$	$3 + 9 = 12$	0
7	9	$0.5(7) = 3.5$	$0.5(12) + 0.5(8) - 0.25(10) = 7.5$	$3.5 + 7.5 = 11$	-2
8	13	$0.5(8) = 4$	$0.5(9) + 0.5(11) - 0.25(8) = 8$	$4 + 8 = 12$	1
9	16	$0.5(9) = 4.5$	$0.5(13) + 0.5(14) - 0.25(11) = 10.75$	$4.5 + 10.75 = 15.25$	0.75
10	14	$0.5(10) = 5$	$0.5(16) + 0.5(12) - 0.25(14) = 10.5$	$5 + 10.5 = 15.5$	-1.5
11	11	$0.5(11) = 5.5$	$0.5(14) + 0.5(9) - 0.25(12) = 8.5$	$5.5 + 8.5 = 14$	-3
12	15	$0.5(12) = 6$	$0.5(11) + 0.5(13) - 0.25(9) = 9.75$	$6 + 9.75 = 15.75$	-0.75
13	18	$0.5(13) = 6.5$	$0.5(15) + 0.5(16) - 0.25(13) = 12.25$	$6.5 + 12.25 = 18.75$	-0.75
14	16	$0.5(14) = 7$	$0.5(18) + 0.5(14) - 0.25(16) = 12$	$7 + 12 = 19$	-3
15	13	$0.5(15) = 7.5$	$0.5(16) + 0.5(11) - 0.25(14) = 10$	$7.5 + 10 = 17.5$	-4.5
16	17	$0.5(16) = 8$	$0.5(13) + 0.5(15) - 0.25(11) = 11.25$	$8 + 11.25 = 19.25$	-2.25
17	—	$0.5(17) = 8.5$	$0.5(17) + 0.5(18) - 0.25(15) = 13.75$	$8.5 + 13.75 = 22.25$	—

Here $z_t = t$ is collinear with y's linear trend, so $\beta_1 z_t$ adds an **extra** drift on top of the AR/seasonal structure — forecasts now overshoot. With a less collinear regressor, β_1 would correct part of y that AR/seasonal lags miss.

161.8. VAR

Vector autoregression

$$\mathrm{VAR}(p)$$

$$\Phi(B)\boldsymbol{x}_t = \boldsymbol{c} + \boldsymbol{\varepsilon}_t$$

$$\Phi(B) = \boldsymbol{I} - \Phi_1 B - \ldots - \Phi_p B^p$$

i.e.,

$$\boldsymbol{x}_t = \boldsymbol{c} + \Phi_1 \boldsymbol{x}_{t-1} + \ldots + \Phi_p \boldsymbol{x}_{t-p} + \boldsymbol{\varepsilon}_t$$

Each component depends on p lags of all K series.

Parameters: $\Phi_1, \ldots, \Phi_p$ (K^2 each), $\boldsymbol{c}$ (K), $\boldsymbol{\Sigma}$ ($K(K+1)/2$)
Orders: p, K (series)

Example: VAR(1) , $K = 2$

Given

- Order: $p = 1$, $K = 2$
- Coefficient matrix:

$$\Phi_1 = \begin{bmatrix} 0.4 & 0.2 \\ 0.2 & 0.4 \end{bmatrix}$$

- Intercept: $\boldsymbol{c} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$
- Two series stacked into a vector $\boldsymbol{x}_t = \begin{bmatrix} x_{1,t} \\ x_{2,t} \end{bmatrix}$:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
x_(1\,)$_t$	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
x_(2\,)$_t$	8	9	7	6	10	11	9	8	12	13	11	10	14	15	13	12

Step 1 — formula

Substitute $p = 1$ into the VAR(p) recursion:

$$\boldsymbol{x}_t = \boldsymbol{c} + \Phi_1 \boldsymbol{x}_{t-1} + \boldsymbol{\varepsilon}_t$$

Forecast (set $\boldsymbol{\varepsilon}_t = \boldsymbol{0}$):

$$\hat{\boldsymbol{x}}_t = \boldsymbol{c} + \Phi_1 \boldsymbol{x}_{t-1}$$

Componentwise (write out the matrix-vector product):

$$\begin{bmatrix} \hat{x}_{1,t} \\ \hat{x}_{2,t} \end{bmatrix} = \begin{bmatrix} c_1 \\ c_2 \end{bmatrix} + \begin{bmatrix} 0.4 & 0.2 \\ 0.2 & 0.4 \end{bmatrix} \begin{bmatrix} x_{1,t-1} \\ x_{2,t-1} \end{bmatrix}$$

$$\hat{x}_{1,t} = c_1 + 0.4 x_{1,t-1} + 0.2 x_{2,t-1}$$

$$\hat{x}_{2,t} = c_2 + 0.2 x_{1,t-1} + 0.4 x_{2,t-1}$$

Innovation:

$$\varepsilon_t = x_t - \hat{x}_t$$

Step 2 — apply at $t = 2$

Plug in $c = 0$, Φ_1, $x_1 = \begin{bmatrix} 12 \\ 8 \end{bmatrix}$:

$$\hat{x}_{1,2} = 0.4 \cdot 12 + 0.2 \cdot 8 = 4.8 + 1.6 = 6.4$$

$$\hat{x}_{2,2} = 0.2 \cdot 12 + 0.4 \cdot 8 = 2.4 + 3.2 = 5.6$$

$$\varepsilon_2 = \begin{bmatrix} 10 \\ 9 \end{bmatrix} - \begin{bmatrix} 6.4 \\ 5.6 \end{bmatrix} = \begin{bmatrix} 3.6 \\ 3.4 \end{bmatrix}$$

Step 3 — iterate

t	x_t	$\hat{x}_{1,t} = 0.4x_{1,t-1} + 0.2x_{2,t-1}$	$\hat{x}_{2,t} = 0.2x_{1,t-1} + 0.4x_{2,t-1}$	ε_t
2	$\begin{bmatrix} 10 \\ 9 \end{bmatrix}$	$0.4(12) + 0.2(8) = 6.4$	$0.2(12) + 0.4(8) = 5.6$	$\begin{bmatrix} 3.6 \\ 3.4 \end{bmatrix}$
3	$\begin{bmatrix} 8 \\ 7 \end{bmatrix}$	$0.4(10) + 0.2(9) = 5.8$	$0.2(10) + 0.4(9) = 5.6$	$\begin{bmatrix} 2.2 \\ 1.4 \end{bmatrix}$
4	$\begin{bmatrix} 11 \\ 6 \end{bmatrix}$	$0.4(8) + 0.2(7) = 4.6$	$0.2(8) + 0.4(7) = 4.4$	$\begin{bmatrix} 6.4 \\ 1.6 \end{bmatrix}$
5	$\begin{bmatrix} 14 \\ 10 \end{bmatrix}$	$0.4(11) + 0.2(6) = 5.6$	$0.2(11) + 0.4(6) = 4.6$	$\begin{bmatrix} 8.4 \\ 5.4 \end{bmatrix}$
6	$\begin{bmatrix} 12 \\ 11 \end{bmatrix}$	$0.4(14) + 0.2(10) = 7.6$	$0.2(14) + 0.4(10) = 6.8$	$\begin{bmatrix} 4.4 \\ 4.2 \end{bmatrix}$
7	$\begin{bmatrix} 9 \\ 9 \end{bmatrix}$	$0.4(12) + 0.2(11) = 7$	$0.2(12) + 0.4(11) = 6.8$	$\begin{bmatrix} 2 \\ 2.2 \end{bmatrix}$
8	$\begin{bmatrix} 13 \\ 8 \end{bmatrix}$	$0.4(9) + 0.2(9) = 5.4$	$0.2(9) + 0.4(9) = 5.4$	$\begin{bmatrix} 7.6 \\ 2.6 \end{bmatrix}$
9	$\begin{bmatrix} 16 \\ 12 \end{bmatrix}$	$0.4(13) + 0.2(8) = 6.8$	$0.2(13) + 0.4(8) = 5.8$	$\begin{bmatrix} 9.2 \\ 6.2 \end{bmatrix}$
10	$\begin{bmatrix} 14 \\ 13 \end{bmatrix}$	$0.4(16) + 0.2(12) = 8.8$	$0.2(16) + 0.4(12) = 8$	$\begin{bmatrix} 5.2 \\ 5 \end{bmatrix}$
11	$\begin{bmatrix} 11 \\ 11 \end{bmatrix}$	$0.4(14) + 0.2(13) = 8.2$	$0.2(14) + 0.4(13) = 8$	$\begin{bmatrix} 2.8 \\ 3 \end{bmatrix}$
12	$\begin{bmatrix} 15 \\ 10 \end{bmatrix}$	$0.4(11) + 0.2(11) = 6.6$	$0.2(11) + 0.4(11) = 6.6$	$\begin{bmatrix} 8.4 \\ 3.4 \end{bmatrix}$
13	$\begin{bmatrix} 18 \\ 14 \end{bmatrix}$	$0.4(15) + 0.2(10) = 8$	$0.2(15) + 0.4(10) = 7$	$\begin{bmatrix} 10 \\ 7 \end{bmatrix}$
14	$\begin{bmatrix} 16 \\ 15 \end{bmatrix}$	$0.4(18) + 0.2(14) = 10$	$0.2(18) + 0.4(14) = 9.2$	$\begin{bmatrix} 6 \\ 5.8 \end{bmatrix}$

15	$\begin{bmatrix}13\\13\end{bmatrix}$	$0.4(16)+0.2(15)=9.4$	$0.2(16)+0.4(15)=9.2$	$\begin{bmatrix}3.6\\3.8\end{bmatrix}$
16	$\begin{bmatrix}17\\12\end{bmatrix}$	$0.4(13)+0.2(13)=7.8$	$0.2(13)+0.4(13)=7.8$	$\begin{bmatrix}9.2\\4.2\end{bmatrix}$
17	—	$0.4(17)+0.2(12)=9.2$	$0.2(17)+0.4(12)=8.2$	—

161.9. VMA

Vector moving average

$$\text{VMA}(q)$$

$$\boldsymbol{x}_t = \boldsymbol{c} + \boldsymbol{\Theta}(B)\boldsymbol{\varepsilon}_t$$

$$\boldsymbol{\Theta}(B) = \boldsymbol{I} + \boldsymbol{\Theta}_1 B + \ldots + \boldsymbol{\Theta}_q B^q$$

i.e.,

$$\boldsymbol{x}_t = \boldsymbol{c} + \boldsymbol{\varepsilon}_t + \boldsymbol{\Theta}_1 \boldsymbol{\varepsilon}_{t-1} + \ldots + \boldsymbol{\Theta}_q \boldsymbol{\varepsilon}_{t-q}$$

Each component is a linear combination of past shocks across all series.

Parameters: $\boldsymbol{\Theta}_1, \ldots, \boldsymbol{\Theta}_q$ (K^2 each), $\boldsymbol{c}$ (K), $\boldsymbol{\Sigma}$ ($K(K+1)/2$)
Orders: q, K (series)

Example: $\text{VMA}(1)$, $K = 2$

Given

- Order: $q = 1$, $K = 2$
- Coefficient matrix:

$$\boldsymbol{\Theta}_1 = \begin{bmatrix} 0.4 & 0.2 \\ 0.2 & 0.4 \end{bmatrix}$$

- Intercept: $\boldsymbol{c} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$
- Initial innovation: $\boldsymbol{\varepsilon}_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$
- Two series stacked into $\boldsymbol{x}_t = \begin{bmatrix} x_{1,t} \\ x_{2,t} \end{bmatrix}$:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
$\text{x_(1\,)}_t$	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
$\text{x_(2\,)}_t$	8	9	7	6	10	11	9	8	12	13	11	10	14	15	13	12

Step 1 — formula

Substitute $q = 1$ into the VMA(q) recursion:

$$\boldsymbol{x}_t = \boldsymbol{c} + \boldsymbol{\varepsilon}_t + \boldsymbol{\Theta}_1 \boldsymbol{\varepsilon}_{t-1}$$

Forecast (set $\boldsymbol{\varepsilon}_t = \mathbf{0}$):

$$\hat{\boldsymbol{x}}_t = \boldsymbol{c} + \boldsymbol{\Theta}_1 \boldsymbol{\varepsilon}_{t-1}$$

Componentwise:

$$\hat{x}_{1,t} = c_1 + 0.4\varepsilon_{1,t-1} + 0.2\varepsilon_{2,t-1}$$

$$\hat{x}_{2,t} = c_2 + 0.2\varepsilon_{1,t-1} + 0.4\varepsilon_{2,t-1}$$

Innovation:

$$\boldsymbol{\varepsilon}_t = \boldsymbol{x}_t - \hat{\boldsymbol{x}}_t$$

Step 2 — apply at $t = 1$

Plug in $\boldsymbol{c} = \mathbf{0}$, $\boldsymbol{\Theta}_1$, $\boldsymbol{\varepsilon}_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$:

$$\hat{\boldsymbol{x}}_1 = \mathbf{0} + \boldsymbol{\Theta}_1 \mathbf{0} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$\boldsymbol{\varepsilon}_1 = \boldsymbol{x}_1 - \hat{\boldsymbol{x}}_1 = \begin{bmatrix} 12 \\ 8 \end{bmatrix} - \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 12 \\ 8 \end{bmatrix}$$

Step 3 — iterate

Each row uses the **previous** innovation $\boldsymbol{\varepsilon}_{t-1}$. Values rounded to 4 decimal places at every step (computation continues from the rounded value).

t	$\boldsymbol{x}_t$	$\hat{x}_{1,t} = 0.4\varepsilon_{1,t-1} + 0.2\varepsilon_{2,t-1}$	$\hat{x}_{2,t} = 0.2\varepsilon_{1,t-1} + 0.4\varepsilon_{2,t-1}$	$\boldsymbol{\varepsilon}_t$
1	$\begin{bmatrix} 12 \\ 8 \end{bmatrix}$	$0.4(0) + 0.2(0) = 0$	$0.2(0) + 0.4(0) = 0$	$\begin{bmatrix} 12 \\ 8 \end{bmatrix}$
2	$\begin{bmatrix} 10 \\ 9 \end{bmatrix}$	$0.4(12) + 0.2(8) = 6.4$	$0.2(12) + 0.4(8) = 5.6$	$\begin{bmatrix} 3.6 \\ 3.4 \end{bmatrix}$
3	$\begin{bmatrix} 8 \\ 7 \end{bmatrix}$	$0.4(3.6) + 0.2(3.4) = 2.12$	$0.2(3.6) + 0.4(3.4) = 2.08$	$\begin{bmatrix} 5.88 \\ 4.92 \end{bmatrix}$
4	$\begin{bmatrix} 11 \\ 6 \end{bmatrix}$	$0.4(5.88) + 0.2(4.92) = 3.336$	$0.2(5.88) + 0.4(4.92) = 3.144$	$\begin{bmatrix} 7.664 \\ 2.856 \end{bmatrix}$
5	$\begin{bmatrix} 14 \\ 10 \end{bmatrix}$	$0.4(7.664) + 0.2(2.856) = 3.6368$	$0.2(7.664) + 0.4(2.856) = 2.6752$	$\begin{bmatrix} 10.3632 \\ 7.3248 \end{bmatrix}$
6	$\begin{bmatrix} 12 \\ 11 \end{bmatrix}$	$0.4(10.3632) + 0.2(7.3248) = 5.6102$	$0.2(10.3632) + 0.4(7.3248) = 5.0026$	$\begin{bmatrix} 6.3898 \\ 5.9974 \end{bmatrix}$

7	$\begin{matrix}9\\9\end{matrix}$	$0.4(6.3898) + 0.2(5.9974) = 3.7554$	$0.2(6.3898) + 0.4(5.9974) = 3.6769$	$\begin{matrix}5.2446\\5.3231\end{matrix}$
8	$\begin{matrix}13\\8\end{matrix}$	$0.4(5.2446) + 0.2(5.3231) = 3.1625$	$0.2(5.2446) + 0.4(5.3231) = 3.1781$	$\begin{matrix}9.8375\\4.8219\end{matrix}$
9	$\begin{matrix}16\\12\end{matrix}$	$0.4(9.8375) + 0.2(4.8219) = 4.8994$	$0.2(9.8375) + 0.4(4.8219) = 3.8963$	$\begin{matrix}11.1006\\8.1037\end{matrix}$
10	$\begin{matrix}14\\13\end{matrix}$	$0.4(11.1006) + 0.2(8.1037) = 6.061$	$0.2(11.1006) + 0.4(8.1037) = 5.4616$	$\begin{matrix}7.939\\7.5384\end{matrix}$
11	$\begin{matrix}11\\11\end{matrix}$	$0.4(7.939) + 0.2(7.5384) = 4.6833$	$0.2(7.939) + 0.4(7.5384) = 4.6032$	$\begin{matrix}6.3167\\6.3968\end{matrix}$
12	$\begin{matrix}15\\10\end{matrix}$	$0.4(6.3167) + 0.2(6.3968) = 3.806$	$0.2(6.3167) + 0.4(6.3968) = 3.822$	$\begin{matrix}11.194\\6.178\end{matrix}$
13	$\begin{matrix}18\\14\end{matrix}$	$0.4(11.194) + 0.2(6.178) = 5.7132$	$0.2(11.194) + 0.4(6.178) = 4.71$	$\begin{matrix}12.2868\\9.29\end{matrix}$
14	$\begin{matrix}16\\15\end{matrix}$	$0.4(12.2868) + 0.2(9.29) = 6.7727$	$0.2(12.2868) + 0.4(9.29) = 6.1734$	$\begin{matrix}9.2273\\8.8266\end{matrix}$
15	$\begin{matrix}13\\13\end{matrix}$	$0.4(9.2273) + 0.2(8.8266) = 5.4562$	$0.2(9.2273) + 0.4(8.8266) = 5.3761$	$\begin{matrix}7.5438\\7.6239\end{matrix}$
16	$\begin{matrix}17\\12\end{matrix}$	$0.4(7.5438) + 0.2(7.6239) = 4.5423$	$0.2(7.5438) + 0.4(7.6239) = 4.5584$	$\begin{matrix}12.4577\\7.4416\end{matrix}$

161.10. VARMA

Vector autoregressive moving average

$\mathrm{VARMA}(p, q)$

$$\boldsymbol{\Phi}(B)\boldsymbol{x}_t = \boldsymbol{c} + \boldsymbol{\Theta}(B)\boldsymbol{\varepsilon}_t$$

$$\boldsymbol{\Phi}(B) = \boldsymbol{I} - \boldsymbol{\Phi}_1 B - \ldots - \boldsymbol{\Phi}_p B^p$$

$$\boldsymbol{\Theta}(B) = \boldsymbol{I} + \boldsymbol{\Theta}_1 B + \ldots + \boldsymbol{\Theta}_q B^q$$

Vector analog of ARMA. Identifiability requires care (e.g., echelon forms).

Parameters: $\boldsymbol{\Phi}_1, \ldots, \boldsymbol{\Phi}_p$, $\boldsymbol{\Theta}_1, \ldots, \boldsymbol{\Theta}_q$, $\boldsymbol{c}$, $\boldsymbol{\Sigma}$
Orders: p, q, K

Example: $\mathrm{VARMA}(1,1)$, $K = 2$

Given

- Orders: $p = 1$, $q = 1$, $K = 2$
- Coefficient matrices:

$$\Phi_1 = \begin{bmatrix} 0.4 & 0.2 \\ 0.2 & 0.4 \end{bmatrix} \quad \Theta_1 = \begin{bmatrix} 0.4 & 0.2 \\ 0.2 & 0.4 \end{bmatrix}$$

- Intercept: $\boldsymbol{c} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$
- Initial conditions: $\boldsymbol{x}_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$, $\boldsymbol{\varepsilon}_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$
- Two series stacked into $\boldsymbol{x}_t = \begin{bmatrix} x_{1,t} \\ x_{2,t} \end{bmatrix}$:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
$\mathrm{x_(1\backslash,)}_t$	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
$\mathrm{x_(2\backslash,)}_t$	8	9	7	6	10	11	9	8	12	13	11	10	14	15	13	12

Step 1 — formula

Substitute $p = 1$, $q = 1$ into the VARMA recursion:

$$\boldsymbol{x}_t = \boldsymbol{c} + \Phi_1 \boldsymbol{x}_{t-1} + \boldsymbol{\varepsilon}_t + \Theta_1 \boldsymbol{\varepsilon}_{t-1}$$

Forecast (set $\boldsymbol{\varepsilon}_t = \boldsymbol{0}$):

$$\hat{\boldsymbol{x}}_t = \boldsymbol{c} + \Phi_1 \boldsymbol{x}_{t-1} + \Theta_1 \boldsymbol{\varepsilon}_{t-1}$$

Componentwise:

$$\hat{x}_{1,t} = c_1 + 0.4 x_{1,t-1} + 0.2 x_{2,t-1} + 0.4 \varepsilon_{1,t-1} + 0.2 \varepsilon_{2,t-1}$$

$$\hat{x}_{2,t} = c_2 + 0.2 x_{1,t-1} + 0.4 x_{2,t-1} + 0.2 \varepsilon_{1,t-1} + 0.4 \varepsilon_{2,t-1}$$

Innovation:

$$\boldsymbol{\varepsilon}_t = \boldsymbol{x}_t - \hat{\boldsymbol{x}}_t$$

Step 2 — apply at $t = 1$

With $\boldsymbol{x}_0 = \boldsymbol{0}$, $\boldsymbol{\varepsilon}_0 = \boldsymbol{0}$, every product is zero:

$$\hat{\boldsymbol{x}}_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad \boldsymbol{\varepsilon}_1 = \boldsymbol{x}_1 - \hat{\boldsymbol{x}}_1 = \begin{bmatrix} 12 \\ 8 \end{bmatrix}$$

Step 2b — apply at $t = 2$

Plug in $\boldsymbol{x}_1 = \begin{bmatrix} 12 \\ 8 \end{bmatrix}$, $\boldsymbol{\varepsilon}_1 = \begin{bmatrix} 12 \\ 8 \end{bmatrix}$:

$$\hat{x}_{1,2} = 0.4(12) + 0.2(8) + 0.4(12) + 0.2(8) = 4.8 + 1.6 + 4.8 + 1.6 = 12.8$$

$$\hat{x}_{2,2} = 0.2(12) + 0.4(8) + 0.2(12) + 0.4(8) = 2.4 + 3.2 + 2.4 + 3.2 = 11.2$$

$$\varepsilon_2 = \begin{bmatrix} 10 \\ 9 \end{bmatrix} - \begin{bmatrix} 12.8 \\ 11.2 \end{bmatrix} = \begin{bmatrix} -2.8 \\ -2.2 \end{bmatrix}$$

Step 3 — iterate

Two contributions per row: AR part $\Phi_1 x_{t-1}$ and MA part $\Theta_1 \varepsilon_{t-1}$. Values rounded to 4 decimal places.

t	x_t	$\Phi_1 x_{t-1}$	$\Theta_1 \varepsilon_{t-1}$	$\hat{x}_t$ / ε_t
1	$\begin{bmatrix} 12 \\ 8 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$ / $\begin{bmatrix} 12 \\ 8 \end{bmatrix}$
2	$\begin{bmatrix} 10 \\ 9 \end{bmatrix}$	$\begin{bmatrix} 6.4 \\ 5.6 \end{bmatrix}$	$\begin{bmatrix} 6.4 \\ 5.6 \end{bmatrix}$	$\begin{bmatrix} 12.8 \\ 11.2 \end{bmatrix}$ / $\begin{bmatrix} -2.8 \\ -2.2 \end{bmatrix}$
3	$\begin{bmatrix} 8 \\ 7 \end{bmatrix}$	$\begin{bmatrix} 5.8 \\ 5.6 \end{bmatrix}$	$\begin{bmatrix} -1.56 \\ -1.44 \end{bmatrix}$	$\begin{bmatrix} 4.24 \\ 4.16 \end{bmatrix}$ / $\begin{bmatrix} 3.76 \\ 2.84 \end{bmatrix}$
4	$\begin{bmatrix} 11 \\ 6 \end{bmatrix}$	$\begin{bmatrix} 4.6 \\ 4.4 \end{bmatrix}$	$\begin{bmatrix} 2.072 \\ 1.888 \end{bmatrix}$	$\begin{bmatrix} 6.672 \\ 6.288 \end{bmatrix}$ / $\begin{bmatrix} 4.328 \\ -0.288 \end{bmatrix}$
5	$\begin{bmatrix} 14 \\ 10 \end{bmatrix}$	$\begin{bmatrix} 5.6 \\ 4.6 \end{bmatrix}$	$\begin{bmatrix} 1.6736 \\ 0.7504 \end{bmatrix}$	$\begin{bmatrix} 7.2736 \\ 5.3504 \end{bmatrix}$ / $\begin{bmatrix} 6.7264 \\ 4.6496 \end{bmatrix}$
6	$\begin{bmatrix} 12 \\ 11 \end{bmatrix}$	$\begin{bmatrix} 7.6 \\ 6.8 \end{bmatrix}$	$\begin{bmatrix} 3.6206 \\ 3.2051 \end{bmatrix}$	$\begin{bmatrix} 11.2206 \\ 10.0051 \end{bmatrix}$ / $\begin{bmatrix} 0.7794 \\ 0.9949 \end{bmatrix}$
7	$\begin{bmatrix} 9 \\ 9 \end{bmatrix}$	$\begin{bmatrix} 7 \\ 6.8 \end{bmatrix}$	$\begin{bmatrix} 0.5108 \\ 0.5538 \end{bmatrix}$	$\begin{bmatrix} 7.5108 \\ 7.3538 \end{bmatrix}$ / $\begin{bmatrix} 1.4892 \\ 1.6462 \end{bmatrix}$
8	$\begin{bmatrix} 13 \\ 8 \end{bmatrix}$	$\begin{bmatrix} 5.4 \\ 5.4 \end{bmatrix}$	$\begin{bmatrix} 0.9249 \\ 0.9563 \end{bmatrix}$	$\begin{bmatrix} 6.3249 \\ 6.3563 \end{bmatrix}$ / $\begin{bmatrix} 6.6751 \\ 1.6437 \end{bmatrix}$
9	$\begin{bmatrix} 16 \\ 12 \end{bmatrix}$	$\begin{bmatrix} 6.8 \\ 5.8 \end{bmatrix}$	$\begin{bmatrix} 2.9988 \\ 1.9925 \end{bmatrix}$	$\begin{bmatrix} 9.7988 \\ 7.7925 \end{bmatrix}$ / $\begin{bmatrix} 6.2012 \\ 4.2075 \end{bmatrix}$
10	$\begin{bmatrix} 14 \\ 13 \end{bmatrix}$	$\begin{bmatrix} 8.8 \\ 8 \end{bmatrix}$	$\begin{bmatrix} 3.3220 \\ 2.9232 \end{bmatrix}$	$\begin{bmatrix} 12.122 \\ 10.9232 \end{bmatrix}$ / $\begin{bmatrix} 1.878 \\ 2.0768 \end{bmatrix}$
11	$\begin{bmatrix} 11 \\ 11 \end{bmatrix}$	$\begin{bmatrix} 8.2 \\ 8 \end{bmatrix}$	$\begin{bmatrix} 1.1666 \\ 1.2071 \end{bmatrix}$	$\begin{bmatrix} 9.3666 \\ 9.2071 \end{bmatrix}$ / $\begin{bmatrix} 1.6334 \\ 1.7929 \end{bmatrix}$
12	$\begin{bmatrix} 15 \\ 10 \end{bmatrix}$	$\begin{bmatrix} 6.6 \\ 6.6 \end{bmatrix}$	$\begin{bmatrix} 1.0119 \\ 1.0438 \end{bmatrix}$	$\begin{bmatrix} 7.6119 \\ 7.6438 \end{bmatrix}$ / $\begin{bmatrix} 7.3881 \\ 2.3562 \end{bmatrix}$
13	$\begin{bmatrix} 18 \\ 14 \end{bmatrix}$	$\begin{bmatrix} 8 \\ 7 \end{bmatrix}$	$\begin{bmatrix} 3.4265 \\ 2.4201 \end{bmatrix}$	$\begin{bmatrix} 11.4265 \\ 9.4201 \end{bmatrix}$ / $\begin{bmatrix} 6.5735 \\ 4.5799 \end{bmatrix}$
14	$\begin{bmatrix} 16 \\ 15 \end{bmatrix}$	$\begin{bmatrix} 10 \\ 9.2 \end{bmatrix}$	$\begin{bmatrix} 3.5454 \\ 3.1467 \end{bmatrix}$	$\begin{bmatrix} 13.5454 \\ 12.3467 \end{bmatrix}$ / $\begin{bmatrix} 2.4546 \\ 2.6533 \end{bmatrix}$
15	$\begin{bmatrix} 13 \\ 13 \end{bmatrix}$	$\begin{bmatrix} 9.4 \\ 9.2 \end{bmatrix}$	$\begin{bmatrix} 1.5125 \\ 1.5523 \end{bmatrix}$	$\begin{bmatrix} 10.9125 \\ 10.7523 \end{bmatrix}$ / $\begin{bmatrix} 2.0875 \\ 2.2477 \end{bmatrix}$

16		17 12			7.8 7.8			1.2845 1.3166			9.0845 9.1166	/	7.9155 2.8834			

161.11. VARIMA

Vector ARIMA

$$\mathrm{VARIMA}(p, d, q)$$

$$\boldsymbol{\Phi}(B)(1 - B)^d \boldsymbol{x}_t = \boldsymbol{c} + \boldsymbol{\Theta}(B)\boldsymbol{\varepsilon}_t$$

Differencing applied component-wise to each of the K series.
In practice, cointegrated systems use VECM (Vector Error Correction) rather than VARIMA — VECM models the long-run equilibrium directly.

Parameters: $\boldsymbol{\Phi}_1, \ldots, \boldsymbol{\Phi}_p$, $\boldsymbol{\Theta}_1, \ldots, \boldsymbol{\Theta}_q$, $\boldsymbol{c}$, $\boldsymbol{\Sigma}$
Orders: p, d, q, K

Example: $\mathrm{VARIMA}(1, 1, 1)$, $K = 2$

Given

- Orders: $p = 1$, $d = 1$, $q = 1$, $K = 2$
- Coefficient matrices:

$$\boldsymbol{\Phi}_1 = \begin{bmatrix} 0.4 & 0.2 \\ 0.2 & 0.4 \end{bmatrix} \quad \boldsymbol{\Theta}_1 = \begin{bmatrix} 0.4 & 0.2 \\ 0.2 & 0.4 \end{bmatrix}$$

- Intercept: $\boldsymbol{c} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$
- Initial conditions: $\boldsymbol{d}_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$, $\boldsymbol{\varepsilon}_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$
- Two series stacked into $\boldsymbol{x}_t = \begin{bmatrix} x_{1,t} \\ x_{2,t} \end{bmatrix}$:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
$\mathrm{x_(1\backslash,)}_t$	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
$\mathrm{x_(2\backslash,)}_t$	8	9	7	6	10	11	9	8	12	13	11	10	14	15	13	12

Step 1 — formula

Substitute orders into the VARIMA recursion:

$$(\boldsymbol{I} - \boldsymbol{\Phi}_1 B)(1 - B)\boldsymbol{x}_t = \boldsymbol{c} + (\boldsymbol{I} + \boldsymbol{\Theta}_1 B)\boldsymbol{\varepsilon}_t$$

Difference — apply $(1 - B)$ component-wise:

$$\boldsymbol{d}_t := (1 - B)\boldsymbol{x}_t = \boldsymbol{x}_t - \boldsymbol{x}_{t-1}$$

In differenced space, the model is VARMA$(1, 1)$:

$$\boldsymbol{d}_t = \boldsymbol{c} + \boldsymbol{\Phi}_1 \boldsymbol{d}_{t-1} + \boldsymbol{\varepsilon}_t + \boldsymbol{\Theta}_1 \boldsymbol{\varepsilon}_{t-1}$$

Forecast the difference (set $\boldsymbol{\varepsilon}_t = \mathbf{0}$):

$$\hat{\boldsymbol{d}}_t = \boldsymbol{c} + \boldsymbol{\Phi}_1 \boldsymbol{d}_{t-1} + \boldsymbol{\Theta}_1 \boldsymbol{\varepsilon}_{t-1}$$

Undifference — convert back to a forecast for $\boldsymbol{x}_t$:

$$\hat{\boldsymbol{x}}_t = \boldsymbol{x}_{t-1} + \hat{\boldsymbol{d}}_t$$

Innovation:

$$\boldsymbol{\varepsilon}_t = \boldsymbol{d}_t - \hat{\boldsymbol{d}}_t = \boldsymbol{x}_t - \hat{\boldsymbol{x}}_t$$

Step 2 — apply at $t = 2$

First difference: $\boldsymbol{d}_2 = \boldsymbol{x}_2 - \boldsymbol{x}_1 = \begin{bmatrix} 10 \\ 9 \end{bmatrix} - \begin{bmatrix} 12 \\ 8 \end{bmatrix} = \begin{bmatrix} -2 \\ 1 \end{bmatrix}$.

Plug in $\boldsymbol{\Phi}_1$, $\boldsymbol{\Theta}_1$, $\boldsymbol{d}_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$, $\boldsymbol{\varepsilon}_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$:

$$\hat{\boldsymbol{d}}_2 = \boldsymbol{\Phi}_1 \begin{bmatrix} 0 \\ 0 \end{bmatrix} + \boldsymbol{\Theta}_1 \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$\hat{\boldsymbol{x}}_2 = \boldsymbol{x}_1 + \hat{\boldsymbol{d}}_2 = \begin{bmatrix} 12 \\ 8 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 12 \\ 8 \end{bmatrix}$$

$$\boldsymbol{\varepsilon}_2 = \boldsymbol{x}_2 - \hat{\boldsymbol{x}}_2 = \begin{bmatrix} 10 \\ 9 \end{bmatrix} - \begin{bmatrix} 12 \\ 8 \end{bmatrix} = \begin{bmatrix} -2 \\ 1 \end{bmatrix}$$

Step 3 — iterate

Pipeline at each t: difference → VARMA-forecast → undifference. Values rounded to 4 decimal places.

t	$\boldsymbol{d}_t$	$\hat{\boldsymbol{d}}_t = \boldsymbol{\Phi}_1 \boldsymbol{d}_{t-1} + \boldsymbol{\Theta}_1 \boldsymbol{\varepsilon}_{t-1}$	$\hat{\boldsymbol{x}}_t = \boldsymbol{x}_{t-1} + \hat{\boldsymbol{d}}_t$	$\boldsymbol{\varepsilon}_t$
2	$\begin{bmatrix} -2 \\ 1 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 12 \\ 8 \end{bmatrix}$	$\begin{bmatrix} -2 \\ 1 \end{bmatrix}$
3	$\begin{bmatrix} -2 \\ -2 \end{bmatrix}$	$\begin{bmatrix} -0.6 \\ 0 \end{bmatrix} + \begin{bmatrix} -0.6 \\ 0 \end{bmatrix} = \begin{bmatrix} -1.2 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 8.8 \\ 9 \end{bmatrix}$	$\begin{bmatrix} -0.8 \\ -2 \end{bmatrix}$
4	$\begin{bmatrix} 3 \\ -1 \end{bmatrix}$	$\begin{bmatrix} -1.2 \\ -1.2 \end{bmatrix} + \begin{bmatrix} -0.72 \\ -0.96 \end{bmatrix} = \begin{bmatrix} -1.92 \\ -2.16 \end{bmatrix}$	$\begin{bmatrix} 6.08 \\ 4.84 \end{bmatrix}$	$\begin{bmatrix} 4.92 \\ 1.16 \end{bmatrix}$
5	$\begin{bmatrix} 3 \\ 4 \end{bmatrix}$	$\begin{bmatrix} 1 \\ 0.2 \end{bmatrix} + \begin{bmatrix} 2.2 \\ 1.448 \end{bmatrix} = \begin{bmatrix} 3.2 \\ 1.648 \end{bmatrix}$	$\begin{bmatrix} 14.2 \\ 7.648 \end{bmatrix}$	$\begin{bmatrix} -0.2 \\ 2.352 \end{bmatrix}$

6	$\begin{bmatrix}-2\\1\end{bmatrix}$		$\begin{bmatrix}2\\2.2\end{bmatrix}$	$+$	$\begin{bmatrix}0.3904\\0.9008\end{bmatrix}$	$=$	$\begin{bmatrix}2.3904\\3.1008\end{bmatrix}$		$\begin{bmatrix}16.3904\\13.1008\end{bmatrix}$		$\begin{bmatrix}-4.3904\\-2.1008\end{bmatrix}$
7	$\begin{bmatrix}-3\\-2\end{bmatrix}$		$\begin{bmatrix}-0.6\\-0\end{bmatrix}$	$+$	$\begin{bmatrix}-2.1763\\-1.7184\end{bmatrix}$	$=$	$\begin{bmatrix}-2.7763\\-1.7184\end{bmatrix}$		$\begin{bmatrix}9.2237\\9.2816\end{bmatrix}$		$\begin{bmatrix}-0.2237\\-0.2816\end{bmatrix}$
8	$\begin{bmatrix}4\\-1\end{bmatrix}$		$\begin{bmatrix}-1.6\\-1.4\end{bmatrix}$	$+$	$\begin{bmatrix}-0.1458\\-0.1573\end{bmatrix}$	$=$	$\begin{bmatrix}-1.7458\\-1.5573\end{bmatrix}$		$\begin{bmatrix}7.2542\\7.4427\end{bmatrix}$		$\begin{bmatrix}5.7458\\0.5573\end{bmatrix}$
9	$\begin{bmatrix}3\\4\end{bmatrix}$		$\begin{bmatrix}1.4\\0.4\end{bmatrix}$	$+$	$\begin{bmatrix}2.4098\\1.3721\end{bmatrix}$	$=$	$\begin{bmatrix}3.8098\\1.7721\end{bmatrix}$		$\begin{bmatrix}16.8098\\9.7721\end{bmatrix}$		$\begin{bmatrix}-0.8098\\2.2279\end{bmatrix}$
10	$\begin{bmatrix}-2\\1\end{bmatrix}$		$\begin{bmatrix}2\\2.2\end{bmatrix}$	$+$	$\begin{bmatrix}0.1217\\0.7292\end{bmatrix}$	$=$	$\begin{bmatrix}2.1217\\2.9292\end{bmatrix}$		$\begin{bmatrix}18.1217\\14.9292\end{bmatrix}$		$\begin{bmatrix}-4.1217\\-1.9292\end{bmatrix}$
11	$\begin{bmatrix}-3\\-2\end{bmatrix}$		$\begin{bmatrix}-0.6\\0\end{bmatrix}$	$+$	$\begin{bmatrix}-2.0345\\-1.596\end{bmatrix}$	$=$	$\begin{bmatrix}-2.6345\\-1.596\end{bmatrix}$		$\begin{bmatrix}11.3655\\11.404\end{bmatrix}$		$\begin{bmatrix}-0.3655\\-0.404\end{bmatrix}$
12	$\begin{bmatrix}4\\-1\end{bmatrix}$		$\begin{bmatrix}-1.6\\-1.4\end{bmatrix}$	$+$	$\begin{bmatrix}-0.227\\-0.2347\end{bmatrix}$	$=$	$\begin{bmatrix}-1.827\\-1.6347\end{bmatrix}$		$\begin{bmatrix}9.173\\9.3653\end{bmatrix}$		$\begin{bmatrix}5.827\\0.6347\end{bmatrix}$
13	$\begin{bmatrix}3\\4\end{bmatrix}$		$\begin{bmatrix}1.4\\0.4\end{bmatrix}$	$+$	$\begin{bmatrix}2.4577\\1.4193\end{bmatrix}$	$=$	$\begin{bmatrix}3.8577\\1.8193\end{bmatrix}$		$\begin{bmatrix}18.8577\\11.8193\end{bmatrix}$		$\begin{bmatrix}-0.8577\\2.1807\end{bmatrix}$
14	$\begin{bmatrix}-2\\1\end{bmatrix}$		$\begin{bmatrix}2\\2.2\end{bmatrix}$	$+$	$\begin{bmatrix}0.093\\0.7008\end{bmatrix}$	$=$	$\begin{bmatrix}2.093\\2.9008\end{bmatrix}$		$\begin{bmatrix}20.093\\16.9008\end{bmatrix}$		$\begin{bmatrix}-4.093\\-1.9008\end{bmatrix}$
15	$\begin{bmatrix}-3\\-2\end{bmatrix}$		$\begin{bmatrix}-0.6\\0\end{bmatrix}$	$+$	$\begin{bmatrix}-2.0174\\-1.5789\end{bmatrix}$	$=$	$\begin{bmatrix}-2.6174\\-1.5789\end{bmatrix}$		$\begin{bmatrix}13.3826\\13.4211\end{bmatrix}$		$\begin{bmatrix}-0.3826\\-0.4211\end{bmatrix}$
16	$\begin{bmatrix}4\\-1\end{bmatrix}$		$\begin{bmatrix}-1.6\\-1.4\end{bmatrix}$	$+$	$\begin{bmatrix}-0.2372\\-0.2449\end{bmatrix}$	$=$	$\begin{bmatrix}-1.8372\\-1.6449\end{bmatrix}$		$\begin{bmatrix}11.1628\\11.3551\end{bmatrix}$		$\begin{bmatrix}5.8372\\0.6449\end{bmatrix}$

161.12. VARIMAX

Vector ARIMA with exogenous regressors

$$\mathrm{VARIMAX}(p, d, q)$$

$$\boldsymbol{\Phi}(B)(1-B)^d \boldsymbol{y}_t = \boldsymbol{c} + \boldsymbol{B}\boldsymbol{x}_t + \boldsymbol{\Theta}(B)\boldsymbol{\varepsilon}_t$$

$\boldsymbol{y}_t \in \mathbb{R}^K$ = endogenous vector series
$\boldsymbol{x}_t \in \mathbb{R}^k$ = exogenous regressor vector
$\boldsymbol{B} \in \mathbb{R}^{K \times k}$ = matrix of regression coefficients
Each of the K series can depend on the same set of k exogenous variables.

Parameters: $\boldsymbol{\Phi}_1, \ldots, \boldsymbol{\Phi}_p$, $\boldsymbol{\Theta}_1, \ldots, \boldsymbol{\Theta}_q$, $\boldsymbol{B}$ $(K \times k)$, $\boldsymbol{c}$, $\boldsymbol{\Sigma}$
Orders: p, d, q, K, k (regressors)

Example: $\mathrm{VARIMAX}(1, 0, 1)$, $K = 2$, $k = 1$

Given

- Orders: $p = 1$, $d = 0$, $q = 1$, $K = 2$, $k = 1$ (one exog)

- Coefficient matrices:

$$\boldsymbol{\Phi}_1 = \begin{bmatrix} 0.4 & 0.2 \\ 0.2 & 0.4 \end{bmatrix} \quad \boldsymbol{\Theta}_1 = \begin{bmatrix} 0.4 & 0.2 \\ 0.2 & 0.4 \end{bmatrix} \quad \boldsymbol{B} = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}$$

- Intercept: $\boldsymbol{c} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$
- Initial conditions: $\boldsymbol{y}_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$, $\boldsymbol{\varepsilon}_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$
- Endogenous $\boldsymbol{y}_t = \begin{bmatrix} y_{1,t} \\ y_{2,t} \end{bmatrix}$:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
$\mathrm{y_(1\backslash,)}_t$	12	10	8	11	14	12	9	13	16	14	11	15	18	16	13	17

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
$\mathrm{y_(2\backslash,)}_t$	8	9	7	6	10	11	9	8	12	13	11	10	14	15	13	12

- Exogenous z_t:

t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
z_t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

Step 1 — formula

Substitute orders into the VARIMAX recursion (with $d = 0$, no differencing):

$$(\boldsymbol{I} - \boldsymbol{\Phi}_1 B)\boldsymbol{y}_t = \boldsymbol{c} + \boldsymbol{B}z_t + (\boldsymbol{I} + \boldsymbol{\Theta}_1 B)\boldsymbol{\varepsilon}_t$$

Forecast (set $\boldsymbol{\varepsilon}_t = \boldsymbol{0}$):

$$\hat{\boldsymbol{y}}_t = \boldsymbol{c} + \boldsymbol{\Phi}_1 \boldsymbol{y}_{t-1} + \boldsymbol{B}z_t + \boldsymbol{\Theta}_1 \boldsymbol{\varepsilon}_{t-1}$$

Three contributions: AR product, exogenous regression $\boldsymbol{B}z_t$, MA product.

Componentwise:

$$\hat{y}_{1,t} = c_1 + 0.4y_{1,t-1} + 0.2y_{2,t-1} + 0.5z_t + 0.4\varepsilon_{1,t-1} + 0.2\varepsilon_{2,t-1}$$

$$\hat{y}_{2,t} = c_2 + 0.2y_{1,t-1} + 0.4y_{2,t-1} + 0.5z_t + 0.2\varepsilon_{1,t-1} + 0.4\varepsilon_{2,t-1}$$

Innovation:

$$\boldsymbol{\varepsilon}_t = \boldsymbol{y}_t - \hat{\boldsymbol{y}}_t$$

Step 2 — apply at $t = 2$

Compute the three contributions separately, then sum.

AR part: $\boldsymbol{\Phi}_1 \boldsymbol{y}_1 = \boldsymbol{\Phi}_1 \begin{bmatrix} 12 \\ 8 \end{bmatrix} = \begin{bmatrix} 0.4(12)+0.2(8) \\ 0.2(12)+0.4(8) \end{bmatrix} = \begin{bmatrix} 6.4 \\ 5.6 \end{bmatrix}$

Exog part: $\boldsymbol{B}z_2 = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix} \cdot 2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$

MA part: $\boldsymbol{\Theta}_1\varepsilon_1$. With $\varepsilon_0 = \mathbf{0}$, the $t = 1$ step gives $\hat{\boldsymbol{y}}_1 = \boldsymbol{B}z_1 = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}$ and $\varepsilon_1 = \begin{bmatrix} 12 \\ 8 \end{bmatrix} - \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix} = \begin{bmatrix} 11.5 \\ 7.5 \end{bmatrix}$.

$$\boldsymbol{\Theta}_1\varepsilon_1 = \begin{bmatrix} 0.4(11.5) + 0.2(7.5) \\ 0.2(11.5) + 0.4(7.5) \end{bmatrix} = \begin{bmatrix} 6.1 \\ 5.3 \end{bmatrix}$$

Sum: $\hat{\boldsymbol{y}}_2 = \begin{bmatrix} 6.4 \\ 5.6 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \end{bmatrix} + \begin{bmatrix} 6.1 \\ 5.3 \end{bmatrix} = \begin{bmatrix} 13.5 \\ 11.9 \end{bmatrix}$

$\varepsilon_2 = \begin{bmatrix} 10 \\ 9 \end{bmatrix} - \begin{bmatrix} 13.5 \\ 11.9 \end{bmatrix} = \begin{bmatrix} -3.5 \\ -2.9 \end{bmatrix}$

Step 3 — iterate

Three contributions per row: AR $\boldsymbol{\Phi}_1\boldsymbol{y}_{t-1}$, exog $\boldsymbol{B}z_t$, MA $\boldsymbol{\Theta}_1\varepsilon_{t-1}$. Values rounded to 4 decimal places.

t	$\boldsymbol{y}_t$	$\boldsymbol{\Phi}_1\boldsymbol{y}_{t-1}$	$\boldsymbol{B}z_t$	$\boldsymbol{\Theta}_1\varepsilon_{t-1}$	$\hat{\boldsymbol{y}}_t$ / ε_t
1	$\begin{bmatrix} 12 \\ 8 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}$ / $\begin{bmatrix} 11.5 \\ 7.5 \end{bmatrix}$
2	$\begin{bmatrix} 10 \\ 9 \end{bmatrix}$	$\begin{bmatrix} 6.4 \\ 5.6 \end{bmatrix}$	$\begin{bmatrix} 1 \\ 1 \end{bmatrix}$	$\begin{bmatrix} 6.1 \\ 5.3 \end{bmatrix}$	$\begin{bmatrix} 13.5 \\ 11.9 \end{bmatrix}$ / $\begin{bmatrix} -3.5 \\ -2.9 \end{bmatrix}$
3	$\begin{bmatrix} 8 \\ 7 \end{bmatrix}$	$\begin{bmatrix} 5.8 \\ 5.6 \end{bmatrix}$	$\begin{bmatrix} 1.5 \\ 1.5 \end{bmatrix}$	$\begin{bmatrix} -1.98 \\ -1.86 \end{bmatrix}$	$\begin{bmatrix} 5.32 \\ 5.24 \end{bmatrix}$ / $\begin{bmatrix} 2.68 \\ 1.76 \end{bmatrix}$
4	$\begin{bmatrix} 11 \\ 6 \end{bmatrix}$	$\begin{bmatrix} 4.6 \\ 4.4 \end{bmatrix}$	$\begin{bmatrix} 2 \\ 2 \end{bmatrix}$	$\begin{bmatrix} 1.424 \\ 1.24 \end{bmatrix}$	$\begin{bmatrix} 8.024 \\ 7.64 \end{bmatrix}$ / $\begin{bmatrix} 2.976 \\ -1.64 \end{bmatrix}$
5	$\begin{bmatrix} 14 \\ 10 \end{bmatrix}$	$\begin{bmatrix} 5.6 \\ 4.6 \end{bmatrix}$	$\begin{bmatrix} 2.5 \\ 2.5 \end{bmatrix}$	$\begin{bmatrix} 0.8624 \\ -0.0608 \end{bmatrix}$	$\begin{bmatrix} 8.9624 \\ 7.0392 \end{bmatrix}$ / $\begin{bmatrix} 5.0376 \\ 2.9608 \end{bmatrix}$
6	$\begin{bmatrix} 12 \\ 11 \end{bmatrix}$	$\begin{bmatrix} 7.6 \\ 6.8 \end{bmatrix}$	$\begin{bmatrix} 3 \\ 3 \end{bmatrix}$	$\begin{bmatrix} 2.6072 \\ 2.1918 \end{bmatrix}$	$\begin{bmatrix} 13.2072 \\ 11.9918 \end{bmatrix}$ / $\begin{bmatrix} -1.2072 \\ -0.9918 \end{bmatrix}$
7	$\begin{bmatrix} 9 \\ 9 \end{bmatrix}$	$\begin{bmatrix} 7 \\ 6.8 \end{bmatrix}$	$\begin{bmatrix} 3.5 \\ 3.5 \end{bmatrix}$	$\begin{bmatrix} -0.6812 \\ -0.6381 \end{bmatrix}$	$\begin{bmatrix} 9.8188 \\ 9.6619 \end{bmatrix}$ / $\begin{bmatrix} -0.8188 \\ -0.6619 \end{bmatrix}$
8	$\begin{bmatrix} 13 \\ 8 \end{bmatrix}$	$\begin{bmatrix} 5.4 \\ 5.4 \end{bmatrix}$	$\begin{bmatrix} 4 \\ 4 \end{bmatrix}$	$\begin{bmatrix} -0.4599 \\ -0.4286 \end{bmatrix}$	$\begin{bmatrix} 8.9401 \\ 8.9714 \end{bmatrix}$ / $\begin{bmatrix} 4.0599 \\ -0.9714 \end{bmatrix}$
9	$\begin{bmatrix} 16 \\ 12 \end{bmatrix}$	$\begin{bmatrix} 6.8 \\ 5.8 \end{bmatrix}$	$\begin{bmatrix} 4.5 \\ 4.5 \end{bmatrix}$	$\begin{bmatrix} 1.4297 \\ 0.4234 \end{bmatrix}$	$\begin{bmatrix} 12.7297 \\ 10.7234 \end{bmatrix}$ / $\begin{bmatrix} 3.2703 \\ 1.2766 \end{bmatrix}$
10	$\begin{bmatrix} 14 \\ 13 \end{bmatrix}$	$\begin{bmatrix} 8.8 \\ 8 \end{bmatrix}$	$\begin{bmatrix} 5 \\ 5 \end{bmatrix}$	$\begin{bmatrix} 1.5634 \\ 1.1647 \end{bmatrix}$	$\begin{bmatrix} 15.3634 \\ 14.1647 \end{bmatrix}$ / $\begin{bmatrix} -1.3634 \\ -1.1647 \end{bmatrix}$

11	11 11	8.2 8	5.5 5.5	−0.7783 −0.7386	12.9217 12.7614	/	−1.9217 −1.7614
12	15 10	6.6 6.6	6 6	−1.1209 −1.0889	11.4791 11.5111	/	3.5209 −1.5111
13	18 14	8 7	6.5 6.5	1.0861 0.0598	15.5861 13.5598	/	2.4139 0.4402
14	16 15	10 9.2	7 7	1.0536 0.6589	18.0536 16.8589	/	−2.0536 −1.8589
15	13 13	9.4 9.2	7.5 7.5	−1.1932 −1.1543	15.7068 15.5457	/	−2.7068 −2.5457
16	17 12	7.8 7.8	8 8	−1.5919 −1.5597	14.2081 14.2403	/	2.7919 −2.2403

162. System Dynamics

163. System Dynamics

A discipline for modeling and simulating **complex dynamic systems** with **feedback**. Developed by Jay Forrester (MIT, 1956) to understand industrial dynamics, urban dynamics, and social systems. Codified by John Sterman (**Business Dynamics**, 2000).

163.1. Core concepts

- **Stocks and flows**: state variables (stocks) change via rates (flows); foundation in ODEs
- **Causal loop diagrams**: graphical representation of feedback structures
- **Feedback loops**: reinforcing (R) vs balancing (B); positive vs negative feedback
- **Delays**: material, information, perception
- **Nonlinearity**: table functions, saturation effects

163.2. When to use

- **Long-term, complex dynamics** with non-obvious feedbacks
- **Policy analysis**: predict consequences of changes before implementing
- **Systems with delays** where intuition fails (planning, supply chains, public health)
- **Stakeholder communication**: visual model helps shared mental models

163.3. When not to use

- **Short-term operational decisions**: discrete-event simulation or OR more appropriate
- **Detailed scheduling**: discrete techniques, not continuous SD
- **Statistical estimation**: regression / time series, not SD
- **Single-decision optimization**: OR / DP, not SD

163.4. Vs other approaches

	System Dynamics	Other
Time	Continuous (ODEs)	Discrete or instantaneous
Focus	Feedback, accumulation	Optimization, statistical fit
Granularity	Aggregate (whole system)	Individual entities
Outcome	Trajectory over time	Optimal decision now
Compare with	OR / DP (optimization), discrete-event sim (entities), agent-based (individuals)	

163.5. Classic SD applications

- **Beer Distribution Game** — supply-chain bullwhip
- **Industrial dynamics** — order-inventory-capacity cycles
- **Bass diffusion** — new product adoption
- **SIR / SEIR** — epidemic spread
- **Population** — predator-prey, demographic transitions

- **Logistic growth** — capacity-limited expansion
- **Public policy** — urban planning, climate scenarios, healthcare

163.6. Software

- **Vensim** (industry standard, free PLE version)
- **Stella / iThink** (Apple-era educational)
- **AnyLogic** (also discrete + agent-based)
- **Python: PySD** (parses Vensim models)

163.7. See also

- **Stocks and Flows**
- **Causal Loop Diagrams**
- **Feedback Loops**
- **Delays**
- **Numerical Integration**
- **Bullwhip in SD**
- **Beer Game**

164. Stocks Flows

The foundation of system dynamics modeling. Two element types:

- **Stocks** (or **levels**) — accumulations: inventory, population, money
- **Flows** (or **rates**) — rates of change: orders, births, spending

Stocks change **only** through flows. Flows change instantaneously based on system state.

164.1. Mathematical foundation

For a stock $S(t)$ with inflow rate $f_{\text{in}}(t)$ and outflow rate $f_{\text{out}}(t)$:

$$\frac{dS}{dt} = f_{\text{in}}(t) - f_{\text{out}}(t)$$

In integral form:

$$S(t) = S(t_0) + \int_{t_0}^{t} [f_{\text{in}}(\tau) - f_{\text{out}}(\tau)] d\tau$$

Each stock obeys this **conservation law**: change in stock = net flow.

164.2. Units

Dimensional consistency is mandatory:

$$[\text{stock}] = [\text{flow}] \cdot [\text{time}]$$

- Inventory in **units**; production rate in **units / day**; integration over time recovers units.
- Money in **dollars**; spending in **dollars / month**.

- Population in **people**; birth rate in **people / year**.

A model with mismatched units is wrong; checking units catches half of all SD modeling errors.

164.3. Three classes of stock-flow systems

First-order linear:

$$\cdot S = -kS + f_{\text{in}}$$

Equivalently: stock decays toward $\frac{f_{\text{in}}}{k}$ with time constant $\frac{1}{k}$. Examples: exponential decay (radioactive substance), goal-seeking (workforce hiring toward target).

Second-order linear: two stocks, coupled

$$\cdot S_1 = aS_1 + bS_2$$
$$\cdot S_2 = cS_1 + dS_2$$

Eigenvalues determine behavior: stable / unstable spirals, exponential growth/decay, oscillation. Predator-prey, mass-spring, etc.

Nonlinear: $\cdot S = f(S)$ with f nonlinear — limit cycles, chaos, bifurcations. Most real-world systems.

164.4. Bathtub metaphor

A common teaching tool: a bathtub with a tap (inflow) and drain (outflow). Water level = stock. Tap and drain rates = flows.

Even simple bathtubs are non-intuitive: most people overestimate how fast filling a bathtub increases water level, because they fail to integrate the flow over time.

164.5. Stocks have memory; flows don't

If you turn off the tap (set inflow to zero), the bathtub doesn't suddenly empty — water remains. Same with inventory after halting production, or accumulated CO2 after stopping emissions. **Stocks persist**.

Conversely, a flow can change instantaneously: stop the inflow → inflow rate is zero immediately.

This distinction is fundamental: policy interventions on **flows** have **delayed** effects on **stocks**.

164.6. Connection to differential equations

Stocks and flows = the language of **ordinary differential equations** (ODEs). SD diagrams are just an alternative notation for systems of ODEs. The visual language helps:

- Identify state variables (= stocks)
- Track causal pathways (= flow equations)
- Communicate model structure to non-modelers

164.7. See also

- **Causal Loop Diagrams** — qualitative version
- **Feedback Loops**
- **Numerical Integration** — for simulation
- **Differential Equations**

165. Causal Loop Diagrams

A **qualitative** diagram showing causal relationships between variables in a system. Used for understanding feedback structure without building a quantitative model.

165.1. Anatomy

Variables (nodes): named quantities, e.g., “Population”, “Birth rate”.

Links (arrows): cause → effect relationships, labeled with polarity:

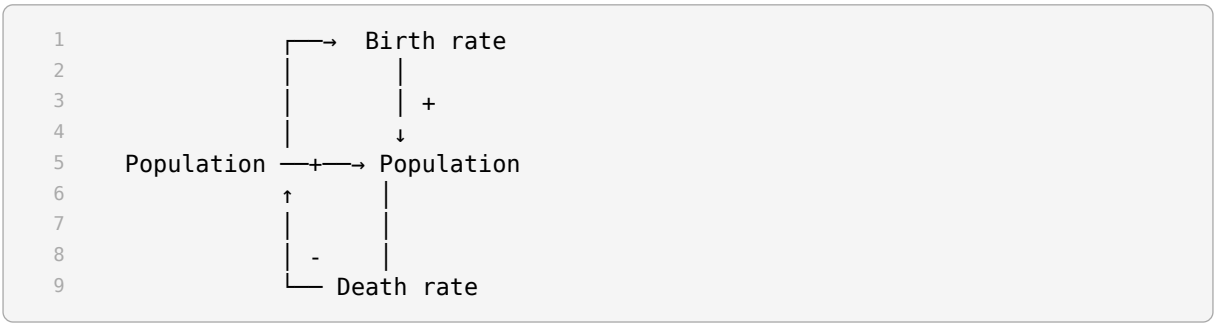
- **Positive link** (+): cause increases → effect increases (and decreases → decreases). Same direction.
- **Negative link** (-): cause increases → effect **decreases** (opposite direction).

165.2. Loop polarity

Trace a loop and multiply link polarities:

- **Reinforcing loop** (R): odd number of negative links ‘multiplied out’ to positive net effect. Self-amplifying.
- **Balancing loop** (B): even number of negatives → net negative. Goal-seeking, stabilizing.

165.3. Example: population growth



Loops:

- **Population → Births → Population**: reinforcing (R) — more people → more births → even more people
- **Population → Deaths → Population**: balancing (B) — more people → more deaths → less population

Behavior depends on which loop dominates. With more births than deaths: exponential growth. With more deaths than births: exponential decay. Balance: steady state.

165.4. Common patterns / archetypes

- **Reinforcing loop**: exponential growth or collapse (population, viral spread, runaway inflation)
- **Balancing loop**: goal-seeking, homeostasis (thermostat, predator-prey, supply-demand equilibrium)
- **Limits to growth**: R + B = S-curve (logistic growth, market penetration)
- **Shifting the burden**: short-term fix, long-term fundamental solution underused
- **Tragedy of the commons**: individual benefit ≠ collective benefit
- **Eroding goals**: gap → lower goal → narrower gap → success theater

165.5. From CLD to quantitative model

A CLD shows **structure**, not **magnitudes**. To predict behavior quantitatively, convert each link into an equation (a stock-flow system):

- **Variables** become stocks or auxiliaries
- **Positive links** with cause = x, effect = $f(x)$ where f is increasing
- **Negative links**: f is decreasing
- Define **table functions** for non-linear effects
- Quantify all parameters

165.6. Common pitfalls

- **Drawing too many links**: keep CLDs focused — 5–15 variables, 10–25 links max
- **Confusing correlation with causation**: only draw a link if A **causes** B (not just correlates)
- **Implicit time scales**: real links have delays; CLDs don't show them — note carefully
- **Bypassing CLDs** and jumping to stock-flow: tempting but loses high-level structure

165.7. See also

- **Stocks and Flows** — quantitative version
- **Feedback Loops**
- **System Dynamics** — overview

166. Feedback Loops

Closed causal chains where an effect comes back to influence its cause. The engine of dynamic behavior in system dynamics.

166.1. Two types

Reinforcing loops (R, positive feedback): the loop **amplifies** changes. Math: $\cdot\ x = kx$ with $k > 0$. Solution: exponential growth $x(t) = x_0 e^{kt}$.

Balancing loops (B, negative feedback): the loop **resists** changes. Math: $\cdot\ x = -kx + kx^*$ with $k > 0$. Solution: exponential approach to target x^* with time constant $\frac{1}{k}$.

166.2. Behavior patterns

Loop structure	Behavior	Example
Pure reinforcing	Exponential growth/collapse	Compound interest, viral spread
Pure balancing	Goal-seeking to steady state	Thermostat, predator-prey equilibrium
R + B (limit)	S-curve (logistic)	Market penetration, learning curves
Two B with delay	Oscillation (damped or sustained)	Inventory cycles, beer game
R + B unstable	Overshoot and collapse	Resource depletion

166.3. First-order behavior

Reinforcing ($\cdot x = gx$):

$$x(t) = x_0 e^{gt}$$

- Doubling time: $T_2 = \frac{\ln(2)}{g} \approx \frac{0.69}{g}$
- Halving time (if $g < 0$): same formula, opposite sign

Balancing ($\cdot x = \frac{x^* - x}{\tau}$):

$$x(t) = x^* + (x_0 - x^*)e^{-\frac{t}{\tau}}$$

- Time constant τ: time to reach 63% of the way to x^*
- Settling time (within 5%): $\approx 3\tau$
- Settling time (within 1%): $\approx 5\tau$

166.4. Combined loops: limits to growth

Reinforcing growth ($\cdot x = gx$) plus balancing loop tied to capacity K:

$$\cdot x = rx\left(1 - \frac{x}{K}\right) \quad \text{(logistic growth)}$$

Initially R dominates $\rightarrow$ exponential growth. As $x \rightarrow K$, the balancing factor $\left(1 - \frac{x}{K}\right)$ shrinks $\cdot x$. Asymptote: $x \rightarrow K$. S-curve.

See Logistic Growth.

166.5. Second-order: oscillation

A loop with **delays** produces oscillation. Simple example: $\cdot\cdot x + 2\zeta\omega \cdot x + \omega^2 x = 0$ (damped harmonic oscillator).

Damping ζ	Behavior	Pattern
0	Sustained oscillation	sine wave
$0 < \zeta < 1$	Damped oscillation	decaying sine
$\zeta = 1$	Critical damping	smoothest approach
$\zeta > 1$	Over-damped	no oscillation, slow approach

166.6. Why feedback loops matter

Most “surprising” system behavior comes from feedback:

- **Bullwhip in supply chains**: balancing loops with delays amplify variance
- **Boom-and-bust cycles**: capacity adjustment delays
- **Population dynamics**: birth/death feedbacks
- **Climate**: ice-albedo, methane release reinforcing loops; weathering balancing loops

Without feedback, a system can’t generate dynamic patterns; it’s just open-loop input → output.

166.7. Identifying loops in a system

1. List variables and how they affect each other
2. Trace cycles in the diagram
3. Compute polarity of each cycle (R or B)
4. Identify dominant loops in each behavior phase

The same model can have R-dominant phase (early growth) and B-dominant phase (saturation).

166.8. See also

- **Causal Loop Diagrams**
- **Stocks and Flows**
- **Logistic Growth**
- **System Dynamics**

167. Delays

Time lags between cause and effect. Critical for explaining oscillation and instability in feedback systems.

167.1. Three kinds

Material delays: physical movement / transformation takes time (shipping, manufacturing, aging).

Information delays: perception / measurement lags (reporting, accounting cycles).

Decision delays: time between observing data and acting (administrative).

167.2. Pipeline (pure) delay

Constant time lag D:

$$O(t) = I(t - D)$$

Output is just input shifted by D. Variance preserved.

167.3. First-order exponential delay

$$\cdot S = I - \frac{S}{D}, \quad O = \frac{S}{D}$$

A stock S holds material in transit; outflow rate is $\frac{S}{D}$. Mean delay D, but each unit's transit time is random (exponentially distributed). High variance.

167.4. Higher-order Erlang delays

Cascade n first-order delays in series, each of length $\frac{D}{n}$:

$$\cdot S_k = (S_{k-1} - S_k)\left(\frac{n}{D}\right), \; k = 1, ..., n$$

Output = $S_n \cdot \left(\frac{n}{D}\right)$.

The impulse response is **Erlang-distributed**: mean D, variance $\frac{D^2}{n}$. As $n \to \infty$, variance $\to 0$ and the cascade approaches a pure pipeline delay.

n	Behavior
1	Pure first-order (high variance, smooth)
3	Visible peak, moderate spread (typical SD default)
10	Tight pulse, low variance
∞	Pipeline delay (no spread)

In Vensim: `DELAY1` (order 1), `DELAY3` (order 3), `DELAYN` (order n).

167.5. Why delays drive oscillation

Imagine a thermostat that responds **late**. The room overheats; the system finally responds, cooling too much; oscillation. The delay turns a balancing loop into an oscillator.

Generic structure:

$$\cdot x = \frac{x^* - \hat{x}}{\tau}, \; \hat{x} = \text{delayed perception of } \; x$$

When $\tau \gg$ delay, smooth approach. When $\tau \approx$ delay, oscillation. When $\tau \ll$ delay, instability.

167.6. Common SD examples

- **Beer game**: each echelon has ordering + shipping delays → bullwhip oscillation
- **Capacity planning**: long construction lead times → boom-bust cycles
- **Workforce**: hiring + training delay → over/under-staffing oscillation
- **Public policy**: legislation lag → policy ineffectiveness

167.7. See also

- **Feedback Loops**
- **Smoothing Operators**
- **Second-Order Systems** — formal oscillation analysis

- **Beer Game**

168. Second-Order

A two-stock system with delay between input and output. The classic **oscillator** structure in system dynamics.

168.1. Generic equation

$$\cdot\cdot x + 2\zeta\omega_n \cdot x + \omega_n^2 x = \omega_n^2 x^*$$

where:

- ω_n: **natural frequency** (radians/time)
- ζ: **damping ratio** (dimensionless)
- x^*: target / equilibrium

168.2. Behavior by ζ

ζ	Type	Behavior
0	Undamped	Pure sinusoidal oscillation, no decay
$0 < \zeta < 1$	Underdamped	Decaying oscillation
$\zeta = 1$	Critically damped	Smoothest non-oscillatory approach
$\zeta > 1$	Overdamped	Slow non-oscillatory approach

168.3. Damped frequency and period

For underdamped systems:

$$\omega_d = \omega_n\sqrt{1-\zeta^2} \quad \text{(damped frequency)}$$

$$T_d = 2\frac{\pi}{\omega_d} \quad \text{(period of oscillation)}$$

168.4. Where 2nd-order systems arise

Whenever a balancing loop has a **delay** equal to (or exceeding) the loop's time constant:

- **Inventory control with order delay**: targeting inventory level via delayed orders
- **Production-inventory**: production responds to inventory gap with lead time
- **Workforce**: hiring decisions based on delayed performance signals
- **Predator-prey**: population responds to delayed food availability

168.5. Beer game equations as 2nd-order

In the Sterman beer game model, each echelon's order rate depends on:

- Current inventory gap
- **Delayed** perception of demand
- Delayed shipments from upstream

Combining gives a 2nd-order ODE in inventory and orders → oscillation visible in real beer-game data and field supply chains.

168.6. Stability via Routh-Hurwitz

For a 2nd-order linear system $a \cdot \cdot x + b \cdot x + cx = 0$, stable iff a, b, c all have the same sign (Routh-Hurwitz criterion). For SD systems, $a > 0$ typically; need $b > 0$ (damping) and $c > 0$ (restoring force) for stability.

Linearization around a fixed point gives this form locally; eigenvalues of the Jacobian determine stability.

168.7. See also

- **Feedback Loops** — 1st-order foundations
- **Delays** — what creates oscillation
- **Phase Plane** — visualization
- **Beer Game** — application

169. Smoothing

Operators that compute smoothed (averaged) versions of input signals. Implemented as exponential delays but used to model **perception** rather than physical transit.

169.1. First-order smoothing (exponential smoothing)

$$d\frac{\hat{X}}{d}t = \frac{X - \hat{X}}{\tau}$$

$\hat{X}$ tracks X with smoothing time τ — equivalent to single-exponential smoothing.

In discrete time: $\hat{X}_t = \alpha X_t + (1 - \alpha)\hat{X}_{t-1}$ with $\alpha = \Delta\frac{t}{\tau}$.

In Vensim: `SMOOTH(input, tau)`.

169.2. Higher-order smoothing

n first-order smooths in series — each one smoother. `SMOOTH3` is third-order (very common default in SD models). Removes more noise but adds more phase lag.

169.3. TREND function

Computes a smoothed **fractional rate of change**:

$$\mathrm{TREND}\big(X, \tau_p, \tau_t\big) = \frac{X - \hat{X}_p}{\hat{X}_p \cdot \tau_t}$$

where $\hat{X}_p$ is X smoothed with time τ_p. Gives "perceived growth rate" — used to model planners' expectations.

In Vensim: `TREND(input, perception_time, trend_time)`.

169.4. FORECAST function

Extrapolates the input forward using TREND:

$$\text{FORECAST}(X, h) = X \cdot (1 + h \cdot \text{TREND})$$

Forecast h time units ahead based on current trend.

Used to model planning under expectations — what someone **thinks** future demand will be.

169.5. Why smoothing models decision-making

People (and organizations) don't react to raw data; they react to **smoothed** versions:

- **Expected demand**: smoothed over weeks / months
- **Perceived inflation**: long-term average, not daily news
- **Expected supply lead time**: averaged over past orders

Smoothing operators in SD capture this **adaptive expectation** behavior. Combined with delays and stocks, they generate the "policy resistance" and stability problems that define Sterman's beer-game style dynamics.

169.6. Phase lag

Smoothing introduces a **delay** in perception:

- **First-order smooth**: lag is approximately τ
- **Third-order smooth**: lag approximately $3\frac{\tau}{3} = \tau$ but with much sharper response
- Higher orders: tighter pulse, slower mean response

Phase lag = part of why supply-chain bullwhip happens: each echelon's reaction is **delayed** relative to actual demand.

169.7. See also

- **Delays** — same mechanism, different framing
- **Stock Management** — uses smoothing for expected demand
- **Exponential Smoothing (ETS)** — discrete-time analog

170. Stock Management

The generic structure for inventory / stock control in system dynamics: order based on **expected loss** + **inventory gap** + **supply-line gap**.

170.1. The classical formula

$$\text{Order} = \text{Expected loss} + \frac{S^* - S}{\tau_S} + \alpha \frac{SL^* - SL}{\tau_{SL}}$$

where:

- S: current stock
- S^*: target stock
- SL: current supply line (on-order, not yet received)
- SL^*: target supply line
- τ_S: time to correct stock gap
- τ_{SL}: time to correct supply-line gap
- $\alpha \in [0, 1]$: **supply-line weighting** (how much you account for orders already placed)
- Expected loss: replacement for expected demand

170.2. The supply-line neglect parameter

α is critical:

- $\alpha = 1$: fully account for on-order inventory → stable, no over-ordering
- $\alpha = 0$: **ignore the supply line entirely** → over-order while waiting → oscillation

Sterman's beer-game experiments find $\alpha \approx 0.3$ on average — most subjects substantially neglect the supply line.

170.3. Anchoring and adjustment

Generic decision-rule structure:

$$\text{Decision} = \text{Anchor} + \alpha \cdot (\text{Cue} - \text{Anchor})$$

- **Anchor**: the default level (e.g., expected demand)
- **Cue**: corrective signal (e.g., inventory gap)
- α: partial-adjustment coefficient

This is the **anchoring-and-adjustment heuristic** (Tversky-Kahneman) operationalized as a formula. Captures bounded rationality.

170.4. Why this generates bullwhip

The chain of reasoning that produces bullwhip:

1. Retailer sees demand spike → expected demand rises
2. Inventory gap grows → orders bigger (anchor + correction)
3. Supply-line neglect → keeps ordering even though previous orders are in transit
4. Wholesaler sees retailer's amplified orders → repeats the pattern
5. Manufacturer sees wholesaler's even more amplified orders → repeats

Each echelon amplifies the variance. The math: see Bullwhip in SD.

170.5. Real-world example

A factory targets 100 units of inventory ($S^* = 100$). Currently has 60 ($S = 60$). On-order 50 units ($SL = 50$). Expected demand 20 units/period.

$$\text{Order} = 20 + \frac{100 - 60}{4} + 0.3 \cdot \frac{100 - 50}{4} = 20 + 10 + 3.75 = 33.75$$

Settings: $\tau_S = \tau_{SL} = 4$ periods, $\alpha = 0.3$, $SL^* = 100$ (an order-of-magnitude approximation: target supply-line equal to target stock).

170.6. See also

- **Feedback Loops**
- **Smoothing** — for expected demand
- **Bullwhip in SD**
- **Beer Game**
- **Bullwhip Effect (Supply Chain)**

171. Bullwhip SD

The **quantitative** analysis of supply-chain order amplification, from the system-dynamics perspective. Closed-form result by Chen, Drezner, Ryan, Simchi-Levi (2000) for the **order-up-to** (OUT) policy with AR(1) demand.

171.1. Chen-Drezner-Ryan-Simchi-Levi formula

For demand D_t following an AR(1) process and an OUT policy with smoothing parameter p and lead time L:

$$\frac{\mathrm{Var}(\text{orders})}{\mathrm{Var}(\text{demand})} = 1 + \frac{2L}{p} + \frac{2L^2}{p^2}$$

The order variance is always **at least** the demand variance, and grows quadratically with $\frac{L}{p}$ ratio. For two-echelon chains, this multiplies stage by stage.

171.2. Key insights

- **Lead time matters most**: doubling L roughly quadruples the bullwhip
- **Smoothing matters too**: longer smoothing (p) reduces bullwhip but adds delay
- **No exact balance**: even optimal p doesn't achieve $\mathrm{Var}(O) = \mathrm{Var}(D)$ — always amplified

171.3. Sources of bullwhip (Lee-Padmanabhan-Whang 1997)

Four causes, each contributing variance amplification:

1. **Demand signal processing**: each echelon uses smoothed orders (not actual demand) — like the Chen-DRSL formula
2. **Order batching**: large infrequent orders smooth retailer's demand into spikes upstream
3. **Price fluctuations**: forward-buying during promotions creates spikes
4. **Rationing & shortage gaming**: customers over-order during shortages, creating ghost demand

Each cause is amplified by long lead times.

171.4. SD beer-game replication

The Sterman beer-game model implements this dynamically:

- Each echelon's decision-rule (anchoring-adjustment with supply-line neglect) propagates variance
- Initial customer-demand step (small) gets amplified at each echelon
- Final factory order is 5-10x the customer-demand variance — matching experimental data

This is the **quantitative validation** of the SD bullwhip explanation.

171.5. Mitigations

What reduces bullwhip:

- **Short lead times**: L down → quadratic reduction
- **Information sharing**: upstream sees real customer demand, not amplified orders (CPFR, VMI)
- **Order smoothing**: don't fully correct gaps instantly
- **Stable pricing**: avoid promotion-induced spikes
- **Capacity allocation rules**: use historical demand, not current orders (avoid rationing game)

171.6. See also

- **Stock Management** — the decision rule
- **Beer Game** — simulation
- **Bullwhip Effect (Supply Chain)** — broader view
- **Delays** — what enables amplification

172. Beer Game

The **Beer Distribution Game** (MIT Sloan, 1960s) — a tabletop / simulation exercise demonstrating the bullwhip effect. The most-played management game ever.

172.1. Setup

Four-stage supply chain: Retailer → Wholesaler → Distributor → Factory. Each stage:

- Receives orders from downstream
- Ships goods to downstream
- Places orders upstream
- Carries inventory; pays **holding cost** and **backorder cost**

There's a **2-week shipping delay** between stages and **1-week order delay**. Initially, demand is 4 cases/week (stable).

172.2. The shock

After 4 weeks, retail customer demand **steps up** to 8 cases/week and stays there.

172.3. The result (consistent across hundreds of plays)

Despite the step being small (4 → 8), upstream stages see:

- Factory orders peak at 20-40 cases/week
- All stages alternate between massive backorders and massive overstocks
- Total system cost is **10-30× higher** than the optimum

Even MIT students consistently exhibit this behavior. The bullwhip is **structural**, not a personal failure.

172.4. Sterman's decision-rule estimate

Sterman (1989) fit each player's order rule to:

$$\text{Order}_t = \max\left\{0, \hat{D}_t + \alpha_S(S^* - S_t) + \alpha_{SL}(SL^* - SL_t)\right\}$$

where $\hat{D}_t$ is smoothed demand. He found:

- $\alpha_S \approx 0.36$ (modest correction speed)
- $\alpha_{SL} \approx 0.26$ — but more critically:
- Players **underweight** the supply line (most attention on S, less on SL)

This **supply-line neglect** drives oscillation: players keep ordering even though previous orders are en route.

172.5. What works

After playing several rounds, players learn:

1. **Slow down**: smaller corrections per period
2. **Pay attention to the supply line**: full α_{SL}
3. **Smooth demand**: don't react to every blip
4. **Information sharing**: when upstream sees end-customer demand (not just downstream orders), bullwhip largely vanishes

172.6. Mathematical structure

Each echelon is a 2nd-order delayed feedback system (here). Four cascaded → high-order oscillator. With shock input + supply-line neglect → sustained large oscillation.

172.7. Variations

- **Beer Game online**: Powersim, Forio, MIT's online version
- **Reduced-delay variant**: shorter lead times → less bullwhip
- **Information-sharing variant**: all stages see end-demand → bullwhip eliminated

172.8. Why it matters

The beer game is the **empirical** answer to "do real people exhibit bullwhip?" Answer: yes, robustly. It validates the SD theory and motivates:

- Lead-time reduction
- Information sharing (CPFR, VMI, EDI)
- Order-policy training
- Centralized planning

172.9. See also

- **Bullwhip in SD** — closed-form analysis
- **Stock Management** — the decision rule
- **Bullwhip Effect**
- **Delays**

173. Bass Diffusion

A simple SD model of new-product adoption. Frank Bass (1969). Predicts the characteristic **S-shaped** adoption curve from two parameters.

173.1. Model

Let $A(t)$ = cumulative adopters, N = total market size. Adoption rate:

$$\cdot A = \left(p + q\frac{A}{N}\right)(N - A)$$

Two parameters:

- p: **coefficient of innovation** — adoption driven by external influence (advertising, news, intrinsic appeal)
- q: **coefficient of imitation** — adoption driven by word-of-mouth (existing adopters influencing potential adopters)

173.2. Structure

Two interpreting terms:

- $p(N - A)$: **innovation** — fraction of remaining non-adopters who'd adopt based on external info
- $q\left(\frac{A}{N}\right)(N - A)$: **imitation** — non-adopters influenced by the proportion $\frac{A}{N}$ of current adopters; total imitation pressure proportional to remaining market

Together: a **limits-to-growth** archetype with reinforcing word-of-mouth and balancing market exhaustion.

173.3. Behavior

Early on ($A \ll N$): adoption is mostly pN — small, driven by innovation only.

Middle: A approaches $\frac{N}{2}$; imitation term peaks, growing very fast.

Late: $N - A$ shrinks; growth slows.

Peak adoption rate at:

$$A^* = \frac{N(q-p)}{2q}$$

Time of peak adoption:

$$t^* = \frac{\ln\left(\frac{q}{p}\right)}{p+q}$$

For typical $p \approx 0.01 - 0.05$, $q \approx 0.3 - 0.5$: peak around 5-10 years after launch.

173.4. Empirical fit

Bass fit the model to historical sales of color TVs, washing machines, and dozens of other durables. Excellent S-curve fits with just p, q as parameters.

Common parameter values:

- **Consumer electronics**: $p \approx 0.03$, $q \approx 0.4$
- **Pharmaceuticals**: $p \approx 0.001$, $q \approx 0.5$
- **Software platforms**: $p \approx 0.01 - 0.05$, $q \approx 0.5 - 0.7$

173.5. Generalizations

- **Generalized Bass**: include effect of price, advertising $\rightarrow p(t), q(t)$ time-varying
- **Repeat purchase**: cumulative installed base $\neq$ cumulative purchases
- **Substitute / next-generation** products: previous-generation forms downstream pool

173.6. Why it matters

The Bass model is **the** default for:

- **New-product launch** forecasting
- **Marketing budget** allocation across product lifecycle stages
- **Technology adoption** studies (Rogers' **Diffusion of Innovations**)

Most consumer-electronics product launches use Bass as the baseline forecast.

173.7. See also

- **Logistic Growth** — similar S-curve, simpler model
- **Feedback Loops** — R + B = S-curve archetype
- **System Dynamics overview**

174. Logistic Growth

The simplest **limits-to-growth** model. A population (or any quantity) grows exponentially when small, then saturates as it approaches a **carrying capacity** K.

174.1. Model

$$\cdot N = rN\left(1 - \frac{N}{K}\right)$$

- r: intrinsic growth rate (per unit time)
- K: carrying capacity (max sustainable level)
- N: current level

174.2. Closed-form solution

$$N(t) = \frac{K}{1 + Ae^{-rt}}, \quad A = \frac{K - N_0}{N_0}$$

S-curve from N_0 asymptoting to K.

174.3. Phases

Phase	Behavior	Where you are
Exponential	$\cdot N \approx rN$	$N \ll K$, factor $\left(1 - \frac{N}{K}\right) \approx 1$
Inflection	$\cdot N$ peaks	$N = \frac{K}{2}$, maximum growth rate
Saturation	$\cdot N$ decays	$N \to K$, growth tapers off

174.4. Structurally

Reinforcing growth ($\cdot N = rN$) plus balancing limit ($1 - \frac{N}{K}$). The R+B = S-curve archetype.

174.5. Where it shows up

- **Population biology**: limited food supply, predation
- **Bacterial growth**: nutrient depletion
- **Market penetration**: saturating consumer base
- **Technology adoption**: total addressable market
- **Renewable resources** under harvest: $\cdot R = rR\left(1 - \frac{R}{K}\right) - hR$ (logistic + harvest)
- **Tumor growth** (Gompertz model is a variant)

174.6. Limitations

- **Symmetric**: growth and saturation phases are mirror images — real systems often asymmetric
- **Static K**: real carrying capacities change (technology, environment)
- **No overshoot**: real systems often overshoot K then collapse — see oscillation / collapse archetypes

174.7. Compared to Bass diffusion

	Logistic	Bass
Initial growth	Exponential (rN)	Innovation rate (pN, constant)
Driver	Word-of-mouth only (r effectively combines)	Innovation + imitation separately
Use case	Pure peer-driven growth	New product with external + internal drivers

Bass is logistic-like but separates the two drivers — better for marketing analysis.

174.8. Parameter estimation

Given time-series data (t_i, N_i):

1. Plot $N(t)$ — confirm S-shape
2. Estimate K from asymptote (or as a parameter to fit)
3. Linearize: $\ln(K - N) - \ln N = \ln A - rt$, fit a line; slope $= -r$, intercept $= \ln A$

Or just nonlinear-least-squares fit the closed form.

174.9. See also

- **Bass Diffusion** — refinement
- **Feedback Loops** — the R+B archetype
- **Differential Equations** — the parent topic

175. SIR SEIR

Compartmental models of **epidemic spread**. The mathematical foundation of mainstream epidemiology.

175.1. SIR model

Three stocks:

- S: Susceptible
- I: Infected
- R: Recovered (immune)

Equations:

$$\cdot S = -\beta S \frac{I}{N}$$

$$\cdot I = \beta S \frac{I}{N} - \gamma I$$

$$\cdot R = \gamma I$$

with:

- β: transmission rate (contacts × probability per contact)
- γ: recovery rate ($\frac{1}{\gamma}$ = mean infectious period)
- $N = S + I + R$ (total population, conserved)

175.2. Basic reproduction number R_0

The most-cited epidemic metric:

$$R_0 = \frac{\beta}{\gamma}$$

Interpretation: average number of secondary infections from one infected person in a fully-susceptible population.

- $R_0 > 1$: epidemic grows
- $R_0 < 1$: epidemic dies out
- $R_0 = 1$: marginal — slow spread

For COVID-19 wild-type: $R_0 \approx 2.5 - 3$. For measles: $R_0 \approx 12 - 18$. For influenza: $R_0 \approx 1 - 2$.

175.3. Herd immunity threshold

The fraction of population that must be immune to halt spread:

$$1 - \frac{1}{R_0}$$

For $R_0 = 3$: 67% needs to be immune. For $R_0 = 18$ (measles): 94%.

Vaccines / prior infection drive immune fraction up; once you cross the threshold, sustained transmission is impossible (in this simple model).

175.4. SEIR: add Exposed

Adds an **Exposed** (infected but not yet contagious) compartment:

$$\cdot E = \beta S \frac{I}{N} - \sigma E$$

$$\cdot I = \sigma E - \gamma I$$

with $\frac{1}{\sigma}$ = mean incubation period.

For COVID-19, SEIR is more realistic (significant incubation period).

175.5. Behavior

Epidemic curve typically S-shaped (in cumulative cases) and bell-shaped (in new cases per day):

- Exponential growth phase: $I(t) \approx I_0 e^{(\beta-\gamma)t}$
- Peak when $S(t) = \frac{N}{R_0}$ (= herd-immunity threshold)
- Decay phase

175.6. Variants

- **SIRS**: lose immunity over time (return $R \to S$); endemic equilibrium possible
- **SIS**: no immunity at all (return $I \to S$); endemic equilibrium with constant I
- **Age-structured**: multiple age compartments with different mixing rates
- **Spatial**: include geographic structure
- **Network**: contacts on a graph; heterogeneous mixing

175.7. Where used

- **Public health planning**: vaccination targets, lockdown timing
- **Pharmaceutical R&D**: market size estimation for vaccines / antivirals
- **Risk modeling**: insurance, business continuity
- **Behavioral economics**: viral information spread (memes, news)

175.8. Limitations

- **Well-mixed** assumption: real contacts are heterogeneous
- **Constant parameters**: real R_0 changes with seasonality, behavior, variants
- **Deterministic**: small populations need stochastic models (Gillespie, branching processes)

175.9. See also

- **Logistic Growth** — similar S-curve dynamics
- **Stocks and Flows** — modeling language
- **Feedback Loops** — the underlying R+B structure

176. Lotka-Volterra

The classical **predator-prey** model. Two species:

$$\dot{x} = \alpha x - \beta x y \qquad \text{prey}$$

$$\dot{y} = \delta x y - \gamma y \qquad \text{predator}$$

where:

- x: prey population, y: predator population
- α: prey growth rate (unlimited food, no predator)
- β: predation rate
- δ: predator growth per prey eaten
- γ: predator death rate (no prey)

176.1. Behavior: closed orbits in the phase plane

Equilibrium: $(x^*, y^*) = \left(\frac{\gamma}{\delta}, \frac{\alpha}{\beta}\right)$.

Around the equilibrium, the system **oscillates** in closed orbits (in (x, y) space). Pre-mathematical solution by Lotka (1925) and Volterra (1926).

Energy-like invariant:

$$H(x, y) = \delta x + \beta y - \gamma \ln x - \alpha \ln y = \text{const}$$

Trajectories follow contours of constant H.

176.2. Oscillation dynamics

Cycle structure:

1. Prey abundant → predators thrive (predator pop rises)
2. Predators abundant → prey decline (prey pop falls)
3. Prey scarce → predators decline (predator pop falls)
4. Predators scarce → prey rebound (prey pop rises)
5. Cycle repeats

Lynx-hare population data (Hudson Bay records) shows this pattern beautifully.

176.3. Limitations and extensions

The pure Lotka-Volterra has issues:

- **Closed orbits** (mathematically) → no damping. Real systems usually damp out or destabilize.
- **Unlimited prey** if no predator. Add a carrying capacity:

$$\cdot x = \alpha x\left(1 - \frac{x}{K}\right) - \beta xy$$

This logistic version has **stable spiral** or **limit cycle** depending on parameters.

- **Functional response** (Holling Type II): predator becomes saturated:

$$\cdot x = \alpha x - \frac{\beta xy}{1 + hx}$$

with h = handling time. Can produce limit cycles or even chaos.

176.4. Beyond two species

Generalizes to n-species:

$$\cdot x_i = x_i\left(r_i + \sum_j a_{ij}x_j\right)$$

with a_{ij} = interaction matrix. Models food webs, ecological communities.

176.5. Outside biology

The model is structurally fundamental — appears wherever two stocks interact with the "predator gains by consuming prey" pattern:

- **Sales force vs prospect pool**: salespeople convert prospects (prey) and decline without them
- **Consumer behavior**: cyclical fashion (predator) consumes available trends (prey)
- **Marketing budgets vs customer base**

176.6. See also

- **Feedback Loops** — oscillation from coupled feedback
- **Phase Plane** — visualization of trajectories
- **SIR/SEIR** — related compartmental dynamics

177. Phase Plane

A graphical method for analyzing 2-state dynamical systems. Plot the two state variables on axes; the system's trajectory traces a curve in this plane.

177.1. For a 2-state system

$$\cdot x = f(x, y), \ \cdot y = g(x, y)$$

Each (x, y) point has a **velocity vector** $(\cdot x, \cdot y)$. The phase portrait is the field of these vectors plus typical trajectories.

177.2. Key features

- **Equilibria / fixed points**: (x^*, y^*) where $f = g = 0$. The system rests there.
- **Nullclines**: curves where $\cdot x = 0$ (the x-nullcline) or $\cdot y = 0$ (the y-nullcline). Equilibria lie at intersections.
- **Trajectories**: integral curves of the vector field
- **Separatrices**: trajectories that divide the plane into regions of different long-term behavior

177.3. Linearizing at an equilibrium

Near (x^*, y^*), Taylor-expand f, g:

$$\begin{bmatrix} \cdot x \\ \cdot y \end{bmatrix} \approx J \begin{bmatrix} x - x^* \\ y - y^* \end{bmatrix}$$

where J is the Jacobian:

$$J = \begin{bmatrix} \frac{\partial f}{\partial x} & \frac{\partial f}{\partial y} \\ \frac{\partial g}{\partial x} & \frac{\partial g}{\partial y} \end{bmatrix}_{(x^*, y^*)}$$

The **eigenvalues** of J classify the equilibrium.

177.4. Classification by eigenvalues

Eigenvalues	Type	Behavior
Both real, both < 0	Stable node	Trajectories approach (x^*, y^*) from all directions
Both real, both > 0	Unstable node	Trajectories diverge from (x^*, y^*)
Real, opposite signs	Saddle	Stable along one direction, unstable along other; separatrix
Complex, real part < 0	Stable spiral	Trajectories spiral inward
Complex, real part > 0	Unstable spiral	Trajectories spiral outward
Pure imaginary	Center	Closed orbits around equilibrium (oscillation)

The **trace-determinant plane** visualizes these regimes:

- $\tau = \mathrm{tr}(J), \Delta = \det(J)$
- $\Delta > 0, \tau < 0$: stable (node or spiral)
- $\Delta > 0, \tau > 0$: unstable (node or spiral)
- $\Delta < 0$: saddle (always unstable)
- $\Delta > 0, \tau^2 < 4\Delta$: spiral; otherwise: node

177.5. Lotka-Volterra as example

$$\cdot x = \alpha x - \beta xy, \ \cdot y = \delta xy - \gamma y$$

Equilibrium at $\left(\frac{\gamma}{\delta}, \frac{\alpha}{\beta}\right)$.

Jacobian at equilibrium:

$$J = \begin{bmatrix} 0 & -\beta\frac{\gamma}{\delta} \\ \delta\frac{\alpha}{\beta} & 0 \end{bmatrix}$$

Trace $= 0$, $\det > 0$. Pure imaginary eigenvalues → **center** → closed orbits. (Confirms Lotka-Volterra's well-known oscillation.)

177.6. Limit cycles and chaos

For **nonlinear** systems with appropriate structure, isolated closed orbits (**limit cycles**) appear. Examples:

- van der Pol oscillator
- Predator-prey with Holling functional response
- Belousov-Zhabotinsky chemical reaction

In dimension ≥ 3, chaos becomes possible (Lorenz attractor, etc.). 2-D systems can't be chaotic by Poincaré-Bendixson theorem.

177.7. Routh-Hurwitz stability for higher dimensions

For 3+ states, eigenvalue computation generalizes. The **Routh-Hurwitz criterion** gives an algorithmic stability test from coefficient signs of the characteristic polynomial.

177.8. See also

- **Feedback Loops**
- **Second-Order Systems**
- **Lotka-Volterra** — phase plane example
- **Differential Equations**
- **Eigenvectors / Eigenvalues** — for the Jacobian

178. Numerical Integration

Methods for **numerically integrating** the ODEs of a system-dynamics model. Critical for simulation accuracy.

178.1. Euler method (default for SD)

$$S_{t+\Delta t} = S_t + \Delta t \cdot (\text{net flow}_t)$$

Forward Euler. Simple, fast, error per step $O(\Delta t^2)$, total error $O(\Delta t)$.

For SD models, Euler with a small enough Δt is usually good enough.

178.2. Time step (Δt) selection rule

$$\Delta t \leq \frac{\min(\tau_i)}{4}$$

where τ_i are the time constants of the system. Rule of thumb: divide the **fastest** time constant by 4-10.

Why? At $\Delta t = \frac{\tau}{4}$, Euler captures the dominant time scale; faster than that, all time scales are well-resolved.

178.3. Runge-Kutta 2 (RK2 / midpoint)

$$k_1 = f(S_t, t)$$

$$k_2 = f\left(S_t + \left(\Delta\frac{t}{2}\right)k_1, t + \Delta\frac{t}{2}\right)$$

$$S_{t+\Delta t} = S_t + \Delta t \cdot k_2$$

Error per step $O(\Delta t^3)$, total $O(\Delta t^2)$. Twice the work of Euler per step, better accuracy.

178.4. Runge-Kutta 4 (RK4)

The “classical” RK4:

$$k_1 = f(S_t, t)$$

$$k_2 = f\left(S_t + \left(\Delta\frac{t}{2}\right)k_1, t + \Delta\frac{t}{2}\right)$$

$$k_3 = f\left(S_t + \left(\Delta\frac{t}{2}\right)k_2, t + \Delta\frac{t}{2}\right)$$

$$k_4 = f(S_t + \Delta t \cdot k_3, t + \Delta t)$$

$$S_{t+\Delta t} = S_t + \left(\Delta\frac{t}{6}\right)(k_1 + 2k_2 + 2k_3 + k_4)$$

Error per step $O(\Delta t^5)$, total $O(\Delta t^4)$. Four times the work per step, much higher accuracy.

178.5. Which to use?

Method	Per-step cost	Per-step error	Best for
Euler	1 eval	$O(\Delta t^2)$	Most SD models, small Δt
RK2	2 evals	$O(\Delta t^3)$	Moderate accuracy needs

Method	Per-step cost	Per-step error	Best for
RK4	4 evals	$O(\Delta t^5)$	High-accuracy / oscillatory systems

For smooth SD models with $\Delta t = \frac{\tau_{\min}}{4}$: Euler is fine. For **oscillatory** or **stiff** systems: RK4 or adaptive methods.

178.6. Stiff systems

When time constants span orders of magnitude (e.g., $\tau_1 = 1$, $\tau_2 = 1000$), explicit methods (Euler, RK) need tiny Δt to stay stable on the fast time scale, wasting effort on the slow.

Implicit methods (backward Euler, BDF) trade per-step cost for stability — large Δt allowed.

In SD, stiff systems are rare; explicit methods are usually sufficient.

178.7. Initialization in equilibrium

Common SD practice: set initial stocks so that **all flows balance** at $t = 0$ (system at steady state). Then study the response to a perturbation.

This isolates the **dynamic** response from initial-condition transients.

178.8. See also

- **Stocks and Flows** — the equations being integrated
- **System Dynamics overview**
- **Differential Equations**

179. Validation

Tests for confirming that a system-dynamics model represents reality usefully. Sterman (2000, Ch. 21) gives a canonical 12-test battery, divided into **structure-oriented** and **behavior-oriented** tests.

179.1. Structure-oriented tests (does the model capture what it should?)

- **Boundary adequacy**: are the right things inside vs outside the model?
- **Structure assessment**: do the relationships match real-world causal structure?
- **Dimensional consistency**: units balance in every equation
- **Parameter assessment**: parameter values consistent with literature and expert estimates
- **Extreme conditions test**: behavior at extremes (zero inventory, infinite capacity) sensible
- **Integration error test**: refine Δt; if behavior changes, integrator is too coarse

179.2. Behavior-oriented tests (does the model produce realistic output?)

- **Behavior reproduction**: match historical / observed data
- **Behavior anomaly**: if you simulate a known anomaly (delay, shock), the model exhibits it
- **Family member**: same structure should explain same phenomena across similar systems
- **Surprise behavior**: model should reveal **new** dynamic patterns not built-in
- **Sensitivity analysis**: behavior changes "reasonably" with parameter perturbations
- **System improvement**: model leads to useful policy insight

179.3. Quantitative fit: Theil's U decomposition

For predicted $\hat{y}_t$ vs actual y_t:

$$U = \sqrt{\frac{\text{MSE}}{\sigma_{\hat{y}}^2 + \sigma_y^2 + (|y| - |\hat{y}|)^2}}$$

Decomposes MSE into three components:

- U^M (bias): systematic mean error
- U^S (variance): scale of variability
- U^C (covariance): direction / phase

$U^M + U^S + U^C = 1$. Ideal model: U^M and U^S small, U^C large — model captures direction even if magnitudes slightly off.

179.4. Behavior modes matter more than point fits

For SD models, **qualitative behavior** (does it overshoot? oscillate? saturate?) is usually more valuable than precise fit. A model that predicts the **right pattern** of bullwhip with the **wrong amplitude** is better than one with right amplitude but no pattern.

Sterman emphasizes: don't fixate on RMSE; check that the **behavior modes** match.

179.5. Calibration methods

For data-fitting:

- **FIMLOF** (Full Information Maximum Likelihood with Optimal Filtering): rigorous statistical calibration including Kalman-filter-style observation noise
- **Vensim Powell search**: gradient-based optimization on RMSE
- **Bayesian calibration**: posterior distributions over parameters given data

For **decision-rule parameters** (e.g., beer-game α_S, α_{SL}): typically estimated by least-squares from gameplay data.

179.6. Common pitfalls

- **Over-fitting** to historical data — many degrees of freedom in an SD model

- **Ignoring extreme tests** — model “fits” but breaks at boundaries
- **Skipping unit checks** — about half of model bugs are unit errors
- **Confidence over-reach** — SD models are **insightful**, not **predictive** in the forecasting sense

179.7. Sensitivity analysis types

- **Univariate**: vary one parameter at a time; identify high-leverage variables
- **Monte Carlo / Latin Hypercube**: sample parameter combinations; build envelopes of model behavior
- **Tornado diagram**: rank parameters by impact on a metric

179.8. See also

- **System Dynamics overview**
- **Numerical Integration** — integration-error test
- **Monte Carlo Simulation** — for sensitivity

180. Operations Research

181. Optimization

181.1. Cheatsheet

181.1.1. Linear Programming

Function Type	Method	Example
Maximization to Minimization	$\max \quad f(x) \iff$ $\min \ -f(x)$	$\max \quad 3x + 2y \iff$ $\min -3x - 2y$
$\geq$ Constraint to $\leq$	$g_i(x) \geq \quad b_i \iff$ $-g_i(x) \leq -\ b_i$	$x + y \geq \quad 5 \iff$ $-x - y \leq -5$
Equality Constraint	$g_i(x) = b_i \iff$ $g_i(x) \geq b_i \wedge$ $g_i(x) \leq b_i$	$x + y = 4 \iff$ $x + y \geq 4 \wedge$ $x + y \leq 4$
Slack Variable		$x + y \leq 6$ Add slack variable $s \geq 0$: $x + y + s = 6$ s is how much less is being used than the maximum limit of 6
Surplus Variable		$x + y \geq 6$ Add slack variable $s \geq 0$: $x + y - s = 6$ s is the amount above the minimum required level 6

181.1.2. Integer Programming

Function Type	Method	Example
At **least** k of the items in S	$\sum_{i \in S} x_i \geq k$	Produce at least 3 products: $\sum_{i \in S} x_i \geq 3$

Function Type	Method	Example
At **most** k of the items in S	$\sum_{i \in S} x_i \leq k$	Open at most 2 facilities: $\sum_{i \in S} x_i \leq 2$
Exactly k of the items in S	$\sum_{i \in S} x_i = k$	Choose exactly one location: $\sum_{i \in S} x_i = 1$
If $x_a = 1$, then $x_b = 1$	$x_a \leq x_b$	If project A is done, project B must also be done
If $x_a = 1$, then $x_b = 0$	$x_a + x_b \leq 1$	Projects A and B are mutually exclusive
If $x_a = 1$, then $x_b + x_c \leq 0$	$x_b + x_c \leq M(1 - x_a)$	If project A is selected, neither project B nor C can be selected
If $x_j = 0$, then at least k items in S must be selected	$\sum_{i \in S} x_i \geq k(1 - x_j)$	You have a main server represented by x_j. If it is down or not used ($x_j = 0$), then at least k backup servers (in set S) must be activated
At least k of m constraints satisfied	$z_i = \begin{cases} 1 \text{ if } g_i(x) \leq b_i \text{ enforced} \\ 0 \text{ if } g_i(x) \leq b_i \text{ relaxed} \end{cases}$ for $i = 1, ..., m$ $g_i(x) - b_i \leq M_i(1 - z_i) \quad \forall i = 1, ..., m$ $\sum_{i=1}^{m} z_i \geq k$ $M_i \geq \max_x (g_i(x) - b_i)$	Enforce any k of the m constraints by relaxing up to $m - k$ of them
Fixed (Setup) Costs	$y = \begin{cases} 1 \text{ if is used} \\ 0 \text{ if not used} \end{cases}$ $x \geq 0$ $x \leq M \cdot y$	If facility is used ($y = 1$), then allow production up to M; else $x = 0$

Function Type	Method	Example
Balanced Flow	$\sum_{\substack{i \in V \\ i \neq k}} x_{ik} = d_k$ $\sum_{\substack{i \in V \\ i \neq k}} x_{jk} = d_k$	Each node k receives and sends exactly d_k units of flow. If $d_k = 1$, this is a TSP or assignment
Unbalanced Flow	$\sum_{\substack{i \in V \\ i \neq k}} x_{ik} - \sum_{\substack{i \in V \\ i \neq k}} x_{jk} = b_k$	Each node k has net inflow minus outflow equal to b_k: • $b_k > 0$: demand • $b_k < 0$: supply • $b_k = 0$: transshipment
Subtours (MTZ)	$1 \leq u_i \leq n-1 \quad \forall i = 2, ..., n$ $u_i - u_j + (n-1)x_{ij} \leq n-2$ $\forall i \neq j, i, j \in \{2, ..., n\}$	
Subtours (SEC)	$\sum_{\substack{i \in S \\ j \neq j}} \sum_{j \in S} x_{ij} \leq \lvert S \rvert - 1 \quad \forall S \subseteq V, \lvert S \rvert \geq 2$	For every subset S of the nodes V that has at least 2 nodes, the total number of arcs (or paths) within that subset — from node i to node j, where $i \neq j$ must be less than or equal to $\lvert S \rvert - 1$

181.1.3. Non-Linear Programming

Function Type	Example	Linearization Method	Example
Binary × Binary Constraint	$z = x \cdot y$ $x, y \in \{0, 1\}$	$z \leq x$ $z \leq y$ $z \geq x + y - 1$ $x \in \{0, 1\}$	$z = 1 \iff$ $x = y = 1$
Binary × Continuous Constraint	$z = x \cdot y$ $x \in \{0, 1\}$ $y \in [L, U]$	$z \leq Ux$ $z \geq Lx$ $z \leq y - L(1 - x)$ $z \geq y - U(1 - x)$	
Continuous × Continuous Constraint	$z = x \cdot y$ $x \in [L_x, U_x]$ $y \in [L_y, U_y]$	$z \geq L_x y + L_y x - L_x L_y$ $z \geq U_x y + U_y x - U_x U_y$ $z \geq L_x y + U_y x - L_x U_y$ $z \geq U_x y + L_y x - U_x L_y$	
Absolute Value	$z = \lvert x \rvert$	$z \geq x$ $z \geq -x$	$z = \lvert x_1 - x_2 \rvert$ $\iff$ $z \geq x_1 - x_2$ $z \geq x_2 - x_1$
Max Constraint	$y \geq \max(x_1, ..., x_n)$	$y \geq x_1 \quad \forall i = 1, ..., n$	$y + x_1 + 3 \geq \max(x_1 - x_3, 2x_2 + 4)$ $\iff$ $\begin{cases} y + x_1 + 3 \geq x_1 - x_3 \\ y + x_1 + 3 \geq 2x_2 + 4 \end{cases}$
Min Constraint	$y \leq \min(x_1, ..., x_n)$	$y \leq x_1 \quad \forall i = 1, ..., n$	$y + x_1 \leq \min(x_1 - x_3, 2x_2 + 4, 0)$ $\iff$ $\begin{cases} y + x_1 \leq x_1 - x_3 \\ y + x_1 \leq 2x_2 + 4 \\ y + x_1 \leq 0 \end{cases}$
Max Min Objective	$\max \min(x_1, ..., x_n)$	$y = \min(x_1, ..., x_n)$ $y \leq x_i \quad \forall i = 1, ..., n$	
Min Max Objective	$\min \max(x_1, ..., x_n)$	$y = \max(x_1, ..., x_n)$ $y \geq x_i \quad \forall i = 1, ..., n$	
Min Min Objective	$\min \min(x_1, ..., x_n)$		
Max Max Objective	$\max \max(x_1, ..., x_n)$		

181.1.4. Algorithms

Algorithm	Update Rule	
Gradient Descent	$x_{k+1} = x_k - \alpha \nabla f(x_k)$	
Newton's Method	$x_{k+1} = x_k - \left[\nabla^2 f(k)\right]^{-1} \nabla f(x_k)$	

Newton's Method

Approximation	Taylor Expansion	Newton Update
Linear (1D)	$f'(x_{k+1}) \approx f'(x_k) + f''(x_k)(x_{k+1} - x_k)$	$x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}$
Linear (nD)	$\nabla f(\mathbf{x}_{k+1}) \approx \nabla f(\mathbf{x}_k) + H_k(\mathbf{x}_{k+1} - \mathbf{x}_k)$	$\mathbf{x_{k+1}} = \mathbf{x}_k - H_k^{-1} \nabla f(\mathbf{x}_k)$
Quadratic (1D)	$f_Q(x) = f(x_k) + f'(x_k)(x - x_k) + \frac{1}{2} f''(x_k)(x - x_k)^2$	$x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}$
Quadratic (nD)	$m_k(\mathbf{p}) = f(\mathbf{x}_k) + \nabla f(\mathbf{x}_k)^T \ \mathbf{p} + \frac{1}{2} \mathbf{p}^T H_k \ \mathbf{p}$ $\mathbf{p} = \mathbf{x} - \mathbf{x}_k$	$\mathbf{x_{k+1}} = \mathbf{x}_k - H_k^{-1} \nabla f(\mathbf{x}_k)$

181.2. MILP Idioms

181.2.1. MILP Modeling Idioms

Phrasebook of LP/MILP modeling idioms — small algebraic constructs that encode logic, products, piecewise terms, and other structure as linear constraints.

Network / flow	
$\sum_j x_{ij} - \sum_j x_{ji} = b_i$	conservation: b_i source/sink/0
$x_{ij} - x_{ji}$	net arc flow (single signed var)
$0 \le x_{ij} \le u_{ij}$	arc capacity
$\sum_{i \in S, j \notin S} x_{ij}$	flow across a cut
Conditional (Big-M)	
$a^T x \le b + M(1 - y)$	active only if $y = 1$
$a^T x \le b_1 + My$ $a^T x \le b_2 + M(1 - y)$	either/or
tight M	loose M ruins relaxation

Binary ↔ continuous	
$x \le My$	fixed-charge on/off
$ly \le x \le uy$	semi-continuous: 0 or $[l, u]$
$x \ge \varepsilon y$	force strictly positive
Binary logic	
$\sum y_i \le 1$	at most one
$\sum y_i = 1$	exactly one (SOS1)
$\sum y_i \ge 1$	OR
$\sum y_i \le k$ $\sum y_i \ge k$	cardinality
$y_a \le y_b$	implication $a \Rightarrow b$
Products	
$z \le y_1$ $z \le y_2$ $z \ge y_1 + y_2 - 1$	bin × bin
$z \le uy$ $z \le x$ $z \ge x - u(1-y)$ $z \ge 0$	bin × cont, $x \in [0, u]$
McCormick envelope	cont × cont relaxation
Abs / norms	
$x = x^+ - x^-$ $x^+, x^- \ge 0$	$\lvert x \rvert = x^+ + x^-$ (L1)
$t \ge x$ $t \ge -x$	$\lvert x \rvert$, epigraph
Min / max	
$t \ge f_i(x) \forall i$	min $t \Rightarrow$ minimax
$t \le f_i(x) \forall i$	max $t \Rightarrow$ maximin
Piecewise-linear	
λ-method + SOS2	convex combo of breakpoints
δ-method	incremental encoding
Sequencing	
$u_i - u_j + nx_{ij} \le n - 1$	MTZ subtour elim.
$s_j \ge s_i + d_i$	precedence
Covering family	
$\sum a_{ij} x_j \ge 1$	set covering
$\sum a_{ij} x_j \le 1$	set packing
$\sum a_{ij} x_j = 1$	partitioning
$\sum w_i x_i \le W$	knapsack
Slack / deviation	

$a^T x + s = b$ $s \geq 0$	slack (≤ → =); surplus $-s$
$f(x) + d^- - d^+ = \text{tgt}$	goal prog. / soft
Temporal	
$I_t = I_{t-1} + p_t - d_t$	inventory balance
Transforms	
$\sum x = 1, x \geq 0$	unit simplex
Charnes–Cooper	linear-fractional → LP
$y_i \geq y_{i+1}$	symmetry breaking

181.3. Standard Form

$$\begin{aligned} \min \quad & c^T x \\ s.t. \quad & Ax = b \\ & x \geq 0 \end{aligned}$$

Original Constraint	Conversion to Equaliy	Variable
$a_i^T x \leq b_i$	$a_i^T x + s_i = b_i$	Slack $s_i \geq 0$
$a_i^T x \geq b_i$	$a_i^T x - s_i = b_i$	Surplus $s_i \geq 0$
$a_i^T x = b_i$	no change	—

Example

$$\begin{aligned} \max \quad & 4x_1 + 3x_2 \\ s.t. \quad & 2x_1 + x_2 \leq 8 \\ & x_1 + 2x_2 \leq 6 \\ & x_1, x_2 \geq 0 \end{aligned}$$

Step 1. Convert inequality ($\leq$) to equality ($=$)

Add slack variables $s_1, s_2 \geq 0$

$$\begin{aligned} \max \quad & 4x_1 + 3x_2 \\ s.t. \quad & 2x_1 + x_2 + s_1 = 8 \\ & x_1 + 2x_2 + s_2 = 6 \\ & x_1, x_2, s_1, s_2 \geq 0 \end{aligned}$$

Step 2. Convert Max to Min

Multiply the objective by –1:

$$
\begin{aligned}
\min \quad & -4x_1 - 3x_2 \\
s.t. \quad & 2x_1 + x_2 + s_1 = 8 \\
& x_1 + 2x_2 + s_2 = 6 \\
& x_1, x_2, s_1, s_2 \geq 0
\end{aligned}
$$

Example

$$
\begin{aligned}
\min \quad & 4x_1 + 3x_2 \\
s.t. \quad & 2x_1 + 3x_2 \geq 6 \\
& x_1 + 4x_2 \geq 8 \\
& x_1, x_2 \geq 0
\end{aligned}
$$

Step 1. Convert inequality ($\geq$) to equality ($=$)

Add slack variables $s_1, s_2 \geq 0$

$$
\begin{aligned}
\min \quad & 4x_1 + 3x_2 \\
s.t. \quad & 2x_1 + 3x_2 - s_1 = 6 \\
& x_1 + 4x_2 - s_2 = 8 \\
& x_1, x_2, s_1, s_2 \geq 0
\end{aligned}
$$

181.4. Linear Programming

181.4.1. Mathematical Programs

181.4.1.1. Formulation

$$
\begin{aligned}
\min \quad & f(x_1, x_2, ..., x_n) && && \text{(objective function)} \\
s.t. \quad & g_i(x_1, x_2, ..., x_n) \leq b_i && \forall i \in 1, ..., m && \text{(constraints)} \\
& x_j \in \mathbb{R} && \forall j \in 1, ..., n && \text{(decision variables)}
\end{aligned}
$$

- m: Number of constraints
- n: Number of variables
- $x_1, x_2, ..., x_n$: decision variables
- $f : \mathbb{R}^n \to \mathbb{R}$
- $g_i : \mathbb{R}^n \to \mathbb{R}$

$$
\begin{aligned}
\min \quad & \sum_{j=1}^{n} c_j x_j \\
s.t. \quad & \sum_{j=1}^{n} A_{ij} x_j \leq b_i \quad \forall i = 1, ..., m
\end{aligned}
$$

- A_{ij}: constraint coefficients

- b_i: right-hand-side values (RHS)
- c_j: objective coefficients

Vector

$$\begin{aligned} \min \quad & c^T x \\ s.t. \quad & a_i^T x \leq b_i \quad \forall i = 1, ..., m \end{aligned}$$

- $a_i \in \mathbb{R}^n$
- $b_i \in \mathbb{R}$
- $c \in \mathbb{R}^n$
- $x \in \mathbb{R}^n$

Matrix

$$\begin{aligned} \min \quad & c^T x \\ s.t. \quad & Ax \leq b \end{aligned}$$

- $A \in R^{m \times n}$
- $b \in \mathbb{R}^m$

$$A = \begin{bmatrix} a_{11} & a_{12} & ... & a_{1n} \\ a_{21} & a_{22} & ... & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & ... & a_{mn} \end{bmatrix}$$

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \quad c = \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_n \end{bmatrix} \quad b = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

$$Ax = \begin{bmatrix} a_{11} & a_{12} & ... & a_{1n} \\ a_{21} & a_{22} & ... & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & ... & a_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} a_{11}x_1 & a_{12}x_2 & ... & a_{1n}x_n \\ a_{21}x_1 & a_{22}x_2 & ... & a_{2n}x_n \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1}x_1 & a_{m2}x_2 & ... & a_{mn}x_n \end{bmatrix} \leq \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

$$c^T x = [c_1, c_2, ..., c_n] \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \sum_{j=1}^{n} c_j x_j$$

181.4.1.2. Transformation

- **Min and Max**

$$\max f(x) \Leftrightarrow \min -f(x)$$

- **$=$ and $\geq$**

$$g_i(x) \geq b_i \Leftrightarrow -g_i(x) \leq -b_i$$

$$g_i(x) = b_i \Leftrightarrow g_i(x) \geq b_i \wedge g_i(x) \leq b_i$$

- **E.g.,**

$$\begin{array}{lrcrcl} \max & x_1 & - & x_2 & & \\ s.t. & -2x_1 & + & x_2 & \geq & -3 \\ & x_1 & + & 4x_2 & = & 5 \end{array} \quad \Leftrightarrow \quad \begin{array}{lrcrcl} \min & -x_1 & + & x_2 & & \\ s.t. & 2x_1 & - & x_2 & \leq & 3 \\ & x_1 & + & 4x_2 & \leq & 5 \\ & -x_1 & + & 4x_2 & \leq & -5 \end{array}$$

181.4.1.3. Constraints

- **Sign Constraints**

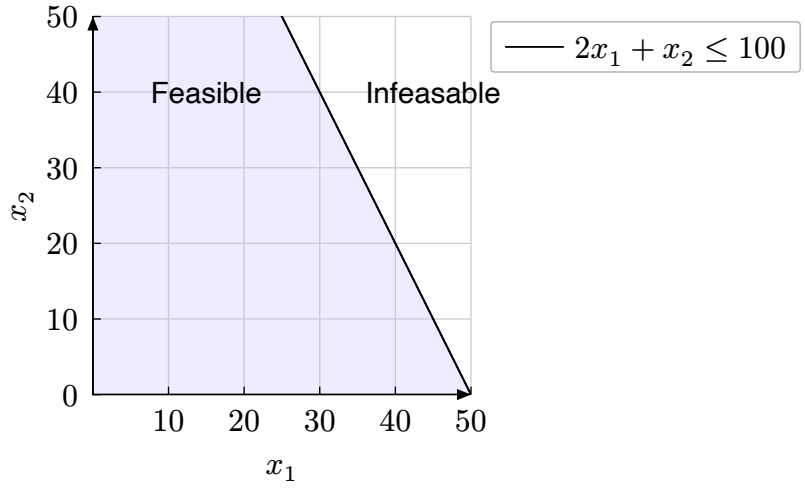
$$x_i \geq 0 \text{ or } x_i \leq 0$$

- **Functional Constraints**

$$\sum_{i=1}^{n} c_i x_i \leq b_i$$

181.4.1.4. Feasible Solutions

- Feasible solution: Satisfies all constants
- Infeasable solution: violates at least one constraint



181.4.1.5. Feasible Region and Optimal Solution

- Feasible Region: Set of feasible solutions
- Optimal Solution: Attains largest (maximization) or smallest (minimization) objective value

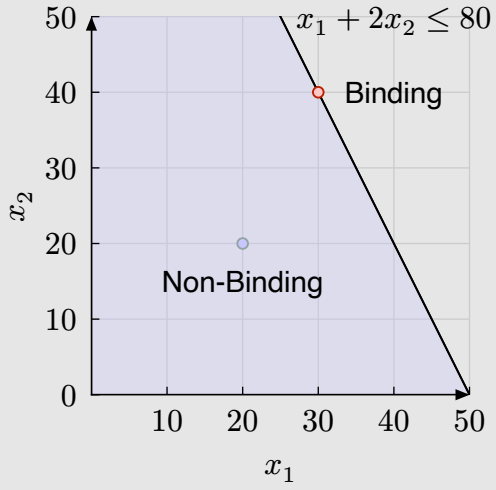
181.4.1.6. Binding Constraint

Let $g(\cdot) \leq b$ be an inequality constraint and $|x|$ be a solution. $g(\cdot) \leq b$ is binding at $|x|$ if $g(|x|) = b$

Example

$2x_1 + x_2 \leq 100$ is **binding** (**active**) at the point $(x_1, x_2) = (30, 40)$

$2x_1 + x_2 \leq 100$ is **non-binding** (**inactive**) at the point $(x_1, x_2) = (20, 20)$



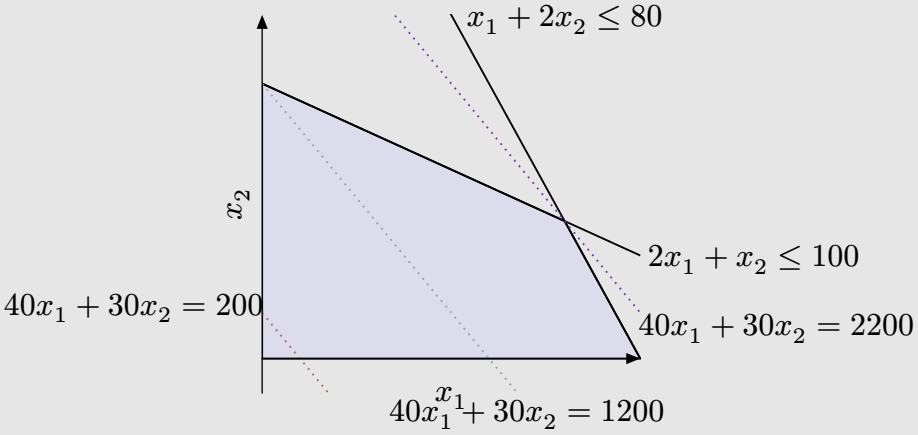
181.4.1.7. Strict and Weak Inequalities

- Strict: The two sides **cannot be equal** (e.g., $x_1 + x_2 > 5$)
- Weak: The two sides **can be equal** (e.g., $x_1 + x_2 \geq 5$)

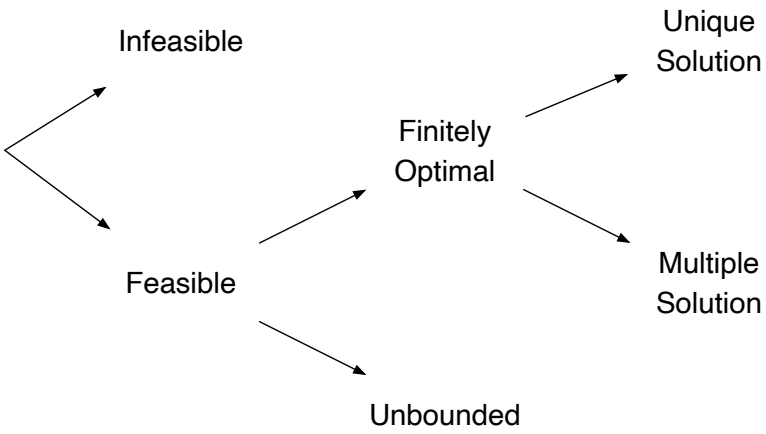
181.4.1.8. Graphical Approach

Example

$$\begin{array}{lrcrcr} \max & 40x_1 & + & 30x_2 & & \\ s.t. & 2\ x_1 & + & x_2 & \leq & 100 \\ & x_1 & + & 2x_2 & \leq & 80 \end{array}$$



181.4.1.9. Types of LP



181.4.1.10. LP Formulation

Example

Production

We produce desks and tables

- Producing a desk requires 3 units of wood, 1 hour of labor and 50 minutes of machine time
- Producing a table requires 5 units of wood, 2 hours of labor and 20 minutes of machine time

For each day, we have

- 200 workers that each work 8 hours
- 50 machines that each run for 16 hours
- A supply of 3600 units of wood

Desks and tables are sold $700 and $900 per unit, respectively

Everything that is produced is sold

1. Define Variables

x_1: number of desks produced per day

x_2: number of tables produced per day

2. Formulate Objective Function

$$\max \quad 700x_1 + 900x_2$$

3. Formulate Constraints

Resource	Consumption per	Total supply

	Desk	Table	
Wood	3 units	5 units	3600 units
Labor	1 hours	2 hours	200 workers $\times$ 8 hours = 1600 hours
Machine time	50 minutes	20 minutes	50 machines $\times$ 16 hours = 800 hours

$$
\begin{aligned}
3x_1 &+ 5x_2 \leq 3600 \\
x_1 &+ 2x_2 \leq 1600 \\
50x_1 &+ 20x_2 \leq 48000
\end{aligned}
$$

4. Complete Formulation

$$
\begin{aligned}
\max \quad & 700x_1 + 900x_2 & & & \\
s.t. \quad & 3x_1 + 5x_2 \leq 3600 & & & (\text{wood}) \\
& x_1 + 2x_2 \leq 1600 & & & (\text{labor}) \\
& 50x_1 + 20x_2 \leq 48000 & & & (\text{machine}) \\
& x_1 \geq 0 & & & \\
& x_2 \geq 0 & & &
\end{aligned}
$$

Compact Formulation

$$
\begin{aligned}
\max \quad & \sum_{j=1}^{n} P_j x_j \\
s.t. \quad & \sum_{j=1}^{n} A_{ij} x_j \leq R_i \quad \forall i = 1, ..., m \\
& x_j \geq 0 \quad \forall j = 1, ..., n
\end{aligned}
$$

Where:

- n: number of products
- m: number of resources
- j: indices of products
- i: indices of resources
- P_j: unit sale price of product j
- R_i: supply limit of resource i
- A_{ij}: unit of resource i to produce one unit of product j
- $i = 1, ..., m$
- $j = 1, ..., n$
- x_j: production quantity for product j, $j = 1, ..., n$

model.py

```python
from pyomo.environ import *
from pyomo.dataportal import DataPortal

model = AbstractModel()

# Sets
model.Products = Set()
model.Resources = Set()

# Parameters
model.Profit = Param(model.Products)
model.Supply = Param(model.Resources)
model.Consumption = Param(model.Resources, model.Products)

# Variables
model.x = Var(model.Products, domain=NonNegativeReals)

# Objective: Maximize profit
def objective_rule(model):
    return sum(model.Profit[j] * model.x[j] for j in model.Products)
model.OBJ = Objective(rule=objective_rule, sense=maximize)

# Constraints: Do not exceed resource supply
def constraint_rule(model, i):
    return sum(model.Consumption[i, j] * model.x[j] for j in
model.Products) <= model.Supply[i]
model.ResourceConstraint = Constraint(model.Resources,
rule=constraint_rule)

# Load data from .dat file
data = DataPortal()
data.load(filename='data.dat', model=model)

# Create an instance of the model
instance = model.create_instance(data)

# Create solver
solver = SolverFactory('glpk')
solver.options['tmlim'] = 60

# Solve with solver timeout (optional)
results = solver.solve(instance, tee=True)

# Display results
instance.display()
```

data.dat

```
set Products := Desk Table ;
set Resources := Wood Labor Machine ;
```

```
param Profit :=
Desk   700
Table  900 ;

param Supply :=
Wood    3600
Labor   1600
Machine 48000 ;

param Consumption:
          Desk Table :=
Wood        3     5
Labor       1     2
Machine    50    20 ;

```

Output:

```
  Variables:
  x : Size=2, Index=Products
      Key   : Lower : Value            : Upper : Fixed : Stale : Domain
      Desk :     0 : 884.210526315789 :  None : False : False : NonNegativeReals
      Table :     0 : 189.473684210526 :  None : False : False : NonNegativeReals

Objectives:
  OBJ : Size=1, Index=None, Active=True
      Key  : Active : Value
      None :   True : 789473.6842105257

Constraints:
  ResourceConstraint : Size=3
      Key     : Lower : Body              : Upper
        Labor :  None : 1263.157894736841 :  1600.0
      Machine :  None : 47999.99999999997 : 48000.0
        Wood :  None : 3599.999999999997 :  3600.0
```

)

Example

Multi Period (Inventory)

We produce and sell products

For the coming 4 days, the demand will be

- Days 1: 100

- Days 2: 150
- Days 3: 200
- Days 4: 170

The unit production costs are different for different days

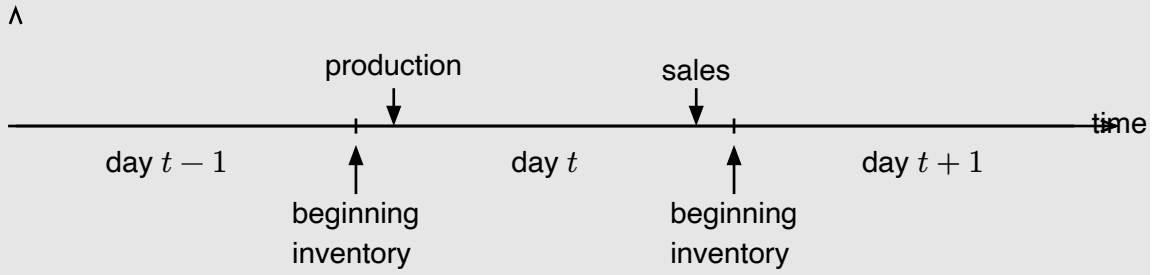
- Days 1: \$9
- Days 2: \$12
- Days 3: \$10
- Days 4: \$12

Prices are all fixed. So maximizing profit is equivalent to minimizing costs.

We may store products and sell them later. Inventory cost is \$1 per unit per day.

E.g., producing 620 units on day 1 to fulfill all demands:

$$\$9 \times 620 + \$1 \times 150 + \$2 \times 200 + \$3 \times 170 = \$6,640$$



$$I_{t-1} + x_t - d_t = I_t$$

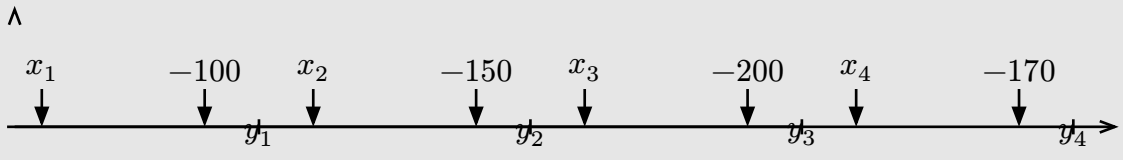
- I_{t-1}: inevntory at the end of day $t-1$ (i.e., beginning inventory for day t)
- x_t: units produced on day t
- d_t: demand (or sales) on dat t
- I_t: inventory at the end of day t

Inventory costs are calculated according to **ending inventory**

- x_t: production quantity of day t, $t = 1, ..., 4$
- y_t: **ending** inventory of day t, $t = 1, ..., 4$

Objective function:

$$\min \quad 9x_1 + 12x_2 + 10x_3 + 12x_4 + y_1 + y_2 + y_3 + y_4$$



Inventory balancing constraints:

- Day 1: $x_1 - 100 = y_1$

- Day 2: $y_1 + x_2 - 150 = y_2$
- Day 3: $y_2 + x_3 - 200 = y_3$
- Day 4: $y_3 + x_4 - 170 = y_4$

Demand fulfillment constraints:

- $x_1 \geq 100$
- $y_1 + x_2 \geq 150$
- $y_2 + x_3 \geq 200$
- $y_3 + x_4 \geq 170$

Non-negativity constraints:

- $y_t \geq 0$
- $x_t \geq 0$

Complete Formulation

$$
\begin{aligned}
\min \quad & 9x_1 + 12x_2 + 10x_3 + 12x_4 + y_1 + y_2 + y_3 + y_4 \\
s.t. \quad & x_1 - 100 = y_1 \\
& y_1 + x_2 - 150 = y_2 \\
& y_2 + x_3 - 200 = y_3 \\
& y_3 + x_4 - 170 = y_4 \\
& x_1 \geq 100 \\
& y_1 + x_2 \geq 150 \\
& y_2 + x_3 \geq 200 \\
& y_3 + x_4 \geq 170 \\
& x_t, y_t \geq 0 \quad \forall t = 1, ..., 4
\end{aligned}
$$

Simplification (Redundant Constraints)

- Inventory balancing & non-negativity imply fulfillment
  - E.g., in day 1, $x_1 - 100 = y_1$ and $y_1 \geq 0$ means $x_1 \geq 100$

$$
\begin{aligned}
\min \quad & 9x_1 + 12x_2 + 10x_3 + 12x_4 + y_1 + y_2 + y_3 + y_4 \\
s.t. \quad & x_1 - 100 = y_1 \\
& y_1 + x_2 - 150 = y_2 \\
& y_2 + x_3 - 200 = y_3 \\
& y_3 + x_4 - 170 = y_4 \\
& x_t, y_t \geq 0 \quad \forall t = 1, ..., 4
\end{aligned}
$$

- No need to have ending inventory in period 4 (costly but useless)

$$
\begin{aligned}
\min \quad & 9x_1 + 12x_2 + 10x_3 + 12x_4 + y_1 + y_2 + y_3 + y_4 \\
s.t. \quad & x_1 - 100 = y_1 \\
& y_1 + x_2 - 150 = y_2 \\
& y_2 + x_3 - 200 = y_3 \\
& y_3 + x_4 - 170 = 0 \\
& x_t \geq 0 \quad \forall t = 1, ..., 4 \\
& y_t \geq 0 \quad \forall t = 1, ..., 3
\end{aligned}
$$

Compact Formulation

$$
\begin{aligned}
\min \quad & \sum_{t=1}^{4} (C_t x_t + y_t) \\
s.t. \quad & y_t - 1 + x_t - D_t = y_t \quad \forall t = 1, ..., 4 \\
& y_0 = 0 \\
& x_t, y_t \geq 0 \quad \forall t = 1, ..., 4
\end{aligned}
$$

Where:

- D_t: demand on day t
- y_t: ending inventory of day t
- C_t: unit production cost on day t

Example

Personnel Scheduling

Scheduling employees

Each employee must work for 5 consecutive days and then take 2 consecutive rest days

Number of employees required for each day

Mon	Tue	Wed	Thu	Fri	Sat	Sun
110	80	150	30	70	160	120

Seven shifts

- Mon to Fri
- Tue to Sat
- …

Minimize number of employees hired

Let x_i be the number of employees assigned to work from day i for 5 consecutive days

Objective function:

$$
\min x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_7
$$

Demand Fullfilment Constraints

- 110 employeeds needed Monday

$$x_1 + x_4 + x_5 + x_6 + x_7 \geq 110$$

An employee that works on Tues (t_2) or Wed (t_3) does not work on Mon (t_1)

- 80 employees needed Tuesday

$$x_1 + x_2 + x_5 + x_6 + x_7 \geq 80$$

An employee that works on Wed (t_3) or Thu (t_4) does not work on Tue (t_2)

- 120 employees needed Sunday

$$x_3 + x_4 + x_5 + x_6 + x_7 \geq 120$$

An employee that works on Mon (t_1) or Tue (t_2) does not work on Sun (t_7)

Non-Nagativity Constraints

$$x_i \geq 0 \quad \forall i = i, ..., 7$$

Complete Formulation

$$\begin{array}{lrrrrrrrrrrr}
\min & x_1 & + & x_2 & + & x_3 & + & x_4 & + & x_5 & + & x_6 & + & x_7 & \\
s.t. & x_1 & + & & & & & x_4 & + & x_5 & + & x_6 & + & x_7 & \geq 110 \\
 & x_1 & + & x_2 & + & & & & & x_5 & + & x_6 & + & x_7 & \geq 80 \\
 & x_1 & + & x_2 & + & x_3 & + & & & & & x_6 & + & x_7 & \geq 150 \\
 & x_1 & + & x_2 & + & x_3 & + & x_4 & + & & & & & x_7 & \geq 30 \\
 & x_1 & + & x_2 & + & x_3 & + & x_4 & + & x_5 & & & & & \geq 70 \\
 & & & x_2 & + & x_3 & + & x_4 & + & x_5 & + & x_6 & & & \geq 160 \\
 & & & & & x_3 & + & x_4 & + & x_5 & + & x_6 & + & x_7 & \geq 120
\end{array}$$

$$x_i \geq 0 \forall i = 1, ..., 7$$

model.py

```python
from pyomo.environ import *
from pyomo.dataportal import DataPortal

model = AbstractModel()

# Sets
model.Days = Set(ordered=True)           # Days 1..7 (Mon..Sun)
model.Shifts = Set(ordered=True)         # Shifts starting on days
1..7

# Parameters
model.Demand = Param(model.Days)         # Daily staffing requirements
model.Cover = Param(model.Days, model.Shifts, within=Binary)  #
Coverage matrix (1 if shift j covers day i)

# Variables
```

```
model.x = Var(model.Shifts, domain=NonNegativeReals)

# Objective: Minimize total employees hired
def obj_rule(model):
    return sum(model.x[s] for s in model.Shifts)
model.OBJ = Objective(rule=obj_rule, sense=minimize)

# Constraints: Cover daily demand
def demand_rule(model, d):
    return sum(model.Cover[d, s] * model.x[s] for s in model.Shifts)
>= model.Demand[d]
model.DemandConstraint = Constraint(model.Days, rule=demand_rule)

# Load data from .dat file
data = DataPortal()
data.load(filename='data.dat', model=model)

# Create an instance of the model
instance = model.create_instance(data)

# Create solver
solver = SolverFactory('glpk')
solver.options['tmlim'] = 60

# Solve with solver timeout (optional)
results = solver.solve(instance, tee=True)

# Display results
instance.display()
```

data.dat

```
set Days := Mon Tue Wed Thu Fri Sat Sun ;
set Shifts := s1 s2 s3 s4 s5 s6 s7 ;

param Demand :=
Mon 110
Tue 80
Wed 150
Thu 30
Fri 70
Sat 160
Sun 120 ;

# Each shift covers 5 consecutive days starting on the shift's day
# 1 if shift s_j covers day d_i, 0 otherwise

param Cover:
        s1 s2 s3 s4 s5 s6 s7 :=
Mon     1  0  0  1  1  1  1
Tue     1  1  0  0  1  1  1
Wed     1  1  1  0  0  1  1
Thu     1  1  1  1  0  0  1
```

```
Fri     1  1  1  1  1  0  0
Sat     0  1  1  1  1  1  0
Sun     0  0  1  1  1  1  1 ;
```

Output:

```
  Variables:
  x : Size=7, Index=Shifts
      Key : Lower : Value            : Upper : Fixed : Stale : Domain
        s1 :     0 : 3.3333333333333 :  None : False : False :
NonNegativeReals
        s2 :     0 :            40.0 :  None : False : False :
NonNegativeReals
        s3 :     0 : 13.333333333333 :  None : False : False :
NonNegativeReals
        s4 :     0 :             0.0 :  None : False : False :
NonNegativeReals
        s5 :     0 : 13.333333333333 :  None : False : False :
NonNegativeReals
        s6 :     0 : 93.333333333333 :  None : False : False :
NonNegativeReals
        s7 :     0 :             0.0 :  None : False : False :
NonNegativeReals

Objectives:
  OBJ : Size=1, Index=None, Active=True
      Key  : Active : Value
      None :   True : 163.33333333333323

Constraints:
  DemandConstraint : Size=7
      Key : Lower : Body               : Upper
      Fri :  70.0 :  69.99999999999993 :  None
      Mon : 110.0 : 109.99999999999993 :  None
      Sat : 160.0 :  159.9999999999999 :  None
      Sun : 120.0 :  119.9999999999999 :  None
      Thu :  30.0 :  56.66666666666663 :  None
      Tue :  80.0 : 149.99999999999994 :  None
      Wed : 150.0 : 149.99999999999994 :  None
```

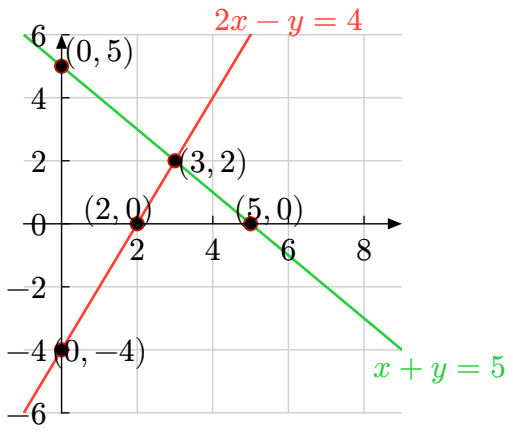
181.5. Simplex Method

181.5.0.1. Linear Algebra Review

$$x + y = 5$$
$$2x - y = 4$$

Row View

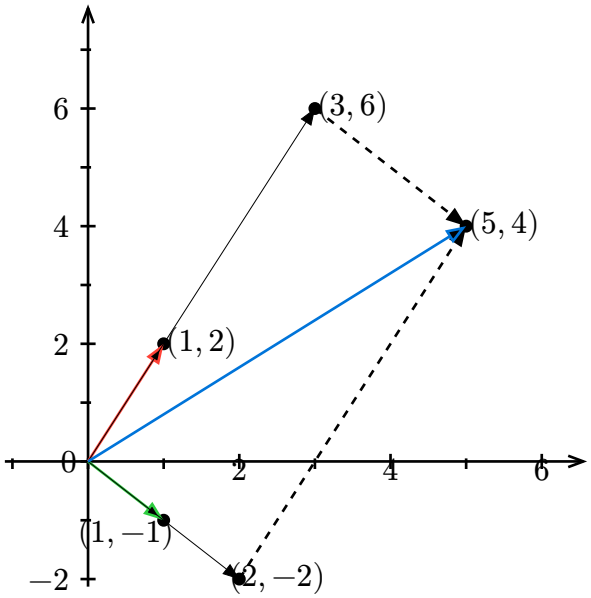
Solution: Intersection of n lines or (hyper)planes



Column View

Solution: Combination of LHS column vectors to form the RHS vector

$$x \begin{bmatrix} 1 \\ 2 \end{bmatrix} + y \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 5 \\ 4 \end{bmatrix}$$



A linear system is **singular** if there is no unique solution

- Row view: the n (hyper)planes do not intersect at exactly one point
- Column view: the n vectors do not span a complete n-dimensional space

181.5.0.2. Gaussian Elimination

Given a system $Ax = b$, or augmented matrix $[A \mid b]$, use the following 3 rules:

- Row Swapping: $R_i \longleftrightarrow R_j$
- Row Scaling: $k \cdot R_i \longrightarrow R_i$
- Row Replacement: $R_i - k \cdot R_j \longrightarrow R_i$

To solve the system:

1. Transform the augmented matrix into row echelon form (REF) using these operations:

- The first nonzero entry (pivot) in each row is to the right of the pivot in the row above
- All entries below a pivot are zero
- Rows of all zeros (if any) are at the bottom

2. Back-substitute starting from the last nonzero row to find the solution

Example

Non-singular Case

System of equations

$$\begin{aligned} 2y &= 4 \\ 1x + 1y &= 3 \end{aligned}$$

Augmented matrix

$$\left[\begin{array}{cc|c} 0 & 2 & 4 \\ 1 & 1 & 3 \end{array}\right]$$

Swapping ($R_1 \longleftrightarrow R_2$)

$$\left[\begin{array}{cc|c} 1 & 1 & 3 \\ 0 & 2 & 4 \end{array}\right]$$

Scaling ($\frac{1}{2}R_2 \longrightarrow R_2$)

Row Echelon Form (REF)

$$\left[\begin{array}{cc|c} 1 & 1 & 3 \\ 0 & 1 & 2 \end{array}\right]$$

Replacement ($R_1 - R_2 \longrightarrow R_1$)

Reduced Row Echelon Form (RREF)

$$\left[\begin{array}{cc|c} 1 & 0 & 1 \\ 0 & 1 & 2 \end{array}\right]$$

Result

$$x = 1, \quad y = 2$$

Example

Singular Case

System of equations

$$\begin{aligned} x+ \quad y &= 2 \\ 2x + 2y &= 4 \end{aligned}$$

Augmented matrix

$$\left[\begin{array}{cc|c} 1 & 1 & 2 \\ 2 & 2 & 4 \end{array}\right]$$

Replacement ($R_2 - 2R_1 \longrightarrow R_2$)

$$\left[\begin{array}{cc|c} 1 & 1 & 2 \\ 0 & 0 & 0 \end{array}\right]$$

Infinitely many solutions E.g.

- $x = 0, \quad y = 2$
- $x = 1, \quad y = 1$
- $x = 2, \quad y = 0$
- etc.

Rows are **linearly dependent**

The coefficient matrix

$$A = \begin{bmatrix} 1 & 1 \\ 2 & 2 \end{bmatrix}$$

A has **determinant** 0, so it's **not invertible**

181.5.0.3. Inverse

$$A^{-1}A = I \quad \wedge \quad AA^{-1}A = I$$

A^{-1} is **unique**

Finding A^{-1}: Gauss-Jordan Elimination

A square matrix is **nonsingular** iff it is **invertible**

$$[A \mid I] \longrightarrow [I \mid A^{-1}]$$

Example

$$A = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$$

$$[A \mid I] = \left[\begin{array}{cc|cc} 1 & 2 & 1 & 0 \\ 2 & 1 & 0 & 1 \end{array}\right]$$

Replacement ($R_2 - 2R_1 \longrightarrow R_2$)

$$\left[\begin{array}{cc|cc} 1 & 2 & 1 & 0 \\ 0 & -3 & -2 & 1 \end{array}\right]$$

Scale ($-\frac{1}{3}R_2 \longrightarrow R_2$)

$$\left[\begin{array}{cc|cc} 1 & 2 & 1 & 0 \\ 0 & 1 & \frac{2}{3} & -\frac{1}{3} \end{array}\right]$$

Replacement ($R_1 - 2R_2 \longrightarrow R_1$)

$$\left[\begin{array}{cc|cc} 1 & 0 & -\frac{1}{3} & \frac{2}{3} \\ 0 & 1 & \frac{2}{3} & -\frac{1}{3} \end{array}\right]$$

Result

$$A^{-1} = \begin{bmatrix} -\frac{1}{3} & \frac{2}{3} \\ \frac{2}{3} & -\frac{1}{3} \end{bmatrix}$$

181.5.0.4. Linear Dependence and Independence

A set of m n-dimensional vectors $x_1, x_2, ..., x_m$ are **linearly dependent** if there exists a non-zero vector $w \in \mathbb{R}^m$ such that:

$$w_1 x_1 + w_2 x_2 + ... + w_m x_m = 0$$

That is, at least one of the vectors can be written as a linear combination of the others.

Example

Dependent

Let:

$$x_1 = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad x_2 = \begin{bmatrix} 2 \\ 4 \end{bmatrix}$$

Augmented:

$$\left[\begin{array}{cc|c} 1 & 2 & 0 \\ 2 & 4 & 0 \end{array}\right]$$

Elimination ($R_2 - 2R_1 \longrightarrow R_2$)

$$\left[\begin{array}{cc|c} 1 & 2 & 0 \\ 0 & 0 & 0 \end{array}\right]$$

Independent

Let:

$$x_1 = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad x_2 = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$$

Augmented:

$$\left[\begin{array}{cc|c} 1 & 3 & 0 \\ 2 & 1 & 0 \end{array}\right]$$

Elimination ($R_2 - 2R_1 \longrightarrow R_2$)

$$\left[\begin{array}{cc|c} 1 & 2 & 0 \\ 0 & -5 & 0 \end{array}\right]$$

Back-substitute

- $-5w_2 = 0 \Rightarrow w_2 = 0$
- $w_1 + 3w_2 = 0 \Rightarrow w_1 = 0$

Only solution is $w_1 = w_2 = 0$

In Gaussian elimination, a **row of all zeros**

1. **Homogeneous** System (RHS = 0)

$$\left[\begin{array}{cc|c} 1 & 2 & 0 \\ 0 & 0 & 0 \end{array}\right]$$

$$x_1 + 2x_2 = 0$$
$$0 = 0$$

- ✅ Represents a **redundant** equation (i.e., no new information) **dependent** set of vectors
- ✅ Does not affect consistency: the system is always **consistent**
- ✅ The system has fewer pivots than variables, hence **infinitely many solutions**

2. **Inconsistent** System (Nonzero RHS)

$$\left[\begin{array}{cc|c} 1 & 2 & 3 \\ 0 & 0 & 5 \end{array}\right]$$

$$x_1 + 2x_2 = 3$$
$$0 = 5$$

- ❌ The second row leads to a **contradiction**
- ❌ The system is **inconsistent** (**no solution**)

181.5.0.5. Extreme Points

Given that x, x_1, and x_2 are points in the set S:

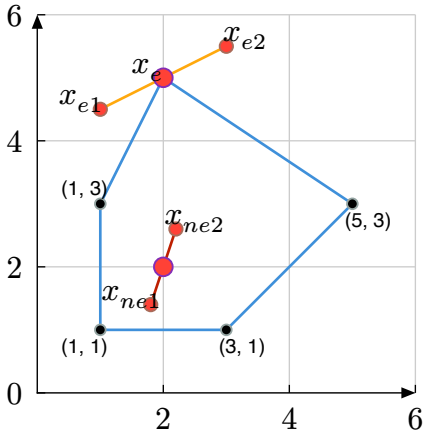
- If you can find two different points x_1 and x_2 in S such that x lies somewhere strictly between them on the straight line connecting x_1 and x_2, then x is not an extreme point
- If no such pair of points exists—meaning you cannot place x anywhere strictly between two other points in S on a straight line—then x is an extreme point

Formal Definition

For a set $S \in \mathbb{R}^n$, a point x is an extreme point if there does not exist a three-tuple (x_1, x_2, λ) such that $x_1 \in S$, $x_2 \in S$, $\lambda \in (0, 1)$, and

$$x = \lambda x_1 + (1 - \lambda)x_2$$

A point x in set $S \subseteq \mathbb{R}^n$ is extreme if it cannot be written as a strict **convex combination** of two different points $x_1, x_2 \in S$.



Convex Combination

Let $x_1, x_2, ..., x_k \in \mathbb{R}^n$. A convex combination of these vectors is any point of the form:

$$x = \lambda_1 x_1 + \lambda_2 x_2 + ... + \lambda_k x_k$$

where:

- $\lambda_i \geq 0 \quad \forall i$
- $\sum_{i=1}^{k} \lambda_i = 1$

x lies strictly between x_1 and x_2 on the line segment connecting them, not equal to either one

181.5.0.6. Slack and Suplus

1. Slack

Unused capacity

Example

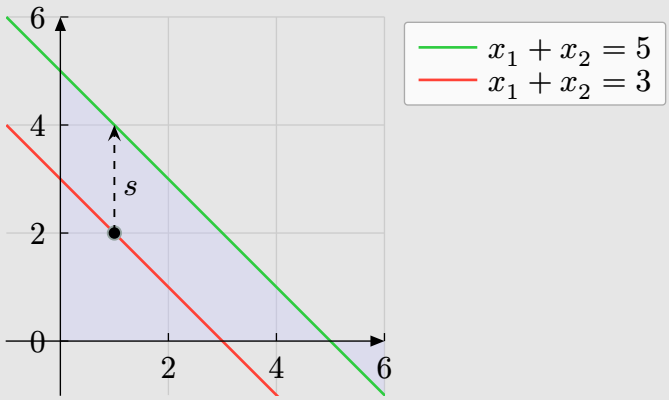
Given a constraint:

$$x_1 + x_2 \leq 5$$

The total **must not exceed** 5. To convert this into an equation, we **add** a slack variable $s \geq 0$:

$$x_1 + x_2 + s = 5$$

- If $x_1 + x_2 = 3$, then $s = 2$
- If $x_1 + x_2 = 5$, then $s = 0$



2. Excess (surplus)

Excess above the required amount

Example

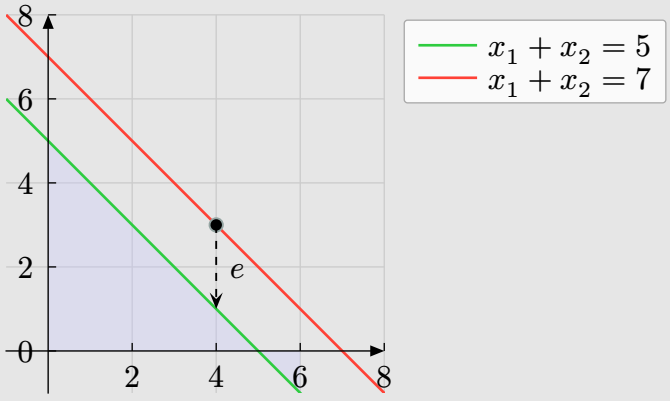
Given a constraint:

$$x_1 + x_2 \geq 5$$

The total **must be at least** 5. To convert this into an equation, we **subtract** a excess variable $s \geq 5$:

$$x_1 + x_2 - e = 5$$

- If $x_1 + x_2 = 7$, then $e = 2$
- If $x_1 + x_2 = 5$, then $e = 0$



Objective Function		
Max	✅	$\max z = c^T x$
Min ⟶ Max	$-z$	$\min z = c^T \boldsymbol{x}$ $\Longleftrightarrow$ $\max z = -c^T \boldsymbol{x}$
Constraint Forms		
$\leq$	Slack	$a_1 x_1 + a_2 x_2 \leq b$ $a_1 x_1 + a_2 x_2 + s = b$
$\geq$	Excess + Artificial	$a_1 x_1 + a_2 x_2 \geq b$ $a_1 x_1 + a_2 x_2 - e + a = b$
$=$	Artificial	$a_1 x_1 + a_2 x_2 = b$ $a_1 x_1 + a_2 x_2 + a = b$
Negative b	Multiply by –1	$a_1 x_1 + a_2 x_2 \leq -b$ $-a_1 x_1 - a_2 x_2 \geq b$
Variable Bounds		
Non-negativity		$x_i, s_i, a_i \geq 0 \quad \forall i$
Lower Bound ≠ 0	Substitution	$x_1 \geq 5$ $y_1 = x_1 - 5$ $x_1 = y_1 + 5$ $y_1 \geq 0$
Unrestricted Variables	Replace with difference	x_j unrestricted $x_j = x_j' - x_j''$ $x_j', x_j'' \geq 0$
Variable Roles		
Basic Variables	Usually non-zero	Part of the current solution (solved from the constraints)
Non-Basic Variables	Always zero	Temporarily set to 0 so basic variables can be solved

<table>
<tr><th colspan="3">Simplex</th></tr>
<tr><td></td><td></td><td></td></tr>
<tr><td></td><td></td><td></td></tr>
<tr><td></td><td></td><td></td></tr>
<tr><td>Selection:
Entering variable
Pivot column selection</td><td>For maximization problems:
Select the column with the most negative value in the objective row
This corresponds to the variable that will increase the objective function the most
For minimization problems:
Select the column with the most positive value in the objective row
This corresponds to the variable that will decrease the objective function the most</td><td></td></tr>
<tr><td>Selection:
Leaving variable
Pivot row</td><td>1. Calculate ratios for each row (except the objective row)
For each row: b_i/p_i
Only consider rows where the pivot column element $p_i > 0$
Ignore rows where the pivot column element $p_i \leq 0$</td><td></td></tr>
</table>

	2. Select the **minimum ratio**: • The row with the **smallest non-negative ratio** becomes the pivot row • The basic variable in this row is the **leaving variable** (exits the basis)	

,

Example

$$\begin{aligned} \max \quad & z = 3x_1 + 2x_2 \\ s.t. \quad & x_1 + x_2 \leq 4 \\ & x_1 \leq 2 \\ & x_2 \leq 3 \\ & x_1, x_2 \geq 0 \end{aligned}$$

Step 1: Convert to Standard Form

$$\begin{aligned} \max \quad & z = 3x_1 + 2x_2 \\ s.t. \quad & x_1 + x_2 + s_1 = 4 \\ & x_1 + s_2 = 2 \\ & x_2 + s_3 = 3 \\ & x_1, x_2, s_1, s_2, s_3 \geq 0 \end{aligned}$$

Objective

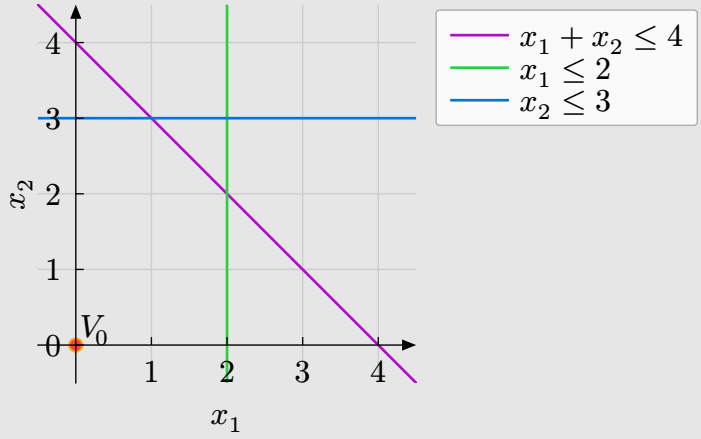
$$z - 3x_1 - 2x_2 = 0$$

Step 2: Initial Tableau

Basis	x_1	x_2	s_1	s_2	s_3	b
s_1	1	1	1	0	0	4
s_2	1	0	0	1	0	2
s_3	0	1	0	0	1	3
z	–3	–2	0	0	0	0

- Basic Variables: $s_1 = 4$, $s_2 = 2$, $s_3 = 3$
- Non-basic variables: $x_1 = 0$, $x_2 = 0$

- $V_0 = (0, 0)$
- Objective: $z = 3x_1 + 2x_2 = 3(0) + 2(0) = 0$



Step 3: Choose entering variable

- Look at the bottom row (z):
  - Coefficients of $x_1 = -3$, $x_2 = -2$
- Most negative: $x_1 \longrightarrow$ Enter the basis

Basis	x_1	x_2	s_1	s_2	s_3	b
s_1	1	1	1	0	0	4
s_2	1	0	0	1	0	2
s_3	0	1	0	0	1	3
z	–3	–2	0	0	0	0

Step 4: Determine leaving variable

Look at ratios of b/x_1:

Basis	x_1	x_2	s_1	s_2	s_3	b	b/x_1
s_1	1	1	1	0	0	4	$4/1 = 4$
s_2	1	0	0	1	0	2	$2/1 = 2$
s_3	0	1	0	0	1	3	$3/0 = \infty$
z	–3	–2	0	0	0	0	

- Minimum Ratio
  - s_2 leaves, x_1 enters

Basis	x_1	x_2	s_1	s_2	s_3	b
s_1	1	1	1	0	0	4
x_1	1	0	0	1	0	2
s_3	0	1	0	0	1	3
z	–3	–2	0	3	0	0

Step 5: Pivot on row 2, column x_1

Now eliminate x_1 from other rows:

Basis	x_1	x_2	s_1	s_2	s_3	b
s_1	1	1	1	0	0	4
x_1	1	0	0	1	0	2
s_3	0	1	0	0	1	3
z	–3	–2	0	3	0	0

$R_{s_1} := R_{s_1} - R_{x_1}$

Basis	x_1	x_2	s_1	s_2	s_3	b
s_1	1	1	1	0	0	4
x_1	1	0	0	1	0	2
s_3	0	1	0	0	1	3
z	–3	–2	0	3	0	0

$$R_{s_1} := R_{s_1} - R_{x_1} \implies \begin{aligned} x_1 &: \ 1-1=0 \\ x_2 &: \ 1-0=1 \\ s_1 &: \ 1-0=1 \\ s_2 &: \ 0-1=-1 \\ s_3 &: \ 0-0=0 \\ b &: \ 4-2=2 \end{aligned}$$

Update R_{s_1}

Basis	x_1	x_2	s_1	s_2	s_3	b
s_1	0	1	1	–1	0	2
x_1	1	0	0	1	0	2
s_3	0	1	0	0	1	3
z	–3	–2	0	3	0	0

$R_z := R_z - 3R_{x_1}$

Basis	x_1	x_2	s_1	s_2	s_3	b
s_1	0	1	1	–1	0	2
x_1	1	0	0	1	0	2
s_3	0	1	0	0	1	3
z	–3	–2	0	3	0	0

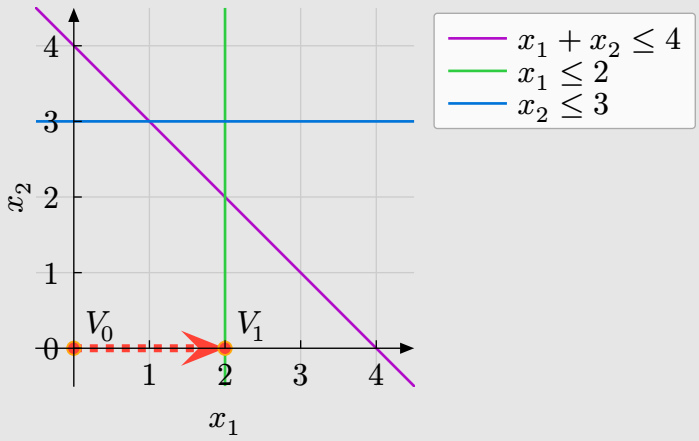
$$R_z := R_z - 3R_{x_1} \implies \begin{aligned} x_1 &: \quad -3 + 3(1) = 0 \\ x_2 &: \quad -2 + 3(0) = -2 \\ s_1 &: \quad 0 + 3(0) = 0 \\ s_2 &: \quad 0 + 3(1) = 3 \\ s_3 &: \quad 0 + 3(0) = 0 \\ b &: \quad 0 + 3(2) = 6 \end{aligned}$$

Update R_z

Basis	x_1	x_2	s_1	s_2	s_3	b
s_1	0	1	1	–1	0	2
x_1	1	0	0	1	0	2
s_3	0	1	0	0	1	3
z	0	–2	0	3	0	6

- Basic Variables: $x_1 = 2$, $s_1 = 2$, $s_3 = 3$
- Non-basic variables: $x_2 = 0$
  - $V_1 = (2, 0)$
  - Objective: $z = 3x_1 + 2x_2 = 3(2) + 2(0) = 6$

Basis	x_1	x_2	s_1	s_2	s_3	b
s_1	0	1	1	–1	0	2
x_1	1	0	0	1	0	2
s_3	0	1	0	0	1	3
z	0	–2	0	3	0	6



Step 3: Choose entering variable

- Look at the bottom row (z):
  - Coefficients of $x_2 = -2$
- Most negative: $x_2 \longrightarrow$ Enter the basis

Basis	x_1	x_2	s_1	s_2	s_3	b
s_1	0	1	1	–1	0	2
x_1	1	0	0	1	0	2
s_3	0	1	0	0	1	3
z	0	–2	0	3	0	6

Step 4: Determine leaving variable

Look at ratios of b/x_2:

Basis	x_1	x_2	s_1	s_2	s_3	b	b/x_2
s_1	0	1	1	–1	0	2	$2/1 = 2$
x_1	1	0	0	1	0	2	$2/0 = \infty$
s_3	0	1	0	0	1	3	$3/1 = 3$
z	0	–2	0	3	0	6	

- s_1 leaves, x_2 enters

Basis	x_1	x_2	s_1	s_2	s_3	b
x_2	0	1	1	–1	0	2
x_1	1	0	0	1	0	2
s_3	0	1	0	0	1	3
z	0	–2	0	3	0	6

Step 5: Pivot on row 1, column x_2

Basis	x_1	x_2	s_1	s_2	s_3	b
x_2	0	1	1	–1	0	2
x_1	1	0	0	1	0	2
s_3	0	1	0	0	1	3
z	0	–2	0	3	0	6

Now eliminate x_2 from other rows:

Basis	x_1	x_2	s_1	s_2	s_3	b
x_2	0	1	1	–1	0	2
x_1	1	0	0	1	0	2
s_3	0	1	0	0	1	3
z	0	–2	0	3	0	6

$R_{s_3} := R_{s_3} - R_{x_2}$

Basis	x_1	x_2	s_1	s_2	s_3	b
x_2	0	1	1	–1	0	2
x_1	1	0	0	1	0	2
s_3	0	1	0	0	1	3
z	0	–2	0	3	0	6

$$R_{s_3} := R_{s_3} - R_{x_2} \implies \begin{aligned} x_1 &: \ 0-0=0 \\ x_2 &: \ 1-1=0 \\ s_1 &: \ 0-1=-1 \\ s_2 &: \ 0-(-1)=1 \\ s_3 &: \ 1-0=1 \\ b &: \ 3-2=1 \end{aligned}$$

Basis	x_1	x_2	s_1	s_2	s_3	b
x_2	0	1	1	–1	0	2
x_1	1	0	0	1	0	2
s_3	0	0	–1	1	1	1
z	0	–2	0	3	0	6

$$R_z = R_z - (-2)R_{x_2} = R_z + 2R_{x_2}$$

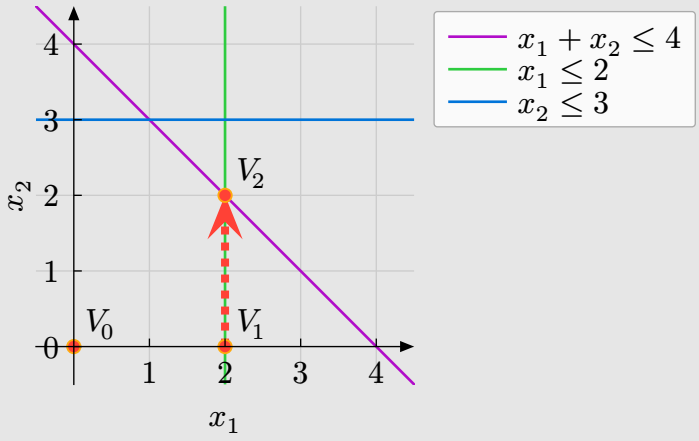
Basis	x_1	x_2	s_1	s_2	s_3	b
x_2	0	1	1	–1	0	2
x_1	1	0	0	1	0	2
s_3	0	0	–1	1	1	1
z	0	–2	0	3	0	6

$$R_z = R_z + 2R_{x_2} \implies \begin{aligned} x_1 &: \ 0+2(0)=0 \\ x_2 &: \ -2+2(1)=0 \\ s_1 &: \ 0+2(1)=2 \\ s_2 &: \ 3+2(-1)=1 \\ s_3 &: \ 0+2(0)=0 \\ b &: \ 6+2(2)=10 \end{aligned}$$

Basis	x_1	x_2	s_1	s_2	s_3	b
x_2	0	1	1	–1	0	2
x_1	1	0	0	1	0	2

s_3	0	0	−1	1	1	1
z	0	0	2	1	0	10

- Basic Variables: $x_1 = 2$, $x_2 = 2$, $s_3 = 1$
- Non-basic variables: $s_1 = 0$, $s_2 = 0$
  - $V_2 = (2, 2)$
  - Objective: $z = 3x_1 + 2x_2 = 3(2) + 2(2) = 10$



Example

Problem

$$
\begin{aligned}
\min \quad & z = x_1 + 2x_2 & \\
s.t. \quad & x_1 + x_2 \leq 1 & \text{(slack)} \\
& 2x_1 - x_2 \geq 2 & \text{(surplus + artificial)} \\
& -x_1 + 3x_2 \leq -1 & \text{(negative } b\text{)} \\
& x_1 - x_2 = 1 & \text{(artificial)} \\
& x_1 \geq 0 & \\
& x_2 & \text{(unrestricted)}
\end{aligned}
$$

Standard Form

Step 1: Handle Unrestricted Variable x_2

Since x_2 is unrestricted:

$$
\begin{aligned}
x_2 &= x_2' - x_2'' \\
x_2', x_2'' &\geq 0
\end{aligned}
$$

Rewrite the objective:

$$
\begin{aligned}
z &= x_1 + 2(x_2' - x_2'') \\
&= x_1 + 2x_2' - 2x_2''
\end{aligned}
$$

Step 2: Slack Constraint ($\leq$)

$$x_1 + x_2 \leq 1$$

Rewrite with $x_2' - x_2''$:

$$x_1 + (x_2' - x_2'') \leq 1$$

Add slack variable:

$$x_1 + x_2' - x_2'' + s_1 = 1$$
$$s_1 \geq 0$$

Step 3: Surplus + Artificial Constraint ($\geq$)

$$2x_1 - x_2 \geq 2$$

Rewrite with $x_2' - x_2''$:

$$2x_1 - (x_2' - x_2'') \geq 2 \implies 2x_1 - x_2' + x_2'' \geq 2$$

Convert to equality by subtracting a surplus variable and adding an artificial variable:

$$2x_1 - x_2' + x_2'' - s_2 + a_1 = 2$$
$$s_2, a_1 \geq 0$$

Step 4: Negative RHS Constraint

$$-x_1 + 3x_2 \leq -1$$

Rewrite with $x_2' - x_2''$:

$$-x_1 + 3(x_2' - x_2'') \leq -1 \implies -x_1 + 3x_2' - 3x_2'' \leq -1$$

Multiply both sides by -1 to get a positive RHS and flip inequality:

$$x_1 - 3x_2' + 3x_2'' \geq 1$$

Convert to equality by subtracting a surplus variable and adding an artificial variable:

$$x_1 - 3x_2' + 3x_2'' - s_3 + a_2 = 1$$
$$s_3, a_2 \geq 0$$

Step 5: Artificial Equality Constraint ($=$)

$$x_1 - x_2 = 1$$

Rewrite with $x_2' - x_2''$:

$$x_1 - (x_2' - x_2'') = 1 \implies x_1 - x_2' + x_2'' = 1$$

Add an artificial variable:

$$x_1 - x_2' + x_2'' + a_3 = 1$$
$$a_3 \geq 0$$

Final Standard Form

$$
\begin{aligned}
\min \quad & z = x_1 + 2x_2' - 2x_2'' \\
s.t. \quad & x_1 + x_2' - x_2'' + s_1 = 1 \\
& 2x_1 - x_2' + x_2'' - s_2 + a_1 = 2 \\
& x_1 - 3x_2' + 3x_2'' - s_3 + a_2 = 1 \\
& x_1 - x_2' + x_2'' + a_3 = 1 \\
& x_1, x_2', x_2'', s_1, s_2, s_3, a_1, a_2, a_3 \geq 0
\end{aligned}
$$

Phase I

Objective

$$
\begin{aligned}
\min \quad & w = a_1 + a_2 + a_3 \\
s.t. \quad & x_1 + x_2' - x_2'' + s_1 = 1 && \text{(slack)} \\
& 2x_1 - x_2' + x_2'' - s_2 + a_1 = 2 && \text{(surplus + artificial)} \\
& x_1 - 3x_2' + 3x_2'' - s_3 + a_2 = 1 && \text{(negative } b\text{)} \\
& x_1 - x_2' + x_2'' + a_3 = 1 && \text{(artificial)} \\
& x_1, x_2', x_2'', s_1, s_2, s_3, a_1, a_2, a_3 \geq 0 && \text{(unrestricted)}
\end{aligned}
$$

Deriving Phase I Objective Row R_w

1. Objective function before substitution

$$w = a_1 + a_2 + a_3$$

2. Express each artificial variable in terms of other variables from their constraints

- R_{a_1}

$$
\begin{aligned}
2x_1 - x_2' + x_2'' - s_2 + a_1 &= 2 \\
a_1 &= -2x_1 + x_2' - x_2'' + s_2 + 2
\end{aligned}
$$

- R_{a_2}

$$
\begin{aligned}
x_1 - 3x_2' + 3x_2'' - s_3 + a_2 &= 1 \\
a_2 &= -x_1 + 3x_2' - 3x_2'' + s_3 + 1
\end{aligned}
$$

- R_{a_3}

$$
\begin{aligned}
x_1 - x_2' + x_2'' + a_3 &= 1 \\
a_3 &= -x_1 + x_2' - x_2'' + 1
\end{aligned}
$$

3. Substitute into w:

$$
\begin{aligned}
w &= a_1 + a_2 + a_3 \\
&= (-2x_1 + x_2' - x_2'' + s_2 + 2) + (-x_1 + 3x_2' - 3x_2'' + s_3 + 1) + (-x_1 + x_2' - x_2'' + 1) \\
&= (-2x_1 - x_1 - x_1) + (x_2' + 3x_2' + x_2') + (-x_2'' - 3x_2'' - x_2'') + (s_2 + s_3) + (2 + 1 + 1) \\
&= -4x_1 + 5x_2' - 5x_2'' + s_2 + s_3 + 4
\end{aligned}
$$

4. Rewrite as a constraint (bring all variables to the left):

$$w + 4x_1 - 5x_2' + 5x_2'' - s_2 - s_3 = 4$$
$$w = -4x_1 + 5x_2' - 5x_2'' + s_2 + s_3 + 4$$

So in tableau row R_w:

Basis	x_1	x_2'	x_2''	s_1	s_2	s_3	a_1	a_2	a_3	b
s_1	1	1	–1	1	0	0	0	0	0	1
a_1	2	–1	1	0	–1	0	1	0	0	2
a_2	1	–3	3	0	0	–1	0	1	0	1
a_3	1	–1	1	0	0	0	0	0	1	1
w	4	–5	5	0	–1	–1	0	0	0	4

Perform First Pivot

1. Pivot Column

Look at the w row (objective row) for the most negative coefficient:

Basis	x_1	x_2'	x_2''	s_1	s_2	s_3	a_1	a_2	a_3	b
s_1	1	1	–1	1	0	0	0	0	0	1
a_1	2	–1	1	0	–1	0	1	0	0	2
a_2	1	–3	3	0	0	–1	0	1	0	1
a_3	1	–1	1	0	0	0	0	0	1	1
w	4	–5	5	0	–1	–1	0	0	0	4

The most negative is $-5 \longrightarrow$ pivot column is x_2'

2. Ratio

- Compute ratios (b/x_2') for rows with positive pivot entries:

Basis	x_1	x_2'	x_2''	s_1	s_2	s_3	a_1	a_2	a_3	b	b/x_2'
s_1	1	1	–1	1	0	0	0	0	0	1	1
a_1	2	–1	1	0	–1	0	1	0	0	2	✗
a_2	1	–3	3	0	0	–1	0	1	0	1	✗
a_3	1	–1	1	0	0	0	0	0	1	1	✗
w	4	–5	5	0	–1	–1	0	0	0	4	

- Minimum ratio is 1, so we pivot on row s_1:

Basis	x_1	x_2'	x_2''	s_1	s_2	s_3	a_1	a_2	a_3	b	b/x_2'
s_1	1	1	–1	1	0	0	0	0	0	1	1
a_1	2	–1	1	0	–1	0	1	0	0	2	✗

a_2	1	–3	3	0	0	–1	0	1	0	1	✗
a_3	1	–1	1	0	0	0	0	0	1	1	✗
w	4	–5	5	0	–1	–1	0	0	0	4	

3. Pivot $s_1 \longrightarrow x_2'$

Basis	x_1	x_2'	x_2''	s_1	s_2	s_3	a_1	a_2	a_3	b
x_2'	1	1	–1	1	0	0	0	0	0	1
a_1	2	–1	1	0	–1	0	1	0	0	2
a_2	1	–3	3	0	0	–1	0	1	0	1
a_3	1	–1	1	0	0	0	0	0	1	1
w	4	–5	5	0	–1	–1	0	0	0	4

4. Gauss-Jordan Elimination

Example

Step 3: Pivot $a_1 \longrightarrow x_1$

Basis	x_1	x_2'	x_2''	s_1	s_2	s_3	a_1	a_2	a_3	b
s_1	1	1	–1	1	0	0	0	0	0	5
x_1	3	–1	1	0	–1	0	1	0	0	4
a_2	1	–2	2	0	0	–1	0	1	0	6
a_3	1	–1	1	0	0	0	0	0	1	2
w	–5	4	–4	0	1	1	0	0	0	12

Gauss-Jordan Elimination

$$R_{x_1} = \frac{1}{3} R_{x_1}$$

Basis	x_1	x_2'	x_2''	s_1	s_2	s_3	a_1	a_2	a_3	b
s_1	1	1	–1	1	0	0	0	0	0	5
x_1	1	–1/3	1/3	0	–1/3	0	1/3	0	0	4/3
a_2	1	–2	2	0	0	–1	0	1	0	6
a_3	1	–1	1	0	0	0	0	0	1	2
w	–5	4	–4	0	1	1	0	0	0	12

$$R_{a_2} = R_{a_2} - 1 R_{x_1}$$

Basis	x_1	x_2'	x_2''	s_1	s_2	s_3	a_1	a_2	a_3	b

s_1	1	1	–1	1	0	0	0	0	0	5
x_1	1	–1/3	1/3	0	–1/3	0	1/3	0	0	4/3
a_2	0	–5/3	5/3	0	1/3	–1	–1/3	1	0	14/3
a_3	1	–1	1	0	0	0	0	0	1	2
w	–5	4	–4	0	1	1	0	0	0	12

$$R_{a_3} = R_{a_3} - 1R_{x_1}$$

Basis	x_1	x_2'	x_2''	s_1	s_2	s_3	a_1	a_2	a_3	b
s_1	1	1	–1	1	0	0	0	0	0	5
x_1	1	–1/3	1/3	0	–1/3	0	1/3	0	0	4/3
a_2	0	–5/3	5/3	0	1/3	–1	–1/3	1	0	14/3
a_3	0	–2/3	2/3	0	1/3	0	–1/3	0	1	2/3
w	–5	4	–4	0	1	1	0	0	0	12

$$R_w = R_w + 5R_{x_1}$$

Basis	x_1	x_2'	x_2''	s_1	s_2	s_3	a_1	a_2	a_3	b
s_1	1	1	–1	1	0	0	0	0	0	5
x_1	1	–1/3	1/3	0	–1/3	0	1/3	0	0	4/3
a_2	0	–5/3	5/3	0	1/3	–1	–1/3	1	0	14/3
a_3	0	–2/3	2/3	0	1/3	0	0-1/3	0	1	2/3
w	0	7/3	–7/3	0	–2/3	1	5/3	0	0	56/3

$$R_{s_1} = R_{s_1} - 1R_{x_1}$$

Basis	x_1	x_2'	x_2''	s_1	s_2	s_3	a_1	a_2	a_3	b
s_1	0	4/3	–4/3	1	1/3	0	–1/3	0	0	11/3
x_1	1	–1/3	1/3	0	–1/3	0	1/3	0	0	4/3
a_2	0	–5/3	5/3	0	1/3	–1	–1/3	1	0	14/3
a_3	0	–2/3	2/3	0	1/3	0	–1/3	0	1	2/3
w	0	7/3	–7/3	0	–2/3	1	5/3	0	0	56/3

Step 3: Perform the Second Pivot

1. Look at the w row (objective row) for the most negative coefficient:

Basis	x_1	x_2'	x_2''	s_1	s_2	s_3	a_1	a_2	a_3	b
s_1	0	4/3	–4/3	1	1/3	0	–1/3	0	0	11/3
x_1	1	–1/3	1/3	0	–1/3	0	1/3	0	0	4/3

a_2	0	–5/3	5/3	0	1/3	–1	–1/3	1	0	14/3
a_3	0	–2/3	2/3	0	1/3	0	0-1/3	0	1	2/3
w	0	7/3	–7/3	0	–2/3	1	5/3	0	0	56/3

- The most negative is $-\frac{7}{3}$ for x_2''
  - Pivot column is x_2''

2. Ratio
   - Compute ratios (b/x_2'') for rows with positive pivot entries:

Basis	x_1	x_2'	x_2''	s_1	s_2	s_3	a_1	a_2	a_3	b	b/x_2''
s_1	0	4/3	–4/3	1	1/3	0	–1/3	0	0	11/3	✗
x_1	1	–1/3	1/3	0	–1/3	0	1/3	0	0	4/3	$\frac{4}{3} \div \frac{1}{3} = 4$
a_2	0	–5/3	5/3	0	1/3	–1	–1/3	1	0	14/3	$\frac{14}{3} \div \frac{5}{3} = 2.8$
a_3	0	–2/3	2/3	0	1/3	0	0-1/3	0	1	2/3	$\frac{2}{3} \div \frac{2}{3} = 1$
w	0	7/3	–7/3	0	–2/3	1	5/3	0	0	56/3	

   - Minimum ratio is 1, so we pivot on row a_3:

Basis	x_1	x_2'	x_2''	s_1	s_2	s_3	a_1	a_2	a_3	b	b/x_2''
s_1	0	4/3	–4/3	1	1/3	0	–1/3	0	0	11/3	✗
x_1	1	–1/3	1/3	0	–1/3	0	1/3	0	0	4/3	$\frac{4}{3} \div \frac{1}{3} = 4$
a_2	0	–5/3	5/3	0	1/3	–1	–1/3	1	0	14/3	$\frac{14}{3} \div \frac{5}{3} = 2.8$
a_3	0	–2/3	2/3	0	1/3	0	0-1/3	0	1	2/3	$\frac{2}{3} \div \frac{2}{3} = 1$
w	0	7/3	–7/3	0	–2/3	1	5/3	0	0	56/3	

Step 4: Pivot $a_3 \longrightarrow x_2''$

Basis	x_1	x_2'	x_2''	s_1	s_2	s_3	a_1	a_2	a_3	b
s_1	0	4/3	–4/3	1	1/3	0	–1/3	0	0	11/3
x_1	1	–1/3	1/3	0	–1/3	0	1/3	0	0	4/3
a_2	0	–5/3	5/3	0	1/3	–1	–1/3	1	0	14/3
x_2''	0	–2/3	2/3	0	1/3	0	–1/3	0	1	2/3
w	0	7/3	–7/3	0	–2/3	1	5/3	0	0	56/3

Gauss-Jordan Elimination

$$R_{x_2''} = R_{x_2''}/(2/3)$$

Basis	x_1	x_2'	x_2''	s_1	s_2	s_3	a_1	a_2	a_3	b
s_1	0	4/3	–4/3	1	1/3	0	–1/3	0	0	11/3

x_1	1	–1/3	1/3	0	–1/3	0	1/3	0	0	4/3
a_2	0	–5/3	5/3	0	1/3	–1	–1/3	1	0	14/3
x_2''	0	–1	1	0	1/2	0	–1/2	0	3/2	1
w	0	7/3	–7/3	0	–2/3	1	5/3	0	0	56/3

$$R_{s_1} = R_{s_1} - \left(-\frac{4}{3}\right) R_{x_2''} = R_{s_1} + \frac{4}{3} R_{x_2''}$$

Basis	x_1	x_2'	x_2''	s_1	s_2	s_3	a_1	a_2	a_3	b
s_1	0	0	0	1	1	0	–1	0	2	5
x_1	1	–1/3	1/3	0	–1/3	0	1/3	0	0	4/3
a_2	0	–5/3	5/3	0	1/3	–1	–1/3	1	0	14/3
x_2''	0	–1	1	0	1/2	0	–1/2	0	3/2	1
w	0	7/3	–7/3	0	–2/3	1	5/3	0	0	56/3

$$R_{x_1} = R_{x_1} - \frac{1}{3} R_{x_2''}$$

Basis	x_1	x_2'	x_2''	s_1	s_2	s_3	a_1	a_2	a_3	b
s_1	0	0	0	0	–1/2	0	1/2	0	–1/2	1
x_1	1	0	0	0	–1/2	0	1/2	0	–1/2	1
a_2	0	–5/3	5/3	0	1/3	–1	–1/3	1	0	14/3
x_2''	0	–1	1	0	1/2	0	–1/2	0	3/2	1
w	0	7/3	–7/3	0	–2/3	1	5/3	0	0	56/3

$$R_{a_2} = R_{a_2} - \frac{5}{3} R_{x_2''}$$

Basis	x_1	x_2'	x_2''	s_1	s_2	s_3	a_1	a_2	a_3	b
s_1	0	0	0	0	–1/2	0	1/2	0	–1/2	1
x_1	1	0	0	0	–1/2	0	1/2	0	–1/2	1
a_2	0	0	0	0	–1/2	–1	1/2	1	–5/2	3
x_2''	0	–1	1	0	1/2	0	–1/2	0	3/2	1
w	0	7/3	–7/3	0	–2/3	1	5/3	0	0	56/3

$$R_w = R_w - \left(-\frac{7}{3}\right) R_{x_2''} = R_w + \frac{7}{3} R_{x_2''}$$

Basis	x_1	x_2'	x_2''	s_1	s_2	s_3	a_1	a_2	a_3	b
s_1	0	0	0	0	–1/2	0	1/2	0	–1/2	1

x_1	1	0	0	0	–1/2	0	1/2	0	–1/2	1
a_2	0	0	0	0	–1/2	–1	1/2	1	–5/2	3
x_2''	0	–1	1	0	1/2	0	–1/2	0	3/2	1
w	0	0	0	0	1/2	1	1/2	0	7/2	21

Step 5: Perform the Third Pivot

1. Look at the w row (objective row) for the most negative coefficient:

Basis	x_1	x_2'	x_2''	s_1	s_2	s_3	a_1	a_2	a_3	b
s_1	0	0	0	0	–1/2	0	1/2	0	–1/2	1
x_1	1	0	0	0	–1/2	0	1/2	0	–1/2	1
a_2	0	0	0	0	–1/2	–1	1/2	1	–5/2	3
x_2''	0	–1	1	0	1/2	0	–1/2	0	3/2	1
w	0	0	0	0	–1/6	1	1/2	0	7/2	21

   - The most negative is $-\frac{1}{6}$ for s_2
     - Pivot column is s_2

2. Ratio
   - Compute ratios (b/s_2) for rows with positive pivot entries:

Basis	x_1	x_2'	x_2''	s_1	s_2	s_3	a_1	a_2	a_3	b	b/s_2
s_1	0	0	0	0	–1/2	0	1/2	0	–1/2	1	✗
x_1	1	0	0	0	–1/2	0	1/2	0	–1/2	1	✗
a_2	0	0	0	0	–1/2	–1	1/2	1	–5/2	3	✗
x_2''	0	–1	1	0	1/2	0	–1/2	0	3/2	1	2
w	0	0	0	0	–1/6	1	1/2	0	7/2	21	

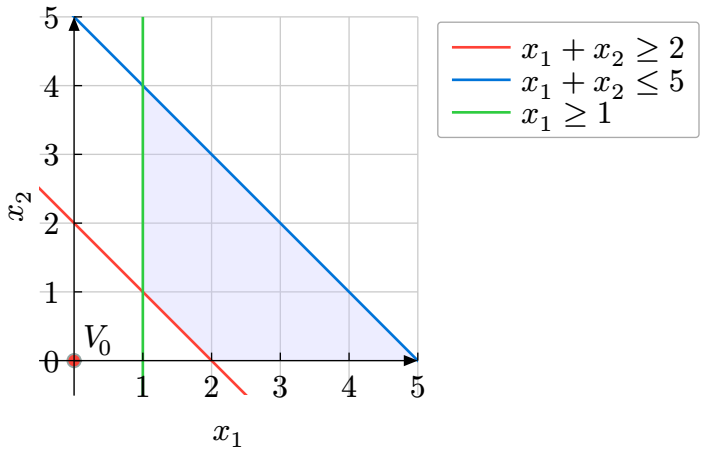
Two-Phase

$$
\begin{aligned}
\max \quad & z = 2x_1 + x_2 \\
s.t. \quad & x_1 + x_2 \geq 2 \\
& x_1 \geq 1 \\
& x_1 + x_2 \leq 5 \\
& x_1, x_2 \geq 0
\end{aligned}
$$

$$
\begin{array}{lrcrcrcrcrcl}
\max & z = 2x_1 & + & x_2 & + & 0e_1 & + & 0e_2 & + & 0s_1 & & \\
s.t. & x_1 & + & x_2 & - & e_1 & & & & & \geq & 2 \\
& x_1 & & & & & - & e_2 & & & \geq & 1 \\
& x_1 & + & x_2 & & & & & + & s_1 & \leq & 5
\end{array}
$$

$$
z - 2x_1 - x_2 = 0
$$

Basis	x_1	x_2	e_1	e_2	s_1	b
???	1	1	-1	0	0	2
???	1	0	0	-1	0	1
s_1	1	1	0	0	1	5
z	–2	–1	0	0	0	0



$$
\begin{array}{lrcrcrcrcrcl}
\min & w = 0x_1 & + & 0x_2 & + & 0e_1 & + & 0e_2 & + & 0s_1 & & \\
s.t. & x_1 & + & x_2 & - & e_1 & & & & & \geq & 2 \\
& x_1 & & & & & - & e_2 & & & \geq & 1 \\
& x_1 & + & x_2 & & & & & + & s_1 & \leq & 5
\end{array}
$$

Degeneracy

Degeneracy occurs when a basic variable takes the value zero in a basic feasible solution. This leads to ties in the minimum ratio test, which determines the leaving variable during a pivot.

Example

Problem

$$
\begin{aligned}
\max \quad & z = x_1 + 2x_2 \\
s.t. \quad & x_1 + x_2 \leq 3 \\
& x_2 \leq 2 \\
& \frac{1}{2}x_1 + x_2 \leq 2.5 \\
& x_1, x_2 \geq 0
\end{aligned}
$$

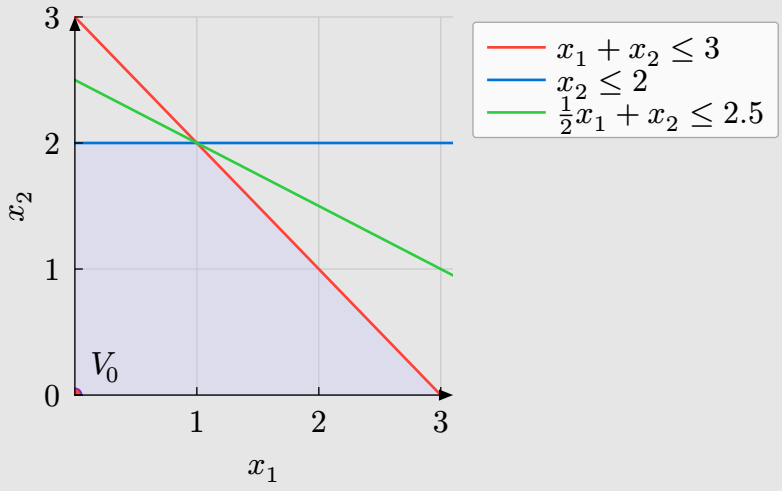
Standard Form

$$
\begin{aligned}
z - x_1 - 2x_2 &= 0 \\
x_1 + x_2 + s_1 &= 3 \\
x_{2_{s_2}} &= 2 \\
\frac{1}{2}x_1 + x_2 + s_3 & \\
x_1, x_2, s_1, s_2, s_3 &\underset{=}{>} 0
\end{aligned}
$$

Initialize Tableau

Basis	x_1	x_2	s_1	s_2	s_3	b
s_1	1	1	1	0	0	3
s_2	0	1	0	1	0	2
s_3	1/2	1	0	0	1	2.5
w	–1	–1	0	0	0	0

$$
\begin{aligned}
x_1 &= 0 \\
x_2 &= 0
\end{aligned}
$$



Minimum Test

Basis	x_1	x_2	s_1	s_2	s_3	b
s_1	1	1	1	0	0	3

s_2	0	1	0	1	0	2
s_3	1/2	1	0	0	1	2.5
w	−1	−1	0	0	0	0

Ratio Test

Basis	x_1	x_2	s_1	s_2	s_3	b
s_1	1	1	1	0	0	3
s_2	0	1	0	1	0	2
s_3	1/2	1	0	0	1	2.5
w	−1	−1	0	0	0	0

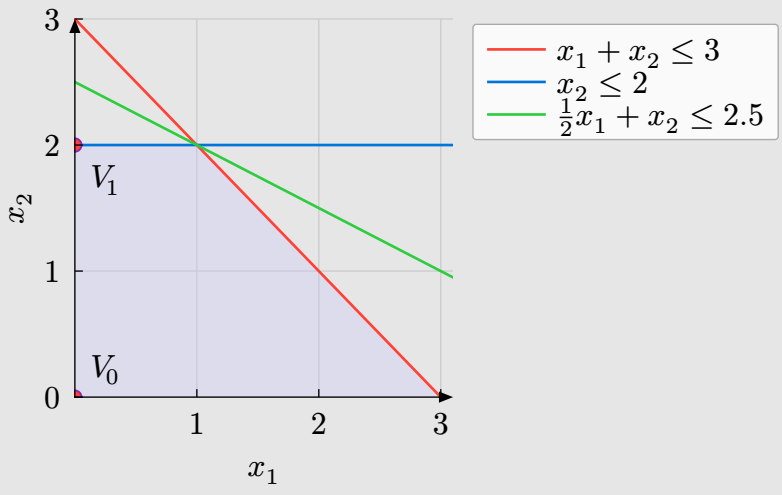
Pivot

Basis	x_1	x_2	s_1	s_2	s_3	b
s_1	1	1	1	0	0	3
x_2	0	1	0	1	0	2
s_3	1/2	1	0	0	1	2.5
w	−1	−1	0	0	0	0

Gaussian Elimination

Basis	x_1	x_2	s_1	s_2	s_3	b
s_1	1	0	1	−1	0	1
x_2	0	1	0	1	0	2
s_3	1/2	0	0	−1	1	0.5
w	−1	0	0	2	0	4

$$x_1 = 0$$
$$x_2 = 2$$



Minimum Test

Basis	x_1	x_2	s_1	s_2	s_3	b
s_1	1	0	1	–1	0	1
x_2	0	1	0	1	0	2
s_3	1/2	0	0	–1	1	0.5
w	–1	0	0	2	0	4

Ratio Test

Basis	x_1	x_2	s_1	s_2	s_3	b
s_1	1	0	1	–1	0	1
x_2	0	1	0	1	0	2
s_3	1/2	0	0	–1	1	0.5
w	–1	0	0	2	0	4

$$\left.\begin{aligned} \tfrac{1}{1} &= 1 \\ \tfrac{0.5}{0.5} &= 1 \end{aligned}\right\} \text{Tie}$$

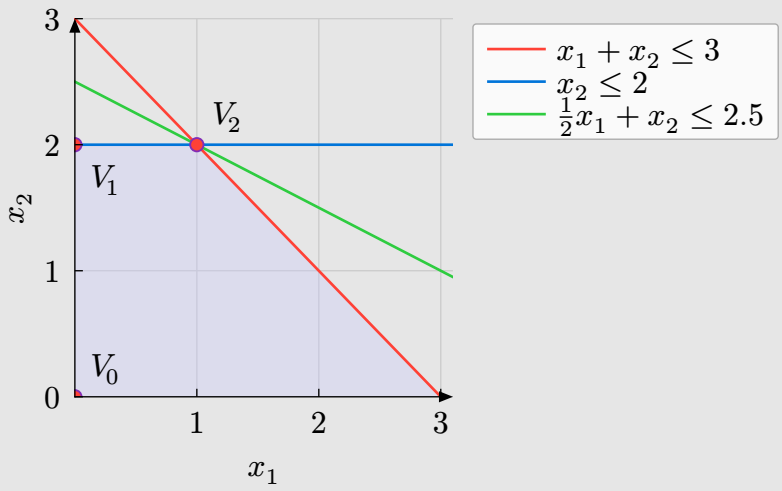
Pivot

Basis	x_1	x_2	s_1	s_2	s_3	b
x_1	1	0	1	–1	0	1
x_2	0	1	0	1	0	2
s_3	1/2	0	0	–1	1	0.5
w	–1	0	0	2	0	4

Gaussian Elimination

Basis	x_1	x_2	s_1	s_2	s_3	b
x_1	1	0	1	–1	0	1
s_2	0	1	0	0	0	2
s_3	0	0	–1/2	–1/2	1	0
w	0	0	1	1	0	5

If after elimination a basic variable has a value of 0, it indicates degeneracy.



Degeneracy comes from redundant constraints

Bland's Rule: Choose the variable with the lowest index in the basis

- x_1 over x_2
- x_2 over s_1

Unbounded Solution

When identifying the entering variable, look at its column in the tableau. If all entries in that column (above the objective row) are ≤ 0, the problem is unbounded

Can't perform ratio test $\longrightarrow$ no constraint limits the entering variable $\longrightarrow$ objective function increases without bound

Example

Problem

$$\begin{aligned} \max \quad & x_1 + x_2 \\ s.t. \quad & x_1 - x_2 \leq 1 \\ & x_1, x_2 \geq 0 \end{aligned}$$

Standard Form

$$\begin{aligned} \max \quad & z - x_1 - x_2 = 0 \\ s.t. \quad & x_1 - 2x_2 + s_1 = 1 \\ & x_1, x_2, s_1 \geq 0 \end{aligned}$$

Initialize Tableau

Basis	x_1	x_2	s_1	b
s_1	1	−1	1	1
z	−1	−1	0	0

Minimum and Ratio Test

Basis	x_1	x_2	s_1	b
s_1	1	–1	1	1
z	–1	–1	0	0

Pivot

Basis	x_1	x_2	s_1	b
x_1	1	–1	1	1
z	–1	–1	0	0

Gaussian Elimination

Basis	x_1	x_2	s_1	b
x_1	1	–1	1	1
z	0	–2	1	1

Minimum and Ratio Test

Basis	x_1	x_2	s_1	b
x_1	1	–1	1	1
z	0	–2	1	1

Check for Optimality

Objective row has –2 for x_2 $\longrightarrow$ not optimal

Entering variable: x_2 (most negative coefficient)

Unboundedness Detection

Looking at the x_2 column:

Entry in constraint row: –1 (which is ≤ 0)

All entries in x_2 column are ≤ 0

Conclusion: Cannot perform ratio test $\longrightarrow$ Problem is unbounded

Alternative Solutions

Example

Problem

$$\begin{aligned} \max \quad & 2x_1 + 4x_2 \\ s.t. \quad & x_1 + x_2 \geq 3 \\ & \frac{1}{2}x_1 + x_2 \geq 2.5 \\ & x_1, x_2 \geq 0 \end{aligned}$$

Standard Form

$$\begin{aligned} z - 2x_1 - x_2 &= 0 \\ x_1 + x_2 + s_1 &= 3 \\ \frac{1}{2}x_1 + x_2 + s_2 &= 2.5 \\ x_1, x_2, x_3 &\geq 0 \end{aligned}$$

Initialize Tableau

Basis	x_1	x_2	s_1	s_2	b
s_1	1	1	1	0	3
s_2	1/2	1	0	1	2.5
z	–2	–4	0	0	0

Minimum Test

Basis	x_1	x_2	s_1	s_2	b
s_1	1	1	1	0	3
s_2	1/2	1	0	1	2.5
z	–2	–4	0	0	0

Ratio Test

Basis	x_1	x_2	s_1	s_2	b
s_1	1	1	1	0	3
s_2	1/2	1	0	1	2.5
z	–2	–4	0	0	0

Pivot

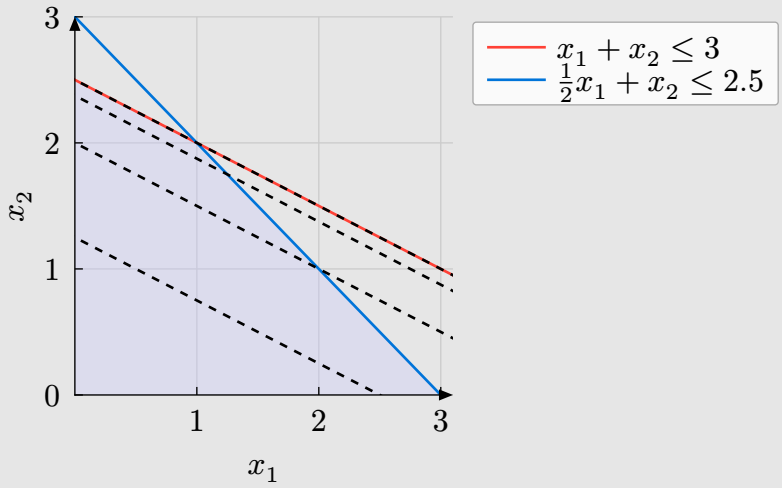
Basis	x_1	x_2	s_1	s_2	b
s_1	1	1	1	0	3
x_2	1/2	1	0	1	2.5
z	–2	–4	0	0	0

Gaussian Elimination

Basis	x_1	x_2	s_1	s_2	b

s_1	1/2	0	1	–1	0.5
x_2	1/2	1	0	1	2.5
z	0	0	0	4	10

In a maximization or minimization linear programming problem, if there is a 0 in the z row (objective function row) of the final (optimal) simplex tableau in a non-basic column (i.e. a variable not currently in the solution), then there are multiple optimal solutions.



181.5.0.7. Standard Form

- **Equalities**: Convert inequalities (using slack, surplus, or artificial variables)
- **Non-negative variables**: If a variable is unrestricted or can be negative, replace it with the difference of two non-negative variables

Example

$$2x_1 + 3x_2 \leq -4$$
$$\iff$$
$$-2x_1 - 3x_2 \geq 4$$

Nonpositive

$$2x_1 + 3x_2 \leq 4, \quad x_1 \leq 0$$
$$\iff$$
$$-2x_1 + 3x_2 \leq 4, \quad x_1 \geq 0$$

Free (Unrestricted)

$$2x_1 + 3x_2 \le 6, \quad x_1 \text{ urs}$$
$$\Leftrightarrow$$
$$2x_1' - 2x_1'' + 3x_2 \le 4, \quad x_1', x_1'' \ge 0$$

181.5.0.8. Simplex Method

Example

$$\begin{aligned} \max \quad & 7x_1 + 6x_2 \\ s.t. \quad & 2x_1 + 4x_2 \le 16 \\ & 3x_1 + 2x_2 \le 12 \\ & x_1, x_2 \ge 0 \end{aligned}$$

$$\begin{aligned} \max \quad & 7x_1 + 6x_2 + 0s_1 + 0s_2 \\ s.t. \quad & 2x_1 + 4x_2 + s_1 = 16 \\ & 3x_1 + 2x_2 + s_2 = 12 \end{aligned} \qquad x_1, x_2, s_1, s_2 \ge 0$$

Basis	c_B	x_1 7	x_2 6	s_1 0	s_2 0	b
s_1	0	2	4	1	0	16
s_2	0	3	2	0	1	12
	z_j	$0 \times 2 + 0 \times 3 = 0$	0	0	0	0
	$c_j - z_j$	7	6	0	0	

$$x_1 = 0, x_2 = 0, s_1 = 16, s_2 = 12$$

Basis	c_B	x_1 7	x_2 6	s_1 0	s_2 0	b	Ratio
s_1	0	2	4	1	0	16	16/2 = 8
s_2	0	3	2	0	1	12	12/3 = 4
	z_j	0	0	0	0	0	
	$c_j - z_j$	7	6	0	0		

---

1. Formulation

Objective Function

$$
\begin{aligned}
Z &= c_1x_1 + c_2x_2 + ... + c_nx_n \\
&= c^Tx
\end{aligned}
$$

Constraints

$$
\begin{aligned}
a_{11}x_1 + a_{12}x_2 + ... + a_{1n}x_n &\leq b_1 \\
a_{21}x_1 + a_{22}x_2 + ... + a_{2n}x_n &\geq b_2 \\
&\vdots \\
a_{m1}x_1 + a_{m2}x_2 + ... + a_{mn}x_n &= b_m \\
x_1, x_2, ..., x_n &\geq 0
\end{aligned}
$$

Or,

$$
\begin{aligned}
Ax &\leq, =, \geq b \\
x &\geq 0
\end{aligned}
$$

- $\leq$ for maximization
- $\geq$ for minimization

2. Convert to Canonical Form

In **slack variable form**

$$
\begin{aligned}
a_{11} + x_1 + a_{12}x_2 + ... + a_{1n}x_n + s_1 &= b_1 \\
a_{21} + x_1 + a_{22}x_2 + ... + a_{2n}x_n + s_2 &= b_2 \\
&\vdots \\
a_{m1}x_1 + a_{m2}x_2 + ... + a_{mn}x_n + s_m &= b_m \\
x_1, x_2, ..., x_n &\geq 0 \\
s_1, s_2, ..., s_m &\geq 0
\end{aligned}
$$

Or **matrix notation**,

$$
\begin{aligned}
\max \quad & z = c^Tx \\
s.t. \quad & Ax + Is = b_i \\
& x \geq 0 \\
& s \geq 0
\end{aligned}
$$

$$
\begin{aligned}
\max \quad z \quad &= \quad c^T \quad x \\
&= [c_1, c_2, ..., c_n] \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}
\end{aligned}
$$

$$
A \quad x \quad + \quad I \quad s \quad = \quad b
$$

$$\begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} + \begin{bmatrix} 1 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{bmatrix} \begin{bmatrix} s_1 \\ s_2 \\ \vdots \\ s_m \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix}$$

3. Set up Simplex Tableau

	x_1	x_2	...	x_n	s_1	s_2	...	s_m	RHS
Constraint 1	a_{11}	a_{12}	...	$a_1 n$	1	0	...	0	b_1
Constraint 2	a_{21}	a_{22}	...	$a_2 n$	0	1	...	0	b_2
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$
Constraint n	a_{m1}	a_{m2}	...	a_{mn}	0	0	...	1	b_m
Objective Function	$-c_1$	$-c_2$	...	$-c_n$	0	0	...	0	0

- $-$ for maximization
- $+$ for minimization

4. Perform Pivot Operation

- **Identify the Pivot Column**: Choose the column with the most:
  - Maximization: Negative coefficient
  - Minimization: Positive coefficient (or multiply the objective by −1 and treat it as a maximization)

in the objective function row (this indicates which variable will enter the basis).

- **Identify the Pivot Row**: Calculate the ratio of RHS to the pivot column's coefficients (only where coefficients are positive). The row with the smallest ratio determines the variable to leave the basis.
- **Pivot**: Perform row operations to change the pivot element to 1 and other elements in the pivot column to 0. This involves dividing the pivot row by the pivot element and then adjusting the other rows to zero out the pivot column.

5. Iterate

Repeat the pivot operations until there are no more negative coefficients in the objective function row, indicating that the current solution is optimal.

6. Read the Solution

The final tableau will give the values of the variables at the optimal solution. The basic variables are the variables corresponding to the columns with the identity matrix in the final tableau, while non-basic variables are set to zero.

Example

$$\text{Maximize}: \quad Z = 3x_1 + 2x_2$$

s.t.

$$x_1 + x_2 \leq 4$$

$$2x_1 + x_2 \leq 5$$

$$x_1, x_2 \geq 0$$

1. Convert to Canonical Form

Introduce slack variables s_1 and s_2 to convert inequalities into equalities:

$$x_1 + x_2 + s_1 = 4$$

$$2x_1 + x_2 + s_2 = 5$$

$$x_1, x_2, s_1, s_2 \geq 0$$

Initialize Simplex Tableau

	x_1	x_2	s_1	s_2	RHS
Constraint 1	1	1	1	0	4
Constraint 2	2	1	0	1	5
Objective Function	–3	–2	0	0	0

2. Identify the Pivot Column

In the objective function row, the coefficients are $[-3, -2, 0, 0]$. The most negative coefficient is -3, which is in the x_1 column. Therefore, x_1 will enter the basis.

3. Identify the Pivot Row

Calculate the ratio of RHS to the pivot column's coefficients (where the coefficients are positive):

- For Row 1: $\frac{4}{1} = 4$ (pivot column coefficient is 1)
- For Row 2: $\frac{5}{2} = 2.5$ (pivot column coefficient is 2)

The smallest ratio is 2.5, so Row 2 will be the pivot row. This means that s_2 will leave the basis.

4. Pivot

Perform row operations to make the pivot element 1 and zero out other elements in the pivot column.

Pivot Element: The element at the intersection of Row 2 and x_1 column is 2.

Steps:

1. Make Pivot Element 1: Divide all elements in Row 2 by the pivot element (2):

$$\begin{aligned}\text{New Row 2} &= \frac{1}{2} \times \text{Old Row 2} \\ &= [1, 0.5, 0, 0.5, 2.5]\end{aligned}$$

2. Zero Out Pivot Column in Other Rows:
   - For Row 1: Subtract 1 times New Row 2 from Row 1:

$$\begin{aligned}\text{New Row 1} &= \text{Old Row 1} - [1, 0.5, 0, 0.5, 2.5] \\ &= [0, 0.5, 1, -0.5, 1.5]\end{aligned}$$

   - For Objective Function: Add 3 times New Row 2 to the Objective Function row

$$\begin{aligned}\text{New Objective Function} &= \text{Old Objective Function} + 3 \times [1, 0.5, 0, 0.5, 2.5] \\ &= [0, -0.5, 0, 1.5, 7.5]\end{aligned}$$

Updated Simplex Tableau

	x_1	x_2	s_1	s_2	RHS
Constraint 1	0	0.5	1	–0.5	1.5
Constraint 2	1	0.5	0	0.5	2.5
Objective Function	0	–0.5	0	1.5	7.5

181.5.1. Matrix Notation

$$\begin{aligned}\max \quad & c^T x \\ s.t. \quad & Ax = b \\ & x \geq 0\end{aligned}$$

$$\begin{aligned}A &\in \mathbb{R}^{m \times n} \\ b &\in \mathbb{R}^{m \times 1} \\ c &\in \mathbb{R}^{n \times 1} \\ x &\in \mathbb{R}^{n \times 1}\end{aligned}$$

$$\begin{aligned}\max \quad & c_B^T x_B + c_N^T x_N \\ s.t. \quad & Bx_B + Nx_N = b \\ & x_B, x_N \geq 0\end{aligned}$$

$$\begin{aligned}c^T &= [c_B^T, c_N^T] \\ A &= [B, N]\end{aligned}$$

Where:

- x_B: Basic variables (in the basis)
- x_N: Non-basic variables (not in the basis)
- c_B: Coefficients of basic variables in the objective function
- c_N: Coefficients of non-basic variables in the objective function
- B: Columns of A corresponding to **basic** variables
- N: Columns of A corresponding to **non-basic** variables

- b: Right-hand side constants

Example

Standard form:

$$\begin{aligned} \max \quad & x_1 \\ s.t. \quad & 2x_1 - x_2 + x_3 = 4 \\ & 2x_1 + x_2 + x_4 = 8 \\ & x_2 + x_3 = 3 \\ & x_1, x_2, x_3, x_4 \geq 0 \quad \forall i = 1, ..., 5 \end{aligned}$$

$$c^T = [1, 0, 0, 0, 0]$$

$$A = \begin{bmatrix} 2 & -1 & 1 & 0 & 0 \\ 2 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 \end{bmatrix}$$

$$b = \begin{bmatrix} 4 \\ 8 \\ 3 \end{bmatrix}$$

Given $x_B = (x_1, x_4, x_5)$ and $x_N = (x_2, x_3)$:

$$c_B^T = [1, 0, 0], \qquad c_N^T = [0, 0]$$

$$B = \begin{bmatrix} 2 & 0 & 0 \\ 2 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \qquad N = \begin{bmatrix} -1 & 1 \\ 1 & 0 \\ 1 & 0 \end{bmatrix}$$

$$b = \begin{bmatrix} 4 \\ 8 \\ 3 \end{bmatrix}$$

Objective Function

$$1x_1 + 0x_2 + 0x_3 + 0x_4 + 0x_5 = 0$$

$$x_B^T x_b + x_N^T x_N = 0$$

Constraints

$$
\begin{aligned}
2x_1 - 1x_2 + 1x_3 + 0x_4 + 0x_5 &= 4 \\
2x_1 + 1x_2 + 0x_3 + 1x_4 + 0x_5 &= 8 \\
0x_1 + 1x_2 + 0x_3 + 0x_4 + 1x_5 &= 3
\end{aligned}
$$

$$
A_B x_B + A_N x_N = b
$$

Problem

$$
\begin{aligned}
\max \quad & c_B^T x_B + c_N^T x_N \\
s.t. \quad & B x_B + N x_N = b \\
& x_B, x_N \geq 0
\end{aligned}
$$

Rearange the the terms in the constrains:

$$
\begin{aligned}
\max \quad & c_B^T x_B + c_N^T x_N \\
s.t. \quad & x_B = B^{-1}(b - N x_N) \\
& x_B, x_N \geq 0
\end{aligned}
$$

Replace x_B in the objective function:

$$
\begin{aligned}
\max \quad & c_B^T [B^{-1}(b - N x_N)] + c_N^T x_N \\
s.t. \quad & x_B = B^{-1}(b - N x_N) \\
& x_B, x_N \geq 0
\end{aligned}
$$

Rearrange the terms in the objectcive function:

$$
\begin{aligned}
\max \quad & c_B^T B^{-1} b - (c_B^T B^{-1} N - c_N^T) x_N \\
s.t. \quad & x_B = B^{-1}(b - N x_N) \\
& x_B, x_N \geq 0
\end{aligned}
$$

The standard form LP becomes:

$$
\begin{aligned}
\max \quad & c_B^T B^{-1} b - (c_B^T B^{-1} N - c_N^T) x_N \\
s.t. \quad & x_B = B^{-1}(b - N x_N) \\
& x_B, x_N \geq 0
\end{aligned}
$$

Rearrange the terms of the constrains:

$$
\begin{aligned}
\max \quad & c_B^T B^{-1} b - (c_B^T B^{-1} N - c_N^T) x_N \\
s.t. \quad & I x_B + B^{-1} N x_N = B^{-1} b \\
& x_B, x_N \geq 0
\end{aligned}
$$

Ignore the sign constraints and let z be the objectctive value:

$$
\begin{aligned}
z \quad &+ \quad (c_B^T B^{-1} N - c_N^T) x_N \quad &= \quad c_B^T B^{-1} b \\
I x_B \quad &+ \quad B^{-1} N x_N \quad &= \quad B^{-1} b
\end{aligned}
$$

The Simplex Tableau is:

0	$c_B^T B^{-1} N - c_N^T$	$c_B^T B^{-1} b$	0
I	$B^{-1} N x_N$	$B^{-1} b$	$1, ..., m$
basic	non-basic	RHS	

Example

Step 1. Standard form

$$\begin{aligned} \max \quad & x_1 \\ s.t. \quad & 2x_1 - x_2 + x_3 = 4 \\ & 2x_1 + x_2 + x_4 = 8 \\ & x_2 + x_5 = 3 \\ & x_i \geq 0 \quad \forall i = 1, ..., 5 \end{aligned}$$

Matrix notation:

$$c^T = [1, 0, 0, 0, 0], \qquad A = \begin{bmatrix} 2 & -1 & 1 & 0 & 0 \\ 2 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 \end{bmatrix}, \qquad b = \begin{bmatrix} 4 \\ 8 \\ 3 \end{bmatrix}$$

Step 2. Initial Basis

$$B = (x_1, x_4, x_5) \qquad N = (x_2, x_3)$$

$$B = \begin{bmatrix} 2 & 0 & 0 \\ 2 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \qquad N = \begin{bmatrix} -1 & 1 \\ 1 & 0 \\ 1 & 0 \end{bmatrix}$$

Cost split:

$$c_B^T = [1, 0, 0] \qquad c_N^T = [0, 0]$$

Compute inverse of B:

$$B^{-1} = \begin{bmatrix} \frac{1}{2} & 0 & 0 \\ -1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \qquad x_B = B^{-1} b = \begin{bmatrix} \frac{1}{2} & 0 & 0 \\ -1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 4 \\ 8 \\ 3 \end{bmatrix} = \begin{bmatrix} 2 \\ 4 \\ 3 \end{bmatrix}$$

So the basic feasible solution (BFS) is:

$$x = (x_1, x_2, x_3, x_4, x_5) = (2, 0, 0, 4, 3)$$

$$z = c_B^T B^{-1} b = [1, 0, 0] \begin{bmatrix} 2 \\ 4 \\ 3 \end{bmatrix} = 2$$

Step 3. Reduced costs and entering variable

$$\begin{aligned} \bar{c}_N^T &= c_B^T B^{-1} N - c_N^T \\ &= [1 \quad 0 \quad 0] \begin{bmatrix} \frac{1}{2} & 0 & 0 \\ -1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} -1 & 1 \\ 1 & 0 \\ 1 & 0 \end{bmatrix} - [0 \quad 0] \\ &= \begin{bmatrix} -\frac{1}{2} & \frac{1}{2} \end{bmatrix} \end{aligned}$$

Since $\bar{c}_2 = -\frac{1}{2} < 0$, the entering variable is x_2

Step 4. Direction, ratio test and leaving variable

For $x_B = (x_1, x_4, x_5)$, we have:

$$d = B^{-1} N_2 = \begin{bmatrix} \frac{1}{2} & 0 & 0 \\ -1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} -1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} -\frac{1}{2} \\ 2 \\ 1 \end{bmatrix}$$

Current basic solution:

$$x_B = A_B^{-1} b = \begin{bmatrix} 2 \\ 4 \\ 3 \end{bmatrix}$$

The minimum ratios test:

$$\frac{x_4}{2} = \frac{4}{2} = 2 \qquad \frac{x_5}{1} = \frac{3}{1} = 3$$

Smallest ratio is $2 \Rightarrow x_4$ leaves the basis

Step 5. Pivot to the new basis

$$B = (x_1, x_2, x_5) \qquad N = (x_3, x_4)$$

$$B = \begin{bmatrix} 2 & -1 & 1 \\ 2 & 1 & 0 \\ 0 & 1 & 0 \end{bmatrix} \qquad N = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}$$

Cost split:

$$c_B^T = [1, 0, 0] \qquad c_N^T = [0, 0]$$

Compute inverse of B:

$$B^{-1} = \begin{bmatrix} \frac{1}{2} & \frac{1}{4} & 0 \\ -\frac{1}{2} & 1 & 0 \\ \frac{1}{2} & -\frac{1}{2} & 1 \end{bmatrix} \qquad x_B = B^{-1}b = \begin{bmatrix} \frac{1}{2} & \frac{1}{4} & 0 \\ -\frac{1}{2} & 1 & 0 \\ \frac{1}{2} & -\frac{1}{2} & 1 \end{bmatrix} \begin{bmatrix} 4 \\ 8 \\ 3 \end{bmatrix} = \begin{bmatrix} 3 \\ 2 \\ 1 \end{bmatrix} = \begin{bmatrix} x_1 \\ x_2 \\ x_5 \end{bmatrix}$$

So the new basic feasible solution (BFS) is:

$$x = (x_1, x_2, x_3, x_4, x_5) = (3, 2, 0, 0, 1)$$

$$z = c_B^T x_B = [1, 0, 0] \begin{bmatrix} 3 \\ 2 \\ 1 \end{bmatrix} = 3$$

Step 6. Check optimality (reduced costs)

$$\begin{aligned} \bar{c}_N^T &= c_B^T B^{-1} N - c_N^T \\ &= [1 \quad 0 \quad 0] \begin{bmatrix} \frac{1}{4} & \frac{1}{4} & 0 \\ -\frac{1}{2} & \frac{1}{2} & 0 \\ \frac{1}{2} & -\frac{1}{2} & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix} - [0 \quad 0] \\ &= \begin{bmatrix} \frac{1}{4} & \frac{1}{4} \end{bmatrix} \end{aligned}$$

No negative reduced costs $\Rightarrow$ optimal

$$x^* = (3, 2, 0, 0, 1), \qquad z^* = 3$$

Example

Problem:

$$\begin{aligned} \max \quad & 2x_1 + 3x_2 \\ s.t. \quad & x_1 + x_2 \leq 4 \\ & x_1 + 2x_2 \leq 6 \\ & x_1, x_2 \geq 0 \end{aligned}$$

Standard form:

$$\begin{aligned} \max \quad & 2x_1 + 3x_2 \\ s.t. \quad & x_1 + x_2 + s_1 = 4 \\ & x_1 + 2x_2 + s_2 = 6 \\ & x_1, x_2, s_1, s_2 \geq 0 \end{aligned}$$

Let $x_B = (s_1, s_2)$ and $x_N = (x_1, x_2)$

$$c_B = [0, 0] \qquad c_N = [2, 3]$$

$$A_B = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \qquad A_N = \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}$$

$$b = \begin{bmatrix} 4 \\ 6 \end{bmatrix}$$

$$A_B^{-1} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$A_B^{-1} A_N = \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}$$

$$A_B^{-1} b = \begin{bmatrix} 4 \\ 6 \end{bmatrix}$$

$$c_B^T A_B^{-1} A_N - c_N^T = [-2 \quad -3]$$

$$c_B^T A_B^{-1} b = 0$$

-2	-3	0	0	0
1	1	1	0	4
1	2	0	1	6
basic		non-basic		RHS

Let $x_B = (x_1, x_2)$ and $x_N = (s_1, s_2)$

$$c_B = \begin{bmatrix} 2 \\ 3 \end{bmatrix} \qquad c_N = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$A_B = \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix} \qquad A_N = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$b = \begin{bmatrix} 4 \\ 6 \end{bmatrix}$$

0	0	1	1	10
1	0	2	-1	2
0	1	-1	1	2
basic		non-basic		RHS

$A_B^{-1} = \begin{bmatrix} 2 & -1 \\ -1 & 1 \end{bmatrix}$ $A_B^{-1}A_N = \begin{bmatrix} 2 & -1 \\ -1 & 1 \end{bmatrix}$ $A_B^{-1}b = \begin{bmatrix} 2 \\ 2 \end{bmatrix}$ $c_B^T A_B^{-1} A_N - c_N^T = [1 \quad 1]$ $c_B^T A_B^{-1} b = 10$	

Example

Step 1. Canonical form

$$\begin{array}{lrcrcl} \max & x_1 & + & 3x_2 & & \\ s.t. & -x_1 & + & x_2 & \leq & 3 \\ & -x_1 & + & 2x_2 & \leq & 8 \\ & 3x_1 & + & x_2 & \leq & 18 \\ & x_1, x_2 \geq 0 & & & & \end{array}$$

Step 2. Standard form

$$\begin{array}{lrcrcrcrcl} \max & x_1 & + & 3x_2 & & & & & & \\ s.t. & -x_1 & + & x_2 & + & s_1 & & & = & 3 \\ & -x_1 & + & 2x_2 & & & + & s_2 & = & 8 \\ & 3x_1 & + & x_2 & & & & + \quad s_3 & = & 18 \\ & x_1, x_2, s_1, s_2 \geq 0 & & & & & & & & \end{array}$$

Matrix notation:

$$c^T = [1 \quad 3 \quad 0 \quad 0 \quad 0]$$

$$A = \begin{bmatrix} -1 & 1 & 1 & 0 & 0 \\ -1 & 2 & 0 & 1 & 0 \\ 3 & 1 & 0 & 0 & 1 \end{bmatrix} \qquad b = \begin{bmatrix} 3 \\ 8 \\ 18 \end{bmatrix}$$

Step 3. Initial Basis

$$B = (s_1, s_2, s_3) \qquad N = (x_2, x_2)$$

$$B = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \qquad N = \begin{bmatrix} -1 & 1 \\ -1 & 2 \\ 3 & 1 \end{bmatrix}$$

$$c_B^T = [0 \quad 0 \quad 0] \qquad c_N^T = [1 \quad 3]$$

Compute B^{-1}:

$$B^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

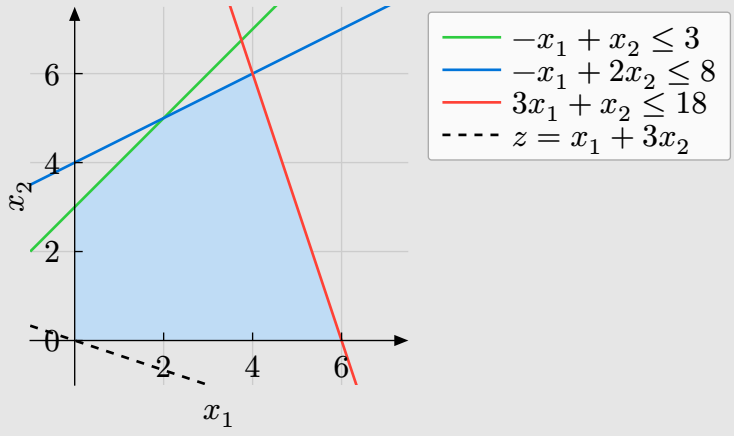
Compute basic feasible solution:

$$x_B = B^{-1}b = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 3 \\ 8 \\ 18 \end{bmatrix} = \begin{bmatrix} 3 \\ 8 \\ 18 \end{bmatrix} = \begin{bmatrix} s_1 \\ s_2 \\ s_3 \end{bmatrix}$$

Current basic feasible solution:

$$x = (x_1, x_2, s_1, s_2, s_3) = (0, 0, 3, 8, 18)$$

$$z = c_B^T B^{-1} b = c_B^T x_B = [0 \quad 0 \quad 0] \begin{bmatrix} 3 \\ 8 \\ 18 \end{bmatrix} = 0$$



Step 4. Reduced costs and **entering variable**

$$\begin{aligned}\bar{c}_N^T &= c_B^T B^{-1} N - c_N^T \\ &= \begin{bmatrix}0 & 0 & 0\end{bmatrix}\begin{bmatrix}1 & 0 & 0\\ 0 & 1 & 0\\ 0 & 0 & 1\end{bmatrix}\begin{bmatrix}-1 & 1\\ -1 & 2\\ 3 & 1\end{bmatrix} - \begin{bmatrix}1 & 3\end{bmatrix} \\ &= \begin{bmatrix}-1 & -3\end{bmatrix}\end{aligned}$$

Since $\bar{c}_1 = -1 < 0$, the **entering variable** is x_1

Step 5. Direction, ratio test and **leaving variable**

For $x_B = (s_1, s_2, s_3)$, we have:

$$d = B^{-1}N_1 = \begin{bmatrix}1 & 0 & 0\\ 0 & 1 & 0\\ 0 & 0 & 1\end{bmatrix}\begin{bmatrix}-1\\ 1\\ 3\end{bmatrix} = \begin{bmatrix}-1\\ -1\\ 3\end{bmatrix}$$

Current basic solution:

$$x_B = B^{-1}b = \begin{bmatrix}3\\ 8\\ 18\end{bmatrix}$$

The minimum ratios test:

$$\frac{s_3}{3} = \frac{18}{3} = 6$$

Smallest ratio is $3 \Rightarrow s_3$ **leaves the basis** (only consider ratios where the corresponding $d_i > 0$)

Step 6. Pivot to the new basis

$$B = (s_1, s_2, x_1) \qquad N = (s_3, x_2)$$

$$B = \begin{bmatrix}1 & 0 & -1\\ 0 & 1 & -1\\ 0 & 0 & 3\end{bmatrix} \qquad N = \begin{bmatrix}0 & 1\\ 0 & 2\\ 1 & 1\end{bmatrix}$$

$$c_B^T = \begin{bmatrix}0 & 0 & 1\end{bmatrix} \qquad c_N^T = \begin{bmatrix}0 & 3\end{bmatrix}$$

Compute B^{-1}:

$$B^{-1} = \begin{bmatrix}1 & 0 & 0.33\\ 0 & 1 & 0.33\\ 0 & 0 & 0.33\end{bmatrix}$$

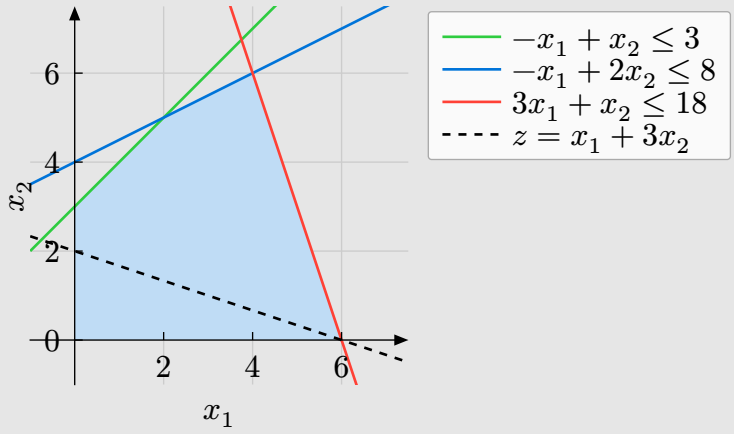
Compute basic feasible solution:

$$x_B = B^{-1}b = \begin{bmatrix} 1 & 0 & 0.33 \\ 0 & 1 & 0.33 \\ 0 & 0 & 0.33 \end{bmatrix} \begin{bmatrix} 3 \\ 8 \\ 18 \end{bmatrix} = \begin{bmatrix} 8.94 \\ 13.94 \\ 5.94 \end{bmatrix} = \begin{bmatrix} s_1 \\ s_2 \\ x_1 \end{bmatrix}$$

Current basic feasible solution:

$$x = (x_1, x_2, s_1, s_2, s_3) = (5.94, 0, 8.94, 13.94, 0)$$

$$z = c_B^T x_B = [0 \quad 0 \quad 1] \begin{bmatrix} 8.94 \\ 13.94 \\ 5.94 \end{bmatrix} = 5.94$$



Step 7. Reduced costs and **entering variable**

$$\begin{aligned} \bar{c}_N^T &= c_B^T B^{-1} N - c_N^T \\ &= [0 \quad 0 \quad 1] \begin{bmatrix} 1 & 0 & 0.33 \\ 0 & 1 & 0.33 \\ 0 & 0 & 0.33 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 0 & 2 \\ 1 & 1 \end{bmatrix} - [0 \quad 3] \\ &= [0.33 \quad -2.67] \end{aligned}$$

Since $\bar{c}_2 = -2.67 < 0$, the **entering variable** is x_2

Step 8. Direction, ratio test and **leaving variable**

For $x_B = (s_1, s_2, x_1)$, we have:

$$d = B^{-1} N_2 = \begin{bmatrix} 1 & 0 & 0.33 \\ 0 & 1 & 0.33 \\ 0 & 0 & 0.33 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 1.33 \\ 2.33 \\ 0.33 \end{bmatrix}$$

Current basic solution:

$$x_B = B^{-1}b = \begin{bmatrix} 8.94 \\ 13.94 \\ 5.94 \end{bmatrix}$$

The minimum ratios test:

$$\frac{s_1}{1} = \frac{8.94}{1.33} = 6.75, \qquad \frac{s_2}{2} = \frac{13.94}{2.33} = 6, \qquad \frac{x_1}{1} = \frac{5.94}{0.33} = 18$$

Smallest ratio is $\frac{13.94}{2} \Rightarrow s_2$ **leaves the basis** (only consider ratios where the corresponding $d_i > 0$)

Step 9. Pivot to the new basis

$$B = (s_1, x_2, x_1) \qquad N = (s_2, s_3)$$

$$B = \begin{bmatrix} 1 & 1 & -1 \\ 0 & 2 & -1 \\ 0 & 1 & 3 \end{bmatrix} \qquad N = \begin{bmatrix} 0 & 0 \\ 1 & 1 \\ 0 & 1 \end{bmatrix}$$

$$c_B^T = [0 \quad 3 \quad 1] \qquad c_N^T = [0 \quad 0]$$

Compute B^{-1}:

$$B^{-1} = \begin{bmatrix} 1 & -0.57 & 0.14 \\ 0 & 0.43 & 0.14 \\ 0 & -0.14 & 0.29 \end{bmatrix}$$

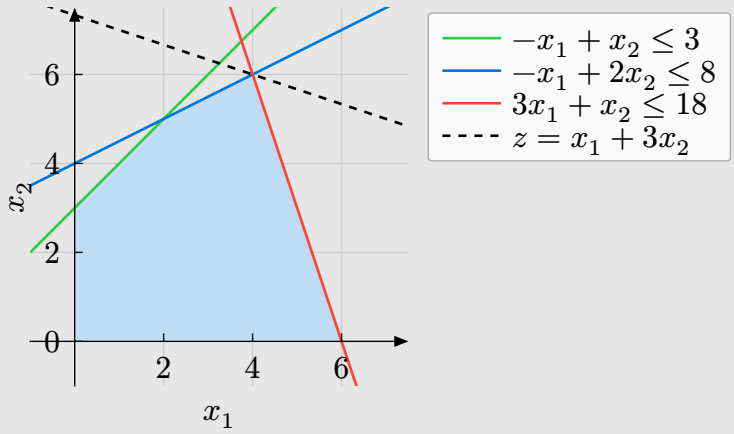
Compute basic feasible solution:

$$x_B = B^{-1}b = \begin{bmatrix} 1 & -0.57 & 0.14 \\ 0 & 0.43 & 0.14 \\ 0 & -0.14 & 0.29 \end{bmatrix} \begin{bmatrix} 3 \\ 8 \\ 18 \end{bmatrix} = \begin{bmatrix} 0.96 \\ 5.96 \\ 4.1 \end{bmatrix} = \begin{bmatrix} s_1 \\ x_2 \\ x_1 \end{bmatrix}$$

Current basic feasible solution:

$$x = (x_1, x_2, s_1, s_2, s_3) = (4, 6, 1, 0, 0)$$

$$z = c_B^T x_B = [0 \quad 3 \quad 1] \begin{bmatrix} 0.96 \\ 5.96 \\ 4.1 \end{bmatrix} = 22$$



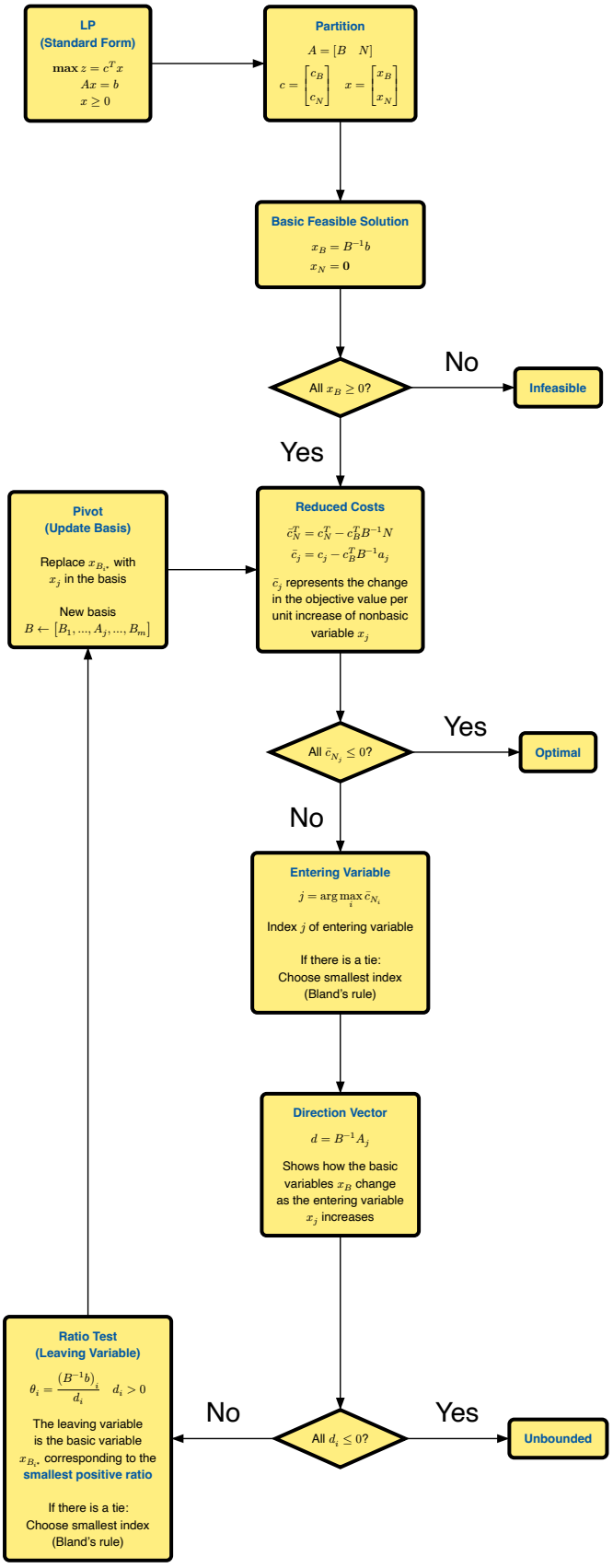
```python
Nv = np.array(['x1', 'x2'])
Bv = np.array(['s1', 's2', 's3'])

A = np.array([
    [-1, 1, 1, 0, 0],
    [-1, 2, 0, 1, 0],
    [3, 1, 0, 0, 1],
])

c = np.array([1, 3, 0, 0, 0])

b = np.array([3, 8, 18])

cN = c[:2]
cB = c[2:]

N = A[:,:2]
B = A[:,2:]

print("Nv"), print(Nv)
print()
print("N"), print(N)
print()
print("cN"), print(cN)
print()
print("Bv"), print(Bv)
print()
print("B"), print(B)
print()
print("cB"), print(cB)
print()
print("b"), print(b)
print()

Binv = np.linalg.inv(B)
bfs = Binv @ b
z = cB @ bfs

print("B^(-1)"), print(Binv)
print()
print("BFS"), print(bfs)
print()
```

```
print("z"), print(z)
print()
```

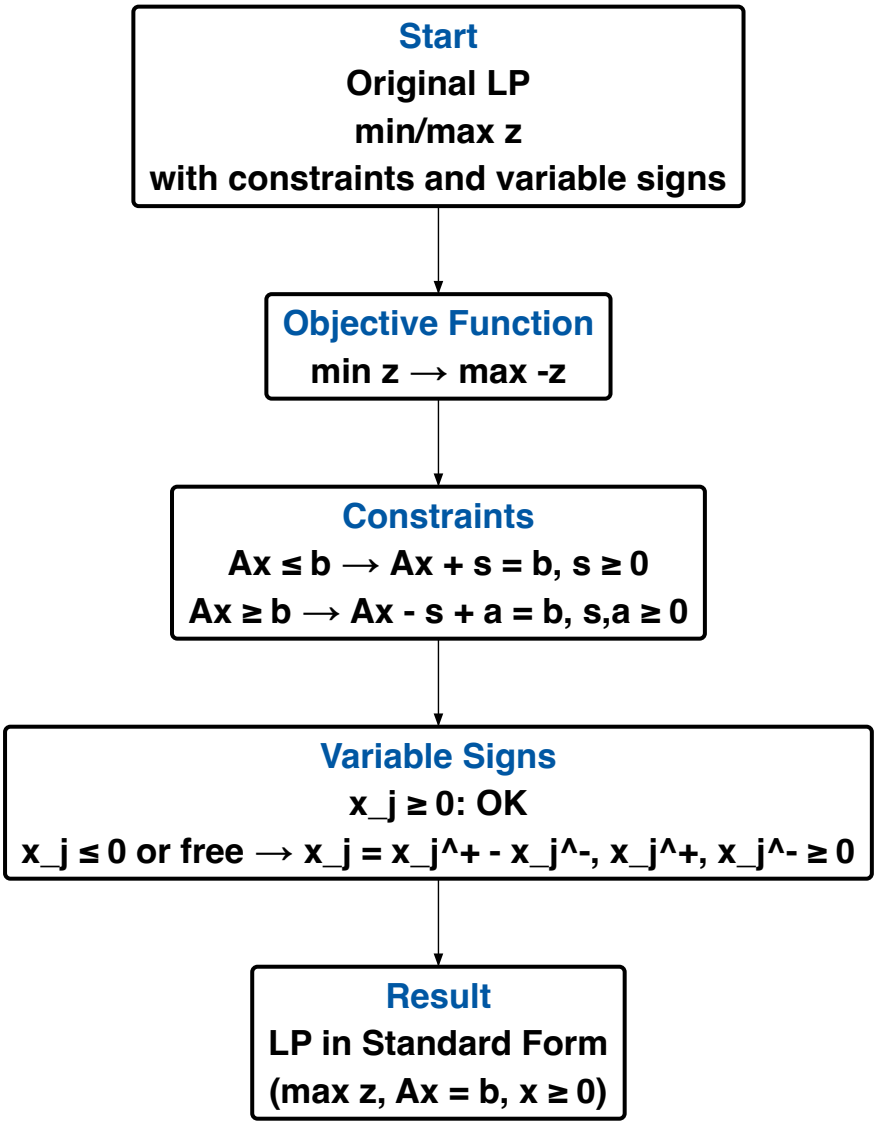
```
reduced_cost = cB @ Binv @ N - cN

print("bar(c)_N^T"), print(reduced_cost)
print()

entering_var_index = 0

print("Entering var"), print(Nv[entering_var_index])
print()

d = Binv @ N[:,entering_var_index]
ratios = bfs / d

print(f"Ratios"), print(ratios)
print()

exiting_var_index = 2

print(f"Exiting variable"), print(Bv[exiting_var_index])
print()

B[:, exiting_var_index], N[:, entering_var_index] = N[:,
entering_var_index].copy(), B[:, exiting_var_index].copy()
cB[exiting_var_index], cN[entering_var_index] =
cN[entering_var_index].copy(), cB[exiting_var_index].copy()
Bv[exiting_var_index], Nv[entering_var_index] =
Nv[entering_var_index].copy(), Bv[exiting_var_index].copy()

print("Nv"), print(Nv)
print()
print("N"), print(N)
print()
print("cN"), print(cN)
print()
print("Bv"), print(Bv)
print()
print("B"), print(B)
print()
print("cB"), print(cB)
print()

Binv = np.linalg.inv(B)
bfs = Binv @ b
z = cB @ bfs

print("B^(-1)"), print(Binv)
print()
print("BFS"), print(bfs)
print()
print("z"), print(z)
print()

```

```
reduced_cost = cB @ Binv @ N - cN

print("bar(c)_N^T"), print(reduced_cost)
print()

entering_var_index = 1

print("Entering var"), print(Nv[entering_var_index])
print()

d = Binv @ N[:,entering_var_index]
ratios = bfs / d

print(f"Ratios"), print(ratios)
print()

exiting_var_index = 1

print(f"Exiting variable"), print(Bv[exiting_var_index])
print()

B[:, exiting_var_index], N[:, entering_var_index] = N[:,
entering_var_index].copy(), B[:, exiting_var_index].copy()
cB[exiting_var_index], cN[entering_var_index] =
cN[entering_var_index].copy(), cB[exiting_var_index].copy()
Bv[exiting_var_index], Nv[entering_var_index] =
Nv[entering_var_index].copy(), Bv[exiting_var_index].copy()

print("Nv"), print(Nv)
print()
print("N"), print(N)
print()
print("cN"), print(cN)
print()
print("Bv"), print(Bv)
print()
print("B"), print(B)
print()
print("cB"), print(cB)
print()

Binv = np.linalg.inv(B)
bfs = Binv @ b
z = cB @ bfs

print("B^(-1)"), print(Binv)
print()
print("BFS"), print(bfs)
print()
print("z"), print(z)
print()
```

181.6. Simplex Cheatsheet



1. Convert to standard form



$$\begin{cases} z = \min c^T x \\ z = \max c^T x \end{cases} \quad \begin{array}{l} Ax = b \\ Ax \le b \\ Ax \ge b \\ x \ge 0 \\ x \le 0 \\ x \text{ free} \end{array} \quad \rightarrow \quad \begin{array}{l} z = \max c^T x \\ Ax = b \\ x \ge 0 \end{array}$$

Objective Function

$$\min z \quad \rightarrow \quad \max -(z)$$

Constraints

Slack $Ax \le b \quad \rightarrow \quad \begin{array}{l} Ax + s = b \\ s \ge 0 \end{array}$

Surplus $Ax \ge b \quad \rightarrow \quad \begin{array}{l} Ax - s + a = b \\ s, a \ge 0 \end{array}$

Negative/Free Variables

$$\begin{array}{l} x_j \le 0 \\ x_j \text{ free} \end{array} \quad \rightarrow \quad \begin{array}{l} x_j = x_j^+ - x_j^- \\ x_j^+, x_j^- \ge 0 \end{array}$$

2. Partition

$$A = [B \quad N] \qquad x = \begin{bmatrix} x_B \\ x_N \end{bmatrix} \qquad c = \begin{bmatrix} c_B \\ c_N \end{bmatrix}$$

Constraints:

$$Bx_b + Nx_n = b$$

3. Basic Feasible Solution

$$x_B = B^{-1}b$$

4. Reduced Costs

$$\bar{x}_N = c_N^T - c_B^T B^{-1} N$$

- If $\bar{c}_j > 0$: x_j can enter the basis
- If $\bar{c}_j \leq 0$: optimal

5. Choose Entering Variable

Select $x_j \in N$ with the largest $\bar{c}_j$.

If there's a tie, pick the smallest index (Bland's rule).

6. Pivot / Ratio Test

Let $d = B^{-1}A_j$ be the column of the entering variable expressed in basic coordinates Only $d_i > 0$ maintain feasibility

$\theta_i = \frac{(B^{-1}b)_i}{d_i}, \quad d_i > 0$

- The smallest θ_i identifies the leaving variable x_{B_i}.
- If all $d_i \leq 0$: the LP is unbounded

181.7. Input / Output

181.7.1. Input Output Model (Leontief)

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix} \qquad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \qquad \mathbf{d} = \begin{bmatrix} d_1 \\ d_2 \\ \vdots \\ d_n \end{bmatrix}$$

Where:

- a_{ij}: units of good i needed to produce 1 unit of good j
- $\mathbf{x}$: total outputs for each sector
- $\mathbf{d}$: vector of goods used **externally by consumers**
- $\mathbf{A}\ \mathbf{x}$: vector of goods used **internally in production**

Technical coefficient: the amount of input required per unit of output

Balance Equation

Total production = production used as inputs for other sectors+final consumption

$$\mathbf{x} = \mathbf{A}\ \mathbf{x} + \mathbf{d}$$

Rewrite the Equation

1. Subtract $\mathbf{A}\ \mathbf{x}$ from both sides:

$$\mathbf{x} - \mathbf{A}\ \mathbf{x} = \mathbf{d}$$

2. Factor out $\mathbf{x}$:

$$\mathbf{x}\ (\mathbf{I} - \mathbf{A}) = \mathbf{d}$$

Given final demand $\mathbf{d}$ and input requirements $\mathbf{A}$, what total production $\mathbf{x}$ is needed?

2. Solving for Production

$$\mathbf{x} = (\mathbf{I} - \mathbf{A})^{-1}\ \mathbf{d}$$

This gives exactly how much each sector must produce to satisfy both internal (inputs) and external (final demand) needs.

$(I - A)^{-1}$: Leontief inverse, each element shows the total output needed from sector i per unit of final demand in sector j, accounting for indirect effects

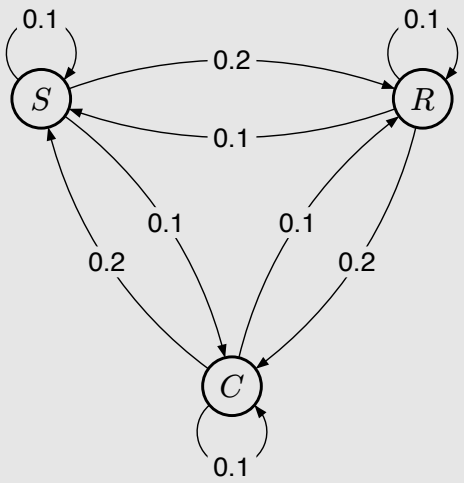
Example

- Steel (S)
- Rubber (R)
- Chemicals (C)

O / I	Steel	Rubber	Chemicals
Steel	0.1	0.2	0.1
Rubber	0.1	0.1	0.2
Chemicals	0.2	0.1	0.1

Resource	Demand
Steel	10
Rubber	20
Chemicals	30

$$\mathbf{A} = \begin{bmatrix} 0.1 & 0.2 & 0.1 \\ 0.1 & 0.1 & 0.2 \\ 0.2 & 0.1 & 0.1 \end{bmatrix} \qquad \mathbf{d} = \begin{bmatrix} 10 \\ 20 \\ 30 \end{bmatrix}$$



$$\mathbf{x} = (\mathbf{I} - \mathbf{A})^{-1}\ \mathbf{d}$$

$$\mathbf{x} = \left(\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} - \begin{bmatrix} 0.1 & 0.2 & 0.1 \\ 0.1 & 0.1 & 0.2 \\ 0.2 & 0.1 & 0.1 \end{bmatrix} \right)^{-1} \begin{bmatrix} 10 \\ 20 \\ 30 \end{bmatrix}$$

$$\mathbf{x} = \begin{bmatrix} 23.42 \\ 34.23 \\ 42.34 \end{bmatrix}$$

- Steel ($\boldsymbol{x_1 = 23.42}$): **total units** needed for final demand and as inputs to other sectors
- Rubber ($\boldsymbol{x_2 = 34.23}$): **total units** for final demand plus inputs to all sectors
- Chemicals ($\boldsymbol{x_2 = 42.34}$): **total units** including intermediate uses

181.7.1.1. Labor

1. Direct Labor

Each sector requires some amount of direct labor to produce a unit of output. Direct labor coefficient:

- l_k: hours of labor needed to produce **one unit of good** j

2. Indirect Labor

Producing goods often requires inputs from other sectors, which themselves required labor. To account for this:

$$\boldsymbol{\lambda} = \mathbf{l}(\mathbf{I} - \mathbf{A})^{-1}$$

- $\mathbf{l}$: vector of direct labor coefficients
- $\mathbf{A}$: input-output matrix
- $\boldsymbol{\lambda}$: **total (direct and indirect) labor required per unit** of each good

Interpretation: $\lambda_j$is the total socially necessary labor time embedded in one unit of good j

3. Labor as a “Currency”

- People earn labor vouchers corresponding to the hours they work.
- Each good has a price in labor time: λ_j hours per unit.
- If you work h hours, you get h labor vouchers to spend.

Budget constraint for a worker:

$$\sum_j \lambda_j c_j \leq h$$

Where:

- c_j: units of good j the worker wants to consume
- λ_j: labor time required to produce one unit of good j (includes direct and indirect) labor
- h: total labor hours the worker has contributed (i.e., the number of labor vouchers they possess)

Example

Suppose:

- Good 1: $\lambda_1 = 2 \text{ hours/unit}$
- Good 2: $\lambda_1 = 5 \text{ hours/unit}$
- Worker labor: $h = 20 \text{ hours}$

Constraint:

$$2c_1 + 5c_2 \leq 20$$

- If the worker wants 4 units of Good 1: $2 \times 4 = 8 \text{ hours}$
- Then they could afford at most $(20 - 8) \div 5 = 2.4 \text{ units}$ of Good 2

So their consumption plan is limited by their labor contribution

4. Labor Planning

Labor also acts as a constraint for the whole economy:

$$\mathbf{I}\,\mathbf{x} \leq L$$

Where:

- $\mathbf{x}$: vector of production
- L: total labor available in society
- This ensures that planned production doesn't require more labor than society can provide

Example

1. Stepup

Suppose the economy produces Bread (B) and Clothes (C). Let's define:

- $\mathbf{x} = (x_B, x_C)$: total production of each good
- $\mathbf{d} = (d_B, d_C)$: final demand (how much society wants to consume)

- $\mathbf{A} = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.1 \end{bmatrix}$: input-output matrix
- $\mathbf{l} = (2, 5)$: direct labor per unit (hours) for Bread and Clothes
- $L = 100$: total labor available in society

2. Constraints

Input-Output Balance: production must satisfy both internal (inputs) and final demand

$$x_B \geq 0.1x_B + 0.2x_C + d_B$$
$$x_C \geq 0.3x_B + 0.1x_C + d_C$$

Labor Constraint: total labor used cannot exceed available labor

$$2x_B + 5x_C \leq 100$$

Non-negativity: you can't produce negative amounts

$$x_B \geq 0 \qquad x_C \geq 0$$

3. Objective: Maximize production

$$\max \quad w_B d_B + w_C d_C$$

Where:

- w_B, w_C: societal priority weights (e.g., bread might be more essential than clothes)
- d_B, d_C: quantities actually supplied to meet demand

Interpretation

- The LP decides how much of each product to product (x_B, x_C)
- It respects labor limits ($2x_B + 5x_C \leq 100$)
- It respects interdependencies (inputs for other products) via the matrix A
- It maximizes fulfillment of product demand, possibly weighted by social priorities

So, no abstract utility function needed — the LP is about physical goods, labor, and inputs

181.8. Integer Programming

181.8.1. IP (Integer Programming)

181.8.2. Selection and Logical Relations on Binary Variables $x_i \in \{0, 1\}$

181.8.2.1. Cardinality Constraints

Given a subset $S \subseteq \{1, ..., n\}$:

- At least k of the items in S

$$\sum_{i \in S} x_i \geq k$$

Example

We want to select at least 2 of items 1, 2, 3

$$x_1 + x_2 + x_3 \geq 2$$

- At most k of the items in S

$$\sum_{i \in S} x_i \leq k$$

Example

We want to select at most 2 of items 1, 2, 3

$$x_1 + x_2 + x_3 \leq 2$$

- Exactly k of the items in S

$$\sum_{i \in S} x_i = k$$

Example

We want to select exactly 2 of items 1, 2, 3

$$x_1 + x_2 + x_3 = 2$$

181.8.2.2. Logical OR

Given a subset $S \subseteq \{1, ..., n\}$:

- At least 1 item in S

$$\sum_{i \in S} x_i \geq 1$$

Example

Select item 1 or item 2, or both

$$x_1 + x_2 \geq 1$$

181.8.2.3. Conditional Selection: "If X, then Y"

(Implication Constraints)

- If $x_a = 1$, then $x_b = 1$

$$x_a \leq x_b$$

Example

If item 1 is selected, then item 2 must also be selected

$$x_1 \leq x_2$$

x_a	x_b	Valid?
0	0	✅
0	1	✅
1	0	✅
1	1	❌

- If $x_a = 1$, then $x_b = 0$

$$x_a + x_b \leq 1$$

Example

If you choose project A, then you cannot choose project B (mutually exclusive).

x_a	x_b	Valid?
0	0	✅
0	1	✅
1	0	✅
1	1	❌

- If $x_a = 1$, then $x_b + x_c \leq 0$ (generalized)

$$x_b + x_c \leq M(1 - x_a)$$

Example

For binary variables $M = 2$ works

If you choose project A, then you must not choose either project B or C.

x_a	x_b	x_c	Valid?
1	0	0	✅
1	1	0	❌
1	1	1	❌
0	1	1	✅

0	0	1	✅

181.8.2.4. Conditional OR: “If not X, then select Y and Z”

- If $x_j = 0$, then at least k items in S must be selected (generalized)

$$\sum_{i \in S} x_i \geq k(1 - x_j)$$

Example

If you do not choose item 1, then you must choose both item 2 and item

If $x_1 = 0$, then $x_2 = x_3 = 1$

$$x_2 + x_3 \geq 2(1 - x_1)$$

x_1	x_2	x_3	LHS	RHS	Valid?
0	1	1	2	2	✅
0	1	0	1	2	❌
0	0	0	0	2	❌
1	0	0	0	0	✅
1	1	0	1	0	✅
1	1	1	2	0	✅

181.8.2.5. XOR / Exclusive Conditions

- Exactly one item in a set S

$$\sum_{i \in S} x_i = 1$$

- Select one or the other but not both (i.e., $x_i \oplus x_k$)

$$x_a + x_b = 1$$

Example

You can select item 1 or item 2, but not both

x_1	x_2	Sum	Valid?
0	1	1	✅
1	0	1	✅
1	1	2	❌

0	0	0	✗

181.8.3. Constraint Activation and Relaxation via Binary Indicators

$z_i \in \{1, 0\}$

181.8.3.1. Base Case: At Least One of Two Constraints Satisfied

You are given 2 constraints

$$g_1(x) \leq b_1$$
$$g_2(x) \leq b_2$$

We want at least one of these two constraints to be satisfied.

Binary Indicator Variables

Define:

$$z_i = \begin{cases} 1 & \text{if } g_{i(x)} \leq b_i \text{ is enforced} \\ 0 & \text{if } g_{i(x)} \leq b_i \text{ may be violated} \end{cases} \quad \text{for } i = 1, 2$$

Let M_1 and M_2 be upper bounds on how much each constraint's LHS $g_{i(x)}$ can exceed b_i:

$$M_i \geq \max_x \left(g_{i(x)} - b_i \right)$$

Big-M Reformulation

We relax each constraint as follows:

$$g_1(x) - b_1 \leq M_1(1 - z_1)$$
$$g_2(x) - b_2 \leq M_2(1 - z_2)$$

When $z_i = 1$, the original constraint is enforced

When $z_i = 0$, the original constraint can be violated by up to M_i

At Least One Constraint Enforced

We require:

$$z_1 + z_2 \geq 1$$

181.8.3.2. General Formulation for m Constraints with At Least k Satisfied

1. Introduce a binary variable $z_i \in \{0, 1\}$ for each constraint $g_{i(x)} \leq b_i$, where:
   - $z_i = 1$: the constraint is enforced (i.e., $g_{i(x)} \leq b_i$)
   - $z_i = 0$: the constraint may be violated
2. Use "big-M" constraint to relax $g_{i(x)} \leq b_i$ when $z_i = 0$, like:

$$g_{i(x)} - b_i \leq M_i(1 - z_i), \quad \forall i = 1, ..., m$$

Where $M_i \geq \max_x \left(g_{i(x)} - b_i \right)$

3. Impose a condition that at least k of the m constraints must be satisfied:

$$\sum_{i=1}^{m} z_i \geq k$$

181.8.4. Fixed Charge Constraints (Setup Costs)

- n: Number of factories
- K_i: Capacity of factory i
- C_i: Unit production cost at factory i
- S_i: Fixed setup cost for factory i
- D: Total Demand

Decision Variables

- x_i: production quantity at factory i, $i = 1, ..., n$
- y_i: Binary variable indicating whether factory i is used

$$y_i = \begin{cases} 1 & \text{if factory } i \text{ is used (i.e., } x_i > 0\ ) \\ 0 & \text{if factory } i \text{ is not used (i.e, } x_i = 0\ ) \end{cases}$$

Objective Function

$$\min \underbrace{\sum_{i=1}^{n} C_i x_i}_{\text{Production Cost}} + \underbrace{\sum_{i=1}^{n} S_i y_i}_{\text{Setup Cost}}$$

Capacity Constraints

$$x_i \leq K_i \quad \forall i = 1, ..., n$$

Demand Fullfilment

$$\sum_{i=1}^{n} x_i \geq D$$

Logical Linking of x_i and y_i

To ensure $y_i = 1$ if any production occurs at factory i:

$$x_i \leq K_i y_i \quad \forall i = 1, ..., n$$

- If $x_i > 0$, this forces $y_i = 1$
- If $x_i = 0$, y_i can be 0 or 1 (but to minimize cost, y_i will be 0)

Binary & Non-negative Constraints

$$y_i \in \{0, 1\}, \quad x_i \geq 0 \quad \forall i = 1, ..., n$$

General Form (Big-M Formulation)

$$x \leq My$$

Where M must be set to the upper bound of x

181.8.5. Facility Location

Problem	Definition
Set Covering	Select the minimum number of facilities such that every demand point is within a specified coverage distance of at least one facility
Maximum Covering	Select a fixed number of facilities to maximize the number of demand points covered within a specified distance
Fixed Charge Location	Determine which facilities to open and how to assign demand to them to minimize the total cost

181.8.5.1. Set Covering Problem

Question: How to allocate as few facilities as possible to cover all demand nodes?

- I: Set of demand nodes
- J: Set of location nodes
- $d_{ij} > 0$: Distance between demand node $i \in I$ and location node $j \in J$
- s: Maximum allowable distance (a facility at j can cover demand node i if $d_{ij} < s$)
- Demand i is "covered" by location j if $d_{ij} < s$
- $w_j > 0$: Cost of building facility at location j (usually set to $w_j = 1$ for minimizing number of facilities, but can reflect other costs)
- $a_{ij} = \begin{cases} 1 & \text{if } d_{ij} < s \\ 0 & \text{otherwise} \end{cases}$: Indicator of whether location j can cover demand i
- $x_j = \begin{cases} 1 & \text{if facility is built at location } j \\ 0 & \text{otherwise} \end{cases}$

Complete Formulation

$$\begin{aligned} \min \quad & \sum_{j \in J} w_j x_j \\ s.t. \quad & \sum_{j \in J} a_{ij} x_j \geq 1 \qquad \forall i \in I \\ & x_j \in \{0, 1\} \qquad \forall j \in J \end{aligned}$$

181.8.5.2. Maximum Covering Problem

Question: How to allocate at most p facilities to cover as many demand nodes as possible?

- I: Set of demand nodes
- J: Set of location nodes
- $d_{ij} > 0$: Distance between demand node $i \in I$ and location node $j \in J$
- $p \in \mathbb{N}$: maximum number of facilities
- $w_j > 0$: Cost of building facility at location j (usually set to $w_j = 1$ for minimizing number of facilities, but can reflect other costs)

- $x_j = \begin{cases} 1 & \text{if facility is built at location } j \\ 0 & \text{otherwise} \end{cases}$
- $a_{ij} = \begin{cases} 1 & \text{if } d_{ij} < s \\ 0 & \text{otherwise} \end{cases}$: Indicator of whether location j can cover demand i
- $y_i = \begin{cases} 1 & \text{if demand } i \in I \text{ is covered by any facility} \\ 0 & \text{otherwise} \end{cases}$

Complete Formulation

$$
\begin{aligned}
\min \quad & \sum_{i \in I} w_i y_i \\
s.t. \quad & \sum_{j \in J} a_{ij} x_j \geq y_i \quad \forall i \in I \\
& \sum_{j \in J} x_j \leq p \quad \forall j \in J \\
& x_j \in \{0,1\} \quad \forall j \in J \\
& y_i \in \{0,1\} \quad \forall i \in I
\end{aligned}
$$

Fixed Charge Location Problems

Question: How to allocate some facilities to minimize the total shipping and construction costs?

Uncapacitated Facility Location Problem (UFL)

- I: Set of demand nodes
- J: Set of location nodes
- $h_i > 0$: Demand size at demand node i
- d_{ij}: Unit shipping cost from location node j to demand node i
- $f_j > 0$: Fixed construction cost at location j
- $x_j = \begin{cases} 1 & \text{if facility is built at location } j \\ 0 & \text{otherwise} \end{cases}$
- $y_{ij} = \begin{cases} 1 & \text{if demand } i \in I \text{ is served by facility at location } j \in J \\ 0 & \text{otherwise} \end{cases}$

Complete Formulation

$$
\begin{aligned}
\min \quad & \underbrace{\sum_{i \in I} \sum_{j \in J} h_i d_{ij} y_{ij}}_{\text{Shipping Costs}} + \underbrace{\sum_{j \in J} f_j x_j}_{\text{Fixed Costs}} \\
s.t. \quad & y_{ij} \leq x_j \quad \forall i \in I, j \in J \\
& \sum_{j \in J} y_{ij} = 1 \quad \forall i \in I \\
& x_j \in \{0,1\} \quad \forall j \in J \\
& y_{ij} \in \{0,1\} \quad \forall i \in I
\end{aligned}
$$

Capacitated Facility Location Problem (CFL)

If locations have a capacity, we may add constraint

$$\sum_{i \in I} h_i y_{ij} \leq K_j \quad \forall j \in J$$

Where:

- $K_j > 0$: Capacity of location j

Example

$$\min \quad \underbrace{\sum_{j=1}^{5} f_j x_j}_{\text{Fixed Costs}} + \underbrace{\sum_{i=1}^{5} \sum_{j=1}^{5} c_{ij} y_{ij}}_{\text{Shipping Costs}}$$

$$\begin{aligned}
s.t. \quad & \sum_{i=1}^{5} y_{ij} \leq K_j x_j && \forall j = 1, ..., 5 && \text{(Capacity Constraint)} \\
& \sum_{j=1}^{5} y_{ij} \geq D_i && \forall i = 1, ..., 5 && \text{(Demand Constraint)} \\
& x_j \in \{0, 1\} && \forall j = 1, ..., 5 && \text{(Binary Constraint)} \\
& y_{ij} \geq 0 && \forall i = 1, ..., 5, j = 1, ..., 5 && \text{(Positivity Constraint)}
\end{aligned}$$

Parameters

- f_j: weekly operating cost of distribution center j
- c_{ij}: shipping cost per book from distribution center j to region i
- K_j: capacity of distribution center j
- D_i: book demand of region i

Decision variable

- x_{ij}: binary variable

$$x_{ij} = \begin{cases} 1 & \text{if a distribution center is build at location } j \\ 0 & \text{otherwise} \end{cases}$$

- y_{ij}: number of books shipped from distribution center j to region i

Supply

	WA	NV	NE	PA	FL	Demand (D_i)
NorthWest	y_{11}	y_{12}	y_{13}	y_{14}	y_{15}	8000
SouthWest	y_{21}	y_{22}	y_{23}	y_{24}	y_{25}	12000
MidWest	y_{31}	y_{32}	y_{33}	y_{34}	y_{35}	9000
SouthEast	y_{41}	y_{42}	y_{43}	y_{44}	y_{45}	14000
NorthEast	y_{51}	y_{52}	y_{53}	y_{54}	y_{55}	17000

Fixed Costs (f_j) and Capacity (K_j)

	x_1	x_2	x_3	x_4	x_5
Operation Cost (f_i)	40000	30000	25000	40000	30000
Capacity (K_i)	20000	20000	15000	25000	15000

Shipping Cost (c_j)

	WA	NV	NE	PA	FL
NorthWest	2.4	3.25	4.05	5.25	6.95
SouthWest	3.5	2.3	3.25	6.05	5.85
MidWest	4.8	3.4	2.85	4.3	4.8
SouthEast	6.8	5.25	4.3	3.25	2.1
NorthEast	5.75	6	4.75	2.75	3.5

Binary Decision Variable

WA	x_1
NV	x_2
NE	x_3
PA	x_4
FL	x_5

model.py

model.py

```python
from pyomo.environ import *
from pyomo.dataportal import DataPortal

model = AbstractModel()

# Sets
model.I = Set(doc='Regions (demand points)')
model.J = Set(doc='Candidate distribution centers')

# Parameters
model.D = Param(model.I, within=NonNegativeReals, doc='Demand at region i')
model.K = Param(model.J, within=NonNegativeReals, doc='Capacity at distribution center j')
model.f = Param(model.J, within=NonNegativeReals, doc='Fixed cost to open center j')
model.c = Param(model.I, model.J, within=NonNegativeReals, doc='Shipping cost from j to i')

# Decision Variables
```

```
model.x = Var(model.J, within=Binary, doc='1 if center j is opened')
model.y = Var(model.I, model.J, within=NonNegativeReals, doc='Units shipped from j to i')

# Objective Function: Minimize total cost
def total_cost_rule(model):
    fixed = sum(model.f[j] * model.x[j] for j in model.J)
    shipping = sum(model.c[i, j] * model.y[i, j] for i in model.I for j in model.J)
    return fixed + shipping
model.TotalCost = Objective(rule=total_cost_rule, sense=minimize)

# Constraints

# Capacity Constraint: Total shipped from center ≤ capacity if opened
def capacity_rule(model, j):
    return sum(model.y[i, j] for i in model.I) <= model.K[j] * model.x[j]
model.CapacityConstraint = Constraint(model.J, rule=capacity_rule)

# Demand Constraint: Total received by region ≥ its demand
def demand_rule(model, i):
    return sum(model.y[i, j] for j in model.J) >= model.D[i]
model.DemandConstraint = Constraint(model.I, rule=demand_rule)

# Load data from .dat file
data = DataPortal()
data.load(filename='data.dat', model=model)

# Create an instance of the model
instance = model.create_instance(data)

# Create solver
solver = SolverFactory('glpk')
solver.options['tmlim'] = 60

# Solve with solver timeout (optional)
results = solver.solve(instance, tee=True)

# Display results
instance.display()
```

data.dat

```
set I := NorthWest SouthWest MidWest SouthEast NorthEast;
set J := WA NV NE PA FL;

param: D :=
NorthWest 8000
SouthWest 12000
MidWest 9000
SouthEast 14000
```

```
NorthEast 17000;

param: f :=
WA 40000
NV 30000
NE 25000
PA 40000
FL 30000;

param: K :=
WA 20000
NV 20000
NE 15000
PA 25000
FL 15000;

param c:
        WA     NV     NE     PA     FL :=
NorthWest 2.4   3.25   4.05   5.25   6.95
SouthWest 3.5   2.3    3.25   6.05   5.85
MidWest   4.8   3.4    2.85   4.3    4.8
SouthEast 6.8   5.25   4.3    3.25   2.1
NorthEast 5.75  6      4.75   2.75   3.5 ;

```

Output:

```
Variables:
x : 1 if center j is opened
    Size=5, Index=J
    Key : Lower : Value : Upper : Fixed : Stale : Domain
     FL :     0 :   1.0 :     1 : False : False : Binary
     NE :     0 :   0.0 :     1 : False : False : Binary
     NV :     0 :   1.0 :     1 : False : False : Binary
     PA :     0 :   1.0 :     1 : False : False : Binary
     WA :     0 :   0.0 :     1 : False : False : Binary
y : Units shipped from j to i
    Size=25, Index=I*J
    Key                : Lower : Value   : Upper : Fixed : Stale :
Domain
      ('MidWest', 'FL') :     0 :  1000.0 :  None : False : False :
NonNegativeReals
      ('MidWest', 'NE') :     0 :     0.0 :  None : False : False :
NonNegativeReals
      ('MidWest', 'NV') :     0 :     0.0 :  None : False : False :
NonNegativeReals
      ('MidWest', 'PA') :     0 :  8000.0 :  None : False : False :
NonNegativeReals
      ('MidWest', 'WA') :     0 :     0.0 :  None : False : False :
NonNegativeReals
```

```
    ('NorthEast', 'FL') :     0 :     0.0 :  None : False : False :
NonNegativeReals
    ('NorthEast', 'NE') :     0 :     0.0 :  None : False : False :
NonNegativeReals
    ('NorthEast', 'NV') :     0 :     0.0 :  None : False : False :
NonNegativeReals
    ('NorthEast', 'PA') :     0 : 17000.0 :  None : False : False :
NonNegativeReals
    ('NorthEast', 'WA') :     0 :     0.0 :  None : False : False :
NonNegativeReals
    ('NorthWest', 'FL') :     0 :     0.0 :  None : False : False :
NonNegativeReals
    ('NorthWest', 'NE') :     0 :     0.0 :  None : False : False :
NonNegativeReals
    ('NorthWest', 'NV') :     0 :  8000.0 :  None : False : False :
NonNegativeReals
    ('NorthWest', 'PA') :     0 :     0.0 :  None : False : False :
NonNegativeReals
    ('NorthWest', 'WA') :     0 :     0.0 :  None : False : False :
NonNegativeReals
    ('SouthEast', 'FL') :     0 : 14000.0 :  None : False : False :
NonNegativeReals
    ('SouthEast', 'NE') :     0 :     0.0 :  None : False : False :
NonNegativeReals
    ('SouthEast', 'NV') :     0 :     0.0 :  None : False : False :
NonNegativeReals
    ('SouthEast', 'PA') :     0 :     0.0 :  None : False : False :
NonNegativeReals
    ('SouthEast', 'WA') :     0 :     0.0 :  None : False : False :
NonNegativeReals
    ('SouthWest', 'FL') :     0 :     0.0 :  None : False : False :
NonNegativeReals
    ('SouthWest', 'NE') :     0 :     0.0 :  None : False : False :
NonNegativeReals
    ('SouthWest', 'NV') :     0 : 12000.0 :  None : False : False :
NonNegativeReals
    ('SouthWest', 'PA') :     0 :     0.0 :  None : False : False :
NonNegativeReals
    ('SouthWest', 'WA') :     0 :     0.0 :  None : False : False :
NonNegativeReals

Objectives:
  TotalCost : Size=1, Index=None, Active=True
      Key  : Active : Value
      None :   True : 268950.0

Constraints:
  CapacityConstraint : Size=5
      Key : Lower : Body : Upper
      FL :  None :  0.0 :   0.0
      NE :  None :  0.0 :   0.0
      NV :  None :  0.0 :   0.0
      PA :  None :  0.0 :   0.0
      WA :  None :  0.0 :   0.0
```

```
DemandConstraint : Size=5
    Key       : Lower   : Body    : Upper
      MidWest :  9000.0 :  9000.0 :  None
    NorthEast : 17000.0 : 17000.0 :  None
    NorthWest :  8000.0 :  8000.0 :  None
    SouthEast : 14000.0 : 14000.0 :  None
    SouthWest : 12000.0 : 12000.0 :  None
```

181.8.6. Machine Scheduling

	Problem	Definition
One Machine	Single Machine Serial Production	Scheduling jobs one after another on a single machine
Multiple Machines	Multiple Parallel Machines	Scheduling jobs on two or more identical machines working in parallel; each job can be assigned to any machine
	Flow Shop	All jobs follow the same sequence of machines; each job visits every machine in the same order
	Job Shop	Each job has its own specific sequence of machines to follow

- Job Splitting
    - Non-premptive problems

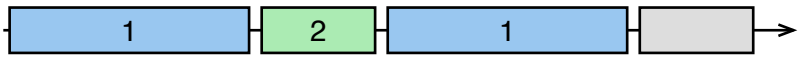
    Once a job starts processing on a machine, it must continue without interruption until completion

    ∧

    - Preemptive problems

    A job can be interrupted and resumed later, possibly on a different machine, without losing progress

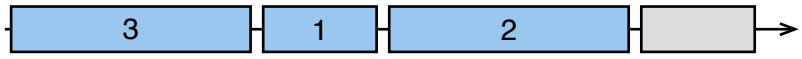
    ∧

- Performance Metrics
    - **Makespan**: The sum of completion times of all jobs, possibly weighted by job importance

- **(Weighted) total completion time**: The sum of completion times of all jobs, possibly weighted by job importance (measures overall throughput)
- **(Weighted) number of delayed jobs**: The total number of jobs completed after they are due, possibly weighted by job priority
- **(Weighted) total lateness**: The sum of differences between each job's completion time and due date (can be negative or positive), possibly weighted
- **(Weighted) total tardiness**: The sum of positive delays (i.e., lateness when a job finishes after its due date), possibly weighted. Negative values (early completions) are treated as zero.
- …

181.8.6.1. Single Machine Serial Production



Notation

- n: Total number of jobs
- $J = \{1, 2, ..., n\}$: Set of jobs
- $j \in J$: A job
- p_j: processing time of job j
- x_j: completion time of job j
- $z_{ij} \in \{0, 1\}$: Binary variable

$$z_{ij} = \begin{cases} 1 & \text{if job } j \text{ is before job } i \\ 0 & \text{otherwise} \end{cases}$$

Job Order Constraints

We must ensure that either job i comes before job j or vice versa, but not both

If job i is before job j ($z_{ij} = 0$)

$$x_j \geq x_i + p_j$$

If job j is before job i ($z_{ij} = 1$):

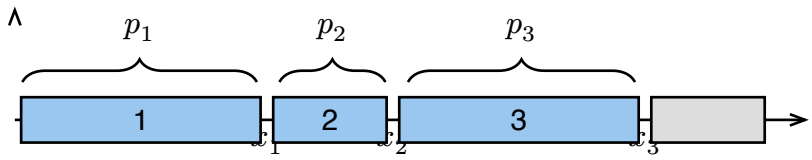
$$x_i \geq x_j + p_i$$

These two cases can be rewritten using a big-M formulation:

$$x_i + p_j - x_j \leq M z_{ij}$$
$$x_j + p_i - x_i \leq M(1 - z_{ij})$$

Where:

- $M = \sum_{j \in J} p_j$



Complete Formulation

$$
\begin{aligned}
\min \quad & \sum_{j \in J} x_j \\
s.t. \quad & x_i + p_j - x_j \leq M z_{ij} && \forall i \in J, j \in J, i < j \\
& x_j + p_i - x_i \leq M(1 - z_{ij}) && \forall i \in J, j \in J, i < j \\
& x_j \geq p_j && \forall j \in J \\
& x_j \geq 0 && \forall j \in J \\
& z_{ij} \in \{0, 1\} && \forall i \in J, j \in J, i < j
\end{aligned}
$$

Interpretation

If jobs are run in order $1, 2, ..., n$, the completion times are:

$$
\begin{aligned}
x_1 &= p_1 \\
x_2 &= p_1 + p_2 \\
x_3 &= p_1 + p_2 + p_3 \\
&\vdots \\
x_n &= \sum_{i=1}^{n} p_i
\end{aligned}
$$

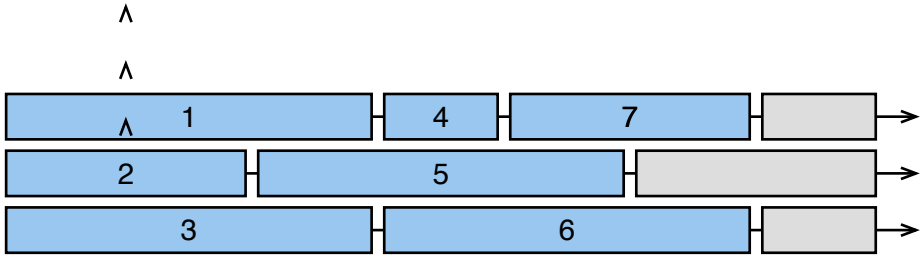
So, the completion time of a job depends on the sum of processing times of all jobs before it in the schedule

Optional: Release Times

If jobs have release times R_j, meaning each job j cannot start before R_j, then we add:

$$x_j \geq R_j + p_j \quad \forall j \in J$$

181.8.6.2. Multiple Parallel Machines



We aim to assign jobs to machines such that:

- Each job is assigned to exactly one machine.
- A job can be processed on any machine.
- The objective is to minimize the makespan, which is the time when the last machine finishes processing.

Notation

- $J = \{1, 2, ..., n\}$: set of jobs
- $I = \{1, 2, ..., m\}$: set of machines
- p_j: processing time of job $j \in J$
- x_{ij}: a binary variable that indicates whether job j is assigned to machine i:

$$x_{ij} = \begin{cases} 1 & \text{if job } j \text{ is assigned to machine } i \\ 0 & \text{otherwise} \end{cases}$$

- The completion time of machine i is the total processing time of all jobs assigned to it

$$C_i = \sum_{j \in J} p_j x_{ij}$$

- The makespan w is the maximum completion time across all machines:

$$w \geq \sum_{j \in J} p_j x_{ij} \quad \forall i \in I$$

Objective

Minimize makespan w

Complete Formulation

$$\begin{aligned} \min \quad & w \\ s.t. \quad & \sum_{j \in J} p_j x_{ij} \leq w \quad && \forall i \in I \\ & \sum_{i \in I} x_{ij} = 1 && \forall j \in J \\ & x_{ij} \in \{0, 1\} && \forall i \in I, j \in J \end{aligned}$$

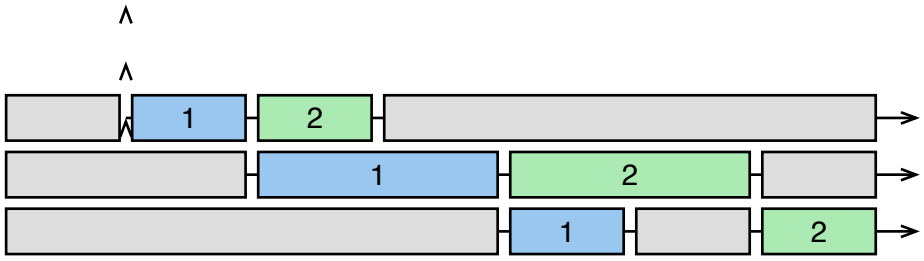
- The first constraint ensures the makespan is at least as large as each machine's workload
- The second ensures each job is assigned to one and only one machine

model.py

```python
from pyomo.environ import *

model = AbstractModel()

model.MACHINES = Set()
model.JOBS = Set()

model.p = Param(model.JOBS, within=PositiveReals)
```

```
9
10 model.x = Var(model.MACHINES, model.JOBS, domain=Binary)
11 model.w = Var(domain=NonNegativeReals)
12
13 def obj_rule(m):
14     return m.w
15 model.Obj = Objective(rule=obj_rule, sense=minimize)
16
17 def job_assignment_rule(m, j):
18     return sum(m.x[i, j] for i in m.MACHINES) == 1
19 model.JobAssignment = Constraint(model.JOBS, rule=job_assignment_rule)
20
21 def machine_completion_rule(m, i):
22     return sum(m.p[j] * m.x[i, j] for j in m.JOBS) <= m.w
23 model.MachineCompletion = Constraint(model.MACHINES,
   rule=machine_completion_rule)
```

181.8.6.3. Flow Shop



181.8.6.4. Job Shop

181.8.7. Vehicle Routing

181.8.7.1. Traveling Salesperson

- Let $G = (V, E)$ be a directed complete graph
- Let d_{ij} be the distance (cost) from node i to j
- Let x_{ij} be a binary decision variable:

$$x_{ij} = \begin{cases} 1 & \text{if } (i,j) \text{ is selected} \\ 0 & \text{otherwise} \end{cases}$$

Objective

Minimize the total travel cost

$$\min \sum_{(i,j) \in E} d_{ij} x_{ij}$$

Constraints

1. Flow Balancing

For node $k \in V$

- One **incoming** edge:

$$\sum_{i \in V, i \neq k} = 1$$

- One **outgoing** edge:

$$\sum_{j \in V, j \neq k} = 1$$

2. Subtours

a. Miller-Tucker-Zemlin (MTZ)

- Let $u_i s$: be the position of node i in the tour (e.g., $u_i = k$ if node i is the kth node to be passed on the tour)

$$\begin{aligned} & u_1 = 1 \\ & 2 \leq u_i \leq n && \forall i \in V \setminus \{1\} \\ & u_i - u_j + 1 \leq (n-1)\big(1 - x_{ij}\big) && \forall (i,j) \in E, i \neq 1, j \neq 1 \end{aligned}$$

b. Subtour Elimination Constraint (SEC)

For every subset $S \subset V$ with $|S| \geq 2$:

$$\sum_{\substack{i \in S, j \in S \\ i \neq j}} x_{ij} \leq |S| - 1 \quad \forall S \subset V, |S| \geq 2$$

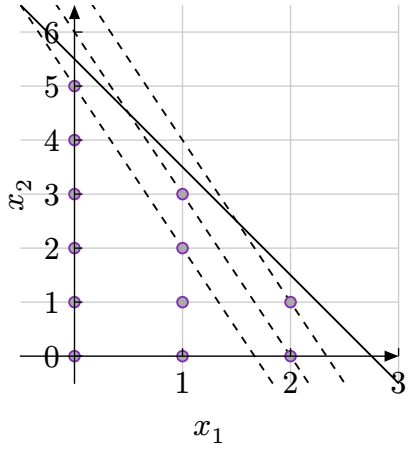
This ensures that no subset of nodes forms a closed tour independent of the main tour.

Complete Formulation

$$\begin{aligned} \min \quad & \sum_{(i,j) \in E} d_{ij} x_{ij} \\ s.t. \quad & \sum_{i \in V, i \neq k} = 1 && \forall k \in V \\ & \sum_{j \in V, j \neq k} = 1 && \forall k \in V \\ & x_{ij} \in \{0,1\} && \forall (i,j) \in E \\ & \text{MTZ or SEC} \end{aligned}$$

181.9. Branch and Bound

Integer Program



Linear Relaxation:

IP $\longrightarrow$ LP

Decompose an IP into mutltiple LPs

Definition 0: Linear Relaxation

For a Given LP, its linear relaxation is the resulting LP after removing all integer constraints

Example

Integer Problem:

$$\begin{aligned} \max \quad & 3x_1 + x_2 \\ s.t. \quad & 4x_1 + 2x_2 \leq 11 \\ & x_i \in \mathbb{Z}_+ \quad \forall i = 1, 2 \end{aligned}$$

Linear Relaxation:

$$\begin{aligned} \max \quad & 3x_1 + x_2 \\ s.t. \quad & 4x_1 + 2x_2 \leq 11 \\ & x_i \geq 0 \quad \forall i = 1, 2 \end{aligned}$$

Example

Integer Program:

$$\begin{aligned} \max \quad & 16x_1 + 22x_2 + 12x_3 + 8x_4 \\ s.t. \quad & 5x_1 + 7x_2 + 4x_3 + 3x_4 \leq 10 \\ & x_i \in \{0, 1\} \quad \forall i = 1, ..., 4 \end{aligned}$$

Linear Relaxation:

$$
\begin{array}{lrcrcrcrcl}
\max & 16x_1 & + & 22x_2 & + & 12x_3 & + & 8x_4 & & \\
s.t. & 5x_1 & + & 7x_2 & + & 4x_3 & + & 3x_4 & \leq & 10 \\
 & x_i \in [0,1] & & \forall i = 1, ..., 4 & & & & & &
\end{array}
$$

$$
x_i \in [0,1] \iff x_i \geq 0 \land x_i \leq 1
$$

Linear relaxation provides a bound

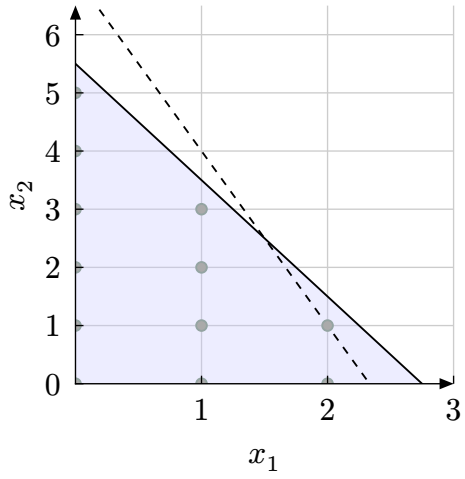
- z^*: Optimal value of the Integer Program (IP)
- z': Optimal value of the Linear Relaxation (LP)

1. Minimization

- For **minimization** IP, linear relaxation provides a **lower bound**
- The feasible region of the LP contains all feasible solutions of the IP (i.e., superset)
- Therefore, the LP can achieve a value that is at most as large as the IP
- Conclusion:

$$
z' \leq z^*
$$

So, linear relaxation provides a lower bound on the optimal value of the integer program

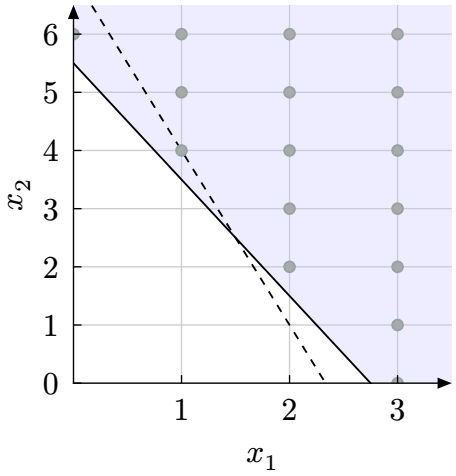


2. Maximization

- For **maximization** IP, linear relaxation provides a **upper bound**
- The LP may find a solution with an objective value greater than or equal to that of any feasible integer solution.
- Conclusion:

$$
z' \geq z^*
$$

- So, linear relaxation provides an upper bound on the optimal value of the integer program



Problem Type	Linear Relaxation Provides	Inequality Between LP and IP
Minimization IP	Lower Bound	$z' \leq z^*$
Maximization IP	Upper Bound	$z' \geq z^*$

- If the linear relaxation is infeasible or unbounded then the IP is infeasible or unbounded
- If an optimal solution to the linear relaxation is feasible,

Let x' be an optimal solutions to the linear relaxation of an IP. If x' is feasible to the IP, it is the optimal to the IP

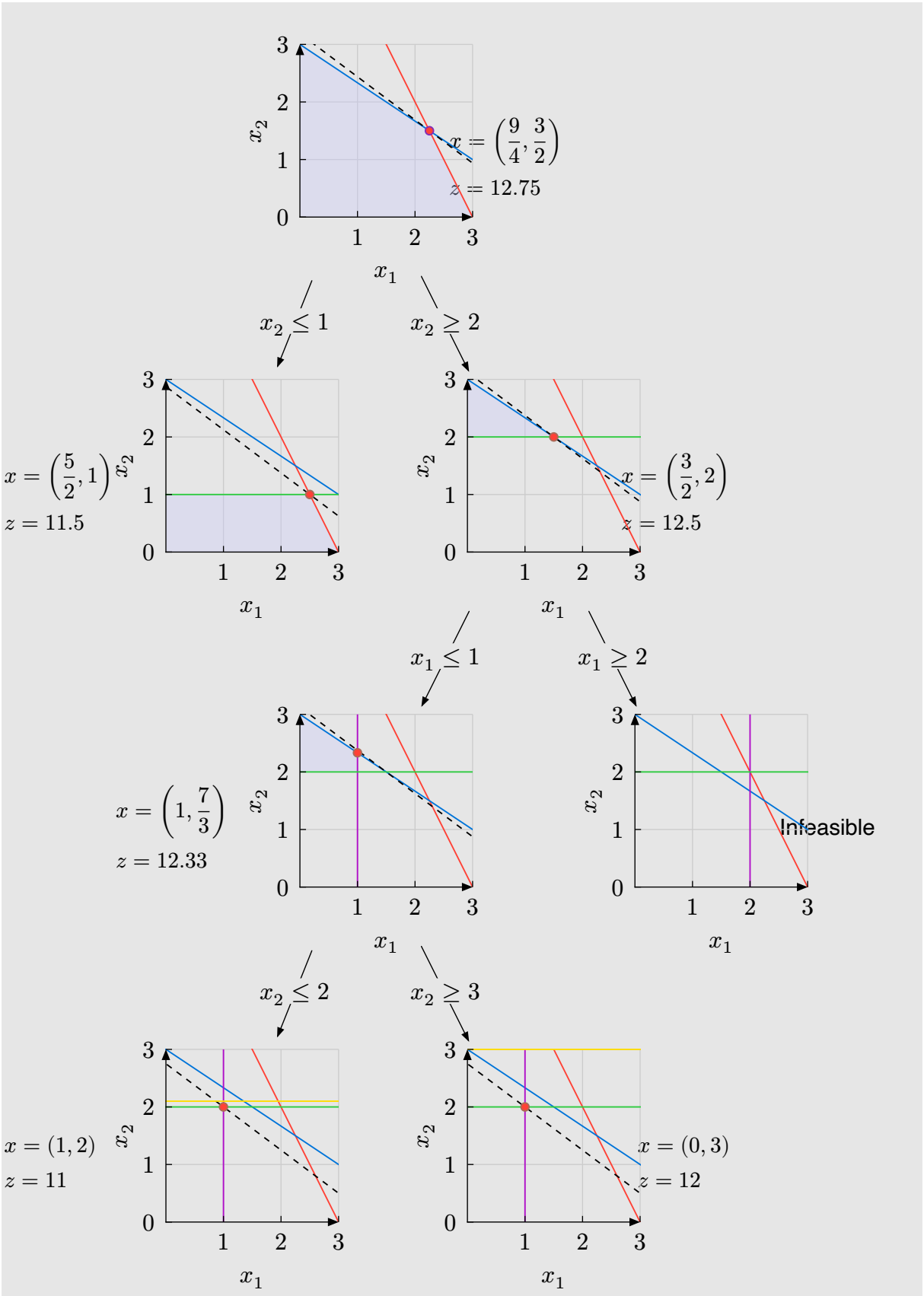
Example

$$
\begin{aligned}
\max \quad & z = 3x_1 + 4x_2 \\
s.t. \quad & 2x_1 + x_2 \leq 6 \\
& 2x_1 + 3x_2 \leq 9 \\
& x_i \in \mathbb{N}_+ \quad \forall i = 1, 2
\end{aligned}
$$

$$
\begin{aligned}
\max \quad & z = 3x_1 + 4x_2 \\
s.t. \quad & 2x_1 + x_2 \leq 6 \\
& 2x_1 + 3x_2 \leq 9 \\
& x_i \geq 0 \quad \forall i = 1, 2
\end{aligned}
$$

$$
x_1 = \frac{9}{4}, \quad x_2 = \frac{3}{2}
$$

$$
z = 12.75
$$


$x = \left(\frac{9}{4}, \frac{3}{2}\right)$
$z = 12.75$
x_1
x_2
$x_2 \leq 1$
$x_2 \geq 2$
$x = \left(\frac{5}{2}, 1\right)$
$z = 11.5$
$x = \left(\frac{3}{2}, 2\right)$
$z = 12.5$
$x_1 \leq 1$
$x_1 \geq 2$
$x = \left(1, \frac{7}{3}\right)$
$z = 12.33$
Infeasible
$x_2 \leq 2$
$x_2 \geq 3$
$x = (1, 2)$
$z = 11$
$x = (0, 3)$
$z = 12$

181.10. Mixed Integer Programming

181.10.1. MIP (Mixed Integer Programming)

Optimizing (maximizing or minimizing) a linear **objective function** subject to linear equality or inequality **constraints**. **Decision variables** can take any continuous real or integer values

Minimize (or Maximize):

$$c^T x$$

Subject to:

- $Ax \leq b$
- $x_i \in \mathbb{Z}$ for some i
- $x_j \in \mathbb{R}$ for the remaining j

Where:

- x is the vector of decision variables
- c is the vector of objective function coefficients
- A is the constraint coefficient matrix
- b is the vector of constraint right-hand side values
- x_i represents integer variables
- x_j represents continuous variables

Example

Objective Function

Maximize

$$c^T x$$

$$\begin{bmatrix} 3 & 2 & -5 & -4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}$$

Constraints

$$Ax \leq b$$

$$\begin{bmatrix} 1 & 2 & 0 & 0 \\ 3 & 1 & 0 & 0 \\ 1 & 0 & -5 & 0 \\ 0 & 1 & 0 & -5 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} \leq \begin{bmatrix} 8 \\ 12 \\ 0 \\ 0 \end{bmatrix}$$

181.11. Non-Linear Programming

181.11.0.1. EOQ

Parameters

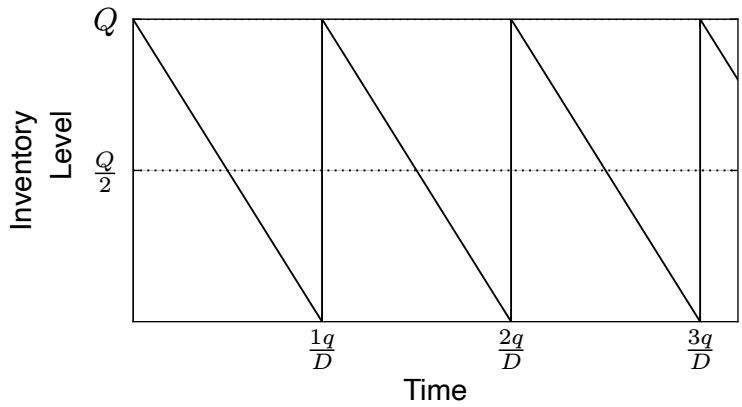
- D: annual demand (units)
- K: unit ordering cost ($)
- h: unit holding cost per year ($)
- p: unit purchasing cost ($)

Decision Variable

- Q: order quantity per order (units)

Objective

Minimize annual total cost



- $\frac{q}{2}$: average inventory level
- $h \times \frac{q}{2} = \frac{hq}{2}$: annual holding cost
- pD: annual purchasing cost
- $\frac{D}{q}$: number of orders in a year
- $K \times \frac{D}{q} = \frac{KD}{q}$: annual ordering cost

Objective Function

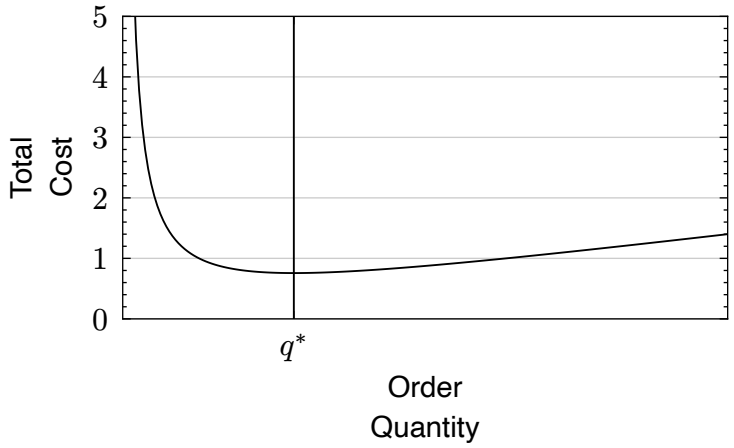
$$\min_{q \geq 0} \underbrace{\frac{KD}{q}}_{\substack{\text{Ordering} \\ \text{Cost}}} + \underbrace{pD}_{\substack{\text{Purchasing} \\ \text{Cost}}} + \underbrace{\frac{hq}{2}}_{\substack{\text{Inventory} \\ \text{Cost}}}$$

pD is just a constant, so:

$$TC(q) = \underbrace{\frac{KD}{q}}_{\substack{\text{Ordering} \\ \text{Cost}}} + \underbrace{\frac{hq}{2}}_{\substack{\text{Inventory} \\ \text{Cost}}}$$

- As q **increases**:
  - Inventory costs increase
  - Ordering cost decease
- As q **decreases**:

- Inventory costs decrease
- Ordering cost increase



181.11.0.2. Portfolio Optimization

Objective

Invest B dollars in n stocks while managing risk and meeting required return

Notation

- B: budget ($)
- R: required minimum expected return
- For each stock $i \in \{1, ..., n\}$
  - p_i: current price
  - μ_i: expected future price
  - σ_i^2: variance of ruturn per share
  - σ_{ij}: Covariance between assets i and j
  - x_i: number of shares to buy (decision variable)

Model

$$\begin{aligned}
\min \quad & \sum_{i=1}^{n} \sigma_i^2 x_i^2 + 2 \sum_{i=1}^{n} \sum_{j=i+1}^{n} \sigma_{ij} x_i x_j && \text{(Total risk)} \\
s.t. \quad & \sum_{i=1}^{n} p_i x_i \leq B && \text{(Budget constraint)} \\
& \sum_{i=1}^{n} \mu_i x_i \geq R && \text{(Return constraint)} \\
& x_i \geq 0 \quad \forall i = 1, ..., n && \text{(No short-selling)}
\end{aligned}$$

181.11.1. Linearizing Maximum/Minimum Functions

1. When the **maximum** function is on the **smaller side** of inequality

General Form

$$y \geq \max(x_1, x_2) \qquad \Longleftrightarrow \qquad y \geq x_1 \wedge y \geq x_2$$

- This rule holds regardless of whether x_1, x_2, y are variables, constants, or expressions

- You're ensuring that y is greater than or equal to all values in the maximum function, so it must be greater than or equal to each term individually

Generalized form with more than two elements

$$y \geq \max(x_1, x_2, ..., x_n) \quad \iff \quad y \geq x_i \quad \forall i = 1, ..., n$$

Example

$$y + x_1 + 3 \geq \max(x_1 - x_3, 2x_2 + 4) \quad \iff \quad \begin{cases} y + x_1 + 3 \geq x_1 - x_3 \\ y + x_1 + 3 \geq 2x_2 + 4 \end{cases}$$

2. When the **minimum** function is on the **larger side** of inequality

General Form

$$y \leq \min(x_1, x_2) \quad \iff \quad y \leq x_1 \wedge y \leq x_2$$

- This rule holds regardless of whether x_1, x_2, y are **variables**, **constants**, or **expressions**
- You're ensuring that y is smaller than or equal to all values in the minimum function, so it must be smaller than or equal to each term individually

Generalized form with more than two elements

$$y \leq \min(x_1, x_2, ..., x_n) \quad \iff \quad y \leq x_i \quad \forall i = 1, ..., n$$

Example

$$y + x_1 \leq \min(x_1 - x_3, 2x_2 + 4, 0) \quad \iff \quad \begin{cases} y + x_1 \leq x_1 - x_3 \\ y + x_1 \leq 2x_2 + 4 \\ y + x_1 \leq 0 \end{cases}$$

Cases Where Linearization Does Not Apply

- $y \leq \max(x_1, x_2) \not\Leftrightarrow y \leq x_1 \wedge y \leq x_2$
- $y \geq \max(x_1, x_2) \not\Leftrightarrow y \geq x_1 \wedge y \geq x_2$
- Maximum or minimum function in an equality

$$y = \max(x_1, x_2) \quad \text{or} \quad y = \min(x_1, x_2)$$

181.11.2. Linearize Objective Function

1. **Minimize** a **Maximum** Function

$$\min \quad \max(x_1, x_2) \quad \iff \quad \begin{aligned} \min \quad & w \\ s.t. \quad & w \geq x_1 \\ & w \geq x_2 \end{aligned}$$

2. **Maximize** a **Minimum** Function

$$\begin{aligned} \max \quad & \min(x_1, x_2, 2x_3 + 5) + x_4 \\ s.t. \quad & 2x_1 + x_2 - x_4 \leq x_3 \end{aligned} \iff \begin{aligned} \max \quad & w + x_4 \\ s.t. \quad & w \leq x_1 \\ & w \leq x_2 \\ & w \leq 2x_3 + 5 \\ & 2x_1 + x_2 - x_4 \leq x_3 \end{aligned}$$

Cases Where Linearization Does Not Apply

- $\max \max(x_1, x_2)$
- $\min \min(x_1, x_2)$

Absolute Function

The absolute value is equivalent to a maximum function:

$$|x| = \max(x, -x)$$

Thus, it can be linearized when it appears on the smaller side of an inequality:

$$|x| \leq y \iff \begin{cases} x \leq y \\ -x \leq y \end{cases}$$

Example

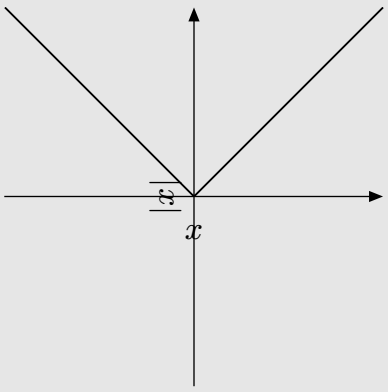
We want to allocate \$1000 to 2 people in a fair way

- **Fairness criterion**: Minimize the difference between the amounts each person receives.
- Let x_i be amount to allocate to person i for $i = 1, 2$

We write the problem as:

$$\begin{aligned} \min \quad & |x_2 - x_1| \\ s.t. \quad & x_1 + x_2 = 1000 \\ & x_i \geq 0 \quad \forall i = 1, 2 \end{aligned}$$

- The absolute value makes the objective **nonlinear**



Linearizing the Problem

We now reformulate the problem to make it linear

Step 1.: Introduce a new variable

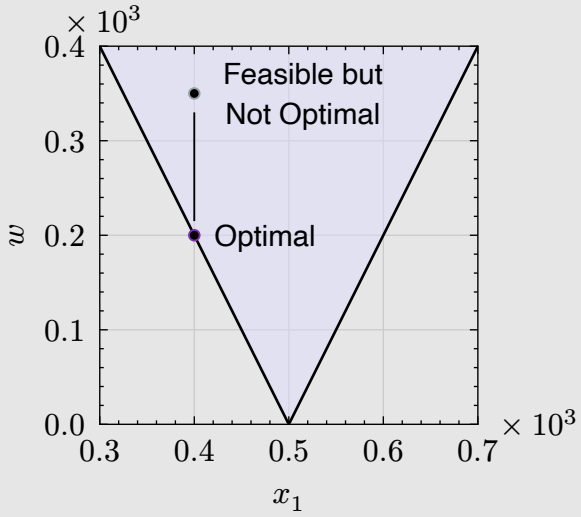
Let w be the absolute difference: $w = |x_2 - x_1|$

$$\begin{aligned} \min \quad & w \\ s.t. \quad & x_1 + x_2 = 1000 \\ & w = |x_2 - x_1| \\ & x_i \geq 0 \quad \forall i = 1, 2 \end{aligned}$$

Step 2. Relax the equality

We replace the equality with an inequality:

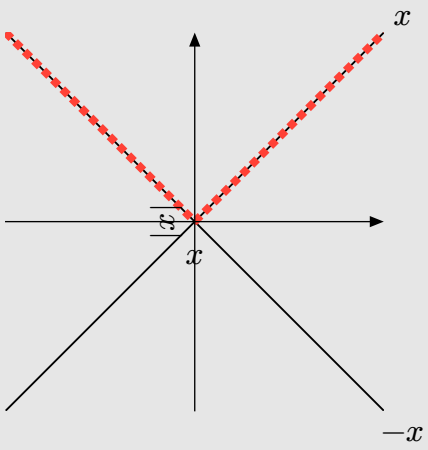
$$\begin{aligned} \min \quad & w \\ s.t. \quad & x_1 + x_2 = 1000 \\ & w \geq |x_2 - x_1| \\ & x_i \geq 0 \quad \forall i = 1, 2 \end{aligned}$$



Step 3. Replace absolute value with maximum

Recall:

$$|x_2 - x_1| = \max(x_2 - x_1, x_1 - x_2)$$



Therefore:

$$w \geq |x_2 - x_1| \quad \Leftrightarrow \quad w \geq \max(x_2 - x_1, x_1 - x_2) \quad \Leftrightarrow \quad w \geq x_2 - x_1 \wedge w \geq x_1 - x_2$$

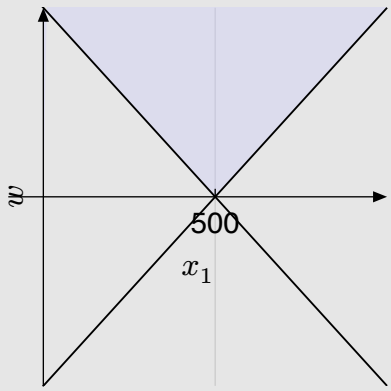
Now the problem is fully linear:

$$\begin{aligned} \min \quad & w \\ s.t. \quad & x_1 + x_2 = 1000 \\ & w \geq x_2 - x_1 \\ & w \geq x_1 - x_2 \\ & x_i \geq 0 \quad \forall i = 1, 2 \end{aligned}$$

Reformulating in One Variable

using $x_2 = 1000 - x_1$, we can express everything in terms of x_1:

$$\begin{aligned} \min \quad & w \\ s.t. \quad & w \geq 1000 - 2x_1 \\ & w \geq 2x_1 - 1000 \\ & x_1 \geq 0 \end{aligned}$$



We are minimizing w, which represents the difference between the two allocations. The solution occurs when the two inequalities intersect — that is, when both are equal:

$$1000 - 2x_1 = 2x_1 - 1000 \quad \Rightarrow \quad x_1 = 500$$

So:

- $x_1 = 500$
- $x_2 = 1000 - x_1 = 500$
- $w = |500 - 500| = 0$

This is the fair allocation: each person receives $500, and the difference between the two amounts is 0.

181.11.3. Linearizing Products of Decision Variables

Products of decision variables can be linearized if:

- A binary and a continuous variable
- Two binary variables

Cannot be linearized if:

- Two continuous variables

181.12. Convexity

Show $f(\theta x + (1-\theta)y) \leq \theta f(x) + (1-\theta)f(y)$ for all x, y in the domain and $\theta \in [0,1]$

Let $D \subseteq \mathbb{R}^n$ be a convex set. A function $f : D \to \mathbb{R}$ is convex if for all $x, y \in D$ and all $\theta \in [0,1]$,

$$f(\theta x + (1-\theta)y) \leq \theta f(x) + (1-\theta)f(y).$$

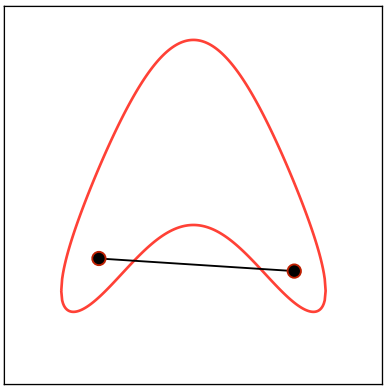
Geometric reading: For every pair of points on the graph, the chord joining them lies on or above the graph over the interval between them.

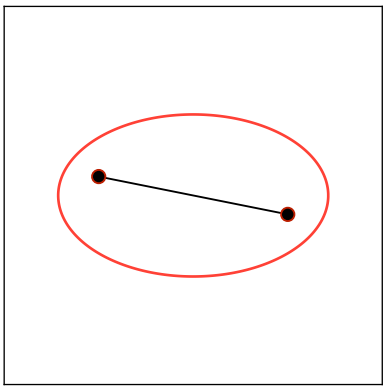
Second-order condition

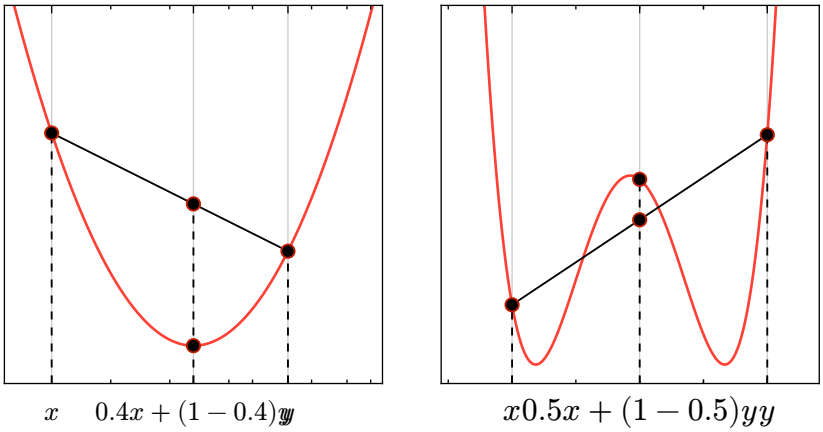
If f is twice differentiable, f is convex iff its Hessian is positive semidefinite everywhere on the domain.

First-order condition

If f is differentiable, f is convex iff $f(y) \geq f(x) + \nabla f(x)^T(y - x)$ for all x, y.







$f(\theta x + (1-\theta)y) \leq \theta f(x) + (1-\theta)f(y)$

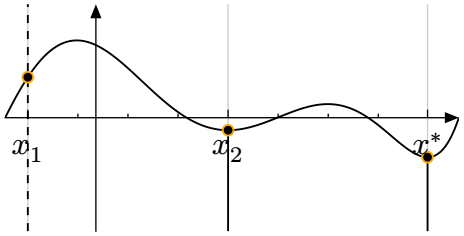
Convex combination:

- Coefficients nonnegative
- Coefficients summing to 1

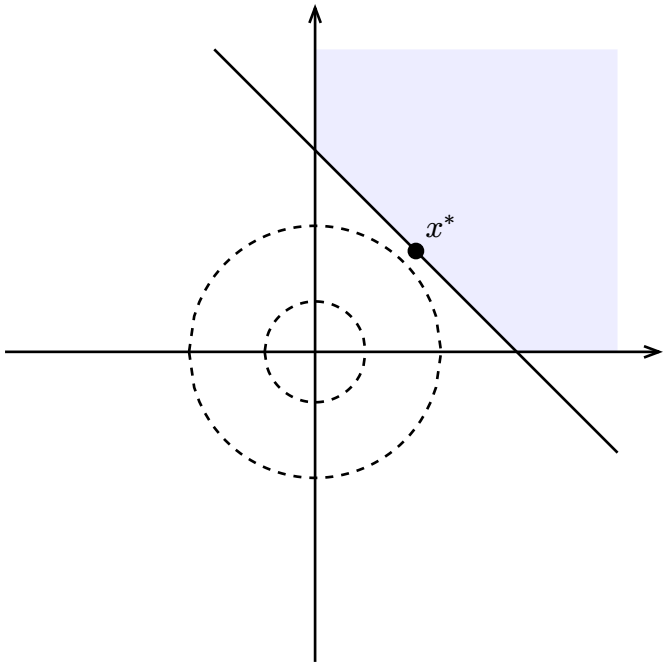
181.13. Convex Analysis

Difficulties of NLP

- A local minimum is not always a global minimum



- An optimal solution may not be an extreme point optimal solution



181.13.0.1. Convexity

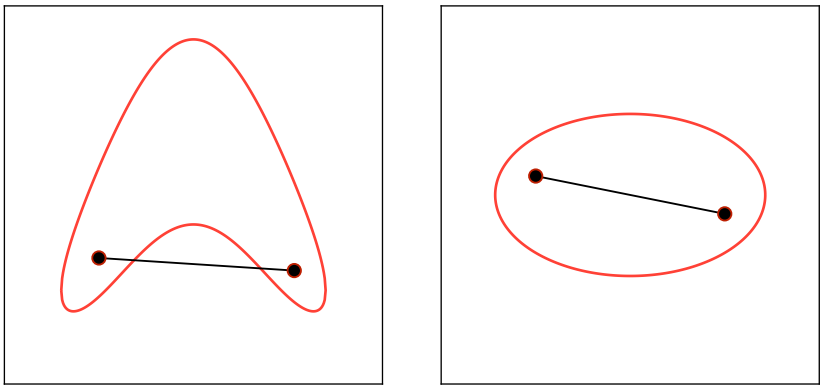
Convex Set

If for whatever two points you select in the set, the line segment connecting them also lies in the set, then the set is convex

A set F is convex if

$$\lambda x_1 + (1-\lambda)x_2 \in F$$

for all $\lambda \in [0,1]$ and $x_1, x_2 \in F$

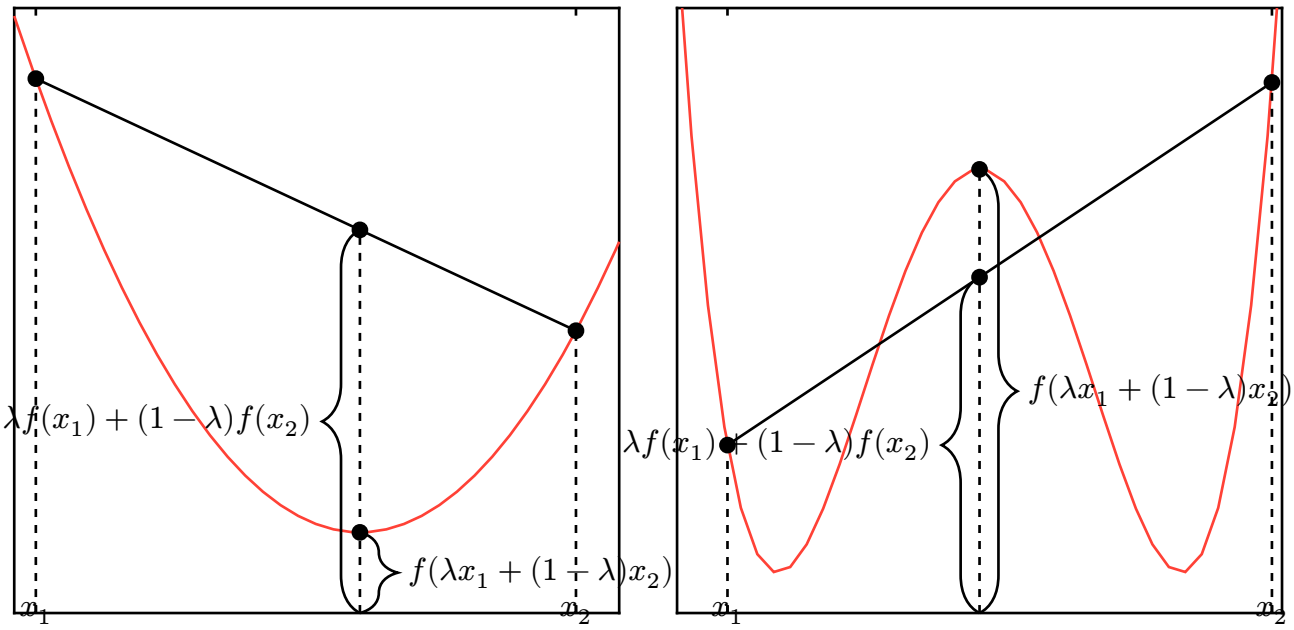


Convex Function

For a convex domain $F \subseteq \mathbb{R}^n$, a function $f : \mathbb{R}^n \to \mathbb{R}$ is convex over F if

$$f(\lambda x_1 + (1-\lambda)x_2) \leq \lambda f(x_1) + (1-\lambda) f(x_2)$$

for all $\lambda \in [0,1]$ and $x_1, x_2 \in F$



Concave Function

For a convex domain $F \in \mathbb{R}^n$, a function $f : \mathbb{R}^n \to \mathbb{R}$ is concave over F if $-f$ is convex

Global Optimality of Convext Functions

Proposition 1. For a convex (concave) function f over a convex doain F, a local minimum (maximum) is a global minimum (maximum)

Two conditions:

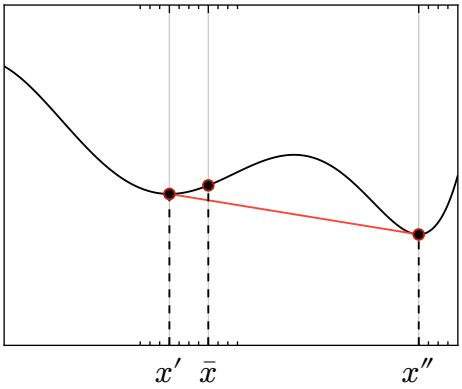
- Function needs to be convex
- Domain (feasible region) need to be convex

Proof

Suppose a local minimum x' is not a global minimum and there exists x'' such that $f(x'') < f(x')$. Consider a small enough $\lambda > 0$ such that $\bar{x} = \lambda x'' + (1-\lambda)x'$ satisfies $f(\bar{x}) > f(x')$. Such $\bar{x}$ exists because x' is a local minimum. Now, note that

$$\begin{aligned} f(\bar{x}) &= f(\lambda x'' + (1-\lambda)x') \\ &> f(x') \\ &= \lambda f(x') + (1-\lambda) f(x') \\ &> \lambda f(x'') + (1-\lambda) f(x') \end{aligned}$$

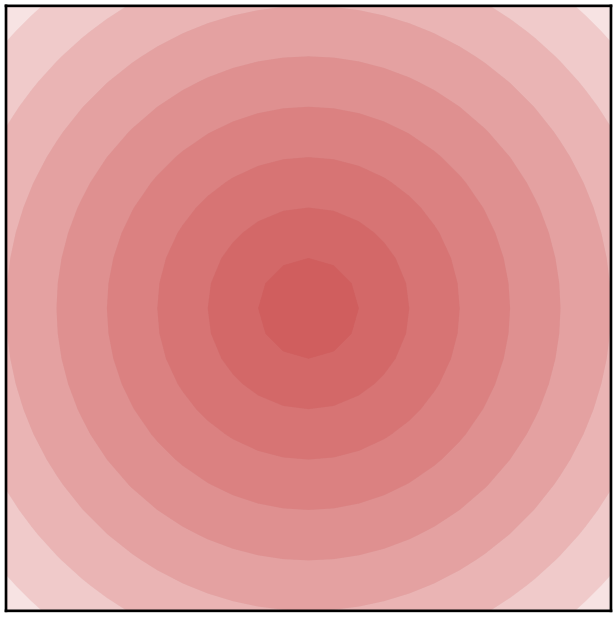
Which violates the fact that $f(\cdot)$ is convex. Therefore, by contradiction, the local minimum x' must be a global minimum.



Minimize a concave function

Proposition 2: For a concave function that has a global minimum over a convex feasible region, there exists a global minimum that is an extreme point

Extreme point: In convex analysis, an extreme point of a convex set C is a point in C that cannot be expressed as a strict convex combination of two other distinct points in C

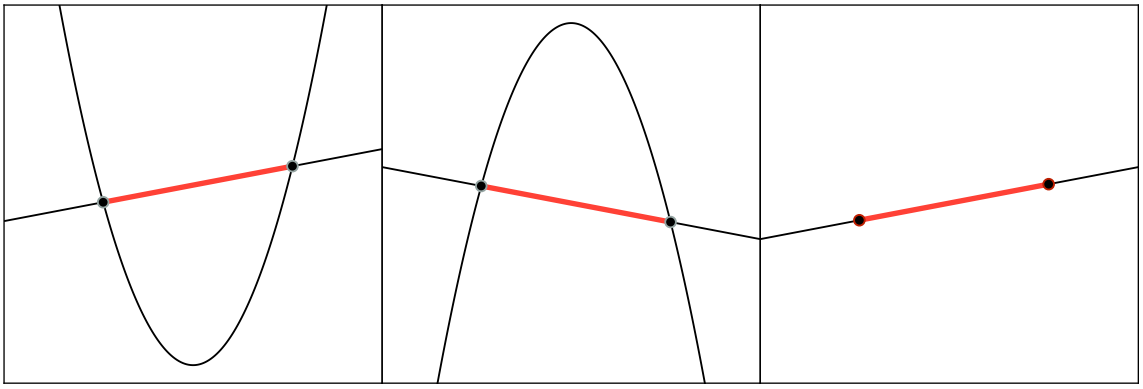


Special Case: LP

When we minimize $f(\cdot)$ over a convez feasible region F:

- If $f(\cdot)$ is **convex**, search for a **local minimum**
- If $f(\cdot)$ is **concave**, search among **extreme points** of F

For any LP we have both



Proposition 3. The feasible region of an LP is convex

The intersections of convex sets are convex

Proposition 4. A linear function $f : \mathbb{R}^n \to \mathbb{R}$ is both convex and concave

Proof:

$$f(\lambda x_1 + (1-\lambda)x_2) = \lambda f(x_1) + (1-\lambda)f(x_2)$$

Let $f(x) = c^T x + b$ be a linear function, $c \in \mathbb{R}^n, b \in \mathbb{R}$, then:

$$\begin{aligned} f(\lambda x_1 + (1-\lambda)x_2) &= c^T(\lambda x_1 + (1-\lambda)x_2) + b \\ &= \lambda(c^T x_1 + b) + (1-\lambda)(c^T x_2 + b) \\ &= \lambda f(x_1) + (1-\lambda)f(x_2) \end{aligned}$$

Convex Programming

Consider a general NLP

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & f(x) \\ s.t. \quad & g_{i(x)} \leq b_i \quad \forall i = 1, ..., m \end{aligned}$$

If:

- The feasible region $F = \{x \in \mathbb{R}^n \mid g_i(x) \leq b_i \forall i = 1, ..., m\}$ (intersection of all the regions satisfying the constraints)
- f is convex over F

A local minimum is a global minimum

Definition 0: Convex Program (CP)

An NLP is convex if its feasible region is convex and its objective function is convex over the feasible region

Convex Programming:

- Minimizing convex function
- Maximizing concave function

Subject to a convex feasible region

Local min = Global min

For an NLP

$$\min_{x \in \mathbb{R}^n} \{f(x) \mid g_i(x) \leq b_i \forall i - 1, ..., m\}$$

if f and g_is are all convex functions, the NLP is a Convex Program

Proof:

We only need to prove that the feasible region is convex, which is implied if $F_i = \{x \in \mathbb{R}^n \mid g_i(x) \leq b_i\}$ is convex for all i. For two points $x_1, x_2 \in F_i$ and an arbitrary $\lambda \in [0, 1]$, we have

$$\begin{aligned} g_i(\lambda x_1 + (1-\lambda)x_2) &\leq \lambda g_i(x_1) + (1-\lambda) g_i(x_2) \\ &\leq \lambda b_i + (1-\lambda) b_i = b_i \end{aligned}$$

Which implies that F_i is convex. repeating this argument for all i completes the proof

If each constraint independently given a convex feasible region, then their intersection is convex

Definition 0: Affine Combination

An affine combination of points $x_1, x_2, ..., x_k \in \mathbb{R}^n$, is any point of the form:

$$\alpha_1 x_1 + \alpha_2 x_2 + ... + \alpha_k x_1 + x_k$$

where the coefficients sum to 1:

$$\alpha_1 + \alpha_2 + ... + \alpha_k = 1$$

This is like a linear combination, but with the extra condition that the weights add up to 1.

For a twice differentiable function $f : \mathbb{R} \to \mathbb{R}$ over an interval (a, b):

- f is convex over (a, b) iff $f''(x) > 0$ for all $x \in (a, b)$
- $\bar{x}$ is a local minimum over (a, b), iff $f'(x) = 0$
- If f is concave over (a, b), x^* is a global minimum over (a, b) iff $f'(\bar{x}) = 0$

First order condition (FOC) $f'(x) = 0$

- FOC is necessary for local optimality
- FOC is sufficient for global optimality if f is convex

Example

Economic Order Quantity (EOQ)

Determine the order quantity in each order

- Demand is deterministic and occurs at a constant rate
- Each order incurs a fixed cost (independent of the order size)
- No shortage allowed
- Lead time is zero
- Holding cost is proportional to the average inventory

- Constant holding cost

Parameters:

- D: annual demand (units/year)
- K: unit ordering cost ($/order)
- h: annual holding cost ($/unit/year)
- p: unit purchase cost ($/unit)

Decision Variable:

- q: order quantity (units/order)

Objective:

- Minimize the total annual cost

Average inventory level:

$$\frac{q}{2}$$

Annual holding cost:

$$H = h\left(\frac{q}{2}\right) = \frac{hq}{2}$$

Annual purchase cost:

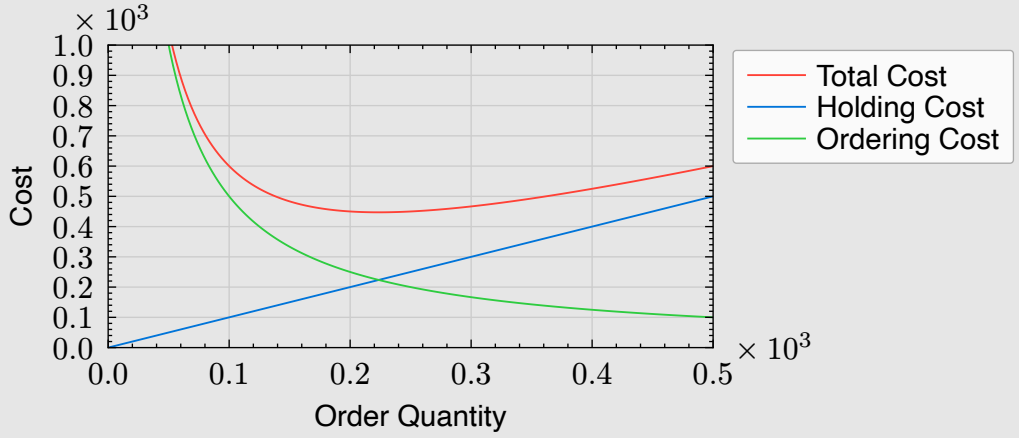
$$pD$$

Annual ordering cost:

$$O = \left(\frac{D}{q}\right)K = \frac{DK}{q}$$

NLP, since pD does not depend on q it does not affect the minimizer:

$$\begin{aligned}\min_{q>0} f(q) &= \left(\frac{D}{q}\right)K + \left(\frac{q}{2}\right)h \\ &= \frac{DK}{q} + \frac{qh}{2}\end{aligned}$$



For:

$$TC(q) = \frac{KD}{q} + \frac{hq}{2}$$

we have

$$TC'(q) = -\frac{KD}{q^2} + \frac{h}{2}$$

and

$$TC''(q) = \frac{2KD}{q^3}$$

Since a twice differentiable function is convex If

$$f''(x) > 0 \quad \forall x \in (a, b)$$

and

$$K > 0, \quad D > 0, \quad q > 0$$

we have

$$TC''(q) = \frac{2KD}{q^3} > 0$$

Therefore, $TC(q)$ is convex

Let q^* be the quantity satisfying the FOC:

$$TC'(q^*) = -\frac{KD}{(q^*)^2} + \frac{h}{2} = 0 \quad \Rightarrow \quad q^* = \sqrt{\frac{2KD}{h}}$$

Implications:

- If order cost K increases, order quantity q^* increases
- If demand D increases, order quantity q^* increases
- If holding cost h increases, order quantity q^* decreases

181.13.1. Multivariate Convex Analysis

An optimal solution either:

- Satisfies the FOC
- Lies on the boundary of the feasible region

If a NLP is a CP, a feasible point satisfying the FOC is optimal (any local minimum is a global minimum)

For a function $f : \mathbb{R}^n \to \mathbb{R}$, its ith partial derivative is $\frac{\partial f(x)}{\partial x_i}$

For a twice differentiable function $f : \mathbb{R}^n \to \mathbb{R}$, if all second order partial derivatives are continuous:

$$\frac{\partial^2 f(x)}{\partial x_i \partial x_j} = \frac{\partial^2 f(x)}{\partial x_j \partial x_i}$$

for all $i = 1, ..., n$ and $j = 1, ..., n$.

Single variate case:

- For $f : \mathbb{R} \to \mathbb{R}$, f is convex iff $f''(x) \geq 0$ for all x

Multivariate case:

$$\nabla f(x) = \begin{bmatrix} \frac{\partial f(x)}{\partial x_1} \\ \vdots \\ \frac{\partial f(x)}{\partial x_n} \end{bmatrix} \qquad \nabla^2 f(x) = H_f = \begin{bmatrix} \frac{\partial^2 f(x)}{\partial x_1^2} & \cdots & \frac{\partial^2 f(x)}{\partial x_1 \partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f(x)}{\partial x_n \partial x_1} & \cdots & \frac{\partial^2 f(x)}{\partial x_n^2} \end{bmatrix}$$

Example

$$f(x_1, x_2, x_3) = x_1^2 + x_2 x_3 + x_3^3$$

$$\nabla f(x) = \begin{bmatrix} \frac{\partial f(x)}{\partial x_1} \\ \frac{\partial f(x)}{\partial x_2} \\ \frac{\partial f(x)}{\partial x_3} \end{bmatrix} = \begin{bmatrix} 2x_1 \\ x_3 \\ x_2 + 3x_3^2 \end{bmatrix}$$

$$\nabla^2 f(x) = \begin{bmatrix} \frac{\partial^2 f(x)}{\partial x_1^2} & \frac{\partial^2 f(x)}{\partial x_1 \partial x_2} & \frac{\partial^2 f(x)}{\partial x_1 \partial x_3} \\ \frac{\partial^2 f(x)}{\partial x_2 \partial x_1} & \frac{\partial^2 f(x)}{\partial x_2^2} & \frac{\partial^2 f(x)}{\partial x_2 \partial x_3} \\ \frac{\partial^2 f(x)}{\partial x_3 \partial x_1} & \frac{\partial^2 f(x)}{\partial x_3 \partial x_2} & \frac{\partial^2 f(x)}{\partial x_3^2} \end{bmatrix} = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 6x_3 \end{bmatrix}$$

You may ask:

- What is the gradient at a point: $\nabla f(3, 2, 1)$
- What is the Hessian at a point: $\nabla^2 f(3, 2, 1)$

Single Variate Function $f : \mathbb{R} \to \mathbb{R}$	• f is convex in $[a, b]$ if $f''(x) \geq 0$ for all $x \in [a, b]$ • $\bar{x}$ is an interior local minimum if $f'(\bar{x}) = 0$ • If f is convex in $[a, b]$, x^* is a global minimum iff $f'(x^*) = 0$
Multi Variate Function $f : \mathbb{R}^n \to \mathbb{R}$	• f is convex in a convex set $F \subseteq \mathbb{R}^n$ if $\nabla^2 f(x)$ is positive semi-definite for all $x \in F$ • $\bar{x}$ is an interior local minimum if $\nabla f(\bar{x}) = \mathbf{0}$ • If f is convex in a convex set F, x^* is a global minimum iff $\nabla f(x^*) = 0$

Definition 0: Positive Semi-Definite (PSD) Matrix

A symmetric matrix A is positive semi-definite if $\mathbf{x}^T A\ \mathbf{x} \geq 0$ for all $\mathbf{x} \in \mathbb{R}^n$

Example

Semi-Definite Matrix

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

$$x^T Ax = \begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

$$= 2x_1^2 + 2x_1x_2 + 2x_2^2$$

$$= 2(x_1 + x_1)^2 + x_1^2 + x_2^2 \geq 0$$

Given a function f, when is its Hessian $\nabla^2 f$ PSD?

For a symmetric matrix A, the following statements are equivalent:

- A is **positive semi-definite**
- All **eigenvalues** of A are non-negative
- All **principal minors** of A are non-negative

A's level-k principal minors is the determinant of a $k \times k$ submatrix whose diagonal is a subset of A's diagonal.

A sufficient condition is for A's **leading** principl minors to all **positive**

For a function f:

1. Find **Hessian** $\nabla^2 f(x)$
2. Find **eigenvalues** or **principal minors** of $\nabla^2 f(x)$
3. Determine over what region $\nabla^2 f(x)$ is **PSD**

The function is convex over that region

Example

$$\min_{x \in \mathbb{R}^2} f(x_1, x_2)$$

$$f(x_1, x_2) = x_1^2 + x_2^2 + x_1x_2 - 2x_1 - 4x_2$$

$$\nabla f(x) = \begin{bmatrix} 2x_1 + x_2 - 2 \\ x_1 + 2x_2 - 4 \end{bmatrix} \qquad \nabla^2 f(x) = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

Find **eigenvalues**

$$Ax = \lambda x \iff (A - \lambda I)x = 0 \iff \det(A - \lambda I) = 0$$

$$\begin{vmatrix} 2-\lambda & 1 \\ 1 & 2-\lambda \end{vmatrix} = 0 \iff 3 - 4\lambda + \lambda^2 = 0 \iff \lambda = 1 \text{ or } 3$$

Or find **leading principal minors**

$$|2| = 2 \quad \text{and} \quad \begin{vmatrix} 2 & 1 \\ 1 & 2 \end{vmatrix} = 3$$

So $\nabla^2 f(x_1, x_2)$ is PSD and thus $\min_{x \in \mathbb{R}^2} f(x_1, x_2)$ is a CP.

The FOC requires $2x_1^* + x_2^* - 2 = 0$ and $x_1^* + 2x_2^* - 4 = 0$

I.e., $(x_1^*, x_2^*) = (0, 2)$

Example

$$\min_{x \in \mathbb{R}^2} f(x_1, x_2)$$

$$f(x_1, x_2) = x_1^3 + 4x_1x_2 + \frac{1}{2}x_2^2 + x_1 + x_2$$

$$\nabla^2 f(x) = \begin{bmatrix} 6x_1 & 4 \\ 4 & 1 \end{bmatrix}$$

- 1st leading principle minor $6x_1 \geq 0$ ($x_1 \geq 0$)
- 2nd leading principal minor $6x_1 - 16 \geq 0$ ($x_1 \geq \frac{8}{3}$)
- $1 \geq 0$

Therefore, the function is convex iff $x_1 \geq \frac{8}{3}$

Example

Two-Product Pricing

A retailer sells product 1 and 2 at prices p_1 and p_2. For product i the demand q_i is:

$$q_1 = a - p_1 + bp_2$$
$$q_2 = a - p_2 + bp_1$$

where $a > 0$ and $b \in [0, 1)$. The retailer sets p_1 and p_2 to maximize its total profit.

If p_1 and p_2 are substitutes of one another then the price of one will affect the demand of the other.

1. Why $b \in [0, 1)$?

If $b \geq 1$: the other product's price will have the equal or greater impact on your own product's price

If $b < 0$: Complimentary products (rather than substitutes), the higher the other product's price, the lower our demand

2. Formulate problem

$$\max_{p_1,p_2} \quad p_1\underbrace{(a - p_1 + bp_2)}_{q_1} + p_2\underbrace{(a + bp_1 - p_2)}_{q_2}$$

Let

$$f(p) = -[p_1(a - p_1 + bp_2) + p_2(a + bp_1 - p_2)$$

3. Is it a CP?

$$\nabla^2 f(p) = \begin{bmatrix} 2 & -2b \\ -2b & 2 \end{bmatrix}$$

Which is PSD if $b \in [0, 1)$ since

- 1st leading principle minor $2 \geq 0$
- 2nd leading principle minor $4 - 4b^2$ $(4(1 - b)(1 + b) \geq 0)$

Therefore $f(p)$ is convex and $-f(p)$, the objective function is concave

The problem is a CP

4. Solve problem

$\nabla f(p) = 0$ requires:

- $-a + 2p_1 - 2bp_2 = 0$
- $-a + 2p_2 - 2bp_1 = 0$

So,

$$p_1 = p_2 = \frac{a}{2(1 - b)}$$

5. How does optimal prices change with a and b?

When

- a increases, the two prices increase: price of product increases when demand increases
- b increases, the two prices increase: effective demand becomes larger

Example

Question 1

For each of the following sets, select all that are convex:
1. $S = \{(x_1, x_2) \in \mathbb{R}^2 \mid x_1^2 + x_2^2 \leq 4, x_1 \geq 0, x_2 \leq 0\}$

✅ $x_1^2 + x_2^2 \leq 4$, a disk of radius 2, is convex and the $x_1 \geq 0, x_2 \leq 0$ lines are also convex

The intersection of convex sets is convex

2. $S = \{(x_1, x_2) \in \mathbb{Z}^2 \mid x_1^2 + x_2^2 \leq 4, x_1 \geq 0, x_2 \leq 0\}$

❌ Integer programs are never convex in the usual sense, because the feasible set of integers is discrete

3. $S = \left\{x \in \mathbb{R}^n \mid \sum_{i=1}^n x_i^2 \leq 4, x_1 \geq 0, x_2 \leq 0\right\}$

✅ $\sum_{i=1}^n x_i^2 \leq 4$, a n-dimensional sphere (hypersphere) of radius 2, is convex and the $x_1 \geq 0, x_2 \leq 0$ lines are also convex

The intersection of convex sets is convex

4. $S = \{x \in \mathbb{R}^n \mid x_1 + x_2 = 10(x_3 + x_4)\}$ where $n \geq 4$

✅ $h(x)$ is a linear function ($h(x) = x_1 + x_2 - 10x_3 - 10x_4$) so $h(x)$ is also affine and is therefore convex

5. $S = \left\{x \in \mathbb{R}^n \mid \max_{i=1,\ldots,n}\{x_i\} = 10 \min_{i=1,\ldots,m}\{x_i\}\right\}$

❌ $\min(x)$ and $\max(x)$

Question 2

For each of the following functions, select all that are convex over the given region:

1. $f(x) = 2x^3 - x^2 - 2x + 1$ for $x \in \mathbb{R}$

$f'(x) = 6x^2 - 2x - 2$

$f''(x) = 12x - 2$

❌

2. $f(x) = \begin{cases} -x & \text{if } x<1 \\ -1 & \text{if } x \geq 1 \end{cases}$ for $x \in \mathbb{R}$

✅

3. $f(x) = x_1^2 - 4x_1x_2 + 3x_2^2 + 3x_1 + 4x_2$ for $x \in \mathbb{R}^2$

❌

4. $f(x) = \sum_{i=1}^n (x_i - a)^2$ where $a > 0$, $x \in \mathbb{R}^n$

✅

5. $f(\alpha, \beta) = \sum_{i=1}^n (\alpha + \beta x_i - y_i)^2$, where $x_i, y_i, \alpha, \beta \in \mathbb{R}$

✅

Question 3

For the NLP

$$\begin{aligned} \min \quad & 2x^3 - x^2 - 2x + 1 \\ s.t. \quad & x \geq -1 \end{aligned}$$

which of the following statements is correct?

1. This is a convex program.

❌

2. There are two local minimizers.

✅

3. The unique global minimizer is a boundary point.

❌

4. This program is unbounded.

❌

5. None of the above.

❌

Question 4

For each of the following matrices, select all that are positive semi-definite:

1.

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

✅

2.

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 3 & 1 \\ 3 & 1 & 2 \end{bmatrix}$$

❌

3.

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 3 & 1 \\ 0 & 0 & 2 \end{bmatrix}$$

✅

4.

$$A = \begin{bmatrix} 1 & 2 & 3 & \dots & n \\ 0 & 2 & 3 & \dots & n \\ 0 & 0 & 3 & \dots & n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & n \end{bmatrix}$$

✅

5.

$$A = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

❌

Question 5

For the following statements, select all that are **incorrect**:

1. An optimal solution to a linear program must be a boundary point.

❌ not guaranteed if the objective is constant over the feasible region

2. An optimal solution to a linear program must be an extreme point.

✅

3. An optimal solution to a nonlinear program can be an interior point.

❌

4. An optimal solution to a nonlinear program must be an interior point.

✅

5. A global optimal solution to a nonlinear program must be a local optimal solution.

❌

181.14. Gradient Descent

The gradient tells us the direction of steepest increase

For a function

$$f(x_1, x_2, ..., x_n)$$

the gradient of f, written ∇f, is:

$$\nabla f(x) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, ..., \frac{\partial f}{\partial x_n} \right)$$

- The **magnitude** of the gradient vector tells you how fast the function is increasing (steepness)
- The **direction** of the gradient vector points to where the function increases most rapidly (direction)

Example

Function

$$f(x_1, x_2) = x_1^2 + x_2^2$$

Compute $\frac{\partial d}{\partial x_1}$

- Differentiate f w.r.t. x_1 (treat x_2 as constant):

$$\frac{\partial}{\partial x_1}(x_1^2 + x_2^2) = \frac{\partial}{\partial x_1}(x_1^2) + \frac{\partial}{\partial x_1}(x_2^2)$$

- Since x_2^2 is constant w.r.t. x_1, its derivative is 0:

$$\frac{\partial}{\partial x_1}(x_2^2) = 0$$

- The derivative of x_1^2 w.r.t. x_1 is:

$$\frac{\partial}{\partial x_1}(x_1^2) = 2x_1$$

- So,

$$\frac{\partial f}{\partial x_1} = 2x_1$$

Compute $\frac{\partial f}{\partial x_2}$

- Differentiate f w.r.t. x_2 (treat x_2 as constant):

$$\frac{\partial}{\partial x_2}(x_1^2 + x_2^2) = \frac{\partial}{\partial x_2}(x_1^2) + \frac{\partial}{\partial x_2}(x_2^2)$$

- Since x_1^2 is constant w.r.t. x_2, its derivative is 0:

$$\frac{\partial}{\partial x_2}(x_1^2) = 0$$

- The derivative of x_2^2 w.r.t. x_2 is:

$$\frac{\partial}{\partial x_2}(x_2^2) = 2x_2$$

So,

$$\frac{\partial f}{\partial x_2} = 2x_2$$

Combine to form gradient

$$\nabla f(x_1, x_2) = (2x_1, 2x_2)$$

Evaluate at a point

$$x = (1, 2)$$

$$\nabla f(1, 2) = (2 \times 1, 2 \times 2) = (2, 4)$$

Interpretation

- The gradient vector $(2, 4)$ at $(1, 2)$ points in the **direction** where the function f **increases most rapidly**
- Its **magnitude** $\sqrt{2^2 + 4^2} = \sqrt{20} \approx 4.47$ measures the **steepness** of that increase
- **Moving one unit along the x_1** axis alone would **increase the function by the partial derivative with respect to x_1**, which is 2
- **Moving one unit along the x_2** axis alone would **increase the function by the partial derivative with respect to x_2**, which is 4
- **Moving from $(1, 2)$ in the direction of $(2, 4)$**, the function value will rise fastest and at a rate roughly proportional to **4.47 per unit distance moved**

181.14.1. Algorithm

Find the minimum of a function

In **gradient descent**, we move in the **opposite direction** of the gradient to minimize a function

Direction of **maximum increase**:

$$\nabla f(x)$$

Direction of **maximum decrease**:

$$-\nabla f(x)$$

1. **Initialize**: Start with an initial guess for the parameters
2. **Compute Gradient**: Find the gradient of the function at the current parameters
3. **Update Parameters**: Adjust the parameters by moving in the opposite direction of the gradient, scaled by the learning rate

4. **Repeat**: Continue the process until the parameters converge to a minimum or the changes are minimal

Update Rule:

$$x^{k+1} \leftarrow x^k - \alpha_k \nabla f\left(x^k\right)$$

Where:

- x^k: current iterate (point in domain of f)
- α_k: **step size** or **learning rate** at iteration k
- $\nabla f\left(x^k\right)$: **gradient** of the function f with respect to θ
- x^{k+1}: next iterate (point in domain of f)

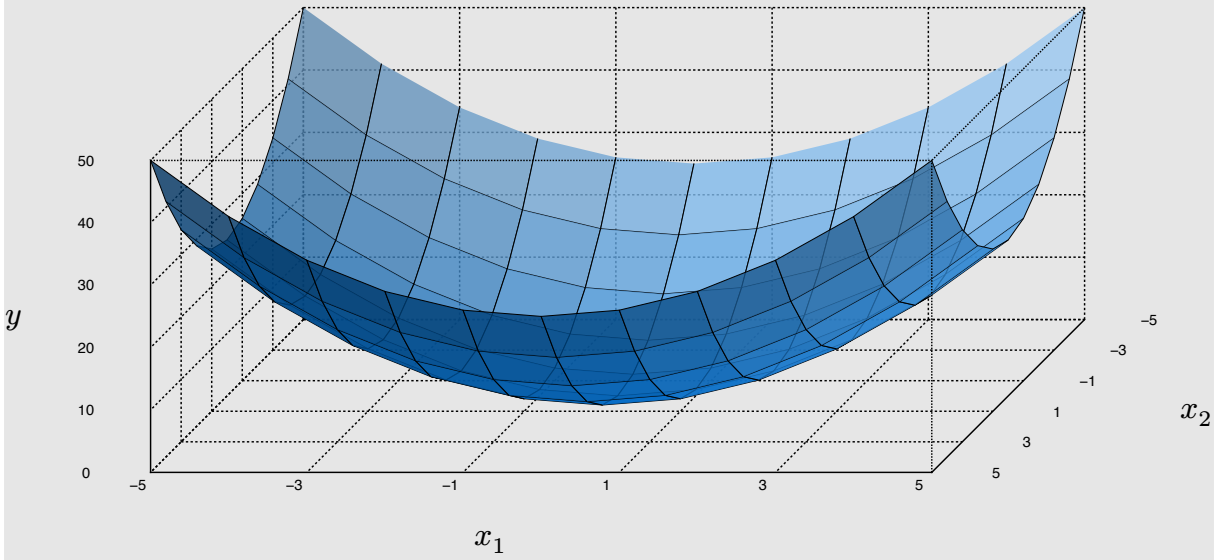
gradient_descent.py

```python
import numpy as np

def gradient_descent(
  f,              # Function to minimize, f(x)
  grad_f,         # Gradient of f, grad_f(x)
  x0,             # Initial guess for x
  alpha=0.01,     # Learning rate
  max_iter=1000,  # Maximum number of iterations
  tol=1e-6        # Tolerance for stopping
):
    x = x0.copy()
    for _ in range(max_iter):
        grad = grad_f(x)
        if np.linalg.norm(grad) < tol:
            break
        x -= alpha * grad
    return x

def f(x):
  """Example: minimize f(x) = x^2 + 2x + 1"""
  return x**2 + 2*x + 1

def grad_f(x):
    """Gradient of f(x) = x^2 + 2x + 1 is f'(x) = 2x + 2"""
    return 2*x + 2

gradient_descent(
    f,
    grad_f,
    x_initial,
    alpha=0.1,
    max_iter=100
)
```

Example

Function

$$x_1^2 + x_2^2$$



Gradient

$$\nabla f(x) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2} \right) = (2x_1, 2x_2)$$

Learning Rate

$$\alpha = 0.1$$

Iteration 1

- Initial Parameter Values:

$$x_1 = 1 \qquad x_2 = 2$$

- Gradient Calculation

$$\nabla f(x) = (2 \times 1, 2 \times 2) = (2, 4)$$

- Parameter Update:

$$x \leftarrow x - \alpha \nabla f(x)$$

$$x_1 \leftarrow 1 - 0.1 \times 2 = 0.8$$
$$x_2 \leftarrow 2 - 0.1 \times 4 = 1.6$$

- Updated Parameter Values:

$$x_1 = 0.8 \qquad x_2 = 1.6$$

Iteration 2

- Current Parameter Values

$$x_1 = 0.8 \qquad x_2 = 1.6$$

- Gradient Calculation:

$$\nabla f(x) = (2 \times 0.8, 2 \times 1.6) = (1.6, 3.2)$$

- Parameter Update:

$$x \leftarrow x - \alpha \nabla f(x)$$

$$x_1 \leftarrow 0.8 - 0.1 \times 1.6 = 0.64$$
$$x_2 \leftarrow 1.6 - 0.1 \times 3.2 = 1.28$$

- Updated Parameter Values:

$$x_1 = 0.64 \qquad x_2 = 1.28$$

Iteration 3

- Current Values

$$x_1 = 0.64 \qquad x_2 = 1.28$$

- Gradient Calculation:

$$\nabla f(x) = (2 \times 0.64, 2 \times 1.28) = (1.28, 2.56)$$

- Parameter Update:

$$x \leftarrow x - \alpha \nabla f(x)$$

$$x_1 \leftarrow 0.64 - 0.1 \times 1.28 = 0.512$$
$$x_2 \leftarrow 1.28 - 0.1 \times 2.56 = 1.024$$

- Updated Parameter Values:

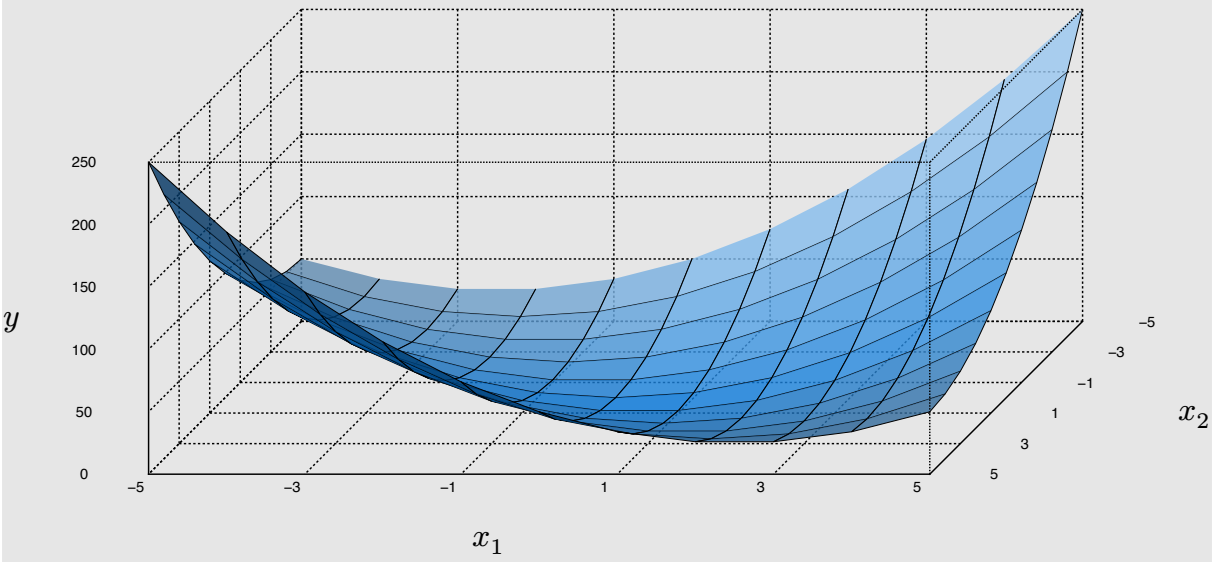
$$x_1 = 0.512 \qquad x_2 = 1.024$$

Example

Problem Setup

Minimize the quadratic function:

$$\min_{x \in \mathbb{R}^2} \quad f(x) = 4x_1^2 - 4x_1x_2 + 2x_2^2 \quad \text{where} \quad x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$



Gradient of $f(x)$

The gradient is:

$$\nabla f(x) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \\ \frac{\partial f}{\partial x_2} \end{bmatrix}$$

Step 1: Compute $\frac{\partial f}{\partial x_1}$

Take the derivative of each term with respect to x_1 (treat x_2 as a constant):

- $\frac{\partial}{\partial x_1}(4x_1^2) = 8x_1$
- $\frac{\partial}{\partial x_1}(-4x_1x_2) = -4x_2$
- $\frac{\partial}{\partial x_1}(2x_2^2) = 0$

So:

$$\frac{\partial f}{\partial x_1} = 8x_1 - 4x_2$$

Step 2: Find $\frac{\partial f}{\partial x_2}$

Take the derivative of each term with respect to x_2 (treat x_1 as a constant):

- $\frac{\partial}{\partial x_2}(4x_1^2) = 0$
- $\frac{\partial}{\partial x_2}(-4x_1x_2) = -4x_1$
- $\frac{\partial}{\partial x_2}(2x_2^2) = 4x_2$

So:

$$\frac{\partial f}{\partial x_2} = -4x_1 + 4x_2$$

So the gradient of $f(x)$ is:

$$\nabla f(x) = \begin{bmatrix} 8x_1 - 4x_2 \\ -4x_1 + 4x_2 \end{bmatrix}$$

Optimal Solution

$$x^* = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$
$$f(x^*) = 0$$

Gradient Descent Iterations

Iteration 1

- Initial guess:

$$x^0 = \begin{bmatrix} 2 \\ 3 \end{bmatrix} \quad \Rightarrow \quad f(x^0) = 4(2^2) - 4(2)(3) + 2(3^2) = 10$$

- Gradient at x^0:

$$\nabla f(x^0) = \begin{bmatrix} 8(2) - 4(3) \\ -4(2) + 4(3) \end{bmatrix} = \begin{bmatrix} 4 \\ 4 \end{bmatrix}$$

- Line Search:

$$x(\alpha_0) = x^0 - \alpha_0 \nabla f(x^0) = \begin{bmatrix} 2 \\ 3 \end{bmatrix} - \alpha_0 \begin{bmatrix} 4 \\ 4 \end{bmatrix}$$
$$= \begin{bmatrix} 2 - 4\alpha_0 \\ 3 - 4\alpha_0 \end{bmatrix}$$

$$\begin{aligned} f(x(\alpha_0)) &= 4{x_1}^2 - 4x_1x_2 + 2{x_2}^2 \\ &= 4(2 - 4\alpha_0)^2 - 4(2 - 4\alpha_0)(3 - 4\alpha_0) + 2(3 - 4\alpha_0)^2 \\ &= 32\alpha_0^2 - 32\alpha_0 + 10 \end{aligned}$$

- Minimizing:

$$\alpha_0 = \operatorname{argmin}\ f(x(\alpha))$$

$$\frac{\partial}{\partial \alpha} f(x(\alpha_0)) = \frac{\partial}{\partial \alpha}(32\alpha_0^2 - 32\alpha_0 + 10) = 0$$

$$= 64\alpha_0 - 32 = 0$$

$$\alpha_0 = \frac{1}{2}$$

- Update Step:

$$x^1 = x^0 - a_0 \nabla f(x^0) = \begin{bmatrix} 2 \\ 3 \end{bmatrix} - \frac{1}{2}\begin{bmatrix} 4 \\ 4 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

- Check Progress:

1. New point:

$$x^1 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

2. Function value decreases $f\left(x^{k+1}\right) < f\left(x^k\right)$

$$f(x^1) = 2 < f(x^0) = 10$$

3. Gradient at new point:

$$\nabla f(x^1) = \begin{bmatrix} 8(0) - 4(1) \\ -4(0) + 4(1) \end{bmatrix} = \begin{bmatrix} -4 \\ 4 \end{bmatrix}$$

4. Gradient magnitude $\|\nabla f\left(x^{k+1}\right)\| < \|\nabla f\left(x^k\right)\|$

$$\|\nabla f(x^1)\| = 4\sqrt{2} = \|\nabla f(x^0)\|$$

Note: Gradient magnitude stays the same in this iteration due to the specific structure of this quadratic function

Iteration 2

- New Point:

$$x^1 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

- Gradient at x_1:

$$\nabla f(x^1) = \begin{bmatrix} 8(0) - 4(1) \\ -4(0) + 4(1) \end{bmatrix} = \begin{bmatrix} -4 \\ 4 \end{bmatrix}$$

$$a_1 = \operatorname{argmin}\ f(x(\alpha_1))$$

- Line search:

$$x(\alpha_1) = x^1 - \alpha_1 \nabla f(x^1) = \begin{bmatrix} 0 \\ 1 \end{bmatrix} - \alpha_1 \begin{bmatrix} -4 \\ 4 \end{bmatrix}$$

$$= \begin{bmatrix} 4\alpha_1 \\ 1 - 4\alpha_1 \end{bmatrix}$$

$$\begin{aligned} f(x(\alpha_1)) &= 4{x_1}^2 - 4x_1x_2 + 2{x_2}^2 \\ &= 4(4\alpha_1)^2 - 4(4\alpha_1)(1 - 4\alpha_1) + 2(1 - 4\alpha_1)^2 \\ &= 160\alpha_1^2 - 32\alpha_1 + 2 \end{aligned}$$

- Minimize:

$$\alpha_1 = \operatorname{argmin}\ f(x(\alpha_1)) = \frac{1}{10}$$

$$\frac{\partial}{\partial \alpha} f(x(\alpha_1)) = \frac{\partial}{\partial \alpha}(160\alpha_1^2 - 32\alpha_1 + 2) = 0$$

$$= 320\alpha_1 - 32 = 0$$

$$\alpha_1 = \frac{1}{10}$$

- Update Step:

$$x^2 = x^1 - \alpha_1 \nabla f(x^1) = \begin{bmatrix} 0 \\ 1 \end{bmatrix} - \frac{1}{10} \begin{bmatrix} -4 \\ 4 \end{bmatrix} = \begin{bmatrix} 0.4 \\ 0.6 \end{bmatrix}$$

- Improvement:

1. New point:

$$x_2 = \begin{bmatrix} 0.4 \\ 0.6 \end{bmatrix}$$

2. Function value decreases $f(x^{k+1}) < f(x^k)$

$$f(x^2) = 0.4 < f(x^1) = 2$$

3. Gradient at new point:

$$\nabla f(x_2) = \begin{bmatrix} 8(0.4) - 4(0.6) \\ -4(0.4) + 4(0.6) \end{bmatrix} = \begin{bmatrix} 0.8 \\ 0.8 \end{bmatrix}$$

4. Gradient magnitude $\|\nabla f(x^{k+1})\| < \|\nabla f(x^k)\|$

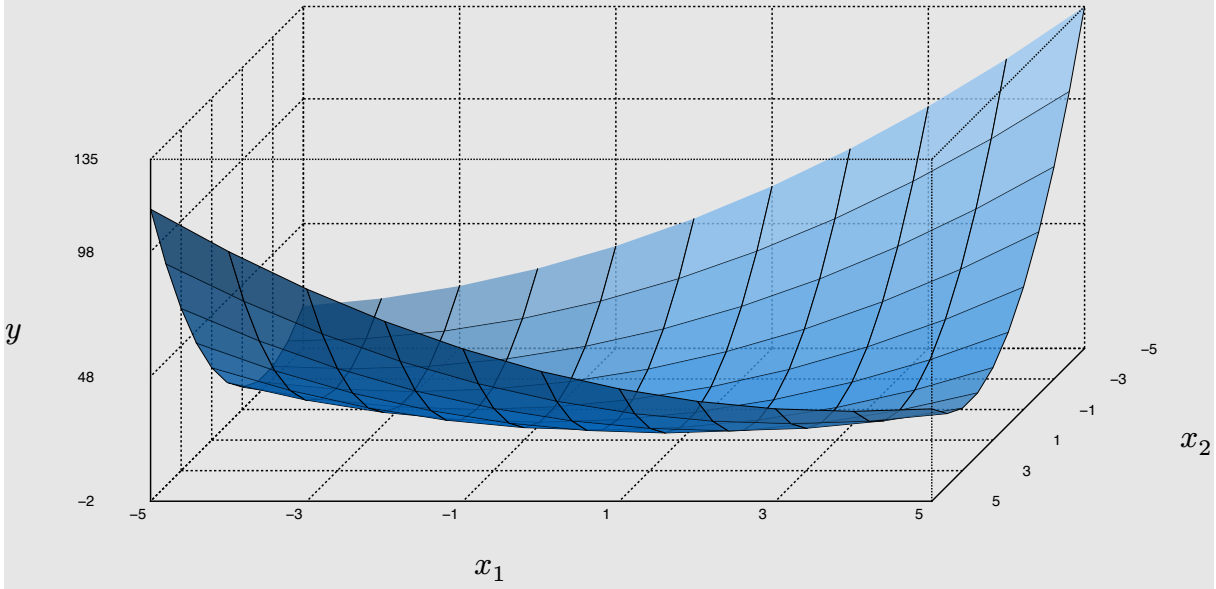
$$\|\nabla f(x^2)\| = \|(0.8, 0.8)\| = \frac{4\sqrt{2}}{5}$$

Example

Problem Setup

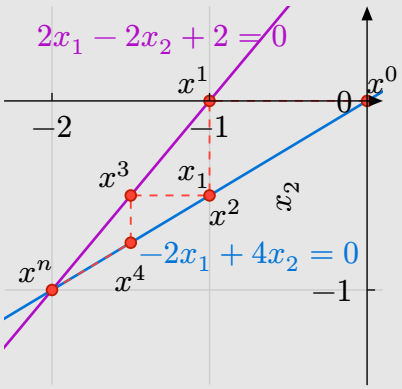
Minimize the quadratic function:

$$\min_{x \in \mathbb{R}^2} \quad f(x) = x_1^2 - 2x_1x_2 + 2x_2^2 + 2x_1 \quad \text{where} \quad x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$



Gradient of $f(x)$

$$\nabla f(x_1, x_2) = \begin{bmatrix} 2x_1 - 2x_2 + 2 \\ -2x_1 + 4x_2 \end{bmatrix}$$



181.15. Newton's Method

Newton's method is an iterative numerical technique to find roots of a function (i.e., solutions to $f(x) = 0$).

Step 1. At each step, approximate $f(x)$ by its tangent line at the current guess x_k:

$$f(x) \approx f(x_k) + f'(x_k)(x - x_k)$$

Step 2. Set this approximation equal to 0:

$$0 = f(x_k) + f'(x_k)(x - x_k)$$

Step 3. Solve for x:

$$x = x_k - \frac{f(x_k)}{f'(x_k)}$$

Step 4. This gives the update rule:

$$x = x_k - \frac{f(x_k)}{f'(x_k)}$$

181.15.1. Newton's Method for Nonlinear Optimization

Used to find the **roots of the first derivative** $f'(x) = 0$, which correspond to the **local minima/maxima of the original function** $f(x)$.

Finding **stationary points** of a function $f(x)$, i.e., points where the gradient (derivative) is zero:

- 1D: $f'(x) = 0$
- nD: $\nabla f(\mathbf{x}) = \mathbf{0}$

It can be derived from either linear approximation of the gradient or quadratic approximation of the function

181.15.1.1. 1D Linear Approximation

1. Start with the first-order (linear) Taylor expansion of the derivative around x_k:

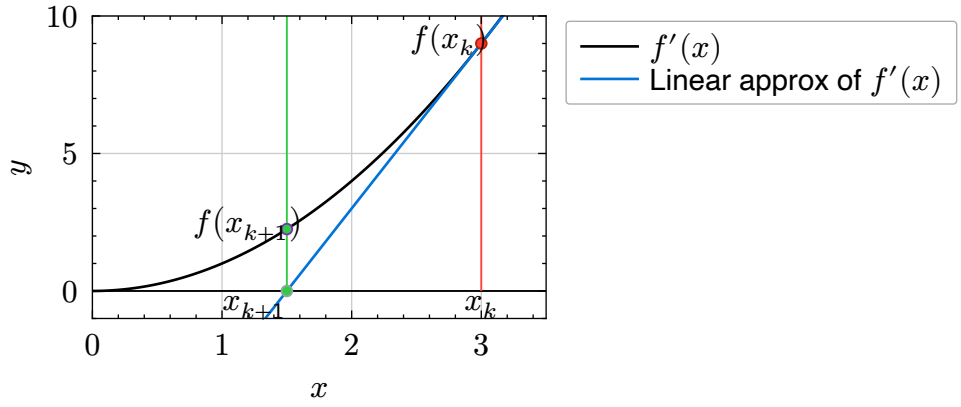
$$f'(x_{k+1}) \approx f'(x_k) + f''(x_k)(x - x_k)$$

2. Stationary point condition: Set the derivative to zero to find a candidate minimum or maximum

$$0 = f'(x_k) + f''(x_k)(x - x_k)$$

3. Solve for the next iterate x_{k+1}:

$$x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}$$



181.15.1.2. nD Linear Approximation

1. Start with the first-order (linear) Taylor expansion of the gradient around x_k:

$$\nabla f(\mathbf{x}_{k+1}) \approx \nabla f(\mathbf{x}_k) + H_k(\mathbf{x}_{k+1} - \mathbf{x}_k)$$

where $H_k = \nabla^2 f(\mathbf{x}_k)$ is the Hessian matrix

2. Stationary point condition: Set the gradient to zero to find a candidate minimum or maximum:

$$\mathbf{0} = \nabla f(\mathbf{x}_k) + H_k(\mathbf{x} + (k+1) - \mathbf{x}_k)$$

3. Solve for the next iterate $\mathbf{x}_{k+1}$:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - H_k^{-1}\nabla f(\mathbf{x}_k)$$

181.15.1.3. 1D Quadratic Approximation

1. Start with the second-order (quadratic) Taylor expansion of the function around x_k:

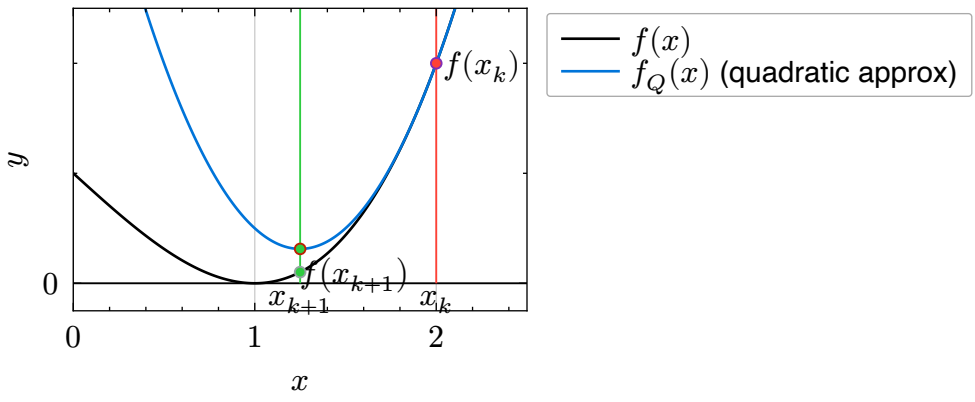
$$f_Q(x) = f(x_k) + f'(x_k)(x - x_k) + \frac{1}{2}f''(x_k)(x - x_k)^2$$

2. Minimize the quadratic approximation: Set the derivative of $f_Q(x)$ to zero:

$$\frac{\mathrm{d}}{\mathrm{d}x}f_Q(x) = f'(x_k) + f''(x_k)(x - x_k) = 0$$

3. Solve for the next iterate x_{k+1}:

$$x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}$$



181.15.1.4. nD Quadratic Approximation

1. Start with the second-order (quadratic) Taylor expansion of the function around x_k:

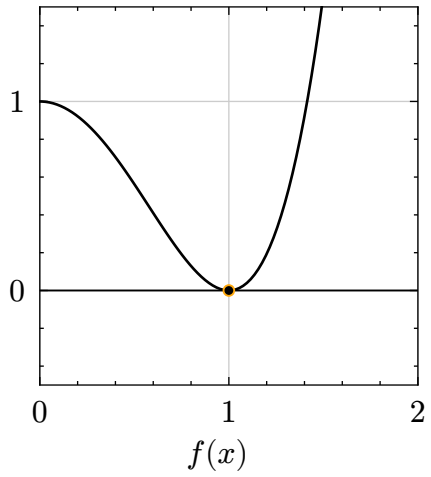
$$m_k(\mathbf{p}) = f(\mathbf{x}_k) + \nabla f(\mathbf{x}_k)^T \ \mathbf{p} + \frac{1}{2}\mathbf{p}^T H_k \ \mathbf{p}$$
$$\mathbf{p} = \mathbf{x} - \mathbf{x}_k$$

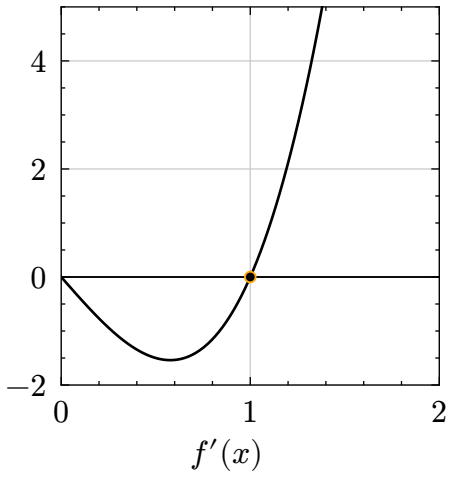
2. Minimize the quadratic approximation: Set the gradient of $m_k(\mathbf{p})$ with respect to $\mathbf{p}$ to zero:

$$\nabla m_k(\mathbf{p}) = \nabla f(\mathbf{x}_k) + H_k \ \mathbf{p} = \mathbf{0}$$

3. Solve for the step $\mathbf{p}_k$ and next iterate $\mathbf{x}_{k+1}$:

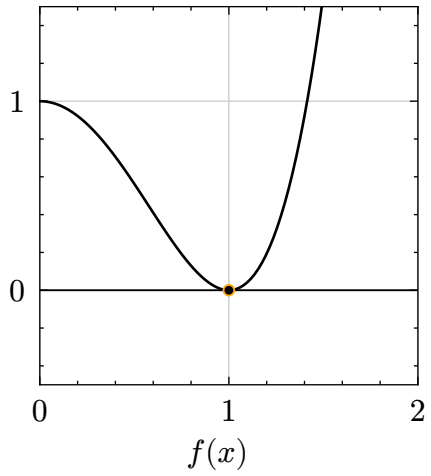
$$\mathbf{p}_k = -H_k^{-1}\nabla f(\mathbf{x}_k), \qquad \mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{p}_k$$

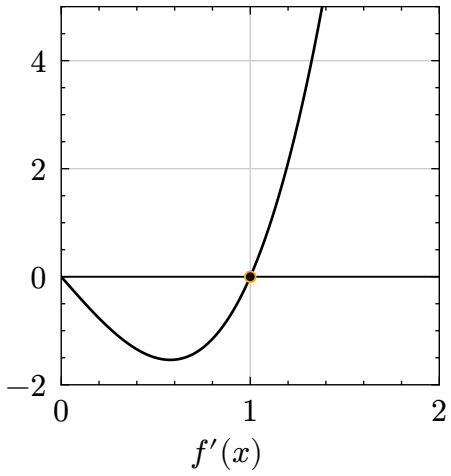




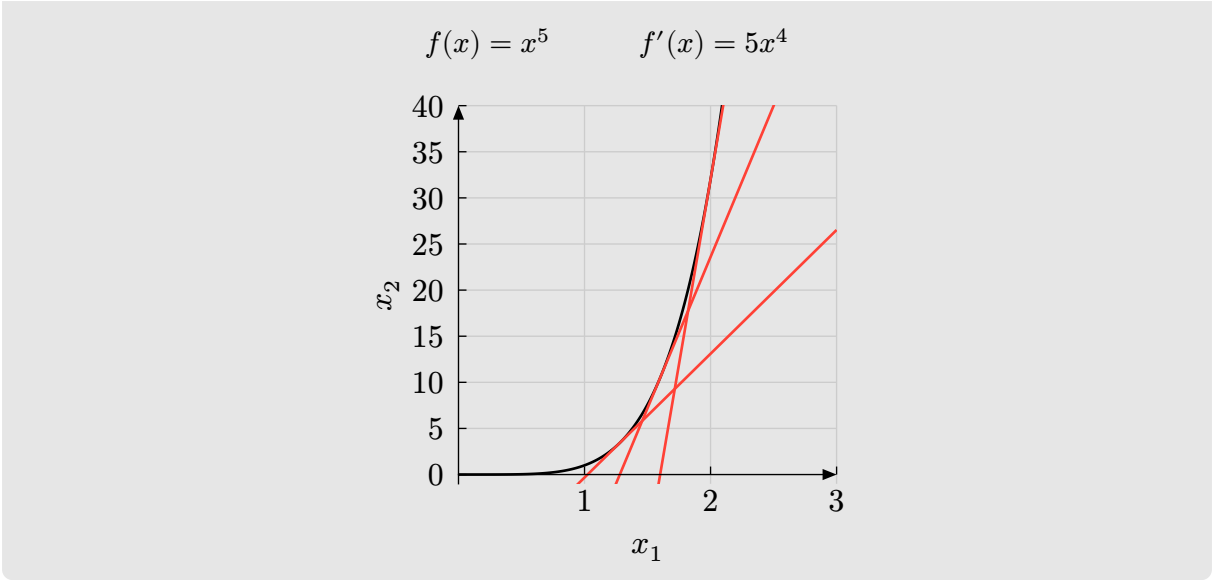
181.15.1.5. Quadratic Approximation

$$f_Q(x) = f(x_k) + f'(x_k)(x - x_k) + \frac{1}{2}f''(x_k)(x - x_k)^2$$





Example



Hello

181.15.2. Multi-Dimensional

second-order iterative algorithm

uses both the

- first derivative (gradient)
- second derivative (Hessian)

Step 1. Start with initial guess x_0

Step 2. At each iteration k:

- Compute gradient $\nabla f(x_k)$
- Compute Hessian $\nabla^2 f(x_k)$
- Update:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - [\nabla^2 f(\mathbf{x}_k)]^{-1} \nabla f(\mathbf{x}_k)$$

$$\nabla f(\mathbf{x}_k) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(\mathbf{x}_k) \\ \frac{\partial f}{\partial x_2}(\mathbf{x}_k) \\ \vdots \\ \frac{\partial f}{\partial x_n}(\mathbf{x}_k) \end{bmatrix} \in \mathbb{R}^n \qquad \nabla^2 f(\mathbf{x}_k) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2}(\mathbf{x}_k) & \frac{\partial^2 f}{\partial x_1 \partial x_2}(\mathbf{x}_k) & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n}(\mathbf{x}_k) \\ \frac{\partial^2 f}{\partial x_2 \partial x_1}(\mathbf{x}_k) & \frac{\partial^2 f}{\partial x_2^2}(\mathbf{x}_k) & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n}(\mathbf{x}_k) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1}(\mathbf{x}_k) & \frac{\partial^2 f}{\partial x_n^2 \partial x_2}(\mathbf{x}_k) & \cdots & \frac{\partial^2 f}{\partial x_n^2}(\mathbf{x}_k) \end{bmatrix} \in \mathbb{R}^{n \times n}$$

Instead of computing the inverse explicitly, we solve the linear system:

$$\nabla^2 f(\mathbf{x}_k) \mathbf{p}_k = \nabla f(\mathbf{x}_k)$$

$$\begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2}(\mathbf{x}_k) & \frac{\partial^2 f}{\partial x_1 \partial x_2}(\mathbf{x}_k) & \dots & \frac{\partial^2 f}{\partial x_1 \partial x_n}(\mathbf{x}_k) \\ \frac{\partial^2 f}{\partial x_2 \partial x_1}(\mathbf{x}_k) & \frac{\partial^2 f}{\partial x_2^2}(\mathbf{x}_k) & \dots & \frac{\partial^2 f}{\partial x_2 \partial x_n}(\mathbf{x}_k) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1}(\mathbf{x}_k) & \frac{\partial^2 f}{\partial x_n^2 \partial x_2}(\mathbf{x}_k) & \dots & \frac{\partial^2 f}{\partial x_n^2}(\mathbf{x}_k) \end{bmatrix} \begin{bmatrix} p_1 \\ p_2 \\ \vdots \\ p_n \end{bmatrix} = \begin{bmatrix} \frac{\partial f}{\partial x_1}(\mathbf{x}_k) \\ \frac{\partial f}{\partial x_2}(\mathbf{x}_k) \\ \vdots \\ \frac{\partial f}{\partial x_n}(\mathbf{x}_k) \end{bmatrix}$$

$$\begin{aligned} \frac{\partial^2 f}{\partial x_1^2}(\mathbf{x}_k)p_1 + \frac{\partial^2 f}{\partial x_1 \partial x_2}(\mathbf{x}_k)p_2 + \dots + \frac{\partial^2 f}{\partial x_1 \partial x_n}(\mathbf{x}_k)p_n &= \frac{\partial f}{\partial x_1}(\mathbf{x}_k) \\ \frac{\partial^2 f}{\partial x_2 \partial x_1}(\mathbf{x}_k)p_1 + \frac{\partial^2 f}{\partial x_2^2}(\mathbf{x}_k)p_2 + \dots + \frac{\partial^2 f}{\partial x_2 \partial x_n}(\mathbf{x}_k)p_n &= \frac{\partial f}{\partial x_2}(\mathbf{x}_k) \\ &\vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1}(\mathbf{x}_k)p_1 + \frac{\partial^2 f}{\partial x_n^2 \partial x_2}(\mathbf{x}_k)p_2 + \dots + \frac{\partial^2 f}{\partial x_n^2}(\mathbf{x}_k)p_n &= \frac{\partial f}{\partial x_n}(\mathbf{x}_k) \end{aligned}$$

Then update:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \mathbf{p}_k$$

Example

$$f(x, y) = x^2 + xy + y^2$$

Step 1. Compute Gradient

$$\nabla f(x, y) = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix} = \begin{bmatrix} 2x + y \\ x + 2y \end{bmatrix}$$

Step 2. Compute Hessian

$$\nabla^2 f(x, y) = \begin{bmatrix} \frac{\partial^2 f}{\partial x^2} & \frac{\partial^2 f}{\partial x \partial y} \\ \frac{\partial^2 f}{\partial y \partial x} & \frac{\partial^2 f}{\partial y^2} \end{bmatrix} =$$

Step 0. Setup

We want to minimize a function $f(x, y)$

At some iteration k, we compute:

- Gradient

$$g = \nabla f(x_k, y_k) = \begin{bmatrix} g_1 \\ g_2 \end{bmatrix}$$

- Hessian

$$H = \nabla^2 f(x_k, y_k) = \begin{bmatrix} a & b \\ b & c \end{bmatrix}$$

We need the Newton step vector p from:

$$Hp = g$$

so that the update is:

$$\begin{bmatrix} x_{k+1} \\ y_{k+1} \end{bmatrix} = \begin{bmatrix} x_k \\ y_k \end{bmatrix} - p$$

Step 1. We need the Newton step p from:

$$\begin{bmatrix} a & b \\ b & c \end{bmatrix} \begin{bmatrix} p_1 \\ p_2 \end{bmatrix} = \begin{bmatrix} g_1 \\ g_2 \end{bmatrix}$$

Which is two equations:

$$ap_1 + bp_2 = g_1$$
$$bp_1 + cp_2 = g_2$$

Step 2. Solve without inversion

This is a simple linear system.

For example, solve the first equation for p_1:

$$p_1 = \frac{g_1 - bp_2}{a}$$

Plug into the second:

$$b\left(\frac{g_1 - bp_2}{a}\right) + cp_2 = g_2$$
$$\frac{bg_1}{a} - \frac{b^2}{a}p_2 + cp_2 = g_2$$
$$\left(c = \frac{b^2}{a}\right)p_2 = g_2 - \frac{bg_1}{a}$$
$$p_2 = \frac{ag_2 - bg_1}{ac - b^2}$$

Then substitute back for p_1:

$$p_1 = \frac{g_1 - bp_2}{a}$$

Step 3. Newton update

$$\begin{bmatrix} x_{k+1} \\ y_{k+1} \end{bmatrix} = \begin{bmatrix} x_k \\ y_k \end{bmatrix} - \begin{bmatrix} p_1 \\ p_2 \end{bmatrix}$$

181.16. Heuristic Algorithms

181.17. Duality

181.17.1. Linear Programming Duality

Every linear program (primal) has a **unique** and **symmetric** dual problem. For any primal LP, there is a unique dual, whose dual is the primal.

Solving the dual gives you information about the primal.

181.17.1.1. General Form

Primal

$$\begin{aligned} \min / \max \quad & \mathbf{c}^T \mathbf{x} \\ s.t. \quad & A\,\mathbf{x} \circ \mathbf{b} \\ & \mathbf{x} \diamond 0 \end{aligned}$$

Dual

$$\begin{aligned} \max / \min \quad & \mathbf{b}^T \mathbf{y} \\ s.t. \quad & A^T \mathbf{y} \diamond \mathbf{c} \\ & \mathbf{y} \circ 0 \end{aligned}$$

Primal

$$\begin{aligned} \min / \max \quad & \sum_{j=1}^{n} c_j x_j \\ s.t. \quad & \sum_{j=1}^{n} a_{ij} x_j \circ b_i, \quad i = 1, ..., m \\ & x_j \diamond 0, \quad j = 1, ..., n \end{aligned}$$

Dual

$$\begin{aligned} \max / \min \quad & w = \sum_{i=1}^{m} u_i b_i \\ s.t. \quad & \sum_{i=1}^{m} a_{ji} u_i \diamond c_j, \quad j = 1, ..., n \\ & u_i \circ 0, \quad i = 1, ..., m \end{aligned}$$

Primal

$$\begin{aligned} \min / \max \quad & c_1 x_1 + c_2 x_2 + c_3 x_3 \\ s.t. \quad & a_{11} x_1 + a_{12} x_2 + a_{13} x_3 \geq b_1 \\ & a_{21} x_1 + a_{22} x_2 + a_{23} x_3 \leq b_2 \\ & a_{31} x_1 + a_{32} x_2 + a_{33} x_3 = b_3 \\ & x_1 \geq 0, \quad x_2 \leq 0, \quad x_3 \text{ urs} \end{aligned}$$

Dual

$$\begin{aligned} \max / \min \quad & b_1 y_1 + b_2 y_2 + b_3 y_3 \\ s.t. \quad & a_{11} y_1 + a_{21} y_2 + a_{31} y_3 \geq c_1 \\ & a_{12} y_1 + a_{22} y_2 + a_{32} y_3 \leq c_2 \\ & a_{13} y_1 + a_{23} y_2 + a_{33} y_3 = c_3 \\ & y_1 \leq 0, \quad y_2 \geq 0, \quad y_3 \text{ urs} \end{aligned}$$

181.17.1.2. Standard Form

Primal

$$\begin{aligned} \min / \max \quad & \mathbf{c}^T \mathbf{x} \\ s.t. \quad & A\,\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq 0 \end{aligned}$$

Dual

$$\begin{aligned} \max / \min \quad & \mathbf{b}^T \mathbf{y} \\ s.t. \quad & A^T \mathbf{y} \geq \mathbf{c} \\ & \mathbf{y} \text{ urs} \end{aligned}$$

Primal

Dual

$$\begin{aligned} \min / \max \quad & \sum_{j=1}^{n} c_j x_j \\ s.t. \quad & \sum_{j=1}^{n} a_{ij} x_j = b_i, \quad i = 1, ..., m \\ & x_j \geq 0, \quad j = 1, ..., n \end{aligned}$$

$$\begin{aligned} \max / \min \quad & w = \sum_{i=1}^{m} y_i b_i \\ s.t. \quad & \sum_{i=1}^{m} a_{ji} y_i \leq c_j, \quad j = 1, ..., n \\ & y_i \text{ urs}, \quad i = 1, ..., m \end{aligned}$$

Primal

$$\begin{aligned} \min / \max \quad & c_1 x_1 + c_2 x_2 + c_3 x_3 \\ s.t. \quad & a_{11} x_1 + a_{12} x_2 + a_{13} x_3 = b_1 \\ & a_{21} x_1 + a_{22} x_2 + a_{23} x_3 = b_2 \\ & a_{31} x_1 + a_{32} x_2 + a_{33} x_3 = b_3 \\ & x_1, x_2, x_3 \geq 0 \end{aligned}$$

Dual

$$\begin{aligned} \max / \min \quad & b_1 y_1 + b_2 y_2 + b_3 y_3 \\ s.t. \quad & a_{11} y_1 + a_{21} y_2 + a_{31} y_3 \geq c_1 \\ & a_{12} y_1 + a_{22} y_2 + a_{32} y_3 \geq c_2 \\ & a_{13} y_1 + a_{23} y_2 + a_{33} y_3 \geq c_3 \\ & y_1, y_2, y_3 \text{ urs} \end{aligned}$$

Obj	max	min	Obj
Constraint	$\leq$ $\geq$ $=$	≥ 0 ≤ 0 urs.	Variable
Variable	≥ 0 ≤ 0 urs.	$\leq$ $\geq$ $=$	Constraint

Example

Minimization

Primal

$$\begin{aligned} \min \quad & 3x_1 + 2x_2 \\ s.t. \quad & x_1 + 2x_2 \geq 4 \\ & 2x_1 + x_2 \geq 5 \\ & x_1 \leq 0, \quad x_2 \text{ urs} \end{aligned}$$

Dual

$$\begin{aligned} \max \quad & 4y_1 + 5y_2 \\ s.t. \quad & y_1 + 2y_2 \geq 3 \\ & 2y_1 + y_2 = 2 \\ & y_1 \leq 0, \quad y_2 \leq 0 \end{aligned}$$

Maximization

Primal

$$\begin{aligned} \max \quad & 3x_1 + 2x_2 \\ s.t. \quad & x_1 + 2x_2 \leq 4 \\ & 2x_1 + x_2 \leq 5 \\ & x_1 \geq 0, \quad x_2 \text{ urs} \end{aligned}$$

Dual

$$\begin{aligned} \min \quad & 4y_1 + 5y_2 \\ s.t. \quad & y_1 + 2y_2 \geq 3 \\ & 2y_1 + y_2 = 2 \\ & y_1 \geq 0, \quad y_2 \geq 0 \end{aligned}$$

181.17.1.3. Weak Duality

1. Dual objective gives a **lower bound** for a **minimization** primal

$$c^T x \geq b^T y$$

2. Dual objective gives an **upper bound** for a **maximization** primal

$$c^T x \leq b^T y$$

Sufficiency of optimality

If $\bar{x}$ and $\bar{y}$ are primal and dual feasible and $c^T \bar{x} \leq \bar{y}^T b$, then $\bar{x}$ and $\bar{y}$ are primal and dual optimal

Example

Given a primal feasible solution $\bar{x}$, if we find a dual feasible solution so that their objective values are identical, $\bar{x}$ is optimal (strong duality)

181.17.1.4. Dual Oprimal solution

If we have solved the primal LP, the dual optimal solution can be obtained

If $\bar{x}$ is primal optimal with basis B, then $\bar{y}^T = c_B^T A_B^{-1}$ is dual optimal

181.17.1.5. Strong Duality

If both primal and dual are feasible and at least one has an optimal solution:

$\bar{x}$ and $\bar{y}$ are primal and dual optimal iff $\bar{x}$ and $\bar{y}$ are primal and dual feasible and

$$c^T \bar{x} = b^T \bar{y}$$

Example

Primal

$$\begin{array}{lrcrcrcl} \max & x_1 & & & & & & \\ s.t. & 2x_1 & - & x_2 & \leq & 4 & & \\ & 2x_1 & + & x_2 & \leq & 8 & & \\ & & & x_2 & \leq & 3 & & \\ & x_1, x_2 \geq 0 & & & & & & \end{array} \quad \Leftrightarrow \quad \begin{array}{lrcrcrcl} \min & 4y_1 & + & 8y_2 & + & 3y_3 & & \\ s.t. & 2y_1 & + & 2y_2 & & & \geq & 1 \\ & -y_1 & + & y_2 & + & y_3 & \geq & 1 \\ & y_1, y_2, y_3 \geq 0 & & & & & & \end{array}$$

Dual

Using the simplex method, we obtain the optimal tableau:

–1	0	0	0	0	0
2	–1	1	0	0	$x_3 = 4$
2	1	0	1	0	$x_4 = 8$
0	1	0	0	1	$x_5 = 3$

$\rightarrow \quad ... \quad \rightarrow$

0	0	1/4	1/4	0	3
1	0	1/4	1/4	0	$x_1 = 3$
0	1	–1/2	1/2	0	$x_2 = 2$
0	0	1/2	–1/2	1	$x_5 = 1$

- The associated optimal basis is $B = (1, 2, 5)$
- The optimal solution is $\bar{x} = (3, 2)$
- The associated objective value is $z^* = 3$

For the standard form primal LP:

$$c^T = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \end{bmatrix} \qquad A = \begin{bmatrix} 2 & -1 & 1 & 0 & 0 \\ 2 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 \end{bmatrix}$$

Given $x_B = (x_1, x_2, x_5)$ and $x_N = (x_2, x_4)$:

$$c_B^T = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix} \qquad A_B = \begin{bmatrix} 2 & -1 & 0 \\ 2 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\bar{y} = c_B^T A_B^{-1} = [1 \quad 0 \quad 0] \begin{bmatrix} \frac{1}{4} & \frac{1}{4} & 0 \\ -\frac{1}{2} & \frac{1}{2} & 0 \\ \frac{1}{2} & -\frac{1}{2} & 1 \end{bmatrix} = \begin{bmatrix} \frac{1}{4} & \frac{1}{4} & 0 \end{bmatrix}$$

For $\bar{y} = \left(\frac{1}{4}, \frac{1}{4}, 0\right)$:

- It is dual feasible: $2\left(\frac{1}{4}\right) + 2\left(\frac{1}{4}\right) > 1$ and $-\frac{1}{4} + \frac{1}{4} + 0 \geq 0$
- Its dual objective value $w = 4\left(\frac{1}{4}\right) + 8\left(\frac{1}{4}\right) = 3 = z^*$

Therefore $\bar{y}$ is **dual optimal**

Example

Minimization

Primal

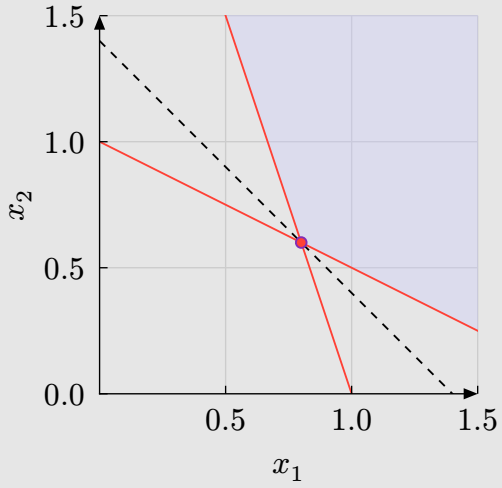
$$\begin{aligned} \min \quad & x_1 + x_2 \\ s.t. \quad & x_1 + 2x_2 \geq 2 \\ & 3x_1 + x_2 \geq 3 \\ & x_1, x_2 \geq 0 \end{aligned}$$

Dual

$$\begin{aligned} \max \quad & 2y_1 + 3y_2 \\ s.t. \quad & y_1 + 3y_2 \leq 1 \\ & 2y_1 + y_2 \leq 1 \\ & y_1, y_2 \geq 0 \end{aligned}$$

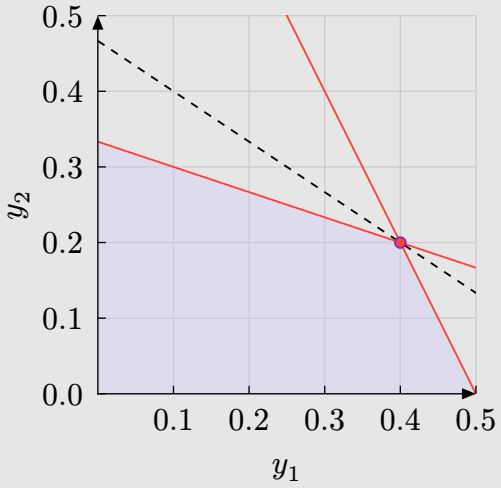
Strong Duality

$$c^T x^* = b^T y^*$$



$x^* = (0.8, 0.6)$

$z^* = 1.4$



$y^* = (0.4, 0.2)$

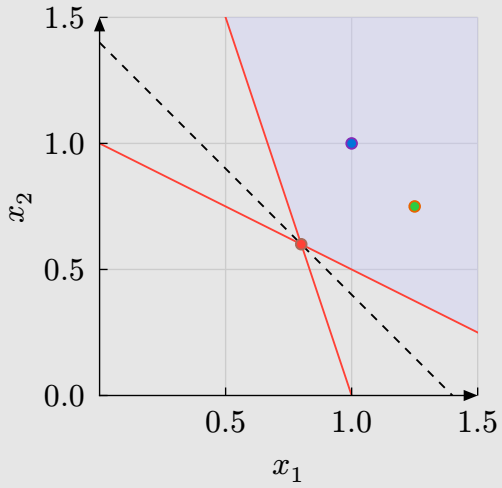
$z^* = 1.4$

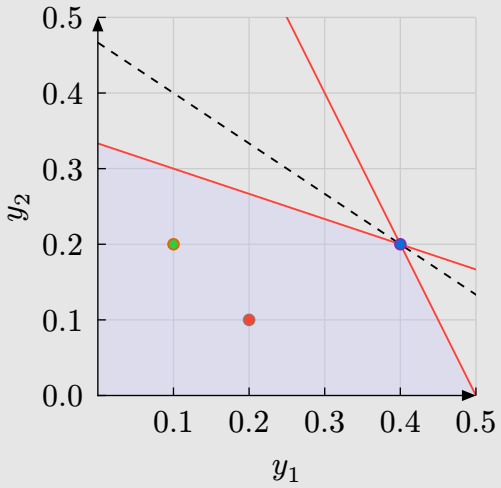
Weak Duality

$$c^T x > b^T y$$

if

$$x \neq x^* \text{ or } y \neq y^*$$





$$x^* = (0.8, 0.6) \qquad y = (0.2, 0.1)$$
$$z^* = 1.4 \quad > \quad w = 1$$

$$x = (1, 1) \qquad y^* = (0.4, 0.2)$$
$$z = 2 \quad > \quad w^* = 1.4$$

$$x = (1, 1) \qquad y = (0.4, 0.2)$$
$$z = 2 \quad > \quad w = 1.4$$

Minimization

Primal

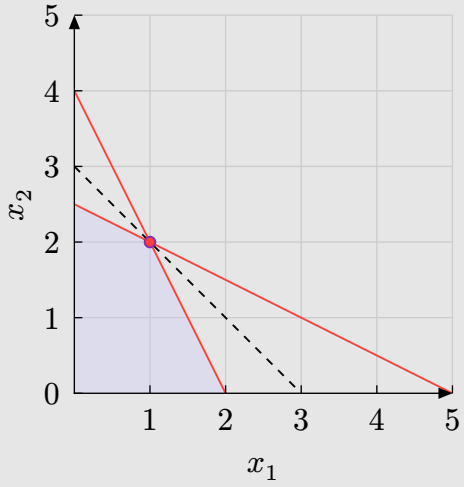
Dual

$$\begin{aligned} \max \quad & x_1 + x_2 \\ s.t. \quad & 2x_1 + x_2 \leq 4 \\ & x_1 + 2x_2 \leq 5 \\ & x_1, x_2 \geq 0 \end{aligned}$$

$$\begin{aligned} \min \quad & 4y_1 + 5y_2 \\ s.t. \quad & 2y_1 + y_2 \geq 1 \\ & y_1 + 2y_2 \geq 1 \\ & y_1, y_2 \geq 0 \end{aligned}$$

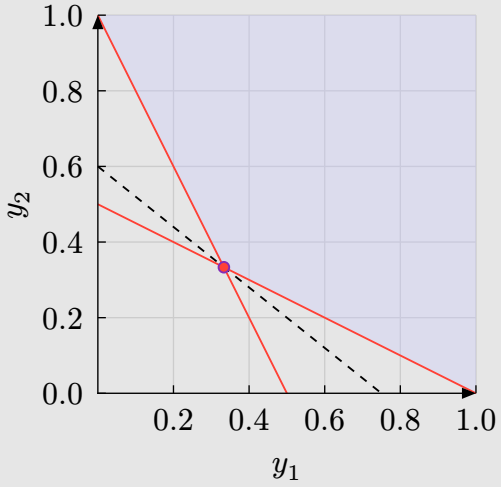
Strong Duality

$$c^T x^* = b^T y^*$$



$x^* = (1, 2)$

$z^* = 3$



$y^* = \left(\frac{1}{3}, \frac{1}{3}\right)$

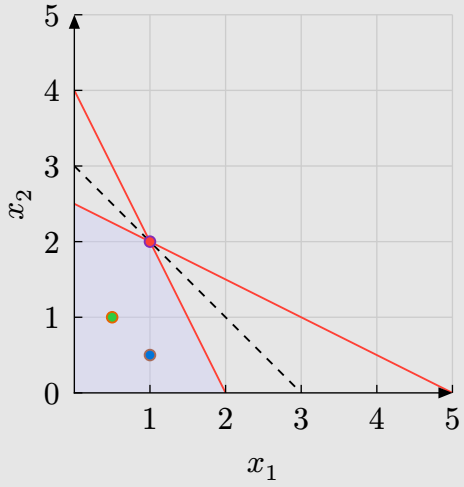
$z^* = 3$

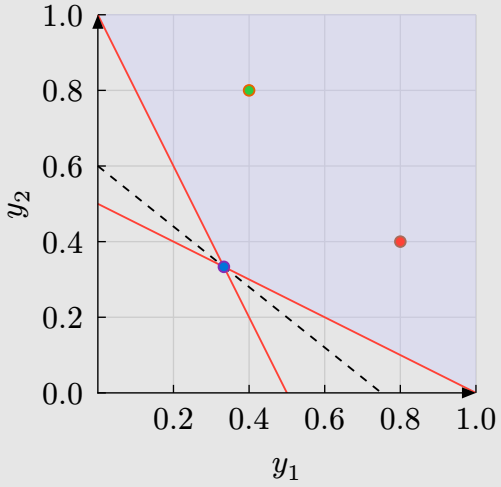
Weak Duality

$$c^T x < b^T y$$

if

$x \neq x^*$ or $y \neq y^*$





$$x^* = (1, 2) \qquad y = (0.8, 0.4)$$
$$z^* = 3 \quad < \quad w = 5.6$$

$$x = (1, 0.5) \qquad y^* = \left(\frac{1}{3}, \frac{1}{3}\right)$$
$$z = 1.5 \quad < \quad w^* = 3$$

$$x = (0.5, 1) \qquad y = (0.4, 0.8)$$
$$z = 1.5 \quad < \quad w = 5.6$$

181.17.1.6. Feasibility-Infeasibility Certificates

- If the primal is **infeasible**, the dual is either **infeasible** or **unbounded**
- If the primal is **unbounded**, the dual is necessarily **infeasible**

Primal	Dual		
	Infeasible	Unbounded	Finitely Optimal
Infeasible	✅	✅	❌
Unbounded	✅	❌	❌
Finitely Optimal	❌	❌	✅

- ✅ means possible and ❌ means impossible
- Primal unbounded $\rightarrow$ no upper bound $\rightarrow$ dual infeasible
- Primal finitely optimal $\rightarrow$ finite objective value $\rightarrow$ dual finitely optimal
- Primal infeasible $\rightarrow$ dual infeasible or unbounded

Example

Infeasible Primal $\rightarrow$ Infeasible Dual

Primal

$$\begin{aligned} \min \quad & 2x_1 + 2x_2 \\ s.t. \quad & x_1 + x_2 \geq 5 \\ & x_1 + x_2 \leq 3 \\ & x_1, x_2 \geq 0 \end{aligned}$$

Dual

$$\begin{aligned} \max \quad & y_1 + y_2 \\ s.t. \quad & y_1 + y_2 \leq 2 \\ & y_1 + y_2 \leq 2 \\ & y_1, y_2 \geq 0 \end{aligned}$$

Infeasible Primal $\rightarrow$ Unbounded Dual

Unbounded Primal $\rightarrow$ Infeasible Primal

181.17.1.7. Strong and Weak Duality

Problem Type	Duality	Weak Duality	Strong Duality
Min Primal **Max** Dual	$c^T x \geq b^T y$	$c^T x > b^T y$ if $x \neq x^*$ or $y \neq y^*$	$c^T x^* = b^T y^*$
Max Primal **Min** Dual	$c^T x \leq b^T y$	$c^T x < b^T y$ if $x \neq x^*$ or $y \neq y^*$	$c^T x^* = b^T y^*$

181.17.1.8. Complimentary Slackness

Slack variables of the dual LP

$$\begin{array}{lll} \max & c^T x & \\ s.t. & Ax = b & \Leftrightarrow \\ & x \geq 0 & \end{array} \qquad \begin{array}{lll} \min & y^T b & \\ s.t. & y^T A \geq c^T & \\ & x \text{ urs} & \end{array}$$

$$\begin{array}{lll} \min & y^T b & \\ s.t. & y^T A - v^T = c^T & \\ & v \geq 0 & \end{array}$$

$\bar{x}$ and $(\bar{y}, \bar{v})$ are primal and dual optimal iff they are feasible and $\bar{v}^T \bar{x} = \mathbf{0}$

Proof

We have

$$\begin{aligned} c^T \bar{x} &= (\bar{y}^T A - \bar{v}^T)\bar{x} \\ &= \bar{y}^T A \bar{x} - \bar{v}^T \bar{x} \\ &= \bar{y}^T b - \bar{v}^T \bar{x} \end{aligned}$$

Therefore $\bar{v}^T \bar{x} = 0$ iff $c^T \bar{x} = \bar{y}^T b$, i.e., $\bar{x}$ and $(\bar{y}, \bar{v})$ are primal and dual optimal according to strong duality.

- Note that $v^T \bar{x} = 0$ iff $\bar{v}_i \bar{x}_i = 0$ for all i as $\bar{x}_i \geq 0$ and $\bar{v} \geq 0$
- If a dual (primal) constraint is **nonbinding**, the corresponding primal (dual) variable is **zero**

Example

Primal	$\Leftrightarrow$	Dual

$$
\begin{aligned}
\max\quad & x_1 \\
s.t.\quad & 2x_1 - x_2 \leq 4 \\
& 2x_1 + x_2 \leq 8 \\
& x_2 \leq 3 \\
& x_1, x_2 \geq 0
\end{aligned}
$$

$$
\begin{aligned}
\min\quad & 4y_1 + 8y_2 + 3y_3 \\
s.t.\quad & 2y_1 + 2y_2 \geq 1 \\
& -y_1 + y_2 + y_3 \geq 0 \\
& y_1, y_2, y_3 \geq 0
\end{aligned}
$$

Let s_i and v_j be the slack variables for the primal and dual LPs:

$$
\begin{aligned}
\max\quad & x_1 \\
s.t.\quad & 2x_1 - x_2 + s_1 = 4 \\
& 2x_1 + x_2 + s_2 = 8 \\
& x_2 + s_3 = 3 \\
& x_1, x_2 \geq 0, \quad s_1, s_2, s_3 \geq 0
\end{aligned}
\quad \Leftrightarrow \quad
\begin{aligned}
\min\quad & 4y_1 + 8y_2 + 3y_3 \\
s.t.\quad & 2y_1 + 2y_2 - v_1 = 1 \\
& -y_1 + y_2 + y_3 - v_2 = 0 \\
& y_1, y_2, y_3 \geq 0, \quad v_1, v_2 \geq 0
\end{aligned}
$$

If optimal solution:

$$
\begin{aligned}
x_1 v_1 &= \mathbf{0} \\
x_2 v_2 &= \mathbf{0} \\
&\vdots
\end{aligned}
$$

Similarly, for an optimal solution:

$$
\begin{aligned}
s_1 y_1 &= \mathbf{0} \\
s_2 y_2 &= \mathbf{0} \\
&\vdots
\end{aligned}
$$

Let $(\bar{x}, \bar{s})$ be primal optimal, we have $(\bar{x}, \bar{s}) = (3, 2, 0, 0, 1)$. Let's find the dual optimal solution $(\bar{y}, \bar{v})$ without solving the LP.

According to complimentary slackness $\bar{x_1}, \bar{x_2}, \bar{s_1} > 0$ imply $\bar{v_1} = \bar{v_2} = \bar{y_3} = 0$ since:

$$
\begin{aligned}
\bar{x_1}\bar{x_1} &= 0 \\
\bar{x_2}\bar{x_2} &= 0 \\
\bar{s_3}\bar{y_3} &= 0
\end{aligned}
$$

The two dual functional equalities are reduced to:

$$
\begin{aligned}
2\bar{y_1} + 2\bar{y_2} &= 1 \\
-\bar{y_1} + \bar{y_2} &= 0
\end{aligned}
$$

Solving the above equaltions results in $\bar{y_1} = \frac{1}{4}$ and $\bar{y_2} = \frac{1}{4}$.

$(\bar{y}, \bar{v})$ is then guaranteed to be dual optimal

($z^* = 3 = w^*$)

If a primal solution is positive, the dual slack must be zero and vis versa.

$$
x_i\Big(c_i - \big(A^T y\big)_i\Big) = \mathbf{0} \quad \forall i
$$

If:

- $x_i > 0$: dual constraint is tight (equality holds)
- $x_i = 0$: dual constraint is slack (inequality not binding)

181.17.1.9. Shadow Prices

What if I have an additional unit of a particular resource?

For each resource there is a maximum price we are willing to pay for one additional unit

Definition: Shadow Price

For an LP that has an optimal colution, the **shadow price of a constraint** if the amount the objective value increases when the RHS of that constraint **increases** by 1, **assuming the current optimal basis remains optimal**

Sign or Shadow Price

Objective Function	Constraint		
	$\leq$	$\geq$	$=$
max	≥ 0	≤ 0	Free
min	≤ 0	≥ 0	Free

- If shifting a constraint does not affect the optimal solution, the shadow price is zero. Shadow prices are zero for constraints that are nonbinding at the optimal solution.

Finding all shadow prices:

- m: number of constraints

Example

The solution to the dual program is the shadow price of the primal

What are the shadow prices?

$$\begin{aligned} \min \quad & 6x_1 + 4x_2 \\ s.t. \quad & x_1 + x_2 \geq 2 \\ & 2x_1 + x_2 \geq 1 \\ & x_1, x_2 \geq 0 \end{aligned}$$

Solve the dual LP

$$\begin{aligned} \max \quad & 2y_1 + y_2 \\ s.t. \quad & y_1 + 3y_2 \leq 6 \\ & y_1 + y_2 \leq 4 \\ & y_1, y_2 \geq 0 \end{aligned}$$

The dual optimal solution is $y^* = (4, 0)$

So shadow prices are 4 and 0 respectively

The shadow price is the absolute increase on the objective value given an increase of 1 on the RHS values

Instead of solving m LPs (one for each constraint increase by 1), we can just solve the dual LP

181.18. Multi-Commodity Network Flow

181.18.1. Minimum Cost Network Flow (MCNF)

1. Maximum Flow
2. Shortest Path

IP $\rightarrow$ LP Relaxation $\rightarrow$ Integer Solution

Network

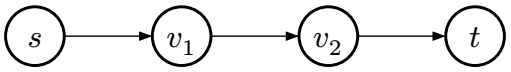
- Nodes (n)
- Edges (u, v)
- Directed
- Undirected

Path

Path from node s to node t is the set of arcs:

$$(s, v_1), (v_1, v_2), (v_2, t)$$

such that s and t are connected



- s: **source**
- t: **destination**

Cycle: Path whose source and destination are the same node

Simple path: A path that is not a cycle

Acyclic network: Network containing no cycle

Flow: actions of sending items through an edge

Flow size: Number of units sent through an edge

Weight: Property on an edge (e.g., distance, cost per unit, etc.)

Weighted network: network whose edges are Weighted

Capacity: Edge constraint

Capacitated network: network whose edges have capacities

Consider the weighted capacitated network $G = (V, E)$

- For node $i \in V$ there is a **supply quantity** b_i
  - $b_i > 0$: i is a **supply** node

- $b_i < 0$: i is a **demand** node
- $b_i = 0$: i is a **transhipment** node
- $\sum_{i \in V} b_i = 0$: Total supplies equal total demands

- For edge $(i, j) \in E$, the weight $c_{ij} \geq 0$ is the **cost** of each unit of flow

Objective: Satisfy all demand while minimizing cost

LP Formulation

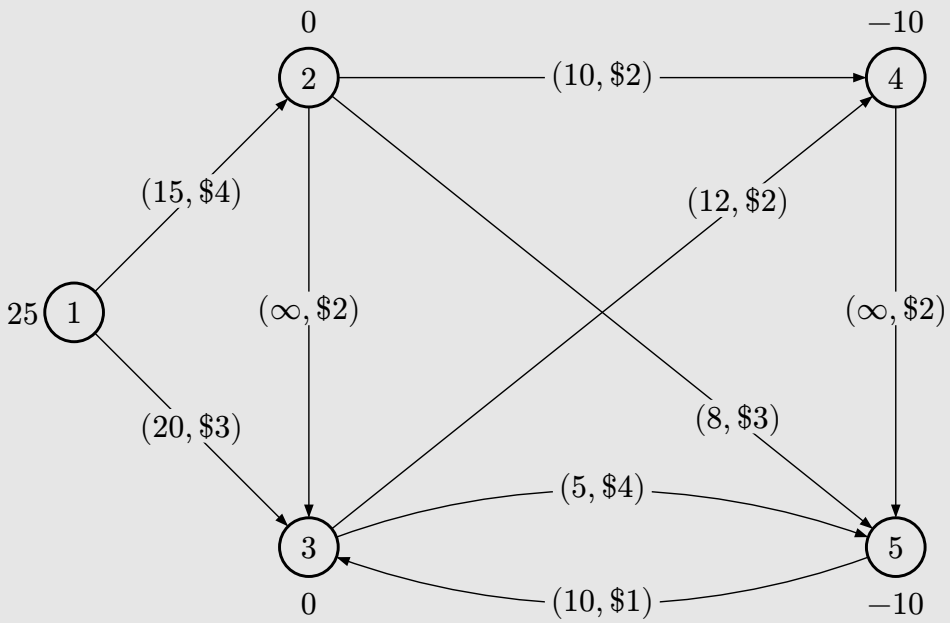
- The number of edges and constraints are equal
- Each variable appears in two constraints (coefficient 1 and coefficient –1)

Example

Decision variables:

x_{ij}: Flow size of edge (i, j)

for all $(i, j) \in E$



Objective function:

$$\min \quad 4x_{12} + 3x_{13} + 2x_{23} + 2x_{24} + 3x_{25} + 2x_{34} + 1x_{35} + 2x_{45} + 1x_{53}$$

Capacity Constraints

$$x_{12} \leq 15$$
$$x_{13} \leq 20$$
$$x_{24} \leq 10$$
$$x_{25} \leq 8$$
$$x_{34} \leq 12$$
$$x_{35} \leq 5$$
$$x_{53} \leq 10$$

Flow Balancing Constraints

- Supply node

$$x_{12} + x_{13} = 25$$

- Transhipment nodes

$$x_{12} = x_{23} + x_{24} + x_{25}$$
$$x_{13} + x_{23} + x_{53} = x_{34} + x_{35}$$

- Demand nodes

$$x_{24} + x_{34} = x_{45} + 10$$
$$x_{25} + x_{35} + x_{45} = x_{53} + 15$$

Complete Formulation

$$\begin{aligned}
\min \quad & 4x_{12} + 3x_{13} + 2x_{23} + 2x_{24} + 3x_{25} + 2x_{34} + x_{35} + 2x_{45} + 4\,x_{53} \\
s.t. \quad & x_{12} + x_{13} = 25 \\
& -x_{12} + x_{23} + x_{24} + x_{25} = 0 \\
& -x_{13} - x_{23} + x_{34} + x_{35} - x_{53} = 0 \\
& -x_{24} - x_{34} + x_{45} = -10 \\
& -x_{25} - x_{35} - x_{45} + x_{53} = -15 \\
& 0 \le x_{12} \le 15 \\
& 0 \le x_{13} \le 20 \\
& 0 \le x_{24} \le 10 \\
& 0 \le x_{25} \le 8 \\
& 0 \le x_{34} \le 12 \\
& 0 \le x_{35} \le 5 \\
& 0 \le x_{53} \le 10
\end{aligned}$$

As long as

- Supply quantities
- Upper bounds

are integers, the solution will be an **integer solution**

The LP relaxation of the IP formulation always gives an integer solution

Total unimodularity gives integer solutions

For a standard form LP $\min\{c^T x \mid Ax = b, x \ge 0\}$, if

- A is totally unimodular and
- $b \in \mathbb{Z}^n$,

then an optimal bfs x^* obtained by the simplex method must satisfy $x^* \in \mathbb{Z}^n$

Proof:

$$x_B = A_B^{-1} b = \frac{1}{\det A_B} A_B^{\text{adj}} b$$

Where:

- A_B^{adj}: adjugate matrix of A_B
- $\left(A_B^{\text{adj}}\right)_{ij}$: determinant of the matrix obtained by removing row j and column i from A_B

If A is totally unimodular, $\det(A_B)$ will be either 1 or –1 for any basis B. x_B is then an integer vector if b is an integer vector.

- If a standard form **LP** has a **totally unimodular coefficient matrix**, an optimal bfs will always be **integer**
- If a standard form **IP** has a **totally unimodular coefficient matrix**, its LP relaxation always gives **integer** solutions

Sufficient conditions for total unimodularity: For matrix A, if:

- all its elements are 1, 0, –1
- each column contains at most two nonzero elements
- rows can be divided into two groups so that for each column two nonzero elements are in the same group if and only if they are different (+ and -)

Then A is totally unimodular

Example

- All its elements are 1, 0, –1
- Each column contains at most two nonzero elements

Constraints:

$$\begin{aligned}
x_{12} + x_{13} &= 25 \\
-x_{12} + x_{23} + x_{24} + x_{25} &= 0 \\
-x_{13} - x_{23} + x_{34} + x_{35} - x_{53} &= 0 \\
-x_{24} - x_{34} + x_{45} &= -10 \\
-x_{25} - x_{35} - x_{45} + x_{53} &= -15
\end{aligned}$$

For simplex:

$$\begin{aligned}
x_{12} + x_{13} &= 25 \\
-x_{12} + x_{23} + x_{24} + x_{25} &= 0 \\
-x_{13} - x_{23} + x_{34} + x_{35} - x_{53} &= 0 \\
+x_{24} + x_{34} - x_{45} &= +10 \\
+x_{25} + x_{35} + x_{45} - x_{53} &= +15
\end{aligned}$$

- Rows can be divided into two groups so that for each column two nonzero elements are in the same group if and only if they are different (+ and -)

$$\left[\begin{array}{ccccccccc} 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -1 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & -1 & -1 & 0 & 0 & 1 & 1 & 0 & -1 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & -1 \end{array}\right]$$

181.18.2. Transportation Problems

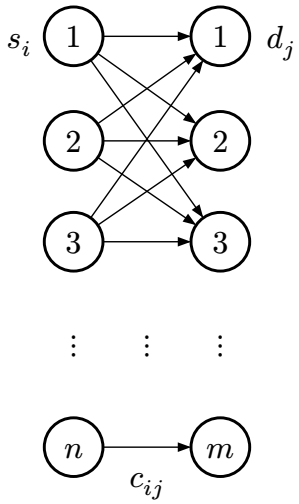
A firm owns n factories that supply one pruduct to m markets

- s_i: Capacity of factory i, $i = 1, ..., n$
- d_j: Demand of market j, $j = 1, ..., m$

Between factory i and market j there is a route

- c_{ij}: Unit cost for shipping one unit from factory i to market j

How to produce and ship the product to fulfill all demands while minimizing the total cost?



if $\sum_{i=1}^{n} s_i = \sum_{j=1}^{n} d_j$

- x_{ij}: Shipping quantity on arc (i, j), $i = 1, ..., n, j = 1, ..., m$

This is a MCNF problem:

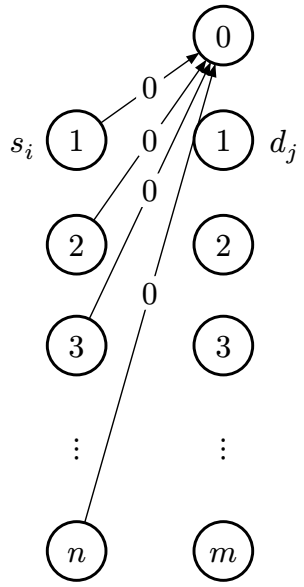
- Factories are supply nodes whose supply quantity is s_i
- Markets are demand nodes whose supply quantity is $-d_j$
- No transhipment nodes
- Arc weights are unit transportation costs c_{ij}
- Arcs have unlimited capacities ($u_{ij} = \infty$)

If capacity is greater than demand:

$$\sum_{i=1}^{n} s_i > \sum_{j=1}^{n} d_j$$

- Create a "virtual market" (market 0) whose demand quantity is $d_0 = \sum_{i=1}^{n} s_i - \sum_{j=1}^{n} d_j$

- Arcs $(i, 0)$ have cost $c_{i0} = 0$
- Shipping to market 0 just means some factory capacity is unused



If different factories have different unit production costs c_i^P

- c_{ij} is updated to $c_{ij} + c_i^P$

If different markets habe different unit retailing costs c_j^R

- c_j is updated to $c_{ij} + c_j^R$

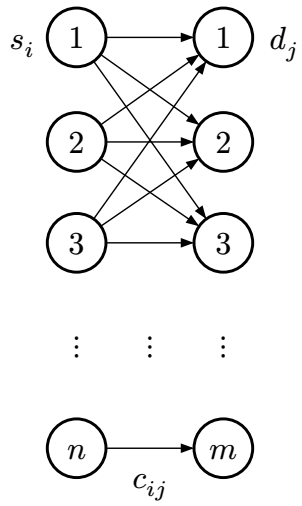
181.18.3. Assignment Problems

- A manager assigns n jobs to m workers
- The assignment is one-to-one (one job to one worker)
    - Jobs cannot be split
- c_{ij}: cost for worker j to complete job i

How to minimize total cost?

Special case of Transportation Problem

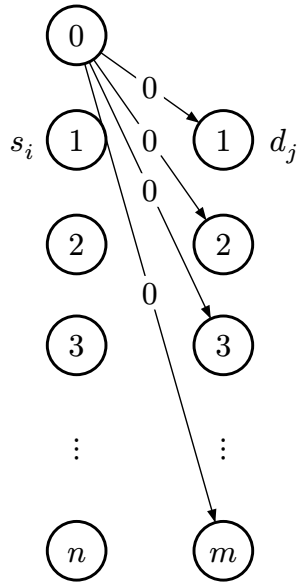
- Jobs are factories and workers are markets
- Each factory produces one item and each market demands one item
- Cost of shipping one item from factory i to maket j is c_{ij}



What if there are fewer jobs then workers:

$$\sum_{i=1}^{n} s_i < \sum_{j=1}^{n} d_j$$

- Create a “virtual job” (job 0)
- Arcs $(i, 0)$ have cost $c_{i0} = 0$
- Assigning job 0 just means some workers are unused



IP Formulation

- I: set of factories / jobs
- J: set of markets / workers

Transportation Problem

Assignment Problem

$$
\begin{aligned}
\min \quad & \sum_{i \in I} \sum_{j \in J} c_{ij} x_{ij} \\
s.t. \quad & \sum_{j=1}^{m} x_{ij} = s_i \quad \forall i \in I \\
& \sum_{i=1}^{n} x_{ij} = d_j \quad \forall j \in J \\
& x_{ij} \in \mathbb{Z}^+ \quad \forall i \in I, j \in J
\end{aligned}
$$

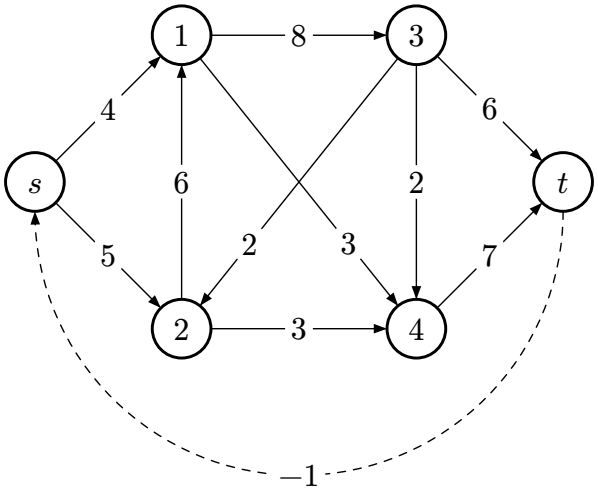
$$
\begin{aligned}
\min \quad & \sum_{i \in I} \sum_{j \in J} c_{ij} x_{ij} \\
s.t. \quad & \sum_{j=1}^{n} x_{ij} = 1 \quad \forall i \in I \\
& \sum_{i=1}^{n} x_{ij} = 1 \quad \forall j \in J \\
& x_{ij} \in \{0, 1\} \quad \forall i \in I, j \in J
\end{aligned}
$$

- These matrices are **totally unimodular**

181.18.4. Transhipment Problems

If there are intermediary nodes in the transportation problem it is a transhipment problem

181.18.5. Maximum Flow Problems



$$
\begin{aligned}
\min \quad & -x_{ts} \\
s.t. \quad & \sum_{(i,k) \in E} x_{ik} - \sum_{(k,j) \in E} x_{kj} = 0 \quad \forall k \in V \\
& x_{ij} \leq u_{ij} \quad \forall (i,j) \in E \\
& x_{ij} \in \mathbb{Z}^+ \quad \forall (i,j) \in E
\end{aligned}
$$

Where:

- x_{ts}: flow size of the
- u_{ij}:

Objective: Maximize number of units sent from s to t given edge capacities

181.18.6. Shortest Path Problems

$$
\begin{aligned}
\min \quad & \sum_{i \in I} \sum_{j \in J} d_{ij} x_{ij} \\
s.t. \quad & \sum_{(s,j) \in E} x_{sj} = 1 \\
& \sum_{(i,t) \in E} x_{it} = 1 \\
& \sum_{(i,k) \in E} x_{ik} - \sum_{(k,j) \in E} x_{kj} = 0 \quad \forall k \in T \\
& x_{ij} \in \{0,1\} \qquad\qquad\qquad\qquad \forall (i,j) \in E
\end{aligned}
$$

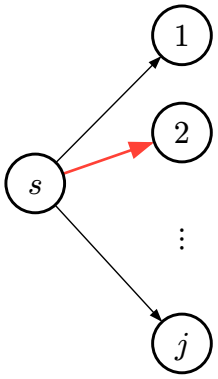
Where:

- s: supply node
- t: demand node
- x_{ij}: weather edge (i,j) is chosen (binary)
- d_{ij}: weight (distance) between nodes i and j

Objective: Get from s to t while minimizing distance

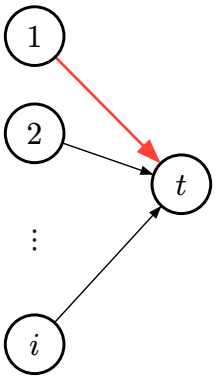
Out of all outgoing edges from s, one must be selected:

$$\sum_{(s,j) \in E} x_{sj} = 1$$



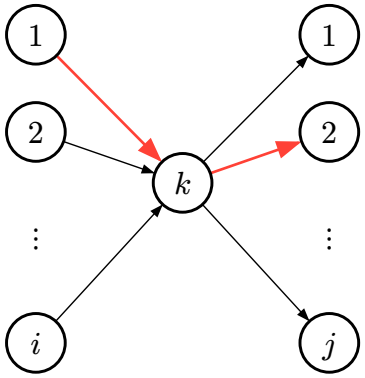
Out of all ingoing edges to t, one must be selected:

$$\sum_{(i,t) \in E} x_{it} = 1$$



For all transhipment nodes T,

$$\sum_{(i,k)\in E} x_{ik} - \sum_{(k,j)\in E} x_{kj} = 0 \quad \forall k \in T$$



Summary

Assignment problems are a special case of **Transportation** problems where:

- $b_i = 1$: net supply at node i

Transportation problems are a special case of **Transhipment** problems where:

- $b_i \neq 0$: net supply at node i

Transhipment problems are a special case of **MCNF** problems where:

- $u_{ij} = \infty$: No capacity on edge (i, j)

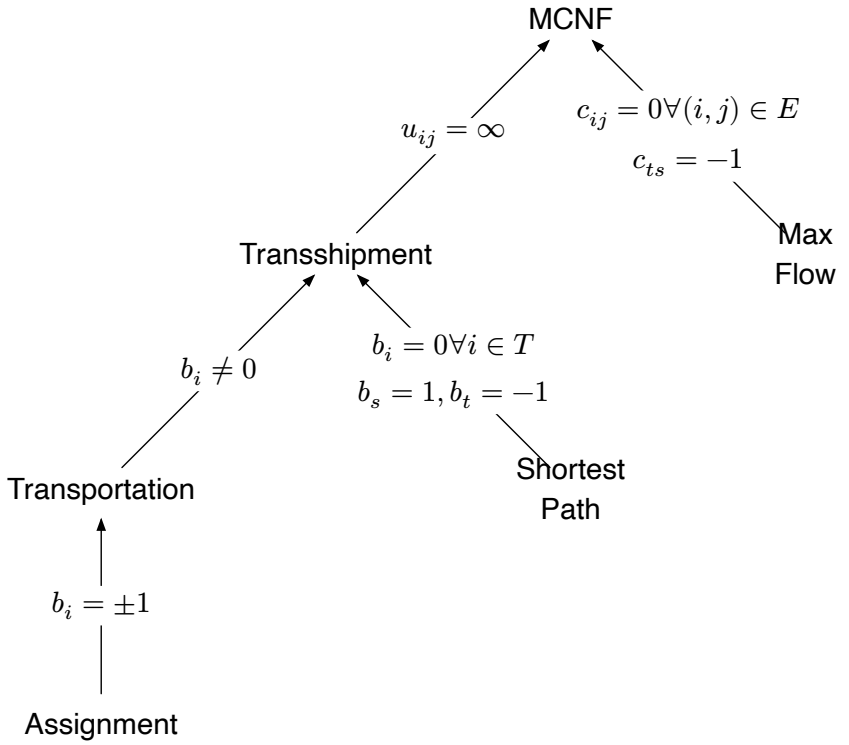
Shortest Path problem is a special case of **Transhipment** problems where:

- $b_s = 1$: One supply node with supply of 1
- $b_t = -1$: One demand node with supply of –1 (demand)
- $b_i = 0 \forall i \in T$: All others are transhipment nodes

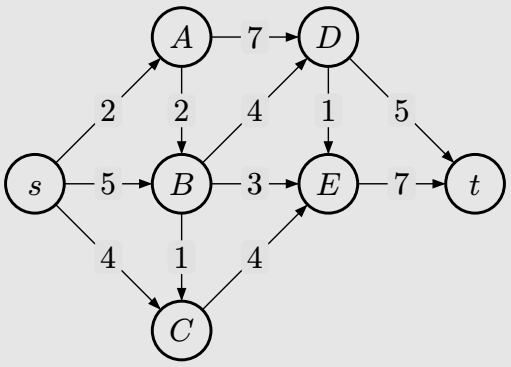
Maximum Flow problems are a special case of **MCNF** problems where:

- $c_{ij} = 0 \forall (i, j) \in E$: All original edges have 0 cost
- $c_{ts} = -1$: One “virtual edge” with cost –1

$$\begin{aligned} \min \quad & c^T x \\ s.t. \quad & Ax = b \\ & x \leq u \\ & x \geq 0 \end{aligned}$$



Example



$$A = \begin{bmatrix} 0 & 2 & 5 & 4 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 7 & 0 & 0 \\ 0 & 0 & 0 & 1 & 4 & 3 & 0 \\ 0 & 0 & 0 & 0 & 0 & 4 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 5 \\ 0 & 0 & 0 & 0 & 0 & 0 & 7 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

181.19. Sensitivity Analysis

181.19.1. Dual Simplex Method

Example

A company is selling 2 products

- Producing 1 unit of product 1 requires 1 unit of resource 1 and 1 unit of resource 2, which can be sold for $2
- Producing 1 unit of product 2 requires 1 unit of resource 1 and 2 unit of resource 2, which can be sold for $3
- Total amount of resources 1 is 4 units
- Total amount of resources 2 is 6 units

$$\begin{array}{lrcrcl} \max & 2x_1 & + & 3x_2 & & \\ s.t. & x_1 & + & x_2 & \leq & 4 \\ & x_1 & + & 2x_2 & \leq & 6 \end{array}$$

$$x_1, x_2 \geq 0$$

–2	–3	0	0	0
1	1	1	0	4
1	2	0	1	6

0	–1	2	0	8
1	1	1	0	4
0	1	–1	1	2

0	0	1	1	10
1	0	2	–1	2
0	1	–1	1	2

- An optimal solution $(x_1^*, x_2^*) = (2, 2)$
- The objective is $z^* = 10$

Additional activity

The company now produces a 3rd product.

- 1 unit of product 3 requires 1 unit of resource 2 and is sold for $8

$$
\begin{array}{lrcrcrcl}
\max & 3x_1 & + & 2x_2 & + & 8x_3 & & \\
s.t. & x_1 & + & 2x_2 & & & \leq & 4 \\
 & 2x_1 & + & 4x_2 & + & x_3 & \leq & 6
\end{array}
$$

$$
x_1, x_2, x_3 \geq 0
$$

We don't need to solve a new linear program

The decision variable set changes from (x_1, x_2) to (x_1, x_2, x_3), the solution $(x_1^*, x_2^*, 0)$ is feasible

All we need to do is to check whether we should produce some product 3

- If no, the current solution $(x_1^*, x_2^*, 0)$ is optimal
- If yes, we increase the nonbasic variable x_3 until one basic variable becomes 0

All we need is the (vector of) **reduced costs**:

$$
c_B^T B^{-1} N - c_N
$$

Where:

- B: basis matrix
- N: non-basis matrix
- c_B: vector of objective coefficients corresponding to the basic variables
- c_N: vector of objective coefficients corresponding to the non-basic variables

For a single nonbasic variable (x_3), it simplifies to:

$$
c_B^T B^{-1} A_j - c_j
$$

After we solve the original problem, we have $B^* = (x_1, x_2)$ and $N^* = (s_1, s_2)$

- The optimal basis is B^*

When we add a new decision variable x_3, it is 0 (**nonbasic**) at the beginning

Therefore, to solve the new problem, we may start from the basis $B = (x_1, x_2)$ and the set of nonbasic variables $N = (x_3, s_1, s_2)$

We start with the optimal tableau

0	0	1	1	10
1	0	2	–1	2
0	1	–1	1	2

basic nonbasic

0	0	?	1	1	10

1	0	?	2	−1	2
0	1	?	−1	1	2
basic		nonbasic			

The vecots of constraint coefficients for nonbasic variables is:

$$B^{-1}N$$

for column j:

$$B^{-1}A_j$$

Where A_j is the coefficient column for x_3:

$$\begin{bmatrix} 2 & -1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \end{bmatrix}$$

And the vector if reduced costs for nonbasic variables is

$$c_B^T B^{-1} N - c_N^T$$

For column j, that value is

$$c_B^T B^{-1} A_j - c_j$$

$$\begin{bmatrix} 2 & 3 \end{bmatrix} \begin{bmatrix} -1 \\ 1 \end{bmatrix} - 8 = -7$$

0	0	−7	1	1	10
1	0	−1	2	−1	2
0	1	1	−1	1	2
basic		nonbasic			

From this new tableau keep iterating

0	0	−7	1	1	10
1	0	−1	2	−1	2
0	1	1	−1	1	2

0	7	0	−6	8	24
1	1	0	1	0	4

0	1	1	−1	1	2

6	13	0	0	8	28
1	1	0	1	0	4
1	2	1	0	1	6

$(x_1^{**}, x_2^{**}, x_3^{**}) = (0, 0, 6)$ with $z^{**} = 48$

Allows asking **what if** questions:

- What if it is $5 instead of $8
- What if it takes 2 of resource on instead of 1?
- What if it takes 1 of resource 2 instead of 0?

Additional Constraints

Primal

$$\begin{array}{lrcrcl} \max & 2x_1 & + & 3x_2 & & \\ s.t. & x_1 & + & x_2 & \leq & 4 \\ & x_1 & + & 2x_2 & \leq & 6 \\ & x_1, x_2 \geq 0 & & & & \end{array}$$

We add a new constraint:

Dual

$$\begin{array}{lrcrcl} \max & 2x_1 & + & 3x_2 & & \\ s.t. & x_1 & + & x_2 & \leq & 4 \\ & x_1 & + & 2x_2 & \leq & 6 \\ & x_1 & & & \leq & 1 \\ & x_1, x_2 \geq 0 & & & & \end{array}$$

We may plug in in the original optimal solution $(x_1^*, x_2^*) = (2, 2)$ into the new constraint

- If it is feasible, it is optimal for the new problem
- In our example, it is not feasible because $2 > 1$

The original optimal tableau

0	0	1	1	10
1	0	2	−1	2
0	1	−1	1	2

The new constraint $x_1 \leq 1$ introduces a new slack variable (s_3)

Include s_3 to be a basic variable

Let $B = (x_1, x_2, s_3)$ and $N = (s_1, s_2)$

We have

$$c_B = \begin{bmatrix} 2 \\ 3 \\ 0 \end{bmatrix}, \quad c_N = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad B = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 2 & 0 \\ 1 & 0 & 1 \end{bmatrix}, \quad N = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}, \quad b = \begin{bmatrix} 4 \\ 6 \\ 1 \end{bmatrix}$$

We then have

$$B^{-1} = \begin{bmatrix} 2 & -1 & 0 \\ -1 & 1 & 0 \\ -2 & 1 & 1 \end{bmatrix}, \quad B^{-1}N = \begin{bmatrix} 2 & -1 \\ -1 & 1 \\ -2 & 1 \end{bmatrix}, \quad c_B^T B^{-1} N - c_N^T = [1 \quad 1], \quad c_B^T B^{-1} b = 10$$

0	0	1	1	0	10
1	0	2	–1	0	2
0	1	–1	1	0	2
0	0	–2	1	1	–1
basic		nonbasic		basic	

This is an invalid simplex tableau (RHS column column contains a negative value)

- This means $B = (x_1, x_2, s_3)$ is infeasible (as we already know)

Linear Programming Duality

We know a primal constraint is a dual variable

If a primal LP has one new constraint, it's dual LP will have one new variable

Primal

$$\begin{aligned} \max \quad & 2x_1 + 3x_2 \\ s.t. \quad & x_1 + x_2 \leq 4 \\ & x_1 + 2x_2 \leq 6 \\ & x_1, x_2 \geq 0 \end{aligned}$$

Dual

$$\begin{aligned} \min \quad & 4y_1 + 6y_2 \\ s.t. \quad & y_1 + y_2 \geq 2 \\ & y_1 + 2y_2 \geq 3 \\ & y_1, y_2 \geq 0 \end{aligned}$$

We add a new constraint:

Primal LP with new constraint

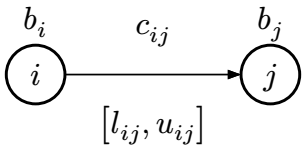
Dual LP with new variable

$$
\begin{aligned}
\max \quad & 2x_1 + 3x_2 \\
s.t. \quad & x_1 + x_2 \leq 4 \\
& x_1 + 2x_2 \leq 6 \\
& x_1 \leq 1 \\
& x_1, x_2 \geq 0
\end{aligned}
\qquad
\begin{aligned}
\min \quad & 4y_1 + 6y_2 + y_3 \\
s.t. \quad & y_1 + y_2 + y_3 \geq 2 \\
& y_1 + 2y_2 \geq 3 \\
& y_1, y_2, y_3 \geq 0
\end{aligned}
$$

181.20. Network Flow

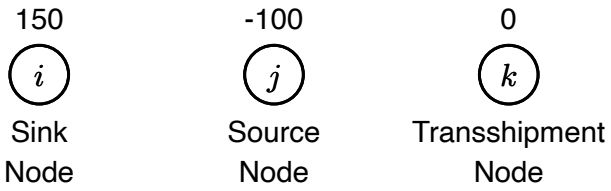
Types of Problems:

- Maximum Flow
- Minimum Cost Flow
- Multi-Commodity Flow



Node i

- Sink node: $b_i > 0$ — Has demand of b_i units
- Source node: $b_i < 0$ — Has supply of $-b_i$ units
- Transsipment node: $b_i = 0$ — Neither supply or demand



Units shipped from node i to node j:

$$x_{ij}$$

Minimize:

$$Z = \sum_{i,j} c_{ij} x_{ij}$$

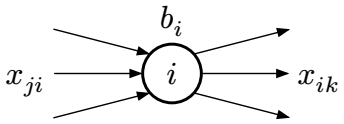
s.t.

$$\sum_k x_{ki} - \sum_l x_{il} = \text{or} \leq \text{or} \geq b_1$$

$$l_{ij} \leq x_{ij} \leq u_{ij}$$

Where:

- c_{ij}: Unit cost of flow from node i to node j
- b_i: Demand on node i
- l_{ij}: flow lower bound from i to j
- u_{ij}: Flow upper bound from i to j
- $\sum_k x_{ki}$: Inflow to i
- $\sum_l x_{il}$: Outflow from j



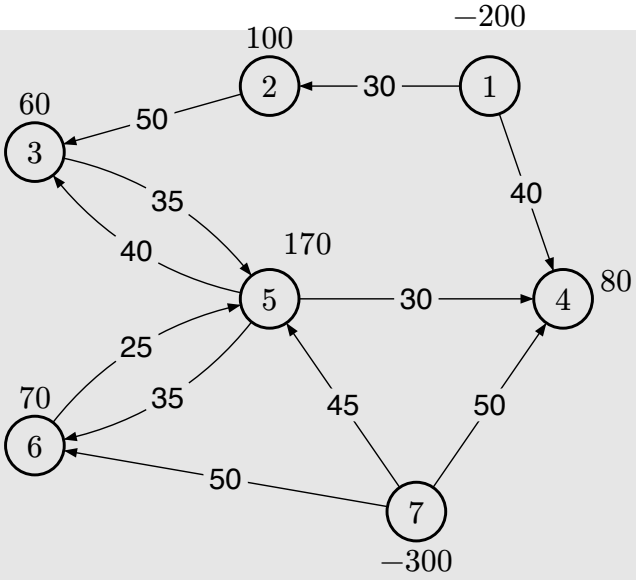
If:

- Total Supply $=$ Total Demand [Inflow to i] - [Outflow from i] $= b_i$
- Total Supply $>$ Total Demand [Inflow to i] - [Outflow from i] $\geq b_i$
- Total Supply $<$ Total Demand [Inflow to i] - [Outflow from i] $\leq b_i$

Important:

- One decision variable x_{ij} for each edge (i, j)
- One flow balancing constraint for each node i

Example



Minimize

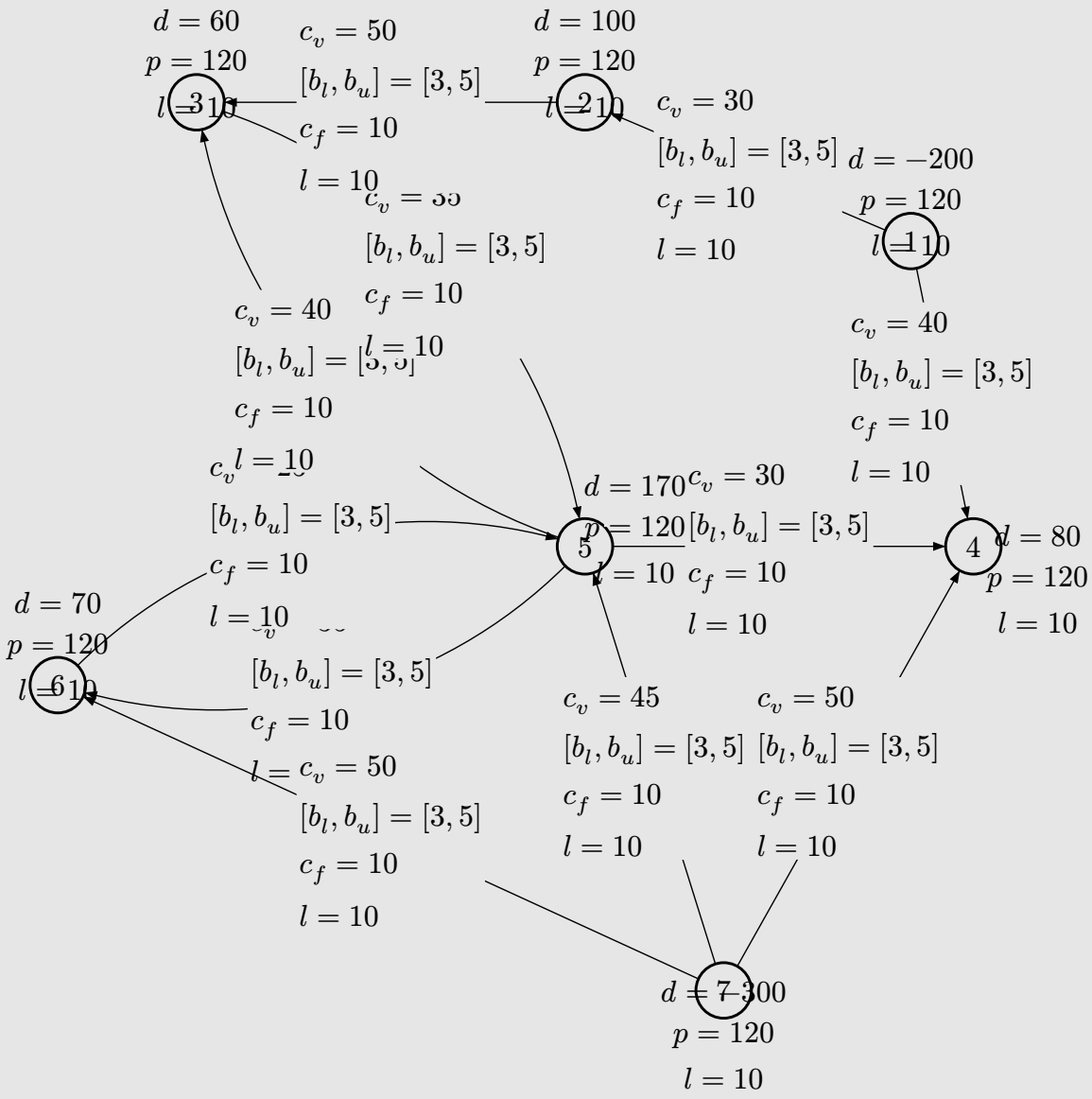
$$Z = 30x_{12} + 40x_{14} + 50x_{23} + 35x_{35} + 40x_{53}$$
$$+30x_{54} + 35x_{56} + 25x_{65} + 50x_{74} + 45x_{75} + 50x_{76}$$

s.t.

$$x_{12} + x_{14} \leq 200 \quad \text{(Node 1)}$$
$$x_{12} + x_{23} \geq 100 \quad \text{(Node 2)}$$
$$x_{23} + x_{53} - x_{35} \geq 60 \quad \text{(Node 3)}$$
$$x_{14} + x_{54} + x_{74} \geq 80 \quad \text{(Node 4)}$$
$$x_{35} + x_{65} + x_{75} - x_{53} - x_{54} - x_{56} \geq 170 \quad \text{(Node 5)}$$
$$x_{56} + x_{76} - x_{65} \geq 70 \quad \text{(Node 6)}$$
$$x_{76} + x_{75} + x_{74} \leq 300 \quad \text{(Node 7)}$$

$$x_{ij} \geq 0 \quad \forall (i,j) \in E$$

Example



Minimize:

$$\sum_{ij}\left(c_{ij}^{f} \cdot y_{ij}\right) + \sum_{ij}\left(c_{ij}^{v} \cdot x_{ij}\right) + \sum_{i}\left(c_{i}^{f} \cdot y_{i}\right) + \sum_{i}(p_i \cdot s_i) +$$

Where:

- $\sum_{ij}\left(c_{ij}^{f} \cdot y_{ij}\right)$: Edge fixed cost contribution
- $\sum_{ij}\left(c_{ij}^{v} \cdot x_{ij}\right)$: Variable cost contribution
- $\sum_{i}\left(c_{i}^{f} \cdot y_{i}\right)$: Node fixed cost contribution
- $\sum_{i}(p_i \cdot s_i)$: Penalty contribution
- $\sum_{ij}\left(l_{ij} \cdot x_{ij}\right)$: Edge lead time weighted by flow
- $\sum_{i} l_i \cdot \sum_{j} x_{ij}$: Node service time weighted by flow

s.t.

$\sum_j x_{ji} - \sum_j x_{ij} = \text{or} \leq \text{or} \geq d_i$	Flow Conservation
$b_{ij}^l \leq x_{ij} \leq b_{ij}^u$	Lower & Upper Flow Bound
$x_{ij} \leq M \cdot y_{ij}$	Fixed Cost Route
$\sum_i (x_{ji} + x_{ij}) \leq M \cdot y_i$	Fixed Cost Node
$\sum_i x_{ij} + s_j \geq d_j$	Unmet Demand Penalty
$x_{ij} \geq 0 \quad \forall (i,j) \in E$	Non Negative Flow

Node Properties:

- **Node Type**: Source, sink, or intermediary.
- **Supply (Source)**: Flow capacity of source nodes.
- **Holding Cost (Intermediary)**: Inventory cost for stored goods.
- **Service Time**: Processing time at the node.
- **Demand**: Required flow at sink nodes.
- **Storage Capacity (Intermediary)**: Maximum amount of goods that can be held at a node
- **Penalty for Unfulfilled Demand (Sink)**: Cost for unmet demand.
- **Disruption Risk (All Nodes)**: Probability of a node being unavailable due to unforeseen circumstances

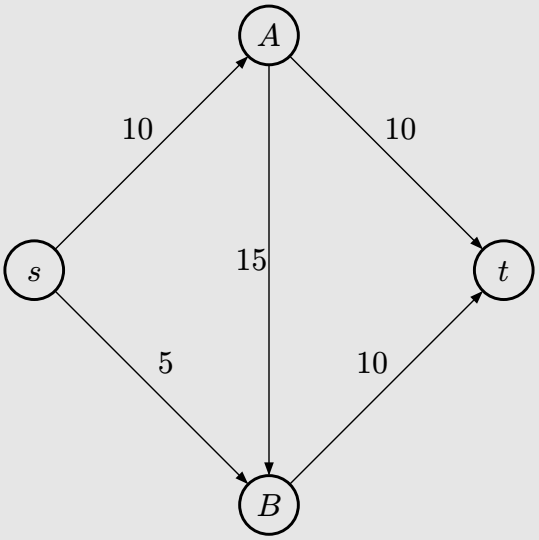
Edge Properties:

- **Edge Type**: Transport mode (air, water, road, rail).
- **Fixed Cost**: Cost incurred for using the edge, regardless of flow.
- **Reliability**: Probability of edge availability.
- **Flow Bounds**: Minimum and maximum allowable flow.
- **Unit Cost**: Cost per unit of flow.
- **Lead Time**: Time it takes for flow to travel along the edge.
- **Environmental Impact**: Account for the carbon footprint of using certain transport modes.

181.21. Ford–Fulkerson

Find augmenting paths in the network and increase the flow until no more augmenting paths can be found

Example



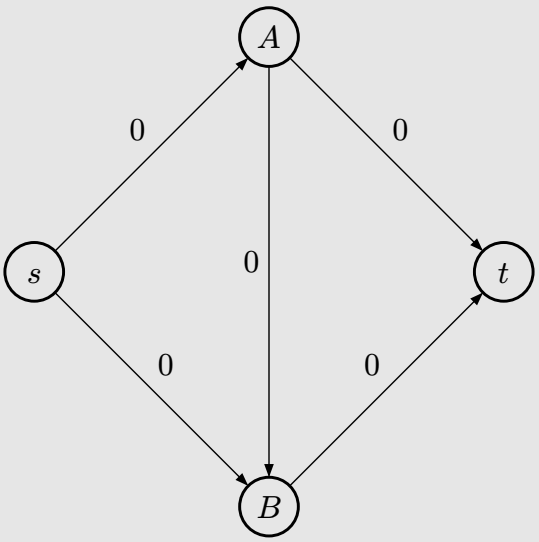
- s: Source
- A & B: Intermediate nodes
- t: Sink

Capacities:

- $s \to t$: 10
- $s \to B$: 5
- $A \to B$: 15
- $A \to t$: 10
- $B \to t$: 10

Step 1: Initialize flow to 0

All flows through the edges are initially set to 0.



Step 2: Find an augmenting path

Find an augmenting path using Depth-First Search (DFS). Start from the source s

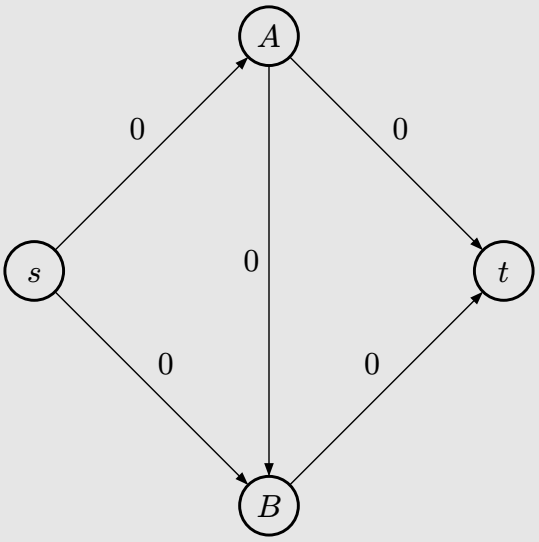
$$s \to A \to t$$

The minimum capacity along this path is 10 (bottleneck on edge $A \to t$)

We can push a flow of 10 units along this path.

Step 3: Update the residual graph

- $s \to A$: Capacity becomes $10 - 10 = 0$ (no residual capacity)
- $A \to t$: Capacity becomes $10 - 10 = 0$ (no residual capacity)



Step 4: Find another augmenting path

Find another augmenting path.

We cannot use $s \to A$ or $A \to t$ because their capacities are 0.

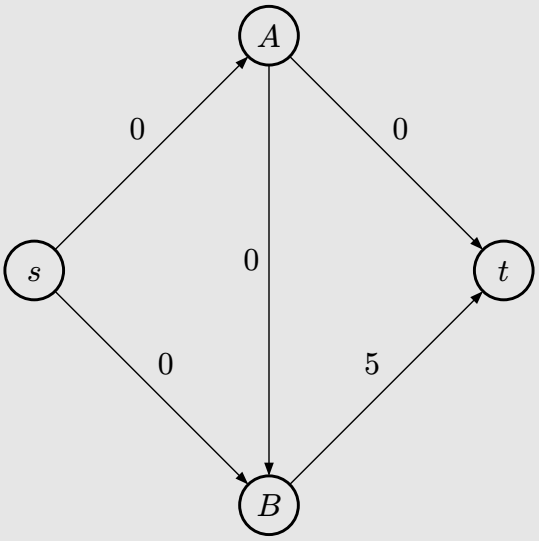
$$s \to B \to t$$

The minimum capacity along this path is 5 (bottleneck on edge $s \to B$)

We can push a flow of 5 units along this path

Step 5: Update the residual graph

- $s \to B$: Capacity becomes $5 - 5 = 0$
- $B \to t$: Capacity becomes $10 - 5 = 5$



Step 6: Termination

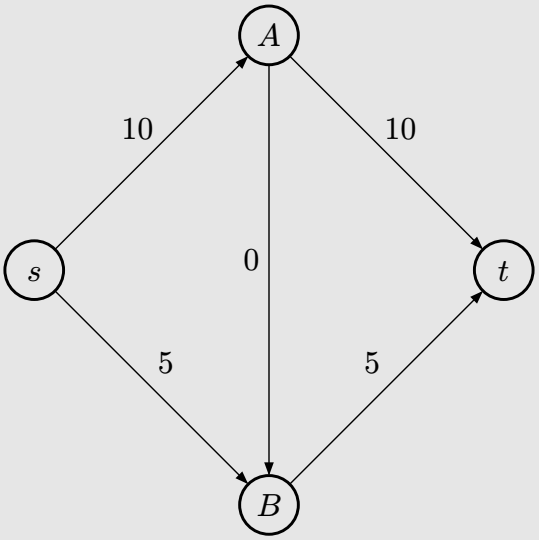
No more augmenting paths from s to t can be found, as all edges from s are fully saturated

Maximum flow:

- $s \to A \to t$: 10 units
- $s \to B \to t$: 5 units

Thus, the maximum flow from source s to sink t is 15 units

Step 7: Final Flow Distribution



181.22. Edmonds–Karp

181.23. Transportation Problem

A classical LP: ship supply from m sources to n destinations at minimum cost.

181.23.1. Formulation

Decision variable x_{ij} = units shipped from source i to destination j.

$$\min \sum_{i=1}^{m} \sum_{j=1}^{n} c_{ij} x_{ij}$$

subject to:

$$\sum_{j} x_{ij} = s_i, \quad i = 1, ..., m \qquad (\text{supply at source } i = s_i\ )$$

$$\sum_{i} x_{ij} = d_j, \quad j = 1, ..., n \qquad (\text{demand at destination } j = d_j\ )$$

$$x_{ij} \geq 0$$

Balanced: $\sum s_i = \sum d_j$. If unbalanced, add a dummy source / sink.

181.23.2. Structure

- mn decision variables, $m + n$ equality constraints (one redundant) $\longrightarrow$ $m + n - 1$ basic variables in any BFS
- Constraint matrix is **totally unimodular** — LP relaxation always has integer optimum (with integer supply/demand)
- Special structure $\longrightarrow$ much faster than general simplex

181.23.3. Solution algorithms

1. **Northwest-corner rule** — initial BFS by greedy fill from upper-left
2. **Vogel's approximation method (VAM)** — better initial BFS using opportunity costs
3. **MODI / stepping stone** — improvement iterations on a BFS to optimality
4. **Transportation simplex** — specialized simplex exploiting structure; $O((m+n)^2)$ per pivot vs $O((mn)^2)$ for general

For large instances: just use network simplex.

181.23.4. Special cases

- **Assignment problem** — $m = n$, all $s_i = d_j = 1$, $x \in \{0, 1\}$
- **Transshipment** — allow intermediate nodes
- **MCNF** — generalization with arc capacities (here)

181.23.5. See also

- **Assignment Problem**
- **Transshipment**
- **Network Simplex**

- **Linear Programming**

181.24. Assignment Problem

The special case of the transportation problem where n agents are assigned to n tasks one-to-one.

181.24.1. Formulation

Cost c_{ij} to assign agent i to task j. Decision variable $x_{ij} \in \{0, 1\}$.

$$\min \sum_{i=1}^{n} \sum_{j=1}^{n} c_{ij} x_{ij}$$

s.t.:

$$\sum_{j} x_{ij} = 1, \quad i = 1, ..., n \qquad \text{(each agent gets one task)}$$

$$\sum_{i} x_{ij} = 1, \quad j = 1, ..., n \qquad \text{(each task gets one agent)}$$

$$x_{ij} \in \{0, 1\}$$

181.24.2. Unbalanced (dummy)

When agents and tasks differ in number (n agents, m tasks, $n > m$), not everyone can be assigned. Relax the agent constraint to $\leq$:

$$\begin{aligned} \min & \sum_{ij} c_{ij} x_{ij} \\ \text{s.t.} & \sum_{j} x_{ij} \leq 1 \quad \forall i \quad \text{(agent does at most one task)} \\ & \sum_{i} x_{ij} = 1 \quad \forall j \quad \text{(every task is covered)} \\ & x_{ij} \geq 0 \end{aligned}$$

Equivalently, restore balance with a **dummy task** (column $m + 1$, zero cost) that absorbs the $n - m$ unassigned agents:

$$\begin{aligned} \min & \sum_{i=1}^{n} \sum_{j=1}^{m+1} c_{ij} x_{ij} \\ \text{s.t.} & \sum_{j=1}^{m+1} x_{ij} = 1 \quad \forall i \in \{1, ..., n\} \\ & \sum_{i=1}^{n} x_{ij} = 1 \quad \forall j \in \{1, ..., m\} \\ & \sum_{i=1}^{n} x_{i,m+1} = n - m \\ & x_{ij} \geq 0 \end{aligned}$$

181.24.3. LP relaxation gives integer solution

Constraint matrix is totally unimodular → LP relaxation has integer optimum. Can drop the binary constraint and solve as an LP; equivalently use the Hungarian algorithm in $O(n^3)$.

181.24.4. Applications

- **Job-machine assignment** — minimize total processing time
- **Crew scheduling** — assign pilots to flights
- **Bipartite matching** — pair customers to drivers
- **Image processing / computer vision** — find correspondences between point sets
- **Subproblem** in branch-and-bound for VRP, TSP

181.24.5. Generalizations

- **Generalized assignment** — costs and capacities, NP-hard
- **Quadratic assignment problem** — cost depends on pairs of assignments, NP-hard
- **Bottleneck assignment** — minimize **max** assignment cost (minimax variant)

181.24.6. See also

- **Hungarian Algorithm** — the standard solver
- **Transportation Problem** — the generalization
- **Unimodularity** — why LP relaxation works

181.25. Hungarian Algorithm

The classical $O(n^3)$ algorithm for the assignment problem. Kuhn (1955), based on work by Hungarian mathematicians Egerváry & König.

181.25.1. Algorithm (intuition)

Given an $n \times n$ cost matrix C:

1. **Row reduction**: subtract each row's minimum from that row
2. **Column reduction**: subtract each column's minimum from that column
3. **Cover all zeros** using the minimum number of horizontal / vertical lines
4. **If lines = n**: an optimal assignment exists among the zeros — extract it
5. **If lines < n**: adjust the matrix (subtract the minimum uncovered value from uncovered entries, add it to entries at line intersections) and go back to step 3

Repeat until step 4 succeeds. Termination in $O(n^3)$.

181.25.2. Why it works

Each step preserves the **relative cost structure**: subtracting a constant from a row doesn't change which assignment is optimal. The reductions create zeros — and an assignment using only zero-cost cells (when one exists) is necessarily optimal because all assignments have the same total constant subtracted.

The "cover" check finds when enough zeros exist to form a complete assignment. The adjustment step exposes more zeros without changing optimal-assignment structure.

181.25.3. Worked example

$$C = \begin{bmatrix} 9 & 2 & 7 & 8 \\ 6 & 4 & 3 & 7 \\ 5 & 8 & 1 & 8 \\ 7 & 6 & 9 & 4 \end{bmatrix}$$

After row reduction (subtract row mins $2, 3, 1, 4$):

$$\begin{bmatrix} 7 & 0 & 5 & 6 \\ 3 & 1 & 0 & 4 \\ 4 & 7 & 0 & 7 \\ 3 & 2 & 5 & 0 \end{bmatrix}$$

After column reduction (subtract column mins $3, 0, 0, 0$):

$$\begin{bmatrix} 4 & 0 & 5 & 6 \\ 0 & 1 & 0 & 4 \\ 1 & 7 & 0 & 7 \\ 0 & 2 & 5 & 0 \end{bmatrix}$$

Cover all zeros: rows 2, 4 and column 3 cover all zeros — 3 lines for an $n = 4$ matrix $\rightarrow$ need to adjust.

(Continue iterations…)

Final assignment: agent 1 $\rightarrow$ task 2, agent 2 $\rightarrow$ task 1, agent 3 $\rightarrow$ task 3, agent 4 $\rightarrow$ task 4. Total cost $= 2 + 6 + 1 + 4 = 13$.

181.25.4. Why $O(n^3)$

Each “augmenting path” step (finding more zeros to expose) costs $O(n^2)$ via a labeling procedure. At most n augmenting paths needed (each adds at least one matched pair). Total $O(n^3)$.

Modern variants:

- **Jonker–Volgenant** — $O(n^3)$ with smaller constants
- **Auction algorithm** (Bertsekas) — different paradigm, also $O(n^3)$ but parallelizable
- **Sparse Hungarian** — $O(m\sqrt{n})$ for sparse graphs

181.25.5. See also

- **Assignment Problem** — the problem being solved
- **Transportation Problem** — generalization

181.26. Transshipment Problem

Generalization of the transportation problem allowing **intermediate transshipment nodes** — locations that neither produce nor consume, but pass flow through.

181.26.1. Formulation

Three node types:

- **Supply** nodes ($b_i > 0$): produce b_i units
- **Demand** nodes ($b_i < 0$): consume $|b_i|$ units
- **Transshipment** nodes ($b_i = 0$): inflow = outflow

For each arc $(i, j) \in E$, decision $x_{ij} \geq 0$ = flow on arc, with cost c_{ij} per unit.

$$\min \sum_{(i,j)\in E} c_{ij} x_{ij}$$

s.t. flow conservation at each node:

$$\sum_{j:(j,i)\in E} x_{ji} - \sum_{j:(i,j)\in E} x_{ij} = -b_i, \quad \forall i \in V$$

(Inflow minus outflow = $-b_i$: positive for sinks, negative for sources, zero for transshipment.)

181.26.2. Capacitated variant

Now cap each arc: flow can't exceed an edge's capacity u_{ij}. Same objective and conservation, plus a per-arc upper bound:

$$\min \sum_{(i,j)\in E} c_{ij} x_{ij}$$

$$\sum_{j:(j,i)\in E} x_{ji} - \sum_{j:(i,j)\in E} x_{ij} = -b_i, \quad \forall i \in V$$

$$0 \leq x_{ij} \leq u_{ij}, \quad \forall (i,j) \in E$$

- lower bound 0: no reverse flow on the arc
- upper bound u_{ij}: the arc can't carry more than its capacity

181.26.3. Reduction to transportation problem

Any transshipment problem with n nodes can be converted into a transportation problem on a complete bipartite graph (every source potentially connects to every demand via possible transshipment paths) — at the cost of larger problem size.

In practice solve directly as an LP, or use network simplex.

181.26.4. Why it matters

The transshipment formulation is the **canonical model** for:

- **Multi-echelon distribution**: factory → DC → store
- **Hub-and-spoke networks**: airlines, parcel logistics
- **Inter-regional sourcing**: route through cheapest intermediate even when not on the direct path

- **Min-cost flow generalization** — see MCNF

181.26.5. See also

- **Transportation Problem** — special case (no intermediates)
- **Multi-Commodity Network Flow**
- **Network Simplex**

181.27. Min-Cost Flow Algorithms

Algorithms that solve the **minimum-cost flow** problem more efficiently than the generic LP. Given a network with arc capacities u_{ij} and costs c_{ij}, find feasible flow satisfying supply / demand at minimum total cost.

For the problem formulation see Min-cost flow LP.

181.27.1. Negative-cycle canceling

Idea: start with any feasible flow. If the residual graph has a negative-cost cycle, push flow around it to lower the total cost. Repeat until no negative cycle exists → optimal.

```
Initialize x with any feasible flow (e.g., zero flow if no demand)
While residual graph has a negative-cost cycle C:
    delta ← min residual capacity along C
    push delta units around C
Return x
```

- **Correctness**: optimum ⟺ no negative-cycle in residual (by LP duality / Bellman-Ford optimality)
- **Complexity**: polynomial only with careful pivot rules (e.g., minimum-mean-cycle canceling, $O(VE^2 \log V)$)
- **Use**: easy to implement, slow in practice for large instances

181.27.2. Successive shortest paths (SSP)

Idea: route the flow one unit at a time along the **shortest path** in the residual network (using current reduced costs).

```
Initialize x = 0, π = 0 (node potentials), b' = b (residual supply)
While there is unsatisfied supply:
    Find shortest path P from a remaining source to a remaining sink in
residual
    delta ← min residual capacity along P
    Push delta along P, update b'
    Update node potentials π to maintain non-negative reduced costs
```

- **Correctness**: maintains **optimality of the current flow** throughout — each new path is optimal augmentation
- **Complexity**: $O(V^2U)$ with U = max supply; or $O(V^2E)$ with Dijkstra
- **Use**: efficient when supply is small or paths are short

181.27.3. Cycle-canceling vs SSP

	Cycle-canceling	SSP
Starts from	feasible flow	zero flow
Maintains	feasibility	optimality
Iteration	find negative cycle	find shortest path
Stops when	no negative cycle	supply satisfied
Strongly polynomial?	yes (with care)	yes

181.27.4. Other methods

- **Network simplex** — specialization of simplex; very fast in practice
- **Out-of-kilter algorithm** — historical, more complex
- **Push-relabel** — efficient for max flow / min-cost extensions
- **Cost-scaling** — strongly polynomial, $O(VE \log(VC))$

181.27.5. Practical recommendation

For most instances, use **network simplex** (in commercial solvers like CPLEX, Gurobi, or open-source libraries). It's the standard production algorithm for MCNF.

181.27.6. See also

- **Min-Cost Flow** — problem formulation
- **Network Simplex**
- **Network Flow** — max-flow variant
- **Dijkstra** — shortest-path subroutine

181.28. Network Simplex

A specialization of the simplex method for network flow problems (transportation, transshipment, min-cost flow). Exploits the **tree structure** of basic feasible solutions for $50 - 100\times$ speed-up over generic simplex.

181.28.1. Key insight: basis = spanning tree

For a network with n nodes, a basic feasible solution corresponds to a **spanning tree** of the network (plus a choice of source / sink directions). There are $n - 1$ basic variables (one per tree arc), exactly matching the LP's basis dimension.

Non-basic arcs have flow zero or equal to their capacity.

181.28.2. Why this matters

Operations that are $O(n^3)$ in general simplex become $O(n)$ on a tree:

- **Pivot** (replace one basic arc with one non-basic arc) → just update the tree path
- **Dual prices** → propagate from root to leaves of the tree
- **Reduced costs** → compute from dual prices via $|c|_{ij} = c_{ij} - \pi_i + \pi_j$

181.28.3. Algorithm

```
Initialize a feasible spanning tree T (e.g., via big-M or two-phase)
While there exists a non-basic arc (i, j) with reduced cost < 0:
    Add (i, j) to T forming a cycle
    Push flow around the cycle until some basic arc hits zero or capacity
    Remove that arc from T (it leaves the basis)
Return optimal flow
```

181.28.4. Complexity

- **In theory**: not polynomial (can cycle in degenerate cases without Bland-style rules)
- **In practice**: extremely fast — $O(n)$ to $O(n \log n)$ per pivot, sub-linear pivots needed on real instances
- **State-of-the-art** MCNF solver in CPLEX, Gurobi, MINOS, etc.

181.28.5. When to use

- **Min-cost flow** problems
- **Transportation / transshipment / assignment** (which are MCNF special cases)
- Always faster than generic LP simplex for network problems

181.28.6. See also

- **Simplex Method** — the general parent
- **Min-Cost Flow** — problem solved
- **Transportation Problem** — special case
- **Min-Cost Flow Algorithms** — alternative methods

181.29. Min Spanning Tree

Given a connected, undirected, weighted graph, find a **spanning tree** (acyclic connected subgraph touching every vertex) of minimum total edge weight.

181.29.1. Two classic algorithms

Kruskal's algorithm (1956): edge-based, $O(E \log E)$

```
Sort edges by weight ascending
Initialize T = empty (every node is its own component)
For each edge (u, v) in sorted order:
    If u and v are in different components:
        Add (u, v) to T
        Merge their components
Return T
```

Prim's algorithm (1957): vertex-based, $O((V + E) \log V)$ with binary heap, $O(E + V \log V)$ with Fibonacci heap

```
Initialize T = {arbitrary starting vertex}
While T doesn't span the graph:
    Find min-weight edge (u, v) with u ∈ T, v ∉ T
```

```
    Add v and edge (u, v) to T
Return T
```

181.29.2. Why it's MST (correctness)

The **cut property**: for any cut of the graph, the minimum-weight edge crossing the cut belongs to some MST. Both Kruskal and Prim repeatedly select such cut-crossing minimum edges.

The **cycle property**: for any cycle, the maximum-weight edge does **not** belong to any MST. (Kruskal implicitly enforces this by skipping cycle-creating edges.)

181.29.3. Properties

- **Unique MST** iff all edge weights are distinct
- **Tree**: $V - 1$ edges, no cycles
- **Total weight**: sum of MST edges; minimum among all spanning trees
- **Multiple MSTs**: with ties, any choice that breaks cycles consistently gives the same total weight

181.29.4. Where it shows up

- **Network design**: minimum-cost wiring / connection (telephone, power, road)
- **Clustering**: single-linkage clustering = MST then prune longest edges
- **Approximation algorithms**: 2-factor approx for metric TSP uses MST traversal; Christofides is more refined
- **Image segmentation**: graph cuts based on MST

181.29.5. Comparison

	Kruskal	Prim
Data structure	union-find	priority queue
Best for	sparse graphs	dense graphs
Complexity	$O(E \log E)$	$O(E + V \log V)$ (Fibonacci)
Online use	needs all edges upfront	grows tree from any start

181.29.6. See also

- **TSP** — uses MST in lower bounds & approximations
- **Floyd-Warshall** — all-pairs shortest path (related)
- **Dijkstra** — single-source shortest path (related, different structure)

181.30. Floyd-Warshall

All-pairs shortest path algorithm — finds the shortest distance from every node to every other node in $O(V^3)$.

Useful when you need the distance matrix for **many** queries, vs running Dijkstra from each source ($O(V(V + E) \log V)$).

181.30.1. Algorithm

Dynamic programming. Let $d_k(i, j)$ = shortest path from i to j using only intermediate nodes from $\{1, ..., k\}$:

$$d_k(i,j) = \min(d_{k-1}(i,j),\ d_{k-1}(i,k) + d_{k-1}(k,j))$$

Either use node k as a stop (split at k) or don't (same path).

```
Initialize D[i][j] = edge weight (or ∞ if no edge), D[i][i] = 0
For k = 1, 2, …, V:
    For i = 1, …, V:
        For j = 1, …, V:
            D[i][j] ← min(D[i][j], D[i][k] + D[k][j])
Return D
```

Three nested loops → $O(V^3)$. Space $O(V^2)$ (the matrix). Update in place — no need for separate "old" and "new" matrices.

181.30.2. Edge cases

- **Negative edges**: handled (unlike Dijkstra)
- **Negative-cycle detection**: after the run, if any $D[i][i] < 0$, the graph has a negative cycle reachable from i
- **Disconnected nodes**: stay at ∞

181.30.3. Compared to single-source alternatives

	Floyd-Warshall	V × Dijkstra	V × Bellman-Ford
All-pairs cost	$O(V^3)$	$O(V(V+E)\log V)$	$O(V^2E)$
Negative edges?	yes	no	yes
Memory	$O(V^2)$	$O(V^2)$	$O(V^2)$
Dense graphs ($E \approx V^2$)	$O(V^3)$ — best	$O(V^3 \log V)$	$O(V^4)$
Sparse graphs ($E \approx V$)	$O(V^3)$	$O(V^2 \log V)$ — best	$O(V^3)$

Floyd-Warshall is best for **dense** graphs with potentially negative edges.

181.30.4. Path reconstruction

To recover paths (not just distances), maintain a **predecessor** matrix $\pi[i][j]$ alongside D:

```
When updating D[i][j] via D[i][k] + D[k][j]:
    π[i][j] ← π[k][j]
```

Then trace back from j to i via π.

181.30.5. Applications

- **Network routing**: distance tables for routing protocols
- **Logistics**: precompute shipping costs between all warehouses
- **Game AI / graph queries**: when distance matrix is queried millions of times

- **Transitive closure**: same algorithm with boolean OR / AND replaces min / +

181.30.6. See also

- **Dijkstra** — single-source, non-negative edges
- **Dynamic Programming** — the framework
- **Min Spanning Tree** — different problem, related techniques

181.31. TSP

The **Traveling Salesman Problem**: visit n cities exactly once and return to start, minimizing total distance.

Famous for being **easy to state, hard to solve**: NP-hard, no known polynomial-time exact algorithm. But also one of the most-studied problems in combinatorial optimization.

181.31.1. Formulation (DFJ — Dantzig-Fulkerson-Johnson, 1954)

Decision variable $x_{ij} \in \{0, 1\}$: 1 if the tour uses edge (i, j), else 0.

$$\min \sum_{i,j} c_{ij} x_{ij}$$

s.t.:

$$\sum_{j} x_{ij} = 1, \quad \forall i \qquad \text{(leave each city once)}$$

$$\sum_{i} x_{ij} = 1, \quad \forall j \qquad \text{(enter each city once)}$$

$$\sum_{i \in S, j \notin S} x_{ij} \geq 1, \quad \forall S \subset V, 2 \leq |S| \leq n-1 \qquad \text{(subtour elimination)}$$

$$x_{ij} \in \{0, 1\}$$

The subtour-elimination constraints are **exponential** in number ($2^n - 2$ subsets). In practice, add them lazily as cutting planes when violated.

181.31.2. Alternative formulation (MTZ — Miller-Tucker-Zemlin)

Polynomially many constraints using **node potentials** u_i:

$$u_i - u_j + n x_{ij} \leq n - 1, \quad \forall i, j \in \{2, ..., n\}, i \neq j$$

Forces a single tour without exponentially many cuts. LP relaxation is **weaker** than DFJ, so fewer cuts needed but worse bounds.

181.31.3. Computational reality

- **Concorde solver** (Applegate, Bixby, Chvátal, Cook): exact TSP up to $\sim 85,000$ cities
- **Practical heuristics** (LKH-3, OR-tools): near-optimal solutions for millions of cities in minutes
- **NP-hard** — but the constants are favorable in practice

181.31.4. Lower bounds (for branch-and-bound)

- **MST lower bound**: total tour length $\geq$ MST weight (twice the MST = upper bound, see Christofides)
- **1-tree bound** (Held-Karp): drop one node, MST on the rest, plus the two cheapest edges from the dropped node
- **LP relaxation** of DFJ

181.31.5. Heuristics

- **Nearest neighbor**: start anywhere, go to the closest unvisited city. Bad — often 25% above optimal
- **Greedy edge**: keep adding cheapest edge that doesn't create a cycle (until tour complete). Bad — similar quality
- **Insertion heuristics**: build tour by inserting one city at a time (cheapest, nearest, farthest)
- **2-opt, 3-opt**: local search swapping pairs / triples of edges — see TSP 2-opt
- **Lin-Kernighan**: variable-depth swap; the gold standard for **heuristic** TSP
- **Christofides**: 1.5-approximation for **metric** TSP (triangle inequality)

181.31.6. Variants

- **Symmetric TSP**: $c_{ij} = c_{ji}$ — most studied
- **Asymmetric TSP** (ATSP): different costs each direction
- **Euclidean TSP**: cities are points in the plane; admits PTAS (Arora 1998)
- **Metric TSP**: triangle inequality; 1.5-approx by Christofides
- **Vehicle Routing Problem (VRP)**: multiple vehicles, capacities — see VRP

181.31.7. See also

- **TSP 2-opt** — local search heuristic
- **Christofides** — 1.5-approximation
- **VRP** — multi-vehicle generalization
- **Clarke-Wright Savings** — heuristic
- **MST** — used in lower bounds

181.32. TSP 2opt

A simple but effective **local search** heuristic for the TSP. Iteratively replaces pairs of edges with a **cheaper** pair until no improvement is possible.

181.32.1. The 2-opt move

Given a tour $\ldots \to A \to B \to \ldots \to C \to D \to \ldots$, the 2-opt move **reverses** the segment $B \to \ldots \to C$, giving $\ldots \to A \to C \to \ldots \to B \to D \to \ldots$.

Geometrically: it removes two non-adjacent edges (A, B) and (C, D), and adds (A, C) and (B, D) — **uncrossing** the tour.

Improvement condition: do the swap iff

$$c(A, C) + c(B, D) < c(A, B) + c(C, D)$$

181.32.2. Algorithm

```
T ← initial tour (e.g., nearest neighbor)
Repeat:
    For each pair of non-adjacent edges (A,B), (C,D) in T:
        If c(A,C) + c(B,D) < c(A,B) + c(C,D):
            Perform 2-opt swap
            Break (or continue to next pair)
Until no improvement found
Return T
```

181.32.3. Complexity

- **Per pass**: $O(n^2)$ pairs to check
- **Number of passes**: typically $O(n)$ in practice
- **Total**: $O(n^3)$ to reach a 2-opt local optimum

181.32.4. Why it works

Each successful swap strictly decreases tour length. Since there are finitely many tours and the objective is bounded below, the process terminates.

181.32.5. 2-opt vs 3-opt vs Lin-Kernighan

Heuristic	Move type	Quality
2-opt	reverse one segment	5% above optimal
3-opt	3-edge swap, multiple reconnections	3% above optimal
Lin-Kernighan	variable-depth swaps	1% above optimal — gold standard

Lin-Kernighan dynamically tries 2-opt, 3-opt, …, k-opt moves with backtracking. **LKH-3** (Helsgaun's variant) is the state-of-the-art heuristic.

181.32.6. Practical tips

- **Start from a good initial tour** (savings algorithm, nearest-neighbor + 2-opt is standard)
- **Use neighborhood lists** — limit to k nearest neighbors per node to avoid $O(n^2)$ pair checks
- **First-improvement vs best-improvement** — first-improvement (take the first beneficial swap) is usually faster
- **Restart from multiple initial tours** — local optima vary; multiple runs improve average quality

181.32.7. See also

- **TSP** — problem
- **Christofides** — approximation with worst-case guarantee
- **Clarke-Wright** — good initial tour for TSP / VRP
- **Heuristics** — broader local-search family

181.33. Christofides

A $\frac{3}{2}$-approximation algorithm for the **metric** TSP — produces tours guaranteed within $1.5\times$ optimal.

For 40 years, this was the best-known approximation ratio. In 2020, Karlin-Klein-Gharan improved it slightly (to $1.5 - 10^{-36}$). Christofides' algorithm remains the standard reference point.

181.33.1. The algorithm

Given a complete graph with distances satisfying the **triangle inequality** ($c_{ik} \leq c_{ij} + c_{jk}$):

1. **Build MST** — minimum spanning tree T of the graph
2. **Find odd-degree nodes** in T — call this set O. (By handshake lemma, $|O|$ is even.)
3. **Min-weight perfect matching** on O — solve a matching problem on the subgraph induced by O
4. **Combine MST + matching** — gives a multigraph where every vertex has even degree
5. **Find Eulerian circuit** on this multigraph — visits every edge exactly once
6. **Shortcut** — convert the Eulerian circuit to a TSP tour by skipping repeated visits (uses triangle inequality)

The output is a Hamiltonian tour with total length $\leq 1.5 \cdot$ OPT.

181.33.2. Why $\frac{3}{2}$?

- **MST cost**: $\leq$ OPT (any TSP tour minus one edge is a spanning tree)
- **Matching cost**: $\leq \frac{1}{2}$ OPT (a TSP tour on the odd-degree nodes can be decomposed into two perfect matchings; the cheaper is $\leq \frac{1}{2}$ OPT)
- **Total**: MST + matching $\leq$ OPT $+\frac{1}{2}$ OPT $= 1.5$ OPT
- **Shortcutting only shortens** — under triangle inequality, replacing $A \to B \to C$ with $A \to C$ doesn't increase length

181.33.3. Complexity

- MST: $O(E \log V)$ — see MST
- Min-weight matching: $O(|O|^3)$ — most expensive step in practice
- Eulerian circuit: $O(E)$
- **Total**: $O(V^3)$, dominated by matching

The matching step makes Christofides slower than simpler heuristics like 2-opt on large instances.

181.33.4. When triangle inequality fails

If the graph doesn't satisfy triangle inequality (e.g., asymmetric TSP with arbitrary costs):

- No constant-factor approximation exists unless P = NP
- Christofides doesn't apply

For **Euclidean TSP** in the plane:

- Better approximations exist (PTAS by Arora 1998)
- Christofides is rarely the best practical choice (LKH-3 is usually better)

181.33.5. Worth knowing in 2026

- **2-opt / LK** heuristics give better solutions in practice (no guarantee but tighter results)
- Christofides is the **theoretical benchmark** — used in proofs, complexity arguments
- **Metric TSP**: still the only easy worst-case bound to cite

181.33.6. See also

- **TSP** — problem
- **TSP 2-opt** — heuristic alternative
- **MST** — used in step 1

181.34. VRP

The **Vehicle Routing Problem**: given a fleet of vehicles based at a depot, serve a set of customers with minimum total cost (distance / time). Generalization of TSP to multiple vehicles.

Core supply-chain problem: last-mile delivery, beverage distribution, waste collection, school bus routing, mobile maintenance.

181.34.1. Capacitated VRP (CVRP)

Standard formulation. Each vehicle has capacity Q. Each customer i has demand q_i.

Decision variable x_{ijk}: 1 if vehicle k traverses arc (i, j).

$$\min \sum_{k} \sum_{i,j} c_{ij} x_{ijk}$$

s.t.:

- Each customer visited exactly once
- Vehicle starts and ends at the depot
- Total demand on each vehicle's route $\leq Q$
- Subtour elimination (like TSP)

181.34.2. Why it's harder than TSP

- **NP-hard** even worse than TSP (TSP is the single-vehicle case)
- **Combinatorial explosion**: must simultaneously partition customers into routes **and** route each
- **Exponential** number of subtour constraints, plus capacity constraints

181.34.3. Variants

- **VRP-TW (with time windows)** — each customer has a delivery window $[a_i, b_i]$; see VRPTW
- **Pickup-and-delivery VRP** (PDVRP) — pickups must precede deliveries on the same route
- **Heterogeneous fleet VRP** — vehicles with different capacities / costs
- **Stochastic VRP** — random demand or travel times

- **Periodic VRP** — multi-day delivery schedules with frequency constraints
- **Open VRP** — vehicles don't return to depot (typical for last-mile contractors)
- **Multi-depot VRP** — vehicles from multiple depots

181.34.4. Solution methods

Exact (small instances, $n \leq 200$):

- **Branch-and-cut** on the LP formulation
- **Branch-and-price** (column generation)
- **Dynamic programming** for very small fleets

Heuristics:

- **Constructive**: Clarke-Wright savings (the classic), Sweep, Fisher-Jaikumar
- **Improvement**: 2-opt, Or-opt, 3-opt within routes; inter-route swaps
- **Metaheuristics**: tabu search, simulated annealing, genetic algorithms, large-neighborhood search
- **Modern**: Adaptive Large Neighborhood Search (ALNS), HGS (Hybrid Genetic Search)

Open-source solvers:

- **OR-Tools** (Google) — fast LS-based heuristics
- **VROOM** — open-source CVRP / VRPTW solver
- **VRPSolver** — academic branch-cut-price

181.34.5. Continuous approximation

For large-scale VRP design (how many vehicles? how big are routes?), **Daganzo's continuous approximation** gives clean formulas. See Daganzo.

181.34.6. See also

- **TSP** — single-vehicle case
- **VRPTW** — with time windows
- **Clarke-Wright** — standard heuristic
- **Daganzo Continuous Approximation**
- **Assignment Problem** — sub-problem in some methods

181.35. VRP Time Windows

VRP with Time Windows (VRPTW): each customer i specifies a **time window** $[a_i, b_i]$ during which service must begin.

The most common real-world VRP variant. Models meal delivery, parcel logistics, home services, dialysis transport, etc.

181.35.1. Formulation additions

On top of CVRP:

Let t_i = service start time at customer i, s_i = service duration, t_{ij} = travel time from i to j.

Time window constraints:

$$a_i \le t_i \le b_i, \quad \forall i$$

Time propagation:

$$t_j \ge t_i + s_i + t_{ij} - M(1 - x_{ijk})$$

(If vehicle k goes $i \to j$, then j is served at least $t_i + s_i + t_{ij}$.)

Early arrival waiting is usually allowed: arrive before a_i, wait, then serve at a_i.

181.35.2. Hard vs soft windows

- **Hard windows**: $t_i \in [a_i, b_i]$ strict — infeasibility if violated
- **Soft windows**: penalty for arriving outside the window, but feasible
- **Multiple windows**: a customer has several acceptable service intervals

Soft windows are easier to solve (relaxes infeasibility); hard windows model real commitments.

181.35.3. Why VRPTW is much harder than CVRP

- **Tighter feasibility**: many tours that fit capacity violate time windows
- **Sequence-dependent feasibility**: $i \to j \to k$ may be infeasible even if each pair $i \to j$ and $j \to k$ is feasible alone
- **Larger LP relaxations**: more constraints, more weakening of bounds
- **Column generation** (the standard exact method) needs **Resource-Constrained Shortest Path** sub-problems — themselves NP-hard

181.35.4. Solution methods

Constructive:

- **Solomon's insertion heuristic** (I1): build routes by inserting customers based on a parameterized "insertion cost"
- **Time-oriented nearest neighbor**

Improvement:

- **2-opt-star**: 2-opt that respects time windows
- **Or-opt**: move chains of 1-3 customers
- **Cross-exchange**: swap segments between routes

Metaheuristics:

- **ALNS** (Adaptive Large Neighborhood Search) — typically best in benchmarks
- **Tabu search**

Exact:

- **Branch-and-price-and-cut** with elementary RCSP sub-problems
- Solomon benchmark instances ($n = 100$): solved to optimality

181.35.5. Applications

- **Food delivery** (DoorDash, Uber Eats): tight 30-60 min windows
- **Parcel last-mile** (FedEx, UPS): standard 9-5 windows, dense urban routing
- **Healthcare logistics**: dialysis pickup, home nursing
- **Cash-in-transit**: armored car pickups

- **Field service** (HVAC, telecom): morning / afternoon appointment windows

181.35.6. See also

- **VRP** — without time windows
- **TSP** — single-vehicle ancestor
- **Clarke-Wright** — initial routes

181.36. Clarke-Wright

The **savings algorithm** (Clarke & Wright, 1964) — the classical constructive heuristic for VRP. Simple, fast, surprisingly good for capacitated VRP.

181.36.1. The savings concept

Two single-customer routes: depot $\to i \to$ depot and depot $\to j \to$ depot. Cost = $2c_{0i} + 2c_{0j}$.

Merge into depot $\to i \to j \to$ depot. Cost = $c_{0i} + c_{ij} + c_{0j}$.

Saving:

$$s_{ij} = (2c_{0i} + 2c_{0j}) - (c_{0i} + c_{ij} + c_{0j}) = c_{0i} + c_{0j} - c_{ij}$$

By triangle inequality $s_{ij} \geq 0$. Large savings ⟶ customers i and j are close to each other and far from the depot.

181.36.2. Algorithm

```
Initialize: one route per customer (depot → i → depot)
Compute s[i,j] for all pairs and sort descending
For each (i, j) in sorted order:
    If merging routes containing i and j is feasible:
        - i and j must be on the same "endpoint" of their routes
        - i and j must be on different routes (not the same one)
        - Merged route's total demand ≤ Q (capacity)
    Then merge them
Return routes
```

Sort: $O(n^2 \log n)$. Merge checks: $O(n^2)$ pairs. Total: $O(n^2 \log n)$.

181.36.3. Two variants

Parallel (the original): consider all savings in one sorted list; merge eagerly. Multiple routes grow simultaneously.

Sequential: process one route at a time; finish growing it before starting another. Tends to produce longer routes; sometimes better, sometimes worse than parallel.

181.36.4. Variants & enhancements

- **Modified savings**: $s_{ij} = c_{0i} + c_{0j} - \lambda c_{ij}$ with $\lambda > 1$ — biases toward shorter inter-customer edges
- **Probabilistic Clarke-Wright**: randomize merge order to find better solutions

- **Followed by 2-opt within routes**: standard polish

181.36.5. Real-world significance

For 60 years (1964–today), Clarke-Wright + 2-opt has been the standard **quick-and-dirty** VRP heuristic:

- Implementable in 100 lines of code
- Fast on 1000+ customer instances
- Within 5-10% of optimum for capacitated VRP
- Easy to adapt for time windows, heterogeneous fleets, etc.

Modern heuristics (ALNS, HGS) do better but at significantly more complexity.

181.36.6. Daganzo continuous-approximation alternative

For very large fleets, instead of optimizing each route, use Daganzo's continuous-approximation formulas to design route structure. See Daganzo.

181.36.7. See also

- **VRP** — problem solved
- **TSP** — single-vehicle ancestor
- **VRPTW** — extension to time windows
- **Daganzo** — large-scale alternative

181.37. Daganzo Continuous

A **continuous-approximation** approach to large-scale routing and logistics design (Daganzo, 1984+). Replaces discrete combinatorial optimization with closed-form formulas — sacrifices exactness for **insight** and **fast strategic design**.

181.37.1. The key formula: route length

For n customers uniformly distributed in a region of area A, the **expected length of an optimal TSP tour** satisfies:

$$L \approx k\sqrt{nA}$$

with $k \approx 0.71$ (the **Beardwood-Halton-Hammersley constant**). For uniform Euclidean instances, this constant has been studied extensively.

For **VRP**: each vehicle visits $\approx Q$ customers (where Q is capacity in number of customers). Number of routes $= \frac{n}{Q}$. Each route length:

$$L_{\text{route}} \approx 2r + 0.57\sqrt{QA_{\text{zone}}}$$

where r is depot-to-zone distance and A_{zone} is the area of the zone the route covers.

181.37.2. Why useful

- **Strategic design**: how many vehicles? How big should zones be? Optimal depot location?
- **Sensitivity analysis**: $d\frac{L}{d}n$, etc.
- **Fast benchmarking**: compare network designs without solving thousands of VRPs

181.37.3. Square-root facility-location model

For a region with demand density ρ and warehouse fixed cost f, the optimal **number of facilities** N^* minimizes:

$$Nf + 0.71\sqrt{\rho \frac{A}{N}} \text{ (transport cost per customer scaled by sqrt(area / N))}$$

Differentiating and solving:

$$N^* \propto \sqrt{\rho A f^{-\frac{2}{3}}}$$

— larger regions / denser demand / lower facility cost → more facilities. **Square-root scaling**: doubling demand doesn't double facility count; it multiplies by $\sqrt{2}$.

181.37.4. Other Daganzo formulas

- **Headway design** in transit / dispatching: $H^* = \sqrt{2\frac{c_s + c_h}{\lambda c_w}}$ for cost-of-waiting / dispatch trade-off
- **Hub-spoke vs direct shipping**: cost crossover conditions
- **One-to-many vs many-to-many distribution**

These give **order-of-magnitude correct** answers in seconds, useful when:

- The detailed instance isn't known yet (strategic phase)
- Quick comparison of network designs is needed
- Sensitivity to a parameter matters more than exact cost

181.37.5. Limitations

- **Uniform / smooth demand** assumed — clustering breaks formulas
- **Continuous approximation** — discrete fleet sizes round
- **No time windows / capacities** in basic forms (extensions exist)
- Less useful for **operational** (day-to-day) routing than for **strategic** design

181.37.6. Where it shows up

- **Logistics network design** — placing DCs, sizing warehouses
- **Public transit planning** — bus / rail / paratransit
- **Last-mile design** — Amazon, UPS strategic network choices
- **Reference for VRP heuristic benchmarking**

181.37.7. See also

- **VRP** — operational counterpart
- **Clarke-Wright** — operational heuristic
- **Location Pooling** — strategic trade-off Daganzo formalizes

181.38. Facility Location

A family of combinatorial optimization problems: **where** should warehouses / stores / hospitals / cell towers be located to best serve known demand?

181.38.1. Variants

The variants differ by:

- **Objective**: minimize total cost, minimize maximum distance, maximize coverage
- **Constraints**: facility capacity, customer assignment rules
- **Discrete vs continuous**: candidate locations are finite (p-median) or anywhere in space (center of gravity)

Variant	Objective	Constraint
UFLP	min $\sum f_j y_j + \sum c_{ij} x_{ij}$	uncapacitated
CFLP	same	capacity per facility
p-median	min total assigned distance	exactly p facilities
p-center	min max distance	exactly p facilities
Set covering	min number of facilities	every demand covered
Max covering	max demand covered	exactly p facilities
Center of gravity	min weighted sum of distances	single facility, continuous

181.38.2. Common decision variables

For discrete location problems:

$$y_j = \begin{cases} 1 \text{ if facility } j \text{ is opened} \\ 0 \text{ otherwise} \end{cases}$$

$$x_{ij} = \text{fraction of demand at } i \text{ served by facility } j \text{ (or } 0/1 \text{ for single-source)}$$

181.38.3. Trade-offs

- **Many facilities**: low transport cost, high fixed cost, **less** risk pooling
- **Few facilities**: high transport cost, low fixed cost, more pooling benefit
- **Capacity-constrained**: forces some demand to nearer facilities even when distant ones are cheaper

181.38.4. Where it shows up

- **Supply chain network design** — DC placement
- **Public service** — fire stations, ambulances, schools (p-center for response time)
- **Telecom** — cell tower placement (set covering)
- **Retail** — store location (max covering under budget)
- **Hub-and-spoke logistics** — depots, airports

181.38.5. See also

- **UFLP** / **CFLP** — fixed-charge formulations
- **p-median** / **p-center**
- **Set Covering** / **Max Covering**
- **Center of Gravity** — continuous case
- **Daganzo Continuous** — strategic scale

181.39. UFLP

Uncapacitated Facility Location Problem: open a subset of candidate facilities and assign each customer to one open facility, minimizing fixed + transport costs. No capacity limits.

181.39.1. Formulation

Sets: customers I, candidate facilities J. Costs: f_j (open facility j), c_{ij} (assign customer i to facility j).

Decision variables:

- $y_j \in \{0,1\}$ — open facility j?
- $x_{ij} \in [0,1]$ — fraction of customer i's demand served by j (or $x \in \{0,1\}$ for single-sourcing)

$$\min \sum_j f_j y_j + \sum_i \sum_j c_{ij} x_{ij}$$

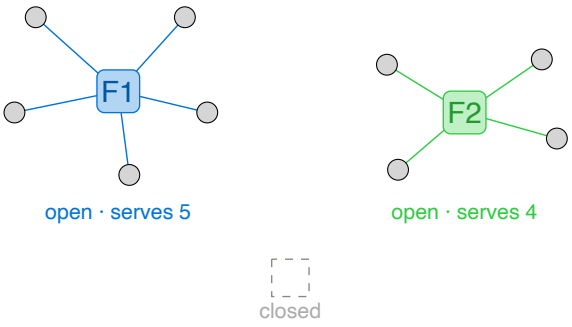
s.t.:

$$\sum_j x_{ij} = 1, \quad \forall i \qquad \text{(every customer fully assigned)}$$

$$x_{ij} \le y_j, \quad \forall i,j \qquad \text{(can only assign to open facilities)}$$

$$y_j \in \{0,1\},\ x_{ij} \ge 0$$

181.39.2. Picture



181.39.3. Structure

Once y is fixed (which facilities are open), the assignment is trivial — each customer chooses its cheapest open facility. So UFLP reduces to choosing a subset of facilities.

For $|J| = n$ candidates: 2^n subsets — NP-hard in general, but tractable up to 1000 facilities with MIP solvers.

181.39.4. Erlenkotter's dual ascent

A classical specialized algorithm (Erlenkotter 1978):

1. Start with weak dual solution
2. Iteratively raise dual variables while maintaining feasibility
3. At each step, dual values determine which facilities **must** be open
4. Heuristic primal construction from dual

Often achieves provably-optimal solutions on instances with 1000+ candidates. Predates modern MIP solvers but still competitive.

181.39.5. LP relaxation

The LP relaxation (drop $y \in \{0, 1\}$ to $y \in [0, 1]$) is a **strong** relaxation — gap typically under 5%. Good starting point for branch-and-cut.

181.39.6. Approximation algorithms

UFLP admits constant-factor approximations (the problem is APX-hard but admits PTAS in some special cases):

- **Greedy** (Mahdian, Markakis, Saberi, Vazirani 2003): 1.61-approximation
- **LP-rounding** / **primal-dual**: 1.5-approximation for **metric** UFLP (Byrka 2007)
- **Open-source**: 1.488-approximation (Li 2013) — current best

For metric UFLP (costs satisfy triangle inequality), 1.46 is the lower bound on what's achievable (Sviridenko-Vyacheslav 2010).

181.39.7. See also

- **CFLP** — with capacities
- **Facility Location overview**
- **p-median** — fixed number of facilities, no fixed cost
- **Lagrange relaxation** — bounds for branch-and-bound

181.40. CFLP

Capacitated Facility Location Problem: like UFLP, but each facility has a maximum capacity it can serve.

181.40.1. Formulation

Add capacity u_j for each facility j. Customer i has demand d_i.

$$\min \sum_j f_j y_j + \sum_i \sum_j c_{ij} x_{ij}$$

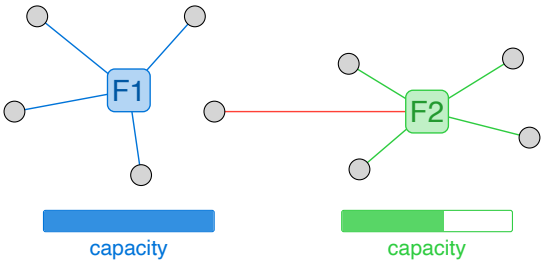
s.t.:

$$\sum_j x_{ij} = 1, \quad \forall i \qquad \text{(customer fully served)}$$

$$\sum_i d_i x_{ij} \le u_j y_j, \quad \forall j \qquad \text{(capacity)}$$

$$y_j \in \{0, 1\},\ x_{ij} \in [0, 1] \ \text{(split delivery) or } \{0, 1\} \ \text{(single sourcing)}$$

181.40.2. Picture



The red link marks an over-capacity assignment that forces a customer onto a farther facility.

181.40.3. Two variants

Split-delivery CFLP: $x_{ij} \in [0, 1]$ — a customer can be served by multiple facilities. LP relaxation is tractable.

Single-source CFLP ($x_{ij} \in \{0, 1\}$): each customer goes to exactly one facility. **Much harder** — even checking feasibility is NP-hard.

181.40.4. Why harder than UFLP

- **Capacity coupling**: opening a small far facility might be forced just to handle overflow from a nearby capacitated one
- **LP relaxation weaker** — capacity constraints reduce the integrality of the LP
- **No simple greedy approximation**

181.40.5. Lagrangian relaxation

Standard solution approach (Geoffrion 1974):

1. **Dualize the customer-assignment constraints** $\sum_j x_{ij} = 1$
2. The relaxed problem **decomposes** by facility — each facility solves a simple knapsack
3. Iteratively update Lagrange multipliers via subgradient method
4. Combine with branch-and-bound for the integer y

Gives strong bounds (often within 1-2% of optimum) and a feasible primal heuristic.

181.40.6. Approximation

For **metric** CFLP, constant-factor approximations exist:

- **Local search** — Korupolu, Plaxton, Rajaraman: 7-approximation
- **Improved local search** — 5.83-approximation (Chudak, Williamson)
- Current best (LP-rounding): 5-approximation (Bansal-Garg-Gupta 2012)

Practical solvers (MIP, Lagrangian) usually beat the worst-case approximation by far on real instances.

181.40.7. Where it shows up

- **Warehouse network design** — each DC has capacity (workforce, square footage, throughput)
- **Plant location** — plants have production limits
- **Cloud / data center sizing** — placement + capacity

- **Healthcare networks** — hospital bed capacity drives location choice

181.40.8. See also

- **UFLP** — uncapacitated case
- **Facility Location overview**
- **Lagrange Relaxation** — standard solution method

181.41. p-Median

p-Median Problem: place exactly p facilities among candidates to minimize the **total** assigned distance / cost. No fixed facility cost — the constraint is "exactly p".

181.41.1. Formulation

Choose p facilities from candidate set J to minimize:

$$\min \sum_i d_i \sum_j c_{ij} x_{ij}$$

s.t.:

$$\sum_j x_{ij} = 1, \quad \forall i$$

$$x_{ij} \le y_j, \quad \forall i, j$$

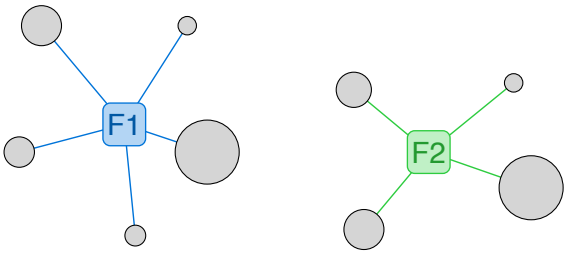
$$\sum_j y_j = p$$

$$y_j \in \{0,1\},\ x_{ij} \ge 0$$

(Here d_i is the demand weight of customer i.)

181.41.2. Picture

Customer disc size $\propto$ demand weight d_i — $p = 2$ facilities chosen to minimize total weighted distance.



181.41.3. Difference from UFLP

- **UFLP**: facility costs are **variable** — pay f_j to open. Optimizer picks the right number.
- **p-Median**: exactly p facilities — number is fixed by the problem. No per-facility fixed cost.

In practice, the two are related: solving p-median for varying p traces out the **cost-vs-number-of-facilities** trade-off curve that UFLP would balance via the fixed costs.

181.41.4. Solution methods

NP-hard. Methods:

- **Branch-and-bound on MIP**
- **LP relaxation** — strong; relaxation often gives integer optima
- **Lagrangian relaxation** — dualize $\sum_j y_j = p$ to get p-median dualizing the cardinality
- **Greedy / interchange heuristics** (Teitz-Bart, swap-based local search)
- **Approximation**: 3-approximation for **metric** p-median (Lin-Vitter, Charikar-Guha)

181.41.5. Applications

- **Distribution center placement** — find best p of n candidates
- **Service location** — fire stations, branches
- **Cell phone tower placement** — when budget fixes count p
- **Vendor consolidation** — pick p suppliers from larger pool

181.41.6. Variants

- **Continuous p-median** (1-median = center of gravity multi-facility versions: Weiszfeld iteration generalized)
- **p-center** (p-center) — minimize **max** distance instead of total
- **Capacitated p-median** — facilities have capacity limits
- **Multi-objective** — trade off coverage / equity / total cost

181.41.7. See also

- **UFLP** — variable count, fixed costs
- **p-center** — minimax variant
- **Center of Gravity** — $p = 1$ continuous case
- **Facility Location overview**

181.42. p-Center

p-Center Problem: place p facilities to **minimize the maximum** distance any customer must travel to its nearest facility. A **minimax** objective — focuses on the worst-served customer, not average.

181.42.1. Formulation

$$\min_{y,x,z} \; z$$

s.t.:

$$z \geq \sum_j c_{ij} x_{ij}, \quad \forall i \qquad (\text{z bounds max assigned distance})$$

$$\sum_j x_{ij} = 1, \quad \forall i$$

$$x_{ij} \le y_j, \ \sum_j y_j = p, \ y_j \in \{0,1\}$$

Variable z is the maximum cost; minimizing z subject to constraints is **minimax**.

181.42.2. Why minimax matters

- **Emergency services**: response time to the worst-located resident matters more than average
- **Equity**: avoid concentrating service in dense areas at the expense of remote customers
- **Service-level agreements**: "no customer more than X miles away" is a hard constraint

vs **p-median** (p-median) which minimizes **total** (= average × N) distance.

181.42.3. Solution methods

Decision-version of p-center: "is there a placement with max distance $\le r$?" This is a **set covering problem**: each facility covers all customers within r, and you need p facilities whose covers union to the full customer set.

Algorithm: binary search on r.

1. Sort all distinct c_{ij} values
2. Binary search: for candidate r, check if a feasible p-coverage exists (set covering feasibility)
3. The smallest feasible r is the optimum

Set covering itself is NP-hard, so this gives an **NP-hard subroutine** — but the binary search keeps the number of feasibility checks small.

181.42.4. Approximation

For **metric** p-center (triangle inequality), 2-approximation is achievable (Hochbaum-Shmoys 1985): greedy farthest-first.

```
Choose first facility: any node
Repeat p-1 times:
    Choose the unselected node FARTHEST from all already-selected facilities
```

This achieves max distance $\le 2 \cdot$ OPT. Tight — no better than 2 is possible unless P = NP.

181.42.5. Applications

- **Fire / ambulance station placement** — minimize worst-case response time
- **Distribution center location** under SLA — "all customers within 200 miles"
- **Schools** — equitable access (no child more than X miles)
- **Cellular networks** with coverage radius

181.42.6. See also

- p-median — total-distance variant
- Set Covering — the feasibility subproblem
- Facility Location overview

181.43. Set Covering

Set Covering Location Problem (LSCP): minimize the number of facilities needed to cover every customer at least once. “Cover” = customer is within an acceptable distance $d_{\max}$ of some facility.

181.43.1. Formulation

Let $N_i = \{j : c_{ij} \le d_{\max}\}$ — candidate facilities that can cover customer i.

$$\min \sum_j y_j$$

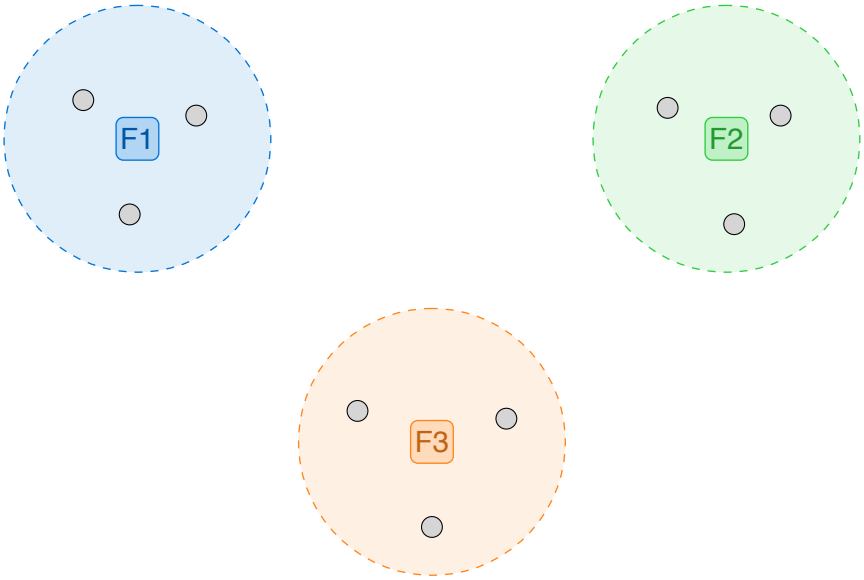
s.t.:

$$\sum_{j \in N_i} y_j \ge 1, \quad \forall i \qquad \text{(each customer covered by } \ge 1 \text{ facility)}$$

$$y_j \in \{0, 1\}$$

181.43.2. Picture

Each facility covers a disc of radius $d_{\max}$; every customer must fall inside at least one open disc.



181.43.3. The general Set Cover problem

LSCP is a special case of Set Cover: given a collection of subsets covering a universe, find the minimum number to use.

Set Cover is one of **Karp's 21 NP-complete problems** — a foundational hard problem in complexity theory.

181.43.4. Greedy approximation

The natural greedy algorithm:

```
At each step: pick the facility that covers the most uncovered customers
Stop when all customers are covered
```

Achieves an H_n-approximation (where $H_n \approx \ln n$ is the harmonic sum). And this is **optimal** — no polynomial algorithm achieves better than $(1 - o(1)) \ln n$ unless P = NP.

181.43.5. Linear programming relaxation

Drop $y_j \in \{0, 1\}$ to $y_j \in [0, 1]$. Solve the LP:

- LP gives a lower bound on integer optimum
- LP value can be a fractional cover (some y_j partially selected)
- **Randomized rounding** converts LP solution to integer solution with $O(\log n)$-factor approximation

181.43.6. Variants

- **Weighted set cover**: each facility j has cost w_j. Minimize $\sum_j w_j y_j$
- **Maximum coverage** (here): inverted — fix facility count p, **maximize** coverage
- **Partial cover**: cover at least k of n customers

181.43.7. Where it shows up

- **Fire / EMS placement** — every household within $d_{\max}$ minutes
- **Cell tower placement** — every demand area covered
- **Sensor placement** — every region monitored
- **Scheduling / shift staffing** — every hour staffed (set-cover on time intervals)
- **Vaccine distribution** — every village within reach

181.43.8. See also

- **Max Covering** — inverted variant
- **p-center** — uses set covering as feasibility subroutine
- **Facility Location overview**

181.44. Max Covering

Max Covering Location Problem (MCLP): place exactly p facilities to **maximize the demand covered**. Inversion of set covering: fix count, maximize coverage.

181.44.1. Formulation

Let d_i = demand at customer i. Let $N_i = \{j : c_{ij} \le d_{\max}\}$ — facilities that can cover i.

Decision variables:

- y_j = open facility j?

- z_i = customer i covered? (auxiliary)

$$\max \sum_i d_i z_i$$

s.t.:

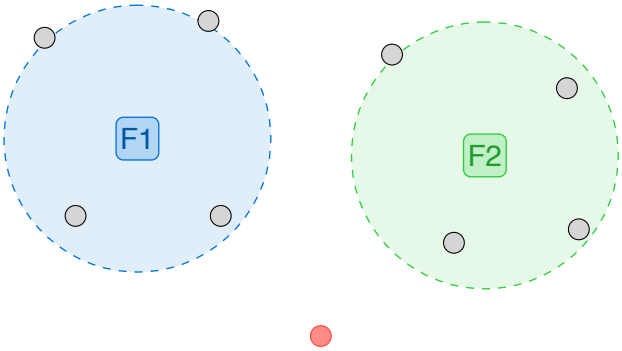
$$z_i \leq \sum_{j \in N_i} y_j, \quad \forall i \qquad \text{(i covered only if at least one nearby facility is open)}$$

$$\sum_j y_j = p$$

$$y_j, z_i \in \{0, 1\}$$

181.44.2. Picture

With $p = 2$ facilities and a fixed coverage radius, a low-demand outlier (red) is left uncovered — the budget is spent capturing the most demand.



181.44.3. Connection to p-center and set covering

Problem	Objective	Fixed
Set covering	minimize number of facilities	cover all demand within $d_{\max}$
p-center	min $d_{\max}$	p facilities, cover all
Max covering	max demand covered	p facilities and $d_{\max}$ fixed

Picks two corners (fix p and $d_{\max}$) and maximizes coverage. Useful when budget **and** service distance are both constraints.

181.44.4. Greedy approximation

Greedy is a $1 - \frac{1}{e} \approx 0.63$ approximation:

```
While p facilities not yet picked:
    Pick the facility that adds the most new covered demand
```

This is **optimal** for max-coverage problems (Nemhauser-Wolsey-Fisher 1978) — no polynomial-time algorithm exceeds $1 - \frac{1}{e}$ unless P = NP.

181.44.5. Submodularity

The function $f(S) =$ "demand covered by set of facilities S" is **monotone submodular**:

$$f(S \cup \{j\}) - f(S) \geq f(T \cup \{j\}) - f(T) \quad \text{for } S \subset T$$

— adding j to a smaller set gives at least as much marginal value as adding it to a bigger set (**diminishing returns**).

The $1 - \frac{1}{e}$ bound is the classical Nemhauser bound for monotone submodular maximization with a cardinality constraint.

181.44.6. Applications

- **Retail** — pick p store locations to capture max market share within an acceptable drive time
- **Cell towers** — p towers to cover the most population
- **Recall logistics** — pick p warehouses to recall product from
- **Influence maximization** — viral marketing, p seed users to maximize reach

181.44.7. See also

- **Set Covering** — dual variant
- **p-center**
- **Facility Location overview**

181.45. Center of Gravity

The **single-facility continuous-space** location problem: place **one** facility anywhere in the plane to minimize total weighted distance to customers.

The optimal location is the **Weber point** (also called the geometric median).

181.45.1. Two flavors

Weighted centroid (squared Euclidean distance — easy):

$$\min \sum_i d_i \, \|x - p_i\|^2 \qquad \Rightarrow \; x^* = \frac{\sum_i d_i p_i}{\sum_i d_i}$$

— closed-form. The **weighted mean**. Used as a **quick first approximation** in network design.

Weber point (Euclidean distance — harder):

$$\min \sum_i d_i \, \|x - p_i\|$$

No closed form. Must iterate. The function is convex; standard methods converge.

181.45.2. Weiszfeld's algorithm

For the Weber point, **Weiszfeld iteration**:

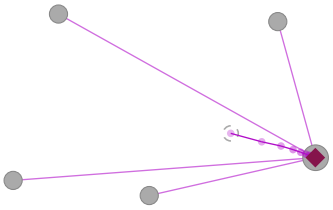
$$x^{(k+1)} = \frac{\sum_i d_i \frac{p_i}{\|} x^{(k)} - p_i\|}{\sum_i \frac{d_i}{\|} x^{(k)} - p_i\|}$$

— a weighted average where weights are **inverse distances**: closer points pull more strongly.

```
Initialize x⁽⁰⁾ at the centroid
Repeat:
    Compute weights w_i = d_i / ||x⁽ᵏ⁾ - p_i||
    x⁽ᵏ⁺¹⁾ ← (Σ w_i p_i) / Σ w_i
Until convergence
```

Converges linearly. Caveat: if any iterate lands **exactly** on a customer point, the formula breaks (denominator $= 0$). Mitigation: small perturbation or alternate update rule when this occurs.

Iterating from the centroid (dashed ring), the trail converges to the Weber point (diamond); spokes connect the optimum to each customer (sized by weight):



181.45.3. Geometric insight

The Weber point is **Pareto-optimal**: no infinitesimal move benefits any single weighted customer without harming the total. The gradient is zero — that's exactly the Weiszfeld fixed-point condition.

181.45.4. Why use L^1 instead?

Sometimes **Manhattan distance** $\sum_i d_i\big(|x_1 - p_{i,1}| + |x_2 - p_{i,2}|\big)$ is more appropriate (grid streets, axis-aligned travel). Then the optimum is the **coordinate-wise median**:

$$x_1^* = \text{weighted median of } p_{i,1}, \quad x_2^* = \text{weighted median of } p_{i,2}$$

— much simpler to compute than the Euclidean Weber point.

181.45.5. Where it shows up

- **Distribution-center placement** — first approximation before discrete refinement
- **Field service hubs**
- **Hospital emergency response**
- **Server placement** in computer networks

In practice, center of gravity gives a continuous-space starting point that's then snapped to the nearest feasible candidate (highway access, zoning, real estate availability).

181.45.6. See also

- p-median — discrete, multi-facility
- Facility Location overview
- Daganzo Continuous Approximation — strategic-level

181.46. Stochastic Programming

Optimization with uncertain parameters whose probability distribution is known. Distinguishes:

- Here-and-now decisions made before the random data is revealed
- Wait-and-see / recourse decisions made after observing the random outcome

181.46.1. Two-stage stochastic LP

$$\min_x c^T x + E_\xi[Q(x,\xi)]$$

where:

$$Q(x,\xi) = \min_y \{q(\xi)^T y : T(\xi)x + Wy = h(\xi),\ y \geq 0\}$$

- x: first-stage (here-and-now) decision — fixed before ξ is observed
- ξ: random parameter
- $y = y(\xi)$: second-stage (recourse) decision — chosen after observing ξ
- $Q(x,\xi)$: optimal recourse cost given x and ξ

The expectation E_ξ aggregates over the distribution of ξ.

181.46.2. Subtopics

- Two-stage recourse — the core model
- Scenario trees — multi-stage extension
- Chance constraints — probabilistic feasibility
- Sample average approximation — solve via Monte Carlo
- EVPI vs VSS — value of information / value of stochastic modeling

181.46.3. When to use

- Decisions must be made before uncertainty resolves — capacity planning, network design, hiring
- Some adjustments are possible later — production levels, shipping, hiring temps
- You want explicit risk management not just nominal expected-value optimization

181.46.4. Alternatives

- Dynamic programming: same as stochastic programming for sequential decisions, but explodes with state-space dimension. SP scales better in continuous variables.
- Robust optimization: hedges against worst-case uncertainty instead of expected case. Less data-hungry; more conservative.
- Scenario analysis: pick a few representative scenarios; solve each deterministically. Crude but easy.

181.46.5. See also

- **Two-stage Recourse**
- **Scenario Trees**
- **Chance Constraints**
- **Sample Average Approximation**
- **EVPI vs VSS**
- **Dynamic Programming**

181.47. Two-stage Recourse

The simplest stochastic program: decide x now, observe random ξ, then adapt with second-stage decision y.

181.47.1. Formulation

$$\min_{x} \; c^T x + E_{\xi}[Q(x, \xi)]$$

where the **recourse function**:

$$Q(x, \xi) = \min_{y} \{ q(\xi)^T y : T(\xi) x + W y = h(\xi), y \geq 0 \}$$

x is **here-and-now** (e.g., build capacity); y is **wait-and-see** (e.g., produce at each plant once demand is known).

181.47.2. Deterministic equivalent (scenarios)

For a finite scenario set $\xi^1, \ldots, \xi^S$ with probabilities p^s:

$$\min_{x, y^1, \ldots, y^S} c^T x + \sum_{s} p^s q(\xi^s)^T y^s$$

s.t.: $T(\xi^s) x + W y^s = h(\xi^s), \quad y^s \geq 0, \quad \forall s.$

A **single large LP** with $|x| + |y| \cdot S$ variables. Can be solved with simplex / interior-point. Problematic when S is large.

181.47.3. Decomposition methods

For large S:

- **L-shaped method (Benders decomposition)**: alternate between a master problem on x and recourse subproblems for each scenario
- **Stochastic dual dynamic programming (SDDP)**: for multi-stage problems, builds piecewise-linear approximations of cost-to-go
- **Progressive hedging**: bundle scenarios and enforce non-anticipativity via Lagrangian relaxation

181.47.4. Worked example

You can install solar capacity x now (cost c per unit). Tomorrow, demand D is realized — uncertain, two scenarios:

- High demand ($p = 0.6$): $D = 100$, recourse buys grid power at 0.20 per unit shortage
- Low demand ($p = 0.4$): $D = 40$, recourse sells excess at 0.10 per unit

Decision: how much x to install? Two-stage SP:

$$\min_x cx + 0.6[0.20 \max(0, 100 - x)] + 0.4[-0.10 \max(0, x - 40)]$$

(Recourse function is the inner min — here it's just the cost given the realized demand.)

Optimal x^* balances the cost of capacity now against expected shortage/surplus cost later. Different from optimizing for the **expected** demand $0.6 \cdot 100 + 0.4 \cdot 40 = 76$.

181.47.5. Why expected-demand optimization is wrong

Optimizing the expected (deterministic) problem misses the **cost asymmetry** between scenarios. Shortage at 0.20 per unit might be much more expensive than surplus at 0.10 per unit savings. SP captures this directly; deterministic doesn't.

This gap is the **Value of Stochastic Solution** (VSS) — see EVPI vs VSS.

181.47.6. See also

- **Stochastic Programming**
- **Scenario Trees**
- **EVPI vs VSS**
- **Decision Trees** — discrete-decision analog

181.48. Scenario Trees

The data structure that represents multi-stage uncertainty in stochastic programming. A **branching tree** where each node represents a state of information at a point in time.

181.48.1. Structure

- **Root**: initial state, $t = 0$, all decisions ahead
- **Internal nodes**: information realized so far at time t
- **Branches**: random outcomes from each node
- **Leaves**: complete realization paths from root to leaf

Each path from root to leaf is a **scenario**. The tree assigns probabilities to outcomes; each leaf's path-probability is the product of branch probabilities.

181.48.2. Example: 3-stage tree

Stage 1 (root): uncertain demand $D_1 \in \{\text{high}, \text{low}\}$ Stage 2: uncertain interest rate $R_2 \in \{\text{up}, \text{down}\}$ Stage 3: terminal payoff depending on both

```
                        ┌── high R → leaf (D=high, R=up)
        ┌── D high (.6)─┤
        │               └── low R → leaf (D=high, R=down)
root ──┤
        │               ┌── high R → leaf (D=low, R=up)
```

```
        └── D low (.4)─┤
                       └── low R → leaf (D=low, R=down)
```

4 scenarios. Decisions at each node, conditional on the information available at that point.

181.48.3. Non-anticipativity

A critical constraint: decisions at a node can depend **only** on information realized **up to that node**. Two scenarios sharing a prefix must have **identical** decisions for that prefix.

Formally: $x_t^{\text{scenario A}} = x_t^{\text{scenario B}}$ if A and B agree up to stage t.

Implementation:

- **Per-node variables** (vs per-scenario): naturally encodes non-anticipativity
- **Per-scenario variables with explicit constraints**: $x_t^s = x_t^{s'}$ for s, s' in the same subtree
- **Progressive hedging**: dualize the non-anticipativity to decompose

181.48.4. Curse of dimensionality

Number of leaves = $\prod$ of branching factors. A 5-stage tree with 5 outcomes per stage has $5^5 = 3,125$ leaves; with 10 outcomes, $10^5 = 100,000$ leaves.

Mitigations:

- **Scenario reduction** — cluster similar scenarios, pick representative ones (Heitsch-Römisch)
- **Sampling** — Monte Carlo-style — see SAA
- **Decomposition** — solve subproblems per scenario, coordinate via Lagrangian (PH, SDDP)
- **Approximate dynamic programming**

181.48.5. Where it shows up

- **Investment planning** — multi-year capital allocation under uncertain returns
- **Power systems** — unit commitment under stochastic demand & renewables
- **Supply chain network design** — multi-year facility planning
- **Financial planning** — asset-liability matching for pensions / insurance
- **Inventory** — multi-period stochastic inventory

181.48.6. See also

- **Stochastic Programming** — broader framework
- **Two-stage Recourse** — simplest case
- **SAA** — Monte Carlo approximation
- **Dynamic Programming** — alternative view

181.49. Chance Constraints

Constraints required to hold **with high probability** rather than always. Useful when:

- Always-feasibility is too expensive

- The data is uncertain and 100% reliability isn't needed (or even possible)
- Service-level requirements are stated probabilistically ("95% of orders shipped on time")

181.49.1. Formulation

For uncertain parameters ξ:

$$P(a(\xi)^T x \leq b(\xi)) \geq 1 - \alpha$$

The constraint $a(\xi)^T x \leq b(\xi)$ must hold with probability at least $1 - \alpha$. α is the **risk level** — typically $\alpha = 0.05$ or 0.01.

Equivalent: violate the constraint with probability at most α.

181.49.2. Joint vs individual chance constraints

Individual: each constraint i holds with high probability, separately:

$$P(a_i^T x \leq b_i) \geq 1 - \alpha_i, \quad \forall i$$

Joint: **all** constraints hold simultaneously with high probability:

$$P(a_i^T x \leq b_i, \forall i) \geq 1 - \alpha$$

Joint is stronger — and harder to handle. Often individual is what's actually intended.

181.49.3. Reformulation under normality

If $a^T x - b$ is normally distributed (with mean $\mu(x)$ and variance $\sigma^2(x)$):

$$P(a^T x \leq b) = \Phi\left(\frac{b - \mu(x)}{\sigma(x)}\right) \geq 1 - \alpha$$

$$\Rightarrow \mu(x) + \Phi^{-1}(1 - \alpha)\sigma(x) \leq b$$

The chance constraint becomes a **deterministic** constraint involving $\sigma(x)$ — possibly convex (e.g., if $\sigma(x)$ is linear in x and $\alpha \leq 0.5$). This is a **second-order cone constraint**, solvable by interior-point methods.

181.49.4. Reformulation via sampling

Without normality, use scenarios $\xi^1, ..., \xi^N$:

$$P(\{a(\xi)^T x \leq b(\xi)\}) \approx \frac{1}{N} \left|\left\{s : a(\xi^s)^T x \leq b(\xi^s)\right\}\right|$$

Enforcing the chance constraint $\geq 1 - \alpha$ on the sample becomes an MILP with binary indicators per scenario — exponential growth, manageable up to $N \approx 1000$.

181.49.5. Equivalence to VaR / CVaR

In risk management, chance constraints relate to:

- **Value at Risk (VaR)**: the $(1 - \alpha)$-quantile of the loss distribution. The chance constraint $P(\text{loss} \leq V) \geq 1 - \alpha$ is essentially $V \geq \text{VaR}_\alpha$.

- **Conditional VaR (CVaR)**: expected loss beyond VaR. Tighter, convex, easier to optimize.

In practice, CVaR-based reformulations of chance constraints are more numerically tractable.

181.49.6. Where chance constraints matter

- **Service-level inventory**: 95% in-stock probability per cycle
- **Robust facility location**: feasible under high-probability demand realizations
- **Portfolio optimization**: $P(\text{loss} \geq V) \leq \alpha$
- **Power systems**: probabilistic generation-load balance
- **Network design**: links carry expected traffic with high probability

181.49.7. See also

- **Stochastic Programming**
- **Two-stage Recourse**
- **SAA**
- **Cycle Service Level** — inventory application

181.50. Sample Average Approximation

Sample Average Approximation (SAA) — replace an intractable expectation in a stochastic program with a Monte Carlo average over scenarios.

181.50.1. Problem

Original stochastic program:

$$z^* = \min_x f(x) + E_\xi[Q(x, \xi)]$$

The expectation is hard (high-dimensional integral; complex distribution).

181.50.2. SAA approximation

Generate N i.i.d. samples $\xi^1, ..., \xi^N$ from the distribution of ξ. Solve:

$$\hat{z}_N = \min_x f(x) + \frac{1}{N} \sum_{s=1}^{N} Q(x, \xi^s)$$

This is just a **deterministic** mathematical program — with N scenarios. Solvable by standard tools (simplex, branch-and-bound, etc.).

181.50.3. Convergence

As $N \to \infty$:

$$\hat{z}_N \to z^* \quad \text{(strong law of large numbers)}$$

with the optimal solution $\hat{x}_N$ also converging to the true optimum x^*. Rate: $O\left(\frac{1}{\sqrt{N}}\right)$ by CLT — the error variance decays as $\frac{1}{N}$, standard error as $\frac{1}{\sqrt{N}}$.

181.50.4. Confidence intervals

The objective value $\hat{z}_N$ from a single SAA run is a **random** estimate. Standard confidence interval:

$$\text{CI} = \left[\hat{z}_N - 1.96\frac{\sigma_N}{\sqrt{N}}, \hat{z}_N + 1.96\frac{\sigma_N}{\sqrt{N}}\right]$$

where σ_N is the sample standard deviation of the recourse values $Q(\hat{x}_N, \xi^s)$.

181.50.5. Practical SAA algorithm

1. Generate N samples
2. Solve SAA problem $\rightarrow \hat{x}_N$
3. Independently evaluate $\hat{x}_N$ on M fresh samples (out-of-sample, $M \gg N$) to estimate true $f(\hat{x}_N) + E[Q(\hat{x}_N, \xi)]$
4. Compare in-sample $\hat{z}_N$ vs out-of-sample estimate. If similar, $\hat{x}_N$ is a good solution. If much worse out-of-sample, increase N.

This **out-of-sample validation** is essential — in-sample optimization can over-fit to the scenarios used.

181.50.6. Choosing N

Trade-off:

- **Larger N**: better approximation, but exploding problem size (number of recourse variables = $|y| \cdot N$)
- **Smaller N**: faster solve, more sample error

Heuristics:

- **Replication**: solve SAA with several smaller N's, take the best. Cheaper than one huge N.
- **Adaptive sampling**: start small, grow N until confidence interval is tight enough
- **Stratified sampling**: sample more from high-impact regions of the distribution to reduce variance

181.50.7. Variance reduction

Plain Monte Carlo has variance $\frac{\sigma^2}{N}$. Tricks to do better:

- **Antithetic variates** — use ξ and $-\xi$ together
- **Importance sampling** — sample more from rare-but-impactful scenarios
- **Common random numbers** — across optimization iterations
- **Quasi-Monte Carlo** (Halton, Sobol sequences) — $O\left(\frac{1}{N}\right)$ convergence instead of $O\left(\frac{1}{\sqrt{N}}\right)$ for smooth integrands

181.50.8. When SAA shines

- **Continuous distributions** of ξ (chance-constrained SP can be cumbersome)
- **Distributions without closed-form moments**
- **Black-box simulators** — easy to sample, hard to integrate analytically

181.50.9. See also

- **Stochastic Programming**
- **Scenario Trees**
- **Monte Carlo Simulation**
- **Two-stage Recourse**

181.51. EVPI vs VSS

Two complementary metrics for **how much uncertainty modeling adds** to a decision problem:

- **EVPI** (Expected Value of Perfect Information) — how much you'd gain by knowing ξ exactly before deciding
- **VSS** (Value of Stochastic Solution) — how much you gain by modeling uncertainty **at all**

181.51.1. Three problem versions

For a stochastic program with random parameter ξ:

	Problem	Decision
SP	$\min_x E_\xi[f(x,\xi)]$	accounts for uncertainty
WS (wait-and-see)	$E_\xi[\min_x f(x,\xi)]$	decide **after** observing ξ
EV (expected value)	$\min_x f(x, E[\xi])$	decide using **average** ξ, ignore variability

Their optimal values: z^{SP}, z^{WS}, z^{EV}.

181.51.2. EVPI

$$\mathrm{EVPI} = z^{\mathrm{SP}} - z^{\mathrm{WS}}$$

(For minimization: lower is better, so EVPI is positive when WS < SP, meaning perfect info would help.)

Interpretation: **upper bound on the value of information**. Knowing ξ in advance enables tailored optimal decisions per scenario — usually does better than committing to one decision before observing.

181.51.3. VSS

$$\mathrm{VSS} = \mathrm{EEV} - z^{\mathrm{SP}}$$

where EEV (Expected result of the Expected-value solution) is the expected cost of using the EV-optimal x:

$$\mathrm{EEV} = E_\xi[f(x^{\mathrm{EV}}, \xi)]$$

Interpretation: **value of modeling uncertainty**. If you ignore variability and just use mean parameters, your decision can be poor when actual ξ deviates. VSS measures this loss.

181.51.4. Why both matter

Situation	EVPI	VSS
Worth investing in better forecasts?	yes if EVPI is large	—
Worth doing stochastic programming instead of EV?	—	yes if VSS is large
EVPI small, VSS small	decision is robust to uncertainty	
EVPI large, VSS small	forecasting is helpful, but you'd make the same decision anyway	
EVPI small, VSS large	can't improve via forecasting, but must explicitly model variability	
Both large	strongest case for both better data **and** stochastic modeling	

181.51.5. Worked example

From two-stage recourse example:

- z^{SP}: solve stochastic program → optimal x^{SP} and expected cost
- z^{WS}: best x in each scenario separately, average over scenarios
- z^{EV}: solve with $E[D]$ replacing D; then compute **actual** expected cost = EEV

A small calculation gives concrete EVPI and VSS. The relative magnitudes depend on:

- Spread of scenarios (larger spread → larger EVPI/VSS)
- Cost asymmetry between scenarios (asymmetric → larger VSS)
- Convexity of cost in ξ (concave → larger EVPI, by Jensen)

181.51.6. Sanity check identities

- **EVPI** ≥ 0 always (perfect info can't hurt)
- **VSS** ≥ 0 always (stochastic modeling can't hurt)
- **VSS** $\leq \mathrm{EEV} - z^{\mathrm{WS}}$ (stochastic is no better than perfect info)

181.51.7. See also

- **Stochastic Programming**
- **Two-stage Recourse**
- **EVPI (Decision Analysis)** — discrete analog
- **EVSI** — value of sample info

181.52. Lagrange Relaxation

2 types of points:

- **Interior** point
- **Boundary** point

181.52.1. Single Variate

For constrained single variate:

- **Check convexity**: Verify whether the objective function is convex by inspecting the second derivative. Convexity ensures that any stationary point is a global minimum
- **Find unconstrained minimizer**: Solve the first-order condition (FOC), $f'(x) = 0$, to identify the unconstrained optimal solution x^*.
- Apply feasibility:
    - If x^* lies within the feasible region, it is the constrained optimum.
    - If x^* is infeasible, the constrained optimum is attained at the feasible boundary point closest to x^* (since convexity guarantees the function grows monotonically away from the minimizer).

Example

$$\min_{x \geq 0} f(x) = x^2 + 2x - 3$$

Derivatives:

$$f'(x) = 2x + 2$$
$$f''(x) = 2$$

Thus, f is convex.

FOC solution:

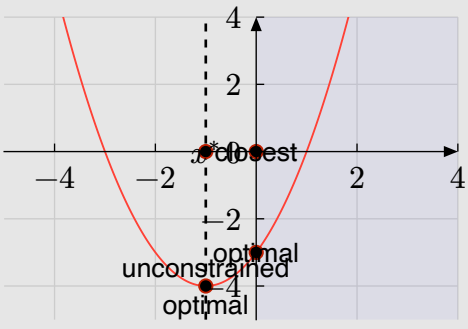
$$f'(x) = 0 \quad \Rightarrow \quad x^* = -1$$

This is the unconstrained minimizer, but it violates $x \geq 0$:

- Since $x^* = -1$ lies outside the feasible set, the closest feasible point is the boundary $x = 0$.

Evaluation:

$$f(0) = -3$$



So the constrained optimum is at $x = 0$ with objective value -3.

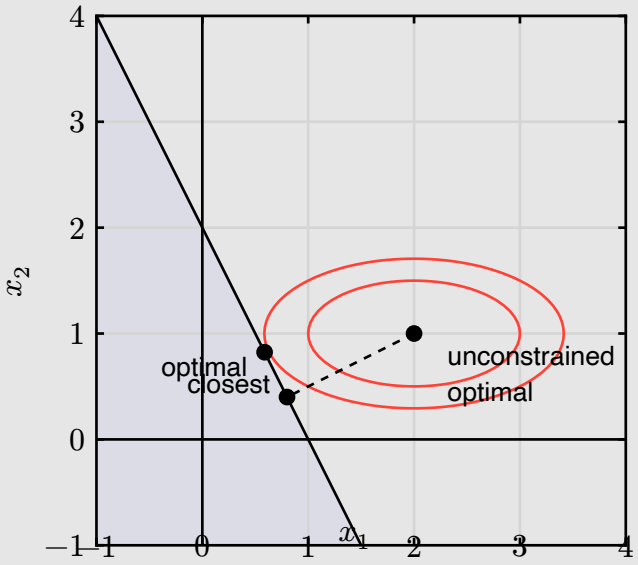
181.52.2. Multi Variate

Example

$$\begin{aligned} \min_{x \in \mathbb{R}^2} \quad & f(x) = (x_1 - 2)^2 + 4(x_2 - 1)^2 \\ s.t. \quad & x_1 + 2x_2 \leq 2 \end{aligned}$$

For this CP, the FOC solution $x = (2, 1)$ is infeasible

The closest feasible solution is **not** optimal



$$\begin{aligned} z^* = \max_{x \in \mathbb{R}^n} \quad & f(x) \\ s.t. \quad & g_{i(x)} \leq b_i \quad \forall i = 1, ..., m \end{aligned}$$

Replace **hard** constraints with **soft** constraints

$$\begin{aligned} z^L(\lambda) = \max_{x \in \mathbb{R}^n} \quad & f(x) + \sum_{i=1}^{m} \lambda_i \left[b_i - g_{i(x)} \right] \\ & \lambda_i \geq 0 \end{aligned}$$

Lagrangian

$$\mathcal{L}(x \mid \lambda) = f(x) + \sum_{i=1}^{m} \lambda_i \left[b_i - g_{i(x)} \right]$$

λ_i is the Lagrangian multiplier

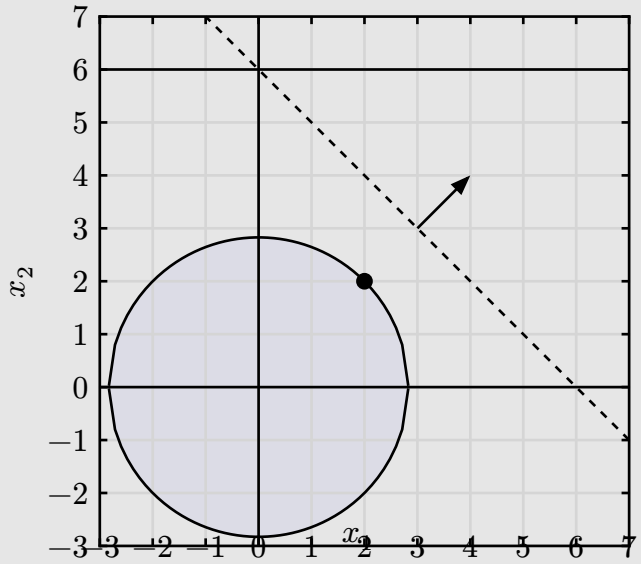
For a minimization problem

$$z^* = \min_{x \in \mathbb{R}^n} \quad \left\{ f(x) \mid g_{i(x)} \leq b_i \quad \forall i = 1, ..., m \right\}$$

$$\begin{aligned} z^L(\lambda) = \max_{x \in \mathbb{R}^n} \quad & f(x) + \sum_{i=1}^{m} \lambda_i \left[b_i - g_{i(x)} \right] \\ & \lambda_i \leq 0 \end{aligned}$$

Example

$$\begin{aligned} z^* = \max \quad & x_1 + x_2 \\ s.t. \quad & x_1^2 + x_2^2 \leq 8 \\ & x_2 \leq 6 \end{aligned}$$



Original NLP

$$z^* = \max_{x \in \mathbb{R}^2} \{x_1 + x_2 \mid x_1^2 + x_2^2 \leq 8, x_2 \leq 6\}$$

Given the Lagrangian multipliers $\lambda = \{\lambda_1, \lambda_2\} \geq 0$, the Lagrangian is:

$$\mathcal{L}(x \mid \lambda) = \underbrace{x_1 + x_2}_{f} + \lambda_1 \underbrace{(8 - x_1^2 - x_2^2)}_{b_1 - g_1} + \lambda_2 \underbrace{(6 - x_2)}_{b_2 - g_2}$$

We may then solve

$$z^L(\lambda) = \max_{x \in \mathbb{R}^2} \mathcal{L}(x \mid \lambda)$$

given any $\lambda \geq 0$.

E.g.:

-

181.52.3. Bounds

Lagrance relaxation provides bounds for the original NLP

Weak Duality of Lagrangian Relaxation

$$z^L(\lambda) \geq z^* \quad \forall \lambda \geq 0$$

Proof

$$
\begin{aligned}
z^* = \max_{x \in \mathbb{R}^n} &= \{f(x) \mid g_i(x) \leq b_i \quad \forall i = 1, ..., m\} \\
&\leq \max_{x \in \mathbb{R}^n} = \left\{ f(x) + \sum_{i=1}^{m} \lambda_i [b_i - g_i(x)] \mid g_i(x) \leq b_i \quad \forall i = 1, ..., m \right\} \\
&\leq \max_{x \in \mathbb{R}^n} = \left\{ f(x) + \sum_{i=1}^{m} \lambda_i [b_i - g_i(x)] \right\} = z^L(\lambda)
\end{aligned}
$$

Given that $x^L(\lambda) \geq z^*$ for all $\lambda \geq 0$, the **Lagrange dual program** is defined as:

$$
\min_{\lambda \geq 0} z^L(\lambda)
$$

The lagrange multipliers are the **dual variables** in NLP

181.53. Lagrangian

$$
\nabla f = \lambda \nabla g
$$
$$
g = 0
$$

Constrained optimization problem:

$$
\begin{aligned}
\min \quad & f(x) \\
s.t. \quad & g_i(x) = 0 \quad \forall i = 1, ..., m \\
& h_j(x) = 0 \quad \forall j = 1, ..., p
\end{aligned}
$$

- $x \in \mathbb{R}^n$: decision variables
- $f(x)$: objective function
- $g_i(x) = 0$: equality constraints
- $h_i(x) \leq 0$: inequality constraints

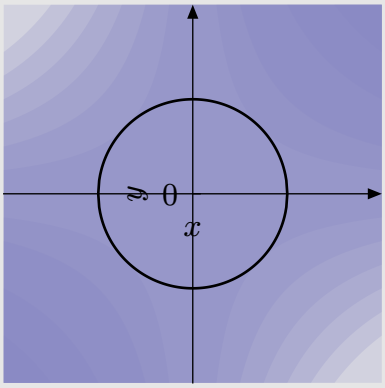
Lagrangian $\mathcal{L}(x, \lambda, \mu)$:

$$
\mathcal{L}(x, \lambda, \mu) = \underbrace{f(x)}_{\text{objective function}} + \underbrace{\sum_{i=1}^{m} \lambda_i g_i(x)}_{\text{equality constraints}} + \underbrace{\sum_{j=1}^{p} \mu_j h_j(x)}_{\text{inequality constraints}}
$$

- λ_i: Lagrangian multipliers for equality constraints
- μ_i: Lagrangian multipliers for inequality constraints

Example

Example



181.54. KKT Conditions

Conditions that a solution must satisfy in order to be optimal for a nonlinear optimization problem

Conditions are necessary for optimality, and sufficient if:

- f is convex
- g_i are convex
- h_j linear

181.54.1. Setup

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & f(x) \\ s.t. \quad & g_i(x) \leq 0 \quad \forall i = 1, ..., m \\ & h_j(x) = 0 \quad \forall j = 1, ..., p \end{aligned}$$

- $f(x)$: objective function
- $g_i(x)$: inequality constraints
- $h_j(x)$: equality constraints

181.54.2. KKT Multipliers

We introduce:

- $\lambda_i \geq 0$: for each inequality constraint (Lagrange multipliers)
- $\mu_i \in \mathbb{R}$: for each equality constraint

The Lagrangian is:

$$\mathcal{L}(x, \lambda, \mu) = f(x) + \sum_{i=1}^{m} \lambda_i g_i(x) + \sum_{j=1}^{p} \mu_j h_j(x)$$

For n variables and m constrains:

- n primal variables (x) and m dual variables (λ)
- n equalities for dual feasibility
- m equalities for complementary slackness

181.54.3. KKT Conditions

At local optimum x^* there exists multipliers (λ^*, μ^*) such that:

1. **Stationarity**

Minimization:

$$\nabla f(x^*) + \sum_{i=1}^{m} \lambda_i^* \nabla g_i(x^*) + \sum_{j=1}^{p} \mu_j^* \nabla h_j(x^*) = 0$$

Stationarity is like a generalized version of "set the derivative to zero", but accounting for the constraints

2. **Primal feasibility**

$$g_i(x^*) \leq 0$$
$$h_j(x^*) = 0$$

The solution must satisfy all the original constraints — it must lie inside the feasible region

3. **Dual Feasibility**

$$\lambda_i \geq 0 \quad \forall i$$

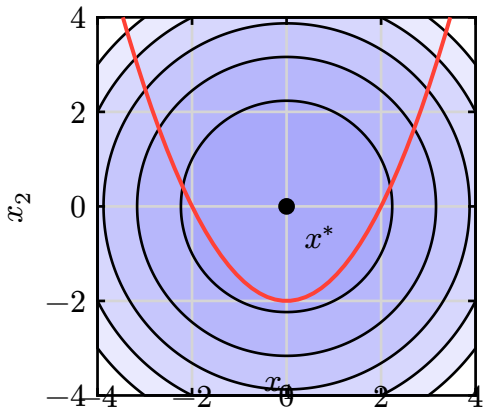
4. **Complementary Slackness**

$$\lambda_i g_i(x^*) = 0 \quad \forall i$$

Complementary slackness only applies to inequalities

Inactive (non-binding) constraint

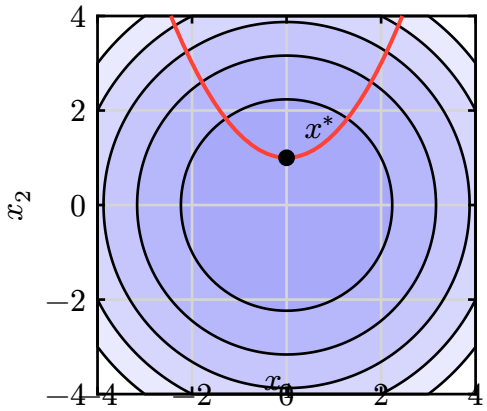
$$g_i(x^*) < 0 \quad \Rightarrow \quad \lambda_i^* = 0$$



- The constraint does not affect the optimum
- The feasible region is "loose" at the optimum — the optimum lies strictly inside it
- Economically, the shadow price is zero: relaxing the constraint wouldn't change the objective

Active (binding) constraint

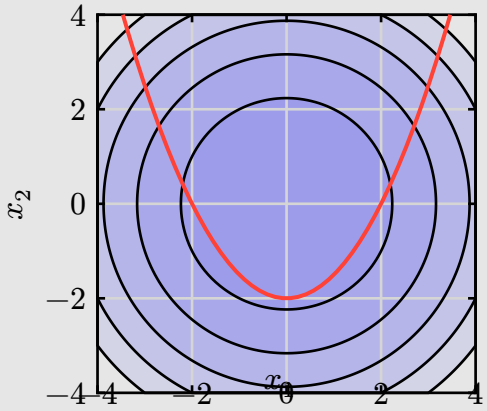
$$g_i(x^*) = 0 \quad \Rightarrow \quad \lambda_i^* \geq 0$$



- The constraint is tight at the optimum — it holds with equality
- λ_i^* is the shadow price:
    - It measures how much the objective value would improve per unit relaxation of the constraint
        - The objective would decrease by λ_i^* per unit of relaxation (if convex)

Example

$$\begin{aligned} \min \quad & x_1^2 + x_2^2 \\ s.t. \quad & \frac{x_1^2}{2} - x_2 \leq 2 \end{aligned}$$



Step 1. Standardize constraints

Constraints already in the form $g(x) \leq 0$ for minimization

Sign convention $\lambda \geq 0$

Step 2. Formulate Lagrangian

$$\mathcal{L}(x_1, x_2, \lambda) = x_1^2 + x_2^2 + \lambda \left(\frac{x_1^2}{2} - x_2 - 2 \right)$$

Step 3. Stationarity

$$\nabla_x \mathcal{L} = \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial x_1} \\ \frac{\partial \mathcal{L}}{\partial x_2} \end{bmatrix} = \begin{bmatrix} 2x_1 + \lambda x_1 \\ 2x_2 - \lambda \end{bmatrix} = 0$$

Solve for x_1:

$$\begin{aligned} 2x_1 + \lambda x_1 &= 0 \\ x_1(2 + \lambda) &= 0 \\ x_1(2 + \lambda) &= \frac{0}{2 + \lambda} \\ x_1 &= 0 \end{aligned}$$

Solve for x_2

$$\begin{aligned} 2x_2 - \lambda &= 0 \\ x_2 &= \frac{\lambda}{2} \end{aligned}$$

Candidate point:

$$(x_1, x_2) = \left(0, \frac{\lambda}{2}\right)$$

Step 4. Primal Feasibility

Constraint:

$$g(x_1, x_2) = \frac{x_1^2}{2} - x_2 - 2 \leq 0$$

Substitute $x_1 = 0$ and $x_2 = \frac{\lambda}{2}$

$$\begin{aligned} \frac{0^2}{2} - \frac{\lambda}{2} - 2 &\leq 0 \\ -\frac{\lambda}{2} - 2 &\leq 0 \\ -\frac{\lambda}{2} &\leq 2 \\ \lambda &\geq -4 \end{aligned}$$

Step 5. Dual Feasibility

$$\lambda \geq 0$$

Step 6. Complementary Slackness

Constraint:

$$g(x_1, x_2) = \frac{x_1^2}{2} - x_2 - 2 \leq 0$$

Condition:

$$\lambda \cdot g(x^*) = 0$$

Candidate point:

$$(x_1, x_2) = \left(0, \frac{\lambda}{2}\right)$$

Evaluate $g(x^*)$:

$$\lambda \cdot g(x^*) = \lambda\left(\frac{x_1^2}{2} - x_2 - 2\right) = 0$$

Substitute $x_1 = 0$ and $x_2 = \frac{\lambda}{2}$

$$\lambda\left(\frac{0^2}{2} - \frac{\lambda}{2} - 2\right) = 0$$

$$\lambda\left(-\frac{\lambda}{2} - 2\right) = 0$$

Solve:

$$\lambda\left(-\frac{\lambda}{2} - 2\right) = 0 \quad \Rightarrow \quad \lambda = 0 \text{ or } \lambda = -4$$

But because of Dual Feasibility $\lambda \geq 0$, so $\lambda = 0$

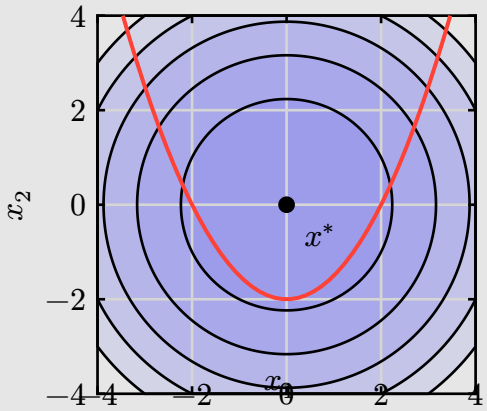
Constraint is non-binding at optimum:

- $\lambda = 0$
- $g(x^*) < 0$

$$\begin{aligned} g(x^*) = \frac{x_1^2}{2} - x_2 - 2 &\leq 0 \\ \frac{0^2}{2} - \frac{\lambda}{2} - 2 &\leq 0 \\ \frac{0^2}{2} - \frac{0}{2} - 2 &\leq 0 \\ -2 &\leq 0 \end{aligned}$$

Step 7. Solve for Optimum

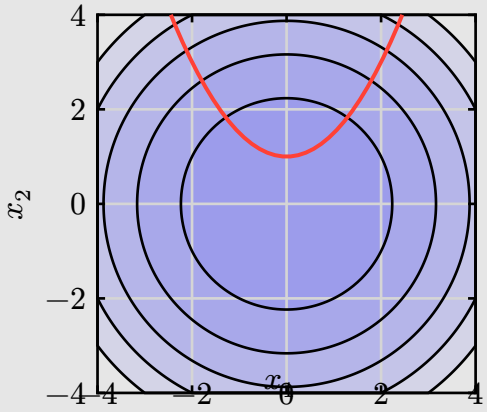
$$\begin{aligned} (x_1, x_2) &= \left(0, \frac{\lambda}{2}\right) \\ &= \left(0, \frac{0}{2}\right) \\ &= (0, 0) \end{aligned}$$



Step 8. Check Convexity

Example

$$\begin{aligned} \min \quad & x_1^2 + x_2^2 \\ s.t. \quad & \frac{x_1^2}{2} - x_2 \leq -1 \end{aligned}$$



Step 1. Standardize constraints

Constraints already in the form $g(x) \leq 0$ for minimization

Sign convention $\lambda \geq 0$

Step 2. Formulate Lagrangian

$$\mathcal{L}(x_1, x_2, \lambda) = x_1^2 + x_2^2 + \lambda \left(\frac{x_1^2}{2} - x_2 + 1 \right)$$

Step 3. Stationarity

$$\nabla_x \mathcal{L} = \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial x_1} \\ \frac{\partial \mathcal{L}}{\partial x_2} \end{bmatrix} = \begin{bmatrix} 2x_1 + \lambda x_1 \\ 2x_2 - \lambda \end{bmatrix}$$

Solve for x_1:

$$
\begin{aligned}
2x_1 + \lambda x_1 &= 0 \\
x_1(2+\lambda) &= 0 \\
x_1 &= \frac{0}{2+\lambda} \\
x_1 &= 0
\end{aligned}
$$

Solve for x_2:

$$
\begin{aligned}
2x_2 - \lambda &= 0 \\
2x_2 &= \lambda \\
x_2 &= \frac{\lambda}{2}
\end{aligned}
$$

Candidate point:

$$
(x_1, x_2) = \left(0, \frac{\lambda}{2}\right)
$$

Step 4. Primal Feasibility

Constraint:

$$
g(x_1, x_2) = \frac{x_1^2}{2} - x_2 + 1 \leq 0
$$

Substitute $x_1 = 0$ and $x_2 = \frac{\lambda}{2}$:

$$
\begin{aligned}
\frac{x_1^2}{2} - x_2 + 1 &\leq 0 \\
\frac{0^2}{2} - \frac{\lambda}{2} + 1 &\leq 0 \\
-\frac{\lambda}{2} + 1 &\leq 0 \\
-\frac{\lambda}{2} &\leq -1 \\
\lambda &\geq 2
\end{aligned}
$$

Step 5. Dual Feasibility

$$
\lambda \geq 0
$$

Step 6. Complementary Slackness

Constraint:

$$
g(x_1, x_2) = \frac{x_1^2}{2} - x_2 + 1 \leq 0
$$

Condition:

$$
\lambda \cdot g(x^*) = 0
$$

Candidate point:

$$(x_1, x_2) = \left(0, \frac{\lambda}{2}\right)$$

Evaluate $g(x^*)$:

$$\lambda \cdot g(x^*) = \lambda\left(\frac{x_1^2}{2} - x_2 + 1\right) = 0$$

Substitute $x_1 = 0$ and $x_2 = \frac{\lambda}{2}$

$$\lambda\left(\frac{0^2}{2} - \frac{\lambda}{2} + 1\right) = 0$$

$$\lambda\left(-\frac{\lambda}{2} + 1\right) = 0$$

Solve:

$$\lambda\left(-\frac{\lambda}{2} + 1\right) = 0 \quad \Rightarrow \quad \lambda = 0 \text{ or } \lambda = 2$$

But because of Primal Feasibility, if $\lambda = 0$, $(x_1, x_2) = (0, 0)$ and $g(0, 0) = 1 \nleq 0$, thus $\lambda = 2$

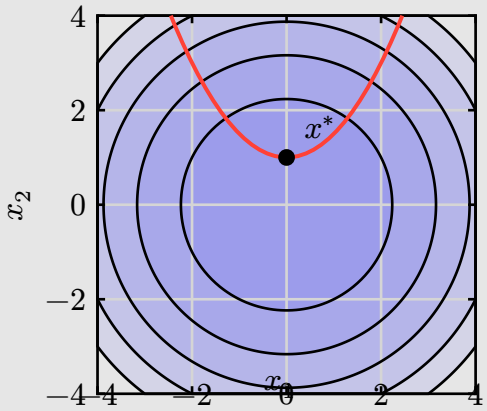
Constraint is binding at optimum:

- $\lambda = 2$
- $g(x^*) = 0$

$$\begin{aligned} g(x^*) = \frac{x_1^2}{2} - x_2 + 1 &= 0 \\ \frac{0^2}{2} - \frac{\lambda}{2} + 1 &= 0 \\ \frac{0^2}{2} - \frac{2}{2} + 1 &= 0 \\ 0 &= 0 \end{aligned}$$

Step 7. Solve for Optimum

$$\begin{aligned} (x_1, x_2) &= \left(0, \frac{\lambda}{2}\right) \\ &= \left(0, \frac{2}{2}\right) \\ &= (0, 1) \end{aligned}$$



Step 8. Check Convexity

181.54.4. Calculating Lagrangian Multipliers

To find the multipliers (λ^*, μ^*) and the optimal point x^*, solve the KKT system of equations:

$$\begin{cases} \nabla_x \mathcal{L}(x, \lambda, \mu) = 0 & \\ g_i(x) \leq 0 & \forall i = 1, ..., m \\ h_j(x) = 0 & \forall i = 1, ..., p \\ \lambda_i \geq 0 & \forall i = 1, ..., m \\ \lambda_i g_i(x) = 0 & \forall i = 1, ..., m \end{cases}$$

Example

Retailer Maximizing Profit under a Capacity Constraint

A retailer sells product 1 and 2 with quantities q_1 and q_2. For product i the market-clearing prices are:

$$p_i = a_i - b_i q_i \quad i = 1, 2$$

Where $a_i, b_i > 0$.

The retailer maximizes **total profit** subject to a **capacity constraint**:

$$q_1 + q_2 \leq K \quad q_1, q_2 \geq 0$$

1. Formulation

$$\begin{aligned} \max_{q_1, q_2 \geq 0} \quad & q_1 \overbrace{(a_1 - b_1 q_1)}^{p_1} + q_2 \overbrace{(a_2 - b_2 q_2)}^{p_2} \\ s.t. \quad & q_1 + q_2 \leq K \end{aligned}$$

The objective is concave because

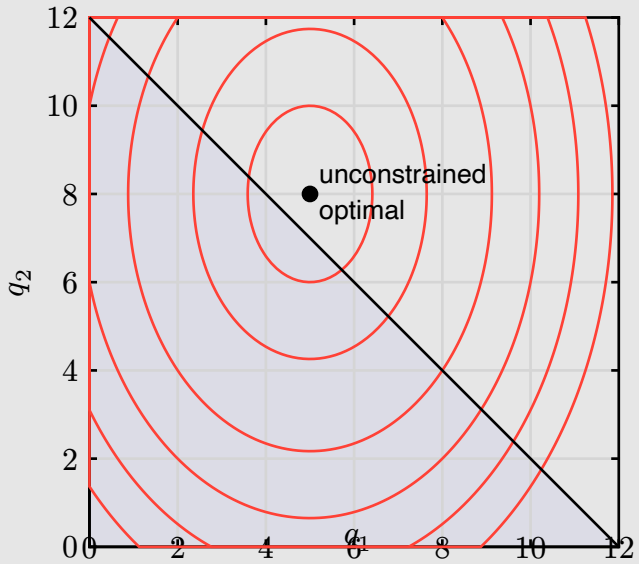
$$-\nabla^2 f(q_1, q_2) = \begin{bmatrix} 2b_1 & 0 \\ 0 & 2b_2 \end{bmatrix}$$

is positive semi-definite.

The unconstrained FOC solution is:

$$q_1 = \frac{a_1}{2b_1}, \quad q_2 = \frac{a_2}{2b_2}$$

If $q_1 + q_2 \leq K$, this solution is feasible.



2. Lagrangian

Introduce KKT multiplier $\lambda \geq 0$ for the inequality constraint:

$$\mathcal{L}(q_1, q_2, \lambda) = \overbrace{q_1(a_1 - b_1 q_1) + q_2(a_2 - b_2 q_2)}^{f} + \lambda \overbrace{(K - q_1 - q_2)}^{b-g}$$

3. KKT Conditions

1. Stationarity

$$\frac{\partial}{\partial q_1}\mathcal{L} = a_1 - 2b_1 q_1 - \lambda = 0$$
$$\frac{\partial}{\partial q_2}\mathcal{L} = a_2 - 2b_2 q_2 - \lambda = 0$$

2. Primal Feasibility

$$q_1 + q_2 \leq K \quad q_1, q_2 \geq 0$$

3. Dual Feasibility

$$\lambda \geq 0$$

4. Complementary Slackness

$$\lambda(K - q_1 - q_2) = 0$$

4. Solving KKT system

Case 1: Unconstrained ($\lambda = 0$)

$$\max_{q_1, q_2 \geq 0} \quad f(q_1, q_2) = q_1(a_1 - b_1 q_1) + q_2(a_2 - b_2 q_2)$$

$$f(q_1, q_2) = a_1 q_1 - b_1 q_1^2 + a_2 q_2 - b_2 q_2^2$$

First-Order Condition

$$\frac{\partial f}{\partial q_1} = a_1 - 2b_1 q_1$$

$$\frac{\partial f}{\partial q_2} = a_2 - 2b_2 q_2$$

Set derivatives to 0 (stationarity)

$$a_1 - 2b_1 q_1 = 0 \quad \Rightarrow \quad q_1^* = \frac{a_1}{2b_1}$$

$$a_2 - 2b_2 q_2 = 0 \quad \Rightarrow \quad q_2^* = \frac{a_2}{2b_2}$$

Unconstrained Solution:

$$q_1^* = \frac{a_1}{2b_1}$$

$$q_2^* = \frac{a_2}{2b_2}$$

The second derivative check confirms it's a maximum because:

$$\frac{\partial^2 f}{\partial q_1^2} = -2b_1 < 0$$

$$\frac{\partial^2 f}{\partial q_2^2} = -2b_2 < 0$$

Feasible if

$$\frac{a_1}{2b_1} + \frac{a_2}{2b_2} \leq K$$

Case 2: Binding Constraint ($\lambda > 0$)

We now include the capacity constraint:

$$q_1 + q_2 = K$$

If it binds (active) then

$$q_1 + q_2 = K$$

Lagrangian

$$\mathcal{L}(q_1, q_2, \lambda) = \overbrace{q_1(a_1 - b_1 q_1) + q_2(a_2 - b_2 q_2)}^{f(q_1,q_2)} + \lambda \overbrace{(K - q_1 - q_2)}^{b - g(q_1,q_2)}$$

First-Order Condition

$$\frac{\partial \mathcal{L}}{\partial q_1} = a_1 - 2b_1 q_1 - \lambda$$
$$\frac{\partial \mathcal{L}}{\partial q_2} = a_2 - 2b_2 q_2 - \lambda$$
$$\frac{\partial \mathcal{L}}{\partial \lambda} = K - q_1 - q_2$$

Set derivatives to 0 (stationarity)

$$a_1 - 2b_1 q_1 - \lambda = 0$$
$$a_2 - 2b_2 q_2 - \lambda = 0$$
$$K - q_1 - q_2 = 0$$

When we take the derivative of the Lagrangian with respect to the multiplier λ we recover the constraint itself.

Solve system of equations

$$\begin{cases} a_1 - 2b_1 q_1 - \lambda = 0 \\ a_2 - 2b_2 q_2 - \lambda = 0 \\ q_1 + q_2 = K \end{cases}$$

Solution:

$$q_1 = \frac{2b_2 K + a_1 - a_2}{2(b_1 + b_2)} \qquad q_2 = \frac{2b_1 K + a_2 - a_1}{2(b_1 + b_2)}$$

Feasible if

$$\frac{a_1}{2b_1} + \frac{a_2}{2b_2} > K$$

5. Optimal Solution

$$(q_1^*, q_2^*) = \begin{cases} \left(\frac{a_1}{2b_1}, \frac{a_2}{2b_2}\right) & \text{if } \frac{a_1}{2b_1} + \frac{a_2}{2b_2} \leq K \\ \left(\frac{2b_2 K + a_1 - a_2}{2(b_1+b_2)}, \frac{2b_1 K + a_2 - a_1}{2(b_1+b_2)}\right) & \text{otherwise} \end{cases}$$

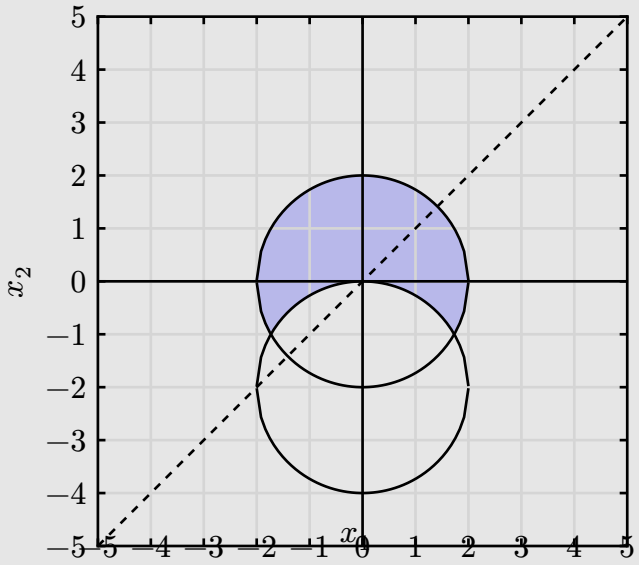
Intuition:

- If capacity K is large enough, the retailer produces unconstrained quantities.
- If K is tight, the total production hits the limit, and the optimal quantities are shared according to the relative parameters a_i, b_i

Example

$$
\begin{aligned}
\max \quad & f(x) = x_1 - x_2 \\
s.t. \quad & g_1(x) = x_1^2 + x_2^2 \leq 4 \\
& g_2(x) = -x_1^2 - (x_2 + 2)^2 \leq -4
\end{aligned}
$$

This NLP is **nonconvex**



1 Lagrangian

Introduce multipliers $\lambda_1, \lambda_2 \geq 0$ for the inequalities. The Lagrangian is

$$
\mathcal{L}(x_1, x_2, \lambda) = \overbrace{x_1 - x_2}^{f} + \lambda_1 \overbrace{(4 - x_1^2 - x_2^2)}^{g_1} + \lambda_2 \overbrace{\left(-4 + x_1^2 + (x_2 + 2)^2\right)}^{g_2}
$$

The solution $\bar{x}$ is a local maximum only if there exists λ such that

2 KKT Conditions

1. Primal feasibility

$$
\begin{aligned}
& x_1^2 + x_2^2 \leq 4 && \text{(PF-1)} \\
& -x_1^2 - (x_2 + 2)^2 \leq -4 && \text{(PF-2)}
\end{aligned}
$$

2. Dual Feasibility

$$
\begin{aligned}
& \lambda_1 \geq 0 && \text{(DFS-1)} \\
& \lambda_2 \geq 0 && \text{(DFS-2)}
\end{aligned}
$$

3. Stationarity

$$
\begin{aligned}
& \frac{\partial \mathcal{L}}{\partial x_1} = 1 - 2(\lambda_1 - \lambda_2)x_1 = 0 && \text{(DFF-1)} \\
& \frac{\partial \mathcal{L}}{\partial x_2} = -1 - 2(\lambda_1 - \lambda_2)x_2 + 4\lambda_2 = 0 && \text{(DFF-2)}
\end{aligned}
$$

4. Complimentary Slackness

$$\lambda_1(4 - x_1^2 - x_2^2) = 0 \qquad \text{(CS-1)}$$
$$\lambda_2\left(-4 + x_1^2 + (x_2+2)^2\right) = 0 \qquad \text{(CS-2)}$$

3️⃣ Examine all 4 cases for (λ_1, λ_2)

Case 1. $(\lambda_1 > 0, \lambda_2 > 0)$

Step 1: Complementary Slackness

Since both multipliers are positive, the corresponding constraints must be active:

$$x_1^2 + x_2^2 = 4 \qquad \text{(CS-1)}$$
$$x_1^2 + (x_2+2)^2 = 4 \qquad \text{(CS-2)}$$

Solve this system → two candidate points:

$$(x_1, x_2) = \left(\sqrt{3}, -1\right) \quad \text{and} \quad \left(-\sqrt{3}, -1\right)$$

Step 2: Primal Feasibility

1. For $\left(\sqrt{3}, -1\right)$

$$x_1^2 + x_2^2 = \sqrt{3}^2 + (-1)^2 \le 4 \qquad \text{(PF-1)}$$
$$-x_1^2 - (x_2+2)^2 = -\left(\sqrt{3}\right)^2 - (-1+2)^2 \le -4 \qquad \text{(PF-2)}$$

2. For $\left(-\sqrt{3}, -1\right)$

$$x_1^2 + x_2^2 = \left(-\sqrt{3}\right)^2 + (-1)^2 \le 4 \qquad \text{(PF-1)}$$
$$-x_1^2 - (x_2+2)^2 = -\left(-\sqrt{3}\right)^2 - (-1+2)^2 \le -4 \qquad \text{(PF-2)}$$

✅ Both satisfy primal feasibility.

Step 3: Stationarity / First-order conditions (DFF)

Plug each candidate point into the gradient equations and solve the system:

$$1 - 2(\lambda_1 - \lambda_2)x_1 = 0 \qquad \text{(DFF-1)}$$
$$-1 - 2(\lambda_1 - \lambda_2)x_2 + 4\lambda_2 = 0 \qquad \text{(DFF-2)}$$

For $\left(\sqrt{3}, -1\right)$:

$$1 - 2(\lambda_1 - \lambda_2)\left(\sqrt{3}\right) = 0$$
$$-1 - 2(\lambda_1 - \lambda_2)(-1) + 4\lambda_2 = 0$$

Solving the system we get:

$$\lambda_1 = \frac{1}{4} + \frac{1}{4\sqrt{3}}$$
$$\lambda_2 = \frac{1}{4} - \frac{1}{4\sqrt{3}}$$

✅ Both $\lambda_1, \lambda_2 \geq 0 \rightarrow$ dual feasibility satisfied.

For $\left(-\sqrt{3}, -1\right)$:

$$1 - 2(\lambda_1 - \lambda_2)\left(-\sqrt{3}\right) = 0 \Longrightarrow \lambda_1 - \lambda_2 = -\frac{1}{2\sqrt{3}}$$

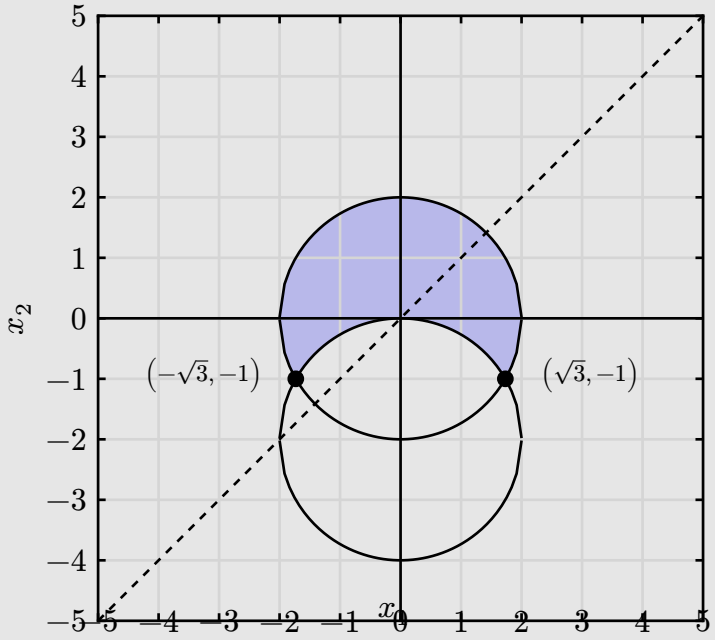
$$-1 - 2(\lambda_1 - \lambda_2)(-1) + 4\lambda_2 = 0 \Longrightarrow \lambda_2 = \frac{1}{4} + \frac{1}{4\sqrt{3}}$$

Solving the system we get:

$$\lambda_1 = \frac{1}{4} - \frac{1}{4\sqrt{3}}$$

$$\lambda_2 = \frac{1}{4} + \frac{1}{4\sqrt{3}}$$

✅ Both $\lambda_1, \lambda_2 \geq 0 \rightarrow$ dual feasibility satisfied.



✅ Both $\left(\sqrt{3}, -1\right)$ and $\left(-\sqrt{3}, -1\right)$ satisfy dual feasibility $\rightarrow$ both are KKT points.

Case 2. $(\lambda_1 > 0, \lambda_2 = 0)$

Step 1. Assumptions (Activity / Inactivity)

- Constraint 1: Active $(x_1^2 + x_2^2 = 4)$
- Constraint 2: Inactive $(x_1^2 + (x_2 + 2)^2 = 4)$

Step 2. Stationarity Condition

The Lagragian:

$$\mathcal{L}(x_1, x_2, \lambda_1, \lambda_2) = (x_1 - x_2) + \lambda_1(4 - x_1^2 - x_2^2) + \lambda_2\left(-4 + x_1^2 + (x_2 + 2)^2\right)$$

Compute the gradients with respect to x_1 and x_2:

$$\begin{cases} \frac{\partial \mathcal{L}}{\partial x_1} = 1 - 2\lambda_1 x_1 + 2\lambda_2 x_1 = 0 \\ \frac{\partial \mathcal{L}}{\partial x_2} = -1 - 2\lambda_1 x_2 + 2\lambda_2 (x_2 + 2) = 0 \end{cases}$$

Since $\lambda_2 = 0$, the equations simplify to:

$$\begin{cases} 1 - 2\lambda_1 x_1 = 0 \\ -1 - 2\lambda_1 x_2 = 0 \end{cases}$$

Simplify:

$$\begin{cases} x_1 = \frac{1}{2\lambda_1} \\ x_2 = -\frac{1}{2\lambda_1} \end{cases} \quad \Rightarrow \quad x_1 = -x_2$$

Step 3. Substitute into Active Constraint

Using $x_1^2 + x_2^2 = 4$ and $x_2 = -x_1$:

$$x_1^2 + (-x_1)^2 = 2x_1^2 = 4 \quad \Rightarrow \quad x_1^2 = 2 \quad \Rightarrow \quad x_1 = \pm\sqrt{2}, \quad x_2 = \mp\sqrt{2}$$

Two candidate solutions x_1 and x_2:

$$\left(-\sqrt{2}, \sqrt{2}\right) \quad \text{and} \quad \left(\sqrt{2}, -\sqrt{2}\right)$$

Step 4. Solve for λ_1 and enforce $\lambda_1 > 0$

$$1 + 2\lambda_1 x_1 = 0 \quad \Rightarrow \lambda_1 = -\frac{1}{2x_1}$$

Substitute (x_1, x_2) into the stationarity condition:

$$1 + 2\lambda_1 x_1$$

- For $(x_1, x_2) = \left(\sqrt{2}, -\sqrt{2}\right)$:

$$\lambda_1 = \frac{1}{2\sqrt{2}} > 0 \rightarrow \text{✅ } \textbf{Valid } (\lambda_1 \geq 0)$$

- For $(x_1, x_2) = \left(-\sqrt{2}, \sqrt{2}\right)$:

$$\lambda_1 = \frac{1}{2\left(-\sqrt{2}\right)} > 0 \rightarrow \text{❌ } \textbf{Reject } (\text{violates } \lambda_1 \geq 0)$$

So, the surviving candidate is:

$$(x_1, x_2) = \left(\sqrt{2}, -\sqrt{2}\right), \qquad \lambda_1 = \frac{1}{2\sqrt{2}}, \lambda_2 = 0$$

Step 5. Check the (inactive) constraint 2 for primal feasibility

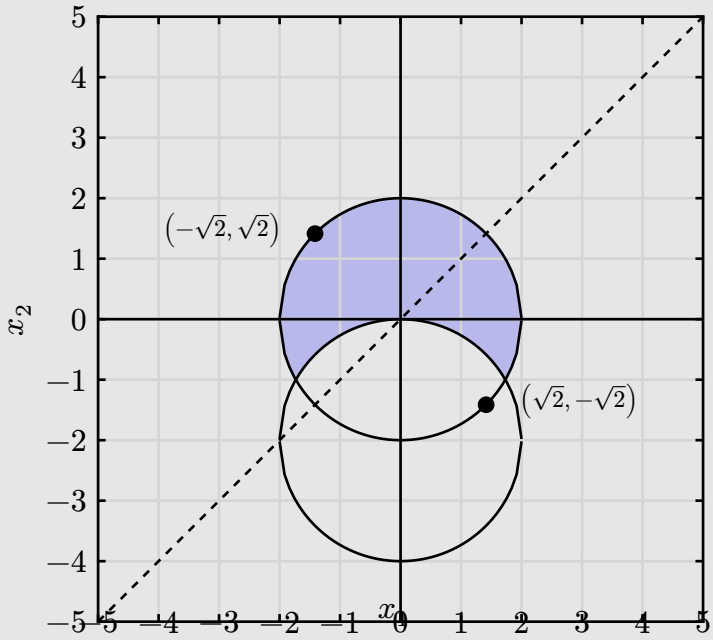
Constraint 2:

$$g_2(x) = -x_1^2 - (x_2 + 2)^2 + 4 \leq 0$$

Plug $x_1 = \sqrt{2}$ and $x_2 = -\sqrt{2}$:

$$g_2(x) = -\left(\sqrt{2}\right)^2 - \left(-\sqrt{2} + 2\right)^2 + 4 > 0$$

❌ This violates the inequality $g_2(x) \leq 0$



❌ $(x_1, x_2) = \left(\sqrt{2}, -\sqrt{2}\right)$ is rejected because it violates the inequality $g_2(x) \leq 0$

❌ $(x_1, x_2) = \left(-\sqrt{2}, \sqrt{2}\right)$ is rejected because it gives $\lambda < 0$

Case 3. ($\lambda_1 = 0, \lambda_2 > 0$)

Step 1. Assumptions (Activity / Inactivity)

- Constraint 1: Inactive ($x_1^2 + x_2^2 = 4$)
- Constraint 2: Active ($x_1^2 + (x_2 + 2)^2 = 4$)

Step 2. Stationarity Condition

The Lagragian:

$$\mathcal{L}(x_1, x_2, \lambda_1, \lambda_2) = (x_1 - x_2) + \lambda_1(4 - x_1^2 - x_2^2) + \lambda_2\left(-4 + x_1^2 + (x_2 + 2)^2\right)$$

Compute the gradients with respect to x_1 and x_2:

$$\begin{cases} \frac{\partial \mathcal{L}}{\partial x_1} = 1 - 2\lambda_1 x_1 + 2\lambda_2 x_1 = 0 \\ \frac{\partial \mathcal{L}}{\partial x_2} = -1 - 2\lambda_1 x_2 + 2\lambda_2(x_2 + 2) = 0 \end{cases}$$

Since $\lambda_1 = 0$, the equations simplify to:

$$\begin{cases} 1 + 2\lambda_2 x_1 = 0 \\ -1 + 2\lambda_2 (x_2 + 2) = 0 \end{cases}$$

Simplify:

$$\begin{cases} x_1 = -\frac{1}{2\lambda_2} \\ x_2 = \frac{1}{2\lambda_2} - 2 \end{cases} \quad \Rightarrow \quad x_1 = -x_2$$

Step 3. Substitute into Active Constraint

$$x_1^2 + (x_2 + 2)^2 = 4$$

Substituting x_1 and x_2:

$$\left(-\frac{1}{2\lambda_2}\right)^2 + \left(\frac{1}{2\lambda_2} - 2 + 2\right)^2 = 4 \quad \Rightarrow \quad \lambda_2 = \pm\frac{1}{2\sqrt{2}}$$

Two candidate solutions x_1 and x_2:

$$\left(-\sqrt{2}, \sqrt{2} - 2\right) \quad \text{and} \quad \left(\sqrt{2}, -\sqrt{2} - 2\right)$$

Substitute (x_1, x_2) into the stationarity condition:

$$-1 + 2\lambda_2 (x_2 + 2)$$

- For $(x_1, x_2) = \left(\sqrt{2}, -\sqrt{2} - 2\right)$:

$$\lambda_2 = -\frac{1}{2\sqrt{2}} < 0 \rightarrow \text{❌ } \textbf{Reject} \text{ (violates } \lambda_1 \geq 0)$$

- For $(x_1, x_2) = \left(-\sqrt{2}, \sqrt{2} - 2\right)$:

$$\lambda_2 = \frac{1}{2\sqrt{2}} > 0 \rightarrow \text{✅ } \textbf{Valid} \ (\lambda_1 \geq 0)$$

So, the surviving candidate is:

$$(x_1, x_2) = \left(-\sqrt{2}, \sqrt{2} - 2\right), \qquad \lambda_1 = 0, \lambda_2 = \frac{1}{2\sqrt{2}}$$

Step 5. Check the (inactive) constraint 1 for primal feasibility

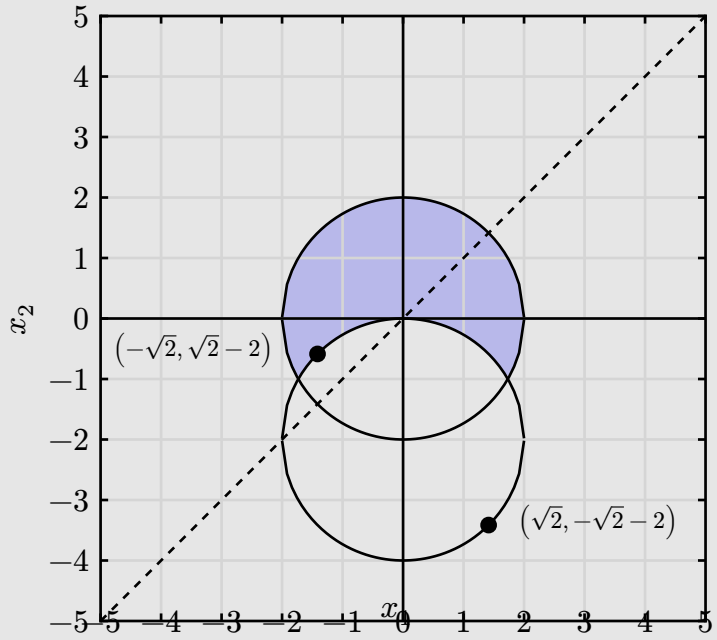
Constraint 1:

$$g_1(x) = x_1^2 + x_2^2 \leq 4$$

Plug $x_1 = -\sqrt{2}$ and $x_2 = \sqrt{2} - 2$:

$$g_1(x) = -\left(\sqrt{2}\right)^2 - \left(\sqrt{2} - 2\right)^2 + 4 \leq 0$$

✅ Satisfies primal feasibility. Constraint 1 is inactive (strict inequality), as required.



✅ $(x_1, x_2) = \left(-\sqrt{2}, \sqrt{2}-2\right)$ is a valid KKT point under this assumption.

❌ $(x_1, x_2) = \left(\sqrt{2}, -\sqrt{2}-2\right)$ is rejected because it gives $\lambda < 0$

Case 4. $(\lambda_1 = 0, \lambda_2 = 0)$

Step 1. Assumptions (Activity / Inactivity)

- Both multipliers are zero: $\lambda_1 = 0, \lambda_2 = 0$
- No constraints are forced to be active by complementary slackness

Step 2. Stationarity Condition

With $\lambda_1 = 0$ and $\lambda_2 = 0$, the stationarity equations simplify to:

$$\begin{cases} 1 - 2(\lambda_1 - \lambda_2)x_1 = 1 = 0 \\ -1 - 2(\lambda_1 - \lambda_2)x_2 + 4\lambda_2 = -1 = 0 \end{cases}$$

Step 3. Stationary Check

Both equations $1 = 0$ and $-1 = 0$ are impossible

No solution exists under this assumption

❌ No KKT points exist

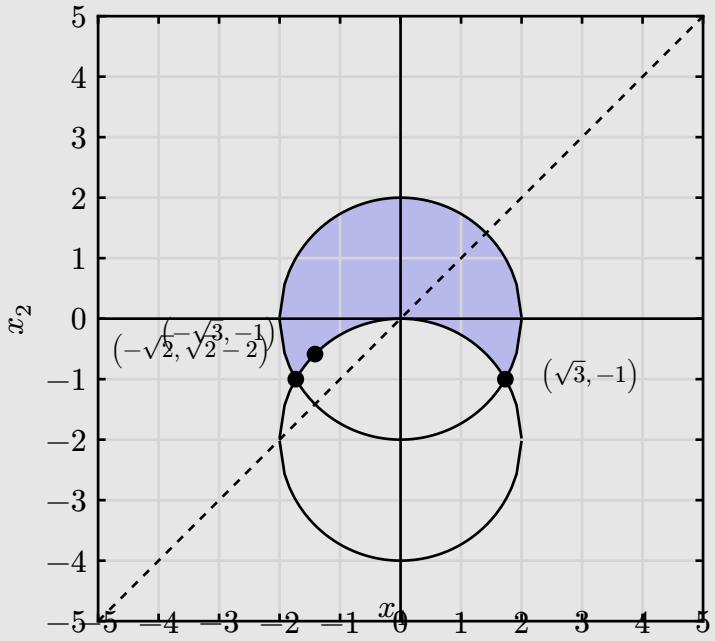
4️⃣ Summary

- $\lambda_1 > 0$ and $\lambda_2 > 0$:
  - $\left(\sqrt{3}, -1\right)$
  - $\left(-\sqrt{3}, 1\right)$
- $\lambda_1 = 0$ and $\lambda_2 > 0$:

- $\left(-\sqrt{2}, \sqrt{2}-2\right)$

These are the only candidates for local maxima (and thus global maxima)

Necessary, but not sufficient for non-convex NLPs



181.54.5. Sensitivity Analysis and Shadow Prices

Lagrange multipliers measure how senitive the objective function is to changes in the constraints

181.54.5.1. Shadow Prices

The shadow price of a constraint is the value of the corresponding Lagrange multiplier at the optimal solution:

$$\text{ShadowPrice}_i = \lambda_i^*$$

It measures how much the objective function would improve if the constraint were relaxed by one unit. For example:

$$x_1 + x_2 \leq 12 \quad \rightarrow \quad x_1 + x_2 \leq 13$$

The corresponding shadow price tells us the change in the optimal objective due to this relaxation. The magnitude of the Lagrange multiplier reflects the influence of its associated constraint on the optimal solution.

	Constraint Status	Shadow Price (λ^*)	Impact on Objective
Binding / Active	$g_i(x^*) = 0$	$\lambda^* > 0$	Relaxing improves objective
Inactive / Slack	$g_i(x^*) < 0$	$\lambda^* = 0$	Relaxing has no effect

Example

Primal NLP

$$z^* = \max_{x \in \mathbb{R}^n} \{ f(x) \mid g_i(x) \leq b_i \quad \forall i = 1, ..., m \}$$

Lagrange **Dual** Program

$$w^* = \min_{\lambda \geq 0} z^L(\lambda) = \min_{\lambda \geq 0} \left\{ \max_{x \in \mathbb{R}^n} f(x) + \sum_{i=1}^{m} \lambda_i [b_i - g_i(x)] \right\}$$

Convexity

The Lagrange Dual Program function $z^L(\lambda)$ is always convex over $\lambda \in [0, \infty)^n$

Weak Duality

$$w^* \geq z^* \quad \forall \lambda \geq 0$$

Strong Duality

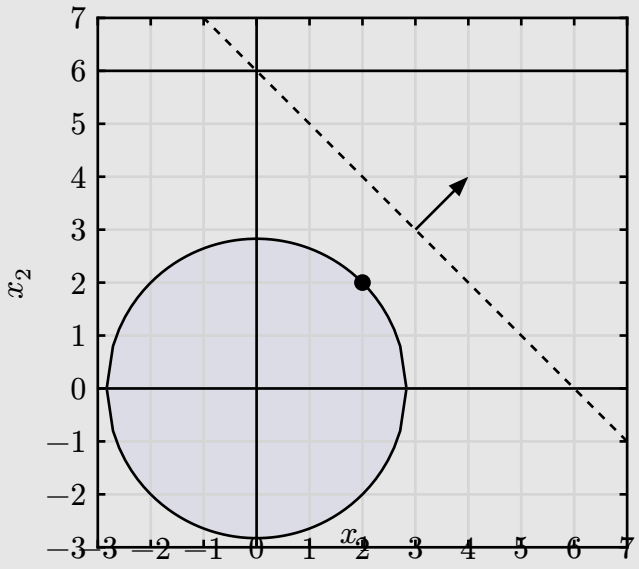
$w^* = z^*$ is the primal NLP is a “regular” **convex** program

Example

$$\begin{array}{rrrrrl} z^* & = & \max & x_1 & +x_2 & \\ & & s.t. & x_1^2 & +x_2^2 & \leq \quad 8 \\ & & & & x_2 & \leq \quad 6 \end{array}$$

Optimal solution

$$x^* = (2, 2)$$
$$z^* = 4$$



Lagragian relaxation

$$w^* = \min_{\lambda_1 \geq 0, \lambda_2 \geq 0} z^L(\lambda) = \min_{\lambda_1 \geq 0, \lambda_2 \geq 0} \max_{x \in \mathbb{R}^2} \mathcal{L}(x \mid \lambda)$$

Where

$$\mathcal{L}(x \mid \lambda) = x_1 + x_2 + \lambda_1(8 - x_1^2 + x_2^2) + \lambda_2(6 - x_2)$$

To solve the Lagrange dual program:

$$\max_{x \in \mathbb{R}^2} \mathcal{L}(x \mid \lambda) \max_{x \in \mathbb{R}^2} \{x_1 + x_2 + \lambda_1(8 - x_1^2 - x_2^2) + \lambda_2(6 - x_2)\}$$

First-order condition:

$$x_1 = \frac{1}{2\lambda_1} \quad \text{and} \quad x_2 = \frac{1 - \lambda_2}{2\lambda_1}$$

Plugging into $\mathcal{L}(x \mid \lambda)$:

$$z^L(\lambda) = \max_{x \in \mathbb{R}^2} \mathcal{L}(x \mid \lambda) = \frac{1 + (1 - \lambda_2)^2}{4\lambda_1} + 8\lambda_1 + 6\lambda_2$$

The Lagrangian dual program is to look for $\lambda_1 \geq 0$ and $\lambda_2 \geq 0$ to minimize $z^L(\lambda)$

The Lagrange dual program:

$$\min_{\lambda_1 \geq 0, \lambda_2 \geq 0} \frac{1 + (1 - \lambda_2)^2}{4\lambda_1} + 8\lambda_1 + 6\lambda_2$$

$z^L(\lambda)$ is convex over $(0, \infty)^2$:

$$\nabla z^L(\lambda) = \begin{bmatrix} -\frac{1+(1-\lambda_2)^2}{4\lambda_1^2} + 8 \\ -\frac{1-\lambda_2}{2\lambda_1} + 6 \end{bmatrix}, \quad \nabla^2 z^L(\lambda) = \begin{bmatrix} \frac{1+(1-\lambda_2)^2}{2\lambda_1^2} & \frac{1-\lambda_2}{2}\lambda_1^2 \\ \frac{1-\lambda_2}{2\lambda_1^2} & \frac{1}{2\lambda_1} \end{bmatrix}$$

Since:

$\frac{1+(1-\lambda_2)^2}{2\lambda_1^2} > 0$ and $\det(\nabla^2 z^L(\lambda)) = \frac{1}{2\lambda_1} > 0$, $z^L(\lambda)$ is convex

To solve

$$\min_{\lambda_1 \geq 0, \lambda_2 \geq 0} \frac{1 + (1 - \lambda_2)^2}{4\lambda_1} + 8\lambda_1 + 6\lambda_2$$

Apply KKT conditions

λ_1 cannot be binding ($\lambda_1 = 0$), because division by 0, so ignore it

Let $\mu \geq 0$ be the Lagrange multiplier for $\lambda_2 \geq 0$, the Lagrange is

$$\frac{1 + (1 - \lambda_2)^2}{4\lambda_1} + 8\lambda_1 + 6\lambda_2 - \mu\lambda_2$$

The KKT condition requires an optional solution to satisfy (FOC)

$$-\frac{1+(1-\lambda_2)^2}{4\lambda_1^2}+8=0,\quad -\frac{1-\lambda_2}{2\lambda_1}+6-\mu=0,\quad \mu\lambda=0$$

- Suppose $\mu > 0$
  - Implies $\lambda_2 = 0$
  - $-\frac{1+(1-\lambda_2)^2}{4\lambda_1^2}+8=0$ requires $\lambda_1 = \frac{1}{4}$
  - $-\frac{1-\lambda_2}{2\lambda_1}+6-\mu=0$ requires $\mu = 4$, which is feasible
- Suppose $\mu = 0$
  - $-\frac{1+(1-\lambda_2)^2}{4\lambda_1^2}+8=0$ requires $\lambda_2 = 1 - 12\lambda_1$
  - Plugging into $-\frac{1+(1-\lambda_2)^2}{4\lambda_1^2}+8=0$ results on $1 + 112\lambda_1^2$ which is impossible
- The only KKT point is $(\lambda_1, \lambda_2) = \left(\frac{1}{4}, 0\right)$
- Plugging into $z^L(\lambda)$ gives us $w^* = 4$ which exactly equals z^*

So strong duality holds, $w^* = z^*$

181.54.6. Lagrange Duality and LP Duality

LP duality is a special case of Lagrange Duality

For a general LP

$$\begin{aligned} \max \quad & c^T x \\ s.t. \quad & Ax = b \\ & x \geq 0 \end{aligned}$$

Where $A \in \mathbb{R}^{m \times n}$ (m constraints and n variables)

Let $\lambda \in \mathbb{R}^m$ be the Lagrange multipliers, the Lagrange relaxation is

$$\begin{aligned} z^*(\lambda) &= \max_{x \geq 0} c^T x + \lambda^T (b - Ax) \\ &= A^T b + \max_{x \geq 0} (c^T - \lambda^T A) x \end{aligned}$$

Because of equality constraint, λ is unrestricted in sign

Lagrange dual program

$$\min_{\lambda} z^L(\lambda) = \min_{\lambda} \left\{ \lambda^T b + \max_{x \geq 0} (c^T - \lambda^T A) x \right\}$$

Search for λ that minimizes $z^L(\lambda)$

Dual program is only meaningful if $c^T \leq \lambda^T A$, because if $\left(c^T\right)_i > \left(\lambda^T A\right)_i$ for any i, $\max_{x \geq 0} (c^T - \lambda^T A) x$ will be unbounded because we can increase x_i to ∞

Thus, no choice of λ that violates $c^T \leq \lambda^T A$ may be optimal to the Lagrange dual program

The Lagrange dual program

$$\min_{\lambda} z^*(\lambda) = \min_{\lambda_i c^T \leq \lambda^T A} \left\{ \lambda^T b + \max_{x \geq 0} (c^T - \lambda^T A)x \right\}$$

If λ satisfies $c^T \leq \lambda^T A$ thenwe know $\max_{x \geq 0} (c^T - \lambda^T A)x = 0$

The Lagrange dual program becomes

$$\begin{aligned} \min_{\lambda \text{ urs}} \quad & \lambda^T b \\ s.t. \quad & \lambda^T A \geq c^T \end{aligned}$$

Which is exactly the dual LP of

$$\begin{aligned} \max_{x \geq 0} \quad & c^T x \\ s.t. \quad & Ax = b \end{aligned}$$

Example

$$\begin{aligned} \min \quad & (x_1 - 4)^2 + (x_2 - 2)^2 \\ s.t. \quad & 2x_1 + x_2 \leq 6 \end{aligned}$$

1. What are the leading principle minors of the Hessian matrix of the objective function?

$$\nabla f(x_1, x_2) = \begin{bmatrix} 2(x_1 - 4) \\ 2(x_2 - 2) \end{bmatrix} \qquad \nabla^2 f(x_1, x_2) = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$$

2 and 4 ✅

2.

$$\mathcal{L}(x_1, x_2 \mid \lambda) = (x_1 - 4)^2 + (x_2 - 2)^2 + \lambda(6 - 2x_2 - 2)$$

$\lambda \leq 0$ ✅

3. According to the FOC of the Lagragian, what is a necessary condition for an optimal solution

Case 1. $\lambda > 0$

Constraint active: $2x_2 - 2 \leq 6$

FOC (gradient w.r.t. x_1 and x_2):

$$\nabla \mathcal{L}(x_1, x_2 \mid \lambda) = \begin{bmatrix} 2(x_1 - 4) \\ 2(x_2 - 2) - 2\lambda \end{bmatrix}$$

$$2(x_1 - 4) = 0$$
$$x_1 = 4$$

$$2(x_2 - 2) - 2\lambda = 0$$
$$2x_2 - 4 - 2\lambda = 0$$
$$x_2 = \frac{2\lambda + 4}{2}$$
$$x_2 = \lambda + 2$$

$$x_1 = 4$$
$$x_2 = \lambda + 2$$

Substitute into active constraint

$$6 - 2x_2 - 2 = 0$$
$$4 - 2x_2 = 0$$
$$4 - 2(\lambda + 2) = 0$$
$$4 - 2\lambda + 4 = 0$$
$$2\lambda = 0$$
$$\lambda = \frac{0}{-2}$$
$$\lambda = 0$$

Find λ:

$\lambda = 0$

Which violates $\lambda > 0$

Case 2. $\lambda = 0$

$$x_1 = 4$$
$$x_2 = \lambda + 2$$
$$x_2 = 0 + 2$$

$$(x_1, x_2) = (4, 2)$$

$\boldsymbol{x_1 - 2x_2 = 0}$ ✅

Case 2. $\lambda = 0$

4. What is a local optimal solution to the nonlinear program?

$\boldsymbol{(x_1, x_2) = \left(\frac{12}{5}, \frac{6}{5}\right)}$ ✅

5.

For a linear program, linear programming duality and Lagrange duality is equivalent (i.e., the dual programs obtained through the two ways are identical). ✅

For an unconstrained nonlinear program, the KKT condition is equivalent to the first-order condition. ✅

For an unconstrained nonlinear program, the KKT condition is necessary and sufficient. ✅

181.55. Dynamic Programming

A method for solving multi-stage decision problems by breaking them into overlapping sub-problems, each solved once and reused.

181.55.1. When DP applies

Required ingredients:

1. **Stages** — the problem decomposes into a sequence of decisions $t = 0, 1, ..., T-1$
2. **State** — a summary of the past sufficient for future decisions (Markov property)
3. **Additive cost** — total cost = sum of per-stage costs $\sum c_t(s_t, a_t)$
4. **Known transition** — $s_{t+1} = f_t(s_t, a_t)$ deterministic, or $P_t(s' \mid s, a)$ stochastic
5. **Tractable state set** — table fits in memory (or use approximation)

If any breaks, plain DP doesn't apply.

181.55.2. Problem class

At stage t in state s_t:

- Choose action $a_t \in \mathcal{A}(s_t)$
- Pay immediate cost $c_t(s_t, a_t)$
- System transitions to $s_{t+1} = f_t(s_t, a_t)$

After the last decision: terminal cost $c_T(s_T)$.

Total cost of a plan $\pi = (a_0, ..., a_{T-1})$:

$$J^\pi(s_0) = \sum_{t=0}^{T-1} c_t(s_t, a_t) + c_T(s_T)$$

Goal: $\pi^* = \arg\min_\pi J^\pi(s_0)$.

181.55.3. Why naive enumeration fails

K actions per stage, T stages $\to K^T$ plans. $K = 5, T = 20 \to 5^{20} \approx 10^{14}$. DP shrinks this to $O(T \ |\mathcal{S}| \ |\mathcal{A}|)$ via the Bellman recursion.

181.55.4. Core ideas

- **Bellman equation** — the recursion that breaks the problem into sub-problems
- **Backward induction** — solve from the end back
- **Forward induction** — dual direction (cost-to-arrive)
- **Stochastic DP** — random transitions, expected cost
- **Value iteration** / **Policy iteration** — infinite-horizon MDPs

181.55.5. Curse of dimensionality

State has d components with n values each $\rightarrow |\mathcal{S}| = n^d$. DP cost grows exponentially in **dimension**, not in T.

Mitigations: state aggregation, approximate DP, reinforcement learning, function approximation.

181.55.6. Common applications

- **Shortest path on DAG** — $O(V + E)$
- **0-1 Knapsack** — pseudo-polynomial $O(nW)$
- **Equipment replacement**
- **Inventory**: Wagner-Whitin lot-sizing
- **Resource allocation** across activities
- **Optimal stopping** — secretary problem, asset selling
- **Revenue management** — see Bellman applied to capacity-time
- **Reinforcement learning** — MDP foundation

181.55.7. See also

- **Bellman Equation**
- **Backward Induction**
- **Stochastic DP**
- **Value Iteration**
- **Policy Iteration**

181.56. Bellman Equation

The recursion at the heart of dynamic programming. Defines the optimal cost-to-go $J_t(s)$ — minimum cost to complete the problem from stage t in state s — as a function of cost-to-go from the next stage.

181.56.1. Principle of optimality (Bellman, 1957)

> "An optimal policy has the property that whatever the initial state and initial decision, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision."

Restated: if $(a_0^*, a_1^*, ..., a_{T-1}^*)$ is optimal from s_0, then $(a_1^*, ..., a_{T-1}^*)$ is optimal from $s_1 = f_0(s_0, a_0^*)$.

Why true (contradiction): if the suffix weren't optimal, swap it for the better one $\rightarrow$ strictly lower total cost $\rightarrow$ contradicts optimality of the full plan. ∎

181.56.2. Deriving the recursion

The total cost decomposes:

$$J^\pi(s_0) = \underbrace{c_0(s_0, a_0)}_{\text{stage } 0} + \underbrace{\sum_{t=1}^{T-1} c_t(s_t, a_t) + c_T(s_T)}_{\text{sub-problem of same kind, from } s_1}$$

The bracketed remainder is the same problem starting from s_1. By the principle of optimality, the optimal remainder is $J_1(s_1)$ — the optimal cost-to-go from s_1.

So at stage t in state s, you pay $c_t(s, a)$ now and face the optimal-remaining problem from $f_t(s, a)$:

$$J_t(s) = \min_{a \in \mathcal{A}(s)} \{c_t(s, a) + J_{t+1}(f_t(s, a))\}$$

This is the **Bellman equation** — direct consequence of additivity + principle of optimality.

181.56.3. Base case

$$J_T(s) = c_T(s)$$

(no decisions left; the only cost is the terminal one.)

181.56.4. Optimal action

$$a_t^*(s) = \arg\min_a \{c_t(s, a) + J_{t+1}(f_t(s, a))\}$$

The optimal total cost from s_0 is $J_0(s_0)$.

181.56.5. Worked example: shortest path through a layered graph

Stages $0 \to 1 \to 2 \to 3$, states $\{S\}, \{A, B\}, \{C, D\}, \{G\}$. Edge costs:

from\to	A	B	C	D	G
S	4	2			
A			5	3	
B			1	6	
C					2
D					4

Stage 3 (base case): $J_3(G) = 0$.

Stage 2:

- $J_2(C) = 2 + 0 = 2$
- $J_2(D) = 4 + 0 = 4$

Stage 1:

- $J_1(A) = \min\{5 + 2, 3 + 4\} = \min\{7, 7\} = 7$ (tie)
- $J_1(B) = \min\{1 + 2, 6 + 4\} = \min\{3, 10\} = 3$, action: go to C

Stage 0:

- $J_0(S) = \min\{4 + 7, 2 + 3\} = \min\{11, 5\} = 5$, action: go to B

Optimal path: $S \to B \to C \to G$, cost 5.

181.56.6. Two variants

- **Backward induction**: solve $J_T \to J_{T-1} \to \ldots \to J_0$.
- **Forward induction**: dual recursion on cost-to-arrive $V_t(s)$.

Both compute the same optimum — choose whichever fits the data flow.

181.56.7. Stochastic case

When transitions are random ($s_{t+1} \sim P_t(\cdot \mid s_t, a_t)$), the Bellman equation becomes:

$$J_t(s) = \min_a \left\{ c_t(s,a) + \sum_{s'} P_t(s' \mid s,a)\, J_{t+1}(s') \right\}$$

— expectation over the next state replaces the deterministic transition. See Stochastic DP.

181.56.8. See also

- **Dynamic Programming** — the broader framework
- **Backward Induction** — algorithm
- **Stochastic DP** — random case

181.57. Backward Induction

The standard algorithm for solving finite-horizon dynamic programming problems. Computes the optimal cost-to-go $J_t(s)$ from the back ($t = T$) toward the start ($t = 0$).

181.57.1. Algorithm

```
For each terminal state s ∈ S:
    J[T][s] ← c_T(s)

For t = T-1, T-2, …, 0:
    For each state s ∈ S:
        J[t][s]  ← min over a of { c_t(s,a) + J[t+1][ f_t(s,a) ] }
        π[t][s]  ← argmin over a of the same expression

Return J[0][s₀] and policy {π[t][s]}
```

181.57.2. Why backward

At stage t you need to know J_{t+1} to evaluate the Bellman equation:

$$J_t(s) = \min_a \{ c_t(s,a) + J_{t+1}(f_t(s,a)) \}$$

So compute stages in reverse: J_T first (base case), then J_{T-1}, then J_{T-2}, …, then J_0.

181.57.3. Complexity

$$O(T \cdot |\mathcal{S}| \cdot |\mathcal{A}|)$$

Polynomial in T, not exponential. (Naive enumeration is $|\mathcal{A}|^T$.)

Memory: $O(T \cdot |\mathcal{S}|)$ for the full table, or $O(|\mathcal{S}|)$ if you only need J_0 and can discard each J_{t+1} after using it (but then you lose the policy).

181.57.4. When backward is the right choice

- **Finite horizon** with a clean terminal cost $c_T(s)$

- **Computing the optimal value from a known start state** s_0
- **Producing the optimal action at every reachable state** (policy)
- **Dense state graphs**: most states are reachable

For sparse reachable sets, **memoization** (top-down recursive DP with caching) avoids computing unreached states.

181.57.5. Example: Wagner-Whitin lot-sizing

Demand d_t in each period $t = 1, ..., T$. Setup cost K, holding cost h per unit per period. Choose periods to place orders to minimize total cost.

State: $s_t \in \{0, 1, ..., T\}$ = the period of the **last** order (so demand from $s_t + 1$ to t is covered by that order).

The zero-inventory property says it's optimal to order in period j iff j is a previous "last-order" period: $J_T(T)$ = min total cost given the last order was in period T.

Backward induction over T states × T periods → $O(T^2)$. This is the classic **Wagner-Whitin algorithm** — see Wagner-Whitin.

181.57.6. Tabulation vs memoization

Two implementations of the same logic:

Tabulation (bottom-up)	**Memoization (top-down)**
Fill full table $J_T, J_{T-1}, ..., J_0$ in order	Recursive call from $J_0(s_0)$, cache results
Visits every state	Only states actually reached
Best for dense reachable sets	Best for sparse reachable sets

Both compute the same $J_t(s)$.

181.57.7. See also

- **Bellman Equation**
- **Forward Induction** — the dual approach
- **Dynamic Programming**

181.58. Forward Induction

The dual of backward induction. Instead of computing the optimal cost-**from** each state, compute the optimal cost **to arrive at** each state from s_0.

181.58.1. Setup

Define $V_t(s)$ = minimum cost to reach state s at stage t from the start state s_0.

Base case:

$$V_0(s_0) = 0, \quad V_0(s) = +\infty \text{ for } s \neq s_0$$

Inductive step: to land at s' at stage $t + 1$, you must come from some predecessor s at stage t and choose action a with $f_t(s, a) = s'$:

$$V_{t+1}(s') = \min_{(s,a)\,:\,f_t(s,a)=s'} \{V_t(s) + c_t(s,a)\}$$

Optimal value: $\min_s [V_T(s) + c_T(s)]$.

181.58.2. Algorithm

```
V[0][s₀] ← 0; V[0][s] ← ∞ for s ≠ s₀

For t = 0, 1, …, T-1:
    For each s ∈ S:
        For each action a ∈ A(s):
            s' ← f_t(s, a)
            cost ← V[t][s] + c_t(s, a)
            if cost < V[t+1][s']:
                V[t+1][s'] ← cost
                pred[t+1][s'] ← (s, a)  # for path reconstruction

Optimal value: min over s of (V[T][s] + c_T(s))
Trace back via `pred` to recover the optimal path.
```

Cost is the same as backward induction: $O(T\ |\mathcal{S}|\ |\mathcal{A}|)$.

181.58.3. Backward vs forward

	Backward	**Forward**
Variable	$J_t(s)$ = cost-to-go from s	$V_t(s)$ = cost-to-arrive at s
Direction	$T \to 0$	$0 \to T$
Best when	Terminal cost known/small; want action at every state	Starting state fixed; want best path to every state
Yields	Optimal action at every (t, s)	Optimal path to every reachable state

Both compute the same optimum.

181.58.4. Example: shortest path on a DAG

The classic forward DP. Topologically sort the DAG; visit each node in order, computing $V[s']$ as the minimum of $V[s] + c(s, s')$ over all predecessors s.

This is exactly what Bellman-Ford / Dijkstra-on-DAGs does — see Dijkstra (which is a forward DP with priority-queue ordering).

181.58.5. When forward is the right choice

- **Single source, all-targets**: e.g., shortest paths from s_0 to every other node
- **Streaming or layered processing**: data arrives stage by stage, can't see the end
- **Predecessor tracking is natural**: reconstruct paths via `pred[t][s]`

181.58.6. See also

- **Backward Induction** — dual
- **Bellman Equation**
- **Dijkstra** — forward DP with priority queue

- **Bellman-Ford** — forward DP, handles negative edges

181.59. Stochastic DP

Extends deterministic DP to problems with random transitions. The framework underlying **Markov Decision Processes** (MDPs), reinforcement learning, and most multi-period stochastic optimization in operations.

181.59.1. Setup

At stage t in state s_t:

- Choose action $a_t \in \mathcal{A}(s_t)$
- Pay immediate cost $c_t(s_t, a_t)$
- **Next state is random**: $s_{t+1} \sim P_t(\cdot \mid s_t, a_t)$ — transition probability depending on current state and action

After the last decision: terminal cost $c_T(s_T)$.

181.59.2. Policy (not plan)

A fixed **plan** $(a_0, a_1, ..., a_{T-1})$ no longer works — you don't know which state will realize at each step. Instead, a **policy** is a function:

$$\pi_t : \mathcal{S} \to \mathcal{A}, \ \pi_t(s) \text{ is the action taken in state } s \text{ at stage } t$$

Total cost is a random variable; we minimize its expectation:

$$J^{\pi}(s_0) = E\left[\sum_{t=0}^{T-1} c_t(s_t, \pi_t(s_t)) + c_T(s_T)\right]$$

181.59.3. Stochastic Bellman equation

Same argument as the deterministic case (additivity + principle of optimality, now applied to **expected** cost):

$$J_t(s) = \min_{a \in \mathcal{A}(s)} \left\{ c_t(s,a) + \sum_{s'} P_t(s' \mid s, a)\, J_{t+1}(s') \right\}$$

Equivalently:

$$J_t(s) = \min_a \Big\{ c_t(s,a) + E_{s' \sim P_t(\cdot \mid s,a)}\big[J_{t+1}(s')\big] \Big\}$$

The expectation over s' replaces the deterministic transition $f_t(s,a)$ from the deterministic Bellman.

181.59.4. Side-by-side comparison

$$\text{deterministic:} \quad J_t(s) = \min_a \{ c_t(s,a) + J_{t+1}(f_t(s,a)) \}$$

$$\text{stochastic:} \quad J_t(s) = \min_a \{ c_t(s,a) + E[J_{t+1}(s')] \}$$

181.59.5. Worked example

From state B at stage 1, action "right" leads to:

- C with probability 0.7 (immediate cost 1, $J_2(C) = 2$)
- D with probability 0.3 (immediate cost 6, $J_2(D) = 4$)

$$Q_1(B, \text{right}) = 0.7\,(1 + 2) + 0.3\,(6 + 4) = 2.1 + 3.0 = 5.1$$

If action "left" gives 4.5, optimal action at $(t = 1, B)$ = left.

181.59.6. Examples

- **(s, S) inventory policy** — order up to S when inventory drops below s; optimality is a stochastic-DP result
- **Multi-period newsvendor** — stochastic demand each period, carry inventory between periods
- **Revenue management** — DP on (remaining capacity, time-to-go)
- **Optimal stopping** — secretary problem, asset selling, exercise of American options
- **Reinforcement learning** — model-free / sample-based stochastic DP

181.59.7. Finite vs infinite horizon

- **Finite horizon**: just solve backward in time as in deterministic case
- **Infinite horizon**: leads to value iteration / policy iteration

181.59.8. Curse of dimensionality + curse of randomness

Stochastic DP inherits the dimensionality curse from deterministic DP, plus the cost of computing expectations over the next-state distribution. Approximations:

- **Sampling**: replace exact expectation with Monte Carlo estimate
- **Approximate DP / Neuro-DP**: function approximation for J_t
- **Reinforcement learning**: sample-based, model-free

181.59.9. See also

- **Bellman Equation** — deterministic version
- **Value Iteration**
- **Policy Iteration**
- **Markov Chains** — underlying state-transition model

181.60. Value Iteration

The standard algorithm for solving **infinite-horizon** Markov Decision Processes. Iteratively applies the Bellman equation until the value function converges.

181.60.1. Infinite-horizon discounted MDP

States $\mathcal{S}$, actions $\mathcal{A}$, transition $P(s' \mid s, a)$, immediate cost $c(s, a)$, discount factor $\gamma \in (0, 1)$.

Objective: minimize expected discounted total cost over an infinite horizon:

$$J^{\pi}(s) = E\left[\sum_{t=0}^{\infty} \gamma^t \, c(s_t, \pi(s_t)) \mid s_0 = s\right]$$

Since rewards / costs in the far future are exponentially discounted, the sum converges.

181.60.2. Bellman optimality equation (infinite horizon)

$$J^*(s) = \min_{a \in \mathcal{A}} \left\{ c(s,a) + \gamma \sum_{s'} P(s' \mid s,a) \, J^*(s') \right\}$$

This is a **fixed-point equation**: $J^* = \mathcal{T} J^*$ where $\mathcal{T}$ is the **Bellman operator**. The operator is a contraction (with contraction factor γ), so the fixed point is unique and reachable by iteration.

181.60.3. Algorithm

```
Initialize J(s) ← 0 (or any function)

Repeat until convergence:
    For each s ∈ S:
        J'(s) ← min over a of { c(s, a) + γ Σ_{s'} P(s' | s, a) J(s') }
    J ← J'

Return optimal policy:
    π*(s) ← argmin over a of { c(s, a) + γ Σ_{s'} P(s' | s, a) J(s') }
```

Convergence test: stop when $\max_s |J'(s) - J(s)| < \varepsilon$.

181.60.4. Why it converges

The Bellman operator $\mathcal{T}$ is a **γ-contraction** under the sup-norm:

$$\|\mathcal{T} J_1 - \mathcal{T} J_2\|_\infty \le \gamma \, \|J_1 - J_2\|_\infty$$

By Banach fixed-point theorem, iterating from any starting J converges geometrically to the unique fixed point J^*:

$$\|J^{(k)} - J^*\|_\infty \le \gamma^k \, \|J^{(0)} - J^*\|_\infty$$

181.60.5. Convergence rate

Geometric in γ. To get within ε of J^* takes about

$$k = \frac{\log \varepsilon}{\log \gamma} \text{ iterations}$$

If γ is close to 1 (long-horizon problems), convergence is slow. **Acceleration tricks**:

- **Gauss-Seidel updates** (update $J(s)$ in-place using already-updated neighbors)
- **Prioritized sweeping** (update states with large Bellman residuals first)
- **Approximate value iteration** (function approximation for large state spaces)

181.60.6. Cost per iteration

$O(|\mathcal{S}|^2 \cdot |\mathcal{A}|)$ — for each state ($|\mathcal{S}|$), for each action ($|\mathcal{A}|$), sum over next states ($|\mathcal{S}|$).

181.60.7. Value iteration vs policy iteration

	Value iteration	Policy iteration
Iterates on	Value function J	Policy π
Per iteration cost	Cheap ($O(\lvert\mathcal{S}\rvert^2 \lvert\mathcal{A}\rvert)$)	Expensive (full policy evaluation: solve linear system)
Iterations to converge	Many (geometric in γ)	Few (often $\leq \lvert\mathcal{S}\rvert$ iterations)
Total cost	Comparable in practice	Comparable
Anytime?	Yes — returns a usable J at any point	Only at policy-iteration end

See Policy Iteration for the alternative.

181.60.8. Example application

Inventory management with infinite horizon: state = on-hand inventory, action = order quantity, transition = stochastic demand. Value iteration finds the optimal (s, S) policy.

181.60.9. See also

- **Bellman Equation**
- **Stochastic DP**
- **Policy Iteration**
- **Markov Chains**

181.61. Policy Iteration

An alternative to value iteration for infinite-horizon MDPs. Alternates between **evaluating** the current policy and **improving** it.

181.61.1. Algorithm

```
Initialize π(s) ← arbitrary action

Repeat:
    # Policy evaluation: compute J^π by solving the linear system
    #     J^π(s) = c(s, π(s)) + γ Σ_{s'} P(s' | s, π(s)) J^π(s')
    # i.e., (I - γ P_π) J^π = c_π        (|S| × |S| linear system)

    # Policy improvement: greedy w.r.t. J^π
    For each s ∈ S:
        π'(s) ← argmin over a of { c(s, a) + γ Σ_{s'} P(s' | s, a) J^π(s') }

    If π' = π: stop
    π ← π'

Return π*
```

181.61.2. Two phases per iteration

1. **Policy evaluation**: solve $(I - \gamma P_\pi) J^\pi = c_\pi$ for the value function of the current policy π. This is a $\lvert\mathcal{S}\rvert \times \lvert\mathcal{S}\rvert$ linear system — exact, no approximation.

2. **Policy improvement**: greedy update — pick the action minimizing the Bellman expression **using the current** J^π.

If policy improvement changes nothing, the current π is optimal (Bellman optimality is satisfied).

181.61.3. Why it terminates in finite steps

Each iteration **strictly improves** the policy (in value) unless already optimal. There are only $|\mathcal{A}|^{|\mathcal{S}|}$ deterministic policies, so the algorithm must terminate. In practice, it converges in **very few iterations** — often $O(|\mathcal{S}|)$ or fewer, regardless of γ.

181.61.4. Cost per iteration

- **Policy evaluation**: $O(|\mathcal{S}|^3)$ (solve linear system) or fewer with iterative methods
- **Policy improvement**: $O(|\mathcal{S}|^2 \cdot |\mathcal{A}|)$

So per iteration, policy iteration is **more expensive** than value iteration. But it needs **fewer iterations**. Total cost is comparable.

181.61.5. Modified policy iteration

A middle ground: don't fully solve the linear system in step 1. Instead, do k value-iteration-like sweeps to **approximate** J^π:

```
Repeat k times:
    For each s ∈ S:
        J(s) ← c(s, π(s)) + γ Σ_{s'} P(s' | s, π(s)) J(s')
```

Then improve. With $k = 1$: equivalent to value iteration. With $k = \infty$: full policy iteration. Tuning k trades off per-iteration cost vs number of iterations.

181.61.6. Comparison with value iteration

	Value iteration	**Policy iteration**
Per-iter cost	low	high
Number of iterations	many	few
Convergence	asymptotic ($J \to J^*$)	exact terminator (finite policy space)
Memory	$\|\mathcal{S}\|$ for J	$\|\mathcal{S}\|$ for J and π
Practical	simpler, more common in DP literature	favored for small / mid state spaces

181.61.7. Why policy iteration converges fast

The **policy improvement theorem** says that greedy update produces a policy with value at least as good **componentwise** — strictly better at some state if the policy wasn't already optimal. In high-discount problems, value iteration can take thousands of iterations; policy iteration usually takes 10–50.

181.61.8. Linear-programming reformulation

Both value iteration and policy iteration solve the **same** fixed-point problem:

$$\min_J \sum_s \mu_0(s) J(s) \quad \text{s.t. } J(s) \le c(s,a) + \gamma \sum_{s'} P(s' \mid s,a) J(s'), \ \forall (s,a)$$

This LP has $|\mathcal{S}|$ variables and $|\mathcal{S}| \cdot |\mathcal{A}|$ constraints. Solving it directly is a third approach (and the connection to LP duality reveals deep structure of MDPs).

181.61.9. See also

- **Value Iteration** — the alternative
- **Bellman Equation**
- **Stochastic DP**
- **Linear Programming** — LP reformulation

181.62. Regression

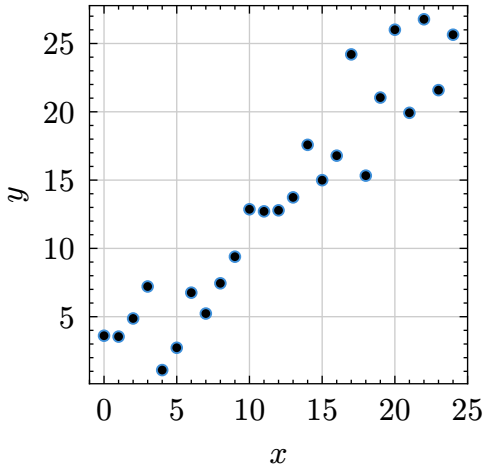
181.62.1. Linear Regression

181.62.1.1. Simple Linear Regression

181.62.1.1.1. Data and Model

We observe a dataset of n samples:

$$(x_i, y_i) \quad i = 1, ..., n$$



We assume a linear relationship between x and y:

$$y = \alpha + \beta x_i + \varepsilon_i$$

Where:

- α: **intercept**
- β: **slope**
- ε_i: **error term** (noise)

181.62.1.1.2. Optimization Problem

The parameters α and β are estimated by minimizing the **sum of square errors** (SSE)

$$\min_{\alpha,\beta} f(\alpha,\beta) = \sum_{i=1}^{n} [y_i - (\alpha + \beta x_1)]^2$$

This is an **unconstrained convex optimization problem**

181.62.1.1.3. Gradient and Hessian

Compute the partial derivatives:

$$\frac{\partial f}{\partial \alpha} = -2 \sum_{i=1}^{n} [y_i - (\alpha + \beta x_1)]$$

$$\frac{\partial f}{\partial \beta} = -2 \sum_{i=1}^{n} [y_i - (\alpha + \beta x_1)] x_i$$

Thus, the **gradient** is:

$$\nabla f(\alpha,\beta) = \begin{bmatrix} -2 \sum_{i=1}^{n} [y_i - (\alpha + \beta x_1)] \\ -2 \sum_{i=1}^{n} [y_i - (\alpha + \beta x_1)] x_i \end{bmatrix}$$

and the **Hessian matrix** is:

$$\nabla^2 f(\alpha,\beta) = \begin{bmatrix} 2n & 2 \sum_{i=1}^{n} x_i \\ 2 \sum_{i=1}^{n} x_i & 2 \sum_{i=1}^{n} x_i^2 \end{bmatrix}$$

$$= 2 \begin{bmatrix} n & \sum_{i=1}^{n} x_i \\ \sum_{i=1}^{n} x_i & \sum_{i=1}^{n} x_i^2 \end{bmatrix}$$

181.62.1.1.4. Convexity

The Hessian is constant (does not depend on α, β) and symmetric. We check positive semidefiniteness via the determinant:

$$\det(\nabla^2 f) = 4 \left(n \sum_{i=1}^{n} x_i^2 - \left(\sum_{i=1}^{n} x_i \right)^2 \right) = 4 \sum_{i<j} (x_i - x_j)^2 \geq 0$$

Sincle this expression is nonnegative for all x_i, f is **convex**

181.62.1.1.5. Optimality Conditions

Setting the gradient equal to zero gives the normal equaltions:

$$\begin{cases} n\alpha + \left(\sum_{i=1}^{n} x_i \right) \beta & = \sum_{i=1}^{n} y_i \\ \left(\sum_{i=1}^{n} x_i \right) \alpha + \left(\sum_{i=1}^{n} x_i^2 \right) \beta = \sum_{i=1}^{n} x_i y_i \end{cases}$$

Solving this linear system yields the closed-form solution:

$$\hat{\beta} = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}$$

$$\hat{\alpha} = \bar{y} - \hat{\beta} \bar{x}$$

where:

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i \qquad \bar{y} = \frac{1}{n}\sum_{i=1}^{n} y_i$$

181.62.1.1.6. Convexity by Decomposition

Alternatively, decompose $f(\alpha, \beta)$ as:

$$f(\alpha, \beta) = \sum_{i=1}^{n} \left[y_i - (\alpha + \beta x_i)\right]^2 = \sum_{i=1}^{n} f_i(\alpha, \beta)$$

$$f_i(\alpha, \beta) = \left[y_i - (\alpha + \beta x_i)\right]^2$$

Expanding f_i:

$$f_i(\alpha, \beta) = y_i^2 + \alpha^2 + \beta^2 x_i^2 - 2\alpha y_i - 2\beta x_i y_i + 2\alpha\beta x_i$$

Each f_i is a quadratic function in (α, β)

Each f_i has Hessian:

$$\nabla^2 f_i(\alpha, \beta) = \begin{bmatrix} 2 & 2x_i \\ 2x_i & 2x_i^2 \end{bmatrix} = 2\begin{bmatrix} 1 & x_i \\ x_i & x_i^2 \end{bmatrix}$$

Since each $\nabla^2 f_i$ is positive semidefinite, and sums of convex functions are convex, f is convex

Example

181.62.1.2. Multiple Linear Regression

181.62.1.2.1. Data and Model

For n samples and p predictors:

$$\left(x_1^{(i)}, x_2^{(i)}, ..., x_p^{(i)}, y_i\right) \quad i = 1, ..., n$$

The model is:

$$\begin{aligned} y_i &= \alpha + \beta_1 x_1^{(i)} + ... + \beta_p x_p^{(i)} + \varepsilon_i \\ &= \alpha + \boldsymbol{\beta}^T \mathbf{x}^{(i)} + \varepsilon_i \end{aligned}$$

181.62.1.2.2. Optimization Problem

$$\min_{\alpha,\beta} f(\alpha, \beta)$$

where

$$f(\alpha, \beta) = \sum_{i=1}^{n} \left[y_i - \left(\alpha + \sum_{j=1}^{p} \beta_j x_j^i \right) \right]^2$$
$$= \sum_{i=1}^{n} \left[y_i - \left(\alpha + \boldsymbol{\beta}^T \mathbf{x}^{(i)} \right) \right]^2$$

In matrix form, letting $X \in \mathbb{R}^{n \times p}$ be the data matrix, $\boldsymbol{y} \in \mathbb{R}^n$, and $\mathbf{1}$ the column of ones:

$$\min_{\alpha, \beta} \quad \|\boldsymbol{y} - \alpha \mathbf{1} - X\beta\|_2^2$$

The closed-form solution is:

$$\hat{\beta} = (X^T X)^{-1} X^T (\mathbf{y} - \bar{y}\mathbf{1})$$
$$\hat{\alpha} = \bar{y} - \hat{\beta}^T \bar{\mathbf{x}}$$

provided $X^T X$ is invertible

181.62.1.3. Regularization

Regularization penalizes large coefficients to prevent overfitting and improve numerical stability.

Let $\lambda > 0$ be a regularization parameter.

181.62.1.3.1. Ridge Regression (L2 Regularization)

$$\min_{\alpha, \beta} \sum_{i=1}^{n} \left[y_i - (\alpha + \boldsymbol{\beta}^T \mathbf{x}^i) \right]^2 + \lambda \sum_{j=1}^{p} \beta_j^2$$

In matrix form:

$$\min_{\beta} \quad \|\mathbf{y} - X\boldsymbol{\beta}\|_2^2 + \lambda \|\boldsymbol{\beta}\|_2^2$$

Closed-form solution (for centered data):

$$\hat{\boldsymbol{\beta}} = \left(X^T X + \lambda I_p \right)^{-1} X^T \mathbf{y}$$

Ridge regression shrinks coefficients toward zero but does not set them exactly to zero.

181.62.1.3.2. LASSO Regression (L1 Regularization)

$$\min_{\alpha, \beta} \sum_{i=1}^{n} \left[y_i - (\alpha + \beta^T x^i) \right]^2 + \lambda \sum_{j=1}^{p} |\beta_j|$$

LASSO performs feature selection because some coefficients can become exactly zero.

This problem is still convex but non-differentiable due to the absolute value term.

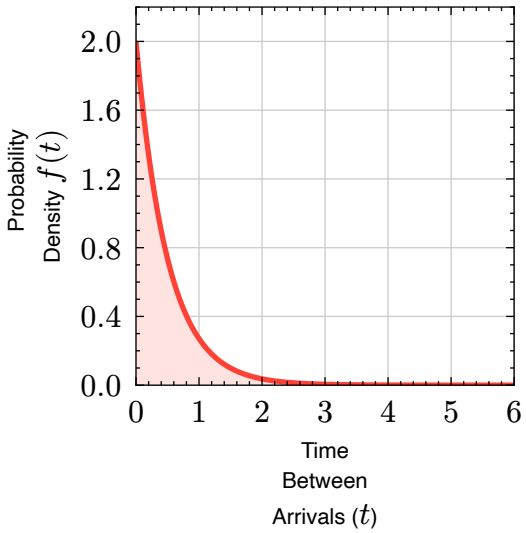
Example

181.63. Support Vector Machine

182. Queuing Theory

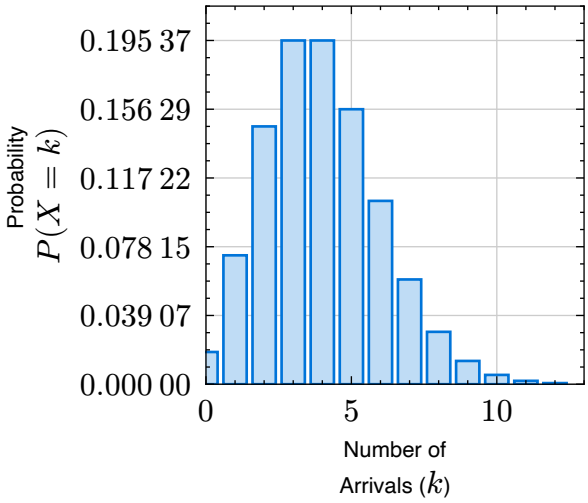
182.1. M/M/1

- Arrival rate (λ): Average number of customers arriving per unit of time
- Service rate (μ): Average number of customers served per unit of time



Exponential Distribution: Inter-arrival Times

Rate parameter λ = 2 (same as above)



Poisson Distribution: Number of Arrivals in Time Period $t = 2$

Rate parameter $\lambda = 2$, so $\lambda t = 4$

Key Relationship

Both distributions share the same rate parameter λ = 2

- If inter-arrival $\times \sim \text{Exponential}(\lambda)$, then arrivals in time $t \sim \text{Poisson}(\lambda t)$
- Mean inter-arrival time = $\frac{1}{\lambda}$ = 0.5
- Mean arrivals per unit time = λ = 2
- Mean arrivals in time period 2 = λt = 4

1. Utilization (ρ)

Fraction of time the server is busy

$$\rho = \frac{\lambda}{\mu}$$

2. Average Number of Customers in the System (L)

Average number of customers (both waiting and being served) in the system

$$L = \frac{\rho}{1 - \rho}$$

3. Average Number of Customers in the Queue (L_q)

Average number of customers waiting in the queue

$$L_q = \frac{\rho^2}{1 - \rho}$$

4. Average Time a Customer Spends in the System (W)

Average time a customer spends in the system (from arrival until they are done being served)

$$W = \frac{1}{\mu - \lambda}$$

5. Average Waiting Time in the Queue (W_q)

Average time a customer spends just waiting in line before being served

$$W_q = \frac{\rho}{\mu - \lambda}$$

6. Probability that the System is Empty (P_0)

Probability that there are zero customers in the system (no one is being served and no one is waiting)

$$P_0 = 1 - \rho$$

7. Probability that n Customers are in the System (P_n)

Probability that there are n customers in the system (either waiting or being served)

$$P_n = (1 - \rho) \cdot \rho^n$$

8. Probability the Queue is Full (if the Queue has Limited Capacity) ($P_{n_{\max}}$)

Probability that the system is at full capacity

$$P_{n_{\max}} = (1 - \rho) \cdot p^{n_{\max}}$$

9. System Throughput

Rate at which customers are served and leave the system

$$\text{Throughput} = \lambda$$

10. Expected Time in Service (W_s)

Average time a customer spends actually being served (not including waiting time)

$$W_s = \frac{1}{\mu}$$

11. Idle Time ($1 - \rho$)

Fraction of time that the server is idle (i.e., not serving any customers)

$$\text{Idle Time} = 1 - \rho$$

12. Probability of Having to Wait in the Queue (P_w)

Probability that an arriving customer will have to wait before being served, i.e., that the server is busy when the customer arrives

$$P_w = \rho$$

13. Variance of the Number of Customers in the System ($\mathrm{Var}(L)$)

Variance of the number of customers in the system

$$\mathrm{Var}(L) = \frac{\rho}{(1-\rho)^2}$$

Example

A bank with a single teller

- **Arrival rate (λ)**: On average, 4 customers arrive every 10 minutes ($\lambda = 4$ custommers per 10 minutes)
- **Service Rate (μ)**: The teller can serve 6 customers every 10 minutes ($\mu = 6$ customers per 10 minutes)

1. **Utilization (ρ)**

$$\rho = \frac{\lambda}{\mu} = \frac{4}{6} = 0.67$$

The teller is busy 67% of the time. The remaining 33% of the time, the teller is idle, waiting for the next customer

2. Average Number of Customers in the System (L)

$$L = \frac{\rho}{1-\rho} = \frac{0.67}{1-0.67} = 2$$

On average, there are 2 customers in the coffee shop at any given time, either being served or waiting in line

3. Average Number of Customers in the Queue (L_q)

$$L_q = \frac{\rho^2}{1-\rho} = \frac{0.67^2}{1-0.67} = 1.33$$

On average, about 1.33 customers are waiting in line at any time

4. Average Time a Customer Spends in the System (W)

$$W = \frac{1}{\mu - \lambda} = \frac{1}{6} - 4 = 0.5$$

On average, a customer spends 5 minutes (0.5×10 minutes) in the shop (including both waiting in line and getting served)

5. Average Waiting Time in the Queue (W_q)

$$W_q = \frac{\rho}{\mu - \lambda} = \frac{0.67}{6 - 4} = 0.33$$

On average, a customer waits 3.3 minutes (0.33×10 minutes) in line before being served by the teller

6. Probability that the System is Empty (P_0)

$$P_0 = 1 - \rho = 1 - 0.67 = 0.33$$

There is a 33% chance that the coffee shop is empty, meaning there is no customer in the queue or being served

7. Probability that n Customers are in the System (P_n)

$$P_n = (1 - \rho) \cdot \rho^n = (1 - 0.67) \cdot 0.67^2 = 0.148$$

There is a 14.8% chance that exactly 2 customers are either in line or being served

8. Probability the Queue is Full (if the Queue has Limited Capacity) ($P_{n_{\max}}$)

$$P_{n_{\max}} = (1 - \rho) \cdot p^{n_{\max}} = (1 - 0.67) \cdot 0.67^5 = 0.028$$

There is a 2.8% chance that the system is full, and no new customers can enter

9. System Throughput

$$\text{Throughput} = \lambda = 4$$

The coffee shop serves 4 customers every 10 minutes, on average

10. Expected Time in Service (W_s)

$$W_s = \frac{1}{\mu} = \frac{1}{6} = 0.167$$

On average, a customer spends 1.67 minutes being served by the teller

11. Idle Time ($1 - \rho$)

$$\text{Idle Time} = 1 - \rho = 1 - 0.67 = 0.33$$

The teller is idle 33% of the time

12. Probability of Having to Wait in the Queue (P_w)

$$P_w = \rho = 0.67$$

There is a 67% chance that a customer will have to wait when they arrive

13. Variance of the Number of Customers in the System ($\text{Var}(L)$)

$$\text{Var}(L) = \frac{\rho}{(1 - \rho)^2} = \frac{0.67}{(1 - 0.67)^2} = 6.12^2$$

The queue length varies significantly, with a variance of 6.12 customers

Costs

- **Cost per Waiting Customer per Unit Time (C_w)**: cost incurred per customer for each unit of time they spend waiting in the queue

$$\text{Total Waiting Cost} = L_q \times C_w \times \text{Unit Time}$$

- **Cost per Idle Server per Unit Time (C_s)**: cost incurred per unit time when the server is not serving customers

$$\text{Total Idle Cost} = (1 - \rho) \times C_s \times \text{Unit Time}$$

Total Cost

$$\text{Total Cost} = \text{Total Waiting Cost} + \text{Total Idle Cost}$$

Example

- Arrival Rate (λ): 4 customers per 10 minutes
- Service Rate (μ): 6 customers per 10 minutes
- Cost per Waiting Customer per Hour (C_w): $10
- Cost per Idle Server per Hour (C_s): $20
- Operational Time: 1 hour

1. Utilization

$$\rho = \frac{\lambda}{\mu} = \frac{4}{6} = 0.67$$

2. Average Number of Customers in Queue (L_q)

$$L_q = \frac{\rho^2}{1 - \rho} = \frac{0.67^2}{1 - 0.67} = 1.33 \text{ cusommers}$$

3. Total Waiting Cost

$$\text{Total Waiting Cost} = L_q \times C_w \times \text{Unit Time} = 1.33 \times 10 \times 1 = \$13.33$$

4. Idle Time

$$\text{Idle Time} = 1 - \rho = 1 - 0.67 = 0.33$$

5. Total Idle Cost

$$\text{Total Idle Cost} = (1 - \rho) \times C_s \times \text{Unit Time} = 0.33 \times 20 \times 1 = \$6.67$$

6. Total Cost

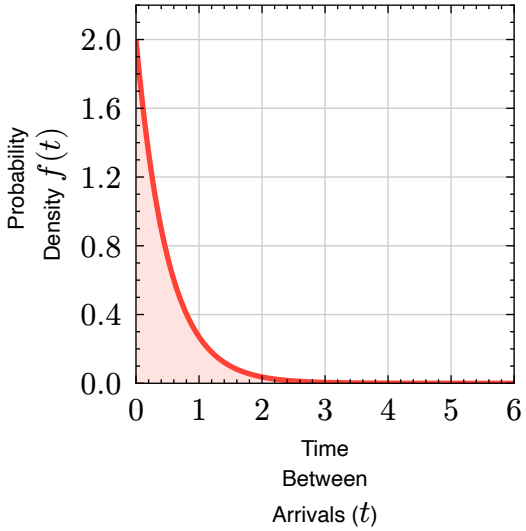
$$\text{Total Cost} = \text{Total Waiting Cost} + \text{Total Idle Cost} = 13.33 + 6.67 = \$20$$

182.2. M/M/c

- M: Memoryliss interarrival times
- M: Memoryless service times
- c: number of servers

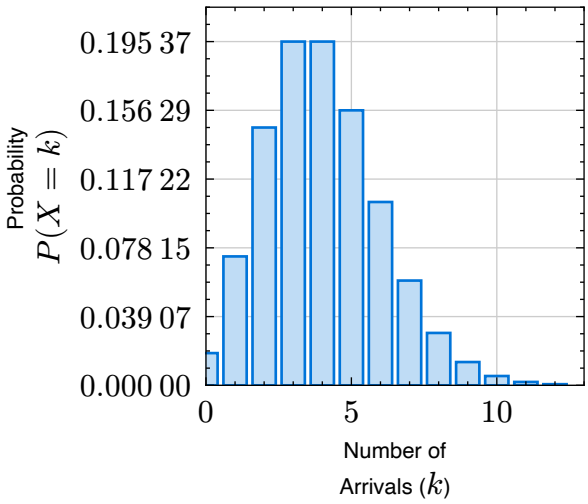
Input parameters:

- c: number of parallel servers in the system
- μ: average service rate of μ (mu) per server
    - Exponential distribution: μ customers per unit of time
- λ: average rate of arrivals per unit of time
    - Poisson process: number of arrivals in a given time period
    - Exponential distribution: time between successive arrivals



Exponential Distribution: Inter-arrival Times

Rate parameter λ = 2 (same as above)

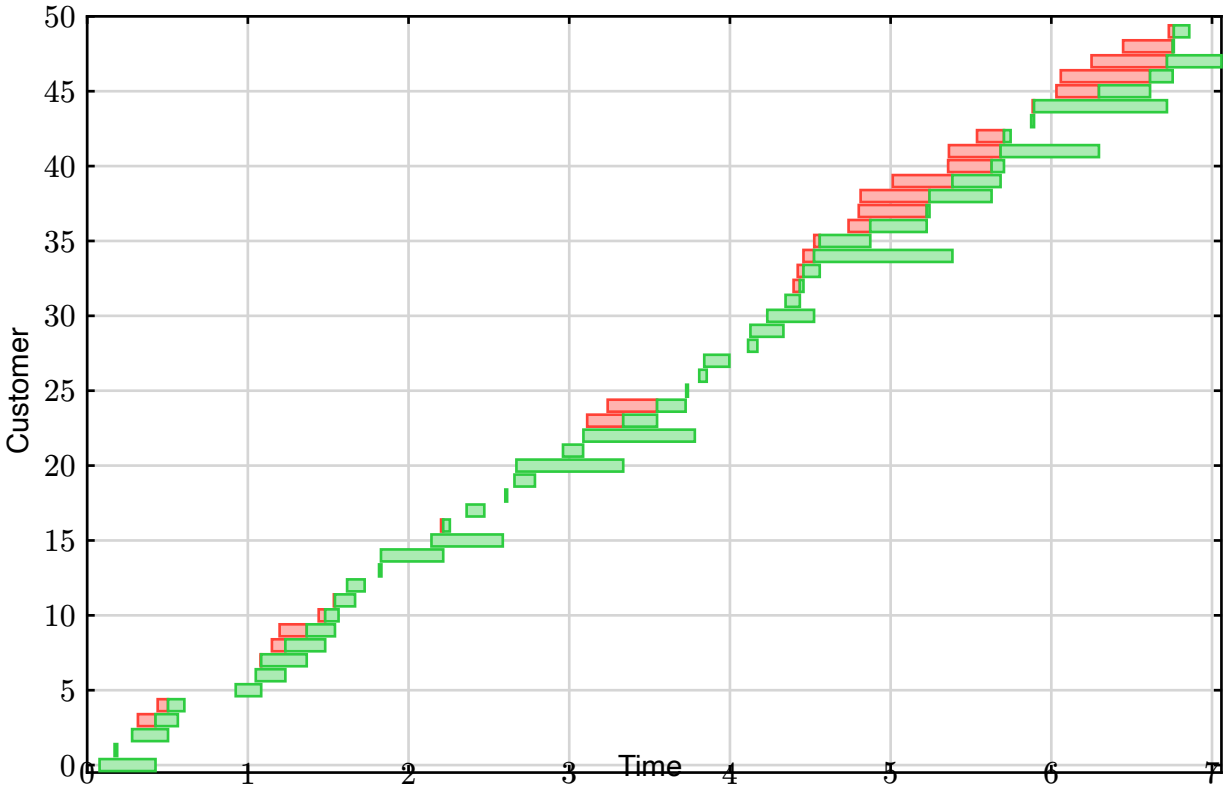


Poisson Distribution: Number of Arrivals in Time Period $t = 2$

Rate parameter $\lambda = 2$, so $\lambda t = 4$

System Stability

- $\lambda > c\mu$:
    - Arrival rate greater than service rate
    - Queue will grow infinitely
- $\lambda < c\mu$:
    - Without variability: no queue or wait time
    - With variability: may have wait time
- $\lambda = c\mu$:



Performance Metrics

1. Utilization (ρ)

$$\rho = \frac{\lambda}{c \cdot \mu}$$

Where:

- λ: arrival rate (mean number of arrivals per unit of time)
- μ: service rate (mean number of services completed per server per unit of time)

2. Probability of Zero Customers in the System (Idle Probability) (P_0)

Probability that no customers are in the system

$$P_0 = \left(\sum_{n=0}^{c-1} \frac{\left(\frac{\lambda}{\mu}\right)^2}{n!} + \frac{\left(\frac{\lambda}{\mu}\right)^c}{c!(1-p)} \right)^{-1}$$

3. Probability that All Servers are Busy (Blocking Probability or Queueing Probability) (P_w)

Probability that all c servers are busy and a customer will have to wait in the queue

$$P_w = \frac{\frac{\left(\frac{\lambda}{\mu}\right)^c}{c!}}{\sum_{n=0}^{c-1} \frac{\left(\frac{\lambda}{\mu}\right)^n}{n!} + \frac{\left(\frac{\lambda}{\mu}\right)^c}{c!(1-p)}}$$

4. Average Number of Customers in the System (L)

Average number of customers in the system (both in the queue and being served)

$$L = \frac{\lambda}{\mu} + \frac{P_w \cdot \left(\frac{\lambda}{\mu}\right)}{c(1-p)}$$

5. Average Number of Customers in the Queue (L_q)

Average number of customers waiting in the queue (not being served)

$$L_q = P_w \cdot \left(\frac{\rho \cdot \left(\frac{\lambda}{\mu}\right)}{1-\rho}\right)$$

6. Average Time a Customer Spends in the System (W)

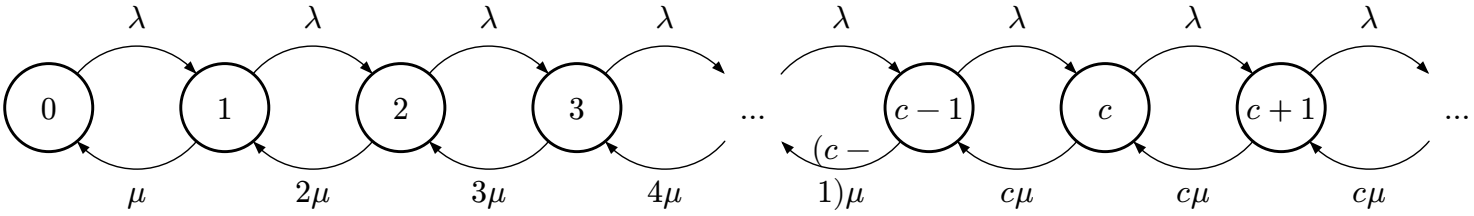
Average time a customer spends in the system (both waiting and being served) (Little's Law)

$$W = \frac{L}{\lambda}$$

7. Average Time a Customer Spends in the Queue (W_q)

Average waiting time a customer spends in the queue before being served

$$W_q = \frac{L_q}{\lambda} = P_w \cdot \frac{1}{c\mu(1-p)}$$



182.3. Little's Law

$$L = \lambda W$$

Where:

- L: average number of items (customers, tasks, etc.) in the system
- λ: average arrival rate (items per unit time)
- W: average time an item spends in the system

Example

Bank where customers arrive at an average rate of 10 customers per hour ($\lambda = 10$ customers / hour). Each customer spends an average of 12 minutes in the bank ($W = \frac{12}{60} = 0.2$ hours).

$$L = \lambda W = 10 \times 0.2 = 2$$

On average, there are 2 customers in the bank at any given time (includes being served and those waiting in line)

182.4. Kingman VUT

A foundational approximation for the **G/G/1** queue (general arrival distribution, general service distribution, single server). The VUT equation:

$$W_q \approx \underbrace{\frac{c_a^2 + c_s^2}{2}}_{V} \underbrace{\frac{\rho}{1-\rho}}_{U} \underbrace{\frac{1}{\mu}}_{T}$$

— Variability × Utilization × Time.

182.4.1. Components

- c_a: coefficient of variation of inter-arrival times ($\frac{\text{std}}{\text{mean}}$)
- c_s: coefficient of variation of service times
- $\rho = \frac{\lambda}{\mu}$: utilization (arrival rate / service rate)
- $\frac{1}{\mu}$: mean service time
- W_q: expected wait in queue (before service starts)

182.4.2. Three factors — three levers

1. **Variability** $V = \frac{c_a^2+c_s^2}{2}$
   - Reduce arrival variability (smooth incoming flow, level-load production)
   - Reduce service variability (standardize, reduce setup-time variation)
   - Halve variability → halve queue time
2. **Utilization** $U = \frac{\rho}{1-\rho}$
   - Critical: $U \to \infty$ as $\rho \to 1$
   - Going from $\rho = 0.8$ to $\rho = 0.9$ doubles the queue
   - Going to $\rho = 0.95$ doubles again
3. **Time** $T = \frac{1}{\mu}$
   - Faster service rate (lower $\frac{1}{\mu}$) shrinks queues directly
   - Both reduces T and lowers ρ at fixed λ

182.4.3. Why it's an approximation

The VUT formula is **exact** only for **M/M/1** (Poisson arrivals + exponential service, $c_a = c_s = 1$):

$$W_q^{\text{M/M/1}} = \frac{1+1}{2} \cdot \frac{\rho}{1-\rho} \cdot \frac{1}{\mu} = \frac{\rho}{\mu(1-\rho)} \checkmark$$

For other distributions, VUT is a **Kingman approximation** — derived from upper bounds (Kingman 1962). Tight when traffic is heavy ($\rho \to 1$); less accurate when load is light.

182.4.4. Generalization: G/G/c (multi-server)

For c identical servers, the **Sakasegawa approximation**:

$$W_q^{\mathrm{G/G/c}} \approx \frac{c_a^2 + c_s^2}{2} \cdot \frac{\rho^{\sqrt{2(c+1)}-1}}{c(1-\rho)} \cdot \frac{1}{\mu}$$

See Sakasegawa.

182.4.5. Variability-Utilization-Time intuition

This is the **single most important formula** in factory physics. It says:

- **Queues grow non-linearly in utilization** — 80% utilization is dramatically different from 95%
- **Variability is half the battle** — even at high utilization, low variability keeps queues small
- **Capacity and demand both matter** — but variability multiplies their effect

182.4.6. Common applications

- **Manufacturing lines**: setup variability, breakdown variability dominate
- **Call centers**: arrival peaks + service-time spread
- **Hospitals**: patient arrival randomness + variable treatment times
- **Computer systems**: request bursts + response-time variance

182.4.7. See also

- **Sakasegawa** — multi-server extension
- **Linking Equation** — variance propagation
- **M/M/1** — exact case
- **Little's Law** — $L = \lambda W$
- **Factory Physics**

182.5. Sakasegawa

Generalization of the VUT formula to **G/G/c** — multi-server queues with general arrival and service distributions.

182.5.1. Formula

$$W_q \approx \frac{c_a^2 + c_s^2}{2} \cdot \frac{\rho^{\sqrt{2(c+1)}-1}}{c(1-\rho)} \cdot \frac{1}{\mu}$$

where now:

- c: number of servers
- $\rho = \frac{\lambda}{c\mu}$: per-server utilization
- W_q: expected wait time

The factor $\frac{\rho^{\sqrt{2(c+1)}-1}}{c(1-\rho)}$ replaces the single-server $\frac{\rho}{1-\rho}$.

182.5.2. Why the exponent

For $c = 1$: $\sqrt{2 \cdot 2} - 1 = \sqrt{4} - 1 = 1 \to \rho^1 = \rho$, recovering Kingman VUT.

For $c \to \infty$: $\sqrt{2(c+1)} - 1 \to \infty$, and ρ^{large} goes to 0 rapidly → queues vanish at fixed ρ. As you add servers, you absorb variability **more efficiently** than just dividing the work — a fundamental insight.

182.5.3. Practical impact

Two M/M/1 servers handling $\frac{\lambda}{2}$ each vs one M/M/2 server handling λ:

Setup	ρ	$\frac{W_q}{\frac{1}{\mu}}$	Comparison
Two separate M/M/1	$\rho_1 = \frac{\lambda}{2\mu}$	$\frac{\rho_1}{1-\rho_1}$	each server has own queue
Pooled M/M/2	$\rho_2 = \frac{\lambda}{2\mu}$	$\frac{\rho_2^{\sqrt{6}-1}}{2(1-\rho_2)}$ — lower	shared queue, better

Pooling reduces waiting time even at the same per-server utilization. This is the queueing analog of risk pooling.

182.5.4. Limitations

- **Approximation**: tight in heavy traffic, less accurate when ρ is small
- **Identical servers** assumed
- **No abandonment**: customers can't leave; for that see Erlang A

182.5.5. Where used

- **Call center sizing**: how many agents for given call volume and service-time distribution
- **Web service capacity planning**: request handlers
- **Bank teller / DMV staffing**
- **Manufacturing**: multiple parallel machines

182.5.6. See also

- **Kingman VUT** — single-server case
- **Linking Equation** — variance propagation across stations
- **Erlang C** — exact M/M/c waiting probability
- **Erlang B** — exact M/M/c/c (loss)

182.6. Linking Equation

A simple but powerful approximation for **how variability propagates through tandem queues**. The departure-process variability from one station is the arrival variability for the next.

182.6.1. Formula

For a single station with utilization ρ, arrival CoV c_a, and service CoV c_s, the **departure** CoV c_d is:

$$c_d^2 = (1 - \rho^2)c_a^2 + \rho^2 c_s^2$$

— a weighted average of arrival and service variability, weighted by ρ^2 and $1 - \rho^2$.

182.6.2. Two extreme regimes

Low utilization ($\rho \to 0$): $c_d \approx c_a$. The station is mostly idle; departures inherit the arrival pattern.

High utilization ($\rho \to 1$): $c_d \approx c_s$. The station is always busy; departures inherit the service-time pattern.

In between: blend.

182.6.3. Tandem queues: variance flows downstream

Tandem layout:

```
Station 1 → Station 2 → Station 3 → ... → Station N
```

Arrival CoV at station 2 = departure CoV from station 1 = $c_d^{(1)}$.

Applying the linking equation repeatedly:

$$c_a^{(k+1)} = c_d^{(k)}$$

So variance **propagates** downstream. Stations near the bottleneck (high ρ) propagate their service variability strongly; idle stations pass through arrival variability.

182.6.4. Why it matters: the Hopp-Spearman corollary

In a **balanced** line (all stations have similar ρ and c_s), variability builds up but stays bounded. In an **unbalanced** line, one bad station injects variance that downstream stations can't fully absorb — leading to queue blowups far from the original problem.

This is part of why **line balancing** (here) and **variance reduction** (eliminating setups, breakdowns) are foundational manufacturing improvements.

182.6.5. Bottleneck analysis

The linking equation lets you predict where queues will build **before** operations start:

1. Compute ρ at each station
2. Apply the linking equation from upstream to downstream
3. Combine with VUT / Sakasegawa to estimate queue lengths

Identifies bottleneck location and severity quantitatively.

182.6.6. Limitations

- **Approximation** — exact only for special distributions
- **Independence assumed** — auto-correlation in service or arrivals breaks the formula
- **Open queues** — doesn't handle blocking / starvation directly (need finite-buffer extensions)

For finite buffers: see two-machine finite-buffer models (Buzacott-Shanthikumar) — more complex but capture blocking effects.

182.6.7. See also

- **Kingman VUT**
- **Sakasegawa**
- **Best/Worst/PWC** — Hopp-Spearman line performance
- **Line Balancing**

182.7. Erlang B

The **Erlang B formula**: probability that all servers in an M/M/c/c queue (Poisson arrivals, exponential service, c servers, c capacity = no queue) are busy. Arriving customers facing all-busy are **blocked / lost**.

182.7.1. Formula

$$B(c, a) = \frac{\frac{a^c}{c!}}{\sum_{k=0}^{c} \frac{a^k}{k!}}$$

where $a = \frac{\lambda}{\mu}$ is the **offered load** (in Erlangs).

$B(c, a)$ is the **blocking probability** — proportion of arrivals turned away.

182.7.2. Telephony origin

Developed by A. K. Erlang for telephone networks (1917). c servers = trunk lines; blocked calls are dropped (no queue — caller hears busy signal).

Used for over a century to **size phone networks**: how many trunk lines do I need to keep blocking probability below some threshold (typically 1% or 0.1%)?

182.7.3. Worked example

$\lambda = 50$ calls/hour, average call lasts 3 minutes = 0.05 hr. So $a = 50 \cdot 0.05 = 2.5$ Erlangs.

How many trunks to keep blocking below 1%?

c	$B(c, a) \approx$
3	35%
5	9%
7	1.3%
8	0.4%

Need $c = 8$ trunks for $< 1\%$ blocking with offered load 2.5 Erlangs.

182.7.4. Recursive formula (numerically stable)

$$B(c, a) = \frac{aB(c-1, a)}{c + aB(c-1, a)}, \quad B(0, a) = 1$$

This recursion avoids the factorials that overflow in direct computation. Standard implementation.

182.7.5. Erlang B vs Erlang C

Model	System	Blocked customer
Erlang B (M/M/c/c)	c servers, no queue	lost (dropped)
Erlang C (M/M/c)	c servers, infinite queue	waits

Erlang B is the **blocking probability** (proportion lost). Erlang C is the **waiting probability** (proportion who must wait > 0).

For the same parameters, Erlang B < Erlang C.

182.7.6. Where it shows up

- **Telephone trunk sizing** (historical, but still relevant for VoIP gateways)
- **Connection limits** on servers (max concurrent connections)
- **Hospital bed capacity** (refused-admission probability)
- **Parking lot sizing** (turn-away rate)
- **Toll booth design**

182.7.7. Insensitivity to service distribution

Surprisingly: Erlang B is **insensitive** to the service-time distribution — only the **mean** matters. So even if call durations are highly variable, Erlang B's blocking probability formula stays exact. Holds because of **PASTA** (Poisson Arrivals See Time Averages) + reversibility.

182.7.8. See also

- **Erlang C** — with queue
- **M/M/1** — single-server with queue
- **M/M/c** — multi-server with queue
- **Erlang A** — with abandonment

182.8. Erlang C

The **Erlang C formula**: probability that an arriving customer has to **wait** in an M/M/c queue (Poisson arrivals, exponential service, c servers, infinite buffer).

The exact analog of M/M/c queue's "no-empty-server probability" at steady state.

182.8.1. Formula

$$C(c,a) = \frac{\frac{(ca)^c}{c!}}{\frac{(ca)^c}{c!} + (c-ca)\sum_{k=0}^{c-1}\frac{(ca)^k}{k!}}$$

Wait — common convention uses **offered load** $a = \frac{\lambda}{\mu}$ (in Erlangs) and **number of servers** c, with utilization $\rho = \frac{a}{c} < 1$:

$$C(c,a) = \frac{\frac{a^c}{c!}}{(1-\rho)\sum_{k=0}^{c-1}\frac{a^k}{k!} + \frac{a^c}{c!}}$$

$C(c,a)$ is the **probability of waiting** (proportion who don't get served immediately).

182.8.2. Expected wait

Given $C(c, a)$ is the wait probability, the **expected wait in queue**:

$$E[W_q] = \frac{C(c, a)}{c\mu - \lambda}$$

Same form as M/M/1 expected wait, just multiplied by the probability of waiting (since non-waiters contribute zero).

182.8.3. Where it shows up

- **Call center staffing**: how many agents to keep wait probability below threshold
- **Bank tellers / DMV staffing**
- **Help desk / IT support** sizing
- **Server farm capacity**

182.8.4. Worked example

Help desk: 30 calls per hour, average call lasts 5 minutes = $\frac{1}{12}$ hr. Offered load: $a = 30 \cdot \frac{1}{12} = 2.5$ Erlangs.

c (agents)	$\rho = \frac{a}{c}$	$C(c, a) \approx$
3	0.83	60%
4	0.625	27%
5	0.5	13%
6	0.42	6%

To keep wait probability under 10%, need $c = 6$ agents. Wait time at $c = 6$: $E[W_q] = \frac{0.06}{6 \cdot 12 - 30} = \frac{0.06}{42} \approx 86$ seconds.

182.8.5. Erlang C in call-center practice

The industry standard formula. Variants:

- **Square-root staffing rule** (Halfin-Whitt regime): $c \approx \rho + \beta\sqrt{\rho}$ for some quality-of-service constant β
- **Erlang A**: add abandonment — customers leave after waiting too long; see Erlang A
- **Skill-based routing**: agents are heterogeneous; needs simulation

182.8.6. Limitations

- **Exponential service times** — real call durations are heavier-tailed (often lognormal)
- **Poisson arrivals** — works for unscheduled arrivals; not for scheduled appointments
- **Infinite buffer** — real callers abandon; use Erlang A

For most call-center sizing, Erlang C is good enough. For accuracy use simulation.

182.8.7. See also

- **Erlang B** — without queue (blocking)
- **M/M/c** — the queue model itself
- **Erlang A** — with abandonment

- **Square-Root Staffing**

182.9. MMc Abandonment

Erlang A (also written M/M/c+M): M/M/c queue with **abandonment** — customers leave after waiting too long. More realistic for call centers, where impatient callers hang up.

182.9.1. Setup

- λ: arrival rate (Poisson)
- μ: service rate per server (exponential)
- c: number of servers
- θ: abandonment rate (exponential patience time, mean $\frac{1}{\theta}$)

182.9.2. Why it matters

Erlang C assumes infinite patience — every customer eventually gets served. Erlang A models the **real** situation where customers hang up at some rate.

Without abandonment: a slightly-overloaded system has **unbounded** waits. With abandonment: a slightly-overloaded system has bounded waits **and** some abandonment rate.

182.9.3. Approximate formulas

Exact Erlang A is nonelementary — uses incomplete gamma function. The most useful results:

Abandonment probability:

$$P(\text{abandon}) \approx C(c, a) \cdot \frac{\rho - 1}{\dots}$$

(Specific formulas are technical; key insight is monotonicity: more $\theta \to$ faster abandonment $\to$ less waiting.)

Expected wait given service:

$$E[W_q] \approx \frac{C(c, a)}{c\mu - \lambda + \theta(c - \rho c)}$$

(Approximate; sharper formulas in queueing texts.)

182.9.4. Garnett-Mandelbaum-Reiman scaling

In heavy traffic ($\rho \approx 1$), Erlang A admits a clean scaling limit (Halfin-Whitt regime extended). Lets you size:

- Quality-driven regime ($\beta > 0$): wait > 0 rare; small abandonment
- Efficiency-driven regime ($\beta < 0$): wait probable, abandonment substantial
- Quality-and-efficiency-driven (QED, $\beta \approx 0$): balanced

Used in modern call-center capacity planning.

182.9.5. Call-center application

Erlang A predicts **service level** (= P(answered within X seconds, before abandoning)) directly. For typical inbound centers:

- Target: SL = 80% of calls answered within 20 seconds
- Erlang A converts agent count + abandonment rate + call duration into SL

This is what tools like Genesys, Five9, and Erlang calculators compute.

182.9.6. Compared to Erlang C

	Erlang C (M/M/c)	Erlang A (M/M/c+M)
Abandonment	no	yes
Customers behavior	infinite patience	exponential patience with mean $\frac{1}{\theta}$
Overload regime	unbounded queue	bounded by abandonment
Realism	optimistic on staffing	closer to reality
Computation	closed form	more complex, often approximated

Erlang A **recommends fewer agents** than Erlang C at the same target SL, because abandonment shortens queues — closer to reality.

182.9.7. Patience modeling

Real patience is rarely exponential — often:

- **Lognormal** — most calls abandon quickly, but a tail waits long
- **Mixture** — different customer segments with different patience
- **Time-varying** — patience changes with wait

These need simulation. Erlang A with exponential patience is the analytical starting point.

182.9.8. See also

- **Erlang C** — without abandonment
- **Erlang B** — no queue at all
- **Square-Root Staffing**

182.10. Square-Root Staffing

A simple rule of thumb for sizing call-center / service-system capacity. **Halfin-Whitt** asymptotic regime (1981).

182.10.1. The rule

Given offered load $a = \frac{\lambda}{\mu}$ Erlangs, optimal number of servers is approximately:

$$c \approx a + \beta\sqrt{a}$$

— a base of a servers (deterministic capacity) plus a **safety margin** $\beta\sqrt{a}$ that grows as $\sqrt{a}$.

182.10.2. Three quality regimes (parameterized by β)

- **Efficiency-driven** ($\beta < 0$): servers saturated, customers wait often. Wait time scales as $\frac{1}{\sqrt{a}}$.
- **Quality-driven** ($\beta > 2$ or so): servers idle often, customers rarely wait. Wait time scales as $a^{-\frac{1}{2}}$ with very small constant.
- **Quality-and-Efficiency-Driven** (QED, $\beta \approx 0.5 - 1.5$): balanced — wait probability and idle probability both moderate.

For typical call centers targeting "80% of calls answered within 20 sec", $\beta \approx 0.7 - 1.0$.

182.10.3. Why $\sqrt{a}$?

In the heavy-traffic limit, queueing behavior is governed by **central limit-style fluctuations**. The natural fluctuation scale of an offered load a is $\sqrt{a}$ — same as the standard deviation of a Poisson(a) distribution.

If you provision exactly a servers, fluctuations of size $\sqrt{a}$ overwhelm capacity routinely. If you provision $a + \sqrt{a}$, you absorb typical fluctuations. Bigger $\beta \to$ more safety.

182.10.4. Example

Call center: 50 calls/min, average 3 min per call $\to a = 150$ Erlangs.

Strategy	β	$c = a + \beta\sqrt{a}$	Behavior
No safety	0	150	50% chance customer waits
Modest	0.5	156	30% wait probability
Quality	1.5	168	10% wait probability
Premium	2.5	181	< 3% wait probability

(Wait probabilities approximate, depend on abandonment, service distribution, etc.)

182.10.5. Practical use

1. Estimate offered load $a = \frac{\lambda}{\mu}$
2. Choose target service level (e.g., 80% answered in 20 sec)
3. Look up corresponding β (or simulate to calibrate)
4. Staff $c \approx a + \beta\sqrt{a}$

Compare to Erlang C / Erlang A exact calculations — typically very close.

182.10.6. Why it's the right scale

For a fixed **fraction-of-time-waiting** target (constant probability of waiting), β stays **constant** as offered load grows. So if you double traffic (a doubles), safety margin only grows as $\sqrt{2} \approx 1.4$x — capacity grows by $\sqrt{a}$ less than linearly.

This is the **economies of scale** of pooling: big call centers need **proportionally less** slack capacity than small ones.

182.10.7. See also

- **Erlang C** — exact wait probability

- **Erlang B** — no queue / blocking
- **Erlang A** — with abandonment
- **Sakasegawa** — G/G/c approximation

183. Decision Analysis

183.1. Decision Analysis

A framework for making **decisions under uncertainty**. Distinguishes:

- **States of nature**: random outcomes outside your control (demand, weather, exchange rates)
- **Acts / decisions**: things you choose (build big plant? buy insurance? launch product?)
- **Payoffs**: cost or reward depending on (act, state) combination
- **Probabilities**: belief about likelihood of each state

Decision analysis asks: **given uncertainty, how should I choose?**

183.1.1. Core components

1. **Decision tree** — graphical representation of the decision problem
2. **EMV** — Expected Monetary Value, the standard decision criterion
3. **EVPI** — value of perfectly knowing the state in advance
4. **EVSI** — value of imperfect (sample) information
5. **Decision criteria** — alternatives to EMV under deep uncertainty
6. **Utility theory** — risk-averse decision making

183.1.2. Steps in a decision analysis

1. **Frame**: identify objectives, alternatives, uncertainties, payoffs
2. **Structure**: build a decision tree (or influence diagram)
3. **Quantify**: assign probabilities and payoff values
4. **Evaluate**: compute expected value of each strategy
5. **Sensitivity analysis**: how does the answer change with input changes?
6. **Recommend**: highest expected value (or by other criterion)

183.1.3. When to use

- **Strategic decisions** (capital projects, market entry) with multi-million-dollar payoffs
- **Decisions involving uncertain outcomes** with quantifiable probabilities
- **Sequence decisions** (act, observe, decide again) — decision trees are natural
- **Comparison of alternatives** under uncertainty

183.1.4. Limits

- **Probability estimation** is hard — anchoring, overconfidence biases
- **Risk aversion** not captured by EMV alone — use utility theory
- **Deep uncertainty** (probabilities themselves unknown) — see non-probabilistic criteria

- **Behavioral / strategic** considerations (game theory) outside this framework

183.1.5. See also

- **Decision Trees**
- **EMV**
- **EVPI** / **EVSI**
- **Decision Criteria**
- **Utility Theory**

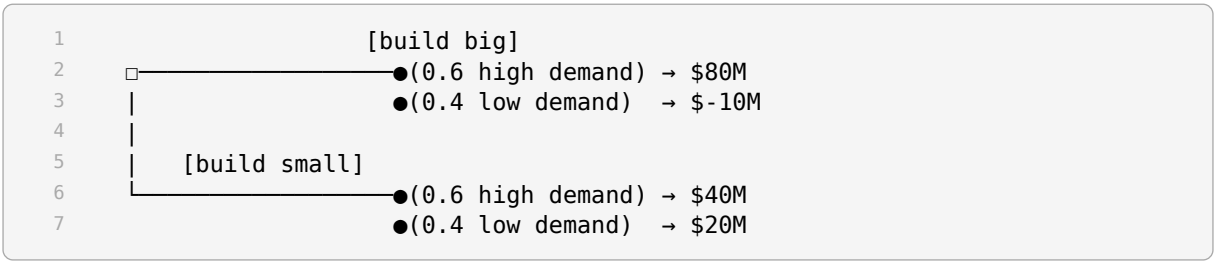
183.2. Decision Trees

A **decision tree** is the standard graphical model for decision analysis. Branches represent decisions or random outcomes; leaves carry payoffs; the tree's root is "now".

183.2.1. Node types

- **Decision nodes** (squares ?): branches are **acts** you choose
- **Chance nodes** (circles ◯): branches are **states of nature**, each with a probability
- **Terminal nodes** (triangles ▲): each leaf has a **payoff** (cost or revenue)

183.2.2. Example: build big plant or small?



183.2.3. Folding-back (rollback) algorithm

The algorithm to **solve** a decision tree: work from leaves backward to root.

```
At each chance node: compute expected payoff
    E[payoff] = Σ p_i × payoff_i over branches
At each decision node: pick the branch with the best expected payoff
    Tag the node with that branch's value (and which act to pick)
At the root: read off the optimal first decision and its expected value
```

This is just backward induction applied to a decision tree.

183.2.4. Solving the example

Big plant: $E[\text{payoff}] = 0.6 \cdot 80 + 0.4 \cdot (-10) = 48 - 4 = 44$ (millions).

Small plant: $E[\text{payoff}] = 0.6 \cdot 40 + 0.4 \cdot 20 = 24 + 8 = 32$ (millions).

Decision: build big. Expected value: 44M.

183.2.5. Multi-stage trees

Decisions can be sequenced: act, observe state, act again. Each stage adds a layer of (decision → chance) before reaching a payoff. Fold back as before — each chance / decision node still uses local expected value / argmax.

183.2.6. Sensitivity analysis

After solving, vary inputs (probabilities, payoffs) and re-solve. Identify which inputs matter most. Standard tools:

- **Tornado diagrams** — rank inputs by impact on decision
- **Spider plots** — show decision threshold crossings
- **Probability sensitivity** — at what p does the recommended decision change?

183.2.7. Common applications

- **Capital investment** — go / no-go, big / small project
- **Drug development** — phase-by-phase, abandon if results poor
- **Insurance / risk management** — buy coverage vs self-insure
- **Marketing campaigns** — pilot test then full launch
- **Litigation** — settle vs go to trial

183.2.8. See also

- **EMV** — the standard criterion
- **EVPI** — value of resolving uncertainty
- **EVSI** — value of imperfect info
- **Backward Induction** — same algorithm

183.3. EMV

Expected Monetary Value (EMV) — the standard decision criterion under decision analysis: pick the act with the highest expected payoff.

For act a_i with payoff V_{ij} in state s_j (probability p_j):

$$\mathrm{EMV}(a_i) = \sum_j p_j V_{ij}$$

Optimal act: $a^* = \arg\max_i \mathrm{EMV}(a_i)$.

183.3.1. Example

	State s_1 (high, $p = 0.6$)	State s_2 (low, $p = 0.4$)	EMV
Act a_1 (build big)	80	-10	$0.6 \cdot 80 + 0.4 \cdot (-10) = 44$
Act a_2 (build small)	40	20	$0.6 \cdot 40 + 0.4 \cdot 20 = 32$
Act a_3 (don't build)	0	0	0

(All values in millions of dollars.) Optimal by EMV: a_1, EMV $= 44$.

183.3.2. When EMV is the right criterion

- **Repeated decisions**: long-run average is the right measure; gambler's-ruin issues smoothed out
- **Risk-neutral** decision-maker: dollars are interchangeable
- **Decisions small relative to wealth**: linear approximation valid
- **Probabilities are reasonably well-known**: estimation error is small relative to payoffs

183.3.3. When EMV is not the right criterion

- **One-shot decision** with large downside: someone facing potential ruin will rationally avoid the high-EMV gamble (insurance, hedging)
- **Risk-averse decision-maker**: use **utility theory** — payoffs run through a **concave** utility function before averaging — see Utility Theory
- **Deep uncertainty**: probabilities not just unknown but unknowable — use non-probabilistic criteria
- **Multiple objectives**: not just money — see multi-criteria decision analysis

183.3.4. The St. Petersburg paradox

A coin is flipped until tails. If n flips, payoff $= 2^n$. Expected payoff: $\sum_n \left(\frac{1}{2}\right)^n \cdot 2^n = \sum_n 1 = \infty$.

But no rational person pays infinite money to play. Resolves: **utility** is concave (log-utility famously), and expected utility is finite. Bernoulli's solution from 1738 — first hint that EMV alone isn't enough.

183.3.5. See also

- **Decision Trees**
- **Decision Analysis**
- **EVPI** / **EVSI**
- **Utility Theory** — when EMV isn't enough

183.4. EVPI

Expected Value of Perfect Information (EVPI) — how much more you'd earn by **knowing the state of nature in advance**. The upper bound on what information is worth.

183.4.1. Definition

Let $\mathrm{EMV}^* = \max_i \mathrm{EMV}(a_i)$ — best expected value under uncertainty.

With perfect information (you know s_j before deciding): pick the **best act for each state**. Expected value:

$$\text{EV under PI} = \sum_j p_j \max_i V_{ij}$$

(Note: $\max$ inside the sum vs $\max$ of sums — perfect info lets you tailor act to state.)

$$\text{EVPI} = \text{EV under PI} - \text{EMV}^*$$

183.4.2. Example

From EMV example:

	s_1 (high)	s_2 (low)
a_1 (big)	80	-10
a_2 (small)	40	20
a_3 (no build)	0	0
Best in this state	80 (a_1)	20 (a_2)

$\text{EV under PI} = 0.6 \cdot 80 + 0.4 \cdot 20 = 48 + 8 = 56$

$\text{EMV}^* = 44$ (from a_1)

$\text{EVPI} = 56 - 44 = 12$

So perfectly knowing the market state in advance is worth 12M to this decision.

183.4.3. Why it's an upper bound

Real information sources (market research, pilot tests, expert forecasts) are **imperfect** — they update probabilities but don't reveal the state directly. The value of imperfect information (EVSI) is **always** less than EVPI:

$$\text{EVSI} \leq \text{EVPI}$$

If EVPI itself is small, **no information** is worth much — don't bother investing in market research.

183.4.4. Usage

- **Set a budget** for information gathering: $\text{EVPI} = 12$M means no research is worth more than 12M
- **Identify high-stakes uncertainties**: high EVPI signals where uncertainty matters most
- **Drive sensitivity analysis**: if EVPI is low, the decision is robust to uncertainty about that state

183.4.5. Decomposition by uncertainty

If there are multiple sources of uncertainty (demand **and** exchange rate), you can compute EVPI for each separately, or jointly. They're not additive in general.

183.4.6. See also

- **EVSI** — value of imperfect information
- **EMV**
- **Decision Trees**

183.5. EVSI

Expected Value of Sample Information (EVSI) — value of **imperfect** information, like a market survey that updates (but doesn't fully reveal) the state of nature.

The realistic counterpart of EVPI what most real-world decisions face.

183.5.1. Setup

You can run a test or survey before deciding. Each test outcome z:

- Occurs with some probability $P(z)$
- Updates your beliefs about state s via **Bayes' theorem**:

$$P(s \mid z) = \frac{P(z \mid s)P(s)}{P(z)}$$

After observing z, you make the **optimal** decision **given the updated beliefs**:

$$V(z) = \max_a \sum_s P(s \mid z) V_{a,s}$$

183.5.2. EVSI formula

$$\text{EV with sample info} = \sum_z P(z)V(z)$$

$$\text{EVSI} = \text{EV with sample info} - \text{EMV}^*$$

EVSI is always between 0 and EVPI:

$$0 \leq \text{EVSI} \leq \text{EVPI}$$

— at most EVPI (since no info beats perfect info), at least 0 (you can always ignore the info).

183.5.3. Example

States s_1 (high demand), s_2 (low demand), with $P(s_1) = 0.6$.

Test reliability:

- If s_1, test says "high" with $P = 0.8$
- If s_2, test says "high" with $P = 0.3$

$P(\text{test high}) = 0.6 \cdot 0.8 + 0.4 \cdot 0.3 = 0.60$ $P(\text{test low}) = 0.40$

Updates:

- $P(s_1 \mid \text{high}) = \frac{0.6 \cdot 0.8}{0.60} = 0.80$
- $P(s_1 \mid \text{low}) = \frac{0.6 \cdot 0.2}{0.40} = 0.30$

Given "test high" (probs $0.80, 0.20$):

- a_1: $0.8 \cdot 80 + 0.2 \cdot (-10) = 62$
- a_2: $0.8 \cdot 40 + 0.2 \cdot 20 = 36$
- **Best**: a_1, value 62

Given "test low" (probs $0.30, 0.70$):

- a_1: $0.3 \cdot 80 + 0.7 \cdot (-10) = 17$
- a_2: $0.3 \cdot 40 + 0.7 \cdot 20 = 26$

- **Best**: a_2, value 26

$\text{EV with sample info} = 0.60 \cdot 62 + 0.40 \cdot 26 = 37.2 + 10.4 = 47.6$

$\text{EVSI} = 47.6 - 44 = 3.6$ (vs EVPI $= 12$)

So this test is worth at most $\backslash 3.6$M. If it costs more, skip it.

183.5.4. Why EVSI < EVPI

The sample provides **probabilistic** updating, not certainty. Some test outcomes still leave significant ambiguity. EVPI assumes you'd be told **exactly** which state — much stronger.

183.5.5. Use cases

- **Market research budgeting**: should I commission a $\backslash 100$K consumer survey? Compare EVSI to cost.
- **Pilot programs**: small-scale rollout to learn before full commitment
- **Diagnostic tests**: in healthcare, sequential decisions to test before treating
- **A/B testing**: when is the next experiment worth running?

183.5.6. Bayesian decision-making

EVSI is fundamentally **Bayesian**: it requires prior probabilities, likelihood functions, and updating. The Bayesian decision theory framework subsumes EVPI and EVSI as special cases of expected utility under updated beliefs.

183.5.7. See also

- **EVPI** — upper bound
- **EMV**
- **Bayes' Rule** — the updating
- **Decision Trees**

183.6. Decision Criteria

Five classical criteria for decisions under uncertainty when **probabilities are unknown** or **deeply uncertain** (probability estimates aren't trustworthy). Used when EMV doesn't apply.

183.6.1. Setup

Acts a_i, states s_j, payoff matrix V_{ij}. No probabilities. How to choose?

Five rules give five different answers, reflecting different attitudes toward uncertainty.

183.6.2. Maximin (pessimist)

Worst-case thinking. For each act, find the worst possible payoff. Pick the act with the **best worst case**:

$$a^* = \arg\max_i \left(\min_j V_{ij} \right)$$

Very conservative — wars, safety-critical systems, fiduciary obligations.

183.6.3. Maximax (optimist)

Best-case thinking. For each act, find the best possible payoff. Pick the act with the **best best case**:

$$a^* = \arg\max_i \left(\max_j V_{ij} \right)$$

Aggressive — startup founders, venture capital, lottery players.

183.6.4. Minimax regret (Savage)

Avoid kicking-yourself thinking. For each (act, state), compute the **regret** — how much better you'd have done with the best act for that state:

$$R_{ij} = \max_k V_{kj} - V_{ij}$$

Then minimize the **maximum regret**:

$$a^* = \arg\min_i \left(\max_j R_{ij} \right)$$

Used when **retrospective comparison** is the bigger concern than absolute outcome — diplomatic / political decisions.

183.6.5. Hurwicz criterion (compromise)

Linear combination of optimism and pessimism. Coefficient of optimism $\alpha \in [0, 1]$:

$$H(a_i) = \alpha \max_j V_{ij} + (1 - \alpha) \min_j V_{ij}$$

$$a^* = \arg\max_i H(a_i)$$

$\alpha = 1$: maximax. $\alpha = 0$: maximin. $\alpha = 0.5$: balanced.

Calibrate α to the decision-maker's risk attitude (or context: α high in opportunity-rich environments, low in threat-heavy ones).

183.6.6. Laplace (insufficient reason)

Pretend each state is equally likely (uniform prior), then apply EMV:

$$L(a_i) = \frac{1}{n} \sum_j V_{ij} \qquad \text{(simple average over states)}$$

$$a^* = \arg\max_i L(a_i)$$

Equivalent to **Bayesian decision** under uniform prior. The first thing to try when you have no information about state probabilities.

183.6.7. Comparison example

Payoffs:

	s_1	s_2	s_3
a_1	50	-10	30
a_2	20	40	10
a_3	0	0	100

	Maximin	Maximax	Min Regret	Hurwicz ($\alpha = 0.6$)	Laplace
a_1	-10	50	0 (regret table needed)	$0.6 \cdot 50 + 0.4 \cdot (-10) = 26$	$\frac{1}{3} \cdot (50 - 10 + 30) = 23.3$
a_2	10	40		28	23.3
a_3	0	100		60	33.3

Different criteria, different winners — picks a_2 (Maximin) vs a_3 (Maximax / Hurwicz / Laplace).

183.6.8. Which to use?

No single right answer — depends on:

- Stakes (high-stakes one-shot → maximin)
- Repeatability (many similar decisions → Laplace/EMV)
- Reputation cost (criticism risk → minimax regret)
- Risk preference (default to Hurwicz with calibrated α)

183.6.9. See also

- **EMV** — when probabilities are known
- **Utility Theory** — handles risk aversion under known probabilities
- **Decision Analysis overview**

183.7. Utility Theory

A framework for decisions under **risk aversion**. Instead of maximizing expected payoff (EMV), maximize **expected utility** — where the utility function $u(x)$ encodes the decision-maker's preferences over wealth.

183.7.1. Why EMV is incomplete

Consider a fair coin flip: win 1M or lose 1M. EMV $= 0$. But most people refuse this gamble — the 1M downside is **worse** in subjective terms than the 1M upside is **better**.

This is **risk aversion**: declining marginal utility of wealth.

183.7.2. Utility function

$u : \mathbb{R} \to \mathbb{R}$ maps wealth to utility. The decision-maker maximizes:

$$E[u(W)] = \sum_j p_j u(W_j)$$

instead of $E[W] = \sum_j p_j W_j$.

For a risk-averse decision-maker, u is **concave**: $u''(x) \leq 0$.

183.7.3. Common forms

Linear — risk-neutral: $u(x) = x$. Reduces to EMV.

Logarithmic (Bernoulli 1738): $u(x) = \log(x)$. Strongly risk-averse for losses; resolves the St. Petersburg paradox.

Exponential: $u(x) = -e^{-\frac{x}{R}}$ for some risk parameter $R > 0$. Constant absolute risk aversion (CARA).

Power: $u(x) = \frac{x^{1-r}-1}{1-r}$ for $r > 0$. Constant relative risk aversion (CRRA).

Quadratic: $u(x) = x - ax^2$. Mean-variance — used implicitly in Markowitz portfolio theory.

183.7.4. Risk aversion measures

Arrow-Pratt absolute risk aversion:

$$A(x) = -u'' \frac{x}{u'}(x)$$

Arrow-Pratt relative risk aversion:

$$R(x) = xA(x) = -xu'' \frac{x}{u'}(x)$$

Used to **parameterize** and **measure** risk aversion empirically. Typical values for stock-market investors: $R \approx 1$ to 5.

183.7.5. Certainty equivalent

The **certainty equivalent** (CE) of a gamble X is the certain amount c with the same utility:

$$u(c) = E[u(X)] \qquad \Rightarrow\ c = u^{-1}(E[u(X)])$$

The **risk premium** is the amount the decision-maker would pay to avoid the risk:

$$\pi = E[X] - c$$

For a risk-averse DM (u concave): $c < E[X] \rightarrow \pi > 0$. The DM is willing to give up expected return to reduce variance.

183.7.6. Insurance application

You face a risk of losing L with probability p.

- **Expected loss**: pL
- **Certainty equivalent of loss**: c where $u(W - c) = (1 - p)u(W) + pu(W - L)$
- **Maximum premium you'd pay**: c — could be much greater than pL if you're risk-averse

This is **why insurance exists** — risk-averse buyers transfer risk to risk-pooling insurers.

183.7.7. Calibration: lottery method

To estimate someone's utility function:

1. Anchor: $u(W_{\min}) = 0$, $u(W_{\max}) = 1$

2. For intermediate W: ask "you're indifferent between W guaranteed and a 50/50 lottery between $W_{\min}$ and $W_{\max}$ — but at what W?"
3. Result: $u(W) = 0.5$ at the indifference point
4. Repeat to fill in the curve

Subjective and biased — but operationally usable.

183.7.8. See also

- **EMV** — risk-neutral baseline
- **Decision Trees** — apply utility to payoffs at leaves
- **Economics: Utility** — classical economic foundation
- **Decision Criteria** — non-probabilistic alternative

184. Control Charts

184.1. Cheatsheet

Chart	Tracks	Data Typse	Sample Size	Statistical Basis	Use Case
c	Number of defects per unit	Count (discrete)	Fixed	Poisson	When you count defects per inspection unit (e.g., scratches, cracks) and sample size is constant
u	Number of defects per unit (standardized)	Count per unit (continuous)	Variable	Poisson	Same as c-chart but for variable sample sizes (defects per unit area, time, etc.)
np	Number of defective units	Count (discrete)	Fixed	Binomial	Each item is either defective or not; count how many

Chart	Tracks	Data Typse	Sample Size	Statistical Basis	Use Case
					items fail inspection
p	Proportion of defective units	Proportion (0-1)	Variable	Binomial	Same as np-chart but with variable sample size — tracks percentage defective
$\bar{x}$	Mean of continuous variable	Continuous	Grouped samples	Normal	Monitors process average (e.g., length, weight, temperature) over time
R	Range of values in a sample	Continuous	Grouped samples	Range distribution (based on Normal)	Used with x̄-chart to monitor variation in process (spread)

184.2. C-Chart

Count of defects (fixed unit size)

Number of defects observed in each sample or inspection unit

$$\bar{c} = \frac{1}{k}\sum_{i=1}^{k} c_i$$

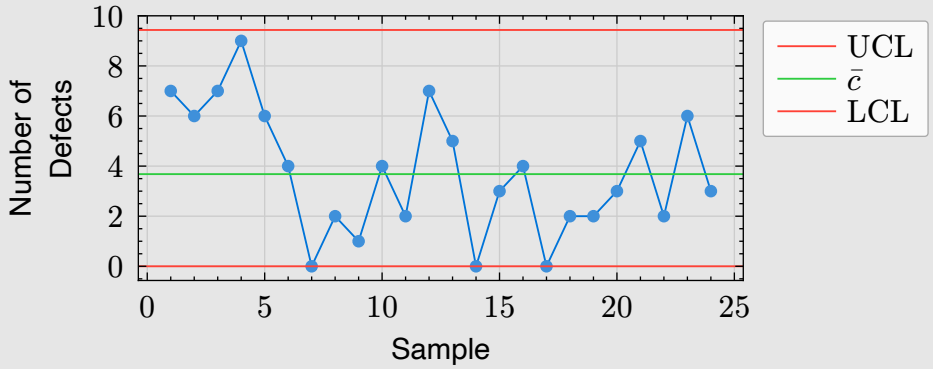
$$\mathrm{UCL}_c = \bar{c} + 3\sqrt{\bar{c}}$$
$$\mathrm{LCL}_c = \bar{c} - 3\sqrt{\bar{c}}$$

Where:

- c: number of defects

- k: number of samples

Example



This C-chart displays the number of defects identified in each of 25 inspected units, where each unit is of fixed size. The center line represents the average number of defects across all samples, calculated as:

$$\bar{c} = \frac{1}{k}\sum_{i=1}^{k} c_i$$

Control limits are drawn at ±3 standard deviations from the mean, using the square root of $\bar{c}$ to account for Poisson-distributed defect counts:

$$\mathrm{UCL}_c = \bar{c} + 3\sqrt{\bar{c}} \qquad \mathrm{LCL}_c = \max\left(0, \bar{c} - 3\sqrt{\bar{c}}\right)$$

184.3. P-Chart

The p-chart monitors the proportion of defectives in each sample, accounting for variable sample sizes.

$$\mathrm{UCL}_{p_i} = \bar{p} + 3\sqrt{\frac{\bar{p}(1-\bar{p})}{n_i}}$$

$$\mathrm{LCL}_{p_i} = \bar{p} - 3\sqrt{\frac{\bar{p}(1-\bar{p})}{n_i}}$$

Where:

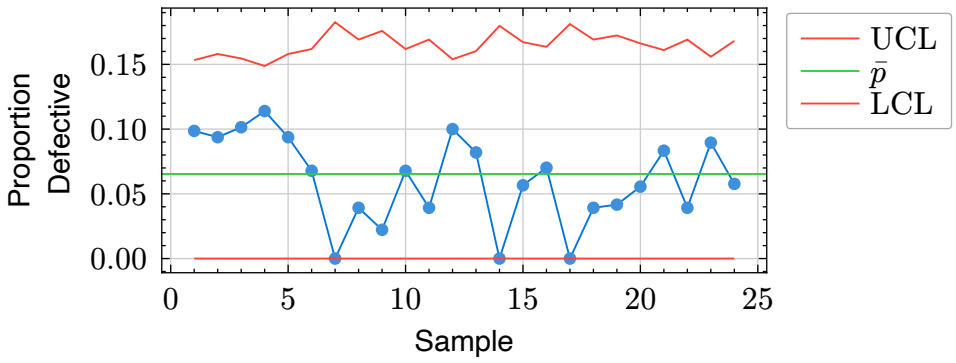
- d_i: The number of defective items observed in sample i
- p_i: sample proportion defective

$$p_i = \frac{d_i}{n_i}$$

- $\bar{p}$: overall proportion defective

$$\bar{p} = \frac{\sum d_i}{\sum n_i}$$

- n_i: sample size for sample i



184.4. U-Chart

Defects per unit (variable unit size)

$$\mathrm{UCL}_u = \bar{u} + 3\sqrt{\frac{\bar{u}}{n_i}}$$

$$\mathrm{LCL}_u = \bar{u} - 3\sqrt{\frac{\bar{u}}{n_i}}$$

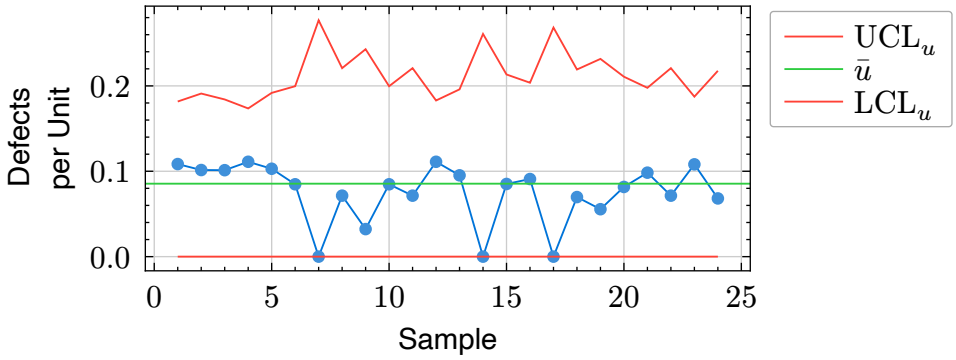
Where:

- c_i: number of defects in sample i
- n_i: sample size of group i
- u_i: defects per unit

$$u_i = \frac{c_i}{n_i}$$

- $\bar{u}$:

$$\frac{\sum c_i}{\sum n_i}$$



184.5. NP-Chart

184.5.1. NP-charts (Number Proportion)

Number of defective items (constant sample size)

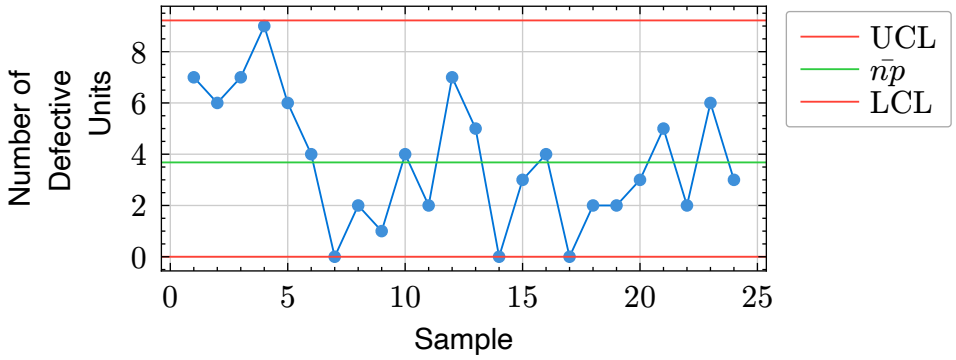
$$\text{UCL}_{np} = n\bar{p} + 3\sqrt{n\bar{p}(1-\bar{p})}$$
$$\text{LCL}_{np} = n\bar{p} - 3\sqrt{n\bar{p}(1-\bar{p})}$$

Where:

- n: count of defective items in each sample
- p: proportion of defectives
- $\bar{p}$:

$$\bar{p} = \frac{1}{k}\sum_{i=1}^{k}\frac{d_i}{n}$$
$$= \frac{\sum_{i=1}^{k} d_i}{n \cdot k}$$

- $n\bar{p} = n\bar{p}$



184.6. X-Chart

184.6.1. $\bar{X}$-chart

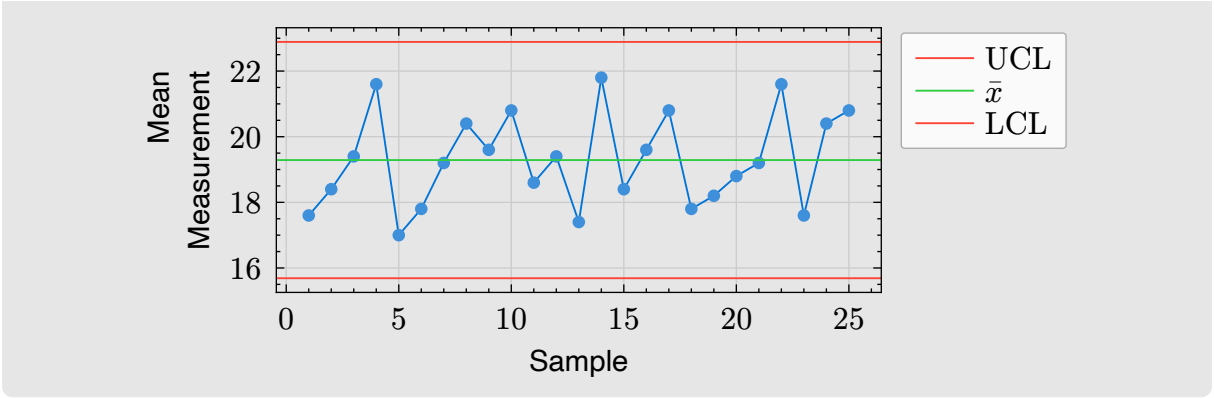
Used to monitor the average of a continuous quality characteristic (e.g., weight, length, temperature), assuming constant sample size per subgroup.

$$\text{UCL}_{\bar{x}} = \bar{x} + A_2 \cdot \bar{R}$$
$$\text{UCL}_{\bar{x}} = \bar{x} - A_2 \cdot \bar{R}$$

Where:

- $\bar{x}_i$: average of subgroup i
- $\bar{x}$: grand mean of all subgroup means
- $\bar{R}$: average range of all subgroups
- A_2: constant dependent on sample size n (e.g., $A_2 = 0.577$ for $n = 5$)

Example

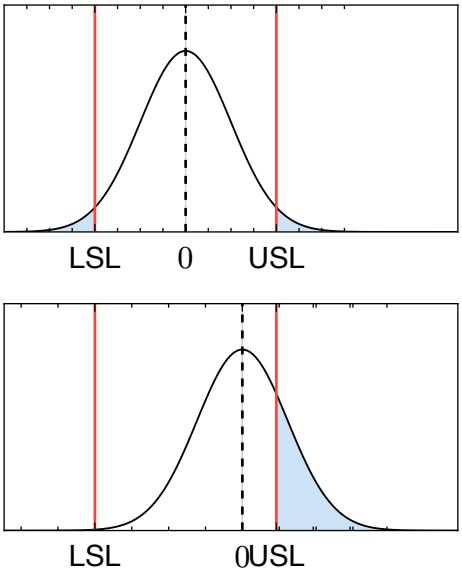


185. Process Capability

185.1. Cheatsheet

Index	Formula	Variation Used	Assumes Process Centering	Measures
C_p	$\frac{\text{USL} - \text{LSL}}{6\sigma_{\text{within}}}$	Short-term (σ_{within})	Yes	Pure “noise” under stable conditions (same machine, same operator, same material, no maintenance events)
C_{pk}	$\min\left(\frac{\text{USL} - \mu}{3\sigma_{\text{within}}}, \frac{\mu - \text{LSL}}{3\sigma_{\text{within}}}\right)$	Short-term (σ_{within})	No	Pure “noise” under stable conditions (same machine, same operator, same material, no maintenance events)
P_p	$\frac{\text{USL} - \text{LSL}}{6\sigma_{\text{overall}}}$	Long-term (σ_{overall})	Yes	Real-life variability from all causes (data across multiple shifts, operators, material batches, maintenance)
P_{pk}	$\min\left(\frac{\text{USL} - \mu}{3\sigma_{\text{overall}}}, \frac{\mu - \text{LSL}}{3\sigma_{\text{overall}}}\right)$	Long-term (σ_{overall})	No	Real-life variability from all causes (data across multiple shifts, operators, material batches, maintenance)
C_{pm}	$\frac{\text{USL} - \text{LSL}}{6\sqrt{\sigma_{\text{within}} + (\mu - T)^2}}$	• Short-term (σ_{within}) • Deviation from target (T)	No	Penalizes deviation from target (T); reflects capability under stable conditions when target matters

Centered v. Decentered Process



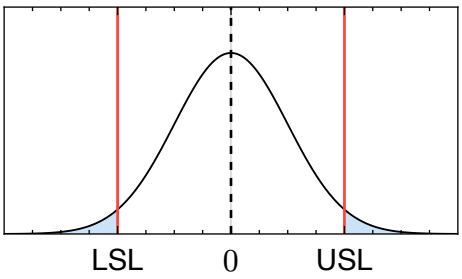
185.2. Cp

185.2.1. C_p (Process Capability Index)

Measure how well a process can produce outputs within specified limits

$$C_p = \frac{\text{USL} - \text{LSL}}{6\sigma}$$

Assumption: Process is **centered** within the specification limits



C_p Value	Meaning	Quality Impact
$C_p < 1$	Process variation exceeds specs	Defects likely, even if centered
$C_p = 1$	Process barely fits within specs	OK if perfectly centered
$C_p > 1$	Process well within specs	Robust, fewer defects
$C_p = 2$	Six Sigma level	World-class performance

Example

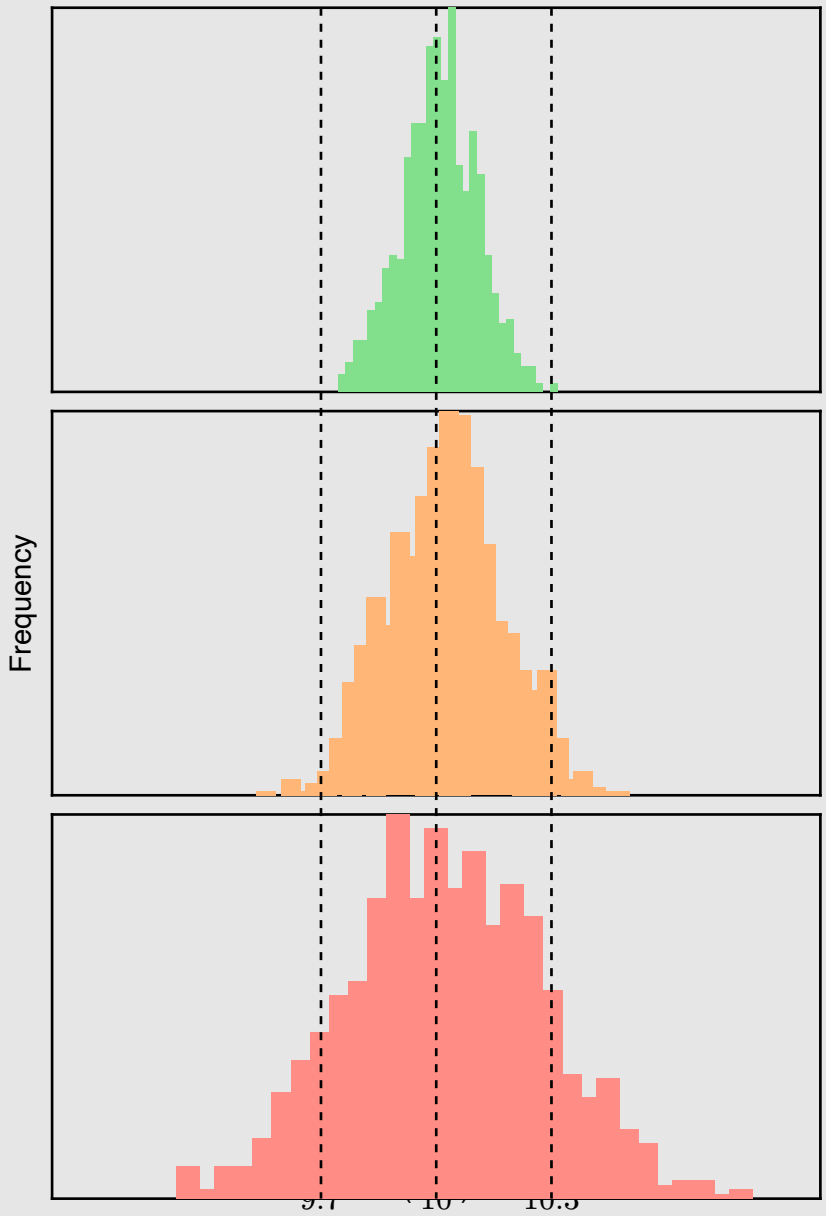
A manufacturing process produces metal rods with a target diameter of 10.0 mm. The specification limits are:

- Target: 10 mm
- Lower Specification Limit (LSL): 9.7 mm
- Upper Specification Limit (USL): 10.3 mm
- Specification Width (Tolerance): 0.6 mm

Let's analyze three different processes with varying levels of capability.

Process	Mean	Std Dev	C_p	Assessment
Process 1	9.998	0.099	1.009	Good
Process 2	10.001	0.147	0.682	Marginal
Process 3	9.994	0.255	0.393	Poor

Process 1	Process 2	Process 3	Six Sigma
$C_p = \frac{\text{USL} - \text{LSL}}{6\sigma}$	$C_p = \frac{\text{USL} - \text{LSL}}{6\sigma}$	$C_p = \frac{\text{USL} - \text{LSL}}{6\sigma}$	$C_p = \frac{\text{USL} - \text{LSL}}{6\sigma}$
$= \frac{10.3 - 9.7}{6 \times 0.099}$	$= \frac{10.3 - 9.7}{6 \times 0.147}$	$= \frac{10.3 - 9.7}{6 \times 0.255}$	$= \frac{10.3 - 9.7}{6 \times 0.05}$
$= \frac{0.6}{0.6}$	$= \frac{0.6}{0.9}$	$= \frac{0.6}{1.5}$	$= \frac{0.6}{0.30}$
$= 1.009$	$= 0.682$	$= 0.393$	$= 2$



Step 3: Interpretation

Process	C_p	Within Spec	Defective	DPMO
Process 1	1.009	96.64%	3.36%	33600
Process 2	0.682	84.13%	15.87%	159700
Process 3	0.393	50.00%	50.00%	500000
Six Sigma	2.0	99.9999998%	0.0000002%	0.002

Higher C_p means fewer defects and better process quality. and lower defects per million opportunities (DPMO)

c_p.py

```
cp_values = [2, 1, 0.1]

for cp in cp_values:
    z = 3 * cp  # spec limit in standard deviations
    prob_defective = 2 * (1 - norm.cdf(z))
    within_specs = 1 - prob_defective
    dpmo = prob_defective * 1_000_000

    print({
      "Cp": cp,
      "Sigma limit (±z)": z,
      "Percent within specs": within_specs * 100,
      "Percent defective": prob_defective * 100,
      "DPMO": dpmo
    })
```

Output:

```
[{'Cp': 2,
  'Sigma limit (±z)': 6,
  'Percent within specs': 99.99999980268245,
  'Percent defective': 1.9731754008489588e-07,
  'DPMO': 0.001973175400848959},
{'Cp': 1,
  'Sigma limit (±z)': 3,
  'Percent within specs': 99.73002039367398,
  'Percent defective': 0.2699796063260207,
  'DPMO': 2699.796063260207},
{'Cp': 0.1,
  'Sigma limit (±z)': 0.30000000000000004,
  'Percent within specs': 23.582284437790534,
  'Percent defective': 76.41771556220947,
  'DPMO': 764177.1556220946}]
```

185.3. Cpk

185.3.1. C_{pk} (Process Capability Index with Centering)

Minimum distance from mean to nearest spec limit divided by 3 standard deviations. Process capability considering centering and variation.

If process is centered:

$$C_{pk} = C_p = \frac{\text{USL} - \text{LSL}}{6\sigma}$$

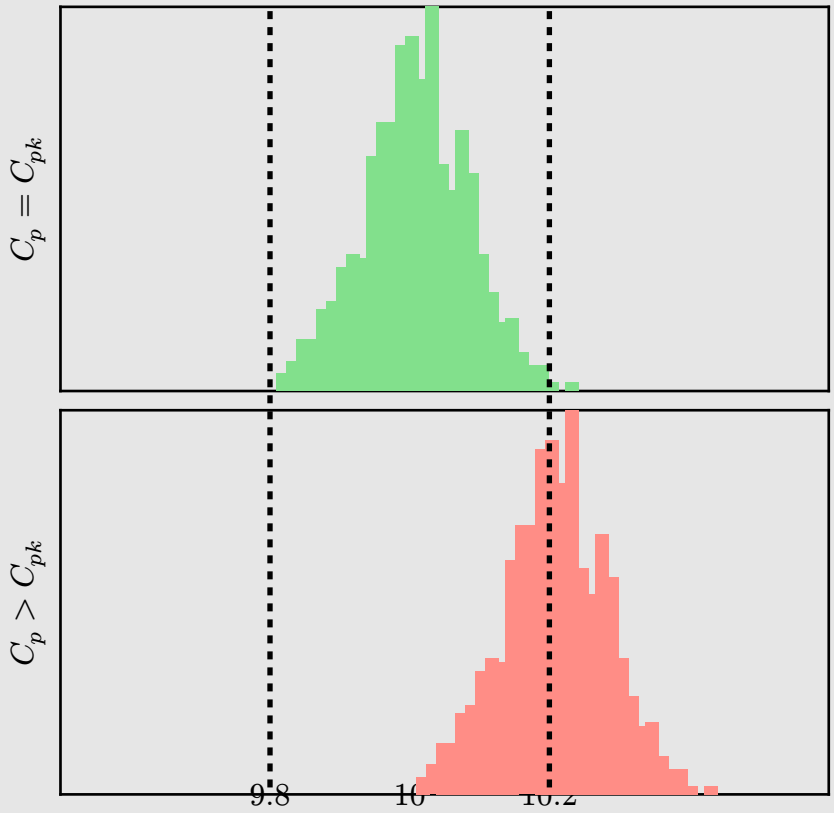
If process mean is off center, the smaller side of the process-to-limit ratio determines C_{pk}:

$$C_{pk} = \text{Min}\left(\frac{\text{USL} - \bar{x}}{3\sigma}, \frac{\bar{x} - \text{LSL}}{3\sigma}\right)$$

If $C_{pk} < C_p$, process is not centered

$C_{\{pk\}}$ Value	Meaning	Quality Impact
$C_{\{pk\}} < 1$	Process not capable, output falls outside specs	Many defects, poor performance
$C_{\{pk\}} = 1$	Process just meets specification limits	Acceptable only if well centered
$C_{\{pk\}} > 1$	Process is capable, centered or nearly so	Fewer defects, reliable quality
$C_{\{pk\}} = 2$	Excellent capability, Six Sigma level	Near-perfect quality

Example



Suppose a company manufactures metal rods, and the specification limits for the diameter of the rods are:

- Target: 10 mm
- Lower Specification Limit (**LSL**): 9.8 mm

- Upper Specification Limit (**USL**): 10.2 mm

The process has:

- A standard deviation σ of 0.05 mm.
- A process mean μ of 10.1 mm.

Step 1: Calculate the distance from the mean to the USL:

$$\frac{\text{USL} - \mu}{3\sigma} = \frac{10.2 \text{ mm} - 10.1 \text{ mm}}{3 \times 0.05 \text{ mm}} = \frac{0.1 \text{ mm}}{0.15 \text{ mm}} = 0.67$$

Step 2: Calculate the distance from the mean to the LSL:

$$\frac{\mu - \text{LSL}}{3\sigma} = \frac{10.1 \text{ mm} - 9.8 \text{ mm}}{3 \times 0.05 \text{ mm}} = \frac{0.3 \text{ mm}}{0.15 \text{ mm}} = 2.00$$

Step 3: Determine C_{pk}:

$$C_{pk} = \min(0.67, 2.00) = 0.67$$

Interpretation:

C_p = 1.33 means the process spread (6 σ) fits 1.33 times within the tolerance range (the distance between the Upper Specification Limit and Lower Specification Limit.

- C_p = 1.00: Process mean is exactly at the midpoint of the specification limits, and the process variation fits exactly within these limits. 99.73% of the output will be within specifications, indicating a capable process (3 sigma process).
- $C_p > 1.00$: The higher the C_{pk}, the more capable and stable the process is, meaning it can consistently produce parts within tolerance with minimal risk of defects.
- $C_p < 1$: The process mean is off-center or the variation is wider than the specification limits, or both. A significant portion of the output may fall outside the specification limits.

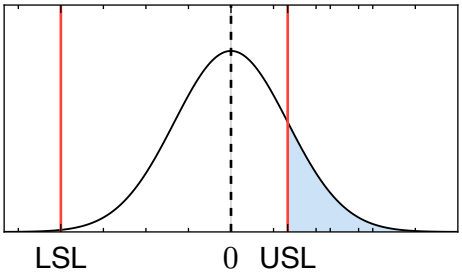
185.4. Cpm

185.4.1. C_{pm} (Taguchi Capability Index)

185.5. Pp

185.5.1. P_p (Process Performance Index)

$$P_p = \frac{\text{USL} - \text{LSL}}{6\sigma_{\text{overall}}}$$



Example

Suppose a company manufactures metal rods, and the specification limits for the diameter of the rods are:

- Upper Specification Limit (**USL**): 10.2 mm
- Lower Specification Limit (**LSL**): 9.8 mm
- Overall standard deviation (σ_{overall}): 0.06 mm

Step 1: Determine the Specification Width

The specification width is the difference between the USL and LSL.

$$\text{Specification Width} = \text{USL} - \text{LSL} = 10.2 \text{ mm} - 9.8 \text{ mm} = 0.4$$

Step 2: Calculate the Process Performance Index P_p

The formula for P_p is:

$$P_p = \frac{\text{Specification Width}}{6\sigma_{\text{overall}}} = \frac{\text{USL} - \text{LSL}}{6\sigma_{\text{overall}}}$$

Substitute the values:

$$P_p = \frac{0.4 \text{ mm}}{6 \times 0.06 \text{ mm}} = \frac{0.4 \text{ mm}}{0.36 \text{ mm}} = 1.11$$

Interpretation:

- A P_p of 1.11 indicates that the process performance, considering all sources of variation over time, is capable but less so than the potential capability indicated by C_p. The value being slightly above 1 suggests that the process can generally produce rods within specifications, but there might be more variability in the process compared to the short-term capability measured by C_p
- Decrease from the C_p value (1.33 to 1.11) reflects the impact of additional variability when evaluating the process over a longer time or under different conditions.

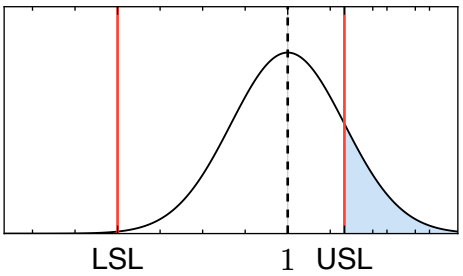
Use:

When you need to evaluate the **actual performance** of a process over a longer period, considering all sources of variation, including shifts, drifts, and other long-term factors.

185.6. Ppk

185.6.1. P_{pk} (Process Performance Index with Centering)

$$P_{pk} = \text{Min}\left(\frac{\text{USL} - \mu_{\text{overall}}}{3\sigma}, \frac{\mu_{\text{overall}} - \text{USL}}{3\sigma}\right)$$



Example

Suppose a company manufactures metal rods, and the specification limits for the diameter of the rods are:

- Upper Specification Limit (**USL**): 10.2 mm
- Lower Specification Limit (**LSL**): 9.8 mm
- Overall standard deviation (σ_{overall}): 0.06 mm
- Overall process mean (μ_{overall}): 10.1 mm

Step 1: Calculate the Distance from the Process Mean to the Specification Limits

Calculate the distance from the overall process mean to both the USL and LSL:

$$\text{USL} - \mu_{\text{overall}} = 10.2 \text{ mm} - 10.1 \text{ mm} = 0.1 \text{ mm}$$

$$\mu_{\text{overall}} - \text{LSL} = 10.1 \text{ mm} - 9.8 \text{ mm} = 0.3 \text{ mm}$$

Step 2: Calculate the Process Performance Index P_k

The formula for P_{pk} is:

$$P_{pk} = \text{Min}\left(\frac{\text{USL} - \mu_{\text{overall}}}{3\sigma}, \frac{\mu_{\text{overall}} - \text{USL}}{3\sigma}\right)$$

Substitute the values:

$$P_{pk} = \text{Min}\left(\frac{0.1 \text{ mm}}{3 \times 0.06 \text{ mm}}, \frac{0.3 \text{ mm}}{3 \times 0.06 \text{ mm}}\right)$$

$$P_{pk} = \text{Min}\left(\frac{0.1 \text{ mm}}{0.18 \text{ mm}}, \frac{0.3 \text{ mm}}{0.18 \text{ mm}}\right)$$

$$P_{pk} = \min(0.56, 1.67) = 0.56$$

Interpretation:

- A P_p of 1.11 indicates that the process performance, considering all sources of variation over time, is capable but less so than the potential capability indicated

by C_p. The value being slightly above 1 suggests that the process can generally produce rods within specifications, but there might be more variability in the process compared to the short-term capability measured by C_p
- Decrease from the C_p value (1.33 to 1.11) reflects the impact of additional variability when evaluating the process over a longer time or under different conditions.

Use:

When you need to evaluate the **actual performance** of a process over a longer period, considering all sources of variation, including shifts, drifts, and other long-term factors.

186. Causes of Variation

186.1. Common

Common Cause Variation: Normal, expected variation inherent to the process; predictable and stable.

Example

- Temperature
- Humidity
- Haterial properties

186.2. Special

Special Cause Variation: Abnormal, unexpected variation due to specific causes; unpredictable and requires immediate action.

External disruptions

Example

- Machine malfunction
- Faulty raw materials
- Staff shortages
- Weather events

187. Acceptance Sampling

188. Critical Path Method (CPM)

What's the shortest possible time to finish the whole project, and which tasks are "tight" (any delay on them delays the whole project)

Let $G = (V, E)$ be a DAG with a duration of $d(v) \geq 0$ on each node. And edge (u, v) means u must finish before v starts. $\text{pred}(v)$ and $\text{succ}(v)$ for in- and out-neighbors.

Forward Pass (one recurrence)

$$\text{EF}(v) = d(v) + \max_{u \in \text{ pred}(v)} \text{EF}(u), \qquad \text{ES}(v) = \text{EF}(v) - d(v)$$

with $\max \varnothing = 0$ (sources get $\text{ES} = 0$).

The makespan is $T = \max_v \text{EF}(v)$.

Backward Pass (dual recurrence)

$$\text{LF}(v) = \min_{w \in \text{ succ}(v)} \text{LS}(w), \qquad \text{LS}(v) = \text{LF}(v) - d(v)$$

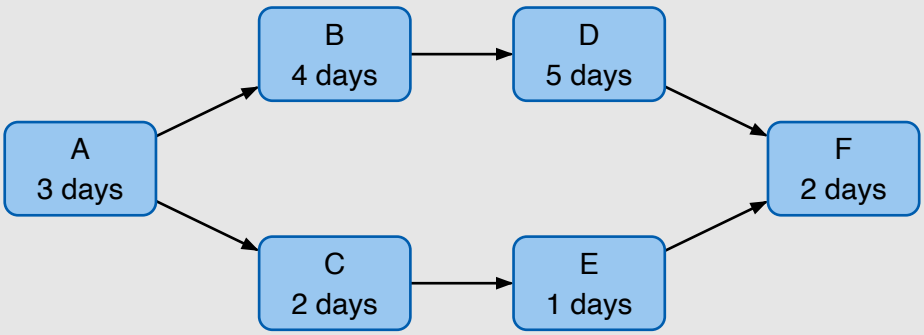
with $\min \varnothing = T$ (sinks get pinned to project end)

Clack & Critical Pass

$$\text{slack}(v) = \text{LS}(v) - \text{ES}(v) = \text{LF}(v) - \text{EF}(v)$$

$$\text{Critical path} = \{v : \text{slack}(v) = 0\}$$

Example



Let's compute. We start the project at time 0.

- A has no predecessors, so ES = 0, and EF = 0 + 3 = 3.
- B depends only on A, so ES = EF(A) = 3, and EF = 3 + 4 = 7.
- C also depends only on A, so ES = EF(A) = 3, and EF = 3 + 2 = 5.
- D depends only on B, so ES = EF(B) = 7, and EF = 7 + 5 = 12.
- E depends only on C, so ES = EF(C) = 5, and EF = 5 + 1 = 6.

189. Supply Chain

190. Planning

190.1. Aggregate Planning

A mid-term (3–18 month) production / staffing plan that matches **aggregated** output to **aggregated** demand, deciding levels of:

- Regular-time production
- Overtime / undertime
- Hiring / layoffs (or contract labor)
- Inventory carry-over
- Backorders / lost sales

Formulated as a linear program.

190.1.1. LP formulation

For periods $t = 1, ..., T$:

Variables:

- P_t: regular-time production
- O_t: overtime production
- H_t: hires this period
- F_t: fires this period
- W_t: workforce at end of period t
- I_t: inventory at end of period t
- B_t: backlog at end of period t

Costs per unit / hire / firing:

- c_P: regular-time labor / production cost
- c_O: overtime premium
- c_H: hiring cost
- c_F: firing / severance cost
- c_I: inventory holding cost
- c_B: backorder cost

Objective:

$$\min \sum_t (c_P P_t + c_O O_t + c_H H_t + c_F F_t + c_I I_t + c_B B_t)$$

Constraints (workforce balance, production capacity, inventory balance):

$$W_t = W_{t-1} + H_t - F_t \qquad \text{(workforce balance)}$$

$$P_t \le k W_t \qquad (\$k\$ = \text{units per worker per period})$$

$$O_t \le o_{\max} W_t \qquad \text{(overtime cap)}$$

$$I_t - B_t = I_{t-1} - B_{t-1} + P_t + O_t - D_t \qquad \text{(net inventory balance)}$$

$$I_t, B_t, P_t, O_t, H_t, F_t, W_t \ge 0$$

190.1.2. Three pure strategies

Strategy	Description	Best when
Chase	Match production to demand each period; vary workforce	Low inventory cost, high hire/fire flexibility, low workforce wages
Level	Constant production; let inventory + backorder absorb demand fluctuations	Stable workforce, low inventory cost, low backorder cost
Mixed	Combination of both, plus overtime / contract	Real-world; what the LP discovers automatically

Pure chase or pure level rarely optimal in practice — LP finds the right mixed strategy.

190.1.3. Example: monthly aggregate plan

12-month plan for a manufacturer. Demand peaks in November. Cost parameters set; LP gives:

- Hire 5 workers in August
- Produce overtime in October-November
- Carry inventory September-October to smooth November peak
- Lay off 3 workers in February
- Total cost: 1.2M

vs pure chase (high hiring/firing): 1.5M vs pure level (high inventory): 1.4M

190.1.4. Connection to MRP

Aggregate planning sets the **envelope** — workforce, total monthly output. The detail of **which specific products** gets handled by:

- **MPS** (Master Production Schedule) — disaggregates aggregate plan into product-week
- **MRP** (Material Requirements Planning) — explodes MPS into parts / materials

See MPS and MRP.

190.1.5. Limitations

- **Linear costs** assumed — real overtime / hiring may have step functions
- **Aggregate units** — assumes products are interchangeable, ignores changeover times
- **Single facility** — multi-plant versions exist but more complex
- **Deterministic demand** — stochastic versions = stochastic programming (see here)

190.1.6. See also

- **MPS** — disaggregation
- **MRP** — material planning
- **Rough-Cut Capacity Planning** — feasibility check
- **Linear Programming**

190.2. MPS

Master Production Schedule (MPS): the **disaggregated** version of aggregate planning. Specifies how many of **each individual product** to make in each **week** (or shorter bucket).

190.2.1. What it specifies

For each end-item i and time bucket t (typically weekly for 6–12 weeks ahead):

- **Planned production quantity** — how many units to start production of
- **Scheduled receipts** — orders already in process
- **Projected on-hand inventory** — running balance
- **Available-to-promise** (ATP) — quantity uncommitted, available for new orders

190.2.2. Time-phased records

A typical MPS row for product i:

Week	1	2	3	4	5	6	7
Forecast	100	120	110	100	130	120	110
Customer orders	80	95	40	10	0	0	0
Projected on-hand	50	50	70	120	70	100	100
MPS quantity	100	120	130	150	80	150	110
ATP	20	25	90	140	80	150	110

ATP at week t = production scheduled for that week minus already-committed customer orders.

190.2.3. How MPS relates to aggregate plan

The aggregate plan says "produce X units total in October." The MPS distributes X across:

- Specific products (model A, B, C)
- Specific weeks (week 1, 2, 3, 4)

Constraint: $\sum_i \text{MPS}_{i,t} \leq$ capacity granted by aggregate plan for period containing t.

190.2.4. Frozen / liquid / planning zones

The MPS is typically divided into three horizons:

- **Frozen zone** (week 1-2): no changes allowed; orders are firm, production committed
- **Slushy / firm zone** (week 3-4): changes require approval
- **Liquid / planning zone** (week 5-8+): can be re-planned freely as new demand information arrives

Frozen-zone duration matches **production lead time** — once you've started making it, you can't easily change.

190.2.5. MPS feasibility: Rough-Cut Capacity Planning (RCCP)

Before committing to an MPS, check that capacity (workforce, machines, materials) is sufficient. See RCCP.

190.2.6. Feeding MRP

The MPS is the **input** to MRP, which explodes each MPS quantity into component requirements via the bill of materials.

190.2.7. Common pitfalls

- **Schedule nervousness**: small forecast changes ripple through MPS → MRP → vendor orders. Solution: stable frozen zone + change-control rules
- **Optimistic MPS**: planning to produce more than capacity allows. Solution: RCCP early
- **Ignoring lead times**: scheduling production for week t without checking material arrives by then

190.2.8. See also

- **Aggregate Planning** — feeds MPS
- **MRP** — fed by MPS
- **RCCP** — feasibility check
- **DRP** — distribution-side counterpart

190.3. MRP

Material Requirements Planning (MRP): explodes the MPS into time-phased requirements for **every part / component / material** needed to make the planned products.

Foundation of every ERP system (SAP, Oracle, NetSuite, etc.).

190.3.1. Inputs

1. **MPS** — planned production quantities, time-phased
2. **Bill of materials (BOM)** — what components go into each product (and how many of each)
3. **Inventory file** — on-hand and scheduled receipts of every part
4. **Lead-time data** — manufacturing / procurement lead time for each part

190.3.2. BOM example

Product **Bicycle**:

- 2 × Wheel
    - 1 × Tire
    - 1 × Rim
    - 30 × Spoke
- 1 × Frame
    - 5 × Steel tube (cut from raw)
    - 4 × Bracket
- 1 × Handlebar

Two-level explosion: Bicycle → Wheel/Frame/Handlebar; then Wheel → Tire/Rim/Spoke; etc.

190.3.3. Three core steps per item

For each part, working **level-by-level from top to bottom** of the BOM:

1. **Gross requirements** — how many needed in each period (driven by parent's planned orders + parent's lead time)
2. **Net requirements** — gross minus available (on-hand + scheduled receipts)
3. **Planned orders** — when to **release** an order to cover net requirements, offset by lead time

```
For each part, time period t:
    Gross[t] = parent_demand[t]
    Net[t] = max(0, Gross[t] - OnHand[t-1] - SchedReceipts[t])
    OnHand[t] = OnHand[t-1] + SchedReceipts[t] - Gross[t] + PlannedReceipt[t]
    Planned release at t - lead_time = Net[t]  (lot-sized; see below)
```

190.3.4. Lot sizing

Plan orders in batches, not unit-by-unit:

- **Lot-for-lot (LFL)**: order exactly the net requirement each period — minimal inventory, max setups
- **Fixed quantity**: always order Q units (e.g., EOQ)
- **Periods of supply (POS)**: cover k periods of demand
- **Silver-Meal** / **Least Unit Cost** / **Part-Period Balancing** — heuristics
- **Wagner-Whitin** — DP-optimal lot-sizing

190.3.5. Time-phased record example

Part: Wheel. Lead time: 2 weeks. On-hand: 50. Lot size: 100.

Week	1	2	3	4	5	6
Gross requirements (from Bicycle)	200	240	220	200	260	240
Scheduled receipts	100	0	0	0	0	0
Projected on-hand	–50	60	–160	40	–220	40
Net requirements	0	0	160	0	220	0
Planned order receipt	0	0	200	0	300	0
Planned order release (lead time –2)	200	0	300	0	0	0

(Numbers illustrative; not perfectly consistent — purpose is to show the structure.)

190.3.6. MRP I → MRP II → ERP

- **MRP I** (1970s): material requirements only
- **MRP II** (1980s): adds capacity requirements, financial integration, scheduling
- **ERP** (1990s+): fully integrated business systems including HR, accounting, CRM

190.3.7. Limitations

- **Deterministic**: ignores demand and lead-time uncertainty (planners add safety stock manually)
- **Schedule nervousness**: small changes ripple
- **Push system**: drives production based on plan, not on-the-ground signal — compare with Kanban / pull
- **Lot sizing tension**: heuristics like EOQ optimize per part, missing cross-part synergies

190.3.8. See also

- **MPS** — input
- **DRP** — same logic, distribution side
- **RCCP** — capacity check
- **Wagner-Whitin** — lot-sizing

190.4. DRP

Distribution Requirements Planning (DRP): MRP applied to the **distribution side** of the supply chain. Plans replenishment of regional warehouses / DCs / retail stores from upstream sources.

Whereas MRP explodes finished-good demand into component requirements, DRP **aggregates** downstream demand into upstream-warehouse requirements.

190.4.1. Setup

Multi-echelon distribution network:

```
                Plant
                  |
            Central DC
              /   \
      Regional DC  Regional DC
       /   |   \      /   |   \
    Store Store Store Store Store Store
```

Each location has its own:

- Forecast (or downstream-driven demand)
- On-hand inventory
- Lead-time from supplier upstream
- Safety stock

DRP computes time-phased orders at each level so the right inventory arrives at the right place at the right time.

190.4.2. Pull-up logic

Each location computes:

$$\text{Gross need}_t = \text{Forecast}_t (\text{for stores; sum of downstream orders for DCs})$$

$$\text{Projected on-hand}_t = \text{Projected on-hand}_{t-1} + \text{Receipts}_t - \text{Gross need}_t$$

$$\text{Planned order release}_{t-L} = \text{Replenishment quantity if projected drops below safety stock}$$

The store's planned order becomes the regional DC's gross need; the regional DC's planned order becomes the central DC's gross need; etc.

190.4.3. Difference from MRP

	MRP	DRP
Direction	Top-down (BOM explosion)	Bottom-up (demand pull-up)
Goal	Plan production / materials	Plan distribution / replenishment
Bill structure	Bill of Materials (parts)	Bill of Distribution (network)
Demand source	MPS (planned), forecast	Customer demand (forecast or actual)
Time-phasing	Lead-time-offset for production	Lead-time-offset for shipping

Both use the same record structure (gross requirements, on-hand, planned orders) but with different drivers.

190.4.4. Why DRP matters

- **Visibility**: every echelon knows what's needed where, time-phased
- **Reduce stockouts** at downstream locations
- **Smooth upstream demand** (somewhat — bullwhip still bites)
- **Coordinate transportation**: bunch orders into fewer, larger shipments

190.4.5. Bullwhip implications

DRP can **amplify** the bullwhip effect: lot-sized replenishment + safety stock at each echelon → upstream sees orders far more variable than actual end-customer demand.

Mitigations:

- **Smaller lot sizes** (more frequent, smaller orders)
- **Information sharing** (each echelon sees true end-customer demand, not just downstream orders)
- **VMI** (vendor-managed inventory) — upstream supplier sees real downstream consumption

190.4.6. Connection to S&OP

DRP feeds into **Sales & Operations Planning**: balancing distribution-network requirements with production capability.

190.4.7. See also

- **MRP** — the production-side counterpart
- **MPS** — central production plan
- **Bullwhip Effect** — DRP can amplify it
- **Multi-Echelon Inventory**

190.5. RCCP

Rough-Cut Capacity Planning (RCCP): a **feasibility check** on the MPS. Estimates whether key resources (workforce, machines, materials) can support the planned production **before** committing to MRP.

190.5.1. Why RCCP

MPS plans **what to produce**. Without RCCP, the plan might be infeasible — exceed available capacity → MRP will issue impossible orders → cascading delays.

RCCP catches this at the strategic / tactical level **before** detailed MRP runs.

190.5.2. Bills of capacity / bills of resources

A **bill of capacity** — like a BOM but for capacity — specifies how much of each **critical resource** each end product consumes.

Example: Product **Bicycle**:

- 0.5 hours on Assembly line A
- 0.3 hours on Welding station
- 1.2 hours of skilled-labor

Plan to produce 1,000 bicycles in week 5:

- Need 500 hours of Line A
- Need 300 hours of Welding
- Need 1,200 hours of skilled labor

190.5.3. Capacity gap check

For each critical resource and each MPS week:

$$\text{Load}_{r,t} = \sum_i \text{MPS}_{i,t} \cdot \text{usage}_{i,r}$$

$$\text{Slack}_{r,t} = \text{Available}_{r,t} - \text{Load}_{r,t}$$

If slack is negative for any (resource, week), the MPS is infeasible.

190.5.4. Capacity vs Loading

	RCCP	**Detailed CRP (Capacity Requirements Planning)**
When	Before MRP	After MRP
Resources	Few critical (work centers, key labor)	All work centers
Granularity	Aggregate, weekly	Per operation, per machine, daily
Speed	Fast (minutes)	Slow (hours)
Action	Adjust MPS if infeasible	Adjust schedule, expedite, overtime

RCCP is the **coarse** feasibility check; CRP is the detailed scheduling that follows MRP.

190.5.5. When RCCP detects infeasibility

If MPS exceeds capacity:

1. **Reduce MPS** — produce less, accept stockouts or backorders
2. **Expand capacity** — overtime, hire temps, weekend shifts
3. **Outsource** — third-party manufacturing
4. **Shift production timing** — produce earlier (carry inventory) or later (backorder)
5. **Substitute products** — produce alternatives that use unused capacity

190.5.6. Limitations

- **Critical-resource focus** — ignores non-critical bottlenecks
- **Aggregate timing** — weekly buckets miss intra-week variations
- **Static** — doesn't model setup times / changeovers

RCCP is **sufficient** for strategic planning; **insufficient** for shop-floor execution.

190.5.7. See also

- **MPS** — what RCCP checks
- **MRP** — downstream of RCCP
- **Aggregate Planning** — upstream of RCCP

190.6. Silver-Meal

A lot-sizing heuristic for time-varying demand (Silver & Meal, 1973). Chooses order intervals to minimize **average cost per period** covered.

Easier than Wagner-Whitin (which is DP-optimal) and often close in quality.

190.6.1. Setup

Demand d_t over T periods. Setup cost K per order, holding cost h per unit per period.

190.6.2. Algorithm

Start with a candidate order in period t covering k periods of demand. Average cost per period:

$$C(k) = \frac{K + h\sum_{j=1}^{k-1} j d_{t+j}}{k}$$

(Setup once + holding cost for demand carried j periods.)

Extend k as long as $C(k)$ decreases. Stop at first k where $C(k+1) > C(k)$. Place an order in t to cover k periods. Start a new order in $t+k$. Repeat until all periods are covered.

190.6.3. Worked example

Demand: $d = (60, 100, 80, 50, 40, 70)$, $K = 100$, $h = 1$.

Start at period 1:

- $C(1) = \frac{100}{1} = 100$
- $C(2) = \frac{100+1\cdot 100}{2} = 100 \leftarrow$ tied

- $C(3) = \frac{100+100+2 \cdot 80}{3} = \frac{360}{3} = 120 \leftarrow$ rose, stop. Order in period 1 covers 2 periods (160 units).

Start at period 3:

- $C(1) = \frac{100}{1} = 100$
- $C(2) = \frac{100+50}{2} = 75 \leftarrow$ decreasing
- $C(3) = \frac{100+50+80}{3} = 76.67 \leftarrow$ rose. Order in period 3 covers 2 periods (130 units).

Start at period 5:

- $C(1) = 100$
- $C(2) = \frac{100+70}{2} = 85 \leftarrow$ decreasing
- ($t + 2$ doesn't exist, $T = 6$). Order in period 5 covers 2 periods (110 units).

Three orders: period 1 (160), period 3 (130), period 5 (110). Total cost computable.

190.6.4. Compared to alternatives

Heuristic	Quality vs WW optimum	Computational cost
Lot-for-lot	worst; high setup cost	trivial
POQ / fixed-quantity (EOQ)	moderate; ignores demand variation	trivial
Least Unit Cost	usually within 5-10%	$O(T^2)$
Silver-Meal	usually within 3-5%	$O(T^2)$
Part-Period Balancing	usually within 5%	$O(T^2)$
Wagner-Whitin	optimal	$O(T^2)$ DP

Silver-Meal is fast and good. Wagner-Whitin is exact and not much slower; usually preferred when implementable.

190.6.5. Limitations

- **Greedy** — no lookahead beyond current order
- **Can miss good solutions** by stopping too early
- **Failure mode**: when demand has long-running upticks, Silver-Meal may stop short of the truly optimal order span

190.6.6. See also

- **Least Unit Cost**
- **Part-Period Balancing**
- **Wagner-Whitin** — DP-optimal
- **MRP** — where lot-sizing fits

190.7. Least Unit Cost

A lot-sizing heuristic similar to Silver-Meal, but minimizing **cost per unit produced** instead of **cost per period covered**.

190.7.1. Algorithm

For an order starting in period t covering k periods:

$$C_{\text{LUC}}(k) = \frac{K + h\sum_{j=1}^{k-1} j d_{t+j}}{\sum_{j=0}^{k-1} d_{t+j}}$$

Numerator: total setup + holding cost. Denominator: total units in the order.

Extend k while $C_{\text{LUC}}(k)$ decreases. Stop when it increases. Order covers k periods.

190.7.2. Compared to Silver-Meal

- **Silver-Meal**: minimize cost per **time** period
- **LUC**: minimize cost per **unit**

When demand is uniform, the two give similar / identical answers. When demand varies, they diverge:

- **Spiky demand peaks**: LUC tends to order bigger batches (drives down per-unit cost via fixed-cost amortization)
- **Smoother demand**: Silver-Meal tends to order more frequently

Neither is strictly better. Both are heuristics, both within 5% of Wagner-Whitin optimum on typical instances.

190.7.3. Worked example

Same data as Silver-Meal: $d = (60, 100, 80, 50, 40, 70)$, $K = 100$, $h = 1$.

Start period 1:

- $C_{\text{LUC}}(1) = \frac{100}{60} = 1.67$
- $C_{\text{LUC}}(2) = \frac{100+100}{60+100} = \frac{200}{160} = 1.25 \leftarrow$ decreasing
- $C_{\text{LUC}}(3) = \frac{100+100+160}{60+100+80} = \frac{360}{240} = 1.50 \leftarrow$ rose, stop. Order 1 covers 2 periods.

(Same answer as Silver-Meal in this case.)

190.7.4. When LUC favors larger orders

If subsequent demand is high (say $d_{t+1} = 1000$), Silver-Meal's **per-period** cost increases fast (high holding cost on big inventory), but LUC's **per-unit** cost may still drop (fixed cost spread over more units).

So LUC tends to **bundle** into larger orders when downstream demand is high.

190.7.5. See also

- **Silver-Meal** — sister heuristic
- **Part-Period Balancing**
- **Wagner-Whitin** — optimal
- **EOQ** — stationary-demand analog

190.8. Part-Period Balancing

A lot-sizing heuristic (DeMatteis, 1968) based on the **EOQ balance condition**: at the EOQ optimum, ordering cost equals holding cost. Replicate this for time-varying demand.

190.8.1. EOQ balance recap

For stationary demand:

- Setup cost per cycle: K
- Holding cost per cycle: $h\frac{Q}{2} \cdot T_{\text{cycle}}$

EOQ optimality: these are equal. The Part-Period Balancing analog: equate setup cost to **part-periods** (cumulative holding cost).

190.8.2. Algorithm

Economic part-period (EPP):

$$\text{EPP} = \frac{K}{h}$$

— the number of unit-periods that would generate holding cost equal to one setup. (One unit held for one period contributes h of holding cost; total holding cost on EPP unit-periods is $h \cdot \text{EPP} = K$.)

For an order starting in period t covering k periods:

$$\text{Part-periods}(k) = \sum_{j=1}^{k-1} j d_{t+j}$$

Extend k until part-periods approaches but doesn't exceed EPP. Stop at the k where:

$$\text{Part-periods}(k+1) > \text{EPP} \quad \text{but} \quad \text{Part-periods}(k) \leq \text{EPP}$$

190.8.3. Worked example

$d = (60, 100, 80, 50, 40, 70)$, $K = 100$, $h = 1$. $\text{EPP} = \frac{100}{1} = 100$.

Start period 1:

- Part-periods after $k = 1$: 0
- After $k = 2$: $1 \cdot 100 = 100$ ← right at EPP, stop
- (If we went to $k = 3$: $100 + 2 \cdot 80 = 260$, way past.)

Order covers 2 periods.

Start period 3:

- Part-periods after $k = 2$: $1 \cdot 50 = 50$ — below EPP
- After $k = 3$: $50 + 2 \cdot 40 = 130$ — past EPP, stop

Take $k = 2$ (130 vs 50 — closer to 100? 130 vs 50: $|130 - 100| = 30$, $|50 - 100| = 50$. Take $k = 3$.). Note: rules differ; some variants pick "closest" rather than "last not exceeding".

Start period 6: only one period left. Order covers 1 period.

(Worked answer depends on tie-breaking rule.)

190.8.4. Variants

- **Standard PPB**: stop at first k where part-periods exceed EPP
- **Look-ahead PPB**: include the contribution of period $t + k$ before deciding
- **Modified PPB**: minimize $|\text{Part-periods} - \text{EPP}|$

190.8.5. Quality

Generally within 5–10% of Wagner-Whitin optimum. Slightly worse than Silver-Meal or LUC in benchmarks, but conceptually simpler — directly mimics EOQ's balance principle.

190.8.6. Why it sometimes wins

When demand patterns are **close to stationary**, PPB hugs the EOQ-like balance more tightly than the period-based or unit-based alternatives.

190.8.7. See also

- **Silver-Meal** — period-based
- **Least Unit Cost** — unit-based
- **Wagner-Whitin** — optimal
- **EOQ** — the balance principle PPB emulates

191. Manufacturing

191.1. Factory Physics

A framework (Hopp & Spearman, **Factory Physics** 1996) that applies **queueing theory** + **Little's Law** + variability analysis to manufacturing. Replaces folk wisdom with quantitative laws.

191.1.1. The three core insights

1. **Three buffers absorb variability** — capacity, inventory, time. You always pay one (or all) of these.
2. **VUT equation**: queue time = Variability × Utilization × Time. Reducing any of the three reduces flow time.
3. **Best/Worst/Practical-worst-case** performance curves: bound any manufacturing line's behavior given basic parameters.

191.1.2. Foundational quantities (a single station)

- r_b: **bottleneck rate** — the maximum sustainable throughput (per hour)
- T_0: **raw processing time** — sum of mean processing times across stages
- w: **WIP level** — work-in-process inventory
- $W_0 = r_b T_0$: **critical WIP** — minimum WIP to achieve r_b throughput

See Critical WIP.

191.1.3. Little's Law applied

For a production line at steady state:

$$\text{WIP} = \text{Throughput} \times \text{Cycle Time}$$

(See Little's Law.)

Given any two, the third is determined. Manufacturing trade-offs come down to managing these three quantities.

191.1.4. Best, worst, practical-worst-case

For any line:

- **Best-case TH**: $\min\left(\frac{w}{T_0}, r_b\right)$
- **Worst-case TH**: $\frac{1}{T_0}$
- **Practical-worst-case TH**: $\left(\frac{w}{W_0+w-1}\right)r_b$

Real lines lie between best (perfectly deterministic) and worst (max variability). PWC gives a "balanced random" baseline. See Best/Worst/PWC.

191.1.5. Levers for improvement

1. **Reduce variability**: smaller setups (SMED), reliable equipment (TPM), uniform service times
2. **Match capacity**: balance line (avoid bottleneck migration)
3. **Right-size WIP**: just above critical WIP — see CONWIP
4. **Pull system control**: kanban / CONWIP / drum-buffer-rope
5. **Eliminate non-value-added activities** (lean / Toyota Production System)

191.1.6. Compared to traditional cost accounting

Traditional manufacturing cost allocation overhead-rates "fully" — pretends each unit absorbs the same overhead. Factory physics shows this is misleading:

- Loading near $\rho = 1$ produces **enormous** queue and cycle-time penalty
- Marginal unit at high utilization costs **much more** (time, capital tied up) than marginal unit at low utilization
- **Capacity has nonlinear value** — overcapacity is good!

191.1.7. See also

- **Best / Worst / PWC**
- **Critical WIP**
- **CONWIP**
- **Kanban Sizing**
- **Takt Time**
- **Line Balancing**
- **Theory of Constraints**
- **Kingman VUT**
- **Little's Law**

191.2. Best Worst PWC

For any production line with bottleneck rate r_b, raw processing time T_0, and WIP level w, throughput is bounded by three curves (Hopp & Spearman). Real lines fall between these curves.

191.2.1. Three curves

Curve	Throughput TH(w)	Cycle Time CT(w)
Best case (deterministic)	$\min\left(\frac{w}{T_0}, r_b\right)$	$\max\left(T_0, \frac{w}{r_b}\right)$
Worst case (max variability)	$\frac{1}{T_0}$	wT_0
Practical-Worst-Case (balanced)	$\left(\frac{w}{W_0+w-1}\right)r_b$	$T_0 + \frac{w-1}{r_b}$

where $W_0 = r_b T_0$ is the **critical WIP**.

191.2.2. Interpretation

Best case: deterministic line, no variability. Throughput grows linearly until hitting bottleneck rate. Cycle time stays at T_0 until WIP exceeds capacity then grows linearly.

Worst case: pathological — all work at one station, batched. Throughput stays at $\frac{1}{T_0}$ no matter how much WIP. Useless line.

PWC: a "balanced random" baseline. Each station equally loaded with exponential variability. Real production lines should land near PWC if well-managed.

191.2.3. Critical WIP

$$W_0 = r_b T_0$$

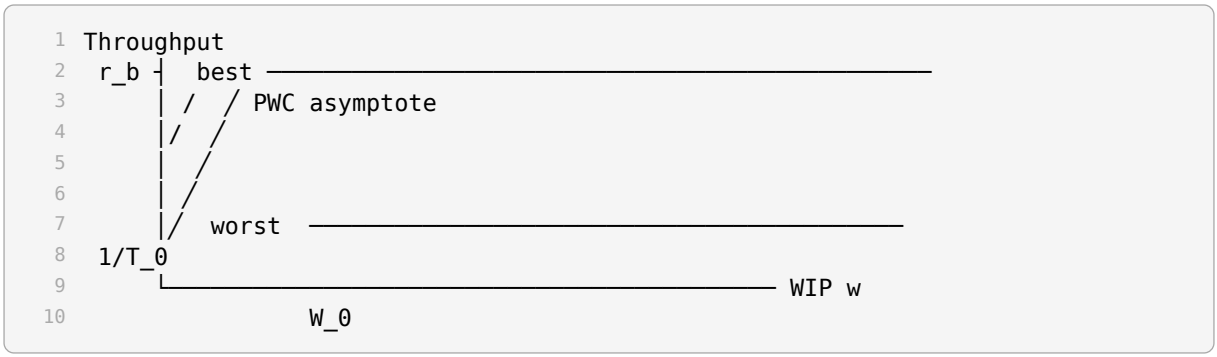
The WIP at which best-case throughput reaches r_b and best-case cycle time equals T_0. Above W_0:

- Best case: cycle time grows linearly with w, throughput stays at r_b
- PWC: cycle time grows linearly too, throughput approaches r_b asymptotically

191.2.4. Take-aways

- If your real line's throughput is well below the PWC curve, you have **more** variability than even PWC assumes — improve setups, breakdowns, batching
- If real cycle time is well above PWC, same — variability is the culprit
- Critical WIP W_0 is the sweet spot for CONWIP (constant WIP) control

191.2.5. Plot intuition



191.2.6. See also

- **Critical WIP**

- **Factory Physics overview**
- **VUT** — the underlying variability driver
- **CONWIP**

191.3. Critical WIP

$$W_0 = r_b T_0$$

The amount of work-in-process inventory at which a deterministic line just barely achieves its bottleneck throughput.

191.3.1. Why it's "critical"

- **Below** W_0: line is **starved** — bottleneck idle some of the time, throughput limited by WIP rather than capacity
- **At** W_0: bottleneck just fully utilized, cycle time = T_0
- **Above** W_0: bottleneck still at max throughput, but cycle time grows (excess WIP just sits in queues)

For **deterministic** lines, W_0 is the **minimum WIP** that achieves maximum throughput. Beyond W_0, you're tying up capital without throughput gain.

191.3.2. Real lines need more WIP

Variability makes the curves wider — to achieve close-to-r_b throughput on a **real** (variable) line, you need WIP somewhat above W_0. How much above depends on variability — see best/worst/PWC curves.

PWC throughput at w WIP:

$$\mathrm{TH}_{\mathrm{PWC}} = \left(\frac{w}{W_0 + w - 1}\right) r_b$$

For $w = 2W_0$: TH = $\left(2\frac{W_0}{2W_0+W_0-1}\right) r_b \approx \left(\frac{2}{3}\right) r_b$. So real lines need $w \gg W_0$ to approach r_b.

191.3.3. Usage

- **CONWIP control**: set WIP cap at $w^* > W_0$, chosen to hit target throughput
- **Lean assessment**: cycle time / T_0 ratio tells you how variability-burdened your line is
- **Capacity planning**: increasing r_b (de-bottlenecking) lowers $W_0 \rightarrow$ less WIP needed

191.3.4. Worked example

Line: bottleneck produces 100 units/hour ($r_b = 100$), raw processing time $T_0 = 3$ hours.

$$W_0 = 100 \cdot 3 = 300 \text{ units}$$

Need at least 300 units of WIP in the line to keep the bottleneck fed.

For a real (variable) line, set CONWIP cap at $w^* = 400 - 500$ units to hit close to r_b.

191.3.5. See also

- **Best/Worst/PWC**
- **CONWIP**

- **Factory Physics**
- **Little's Law**

191.4. CONWIP

Constant Work-In-Process: a production-control discipline that caps WIP at a constant level w. New work enters only when finished work exits — a **pull** system but with one global cap instead of per-station kanbans.

Developed at Northwestern (Spearman, Woodruff, Hopp 1990) as a generalization of kanban.

191.4.1. Mechanic

1. Set WIP cap w^* (typically $w^* > W_0$, the critical WIP)
2. Whenever a unit finishes and exits the line, **authorize** a new unit to enter
3. Total WIP stays constant at w^* always

Compare:

- **Push (MRP-driven)**: release per planned schedule, WIP fluctuates
- **Kanban**: each station holds local kanbans; WIP bounded but more complex
- **CONWIP**: one global cap; simpler, often as effective

191.4.2. Why CONWIP works

The PWC formula:

$$\mathrm{TH} = \left(\frac{w}{W_0 + w - 1}\right) r_b$$

shows throughput approaches r_b as w grows. Setting w generously above W_0 guarantees you're close to bottleneck-limited throughput, while keeping cycle time bounded.

vs **uncapped push**: WIP can grow without bound under variability, blowing up cycle time.

191.4.3. Setting the cap

Trade-off:

- **Too low w**: starvation, throughput below r_b
- **Too high w**: long cycle time, lots of inventory, late deliveries

Rule of thumb: $w^* = W_0 +$ buffer. Buffer sized to cover typical variability — start with $1.5 \cdot W_0$ and adjust based on observed throughput / cycle time.

191.4.4. Implementation

- **Cards / tokens**: physical or digital "authorizations" that flow with units
- **Heijunka box**: time-sliced authorization (with level scheduling)
- **MES system**: software cap enforced on releases

191.4.5. Comparison with Kanban

	Kanban	CONWIP
WIP control	Per-station local	Global single cap
Mix flexibility	Limited (per-part kanbans)	Higher (any part fills the cap)
Implementation	More complex	Simpler
Per-product visibility	Better	Aggregate only
Best for	Repetitive, low-mix	Higher-mix, more flow-flexible

CONWIP is simpler and adapts better to product mix; kanban gives tighter per-product control.

191.4.6. See also

- **Kanban Sizing**
- **Critical WIP**
- **Best/Worst/PWC**
- **Factory Physics**

191.5. Kanban Sizing

The classical Toyota Production System formula for sizing kanban (replenishment-authorization cards) between two production stages:

$$K = \left\lceil \frac{D \cdot \left(T_p + T_w\right) \cdot (1 + \alpha)}{C} \right\rceil$$

where:

- K: number of kanban cards needed
- D: demand rate (units per period)
- T_p: processing lead time (production time for one container)
- T_w: waiting lead time (queue, transport)
- α: safety factor (typically 0.05–0.30)
- C: container size (units per card)

191.5.1. Logic

Each kanban authorizes one container of parts. Total inventory in the loop = $K \cdot C$ units.

Required inventory must cover:

- Demand during the (processing + waiting) lead time: $D\left(T_p + T_w\right)$
- Plus safety: $(1 + \alpha)$ multiplier

Round up so K is an integer.

191.5.2. Example

Assembly line uses gear-boxes at $D = 100$/hour. Supplier has $T_p = 0.5$ hr processing, $T_w = 1$ hr waiting. Container holds $C = 10$ gear-boxes. Safety factor $\alpha = 0.2$.

$$K = \left\lceil \frac{100 \cdot 1.5 \cdot 1.2}{10} \right\rceil = \lceil 18 \rceil = 18 \text{ cards}$$

Total inventory loop: $18 \cdot 10 = 180$ gear-boxes.

191.5.3. Two-card kanban (Toyota)

Toyota's classic system uses **two** card types:

- **Production kanban** (P-kanban): authorizes the producer to make a container
- **Withdrawal kanban** (W-kanban): authorizes downstream to pull from producer's output buffer

P-kanban + W-kanban coordinate the pull. Modern simplified versions use just one card type.

191.5.4. Compared to CONWIP

	Kanban	CONWIP
Granularity	Per-part-type, per-stage	Global cap on all WIP
Setup	More cards to manage	Simpler
Mix flexibility	Lower (each card is specific)	Higher (any part can fill cap)
Best for	Repetitive, stable mix	Variable mix, fewer-part-types

191.5.5. When kanban sizing breaks down

The formula assumes:

- **Stationary demand** — varying demand requires resizing K periodically
- **Reliable supplier** — if upstream stations break down, fixed K won't compensate
- **Small mix** — separate kanbans per part-type only practical for moderate part counts
- **Short lead times** — long lead times require many cards, defeating leanness

In practice, K is recalibrated regularly (monthly / quarterly) based on observed performance.

191.5.6. Lean reduction strategies

The formula highlights levers:

- **Reduce $T_p + T_w$** (faster production, less queueing) → fewer cards needed
- **Smaller C** → more cards needed but smaller batches, more flexibility
- **Lower α** (reduce variability, more reliability) → fewer cards needed

Each improvement reduces inventory while maintaining service.

191.5.7. See also

- **CONWIP** — alternative discipline
- **Takt Time**
- **Heijunka**
- **Factory Physics**

191.6. Takt Time

The **pace of production** needed to match customer demand. From German **Takt** (beat, rhythm).

$$\text{Takt time} = \frac{\text{available production time}}{\text{customer demand}}$$

If customers want 80 units in an 8-hour shift, the line must produce one unit every $\frac{480}{80} = 6$ minutes.

191.6.1. Why takt time matters

Takt is the **target cycle time** for each station and the line overall:

- **Slower than takt**: line can't keep up — backlogs accumulate
- **Faster than takt**: line overproduces — inventory accumulates (waste)
- **At takt**: production synchronized with demand — lean ideal

In lean manufacturing, every station is designed to complete its work **in less than or equal to** the takt time, with engineered slack absorbed by Heijunka boxes / standard WIP.

191.6.2. Calculating takt

Available time: net production time available, after deducting:

- Breaks
- Meetings
- Changeovers (or carefully scheduled outside production)
- Planned maintenance

Customer demand: actual delivery requirement, including:

- Customer orders
- Forecast adjustments
- Safety / replenishment stock targets

191.6.3. Worked example

Factory:

- Two shifts, 8 hours each, 1 hour total breaks per shift
- Net available time: $2 \cdot 7 = 14$ hours/day $= 840$ minutes/day
- Customer demand: 200 units/day

$$\text{Takt time} = \frac{840}{200} = 4.2 \text{ minutes per unit}$$

So each station must complete its task in 4.2 minutes or less. The overall line cadence is one unit out every 4.2 minutes.

191.6.4. Takt time vs cycle time

- **Takt time**: external (driven by demand) — target cadence
- **Cycle time**: internal (driven by process) — actual time per unit

Goal: $\text{cycle time} \leq \text{takt time}$. When equality holds, line runs perfectly **to demand** — no over- or under-production.

191.6.5. Takt time vs throughput

Takt time time **between** units leaving the line

$$\text{Throughput} \quad \text{units per unit time} = \frac{1}{\text{takt}}$$

Inverses. Both convey same information.

191.6.6. Heijunka — level loading

Demand is rarely flat — peaks Monday, valleys Friday, etc. **Heijunka** (here) smooths production by aggregating demand and producing in a level cadence. Takt is then calculated from **average** demand, not peak.

191.6.7. Limitations

- **Volatile demand** — takt changes; need flexibility to adjust
- **Mix complexity** — multiple products with different cycle times need EPEI or sequenced production
- **Setup-heavy lines** — takt time may be longer than nominal cycle time once setups are included

191.6.8. See also

- **Line Balancing** — designing stations to meet takt
- **Heijunka** — level loading
- **Kanban Sizing** — pull control matching takt
- **Little's Law**

191.7. Line Balancing

The task of allocating work to stations on a production line so each station's work time is below the takt time, minimizing idle time and station count.

191.7.1. Setup

- A product is assembled via n **tasks** with processing times $t_1, ..., t_n$
- Tasks have **precedence constraints**: task i must finish before task j starts
- Tasks are grouped into **stations**; each station's total time $\leq \text{takt}$

Goals (often trade-offs):

- **Minimize number of stations** (capital cost)
- **Minimize cycle time** (= maximum station time)
- **Smooth station loads** (workforce equity)

191.7.2. Theoretical bounds

Minimum stations:

$$N^* \geq \left\lceil \frac{\sum_i t_i}{\text{takt}} \right\rceil$$

(Total work ÷ takt time, rounded up.)

Minimum cycle time: $\max_i t_i$ (longest indivisible task sets a floor).

191.7.3. Balance efficiency

$$\text{Balance efficiency} = \frac{\sum_i t_i}{N \cdot \text{cycle time}}$$

100% means every station fully loaded — no idle time. Lower efficiency means wasted capacity.

191.7.4. Heuristics

The problem (Assembly Line Balancing Problem, ALBP) is NP-hard. Common heuristics:

Longest Task Time (LTT): pick assignable tasks (precedences satisfied, fits in remaining station time) in **decreasing order** of t_i.

Ranked Positional Weight (RPW, Helgeson-Birnie 1961):
1. Compute each task's **positional weight** = t_i + sum of t_j for all successors j
2. Assign tasks in decreasing positional weight, respecting precedences

COMSOAL (Arcus 1966): Random assignment with backtracking. Multi-start.

191.7.5. Worked example

10 tasks with times $(5, 3, 4, 6, 4, 5, 4, 3, 5, 4)$, total $= 43$ minutes. Takt time $= 8$ minutes.

Minimum stations: $\lceil \frac{43}{8} \rceil = 6$.

LTT heuristic with simple precedence chain: place tasks into stations one at a time, picking the longest available task that fits.

Result might give 6 or 7 stations, balance efficiency 89%–100%.

191.7.6. Mixed-model line balancing

Real lines produce **multiple products** with different task sets. **Mixed-model** balancing:

- Define product mix (proportion of each)
- Compute **expected** task times weighted by mix
- Balance using weighted times
- Sequence products through stations to avoid bottleneck overload

191.7.7. Single-product vs U-shaped lines

Layout	Pros	Cons
Straight line	simple, fast	workers can't help each other
U-shaped	workers reach multiple stations, can help bottleneck	more complex, requires multi-skilled workers
Cell	flexible, small batches	coordinating multiple cells

U-shaped (cellular) layout is the lean ideal — natural for CONWIP / kanban control.

191.7.8. Variability and balance

Even a perfectly balanced **deterministic** line behaves much worse than the formula suggests when **variability** is added (per VUT). Real balancing requires both:

- **Mean balance**: station load averages match
- **Variance balance**: station variability matches

Otherwise the high-variance station becomes a “virtual bottleneck” that drives queueing throughout.

191.7.9. See also

- **Takt Time** — the constraint
- **Best/Worst/PWC**
- **Factory Physics**
- **CONWIP**

191.8. Theory of Constraints

Theory of Constraints (TOC, Goldratt 1984): a management philosophy that says system performance is determined by the **bottleneck** (the constraint). Identify the bottleneck, exploit it, subordinate everything else.

191.8.1. The five focusing steps

1. **Identify** the constraint — the bottleneck resource limiting system throughput
2. **Exploit** — maximize utilization of the bottleneck (no idle time, perfect quality, no setups)
3. **Subordinate** — every other resource serves the bottleneck (don’t push more than the bottleneck can absorb)
4. **Elevate** — invest to increase bottleneck capacity (add equipment, overtime, etc.)
5. **Return to step 1** — when the bottleneck moves elsewhere, repeat

191.8.2. Throughput accounting

TOC replaces traditional cost accounting (which over-emphasizes unit costs) with three measures:

- **Throughput** (T): sales revenue minus truly variable costs
- **Investment / Inventory** (I): money tied up in the system
- **Operating Expense** (OE): money to convert inventory into throughput

Net profit: $T - OE$

Return on investment: $\frac{T-OE}{I}$

Goal: increase T first, decrease I second, decrease OE third. Traditional accounting often prioritizes OE reduction, which can hurt T if you cut the bottleneck.

191.8.3. Product mix problem (with constraint)

Given multiple products competing for a single bottleneck, maximize total throughput:

$$\max \sum_i T_i x_i \qquad \text{s.t.} \sum_i b_i x_i \leq \text{bottleneck capacity}$$

where b_i is bottleneck-minutes per unit of product i. Optimal: produce in decreasing order of $\frac{T_i}{b_i}$ (**throughput per bottleneck minute**).

This is a **single-constraint LP** — explicit solution by ratio ranking.

191.8.4. Drum-Buffer-Rope (DBR) scheduling

A scheduling discipline derived from TOC:

- **Drum**: the bottleneck sets the pace
- **Buffer**: small protective inventory in front of the bottleneck — never let it starve
- **Rope**: communicates bottleneck demand backward to release new work into the system (a "rope" tying the entry point to the bottleneck)

DBR is essentially CONWIP but with the WIP cap chosen to keep just the **right amount** of buffer in front of the bottleneck.

191.8.5. Compared to Lean

	TOC	Lean / Toyota
Focus	Bottleneck — what limits throughput	Waste — what doesn't add value
Inventory at bottleneck	Buffer (protective)	Reduce (Just-in-Time)
Improvement target	Throughput per bottleneck-minute	Cycle time, waste elimination
Best for	Complex job shops, bottleneck-driven systems	Repetitive manufacturing, low variability

Many production systems benefit from **both** — Lean to reduce variability and waste, TOC to manage the residual bottleneck. The two are complementary, not competing.

191.8.6. See also

- **Factory Physics** — quantitative bottleneck analysis
- **CONWIP** — DBR's modern equivalent
- **Line Balancing**
- **Best/Worst/PWC**

191.9. Learning Curves

The empirical observation that **cost or time per unit decreases by a constant percentage** every time cumulative production doubles. Discovered by T. P. Wright (1936) studying aircraft production at Curtiss-Wright.

191.9.1. Wright's formula

Time (or cost) for the n-th unit:

$$T_n = T_1 n^{-b}$$

where:

- T_1: time for the first unit
- b: learning exponent
- n: cumulative unit number

The **learning rate** L — the proportion of time the $2n$-th unit takes relative to the n-th:

$$L = \frac{T_{2n}}{T_n} = \frac{(2n)^{-b}}{n^{-b}} = 2^{-b}$$

So $b = -\log_2(L) = -\ln \frac{L}{\ln} 2$.

191.9.2. Common learning rates

Learning rate L	Exponent b	Example industry
80%	0.322	Aircraft assembly (Wright's original)
85%	0.234	Automotive
90%	0.152	Electronics, software
95%	0.074	Highly automated; little learning
70%	0.515	Complex assembly, lots of skill

Lower $L \to$ faster learning. $L = 100\%$ means no learning.

191.9.3. Crawford's unit model vs cumulative average

Two conventions:

Wright (unit model): T_n is the time for the **n-th unit specifically**.

Crawford (cumulative average): $|T|_n$ is the **average** time across all units 1 to n. Same L, but different curves and different interpretations.

	Wright unit	**Crawford cumulative average**
Formula	$T_n = T_1 n^{-b}$	$\|T\|_n = T_1 n^{-b}$
Interpretation	time of the n-th unit	average time over first n units

Crawford's average always overstates per-unit improvement vs Wright (averaging includes early high-time units). Industries vary in which they use.

191.9.4. Cumulative cost

Total time / cost for the first n units:

$$C_n = \sum_{k=1}^{n} T_1 k^{-b} \approx T_1 \cdot \frac{n^{1-b}}{1-b} \quad (\text{continuous approximation for large } n)$$

For Crawford model: $C_n = n|T|_n = T_1 n^{1-b}$.

191.9.5. Strategic implications (BCG, 1960s)

Boston Consulting Group famously generalized learning curves to **experience curves** — all costs (not just labor) decline with cumulative volume.

Strategic implications:

- **Cost leadership through scale**: the highest-volume producer has the lowest unit cost
- **Predatory pricing**: price below current cost to grab share, ride the curve to profitability

- **First-mover advantage**: head start on the curve is hard to overtake

Famous critiques (Henderson, BCG): treat learning as a **strategic asset** and a **barrier to entry**.

191.9.6. Limitations

- **Empirical, not causal**: explains pattern, doesn't explain **why** learning happens
- **Discontinuities**: new product / process resets the curve
- **Diminishing returns**: long-run learning saturates
- **Stanford-B / DeJong adjustments**: include an "incompressibility factor" — costs can't fall below some floor

DeJong's model:

$$T_n = T_1\left(M + (1 - M)n^{-b}\right)$$

where $M \in [0, 1]$ is the incompressibility floor.

191.9.7. See also

- **Factory Physics**
- **Takt Time** — performance target as learning advances

191.10. Heijunka

Heijunka (平準化, "leveling"): a Toyota Production System technique for **smoothing** production by mixing products in a level cadence rather than running batches.

Heijunka makes takt time sustainable by absorbing demand fluctuations **inside** the production system.

191.10.1. Volume leveling vs mix leveling

Volume leveling: produce roughly constant total output each period (week, day, shift). Inventory absorbs demand variability over time.

Mix leveling: produce a **mix** of products each period, in **repeating sequence**, rather than running large batches of one product at a time.

Example: producing A, B, C in ratio 4:2:1 per day.

Without heijunka	With heijunka
AAAA AAAA BB BB C	A B A A C A B A
Long batches; A inventory grows then drains	Steady draw on all parts; minimal inventory swings

191.10.2. EPEI (Every Part Every Interval)

A key heijunka metric. EPEI = how often the line produces each product.

If n products are scheduled to run, total setup time is $\sum_i s_i$, and available time per interval is T:

$$\text{EPEI} = T - \sum_i \frac{s_i}{\text{production rate}}$$

Smaller EPEI = more frequent mix changes = more flexibility, but more setups.

Reducing setup times (SMED — Single-Minute Exchange of Dies) is the lever for reducing EPEI.

191.10.3. Heijunka box (load-leveling box)

A physical (or virtual) board that schedules production:

```
Time slot  ──→
Product
A           [kanban][kanban][kanban][kanban]
B           [kanban]──────[kanban]──────
C           [kanban]────────────────────
```

Pull cards from left to right in each slot — produce the indicated product. Maintains the mix across time.

191.10.4. Benefits

- **Smooth upstream demand** — suppliers see steady draws on each part
- **Low inventory swings** — no boom-bust pattern from batch runs
- **Customer-aligned** — production cadence tracks demand
- **Enables JIT** — pull system needs level loading to function

191.10.5. Costs

- **More setups** — each mix change is a changeover (SMED becomes critical)
- **Less efficient** per setup vs long batches
- **Worker training** — operators must adjust between products frequently
- **Tooling cost** — quick-change fixtures, modular setups

191.10.6. When heijunka is harder

- **High product variety** with infrequent demand (long-tail SKUs)
- **Long setup times** not yet reduced via SMED
- **Custom / engineered-to-order** — no repeating sequence possible
- **Volatile demand mix** — sequencing must adapt frequently

In those cases, partial heijunka (level the **high-volume** products, run rest in batches) is a pragmatic compromise.

191.10.7. Implementation steps

1. Calculate average demand per product per period
2. Reduce setup times via SMED
3. Establish a **pitch** (small repeating time slot)
4. Sequence products in the heijunka box per the demand ratio
5. Pull production based on the box; replenish customer pulls

191.10.8. See also

- **Takt Time**
- **Kanban Sizing**
- **Factory Physics**

192. Bullwhip Effect

The phenomenon where **demand variability amplifies up the supply chain**. Small fluctuations at the retail level cause progressively larger swings at the distributor, manufacturer, and raw-material supplier — like a bullwhip cracked at the handle.

Quantitatively measured by the **bullwhip ratio**:

$$\mathrm{BW}_{\text{stage}} = \frac{\mathrm{Var}(\text{orders out of stage})}{\mathrm{Var}(\text{demand into stage})}$$

If $\mathrm{BW} > 1$: variability is amplified. If $\mathrm{BW} = 1$: variability is preserved. **Real supply chains exhibit ratios of 2–5 per stage, compounding multiplicatively as you move upstream.**

192.1. Why it matters

Consequences of bullwhip:

- **Excess inventory** upstream (forced to absorb spikes).
- **Stockouts** and **firefighting** upstream.
- **Capacity over- and under-utilization** (production swings).
- **Higher logistics costs** (expedited shipping during ramp-ups).
- **Worse forecasts** — demand signals are corrupted by bullwhip itself.

Most damaging in industries with long lead times (electronics, pharmaceuticals, automotive) where amplified variance forces high safety stocks at every stage.

192.2. The four classical causes (Lee, Padmanabhan, Whang 1997)

192.2.1. 1. Demand-signal processing

Each stage forecasts based on **orders received**, not **true downstream demand**. When the retailer's demand spikes once, the retailer increases its safety stock — placing a **larger** order to the distributor. The distributor sees a spike and overreacts: even larger order to the manufacturer. The signal is amplified at each step.

Math: if each stage uses simple exponential smoothing or moving averages on **orders** (not demand), they bake the bullwhip from the previous stage into their next forecast — a positive feedback loop.

192.2.2. 2. Order batching

Retailers don't reorder every day — they batch orders into weekly or monthly shipments (driven by EOQ-style fixed costs, transportation economics, or supplier minimum order quantities).

The distributor sees a long pattern of **zero** orders followed by a single **large** order, vs the retailer's continuous demand. **Variance of batched orders >> variance of underlying demand.**

If batch size is Q and underlying demand is smooth: order variance scales with Q (lumpy spikes); demand variance is small. Bullwhip ratio $\approx \frac{Q}{Q/N} = N$ where N is the smoothing factor — easily $10\times$ amplification.

192.2.3. 3. Price fluctuations and forward buying

Promotions, volume discounts, end-of-quarter pushes — all incentivize buyers to **forward-buy** (purchase ahead of need to capture the deal). Order pattern: huge spikes at promotion times, troughs in between.

The retailer's actual sell-through is smooth; their **purchase** pattern is spiky $\rightarrow$ upstream sees the spiky pattern, not the smooth demand. Pure artifact of pricing strategy.

192.2.4. 4. Rationing and shortage gaming

When supply is constrained, the supplier rations: each customer gets a fraction of what they ordered. Rational customer response: **over-order** (order $1/r$ × what you actually need, expecting fraction r to be filled).

When supply normalizes, the inflated orders cancel or get returned, leaving the supplier with phantom demand signals. The 1990s computer-component shortages were full of this.

192.3. Mitigation strategies

Cause	Mitigation
Demand-signal processing	Share **true** downstream demand data with all stages (POS data, EDI, VMI). Each stage forecasts from real demand, not corrupted upstream orders.
Order batching	Reduce setup costs (lean / SMED) so Q shrinks. Coordinate cycle times across stages. Cross-docking. Frequent small deliveries.
Price fluctuations	Everyday-low-pricing (EDLP) instead of promotional pricing. Volume contracts at flat prices. Reduce stockpiling incentives.
Rationing / shortage gaming	Allocate based on past demand history, not current orders (Cisco's 1990s policy). Reduce panic by transparent communication of capacity.

The most effective single intervention: **information sharing**. POS-level visibility flowing upstream eliminates 60-80% of bullwhip in most studies.

192.4. The beer game

The MIT "beer distribution game" is a 4-stage simulation (retailer, wholesaler, distributor, factory) where players try to manage inventory and orders. Even with simple smooth demand, bullwhip emerges naturally — players aren't aware of upstream impacts and overreact based on local information.

Lessons consistently observed:

- Players over-order during ramps.
- Inventory swings grow upstream.
- Total system cost is much higher than optimal.
- Sharing demand information dramatically improves performance.

Example: Bullwhip from order batching

Given (4-stage chain: customer → retailer → wholesaler → factory):

- True customer demand: Poisson, mean 100 units / day, std $\sqrt{100} = 10$.
- Retailer batches orders: places one order per **month** (30 days), order = 30 days of demand $\approx$ 3000 units.
- Wholesaler batches similarly to factory.

Stage variance comparison

- **Customer demand** (per day): mean 100, std 10. Daily variance 100.
- **Retailer's order pattern** (per day): zero for 29 days, 3000 once on day 30. Mean 100/day. **Variance much higher** — the order is lumpy. Approximately: variance of a Poisson-spike pattern $\approx \frac{E[\text{spike}]^2}{30} \approx \frac{3000^2}{30} = 300\{,\}000$.
- **Bullwhip ratio at retailer**: $300\{,\}\frac{000}{100} = 3000$.

Three orders of magnitude amplification **just from monthly batching**.

Cascading

Wholesaler sees retailer's lumpy pattern as **its** demand. Wholesaler batches too — say weekly to factory. Each lump of retailer order triggers a wholesaler order; wholesaler may also batch and order in larger lumps.

By the time the factory sees its order signal, it's variance-amplified far beyond the original Poisson noise at the customer end.

Mitigation

If the wholesaler can see **daily customer demand** (POS data shared upstream): wholesaler forecasts from the smooth Poisson, not from the retailer's lumpy orders. Production at the factory becomes smooth. Inventory across the chain falls. **Same physical products and lead times — just less amplified noise.**

This is the central insight of supply chain coordination: **bullwhip is a coordination failure, not a physical constraint**. Information sharing fixes it cheaply.

193. Classification

193.1. ABC

Classify items by **annual consumption value** (annual demand × unit cost). The Pareto principle says a small fraction of items dominates total value — so spend management effort proportionally.

193.1.1. The Pareto split

- **A items**: top 70–80% of total value, typically 10–20% of SKUs.
- **B items**: next 15–25% of total value, 30% of SKUs.
- **C items**: remaining 5–10% of total value, 50–70% of SKUs.

Exact thresholds vary by industry — pick what makes sense for your inventory.

193.1.2. Why classify

A items deserve tight control: precise demand forecasts, frequent reviews, low safety stock margin, supplier negotiations. Mistakes on A items have outsized financial impact.

C items deserve light control: simple rules (two-bin, periodic review), generous safety stock, automated reorder. Time spent fine-tuning C items earns very little.

B items get medium attention.

193.1.3. Procedure

1. Compute each item's **annual consumption value** = annual demand × unit cost.
2. Sort items by consumption value, descending.
3. Compute **cumulative percentage** of total value down the list.
4. Cut at the chosen thresholds (e.g., 80% / 95% / 100%).

193.1.4. Differential treatment summary

Class	% of value	% of SKUs	Inventory policy
A	70-80%	10-20%	Tight: (s, S) or (Q, r) with low safety stock; weekly review; supplier integration
B	15-25%	30%	Standard: (R, S) with monthly review; modest safety stock
C	5-10%	50-70%	Loose: two-bin or fixed periodic order; generous safety stock

Example

Given: 6 SKUs in a small electronics warehouse.

Item	Annual demand	Unit cost	Annual value
Laptop charger	240	$50	$12,000
Keyboard	360	$30	$10,800
Gaming chair	60	$200	$12,000
Cable	600	$5	$3,000
Sticker pack	1,200	$1	$1,200
Phone case	200	$5	$1,000

Step 1 — sort by annual value (descending)

Item	Annual value	% of total	Cumulative %
Laptop charger	$12,000	30.0%	30.0%
Gaming chair	$12,000	30.0%	60.0%
Keyboard	$10,800	27.0%	87.0%
Cable	$3,000	7.5%	94.5%
Sticker pack	$1,200	3.0%	97.5%
Phone case	$1,000	2.5%	100.0%

Total: $40,000.

Step 2 — cut at thresholds

Using thresholds **80% / 95% / 100%**:

- **A** (cumulative ≤ 80%): Laptop charger, Gaming chair, Keyboard **(3 items, 87% of value, but stop at 80% threshold)** — tighter rule: include the keyboard since it pushes the cumulative just past 80%.
- **B** (next, ≤ 95%): Cable
- **C** (rest): Sticker pack, Phone case

Step 3 — interpret

3 SKUs (50% of item count) carry 87% of inventory value. These are the items where forecasting accuracy and inventory-management precision pay off.

The bottom 3 SKUs (50% of items) carry just 6.5% of value — automate, two-bin, don't waste planning effort.

Why two ties at $12,000?

Laptop charger and gaming chair have **the same total value** but very different demand profiles (240 vs 60 units/year). ABC alone doesn't see this — both look identical. Adding **XYZ analysis** (by demand variability) separates them; combining the two gives the **ABC-XYZ matrix**.

193.2. XYZ

Classify items by **demand variability**, measured via the **coefficient of variation** (CV) of demand over time:

$$\mathrm{CV} = \frac{\sigma}{\mu}$$

where μ is mean periodic demand and σ is its standard deviation.

193.2.1. The variability split

- **X items**: low CV — predictable demand. Forecast easily, lean inventory.
- **Y items**: moderate CV — somewhat variable, perhaps with seasonality or trend.
- **Z items**: high CV — volatile, sporadic, or irregular.

Typical thresholds (industry-dependent):

Class	CV range	Demand pattern
X	< 0.50	Smooth, predictable. Standard normal-distribution methods work.
Y	0.50 - 1.00	Variable but trackable. Use seasonality models, careful safety stock.
Z	> 1.00	Sporadic / lumpy. Traditional forecasting struggles; use intermittent-demand methods (Croston's, etc.) or treat as project-driven.

193.2.2. Why classify by variability

ABC tells you **which items to focus on**; XYZ tells you **how predictable each item is**. Both matter for inventory policy:

- **X items**: tight forecasting works → lean inventory, low safety stock margin.
- **Y items**: forecast error grows → higher safety stock; may benefit from time-series models with seasonal terms.
- **Z items**: forecasts unreliable → either (a) carry generous safety stock and accept overstock, (b) build to order, or (c) consider eliminating the SKU.

XYZ doesn't replace ABC — they're orthogonal. Use them together in the **ABC-XYZ matrix** (next file).

193.2.3. Procedure

1. Collect periodic demand history (monthly is typical; daily for fast movers; quarterly for slow movers).
2. Compute μ and σ for each item.
3. Compute $\mathrm{CV} = \sigma / \mu$.
4. Cut at the chosen thresholds.

Notes:

- Use **enough** history (at least 12 periods, ideally 24+) so CV is stable.
- Detrend if you have a strong trend — otherwise CV is inflated by drift, not noise.

- For seasonal items: deseasonalize first or use within-season variability.

Example

Given (same 6 SKUs as ABC, with 12-month demand histories):

Item	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec	Total
Laptop charger	18	20	19	21	20	19	20	21	19	20	21	22	240
Keyboard	20	25	35	30	28	22	40	45	30	35	28	22	360
Gaming chair	1	0	2	3	1	5	8	12	10	8	6	4	60
Cable	45	50	55	50	48	52	50	53	47	50	52	48	600
Sticker pack	50	100	150	80	120	100	110	90	100	100	100	100	1200
Phone case	10	20	5	30	0	50	25	10	20	15	5	10	200

Step 1 — compute μ and σ for each item

Item	μ (mean / month)	σ (SD)	CV = σ/μ
Laptop charger	20.0	1.1	**0.06**
Keyboard	30.0	7.5	**0.25**
Gaming chair	5.0	3.7	**0.74**
Cable	50.0	2.7	**0.05**
Sticker pack	100.0	25.6	**0.26**
Phone case	16.7	13.6	**0.81**

Step 2 — cut at thresholds (X < 0.5, Y < 1.0, Z ≥ 1.0)

Item	CV	Class
Laptop charger	0.06	X (very predictable)
Cable	0.05	X (very predictable)
Keyboard	0.25	X (predictable)
Sticker pack	0.26	X (predictable)
Gaming chair	0.74	Y (moderately variable, summer-heavy)
Phone case	0.81	Y (variable, on the edge of Z)

Hmm — none of the items hit Z under these thresholds. With only 12 monthly observations, CVs around 0.7–0.8 are common; the "true Z" range (>1.0) usually shows up in **truly intermittent** demand (months of zero, occasional spikes).

Step 3 — interpret

- **Laptop charger, Cable**: stable workhorses. Forecast with simple moving averages or SES; carry minimal safety stock.
- **Keyboard, Sticker pack**: mostly stable but with some seasonality / surge weeks. SES with seasonal adjustment, moderate safety stock.
- **Gaming chair, Phone case**: lumpy. Watch for seasonality; carry more safety stock relative to mean. The gaming chair shows a clear summer pattern — deseasonalize to get a cleaner CV.

Compare to ABC: by value alone, laptop charger and gaming chair were both A items at $12,000/year. By variability, the laptop charger is X (smooth) but the gaming chair is Y (lumpy). They should get **different** inventory policies despite the same value rank — see the ABC-XYZ matrix.

193.3. ABC × XYZ

Combine ABC (consumption value) and XYZ (demand variability) into a **3 × 3 grid**. Each cell suggests a distinct inventory strategy.

193.3.1. The grid

	X (low CV)	Y (moderate CV)	Z (high CV)
A (high value)	**AX**: lean, JIT, supplier integration, low safety stock	**AY**: forecast carefully, moderate safety stock, frequent review	**AZ**: high safety stock + close monitoring; consider supplier flexibility (consignment, drop-ship)
B (medium value)	**BX**: standard policy, automate	**BY**: moderate attention, periodic review	**BZ**: pad safety stock; review whether SKU rationalization helps
C (low value)	**CX**: two-bin / kanban, set-and-forget	**CY**: simple periodic, generous buffer (cheap to overstock)	**CZ**: candidate for elimination — low value AND unpredictable. If kept, build to order or accept high stockout rate

193.3.2. Reading the cells

The **value axis (ABC)** sets **how much management attention** the item deserves. The **variability axis (XYZ)** sets **how much safety stock** (relative to mean demand) the item needs. Combine:

- High value + low variability → **lean**. Spend the management effort on tight forecasting; reap the savings from low safety stock.
- High value + high variability → **expensive to misjudge**. Cannot afford either over- or understock; need close monitoring and possibly contractual risk-sharing with the supplier.
- Low value + high variability → **not worth managing carefully**. Either overstock cheaply or eliminate the SKU.

193.3.3. Procedure

1. Run [ABC analysis](abc.typ): assign each SKU an A / B / C class.
2. Run [XYZ analysis](xyz.typ): assign each SKU an X / Y / Z class.
3. Place each SKU into one of the 9 cells.
4. Pick the inventory policy from the matrix above.

193.3.4. When the matrix isn't enough

The 9-cell grid is a useful starting heuristic but doesn't capture every dimension:

- **Criticality** (a $1 bolt that holds together a $1M machine — see [VED](ved.typ))
- **Movement frequency** / dead stock (see [FSN](fsn.typ))
- **Lead time** (long-lead items need more safety stock regardless of XYZ)

Most operations apply the matrix as a **first-pass policy assignment**, then refine on the dimensions above.

Example

Given (same 6 SKUs from ABC and XYZ):

Combine the two classifications:

Item	ABC	XYZ	Cell + suggested policy
Laptop charger	A	X	**AX** — lean, JIT-friendly, low SS
Gaming chair	A	Y	**AY** — careful forecast (seasonal), moderate SS
Keyboard	A	X	**AX** — lean, automate replenishment
Cable	B	X	**BX** — standard (R, S), automate
Sticker pack	C	X	**CX** — two-bin, set-and-forget
Phone case	C	Y	**CY** — periodic, generous buffer

Resulting matrix populated:

	X	Y	Z
A	Laptop charger Keyboard	Gaming chair	—
B	Cable	—	—

C	Sticker pack	Phone case	—

Step — interpret

- **Top-left (AX)** — laptop charger and keyboard: highest priority, spend the analytics effort here. Lean continuous-review (Q, r) policy with tight safety stock.
- **Center (AY)** — gaming chair: pay for a seasonal forecast; widen safety stock during peak season.
- **Bottom-right empty**: nothing extreme in this dataset. Real catalogs typically have CZ candidates (impulse / niche / one-off items) — those are the SKU-rationalization candidates.

Why the matrix beats ABC alone: if you used ABC alone, the laptop charger and gaming chair would get identical policies (both A). The matrix correctly distinguishes them — the smooth charger gets a lean policy; the lumpy chair gets safety stock and a seasonality model. Same value, different policy.

193.4. FSN

193.4.1. FSN Analysis (Fast / Slow / Non-moving)

Classify items by **movement frequency** — how often they're picked, issued, or sold. Independent of value (ABC) and variability (XYZ).

193.4.1.1. The frequency split

- **F items** (Fast-moving): picked frequently — multiple times per week or daily.
- **S items** (Slow-moving): picked occasionally — a few times per month or quarter.
- **N items** (Non-moving): no issue activity for an extended period (typically 6+ months); effectively **dead stock**.

Thresholds depend on context. Common rules:

Class	Typical threshold	Operational meaning
F	≥ 1 issue / week (or > 12 issues / year)	Place near pickers; standard replenishment cycle.
S	1 issue / month to 1 / week	Place mid-warehouse; periodic review with longer cycles.
N	< 1 issue / 6 months	Quarantine; review for write-off or liquidation.

193.4.1.2. Why classify by movement

FSN is about **operational friction**, not financial impact:

- **Warehouse layout**: F items go into the pick-face / golden zone; N items go into deep storage or get culled.
- **Cycle counting frequency**: F items counted often; N items counted rarely.

- **Obsolescence detection**: a creeping shift from S to N is an early warning that an SKU is dying.
- **Working capital cleanup**: N items tie up cash with zero return — periodic FSN review forces a write-off conversation.

ABC tells you “where is the money sitting?”. FSN tells you “what’s actually moving?”. Both are needed: an **A but N** item (high value, no movement) is **high-priority dead stock** — sell it off, don’t ignore it.

193.4.1.3. Procedure

1. Pull issue / pick / sales records over a window (typically 6–12 months).
2. For each SKU, count the number of issues (or compute **days since last issue**).
3. Classify by your chosen thresholds.
4. Cross-reference with ABC: high-value non-movers are urgent.

193.4.1.4. Adjacent metrics

- **Inventory turnover** = annual demand / average inventory. High = F-class; low = S or N.
- **Days of supply** = average inventory / daily demand. The inverse view.
- **Last-issue date**: most direct test for N-class.

Example

Given: 6-month issue history for 7 SKUs in a small warehouse.

Item	Issues in 6 months	Days since last issue	Notes
Laptop charger	120	0	Daily seller
Cable	80	0	Frequent stock-up
Keyboard	30	3	Moderate
Gaming chair	10	12	Occasional
Phone case	6	25	Slow
Sticker pack	3	60	Slow / sporadic
Old printer cartridge	0	240	Non-moving — likely obsolete

Step 1 — annualize the issue count

Multiply by 2 (since the window was 6 months) for a yearly rate:

Item	Issues / year (annualized)	Class
Laptop charger	240	F (≥ 12 / year)
Cable	160	F
Keyboard	60	F

Gaming chair	20	F (just barely; or S if threshold is daily)
Phone case	12	F / S boundary
Sticker pack	6	S
Old printer cartridge	0	N (no movement in 8 months)

Step 2 — interpret

- **F items** (Laptop charger, Cable, Keyboard, Gaming chair): place in the pick-face. Daily / weekly cycle counts. Standard replenishment.
- **S items** (Phone case, Sticker pack): mid-warehouse storage. Monthly cycle counts. Slower review cadence.
- **N items** (Old printer cartridge): quarantine. Investigate why no movement — discontinued product line? Replaced by a newer SKU? **This is where dead stock hides.**

Step 3 — cross-reference with ABC

From ABC, the laptop charger is A-class. From FSN, F-class. **AF** — high value, fast-moving — is the textbook "manage tightly" cell. Make sure the inventory policy (e.g., (Q, r)) is well-tuned for it.

The old printer cartridge: if it was originally A-class (high unit cost, several cartridges in stock), it's a **high-value non-mover** — write it off or liquidate before the value drops further.

Why FSN matters operationally

Two SKUs with **identical annual value** (ABC = same class) can need very different **physical handling**:

- Laptop charger: 240 issues/year, 1 per day. Pick-face slot, often-replenished.
- Gaming chair: 20 issues/year, weekly. Bulk-storage slot, infrequent picks.

ABC alone misses this distinction. FSN catches it.

193.5. VED

193.5.1. VED Analysis (Vital / Essential / Desirable)

Classify items by **criticality of stockout**. Independent of value or variability — it's about consequence-of-failure.

193.5.1.1. The criticality split

- **V items** (Vital): stockout halts operations or risks safety / regulatory compliance. **No tolerance** for unavailability.
- **E items** (Essential): stockout causes significant disruption but operations can continue (degraded performance, workarounds available).
- **D items** (Desirable): stockout is an inconvenience; operations continue normally with minor impact.

193.5.1.2. Why classify by criticality

ABC, XYZ, and FSN look at **value**, **variability**, and **frequency**. None of these capture **what happens if you run out**. A $1 fuse in a hospital ventilator is **Vital** even though it's low-value, low-variability, and slow-moving.

Criticality is what justifies:

- Higher service levels (target 99.9% for V vs 95% for D)
- Larger safety stocks regardless of holding cost
- Multiple suppliers for V items (sourcing redundancy)
- Emergency procurement contracts for V items

VED is most prevalent in:

- **Healthcare** (drugs, surgical supplies, life-support spares)
- **Manufacturing maintenance** (MRO spares — a $50 bearing that holds together a $500K production line)
- **Aviation / military** (anywhere a single failure has outsized cost)
- **Utilities** (grid, water, telecom — single-point-of-failure components)

193.5.1.3. Procedure

1. For each SKU, ask: **what's the cost of a stockout?**
2. Categorize:
    - **V**: stockout = life, safety, large operational halt, or regulatory violation
    - **E**: stockout = significant downtime / customer impact, but workarounds exist
    - **D**: stockout = minor inconvenience
3. Apply differential service levels and stocking policies.

193.5.1.4. Service-level targets by class

Class	Target CSL	Stocking philosophy
V	99.5–99.99%	**Never stockout**. Multiple suppliers, emergency stock, expedited freight contracts. Safety stock dominates regardless of holding cost.
E	95–99%	Standard high service. Single reliable supplier; safety stock per (Q, r) or (s, S) policies.
D	85–95%	Cost-optimized. Accept occasional stockouts to keep holding costs in check.

193.5.1.5. VED + ABC = a richer matrix

VED is often combined with ABC into a 3×3 grid (similar to ABC-XYZ):

- **AV**: high-value, vital — heavy investment in inventory and supplier diversification.
- **CV**: low-value, vital — overstock liberally; the holding cost is trivial against the stockout cost.
- **AD**: high-value, desirable — most attention to lean inventory, can afford some stockouts.
- **CD**: low-value, desirable — minimum effort; overstock or eliminate.

The asymmetric cells — **low-value vital** especially — are why VED matters: ABC alone says "spend little time on it"; VED says "but never run out."

Example

Given: maintenance spares for a small manufacturing line.

Item	Unit cost	Annual demand	Stockout consequence
Critical PLC controller	$2,000	2 / year	Production line halts; $50K/day lost output
Standard motor bearing	$50	12 / year	Specific machine offline; workaround possible
Lubricant pail	$80	40 / year	Maintenance delayed a few hours
Spare bolt set	$5	200 / year	Small inconvenience; alternative bolts on hand
Calibration weight	$300	1 / year	Required for monthly calibration; no substitute; can pause operations until next cal cycle

Step 1 — assign VED

Item	Class	Reasoning
Critical PLC controller	V	Halts whole line; $50K/day downtime cost
Standard motor bearing	E	One machine offline; workarounds available
Lubricant pail	D	Maintenance delay only; common item
Spare bolt set	D	Substitutes available
Calibration weight	V	Compliance requirement; no substitute

Step 2 — set service levels

Item	VED	CSL target	Implied z
Critical PLC controller	V	99.9%	3.09
Standard motor bearing	E	97%	1.88
Lubricant pail	D	90%	1.28
Spare bolt set	D	90%	1.28
Calibration weight	V	99.5%	2.58

Step 3 — compare safety stocks

Suppose the standard motor bearing had monthly demand σ = 0.5 units, lead time L = 1 month. Safety stock at:

- 95% (default): $z = 1.65 \rightarrow$ SS = 0.83 units $\rightarrow$ round up to 1.
- 97% (E-target): $z = 1.88 \rightarrow$ SS = 0.94 $\rightarrow$ round up to 1.
- 99.9% (if it were V): $z = 3.09 \rightarrow$ SS = 1.55 $\rightarrow$ round up to 2.

Even at the **highest** service level, total SS for a slow-moving item is just 2 units. The **cost difference** is one extra bearing in stock — $50 — vs the **downtime cost** if you stock out. Trivially worth it for E or V class items.

Step 4 — interpret cross-classification with ABC

- **Critical PLC controller**: A-class ($4K annual value), V-class. **AV** — biggest priority. Multiple suppliers, expedited shipping contract, generous safety stock.
- **Calibration weight**: C-class ($300/year), V-class. **CV** — **low value, vital**. The right answer here is **overstock without thinking**. Holding cost is negligible; consequences of stockout are large. Carry 2 units instead of 1.
- **Spare bolt set**: C-class, D-class. **CD** — automated two-bin replenishment. Don't waste planning effort.

The asymmetric cell — CV (low value vital) — is the most common surprise in VED analysis. Operations teams often **under**-stock low-value vital items because ABC says "C-class, don't bother". VED corrects this.

194. Metrics

194.1. Inventory Turnover

How many times the inventory "turns over" (is sold and replaced) per year. The single most-cited inventory metric.

$$\text{Turnover} = \frac{\text{Annual COGS}}{\text{Average inventory value}}$$

Higher = leaner; lower = stagnant.

194.1.1. Definition variants

Three common formulations, depending on what data you have:

Variant	Formula	Use when
Cost-based (most common)	Annual COGS / Avg inventory at cost	You have COGS data — most accurate
Sales-based	Annual sales / Avg inventory at sales price	You only have revenue, not COGS — less precise but works

Unit-based	Annual demand / Avg inventory units	Single-product analysis or ratio comparisons

Cost-based is the standard for financial reporting; sales-based shows up in retail KPIs; unit-based is operational.

194.1.2. Why turnover matters

Turnover combines **demand** and **inventory level** into one number that captures **capital efficiency**:

- High turnover → inventory cycles quickly → low working capital tied up.
- Low turnover → inventory sits → cash trapped, risk of obsolescence.

Compare across products, sites, or time periods to spot trends.

194.1.3. Industry benchmarks

Industry	Typical turnover	Why
Grocery / fresh food	15-25	Perishable, high-frequency replenishment
Apparel retail	4-6	Seasonal cycles, slow-moving items
Electronics retail	6-8	Moderate cycle, fast obsolescence
Industrial distribution	4-8	B2B, slower cycles
Heavy manufacturing	3-5	Long lead times, large WIP
Pharmaceuticals	3-4	Long shelf life, regulatory inventory
Luxury goods	1-2	Low volume, high margin per unit

Use the benchmark to flag outliers. **Far below benchmark** → likely overstocked. **Far above** → may be understocked (frequent stockouts).

194.1.4. Connection to days of supply

The most natural "human-readable" interpretation of turnover is its inverse:

$$\text{Days of supply} = \frac{365}{\text{Turnover}}$$

If turnover = 12, you hold 30 days of inventory. If turnover = 4, you hold 91 days. See days_of_supply.typ.

194.1.5. What turnover doesn't tell you

- **Why** the number is what it is — could be high turnover from good management OR understocking.
- **Distribution across SKUs** — total turnover hides which SKUs are dead stock vs fast movers (use FSN classification).

- **Seasonality** — annual averages mask quarterly swings (use seasonally adjusted turnover or rolling windows).

Turnover is a **summary** metric. Pair with stockout rate, FSN analysis, and days-of-supply distribution for diagnosis.

Example

Given:

- Annual COGS: $2,400,000
- Average inventory at cost (taken at month-ends, averaged): $200,000

Step 1 — turnover

$$\text{Turnover} = \frac{\$2{,}400{,}000}{\$200{,}000} = 12$$

Step 2 — interpret

Inventory turns over 12 times per year — once per month on average. Working capital tied up: $200K, cycling 12× → supports $2.4M in COGS.

Days of supply: $\frac{365}{12} \approx 30$ days.

Step 3 — compare to benchmark

Industry: industrial distribution, benchmark 4-8.

This company at 12 is **well above** benchmark. Two interpretations:
1. **Lean operations**: tight forecasts, fast replenishment, low safety stock. Excellent.
2. **Understocked**: frequent stockouts, lost sales. Bad.

Diagnose with stockout rate and customer-feedback data. If stockout rate is < 2% → great efficiency. If > 10% → being lean is costing sales.

Step 4 — improve turnover (if it's the goal)

Lever 1: shrink average inventory (numerator unchanged). Tighten safety stock, reduce cycle stock (smaller Q, more frequent orders), eliminate dead stock.

Lever 2: grow COGS without growing inventory proportionally — i.e., grow the business.

Lever 1 hits a floor (you need **some** safety stock). Lever 2 is where high-turnover companies actually win — Walmart, Costco, fast fashion all run high turnover by selling a lot from a small inventory footprint.

194.2. Days of Supply

How many days the current inventory will last at the current consumption rate. The "human-readable" cousin of turnover.

$$\text{DOS} = \frac{\text{Average inventory}}{\text{Average daily consumption}}$$

Equivalently:

$$\text{DOS} = \frac{365}{\text{Turnover}}$$

194.2.1. Definition variants

Variant	Formula	When to use
Unit-based	On-hand units / daily demand units	Single SKU operational view
Value-based	Inventory value / daily COGS	Aggregate financial view
Forward-looking	Inventory / **forecasted** daily demand	When demand changes (seasonality, promos)

The forward-looking version is more accurate for planning; the historical version is simpler and what most reports show.

194.2.2. Why DOS rather than turnover?

Turnover is a **ratio** (e.g., 12). DOS is a **time** (e.g., 30 days). Time is more intuitive for operational decisions:

- "We have 45 days of supply" → vivid, actionable.
- "Turnover is 8.1" → harder to feel.

Both convey the same information; just pick the one your stakeholders prefer.

194.2.3. Targets by inventory function

Different functions of inventory imply different DOS targets:

Function	Typical DOS	Reasoning
Cycle stock	$Q/(2d)$	Half the cycle length; depends on EOQ
Safety stock	$z\sigma_L/d$	Buffer for service level
Pipeline stock	L	Lead time itself
Anticipation	varies	Days until peak demand

Total DOS = sum across functions, comparable to total inventory ÷ daily demand.

194.2.4. Reading DOS distribution

A useful diagnostic: plot **DOS by SKU** (histogram).

- **Tail of high-DOS items** (e.g., > 200 days): dead stock candidates. Cross-reference with FSN — these are usually N-class (non-moving) items.
- **Spike at low DOS** (e.g., < 7 days): chronically understocked. Check stockout rate.
- **Healthy bulk**: cluster around the target DOS for the policy.

The aggregate company DOS hides this distribution — it's the average. Plot it.

194.2.5. DOS vs lead time

Compare DOS to lead time:

- **DOS < L**: high stockout risk. The next order won't arrive before current stock runs out.
- **DOS = L**: no buffer. Razor-thin operations.
- **DOS > L**: have at least lead-time coverage; safety stock + cycle stock above L.

Healthy operations target $\text{DOS} \approx 1.5 \times L$ to $2 \times L$, with the excess coming from cycle stock and modest safety stock.

Example

Given:

- Average inventory: 880 units (cycle 388 + safety 30 + pipeline 462)
- Daily demand: 33 units / day
- Lead time: 14 days

Step 1 — total DOS

$$\text{DOS} = \frac{880}{33} \approx 27 \text{ days}$$

Equivalent to turnover of $\frac{365}{27} \approx 13.5$.

Step 2 — DOS by component

- Cycle stock: $\frac{388}{33} \approx 12$ days
- Safety stock: $\frac{30}{33} \approx 1$ day
- Pipeline: $\frac{462}{33} = 14$ days (= L)
- Total: $12 + 1 + 14 = 27$ days ✓

Step 3 — sanity check

- Lead time = 14 days. DOS = 27 days. **Buffer above lead time**: 13 days (cycle + safety).
- Comfortable margin. **No risk of stockout from running out before next order arrives.**

Step 4 — flag analysis

Suppose another SKU shows DOS = 8 days with $L = 14$ days. **DOS < L**: that SKU is in trouble. The next replenishment can't arrive before current stock runs out. **Stockout incoming**.

Action: increase safety stock, reduce Q for faster reorder, or expedite the inbound shipment.

Step 5 — flag dead stock

Suppose another SKU shows DOS = 365 days. That's a **year** of supply on hand. Almost certainly dead stock or vastly overstocked. Action: investigate; cross-check with FSN.

DOS is the diagnostic; FSN, ABC, and stockout rate are the follow-ups.

194.3. Order Cycle Time

194.3.1. (OCT) Order Cycle Time

Measures the total time taken from when a customer places an order to when the order is delivered

$$\text{OCT} = T_{\text{order}} + T_{\text{processing}} + T_{\text{production}} + T_{\text{shipping}}$$

Where:

- T_{order}: Order Entry Time (time it takes to receive and log the order)
- $T_{\text{processing}}$: Order Processing Time (time to check inventory, verify details, and prepare for production or shipment)
- $T_{\text{production}}$: Production Time (time to manufacture or prepare the product)
- T_{shipping}: Shipping Time (time it takes to deliver the product from the warehouse to the customer)

194.4. Perfect Order Rate

Measures the percentage of orders delivered to customers in full, on time, and without any damage

$$\text{Perfect Order Rate} = \frac{\text{Number of Perfect Orders}}{\text{Total Number of Orders}} \times 100\%$$

Where:

- **Number of Perfect Orders**: The number of orders that are delivered on time, complete, and undamaged.
- **Total Number of Orders**: The total number of orders fulfilled within a specific period.

Example

Suppose you received 1,000 orders over a quarter, and 900 of those orders were delivered on time, complete, and without damage.

$$\text{Perfect Order Rate} = \frac{900}{1000} \times 100\% = 90\%$$

194.5. Stockout Rate

The **measured** frequency of stockouts. A KPI for assessing how well an inventory system is actually performing — distinct from the **target** service level used to design the policy.

Three common formulations:

Variant	Formula	What it measures

Cycle stockout rate	(cycles with stockout) / (total cycles)	Frequency of stockout **events**
Time stockout rate	Time out of stock / total time	Fraction of **time** out of stock
Item stockout rate	(SKUs out of stock at moment t) / (total SKUs)	Snapshot view across the catalog

194.5.1. Cycle stockout rate (target = α)

Tracks the fraction of replenishment cycles that hit stockout at least once. Measures the same quantity that **cycle service level** (CSL) targets:

$$\text{Cycle stockout rate} = \frac{\#\text{ cycles with stockout}}{\text{total cycles}} = 1 - \text{CSL}_{\text{actual}}$$

Use when designing policies around the target α and verifying you hit it in operation.

194.5.2. Time stockout rate (target ≈ $1 - \gamma$)

Tracks time fraction out of stock. Equivalent to $1 -$ ready rate (see [ready_rate.typ](../service_levels/ready_rate.typ)).

$$\text{Time stockout rate} = \frac{\text{Days out of stock}}{\text{Total days}}$$

Easy to compute from ERP data — just sum the duration of zero-on-hand periods.

194.5.3. Item stockout rate (catalog-level)

Snapshot of **how many** SKUs are out of stock at a given moment, expressed as a fraction:

$$\text{Item stockout rate}(t) = \frac{\#\text{ SKUs with on-hand} = 0}{\#\text{ active SKUs}}$$

Tracked over time as a daily / weekly time series. Common retail KPI, sometimes called **out-of-stock rate** or **fill-rate-on-shelf**.

194.5.4. Why measure it

The target service level (CSL = 95%, fill rate = 99%) is what you **aim for** during policy design. **Actual stockout rate** tells you whether the system is performing as designed. Common reasons for divergence:

- Demand forecast wrong → safety stock too low.
- Lead times longer than assumed → reorder point too low.
- Demand more variable than assumed → σ_L underestimated.
- Operational errors — late ordering, miscounts, supplier delivery issues.

If actual stockout rate >> target, the **physical** operation is breaking, not the **design**. Different remedy.

194.5.5. Connection to other metrics

Metric	Counts	Stockout-rate equivalent
CSL (α)	Cycles	Cycle stockout rate
Fill rate (β)	Units	1 - Fill rate (unit stockout rate)
Ready rate (γ)	Time	Time stockout rate

Different denominators measuring "how often things go wrong" — choose what your stakeholders care about.

194.5.6. Diagnostic combinations

- **High cycle stockout rate, low time stockout rate**: stockouts are **frequent but short**. Replenishment is reactive but fast.
- **Low cycle stockout rate, high time stockout rate**: stockouts are **rare but long**. When they happen, they linger — usually a supply-side problem (lead time, supplier issue).
- **Item stockout rate growing over time**: catalog-wide degradation — could be a forecasting drift, a supplier-base issue, or demand surge across products.

Example

Given (one quarter of inventory data):

- 100 SKUs, monitored daily for 90 days.
- Combined: 27 stockout **events**. Total stockout **days**: 45 (across all events and SKUs).
- Average number of active SKUs: 100. Snapshot at end of each day shows on average 1.2 SKUs out of stock.

Step 1 — cycle stockout rate

Total cycles: each SKU experiences 12 cycles per quarter (assuming monthly replenishment). Across 100 SKUs: 1200 cycles.

$$\text{Cycle stockout rate} = \frac{27}{1200} \approx 2.25\%$$

Equivalent: actual CSL = 97.75%. If the **target** CSL was 95%, the operation **exceeds** its target.

Step 2 — time stockout rate

Each SKU has 90 SKU-days; total SKU-days = 9,000. Stockout SKU-days = 45.

$$\text{Time stockout rate} = \frac{45}{9000} = 0.5\%$$

Equivalent: actual ready rate = 99.5%.

Step 3 — item stockout rate

Average daily snapshot: 1.2 SKUs out of 100 are out of stock.

$$\text{Item stockout rate} = \frac{1.2}{100} = 1.2\%$$

Step 4 — interpret

Cycle rate (2.25%) is **higher** than time rate (0.5%) — the stockouts are short. On average, $\frac{45}{27} \approx 1.7$ days each. The team responds quickly.

Item rate of 1.2% means at any given snapshot, about 1 SKU in 100 is unavailable. For 100 SKUs that's noticeable; for a 100,000-SKU retailer, that's 1,200 missing items.

Step 5 — drill down

Which SKUs account for the 27 stockout events? If concentrated in 5 SKUs (each stockout 5–6 times), they have policy issues — investigate forecasting, safety stock, lead time. If spread across all 100, the problem is systemic.

Always cross-tab stockout rate against ABC and VED: an A or V class item being out is much more costly than a C or D class.

194.6. GMROI

194.6.1. GMROI (Gross Margin Return On Inventory Investment)

How much gross profit each dollar of inventory generates per year. The retail / merchandising metric of choice — combines **profitability** and **turnover** into one number.

$$\text{GMROI} = \frac{\text{Gross margin}}{\text{Average inventory cost}}$$

Equivalently:

$$\text{GMROI} = \text{Gross margin \%} \times \text{Turnover}$$

A GMROI of 3.0 means: every $1 of inventory generates $3 of gross profit per year.

194.6.1.1. Where it comes from

Standalone, **gross margin %** tells you how much profit you make per dollar of sales. **Turnover** tells you how many times you cycle inventory. Multiplying gives gross profit per dollar of inventory:

$$\frac{\underbrace{\text{Gross margin}}_{\$}}{\underbrace{\text{Avg inventory cost}}_{\$}} = \frac{\underbrace{\text{Gross margin}}_{\$}}{\underbrace{\text{COGS}}_{\$}} \times \frac{\underbrace{\text{COGS}}_{\$}}{\underbrace{\text{Avg inventory cost}}_{\$}}$$

$$= \text{Gross margin \%} \times \text{Turnover}$$

Two levers: improve margins or improve turnover. Both flow to GMROI.

194.6.1.2. Why retailers love it

Compares wildly different products on a **per-dollar-of-shelf-investment** basis:

- **Luxury watches**: 60% margin, 1× turnover → GMROI = 0.6.
- **Cheap apparel**: 30% margin, 6× turnover → GMROI = 1.8.
- **Grocery staples**: 20% margin, 25× turnover → GMROI = 5.0.

Cheap apparel beats luxury watches **per dollar of shelf space**. Grocery staples beat both. This is why grocery and fast fashion retailers dominate inventory-efficient operations even though margins are thin.

194.6.1.3. When GMROI is misleading

- **Doesn't account for shelf space**: a small high-GMROI item might be space-inefficient. Pair with **GMROS** (Gross Margin per Square Foot) for retail decisions.
- **Doesn't capture cross-product effects**: a low-GMROI loss-leader might drive traffic that benefits other items.
- **Sensitive to seasonality**: averaged inventory across a year may misrepresent peak weeks.
- **Capital cost ignored**: GMROI counts only inventory carrying cost implicitly via "average inventory" — not the cost of capital used to fund it.

For executive decisions on shelf allocation, complement GMROI with: GMROS, sales velocity, contribution to traffic, customer mix.

194.6.1.4. Improving GMROI

Three paths:

1. **Raise margins**: better procurement, premium positioning, private label.
2. **Increase turnover**: leaner inventory (smaller Q, lower safety stock), faster product cycles.
3. **Both**: rare but powerful — examples include Costco (low margin, very high turnover) and luxury (high margin, low turnover).

Example

Given (a retail product line):

- Annual sales: $1,000,000
- Annual COGS: $700,000 → gross margin $300,000 → margin % = 30%
- Average inventory at cost: $100,000

Step 1 — turnover

$$\text{Turnover} = \frac{\text{COGS}}{\text{Avg inventory}} = \frac{700}{100} = 7$$

Step 2 — GMROI direct calc

$$\text{GMROI} = \frac{\text{Gross margin}}{\text{Avg inventory}} = \frac{\$300\{,\}000}{\$100\{,\}000} = 3.0$$

Step 3 — verify via the factor identity

Using cost-based turnover and margin-on-COGS:

$$\text{Margin on COGS} = \frac{\text{Margin}}{\text{COGS}} = \frac{300}{700} \approx 0.429$$

$$\text{GMROI} = \text{Margin on COGS} \times \text{Turnover} = 0.429 \times 7 = 3.0\checkmark$$

Equivalently, using margin% on sales and the relation $\text{Margin on COGS} = \text{Margin \%} / (1 - \text{Margin \%}) = \frac{0.30}{0.70} \approx 0.429$.

In practice: state which "turnover" you mean (cost-based vs sales-based) and stick to one definition. Mixing them gives different numbers.

Step 4 — comparison

Compare to a luxury line: margin 60%, turnover 1.5 (annual rotation).

$$\text{GMROI}_{\text{luxury}} = \frac{0.60}{0.40} \times 1.5 = 2.25$$

And to a grocery-style line: margin 20%, turnover 30.

$$\text{GMROI}_{\text{grocery}} = \frac{0.20}{0.80} \times 30 = 7.5$$

Grocery beats luxury per dollar of inventory even though the per-unit margin is much lower. Volume × velocity wins.

195. Service Levels

195.1. Cycle Service Level

195.1.1. Cycle Service Level (Type I, α)

The probability of **no stockout during a replenishment cycle**. Counts **events**: did this cycle have any unmet demand, yes or no?

$$\text{CSL} = 1 - \alpha = P(\text{no stockout during cycle})$$

195.1.1.1. Definition

A **cycle** runs from one order arrival to the next. Stockout = **any** moment in the cycle where on-hand inventory hits zero with demand still present.

For a (Q, r) policy:

- Order placed when inventory position drops to r.
- Order arrives L time later. During those L time units, lead-time demand D_L consumes inventory.
- **No stockout** in this cycle iff $D_L \leq r$.

So:

$$\text{CSL} = P(D_L \leq r)$$

195.1.1.2. Setting r for target CSL

If $D_L \sim \mathcal{N}(\mu_L, \sigma_L^2)$:

$$\text{CSL} = \Phi\left(\frac{r - \mu_L}{\sigma_L}\right) = 1 - \alpha$$

Solve for r:

$$r = \mu_L + z\sigma_L \quad \text{where} \quad z = \Phi^{-1}(1 - \alpha)$$

The familiar "$\mu + z\sigma$" form.

195.1.1.3. What CSL does not count

CSL counts **cycles**, not **units**. A cycle with one unit short and a cycle with 1000 units short both count as one stockout. So a 95% CSL means **5% of cycles experience a stockout**, but says **nothing** about how **severe** those stockouts are.

In practice this matters: a 95% CSL might correspond to 99% fill rate (most stockouts are tiny) or 80% fill rate (the stockouts are catastrophic) depending on the demand-distribution shape.

195.1.1.4. When CSL is the right metric

Use CSL when:

- The **occurrence** of any stockout is what matters (regulatory thresholds, contract penalties tied to "stockout incidents", reputational concerns).
- Demand is fairly smooth — the gap between CSL and fill rate is small.
- You want a simple, interpretable number for executive reporting.

Don't use CSL when stockout **severity** matters more than **frequency** — use fill rate instead.

195.1.1.5. Common z values

CSL ($1 - \alpha$)	α	z	Notes
50%	0.50	0.00	Order at the median
80%	0.20	0.84	Lean
90%	0.10	1.28	Common default
95%	0.05	1.65	Standard target
97.5%	0.025	1.96	Two-sigma
99%	0.01	2.33	High service
99.5%	0.005	2.58	
99.9%	0.001	3.09	Critical items (V-class)

Example

Given (continuous-review (Q, r) policy):

- Mean lead-time demand: $\mu_L = 462$
- Std of lead-time demand: $\sigma_L = 18.7$
- Target CSL: 95%

Step 1 — quantile

$$z = \Phi^{-1}(0.95) \approx 1.65$$

Step 2 — reorder point

$$r = \mu_L + z\sigma_L = 462 + 1.65 \cdot 18.7 \approx 493 \text{ units}$$

Safety stock: $z\sigma_L \approx 31$ units.

Step 3 — interpret

Out of every 100 cycles, expect **about 5** to experience some stockout. Each stockout could be small (1-2 units short) or large (50+ units short, if a demand spike hits) — CSL doesn't distinguish.

If a cycle averages 24 days (= Q/d for $Q = 775$, $d = 33$), then 5% of cycles is **one stockout every ≈ 480 days** on average — about twice every three years.

195.2. Fill Rate

195.2.1. Fill Rate (Type II, β)

The expected **fraction of demand** satisfied directly from on-hand inventory. Counts **units**, not events.

$$\text{Fill rate} = \beta = \frac{E[\text{units served}]}{E[\text{units demanded}]} = 1 - \frac{E[\text{units short}]}{E[\text{units demanded}]}$$

195.2.1.1. Why fill rate, not CSL?

Cycle service level (CSL) treats every stockout — small or large — as a single failure. Fill rate is **unit-weighted**: a cycle that's 1 unit short is treated very differently from a cycle that's 100 units short. Most operational service-level promises to customers (e.g., "we fulfill 99% of unit demand") refer to fill rate.

CSL → frequency of stockout incidents. Fill rate → fraction of demand met.

195.2.1.2. Computing fill rate (continuous demand)

For a (Q, r) policy with normal lead-time demand:

$$\text{Expected shortage per cycle} = E\left[(D_L - r)^+\right] = \sigma_L \cdot L(z)$$

where $z = (r - \mu_L)/\sigma_L$ and $L(z) = \varphi(z) - z(1 - \Phi(z))$ is the **standard normal loss function** (see [newsvendor expected_profit](../newsvendor/expected_profit.typ) for the derivation).

Each cycle covers Q units of demand on average. So:

$$\beta = 1 - \frac{E[\text{shortage per cycle}]}{E[\text{demand per cycle}]} = 1 - \frac{\sigma_L L(z)}{Q}$$

Equivalently:

$$\text{Required } \sigma_L L(z) = (1 - \beta)Q$$

To hit fill rate β, choose r such that $L(z) = (1-\beta)Q/\sigma_L$.

195.2.1.3. Why fill rate is usually higher than CSL

For the same r:

- **CSL** = $P(D_L \le r) = 1-\alpha$
- **Fill rate** = $1 - \sigma_L L(z)/Q$

Whenever Q is much larger than σ_L, the fill rate exceeds the CSL. Reason: most cycles run no stockout at all (contributing 100% fill); the few that **do** stock out are usually short by only a few units (each contributing > 99% fill on its own cycle).

Practical ranges:

- Lean inventory ($Q/\sigma_L \approx 5$): fill rate ≈ CSL.
- Standard ($Q/\sigma_L \approx 20$): fill rate ≈ CSL + 5%.
- Bulk ($Q/\sigma_L \approx 50$): fill rate can be 99% even at CSL = 80%.

195.2.1.4. Setting r for target fill rate

Iterative or table-based. Algorithm:

1. Given target β, solve $L(z) = (1-\beta)Q/\sigma_L$ for z.
2. Use a normal loss function table (or numerically solve z).
3. Set $r = \mu_L + z\sigma_L$.

Most software does this automatically.

Example

Given (same params as CSL example):

- $Q = 775$, $\mu_L = 462$, $\sigma_L = 18.7$
- Target fill rate: $\beta = 99\%$ (more demanding than CSL = 95%)

Step 1 — required loss-function value

$$L(z) = (1-\beta)\frac{Q}{\sigma_L} = (1-0.99)\cdot\frac{775}{18.7} = 0.414$$

Step 2 — invert the loss function

Looking up: $L(0.10) \approx 0.351$, $L(0.05) \approx 0.378$, $L(0.00) \approx 0.399$, $L(-0.05) \approx 0.421$.

So $z \approx -0.04$ gives $L(z) \approx 0.414$. (For high fill rate, we'd usually expect $z > 0$ — but since Q is much larger than σ_L, even a **small** safety stock yields high fill rate. In some cases z even goes **negative** for moderate fill rates.)

Step 3 — reorder point

$$r = \mu_L + z\sigma_L = 462 + (-0.04)\cdot 18.7 \approx 461$$

Almost no safety stock! Setting $r \approx 461$ achieves 99% fill rate because the order quantity 775 is so large compared to lead-time variability 18.7 that stockouts are rare and small.

Step 4 — same setup, target CSL = 95%

Compare: CSL 95% would set $r = 493$ (safety stock 31). Fill rate at $r = 493$:

$$z = \frac{493 - 462}{18.7} \approx 1.66, \quad L(1.66) \approx 0.0207$$

$$\beta = 1 - 18.7 \cdot \frac{0.0207}{775} \approx 1 - 0.0005 \approx 99.95\%$$

Implication: setting CSL to 95% gives a fill rate of essentially 100% in this regime. The CSL target is **much** more demanding than the fill-rate target for moderate-Q regimes.

This explains why retailers usually quote fill rate (99%) rather than CSL (which would have to be e.g. 70% to give the equivalent operational meaning).

195.3. Ready Rate

195.3.1. Ready Rate (Time-based service level)

The probability that on-hand inventory is **positive at any random moment in time**. Counts **time**, not cycles or units.

$$\text{Ready rate} = \gamma = P(\text{on-hand} > 0)$$

195.3.1.1. How it differs from CSL and fill rate

Metric	Counts	What it means	Sample question
CSL (α)	Cycles	P(no stockout in this cycle)	"What fraction of cycles have a stockout?"
Fill rate (β)	Units	E[demand met] / E[demand]	"What fraction of unit demand is met?"
Ready rate (γ)	Time	P(stock > 0 at time t)	"What fraction of time are we in stock?"

All three are "service level" but answer different questions.

195.3.1.2. Ready rate = fraction of time with positive stock

In steady state, the inventory level over a (Q, r) cycle alternates:

- **Positive on-hand** during most of the cycle.
- **Zero on-hand** during any stockout window (the gap from when stock hits zero to when the order arrives).

If T is the average cycle length and W is the **expected stockout duration per cycle**:

$$\gamma = \frac{T - W}{T} = 1 - \frac{W}{T}$$

For (Q, r) with $T = \frac{Q}{d}$:

$$\gamma \approx 1 - \frac{\sigma_L \frac{L(z)}{d}}{\frac{Q}{d}} = 1 - \sigma_L \frac{L(z)}{Q}$$

The same expression as fill rate! Under continuous-demand assumptions, ready rate and fill rate coincide. They diverge only when stockout duration and shortage size aren't proportional (e.g., very lumpy demand patterns).

195.3.1.3. When ready rate is the right metric

Use ready rate when:

- **Server availability**: a service desk that can take orders only when stock is on hand. "Are we ready to serve?"
- **Compliance**: regulations require a minimum % of time in stock (some pharmacies, parts depots).
- **Multi-product systems**: aggregate ready rate = average of individual ready rates, easier to manage.

Don't use ready rate when shortages have widely varying severity — use fill rate to capture how **much** demand was unmet rather than just **whether** you were stocked out at the moment.

195.3.1.4. Approximate equivalence to fill rate

Under standard assumptions (continuous time, smooth demand, no lumpy bursts):

$$\gamma \approx \beta \approx 1 - \sigma_L \frac{L(z)}{Q}$$

In practice, choose whichever is more **interpretable** for your stakeholders:

- Customer-facing metrics → **fill rate** (units of unmet demand)
- Operations / compliance → **ready rate** (time in stock)
- Executive reporting → **CSL** (binary cycle outcome, simpler)

Example

Given (same (Q, r) policy as CSL/fill rate examples):

- $Q = 775$, $\mu_L = 462$, $\sigma_L = 18.7$, $d = 33$/day
- Reorder point set for CSL = 95%: $r = 493$, $z = 1.65$.

Step 1 — expected stockout duration per cycle

Stockout occurs only if $D_L > r$. Expected shortage in units:

$$E[\text{shortage}] = \sigma_L L(z) = 18.7 \cdot 0.0207 \approx 0.39 \text{ units}$$

Convert to time: shortage ≈ 0.39 units, daily demand 33, so:

$$W = \frac{E[\text{shortage}]}{d} \approx \frac{0.39}{33} \approx 0.012 \text{ days} \approx 17 \text{ minutes}$$

Step 2 — ready rate

$$\gamma = 1 - \frac{W}{T} = 1 - \frac{0.012}{\frac{775}{33}} \approx 1 - 0.0005 \approx 99.95\%$$

Step 3 — compare metrics

At the same reorder point $r = 493$:

- **CSL**: 95% (5% of cycles have a stockout)
- **Fill rate**: 99.95% (0.05% of unit demand unmet)
- **Ready rate**: 99.95% (in stock 99.95% of the time)

Fill rate ≈ ready rate (both unit/time-weighted). CSL is the strict outlier — it's an **event** count, blind to severity.

Two stakeholder views:

- **CFO**: "We hit a 5% stockout-cycle rate at this safety stock level — is that too lenient?"
- **Operations**: "We're shipping 99.95% of orders; nobody complains." — fill rate / ready rate.

Both views are correct; just different denominators.

196. Stocks

196.1. Cycle Stock

The portion of inventory that exists **because we order in batches instead of unit-by-unit**. Pure consequence of fixed ordering costs and the EOQ trade-off.

$$\text{Cycle stock (average)} = \frac{Q}{2}$$

196.1.1. Where it comes from

Order Q units. Drain at rate d until empty. Order again. Repeat.

Inventory profile is a sawtooth: peaks at Q just after each order, drops linearly to 0, jumps back to Q. Average over the cycle: $Q/2$.

If we ordered each unit immediately as needed, cycle stock would be 0 — but ordering has a fixed setup cost S, so we batch.

196.1.2. Why $Q/2$?

Linear drainage from Q to 0 over time Q/d. The **time-average** is the area under the triangle divided by the cycle length:

$$\text{Average inventory} = \left(\frac{1}{2}\right) Q \cdot \frac{\frac{Q}{d}}{\frac{Q}{d}} = \frac{Q}{2}$$

Always exactly half the order quantity for **deterministic** demand. For stochastic demand, the expected on-hand mid-cycle is approximately $Q/2$ plus the safety stock buffer.

196.1.3. Cost of cycle stock

Annual holding cost from cycle stock alone:

$$\mathrm{HC}_{\text{cycle}} = h \cdot \frac{Q}{2}$$

This is the standard EOQ holding-cost term. The basic-EOQ optimum balances this against ordering cost $SD/Q \rightarrow$ gives $Q^* = \sqrt{2SD/h}$.

196.1.4. Reduce cycle stock by shrinking Q

Two ways to reduce cycle stock:

1. **Shrink Q** directly — accept more frequent orders, more setup cost.
2. **Reduce setup cost S** — then EOQ shrinks naturally (lean / SMED — single-minute exchange of die — work targets exactly this).

Lean inventory practice = drive S toward zero so Q can shrink, ideally toward 1 unit (just-in-time ordering).

196.1.5. How it composes with other stock types

Cycle stock is just one of **five** inventory components in a typical operation:

Component	Magnitude	Reason
Cycle stock	$Q/2$	Batched ordering
Safety stock	$z\sigma_L$	Demand uncertainty
Pipeline stock	dL	In-transit during lead time
Anticipation stock	planned	Forecasted demand spikes
Decoupling stock	planned	Buffer between production stages

Total average inventory = sum of all components.

Example

Given (the same shared params as EOQ / policies):

- Annual demand: $D = 12000$, daily $d = 33$
- Order cost: S = \$50, holding h = \$2/unit/yr
- Order quantity: $Q^{\mathrm{EOQ}} = 775$

Step 1 — average cycle stock

$$\text{Cycle stock} = \frac{Q}{2} = \frac{775}{2} \approx 388 \text{ units}$$

Step 2 — annual holding cost from cycle stock

$$\mathrm{HC}_{\text{cycle}} = h \cdot \frac{Q}{2} = 2 \cdot 388 \approx$$

\$775 / year

This **equals** the annual ordering cost $S\frac{D}{Q} = 50 \cdot \frac{12000}{775} \approx \$775$ — a property of basic EOQ (the two costs balance at the optimum).

Step 3 — reduce cycle stock

If we halve setup cost ($S \to \$25$ via process improvement), new EOQ:

$$Q^* = \sqrt{2 \cdot 12000 \cdot \frac{25}{2}} \approx 548 \text{ units}$$

Cycle stock drops to $548/2 = 274$ — a 30% reduction. Total holding cost falls to $h \cdot 274 = \$548$/yr.

Halving setup → 30% smaller cycle stock. (Square-root law: cycle stock scales with $\sqrt{S}$.)

Step 4 — driving toward JIT

In the limit $S \to 0$: $Q^* \to 1$, cycle stock $\to 0.5$ unit, you order one (or near-one) unit at a time. This is the JIT / lean ideal.

Real lean operations don't achieve $S = 0$ but get close enough that cycle stock becomes a small fraction of total inventory — most of the remaining inventory is safety stock and pipeline stock.

196.2. Lead Time

Order today, receive after the **lead time** L. What drives safety stock is the **distribution of demand over those L periods**. Central question: why does its standard deviation scale with $\sqrt{L}$, not L?

196.2.1. Setup

Daily demand D_t is i.i.d. with mean μ and standard deviation σ. Take $\mu = 100$, $\sigma = 20$.

196.2.2. Two days

Demand over two days is $D_1 + D_2$ (independent draws).

Means add:

$$\mathbb{E}[D_1 + D_2] = \mu + \mu = 2\mu = 200$$

Standard deviations do **not** add — **variances** do:

$$\text{Var}(D_1 + D_2) = \text{Var}(D_1) + \text{Var}(D_2) = \sigma^2 + \sigma^2 = 2\sigma^2$$

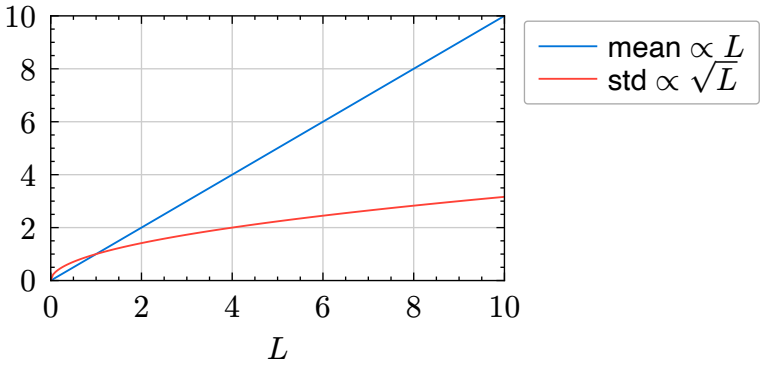
So the spread grows by only $\sqrt{2}$:

$$\text{Std}(D_1 + D_2) = \sqrt{2\sigma^2} = \sigma\sqrt{2}$$

196.2.3. Generalize to L days

$$\mathrm{Var}\left(\sum_{t=1}^{L} D_t\right) = L\sigma^2 \quad \Rightarrow \quad \mathrm{Std}\left(\sum_{t=1}^{L} D_t\right) = \sigma\sqrt{L}$$

- Mean of lead-time demand scales with L: $\mu_L = L\mu$
- Spread scales with $\sqrt{L}$: $\sigma_L = \sigma\sqrt{L}$



This $\sqrt{L}$ is why safety stock $\mathrm{SS} = z \cdot \sigma\sqrt{L}$ grows sub-linearly in lead time — halving L cuts safety stock by only $\sqrt{2}$, not 2.

196.3. Lead Time

Why $\sqrt{L}$?

Setup

Daily demand has mean μ and standard deviation σ

Say:

- $\mu = 100$
- $\sigma = 20$

One day:

- Mean is $\mu = 100$
- Spread is $\sigma = 20$

Two days:

- Demand: $D_1 + D_2$ (each day is independent)

Mean add:

$$\mathbb{E}[D_1 + D_2] = \mu_1 + \mu_2 = 2\mu = 200$$

But standard deviations do **NOT** add, Variances add:

$$\mathrm{Var}(D_1 + D_2) = \mathrm{Var}(D_1) + \mathrm{Var}(D_2) = \sigma_1^2 + \sigma_2^2 = 2\sigma^2$$

So:

$$\mathrm{Std}(D_1 + D_2) = \sqrt{2\sigma^2} = \sigma\sqrt{2}$$

Generalize to L days:

$$\mathrm{Var}\left(\sum_{t=1}^{L} D_t\right) = L \cdot \sigma^2$$

$$\mathrm{Std}\left(\sum_{t=1}^{L} D_t\right) = \sigma\sqrt{L}$$

- Mean scales with L
- Std scales with $\sqrt{L}$

196.4. Safety Stock

The portion of inventory held **as a buffer against uncertainty** — in demand, in lead time, or both. Distinguished from cycle stock (which exists due to **batching**, not uncertainty).

$$\mathrm{SS} = z \cdot \sigma_{\mathrm{LD}}$$

where σ_{LD} is the standard deviation of **lead-time demand** and z is the quantile of the standard normal at the desired service level.

196.4.1. Why this form

The reorder point under (Q, r) is set such that the probability of running out during lead time equals the target stockout probability α:

$$P(D_L > r) = \alpha$$

For normally distributed lead-time demand $D_L \sim \mathcal{N}(\mu_L, \sigma^2_{\mathrm{LD}})$:

$$r = \mu_L + z\sigma_{\mathrm{LD}}, \quad z = \Phi^{-1}(1-\alpha)$$

Decompose:

- μ_L = expected demand during lead time. Just covers the average — no safety.
- $z\sigma_{\mathrm{LD}}$ = the **safety stock**. Buffer above the average for the variability.

196.4.2. Lead-time demand variance — the four cases

The form of σ^2_{LD} depends on which of demand and lead time are random. Combining via the law of total variance:

$$\sigma^2_{\mathrm{LD}} = E[L]\sigma^2_d + (E[d])^2\sigma^2_L$$

where σ_d = std of per-period demand, σ_L = std of lead time, $E[d] = |d|$ and $E[L] = |L|$.

The four classical cases:

Case	σ_{LD}	Where the variance comes from
Constant d, constant L	0	No variance — basic EOQ regime, no safety stock needed
Variable d, constant L	$\sqrt{L} \cdot \sigma_d$	Demand variability over L periods
Constant d, variable L	$\lvert d\rvert \cdot \sigma_L$	Lead time variability scales the deterministic demand

Variable d, variable L	$\sqrt{L\sigma_d^2 + \lvert d\rvert^2\sigma_L^2}$	Both contribute

The combined formula reduces to the special cases when one variance is zero. The $|d|^2$ factor in the lead-time term often dominates when both are random — small percentage variance in lead time can produce **huge** swings in lead-time demand.

196.4.3. Safety stock = $z\sigma_{\text{LD}}$

Once you know σ_{LD}, the safety stock formula is:

$$\text{SS} = z \cdot \sigma_{\text{LD}}$$

Choose z from the desired service level (see [cycle_service_level.typ](../service_levels/cycle_service_level.typ) and [fill_rate.typ](../service_levels/fill_rate.typ)).

196.4.4. Risk pooling reduces safety stock

When inventory is held centrally rather than distributed across n independent locations:

- **Decentralized**: each location holds $z\sigma_d\sqrt{L}$, total = $n \times z\sigma_d\sqrt{L}$.
- **Centralized**: one location holds $z\sigma_d\sqrt{L}\sqrt{n}$ (because aggregate variance scales with n, std with $\sqrt{n}$).

Total safety stock falls by factor $\sqrt{n}$ under centralization (assuming independent demands across locations). This is the **square-root law** of inventory aggregation.

Example

Given (compare four scenarios):

- $|d| = 33$ units / day
- $|L| = 14$ days
- $z = 1.65$ (95% CSL)

Case 1 — constant demand, constant lead time

No variance. **Safety stock = 0**. Just hold $|d||L| = 462$ units of cycle/pipeline; no buffer.

Case 2 — variable demand, constant lead time ($\sigma_d = 5$, $\sigma_L = 0$)

$$\sigma_{\text{LD}} = \sqrt{L} \cdot \sigma_d = \sqrt{14} \cdot 5 \approx 18.7$$

$$\text{SS} = z\sigma_{\text{LD}} = 1.65 \cdot 18.7 \approx 31 \text{ units}$$

Case 3 — constant demand, variable lead time ($\sigma_d = 0$, $\sigma_L = 3$ days)

$$\sigma_{\text{LD}} = |d| \cdot \sigma_L = 33 \cdot 3 = 99$$

$$\text{SS} = 1.65 \cdot 99 \approx 163 \text{ units}$$

Lead time variability dominates demand variability in this case — same z, same demand, but 5× higher safety stock. Because $|d| = 33$ is much larger than $\sigma_d = 5$, the multiplier $|d|$ amplifies σ_L.

Case 4 — variable demand AND variable lead time

$$\sigma_{\text{LD}} = \sqrt{L\sigma_d^2 + |d|^2\sigma_L^2} = \sqrt{14 \cdot 25 + 33^2 \cdot 9} = \sqrt{350 + 9801} = \sqrt{10151} \approx 100.8$$

$$\text{SS} = 1.65 \cdot 100.8 \approx 166 \text{ units}$$

Combined std is **barely larger** than the lead-time-only case — when one source of variance dominates, the other contributes little. Reducing the bigger one (lead-time variability here) gives the biggest safety-stock savings.

Practical takeaway

- Reduce demand-side uncertainty → modest safety-stock savings.
- Reduce **lead-time** uncertainty → often dramatic savings (because $|d|$ multiplies σ_L).
- Centralize inventory → safety stock shrinks by $\sqrt{n}$ (risk pooling).

196.5. Anticipation Stock

Inventory built up **in advance** of a **known** future demand spike. Distinct from safety stock (which buffers **unknown** uncertainty) — anticipation stock buffers **known** but **concentrated** demand.

196.5.1. When you build it

Three classic situations:

1. **Seasonal demand**: ice cream in summer, snowblowers in winter, retail before December. Production capacity is roughly flat year-round but demand is concentrated in a few months. Build inventory in low-demand months and sell it in peak months.
2. **Promotional events**: Black Friday, product launch, advertised sale. Known spike, fixed date. Build before to avoid stockouts during the event.
3. **Planned supply disruption**: known upcoming shutdown (factory maintenance, holiday closure, supplier plant retooling). Build before the gap to maintain customer service through it.

In each case, future demand exceeds future production capacity, so you **can't** meet it just-in-time. Build early.

196.5.2. The build-vs-capacity trade-off

You face two strategies:

Strategy	Pros	Cons
Build anticipation stock	Steady production rate, no capacity expansion needed	Holding cost on the buffer; risk if forecast is wrong
Build to peak capacity	Match production to demand; no excess inventory	High peak capacity → idle capacity off-peak; may need overtime, second shifts, or capital expansion

The right answer depends on the **cost ratio**:

- Cheap to hold inventory + expensive to expand capacity → build anticipation stock.
- Cheap to add capacity (e.g., outsourced labor) + expensive to hold → match capacity to demand.

Most operations use a **blend**: a partial buffer plus some capacity flexibility.

196.5.3. Sizing anticipation stock

Given:

- Production rate p (units / period, what your plant can produce)
- Demand rate by period: d_t, with peak $d_{\text{peak}} > p$

The cumulative gap between demand and production over the peak window must be covered by the buffer:

$$\text{Anticipation stock}_{\text{start}} = \sum_{t \in \text{ peak window}} \max(0, d_t - p)$$

Build this buffer up **before** the peak window starts, then drain during the peak.

196.5.4. When forecast is wrong

Anticipation stock is forecast-driven. If forecast is off:

- **Underbuilt** (forecast too low): stockout in peak. Recovery options: emergency overtime, third-party fulfillment, customer disappointment.
- **Overbuilt** (forecast too high): leftover inventory. Carry to next period, mark down, or write off. Common with seasonal apparel.

This is why fashion retailers struggle so much — anticipation stock + multiplicative demand uncertainty + short selling window. They blend anticipation (initial commitment) with reactive replenishment (postponement strategies, fast fashion).

196.5.5. How it composes with other stock types

Most operations have ZERO anticipation stock most of the time — it builds and drains around specific events. Different from cycle/safety/pipeline stock, which exist continuously.

Total inventory at a moment in time:

$$\text{Total inventory} = \text{Cycle stock} + \text{Safety stock} + \text{Pipeline stock} + \text{Anticipation stock} + \text{Decoupling stock}$$

Anticipation is “spiky” — zero most of the year, large around events.

Example: Seasonal anticipation stock for a snowblower factory

Given:

- Annual demand: 12,000 units
- Demand concentrated in November-February (4 months): 9,600 units (80% of annual)
- Off-peak (March-October, 8 months): 2,400 units (20%)

- Production capacity: 1,000 units / month (12,000/yr — exactly matches average)
- Holding cost: \$2 / unit / month

Step 1 — peak vs production

Peak demand: $\frac{9600}{4} = 2400$ units / month. Production: 1000 units / month. **Gap**: 1400 units / month for 4 months = 5,600 units total **cannot be made just-in-time**.

Step 2 — anticipation stock requirement

Build 5,600 units before November 1. Production capacity in March-October must support both off-peak demand AND building the buffer:

$$\text{Off-peak production needed} = \text{off-peak demand} + \text{buffer} = 2400 + 5600 = 8000 \text{ units}$$

Over 8 off-peak months: $\frac{8000}{8} = 1000$ units / month — **exactly the capacity**.

Convenient: with capacity matched to average demand, anticipation stock fills exactly the off-peak idle capacity.

Step 3 — anticipation-stock holding cost

Buffer builds linearly from 0 (March) to 5,600 (November), then drains linearly to 0 (March again). Average level over the 12-month cycle:

$$\overline{\text{anticipation}} \approx \frac{1}{2} \cdot \frac{1}{2} \cdot 5600 = 1400 \text{ units (rough)}$$

More precisely: the integral of a triangle of height 5600 over 12 months / 12 months = $5600 \cdot \frac{\frac{6}{12}}{2} = 1400$ units average.

Annual holding cost: $h_{\text{month}} \cdot 1400 \cdot 12 = 2 \cdot 1400 \cdot 12 =$ \$33,600 / year.

Step 4 — alternative: build peak capacity instead

Add capacity to make 2,400 units / month (matching peak). Idle most of the year. Capacity costs (extra equipment, plant, labor): say \$100,000 capital cost, depreciated over 10 years → \$10,000/year + variable labor.

Compare:

- Anticipation: \$33,600/year holding cost.
- Capacity expansion: \$10,000+/year capital + idle-time cost.

Capacity expansion wins. But the calculus depends on:

- Higher holding cost → favor capacity.
- Higher capacity cost → favor anticipation.
- More uncertainty → favor capacity (less risk on forecast error).

Real operations almost always **blend**: some anticipation, some flex capacity, some safety stock for forecast error.

196.6. Decoupling Stock

Inventory held **between two production stages** (or between a supplier and a buyer) so each stage can operate independently — without being immediately blocked by problems in the upstream stage.

Conceptually similar to safety stock, but for **internal** stages rather than external customer demand.

196.6.1. The problem decoupling solves

Consider a two-stage line: stage A feeds stage B. If they operate strictly tied (no buffer between):

- Any **slowdown** at A immediately starves B.
- Any **defect** at A goes straight into B.
- Any **breakdown** at A halts the whole line.

Total throughput is bounded by the **minimum** of the two stages' instantaneous availability — not the average. This compounds dramatically with more stages.

Decoupling stock = a buffer between A and B. A pile up parts; B pulls from the pile.

- A stops for an hour: B keeps running off the pile.
- B stops for an hour: A keeps producing onto the pile.

Each stage can handle **its own** short-term variation independently.

196.6.2. When decoupling matters most

The cost of **not** decoupling depends on:

- Variability at each stage (downtime, scrap, cycle-time variation).
- Interaction effects (line balancing, dependence on shared resources).

Decoupling stock is most valuable when:

- Stages have **different** batch sizes or tact times.
- Upstream is unreliable (old equipment, manual operations, supplier delivery variability).
- Downstream demand is more variable than upstream production.

It's **least** valuable in a perfectly flow-balanced, ultra-reliable, automated environment — exactly the conditions Toyota engineered for, which is why TPS deliberately **reduces** decoupling stock to expose problems.

196.6.3. Toyota's view: stock hides problems

Lean / TPS treats decoupling stock as **waste**. A buffer between A and B disguises A's reliability problems — the line keeps running, but you never feel pressure to fix the root cause.

Toyota's approach:

- Reduce decoupling stock.
- Expose the variability problems that emerge.
- Fix the upstream stages until they don't need a buffer.
- Repeat with smaller buffers.

The famous "lower the water to expose the rocks" metaphor: stock = water level, problems = rocks. Lean reduces stock to surface problems and force solutions.

196.6.4. Sizing decoupling stock

Various models:

- **Insurance buffer**: cover expected downtime per cycle. If A breaks down on average τ minutes per shift, hold $\tau \cdot d_B$ units (B's consumption rate × A's expected downtime).
- **Service-level approach**: target P(B never starves) = some level → use safety-stock formulas with $\sigma =$ variability of A's output.
- **Optimization**: balance holding cost of decoupling stock against the expected cost of B starving.

Closed forms exist for some special cases (e.g., M/M/1 stages); more often simulated.

196.6.5. How it composes

Component	Magnitude	Where it lives
Cycle stock	$Q/2$	At each stocking location
Safety stock	$z\sigma_{\text{LD}}$	At each stocking location
Pipeline stock	$\|d\|\|L\|$	In transit between stages
Anticipation stock	planned	Centralized for known events
Decoupling stock	varies (insurance / SS-like)	Between production stages — physically a queue or buffer area

Example: Two-stage manufacturing line

Given:

- Stage A: machining. Cycle time 60 sec/unit. **Average** downtime 5% (3 min/hour).
- Stage B: assembly. Cycle time 60 sec/unit. **Reliable** (negligible downtime).
- Both stages run 8 hours/day.

Step 1 — without decoupling

Tied stages: A's downtime **immediately** idles B. Effective throughput = 0.95 × nominal capacity (A's availability dominates because B has no buffer).

At 60 sec/unit: 480 units/day at full capacity → 456 units/day with 5% A-downtime.

Step 2 — small decoupling buffer

Add a 30-unit buffer between A and B.

Now A's downtime affects B **only if** A is down longer than the buffer can sustain B (30 units × 60 sec = 30 minutes).

In a typical day, A's downtime is split into many short events (e.g., 12 events × 15 sec each rather than one 3-minute outage). A 30-unit buffer easily absorbs all of them. Effective throughput approaches 480/day.

Cost: holding 30 units of WIP at, say, \$20/unit $\Rightarrow$ \$600 capital + small holding cost.
Throughput gain: 24 units/day $\Rightarrow$ over a year, $24 \cdot 250 = 6000$ extra units valued at margin.

Step 3 — Toyota approach: shrink the buffer

Toyota would target **zero** buffer — and then attack the 5% A-downtime root cause until it disappears (preventive maintenance, redesign, jidoka). Once A is reliable enough, the buffer can come out.

The general decoupling-stock decision

At each stage boundary:
1. Estimate the cost of stage starvation (lost throughput × margin).
2. Estimate the cost of holding the buffer.
3. Set buffer size to balance.
4. (If pursuing lean): reduce variability at upstream stages until the buffer is no longer needed, then remove it.

Decoupling stock is the only inventory category Toyota systematically **eliminates** rather than optimizes.

196.7. Pipeline Stock

Inventory that is **in transit** — already ordered and paid for, but not yet received. Sometimes called "in-transit inventory" or "work-in-progress" (WIP) when between production stages.

$$\text{Pipeline stock} = |d| \cdot |L|$$

Average lead-time demand. By Little's Law applied to the inventory system.

196.7.1. Where it comes from

Once you place an order, those units are **yours** (you've paid, they're in the supply chain) — even though they're not on your shelf yet. They're physically held by:

- The supplier (after order placement, before shipment)
- The carrier (in trucks, ships, planes during transit)
- Customs (during import processing)

The financial cost is real: working capital is tied up the entire time.

196.7.2. Derivation via Little's Law

For any stable queue:

$$\text{Average \# in queue} = \text{Arrival rate} \times \text{Average time in queue}$$

Apply to the "in-transit" queue:

- Arrival rate into transit: $|d|$ (units/time, equal to average demand rate).
- Time in transit: $|L|$ (the lead time).

$$\text{Pipeline stock} = |d| \cdot |L|$$

Independent of the order quantity Q or the policy used. **Every** unit you ever consume passes through the pipeline for L time, so on average $|d||L|$ units are always in flight.

196.7.3. Cost of pipeline stock

Holding cost depends on whether you're charged carrying cost on in-transit inventory:

- **Yes** (you've paid the supplier — common with prepay terms): annual cost $h \cdot |d||L|$.
- **No** (consignment / pay-on-receipt): pipeline stock has zero holding cost to you. Common with VMI.

196.7.4. Reduce pipeline stock

The **only** way to reduce pipeline stock is to reduce lead time L:

- Shorter shipping (air vs ocean)
- Closer suppliers (reshoring / nearshoring)
- Skip intermediate inspection / customs steps
- Compress production lead times (lean manufacturing within the supplier)

Cycle stock and safety stock can be tuned by changing Q or z. **Pipeline stock cannot** — it's locked in by lead time and demand rate.

196.7.5. How it composes with other stock types

Component	Magnitude	Reduce by
Cycle stock	$Q/2$	Smaller Q (reduce setup cost S)
Safety stock	$z\sigma_{\text{LD}}$	Higher service or lower variance
Pipeline stock	$\|d\|\|L\|$	Shorter lead time **only**
Anticipation stock	planned	Smoother demand or flexible capacity
Decoupling stock	planned	Tighter inter-stage coordination

Example

Given (shared params):

- Daily demand: $|d| = 33$ units / day
- Lead time: $|L| = 14$ days
- Unit value c = \$10, holding rate $i = 0.20$/year so $h = i \cdot c$ = \$2/unit/year

Step 1 — pipeline stock

$$\text{Pipeline} = |d| \cdot |L| = 33 \cdot 14 \approx 462 \text{ units}$$

This is the average number of units in transit at any moment.

Step 2 — annual cost (assuming you pay upfront)

$$\text{Annual cost}_{\text{pipeline}} = h \cdot 462 =$$

\$924 / year

Step 3 — compare to total inventory

At a (Q, r) policy with $Q = 775$ and 95% CSL ($r = 493$):

- Cycle stock: $Q/2 = 388$ units
- Safety stock: $z\sigma_L = 31$ units
- Pipeline stock: 462 units (this!)
- Total average inventory: $388 + 31 + 462 = 881$ units

Pipeline is the largest component — bigger than cycle stock or safety stock. It's invisible on warehouse counts but very real on the balance sheet.

Step 4 — what does halving lead time do?

Shrink L from 14 days to 7 days (e.g., switch to air freight):

- Pipeline: $33 \cdot 7 = 231$ units (-50%, **–231 units**)
- Safety stock: $1.65 \cdot 5\sqrt{7} \approx 22$ units (slight reduction from $\sqrt{L}$ effect)
- Cycle stock: unchanged (still $Q/2 = 388$)

Halving lead time **cut pipeline + safety stock by 38%**. The savings on holding cost: $h \cdot (462 - 231 + 31 - 22) = 2 \cdot 240 =$ \$480 / year.

Trade-off: air freight may cost \$3000/year more. Whether it's worth it depends on the unit value (high-value goods → easier to justify shorter lead time).

Lean / Toyota argument: lead-time reduction simultaneously cuts pipeline AND safety stock AND working capital tied up. That's why "compress the value stream" is a core lean principle.

197. Newsvendor

197.1. Overview

197.1.1. Newsvendor

Single-period inventory under uncertain demand: order Q **before** seeing demand. After demand realizes, leftover stock is salvaged at $S < C$, and unmet demand is lost.

The trade-off:

- Order **too much** → leftovers, lose $C - S$ per unsold unit.
- Order **too little** → stockouts, lose $P - C$ per missed sale.

197.1.1.1. Assumptions

- Single period (no carryover to next period)
- Single product
- Demand D is random with known distribution F
- Lead time is zero — receive order before demand realizes
- Linear underage and overage costs

197.1.1.2. Setup

- P = sale price per unit

- C = purchase cost per unit
- S = salvage value per unsold unit (with $S < C$)
- D = random demand, with CDF $F(d) = P(D \leq d)$ and density f
- Q = order quantity (decision variable)

Define:

- **Underage cost** (lost margin per missed sale): $C_u = P - C$
- **Overage cost** (loss per unsold unit): $C_o = C - S$

197.1.1.3. Derive Q^* via marginal analysis

Imagine we've decided to order Q units. Should we order one more?

The $(Q+1)$-th unit either **sells** or **doesn't**:

- It **sells** if demand exceeds Q, with probability $P(D > Q) = 1 - F(Q)$. Profit: C_u per unit (margin we'd otherwise lose).
- It **doesn't sell** if demand $\leq Q$, with probability $F(Q)$. Cost: C_o per unit (overage).

Expected marginal profit of ordering one more unit:

$$\Delta(Q) = (1 - F(Q))C_u - F(Q)C_o$$

Order more units while $\Delta(Q) > 0$, stop when $\Delta(Q) = 0$:

$$(1 - F(Q^*))C_u = F(Q^*)C_o$$

$$C_u - F(Q^*)C_u = F(Q^*)C_o$$

$$C_u = F(Q^*)(C_u + C_o)$$

$$F(Q^*) = \frac{C_u}{C_u + C_o}$$

This ratio is the **critical ratio** (CR). Read it as: "What probability of **not** stocking out balances the two costs?"

197.1.1.4. Inverting F to get Q^*

$$Q^* = F^{-1}(\text{CR}) = F^{-1}\left(\frac{C_u}{C_u + C_o}\right)$$

For **any** demand distribution, the recipe is the same: compute CR, look up the quantile.

197.1.1.5. Normal demand specialization

If $D \sim \mathcal{N}(\mu, \sigma^2)$, write $Q^* = \mu + z^*\sigma$ where $z^* = \Phi^{-1}(\text{CR})$ is the standard-normal quantile of CR.

Common z^* values:

- $\text{CR} = 0.5 \to z^* = 0.00$ (order the median)
- $\text{CR} = 0.67 \to z^* \approx 0.44$
- $\text{CR} = 0.84 \to z^* \approx 1.00$
- $\text{CR} = 0.95 \to z^* \approx 1.65$
- $\text{CR} = 0.99 \to z^* \approx 2.33$

When $C_u \gg C_o$ (stockouts much worse than leftovers), CR $\to 1$ and z^* pushes you to over-stock. Conversely when $C_o \gg C_u$, CR $\to 0$ and $z^* < 0$ (under-stock the median).

Example

Given (newsstand selling newspapers):

- Sale price: P = \$3 / unit
- Purchase cost: C = \$1 / unit
- Salvage value: S = \$0 / unit (unsold papers worthless)
- Demand: $D \sim \mathcal{N}(\mu = 100, \sigma = 20)$

Step 1 — underage and overage

$$C_u = P - C = 3 - 1 = 2 \quad (\$/\text{unsold-demand-unit})$$

$$C_o = C - S = 1 - 0 = 1 \quad (\$/\text{leftover})$$

Step 2 — critical ratio

$$\text{CR} = \frac{C_u}{C_u + C_o} = \frac{2}{2+1} = \frac{2}{3} \approx 0.667$$

Each unit ordered should have a $\approx 67\%$ chance of selling — overstocking is twice as cheap as stocking out.

Step 3 — quantile lookup

For normal demand, $z^* = \Phi^{-1}(0.667) \approx 0.44$:

$$Q^* = \mu + z^*\sigma = 100 + 0.44 \cdot 20 \approx 109 \text{ newspapers}$$

Step 4 — interpret

At $Q^* \approx 109$: about a 67% chance of selling out (matching CR), 33% chance of leftovers. Sensible because the underage cost ($C_u = 2$) is twice the overage cost ($C_o = 1$) — the optimum biases toward stocking **more** than the mean (100).

newsvendor.py

```python
import scipy.stats as stats

P, C, S = 3, 1, 0  # sale price, purchase cost, salvage value
mu, sigma = 100, 20  # demand mean and standard deviation

C_u = P - C  # underage cost
C_o = C - S  # overage cost
CR = C_u / (C_u + C_o)  # critical ratio

z_star = stats.norm.ppf(CR)  # quantile
Q_star = mu + z_star * sigma
```

197.1.1.6. Derivation via calculus (integral form)

The marginal argument above has a continuous twin: write expected cost as a function of Q and minimize directly. With overage C_o and underage C_u:

$$\mathcal{C}(Q) = C_o \cdot \mathbb{E}[(Q-D)^+] + C_u \cdot \mathbb{E}[(D-Q)^+]$$

Rewrite the expectations as integrals. Each $(\cdot)^+$ clips the integrand to one side of Q:

$$\mathbb{E}[(Q-D)^+] = \int_0^Q (Q-x) f(x)\,\mathrm{d}x, \qquad \mathbb{E}[(D-Q)^+] = \int_Q^\infty (x-Q) f(x)\,\mathrm{d}x$$

$$\mathcal{C}(Q) = C_o \int_0^Q (Q-x) f(x)\,\mathrm{d}x + C_u \int_Q^\infty (x-Q) f(x)\,\mathrm{d}x$$

Differentiate (Leibniz's rule — the boundary terms vanish because the integrand is 0 at $x = Q$):

$$\frac{\mathrm{d}\mathcal{C}}{\mathrm{d}Q} = C_o F(Q) - C_u(1 - F(Q))$$

Set to zero and solve:

$$C_o F(Q^*) = C_u(1 - F(Q^*)) \quad \Longrightarrow \quad F(Q^*) = \frac{C_u}{C_u + C_o}$$

— the same critical ratio. **Convexity check**: $(\mathrm{d}^2\mathcal{C})/(\mathrm{d}Q^2) = (C_o + C_u) f(Q) \geq 0$, so the stationary point is the global minimum.

197.2. Discrete

197.2.1. Newsvendor with discrete demand

Relax one dimension from basic newsvendor: **demand distribution** is no longer continuous. Demand D takes integer values (Poisson, empirical histograms, etc.). The critical-ratio rule still applies, but the quantile is found by a **discrete lookup** instead of inverting a continuous CDF.

197.2.1.1. Setup

Same costs as basic newsvendor:

- $C_u = P - C$ (underage)
- $C_o = C - S$ (overage)
- $\mathrm{CR} = C_u/(C_u + C_o)$

Demand $D \in \{0, 1, 2, ...\}$ with PMF p_d and CDF $F(d) = P(D \leq d) = \sum_{k=0}^{d} p_k$.

197.2.1.2. Decision rule

For continuous F, $Q^* = F^{-1}(\mathrm{CR})$. For discrete F, the CDF jumps — there may be no Q with $F(Q) = \mathrm{CR}$ exactly. Take the **smallest** Q that gets you to (or above) CR:

$$Q^* = \min\{Q \in \{0, 1, 2, ...\} : F(Q) \geq \mathrm{CR}\}$$

Equivalent statements:

- $F(Q^*) \geq \mathrm{CR}$ (don't undershoot)
- $F(Q^* - 1) < \mathrm{CR}$ (smallest such Q)

Why this is correct: the marginal-analysis derivation from the basic newsvendor still holds. The expected marginal profit of one more unit at level Q is $(1 - F(Q))C_u - F(Q)C_o$. Order while this is ≥ 0, i.e., while $F(Q) \leq \text{CR}$. Stop at the first Q where $F(Q) \geq \text{CR}$.

Example: Empirical demand from sales records

Given: 6 weeks of sales for a perishable item.

- Sale price: P = \$10
- Purchase cost: C = \$4
- Salvage value: S = \$1
- Empirical demand counts:

Demand d	10	12	14	16	18	20
Frequency	1	1	2	1	0	1

Total weeks: 6.

Step 1 — costs and CR

$$C_u = 10 - 4 = 6, \quad C_o = 4 - 1 = 3$$

$$\text{CR} = \frac{6}{6+3} = \frac{2}{3} \approx 0.667$$

Step 2 — empirical CDF

Convert frequencies to probabilities, then accumulate:

d	10	12	14	16	18	20
p_d	$1/6$	$1/6$	$2/6$	$1/6$	0	$1/6$
$F(d)$	$1/6 \approx 0.167$	$2/6 \approx 0.333$	$4/6 \approx 0.667$	$5/6 \approx 0.833$	$5/6$	$6/6 = 1$

Step 3 — find smallest Q with $F(Q) \geq 0.667$

- $F(10) = 0.167 < 0.667$
- $F(12) = 0.333 < 0.667$
- $F(14) = 0.667 \geq 0.667$ ✓ — first to satisfy.

$$Q^* = 14 \text{ units}$$

Step 4 — sanity check via marginal analysis

At $Q = 14$, expected marginal profit of one more unit:

$$\Delta(14) = (1 - 0.667) \cdot 6 - 0.667 \cdot 3 = 2 - 2 = 0 \checkmark$$

Adding a 15th unit would yield $\Delta(15) = (1 - 0.833) \cdot 6 - 0.833 \cdot 3 = 1 - 2.5 < 0$ — not worth it.

Compare to continuous-normal baseline: empirical mean ≈ 14, std ≈ 3.4. If we had assumed normal demand:

$$Q^*_{\text{normal}} \approx 14 + 0.44 \cdot 3.4 \approx 15.5 \to 16 \text{ (rounded)}$$

The discrete answer ($Q^* = 14$) is **smaller** than the continuous approximation because the empirical CDF jumps past the threshold exactly at 14. With small samples, the discrete formula respects the actual data distribution rather than a smoothed assumption.

197.3. Expected Profit

197.3.1. Expected profit and lost sales (newsvendor)

This file doesn't relax a dimension — it computes **additional outputs** of the standard newsvendor model. Once you have Q^*, what's the expected profit, expected leftovers, expected lost sales? The answer for normal demand uses the **standard normal loss function**.

197.3.1.1. Setup

Same as basic newsvendor: $D \sim \mathcal{N}(\mu, \sigma^2)$, decision Q, prices P, C, S.

Define the **standard normal loss function**:

$$L(z) = \varphi(z) - z \cdot (1 - \Phi(z))$$

where φ is the standard normal density, Φ the CDF. $L(z)$ measures the expected shortfall above z for a standard normal — equivalently, $E[(Z - z)^+]$.

Properties:

- $L(0) \approx 0.399$
- $L(z) \to 0$ as $z \to \infty$ (no shortfall expected far in the upper tail)
- $L(z) \to -z$ as $z \to -\infty$ (everything below is "missed demand" if you over-order huge)

For computation, $L(z)$ is tabulated or computed numerically.

197.3.1.2. Expected lost sales

For demand $D = \mu + \sigma Z$ where $Z \sim \mathcal{N}(0, 1)$, lost sales when $D > Q$ amount to $D - Q$. In standard-normal units, $z = \frac{Q-\mu}{\sigma}$:

$$E[\text{lost sales}] = E[(D - Q)^+] = \sigma \cdot L(z)$$

This is the **expected shortfall** in actual units — multiply $L(z)$ by σ to scale back from standard normal.

197.3.1.3. Expected leftovers

By symmetry, expected leftovers $= E[(Q - D)^+]$. By inventory balance:

$$E[(Q - D)^+] = (Q - \mu) + E[(D - Q)^+] = (Q - \mu) + \sigma L(z)$$

197.3.1.4. Expected sales

$$E[\text{sales}] = E[\min(D, Q)] = \mu - \sigma L(z)$$

(Total demand μ minus the part that wasn't satisfied.)

197.3.1.5. Expected profit

Per-unit revenue components:

- Sales bring in $P - C$ profit (margin) per unit sold.
- Leftovers: bought at C, salvaged at S, so loss = $C - S$ per leftover.

$$E[\Pi] = (P - C)E[\text{sales}] - (C - S)E[\text{leftovers}]$$

Expand:

$$E[\Pi] = (P - C)(\mu - \sigma L(z)) - (C - S)((Q - \mu) + \sigma L(z))$$

Plug in $Q = \mu + z\sigma$:

$$E[\Pi] = (P - C)(\mu - \sigma L(z)) - (C - S)(z\sigma + \sigma L(z))$$

$$E[\Pi] = (P - C)\mu - \sigma[(P - C)L(z) + (C - S)(z + L(z))]$$

$$E[\Pi] = (P - C)\mu - \sigma[(P - S)L(z) + (C - S)z]$$

The first term is the **deterministic-demand profit** (sell exactly μ units at margin $P - C$). The second is the **stochastic penalty** — the cost of demand uncertainty, scaled by σ.

197.3.1.6. Bounded loss from uncertainty

For any z, the penalty $\sigma[(P - S)L(z) + (C - S)z]$ is **minimized** exactly when $z = z^* = \Phi^{-1}(\text{CR})$. So the optimal newsvendor minimizes the cost of uncertainty given the cost structure.

Example

Given (same newspaper baseline):

- $P = 3$, $C = 1$, $S = 0$
- $D \sim \mathcal{N}(100, 20^2)$
- $C_u = 2$, $C_o = 1$, $\text{CR} = 2/3$, $z^* = 0.44$, $Q^* \approx 109$

Step 1 — loss-function value

At $z = z^* = 0.44$:

$$L(0.44) = \varphi(0.44) - 0.44 \cdot (1 - \Phi(0.44)) \approx 0.362 - 0.44 \cdot 0.330 \approx 0.218$$

Step 2 — expected lost sales

$$E[\text{lost sales}] = \sigma \cdot L(z^*) = 20 \cdot 0.218 \approx 4.4 \text{ newspapers}$$

Even at the optimum, expect about 4 missed sales per period.

Step 3 — expected leftovers

$$E[\text{leftovers}] = (Q^* - \mu) + \sigma L(z^*) = (109 - 100) + 20 \cdot 0.218 \approx 9 + 4.4 = 13.4$$

Step 4 — expected sales

$$E[\text{sales}] = \mu - \sigma L(z^*) = 100 - 4.4 \approx 95.6 \text{ newspapers}$$

Step 5 — expected profit

$$E[\Pi] = (P - C)E[\text{sales}] - (C - S)E[\text{leftovers}]$$

$$= 2 \cdot 95.6 - 1 \cdot 13.4 = 191.2 - 13.4 \approx$$

$177.8 / period

Compare to deterministic-demand profit

If demand were exactly $\mu = 100$ every period:

$$\Pi_{\text{det}} = (P - C) \cdot \mu = 2 \cdot 100 =$$

$200 / period

The **cost of uncertainty** is $200 - 177.8 = \$22.2$ / period — that's the gap between what you could earn knowing demand exactly vs. what the optimal newsvendor strategy actually earns under the given $\sigma = 20$. Reducing demand uncertainty (better forecasting) directly recoups this gap.

197.4. Shortage Penalty

197.4.1. Newsvendor with explicit shortage penalty

Relax one dimension from basic newsvendor: **excess demand** costs more than just lost margin. In the basic model, $C_u = P - C$ is the foregone profit on a missed sale. But stockouts often have **additional** costs — customer goodwill, expedited replacement, reputation damage, contractual penalties.

Add an **explicit penalty** π per unit of unmet demand, on top of lost margin.

197.4.1.1. Setup

Same as basic newsvendor, plus:

- π = stockout penalty per unmet unit (dollars per unsold-due-to-stockout)

Underage cost now has **two components**:

$$C_u = (P - C) + \pi$$

The **form** of the answer is unchanged — only the value of C_u is larger.

197.4.1.2. Critical ratio

$$\text{CR} = \frac{C_u}{C_u + C_o} = \frac{(P - C) + \pi}{(P - C) + \pi + (C - S)}$$

A larger C_u pushes CR upward → larger z^* → larger Q^*. **More penalty for stocking out → order more.**

197.4.1.3. Bounds on CR

- $\pi = 0$: collapses to basic newsvendor.
- $\pi \to \infty$: $\text{CR} \to 1$ (almost certain to satisfy demand). Q^* grows without bound.
- π very large but finite: Q^* saturates at the upper tail of demand.

197.4.1.4. Decision rule (continuous demand)

$$Q^* = F^{-1}\left(\frac{P - C + \pi}{P - C + \pi + C - S}\right)$$

For normal demand $D \sim \mathcal{N}(\mu, \sigma^2)$:

$$Q^* = \mu + \Phi^{-1}(\mathrm{CR})\sigma$$

Example

Given (same newspaper baseline + a $\$1$ goodwill penalty per missed sale):

- Sale price: P = \$3 / unit
- Purchase cost: C = \$1 / unit
- Salvage: S = \$0
- Stockout penalty: π = \$1 / unit (lost goodwill)
- Demand: $D \sim \mathcal{N}(\mu = 100, \sigma = 20)$

Step 1 — underage and overage

$$C_u = (P - C) + \pi = 2 + 1 = 3$$

$$C_o = C - S = 1$$

Step 2 — critical ratio

$$\mathrm{CR} = \frac{3}{3 + 1} = 0.75$$

Compared to basic newsvendor ($\mathrm{CR} = 2/3 \approx 0.667$), the penalty pushes CR from 67% up to 75%.

Step 3 — quantile and order quantity

$$z^* = \Phi^{-1}(0.75) \approx 0.674$$

$$Q^* = 100 + 0.674 \cdot 20 \approx 113 \text{ newspapers}$$

Step 4 — compare to basic newsvendor

Without penalty: $Q^*_{\text{basic}} \approx 109$. With \$1 stockout penalty: $Q^*_{\pi} \approx 113$.

Order 4 more units to buy down the goodwill cost. Doubling the penalty ($\pi = \$2$) would push $z^* \approx 0.84$ and $Q^* \approx 117$.

The penalty acts like a **thumb on the scale** in favor of overstocking. In settings where stockouts are existential (medical supplies, contractual deadlines), π can dwarf $P - C$ and push the optimal Q^* deep into the upper tail of demand.

197.5. Multiplicative

197.5.1. Newsvendor with multiplicative demand

Relax one dimension from basic newsvendor: the **demand model** is no longer additive. Instead of $D = \mu + \varepsilon$ with additive noise of fixed scale σ, the demand multiplies a deterministic baseline:

$$D = \mu \cdot Z$$

where Z is a positive random factor with mean 1 and known distribution G. In additive form, $\sigma_D = \sigma_Z$ (independent of mean). In multiplicative form, $\sigma_D = \mu\sigma_Z$ — **uncertainty scales with the baseline**.

When does multiplicative make sense? Whenever demand uncertainty is **proportional** — sales of a product 10× as popular have 10× the absolute swing, not the same. Common in retail, fashion, and seasonal goods.

197.5.1.1. Setup

Same costs as basic newsvendor:

- $C_u = P - C$, $C_o = C - S$
- $\mathrm{CR} = C_u/(C_u + C_o)$

Demand: $D = \mu \cdot Z$, where $Z \sim G$ with $E[Z] = 1$.

Decision: Q, but it's natural to write $Q = \mu \cdot z$ (factor relative to baseline).

197.5.1.2. Critical-ratio derivation (unchanged)

The marginal-analysis logic from basic newsvendor is unchanged. Order one more unit while expected marginal profit is positive. The condition $F_{D(Q^*)} = \mathrm{CR}$ still holds, just rewritten in terms of Z:

$$F_{D(Q)} = P(D \le Q) = P(\mu Z \le Q) = P(Z \le Q/\mu) = G(Q/\mu)$$

Set $G(Q^*/\mu) = \mathrm{CR}$:

$$Q^* = \mu \cdot G^{-1}(\mathrm{CR})$$

197.5.1.3. Form of the answer

Write $z^* = G^{-1}(\mathrm{CR})$. Then:

$$Q^* = \mu \cdot z^* \quad (\text{multiplicative})$$

Compare to additive form:

$$Q^* = \mu + \sigma_Z z^* \quad (\text{additive})$$

Same critical ratio, same z^* logic, but **multiplicative scales Q^* as a fraction of μ** instead of adding to it.

197.5.1.4. Lognormal demand specialization

Common choice: $Z \sim \mathrm{Lognormal}(\mu_Z = 0, \sigma_Z^2)$ with $E[Z] = e^{\sigma_Z^2/2}$. Note $E[Z] \neq 1$ in general — if you want $E[Z] = 1$ exactly, use $\mu_Z = -\sigma_Z^2/2$.

In log-space, $\log Z \sim \mathcal{N}(\mu_Z, \sigma_Z^2)$. Then:

$$z^* = \exp(\mu_Z + \sigma_Z \cdot \Phi^{-1}(\mathrm{CR}))$$

Example

Given (same newspaper baseline but multiplicative noise):

- $P = 3$, $C = 1$, $S = 0 \to C_u = 2$, $C_o = 1$, $\mathrm{CR} = 2/3$
- Baseline demand: $\mu = 100$
- Multiplicative noise: Z has $E[Z] = 1$, coefficient of variation $\mathrm{CV} = \sigma_Z = 0.2$ (i.e., 20% relative noise)
- Specifically: $\log Z \sim \mathcal{N}(-0.02, 0.2^2)$ (so $E[Z] \approx 1$)

Step 1 — critical ratio

$$\mathrm{CR} = \frac{2}{3} \approx 0.667$$

(Same as basic newsvendor — the critical ratio depends on costs only.)

Step 2 — quantile in Z-space

$$z^* = \exp(-0.02 + 0.2 \cdot \Phi^{-1}(0.667)) = \exp(-0.02 + 0.2 \cdot 0.430)$$

$$z^* = \exp(-0.02 + 0.086) \approx \exp(0.066) \approx 1.068$$

Step 3 — order quantity

$$Q^* = \mu \cdot z^* = 100 \cdot 1.068 \approx 107 \text{ newspapers}$$

Step 4 — compare to basic (additive) newsvendor

In the basic example, $\sigma_D = 20$ (absolute), so $Q^* \approx 100 + 0.44 \cdot 20 \approx 109$.

In multiplicative form with $\mathrm{CV} = 0.2$, $\sigma_D = \mu \cdot 0.2 = 20$ (same absolute uncertainty **at $\mu = 100$**). $Q^* \approx 107$.

Slight difference (109 vs 107) comes from the lognormal having a different shape than normal — same coefficient of variation, but right-skewed (long right tail). The lognormal puts more probability on the right, so the upper quantile is actually less far above the mean than for a normal of the same σ.

Why multiplicative matters at scale

If baseline demand grows to $\mu = 1000$, additive-form predicts $\sigma_D = 20$ still (a 2% relative swing — implausibly small). Multiplicative-form predicts $\sigma_D = 200$ (a 20% swing, structurally consistent).

At $\mu = 1000$:

- Additive: $Q^* \approx 1000 + 0.44 \cdot 20 = 1009$ (essentially deterministic).
- Multiplicative: $Q^* \approx 1000 \cdot 1.068 = 1068$ (still 6.8% above baseline).

Choose multiplicative when uncertainty is proportional to the baseline; additive when it's a fixed scale (e.g., shipping noise, fixed measurement error).

197.6. Multi-Product

197.6.1. Multi-product newsvendor (risk pooling)

Relax one dimension from basic newsvendor: **number of items** is no longer one. Multiple products $i = 1, ..., n$ with random demands $D_i \sim \mathcal{N}(\mu_i, \sigma_i^2)$.

Two cases, dramatically different:

1. **Independent decisions, independent inventories.** Each product is its own newsvendor. Solve n separate problems.
2. **Pooled inventory.** All products share the same physical pool (interchangeable, or shipped from one location to many destinations). One newsvendor problem on the **aggregate** demand $D_{\text{agg}} = \sum_i D_i$.

Pooling is often **much better** — **risk pooling** / **demand aggregation** reduces relative uncertainty.

197.6.1.1. Independent newsvendors (case 1)

Just n separate basic newsvendors. Each has its own C_u^i, C_o^i, CR_i, Q_i^*. Total expected profit is the sum.

197.6.1.2. Pooled inventory (case 2): aggregate demand statistics

If $D_i \sim \mathcal{N}(\mu_i, \sigma_i^2)$ with pairwise correlation ρ_{ij}:

$$\mu_{\text{agg}} = \sum_{i=1}^{n} \mu_i$$

$$\sigma_{\text{agg}}^2 = \sum_{i=1}^{n} \sigma_i^2 + 2 \sum_{i<j} \rho_{ij} \sigma_i \sigma_j$$

For **independent** products ($\rho_{ij} = 0$):

$$\sigma_{\text{agg}} = \sqrt{\sum_{i=1}^{n} \sigma_i^2}$$

For **identical, independent** products ($\mu_i = \mu$, $\sigma_i = \sigma$):

$$\mu_{\text{agg}} = n\mu, \quad \sigma_{\text{agg}} = \sqrt{n} \cdot \sigma$$

197.6.1.3. Pooling reduces relative uncertainty

Coefficient of variation (CV = σ/μ) for the aggregate vs. individual:

$$\text{CV}_{\text{agg}} = \frac{\sigma_{\text{agg}}}{\mu_{\text{agg}}} = \frac{\sqrt{n}\sigma}{n\mu} = \frac{\text{CV}_{\text{individual}}}{\sqrt{n}}$$

Pooling n identical products cuts relative uncertainty by $\sqrt{n}$. Pool 4 products → half the relative noise. Pool 100 → 10× less.

This translates directly into safety stock savings: the buffer above μ scales with σ, not μ, so doubling n adds inventory only as $\sqrt{2}$, while demand grows as 2.

197.6.1.4. Pooled order decision

Same newsvendor structure, but on aggregate:

$$Q^*_{\text{agg}} = \mu_{\text{agg}} + z^* \cdot \sigma_{\text{agg}}$$

where $z^* = \Phi^{-1}(\text{CR})$ uses the pooled cost structure.

197.6.1.5. When pooling is worse

Risk pooling is **not always** beneficial:

- **Correlated demands** with $\rho_{ij} > 0$: aggregation doesn't reduce variance much. Worst case, perfect correlation ($\rho = 1$): $\sigma_{\text{agg}} = \sum_i \sigma_i$, no benefit.
- **Heterogeneous service-level requirements**: pooling forces a single CR; if products need different service levels, separate inventories may be better.
- **Substitution / customization**: physical pooling requires items to be substitutable. A red and blue jersey can't be pooled if the customer wants a specific color.

Example

Given (4 identical-ish newspaper variants, independent demands):

- 4 products, each: $P_i = 3$, $C_i = 1$, $S_i = 0$, $D_i \sim \mathcal{N}(100, 20^2)$
- Demands independent across products

Step 1 — independent newsvendors (no pooling)

Each product is its own newsvendor, identical to the basic example:

- $\text{CR} = 2/3$, $z^* = 0.44$, $Q^*_i = 100 + 0.44 \cdot 20 \approx 109$
- Total inventory: $4 \cdot 109 = \mathbf{436}$

Step 2 — pooled (case 2)

Aggregate:

- $\mu_{\text{agg}} = 4 \cdot 100 = 400$
- $\sigma_{\text{agg}} = \sqrt{4} \cdot 20 = 40$ (only $\sqrt{n}$, not n)
- $\text{CR} = 2/3$, $z^* = 0.44$
- $Q^*_{\text{agg}} = 400 + 0.44 \cdot 40 \approx \mathbf{418}$

Step 3 — compare

- Independent: 436 units total
- Pooled: 418 units total
- **Save 18 units (≈ 4%)** with the same service level.

Step 4 — see the $\sqrt{n}$ scaling

At $n = 100$ identical products:

- Independent: $100 \cdot 109 = 10900$ units
- Pooled: $\mu_{\text{agg}} + z^*\sigma_{\text{agg}} = 10000 + 0.44 \cdot 200 = 10088$ units
- **Save 7%** and the relative savings **grow with n** — at large n, pooled inventory approaches just μ_{agg} (deterministic limit), while independent inventory still carries each product's full safety buffer.

Why pooling works: when one product over-sells, another likely under-sells. Net demand is more predictable than any one product's demand. Pooling captures this — independent stocks can't.

Real-world consequence: distribution center designs, online retailers vs. physical stores, fungible commodities — all driven by the $\sqrt{n}$ risk-pooling math.

197.7. Capacity-Constrained

197.7.1. Capacity-constrained multi-product newsvendor

Relax two dimensions from basic newsvendor: **number of items** > 1 **and capacity** is constrained. Each product has its own demand and cost structure, but they share a budget, warehouse space, or other resource.

Without the constraint, each product is its own newsvendor — $Q_i^* = \mu_i + z_i^* \sigma_i$ at its own critical ratio $\mathrm{CR}_i = C_u^i / (C_u^i + C_o^i)$. With a shared constraint, those individual optima may not all be achievable — the constraint forces **trade-offs**.

197.7.1.1. Setup

n products. For each i:

- Demand $D_i \sim \mathcal{N}(\mu_i, \sigma_i^2)$
- Costs: $P_i, C_i, S_i \rightarrow C_u^i = P_i - C_i$, $C_o^i = C_i - S_i$
- Resource consumption: v_i per unit of product i (e.g., \$ if budget-constrained, ft³ if space-constrained)

Constraint:

$$\sum_{i=1}^{n} v_i Q_i \leq V$$

If individual optima already satisfy the constraint, use them. The interesting case is when $\sum_i v_i Q_i^{\mathrm{ind}} > V$ — must shrink.

197.7.1.2. Lagrangian

Maximize total expected profit subject to the capacity constraint:

$$\max_{Q_i} \sum_i E[\Pi_i(Q_i)] \quad \text{s.t.} \quad \sum_i v_i Q_i \leq V$$

where $E\left[\Pi_{i(Q_i)}\right]$ is the basic newsvendor expected profit for product i.

Lagrangian (with multiplier $\lambda \geq 0$):

$$L = \sum_i E\left[\Pi_{i(Q_i)}\right] - \lambda \left(\sum_i v_i Q_i - V \right)$$

197.7.1.3. Modified critical ratio

Take $\partial L / \partial Q_i = 0$ for each i:

$$\frac{\partial}{\partial Q_i} E[\Pi_i] = \lambda v_i$$

For continuous demand, $\partial E[\Pi_i]/\partial Q_i = \left(1 - F_{i(Q_i)}\right) C_u^i - F_{i(Q_i)} C_o^i$ (from the marginal-analysis derivation in basic newsvendor). Setting equal to λv_i:

$$\left(1 - F_{i(Q_i)}\right) C_u^i - F_{i(Q_i)} C_o^i = \lambda v_i$$

$$C_u^i - F_{i(Q_i)}(C_u^i + C_o^i) = \lambda v_i$$

$$F_{i(Q_i^*)} = \frac{C_u^i - \lambda v_i}{C_u^i + C_o^i}$$

Compare to basic: $F_{i(Q_i^{\text{basic}})} = C_u^i/(C_u^i + C_o^i)$. The constraint **reduces** each product's effective critical ratio by $\lambda v_i/(C_u^i + C_o^i)$ — products that consume more capacity per unit get **bigger** CR cuts, ordering less.

Effective per-product CR:

$$\text{CR}_i(\lambda) = \frac{C_u^i - \lambda v_i}{C_u^i + C_o^i}$$

When $\lambda v_i \geq C_u^i$, the effective CR goes negative — that product is **not worth ordering at all** given the constraint.

197.7.1.4. Find λ

λ is chosen so that $\sum_i v_i Q_i^*(\lambda) = V$. One equation, one unknown — bisection works (the LHS is monotone decreasing in λ).

197.7.1.5. Algorithm

1. Compute unconstrained $Q_i^{\text{basic}} = \mu_i + \Phi^{-1}\left(\text{CR}_i^{\text{basic}}\right)\sigma_i$.
2. Check $\sum_i v_i Q_i^{\text{basic}} \leq V$. If yes, done.
3. Else, find $\lambda > 0$ such that $\sum_i v_i Q_i^*(\lambda) = V$ (bisection).
4. Each $Q_i^* = \mu_i + \Phi^{-1}\left(\text{CR}_{i(\lambda)}\right)\sigma_i$.

Example

Given (2 newspapers competing for budget):

- Newspaper 1: $D_1 \sim \mathcal{N}(100, 20^2)$, $P_1 = 3$, $C_1 = 1$, $S_1 = 0 \rightarrow C_u^1 = 2$, $C_o^1 = 1$
- Newspaper 2: $D_2 \sim \mathcal{N}(50, 15^2)$, $P_2 = 5$, $C_2 = 2$, $S_2 = 0.5 \rightarrow C_u^2 = 3$, $C_o^2 = 1.5$
- Capacity (budget): $\sum_i C_i Q_i \leq V$ where V = \$150 (so $v_i = C_i$)

Step 1 — unconstrained per-product newsvendors

$$\text{CR}_1 = \frac{2}{3} \approx 0.667 \quad \rightarrow \quad z_1^* \approx 0.44 \quad \rightarrow \quad Q_1^{\text{ind}} \approx 100 + 0.44 \cdot 20 = 108.8$$

$$\text{CR}_2 = \frac{3}{4.5} \approx 0.667 \quad \rightarrow \quad z_2^* \approx 0.44 \quad \rightarrow \quad Q_2^{\text{ind}} \approx 50 + 0.44 \cdot 15 = 56.6$$

Budget required: $1 \cdot 108.8 + 2 \cdot 56.6 = 108.8 + 113.2 = \$222$ — **exceeds \$150 budget**. Constraint binds.

Step 2 — solve for λ

Effective critical ratios as a function of λ:

$$\mathrm{CR}_1(\lambda) = \frac{2 - \lambda \cdot 1}{3}, \quad \mathrm{CR}_2(\lambda) = \frac{3 - \lambda \cdot 2}{4.5}$$

Try $\lambda = 0.4$:

- $\mathrm{CR}_1(0.4) = \frac{1.6}{3} \approx 0.533 \to z_1^* \approx 0.082 \to Q_1 \approx 100 + 1.65 = 101.6$
- $\mathrm{CR}_2(0.4) = \frac{2.2}{4.5} \approx 0.489 \to z_2^* \approx -0.028 \to Q_2 \approx 50 - 0.42 = 49.6$
- Total budget: $1 \cdot 101.6 + 2 \cdot 49.6 = 200.8$ — still over.

Try $\lambda = 0.7$:

- $\mathrm{CR}_1(0.7) = \frac{1.3}{3} \approx 0.433 \to z_1^* \approx -0.168 \to Q_1 \approx 96.6$
- $\mathrm{CR}_2(0.7) = \frac{1.6}{4.5} \approx 0.356 \to z_2^* \approx -0.370 \to Q_2 \approx 44.5$
- Total budget: $96.6 + 89 = 185.6$ — still over.

Try $\lambda = 1.0$:

- $\mathrm{CR}_1(1.0) = \frac{1}{3} \approx 0.333 \to z_1^* \approx -0.431 \to Q_1 \approx 91.4$
- $\mathrm{CR}_2(1.0) = \frac{1}{4.5} \approx 0.222 \to z_2^* \approx -0.764 \to Q_2 \approx 38.5$
- Total budget: $91.4 + 77.0 = 168.4$ — close.

Try $\lambda = 1.15$ (interpolating):

- $\mathrm{CR}_1(1.15) = \frac{0.85}{3} \approx 0.283 \to z_1^* \approx -0.572 \to Q_1 \approx 88.6$
- $\mathrm{CR}_2(1.15) = \frac{0.7}{4.5} \approx 0.156 \to z_2^* \approx -1.013 \to Q_2 \approx 34.8$
- Total budget: $88.6 + 69.6 = 158.2$ — closer.

Try $\lambda = 1.25$:

- $\mathrm{CR}_1(1.25) = \frac{0.75}{3} = 0.25 \to z_1^* \approx -0.674 \to Q_1 \approx 86.5$
- $\mathrm{CR}_2(1.25) = \frac{0.5}{4.5} \approx 0.111 \to z_2^* \approx -1.221 \to Q_2 \approx 31.7$
- Total budget: $86.5 + 63.4 = 149.9 \approx 150$ ✓

$$\lambda^* \approx 1.25$$

Step 3 — final order quantities

$$Q_1^* \approx 87, \quad Q_2^* \approx 32$$

Both shrink below their individual optima (109, 57). Newspaper 2 shrinks proportionally more because its $v_i = 2$ is twice that of newspaper 1 — **the constraint penalizes resource-hungry items more**.

Step 4 — compare to unconstrained

- Unconstrained spend: \$222, both products near their individual optimal CR (0.667).
- Constrained spend: \$150, both products **below** their optimal CR — accepting more risk to fit the budget.

The Lagrange multiplier $\lambda^* \approx 1.25$ is the **shadow price**: each additional dollar of budget would increase total expected profit by approximately \$1.25 (until λ falls back to zero, at which point the constraint stops binding).

198. Multi-Echelon

198.1. Multi-Echelon

A supply chain with **multiple stocking points** between supply and demand — e.g., factory → central warehouse → regional DCs → retail stores. Multi-echelon inventory theory studies how to position stock optimally across these levels.

198.1.1. Why it's harder than single-echelon

Single-echelon models (EOQ, Newsvendor, (Q, r)) treat each location in isolation. Multi-echelon must handle:

- **Coupled lead times**: a downstream stockout cascades upstream
- **Allocation**: when central inventory runs short, who gets it?
- **Information flow**: does each echelon see end-customer demand or just its downstream orders?
- **Risk pooling**: see Risk Pooling

198.1.2. Two main modeling approaches

1. **Clark-Scarf** — serial system; **stochastic** customer demand; reactive policies (chase demand as it materializes). Backward DP on echelon stocks.
2. **Graves-Willems (guaranteed-service)** — general network (assembly, distribution, serial); **bounded** demand; each stage promises a service time. Forward optimization on safety-stock placement.

A third (less common):

3. **Stochastic-service** (Lee-Billington, Ettl) — like Graves-Willems but waiting times at upstream stockouts are themselves stochastic, requiring queueing-style analysis.

For **spare parts** with Poisson demand:

4. **METRIC** (Sherbrooke) — multi-echelon repairable items, uses Palm's theorem
5. **VARI-METRIC** (Graves) — METRIC with variance correction at the depot

198.1.3. Echelon stock vs installation stock

Two ways to count inventory at stage i in a serial system:

- **Installation stock** I_i — what's physically at stage i alone
- **Echelon stock** $E_i = I_i + \sum_{\text{downstream stages}} I_j + \text{in-transit}_{\text{downstream}}$ — total inventory at stage i **and** everything downstream

Echelon-stock policies (base-stock with echelon target) decouple the multi-echelon problem into a sum of single-echelon problems with adjusted penalty costs — the **Clark-Scarf decomposition**.

198.1.4. Where it matters

- **Defense logistics** — spare parts for ships, aircraft (the original METRIC application)
- **Retail / e-commerce** — DC vs store inventory
- **Manufacturing** — raw / WIP / finished-goods buffers
- **Service-parts networks** — repair / overhaul / depot

198.1.5. See also

- **Clark-Scarf**
- **Graves-Willems**
- **Risk Pooling**
- **Base Stock Policy**

198.2. Clark-Scarf

The foundational result for **stochastic** multi-echelon inventory in serial systems (Clark & Scarf 1960). Shows that the multi-echelon problem decomposes into **independent single-echelon problems** — one per stage, each solved as a newsvendor-like base-stock problem with an **induced penalty cost**.

198.2.1. Setup: serial system

Stages numbered 1 (most upstream) to N (most downstream = retailer). Customer demand D hits stage N only. Each stage:

- Holds inventory I_i
- Receives shipments from upstream stage $i-1$ with lead time L_i
- Ships to downstream stage $i+1$
- Stage N also faces backorder cost b per unit per period

Holding cost rates: h_i at stage i (assume $h_1 < h_2 < ... < h_N$ — inventory is more expensive downstream where it carries more added value).

198.2.2. Echelon stock

Define **echelon stock** at stage i:

$$E_i = I_i + \sum_{j=i+1}^{N} I_j + \text{in-transit to } j > i$$

— what's at stage i plus everything below it. Holding cost on echelon stock is incremental:

$$h_i^E = h_i - h_{i-1}$$

(with $h_0 = 0$). This is the **marginal** holding cost as inventory moves from stage $i-1$ to stage i.

198.2.3. Optimal policy: echelon base-stock

The optimal policy is **echelon base-stock**: at each review, stage i orders to bring its echelon stock up to a target S_i^*.

The Clark-Scarf decomposition shows that S_i^* at each stage solves a **single-stage newsvendor problem** with:

- **Demand**: lead-time demand D_{L_i} from the most upstream (stage 1) feeding through
- **Underage cost**: an **induced penalty** $\hat{b}_i$ that propagates upstream
- **Overage cost**: h_i^E (incremental echelon holding cost)

198.2.4. Induced penalty

The recursion (downstream → upstream):

$$\hat{b}_N = b \qquad \text{(retailer faces real backorder cost)}$$

$$\hat{b}_{i-1} = E\Big[\big(D_{L_i} - \text{shortage at stage } i\big)^+\Big] \cdot \hat{b}_i + h_i^E \cdot \ldots$$

Formally: each stage's penalty depends on the optimal cost-to-go from downstream — a backward DP. The decomposition magic is that, despite this coupling, the **form** of each stage's optimal policy is base-stock with parameters depending only on local data + the induced penalty.

198.2.5. Worked sketch (2-stage)

Stages: warehouse (1) → retailer (2). L_1, L_2 lead times. $h_1, h_2 = h_1^E + h_2^E$. Backorder cost b at retailer.

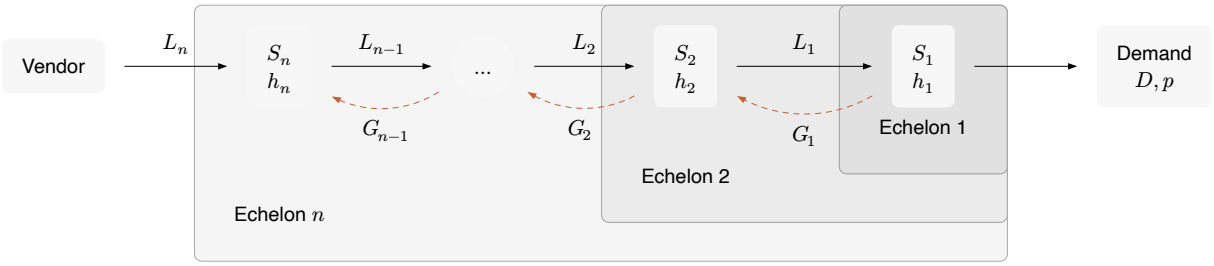
Retailer (stage 2): classical newsvendor with critical ratio $\frac{b}{b+h_2^E}$. Solve for S_2^*.

Warehouse (stage 1): newsvendor with critical ratio $\frac{\hat{b}_1}{\hat{b}_1+h_1^E}$ where $\hat{b}_1$ is the expected cost the warehouse imposes on the retailer when warehouse stockouts cause retailer delays.

198.2.6. Penalty-function recursion

For this construction, index stages 1 = downstream (demand-facing) to n = upstream (vendor) — the reverse of the setup above.

Consider a serial system with n stages, where stage 1 faces stochastic customer demand and each is replenished from stage $j+1$ with deterministic lead time L_j:



Solve stages $1 \to 2 \to n$. Each stage j does three things:

- receives the penalty function from below
- solves its own newsvendor balance

- produces a new penalty function to pass up

Base case (stage 1). No stage below it, so $G_0 = 0$. Stage 1's expected cost as a function of its echelon level y:

$$C_1(y) = h_1 \mathbb{E}\left[\left(y - D_{L_1}\right)^+\right] + p\mathbb{E}\left[\left(D_{L_1} - y\right)^+\right]$$

where D_{L_1} is demand over the lead-time window. This is convex; its minimizer is the fractile:

$$S_1 = F_{L_1}^{-1}\left(\frac{p}{p + h_1}\right)$$

Inductive step (stage j). Stage j has received G_{j-1} from below. Its cost combines its own echelon holding with the penalty it inflicts downstream when it can't fully supply:

$$C_j(y) = h_j \mathbb{E}\left[\left(y - D_{L_j}\right)^+\right] + \mathbb{E}\left[G_{j-1}\left(y - D_{L_j}\right)\right]$$

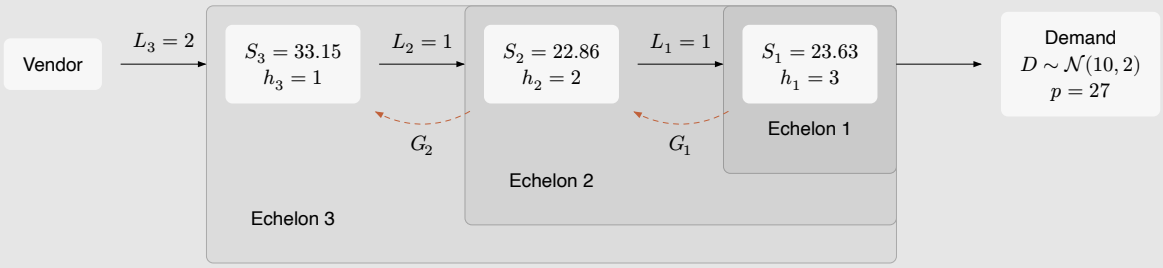
Minimize over y to get $S_j = \arg\min_y C_j(y)$.

Produce the penalty to pass up. The clamp-and-floor operation:

$$G_j(x) = C_j(\min(x, S_j)) - C_j(S_j)$$

Zero when $x \geq S_j$ (stage j isn't constrained), positive and rising as x drops below S_j. Hand G_j to stage $j + 1$.

Example: 3-stage serial system



Stage 1. Window $L_1 + 1 = 2$, fractile $\frac{27}{27+3} = 0.9$, $z = 1.282$:

$$S_1 = 10(2) + 1.282 \cdot 2\sqrt{2} = 20 + 2.63 \approx 23.63$$

Stage 2. Window $L_2 + 1 = 2$, fractile $\frac{27}{27+3+2} = \frac{27}{32} = 0.844$, $z = 1.011$:

$$S_2 = 20 + 1.011 \cdot 2\sqrt{2} = 20 + 2.86 \approx 22.86$$

Stage 3. Window $L_3 + 1 = 3$, fractile $\frac{27}{27+6} = \frac{27}{33} = 0.818$, $z = 0.908$:

$$S_3 = 10(3) + 0.908 \cdot 2\sqrt{3} = 30 + 3.25 \approx 33.15$$

The local stock held at each stage is the echelon-difference: $S_3 - S_2$ at stage 3, $S_2 - S_1$ at stage 2, S_1 at stage 1.

The fractile formula $p/\left(p + \sum_{i<j} h_i\right)$ with window $L_j + 1$ is the exact stage-1 newsvendor and a standard approximation upstream. The **exact** Clark-Scarf S_2, S_3

come from minimizing $C_j(y) = h_j \mathbb{E}\left[\left(y - D_{L_j}\right)^+\right] + \mathbb{E}\left[G_{j-1}\left(y - D_{L_j}\right)\right]$ numerically, since G_{j-1} isn't a clean newsvendor cost.

```python
# pip install stockpyl
from stockpyl.ssm_serial import optimize_base_stock_levels

# Nodes indexed 1=downstream(demand-facing) ... 3=upstream(vendor-facing)
S_star, C_star = optimize_base_stock_levels(
    num_nodes=3,
    echelon_holding_cost=[3, 2, 1],   # h_1, h_2, h_3
    stockout_cost=27,                 # p, charged at downstream node
    lead_time=[1, 1, 2],              # L_1, L_2, L_3 (into each stage)
    demand_mean=10,
    demand_standard_deviation=2,
)
print(S_star)   # exact echelon base-stock levels
print(C_star)   # optimal expected cost
```

198.2.7. Why echelon stocks decouple the problem

If stage i's order pulls from infinite supply (or upstream is always available), each stage is independent. Real life: upstream isn't always available. The induced penalty **charges** upstream for the downstream cost of its stockouts — capturing the coupling exactly.

198.2.8. Limitations

- **Serial systems only**: pure chains, no branching
- **Stationary demand**: i.i.d. demand each period
- **Costly to compute** for many stages and long horizons

For general networks, see Graves-Willems guaranteed-service.

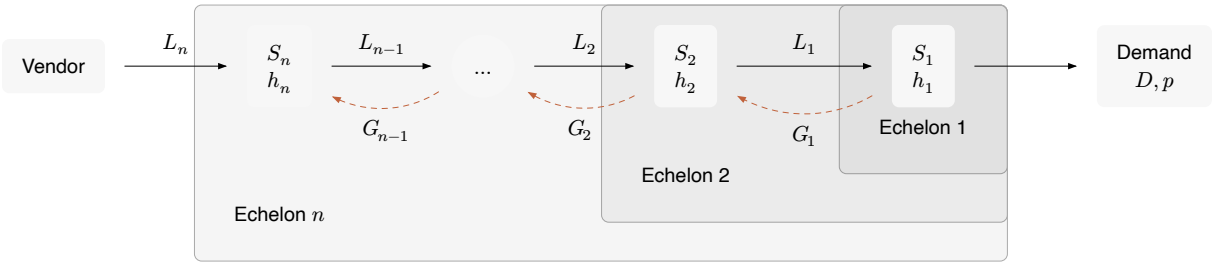
198.2.9. See also

- **Multi-Echelon** — overview
- **Graves-Willems** — alternative for general networks
- **Base Stock Policy**
- **Newsvendor**

198.3. Multi-Echelon Inventory

Setup:

Consider a serial system with N stages, where stage N faces stochastic customer demand and each replenished from stage $j+1$ with deterministic lead time L_j:



Solve stages $1 \to 2 \to n$. Each stage j does three things:

- receives the penalty function from below
- solves its own newsvendor balance
- produces a new penalty function to pass up

Base case (stage 1). No stage bellow it, so $G_0 = 0$. Stage 1's expected cost as a function of its echelon level y:

$$C_1(y) = h_1 \mathbb{E}\left[\left(y - D_{L_1}\right)^+\right] + p\mathbb{E}\left[\left(D_{L_1} - y\right)^+\right]$$

Where D_{L_1} is demand over the lead-time window. This is convex; its minimizer is the fractile:

$$S_1 = F_{L_1}^{-1}\left(\frac{p}{p + h_1}\right)$$

Inductive step (stage j). Stage j has received G_{j-1} from bellow. It's cost combines its own echelon holding with the penalty it inflicts downstream when it can't fully supply:

$$C_j(y) = h_j \mathbb{E}\left[\left(y - D_{L_j}\right)^+\right] + \mathbb{E}\left[G_{j-1}\left(y - D_{L_j}\right)\right]$$

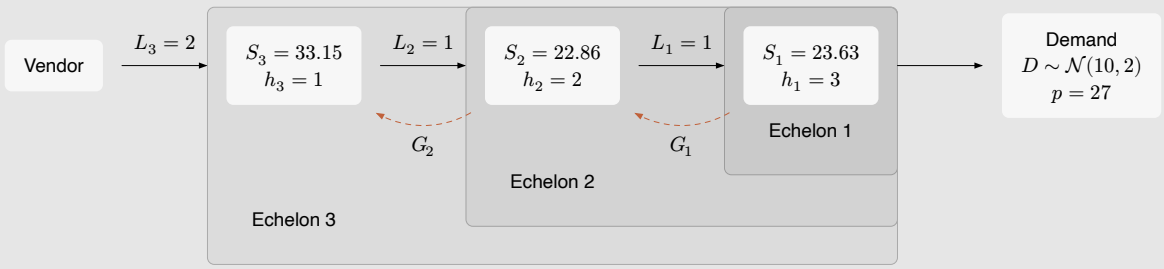
Minimize over y to get $S_j = \arg\min_y C_j(y)$.

Produce the penalty to pass up. The clamp and floor operation:

$$G_j(x) = C_j\left(\min\left(x, S_j\right)\right) - C_j\left(S_j\right)$$

Zero when $x \geq S_j$ (stage j isn't constrained), positive and rising as x drops below S_j. Hand G_j to stage $j + 1$.

Example:



Stage 1. Window $L_1 + 1 = 2$, fractile $\frac{27}{27+3} = 0.9$, $z = 1.282$

$$S_1 = 10(2) + 1.282 \cdot 2\sqrt{2} = 20 + 2.63 \approx 23.63$$

Stage 2. Window $L_2 + 1 = 2$, fractile $\frac{27}{27+3+2} = \frac{27}{32} = 0.844$, $z = 1.011$

$$S_2 = 20 + 1.011 \cdot 2\sqrt{2} = 20 + 2.86 \approx 22.86$$

Stage 3. Window $L_3 + 1 = 3$, fractile $\frac{27}{27+6} = \frac{27}{33} = 0.818$, $z = 0.908$

$$S_3 = 10(3) + 0.908 \cdot 2\sqrt{3} = 30 + 3.25 \approx 33.15$$

- $S_3 - S_2$at stage 3
- $S_2 - S_1$ at stage 2
- S_1 at stage 1

Note:

The fractile formula $\frac{p}{p+\sum_{i<j} h_i}$ with window $L_j + 1$ is the exact stage-1 newsvendor and a standard approximation for upstream stages. The **exact** Clark-Scarf S_2, S_3 come from minimizing $C_j(Y) = h_j \mathbb{E}\left[\left(y - D_{L_j}\right)^+\right] + \mathbb{E}\left[G_{j-1}\left(y - D_{L_j}\right)\right]$ numerically, since G_{j-1} isn't a clean newsvendor cost.

```python
# pip install stockpyl
from stockpyl.ssm_serial import optimize_base_stock_levels

# Nodes indexed 1=downstream(demand-facing) ... 3=upstream(vendor-facing)
S_star, C_star = optimize_base_stock_levels(
    num_nodes=3,
    echelon_holding_cost=[3, 2, 1],   # h_1, h_2, h_3
    stockout_cost=27,                 # p, charged at downstream node
    lead_time=[1, 1, 2],              # L_1, L_2, L_3 (into each stage)
    demand_mean=10,
    demand_standard_deviation=2,
)
print(S_star)   # exact echelon base-stock levels
print(C_star)   # optimal expected cost
```

198.4. Clark-Scarf view (stochastic service)

Echelon inventory position at stage j:

Demand

- μ: 100 units
- σ: 20 units

Lead time:

- $L_{f,w}$: 4 days
- L_f: 5 days

The question: **where do we keep safety stock?**

Safety stock formula:

$$\text{SS} = k \cdot \sigma \cdot \sqrt{L}$$

With $k = 2$ ($\approx 9\%$ service level), $\sigma = 20$, $L = 4$ days:

$$\mathrm{SS}_w = 2 \cdot 20 \cdot \sqrt{4} = 80 \text{ units}$$

198.5. Graves-Willems

The **guaranteed-service** multi-echelon inventory model (Graves & Willems 2000). Each stage promises a **service time** (max lead time it'll deliver in) to its downstream customer. Safety stocks are placed to make those service times achievable given bounded demand.

Used by Procter & Gamble, Hewlett-Packard, Intel, and most modern supply-chain network design software.

198.5.1. Setup

A network of stages (general topology — serial, assembly, distribution, or mixed). For each stage i:

- T_i: internal processing / production time at the stage
- s_i: **outbound service time** to downstream — what stage i commits to deliver in
- S_i: **inbound service time** from upstream — what stage i's suppliers commit to deliver in

Customer-facing stages have $s = 0$ (instant fulfillment) typically.

198.5.2. Net replenishment time

Stage i's **net replenishment time** — how long it must cover with safety stock:

$$\tau_i = S_i + T_i - s_i$$

— inbound time + processing time – outbound promise time. If you can absorb supplier and processing delays without breaking your downstream promise, τ_i is small (less safety stock needed).

Constraint: $\tau_i \geq 0$ (can't promise faster than the work takes).

198.5.3. Safety stock at each stage

Assume demand at the customer-facing stage is bounded by some upper-tail quantile (typical: $\mu + z\sigma$ over the relevant interval). Then **safety stock at stage i**:

$$\mathrm{SS}_i = z\,\sigma_i\,\sqrt{\tau_i}$$

— the standard square-root-of-lead-time form, but with τ_i (net replenishment time) instead of pure lead time. The `sqrt` comes from the variance of demand over a time interval scaling linearly.

Total safety-stock cost across the network:

$$C = \sum_i h_i\,z\,\sigma_i\,\sqrt{\tau_i}$$

198.5.4. Optimization

Decision variables: the inbound / outbound service times s_i, S_i at every stage.

Constraints:

- $s_i, S_i \geq 0$
- $\tau_i = S_i + T_i - s_i \geq 0$
- $S_i = \max_{j \in \text{ upstream}(i)} s_j$ (the inbound time at i is the slowest outbound from any upstream feeder)
- Customer-facing s fixed (often $s = 0$)

Objective: minimize $\sum h_i \sqrt{\tau_i}$.

Algorithm:

- For **trees / serial systems** — dynamic programming, $O(|V|^2)$ time
- For **general networks** — Magnanti et al. extension; MILP

198.5.5. Geometric intuition

Two extreme strategies for each stage:

- $\tau_i = 0$ — hold **no** safety stock here, just pass the delay through (you must trust upstream / promise slower downstream)
- $\tau_i = S_i + T_i$ — hold **enough** safety stock to fully decouple from upstream (you can promise $s_i = 0$)

Optimal placement: tradeoff $\sqrt{\tau_i}$ costs across all stages — concentrate inventory where holding is cheap (upstream, low h_i) and decoupling pays off most.

198.5.6. Risk pooling appears naturally

If two downstream stages share an upstream stage, the upstream stage's safety stock covers their combined variability — naturally captures risk pooling.

198.5.7. When to use

- Generally — anywhere you have a multi-stage supply network and need safety-stock placement
- Works for **assembly** (multiple upstream feed one downstream)
- Works for **distribution** (one upstream feeds multiple downstream)
- Works for **general DAG** networks

198.5.8. Limitations vs Clark-Scarf

- **Bounded (not stochastic) demand**: the cost of z-quantile coverage rather than full distributional optimality
- **Service-time commitment assumed**: real supply chains have stochastic delays even within commitments
- See Stochastic-service for the relaxation

198.5.9. See also

- **Multi-Echelon** — overview
- **Clark-Scarf** — stochastic-demand alternative
- **Stochastic-service**
- **Safety Stock**
- **Risk Pooling**

198.6. Stochastic-service

A relaxation of Graves-Willems guaranteed-service that allows **random wait times** upstream when stockouts occur. Originated with Lee & Billington (1993) and Ettl et al. (2000).

198.6.1. Why relax guaranteed service?

Graves-Willems assumes each stage **always** meets its promised service time, achieved by holding "enough" safety stock for bounded demand. Real supply chains miss promises:

- Demand exceeds the bounded model occasionally
- Suppliers stock out and you wait
- Capacity constrained

Stochastic-service models accept that **upstream stockouts cause downstream delays**, then compute the resulting waiting-time distribution.

198.6.2. The trade-off

	Guaranteed-service	Stochastic-service
Service times	deterministic / committed	random waiting times
Demand model	bounded (z-quantile)	full distribution
Tractability	easy (DP on trees, MILP for nets)	hard — queueing-style analysis
Safety stock	closed form $z\sigma\sqrt{\tau}$	must solve queueing model at each stage

198.6.3. Modeling waiting times

At each upstream stage, when inventory is exhausted, downstream orders wait. The waiting-time distribution depends on:

- Replenishment lead time at the upstream stage
- Demand distribution (downstream orders arriving at upstream)
- Base-stock level at the upstream stage

For each stage i, the **effective lead time** downstream sees is:

$$L_i^{\text{eff}} = L_i^{\text{nominal}} + W_i^{\text{upstream stockout wait}}$$

where W_i is random. Compute $E[W_i]$ and $\mathrm{Var}(W_i)$ via queueing approximations (typically Poisson-process / $\frac{M}{M}{}_1$-style models on the order arrival stream).

198.6.4. METRIC as a special case

For Poisson demand on repairable items, the **METRIC** model gives closed-form expressions for expected backorders at each echelon — see that page for the details. METRIC is the canonical example of a stochastic-service multi-echelon model.

198.6.5. When to use

- **Service-parts networks** — Poisson demand, repair pipelines, low-volume / high-value items

- **Long-lead-time supply chains** — upstream delays are material and frequent
- When guaranteed-service over-promises and you need realistic waiting-time estimates

198.6.6. When NOT to use

- Mass-market consumer goods where guaranteed-service is "close enough"
- Network design phase — guaranteed-service is much faster to optimize
- When you don't have good lead-time / demand distribution data

198.6.7. See also

- **Graves-Willems** — guaranteed-service
- **METRIC** / **VARI-METRIC** — Poisson demand repairables
- **Multi-Echelon overview**

198.7. METRIC

Multi-Echelon Technique for Recoverable Item Control (Sherbrooke 1968). A closed-form model for spare-parts networks with **Poisson demand** and **one-for-one (S - 1, S) base-stock policy**.

The original was developed for US Air Force repairable items — engines, avionics, hydraulics. The model is now standard for any service-parts network: defense, aerospace, semiconductors, medical devices.

198.7.1. Setup

A **two-echelon** network:

- One **depot** (central repair facility)
- J **bases** (forward sites consuming and returning parts)

At each base j:

- **Poisson** demand at rate λ_j
- One-for-one policy: maintain inventory level S_j; every demand triggers a replenishment order to the depot

Failed parts go to the depot for repair (mean repair time T_{depot}), then redistributed.

198.7.2. Palm's theorem (the key fact)

If demand is Poisson and lead times are independent, the **number of orders in the replenishment pipeline** at any moment is Poisson-distributed with mean = (demand rate) × (mean lead time).

For base j:

$$\text{Pipeline orders at base } j \sim \text{Poisson}\big(\lambda_j \, E[L_j]\big)$$

where $E[L_j]$ is the **expected** lead time the base sees — which depends on whether the depot has stock or the base must wait.

198.7.3. Expected backorders

For Poisson pipeline with mean ρ_j and base stock level S_j:

$$E[B_j(S_j)] = \sum_{x=S_j+1}^{\infty} (x - S_j)\, e^{-\rho_j} \frac{\rho_j^x}{x!}$$

— the expected shortfall above S_j in a Poisson distribution. Tables exist; computer-tractable to evaluate.

198.7.4. Effective lead time

If the depot is stocked, base j gets resupplied in standard shipping time T_j^{ship}. If the depot is stocked-out, the base waits for the depot's repair pipeline.

$$E[L_j] = T_j^{\text{ship}} + E\left[W_j^{\text{depot}}\right]$$

where $E\left[W_j^{\text{depot}}\right]$ is the expected wait at the depot, which depends on depot's stock S_0 and the **aggregate** demand $\sum_j \lambda_j$ hitting the depot.

198.7.5. Optimization

Decision variables: S_0 (depot) and S_j for each base.

Objective: minimize total backorders (or expected number of units down) subject to a **budget constraint**:

$$\min \sum_j E[B_j(S_j)] \qquad \text{s.t.} \sum_j c_j S_j + c_0 S_0 \le \text{budget}$$

Solved by **marginal analysis**: at each step, add one unit of inventory at whichever stocking location gives the biggest reduction in expected backorders per dollar.

198.7.6. VARI-METRIC

Sherbrooke's original METRIC approximates depot resupply time as deterministic. **VARI-METRIC** (Graves 1985) fixes this by modeling the depot pipeline as **Negative Binomial** — preserving both the mean **and the variance** of pipeline orders. Sometimes large efficiency gains. See VARI-METRIC.

198.7.7. Where it shows up

- **Military / aerospace logistics** — original application
- **Semiconductor fab spare parts** — single fab + multiple tools
- **Service parts networks** — automotive, industrial equipment
- **Computer / IT spares** — server / network gear

198.7.8. Limitations

- **Poisson demand only** — assumes one-by-one ordering; doesn't handle bulk demand
- **Stationary** — no demand trends or seasonality
- **Two-echelon** — extensions exist for > 2 echelons (Multi-Indenture / Multi-Echelon)

198.7.9. See also

- **VARI-METRIC** — variance-correction extension
- **Stochastic-service** — generalization
- **Poisson Distribution**
- **Base Stock Policy**

198.8. VARI-METRIC

An extension of METRIC (Graves 1985) that captures the **variance** of pipeline orders, not just the mean. The pipeline is modeled as **Negative Binomial** instead of Poisson — same mean, larger variance, more realistic for low-stock scenarios.

198.8.1. Why METRIC under-estimates backorders

METRIC assumes depot resupply time is deterministic and that the resulting **order pipeline is Poisson**. But when the depot occasionally stocks out, downstream waiting times are **random**, inflating pipeline variance.

If true variance $>$ Poisson variance ($=$ mean), the **real** probability of large pipelines is higher than Poisson predicts $\rightarrow$ real backorders are higher.

198.8.2. Negative Binomial pipeline

VARI-METRIC fits the pipeline-order distribution as Negative Binomial with mean ρ and variance $\sigma^2 > \rho$:

$$P(\text{pipeline} = x) = \binom{x + r - 1}{x} (1-p)^x p^r$$

where r, p are chosen to match the **observed (or computed) mean and variance**:

$$\rho = (1-p)\frac{r}{p}, \qquad \sigma^2 = (1-p)\frac{r}{p^2} = \frac{\rho}{p}$$

Solve for (r, p) given target (ρ, σ^2).

198.8.3. Computing the variance

Variance of orders at base j:

$$\sigma_j^2 = \lambda_j E\big[L_j\big] + \lambda_j^2 \operatorname{Var}\big(L_j\big)$$

— two contributions:

1. **Demand variance** during a mean-length lead time (Poisson contribution, $= \lambda E[L]$)
2. **Lead-time variance** itself (when waiting at the depot)

For the depot pipeline (single location aggregating multiple bases), use law of total variance:

$$\operatorname{Var}(\text{pipeline}) = E[\text{pipeline} \mid \text{depot stockout state}] + \operatorname{Var}(\ldots)$$

(Exact formulas are detailed; the point is that variance is **not** equal to the mean.)

198.8.4. Expected backorders with NB pipeline

$$E\left[B_j(S_j)\right] = \sum_{x=S_j+1}^{\infty} (x - S_j)\, P(\text{pipeline} = x)$$

— same form as METRIC, just with the NB instead of Poisson distribution for pipeline orders.

198.8.5. Practical impact

For low-stock items with appreciable depot-side delay, VARI-METRIC predictions are 20–50% more pessimistic than METRIC — and they match empirical data much better.

The optimization (marginal analysis on the budget vs total backorders) is the same algorithm; only the backorder calculation differs.

198.8.6. When METRIC is "good enough" vs VARI-METRIC

- High stock levels (low backorder probability) → METRIC is close
- Low stock / tight budget / many bases → VARI-METRIC corrects substantial under-counting

198.8.7. See also

- **METRIC** — Poisson-pipeline original
- **Multi-Echelon overview**
- **Stochastic-service models**

199. Risk Pooling

199.1. Risk Pooling

The fundamental supply-chain insight that **combining demand streams reduces variability** — so total safety stock to cover combined demand is less than the sum of safety stocks for each stream alone.

Formally: if you pool N independent demand sources, the standard deviation of total demand scales as $\sqrt{N}$, not N. Safety stock (proportional to standard deviation) therefore **scales as $\sqrt{N}$** — a substantial saving when N is large.

199.1.1. The basic identity

For independent random demands $D_1, ..., D_N$ with means μ_i and standard deviations σ_i:

$$E[D_1 + ... + D_N] = \sum_i \mu_i$$

$$\text{Var}(D_1 + ... + D_N) = \sum_i \sigma_i^2$$

$$\sigma_{\text{pool}} = \sqrt{\sum_i \sigma_i^2}$$

For **identical** demands ($\mu_i = \mu$, $\sigma_i = \sigma$):

$$\sigma_{\text{pool}} = \sigma\sqrt{N} \qquad (\text{not} \quad N\sigma)$$

199.1.2. The square-root law

If safety stock $= k\sigma$ for some constant k (e.g., $k = z_{0.95}$ for 95% service), then total pooled safety stock is

$$\text{SS}_{\text{pool}} = k\sigma\sqrt{N}$$

vs unpooled

$$\text{SS}_{\text{separate}} = \sum_i k\sigma = Nk\sigma$$

Saving: factor of $\frac{\sqrt{N}}{N} = \frac{1}{\sqrt{N}}$ in safety stock. With $N = 100$ locations, only 10%(!) of the total separate safety stock is needed when fully pooled.

See Square-Root Law for derivation.

199.1.3. Five flavors of pooling

1. **Location pooling** — centralize inventory across geographies
2. **Product pooling** — share common components / postponement
3. **Lead-time pooling** — substitute inventory for variability when lead times shrink
4. **Capacity pooling** — pool flexible production capacity across products
5. **Virtual pooling / transshipments** — keep separate locations but allow inter-location transfers

199.1.4. With correlated demands

If demands are **positively correlated**, pooling gains shrink. If they're **negatively correlated**, pooling gains expand. See Correlated Pooling for the general formula.

199.1.5. Pooling costs

Pooling isn't free:

- **Transportation cost** — pooled stock is farther from some customers
- **Response time** — central inventory takes longer to reach distant points
- **Coordination overhead** — sharing requires good information systems
- **Demand transparency loss** — aggregated demand loses regional signal

Optimal pooling is a trade-off — see Daganzo's continuous approximation for facility location.

199.1.6. Where it matters

- **Consolidating warehouses** — Walmart's regional DCs vs every-store-own-stock
- **Component commonality** — shared parts across product lines (HP DeskJet postponement case)
- **Make-to-order systems** — pool capacity, not inventory
- **Service-parts networks** — pool depots, especially for slow-moving items

199.1.7. See also

- **Square-Root Law**
- **Correlated Pooling**
- **Location Pooling**
- **Product Pooling**
- **Lead-time Pooling**
- **Safety Stock**

199.2. Square-Root Law

For N independent demand sources with identical mean μ and standard deviation σ, the standard deviation of total demand grows with $\sqrt{N}$ (not N). This is the **square-root law** — the central result of risk pooling.

199.2.1. Statement

Let $D_1, ..., D_N$ be independent, identically distributed random variables with $E[D_i] = \mu, \mathrm{Var}(D_i) = \sigma^2$. Then for $S = \sum_{i=1}^{N} D_i$:

$$E[S] = N\mu$$

$$\mathrm{Var}(S) = N\sigma^2 \qquad (\text{variance is additive})$$

$$\sigma_S = \sigma\sqrt{N} \qquad \left(\text{standard deviation grows as } \sqrt{N}\right)$$

199.2.2. Why variance is additive (proof sketch)

For independent random variables:

$$\mathrm{Var}(D_1 + D_2 + ... + D_N) = \sum_{i=1}^{N} \mathrm{Var}(D_i) + 2\sum_{i<j} \mathrm{Cov}(D_i, D_j)$$

Independence forces $\mathrm{Cov}(D_i, D_j) = 0$, so:

$$\mathrm{Var}(S) = \sum_{i=1}^{N} \sigma^2 = N\sigma^2$$

Taking square root: $\sigma_S = \sigma\sqrt{N}$. ∎

(The non-i.i.d. case with general covariances is the **correlated pooling** formula.)

199.2.3. Safety-stock implication

If safety stock is $z\sigma$ for each of N separate facilities, total:

$$\mathrm{SS}_{\text{separate}} = Nz\sigma$$

If pooled into one facility:

$$\mathrm{SS}_{\text{pool}} = z\,\sigma_S = z\sigma\sqrt{N}$$

Reduction factor:

$$\frac{\text{SS}_{\text{pool}}}{\text{SS}_{\text{separate}}} = \frac{\sqrt{N}}{N} = \frac{1}{\sqrt{N}}$$

N	SS separate	SS pooled	reduction
4	$4z\sigma$	$2z\sigma$	50%
9	$9z\sigma$	$3z\sigma$	67%
25	$25z\sigma$	$5z\sigma$	80%
100	$100z\sigma$	$10z\sigma$	90%

199.2.4. Numerical example

A retailer with $N = 16$ stores. Each store: weekly demand $\mu = 100$, $\sigma = 30$.

Each store needs safety stock $z\sigma = 1.65 \cdot 30 = 49.5$ units for 95% service.

Separate total: $16 \cdot 49.5 = 792$ units.

Pooled (one warehouse covering all):

$$\sigma_{\text{pool}} = 30\sqrt{16} = 120$$
$$\text{SS}_{\text{pool}} = 1.65 \cdot 120 = 198 \qquad \text{units}$$

Reduction: $\frac{198}{792} = 25\%$ of original safety stock. **75% savings on safety stock**.

199.2.5. Generalization to non-identical demands

If σ_i's differ:

$$\sigma_{\text{pool}} = \sqrt{\sum_i \sigma_i^2}$$

If correlations $\rho_{ij} \neq 0$:

$$\sigma^2_{\text{pool}} = \sum_i \sigma_i^2 + 2\sum_{i<j} \rho_{ij}\sigma_i\sigma_j$$

See Correlated Pooling.

199.2.6. Why this is "square-root"

Demand is **sum** of N random variables. By CLT-like arguments, the sum grows as N (in mean) but the **fluctuation** about the mean grows only as $\sqrt{N}$. The fluctuation is what safety stock covers. Hence the savings.

This is the **same** phenomenon as the standard error of a sample mean shrinking as $\frac{1}{\sqrt{n}}$.

199.2.7. Limitations

- **Independence assumed** — positively correlated demands give less benefit
- **Identical σ's assumed** — partial concentration in high-variance locations matters
- **Service level unchanged** — but pooling might allow higher service for same cost
- **Ignores transport cost** — central stock is farther from some customers

See Risk Pooling for trade-offs.

199.2.8. See also

- **Risk Pooling** — overview
- **Correlated Pooling** — non-independent case
- **Location Pooling** — application
- **Safety Stock**

199.3. Correlated Pooling

The generalization of the square-root law to demand sources with non-zero pairwise correlation. The crucial formula:

$$\sigma^2_{\text{pool}} = \sum_{i=1}^{N} \sigma_i^2 + 2 \sum_{i<j} \rho_{ij} \sigma_i \sigma_j$$

where ρ_{ij} is the correlation between demands D_i and D_j.

199.3.1. Derivation

For random variables $D_1, ..., D_N$ with $\text{Var}(D_i) = \sigma_i^2$ and $\text{Cov}(D_i, D_j) = \rho_{ij}\sigma_i\sigma_j$:

$$\text{Var}\left(\sum D_i\right) = \sum_i \text{Var}(D_i) + 2 \sum_{i<j} \text{Cov}(D_i, D_j)$$

$$= \sum_i \sigma_i^2 + 2 \sum_{i<j} \rho_{ij} \sigma_i \sigma_j$$

199.3.2. Three regimes

For identical $\sigma_i = \sigma$ and uniform $\rho_{ij} = \rho$:

$$\sigma^2_{\text{pool}} = N\sigma^2 + N(N-1)\rho\sigma^2 = \sigma^2[N + N(N-1)\rho]$$

$$\sigma_{\text{pool}} = \sigma\sqrt{N + N(N-1)\rho}$$

ρ	Pooled σ as $N \to \infty$	Benefit
$= 0$	$\sigma\sqrt{N}$	classic square-root law (full pooling benefit)
> 0	$\sigma\sqrt{N + N(N-1)\rho} \to \sigma N\sqrt{\rho}$ for large N	reduced benefit — pool variance grows linearly in N
< 0	$\sigma\sqrt{N + N(N-1)\rho}$ can be smaller	enhanced benefit — pool variance grows sub-linearly
$= 1$	σN	no benefit — perfectly correlated demand sums proportionally
$= -\frac{1}{N-1}$	0	perfect cancellation — sum is deterministic

199.3.3. Numerical example

$N = 4$ stores, each $\sigma = 30$. Compare pooling under different correlations:

ρ	σ_{pool}	SS at $z = 1.65$
0 (independent)	$\sigma\sqrt{4} = 60$	99
0.3 (somewhat positive)	$\sqrt{4 + 12 \cdot 0.3} \cdot 30 \approx 86$	142
0.5	$\sqrt{4 + 12 \cdot 0.5} \cdot 30 \approx 95$	156
1 (perfect positive)	$\sqrt{4 + 12} \cdot 30 = 120$	198
-0.2 (slight negative)	$\sqrt{4 - 12 \cdot 0.2} \cdot 30 \approx 51$	84

Compare: separate-stock total SS = $4 \cdot 1.65 \cdot 30 = 198$.

Independent pooling cuts that in half. **Highly correlated** pooling provides **no benefit**. **Negatively correlated** pooling beats independent.

199.3.4. Intuition

When demands move **together** (positive correlation):

- Peaks and troughs reinforce — combined demand is **more** variable per unit of mean
- Pooling does little; you'd need to cover the joint peak anyway

When demands move **opposite** (negative correlation):

- One source up while another down — combined demand is **smoother**
- Pooling is **more** beneficial than for independent demands

199.3.5. Sources of correlation

Type	Typical ρ
Same product, different regions, no shared shocks	near 0
Same product, similar markets / season	moderate $+$
Same product nationwide (TV ads, viral)	high $+$
Substitute products in same store	moderate $-$
Complementary products (winter / summer items)	strong $-$

199.3.6. Practical consequences

- **Diversification works** like in finance: bundle products with low / negative correlation to reduce inventory volatility
- **Common cause events** (weather, holidays, economic shocks) drive correlations up — risk pooling benefit shrinks during them
- **Always estimate empirical ρ** before claiming pooling savings — independence is rarely a safe assumption

199.3.7. See also

- **Risk Pooling** — overview
- **Square-Root Law** — independent case
- **Pearson Correlation**

199.4. Location Pooling

Holding inventory at **one central location** (or fewer central locations) instead of at every demand point. The simplest and most direct application of risk pooling.

199.4.1. Setup

You serve N markets / regions / stores. Demand at each is random with mean μ_i and standard deviation σ_i. Two extremes:

- **Decentralized**: each region holds its own safety stock — independent inventories totalling $\sum z\sigma_i$
- **Centralized**: one warehouse holds aggregate safety stock $z\sqrt{\sum \sigma_i^2}$ (for independent demands), then ships on demand

Centralized cuts safety stock by $\frac{\sqrt{N}}{N} = \frac{1}{\sqrt{N}}$ (the square-root law).

199.4.2. Numerical illustration

$N = 25$ regions, each $\mu = 1000, \sigma = 200$ per week, $z = 1.65$ for 95% service.

	Decentralized	Centralized
Mean demand	$25\{,\}000$	$25\{,\}000$
Std dev	$25 \cdot 200 = 5\{,\}000$ (sum)	$200\sqrt{25} = 1\{,\}000$ (pool)
Safety stock at 95%	$25 \cdot 1.65 \cdot 200 = 8\{,\}250$	$1.65 \cdot 1\{,\}000 = 1\{,\}650$
Savings	—	80%

199.4.3. The trade-offs

Centralization isn't free:

- **Transport cost** — central → customer distance increases (more outbound shipping)
- **Response time** — last-mile lead time grows
- **Demand visibility** — aggregated demand loses regional / time-of-day texture
- **Customer experience** — same-day delivery may be impossible from central locations
- **Single point of failure** — central warehouse outage hits all customers

Some industries (Amazon, parts distribution) trade off these costs explicitly via **partial pooling** — multiple regional DCs rather than one central or N stores.

199.4.4. Optimal degree of pooling

The trade-off is a **facility-location problem**:

- More facilities → less safety stock pooling savings, but lower transport costs
- Fewer facilities → more safety stock pooling savings, but higher transport costs

The classical **square-root facility-location model** (Daganzo continuous approximation; covered in facility location) computes the optimal number of facilities for a given demand density and cost structure.

199.4.5. Other forms

- **Inventory pooling**: physical centralization (one warehouse)
- **Virtual pooling**: separate physical inventories that share via inter-location transfers (transshipments)
- **Substitution-based pooling**: customer accepts a different but similar product
- **Lead-time pooling**: covered separately at lead-time pooling

199.4.6. Real-world examples

- **Walmart regional DCs** — centralized 1980s; revolutionized big-box retail
- **Amazon fulfillment centers** — now mostly partially pooled with last-mile depots
- **Spare parts depots** — defense, automotive
- **Hospital networks** — central pharmacy for slow-moving / expensive drugs

199.4.7. See also

- **Risk Pooling** — overview
- **Square-Root Law**
- **Correlated Pooling**
- **Facility Location**

199.5. Product Pooling

Reducing inventory by **sharing parts / components across product variants** rather than holding stock for each finished SKU separately. Two main mechanisms:

1. **Component commonality** — products share common parts
2. **Postponement (delayed differentiation)** — defer customization until late in the supply chain

Both convert several high-variance finished-good demands into one lower-variance component demand — a textbook application of risk pooling.

199.5.1. Postponement: the HP DeskJet case

Classic Lee-Tang (1996) example. HP shipped printers configured for North America, Europe, Asia from a Singapore factory. Each region got its own SKU (different power supplies, manuals, plugs). Region-specific safety stocks per SKU.

Postponement: ship **generic** printers to regional DCs. Final regional configuration (adding power supply, labeling) done at the DC, right before delivery.

Effect:

- 3 finished SKUs → 1 generic + 3 small customization kits
- Safety stock for the generic stage uses aggregate demand across regions
- Customization kits have very fast lead time → minimal safety stock

Variance accounting:

	Per-region	**Aggregate**
Mean demand	μ_i	$\sum \mu_i$

	Per-region	Aggregate
Variance	σ_i^2	$\sum \sigma_i^2$ (independent)
Coefficient of variation	$\frac{\sigma_i}{\mu_i}$	$\frac{\sqrt{\sum \sigma_i^2}}{\sum} \mu_i$

The aggregate CV is smaller — pooling effect for the generic. Total safety stock falls by a factor $\frac{\sqrt{N}}{N}$ for the components that get postponed.

199.5.2. Component commonality (Baker-Magazine-Nuttle 1986)

Two end products A and B use:

- Some shared components — **commonality**
- Some unique components

For the shared component, demand is $D_A + D_B$ — pooled. For unique components, demand is each product's own — not pooled.

Trade-off:

- More commonality → more pooling savings, but each product is harder to differentiate
- More uniqueness → more product differentiation, more inventory cost

199.5.3. Variance math

Let product demands D_A, D_B be independent with σ_A, σ_B. The shared component (used in **both**) sees:

$$\sigma_{\text{shared}} = \sqrt{\sigma_A^2 + \sigma_B^2}$$

vs separate components:

$$\sigma_{\text{separate, A}} + \sigma_{\text{separate, B}} = \sigma_A + \sigma_B$$

By the triangle inequality $\sqrt{\sigma_A^2 + \sigma_B^2} \leq \sigma_A + \sigma_B$ (and strict inequality when both $\sigma_i > 0$). Equality only when one variance is zero.

So sharing strictly reduces required safety stock for the shared component.

199.5.4. Practical examples

- **Auto industry** — Toyota uses common platforms across models (Corolla, Camry, RAV4 share many parts)
- **Electronics** — Apple uses common batteries, chargers, screens across product lines
- **Apparel** — DKNY makes generic shirts in white, dyes them to color at distribution centers
- **PC manufacturing** — Dell assemble-to-order from common components
- **Pharmacy** — generic drug → repackaged for retail / hospital channels

199.5.5. Limitations

- **Customization later costs more per unit** — postponed assembly is less efficient than dedicated lines

- **Some differentiation can't be deferred** — e.g., physical form factors
- **Information / coordination overhead** — late customization needs accurate downstream demand info

199.5.6. See also

- **Risk Pooling**
- **Square-Root Law**
- **Location Pooling**
- **Safety Stock**

199.6. Lead-time Pooling

Reducing safety stock by **shortening or stabilizing lead times**, not by aggregating demand. Substitutes responsiveness for inventory — a different lever than location pooling.

199.6.1. The lever

Safety stock scales with lead-time demand standard deviation:

$$\text{SS} = z\sqrt{L\sigma_D^2 + \mu_D^2\sigma_L^2}$$

where L is mean lead time and σ_L is lead-time variability. If you cut L or σ_L, safety stock falls.

199.6.2. Two distinct mechanisms

1. Shorter lead time: L falls, lead-time demand variability falls as $\sqrt{L}$. Safety stock for demand-side variability scales as $z\sigma_D\sqrt{L} \to$ cut L by $\frac{1}{4}$, cut SS by $\frac{1}{2}$.

2. Less variable lead time: σ_L falls toward zero. The $\mu_D^2\sigma_L^2$ term shrinks. Big effect when demand is fast and lead-time variability dominated the formula.

Often both effects compound — fast & reliable.

199.6.3. Numerical example

Demand: $\mu_D = 100$/week, $\sigma_D = 30$/week.

Scenario	L	σ_L	LT-demand σ	SS at $z = 1.65$
Slow, variable	8 wk	2 wk	$\sqrt{8 \cdot 900 + 10000 \cdot 4} = \sqrt{47200} \approx 217$	358
Fast, variable	2 wk	1 wk	$\sqrt{2 \cdot 900 + 10000 \cdot 1} = \sqrt{11800} \approx 109$	179
Slow, stable	8 wk	0	$\sqrt{8 \cdot 900} = \sqrt{7200} \approx 85$	140
Fast, stable	2 wk	0	$\sqrt{2 \cdot 900} \approx 42$	69

Going from "slow, variable" to "fast, stable" cuts safety stock by 80%.

199.6.4. How to shrink lead times

- **Air freight vs ocean** — pay premium for time
- **Domestic / near-shore** — vs offshore production
- **Vendor-managed inventory** — supplier holds inventory at your door

- **Just-in-time / Kanban** — pull-based replenishment shortens effective lead time
- **Direct-store delivery** — bypass DCs

199.6.5. How to stabilize lead times

- **Multi-sourcing** — backup suppliers smooth out individual variability
- **Vendor service-level agreements** — explicit, monitored commitments
- **Production buffers** at vendor — they hold the variability, not you
- **Premium expedited service** — pay for guaranteed times

199.6.6. Trade-offs

Faster / more reliable lead times cost more:

- Higher unit cost (premium shipping, near-shoring)
- Higher per-order fixed cost (smaller batches → more orders)
- Supplier relationship investment
- Reduced negotiating leverage

This is one of the core **cost-vs-responsiveness** axes in supply-chain strategy (Fisher's framework: efficient vs responsive supply chains).

199.6.7. When lead-time pooling beats location pooling

If demand correlates strongly across regions (so location pooling gains little), but lead-time variability dominates safety stock, lead-time pooling is the bigger win.

Common in:

- **Fashion / seasonal goods** — short selling season, high demand uncertainty
- **Consumer electronics** — new-product launches
- **Spare parts** — slow movers where SS is dominated by lead-time variance

199.6.8. See also

- **Risk Pooling**
- **Safety Stock**
- **Pipeline Stock**
- **Location Pooling**

200. EOQ

200.1. EOQ

The inventory replenishment trade-off:

- Order **too much** → high **holding cost** (storage, capital tied up)
- Order **too little** → high **order cost** (frequent setups, paperwork)

Strategy of this whole section:

1. Express total annual cost as a function of order quantity Q
2. Take its derivative with respect to Q and set to zero
3. Solve for Q — call this optimum the **Economic Order Quantity** (EOQ)

200.1.1. Assumptions

- Demand D is constant and known
- Replenishment is instantaneous (no lead time)
- Unit cost c is flat (no quantity discounts)
- Single product, single location
- Backorders are allowed at penalty rate b (relaxed in the next subsection)

200.1.2. Cost model (TC)

Total cost over a year is the sum of four pieces. Build them one at a time.

1. Purchase cost. Buy D units per year, each at unit cost c:

$$\mathrm{PC} = cD$$

2. Order cost. Each order costs S to place; ordering Q units at a time means $\frac{D}{Q}$ orders per year:

$$\mathrm{OC} = S\left(\frac{D}{Q}\right)$$

3. Holding cost. With backorders allowed, the average on-hand inventory is $\frac{(Q-B)^2}{2Q}$. Each unit on hand is charged at rate h:

$$\mathrm{HC} = h\frac{(Q-B)^2}{2Q}$$

4. Shortage cost. The average backorder level is $\frac{B^2}{2Q}$. Each backordered unit is charged at rate b:

$$\mathrm{SC} = b\left(\frac{B^2}{2Q}\right)$$

Sum the four pieces into total cost:

$$\mathrm{TC}(Q,B) = \underbrace{cD}_{\mathrm{PC}} + \underbrace{S\left(\frac{D}{Q}\right)}_{\mathrm{OC}} + \underbrace{h\frac{(Q-B)^2}{2Q}}_{\mathrm{HC}} + \underbrace{b\left(\frac{B^2}{2Q}\right)}_{\mathrm{SC}}$$

Variables:

- D = annual demand
- Q = order quantity (the **decision** variable)
- c = unit cost
- S = setup (order) cost
- h = annual holding cost per unit ($= r \cdot v$, where r is the holding cost rate and v is the unit value)
- b = backorder penalty per unit per year
- B = maximum backorder quantity

cD	Purchase cost	Unit cost times annual demand

$S\left(\frac{D}{Q}\right)$	Order cost	Place $\frac{D}{Q}$ orders per year, each costing S
$h\frac{(Q-B)^2}{2Q}$	Holding cost	Average on-hand inventory times holding rate h
$b\left(\frac{B^2}{2Q}\right)$	Shortage cost	Average backorder level times penalty rate b

200.1.3. Simplification (TRC)

Two simplifications take us from TC down to the **Total Relevant Cost** — a function of Q alone.

Step 1 — forbid backorders ($B = 0$). Substitute $B = 0$ everywhere:

$$\mathrm{TC}(Q, 0) = cD + S\left(\frac{D}{Q}\right) + h\frac{(Q-0)^2}{2Q} + b\frac{(0)^2}{2Q}$$

The shortage term zeroes out and the holding term collapses:

$$= cD + S\left(\frac{D}{Q}\right) + h\frac{Q^2}{2Q} + 0 = cD + S\left(\frac{D}{Q}\right) + h\frac{Q}{2}$$

Step 2 — drop the constant. The purchase cost cD does not depend on Q, so it does not move the optimum. Strike it out:

$$\cancel{cD} + S\left(\frac{D}{Q}\right) + h\left(\frac{Q}{2}\right)$$

What remains is the **Total Relevant Cost**:

$$\mathrm{TRC}(Q) = S\left(\frac{D}{Q}\right) + h\left(\frac{Q}{2}\right)$$

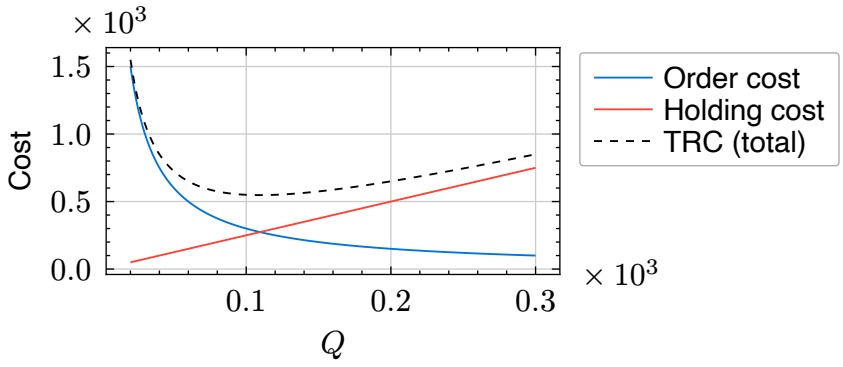


Figure 7: TRC and its two components, as functions of Q. The minimum is where the two component curves cross.

200.1.4. Derive Q^* (first-order condition)

Find the Q that minimizes $\mathrm{TRC}(Q)$:

- Set the derivative with respect to Q to zero (FOC, this subsection).
- Verify the second derivative is positive (SOC, next subsection).

200.1.4.1. Differentiate term by term

Start:

$$\frac{d}{dQ}\,\mathrm{TRC}(Q) = \frac{d}{dQ}\left(S\left(\frac{D}{Q}\right) + h\left(\frac{Q}{2}\right)\right)$$

Split the derivative across the two terms:

$$= \frac{d}{dQ}\left(S\left(\frac{D}{Q}\right)\right) + \frac{d}{dQ}\left(h\left(\frac{Q}{2}\right)\right)$$

Rewrite $\frac{1}{Q}$ as Q^{-1} in the first term:

$$= \frac{d}{dQ}(SDQ^{-1}) + \frac{d}{dQ}\left(\left(\frac{h}{2}\right)Q\right)$$

Apply the power rule to each term — the exponent comes down as a coefficient, then drops by 1:

$$= SD\ {-1}\ Q^{-1-1} + \left(\frac{h}{2}\right)1Q^{1-1}$$

Simplify the exponents:

$$= -SDQ^{-2} + \left(\frac{h}{2}\right)Q^{0}$$

Use $Q^0 = 1$ and tidy:

$$\frac{d}{dQ}\,\mathrm{TRC}(Q) = -\frac{SD}{Q^2} + \frac{h}{2}$$

200.1.4.2. Set to zero, solve for Q

Start (FOC):

$$-\frac{SD}{Q^2} + \frac{h}{2} = 0$$

Move $-\frac{SD}{Q^2}$ to the right side (becomes positive):

$$\frac{h}{2} = \frac{SD}{Q^2}$$

Multiply both sides by Q^2:

$$\left(\frac{h}{2}\right)Q^2 = SD$$

Multiply both sides by $\frac{2}{h}$:

$$Q^2 = \frac{2SD}{h}$$

Take the square root of both sides:

$$Q^* = \texttt{sqrt}\left(\frac{2SD}{h}\right)$$

This Q^* is the **Economic Order Quantity**:

$$\text{EOQ} = Q^* = \sqrt{\frac{2SD}{h}}$$

Example

Plug in $D = 12000$ units/year, $S = 50$ \$/order, $h = 2$ \$/unit/year:

$$Q^* = \sqrt{\frac{2 \cdot 50 \cdot 12000}{2}} = \sqrt{600000} \approx 775 \text{ units}$$

Order 775 units at a time.

200.1.5. Verify it is a minimum (second-order condition)

Differentiate $\frac{d}{dQ}\ \text{TRC}(Q)$ once more.

Start:

$$\frac{d^2}{dQ^2}\ \text{TRC}(Q) = \frac{d}{dQ}\left(-SDQ^{-2} + \frac{h}{2}\right)$$

The constant $\frac{h}{2}$ drops out. Apply the power rule to $-SDQ^{-2}$ — exponent -2 comes down, then drops by 1:

$$= -SD\ -2\ Q^{-2-1} = 2SDQ^{-3}$$

Tidy:

$$\frac{d^2}{dQ^2}\ \text{TRC}(Q) = \frac{2SD}{Q^3}$$

For any $Q > 0$ this is positive $\Rightarrow$ Q^* is a **minimum**. ✓

200.1.6. Derive TRC^* (the minimum cost)

Substitute $Q^* = \sqrt{\frac{2SD}{h}}$ back into TRC.

Start:

$$\text{TRC}^* = S\left(\frac{D}{Q^*}\right) + h\left(\frac{Q^*}{2}\right)$$

Substitute Q^*:

$$= S\frac{D}{\sqrt{\frac{2SD}{h}}} + \left(\frac{h}{2}\right)\sqrt{\frac{2SD}{h}}$$

Use $1/\sqrt{x} = \sqrt{1/x}$ on the first term:

$$= SD\sqrt{\frac{h}{2SD}} + \left(\frac{h}{2}\right)\sqrt{\frac{2SD}{h}}$$

Pull each coefficient inside its radical (squared):

$$= \sqrt{S^2 D^2 \cdot \frac{h}{2SD}} + \sqrt{\frac{h^2}{4} \cdot \frac{2SD}{h}}$$

Cancel inside each radical:

$$= \sqrt{\frac{SDh}{2}} + \sqrt{\frac{SDh}{2}}$$

Two equal terms add:

$$= 2\sqrt{\frac{SDh}{2}}$$

Bring the 2 inside the radical (squared, so it becomes 4):

$$= \sqrt{4 \cdot \frac{SDh}{2}} = \sqrt{2SDh}$$

Result:

$$\mathrm{TRC}^* = \sqrt{2SDh}$$

200.1.7. Key insight: balance at the optimum

At Q^*, **annual order cost equals annual holding cost**. Each is exactly half of TRC^*.

Order cost at Q^*:

$$\mathrm{OC}^* = S\left(\frac{D}{Q^*}\right) = S\frac{D}{\sqrt{\frac{2SD}{h}}} = \sqrt{\frac{S^2 D^2 \cdot h}{2SD}} = \sqrt{\frac{SDh}{2}}$$

Holding cost at Q^*:

$$\mathrm{HC}^* = h\left(\frac{Q^*}{2}\right) = \left(\frac{h}{2}\right)\sqrt{\frac{2SD}{h}} = \sqrt{\frac{h^2}{4} \cdot \frac{2SD}{h}} = \sqrt{\frac{SDh}{2}}$$

Therefore:

$$\mathrm{OC}^* = \mathrm{HC}^* = \sqrt{\frac{SDh}{2}} = \frac{\mathrm{TRC}^*}{2}$$

Mnemonic: **EOQ is the Q where the two cost curves cross.**

200.1.8. How forgiving is EOQ? (sensitivity)

Small input errors produce **much smaller** cost penalties — square-root cushioning. Quantify it.

200.1.8.1. Cost penalty function

Take the ratio of TRC at any Q to TRC^*.

Start:

$$\frac{\mathrm{TRC}(Q)}{\mathrm{TRC}^*} = \frac{S\frac{D}{Q} + h\frac{Q}{2}}{\sqrt{2SDh}}$$

Split the fraction:

$$= \frac{S\frac{D}{Q}}{\sqrt{2SDh}} + \frac{h\frac{Q}{2}}{\sqrt{2SDh}}$$

Two useful identities (each obtained by rearranging $Q^* = \sqrt{(2SD)/h}$):

$$\sqrt{2SDh} = hQ^* \quad \text{and} \quad \sqrt{2SDh} = \frac{2SD}{Q^*}$$

Substitute the **second** identity into the first term, the **first** identity into the second term:

$$= \frac{S\frac{D}{Q}}{\frac{2SD}{Q^*}} + \frac{h\frac{Q}{2}}{hQ^*}$$

Cancel SD in the first fraction, h in the second:

$$= \frac{Q^*}{2Q} + \frac{Q}{2Q^*}$$

Factor out $\frac{1}{2}$:

$$\frac{\mathrm{TRC}(Q)}{\mathrm{TRC}^*} = \frac{1}{2}\left(\frac{Q}{Q^*} + \frac{Q^*}{Q}\right)$$

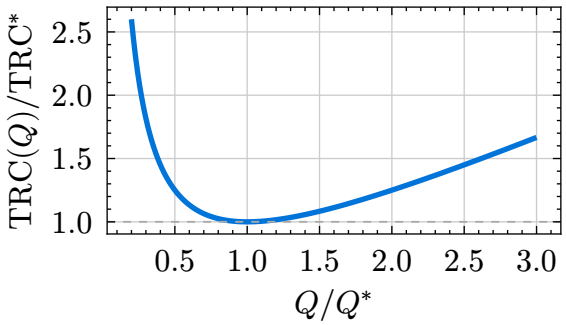


Figure 8: Cost penalty curve: how much extra you pay when $Q \neq Q^*$.

Snapshots:

- $Q = 2Q^*$ or $Q = 0.5Q^* \Rightarrow$ cost is **1.25**× optimum (+25%)
- $Q = Q^* \Rightarrow$ cost is **1.00**× optimum

200.1.8.2. Demand-forecast error

Forecast D', actual D. Define $E = \frac{D}{D'}$. Then $Q'/Q^* = \sqrt{D'/D} = \frac{1}{\sqrt{E}}$, so:

$$\frac{\mathrm{TRC}}{\mathrm{TRC}^*} = \frac{1}{2}\left(\frac{1}{\sqrt{E}} + \sqrt{E}\right)$$

- $E = 0.5$ (actual demand is half the forecast) $\Rightarrow$ cost penalty is just **+6%**. Tiny.

200.1.8.3. Cost-parameter error

Same square-root cushion — a 10% error in S or h produces **much less** than a 10% error in Q^*, since $Q^* \propto \sqrt{S}$ and $Q^* \propto \sqrt{1/h}$.

200.1.9. Practical: power-of-two policies

Real warehouses cannot use an arbitrary cycle time $T^* = \frac{Q^*}{D}$ — they need a fixed, predictable cadence.

Rule. The replenishment interval T must be of the form $T_{\text{base}} \cdot 2^k$ for some integer $k = 0, 1, 2, \ldots$. Example with $T_{\text{base}} = 1$ week: allowed intervals are 1, 2, 4, 8 weeks.

Variables:

- T = chosen replenishment interval
- T^* = optimal interval ($= \frac{Q^*}{D}$)
- T/T^* = how far the chosen schedule deviates from the optimum

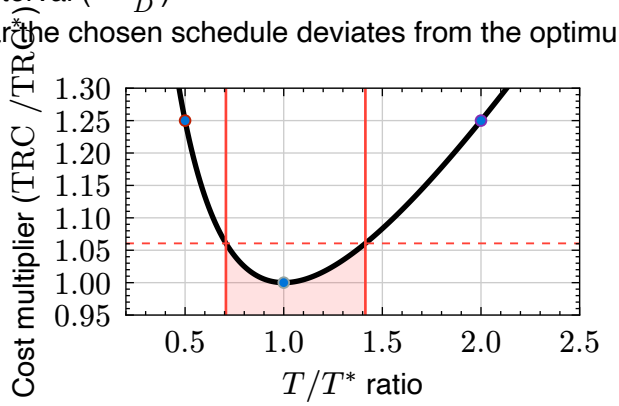


Figure 9: Power-of-two safe zone: $T/T^* \in [0.707, 1.414]$ guarantees ≤ 6% cost penalty.

Safe zone. For any T^*, **some** power-of-two T is always within a factor of $\sqrt{2}$ of it. So $T/T^* \in [0.707, 1.414]$ is always achievable, which from the cost-penalty curve gives at most a **6% cost penalty**.

Why this is great. Powers of two **nest**: a product ordered every 2 months and another every 4 months always sync up. Predictable warehouse rhythm and consolidated shipping costs.

Scenario	Deviation	Cost impact ($\text{TRC}\ /\text{TRC}^*$)
Order quantity	$Q = 2Q^*$ (too large)	1.25 (+25%)
Order quantity	$Q = 0.5Q^*$ (too small)	1.25 (+25%)
Demand forecast	$D = 2D'$ (actual is double)	1.06 (+6%)
Demand forecast	$D = 0.5D'$ (actual is half)	1.06 (+6%)
Replenishment	Power-of-two interval	≤ 1.06 (≤ +6%)

Example

1. Setup: two products with different demand profiles.

Base time period $T_{\text{base}} = 1$ month.

Feature	Product A (fast mover)	Product B (slow mover)
Annual demand D	2400 units	600 units
Order cost S	\$50	\$50
Holding cost h	\$2 / unit / year	\$3 / unit / year

2. Compute the mathematical ideal T^* (using $T^* = Q^*/D$).

Product A:

$$Q_A^* = \sqrt{\frac{2 \cdot 50 \cdot 2400}{2}} \approx 346 \text{ units}$$

$$T_A^* = \frac{346}{2400} \approx 0.144 \text{ years} \approx \mathbf{1.7} \text{ months}$$

Product B:

$$Q_B^* = \sqrt{\frac{2 \cdot 50 \cdot 600}{3}} \approx 141 \text{ units}$$

$$T_B^* = \frac{141}{600} \approx 0.235 \text{ years} \approx \mathbf{2.8} \text{ months}$$

3. Snap to the power-of-two grid.

Allowed intervals: 1, 2, 4, 8, … months. Pick the one inside the 6% safe zone $0.707 \leq T/T^* \leq 1.414$.

- Product A ($T_A^* \approx 1.7$ months):
  - $T = 2$ months $\Rightarrow T/T^* = \frac{2}{1.7} = 1.17 \Rightarrow$ inside the safe zone $\Rightarrow$ choose **2 months**.
- Product B ($T_B^* \approx 2.8$ months):
  - $T = 2$ months $\Rightarrow T/T^* = \frac{2}{2.8} = 0.71 \Rightarrow$ just above the lower bound 0.707 ✓
  - $T = 4$ months $\Rightarrow T/T^* = \frac{4}{2.8} = 1.43 \Rightarrow$ above 1.414 ×
  - Choose **2 months**.

4. Synchronization for free.

Powers of two nest: if A is ordered every 2 months and B every 4 months, B's orders always coincide with one of A's. Predictable rhythm, consolidated shipping.

200.2. EPQ

200.2.1. EPQ (Economic Production Quantity)

Relax one dimension from basic EOQ: replenishment is **no longer instantaneous**. Instead of receiving Q all at once, you produce at finite rate p units/year while demand d drains stock simultaneously.

200.2.1.1. Setup

New variables (beyond basic EOQ):

- p = production rate (units/year), with $p > d$ (otherwise demand outpaces supply)
- d = demand rate (units/year) — same as D in basic EOQ; renamed here to emphasize it's a **rate**, not annual quantity

The inventory profile in each cycle:

- **Buildup phase** (length $\frac{Q}{p}$): produce at rate p, demand at rate d — net accumulation rate $p - d$. Inventory rises to a peak $I_{\max}$.
- **Drawdown phase** (length $\frac{I_{\max}}{d}$): production stops, demand continues at rate d — inventory drains from $I_{\max}$ to 0.

Peak inventory reached at end of buildup:

$$I_{\max} = (p - d) \cdot \left(\frac{Q}{p}\right) = Q \cdot \left(1 - \frac{d}{p}\right)$$

The peak is **less than** Q (the case for basic EOQ where $p = \infty$) because some units are consumed **during** production.

Average on-hand inventory (triangle, height $I_{\max}$):

$$\bar{I} = \frac{I_{\max}}{2} = \frac{Q}{2} \cdot \left(1 - \frac{d}{p}\right)$$

The factor $\left(1 - \frac{d}{p}\right) \leq 1$ shrinks the average vs basic EOQ.

200.2.1.2. Cost model

Total relevant cost (drop the constant cd term):

$$\text{TRC}(Q) = S\left(\frac{d}{Q}\right) + h \cdot \frac{Q}{2} \cdot \left(1 - \frac{d}{p}\right)$$

Only the holding term changes from basic EOQ — multiplied by $\left(1 - \frac{d}{p}\right)$. As $p \to \infty$ (instant production), the factor $\to 1$ and we recover basic EOQ.

200.2.1.3. Derive Q^*

Same calculus as basic EOQ: differentiate, set to zero.

$$\frac{d}{dQ}\,\text{TRC} = -\frac{Sd}{Q^2} + \frac{h}{2} \cdot \left(1 - \frac{d}{p}\right) = 0$$

$$\frac{Sd}{Q^2} = \frac{h}{2} \cdot \left(1 - \frac{d}{p}\right)$$

$$(Q^*)^2 = \frac{2Sd}{h\left(1 - \frac{d}{p}\right)}$$

Take the square root and rewrite $1 - \frac{d}{p} = \frac{p-d}{p}$:

$$Q^* = \sqrt{\frac{2Sd}{h}} \cdot \sqrt{\frac{p}{p-d}}$$

The first factor is the **basic EOQ**; the second is a multiplier > 1 that grows the production batch when the production rate is close to demand (long buildup, little overlap with drawdown).

200.2.1.4. Final formulas

$$Q^* = \sqrt{\frac{2Sd}{h}} \cdot \sqrt{\frac{p}{p-d}} \quad T^* = \frac{Q^*}{d} \quad \mathrm{TRC}^* = \sqrt{2Sdh} \cdot \sqrt{1 - \frac{d}{p}}$$

Sanity check: as $p \to \infty$ (instantaneous), the multiplier $\sqrt{\frac{p}{p-d}} \to 1$ and we recover basic EOQ exactly. As $p \to d^+$ (production barely keeps up), $Q^* \to \infty$ — you should run the line continuously.

Example

Given (shared EOQ params + a finite production rate):

- Annual demand: $d = 12000$ units/year
- Setup cost: S = \$50 / setup
- Holding cost: h = \$2 / unit / year
- Production rate: $p = 24000$ units/year (twice the demand rate)

Step 1 — multiplier from finite production

$$\sqrt{\frac{p}{p-d}} = \sqrt{\frac{24000}{24000 - 12000}} = \sqrt{2} \approx 1.414$$

Step 2 — production batch size

$$Q^* = \sqrt{\frac{2 \cdot 50 \cdot 12000}{2}} \cdot \sqrt{2} = \sqrt{600000} \cdot \sqrt{2} \approx 774.6 \cdot 1.414 \approx 1095 \text{ units}$$

Step 3 — peak inventory and cycle

$$I_{\max} = Q^* \left(1 - \frac{d}{p}\right) = 1095(1 - 0.5) \approx 548 \text{ units}$$

$$T^* = \frac{Q^*}{d} = \frac{1095}{12000} \approx 0.091 \text{ years} \approx 33 \text{ days}$$

Of T^*, the buildup takes $\frac{Q^*}{p} = \frac{1095}{24000} \approx 17$ days; drawdown takes the remaining ≈ 16 days.

Total cost:

$$\mathrm{TRC}^* = \sqrt{2 \cdot 50 \cdot 12000 \cdot 2} \cdot \sqrt{1 - 0.5} \approx 1549 \cdot 0.707 \approx 1095 \ \$/\text{year}$$

Compare to basic EOQ on the same demand and setup costs:

- Basic EOQ ($p = \infty$): $Q^* = 775$, $\mathrm{TRC}^* \approx$ \$1549
- EPQ ($p = 2d$): $Q^* = 1095$ (larger), $\mathrm{TRC}^* \approx$ \$1095 (29% lower)

EPQ is cheaper because some units are consumed **during** production — average on-hand is lower than the basic EOQ assumes.

200.3. Backorders

200.3.1. EOQ with planned backorders

Relax one dimension from basic EOQ: **excess demand** is no longer forbidden — stockouts are allowed at penalty rate b per unit per year. Now there are **two** decisions: order quantity Q and maximum backorder B.

200.3.1.1. Setup

New variables (beyond basic EOQ):

- B = maximum backorder quantity (decision variable)
- b = backorder penalty cost per unit per year

The inventory profile in each cycle:

- Receive Q units, immediately fill the backlog B — on-hand jumps to $Q - B$.
- On-hand drains at rate D over time $\frac{Q-B}{D}$, hits zero, then accumulates backlog up to B over time $\frac{B}{D}$, total cycle $T = \frac{Q}{D}$.

Two area integrals over a cycle, divided by the cycle length $T = \frac{Q}{D}$:

- **Average on-hand**: triangle of height $Q - B$, width $\frac{Q-B}{D} \rightarrow \frac{(Q-B)^2}{2D} \mid T = \frac{(Q-B)^2}{2Q}$
- **Average backlog**: triangle of height B, width $\frac{B}{D} \rightarrow \frac{B^2}{2D} \mid T = \frac{B^2}{2Q}$

200.3.1.2. Cost model

Total relevant cost (drop the constant purchase-cost term):

$$\mathrm{TRC}(Q, B) = S\left(\frac{D}{Q}\right) + h\frac{(Q-B)^2}{2Q} + b\left(\frac{B^2}{2Q}\right)$$

Two changes from basic EOQ TRC, both highlighted in red: the holding term now sees $(Q - B)$ instead of Q (less average on-hand), plus a new shortage term.

200.3.1.3. Derive B^* first (FOC in B)

Holding Q fixed, take $\partial\ \mathrm{TRC}\ /\partial B$:

$$\frac{\partial}{\partial B}\ \mathrm{TRC} = h \cdot 2(Q-B)\frac{(-1)}{2Q} + b \cdot 2\frac{B}{2Q} = -\frac{h(Q-B)}{Q} + b\frac{B}{Q}$$

Set to zero, multiply by Q:

$$-h(Q-B) + bB = 0$$

$$-hQ + hB + bB = 0$$

$$B^* = \frac{hQ}{h+b}$$

So the backlog is the share $h/(h+b)$ of the order quantity.

200.3.1.4. Derive Q^* (FOC in Q, after substituting B^*)

Take $\partial\ \mathrm{TRC}\ /\partial Q$ and set to zero (algebra for the holding term uses the product rule):

$$-\frac{SD}{Q^2} + \frac{h(Q^2-B^2)}{2Q^2} - b\frac{B^2}{2Q^2} = 0$$

Multiply by $2Q^2$:

$$-2SD + h(Q^2 - B^2) - bB^2 = 0$$

$$hQ^2 - (h+b)B^2 = 2SD$$

Substitute $B^* = h\frac{Q}{h+b}$ so $(h+b)(B^*)^2 = h^2\frac{Q^2}{h+b}$:

$$hQ^2 - h^2\frac{Q^2}{h+b} = 2SD$$

$$Q^2 \cdot h\frac{h+b-h}{h+b} = 2SD$$

$$Q^2 \cdot \frac{hb}{h+b} = 2SD$$

$$(Q^*)^2 = \frac{2SD}{h} \cdot \frac{h+b}{b}$$

Take the square root:

$$Q^* = \sqrt{\frac{2SD}{h}} \cdot \sqrt{\frac{h+b}{b}}$$

The first factor is the **basic EOQ**; the second is a multiplier > 1 that grows the order when backorders are cheap (small b).

200.3.1.5. Final formulas

$$Q^* = \sqrt{\frac{2SD}{h}} \cdot \sqrt{\frac{h+b}{b}} \quad B^* = \sqrt{\frac{2SDh}{b(h+b)}} \quad \mathrm{TRC}^* = \sqrt{2SDh} \cdot \sqrt{\frac{b}{h+b}}$$

Sanity check: as $b \to \infty$ (backorders prohibitively expensive), the multiplier $\sqrt{\frac{h+b}{b}} \to 1$ and $B^* \to 0$ — recovers basic EOQ. As $b \to 0$ (backorders are free), $Q^* \to \infty$ and $\mathrm{TRC}^* \to 0$.

Example

Given (shared EOQ params + a backorder penalty):

- Annual demand: $D = 12000$ units/year
- Order cost: S = \$50 / order
- Holding cost: h = \$2 / unit / year
- Backorder penalty: b = \$8 / unit / year

Step 1 — backlog share

$$B^* = \frac{hQ^*}{h+b} = \frac{2}{2+8}Q^* = 0.2Q^*$$

So the optimal backlog is 20% of the order quantity.

Step 2 — order quantity

$$Q^* = \sqrt{\frac{2 \cdot 50 \cdot 12000}{2}} \cdot \sqrt{\frac{2+8}{8}} = \sqrt{600000} \cdot \sqrt{1.25} \approx 774.6 \cdot 1.118 \approx 866 \text{ units}$$

Step 3 — backlog and cost

$$B^* = 0.2 \cdot 866 \approx 173 \text{ units}$$

$$\text{TRC}^* = \sqrt{2 \cdot 50 \cdot 12000 \cdot 2} \cdot \sqrt{\frac{8}{10}} \approx 1549 \cdot 0.894 \approx 1386 \text{ \$/year}$$

Compare to basic EOQ on the same inputs:

- Basic EOQ: $Q^* = 775$, $\text{TRC}^* \approx$ \$1549
- With backorders: $Q^* = 866$ (larger), $B^* = 173$, $\text{TRC}^* \approx$ \$1386 (10% lower)

Allowing planned backorders is worthwhile here because b is only $4 \times h$ — backorders cost a bit more than holding but not so much that we want zero backlog.

200.4. Lost Sales

200.4.1. EOQ with lost sales

Relax one dimension from basic EOQ: **excess demand** is no longer forbidden — but unlike planned backorders, customers facing a stockout **do not wait**. Each unit of unmet demand is permanently lost at penalty π per unit (lost margin + goodwill).

200.4.1.1. Setup

New variables (beyond basic EOQ):

- π = lost-sale cost per unit (\$ per unmet unit)
- τ = stockout duration per cycle (decision variable; could be 0)

The inventory profile in each cycle is:

- Receive Q units, drain at rate D until empty after time Q/D.
- Stay stocked-out for time τ — during this period $D\tau$ units of demand walk away.
- Order again. Cycle length $T = Q/D + \tau$.

200.4.1.2. Cost model

Per cycle:

- **Order**: S
- **Holding**: $h \cdot (Q/2) \cdot (Q/D)$ — average inventory $Q/2$ over time Q/D.
- **Lost-sales**: $\pi D\tau$ — $D\tau$ units lost at π each.

Per unit time, divide by $T = Q/D + \tau$:

$$\text{TC}(Q, \tau) = \frac{S + hQ^2/(2D) + \pi D\tau}{Q/D + \tau}$$

Two decision variables now: Q and τ.

200.4.1.3. When is it optimal to plan stockouts? (FOC in τ)

Take $\partial \text{ TC } / \partial \tau$, quotient rule:

$$\frac{\partial}{\partial \tau}\,\mathrm{TC} = \frac{\pi D \cdot (Q/D + \tau) - (S + hQ^2/(2D) + \pi D\tau) \cdot 1}{(Q/D+\tau)^2}$$

Set numerator to zero, expand $\pi D(Q/D + \tau) = \pi Q + \pi D\tau$:

$$\pi Q + \pi D\tau - S - hQ^2/(2D) - \pi D\tau = 0$$

$$\pi Q - S - hQ^2/(2D) = 0$$

This says: **at the boundary $\tau = 0$**, lost sales become attractive only if $\partial\ \mathrm{TC}\ /\partial\tau \le 0$ there, i.e., $\pi Q \le S + hQ^2/(2D)$.

Plug in $Q = Q^*_{\text{basic}} = \sqrt{2SD/h}$. Then $hQ^2/(2D) = S$, so the condition becomes:

$$\pi \cdot \sqrt{\frac{2SD}{h}} \le 2S \quad \Leftrightarrow \quad \pi \le \sqrt{\frac{2Sh}{D}}$$

So lost sales has a **threshold behavior**:

Lost-sale cost π	**Optimal action**
$\pi \ge \sqrt{2Sh/D}$	Operate as basic EOQ ($\tau^* = 0$, no stockouts).
$\pi < \sqrt{2Sh/D}$	Don't operate — lose all demand. Inventory not worth the trouble.

200.4.1.4. Threshold interpretation

The threshold $\sqrt{2Sh/D}$ is exactly the **basic-EOQ cost per unit served**:

$$\frac{\mathrm{TRC}^*_{\text{basic}}}{D} = \frac{\sqrt{2SDh}}{D} = \sqrt{\frac{2Sh}{D}}$$

So the rule is intuitive: **operate iff the per-unit lost-sale cost exceeds the per-unit cost of running EOQ**. Below that, you'd rather lose the customer than pay the inventory overhead.

200.4.1.5. Final formulas

$$\pi \ge \sqrt{\frac{2Sh}{D}} \quad \Rightarrow \quad Q^* = \sqrt{\frac{2SD}{h}}, \quad \tau^* = 0$$

Below the threshold, the model degenerates — there's no interior planned-stockout solution under deterministic constant demand.

200.4.1.6. Why no interior solution?

Under deterministic constant demand with instantaneous replenishment, there is **no benefit** to planned stockouts — you can always order the right amount at the right time. Lost sales becomes interesting only when demand is **stochastic** (newsvendor / (Q, r) models with lost sales) or when there's a **minimum order quantity** forcing you to order in lumps that don't divide annual demand cleanly.

Example

Given (shared EOQ params + a lost-sale penalty):

- Annual demand: $D = 12000$ units/year
- Order cost: S = \$50 / order
- Holding cost: h = \$2 / unit / year
- Lost-sale penalty: vary π to see threshold

Step 1 — compute the threshold

$$\pi_{\text{threshold}} = \sqrt{\frac{2Sh}{D}} = \sqrt{\frac{2 \cdot 50 \cdot 2}{12000}} = \sqrt{\frac{200}{12000}} \approx$$

\$0.13 / unit

Step 2 — decide

- π = \$1 / unit (typical lost margin): $1 \gg 0.13 \rightarrow$ operate as basic EOQ.
- π = \$0.10 / unit (very cheap to lose): $0.10 < 0.13 \rightarrow$ don't operate.

Step 3 — compare to basic EOQ

- Basic EOQ assumes you always serve demand: $Q^* = 775$, $\text{TRC}^* \approx 1549$ \$/year.
- Lost sales adds a **test**: only operate if π exceeds the per-unit cost of EOQ. For any normal product where lost margin > a few cents per unit, basic EOQ wins and the lost-sales option is never used.

Under deterministic demand, lost sales is a **go / no-go decision**, not a planned-shortage strategy. The interesting variants live in stochastic demand models — see newsvendor and (Q, r) policies.

200.5. Quantity Discounts

200.5.1. EOQ with quantity discounts

Relax one dimension from basic EOQ: **unit price is no longer flat**. The supplier offers per-unit discounts at break quantities — buy more, pay less per unit. Now purchase cost cD depends on Q (via the chosen price tier), so we **cannot** drop it from TC.

Two discount rules are common; we treat both.

200.5.1.1. All-units discount

The discounted price applies to **every** unit once you cross the break.

Tiers indexed $k = 1, 2, ..., K$ with break quantities $0 = Q_0 < Q_1 < ... < Q_{K-1}$ and unit costs $c_1 > c_2 > ... > c_K$:

$$c(Q) = c_k \quad \text{if} \quad Q_{k-1} \le Q < Q_k$$

(Last tier $k = K$ has no upper bound: $Q \ge Q_{K-1}$.)

Holding cost is typically $h_k = i \cdot c_k$ where i is the annual carrying-cost rate (% of unit value), so the **effective holding rate scales with the unit cost**.

200.5.1.2. Cost model

Total cost (now keep purchase cost — it depends on tier):

$$\mathrm{TC}_{k(Q)} = c_k D + S\left(\frac{D}{Q}\right) + h_k\left(\frac{Q}{2}\right) \quad \text{where} \quad h_k = ic_k$$

The new term $c_k D$ is the only addition vs basic-EOQ TRC, but it's the dominant one.

200.5.1.3. Algorithm (all-units case)

1. **For each tier k**, compute the within-tier EOQ:

$$Q_k^* = \sqrt{\frac{2SD}{h_k}} = \sqrt{\frac{2SD}{ic_k}}$$

2. **Feasibility**: is $Q_k^* \in [Q_{k-1}, Q_k)$?
   - If yes: candidate $Q_k = Q_k^*$.
   - If $Q_k^* < Q_{k-1}$ (EOQ too small for this tier): candidate $Q_k = Q_{k-1}$ (snap up to the break).
   - If $Q_k^* \geq Q_k$ (EOQ too large for this tier): tier k is dominated by a lower tier ($k+1$ at least), skip it.
3. **Evaluate TC_k** at each candidate.
4. **Pick the candidate with the smallest TC_k** — that's Q^*.

Geometric intuition: each tier has its own U-shaped TRC curve, sitting on top of a tier-specific constant $c_k D$. Lower tiers have a higher floor but the same shape. The optimum is the lowest **feasible** point across all tiers' curves.

200.5.1.4. Incremental discount (briefly)

Each break only discounts **units beyond** the threshold. So the cost is piecewise linear in Q (no jump at breaks):

$$\mathrm{Purchase}(Q) = \sum_{k=1}^{K} c_k \cdot \max(0, \min(Q, Q_k) - Q_{k-1})$$

The TC curve is continuous (no jumps). Optimization is similar but there's no need to snap to break quantities — you just minimize $\mathrm{TC}(Q)$ piecewise within each segment.

200.5.1.5. Final formulas

For the **all-units** case, no closed-form Q^* — must enumerate tiers as above.

Sanity check: with a single tier ($K = 1$), the algorithm reduces to basic EOQ exactly.

Example

Given (shared EOQ params + a 3-tier discount schedule):

- Annual demand: $D = 12000$ units/year
- Order cost: S = \$50 / order
- Carrying-cost rate: $i = 0.20$ (20%/year)
- Discount schedule:

Tier k	Quantity range	Unit cost c_k
1	$Q < 500$	\$10.00
2	$500 \le Q < 1000$	\$9.50
3	$Q \ge 1000$	\$9.00

Step 1 — within-tier EOQ for each tier

Holding cost is $h_k = ic_k$:

$$Q_1^* = \sqrt{\frac{2 \cdot 50 \cdot 12000}{0.2 \cdot 10}} = \sqrt{600000} \approx 775$$

$$Q_2^* = \sqrt{\frac{2 \cdot 50 \cdot 12000}{0.2 \cdot 9.5}} = \sqrt{631579} \approx 794$$

$$Q_3^* = \sqrt{\frac{2 \cdot 50 \cdot 12000}{0.2 \cdot 9}} = \sqrt{666667} \approx 816$$

Step 2 — feasibility check

- Tier 1 (range $Q < 500$): $Q_1^* = 775$ is **outside** (too large). Tier 1 is dominated. Skip.
- Tier 2 (range $500 \le Q < 1000$): $Q_2^* = 794$ is **inside** ✓. Candidate $Q_2 = 794$.
- Tier 3 (range $Q \ge 1000$): $Q_3^* = 816$ is **outside** (below the break). Snap to break: candidate $Q_3 = 1000$.

Step 3 — evaluate TC at each candidate

$$\mathrm{TC}_2(794) = c_2 D + S\left(\frac{D}{Q}\right) + h_2\left(\frac{Q}{2}\right)$$

$$= (9.5) \cdot 12000 + 50 \cdot \left(\frac{12000}{794}\right) + (0.2 \cdot 9.5) \cdot \left(\frac{794}{2}\right)$$

$$= 114000 + 755.7 + 754.3 \approx 115510$$

$$\mathrm{TC}_3(1000) = (9.0) \cdot 12000 + 50 \cdot \left(\frac{12000}{1000}\right) + (0.2 \cdot 9.0) \cdot \left(\frac{1000}{2}\right)$$

$$= 108000 + 600 + 900 \approx 109500$$

Tier 3 wins.

Step 4 — compare to basic EOQ at the cheapest flat price

- Basic EOQ at $c =$ \$10 (no discount available): $Q^* = 775$, $\mathrm{TC} \approx 120000 + 774 + 775 \approx 121549$.
- With discount (use $Q^* = 1000$): $\mathrm{TC} \approx 109500$. Save \$12049/year by ordering just past the break to capture the volume discount.

The savings come almost entirely from the lower unit cost ($9 vs $10 saves $12000/year on purchase alone). The order/holding penalty for ordering above the unconstrained optimum is small because EOQ is forgiving (square-root cushion).

200.6. Capacity-Constrained

Relax one dimension from basic EOQ: **capacity is no longer unlimited**. A shared resource — warehouse space, working-capital budget, or shelf footprint — caps the total inventory you can hold at any moment. Multiple items compete for the same budget.

200.6.1. Setup

n items indexed $i = 1, ..., n$:

- D_i, S_i, h_i as in basic EOQ (item-specific)
- v_i = capacity consumed per unit of item i (e.g., $ per unit if budget-constrained, ft³ per unit if warehouse-constrained)
- V = total capacity available

Constraint (peak inventory after a fresh order arrives):

$$\sum_{i=1}^{n} v_i Q_i \leq V$$

If individual EOQs already satisfy the constraint, the constraint is **not binding** — use them directly. The interesting case is when $\sum_i v_i Q_i^{\text{basic}} > V$.

200.6.2. Lagrangian formulation

Minimize total cost subject to the capacity constraint:

$$\min_{Q_1,...,Q_n} \sum_i [S_i(D_i/Q_i) + h_i(Q_i/2)] \quad \text{s.t.} \quad \sum_i v_i Q_i \leq V$$

Introduce Lagrange multiplier $\lambda \geq 0$ for the constraint:

$$L(Q_1, ..., Q_n, \lambda) = \sum_i [S_i(D_i/Q_i) + h_i(Q_i/2)] + \lambda \left(\sum_i v_i Q_i - V \right)$$

200.6.3. KKT / FOC in Q_i

For each i, $\partial L / \partial Q_i = 0$:

$$-S_i \frac{D_i}{Q_i^2} + \frac{h_i}{2} + \lambda v_i = 0$$

$$S_i \frac{D_i}{Q_i^2} = \frac{h_i}{2} + \lambda v_i = \frac{h_i + 2\lambda v_i}{2}$$

$$Q_i^*(\lambda) = \sqrt{\frac{2 S_i D_i}{h_i + 2\lambda v_i}}$$

This is **basic EOQ with an inflated holding cost** $h_i + 2\lambda v_i$. The Lagrange multiplier λ raises the effective holding cost on every item **in proportion to its capacity consumption** v_i — items that hog more capacity get penalized more, shrinking their Q_i.

200.6.4. Find λ from the constraint

If the constraint binds, $\sum_i v_i Q_i^*(\lambda) = V$. This is one equation in one unknown λ — solve numerically (or in closed form for some special cases).

Special case: budget constraint with proportional holding. If $v_i = c_i$ (capacity is dollars), $h_i = ic_i$ (carrying-rate model), and we substitute into $Q_i^*(\lambda)$:

$$Q_i^* = \sqrt{\frac{2S_iD_i}{ic_i + 2\lambda c_i}} = \sqrt{\frac{2S_iD_i}{c_i}} \cdot \frac{1}{\sqrt{i + 2\lambda}}$$

Then $v_i Q_i^* = c_i Q_i^* = \sqrt{2S_iD_ic_i}/\sqrt{i + 2\lambda}$. Sum:

$$\sum_i v_i Q_i^* = \left(\frac{1}{\sqrt{i + 2\lambda}}\right) \sum_i \sqrt{2S_iD_ic_i} = V$$

Solve for λ:

$$\sqrt{i + 2\lambda} = \frac{\sum_i \sqrt{2S_iD_ic_i}}{V}$$

$$\lambda = \frac{1}{2}\left[\left(\frac{\sum_i \sqrt{2S_iD_ic_i}}{V}\right)^2 - i\right]$$

For other constraint forms, λ comes out of a 1-D root-finding pass over $\sum_i v_i Q_i^*(\lambda) - V = 0$ (monotone decreasing in λ, so bisection works).

200.6.5. Algorithm

1. Compute unconstrained $Q_i^{\text{basic}} = \sqrt{2S_iD_i/h_i}$ for each i.
2. Check $\sum_i v_i Q_i^{\text{basic}} \le V$. If yes, done — constraint not binding.
3. Else, solve for λ (closed form above, or numerical).
4. Each $Q_i^* = \sqrt{2S_iD_i/(h_i + 2\lambda v_i)}$.

200.6.6. Final formulas

$$Q_i^*(\lambda) = \sqrt{\frac{2S_iD_i}{h_i + 2\lambda v_i}} \quad \text{where} \quad \lambda \ge 0 \text{ satisfies } \sum_i v_i Q_i^*(\lambda) = V$$

Sanity check: if $V \to \infty$ (no constraint), $\lambda \to 0$ and $Q_i^* \to \sqrt{2S_iD_i/h_i}$ — basic EOQs ✓.

Example

Given (2 items sharing a budget-constrained warehouse):

- Item 1: $D_1 = 12000$, S_1 = \$50, c_1 = \$10, $i = 0.20 \to h_1 = 2$
- Item 2: $D_2 = 6000$, S_2 = \$50, c_2 = \$20, $i = 0.20 \to h_2 = 4$

- Constraint: peak inventory value $\sum_i c_i Q_i \le V$ where V = \$10,000

Step 1 — unconstrained EOQs and constraint check

$$Q_1^{\text{basic}} = \sqrt{\frac{2 \cdot 50 \cdot 12000}{2}} = 775$$

$$Q_2^{\text{basic}} = \sqrt{\frac{2 \cdot 50 \cdot 6000}{4}} = 387$$

Peak inventory value $= 10 \cdot 775 + 20 \cdot 387 = 7750 + 7740 = 15490$. Exceeds the \$10,000 budget — **constraint binds**.

Step 2 — find λ via the closed-form

$$\sum_i \sqrt{2 S_i D_i c_i} = \sqrt{2 \cdot 50 \cdot 12000 \cdot 10} + \sqrt{2 \cdot 50 \cdot 6000 \cdot 20} = \sqrt{12000000} + \sqrt{12000000} \approx 3464 + 3464 = 6928$$

$$\sqrt{i + 2\lambda} = \frac{6928}{10000} = 0.6928 \quad \rightarrow \quad i + 2\lambda = 0.48$$

$$2\lambda = 0.48 - 0.20 = 0.28 \quad \rightarrow \quad \lambda = 0.14$$

Step 3 — constrained order quantities

Effective holding cost per item: $h_i + 2\lambda v_i = h_i + 0.28 c_i$:

- Item 1: $h_1^{\text{eff}} = 2 + 0.28 \cdot 10 = 4.8$
- Item 2: $h_2^{\text{eff}} = 4 + 0.28 \cdot 20 = 9.6$

$$Q_1^* = \sqrt{\frac{2 \cdot 50 \cdot 12000}{4.8}} = \sqrt{250000} = 500$$

$$Q_2^* = \sqrt{\frac{2 \cdot 50 \cdot 6000}{9.6}} = \sqrt{62500} = 250$$

Step 4 — verify the constraint

$10 \cdot 500 + 20 \cdot 250 = 5000 + 5000 = 10000$ ✓ exactly hits the budget.

Step 5 — total cost and comparison to basic EOQ

- Item 1 cost: $50 \cdot 12000/500 + 2 \cdot 500/2 = 1200 + 500 = 1700$
- Item 2 cost: $50 \cdot 6000/250 + 4 \cdot 250/2 = 1200 + 500 = 1700$
- Total constrained: **\$3400**

Unconstrained (basic EOQ for each):

- Item 1: $\sqrt{2 \cdot 50 \cdot 12000 \cdot 2} \approx 1549$
- Item 2: $\sqrt{2 \cdot 50 \cdot 6000 \cdot 4} \approx 1549$
- Total unconstrained: **\$3098**

Capacity penalty: \$302/year (about 10% over unconstrained). The Lagrange multiplier $\lambda = 0.14$ acts as a shadow price — each \$ of additional warehouse budget would save approximately λ × marginal value per year.

200.7. Joint Replenishment

200.7.1. Joint replenishment (multi-item EOQ with shared setup)

Relax one dimension from basic EOQ: **number of items** is no longer one. Multiple items share a single supplier (or a single delivery truck, or a single setup operation), so the order cost S is paid **once per coordinated order** — not separately per item.

Single supplier, multiple SKUs: order them all together on the same cadence, save on setup.

200.7.1.1. Setup

n items indexed $i = 1, ..., n$:

- D_i = annual demand for item i
- h_i = holding cost per unit per year for item i
- **Common setup** cost: S (paid once when **any** order goes out, regardless of how many items)
- Decision: a **common cycle time** T — every T years, order each item in the quantity needed for one cycle, $Q_i = D_i T$.

This is the simplest joint-replenishment formulation. (Variants with item-specific minor setups s_i in addition to a major S exist — see Roundy / Federgruen-Zheng — but we'll keep it minimal here.)

200.7.1.2. Cost model

Per cycle:

- **One** setup cost: S
- Holding for item i over the cycle: $h_i \cdot Q_i/2 \cdot T = h_i D_i T/2 \cdot T$. Wait — average inventory $Q_i/2$, time per cycle T, so per-cycle holding cost is $h_i \cdot (Q_i/2) \cdot T = h_i D_i T^2/2$.

Per unit time, divide by T:

$$\mathrm{TC}(T) = \frac{S}{T} + \sum_{i=1}^{n} h_i \frac{D_i T}{2} = \frac{S}{T} + \left(\frac{T}{2}\right) \sum_{i=1}^{n} h_i D_i$$

The single decision T trades off setup cost (decreases with T) against aggregate holding (increases with T).

200.7.1.3. Derive T^*

$$\frac{d}{dT}\ \mathrm{TC} = -\frac{S}{T^2} + \frac{1}{2}\sum_i h_i D_i = 0$$

Solve for T^*:

$$T^{\{*2\}} = \frac{2S}{\sum_i h_i D_i}$$

$$T^* = \sqrt{\frac{2S}{\sum_i h_i D_i}}$$

200.7.1.4. Final formulas

$$T^* = \sqrt{\frac{2S}{\sum_i h_i D_i}} \quad Q_i^* = D_i T^* = D_i \sqrt{\frac{2S}{\sum_j h_j D_j}} \quad \mathrm{TC}^* = \sqrt{2S \sum_i h_i D_i}$$

Sanity check: for a single item ($n = 1$), $T^* = \sqrt{2S/(h_1 D_1)}$, $Q_1^* = D_1 T^* = \sqrt{2SD_1/h_1}$ — exactly basic EOQ ✓.

The aggregation comes through the **single** $\sqrt{\sum_i h_i D_i}$ — items with small $h_i D_i$ contribute little to the optimal cycle, so they're ordered alongside the big-$h_i D_i$ items at no extra setup cost.

200.7.1.5. Why is this cheaper than independent EOQs?

Order each item independently with its own EOQ: each pays S per order, total cost is $\sum_i \sqrt{2SD_i h_i}$.

Joint replenishment: $\sqrt{2S \sum_i h_i D_i}$.

By Cauchy-Schwarz / Jensen, $\sqrt{\sum_i a_i^2} \le \sum_i a_i$, so:

$$\sqrt{2S \sum_i h_i D_i} \le \sum_i \sqrt{2S h_i D_i}$$

Joint always wins. The savings grow with the number of items and the unevenness of $h_i D_i$ across them.

Example

Given (3 items sharing a single supplier with shared setup):

- Common setup: S = \$50 / order
- Item 1: $D_1 = 12000$, h_1 = \$2
- Item 2: $D_2 = 6000$, h_2 = \$3
- Item 3: $D_3 = 3000$, h_3 = \$4

Step 1 — aggregate holding rate

$$\sum_i h_i D_i = 2 \cdot 12000 + 3 \cdot 6000 + 4 \cdot 3000 = 24000 + 18000 + 12000 = 54000$$

Step 2 — common cycle time

$$T^* = \sqrt{\frac{2 \cdot 50}{54000}} = \sqrt{\frac{100}{54000}} \approx 0.0430 \text{ years} \approx 16 \text{ days}$$

Step 3 — order quantities

$$Q_1^* = D_1 T^* = 12000 \cdot 0.043 \approx 516 \text{ units}$$

$$Q_2^* = D_2 T^* = 6000 \cdot 0.043 \approx 258 \text{ units}$$

$$Q_3^* = D_3 T^* = 3000 \cdot 0.043 \approx 129 \text{ units}$$

Step 4 — total cost

$$\text{TC}^* = \sqrt{2 \cdot 50 \cdot 54000} = \sqrt{5400000} \approx 2324$$

$/year

Compare to independent EOQs (each item ordered separately with full S):

- Item 1: $Q_1 = \sqrt{2 \cdot 50 \cdot 12000/2} = 775$, $\text{TC}_1 = \sqrt{2 \cdot 50 \cdot 12000 \cdot 2} \approx 1549$
- Item 2: $Q_2 = \sqrt{2 \cdot 50 \cdot 6000/3} \approx 447$, $\text{TC}_2 = \sqrt{2 \cdot 50 \cdot 6000 \cdot 3} \approx 1342$
- Item 3: $Q_3 = \sqrt{2 \cdot 50 \cdot 3000/4} \approx 274$, $\text{TC}_3 = \sqrt{2 \cdot 50 \cdot 3000 \cdot 4} \approx 1095$
- Total independent: $1549 + $1342 + $1095 = **$3986**.

Joint replenishment saves $1662/year (42% lower) by sharing the setup across items. The savings scale with the number of items — at 10 SKUs of similar size, savings would approach 70%.

200.8. Multi-Echelon

Relax one dimension from basic EOQ: **number of echelons** is no longer one. Inventory is held at multiple stages of a supply chain (warehouse → retailer, supplier → factory → distributor, etc.). Orders cascade up the chain, and each stage has its own setup and holding costs.

Key concept: **echelon inventory** — at each stage, count not just on-hand but also all downstream inventory you've already paid to put in motion.

200.8.1. Setup (two-echelon: warehouse → retailer)

- d = customer demand rate at the retailer
- **Retailer**: setup S_r, holding h_r per unit per year, cycle time T_r
- **Warehouse**: setup S_w, **echelon** holding h_w per unit per year (the **added** cost of holding at the warehouse vs nowhere), cycle time T_w

The retailer pulls from the warehouse every T_r. The warehouse re-orders from the upstream supplier every T_w. **Nesting** requirement: T_w must be an integer multiple of T_r (otherwise the warehouse stocks fractionally, which is wasteful).

200.8.2. Cost model

Annual cost = sum across stages:

$$\text{TC}(T_r, T_w) = \underbrace{\frac{S_r}{T_r} + h_r d \frac{T_r}{2}}_{\text{retailer}} + \underbrace{\frac{S_w}{T_w} + h_w d \frac{T_w}{2}}_{\text{warehouse}}$$

Constraint: $T_w = kT_r$ for some integer $k \geq 1$.

200.8.3. Unconstrained optimum (relax integer k)

If we **ignored** nesting, each stage would independently optimize:

$$T_r^{\text{basic}} = \sqrt{\frac{2S_r}{h_r d}} \quad T_w^{\text{basic}} = \sqrt{\frac{2S_w}{h_w d}}$$

This is just basic EOQ at each stage. But a non-integer ratio T_w/T_r wastes capacity — the warehouse would carry partial cycles. **Nesting** is required for clean operation.

200.8.4. Roundy's power-of-two policies

Restrict cycle times to powers of two of a base period: $T_i = 2^{k_i} T_0$ for integer $k_i \geq 0$, where T_0 is fixed (e.g., 1 day).

Then **any two cycles are nested**: a stage with $k_i = 3$ orders 8 base periods; another with $k_i = 5$ orders every 32 base periods — the second is always a multiple of the first.

Roundy's 98% theorem: the best power-of-two policy achieves **at least 98%** of the unconstrained optimum (worst-case cost ratio $\leq \sqrt{2/\ln(2)}/\left(1 + 1/\sqrt{2}\right) \approx 1.06$). So a 6% penalty is the worst case for using powers of two.

200.8.5. Algorithm

1. Compute unconstrained T_i^{basic} at each stage.
2. Round each to the nearest power of two: $T_i^* = 2^{k_i^*} T_0$ where $k_i^* = \text{round}\left(\log_2\left(T_i^{\text{basic}}/T_0\right)\right)$.
3. Compute resulting cost TC^* and verify it's within 6% of $\sum_i \sqrt{2 S_i h_i d}$.

200.8.6. Final formulas (two-stage, nested with $T_w = kT_r$)

Substitute $T_w = kT_r$ into TC:

$$\text{TC}(T_r, k) = \frac{S_r + S_w/k}{T_r} + (h_r + k h_w) d \frac{T_r}{2}$$

Optimize over T_r for fixed k:

$$T_r^*(k) = \sqrt{\frac{2(S_r + S_w/k)}{(h_r + k h_w) d}}$$

$$\text{TC}^*(k) = \sqrt{2(S_r + S_w/k)(h_r + k h_w) d}$$

Now optimize over integer $k \geq 1$ — small search (typically $k \in \{1, 2, 4, 8\}$).

Example

Given (two-echelon: warehouse → retailer):

- Retail demand: $d = 12000$ units/year
- Retailer: S_r = \$50 / order, h_r = \$2 / unit / year
- Warehouse: S_w = \$200 / order, h_w = \$1 / unit / year (echelon — added cost over nothing)

Step 1 — unconstrained T_i^{basic} at each stage

$$T_r^{\text{basic}} = \sqrt{\frac{2 \cdot 50}{2 \cdot 12000}} \approx 0.0645 \text{ yr} \approx 24 \text{ days}$$

$$T_w^{\text{basic}} = \sqrt{\frac{2 \cdot 200}{1 \cdot 12000}} \approx 0.1826 \text{ yr} \approx 67 \text{ days}$$

Ratio $\frac{T_w^{\text{basic}}}{T_r^{\text{basic}}} \approx 67/24 \approx 2.83$ — roughly 3, but not an integer.

Step 2 — search nested integer k

Try $k = 1, 2, 3, 4$:

$\text{TC}^*(k) = \sqrt{2(S_r + S_w/k)(h_r + kh_w)d}$:

k	$S_r + S_w/k$	$h_r + kh_w$	$\text{TC}^*(k)$
1	50 + 200 = 250	2 + 1 = 3	$\sqrt{2 \cdot 250 \cdot 3 \cdot 12000} \approx 4243$
2	50 + 100 = 150	2 + 2 = 4	$\sqrt{2 \cdot 150 \cdot 4 \cdot 12000} \approx 3795$
3	50 + 67 = 117	2 + 3 = 5	$\sqrt{2 \cdot 117 \cdot 5 \cdot 12000} \approx 3742$
4	50 + 50 = 100	2 + 4 = 6	$\sqrt{2 \cdot 100 \cdot 6 \cdot 12000} \approx 3795$

Optimal $k^* = 3$ (warehouse cycle is 3× the retailer cycle).

Step 3 — cycle times at $k^* = 3$

$$T_r^* = \sqrt{\frac{2(50 + 200/3)}{(2 + 3 \cdot 1) \cdot 12000}} = \sqrt{\frac{2 \cdot 117}{5 \cdot 12000}} \approx 0.0625 \text{ yr} \approx 23 \text{ days}$$

$$T_w^* = 3 \cdot T_r^* \approx 68 \text{ days}$$

Order quantities: retailer $Q_r^* = dT_r^* \approx 750$, warehouse $Q_w^* = dT_w^* \approx 2280$.

Step 4 — compare to basic EOQ at each stage independently

Treating the two stages independently (no nesting):

- Retailer: $\text{TC}_r = \sqrt{2 \cdot 50 \cdot 2 \cdot 12000} \approx 1549$
- Warehouse: $\text{TC}_w = \sqrt{2 \cdot 200 \cdot 1 \cdot 12000} \approx 2191$
- Sum (independent): \$3740/year

Multi-echelon nested: $\text{TC}^*(3) \approx 3742$.

They almost coincide! The 6% Roundy bound is achieved here at 0.05% — the integer-k constraint costs almost nothing because $T_w^{\text{basic}}/T_r^{\text{basic}} \approx 2.83$ was already close to a small integer (3). **This is the Roundy theorem at work**: with two stages, the worst-case nesting penalty is bounded by 6%, and in practice it's usually much less.

The deeper takeaway: **power-of-two policies generalize this beyond two stages** — at any number of echelons, restrict each cycle to $T_i = 2^{k_i} T_0$ and the worst-case penalty stays below 6%. Predictable, nestable, near-optimal.

200.9. Perishables

200.9.1. EOQ for perishable items

Relax one dimension from basic EOQ: items **deteriorate over time**. Each unit decays at constant exponential rate θ — even sitting on the shelf untouched, fraction θ of stock is lost per unit time.

Examples: pharmaceuticals with shelf life, fresh produce, photographic film, blood products.

200.9.1.1. Setup

New variable beyond basic EOQ:

- θ = decay rate (fraction of stock lost per unit time)

Inventory dynamics with continuous demand d and continuous decay:

$$\frac{dI}{dt} = -d - \theta I(t)$$

The stock drops both from sales (rate d) and from spoilage (rate θI). Solve this ODE with $I(0) = Q$:

$$I(t) = \left(Q + \frac{d}{\theta}\right)e^{-\theta t} - \frac{d}{\theta}$$

Cycle ends when $I(T) = 0$:

$$\left(Q + \frac{d}{\theta}\right)e^{-\theta T} = \frac{d}{\theta} \quad \rightarrow \quad T = \left(\frac{1}{\theta}\right)\ln\left(1 + \frac{Q\theta}{d}\right)$$

The cycle is **shorter** than basic EOQ's Q/d — decay shortens how long each order lasts.

200.9.1.2. Cost model — exact form

Per cycle:

- **Order**: S
- **Holding**: $h \cdot \int_0^T I(t)dt$

Compute the integral. Using $I(t) = (Q + d/\theta)e^{-\theta t} - d/\theta$:

$$\int_0^T I(t)dt = -\frac{Q + d/\theta}{\theta} \cdot \left(e^{-\theta T} - 1\right) - dT/\theta$$

After substituting $e^{-\theta T} = d/(Q\theta + d)$ and simplifying:

$$\int_0^T I(t)dt = \frac{Q}{\theta} - \frac{dT}{\theta}$$

The exact TC per unit time is then $S/T + h(Q/(\theta T) - d/\theta)$ — but T is transcendental in Q, so there's no clean closed-form Q^*. Use the small-θ approximation instead.

200.9.1.3. Small-decay approximation

For modest decay ($\theta T \ll 1$), expand T:

$$T = \left(\frac{1}{\theta}\right)\ln\left(1 + Q\frac{\theta}{d}\right) \approx \frac{Q}{d} - \frac{Q^2\theta}{2d^2} + \mathcal{O}(\theta^2)$$

Decay loss per cycle = $Q - dT \approx Q^2\theta/(2d)$ — quadratic in Q.

Annualizing: number of cycles per year is $1/T \approx d/Q + \theta/2$, so:

- Annual order cost: $S/T \approx Sd/Q$
- Annual holding cost: $hQ/2$ (basic EOQ approximation, leading order)
- **Annual decay loss**: $\approx Q\theta/2$ units, valued at unit cost c, costs $cQ\theta/2$

The decay term **shows up as an extra holding-like cost**: it scales with Q. Combine it with the standard holding term:

$$\mathrm{TRC}(Q) \approx S\left(\frac{d}{Q}\right) + (h + c\theta)\left(\frac{Q}{2}\right)$$

The decay rate effectively **raises the holding cost** from h to $h + c\theta$.

200.9.1.4. Derive Q^*

Same calculus as basic EOQ with the inflated holding cost:

$$Q^* \approx \sqrt{\frac{2Sd}{h + c\theta}}$$

200.9.1.5. Final formulas (small-θ approximation)

$$Q^* \approx \sqrt{\frac{2Sd}{h + c\theta}} \quad T^* \approx \frac{Q^*}{d} \quad \mathrm{TRC}^* \approx \sqrt{2Sd(h + c\theta)}$$

Sanity check: as $\theta \to 0$ (no decay), $h + c\theta \to h$ and we recover basic EOQ exactly ✓. As $\theta \to \infty$ (instant decay), $Q^* \to 0$ — order infinitesimally small, infinitely often, so nothing decays.

The result is intuitive: **decay acts like an extra carrying cost of $c\theta$ per unit per unit time**. This captures most of the effect when decay is moderate; for high θ, you'd need the exact transcendental form.

Example

Given (shared EOQ params + a 5%/year decay rate):

- Annual demand: $d = 12000$ units/year
- Order cost: S = \$50 / order
- Holding cost: h = \$2 / unit / year
- Unit cost: c = \$10
- Decay rate: $\theta = 0.05$ /year (5% of stock spoils per year)

Step 1 — effective holding cost

$$h_{\text{eff}} = h + c\theta = 2 + 10 \cdot 0.05 = 2 + 0.5 = 2.5$$

Decay raises the effective holding cost by 25% over the basic EOQ rate.

Step 2 — order quantity

$$Q^* \approx \sqrt{\frac{2 \cdot 50 \cdot 12000}{2.5}} = \sqrt{480000} \approx 693 \text{ units}$$

Step 3 — cycle and cost

$$T^* \approx \frac{Q^*}{d} = \frac{693}{12000} \approx 0.058 \text{ years} \approx 21 \text{ days}$$

$$\text{TRC}^* \approx \sqrt{2 \cdot 50 \cdot 12000 \cdot 2.5} \approx 1732$$

$/year

Decay loss per year: $Q^*\theta/2 \approx 693 \cdot 0.05/2 \approx 17$ units (worth $170).

Compare to basic EOQ on the same params (ignoring decay):

- Basic EOQ ($\theta = 0$): $Q^* = 775$, $\text{TRC}^* \approx$ $1549.
- With decay ($\theta = 0.05$): $Q^* \approx 693$ (smaller), $\text{TRC}^* \approx$ $1732 (12% higher).

Two effects, both pointing the right way:

1. **Order smaller batches** (693 vs 775) so less stock sits long enough to spoil.
2. **Total cost rises** — decay is a real cost you can't optimize away, only minimize.

For higher θ (e.g., fresh produce with $\theta \geq 1$/year), the small-θ approximation breaks down and the exact transcendental form is needed; in practice, perishable inventory is more often modeled with a fixed shelf life L rather than continuous decay.

200.10. Wagner–Whitin

200.10.1. Wagner-Whitin (finite horizon, time-varying demand)

Relax two dimensions from basic EOQ at once: **planning horizon** is now finite, and **demand** is no longer constant — it varies period by period (still known and deterministic). No closed-form Q^*; the answer is a **schedule** of when to order, found by dynamic programming.

200.10.1.1. Setup

- Periods $t = 1, 2, ..., T$ (e.g., months in a finite production plan).
- Known demand d_t in each period.
- Order cost S per order placed (independent of size).
- Holding cost h per unit carried **one period forward**.
- Decision: in which periods to place orders, and how much each order covers.

200.10.1.2. Zero-inventory ordering (ZIO) property

Claim: In an optimal Wagner-Whitin schedule, an order is placed only when starting inventory is zero.

Proof sketch: If you start a period with leftover stock and then order more, you could have ordered less the previous time and placed the same total order one period later — saving holding cost without changing the order count. So orders always coincide with the inventory hitting zero.

Consequence: each order covers a **consecutive block** of periods $[j, k]$ (for some $j \leq k$). The decision space collapses from "how much in each period" to "which blocks to use" — a much smaller problem solvable by DP.

200.10.1.3. Cost of a single order block $[j, k]$

If we order in period j to cover demands $d_j, d_{j+1}, ..., d_k$:

- **Order cost**: S (one order).
- **Holding cost**: each unit consumed in period t ($j \leq t \leq k$) sat in inventory for $t - j$ periods. Total holding for the block:

$$\text{Hold}(j, k) = h \sum_{t=j}^{k} (t - j) d_t$$

Block cost:

$$\text{Cost}(j, k) = S + h \sum_{t=j}^{k} (t - j) d_t$$

200.10.1.4. DP recursion

Let $F(t)$ = minimum cost to cover demand for periods $1, ..., t$. Boundary: $F(0) = 0$.

For each t, the last order block ends at t and starts at some $j \in \{1, ..., t\}$:

$$F(t) = \min_{j=1}^{t} [F(j - 1) + \text{Cost}(j, t)]$$

Solve forward: $F(1), F(2), ..., F(T)$. Track the argmin $j^*(t)$ at each step to reconstruct the schedule.

200.10.1.5. Algorithm

1. Compute "Cost(j, t)" for all pairs $j \leq t$.
2. Forward pass: for each t, compute $F(t)$ and store $j^*(t)$.
3. Backtrack from T: order in $j^*(T)$ covering $[j^*(T), T]$, then recurse on $F(j^*(T) - 1)$.

Complexity: $O(T^2)$ in the number of periods.

Example

Given (small 4-period example):

- 4 periods with demands $d = (10, 20, 5, 15)$
- Order cost: S = \$50 / order
- Holding cost: h = \$1 / unit / period

Step 1 — block costs

$\text{Cost}(j, t) = S + h \sum_{s=j}^{t} (s - j) d_s$:

$j \downarrow \backslash t \rightarrow$	1	2	3	4
1	50	50 + 20 = 70	70 + 10 = 80	80 + 45 = 125

2	—	50	50 + 5 = 55	55 + 30 = 85
3	—	—	50	50 + 15 = 65
4	—	—	—	50

Each cell is the cost of one order placed in period j that covers demand through period t.

Step 2 — DP forward pass

$F(0) = 0.$

$F(1) = F(0) + \text{Cost}(1,1) = 0 + 50 = \mathbf{50}.\ (j^*(1) = 1)$

$$F(2) = \min\begin{cases} F(0) + \text{Cost}(1,2) = 0+70 = 70 \\ F(1) + \text{Cost}(2,2) = 50+50 = 100 \end{cases} = \mathbf{70}.\ (j^*(2) = 1)$$

$$F(3) = \min\begin{cases} F(0) + \text{Cost}(1,3) = 80 \\ F(1) + \text{Cost}(2,3) = 50+55 = 105 \\ F(2) + \text{Cost}(3,3) = 70+50 = 120 \end{cases} = \mathbf{80}.\ (j^*(3) = 1)$$

$$F(4) = \min\begin{cases} F(0) + \text{Cost}(1,4) = 125 \\ F(1) + \text{Cost}(2,4) = 50+85 = 135 \\ F(2) + \text{Cost}(3,4) = 70+65 = 135 \\ F(3) + \text{Cost}(4,4) = 80+50 = 130 \end{cases} = \mathbf{125}.\ (j^*(4) = 1)$$

Step 3 — reconstruct schedule

$j^*(4) = 1 \rightarrow$ order in period 1, covers periods 1–4. Total cost = 125.

Step 4 — compare to basic EOQ on the average demand

Total demand $= 10 + 20 + 5 + 15 = 50$ over 4 periods $\rightarrow$ average $D = 12.5$ per period (or 50/year if $T = 4$ months $= 1/3$ year, $D = 150$/year). EOQ would give a single Q^* and a stationary cycle — but this problem's demand is **not** stationary (period 4 is 3× period 3), so a stationary order cycle wastes either order cost or holding cost.

Wagner-Whitin's answer (one big order in period 1) is **cheaper than any stationary EOQ schedule** because it adapts to the lumpy demand profile.

This is the value of dropping the "infinite horizon, stationary demand" assumption: when demand has structure, the optimal policy reflects that structure, not a one-size-fits-all Q^*.

201. Policies

201.1. Overview

201.1.1. Policy dimensions

Stochastic-demand inventory policies sit at the intersection of **five** design choices. The choices made along these dimensions name the policy.

201.1.1.1. 1. Inventory review

- **Periodic** review (**R**-policies): inventory checked at regular intervals R. Orders are placed only at review points.
- **Continuous** review (**s**-policies): inventory monitored continuously. Orders are triggered immediately when the level falls below a threshold.

201.1.1.2. 2. Order quantity

- **Fixed** order quantity (**Q**): a predetermined, constant quantity is ordered each time (lot-for-lot replenishment).
- **Order-up-to** level (**S**): inventory replenished up to a pre-defined maximum S, adapting to the current stock gap.
- **Reorder-point shorthand** (**s**): the trigger level — used in both Q-style and S-style policies.

201.1.1.3. 3. Flexibility

- **Q**-policies: **rigid** — order size is always the same.
- **S**-policies: **responsive** — order size adapts to the actual inventory level.

201.1.1.4. 4. Lead time

- **Constant**: time between placing an order and receiving it is fixed.
- **Variable**: lead time fluctuates, requiring extra safety stock (see [stocks/safety_stock.typ](../stocks/safety_stock.typ)).

201.1.1.5. 5. Demand

- **Deterministic**: known and constant — yields the EOQ family.
- **Stochastic**: random — yields the policies in this section.

201.1.1.6. Policy comparison table

Policy	Review type	Trigger	Order size
(s, Q) — see [q_r.typ](q_r.typ)	Continuous	Inventory $\leq s$	Fixed batch Q
(s, S) — see [s_S.typ](s_S.typ)	Continuous	Inventory $\leq s$	Order-up-to S
(nQ, r) — see [nQ_r.typ] (nQ_r.typ)	Continuous	Inventory $\leq r$	Smallest nQ above r
$(S-1, S)$ — see [base_stock.typ] (base_stock.typ)	Continuous	Every consumption	One unit (one-for-one)
(R, S) — see [R_S.typ] (R_S.typ)	Periodic (every R)	Always at review	Order-up-to S

$(\boldsymbol{R}, \boldsymbol{s}, \boldsymbol{S})$ — see [R_s_S.typ] (R_s_S.typ)	Periodic (every $\boldsymbol{R}$)	If inventory $\leq \boldsymbol{s}$ at review	Order-up-to $\boldsymbol{S}$
$(\boldsymbol{R}, \boldsymbol{nQ}, \boldsymbol{s})$ — see [R_nQ_s.typ] (R_nQ_s.typ)	Periodic (every $\boldsymbol{R}$)	If inventory $\leq \boldsymbol{s}$ at review	Smallest $\boldsymbol{nQ}$ above $\boldsymbol{s}$

A policy is named by listing its parameters in sorted order: review interval R (if periodic), reorder point s or r, target level S, fixed quantity Q, with n for integer multipliers.

The **(s, Q)** policy is also widely written **(Q, r)** — same thing, different naming convention. Likewise **(R, s)** is a **family** (specifies trigger but not order rule); concrete instances are **(R, s, Q)** and **(R, s, S)**.

201.2. Base Stock

201.2.1. (S–1, S) Policy — base stock / one-for-one

The simplest continuous-review policy: **every consumed unit triggers an immediate replenishment of one unit**. Sometimes called the **base-stock** policy.

Decision rule: maintain inventory position at exactly S. As soon as one unit is sold, place an order for one unit.

Equivalently: trigger an order at $s = S - 1$ (whenever inventory dips below S). Hence the name.

Single parameter:

- S = base-stock level (the only decision)

201.2.1.1. When does this make sense?

Use base-stock when **fixed ordering cost is negligible** (or items are too critical to risk by batching):

- **Spare parts and critical components**: stockouts are catastrophic, individual items are cheap to order.
- **High-value, low-volume items**: holding cost dominates; ordering one-at-a-time is fine.
- **Vendor-managed inventory** with continuous data exchange: ordering happens automatically per consumption.

Don't use base-stock if you have meaningful per-order setup costs — the EOQ logic says batch up.

201.2.1.2. Inventory position vs. on-hand

In (S–1, S):

- **Inventory position** = on-hand + on-order = S always (just before each consumption).
- **On-hand** = $S - \text{outstanding orders}$.

- An **outstanding order** is one consumed within the past L time units that hasn't yet arrived.

If demand during lead time is D_L, outstanding orders just before the next consumption is exactly D_L. So:

$$\text{On-hand} = (S - D_L)^+$$

Stockouts occur when $D_L > S$.

201.2.1.3. Set S — Newsvendor style

Choose S to balance holding cost vs. shortage cost.

For Poisson lead-time demand $D_L \sim \text{Poisson}(\mu_L)$:

$$S^* = \text{smallest integer with } \; P(D_L \le S) \ge \text{CR}$$

Critical ratio: $\text{CR} = C_u/(C_u + C_o)$ where C_u is shortage cost per unit, C_o is holding cost per unit per cycle.

For a **cycle service-level** requirement (probability of no stockout in lead time = $1 - \alpha$):

$$S = \mu_L + z\sigma_L \quad \text{(normal approximation)}$$

For Poisson: $\sigma_L = \sqrt{\mu_L}$, so $S \approx \mu_L + z\sqrt{\mu_L}$.

201.2.1.4. Average inventory

Steady-state expected on-hand:

$$E[\text{on-hand}] = E\left[(S - D_L)^+\right] = S - \mu_L + \sigma_L L(z)$$

where $L(z) = \varphi(z) - z(1 - \Phi(z))$ is the standard normal loss function (see [expected_profit.typ](../newsvendor/expected_profit.typ)).

Annual holding cost: $h \cdot E[\text{on-hand}]$.

201.2.1.5. Final formulas

$$S = \mu_L + z\sigma_L \quad \text{SS} = z\sigma_L \quad E[\text{on-hand}] \approx S - \mu_L + \sigma_L L(z)$$

Example

Given (same policy-comparison params, but interpret as low-volume spare-part regime):

- Demand: $d = 1$ unit / day (Poisson) — instead of 33/day to make the spare-part case realistic
- Lead time: $L = 14$ days
- Service level: 95% $\rightarrow z = 1.645$

Lead-time demand: $D_L \sim \text{Poisson}(\mu_L = 14)$, $\sigma_L = \sqrt{14} \approx 3.74$.

Step 1 — base-stock level

Normal approximation:

$$S = \mu_L + z\sigma_L = 14 + 1.645 \cdot 3.74 \approx 20 \text{ units}$$

Exact (Poisson lookup): smallest S with $P(D_L \le S) \ge 0.95$.

- $P(D_L \le 19) \approx 0.923$
- $P(D_L \le 20) \approx 0.952$ ✓
- $S^* = 20$.

Step 2 — interpret

Maintain inventory position at 20 at all times. Every time a unit is consumed, immediately reorder one. Average backlog of in-transit orders is $\mu_L = 14$. Average on-hand:

$$E[\text{on-hand}] \approx 20 - 14 = 6 \text{ units (plus loss-function correction)}$$

Step 3 — what if demand drops to 1 unit per month **instead of per day?**

$\mu_L = 14 \cdot \left(\frac{1}{30}\right) \approx 0.47$, $\sigma_L = \sqrt{0.47} \approx 0.68$.

- $S = 0.47 + 1.645 \cdot 0.68 \approx 1.6 \to 2$ (round up).
- Most of the time, on-hand = 2; very occasional stockouts.

Compare to (Q, r)

In the **high-volume case** ($d = 33$/day): (Q, r) gives $r = 493$, $Q = 775$. Base-stock would give $S = 493$, but with $S - 1 = 492$ as the trigger — meaning **15 orders per day on average**. Per-order cost dominates → use (Q, r).

In the **low-volume spare-part case** ($d = 1$/day): per-order cost is moot (warehouse already has the receiving infrastructure), and base-stock's responsiveness wins. (S–1, S) is the right choice.

Rule of thumb: base-stock is optimal when $S_{\text{setup}} \to 0$ and converges to r (no batching). For any non-trivial setup cost, batched policies (Q, r) or (s, S) reduce per-unit ordering cost.

201.3. (s, S)

Continuous review, **order-up-to** S. Generalizes (Q, r): instead of always ordering the same Q, top up to a target level.

Decision rule: monitor inventory continuously. When inventory position drops to (or below) the reorder point s, order **enough to bring inventory position back up to** S.

Two parameters:

- s = reorder point (trigger)
- S = order-up-to level

The actual order quantity **varies**: $Q_{\text{actual}} = S - (\text{inventory position when triggered})$. If inventory drops exactly to s before triggering, $Q_{\text{actual}} = S - s$. If demand causes a drop **below** s (overshoot), $Q_{\text{actual}} > S - s$.

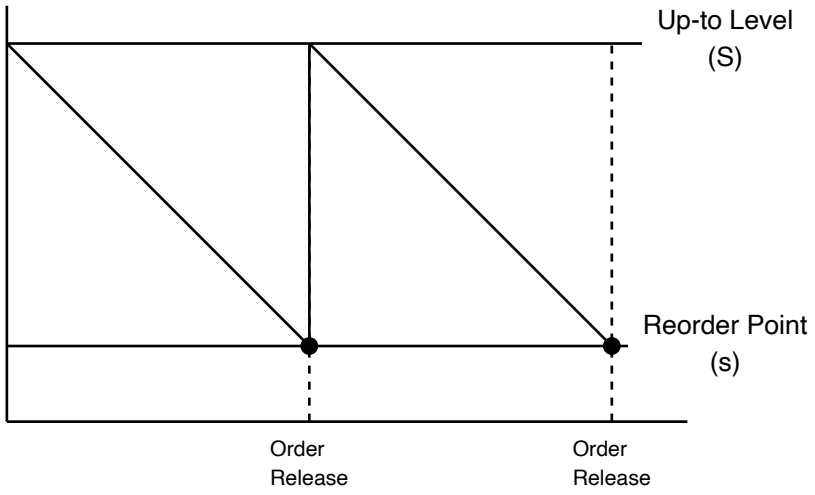
201.3.1. Setup

Same as (Q, r): demand rate d with std σ_d, lead time L, costs S_{setup}, h, service level z.

201.3.2. Difference from (Q, r)

(Q, r) commits to a **fixed** batch Q every order — doesn't matter if inventory dropped to exactly r or much below. (s, S) compensates: if inventory overshoots downward, the order is **larger** to make up.

In practice, **with continuous review and small per-period demand** (smooth depletion), the two are nearly identical. Differences emerge under **lumpy demand** — large discrete withdrawals can drop inventory well below s in one go.



201.3.3. Set s

Same logic as r in (Q, r):

$$s = \mu_L + z\sigma_L$$

201.3.4. Set S

For large fixed cost (most cases), $S - s$ should approximately equal the EOQ:

$$S = s + Q_{\text{EOQ}} = s + \sqrt{\frac{2DS_{\text{setup}}}{h}}$$

Why: when inventory drops exactly to s (no overshoot), this gives the same order quantity as (Q, r). When it overshoots, the order grows automatically.

For low/zero fixed cost: $S = s$ (order one unit at a time — the "base stock" / (S-1, S) policy in the limit; see base_stock.typ).

201.3.5. Final formulas

$$s = \mu_L + z\sigma_L \quad S = s + \sqrt{\frac{2DS_{\text{setup}}}{h}} \quad \text{SS} = z\sigma_L$$

Average inventory under (s, S) is **slightly higher** than (Q, r) because of the overshoot correction — but only by an amount proportional to expected overshoot per cycle, usually small.

Example

Given (same policy-comparison params):

- $D = 12000$/yr, $d = 33$/day, $\sigma_d = 5$, $L = 14$ days
- S_{setup} = \$50, h = \$2/unit/yr, $z = 1.645$
- Demand assumed **smooth** (small per-period withdrawals)

Step 1 — reorder point

$$s = \mu_L + z\sigma_L = 462 + 30.8 \approx 493$$

Identical to r in (Q, r).

Step 2 — order-up-to level

$$Q_{\text{EOQ}} = \sqrt{\frac{2 \cdot 12000 \cdot 50}{2}} \approx 775$$

$$S = s + Q_{\text{EOQ}} = 493 + 775 \approx 1268 \text{ units}$$

Step 3 — order quantity per trigger

Whenever inventory position drops to (or below) 493, order:

$$Q_{\text{actual}} = 1268 - \text{current position}$$

- If position is exactly 493 at the moment of trigger: order $1268 - 493 = 775$ (same as EOQ).
- If position is 480 (overshot below 493 due to a lumpy demand): order $1268 - 480 = 788$ (slightly larger to compensate).

Compare to (Q, r)

Under smooth demand:

- (Q, r): $Q = 775$ exactly, $r = 493$.
- (s, S): $S - s = 775$, expected order ≈ 775, so essentially the same numbers.

Difference appears under lumpy demand: if a single customer order suddenly removes 100 units, dropping inventory from 540 to 440:

- (Q, r) orders 775. New inventory position: $440 + 775 = 1215$.
- (s, S) orders $1268 - 440 = 828$. New inventory position: 1268 (back to target).

(s, S) restores the **target** every time; (Q, r) just adds the same fixed batch. Use (s, S) when demand is lumpy or when you have a target inventory budget.

201.4. (Q, r)

Continuous review, fixed order quantity. The most common stochastic-demand policy in textbooks and practice.

Decision rule: monitor inventory position continuously. When it drops to (or below) the reorder point r, order a fixed quantity Q.

Two parameters:

- Q = order quantity (a **fixed** batch every time)
- r = reorder point (the trigger level)

201.4.1. Setup

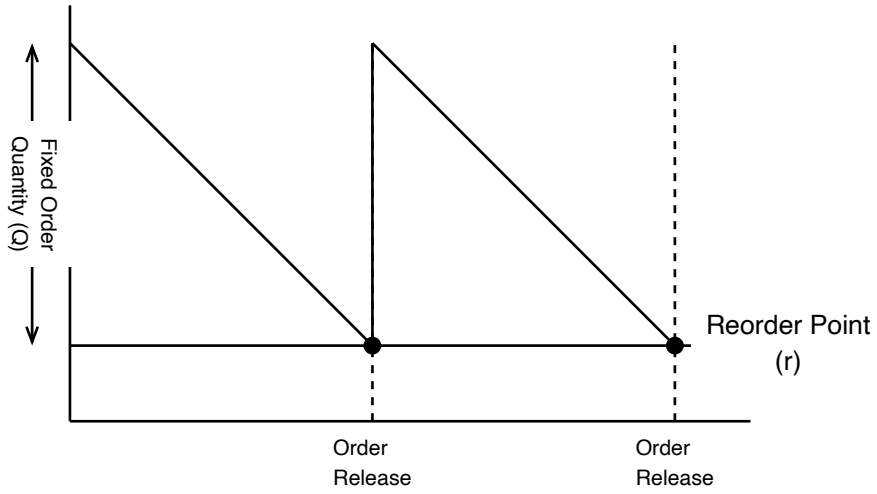
Random demand with rate d per unit time and standard deviation σ_d. Lead time L (constant for now). Holding cost h, fixed order cost S, plus a service-level requirement z (e.g., $z = 1.645$ for 95% cycle service level).

Inventory position $= \text{on-hand} + \text{on-order} - \text{backorders}$. Continuous review means we always know it.

201.4.2. Inventory profile

Sawtooth shape, but with random consumption rate:

- Inventory drains stochastically.
- When position hits r, a Q-unit order is placed; arrives L time later.
- During lead time, demand consumes D_L = random draw from a distribution with mean $\mu_L = dL$ and std $\sigma_L = \sigma_d\sqrt{L}$.
- **Service level** (probability of no stockout per cycle) = $P(D_L \le r)$.



201.4.3. Set r — the reorder point

To meet cycle service level $1 - \alpha$:

$$r = \mu_L + z\sigma_L$$

where:

- $\mu_L = dL$ (expected demand during lead time)
- $\sigma_L = \sigma_d\sqrt{L}$ (std of lead-time demand, assuming i.i.d. periods)
- $z = \Phi^{-1}(1 - \alpha)$ (standard-normal quantile for the desired service level)

The term $z\sigma_L$ is the **safety stock** — buffer for demand variability.

201.4.4. Set Q — the order quantity

When fixed cost S is significant, use the EOQ formula:

$$Q = \sqrt{\frac{2DS}{h}}$$

(Approximate — exact joint optimization of Q and r exists but EOQ is within a few % of optimal.)

201.4.5. Final formulas

$$Q \approx \sqrt{\frac{2DS}{h}} \quad r = \mu_L + z\sigma_L \quad \text{SS} = z\sigma_L$$

Average inventory (approximation): $Q/2 + \text{SS}$, so total holding cost $\approx h(Q/2 + z\sigma_L)$.

Example

Given (shared policy-comparison params):

- Annual demand: D = 12,000 units/year ($d = 33$ units/day)
- Daily demand std: $\sigma_d = 5$ units/day
- Lead time: $L = 14$ days (constant)
- Order cost: S = \$50 / order
- Holding cost: h = \$2 / unit / year
- Service level: 95% $\rightarrow z = 1.645$

Step 1 — order quantity (EOQ)

$$Q = \sqrt{\frac{2 \cdot 12000 \cdot 50}{2}} = \sqrt{600000} \approx 775 \text{ units}$$

Step 2 — lead-time demand statistics

$$\mu_L = dL = 33 \cdot 14 = 462 \text{ units}$$

$$\sigma_L = \sigma_d\sqrt{L} = 5 \cdot \sqrt{14} \approx 18.7 \text{ units}$$

Step 3 — reorder point

$$r = \mu_L + z\sigma_L = 462 + 1.645 \cdot 18.7 \approx 462 + 30.8 \approx 493 \text{ units}$$

Safety stock: $\text{SS} = 30.8$ units.

Step 4 — interpret

Whenever inventory position drops to 493, place an order for 775. The order arrives 14 days later. Expected demand during those 14 days is 462; the extra 31 units of safety stock cover up-to-95th-percentile demand fluctuations.

Average ordering rate: $\frac{D}{Q} = \frac{12000}{775} \approx 15.5$ orders / year. Average cycle length: $\frac{Q}{D} = \frac{775}{12000} \approx 24$ days.

Average inventory: $\frac{Q}{2} + \text{SS} = 388 + 31 \approx 419$ units. Annual holding cost: $h \cdot 419 \approx$ \$838 / year. Annual ordering cost: $S \cdot \frac{D}{Q} = 50 \cdot 15.5 \approx$ \$775 / year. **Total**: $\approx$ \$1613 / year (vs \$1549 for deterministic EOQ — the gap is the cost of safety stock).

201.5. (nQ, r)

Continuous review, **fixed pack size**. Like (Q, r), but the supplier requires orders in multiples of a fixed batch Q (cases, dozens, pallet quantities).

Decision rule: monitor inventory continuously. When inventory position drops to (or below) reorder point r, order the **smallest integer multiple** nQ that brings inventory position back above r.

Three parameters:

- Q = fixed pack size (set by supplier, e.g., case of 24)
- n = number of packs to order this cycle (a **function** of how far inventory dropped, not a free parameter)
- r = reorder point

201.5.1. When does this matter?

(Q, r) assumes you can order any integer quantity Q. (nQ, r) handles the practical case where:

- Supplier ships only full pallets / cases / drums.
- Pack size is **fixed** by packaging or logistics; you choose how many.

If Q is small relative to lead-time demand, (nQ, r) is essentially (Q, r) with rounding noise. If Q is large (e.g., a pallet contains a year's worth of demand), the round-up effect inflates inventory significantly.

201.5.2. Decision logic when triggered

When inventory position falls to $r - \delta$ (for some overshoot $\delta \geq 0$):

$$n = \left\lceil \frac{s - \text{position} + Q}{Q} \right\rceil$$

Or simpler in practice: order the smallest nQ such that the post-order position exceeds some target (e.g., $r + Q$).

In the **small overshoot** case (smooth demand), $n = 1$ almost always — order one pack of Q. (nQ, r) reduces to (Q, r).

201.5.3. Set r

Same as (Q, r):

$$r = \mu_L + z\sigma_L$$

The pack-size constraint affects Q, not r. The reorder trigger logic doesn't change.

201.5.4. Set Q

If Q is **given** (supplier-mandated pack size), no decision. If you can choose pack-size from a discrete set (dozen, gross, case, pallet), pick the closest to $Q_{\text{EOQ}} = \sqrt{2DS_{\text{setup}}/h}$.

201.5.5. Average inventory

Slightly higher than (Q, r): $\approx Q/2 + \text{SS} + \frac{\text{expected overshoot}}{2}$. Overshoot is small under continuous review of smooth demand but grows with demand lumpiness.

201.5.6. Final formulas

$$r = \mu_L + z\sigma_L \quad Q = \text{supplier-fixed pack size} \quad n = \left\lceil \frac{\text{triggered position deficit} + Q}{Q} \right\rceil$$

Example

Given (same policy-comparison params):

- $D = 12000$/yr, $d = 33$/day, $\sigma_d = 5$, $L = 14$ days
- S_{setup} = \$50, h = \$2/unit/yr, $z = 1.645$
- Pack size: $Q = 800$ (one pallet contains 800 units; supplier won't break pallets)

Step 1 — reorder point

$$r = \mu_L + z\sigma_L = 462 + 30.8 \approx 493$$

Step 2 — pack-size sanity check

Unconstrained EOQ would be ≈ 775. Forced pack of 800 is very close — pack-size penalty is negligible.

Step 3 — typical order

Inventory position drops to roughly 493 over time (smooth demand). Order nQ where n is the smallest integer such that the new position exceeds the next reorder cycle's needs.

- At trigger, position ≈ 493.
- One pack: $493 + 800 = 1293$. Above r, well above $r + Q/2 = 893$. ✓
- Order **1 pack** (800 units).

Step 4 — when overshoot matters

If a sudden burst drops inventory from 600 to 200 (a 400-unit lumpy withdrawal that crosses below r):

- One pack brings position to $200 + 800 = 1000$. **Above r ✓**.
- But average inventory just after order is 1000, which is near $S_{\text{target}} \approx 1293$ for the previous cycle — still in normal range.

Now consider $Q = 4000$ (much larger pack):

- One pack: $200 + 4000 = 4200$. Way above r, but we just bought 10 cycles of inventory in one go.
- Average inventory: $\approx \frac{Q}{2} = 2000$ (vs $\frac{Q_{\text{EOQ}}}{2} = 388$ optimal).
- Holding cost more than 5x optimal.

Compare to (Q, r)

Under matched pack size ($Q = 800 \approx Q_{\text{EOQ}}$), (nQ, r) and (Q, r) are nearly identical — same trigger, same order quantity. The difference grows when pack size diverges from the unconstrained optimum.

Use (nQ, r) when: supplier mandates pack sizes; you want predictable shipping units; pack size happens to be close to EOQ. Switch to (Q, r) only if you can break packs.

201.6. (R, S)

Periodic review, **order-up-to** S. The simplest periodic-review policy.

Decision rule: at every review point (every R time units), observe the inventory position and order **just enough to bring it up to** S. No reorder point — order at every review whether or not inventory is depleted.

Two parameters:

- R = review interval (often fixed by external factors — a weekly truck, a monthly billing cycle)
- S = order-up-to level

201.6.1. When to use

Periodic review — either because continuous monitoring is expensive (manual stock-taking) or because deliveries arrive on a fixed cadence (the supplier's truck comes every Monday whether you want it to or not). (R, S) is the workhorse for this regime.

201.6.2. Inventory profile

Sawtooth, with reviews as the cycle anchor:

- At review t: observe position I_t. Order $S - I_t$. Order arrives at $t + L$.
- Between reviews: inventory drains stochastically. **No intervention even if it runs out.**
- **Stockout window**: from order placed until the **next** order arrives — that's $R + L$ (review interval plus lead time).

201.6.3. Set S

The protection window is $R + L$ (not just L as in continuous review!). Demand during this window has:

- Mean: $\mu_{R+L} = d(R + L)$
- Std: $\sigma_{R+L} = \sigma_d\sqrt{R + L}$

For service level $1 - \alpha$:

$$S = \mu_{R+L} + z\sigma_{R+L}$$

The safety stock $z\sigma_{R+L}$ is **larger** than continuous-review safety stock by factor $\sqrt{(R + L)/L}$.

201.6.4. No r in (R, S)

Unlike continuous review, there's no reorder point — every review triggers an order, even if inventory is still high. Order quantity varies cycle to cycle:

$$Q_{\text{actual}} = S - I_t$$

Sometimes $Q_{\text{actual}} \approx 0$ (just received last order); sometimes large (after a busy period). Average: $Q_{\text{actual}} = Rd$ (you replace what was consumed since last review).

201.6.5. Final formulas

$$S = \mu_{R+L} + z\sigma_{R+L} \quad \text{SS} = z\sigma_{R+L} \quad Q_{\text{actual}} = S - I_{\text{current}}$$

Example

Given (same policy-comparison params + a 30-day review):

- $D = 12000$/yr, $d = 33$/day, $\sigma_d = 5$, $L = 14$ days
- Review interval: $R = 30$ days (monthly review)
- h = \$2/unit/yr, $z = 1.645$

Step 1 — protection window

$$R + L = 30 + 14 = 44 \text{ days}$$

Step 2 — demand statistics over the window

$$\mu_{R+L} = d(R + L) = 33 \cdot 44 = 1452 \text{ units}$$

$$\sigma_{R+L} = \sigma_d\sqrt{R + L} = 5 \cdot \sqrt{44} \approx 33.2 \text{ units}$$

Step 3 — order-up-to level

$$S = \mu_{R+L} + z\sigma_{R+L} = 1452 + 1.645 \cdot 33.2 \approx 1452 + 54.6 \approx 1507 \text{ units}$$

Safety stock: $\text{SS} = 54.6$ units.

Step 4 — typical cycle

At each monthly review, suppose inventory position is around $1507 - 33 \cdot 30 = 517$ (steady-state, average consumption between reviews).

- Order: $1507 - 517 = 990$ units.
- Order arrives 14 days later. By then, inventory has drained further.
- Next review: position is back near 517 again (steady state). Order another 990.

Step 5 — compare to (Q, r)

(Q, r) on the same demand: $r = 493$, $Q = 775$, safety stock = 31. (R, S): $S = 1507$, safety stock = 55.

(R, S) carries 24 extra units of safety stock (almost 2x). Why? The protection window is $R + L = 44$ days vs $L = 14$ days. Variance scales linearly with window length, so std scales as $\sqrt{\frac{44}{14}} = 1.77$.

Trade-off: continuous review needs (Q, r) infrastructure (real-time tracking). Periodic review accepts more safety stock in exchange for simpler operations.

201.7. (R, s, S)

Periodic review with a reorder point. Generalizes (R, S) by adding a “do nothing” option when inventory is high.

Decision rule: at every review point (every R time units), observe the inventory position I_t.

- **If** $I_t \leq s$ (inventory low): order $S - I_t$ (bring up to S).
- **Else** ($I_t > s$, inventory still healthy): **don't order**.

Three parameters:

- R = review interval
- s = reorder point
- S = order-up-to level

201.7.1. When to use

Use (R, s, S) when:

- Reviews are periodic (truck arrives weekly), but
- You don't want to order **every** review when inventory is still healthy (saves order cost when consumption was light).

Compared to (R, S): saves order cost in low-consumption periods. Compared to (s, S): adds the periodic-review constraint (simpler operations, less precise).

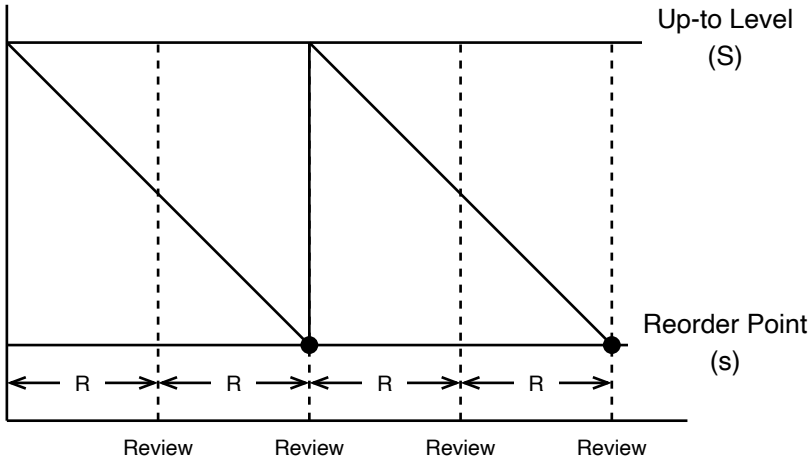
(R, s, S) is widely considered the **most general** periodic policy. It generalizes both (R, S) (set $s = S$, always order) and **zero ordering** (set s very low, never order).

201.7.2. Inventory profile

Sawtooth, with order skipping:

- Some reviews: $I_t > s$, no order. Inventory continues to drain.
- Other reviews: $I_t \leq s$, order up to S.
- Order arrives at next review or before, depending on L vs R.

If R is short relative to mean cycle, most reviews do **not** trigger orders. If R is long, almost every review triggers.



201.7.3. Set s — protection-window logic

Same as (R, S): the protection window is $R + L$, because an order placed at review covers demand until the **next** review's order arrives. So:

$$s = \mu_{R+L} + z\sigma_{R+L}$$

Identical to the S in (R, S)! That's because s in (R, s, S) plays the same protection role as S in (R, S) — it's the level **above which we still have enough buffer**.

201.7.4. Set S

$S - s$ should be the EOQ-style batch size. Heuristic:

$$S = s + Q_{\text{EOQ}} = s + \sqrt{\frac{2DS_{\text{setup}}}{h}}$$

When inventory drops to exactly s between reviews, the next order is for $S - s = Q_{\text{EOQ}}$. With overshoot (inventory below s at review), the order is bigger — same as (s, S).

201.7.5. Computing optimal s, S exactly

Closed forms exist for special cases (Wagner's lower-bound formula); in general, (s^*, S^*) require dynamic programming or simulation. The heuristic above is within a few % of optimal for typical industrial parameter ranges.

201.7.6. Final formulas

$$s = \mu_{R+L} + z\sigma_{R+L} \quad S = s + \sqrt{\frac{2DS_{\text{setup}}}{h}} \quad \text{SS} = z\sigma_{R+L}$$

Example

Given (same policy-comparison params):

- $D = 12000$/yr, $d = 33$/day, $\sigma_d = 5$, $L = 14$ days, $R = 30$ days
- S_{setup} = \$50, h = \$2/unit/yr, $z = 1.645$

Step 1 — reorder point (same as S from (R, S))

$$s = \mu_{R+L} + z\sigma_{R+L} = 1452 + 54.6 \approx 1507$$

Step 2 — order-up-to level

$$Q_{\text{EOQ}} = \sqrt{\frac{2 \cdot 12000 \cdot 50}{2}} \approx 775$$

$$S = s + Q_{\text{EOQ}} = 1507 + 775 \approx 2282$$

Step 3 — typical reviews

Steady-state inventory between reviews drops by about $Rd = 990$ units per review interval.

- **Review 1**: position is 2282 (just received order). Drains to $\approx 2282 - 990 = 1292$ by review 2.
- **Review 2**: position 1292. **Below $s = 1507$** ✓ → order $2282 - 1292 = 990$ units.
- Continues this pattern.

Almost every review triggers an order because R is comparable to the average cycle time. Behaviorally similar to (R, S).

Step 4 — when "don't order" actually fires

Suppose demand is unusually light over a review period (e.g., only 200 units consumed instead of 990). Then inventory at the next review is $2282 - 200 = 2082 > 1507 = s$. No order placed → save \$50.

Then next review: another 200 consumed → 1882, still above s → no order. Then another: 1682 → still above s → no order. Eventually drops below s, then big catch-up order.

(R, s, S) sleeps through low-consumption stretches; (R, S) would have ordered every review regardless.

Compare to (R, S)

- (R, S): always order every review. S at protection-window level (1507). Number of orders = $\frac{365}{R} = 12.2$/year.
- (R, s, S): may skip some reviews. Same s (1507), $S = s + Q_{\text{EOQ}}$ (2282). Order frequency varies; expected $\approx \frac{365}{\frac{Q_{\text{EOQ}}}{d}} = \frac{365}{\frac{775}{33}} = 15.5$/year **but with variability**.

Use (R, s, S) when **order setup cost is significant** and consumption variability is high — the "skip a review" behavior amortizes the setup cost across larger batches.

201.8. (R, nQ, s)

Periodic review, **fixed pack size, with reorder point**. Combines the periodic-review structure of (R, s, S) with the fixed-pack constraint of (nQ, r).

Decision rule: at every review point (every R time units), observe inventory position I_t.

- **If** $I_t \leq s$ (inventory low): order the smallest integer multiple nQ that brings position above s (or above some target).
- **Else**: don't order.

Three parameters:

- R = review interval
- Q = fixed pack size (supplier-mandated)
- s = reorder point

(The "S" of (R, s, S) is replaced by **implicit** $S = s + nQ$ — the resulting position depends on how many packs are needed.)

201.8.1. When to use

The pragmatic real-world combination:

- Reviews are periodic (cycle counts done weekly or monthly).
- Pack size is fixed by the supplier (full case, dozen, pallet).
- Want to skip the order when inventory is still healthy.

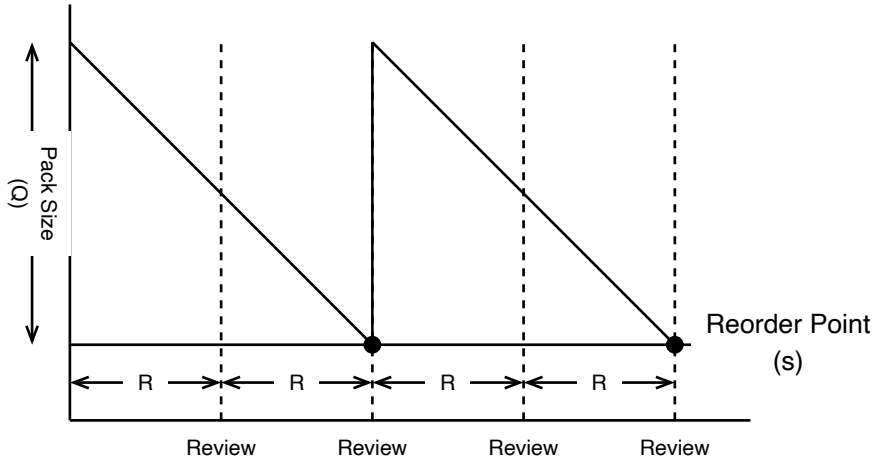
Common in retail (Each store reorders periodically, pack sizes are case-of-N) and pharmacy (DEA-mandated periodic counts of controlled substances, fixed pack size from manufacturer).

201.8.2. Inventory profile

Sawtooth, with two layers of discreteness:

- Some reviews: skip (inventory above s).
- Other reviews: order nQ where n is the integer chosen at trigger time.

Average inventory is **higher** than (R, s, S) because the integer-pack rounding inflates each order. Difference is small if $Q \ll s$, large if Q is comparable to s.



201.8.3. Set s

Same protection-window logic as (R, s, S) and (R, S):

$$s = \mu_{R+L} + z\sigma_{R+L}$$

Pack-size constraint affects the **order quantity**, not the trigger.

201.8.4. Choose n at trigger time

When triggered with position $I_t \le s$:

$$n = \left\lceil \frac{s - I_t + Q}{Q} \right\rceil \quad (\text{smallest } n \text{ s.t. } I_t + nQ > s)$$

A simpler rule: pick n to make post-order position $\approx s + Q/2$ (midpoint of typical s-to-$s+Q$ range). For the smooth-demand case, $n = 1$ usually suffices.

201.8.5. Set Q

Q is **given** by the supplier. If you have flexibility (multiple pack sizes available), pick the one closest to $Q_{\text{EOQ}} = \sqrt{2DS_{\text{setup}}/h}$.

201.8.6. Final formulas

$$s = \mu_{R+L} + z\sigma_{R+L} \quad Q = \text{supplier-fixed pack size} \quad n = \left\lceil \frac{s - I_t + Q}{Q} \right\rceil$$

Example

Given (same policy-comparison params + a fixed pack):

- $D = 12000$/yr, $d = 33$/day, $\sigma_d = 5$, $L = 14$ days, $R = 30$ days
- $z = 1.645$, h = \$2/unit/yr
- Pack size: $Q = 1000$ units (case)

Step 1 — reorder point

$$s = \mu_{R+L} + z\sigma_{R+L} = 1452 + 54.6 \approx 1507$$

Step 2 — typical review with healthy inventory

Inventory at review = 1800 (above $s = 1507$). **Skip — no order**.

Step 3 — typical review with depleted inventory

Inventory at review = 1200 (below s). Compute n:

$$n = \left\lceil \frac{1507 - 1200 + 1000}{1000} \right\rceil = \left\lceil \frac{1307}{1000} \right\rceil = \lceil 1.307 \rceil = 2 \text{ packs}$$

Order $nQ = 2000$ units. Post-order position: $1200 + 2000 = 3200$. Well above s.

Step 4 — typical review with slight depletion

Inventory at review = 1450 (just below s). Compute n:

$$n = \left\lceil \frac{1507 - 1450 + 1000}{1000} \right\rceil = \left\lceil \frac{1057}{1000} \right\rceil = \lceil 1.057 \rceil = 2 \text{ packs}$$

Hmm — even a tiny dip below s orders 2 packs. The formula always rounds up to ensure the post-order position is **above** $s + Q$. A more economical choice would be 1 pack (post-order = 2450, above s = 1507 ✓), but then over time you'd accumulate small deficits.

Refinement: a common variant orders the smallest n such that post-order position exceeds **just** s (not $s + Q$):

$$n_{\text{loose}} = \left\lceil \frac{s - I_t}{Q} \right\rceil = \left\lceil \frac{1507 - 1450}{1000} \right\rceil = \lceil 0.057 \rceil = 1 \text{ pack}$$

Choice of rule depends on tolerance for overshoot vs subsequent shortages. The tighter rule (always above $s + Q$) is safer; the looser rule (just above s) is leaner.

Step 5 — compare to (R, s, S)

(R, s, S) on same params: $s = 1507$, $S = s + Q_{\text{EOQ}} = 2282$.

- At trigger with position 1450, order $S - 1450 = 832$. Post-order: 2282.
- Compare to (R, nQ, s) with $Q = 1000$: order $1 \cdot 1000 = 1000$. Post-order: 2450.

(R, nQ, s) carries about 10% more inventory (2450 vs 2282) because integer packs round up. Acceptable when supplier inflexibility forces the constraint.

Use (R, nQ, s) when: periodic review + fixed pack size + want skip-when-healthy logic. The most operationally realistic of the periodic policies.

202. Graph Theory

203. Overview

Properties of large scale network:

- Powerlaw distribution of edges
- Logarithmically increasing geodesic distance
- High clustering
- Hubs
- Shortcuts
- Small World

Markosova, M. and Nather, P. (2006). Language As A Small World Network. IEEE - ve42.co/rgLanguageNetwork

Lawrence, S. and Lee Giles, C., (1999). Accessibility Of Information On The Web. Springer Science and Business Media LLC - ve42.co/natureErdosRenyi

Albert, R. et al. (1999). Diameter Of The World-Wide Web. Springer Science and Business Media LLC - ve42.co/arxivKleinberg

Réka Albert via Wikipedia - ve42.co/wikiRekaAlbert Barabási-Albert Model via Wikipedia - ve42.co/wikiBarabasiAlbert

https://www.youtube.com/watch?v=t133w4YeD0c by Tenzin Ngodup https://www.youtube.com/watch?v=YiTfNiIn6RI by Complexity Lab

Guimerà, R. et al. (2005). The Worldwide Air Transportation Network: Anomalous Centrality, Community Structure, And Cities' Global Roles. Proceedings of the National Academy of Sciences - ve42.co/pnasBarabasi2004

Gratton, C. et al. (2017). Control Networks And Hubs. Wiley - ve42.co/wileyPsychophysNet

Prisoner's Dilemma via Wikipedia - ve42.co/wikiPrisonersDilemma

Network Science via networksciencebook.com - ve42.co/bookNetworkScience

Six Degrees Of Separation via Wikipedia - ve42.co/wikiSixDegrees

Iraqi Coulda Been Contender In Six Degrees via latimes.com - ve42.co/latimesOHarStorm1999

https://www.youtube.com/watch?v=gujZgDaFp3U by 2veritasium

204. Machine Learning

205. Bias–Variance Tradeoff

205.1. Bias Variance Tradoff

We have:

- $f(x)$: True function
- $y = f(x) + \varepsilon, \text{where } \varepsilon \sim \mathcal{N}(0, \sigma^2)$: Observation with noise
- $\hat{f}(x)$: Model

Bias (systematic error)

$$\text{Bias}(x) = \mathbb{E}\left[\hat{f}(x)\right] - f(x)$$

- Train the same model on many different training sets
- Plot all the fitted curves together
- Compute the average of those fitted curves
- Compare that average curve to the true function $f(x)$.

The difference is the bias

Consistently across datasets

Variance (random error)

$$\text{Variance}(x) = \mathbb{E}\left[\left(\hat{f}(x) - \mathbb{E}\left[\hat{f}(x)\right]\right)^2\right]$$

- Train the same model on many different training sets.
- Plot all the fitted curves together

The spread of those curves around their average prediction at each x is the variance

- Low variance on training set
- High variance on test set

Inconsistent across different data

Example

True Function:

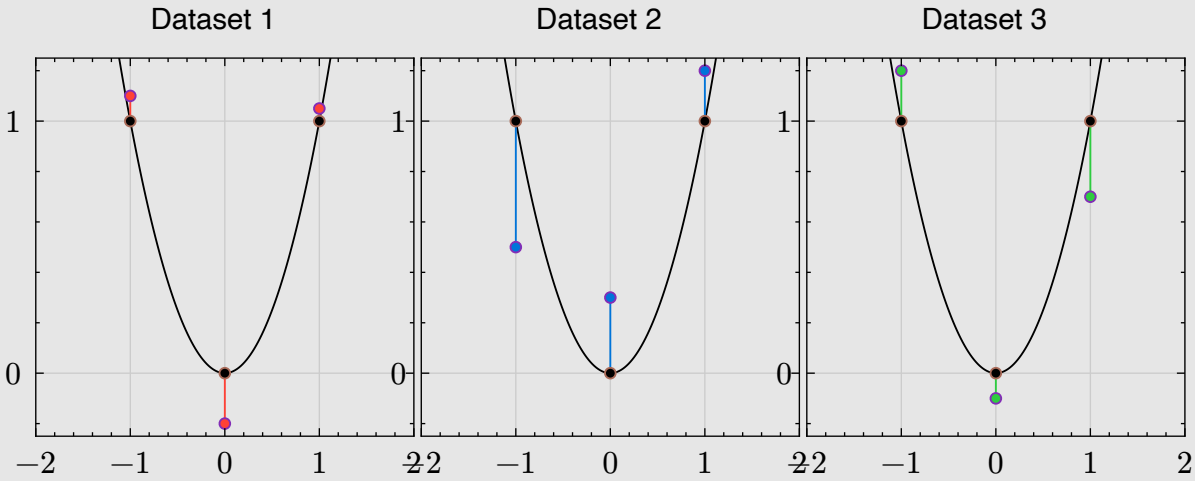
$$x^2$$

Observations:

Observations differ due to noise.

Dataset 1	Dataset 2	Dataset 3
1.1	0.5	1.2
–0.2	0.3	–0.1

1.05	1.2	0.7



$\varepsilon(-1)$	0.1
$\varepsilon(0)$	−0.2
$\varepsilon(1)$	0.05

$\varepsilon(-1)$	−0.5
$\varepsilon(0)$	0.3
$\varepsilon(1)$	0.2

$\varepsilon(-1)$	0.2
$\varepsilon(0)$	−0.1
$\varepsilon(1)$	−0.3

Average across datasets

- $x = -1$

$$\hat{f}_{\text{avg}}(-1) = \frac{1.1 + 0.5 + 1.2}{3} = \frac{2.8}{3} \approx 0.933$$

- $x = 0$

$$\hat{f}_{\text{avg}}(0) = \frac{-0.2 + 0.3 - 0.1}{3} = 0$$

- $x = 1$

$$\hat{f}_{\text{avg}}(1) = \frac{1.05 + 1.2 + 0.7}{3} = \frac{2.95}{3} \approx 0.983$$

Bias

Bias at each point

- $x = -1$

$$\text{Bias}(-1) = 0.933 - 1 = -0.067$$

- $x = 0$

$$\text{Bias}(-0) = 0 - 0 = 0$$

- $x = 1$

$$\text{Bias}(1) = 0.983 - 1 = -0.017$$

Overall Bias

Mean Square Bias

$$\text{Bias}^2 = \frac{1}{n}\sum_{i=1}^{n}\left(\mathbb{E}\left[\hat{f}(x_i)\right] - f(x_i)\right)^2$$

$$\text{Bias}^2 \approx \frac{(-0.067)^2 + 0^2 + (-0.017)^2}{3} = 0.0016$$

Variance

Deviations

- $x = -1$

$$d_1 = 1.1 - 0.933 = 0.166$$
$$d_2 = 0.5 - 0.933 = -0.433$$
$$d_3 = 1.2 - 0.933 = 0.266$$

$$d_1^2 = 0.028$$
$$d_2^2 = 0.188$$
$$d_3^2 = 0.071$$

$$\text{Var}(-1) = \frac{0.028 + 0.188 + 0.071}{3} = 0.096$$

- $x = 0$

$$d_1 = -0.2 - 0 = -0.2$$
$$d_2 = 0.3 - 0 = 0.3$$
$$d_3 = 0.1 - 0 = -0.1$$

$$d_1^2 = 0.04$$
$$d_2^2 = 0.09$$
$$d_3^2 = 0.01$$

$$\text{Var}(-1) = \frac{0.04 + 0.09 + 0.01}{3} = 0.047$$

- $x = 1$

$$d_1 = 1.05 - 0.983 = 0.067$$
$$d_2 = 1.2 - 0.983 = 0.217$$
$$d_3 = 0.7 - 0.983 = -0.283$$

$$d_1^2 = 0.004$$
$$d_2^2 = 0.047$$
$$d_3^2 = 0.08$$

$$\text{Var}(1) = \frac{0.004 + 0.047 + 0.08}{3} = 0.044$$

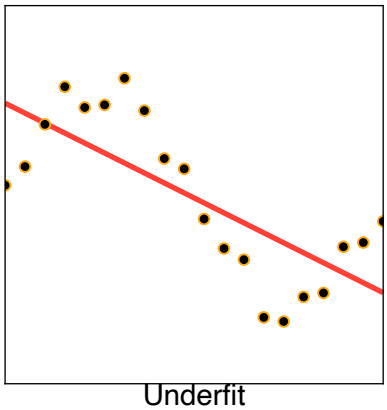
Overall Variance

Average Variance

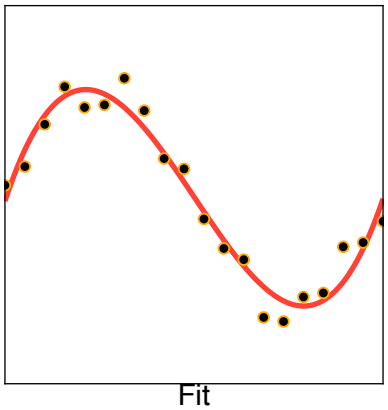
$$\frac{0.096 + 0.047 + 0.044}{3} \approx 0.062$$

Overfitting

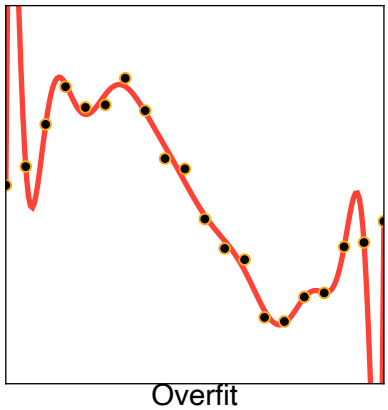
Model Complexity	Bias	Variance	Training Error	Test Error	Situation
Too Simple	High	Low	High	High	Underfitting
Just Right	Moderate	Moderate	Low	Lowest	Good Generalization
Too Complex	Low	High	Very Low	High	Overfitting



Underfit
High Bias
Low Variance

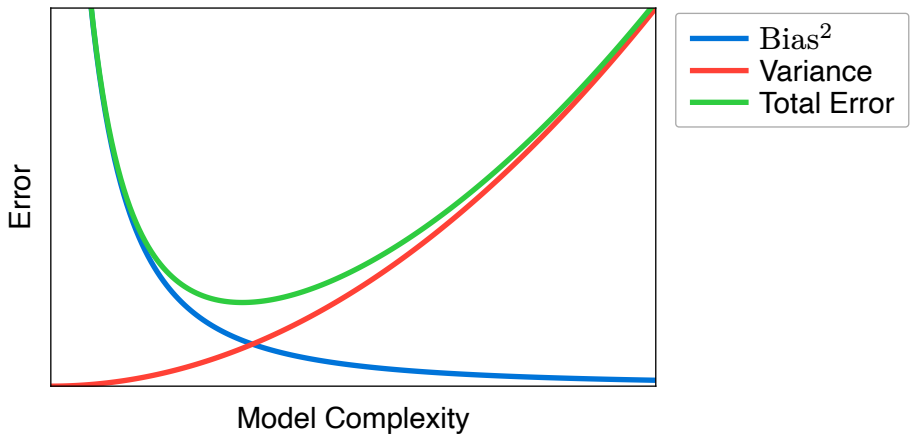


Fit
Low Bias
Low Variance



Overfit
Low Bias
High Variance

Bias-Variance Tradoff



206. Classification

206.1. Support Vector Machine

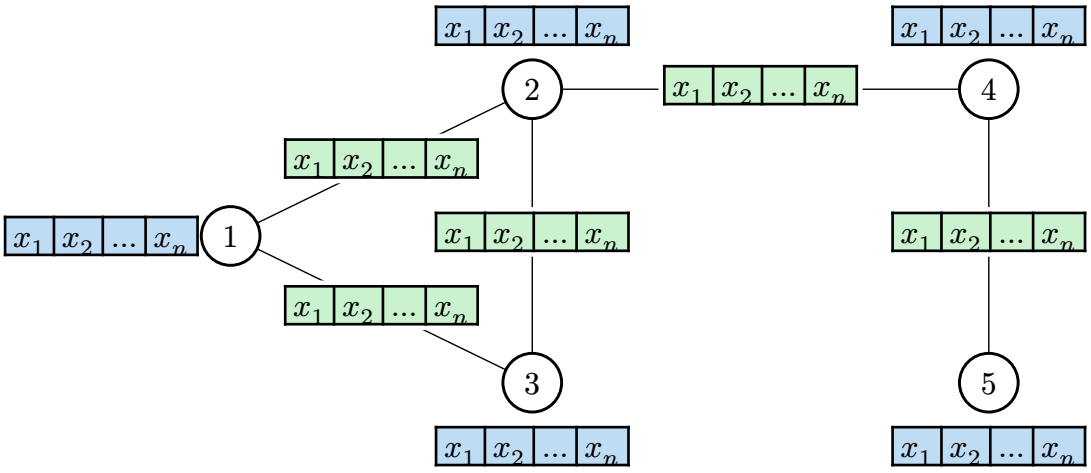
206.2. XGBoost

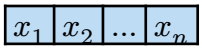
206.2.1. XGBoost (Extreme Gradient Boosting)

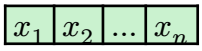
Gradient Boosted Decision Trees

207. Graphs

207.1. Graph Neural Network



Node Embedding: 

Edge Embedding: 

Adjacency Matrix: $$\begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

- Node-level predictions
- Edge-level predictions
- Graph-level predictions

Graph Data

- Size variant
- Isomorphism: permutation invariance
- Grid structure: non-euclidian

207.1.1. Message Passing

Allows nodes in a graph to exchange information and update their representations (embeddings) based on their local neighborhood

$$G = (V, E)$$

Where:

- V: set of nodes ($|V| = n$)
- $E \subseteq V \times V$: set of edges
- $h_v^k \in \mathbb{R}^d$: feature (embedding) of node v at iteration k
- $\mathcal{N}(v)$: set of neighbors of node v

Step 1: Message computation

Each node v receives messages from its neighbors $u \in \mathcal{N}(v)$. A message function M defines how information is sent:

$$\mathbf{m}_v^k = \sum_{u \in \mathcal{N}(v)} M\big(\mathbf{h}_v^{k-1}, \mathbf{h}_u^{k-1}, e_{uv}\big)$$

- $\mathbf{h}_u^{k-1}$: source (neighbor)
- $\mathbf{h}_v^{k-1}$: target (receiver)
- e_{uv}: Optional edge features

Step 2: Aggregation

Aggregation combines incoming messages. This is often already done by the summation, mean, or max:

$$\mathbf{m}_v^k = \text{AGGREGATE}\big(\{\mathbf{h}_u^{k-1} : u \in \mathcal{N}(v)\}\big)$$

Step 3: Update

Each node updates its state with an update function U:

$$\mathbf{h}_v^k = U\big(\mathbf{h}_v^{k-1}, \mathbf{m}_v^k\big)$$

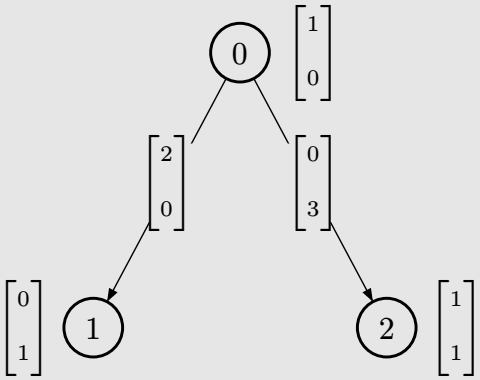
Common choce:

$$\mathbf{h}_v^k = \sigma\big(W \cdot [\mathbf{h}_v^{k-1} \parallel \mathbf{m}_v^k]\big)$$

Where:

- $\parallel$: vector concatenation
- σ: non-linearity (ReLU, tanh)
- W: learnable weights

Example



207.1.2. Invariance

A function is invariant to node permutations if its output stays the same no matter how the nodes are reordered (i.e., relabeled)

$$f(PX, PAP^T) = f(X, A)$$

If you shuffle the node labels, the graph is still the same — just redrawn

If you reorder the rows of X (node features) and both rows and columns of A (adjacency matrix) using the same permutation matrix P, the output does not change

207.1.3. Equivariance

A function is equivariant to node permutations if permuting the input nodes results in the output being permuted in the same way

$$f(PX, PAP^T) = Pf(X, A)$$

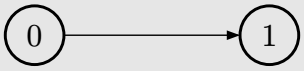
If you shuffle the nodes in the input graph, the output values (e.g., node embeddings or predictions) shuffle in the same way

If you reorder the rows of X (node features) and both rows and columns of A (adjacency matrix) using the same permutation matrix P, the output is reordered in the same way — i.e., its rows are permuted by P too

Example

Let $G = (V, E)$ be a directed graph where

$$V = \{0, 1\}$$
$$E = \{(0, 1)\}$$



Adjacency matrix:

$$A = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$$

Node feature matrix:

$$X = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

Permutation matrix (swaps node 0 and node 1):

$$P = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

Equivariance

Let's define a very simple **equivariant function**:

$$f(X, A) = AX$$

Message passing: sum over neighbors' features

Step 1: Compute output without permutation

$$f(X, A) = AX = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} = \begin{bmatrix} 3 & 4 \\ 0 & 0 \end{bmatrix}$$

Step 2: Compute permuted inputs

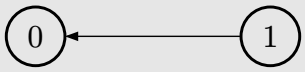
- Permute X

$$PX = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} = \begin{bmatrix} 3 & 4 \\ 1 & 2 \end{bmatrix}$$

- Permute A

$$PAP^T = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix}$$

This is now node 1 pointing to node 0



Step 3: Apply function to permuted inputs

$$f(PX, PAP^T) = (PAP^T)(PX) = \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 3 & 4 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 3 & 4 \end{bmatrix}$$

Step 4: Compare with $Pf(X, A)$

$$f(X, A) = \begin{bmatrix} 3 & 4 \\ 0 & 0 \end{bmatrix} \implies Pf(X, A) = \begin{bmatrix} 0 & 0 \\ 3 & 4 \end{bmatrix}$$

So this function is **equivariant**:

$$f(PX, PAP^T) = Pf(X, A)$$

Invariance

Let's define a simple invariant function:

$$g(X, A) = \mathbf{1}^T AX$$

Step 1: Compute $g(X, A)$

$$AX = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} = \begin{bmatrix} 3 & 4 \\ 0 & 0 \end{bmatrix}$$

Sum over rows:

$$\mathbf{1}^T AX = [3+0 \quad 4+0] = [3 \quad 4]$$

Step 2: Compute permuted inputs

- Permute X

$$PX = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} = \begin{bmatrix} 3 & 4 \\ 1 & 2 \end{bmatrix}$$

- Permute A

$$PAP^T = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix}$$

Step 3: Compute new output

$$PAP^T \cdot PX = \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 3 & 4 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 3 & 4 \end{bmatrix}$$

Step 4: Apply function to permuted inputs

$$\mathbf{1}^T(PAP^T)(PX) = [1 \quad 1]\begin{bmatrix} 0 & 0 \\ & \\ 3 & 4 \end{bmatrix} = [3 \quad 4]$$

Result:

$$g(X, A) = [3 \quad 4] = g(PX, PAP^T)$$

So the function is invariant under node permutation:

$$f(PX, PAP^T) = f(X, A)$$

208. Economics

209. Supply & Demand

210. Utility

Used to represent preferences of consumers (or decision-makers) over goods, services, or outcomes

211. Production Functions

212. Externality

Cost or benefit that affects a third party who is not involved in the economic transaction

- Negative: Cost

  E.g.: A factory produces widgets but also emits pollution that affects nearby residents.

- Positive: Benefit

  E.g., A homeowner plants a garden that beautifies the neighborhood, increasing property values.

Ignoring them can lead to suboptimal solutions

213. Network Effect

Value of a good or service increases with the number of users

- Positive: Increases value

  E.g.: Social networks. More users make the platform more valuable.

- Negative: Reduces value

  E.g.: Traffic networks. More cars may reduce value due to congestion.

214. Algorithms

215. Dijkstra

Step 1: Initialize

- Mark all nodes as unvisited
- Set the distance to the source as 0
- Set the distance to all other nodes as ∞
- Create a priority queue (or a simple list) to keep track of unvisited nodes and their distances.

Example

Graph nodes: A, B, C, D

Start node: A

Distances:

- $A = 0$
- $B = \infty$
- $C = \infty$
- $D = \infty$

Step 2: Select the Unvisited Node with the Smallest Distance

Among all unvisited nodes, pick the one with the smallest known distance. Let's call this the current node.

Step 3: Update Neighboring Distances

For each neighbor of the current node:

1. Calculate the tentative distance:

   distance(current) + weight(current $\longrightarrow$ neighbor)

2. If this tentative distance is less than the known distance, update it.

Example

If $A \longrightarrow B$ has weight 4 and distance to A is 0:

Tentative distance to B = 0 + 4 = 4

If current distance to B is ∞, update to 4

Step 4: Mark the Current Node as Visited

- Once all neighbors are checked, mark the current node as visited
- Visited nodes are not checked again

Step 5: Repeat Until All Nodes Are Visited

- Go back to Step 2 and repeat.

- Stop when all nodes have been visited or the smallest tentative distance among the unvisited nodes is ∞ (meaning unreachable)

Step 6: Reconstruct the Shortest Paths (Optional)

- To find the actual shortest path, track the previous node for each visited node
- Starting from the destination, backtrack through the previous nodes

Example

216. Organisations

217. Actor Network

217.1. Actor-Network Theory (ANT)

218. Bounded Rationality

219. Bureaucracy

219.1. Bureaucratic Theory

220. Cybernetic

221. Embeddedness

222. Functional Theory of Stratification

$$R \propto f(S, I)$$

Where:

- R: reward
- S: scarcity, how many people are qualified to perform a role
- I: importance, how critical the role is to the functioning of society

223. Institutional Theory

224. Network Theories

224.1. World-Systems Theory

(Wallerstein)

224.2. Global Commodity Chains (GCC)

- Producer-driven (autos, electronics)
- Buyer-driven (apparel, retail)

224.3. Global Value Chains (GVC)

Governance types:

- market
- modular
- relational
- captive
- hierarchical

224.4. Global Production Networks (GPN)

225. Resource Dependency

225.1. Resource Dependency Theory (RDT)

$$P_{A \to B} \propto D_{B \to A}$$

Where:

- P: power
- D: dependence

226. Scientific Management

- Work Study (Time and Motion Analysis) “One best way”
- Scientific Selection and Training of Workers
- Division of Labor
- Standardization: Tools and procedures
- Performance-Based Incentives

227. Transaction Costs

Why do firms / organizations exist? Why not many atomized individuals (market)?

Why are there hierarchies (firms / organizations) in markets?

Hierarchy or Market? Make or Buy? Which transactions should be included in the hierarchy, which transactions should be left to the market?

Unit of analysis: Transaction Relation (networks)

Traditional assumption: Using the market has no cost (frictionless)

If trust there would be no cost But the risk of opportunism, shirking exists

Therefore, there are costs:

- Search
- Bargaining
- Monitoring / Enforcement: ensuring that the terms of the agreement are being fulfilled (inspection, quality control, and supervision)
- Information: expenses incurred in acquiring, processing, and disseminating information relevant to the transaction

- Coordination
- Adaptation

Costs are a function of:

- Asset Specificity
- Uncertainty
- Frequency

Efficient alignment hypothesis: Transactions, which vary in their characteristics, should be matched with governance structures that differ in their costs and competencies. Alignment aims to minimize transactions costs.

Questions:

- For you which transactions should be included in the hierarchy and which should be left to the market?
- Furthermore, which transactions included in hierarchies should be in public hierarchies and which in private hierarchies?
- How can this transaction sort approaches be used to predict end symbiosis?

Let the set of transactions be $T = \{1, 2, ..., n\}$ For each transaction $i \in T$, define:

- C_i^M: transaction cost if done via **Market**
- C_i^H: transaction cost if done via **Hierarchy**

Then the total cost across all transactions is:

$$TC = \sum_{i=1}^{n} \min\{C_i^M, C_i^H\}$$

We want to choose the governance $g_i \in \{M, H\}$ for each i that minimized total transaction cost

Transaction costs can be modeled as a function of key characteristics

$$C_i^g = f^{g(S_i, U_i, F_i)}$$

Where:

- S_i: asset specificity
- U_i: Uncertainty
- F_i: frequency
- g: governance structure (M, H)

Assumptions

$$C_i^M = \alpha_M + \beta_M S_i + \gamma_M U_i + \delta_M F_i$$
$$C_i^H = \alpha_H + \beta_H S_i + \gamma_H U_i + \delta_H F_i$$

Where coefficients reflect **how sensitive each governance mode is to specificity, uncertainty, and frequency**. For instance:

- Market is cheap for low specificity, low uncertainty, low frequency
- Hierarchy becomes cheaper as specificity, uncertainty, or frequency increase

Then the optimal governance for transaction i is:

$$g_i^* = \arg \min_{g \in \{M,H\}} C_i^g(S_i, U_i, F_i)$$

So, for each transaction, compute the three potential costs and pick the governance mode with the lowest cost

Symbiosis prediction

Symbiosis arises when two transactions are co-dependent and jointly internalized in the hierarchy:

- Let i and j be linked transactions
- Joint transaction cost:

$$C_{ij}^{\text{joint}} < C_i^M + C_j^M$$

If this inequality holds, hierarchy creates a **net benefit**, predicting **stable interdependence**, i.e., symbiosis

228. Organizational Ecology

229. Certifications

230. CSCP

230.1. Supply Chains, Demand Management, and Forecasting

230.1.1. Introduction to Supply Chain

1. Supply Chain

Network of organizations, people, activities, information and resources involvd in supplying a product or service to a customer. Encompasses everything from procurement of raw materials to final delivery of finished product to the customer.

- Suppliers: Providers of raw materials or components
- Manufacturers: Transform raw materials into finished goods
- Distributors: Warehouses that hold & deliver goodsRetailers: Interface with final customers
- Customer: End users of product or service

2. Supply Chain Management

Overseeing and managing the entire flow of goods and services, ensuring efficient movement from raw materials to finished products.

- Minimize costs
- Optimize inventory levels
- Improving efficiency
- Ensure timely delivery of customers
- Enhance customer satisfaction
- Meet customer demand

Types of supply chains

- Make-to-Stock (MTS): Products are manufactured in advance and stocked based on demand forecast (Coca-Cola)
- Make-to-Order (MTO): Products are made only after a customer order is received (Boeing)
- Assemble-to-Order (ATO): Products are assembled from pre-manufactured components only after a customer order is received (Dell)

3. Flows

- Information: Data exchanged between different parts of the supply chain (orders, inventory levels, product status)
- Product: Physical movement of goods from suppliers to manufacturers, distributors, retails and consumers
- Financial: Payment and credit processes between participants (cost transfers, invoicing, patments)

4. Globalization

Challenges

- Longer lead times
- Currency flucutations
- Regulation compliance
- Geopolitical tensions

5. Competitive advantage

    - Speed to market: Reducing time it takes to get products to market
    - Cost efficiency: Reducing production and transportation costs
    - Felxibility: Ability to adapt to market changes and customer need
    - Customer service: Ensuring timely delivery and product availability

6. Technology & Digitalization

    - AI/ML: Demand forecasing & predictive maintenance
    - IoT: Track shipments & manage inventory
    - Blockchain: transactions transparency and security

7. Sustainability

Minimize environmental & social impact

- Carbon emissions (efficient transportation)
- Energy efficient processes in production
- Minimize waste
- Renewable / recyclable resources
- Fair labor practices / ethical labor practices

8. Risk Management

Supply chain resilience

- Operational risk: Breakdown in manufacturing or distribution process
- Supply risk: Disruption in availability of materials or services
- Demand risk: fluctuation in customer demand
- Environmental risk: Natual disacters / climate change
- Natural disters
- Geopolitical tensions
- Supplier failures
- Demand fluctuation

9. Collaboration

    - Supplier partnerships: Building long term relationships with key suppliers
    - Shared logistics: Coordinating tranportation & warehousing partners
    - Data sharing: sharing real time date, production schedules, dand demand forecasts

10. Supply Chain Integration

    - Horizontal: Collaboration between different firms at the same level of the supply chain (2 manufacturers or 2 distributors)

- vertical: Multiple stages of the supply chain, from raw materials to final product distribution
- Technology: Shared platforms for operations and communication across supply chain

11. Ethical Supply Chains

- Fair wages
- Safe working conditions
- Environmental protection
- Corporate transparency

12. Bullwhip Effect

small fluctuations in customer demand at the retail level become larger and larger as you move upstream (wholesalers → distributors → manufacturers → suppliers)

Casuses: information distortion + delayed reactions

- Delays in demand information moving up the supply chain
- Batch ordering (retailers order large quantities irregularly)
- Price fluctuations (order spikes)
- Overreaction to demand signals by upstream suppliers

230.1.2. Demand Analysis and Patterns

1. Demand Analysis

Predict demand for products and services

- Historical Data: Analyzing past sales & trends
- Market Conditions: Understanding current market dynamics & economic indicators
- Consumer Behavior: Identifying changes in customer preferences and buying patterns

2. Demand Patterns

- Seasonal: Peaks and trophs (hilodays, summer, etc.)
- Cyclical: Economic cycles (recessions & booms)
- Random: Noise
- Trend: Long term movement (up or down)

3. Demand Forecasting Methods

- Qualitative: Expert judgement (e.g., Delphi method)
- Quantitative: Mathematical model, historical data (e.g., time series analysis)

4. Factos affecting demand

- Price
- Consumer preferences
- Income levels
- Competition
- Substitute products

5. Role of Technology in Demand Analysis

- AI/ML

    - Big Data

6. Balancing Supply and Demand
    - Just-in-Time (JIT) inventory: reducing inventory costs by producing just when needed
    - Safety stock: Extra inventory to cushion against demand fluctuation
    - Felxible production systems: Quickly adapting production levels based on demand change
7. Impact of external factors on demand
    - Political: Policy, trade tariffs, international relations
    - Economic: Recession & booms
    - Social: Societal values
    - Environmental: Natural disasters, pandemics
8. Measuring demand accuracy
    - Mean Absolute Deviation (MAD): Measures average magnitude of forecast errors, regardless of direction
    - Mean Squared Error (MSE): Squared errors, giving more weight to larger deviations
    - Tracking Signal: Monitors whether a forecast is consistently over or underestimating demand, detecting bias in forecasting methods
8. Demand Management and Planning
    - Product promotion
    - Pricing strategies
    - Collaborative demand Planning
    - Flexible production

230.1.3. Demand Management

1. Demand Drivers
    - Economic conditions
    - Market trends
    - Technological advancements
    - Consumer preferences
    - Seasonality
    - Marketing
2. Forecasting Demand
    - Quantitative
    - Qualitative
    - Hybrid
3. Collaborative Demand Planning

Collaborate across

- Internal Units (marketing, sales, production)
- External partners (suppliers, retailers)

4. balancing Supply and Demand

Meeting demand without over- or under-production

5. Demand Shaping

Influence demand through strategic initiatives
    - promotions
    - discounts
    - advertising
    - product bundling

6. Technology in Demand Management

    - Big Data
    - AL/ML

7. Inventory Management

    - Overstocking: ties up capital & incurs storage cost
    - Unserstocking: missed sales opportunities

8. Demand Management Chellenges

    - Demand variability
    - Lead times
    - Supply chain disruptions
    - Changes in customer preferences

9. Risk Management

    - Buffer inventory
    - Supplier diversification
    - Contingency planning (natural disasters, economic downturn, etc.)

10. Sales & Operations Planning (S&OP)

Integrated business management process that bring together sales, marketing, supply chain, finance teams to align demand forecasts with operational plans

230.1.4. Forecasting

1. Importance

Planning supply chain activities to meet future demand while optimizing costs

2. Qualitative v. Quantitative

    - Qualitative
    - Quantitative

3. Time Series Forecasting

Predict future demand

    - Moving averages
    - Exponential smoothing
    - Seasonal adjustments

4. Causal Forecasting Models

Cause-and-Effect relationship between demand and various external factors (price, economic indicators, marketing)

5. Forecasting Tools & Software

    - ML
    - Data analytics
    - Real-time data

6. Forecasting Horizon

Time period that the forecast covers

- Short-term (days to weeks)
- Medium-term (months)
- Long-term (years)

Accuracy decreases with longer time horizons

7. Seasonal and Cyclical Demand

    - Seasonal: Predictable variation in demand (weather, holidays)
    - Cyclical: Fluctuations tied to broader economic cycles

8. Collaborative Planning, Forecasting & Replenishment (CPFR)

Businesses work with suppliers, customers, & other partners to jointly forecast demand

9. Forecasting Challenges

Example

MAPE

Data:

Month	Actual	Forecast
1	1200	1250
2	1400	1350
3	1100	1000
4	1300	1200

Steps:

Month	Actual	Forecast	Error $\lvert A - F \rvert$	% Error $\frac{\lvert A-F \rvert}{A}$
1	1200	1250	–50	4.17%

2	1400	1350	50	3.57%
3	1100	1000	100	9.09%
4	1300	1200	100	7.69%

$$\text{MAPE} = \frac{4.17 + 3.57 + 9.09 + 7.69}{4} = 6.6\%$$

Example

EOQ

Data:

Annual Demand (D): 24,000 units

Order Cost (S): $60 / order

Carrying (Holding) Cost: $4/units/year

Steps:

$$\text{EOQ} = \sqrt{\frac{2DS}{H}} = \sqrt{\frac{2 \times 24000 \times 60}{4}} \approx 848$$

$$\text{Orders/year} = \frac{D}{\text{EOQ}} = 24, \frac{000}{848} \approx 28.3$$

$$\text{Ordering Cost} = 28.3 \times 60 = \$1698$$

$$\text{Holding Cost} = (848/2) \times 4 = \$1696$$

$$\text{Total Annual Cost} = \text{Ordering Cost} + \text{Holding Cost} = 1698 + 1696 = 3394$$

Example

Safety Stock (Service Level)

Data:

Average Weekly Demand = 400

Standard Deviation Demand = 50

Lead Time = 2 Weeks

Service Level = 95% $\rightarrow$ z = 1.65

Steps:

$$\sigma_i = \sigma\sqrt{L} = 50 \times \sqrt{2} = 70.7$$

$$\text{Safety Stock} = z \times \sigma_i = 1.65 \times 70.7 = 117 \text{ units}$$

Example

Reorder Point ROP

Data

Average Daily Demand = 60 units

Lead Time = 8 days

Safety Stock = 100 units

Steps

$$\text{ROP} = (\text{Demand} \times \text{Lead Time}) + \text{Safety Stock}$$

$$\text{ROP} = (60 \times 8) + 100 = 480 + 100 = 580 \text{ units}$$

Reorder Point = 580 units

Example

Capacity Utilization

Data

Design Capacity = 10000 units / week

Actual Output = 8500 units / week

Steps:

$$\text{Utilization} = \left(\frac{\text{Actual}}{\text{Design}}\right) \times 100 = \left(\frac{8500}{10000}\right) \times 100 = 85\%$$

Capacity utilization = 85%

Plant is operating efficiently below full load

Example

Inventory Turnover & Days of Supply

Data

COGS = $4,800,000

Average Inventory = $600,000

Steps

$$\text{Inventory Turnover} = \frac{\text{COGS}}{\text{Average Inventory}} = \frac{4800000}{600000} = 8 \text{ turns / year}$$

$$\text{Inventory Turnover} = 8 \times \text{per year} \quad \rightarrow \quad 45.6 \text{ days of supply}$$

Example

Total Landed Cost Comparison

Data:

Supplier A: $20 / unit + $2000 freight per 1000 units

Supplier B: $19 / unit + $3500 freight per 1000 units

Steps

Supplier A: cost / unit = 20 + (2000 / 1000) = $22.00

Supplier B: cost / unit = 19 + (3500 / 1000) = $22.50

Supplier A is cheaper by $0.50 **per unit**

Example

Cycle Time Reduction

Data

Output = 500 units / day

Cycle Time = 1.2 min / unit

Improvement = 15% reduction

Steps

New Cycle Time = 1.2 times (1 - 0.15) = 1.02 min / unit

Available time = 500 times 1.2 = 600 min / day

New output = 600 / 1.02 = 588 units / day

Increase = 588 - 500 = +88 units / day

Example

Cash To Cash Cycle Time

Data

Days of Inventory = 40

Days receivable = 30

Days payable = 25

Steps

C2C = Days Inventory + Days Receivable - Days Payable = 40 + 30 - 25 = 45

Cash to Cash Cycle Time = 45 days

Example

Distribution Network Optimization

Data

Current (1 DC)

- Transportation = $ 300,000
- Facility = $100,000

New (2 DCs)

- Transportation = $220,000
- Facility = $180,000

Steps

Current total = 300,000 + 100,000 = 400,000

New Total = 220,000 + 180,000 = 400,000

Nototal cost change

231. Supply Chain Analytics

231.1. Inventory Turns

Inventory Turns: How many times inventory is turned over per year

Inventory Turns = Cost of Goods Sold / Average Inventory Value

Days of Supply: Average days the item is held in inventory

Days of Supply = Number of Days in Period / Inventory Turns

Efficiency Benchmark

Supply chain strategies

- **Push**: Forecast-driven, build inventory based on demand forecasts
- **Pull**: Customer-order-driven, only produce when there is a demand

Factories -Shipping Inbound-> Distribution Centers -Shipping Outbound> Retail Stores <- Customers

Factory -> DCs -Direct Ship-> Customer

Question: Use stores or direct shipping?

Shipping v. Inventory Trade-off

1. Classical (Store) Model

DCs -Truck Load (TL) -> Stores (Large Inventory) <- Customers

Transport v. Inventory Trade-off

Save shipping cost by TL but require large inventory at stores

Low Value, Long Shelf Life, High Volume Items

2. Continuous Replenishment Model

DCs -Less Than Truck Load (LTL) -> Stores (Small Inventory) <- Customers

Increase Shipping Cost but Reduce Inventory Cost at Stores

3. Direct Shipping Model (Showroom)

Factory -> DCs -Direct Ship-> Customer

No Stores, Direct Shipping

Almost Zero Inventory, High Shipping Cost

High Value, Slow Moving Items

Inventory Risk Pooling Effect

Consolidate inventory into fewer locations, and total safety stock drops — even while serving the same demand

Simple Example Say you sell umbrellas in 2 cities. Each city has:

Average weekly demand: 100 units Std deviation: 50 units You hold 1.65σ safety stock (95% service level)

Decentralized (2 separate warehouses):

Safety stock per city = 1.65 × 50 = 82.5 units Total safety stock = 82.5 × 2 = 165 units

Centralized (1 warehouse serving both cities):

Combined std deviation = √(50² + 50²) = √5000 ≈ 70.7 units Safety stock = 1.65 × 70.7 ≈ 117 units

Savings: 165 → 117 units (29% less inventory)

231.1.1. Push Pull Strategies

Push — you push product into the supply chain speculatively. You bet on your forecast being right. Pull — you wait for a customer order, then pull production/replenishment. No forecast needed, but you need fast response.

The Push-Pull Boundary Most real supply chains are hybrid. The boundary (called the decoupling point) is where you switch from push to pull. Example — car manufacturing:

Steel → stamping → painting → [decoupling point] → final assembly per order Everything upstream is pushed (economies of scale); downstream is pulled (customer config)

Push	Pull

Forecast-driven	Order-driven
Build inventory ahead	Wait for order
Higher inventory	Lower inventory

	Pros	Cons
Push	-Customer doesn't wait • Batch picking / shipping as DCs • Economies of scale	• Significant inventory investment • Risk of obsolescence
Pull	• Lower inventory investment • Faster response to customer demand (latest assets)	• Customer has to wait • Unit picking / packing at DCs • Higher shipping costs • Longer lead times

Tradeoff:

- Shipping, picking / packing
- Customer wait time

vs.

- Inventory investment
- New product introduction

Categorize Products

		Volume	
		Low	High
Price	Expensive	Pull	?
	Inexpensive	?	Push

Data Collection

- Inventory Holding Cost
- Shipping Cost
- Picking / Packing Cost
- Sales Volume (by product)
- Inventory Levels (by product)

Calculate:

- Inventory Cost
- Shipping Cost
- Warehouse Cost

Under Push / Pull by Product

1. Inventory Cost

Estimating Inventory Cost Rates (Per Week Per Unit) At Stores

Capital cost per week per unit + Depreciation per week per unit

Inventory cost per week per unit =

$$\frac{\text{Annual capital cost}}{52} + \frac{\text{Product Value} - \text{Liquidation Value}}{\text{Product Life Cycle}}$$

E.g.,

- Smartphones
  - Product Value: $500
  - Liquidation value: 0%
  - Capital return rate: 8%
  - Inventory cost / unit: $20
  - Life cycle: 26 weeks
- Feature Phone
  - Product Value: $200
  - Liquidation value: 0%
  - Capital return rate: 8%
  - Inventory cost / unit: $8

$\frac{500 \times 0.08}{52} + \frac{500-0}{26} = \$20$

2. Picking / Packing Cost

- Push: Weekly

Cost of N units = Cost of 1st pick/pack cost + subsequent pick/pack cost * (N-1)

- Pull: Unit

1st pick/pack cost * N

231.1.2. Inventory cost rates (per week per unit)

Inventory cost per week (per unit) = Capital cost per week (per unit) + Depreciation per week (per unit)

Capital cost per week (per unit) = Product value * Annual capital return rate / 52

Depreciation per week (per unit) = (Product value - Liquidation value) / Product life-cycle (in weeks)

Example 1: Smart phones

The value of a smart phone is $500 on average. Suppose the annual capital return rate is 8% (if the $500 is invested elsewhere, one may get a return of 8% annually), the liquidation value at the end of the product life-cycle is $0, and the product life-cycle is 26 weeks (half a year). Then for smart phones,

Inventory holding cost per week (per unit) = $500 * 8% / 52 + ($500 - $0) / 26 = $20.00 / unit.

Example 2: Feature phones

The value of a feature phone is $200 on average. Suppose the annual capital return rate is 8%, the liquidation value at the end of the product life-cycle is $0, and the product life-cycle is 26 weeks (half a year). Then for features phones,

Inventory holding cost per week (per unit) = $200 * 8% / 52 + ($200 - $0) / 26 = $8.00 / unit.

231.1.3. Shipping cost rates (per unit)

Overnight express shipping rate (per unit): $12.00 / unit (FedEx benchmark)

Standard 2-day shipping with batch discount:

The 2-day standard shipping cost is usually much cheaper than overnight express, and batch shipping (shipping more than 10 units at a time to the same destination) usually enjoy a discount depending on the carriers and distance.

Using FedEx as a benchmark, the ratio between overnight and 2-day shipping rates is 2.5 / 1, and shipping batch can enjoy a 50% discount per unit. Then the unit shipping cost for 2-day batch shipping is

$12 / 2.5 * 50% = $2.40 / unit.

231.1.4. Warehouse picking / packing cost rates

To pick / pack N (>1) identical products in one order:

1st pick / pack cost = $1.00.

subsequent pick / pack cost = $0.10

232. ACCA F3 — Financial Accounting

232.1. Cheatsheet

2. The context and purpose of financial reporting	
2. The context and purpose of financial reporting	
Define financial reporting	Process of recording, analysing, and summarising financial transactions for decision-making
Types of business entity	Sole trader, partnership, limited liability company (LLC)
Legal differences between entities	Sole trader/partnership: unlimited liability LLC: separate legal entity, limited liability
Advantages/disadvantages of entities	Sole trader: simple but risky Partnership: shared skills but joint liability LLC: protection but regulation

Nature, principles, scope of financial reporting	Produces useful financial info for external users under standards
2. Stakeholders' needs	
Identify the **users** of financial statements and state and differentiate between their **information needs**.	• Investors → returns & growth • Lenders → repayment ability • Suppliers → short-term liquidity • Employees → stability & rewards • Government → tax & compliance
2. Financial statements	
Statement of **financial position**	Shows **assets**, **liabilities**, **equity** at a point in time
Statement of **profit or loss**	Shows performance over a period
Statement of changes in **equity**	Explains movements in equity
Statement of **cash flows**	Explains cash inflows and outflows
Asset	Resources controlled from past events generating future benefits
Liability	Present obligations from past events
Equity	Residual interest: assets – liabilities
Income	Increases in economic benefits
Expense	Decreases in economic benefits
2. Regulatory framework	
Purpose of regulatory system	Ensure consistency, transparency, comparability
IFRS Foundation	Oversees standard-setting

International Accounting Standards Board (**IASB**)	Issues IFRS Accounting Standards
IFRS Advisory Council	Advises IASB
IFRS Interpretations Committee (**IFRIC**)	Interprets IFRS
International Sustainability Standards Board (**ISSB**)	Sustainability standards
2. Duties and responsibilities of those charged with governance	
Define governance, in the context of the preparation of financial statements	System directing and controlling preparation of accounts
Duties and responsibilities of directors in the preparation of the financial statements	Prepare true and fair financial statements
3. Accounting principles, concepts and qualitative characteristics	
3. Key principles and concepts of accounting	
Going concern	Assume business continues indefinitely
Accrual accounting	Record income/expenses when earned/ incurred
Materiality	Ignore immaterial items
Offsetting	No netting unless permitted
Consistency	Same methods over time
Prudence	Caution under uncertainty
Duality	Every transaction has two effects
Business entity	Business separate from owner

Historical cost	Assets recorded at cost
Current value	Assets recorded at fair value
Substance over form	Economic reality > legal form
3. Qualitative characteristics of useful financial information	
Relevance	
Faithful representation	
Comparability	
Verifiability	
Timeliness	
Understandability	
4. The use of double-entry bookkeeping and accounting systems	
Quotation	
Sales order	
Purchase order	
Goods received note	
Goods dispatched note	
Customer (sales) invoice	
Supplier (purchase) invoice	
Supplier statement	
Credit note	

Debit note	
Remittance advice	
Receipt	
Accounting equation	

5. Recording transactions and events
5. Sales and purchases
5. Cash
5. Inventories
5. Tangible non-current assets
5. Depreciation
5. Intangible non-current assets and amortisation
5. Accrued expenses (accruals), prepaid expenses (prepayments), accrued income, and deferred income
5. Receivables and payables
5. Provisions and contingencies
5. Capital structure and finance costs

232.6. Overview

232.6.1. Financial Accounting

232.6.1.1. Objectives

1. Recording transactions
2. Summarizing

Statements

- Profit & Loss
- Financial Position (Balance Sheet)

232.6.1.2. Types of Business Entities:

1. Sole Trader

One person

Owner has unlimited liability

2. Limited Liability Company

Several people

Business separate in law from owners

Owners have limited liability

232.6.1.3. Users of accounting information

- Management
- Owners / Shareholders
- Potential investors
- Lenders
- Employees
- Government
- Public

232.6.1.4. Accounts

	Management	Financial
Purpose	Help management run business	Required by law
Timing	Monthly	Yearly
Requirement	No legal requirement	Legally required
Layout	Useful	International Financial Reporting Standards (IFRS)
Reporting	Internal	External

232.6.1.5. Statement of Financial Position (Balance Sheet)

Every transaction has 2 effects (+ and -)

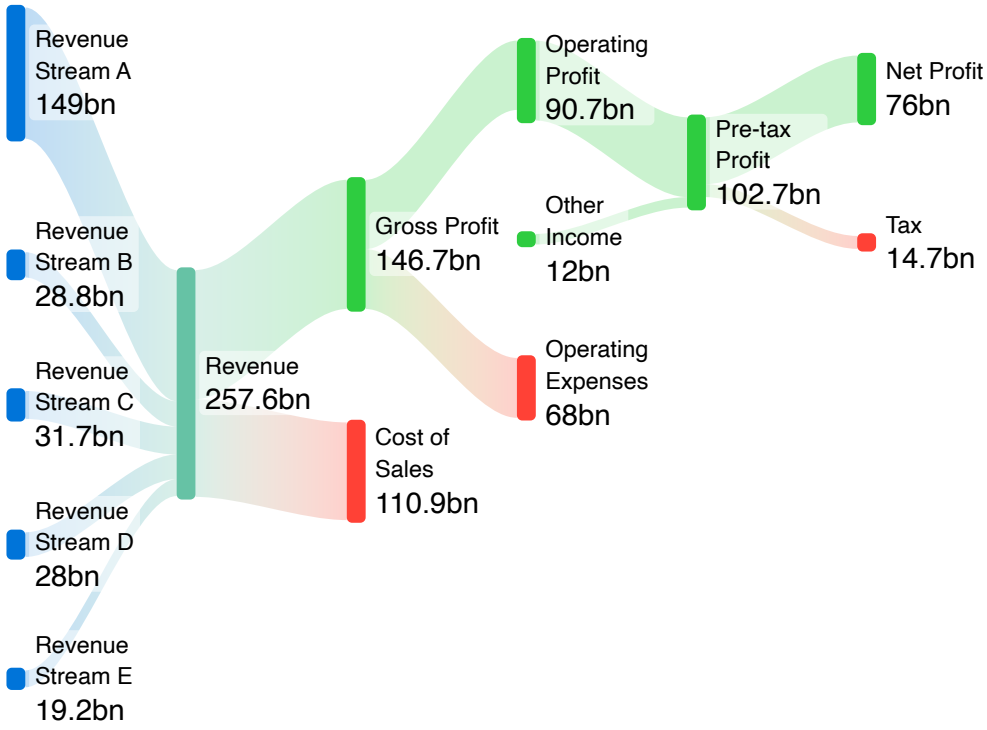
List of:

- Owned
- Owed

- Assets
    - Current
    - Non-current

- Liabilities
    - Current
    - Non-current
- Capital / Equity

232.6.1.6. Statement of Profit & Loss (P&L)



232.6.1.7. Cash Flow Statement

232.6.1.8. Assets & Liabilities

Concept	Definition	Example/Notes
Asset	Resource controlled by the business expected to provide future economic benefits.	Cash, inventory, equipment, receivables.
Liability	Present obligation arising from past events, settlement expected to result in outflow of resources.	Loans, payables, accruals.
Capital Owner's Equity	Owner's claim on business assets after liabilities; money invested + retained profits.	Net assets
Current Asset	Expected to be used or converted to cash within 12 months.	Inventory, receivables, cash.

Non-current Asset	Expected to provide benefit over more than 12 months.	Property, plant, equipment.
Current Liability	Obligation due within 12 months.	Accounts payable, short-term loans.
Non-current Liability	Obligation due after 12 months.	Long-term loans, bonds payable.

232.6.1.9. Revenue & Expenses

Concept	Definition	Example/Notes
Revenue Income	Inflows from selling goods or services in ordinary course of business.	Sales, service income.
Expense	Outflows or use of assets in earning revenue.	Rent, salaries, utilities.
Capital Expenditure CapEx	Spending to acquire or improve non-current assets that provide benefits over multiple periods	Machinery, building.
Revenue Expenditure RevEx	Spending on operating costs whose benefits are consumed within the current accounting period	Repairs, rent, electricity.

Concept	Definition	Example/Notes
Inventory	Goods held for resale or raw materials to make goods.	Retail: finished goods; Manufacturing: raw materials.
Accounts Payable Payable	Money owed to suppliers/ creditors.	Purchase of goods on credit.
Accounts Receivable Receivable	Money owed to the business by customers.	Sales on credit.
Drawings Withdrawals	Assets taken out by owner for personal use.	Cash withdrawn, goods taken for personal use.

Bank Cash	Liquid assets held by the business.	Cash in hand, cash at bank.

Owns = Owes

Assets = Liabilities + Capital (Accounting Equation)

Current vs Non-current is always about the 12-month rule

Revenue - Expenses = Profit / Loss

Capital + Additional Investment - Drawings + Profit - Loss = Closing Capital

232.7. Stakeholders

Identify the **users** of financial statements and state and differentiate between their **information needs**.

Investors	returns & growth
Lenders	repayment ability
Suppliers	short-term liquidity
Employees	stability & rewards
Government	tax & compliance

- **Investors** (shareholders)

Need information on profitability, dividends, growth, and risk to decide whether to buy, hold, or sell shares.

- **Lenders** (banks, loan providers)

Need information on liquidity, cash flows, and solvency to assess the entity's ability to repay loans and interest.

- **Suppliers** (trade payables)

Need information on short-term liquidity and creditworthiness to decide whether to offer credit and on what terms.

- **Employees**

Need information on profitability and stability to assess job security, wages, and pension prospects.

- **Customers**

Need information on financial stability to ensure continuity of supply, especially for long-term contracts.

- **Government** and tax authorities

Need information on profits, revenue, and compliance to assess tax liabilities and enforce regulations.

- **Management** (internal user, but examinable)

Need detailed information on performance, costs, and financial position to plan, control, and make decisions.

232.8. Balance Sheet

232.8.1. Balance Sheet (Statement of Financial Position)

Assets, **liabilities** and **equity** at a single time point

Assets		**Liabilities**	
Current Assets		**Current Liabilities**	
Cash	X	Account Payable	X
Account Receivables	X	Tax Payable	X
Prepaid Expenses	X	Accrued Expenses	X
Total Current Assets	X	Deferred Revenue	X
Non-Current Assets		Total Current Liabilities	X
Property, Equipment	X	**Non-Current Liabilities**	
Intangible	X	Long Term Loans	X
Total Non-Current Assets	X	Total Non-Current Liabilities	X
Total Assets	X	**Total Liabilities**	X
		Equity	
		Capital Contributions	X
		Retained Earnings	X
		Total Equity	X
		Total Liabilities & Equity	X

232.9. Income Statement

Revenue and **expenses** over **time**

Simple:

	MM-DD-YY
Revenue	X
Expenses	(X)

Profit / (Loss) **X / (X)**

Detailed:

	MM-DD-YY
Revenue	X
Cost of Goods Sold	(X)
Gross Profit / (Loss)	**X / (X)**
Operating Expenses	(X)
Operating Profit / (Loss)	**X / (X)**
Tax Expense	(X)
Interest Expense	(X)
Net Profit / (Loss)	**X / (X)**

Revenue:

- Product Sales
- Services Rendered

Expenses:

- Direct (increase in direct proportion to sales)
  - Cost of services
  - Cost of goods sold
  - Product sales - cost of goods/services = gross profit
- Indirect (overhead)
  - Fixed (no correlation to sales)
    - Rent
    - Salaries
    - Insurance
    - Admin
    - Legally
    - Accounting
    - Marketing
    - Depreciation
    - Amortization
  - Variable (loosely correlated)
    - Advertising
    - Commissions
    - Utilities

232.10. Cash Flow Statement

Cash inflows and **outflows** over a period of time

	MM-DD-YY
Cash Inflow	X
Cash Outflow	(X)
Net Cash Flow	**X / (X)**

232.10.1. Direct Method

232.10.2. Indirect Method

233. Appendix

234. Scheduling Code

```
from pyomo.environ import *
from pyomo.dataportal import DataPortal

model = AbstractModel()

# Sets
model.Days = Set(ordered=True)           # Days 1..7 (Mon..Sun)
model.Shifts = Set(ordered=True)         # Shifts starting on days 1..7

# Parameters
model.Demand = Param(model.Days)         # Daily staffing requirements
model.Cover = Param(model.Days, model.Shifts, within=Binary)  # Coverage
matrix (1 if shift j covers day i)

# Variables
model.x = Var(model.Shifts, domain=NonNegativeReals)

# Objective: Minimize total employees hired
def obj_rule(model):
    return sum(model.x[s] for s in model.Shifts)
model.OBJ = Objective(rule=obj_rule, sense=minimize)

# Constraints: Cover daily demand
def demand_rule(model, d):
    return sum(model.Cover[d, s] * model.x[s] for s in model.Shifts) >=
model.Demand[d]
model.DemandConstraint = Constraint(model.Days, rule=demand_rule)

# Load data from .dat file
data = DataPortal()
data.load(filename='data.dat', model=model)

# Create an instance of the model
instance = model.create_instance(data)

# Create solver
solver = SolverFactory('glpk')
solver.options['tmlim'] = 60

# Solve with solver timeout (optional)
results = solver.solve(instance, tee=True)

# Display results
instance.display()
```

```
set Days := Mon Tue Wed Thu Fri Sat Sun ;
set Shifts := s1 s2 s3 s4 s5 s6 s7 ;

param Demand :=
Mon 110
Tue 80
Wed 150
Thu 30
Fri 70
Sat 160
Sun 120 ;
```

```
12
13 # Each shift covers 5 consecutive days starting on the shift's day
14 # 1 if shift s_j covers day d_i, 0 otherwise
15
16 param Cover:
17         s1 s2 s3 s4 s5 s6 s7 :=
18 Mon     1  0  0  1  1  1  1
19 Tue     1  1  0  0  1  1  1
20 Wed     1  1  1  0  0  1  1
21 Thu     1  1  1  1  0  0  1
22 Fri     1  1  1  1  1  0  0
23 Sat     0  1  1  1  1  1  0
24 Sun     0  0  1  1  1  1  1 ;
25
```

235. White Paper

(Project Cybersyn/Synco, Project Red Book, OGAS Project)

- Publicly owned
- Democratically controlled
- Environmentally sustainable
- Fully automated
- Transparent

Supply chain guaranteeing a predefined set of goods and services to all members of society

France is split into 5 strictly nested administrative units:

- National ($\mathcal{N}$)
- Regions ($\mathcal{R}$)
- Departements ($\mathcal{D}$)
- Communes ($\mathcal{C}$)
- IRIS ($\mathcal{I}$)

Objectives:

1. Facility Location
    - **Distribution Centers**: Determine optimal locations of distribution centers at the National, Regional, Departement, Commune, and IRIS levels, subject to demographic demand and geographic accessibility.
    - **Government Stores**: Determine optimal placement of government stores such that all households have equitable access to essential goods.
2. Flow Allocation
    - **Supply Allocation**: Determine how much of each good should be allocated from supply nodes (imports, extraction, or domestic manufacturing plants) to each level of distribution.
    - **Demand Fulfillment**: Ensure that each demand node receives sufficient quantities of goods to meet population needs, respecting fairness and prioritization rules (e.g., hospitals and schools may receive priority allocations).
    - **Reverse Flows**: Incorporate waste recovery, recycling, and reverse logistics so that outputs from demand nodes (waste, byproducts, returns) are routed to processing or extraction nodes when feasible.
3. Systemic Objectives
    - **Equity**: Guarantee universal access across all IRIS units, reducing geographic disparities.
    - **Resilience**: Provide redundancy and flexibility in flows to handle shocks (e.g., disruptions to imports or local production).
    - **Sustainability**: Minimize environmental impact by favoring local production, renewable inputs, and circular waste loops.
    - **Efficiency**: Optimize flows to minimize costs, transport distances, and resource usage, while meeting fairness and sustainability constraints.
    - **Transparency** and **Participation**: Make allocation decisions visible, accountable, and open to democratic oversight at all administrative levels.

        1. Cross-product correlations:

• Let the network learn them via attention or graph connections.

2. Lagged effects:

• Include a long enough lookback window.

3. Continuous updates:

• Use streaming updates or sliding windows.

so the model should account for **spatial** correlations in addition to **temporal** ones (seasonality and trend and product correlations)

Scales There can be a seasonal demand peak at the households, neighborhoods, …, national (levels) scales Correlated with space (GIS)

Seasonality

GeoSpacial Given the administrative hierarchy we need to determine where to place different sized dcs and stores

235.1. Administrative Hierarchy and Demographics

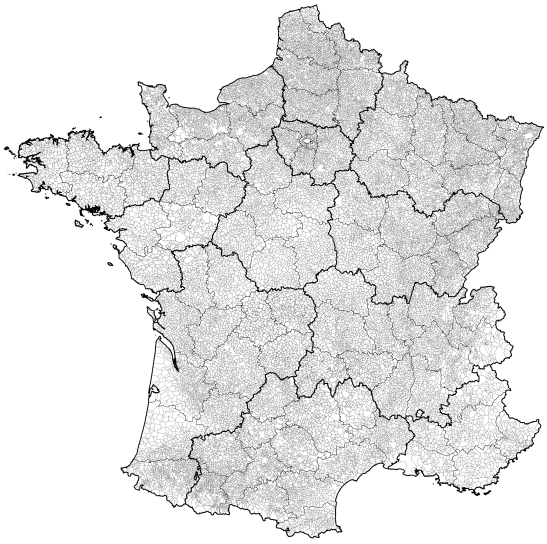


Figure 10: Administrative Hierarchy

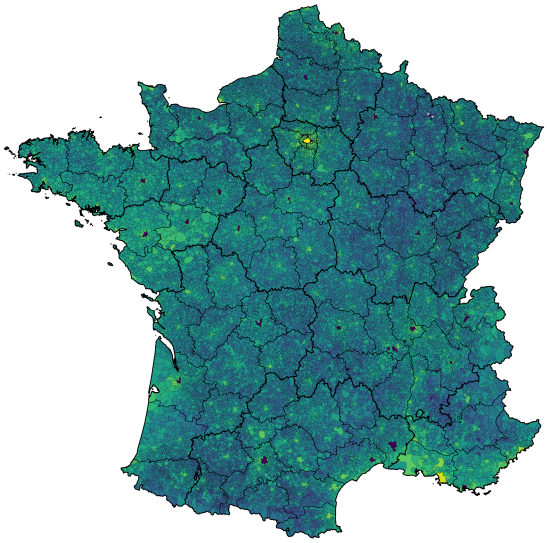


Figure 11: Commune Population Map

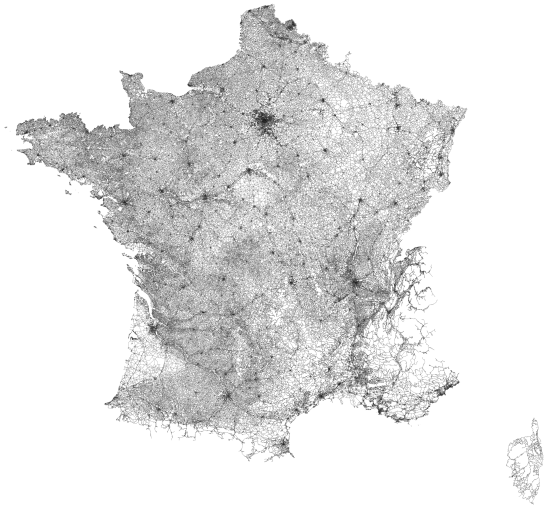


Figure 12: Roads

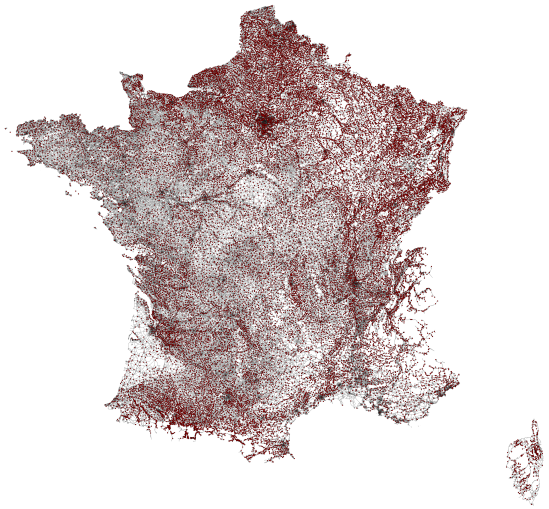


Figure 13: Roads & Commune Hubs

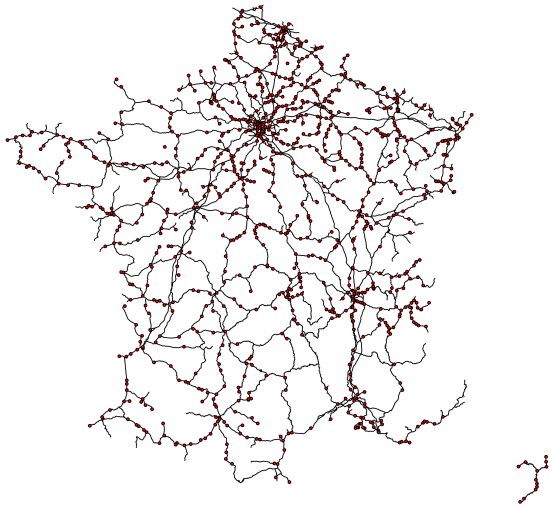


Figure 14: Train Network and Hubs

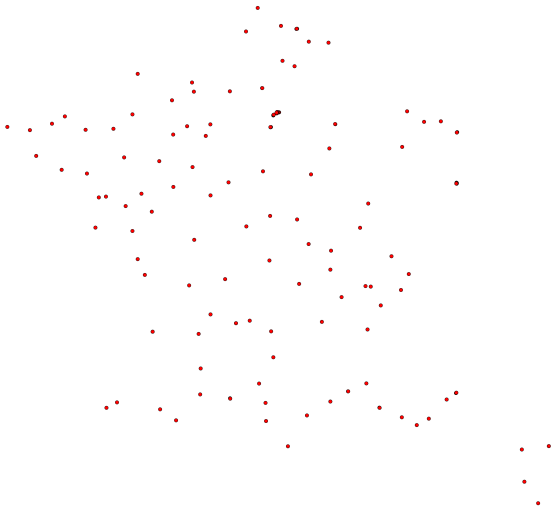


Figure 15: Aeronatic Hubs

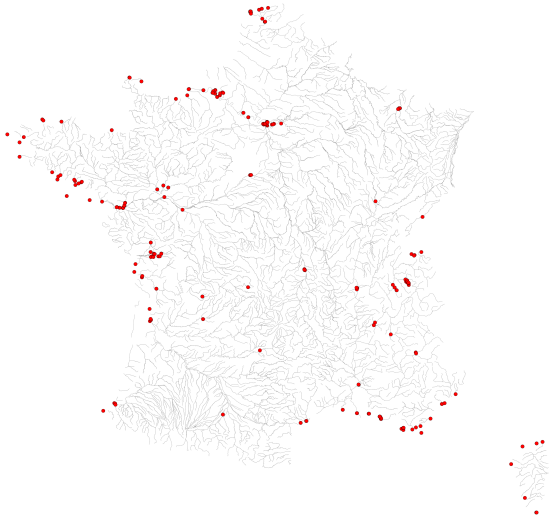


Figure 16: Port Hubs

235.2. Schema

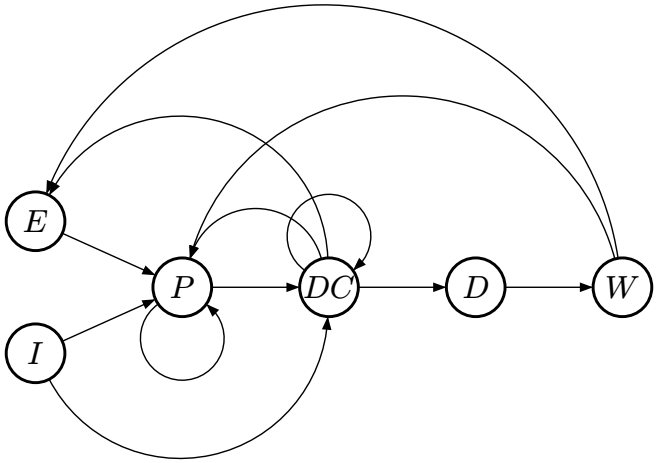
Nodes

Category	Type	Attributes	Description	Code
Extraction				
Supply	Import Facility			

	Manufacturing Plant			
Intermediary	National Distribution Center (NDC)			NDC_
	Regional Distribution Center (RDC)			RDC_
	Department Distribution Center (DDC)			DDC_
	Commune Distribution Center (CDC)			CDC_
Demand	Store			
	Household			
	School			
	Hospital			
Waste				

Mode of Transport	Type	Attributes	Description	Code
Road	Highway National Local			
Rail	Freight Regional			

Air	Flight Path Cargo Route			
Martime	Shipping Coastal			
Inland Waterway	River Canal			



Raw materials are obtained either from Extraction Nodes (E) or imported through Import Nodes (I). They are then sent to Processing Nodes (P), where they undergo one or more processing stages. The resulting products are delivered to Distribution Nodes (DC).

Distribution Nodes can also receive finished products directly from Import Nodes. They may transfer goods to other Distribution Nodes (at different administrative levels), directly to Demand Nodes (D), or back to Processing and Extraction Nodes—for example, when machinery, spare parts, or energy carriers are required to sustain those activities.

Once products are consumed at Demand Nodes, they generate outputs that flow into Waste Nodes (W). Waste Nodes can (1) dispose of waste, (2) redirect it to Processing Nodes for recovery or recycling, or (3) return it to Extraction Nodes through resource recovery activities such as landfill mining or biogas capture.

Data

IRIS	• https://geoservices.ign.fr/contoursiris • https://geoservices.ign.fr/irisge
Administrative Units	• https://geodatafr.github.io/IGN/ADMIN_EXPRESS_Administrative_boundaries/
Roads	• https://transport.data.gouv.fr/datasets/route-500-r
Rail	• https://www.data.gouv.fr/datasets/fichier-de-formes-des-lignes-du-reseau-ferre-national • https://resources.data.sncf.com/explore/dataset/formes-des-lignes-du-rfn/information/

Train Stations	• https://transport.data.gouv.fr/datasets/liste-des-gares?locale=en
Martime Ports	• https://www.data.gouv.fr/datasets/ports-espace-maritime-francais/ • https://data.opendatasoft.com/explore/dataset/caracteristiques-des-ports-en-france-2021%40pndb/
Inland Ports	
Aeroports	• https://www.data.gouv.fr/datasets/aeroports-francais-coordonnees-geographiques/

Stack

Safe Rust

DataBase	• PSQL (`tokio_postgres`, `sqlx`, `bb8`) • ScyllaDB (`scylla`)
API	• gRPC (`tonic`) • REST (`axum`)
Front End	• `dioxus` • `leptos` • `bevy`

236. References

Bibliography